# Première

## Numérique et Sciences Informatiques

Cours • Exercices • Corrigés




**Mathieu Thibaud**

Édition 2026

Version du 8 novembre 2026 — v2026.11.08 (sources : commit 705245b + modifications non validées)

# Sommaire

Présentation de l'année . . . . . 4
Charte d'usage de l'intelligence artificielle . . . . . 10
Panorama de l'année . . . . . 12

## Cours et exercices

1 Les bases de la programmation Python . . . . . 17
Complément : dessiner avec la tortue . . . . . 66
2 Le binaire et l'écriture des nombres . . . . . 90
3 Les types construits . . . . . 137
4 Algorithmique : le parcours séquentiel . . . . . 177
5 Spécifier et mettre au point ses programmes . . . . . 209
6 Les algorithmes de tri . . . . . 247
7 La recherche dichotomique . . . . . 279
8 Les données en tables . . . . . 308
9 Architecture des ordinateurs et systèmes d'exploitation . . . . . 349
10 Le Web : réseaux et interactions homme-machine . . . . . 385
11 Les algorithmes gloutons . . . . . 443
12 Les $k$ plus proches voisins . . . . . 472
Problèmes de synthèse . . . . . 510

## Corrigés

1 Les bases de la programmation Python . . . . . 518
Complément : dessiner avec la tortue . . . . . 545
2 Le binaire et l'écriture des nombres . . . . . 552
3 Les types construits . . . . . 568
4 Algorithmique : le parcours séquentiel . . . . . 581
5 Spécifier et mettre au point ses programmes . . . . . 596
6 Les algorithmes de tri . . . . . 611
7 La recherche dichotomique . . . . . 626
8 Les données en tables . . . . . 639
9 Architecture des ordinateurs et systèmes d'exploitation . . . . . 655
10 Le Web : réseaux et interactions homme-machine . . . . . 668
11 Les algorithmes gloutons . . . . . 691

12 Les $k$ plus proches voisins. . . . . 702
Problèmes de synthèse. . . . . 711
Lexique . . . . . 720
Crédits iconographiques . . . . . 729

# Présentation de l'année

La NSI, ce n'est pas « apprendre à se servir d'un ordinateur » : c'est comprendre **comment il fonctionne** et apprendre à lui **donner des ordres**. Cette année, vous allez écrire vos propres programmes, découvrir ce qui se cache derrière une page Web, un réseau ou une image, et surtout apprendre à **raisonner comme un informaticien**. Bref, vous passez de l'autre côté de l'écran.

**L'esprit NSI**

- **Curiosité** : toujours se demander « mais comment ça marche, en vrai ? »
- **Rigueur** : la machine fait *exactement* ce qu'on lui dit, pas ce qu'on voudrait.
- **Persévérance** : un programme qui « bugue », c'est normal — un bug, ça se corrige, ça ne se subit pas.

## I La NSI, une discipline à part entière

La NSI est une **spécialité scientifique**, au même titre que les mathématiques ou la physique — avec la particularité de se pratiquer beaucoup **sur machine**. Elle s'appuie sur quelques grands piliers qui se répondent toute l'année :

- **programmer** — donner vie à une idée en la transformant en programme ;
- **l'algorithmique** — inventer des méthodes efficaces pour résoudre un problème ;
- **les données** — les représenter, les organiser, les exploiter ;
- **les machines, les réseaux et le Web** — comprendre comment tout cela fonctionne et communique.

En Première, la NSI, c'est **4 heures par semaine**. Une grande partie se passe devant un clavier : ici, on *apprend en faisant*. C'est aussi une discipline qui ouvre grand les portes — vers des études et des métiers très variés, bien au-delà de la seule informatique.

## II Ce que vous allez apprendre cette année

Les chapitres s'enchaînent dans cet ordre : chacun s'appuie sur les précédents. La programmation, elle, sera de toutes les séances.

## A. De la rentrée aux vacances de Toussaint

**Chapitre 1 — Les bases de la programmation Python.** Vous allez apprendre à écrire vos premiers programmes : ranger des informations dans des variables, faire faire des choix à la machine (si… alors…), lui faire répéter des actions, et fabriquer vos propres « fonctions ».

**Complément — Dessiner avec la tortue.** Juste après ce premier chapitre, vos programmes **dessinent** : une « tortue » avance et tourne à l'écran selon vos ordres ; boucles et fonctions suffisent pour tracer polygones, spirales et rosaces… puis pour lui faire écrire des mots entiers (projet « La tortue écrivain »).

**Chapitre 2 — Le binaire et l'écriture des nombres.** Vous allez découvrir comment une machine représente un nombre avec seulement des `0` et des `1`, pourquoi elle ne connaît que ça… et pourquoi, sur un ordinateur, $0{,}1 + 0{,}2$ ne fait pas tout à fait $0{,}3$ !

## B. De la Toussaint à Noël

**Chapitre 3 — Les types construits.** Vous allez apprendre à regrouper des données par paquets : des p-uplets figés, des tableaux (listes) de valeurs, et des « dictionnaires » qui associent une information à une étiquette.

**Chapitre 4 — Algorithmique : le parcours séquentiel.** Vous allez écrire des méthodes pour explorer des données : retrouver un élément, repérer le plus grand, calculer une moyenne… et vous demander si votre méthode est rapide.

## C. De janvier aux vacances d'hiver

**Chapitre 5 — Spécifier et mettre au point ses programmes.** Vous allez apprendre à écrire des programmes fiables : dire précisément ce que fait un programme, le protéger des erreurs, le **tester** et **traquer les bugs** — et vous verrez qu'il existe bien d'autres langages que Python.

**Chapitre 6 — Les algorithmes de tri.** *Vous allez apprendre à trier des données, comme on range un jeu de cartes, et à mesurer le travail que cela demande.*

**Chapitre 7 — La recherche dichotomique.** Vous allez retrouver une valeur très vite dans une liste triée, sans tout parcourir, en coupant la zone de recherche en deux à chaque étape.

**Chapitre 8 — Les données en tables.** Vous allez travailler sur de vraies données (des fichiers comme ceux d'un tableur) : les filtrer, les trier, croiser deux tables. Un avant-goût des bases de données.

## D. Des vacances d'hiver à celles de printemps

**Chapitre 9 — Architecture des ordinateurs et systèmes d'exploitation.** Vous allez ouvrir le capot : comment marche un ordinateur, avec son processeur et sa mémoire, et le rôle du système d'exploitation, que vous piloterez en ligne de commande.

**Chapitre 10 — Le Web : réseaux et interactions homme-machine.** Vous allez découvrir comment les machines s'échangent des informations à travers un réseau, puis comprendre ce qui se passe quand vous cliquez sur une page : ce que fait votre navigateur, ce que fait le serveur à l'autre bout, comment un formulaire envoie vos réponses et ce que le navigateur garde en mémoire (les cookies).

## E. Du printemps à la fin de l'année

**Chapitre 11 — Les algorithmes gloutons.** Vous allez résoudre des problèmes en faisant, à chaque étape, le choix qui semble le meilleur sur le moment : rendre la monnaie, remplir un sac au mieux…

**Chapitre 12 — Les $k$ plus proches voisins.** Une première intelligence artificielle. Vous allez programmer un algorithme qui « devine », c'est-à-dire qui classe un élément d'après ceux qui lui ressemblent le plus. C'est l'une des idées derrière l'IA d'aujourd'hui.

**Remarque**
Tout au long de l'année, on croisera les **grandes idées** et les **personnages** qui ont inventé l'informatique — de Ada Lovelace, qui écrivit le premier programme dès 1843, à celles et ceux qui ont bâti les machines et les réseaux d'aujourd'hui.

## III Comment on travaille

Nos **4 heures par semaine** se partagent entre :

- **du cours**, court et illustré, pour poser les idées et le vocabulaire ;
- **beaucoup de pratique sur machine** : on programme, on teste, on se trompe, on corrige — c'est là qu'on apprend vraiment ;
- **des projets** : environ **un tiers de l'année** y est consacré. Seul ou en équipe, vous concevez et réalisez un vrai programme, de l'idée jusqu'à la version qui marche.

### Nos projets de l'année

Un « projet », ici, ce n'est pas forcément un long travail en autonomie complète — vous débutez en informatique. Ce sont surtout de **vraies applications** de ce qu'on apprend en cours, que l'on construit **pas à pas, ensemble** :

- un **site web interactif** (HTML, CSS, JavaScript), qui prolonge le chapitre « Le Web » ;
- une **analyse de données réelles** : filtrer, trier et croiser un fichier CSV (les prénoms donnés en France), au chapitre « Données en tables » ;
- de la **reconnaissance de caractères** : apprendre à un programme à lire des chiffres manuscrits, puis une **plaque d'immatriculation**, au chapitre « $k$ plus proches voisins » ;
- un **Sudoku** : vérifier qu'une grille respecte les règles (et en fabriquer une), pour mettre en pratique matrices, boucles et méthode — au chapitre « types construits ».

Chacun montre qu'avec quelques notions bien comprises, on réalise déjà des choses **étonnamment concrètes**.

### Comment est organisé un chapitre

Chaque chapitre de ce livre suit le même parcours :

- **Activités préparatoires** : courtes, souvent sans ordinateur, à faire *avant* le cours pour se poser les bonnes questions ;
- **Cours** : les notions, des exemples corrigés et un bilan ;
- **Exercices** : du plus simple au plus exigeant, avec des coups de pouce en fin de rubrique ; environ un tiers sont marqués • essentiel (le minimum pour suivre) ; leurs corrigés sont en fin de livre ;
- **TP et projets** : un travail plus long sur machine qui réinvestit tout le chapitre ;
- **Fiche de révision** : l'essentiel du chapitre en deux pages, à relire avant un test ou un contrôle, avec « Se tester » (questions sur le chapitre), « Et avant ? » (questions sur les chapitres précédents, pour ne pas oublier) et, à partir du chapitre 2, les problèmes de synthèse que vous pouvez désormais traiter ;
- **Défis Advent of Code** : un défi par chapitre, une énigme de programmation tirée d'*Advent of Code* (`adventofcode.com`), un concours en ligne créé et animé depuis 2015 par le développeur **Eric Wastl**, qui publie chaque mois de décembre une série d'énigmes, chacune en deux parties. L'énoncé, en anglais, se lit sur le site (QR code sur chaque

défi ; le navigateur peut le traduire) ; le défi fournit un lexique, un résumé, un petit exemple pour se tester et des coups de pouce. Ces défis sont **facultatifs** et n'utilisent que des notions déjà vues dans l'année, ou présentées dans un encadré « Outil Python » du défi.

**Les défis n'ont volontairement pas de corrigé** dans ce livre : tout le plaisir d'une énigme est de la résoudre soi-même. Le site indique si votre réponse est juste (avec un compte gratuit) ; en cas de blocage, lisez les coups de pouce, puis venez en parler en classe.
En fin de livre, un **lexique** donne le sens des mots importants et des mots du langage Python, avec un lien vers leur définition dans le cours.

## Comment vous êtes évalué

- à chaque chapitre, de courts **tests rapides** (/10) pour vérifier les bases, puis un **contrôle** (/20) ;
- des **projets et travaux sur machine**, qui comptent aussi dans la moyenne.

Le plus important n'est pas de « tout savoir » tout de suite, mais de **progresser** et de comprendre *pourquoi* ça marche.

## L'intelligence artificielle : une charte, trois pastilles

Les assistants d'IA savent écrire du code et des explications ; le but de ce cours est que **vous** sachiez le faire, le vérifier et l'expliquer. Une **charte d'usage de l'IA**, reproduite juste après cette présentation, fixe ce qui est permis. L'usage est décidé **exercice par exercice** et signalé par une pastille à côté du titre :

-  **sans IA** : tests, contrôles, sujets d'épreuve, exercices d'automatisme (lecture de code, questions de cours) ;
-  **IA en appui** : pour déboguer, reformuler ou vérifier, à condition d'écrire soi-même la réponse finale ;
-  **IA intégrée** : l'exercice consiste à critiquer, tester ou améliorer une réponse d'assistant.

Dans tous les cas : vous restez responsable de ce que vous rendez, vous devez pouvoir l'expliquer à l'oral, et vous dites quand l'IA a été utilisée. La complétion automatique de code dans l'éditeur compte comme un assistant.

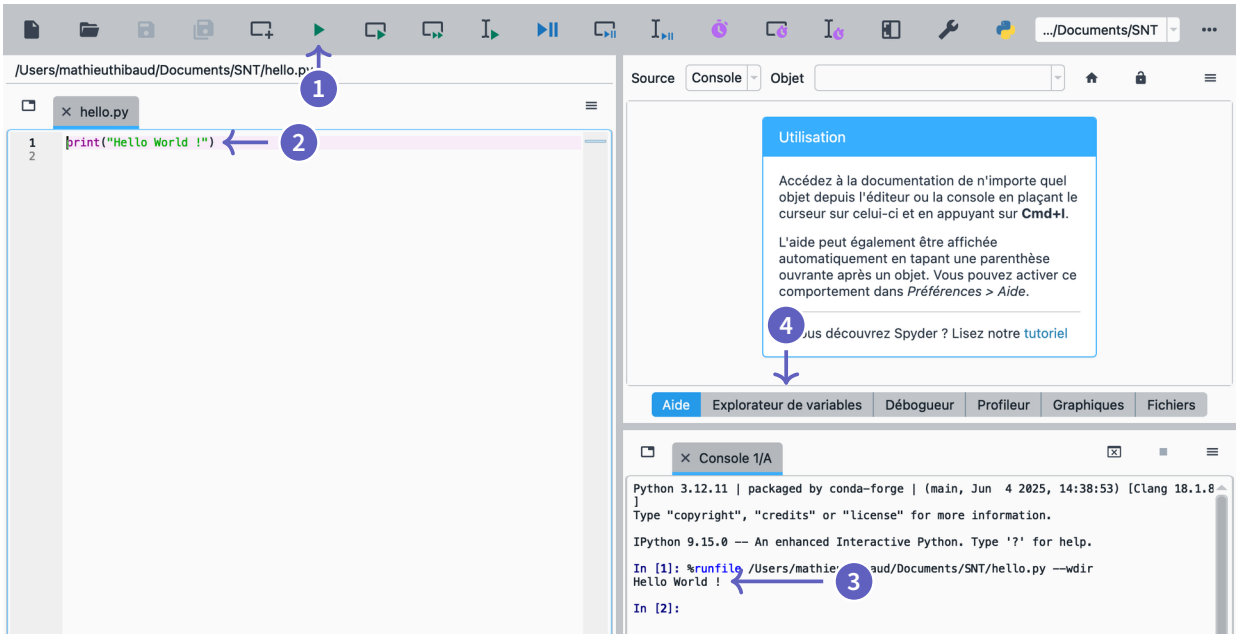
**Remarque** — *Une ressource pour réviser à la maison*
Deux sites tenus par des professeurs de NSI sont d'excellentes ressources, claires, complètes et conformes au programme. Je vous encourage à les consulter pour réviser ou approfondir :

- le site de **Gilles Lassus** : `glassus.github.io` ;
- le site de **David Roche** : `pixees.fr/informatiquelycee`.

# IV Travailler avec ce livre

## A. Avec quel outil ?

Au lycée, on programme avec **Spyder**, un logiciel libre et gratuit. Installez-le aussi chez vous : l'installateur officiel se télécharge sur `spyder-ide.org` et contient Python, il n'y a rien d'autre à installer. En dépannage (ordinateur prêté, installation impossible), **Basthon** fonctionne dans un navigateur, sans installation (`basthon.fr`) : parfait pour un exercice court, moins pratique pour un TP ou un défi qui lit des fichiers, qu'il faut alors déposer dans la page à chaque séance.




Les repères de la capture :

1 le bouton vert ▶ **Exécuter** (ou la touche F5) : il enregistre et lance tout le fichier ;
2 **l'éditeur** (à gauche) : on y écrit le programme, enregistré dans un fichier `.py` ;
3 **la console** (en bas à droite, `In [1]:`) : on y lit ce que le programme affiche, et on peut y taper une instruction pour tester une idée tout de suite, ou appeler une fonction du fichier une fois celui-ci exécuté. Ce qu'on tape dans la console n'est pas enregistré ;
4 l'onglet **Explorateur de variables** : après l'exécution, il montre la valeur de chaque variable (pratique pour comprendre un programme qui ne fait pas ce qu'on attend).

## B. Une méthode pour chaque chapitre

1. **Après chaque séance** (20 à 30 minutes) : relire le passage du cours en *retapant* les exemples dans Spyder, puis faire les exercices • essentiel indiqués par les renvois « Exercices d'application ».
2. **Bloqué ?** Chercher vraiment (5 à 10 minutes), relire le message d'erreur s'il y en a un, puis lire le **coup de pouce** (▷) : d'abord la piste, ensuite seulement le début de solution.
3. **Le corrigé** se lit *après* avoir essayé : comparer, repérer son erreur (les corrigés signalent les *erreurs fréquentes*), puis refaire l'exercice sans regarder quelques jours plus tard.
4. **Deux ou trois jours après** : répondre de mémoire aux questions « Se tester » et « Et avant ? » de la fiche de révision, puis vérifier avec le lien « ▶ réponses ». Revenir régulièrement sur les chapitres passés, c'est ce qui fait tenir les connaissances.
5. **Avant un test** : la fiche de révision et sa liste « Savoir-faire » ; pour chaque case que vous ne pouvez pas cocher, refaire un exercice essentiel.

Les exercices ★★★, les défis et les encadrés « Pour les curieux » servent à aller plus loin : utiles, mais facultatifs.

## C. Lire ce livre sur un ordinateur

- **Tout est cliquable** : le sommaire, le petit sommaire de chaque chapitre, les liens « ▶ corrigé p. . . . » et « ↑ énoncé », les coups de pouce, les numéros d'exercices des renvois et les mentions « chapitre 3 ». L'**onglet de couleur** au bord de la page ramène à la page de titre du chapitre ; le **numéro de page** ramène au sommaire.
- **Les signets** (panneau latéral « Signets » ou « Table des matières » du lecteur PDF) donnent le plan détaillé : chapitre, rubrique, section.
- **Revenir où l'on était** après un lien : le lien « ↑ énoncé » ou l'onglet du chapitre, ou bien la commande « page précédente » du lecteur (Alt + ← dans Acrobat Reader sous Windows, Cmd + [ dans Aperçu sur Mac).
- **Copier un programme** : cliquer sur l'icône ⧉ en haut à droite d'un bloc de code ; une page s'ouvre avec le programme prêt à être copié. Un copier-coller direct depuis le PDF emporte souvent les numéros de ligne et l'indentation est perdue.
- **Chercher un mot** : Ctrl + F (Cmd + F sur Mac).
- Pour lire confortablement, choisir l'affichage « ajuster à la largeur ». Les numéros de page du lecteur sont ceux du livre imprimé.

*Prêt(e)s ? On passe de l'autre côté de l'écran. Bienvenue en NSI !*

# Charte d'usage de l'intelligence artificielle

Les assistants d'IA écrivent du code et des explications, souvent bien, parfois faux, toujours avec assurance. Le but de ce cours n'est pas d'obtenir une réponse : c'est que **vous** sachiez la produire, la vérifier et l'expliquer. L'usage de l'IA n'est donc ni interdit partout ni permis partout : il est **décidé exercice par exercice**, et indiqué par une pastille à côté du titre.

### Ce qui compte comme « IA » dans cette charte

- Les **assistants conversationnels** (chatbots), quel que soit le site ou l'application.
- La **complétion automatique de code** dans l'éditeur (par exemple Copilot dans VS Code) : elle compte comme un assistant, même si l'on ne « pose pas de question ».
- Les **résumés, traductions et réponses générées** affichés en tête des moteurs de recherche.
- Ne comptent pas : le cours, le manuel, la documentation officielle, un moteur de recherche classique, le correcteur orthographique.

## Les trois pastilles

| Pastille | Vous pouvez | Vous ne pouvez pas |
|---|---|---|
|  tests, contrôles, devoirs sur table, exercices d'automatisme | Utiliser le cours et vos notes quand ils sont autorisés ; vos propres essais. | Utiliser un assistant ou la complétion de code, **ni avant ni pendant**. Pour un travail à la maison, s'en servir revient à se priver de l'entraînement que le test mesure. |
|  feuilles d'exercices, préparation à la maison (pastille par défaut) | Faire expliquer un message d'erreur ; reformuler un énoncé ; demander un exemple ; **vérifier** une réponse que vous avez déjà écrite. | Faire écrire la réponse ou le code à votre place, puis le recopier. La réponse finale est **écrite par vous** et vous savez l'expliquer. |
|  exercices conçus avec l'IA | Tout, y compris générer une réponse : l'exercice consiste à la **critiquer**, la **comparer**, la **tester** ou l'**améliorer**. | Rendre la réponse de l'assistant telle quelle : ce qui est évalué, c'est **votre analyse**. |

### Trois règles, quelle que soit la pastille

1. **Vous êtes responsable de ce que vous rendez.** Une erreur de l'assistant est votre erreur.
2. **Vous devez pouvoir expliquer chaque ligne rendue**, à l'oral, sans préparation. Ce que vous ne savez pas expliquer n'est pas votre travail.
3. **Vous dites quand l'IA a été utilisée**, en une ligne en tête du travail : « IA utilisée pour : … ». Le dire n'est jamais pénalisé ; le cacher l'est.

### Pourquoi ces règles

- Un assistant répond avec la même assurance quand il a raison et quand il a tort. Savoir **vérifier** une réponse (la tester, la tracer, la comparer au cours) est précisément une compétence de ce cours : c'est le sens des exercices  .
- Les évaluations, puis les épreuves du bac en Terminale, se passent **sans aucun outil**. S'entraîner avec une aide que vous n'aurez pas le jour J, c'est se préparer à une autre épreuve que la vôtre.
- Sur la plateforme d'entraînement de la classe, les outils d'IA sont réglés par le professeur et suivent la pastille de l'activité.

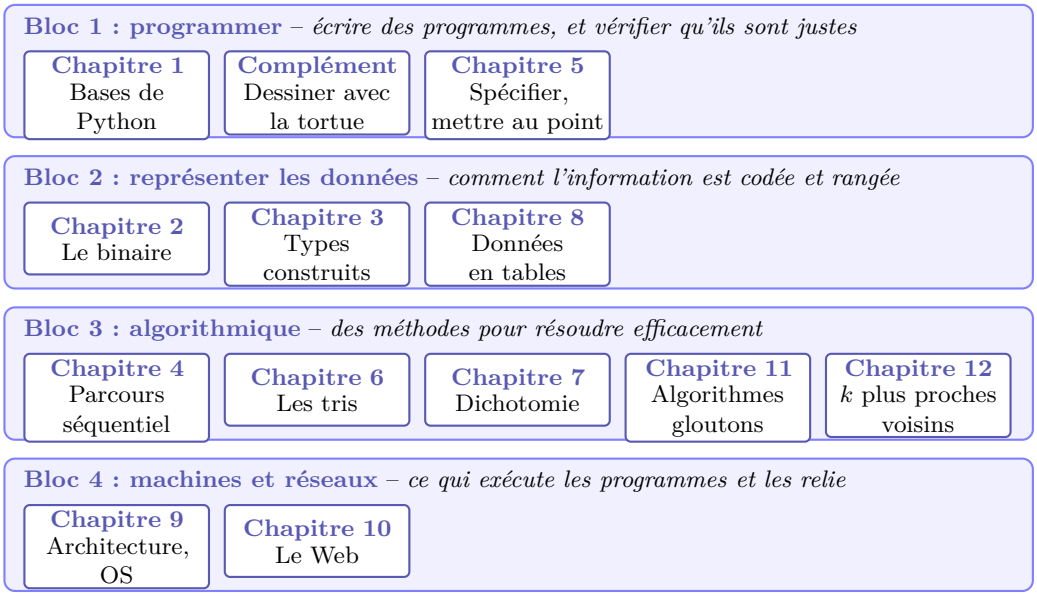
En cas de doute sur ce qui est permis pour un exercice, la règle est simple : **demandez avant**, pas après.

# Panorama de l'année

Les douze chapitres de l'année ne sont pas indépendants : les mêmes outils (la boucle, le tableau, le parcours) reviennent d'un chapitre à l'autre, et deux questions traversent toute l'année : « **mon programme est-il correct ?** » et « **combien coûte-t-il ?** ». Ces pages donnent une vue d'ensemble, à relire à chaque nouveau chapitre. Elles montrent aussi ce que chaque chapitre prépare pour la Terminale.

## Le plan de l'année en quatre blocs

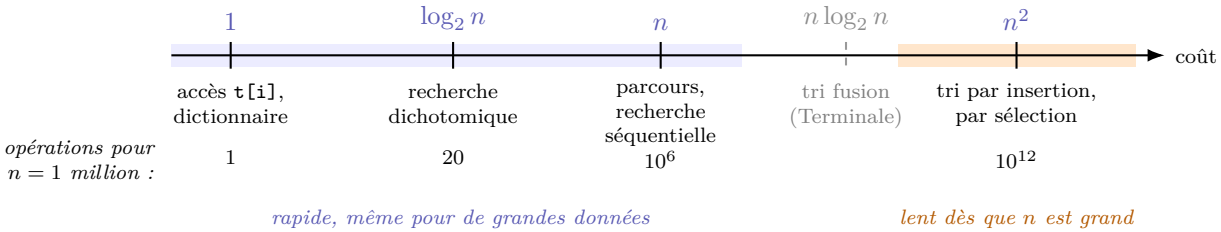
Le programme se partage en quatre grands blocs. Dans chaque bande, on lit les chapitres du bloc ; leur **numéro** indique l'ordre dans lequel on les étudie au fil de l'année.




On pose d'abord les bases : programmer, coder l'information, ranger les données (chapitres 1 à 3). On alterne ensuite **algorithmes** et **données**, on découvre la **machine** et le **réseau** (chapitres 9 et 10), et l'année se termine par deux méthodes d'algorithmique plus ambitieuses : les algorithmes **gloutons** et les $k$ **plus proches voisins**.

## Les deux fils rouges : correction et coût

Un bon programme doit être **correct** (il fait ce qu'on attend) et **efficace** (il ne demande pas trop d'opérations quand les données sont nombreuses). Pour le coût, on range les algorithmes sur une **échelle** ($n$ désigne la taille des données) :




Chaque chapitre apporte des outils pour répondre à l'une des deux questions :

| Outil | L'idée | Chapitre |
|---|---|---|
| Spécifier et tester | Écrire ce que la fonction doit faire (préconditions, post-conditions) et le vérifier par des **assertions** et des **jeux de tests**. Un test réussi ne prouve pas que le programme est juste. | 1 Bases de Python, 5 Mise au point |
| Invariant de boucle | Une propriété vraie à chaque tour de boucle, qui explique pourquoi le résultat final est le bon. | 4 Parcours, 6 Tris |
| Variant de boucle | Une quantité entière, positive, qui diminue strictement à chaque tour : elle prouve que la boucle **se termine**. | 1 Bases de Python, 7 Dichotomie |
| Compter les opérations | Un parcours coûte de l'ordre de $n$, un tri par insertion de l'ordre de $n^2$, une recherche dichotomique de l'ordre de $\log_2 n$. | 4 Parcours, 6 Tris, 7 Dichotomie |
| Accepter un compromis | Un algorithme **glouton** est rapide mais ne donne pas toujours la meilleure solution. | 11 Gloutons |

**Remarque**

Le $\log_2 n$ n'est qu'un **outil de comptage** : c'est le nombre de fois où l'on peut couper $n$ en deux avant d'arriver à 1 (à peu près le nombre de bits de $n$, vu au chapitre 2). Pour un million de données, $\log_2 n \approx 20$.

## Chapitre par chapitre

Pour chaque chapitre : ce qu'il **réutilise**, et où il **resservira**, cette année ou en Terminale (signalé par « Tle »).

| **Chapitre** | **S'appuie sur...** | **Resservira dans...** |
|---|---|---|
| 1 Bases de Python | aucun prérequis | tous les chapitres ; mise au point (5) ; Tle : calculabilité (terminaison) |
| Complément : la tortue | boucles et fonctions (1) | Tle : récursivité (dessins fractals) |
| 2 Le binaire | opérateurs `//`, `%`, booléens (1) | architecture (9) ; Tle : réseaux (adresses IP), cryptographie |
| 3 Types construits | bases de Python (1) | parcours, tris, dichotomie (4, 6, 7) ; données en tables (8) ; Tle : diviser pour régner (image dans une matrice) |
| 4 Parcours séquentiel | tableaux (3) | tris, dichotomie, données en tables, gloutons, $k$ plus proches voisins ; Tle : programmation dynamique |
| 5 Spécifier, mettre au point | assertions et tests (1) ; invariant (4) | tris et dichotomie (6, 7) ; Tle : calculabilité |
| 6 Les tris | parcours : patron, invariant, coût (4) | dichotomie (7) ; trier une table (8) ; $k$ plus proches voisins (12) ; Tle : tri fusion, arbres binaires de recherche |
| 7 Dichotomie | recherche séquentielle et coût (4) ; tableau trié (6) | Tle : arbres binaires de recherche |
| 8 Données en tables | listes et dictionnaires (3) ; parcours (4) | $k$ plus proches voisins (12) ; Tle : bases de données et SQL |
| 9 Architecture, OS | binaire (2) | le Web (10) ; Tle : processus, systèmes sur puce |
| 10 Le Web | architecture (9) ; fonctions (1) | Tle : bases de données (côté serveur), cryptographie (HTTPS) |
| 11 Algorithmes gloutons | parcours, coût, invariant du champion (4) | Tle : programmation dynamique, graphes (Dijkstra) |
| 12 $k$ plus proches voisins | parcours (4), tris (6), données en tables (8) | projet de reconnaissance de caractères ; Tle : sujet possible de Grand Oral (intelligence artificielle) |

**Remarque**

Pour ceux qui continuent la NSI en Terminale, la colonne de droite le montre : chaque chapitre de Première y est réutilisé.

## Une histoire de l'informatique en quelques dates

Chaque date de cette frise correspond à une notion que l'on étudie cette année, dans le chapitre indiqué à droite. Beaucoup sont plus anciennes que les ordinateurs (l'algorithme, le binaire, le premier programme) !


**IX^e s.**
al-Khwârizmî


**1703**
Leibniz


**1843**
Lovelace


**1890**
Hollerith


**1945**
von Neumann


**1947**
le « bug »


**1951**
plus proche voisin


**1967**
la tortue

### Avant les ordinateurs

| | | |
|---|---|---|
| IX^e siècle | Le nom d'**al-Khwârizmî** donne le mot « **algorithme** » | ch. 4 |
| 1703 | **Leibniz** décrit l'**arithmétique binaire** | ch. 2 |
| 1843 | **Ada Lovelace** écrit le **premier programme** (la note G) | ch. 1 |
| 1890 | **Hollerith** : une carte perforée par personne, et des machines à **trier** | ch. 6, 8 |

### Les premiers ordinateurs

| | | |
|---|---|---|
| 1937 | **Shannon** : des circuits fondés sur les **booléens** | ch. 2 |
| 1945 | **von Neumann** décrit l'**architecture** des ordinateurs | ch. 9 |
| 1946 | La **recherche dichotomique** est décrite pour la première fois | ch. 7 |
| 1947 | Le premier « **bug** » : un papillon dans le calculateur Mark II | ch. 5 |
| 1951 | **Fix** et **Hodges** formalisent la règle du **plus proche voisin** | ch. 12 |
| 1952 | **Huffman** invente un **algorithme glouton** de compression | ch. 11 |
| 1953 | **Luhn** invente le **hachage**, au cœur des dictionnaires | ch. 3 |

### L'informatique pour tous

| | | |
|---|---|---|
| 1965 | **Moore** : le nombre de transistors d'une puce double sans cesse | ch. 9 |
| 1967 | Le langage **Logo** et sa **tortue** (Papert et Feurzeig) | compl. |
| 1989 | **Berners-Lee** invente le **Web** au CERN | ch. 10 |
| 1991 | **van Rossum** crée le langage **Python** | ch. 1 |

PREMIÈRE PARTIE

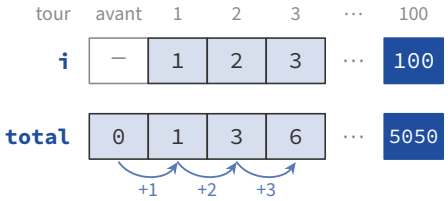
# Cours et exercices

CHAPITRE 1

# Les bases de la programmation Python

```
total = 0
for i in range(1, 101):
    total = total + i
print(total)

5050
```




la trace : à chaque tour, `total` reçoit `total + i`

DANS CE CHAPITRE


- Activités préparatoires . . . . . 18
- Cours . . . . . 20
- Exercices . . . . . 39
- TP et projets . . . . . 53
- Fiche de révision . . . . . 59
- Défi Advent of Code . . . . . 61
- Corrigés . . . . . 518

1

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 L'ordinateur humain

▶ corrigé p. 518

*exécuter un programme à la main, avant de savoir programmer*

Durée : 20 min · Sans ordinateur, crayon et gomme en main, par deux

### Consignes

- Les réponses s'écrivent sur le cahier.
- Vous êtes l'ordinateur. Vous lisez le programme **une ligne à la fois, de haut en bas**, et vous faites exactement ce qui est écrit, sans rien deviner.
- Chaque nom (`a`, `b`, `total`…) désigne une **boîte**, que l'on dessine sur le cahier. Une boîte ne contient qu'**une seule valeur à la fois** : pour en écrire une nouvelle, on gomme l'ancienne.
- `print(...)` signifie : « écrire à l'écran ce qu'il y a entre les parenthèses ». Ici, l'écran, c'est la ligne « Affichage obtenu : … » que vous écrivez sur le cahier.

### Exercice 1 *Des boîtes et une flèche*

Programme 1 :

```
1 a = 7
2 b = a + 3
3 a = 2
4 print(a, b)
```

1. Dessiner sur le cahier deux boîtes `a` et `b`, puis exécuter le programme ligne par ligne en les remplissant (gommer quand une valeur change).
2. Quel est l'affichage obtenu ?
3. À la ligne 3, la boîte `a` change. La boîte `b` a-t-elle changé elle aussi ? Pourquoi ?

Programme 2 :

```
1 n = 5
2 n = n + 1
3 print(n)
```

4. En mathématiques, l'égalité $n = n + 1$ est impossible. Pourtant l'ordinateur l'exécute sans erreur. Qu'affiche-t-il ?
5. Pour exécuter la ligne 2, par quel côté du signe `=` avez-vous commencé : la gauche ou la droite ? Décrire en une phrase ce que fait une ligne de la forme `nom = calcul`.

### Exercice 2 *Échanger deux boîtes*

Programme 3 (il est censé échanger les contenus de `a` et de `b`) :

```
1 a = 1
2 b = 2
3 a = b
4 b = a
5 print(a, b)
```

**6.** Exécuter le programme en dessinant les boîtes `a` et `b` sur le cahier. Quel est l'affichage obtenu ? L'échange a-t-il réussi ?

**7.** Où la valeur `1` est-elle passée ? Proposer une correction du programme, en vous autorisant une troisième boîte `c`.

## Exercice 3 *Répéter sans réécrire*

Nouvelle règle : la ligne `for i in range(1, 5):` signifie « pour `i` valant successivement `1`, `2`, `3`, puis `4`, exécuter les lignes **décalées** juste en dessous ». Chaque passage s'appelle un **tour**.

Programme 4 :

```
1 total = 0
2 for i in range(1, 5):
3     total = total + i
4 print(total)
```

**8.** Recopier et compléter sur le cahier le tableau suivant : valeurs des boîtes à la fin de chaque tour. Donner ensuite l'affichage obtenu.

| | avant la boucle | tour 1 | tour 2 | tour 3 | tour 4 |
|---|---|---|---|---|---|
| `i` | — | | | | |
| `total` | 0 | | | | |

**9.** Combien de fois la ligne `print(total)` a-t-elle été exécutée ? Et la ligne `total = total + i` ?

**10.** On décale la dernière ligne vers la droite, au même niveau que `total = total + i`. Que devient l'affichage ?

**11.** Quelle unique modification faut-il apporter au programme 4 pour calculer $1 + 2 + \cdots + 100$ ?

## Exercice 4 *Ce qu'on a découvert*

**12.** Recopier et compléter sur le cahier avec vos mots.

- Un nom comme `total` désigne ...
- Pour exécuter `nom = calcul`, l'ordinateur ...
- Les lignes décalées sous un `for` ...

1

# COURS

**Programme (BO)** *« Mettre en évidence un corpus de constructions élémentaires. »* Ces constructions sont : *« séquences, affectation, conditionnelles, boucles bornées, boucles non bornées, appels de fonction »*. On y ajoute la *spécification* d'une fonction (prototype, préconditions, postconditions) et l'usage de *jeux de tests*.

**Prérequis** aucun. Il suffit de savoir ouvrir une console ou un éditeur Python en ligne.

**Objectifs** *lire*, *écrire* et *exécuter* un programme simple ; maîtriser les six constructions élémentaires ; *spécifier* et *tester* une fonction. C'est le socle de toute l'année (et de toute la suite).

**Situation de départ**

1. Comment faire échanger leurs contenus à deux « boîtes » d'un programme sans perdre aucune des deux valeurs ?
2. Comment faire calculer $1 + 2 + \cdots + 100$ à un ordinateur sans écrire soi-même les cent nombres ?
3. Un programme qui répète une action peut-il ne jamais s'arrêter ? Comment être sûr qu'il s'arrêtera ?

## I Programmer, c'est quoi ?

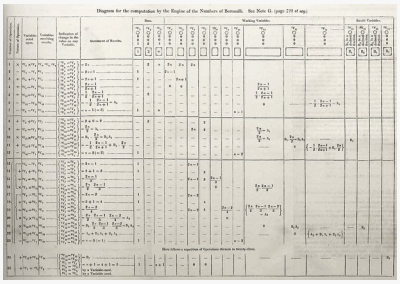
Un **programme** est une suite d'instructions, écrites dans un **langage de programmation**, qu'un ordinateur exécute pour accomplir une tâche. L'idée est plus ancienne que les ordinateurs eux-mêmes : dès 1843, la mathématicienne **Ada Lovelace** rédige ce que l'on considère comme le premier programme de l'histoire, destiné à une machine qui ne sera jamais construite. Le mot **algorithme**, lui, vient du nom du savant persan **al-Khwârizmî** (IX$^{e}$ siècle).

**Pour les curieux** — *Un peu d'histoire* (facultatif)

**Ada Lovelace** (1815–1852), fille du poète Byron, se passionne pour la *machine analytique* imaginée par l'Anglais **Charles Babbage** : une calculatrice mécanique programmable par cartes perforées. En 1843, elle traduit un article sur cette machine et l'enrichit de notes trois fois plus longues que le texte. La dernière, la **note G**, détaille pas à pas comment la machine calculerait les *nombres de Bernoulli*, avec des opérations répétées : c'est le premier programme publié. Mille ans plus tôt, vers 820, **al-Khwârizmî**, savant de la « Maison de la sagesse » de Bagdad, écrit un traité de calcul dont le titre a donné le mot *algèbre* (*al-jabr*), et un autre sur les chiffres indiens dont la traduction latine, *Algoritmi de numero Indorum*, a donné le mot *algorithme*.



*Ada Lovelace (portrait par A. E. Chalon, v. 1840)*



*Le tableau de la note G (1843)*



*L'*Algèbre *d'al-Khwârizmî (copie de 1342, Oxford)*

**Définition 1 — Algorithme et programme**

Un **algorithme** est une méthode : une suite finie et non ambiguë d'étapes qui, à partir de données, produit un résultat. Un **programme** est la *traduction* d'un algorithme dans un langage que la machine peut exécuter.

Nous utiliserons le langage **Python**, créé en 1991 par le Néerlandais **Guido van Rossum**. Il est aujourd'hui l'un des langages les plus enseignés et les plus utilisés au monde, pour sa lisibilité.

**Pour les curieux** — *Pourquoi « Python » ?* (facultatif)
Rien à voir avec le serpent ! Guido van Rossum était fan de la troupe comique britannique *Monty Python*. C'est pourquoi la documentation officielle fourmille encore de références à leurs sketchs (et de `spam`, `eggs` et autres `ham` en guise d'exemples).

## A. La première construction : la séquence

Quand on écrit plusieurs instructions les unes sous les autres, Python les exécute **dans l'ordre**, de haut en bas, une à la fois. C'est la construction la plus simple : la **séquence**.

**Exemple** — *Testez ce programme dans une console ou un éditeur en ligne*
▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 print("Bonjour")
2 print("et")
3 print("au revoir")
```

▶ **Correction.** Les trois lignes s'affichent dans l'ordre où elles sont écrites :

```
Bonjour
et
au revoir
```

**Règle 1 — L'ordre compte**

Dans une séquence, l'ordre des instructions détermine le résultat. Échanger deux lignes peut tout changer : la machine ne « devine » pas votre intention, elle applique vos instructions à la lettre.

**Remarque**
`print(...)` est un **appel de fonction** : on demande à une fonction déjà écrite (`print`) d'*afficher* ce qu'on lui donne entre parenthèses. Nous reviendrons longuement sur les fonctions.

1

# II Variables et affectation

## A. À quoi sert une variable ?

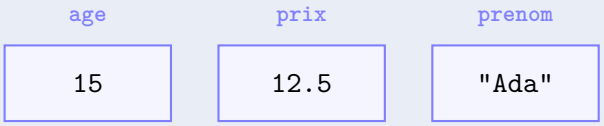
Un programme manipule des **données** : un âge, un prix, un nom, une note… Pour les conserver et les réutiliser, on leur donne un **nom**. Ce nom, associé à une valeur, s'appelle une **variable**.

```
age = 15
prix = 12.5
prenom = "Ada"
```

**Définition 2 — Variable**

Une **variable** est un nom qui désigne un emplacement de la mémoire où est rangée une valeur. On peut *lire* cette valeur (l'utiliser) et l'*écrire* (la modifier).

L'image classique est celle du **casier étiqueté** : le nom de la variable est l'étiquette collée sur le casier, la valeur est ce qui se trouve à l'intérieur. On ne s'intéresse jamais à *l'adresse exacte* du casier dans la mémoire ; l'étiquette suffit.




*Trois casiers en mémoire : l'étiquette (le nom) désigne le contenu (la valeur).*

Ce modèle du casier est **provisoire** : il suffit pour tout ce chapitre. Le chapitre *Les types construits* le précisera : on y verra qu'une variable est plutôt une **étiquette** attachée à une valeur rangée ailleurs en mémoire.

## B. L'affectation : une histoire en deux temps

L'instruction `age = 15` s'appelle une **affectation**. Le symbole `=` ne veut **pas** dire « égal » au sens mathématique : il veut dire « **prend pour valeur** », ou « reçoit ». On lit `age = 15` comme « *age reçoit 15* ».

**Règle 2 — Le mécanisme de l'affectation**

Une affectation `nom = expression` se déroule toujours en **deux temps** :

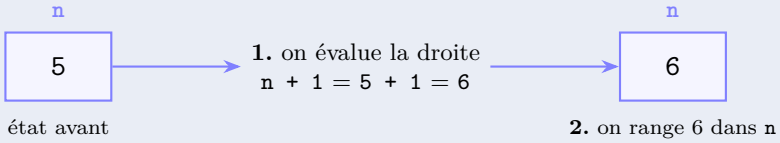
1. Python **évalue d'abord la partie droite** (il calcule la valeur de l'expression) ;
2. puis il **range cette valeur** dans la variable nommée à gauche.

On lit et on calcule à *droite*, on range à *gauche*.

↻ **Retour sur l'activité.** Dans le programme 2 de l'activité « L'ordinateur humain », vous avez déjà commencé par la droite : calculer `n + 1` avec l'ancienne valeur `5`, puis gommer la boîte `n` pour y écrire `6`.

Ce mécanisme éclaire une ligne qui, sinon, semble absurde en mathématiques :

```
n = 5
n = n + 1
```



La valeur de `n` passe donc de `5` à `6`. Cette écriture, très fréquente, s'appelle un **incrément**.

**Exemple** — *Le grand classique : échanger le contenu de deux variables. Prévoyez l'affichage*

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
a = 1
b = 2
a = b
b = a
print(a, b)
```

▶ **Correction.** Beaucoup attendent `2 1`… mais le programme affiche `2 2` ! À la ligne `a = b`, `a` vaut désormais `2` ; la valeur `1` est **perdue**. La ligne suivante range donc `2` (la valeur actuelle de `a`) dans `b`.

**Règle 3 — Échanger deux variables**

Pour ne pas perdre une valeur, on utilise une **variable temporaire** :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
a = 1
b = 2
temp = a      # on met a l'abri la valeur de a
a = b
b = temp
print(a, b)   # affiche : 2 1
```

↻ **Retour sur l'activité.** Le programme 3 de l'activité affichait `2 2` pour cette raison ; la troisième boîte `c` que vous avez proposée pour le corriger joue exactement le rôle de `temp`.

**Remarque**

Avec ce que l'on sait pour l'instant, la variable temporaire est **indispensable** : sans elle, la première affectation `a = b` écraserait la valeur de `a`, alors définitivement perdue. C'est l'image des deux verres, l'un d'eau, l'autre de jus : pour intervertir leurs contenus, il faut un **troisième verre vide**. On verra au chapitre *Les types construits* une écriture plus courte, propre à Python.

▶ **Exercices d'application :** exercices 1 et 2 (lire une trace, échanger deux variables)

## C. Les types de données de base

Chaque valeur a un **type**, qui décrit sa nature et les opérations permises. En classe de première, quatre types de base suffisent :

| Type | Nature | Exemples |
|---|---|---|
| `int` | entier | `15`, `-3`, `0` |
| `float` | nombre à virgule (« flottant ») | `12.5`, `-0.1`, `3.0` |
| `str` | chaîne de caractères (texte) | `"Ada"`, `'bonjour'` |
| `bool` | booléen (vrai/faux) | `True`, `False` |

La fonction `type(...)` renvoie le type d'une valeur :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
type(15)       # <class 'int'>
type(12.5)     # <class 'float'>
type("Ada")    # <class 'str'>
type(3 == 3)   # <class 'bool'>
```

Sur les nombres, Python dispose des opérations usuelles, et de deux opérations propres à la division **entière** :

| `+` | `-` | `*` | `/` | `//` | `%` | `**` |
|---|---|---|---|---|---|---|
| addition | soustraction | multiplication | division | quotient entier | reste | puissance |
| `7 + 2` → `9` | `7 - 2` → `5` | `7 * 2` → `14` | `7 / 2` → `3.5` | `7 // 2` → `3` | `7 % 2` → `1` | `7 ** 2` → `49` |

**Pour les curieux** — *Typage dynamique* (facultatif)
Python est à **typage dynamique** : on ne déclare pas le type d'une variable, il est déduit de la valeur rangée, et il peut changer au fil du programme. Pratique, mais il faut rester vigilant : additionner un `int` et un `str` provoque une erreur.

**Remarque** — *Deux pièges célèbres à connaître*

- **Les entiers de Python sont de taille arbitraire.** Contrairement à beaucoup de langages, Python calcule `2 ** 1000` sans broncher : la seule limite est la mémoire de la machine.
- **Les flottants sont approximatifs.** Testez `0.1 + 0.2` : Python répond `0.30000000000000004` ! Un nombre à virgule est stocké en binaire de manière approchée. **On n'écrit donc jamais `if x == 0.3` sur des flottants** ; on teste plutôt si l'écart est très petit.

## D. Lire une entrée au clavier et convertir

La fonction `input(...)` affiche un message, attend que l'utilisateur tape quelque chose au clavier puis valide, et **renvoie ce qui a été tapé**, **toujours sous forme de chaîne** (`str`), même si l'on a tapé des chiffres. Pour calculer avec, il faut donc **convertir** la valeur dans le bon type :

| Conversion | Rôle | Exemple |
|---|---|---|
| `int(...)` | vers un entier | `int("17")` → `17` |
| `float(...)` | vers un flottant | `float("12.5")` → `12.5` |
| `str(...)` | vers une chaîne | `str(17)` → `"17"` |

```
1 age = int(input("Votre age ? "))   # on tape 16 : age vaut 16
2 print("Dans un an, vous aurez", age + 1, "ans")
```

**Remarque**
Sans `int`, `age` vaudrait la chaîne `"16"`, et `age + 1` provoquerait une erreur : on ne peut pas additionner un `str` et un `int`. De même, `int("deux")` provoque une erreur : la chaîne doit représenter un nombre.

## E. Premier contact avec les chaînes de caractères

Une chaîne est une **suite de caractères**, numérotés à partir de `0`. Voici les opérations dont on se sert dès ce chapitre ; le chapitre *Les types construits* les reprendra en détail.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> mot = "radar"
>>> len(mot)            # nombre de caracteres
5
>>> mot[0]              # caractere en position 0 : le premier
'r'
>>> mot[-1]             # le dernier caractere
'r'
>>> "d" in mot          # ce caractere est-il dans la chaine ?
True
>>> "Ada" + " Lovelace" # concatenation : on met bout a bout
'Ada Lovelace'
>>> "*" * 3             # repetition
'***'
```

Si `mot` a `n` caractères, les positions vont de `0` à `n - 1` : `mot[n]` provoque une erreur.

**Remarque** — *Compter depuis la fin : les positions négatives*
Python accepte aussi des positions **négatives**, qui se comptent **à partir de la fin** : `mot[-1]` est le dernier caractère, `mot[-2]` l'avant-dernier, etc. Ainsi `mot[-1]` est un raccourci pour `mot[len(mot) - 1]`.

| caractère | `r` | `a` | `d` | `a` | `r` |
|---|---|---|---|---|---|
| position | 0 | 1 | 2 | 3 | 4 |
| position négative | -5 | -4 | -3 | -2 | -1 |

Sur un mot vide (`""`), `mot[-1]` provoque une erreur, comme `mot[0]` : il n'y a pas de dernier caractère.

On verra avec la boucle `for` comment parcourir une chaîne caractère par caractère.

## F. Bien nommer ses variables

Un programme se lit bien plus souvent qu'il ne s'écrit. Le choix des noms est donc essentiel.

### Règle 4 — Règles et conventions de nommage

- un nom commence par une lettre ou `_`, puis lettres, chiffres, `_` ; **pas d'espace**, pas d'accent conseillé ;
- Python distingue les majuscules : `note` et `Note` sont deux variables différentes ;
- par convention, on écrit en `minuscules_avec_underscores` (style *snake_case*) ;
- on choisit des noms **parlants** : `prix_total` plutôt que `x` ou `truc` ;
- on ne peut pas utiliser un **mot réservé** du langage (`if`, `for`, `while`, `def`, `return`…).

### Écrire une fonction : le gabarit

Dès les premiers exercices, on vous demande d'« écrire une fonction ». Pour l'instant, utilisez-la comme une **boîte noire** : on lui donne des valeurs (les **paramètres**), elle **renvoie** un résultat. Il suffit de recopier ce gabarit ; la section *Les fonctions* l'approfondira.

```
1 def nom_de_la_fonction(parametre1, parametre2):
2     resultat = ...      # le calcul, avec les parametres
3     return resultat     # la valeur renvoyee
```

1

- la ligne `def` se termine par un **deux-points** `:` ; les lignes suivantes sont **décalées** de 4 espaces ;
- `return` indique la valeur que la fonction renvoie ;
- certaines fonctions des exercices **affichent** (avec `print`) au lieu de renvoyer : on les appelle de la même façon.

**Exemple** — *Le prix d'un achat*
Écrire une fonction `prix_total(prix, quantite)` qui renvoie le prix à payer pour `quantite` articles à `prix` euros pièce.

▶ **Correction.** On suit le gabarit :

```
1 def prix_total(prix, quantite):
2     total = prix * quantite
3     return total
```

On exécute d'abord le fichier : la définition seule n'affiche **rien**. Puis on **appelle** la fonction dans la console, avec des valeurs dont on connaît le résultat, pour la **tester** :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> prix_total(2.5, 4)
10.0
>>> prix_total(3, 0)
0
```

▶ **Exercices d'application :** exercices 3 à 5 (`//` et `%` ; les fonctions s'écrivent avec le gabarit ci-dessus) et 40, 41 (entraînement : `int(input(...))`, longueur et indices d'une chaîne)

## III Les conditionnelles : `if`, `elif`, `else`

Jusqu'ici, tout s'exécutait ligne après ligne. Or un programme doit souvent **choisir** entre plusieurs comportements selon les données. C'est le rôle de la construction **conditionnelle**.

### A. Booléens et conditions

Une **condition** est une expression dont la valeur est un booléen : `True` ou `False`. On la construit avec des **opérateurs de comparaison** :

| `==` | `!=` | `<` | `<=` | `>` | `>=` |
|---|---|---|---|---|---|
| égal | différent | inférieur | inf. ou égal | supérieur | sup. ou égal |

**Règle 5 — `=` n'est pas `==`**

**`=` affecte** une valeur (« reçoit ») ; **`==` teste l'égalité** (« est-il égal à ? ») et renvoie un booléen. Les confondre est l'une des erreurs les plus fréquentes du débutant.

On combine plusieurs conditions avec les **opérateurs booléens** `and`, `or`, `not` :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 age = 16
2 age >= 13 and age <= 17     # True : un adolescent
3 not (age == 18)             # True
```

**À retenir — Évaluation « paresseuse » de `and` et `or`**

Les opérateurs `and` et `or` sont **séquentiels** : Python évalue les conditions *de gauche à droite* et s'arrête dès que le résultat est connu. Dans `a and b`, si `a` est déjà `False`, `b` n'est même pas évalué (le résultat est forcément `False`). On exploite cela pour se protéger, par exemple dans `n != 0 and total / n > 1`.

## B. La syntaxe : le rôle capital de l'indentation

```
1 temperature = 12
2 if temperature < 0:
3     print("Il gele !")
4     print("Couvre-toi.")
5 else:
6     print("Pas de gel.")
7 print("Bonne journee.")      # toujours execute
```

▷ **Je prédis.** Qu'affiche ce programme ? Et si la première ligne devient `temperature = -3` ?
▷ **Je vérifie.** `Pas de gel.` puis `Bonne journee.` ; avec `-3` : `Il gele !`, `Couvre-toi.`, puis `Bonne journee.` La dernière ligne, non indentée, ne dépend pas du test : elle s'affiche toujours.

Deux points de syntaxe sont **obligatoires** en Python : le **deux-points** `:` en fin de ligne `if`/`else`, et l'**indentation** (le décalage vers la droite) des lignes qui dépendent du test.

**Règle 6 — L'indentation *est* la structure**

Là où d'autres langages utilisent des accolades, Python utilise le **décalage** pour délimiter un bloc d'instructions. Toutes les lignes indentées d'un même cran après le `:` forment le bloc exécuté si la condition est vraie. Une indentation fautive change le sens du programme, ou provoque une `IndentationError`. Une seule règle d'or : **on ne mélange jamais espaces et tabulations** (le standard : 4 espaces).

**Pour les curieux** — *Une histoire de goût… devenu règle* (facultatif)
Dans la plupart des langages, l'indentation n'est qu'une politesse pour le lecteur humain. Python a fait un choix radical : la rendre *obligatoire* et *signifiante*. Résultat, un code Python mal rangé ne s'exécute pas — ce qui force chacun à écrire proprement. Certains détestent, beaucoup adorent.

## C. Les cas multiples : `elif`

Pour enchaîner plusieurs tests, on intercale des `elif` (contraction de *else if*). Python teste les conditions dans l'ordre et exécute le **premier** bloc dont la condition est vraie ; les autres sont ignorés.

**Exemple** — *Attribuer une mention à une note sur 20*
Écrire une instruction qui range dans `mention` la mention correspondant à `note` : « Très bien » à partir de 16, « Bien » à partir de 14, « Assez bien » à partir de 12, « Admis » (sans mention) à partir de 10, « Refusé » en dessous.

▶ **Correction.** On enchaîne les tests avec `elif` :

```
1 if note >= 16:
2     mention = "Tres bien"
3 elif note >= 14:
4     mention = "Bien"
5 elif note >= 12:
```

```
    mention = "Assez bien"
elif note >= 10:
    mention = "Admis"
else:
    mention = "Refuse"
```

▷ **Je prédis.** Que vaut `mention` pour `note = 16` ? pour `note = 9.5` ?
▷ **Je vérifie.** `"Tres bien"` : avec `>=`, la borne 16 est *incluse* ; puis `"Refuse"` : aucune condition n'est vraie, c'est le `else` qui s'applique.

**Remarque**
L'ordre est crucial : une note de `17` vérifie *aussi* `note >= 14`, mais comme `note >= 16` est testée d'abord et acceptée, on obtient bien « Très bien ». On va donc *du cas le plus exigeant au moins exigeant*.

### D. Un test très utile : la divisibilité

L'opérateur `%` (le *modulo*) donne le **reste** d'une division entière. Un nombre `a` est divisible par `b` exactement lorsque `a % b` vaut `0`.

```
if n % 2 == 0:
    print("n est pair")
else:
    print("n est impair")
```

▷ **Je prédis.** Qu'affiche ce code pour `n = 0` ? pour `n = -7` ?
▷ **Je vérifie.** `n est pair` : `0 % 2` vaut `0`, et 0 est bien pair ; puis `n est impair` : en Python, `-7 % 2` vaut `1`.

► **Exercices d'application :** exercices 6 à 13 (les conditionnelles)

## IV Les boucles bornées : `for`

Répéter une action est le cœur de la puissance d'un ordinateur. Quand on connaît **à l'avance** le nombre de répétitions, on utilise une **boucle bornée**, la boucle `for`.

### A. Répéter un nombre connu de fois

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
for i in range(5):
    print("Au travail !")
```

Ce programme affiche cinq fois la même ligne. `range(5)` produit la suite des entiers `0, 1, 2, 3, 4` (cinq valeurs, **à partir de 0**, **sans** atteindre 5), et la variable `i` prend successivement chacune de ces valeurs.

**Remarque** — *Bientôt avec la tortue*
Le complément *Dessiner avec la tortue*, qui suit ce chapitre, fera *voir* cette boucle : un carré, c'est `for cote in range(4):` suivi de « avancer, tourner ». Il permettra de s'entraîner aux boucles en dessinant.

**Définition 3 — `range`**

- `range(n)` : les entiers de `0` à `n-1` ;
- `range(a, b)` : les entiers de `a` à `b-1` (la borne `b` est **exclue**) ;
- `range(a, b, pas)` : de `a` à `b-1` en avançant de `pas` (le pas peut être négatif).

**Exemple** — *Prévoyez ce que produit chaque boucle, puis vérifiez*

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 for i in range(1, 6):
2     print(i)
3
4 for i in range(0, 10, 2):
5     print(i)
6
7 for i in range(10, 0, -1):
8     print(i)
```

▶ **Correction.** Chaque `print` affiche un nombre par ligne :

- la 1re boucle affiche `1 2 3 4 5` (la borne `6` est exclue) ;
- la 2e affiche `0 2 4 6 8` (on avance de 2, `10` est exclu) ;
- la 3e affiche `10 9 8 7 6 5 4 3 2 1` : un compte à rebours (pas de `-1`, `0` est exclu).

**Exemple** — *Parcourir une chaîne*

La boucle `for` peut aussi parcourir directement une chaîne : la variable prend successivement chaque caractère.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 for lettre in "Ada":
2     print(lettre)
```

▶ **Correction.** Le programme affiche `A`, puis `d`, puis `a`, un par ligne. Avec un test `if lettre in "aeiouy":` dans la boucle, on peut par exemple repérer les voyelles d'un mot.

**Pour les curieux** — *Pourquoi commencer à 0 ?* (facultatif)

Ce choix déroute au début, mais il est partout en informatique : les cases d'un tableau seront elles aussi numérotées à partir de 0. Le grand informaticien *Edsger Dijkstra* a même écrit une note célèbre pour défendre les intervalles « borne de gauche incluse, borne de droite exclue » : ils rendent les calculs de longueurs plus simples ($b - a$ valeurs).

## B. Le motif de l'accumulateur

La boucle `for` sert rarement à répéter à l'identique : le plus souvent, la variable de boucle *intervient* dans le calcul. Le motif le plus important est l'**accumulateur** : une variable que l'on met à jour à chaque tour.

**Exemple** — *Calculer la somme des entiers de 1 à 100*

▶ **Correction.** On utilise un accumulateur `total` :

```
1 total = 0                 # INITIALISER : l'accumulateur part de 0
2 for i in range(1, 101):   # PARCOURIR : i prend les valeurs 1, 2, ..., 100
3     total = total + i     # METTRE A JOUR : on ajoute i au total courant
4 print(total)              # affiche 5050
```

▷ **Je prédis.** Qu'afficherait ce programme avec `range(1, 1)` à la place de `range(1, 101)` ?

▷ **Je vérifie.** `0` : `range(1, 1)` est vide, la boucle ne fait aucun tour et `total` garde sa valeur initiale.

| tour | `i` | `total` après le tour |
|---|---|---|
| avant | — | 0 |
| 1 | 1 | 1 |
| 2 | 2 | 3 |
| 3 | 3 | 6 |
| … | … | … |
| 100 | 100 | 5050 |

*La **trace** d'exécution : suivre l'évolution des variables tour après tour est un réflexe essentiel.*

↻ **Retour sur l'activité.** Le tableau rempli pour le programme 4 de l'activité était déjà une trace ; `total` y était un accumulateur, qui valait `10` après le tour 4, et remplacer `5` par `101` dans `range(1, 5)` donne exactement le programme ci-dessus.

**Remarque** — *Les étiquettes d'étapes*

Les commentaires en **majuscules** nomment le **rôle** de chaque ligne : `INITIALISER` (l'accumulateur, avant la boucle), `PARCOURIR` (la boucle), `METTRE A JOUR` (à chaque tour) et, dans une fonction, `RENVOYER` (le résultat, après la boucle). On les retrouvera dans les exemples et les corrigés, toujours avec les mêmes mots ; le chapitre *Algorithmique : le parcours séquentiel* en fera une méthode pour écrire n'importe quel parcours.

**Pour les curieux** — *Un peu d'histoire* (facultatif)

D'après la légende, le mathématicien *Carl Friedrich Gauss*, encore écolier, aurait trouvé ce résultat en une poignée de secondes en remarquant que $1 + 100 = 2 + 99 = \cdots = 101$, et qu'il y a 50 telles paires : $50 \times 101 = 5050$. L'ordinateur, lui, additionne bêtement… mais très vite.

## C. Boucler à l'intérieur d'une boucle

Le corps d'une boucle peut contenir… une autre boucle. On parle de **boucles imbriquées**. La boucle intérieure effectue un tour complet pour *chaque* valeur de la boucle extérieure.

**Exemple** — *Prévoyez le nombre d'étoiles affichées*

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 for ligne in range(3):
2     for colonne in range(4):
3         print("*", end="")    # end="" : reste sur la meme ligne
4     print()                  # passe a la ligne suivante
```

▶ **Correction.** La boucle intérieure fait 4 tours pour *chacun* des 3 tours de la boucle extérieure. On obtient un rectangle de $3 \times 4 = 12$ étoiles :

```
****
****
****
```

▶ **Exercices d'application :** exercices 14 à 24 et 42 (les boucles `for`)

# V Les boucles non bornées : `while`

Parfois, on ne sait **pas à l'avance** combien de fois répéter : on veut continuer *tant qu'*une condition reste vraie. C'est la **boucle non bornée**, la boucle `while`.

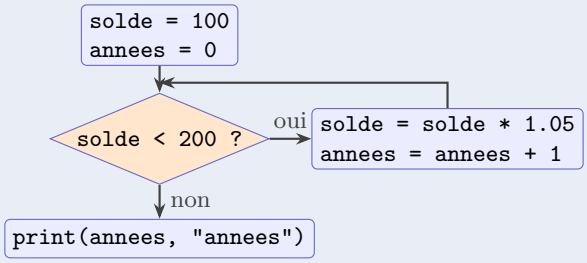
```
solde = 100
annees = 0
while solde < 200:            # tant que le solde n'a pas double
    solde = solde * 1.05      # +5 % par an
    annees = annees + 1
print(annees, "annees")
```

▷ **Je prédis.** Qu'affiche ce programme ?

▷ **Je vérifie.** `15 annees` : après 14 ans, le solde vaut environ 198 ; il ne dépasse 200 qu'au 15e tour (environ 207,89).

**Définition 4 — Boucle `while`**

`while condition:` répète le bloc indenté **tant que** la condition est vraie. La condition est testée *avant chaque tour* : si elle est fausse dès le départ, le bloc n'est jamais exécuté.




## A. Le danger : la boucle infinie

**Règle 7 — Faire évoluer la condition**

Le bloc d'un `while` **doit** modifier, tôt ou tard, une variable de la condition, faute de quoi la condition reste vraie pour toujours : c'est la **boucle infinie**. Le programme se fige et ne s'arrête plus.

```
i = 0
while i < 5:
    print(i)
    # OUPS : on a oublie d'augmenter i
```

Ici `i` vaut toujours `0`, donc `i < 5` reste éternellement vrai : le programme affiche `0` sans fin. (Pour l'interrompre : `Ctrl + C`.) L'oubli de l'incrément est *le* grand classique de la boucle `while`.

## B. Prouver qu'une boucle s'arrête : le variant

Le programme demande explicitement de savoir justifier la **terminaison** d'une boucle non bornée. L'outil s'appelle le **variant de boucle**.

**Définition 5 — Variant de boucle**

Un **variant** est une quantité entière **positive** qui **décroît strictement** à chaque tour de boucle. S'il en existe un, la boucle ne peut pas tourner indéfiniment : une suite d'entiers positifs strictement décroissante ne peut pas se prolonger indéfiniment, donc la boucle finit par s'arrêter.

1

**Exemple** — *Un compte à rebours*

```
n = 10
while n > 0:
    print(n)
    n = n - 1
```

▷ **Je prédis.** Quel est le dernier nombre affiché ? Que vaut `n` après la boucle ?

▷ **Je vérifie.** `1` : quand `n` vaut `0`, la condition est fausse et rien n'est affiché ; après la boucle, `n` vaut `0`.

La quantité `n` est un variant : elle est positive tant qu'on entre dans la boucle, et elle diminue de `1` à chaque tour. Elle atteindra donc `0`, ce qui rend la condition `n > 0` fausse : la boucle se termine.

**Règle 8 — `for` ou `while` ?**

- on connaît le nombre de tours (parcourir 1 à 100, répéter 10 fois…) → `for` ;
- on répète selon une condition dont on ignore quand elle deviendra fausse (jusqu'à ce que l'utilisateur tape « stop », jusqu'à ce que le solde double…) → `while`.

► **Exercices d'application :** exercices 25 à 30 et 43 (les boucles `while`)

# VI Les fonctions

Depuis le début du chapitre, vous écrivez des fonctions en suivant un **gabarit** ; voyons maintenant ce qu'il contient vraiment. Dès qu'un programme grandit, on a besoin de le **découper** en morceaux réutilisables, chacun chargé d'une tâche précise. Ces morceaux sont les **fonctions**. Le programme le dit clairement : la *modularisation* « permet la réutilisation de programmes et la mise à disposition de bibliothèques ».

## A. Définir et appeler une fonction

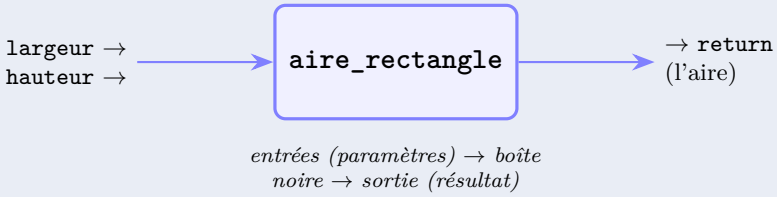
**Définition 6 — Fonction**

Une **fonction** est un bloc d'instructions nommé, que l'on **définit une fois** (mot-clé `def`) et que l'on **appelle** ensuite autant de fois qu'on veut. Elle peut recevoir des **paramètres** (ses entrées) et **renvoyer** un résultat (mot-clé `return`).

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 def aire_rectangle(largeur, hauteur):
2     return largeur * hauteur
3
4 # appels de la fonction :
5 print(aire_rectangle(3, 4))       # affiche 12
6 surface = aire_rectangle(10, 2)   # surface recoit 20
```

On voit une fonction comme une **boîte noire** : on lui fournit des valeurs en entrée (les **arguments**), elle rend une valeur en sortie. Peu importe *comment* elle travaille à l'intérieur : pour l'utiliser, il suffit de connaître ses entrées et sa sortie.




*entrées (paramètres) → boîte noire → sortie (résultat)*

**Règle 9 — `return` n'est pas `print`**

`print` **affiche** une valeur à l'écran (pour un humain) ; `return` **renvoie** une valeur au programme (pour la réutiliser dans un calcul). Une fonction qui `print` mais ne `return` rien renvoie en réalité la valeur spéciale `None`, et son résultat ne peut pas être réutilisé. Confondre les deux est une erreur très fréquente.

**À retenir — `return` termine la fonction**

Dès que Python rencontre un `return`, il **sort immédiatement** de la fonction : les lignes situées après ne sont pas exécutées.

## B. Portée : variables locales et globales

**Règle 10 — Variables locales**

Les paramètres et les variables créées **à l'intérieur** d'une fonction sont **locaux** : ils naissent à l'appel, meurent au `return`, et sont invisibles depuis l'extérieur. Deux fonctions peuvent donc utiliser une variable `i` sans jamais se gêner.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
def double(x):
    resultat = 2 * x     # resultat est LOCAL a la fonction
    return resultat

print(double(5))         # 10
print(resultat)          # NameError : resultat n'existe pas ici
```

Ce cloisonnement est une force : chaque fonction est un petit monde étanche, ce qui rend le programme bien plus facile à comprendre et à corriger.

## C. Spécifier une fonction (au programme)

Écrire une fonction juste ne suffit pas : il faut dire **clairement** ce qu'elle fait, ce qu'elle attend et ce qu'elle garantit. C'est la **spécification**, exigée par le programme.

**Définition 7 — Prototype, précondition, postcondition**

- le **prototype** annonce le nom de la fonction et ses paramètres : `racine(x)` ;
- une **précondition** est une hypothèse sur les arguments, à la charge de l'appelant (*ex. :* `x >= 0`) ;
- une **postcondition** décrit une garantie sur le résultat (*ex. :* « renvoie un nombre positif »).

En pratique, on documente une fonction avec une **docstring** (un texte entre triples guillemets, juste après le `def`) et on peut vérifier une précondition avec une **assertion**. On en donne ici une première idée ; le chapitre *Spécifier et mettre au point ses programmes* y reviendra en détail.

```
def racine(x):
    """Renvoie la racine carree de x.
    Precondition : x >= 0.
    Postcondition : le resultat au carre redonne x."""
    assert x >= 0, "x doit etre positif"
    return x ** 0.5
```

1

▷ **Je prédis.** Que renvoie `racine(9)` ? Que se passe-t-il lors de l'appel `racine(-4)` ?
▷ **Je vérifie.** `3.0`, un flottant (et non `3`) ; `racine(-4)` arrête le programme sur `AssertionError: x doit etre positif`.

> **Remarque**
> Une `assert` qui échoue **arrête** le programme avec un message clair. C'est un garde-fou précieux : mieux vaut un arrêt net et explicite qu'un résultat faux qui passe inaperçu.

## D. Tester une fonction (au programme)

Comment être sûr qu'une fonction est correcte ? On la soumet à un **jeu de tests** : des cas dont on connaît le résultat attendu. En Python, on écrit ces tests avec des `assert` : si tout se passe bien, le programme ne dit rien ; au premier test faux, il s'arrête.

```
1 def maximum(a, b):
2     if a > b:
3         return a
4     return b
5
6 assert maximum(3, 5) == 5
7 assert maximum(5, 3) == 5
8 assert maximum(4, 4) == 4    # le cas d'egalite : a ne pas oublier
```

**Règle 11 — Un jeu de tests ne *prouve* rien… mais protège beaucoup**

Le programme insiste sur un point subtil : *« le succès d'un jeu de tests ne garantit pas la correction d'un programme »*. Passer les tests montre seulement que la fonction est juste **sur les cas testés**. On soigne donc les tests : cas normaux, cas limites (0, valeurs égales, chaîne vide…), cas particuliers.

**Trois réflexes à prendre dès aujourd'hui**

Pour **chaque** fonction que vous écrirez cette année, prenez l'habitude de :

1. écrire une **docstring** (rôle + précondition + postcondition) ;
2. poser un **garde-fou** `assert` sur la précondition ;
3. écrire un petit **jeu de tests** en `assert`, sans oublier les cas limites.

Ces gestes seront approfondis dans les chapitres *Algorithmique : le parcours séquentiel* (invariants de boucle) et *Spécifier et mettre au point ses programmes* (débogage, oracle) — mais on les met en pratique **dès maintenant**.

## E. Réutiliser le travail des autres : les bibliothèques

On ne réécrit pas tout soi-même : une **bibliothèque** (*module*) regroupe des fonctions déjà écrites et testées. On y accède avec `import`.

```
1 import random
2 print(random.randint(1, 6))   # un de : entier au hasard entre 1 et 6
3 print(random.randint(0, 100)) # un entier au hasard entre 0 et 100
```

**Méthode — Utiliser une bibliothèque**

Nul besoin de connaître une bibliothèque par cœur : l'essentiel est de savoir **lire sa documentation** pour trouver la fonction utile et comprendre ce qu'elle attend.

**Pour les curieux** — *En Terminale* (facultatif)
Deux chapitres prolongent les fonctions : *La récursivité* (une fonction peut s'appeler elle-même) et *Programmation objet et paradigmes* (créer ses propres types de données, avec leurs fonctions).

► **Exercices d'application :** exercices 31 à 34 et 44 (les fonctions : spécifier, tester)

# VII Trouver et corriger ses erreurs

Personne n'écrit un programme juste du premier coup. Savoir lire un message d'erreur fait partie du métier.

## A. Les principaux types d'erreur

| Type d'erreur | Quand ? |
|---|---|
| `SyntaxError` | le code est mal formé (`:` ou `)` oublié…) |
| `IndentationError` | indentation incohérente |
| `NameError` | on utilise un nom qui n'existe pas (ou une faute de frappe) |
| `TypeError` | opération entre types incompatibles (`"a" + 1`) |
| `ZeroDivisionError` | division par zéro |

**Pour les curieux** — *Le premier « bug » était un vrai insecte* (facultatif)
En 1947, l'équipe du calculateur Mark II, dont fait partie l'informaticienne *Grace Hopper*, trouve un papillon de nuit coincé dans un relais. Elle le colle dans son cahier de bord avec la mention « premier cas réel de *bug* trouvé », et Grace Hopper rendra l'anecdote célèbre. Le mot est resté ; « déboguer » (*debug*), c'est chasser ces bestioles du code.

## B. Lire un message d'erreur

Quand Python rencontre une erreur, il s'arrête et affiche un message. Ce message impressionne (il est en anglais), mais il est toujours construit de la même façon. Voici un programme `age.py` où l'on a oublié de convertir la saisie :

▷ **Je prédis.** Quel type d'erreur ce programme va-t-il provoquer ? Le prévoir avant de lire la suite.

```
1 age = input("Votre age ? ")
2 print("Dans un an :", age + 1)
```

On l'exécute et on tape `16` :

```
Votre age ? 16
Traceback (most recent call last):
  File "/home/eleve/age.py", line 2, in <module>
    print("Dans un an :", age + 1)
                          ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

Ce message se lit ainsi :

- `Traceback (most recent call last):` : le programme a commencé à s'exécuter, puis s'est arrêté sur une erreur ;
- `File "/home/eleve/age.py", line 2` : le **fichier** et le **numéro de la ligne** où l'erreur s'est produite ;
- vient ensuite la **ligne fautive**, recopiée ; en dessous, le signe `^` (entouré de `~`) montre l'**endroit précis** : ici, le `+` ;
- la **dernière ligne** est la plus importante : le **type d'erreur** (`TypeError`, voir le tableau ci-dessus), puis le **message** : « on ne peut concaténer à une `str` qu'une `str`, pas un `int` ».

Le diagnostic suit : `age` est une chaîne, car `input` renvoie toujours une chaîne. On corrige avec `age = int(input("Votre age ? "))`.

Une faute de **syntaxe** est détectée **avant** l'exécution : Python refuse le programme sans exécuter la moindre ligne, et le message n'a pas de ligne `Traceback`, mais il se lit de la même façon. Dans ce programme `note.py`, il manque le deux-points du `if` :

```
1 note = int(input("Note ? "))
2 if note >= 10
3     print("Admis")
```

La question « Note ? » n'est même pas posée ; Python affiche :

```
  File "/home/eleve/note.py", line 2
    if note >= 10
                ^
SyntaxError: expected ':'
```

Le `^` pointe juste après `10`, là où Python attendait (*expected*) le `:`. La présentation varie un peu selon l'éditeur et la version de Python, mais on retrouve toujours le fichier, le numéro de ligne, la ligne fautive et, en dernier, le type d'erreur suivi du message.

**Méthode — Quand ça ne marche pas**

1. **Lire la dernière ligne** du message : le type d'erreur et le message.
2. **Regarder la ligne indiquée**, puis **la ligne précédente** : une parenthèse ou un guillemet non refermé peut être signalé une ligne trop bas.
3. **Vérifier les détails** : le `:` en fin de ligne `if`, `elif`, `else`, `for`, `while`, `def` ; chaque `(` a sa `)` ; chaque `"` ouvrant a son `"` fermant ; l'**indentation** (4 espaces par niveau).
4. Si le programme ne s'arrête pas mais **donne un mauvais résultat**, ajouter un **`print` de contrôle** qui affiche les valeurs des variables, par exemple `print("i =", i, "total =", total)` dans une boucle ; le retirer une fois l'erreur trouvée.

► **Exercices d'application :** exercices 45 **et** 46 (corriger d'après le message d'erreur)

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Juste après.** Le résultat surprenant de `0.1 + 0.2` s'expliquera au chapitre *Le binaire et l'écriture des nombres*, où `//` et `%` serviront à convertir un entier en binaire.
- **Plus loin dans l'année.** Les chaînes et les `range`, numérotés à partir de 0, annoncent les tableaux des *Types construits* ; l'**accumulateur** deviendra le patron de tous les parcours ; docstrings, `assert` et tests seront approfondis dans *Spécifier et mettre au point ses programmes*.
- **En Terminale.** Le chapitre *Calculabilité* montrera qu'aucun programme ne peut décider, pour *toute* boucle, si elle s'arrête : d'où l'importance de savoir exhiber soi-même un variant.

## Bilan — la carte mémoire

Les **six constructions élémentaires** du programme, réunies :

| Construction | À retenir |
|---|---|
| Séquence | les instructions s'exécutent dans l'ordre, de haut en bas |
| Affectation `=` | « reçoit » : on évalue à droite, on range à gauche |
| Conditionnelle `if/elif/else` | choisir selon une condition (booléen) ; `:` + indentation |
| Boucle bornée `for` | répéter un nombre **connu** de fois (`range`) |
| Boucle non bornée `while` | répéter **tant qu'**une condition est vraie ; gare à l'infini |
| Appel de fonction | `def` pour définir, `return` pour renvoyer ; entrées → sortie |

| Réflexe | À retenir |
|---|---|
| Types de base | `int`, `float`, `str`, `bool` ; gare aux flottants |
| `=` contre `==` | affecter contre tester l'égalité |
| `print` contre `return` | afficher contre renvoyer pour réutiliser |
| Terminaison | un **variant** (entier $\geqslant 0$ qui décroît) garantit l'arrêt |
| Spécifier / tester | docstring, préconditions (`assert`), jeu de tests |

### Retour à la situation de départ

1. Avec une **variable temporaire** : `temp = a`, puis `a = b`, puis `b = temp`. Sans elle, l'affectation `a = b` écrase l'ancienne valeur de `a`, qui est perdue (§ II).
2. Avec une boucle bornée `for i in range(1, 101):` et un **accumulateur** `total`, initialisé à `0`, auquel on ajoute `i` à chaque tour ; le programme affiche `5050` (§ IV).
3. Oui : une boucle `while` dont la condition reste toujours vraie est une **boucle infinie**. Pour garantir l'arrêt, on exhibe un **variant**, entier positif qui décroît strictement à chaque tour (§ V).

## Erreurs fréquentes

- **Confondre `=` et `==`.** `=` *affecte* une valeur, `==` *teste* l'égalité. *Le réflexe :* dans un `if`, c'est `==`.
- **Confondre `print` et `return`.** `print` *affiche* ; `return` *renvoie* une valeur réutilisable. Une fonction sans `return` renvoie `None`.
- **Oublier le `:` ou mal indenter.** En Python, le bloc est défini par l'**indentation** ; le `:` termine la ligne d'un `if`, `for`, `while`, `def`.
- **Une boucle `while` qui ne progresse pas.** Si la variable de condition n'évolue pas, la boucle est **infinie**. *Le réflexe :* un **variant** qui se rapproche de l'arrêt.
- **Oublier que `input` renvoie une chaîne.** Pour calculer, convertir : `int(input(...))`.
- **Confondre `/` et `//`.** `/` donne un **flottant**, `//` le **quotient entier** ; `%` donne le reste.

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

☐ créer (affecter) une variable et reconnaître son **type** (`int`, `float`, `str`, `bool`) ;

☐ écrire une **condition** (`if` / `elif` / `else`) et une expression booléenne ;

☐ écrire une boucle **bornée** (`for`) et **non bornée** (`while`) ;

☐ définir et appeler une **fonction** avec paramètres et `return` ;

☐ convertir (`int` / `float` / `str`) et lire une entrée (`input`) ;

☐ **spécifier** (docstring) et **tester** (`assert`) une fonction ;

☐ justifier qu'une boucle `while` se termine (variant).

# EXERCICES

## Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- ▶ *À programmer et tester* en Python (console ou éditeur en ligne). Le symbole `>>>` figure une saisie dans la console.
- **Réflexe** pour chaque fonction : **(1)** que reçoit-elle (paramètres, *préconditions*) et que **renvoie**-t-elle ? **(2)** l'ai-je **testée** sur des cas simples *et* des cas limites (0, égalité, mot vide…) ?
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

**Remarque**

La **tortue** (`turtle`) fait l'objet d'un **complément séparé**, après ce chapitre : elle n'est pas reprise ici.

### Variables, affectation, types ↑cours

★ **Exercice 1** *Lecture de trace* ▶ corrigé p. 519 • essentiel

Sans machine, donner ce qu'affiche ce programme. Détailler l'évolution de chaque variable.

```
1 x = 5
2 y = 3
3 x = x + y
4 y = x - y
5 x = x - y
6 print(x, y)
```

Que réalise, au final, ce programme sur `x` et `y` ? ▷ coup de pouce

★ **Exercice 2** *Échanger deux variables* ▶ corrigé p. 519

▶ Dans un emploi du temps, `matin = "maths"` et `soir = "NSI"`. On veut **échanger** les deux cours.

1. Un camarade propose : `temp = matin`, puis `soir = matin`, puis `matin = temp`. Que valent `matin` et `soir` à la fin ? Où est l'erreur ?
2. Écrire des instructions correctes (avec une **variable temporaire**) pour qu'à la fin `matin` vaille `"NSI"` et `soir` vaille `"maths"`. Quelle valeur la variable temporaire met-elle à l'abri ?

▷ coup de pouce

★ **Exercice 3** *Convertisseur* ▶ corrigé p. 520

▶

1. Écrire une fonction `celsius_en_fahrenheit(c)` qui renvoie la température `c` (en degrés Celsius) convertie en degrés Fahrenheit (formule : $F = \frac{9}{5}C + 32$).
2. Écrire une fonction `prix_ttc(ht)` qui renvoie le prix TTC d'un article à partir de son prix hors taxe, avec une TVA de 20 %. ▷ coup de pouce

```
>>> celsius_en_fahrenheit(100)
212.0
>>> prix_ttc(50)
60.0
```

★ **Exercice 4** *Rectangle : périmètre et aire* ▶ corrigé p. 520

▶ Écrire deux fonctions `perimetre(largeur, hauteur)` et `aire(largeur, hauteur)` qui renvoient respectivement le périmètre et l'aire d'un rectangle. ▷ coup de pouce

```
>>> perimetre(3, 4)
14
>>> aire(3, 4)
12
```

★★ **Exercice 5** *Le bon usage de `//` et `%`* ▶ corrigé p. 520 • essentiel

▶ Écrire trois fonctions `unites(n)`, `dizaines(n)` et `centaines(n)` qui prennent un entier de trois chiffres au plus et renvoient respectivement son chiffre des unités, des dizaines et des centaines. On n'utilisera que les opérateurs `//` (quotient) et `%` (reste). ▷ coup de pouce

```
>>> unites(347)
7
>>> dizaines(347)
4
>>> centaines(347)
3
>>> dizaines(60)
6
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Les conditionnelles ↑ cours

★ **Exercice 6** *Pair ou impair* ▶ corrigé p. 521

▶ Écrire une fonction `est_pair(n)` qui renvoie `True` si l'entier `n` est pair, `False` sinon. Vérifier le résultat sur quelques valeurs, sans oublier le cas `n = 0`. ▷ coup de pouce

★ **Exercice 7** *Le plus grand des deux* ▶ corrigé p. 521

▶ Écrire une fonction `maximum2(a, b)` qui renvoie le plus grand des deux nombres `a` et `b`, **sans** utiliser la fonction `max`. Ne pas oublier le cas d'égalité. ▷ coup de pouce

★ **Exercice 8** *Le signe d'un nombre* ▶ corrigé p. 522 • essentiel

▶ Écrire `signe(x)` qui renvoie `1` si `x` est strictement positif, `-1` s'il est strictement négatif, et `0` s'il est nul. (On utilisera `if`, `elif`, `else`.) ▷ coup de pouce

★★ **Exercice 9** *Mention au bac* ▶ corrigé p. 522 • essentiel

▶ Écrire une fonction `mention(note)` qui renvoie la mention correspondant à une note sur 20 : « Très bien » ($\geqslant 16$), « Bien » ($\geqslant 14$), « Assez bien » ($\geqslant 12$), « Admis » ($\geqslant 10$, sans mention), « Refusé » sinon.

1. Pourquoi l'**ordre** des tests est-il important ? Que se passerait-il si on testait `note >= 10` en premier ?
2. Que renvoie votre fonction pour `note = 25` (valeur impossible) ? On verra plus loin comment interdire proprement une telle valeur avec une *précondition*. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 10** *Année bissextile* ▶ corrigé p. 522

▶ Écrire une fonction `est_bissextile(annee)` qui renvoie un booléen, sachant qu'une année est bissextile si elle est divisible par 400, *ou* divisible par 4 sans l'être par 100. ▷ coup de pouce

```
>>> est_bissextile(2000)
True
>>> est_bissextile(1900)
False
>>> est_bissextile(2024)
True
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 11** *Le plus grand des trois* ▶ corrigé p. 523

▶ Écrire `maximum3(a, b, c)` qui renvoie le plus grand de trois nombres, sans utiliser `max`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 12** *Quel type de triangle ?* ▶ corrigé p. 523

▶ On donne les longueurs `a`, `b`, `c` des trois côtés d'un triangle (supposées valides, donc strictement positives). Écrire `type_triangle(a, b, c)` qui renvoie `"equilateral"` (trois côtés égaux), `"isocele"` (exactement deux côtés égaux) ou `"scalene"` (aucun côté égal). ▷ coup de pouce

```
>>> type_triangle(3, 3, 3)
'equilateral'
>>> type_triangle(3, 4, 5)
'scalene'
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 13** *Réduction par paliers* ▶ corrigé p. 524

▶ Un magasin applique une réduction selon le montant : `0 %` en dessous de 100 €, `5 %` de 100 à moins de 300 €, `10 %` à partir de 300 €. Écrire `prix_reduit(montant)` qui renvoie le montant après réduction. ▷ coup de pouce

```
>>> prix_reduit(50)
50.0
>>> prix_reduit(200)
190.0
>>> prix_reduit(300)
270.0
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Les boucles bornées (`for`) ↑ cours

★ **Exercice 14** *Remettre l'accumulateur dans l'ordre* ▶ corrigé p. 524

Les lignes de la fonction `somme_entre(a, b)`, qui renvoie $a+(a+1)+\cdots+b$, ont été mélangées et ont perdu leur indentation.

```
total = total + i
return total
def somme_entre(a, b):
total = 0
for i in range(a, b + 1):
```

**1.** Recopier les lignes dans le bon ordre, **avec la bonne indentation**. ▷ coup de pouce

2. ▶ Vérifier que `somme_entre(3, 6)` renvoie `18`.
3. Que renverrait la fonction si la ligne `total = 0` était placée *à l'intérieur* de la boucle ?

★ **Exercice 15** *Sommes* ▶ corrigé p. 524 • essentiel

▶

1. Écrire `somme(n)` qui renvoie $1+2+\cdots+n$ à l'aide d'un accumulateur. Vérifier que `somme(100)` vaut `5050`.
2. Écrire `somme_carres(n)` qui renvoie $1^2+2^2+\cdots+n^2$. ▷ coup de pouce

★ **Exercice 16** *Table de multiplication* ▶ corrigé p. 525

▶ Écrire `table(n)` qui **affiche**, une par ligne, les dix lignes de la table de multiplication de `n`, sous la forme `n x i = ...`. *Rappel :* `print(a, "x", b)` affiche les valeurs séparées par des espaces. ▷ coup de pouce

```
>>> table(7)
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

★★ **Exercice 17** *Construire une chaîne* ▶ corrigé p. 525

▶ Écrire `ligne_etoiles(n)` qui renvoie une chaîne formée de `n` étoiles, en partant d'une chaîne **vide** `""` que l'on complète à chaque tour (`chaine = chaine + "*"`). C'est le même motif d'accumulateur que dans l'exercice « Sommes », appliqué à du texte. ▷ coup de pouce

```
>>> ligne_etoiles(5)
'*****'
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 18** *Factorielle* ▶ corrigé p. 525

▶ Une course réunit `n` coureurs. Combien d'ordres d'arrivée sont possibles ? Le vainqueur peut être l'un des `n` coureurs, le deuxième l'un des `n - 1` restants, et ainsi de suite : il y a $n \times (n-1) \times \cdots \times 2 \times 1$ ordres possibles. Ce produit $1 \times 2 \times \cdots \times n$ s'appelle la **factorielle** de `n`, notée $n!$. Par convention, $0! = 1$ (sans coureur, un seul classement : le classement vide).

Écrire `factorielle(n)` avec un accumulateur. Tester `factorielle(0)` et `factorielle(5)`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 19** *Compter les voyelles* ▶ corrigé p. 526 • essentiel

▶ Écrire `compter_voyelles(mot)` qui renvoie le nombre de voyelles d'une chaîne (on comptera aussi le `y` ; on suppose `mot` écrit en minuscules, sans accent). *Rappel :* on parcourt une chaîne caractère par caractère avec `for lettre in mot:` et on teste `if lettre in "aeiouy":`. ▷ coup de pouce

```
>>> compter_voyelles("bonjour")
3
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 20** *Diviseurs et nombres parfaits* ▶ corrigé p. 526

▶

1. Écrire `somme_diviseurs(n)` qui renvoie la somme de tous les diviseurs de `n` (y compris `1` et `n`).
2. Un entier `n` est **parfait** si la somme de ses diviseurs vaut `2 * n`. Écrire `est_parfait(n)`.
3. Afficher tous les nombres parfaits strictement inférieurs à `100`.

▷ coup de pouce

```
>>> somme_diviseurs(6)
12
>>> est_parfait(28)
True
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 21** *Compter les occurrences d'une lettre* ▶ corrigé p. 527

▶ Écrire `occurrences(caractere, mot)` qui renvoie le nombre de fois que `caractere` apparaît dans `mot`.

▷ coup de pouce

```
>>> occurrences("a", "banana")
3
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 22** *Puissance sans* `**` ▶ corrigé p. 527

▶ Écrire `puissance(a, n)` qui renvoie $a^n$ à l'aide d'une boucle (accumulateur de produit), **sans** utiliser l'opérateur `**`. On rappelle que $a^0 = 1$.

▷ coup de pouce

```
>>> puissance(2, 10)
1024
>>> puissance(5, 0)
1
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 23** *FizzBuzz* ▶ corrigé p. 527 ≈ 15 min

▶ Écrire `fizzbuzz(n)` qui **affiche**, un par ligne, les entiers de `1` à `n`, en affichant à la place `Fizz` pour un multiple de 3 seulement, `Buzz` pour un multiple de 5 seulement, et `FizzBuzz` pour un multiple de 3 *et* de 5.

▷ coup de pouce ▷ début de solution

```
>>> fizzbuzz(5)
1
2
Fizz
4
Buzz
```

★★ **Exercice 24** *Un triangle d'étoiles* ▶ corrigé p. 528

▶ Écrire `triangle(n)` qui **affiche** un triangle de `n` lignes : la ligne numéro `i` contient `i` étoiles.

▷ coup de pouce

```
>>> triangle(3)
*
**
***
```

1

## Les boucles non bornées (`while`) ↑cours

★ **Exercice 25** *Compte à rebours* ▶ corrigé p. 528 • essentiel

▶ Écrire `compte_a_rebours(n)` qui affiche `n`, `n-1`, …, `1` à l'aide d'une boucle `while`. **Justifier** que cette boucle se termine en exhibant un **variant**. ▷ coup de pouce

★★ **Exercice 26** *Décortiquer un entier* ▶ corrigé p. 528 • essentiel

▶ À l'aide de `n % 10` (dernier chiffre) et `n // 10` (on retire le dernier chiffre) :

1. écrire `nombre_de_chiffres(n)` qui renvoie combien de chiffres possède `n` ;
2. écrire `somme_chiffres(n)` qui renvoie la somme des chiffres de `n`. ▷ coup de pouce

```
>>> nombre_de_chiffres(347)
3
>>> somme_chiffres(347)
14
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 27** *Racine entière* ▶ corrigé p. 529

▶ Avec `n` dalles carrées, on veut carreler la plus grande terrasse **carrée** possible : `k` dalles de côté, donc $k \times k$ dalles, sans dépasser les `n` dalles disponibles. Ce plus grand entier `k` tel que $k \times k \leqslant n$ s'appelle la **racine entière** de `n` (avec $n \geqslant 0$). Écrire `racine_entiere(n)` avec une boucle `while`, **sans** utiliser `**` ni de racine carrée toute faite. Justifier la terminaison. ▷ coup de pouce

```
>>> racine_entiere(15)
3
>>> racine_entiere(16)
4
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 28** *Dépasser un seuil* ▶ corrigé p. 529

▶ Léa remplit une tirelire : le jour 1 elle y met 1 €, le jour 2 elle y met 2 €, le jour 3 elle y met 3 €, et ainsi de suite. Au bout de `n` jours, la tirelire contient donc $1 + 2 + \cdots + n$ euros. Écrire `seuil(s)` qui renvoie le premier jour `n` où la tirelire contient **strictement plus** de `s` euros. **Justifier** la terminaison. Tester : `seuil(1000)` doit renvoyer 45. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 29** *PGCD (algorithme d'Euclide)* ▶ corrigé p. 530 ≈ 15 min

▶ Pour calculer le PGCD de deux entiers strictement positifs, on remplace le plus grand par sa différence avec le plus petit, et on recommence jusqu'à ce que les deux soient égaux : cette valeur commune est le PGCD.

1. Écrire `pgcd(a, b)` avec une boucle `while a != b`.
2. **Justifier la terminaison** : trouver une quantité qui décroît strictement à chaque tour et reste positive. ▷ coup de pouce ▷ début de solution

```
>>> pgcd(12, 18)
6
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 30** *Racine numérique* ▶ corrigé p. 530 ≈ 15 min

▶ La **racine numérique** d'un entier s'obtient en additionnant ses chiffres, puis en recommençant sur le résultat, jusqu'à n'obtenir qu'un seul chiffre. Par exemple : $347 \to 3+4+7 = 14 \to 1+4 = 5$. Écrire `racine_numerique(n)` à l'aide d'une boucle `while`. Justifier la terminaison.

▷ coup de pouce ▷ début de solution

```
>>> racine_numerique(347)
5
>>> racine_numerique(12345)
6
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Fonctions : spécifier et tester ↑ cours

★★ **Exercice 31** *Nombres premiers* ▶ corrigé p. 531 • essentiel

▶ Un entier `n` est premier s'il vaut au moins 2 et n'admet aucun diviseur autre que 1 et lui-même.

1. Écrire `est_premier(n)`.
2. Rédiger un **jeu de tests** (`assert`) couvrant : un premier, un non-premier, et les cas limites 0, 1, 2.

▷ coup de pouce

★★ **Exercice 32** *Palindrome* ▶ corrigé p. 531

▶ Un palindrome est un mot qui se lit de la même façon dans les deux sens (« radar », « kayak »). Écrire une fonction booléenne `est_palindrome(mot)` qui le détecte. Prévoir les cas limites (mot vide, une seule lettre). *(On rappelle que* `len(mot)` *est le nombre de caractères de* `mot` *et que* `mot[i]` *est le caractère de position* `i`*, la première position étant* `0`*.)*

▷ coup de pouce

```
>>> est_palindrome("radar")
True
>>> est_palindrome("bonjour")
False
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 33** *Jouer avec les bords d'un mot* ▶ corrigé p. 531

▶

1. Écrire `meme_bord(mot)` qui renvoie `True` si le mot commence et se termine par la même lettre. *(On rappelle que* `mot[0]` *est la première lettre et* `mot[-1]` *la dernière.)*
2. Écrire `memes_bords(mot1, mot2)` qui renvoie `True` si les deux mots commencent par la même lettre **et** finissent par la même lettre.

▷ coup de pouce

```
>>> meme_bord("radar")
True
>>> memes_bords("chat", "chocolat")
True
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1

★★ **Exercice 34** *Le bug qui passe les tests* ▶ corrigé p. 532

On propose la fonction suivante, censée additionner deux nombres :

```
1 def addition(a, b):
2     return a * b
```

1. Vérifier que le jeu de tests `assert addition(2, 2) == 4` et `assert addition(0, 0) == 0` **passe** pourtant sans erreur. Expliquer pourquoi.
2. Proposer un test `assert` qui **démasque** le bug. ▷ coup de pouce
3. Que peut-on en conclure sur cette phrase du programme officiel de NSI : « *le succès d'un jeu de tests ne garantit pas la correction d'un programme* » ?

## Exercices de synthèse ↑ cours

★★★ **Exercice 35** *Les horaires du bus* ▶ corrigé p. 532 ≈ 30 min

Sur une ligne de bus de Monaco, le premier bus part à **6 h 30**, puis il en part un toutes les **12 minutes** ; aucun bus ne part après **21 h 00**. Pour calculer facilement, on compte les heures en **minutes écoulées depuis minuit** : 6 h 30 correspond à $6 \times 60 + 30 = 390$.

1. ▶ Écrire `en_minutes(h, m)` qui renvoie le nombre de minutes écoulées depuis minuit à l'heure `h` h `m`.
2. ▶ Écrire `prochain_bus(t)` qui renvoie l'horaire (en minutes) du premier bus partant à l'instant `t` ou après, et `-1` s'il n'y a plus de bus ce jour-là. On utilisera une boucle `while`. ▷ coup de pouce ▷ début de solution
3. ▶ Écrire `attente(h, m)` qui renvoie le nombre de minutes à attendre le prochain bus quand on arrive à l'arrêt à `h` h `m`, et `-1` s'il n'y a plus de bus.
4. Quel est l'horaire du **dernier** bus de la journée ? L'écrire en heures et minutes à l'aide de `//` et `%`.

```
>>> en_minutes(6, 30)
390
>>> prochain_bus(400)
402
>>> attente(8, 0)
6
>>> attente(22, 15)
-1
```

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 36** *Un mot de passe robuste* ▶ corrigé p. 533 ≈ 30 min

Un site exige des mots de passe **robustes** : au moins **8 caractères**, au moins une **majuscule**, au moins un **chiffre** et au moins un **caractère spécial** parmi `!?@#$%&*`. *(On rappelle que* `len(mdp)` *est le nombre de caractères de* `mdp`*.)*

1. ▶ Écrire `compter_chiffres(mdp)` qui renvoie le nombre de chiffres que contient la chaîne `mdp`.
2. ▶ Écrire `est_robuste(mdp)` qui renvoie `True` si le mot de passe respecte les quatre règles, `False` sinon. ▷ coup de pouce ▷ début de solution
3. Rédiger un **jeu de tests** (`assert`) contenant un mot de passe robuste et, pour **chacune** des quatre règles, un mot de passe qui enfreint **cette seule** règle et respecte les trois autres.

```
>>> compter_chiffres("Monaco98")
2
>>> est_robuste("Monaco98!")
True
>>> est_robuste("monaco98!")
False
```

★★★ **Exercice 37** *Nombres premiers jumeaux* ▶ corrigé p. 533 ≈ 15 min

Deux nombres premiers sont **jumeaux** si leur différence vaut 2 (par exemple 3 et 5, ou 11 et 13).

▶ En réutilisant `est_premier`, écrire `premiers_jumeaux(n)` qui **affiche**, un couple par ligne, tous les couples `p` et `p+2` de premiers jumeaux tels que les deux nombres soient strictement inférieurs à `n`. ▷ coup de pouce ▷ début de solution

```
>>> premiers_jumeaux(20)
3 et 5
5 et 7
11 et 13
17 et 19
```

★★ **Exercice 38** *Une somme selon un assistant d'IA* ▶ corrigé p. 534

Un élève demande à un assistant d'IA : « Écris une fonction Python qui renvoie la somme des entiers impairs de 1 à `n`. » Voici la réponse obtenue.

```
1  def somme_impairs(n):
2      somme = 0
3      for i in range(1, n, 2):
4          somme = somme + i
5      return somme
```

« `range(1, n, 2)` produit `1, 3, 5, ...` jusqu'à `n` ; on ajoute chaque valeur à `somme`, qui est renvoyée à la fin. Par exemple, `somme_impairs(5)` renvoie `1 + 3 + 5`, soit 9. »

1. La réponse est-elle correcte ? Faire la trace de `somme_impairs(5)` (valeurs successives de `i` et de `somme`), puis ▶ vérifier dans la console.
2. Localiser et corriger l'erreur.
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ? ▷ coup de pouce

★★★ **Exercice 39** *Euclide en accéléré* ▶ corrigé p. 534 ≈ 20 min

L'algorithme de l'exercice « PGCD (algorithme d'Euclide) » est décrit par le mathématicien grec **Euclide**, vers 300 av. J.-C., au livre VII de ses *Éléments* (propositions 1 et 2). L'informaticien **Donald Knuth** y voit le « grand-père de tous les algorithmes », le plus ancien algorithme non trivial encore utilisé aujourd'hui (*The Art of Computer Programming*, vol. 2, 1981, p. 318). Il a pourtant un défaut, que l'on va corriger.

1. ▶ Reprendre la fonction `pgcd(a, b)` de l'exercice « PGCD » et y ajouter une variable `tours`, augmentée de 1 à chaque tour de boucle et affichée juste avant le `return`. Combien de tours faut-il pour `pgcd(1000, 3)` ?
2. Dans ce calcul, on retire 3 à 1000 encore et encore, tant que c'est possible. Quelle opération du cours donne **directement** ce qu'il reste à la fin de toutes ces soustractions ?
3. ▶ En déduire `pgcd_rapide(a, b)` : tant que `b` n'est pas nul, on remplace `a` par `b` et `b` par le reste de la division de `a` par `b` ; à la fin, `a` est le PGCD. Afficher aussi le nombre de tours : combien pour `pgcd_rapide(1000, 3)` ? ▷ coup de pouce
4. **Justifier la terminaison** de `pgcd_rapide` : quelle quantité entière, positive, diminue strictement à chaque tour ?

```
>>> pgcd_rapide(12, 18)
3 tours
6
>>> pgcd_rapide(1071, 462)
3 tours
21
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Entraînement ↑ cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

### ★ Exercice 40 *Lire un entier au clavier* ▶ corrigé p. 535 • essentiel

▶ Pour chaque programme, écrire les lignes qui demandent la valeur au clavier avec `input`, la convertissent avec `int`, puis affichent le résultat demandé.

a) Demander l'âge de l'utilisateur et afficher l'âge qu'il aura dans 10 ans.
b) Demander le prix d'un article (un nombre entier d'euros) et afficher le prix de trois articles.
c) Demander un nombre de minutes et l'afficher en heures et minutes (`135` donne `2 h 15 min`).

Dans le programme a), que se passe-t-il si l'on oublie `int` ? ▷ coup de pouce

```
Votre age ? 16
Dans 10 ans, vous aurez 26 ans
```

### ★ Exercice 41 *Longueur, premier et dernier caractère* ▶ corrigé p. 535 • essentiel

1. Sans machine, donner `len(mot)`, `mot[0]` et `mot[len(mot) - 1]` pour chacune des valeurs suivantes de `mot` : a) `"Monaco"` ; b) `"NSI"` ; c) `"z"` ; d) `"Grace Hopper"`. ▶ Vérifier ensuite dans la console.
2. Pourquoi `mot[len(mot)]` provoque-t-il toujours une erreur ?
3. ▶ Écrire `dernier(mot)` qui renvoie le dernier caractère d'un mot non vide, à l'aide de `len`.

▷ coup de pouce

```
>>> dernier("Monaco")
'o'
```

### ★ Exercice 42 *Prédire l'affichage, puis exécuter :* `for` ▶ corrigé p. 535 • essentiel

Pour chaque programme, écrire **sans machine** ce qu'il affiche, puis ▶ l'exécuter. En cas d'écart, trouver l'erreur de raisonnement avant de passer au programme suivant. ▷ coup de pouce

a)

```
for i in range(4):
    print(i * 3)
```

b)

```
for i in range(2, 12, 4):
    print(i)
```

c)

```
total = 0
for i in range(1, 5):
    total = total + i
    print(total)
```

d)

```
mot = ""
for lettre in "NSI":
    mot = lettre + mot
print(mot)
```

★ **Exercice 43** *Prédire l'affichage, puis exécuter : `while`* ▶ corrigé p. 535 • essentiel

Même consigne avec des boucles `while` : prédire **sans machine**, puis ▶ exécuter. ▷ coup de pouce

a)

```
n = 1
while n < 20:
    n = n * 2
print(n)
```

b)

```
n = 10
while n > 0:
    print(n)
    n = n - 3
```

c)

```
i = 5
while i < 5:
    print(i)
    i = i + 1
print("fin")
```

d)

```
x = 100
tours = 0
while x > 1:
    x = x // 2
    tours = tours + 1
print(x, tours)
```

★ **Exercice 44** *Spécifier et tester une fonction* ▶ corrigé p. 536 • essentiel

Pour chacune des deux fonctions ci-dessous : **(1)** écrire sa docstring (rôle, précondition, postcondition) ; **(2)** ajouter un garde-fou `assert` sur la précondition ; **(3)** ▶ écrire un jeu de trois tests `assert`, dont au moins un cas limite. ▷ coup de pouce

a)

```
def prix_billet(age):
    if age < 12:
        return 5
    return 9
```

b)

```
def aire_carre(cote):
    return cote * cote
```

★ **Exercice 45** *Corriger d'après le message d'erreur (1)* ▶ corrigé p. 536 • essentiel

On exécute le fichier `voyelles.py` suivant :

```
1 def compter_voyelles(mot):
2     n = 0
3     for lettre in mot:
4         if lettre in "aeiouy":
5         n = n + 1
6     return n
7
8 print(compter_voyelles("bonjour"))
```

Python affiche :

```
  File "/home/eleve/voyelles.py", line 5
    n = n + 1
    ^
IndentationError: expected an indented block after 'if' statement on line 4
```

1. Quel est le type de l'erreur ? Quelles lignes le message cite-t-il ? Traduire la dernière ligne.
2. Corriger le programme. Qu'affiche-t-il alors ?
3. Le programme a-t-il commencé à s'exécuter avant l'erreur ? Justifier. ▷ coup de pouce

★ **Exercice 46** *Corriger d'après le message d'erreur (2)* ▶ corrigé p. 536 • essentiel

On exécute le fichier `moyenne.py` suivant :

```
1 def moyenne(a, b, c):
2     total = a + b + c
3     return totale / 3
4
5 print(moyenne(12, 15, 9))
```

Python affiche :

```
Traceback (most recent call last):
  File "/home/eleve/moyenne.py", line 5, in <module>
    print(moyenne(12, 15, 9))
          ^^^^^^^^^^^^^^^^^^
  File "/home/eleve/moyenne.py", line 3, in moyenne
    return totale / 3
           ^^^^^^
NameError: name 'totale' is not defined. Did you mean: 'total'?
```

1. Quel est le type de l'erreur ? Dans quelle fonction et à quelle ligne se produit-elle ?
2. Pourquoi le message cite-t-il aussi la ligne 5 ?
3. Corriger le programme et donner ce qu'il affiche alors. ▷ coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Dresser un tableau avec une colonne par variable et une ligne par instruction. Pour chaque affectation, calculer d'abord la partie droite avec les valeurs *du moment*, puis ranger le résultat dans la variable de gauche. ↑énoncé

Ex. 2 q. 2 – piste Dresser un tableau avec une colonne par variable (`matin`, `soir`, `temp`) et une ligne par instruction. Quelle valeur est écrasée par la *première* affectation qui modifie `matin` ou `soir` ? ↑énoncé

Ex. 3 q. 2 – piste Suivre le gabarit de fonction du cours : écrire le calcul avec le paramètre (`c` ou `ht`), puis le renvoyer avec `return`. Ajouter 20 % d'un prix revient à le multiplier par quel nombre ? ↑énoncé

Ex. 4 – piste Faire d'abord le calcul à la main pour le rectangle de 3 sur 4 de l'exemple, en notant les opérations effectuées ; les écrire ensuite avec les paramètres `largeur` et `hauteur`. ↑

Ex. 5 – piste Que donnent `347 % 10` et `347 // 10` ? Comment isoler ensuite le chiffre des dizaines à partir de `34` ? ↑énoncé

Ex. 6 – piste Quel est le reste de la division par 2 d'un nombre pair ? d'un nombre impair ? Le paragraphe « Un test très utile : la divisibilité » du cours traduit cela en Python. ↑énoncé

Ex. 7 – piste Comparer `a` et `b` avec un `if` : combien de cas faut-il distinguer ? En cas d'égalité, peu importe lequel des deux on renvoie. ↑énoncé

Ex. 8 – piste Écrire d'abord en français les trois cas et leur condition, puis les traduire un par un. Tester ensuite avec `12`, `-3` et `0`. ↑énoncé

Ex. 9 q. 2 – piste Reprendre l'exemple du cours sur `elif` : Python exécute le *premier* bloc dont la condition est vraie. Tester à la main la note `18` avec votre ordre de tests. ↑énoncé

Ex. 10 – piste « Divisible par 4 » se traduit par `annee % 4 == 0`. Traduire ensuite la phrase mot à mot avec `or` et `and`, en plaçant des parenthèses. ↑énoncé

Ex. 11 – piste Retenir un « champion provisoire » `m`, égal à `a` au départ, puis le comparer successivement à `b` et à `c`. ↑énoncé

Ex. 12 – piste Dans quel ordre tester les cas ? Si l'on teste d'abord « trois côtés égaux », que reste-t-il à distinguer ensuite ? ↑énoncé

Ex. 13 – piste Choisir d'abord le taux (`0`, `5` ou `10`) avec `if`/`elif`/`else`, puis faire le calcul une seule fois à la fin. Attention aux bornes : `300` relève de quel palier ? ↑énoncé

Ex. 14 q. 1 – piste Une fonction commence par sa ligne `def`. Se demander ensuite ce qui doit être fait une seule fois *avant* la boucle, à *chaque* tour, et une seule fois *après*. ↑énoncé

Ex. 15 q. 2 – piste Reprendre l'exemple du cours « somme des entiers de 1 à 100 » en remplaçant 100 par `n` : attention à la borne exclue du `range`. Pour la question 2, seule la quantité ajoutée à chaque tour change. ↑énoncé

**Ex. 16 – piste** Quelles valeurs la variable de boucle `i` doit-elle prendre ? Écrire le `range` correspondant (borne de droite exclue), puis un seul `print` dans la boucle. ↑énoncé

**Ex. 17 – piste** Combien de tours de boucle faut-il ? Quelle valeur la chaîne doit-elle avoir avant le premier tour ? ↑énoncé

**Ex. 18 – piste** C'est le motif de la somme du cours, avec une multiplication. Mais si l'accumulateur part de `0`, que vaut le produit à la fin ? Par quelle valeur faut-il l'initialiser ? ↑énoncé

**Ex. 19 – piste** C'est un **compteur** : une variable qui part de `0` et augmente de `1` chaque fois que la condition est vraie. ↑énoncé

**Ex. 20 q. 3 – piste** Un diviseur `d` de `n` vérifie `n % d == 0`. Quelles valeurs de `d` faut-il essayer ? Pour la question 3, réutiliser `est_parfait` dans une boucle. ↑énoncé

**Ex. 21 – piste** Même schéma que « Compter les voyelles » : seule la condition testée sur chaque lettre change. ↑énoncé

**Ex. 22 – piste** $a^n = a \times a \times \cdots \times a$ ($n$ facteurs) : combien de tours de boucle, et quelle valeur initiale pour l'accumulateur ? ↑énoncé

**Ex. 23 – piste** Dans quel **ordre** tester les cas ? Pour `15`, lequel des tests `i % 3 == 0` et `i % 5 == 0` serait capté en premier ? ↑énoncé

**Ex. 23 – début de solution**
```
for i in range(1, n + 1):
    if i % 3 == 0 and i % 5 == 0:
        print("FizzBuzz")
    elif ...
```
↑énoncé

**Ex. 24 – piste** Une ligne affichée par tour de boucle : quelles valeurs `i` doit-il prendre ? Pour construire la ligne, réutiliser `ligne_etoiles` de l'exercice « Construire une chaîne », ou tester `"*" * 3` dans la console. ↑énoncé

**Ex. 25 – piste** Tant que quelle condition faut-il encore afficher `n` ? Ne pas oublier de modifier `n` dans la boucle, sinon elle ne s'arrête jamais. Pour le variant : quelle quantité entière positive diminue à chaque tour ? ↑énoncé

**Ex. 26 q. 2 – piste** Tant que `n` n'est pas nul, on traite le dernier chiffre puis on le retire. Attention au cas `n = 0` : combien de chiffres possède-t-il ? ↑énoncé

**Ex. 27 – piste** Partir de `k = 0` et augmenter `k` tant que le *suivant*, `k + 1`, convient encore. Pour la terminaison, chercher une quantité entière positive qui diminue. ↑énoncé

**Ex. 28 – piste** Deux variables évoluent ensemble : l'entier `n` et le total `1 + 2 + ... + n`. Quelle condition écrire dans le `while` pour continuer *tant que* le seuil n'est pas dépassé ? ↑énoncé

**Ex. 29 q. 2 – piste** Faire d'abord à la main `pgcd(12, 18)` : noter les valeurs de `a` et `b` à chaque tour. À chaque tour, lequel des deux modifie-t-on ? ↑énoncé

**Ex. 29 q. 2 – début de solution**
```
while a != b:
    if a > b:
        a = a - b
    else: ...
```
↑énoncé

**Ex. 30 – piste** Une fonction déjà écrite dans l'exercice « Décortiquer un entier » fait l'essentiel du travail. Comment reconnaître qu'un entier n'a plus qu'un seul chiffre ? ↑énoncé

**Ex. 30 – début de solution**
```
def racine_numerique(n):
    while n >= 10:

        ...
```
↑énoncé

**Ex. 31 q. 2 – piste** Traiter d'abord les entiers inférieurs à `2`. Ensuite, chercher un diviseur entre `2` et `n - 1` : dès qu'on en trouve un, la réponse est connue. ↑énoncé

**Ex. 32 – piste** Comparer le premier et le dernier caractère, puis se rapprocher du centre avec deux indices `i` et `j`. Quand faut-il s'arrêter ? ↑énoncé

**Ex. 33 q. 2 – piste** Aucune boucle n'est nécessaire : une seule comparaison (ou deux, reliées par `and`) suffit. Que se passe-t-il avec le mot vide ? ↑énoncé

Ex. 34 q. 2 – piste Comparer `2 + 2` et `2 * 2`, puis `0 + 0` et `0 * 0`. Chercher deux nombres pour lesquels somme et produit diffèrent. ↑énoncé

Ex. 35 q. 2 – piste Partir de l'horaire du premier bus et passer au bus suivant tant qu'il part trop tôt. Une fois sorti de la boucle, ce bus part-il encore avant 21 h 00 ? ↑énoncé

Ex. 35 q. 2 – début de solution
```
def prochain_bus(t):
    depart = 390
    while depart < t:
        depart = depart + 12

    ...
```
↑énoncé

Ex. 36 q. 2 – piste Pour la question 1, reprendre l'exercice « Compter les voyelles » : seule la chaîne de référence change (`"0123456789"` au lieu de `"aeiouy"`). Compter de même les majuscules et les caractères spéciaux. ↑énoncé

Ex. 36 q. 2 – début de solution
```
def est_robuste(mdp):
    majuscules = 0
    speciaux = 0
    for c in mdp:
        if c in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":

            ...
```
↑énoncé

Ex. 37 – piste Parcourir les candidats `p` un par un et tester *deux* conditions avec `and`. Quelle condition garantit que `p + 2` reste strictement inférieur à `n` ? ↑énoncé

Ex. 37 – début de solution
```
for p in range(2, n):
    if p + 2 < n and est_premier(p) and ...
```
↑énoncé

Ex. 38 q. 3 – piste Relire la définition de `range(a, b, pas)` dans le cours : la borne `b` est-elle atteinte ? ↑énoncé

Ex. 39 q. 3 – piste Si l'on écrit `a = b` puis `b = a % b`, l'ancienne valeur de `a` est déjà perdue. Calculer d'abord le reste dans une variable `r`, puis faire les deux affectations. ↑énoncé

Ex. 40 – piste Le paragraphe « Lire une entrée au clavier et convertir » du cours donne le modèle : `input` renvoie toujours une chaîne. Pour c), penser à `//` et `%` avec 60. ↑énoncé

Ex. 41 q. 3 – piste Les positions commencent à `0` : un mot de 3 caractères occupe les positions `0`, `1` et `2`. L'espace compte comme un caractère. ↑énoncé

Ex. 42 – piste Faire un tableau de trace : une ligne par tour, avec la valeur de la variable de boucle et des autres variables. Bien regarder si le `print` est dans la boucle (décalé) ou après. ↑

Ex. 43 – piste La condition est testée *avant* chaque tour : noter à chaque tour la valeur des variables, puis si la condition est encore vraie. Elle peut être fausse dès le départ. ↑énoncé

Ex. 44 – piste Reprendre l'exemple `racine(x)` du paragraphe « Spécifier une fonction » du cours. Un cas limite est une valeur « au bord » : le seuil d'une condition, ou `0`. ↑énoncé

Ex. 45 q. 3 – piste Lire d'abord la **dernière** ligne du message : *expected* signifie « attendu », *indented block* « bloc décalé ». Quelles lignes doivent être décalées sous un `if` ? ↑énoncé

Ex. 46 q. 3 – piste Le message se lit de bas en haut : la dernière ligne donne le type d'erreur et son explication, le bloc juste au-dessus la ligne fautive. Comparer les noms de variables des lignes 2 et 3. ↑énoncé

# TP ET PROJETS

## TP Le nombre mystère, édition de luxe

▶ corrigé p. 537

*un vrai petit jeu, construit fonction par fonction*

Durée : 2 à 3 h (deux séances) · Sur machine, seul ou par deux

### But du TP

Programmer, étape par étape, un **jeu complet** dans la console : l'ordinateur choisit un nombre au hasard, le joueur doit le deviner en un nombre limité d'essais, avec des **indices** (« trop grand », « chaud », « froid »), trois **niveaux**, un **score** et la possibilité de **rejouer**. En défi, on renverse les rôles : c'est l'ordinateur qui devine, et on **prouve** qu'il gagne toujours. Le TP réinvestit tout le chapitre : séquence, affectation, conditionnelles, boucles `for` et `while`, fonctions spécifiées et testées.

### Consignes

- Tout le code s'écrit dans **un seul fichier** `nombre_mystere.py`, que l'on complète au fil des exercices et que l'on **lance après chaque fonction**.
- Les réponses aux questions (sans ▶ ) s'écrivent sur le cahier.
- ▶ signale ce qui est *à programmer et tester*. Chaque fonction reçoit une **docstring** (rôle, et s'il y a lieu précondition et postcondition). Les tests `assert` donnés sont recopiés **en bas du fichier** : s'ils passent, rien ne s'affiche.
- **Pas de liste dans ce TP** : quand il y a plusieurs choses à montrer, on fait des `print` successifs.
- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône ⧉ en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Usage de l'IA, indiqué sur chaque exercice :  aucune ;  en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ;  l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Prise en main : comparer et contrôler

Une partie se joue ainsi : l'ordinateur pense à un nombre `secret` ; le joueur fait une `proposition` ; l'ordinateur répond `"trop petit"`, `"trop grand"` ou `"gagne"`. On commence par les deux briques les plus simples.

### ★ Exercice 1  *La réponse de l'ordinateur*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

▶ Écrire la fonction `comparer(proposition, secret)` qui renvoie la chaîne `"trop petit"`, `"trop grand"` ou `"gagne"`. Elle **renvoie** la réponse (`return`), elle ne l'affiche pas.

```
1 assert comparer(30, 42) == "trop petit"
2 assert comparer(50, 42) == "trop grand"
3 assert comparer(42, 42) == "gagne"
```

Pourquoi est-il préférable de *renvoyer* la réponse plutôt que de l'*afficher* ?

1

★ **Exercice 2** *Une proposition dans les clous*

1. ▶ Écrire `est_dans_intervalle(n, mini, maxi)` qui renvoie `True` si `n` est compris entre `mini` et `maxi` (bornes **incluses**), `False` sinon.
2. Écrire vous-même un **jeu de tests** d'au moins cinq `assert` pour `mini = 1` et `maxi = 100`. Quels cas **limites** ne faut-il surtout pas oublier ?

```
>>> est_dans_intervalle(57, 1, 100)
True
>>> est_dans_intervalle(0, 1, 100)
False
```

## 2 Le cœur du jeu : la boucle de partie

★★ **Exercice 3** *Une première partie*

▶ Écrire, **sans fonction** pour l'instant, un programme qui :

- tire un nombre `secret` entre 1 et 100 avec `random.randint(1, 100)` (ne pas oublier `import random`) ;
- demande une proposition au joueur, **tant qu'**il n'a pas trouvé, et affiche à chaque fois la réponse de `comparer` ;
- compte les essais et affiche à la fin `Bravo, trouve en 6 essai(s) !`

```
Votre proposition : 50
trop grand
Votre proposition : 25
trop petit
...
```

1. Pourquoi une boucle `while` et non une boucle `for` ?
2. Le joueur tape `50` : que vaut `input(...)` ? Que faut-il écrire pour pouvoir le comparer à `secret` ?
3. Cette boucle a-t-elle un **variant** ? Peut-elle ne jamais s'arrêter ?

★★ **Exercice 4** *Un nombre d'essais limité*

On transforme ce programme en une fonction `partie(secret, maxi, essais_max)`. Le joueur propose des nombres entre `1` et `maxi` ; il a droit à `essais_max` essais. La fonction **renvoie** le nombre d'essais utilisés s'il a gagné, et `0` s'il a perdu (elle affiche alors le nombre mystère).

1. ▶ Compléter la fonction. La boucle doit s'arrêter dans **deux** cas : le joueur a trouvé, ou il n'a plus d'essai. ▷ coup de pouce

```
1 def partie(secret, maxi, essais_max):
2     """Joue une partie. Renvoie le nombre d'essais utilises
3     si le joueur gagne, 0 s'il perd."""
4     essais = 0
5     trouve = False
6     while ... and ...:
7         proposition = int(input("Votre proposition : "))
8         ...
9     ...
```

2. ▶ Une proposition **hors limites** (0, 150…) ne doit **pas** être comptée : afficher `Hors limites !` et redemander. Réutiliser `est_dans_intervalle`.

3. Donner un **variant** de la boucle quand toutes les propositions sont dans les limites. Que devient la terminaison si un joueur tape `0` sans arrêt ? Est-ce grave ?
4. Pourquoi ne peut-on pas tester `partie` avec des `assert`, comme `comparer` ? Qu'en déduire sur la façon de découper un programme ?

## 3 Indices et score

★★ **Exercice 5** *Chaud ou froid ?*

Pour aider le joueur, on ajoute un indice selon la distance entre la proposition et le secret : `"brulant"` si cette distance vaut au plus 3, `"chaud"` si elle vaut au plus 10, `"froid"` au-delà.

1. ▶ Écrire `ecart(a, b)` qui renvoie la distance entre `a` et `b`, toujours positive ou nulle (`ecart(3, 10)` et `ecart(10, 3)` valent 7). Utiliser une conditionnelle.
2. ▶ Écrire `indice(proposition, secret)`, en réutilisant `ecart`.
3. ▶ Modifier `partie` pour afficher, à chaque essai raté, la réponse, l'indice et le nombre d'essais restants.

```
assert ecart(3, 10) == 7
assert ecart(10, 3) == 7
assert ecart(5, 5) == 0
assert indice(45, 42) == "brulant"
assert indice(32, 42) == "chaud"
assert indice(31, 42) == "froid"
```

```
Votre proposition : 50
trop grand - chaud - il reste 6 essai(s)
```

Ces six tests suffisent-ils ? Quel test ajouter pour vérifier la frontière entre `"brulant"` et `"chaud"` *de l'autre côté* du secret ?

★★ **Exercice 6** *Compter les points*

Règle du score : une partie perdue rapporte 0 point ; trouver du **premier coup** rapporte 100 points ; sinon, on gagne 10 points, plus 10 points par essai **non utilisé**.

1. Combien de points pour une victoire en 2 essais sur 7 ? en 7 essais sur 7 ?
2. ▶ Écrire `points(essais, essais_max)`, où `essais` est la valeur renvoyée par `partie`. Écrire sa docstring avec la **précondition** sur `essais`, et la vérifier par un `assert` en première ligne.
3. Compléter ce jeu de tests (au moins un test par cas de la règle).

```
assert points(0, 7) == 0      # perdu
assert points(1, 7) == 100    # du premier coup
assert points(2, 7) == ...
assert points(7, 7) == ...
```

## 4 Niveaux et menu

★★ **Exercice 7** *Trois niveaux de difficulté*

| Niveau | Nom | Nombre mystère | Essais autorisés |
|---|---|---|---|
| 1 | facile | entre 1 et 10 | 4 |
| 2 | moyen | entre 1 et 100 | 7 |
| 3 | difficile | entre 1 et 1000 | 10 |

1. ▶ Écrire `afficher_menu()`, qui affiche les trois niveaux, **un par ligne** (trois `print`). Cette fonction ne renvoie rien.

2. ▶ Écrire `borne_max(niveau)` et `essais_autorises(niveau)`, qui renvoient la borne et le nombre d'essais du niveau. Précondition : `niveau` vaut `1`, `2` ou `3` — la garantir par un `assert` avec un message.
3. Que se passe-t-il lors de l'appel `borne_max(4)` ? Pourquoi est-ce préférable à un résultat faux qui passerait inaperçu ?

★★ **Exercice 8** *Un menu à l'épreuve des fautes de frappe*

▶ Écrire `demander_niveau()` qui affiche le menu, demande un niveau et **redemande tant que** la réponse n'est pas `"1"`, `"2"` ou `"3"`. Elle renvoie le niveau sous forme d'**entier**.

```
1 : facile    (entre 1 et 10,   4 essais)
2 : moyen     (entre 1 et 100,  7 essais)
3 : difficile (entre 1 et 1000, 10 essais)
Votre niveau (1, 2 ou 3) : 4
Choix impossible.
Votre niveau (1, 2 ou 3) : 2
```

Pourquoi compare-t-on la saisie aux **chaînes** `"1"`, `"2"`, `"3"` *avant* de la convertir avec `int` ? Que ferait `int("deux")` ?

## 5 Le jeu complet

★★ **Exercice 9** *Rejouer et faire le bilan*

▶ Écrire la fonction principale `jouer()` qui :

- enchaîne les parties **tant que** le joueur répond `o` à la question `Rejouer ? (o/n)` ;
- pour chaque partie : demande le niveau, tire le secret, lance `partie`, calcule et affiche les points gagnés et le total ;
- à la fin, affiche un bilan : nombre de parties jouées, nombre de parties gagnées, total des points.

```
Points gagnes : 30 - total : 30
Rejouer ? (o/n) n
Bilan : 2 partie(s), 1 gagnee(s), 30 points.
```

Puis ajouter tout en bas du fichier la ligne `jouer()`, **après** les tests, et faire tester votre jeu par un voisin.

1. Repérer dans votre programme une **séquence**, une **affectation**, une **conditionnelle**, une boucle **non bornée**, un **appel de fonction**. Où une boucle **bornée** serait-elle utile ? (Voir la partie suivante.)
2. Quelles variables de `jouer` sont des **compteurs** ou des **accumulateurs** ? Avec quelle valeur faut-il les initialiser, et où (avant ou dans la boucle) ? ▷ coup de pouce

## 6 Défi : l'ordinateur devine

On renverse les rôles. Stratégie de l'ordinateur pour un nombre entre `mini` et `maxi` : proposer le **milieu** `(mini + maxi) // 2` ; si c'est trop petit, le secret est entre `milieu + 1` et `maxi` ; si c'est trop grand, entre `mini` et `milieu - 1`. On recommence avec le nouvel intervalle.

Cette stratégie, qui coupe l'intervalle en deux à chaque essai, est l'un des grands classiques de l'algorithmique. Nous l'étudierons en détail au chapitre *La recherche dichotomique* : elle y recevra son nom, nous prouverons qu'elle se termine toujours et nous calculerons précisément son coût.

★★ **Exercice 10** *Jouer la stratégie à la main*

Le secret est 37, entre 1 et 100. Recopier sur le cahier le tableau suivant et le compléter jusqu'à la victoire.

| Essai | `mini` | `maxi` | proposition | réponse |
|---|---|---|---|---|
| 1 | 1 | 100 | 50 | trop grand |
| 2 | | | | |
| 3 | | | | |
| ... | | | | |

★★★ **Exercice 11** *Le pire des cas*

1. ▶ Écrire `essais_ordinateur(secret, maxi)` qui **simule** cette stratégie (le secret est connu de la fonction, qui utilise `comparer` pour obtenir la réponse) et renvoie le nombre d'essais nécessaires.

▷ coup de pouce

```
assert essais_ordinateur(50, 100) == 1
assert essais_ordinateur(25, 100) == 2
assert essais_ordinateur(100, 100) == 7
```

2. ▶ Écrire `pire_cas(maxi)` qui essaie **tous** les secrets possibles de 1 à `maxi` (boucle `for`) et renvoie le plus grand nombre d'essais rencontré. Calculer `pire_cas(10)`, `pire_cas(100)`, `pire_cas(1000)`. Que remarquez-vous en comparant avec le tableau des niveaux ?

▷ début de solution

3. ▶ Écrire `nb_moities(n)` qui compte combien de divisions entières par 2 (`n = n // 2`) il faut effectuer pour que `n` devienne `0` (par exemple, `nb_moities(1)` vaut `1` et `nb_moities(4)` vaut `3`). Vérifier par une boucle d'`assert` que `pire_cas(m) == nb_moities(m)` pour tous les `m` de 1 à 200. Combien d'essais suffiraient pour un nombre entre 1 et un million ?

★★★ **Exercice 12** *À vous de penser, à lui de deviner*

▶ Écrire `ordinateur_devine(maxi)` : le joueur pense à un nombre ; l'ordinateur propose, le joueur répond `+` (c'est plus), `-` (c'est moins) ou `=`. Une réponse inattendue est redemandée sans compter d'essai. Si le joueur **triche** (ses réponses se contredisent), l'intervalle devient vide (`mini > maxi`) : l'ordinateur l'annonce et s'arrête.

```
Je propose 50 (+, - ou =) : +
Je propose 75 (+, - ou =) : -
Je propose 62 (+, - ou =) : +
Je propose 68 (+, - ou =) : =
Trouve en 4 essai(s) !
```

Ajouter ce mode au menu de `jouer()` (par exemple, niveau `4`). Quelles fonctions faut-il modifier ? Lesquelles restent intactes ?

### Bilan à rédiger

Sur une demi-page du cahier (ou dans un commentaire en tête du fichier `nombre_mystere.py`) :

1. Lister les fonctions écrites, avec pour chacune : ce qu'elle reçoit, ce qu'elle renvoie (ou affiche), et si elle est testée par des `assert`. Lesquelles ne peuvent pas l'être, et pourquoi ?
2. Citer un test qui vous a fait trouver une erreur dans votre code. Quelle était l'erreur ?
3. Expliquer en deux phrases pourquoi 7 essais suffisent toujours entre 1 et 100 quand on joue « au milieu ».

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 q. 1 – piste Avec un booléen `trouve` : la boucle continue tant que `not trouve` *et* qu'il reste des essais. Après la boucle, c'est `trouve` qui dit ce qu'il faut renvoyer. ↑énoncé

Ex. 9 q. 2 – piste Trois variables avant la boucle (`nb_parties`, `nb_gagnees`, `total`) et une variable `encore = "o"` pour pouvoir entrer une première fois dans le `while`. ↑énoncé

Ex. 11 q. 1 – piste La boucle de `essais_ordinateur` ressemble à celle de `partie` : un booléen `trouve`, un compteur d'essais ; seuls `mini` et `maxi` changent selon la réponse de `comparer`. ↑énoncé

Ex. 11 q. 2 – début de solution

```
def pire_cas(maxi):
    pire = 0
    for secret in range(1, maxi + 1):
        e = essais_ordinateur(secret, maxi)
        ...
```

↑énoncé

# FICHE DE RÉVISION

## L'essentiel

- **Variable** : un casier étiqueté (modèle provisoire, qui sera précisé au ch. 3) ; le nom désigne la valeur (`age = 15`). **Types** : `int`, `float`, `str`, `bool`.
- **Opérateurs** : `+ - * /` (flottant), `//` (quotient entier), `%` (reste), `**` (puissance) ; comparaisons `== != < <= > >=` ; booléens `and or not`.
- **Condition** : `if ...: / elif ...: / else:`. **Boucles** : `for` (bornée), `while` (non bornée).
- **Fonction** : `def` + paramètres + `return` (renvoie une valeur).

## Une fonction spécifiée et testée

```
def est_pair(n):
    """Renvoie True si l'entier n est pair, False sinon."""
    return n % 2 == 0

assert est_pair(4) == True      # jeu de tests
assert est_pair(7) == False
```

## Les deux boucles

```
for i in range(5):      # i = 0, 1, 2, 3, 4  (borne : on sait combien)
    print(i)

n = 10                  # non bornee : on s'arrete sur une condition
while n > 0:
    print(n)
    n = n - 1           # variant : n decroit -> la boucle se termine
```

## Les deux pièges à ne jamais confondre

- `=` **affecte**, `==` **teste** l'égalité ;
- `print` **affiche**, `return` **renvoie** (sans `return` : la fonction renvoie `None`).

## Erreurs à éviter

- oublier le `:` ou mal indenter le bloc ;
- `while` qui ne progresse pas ⇒ boucle infinie ;
- oublier `int(input(...))` (l'entrée est une chaîne) ;
- confondre `/` (flottant) et `//` (entier).

## Savoir-faire à maîtriser — je sais…

☐ créer (affecter) une variable et reconnaître son **type** (`int`, `float`, `str`, `bool`) ;

☐ écrire une **condition** (`if` / `elif` / `else`) et une expression booléenne ;

☐ écrire une boucle **bornée** (`for`) et **non bornée** (`while`) ;

☐ définir et appeler une **fonction** avec paramètres et `return` ;

☐ convertir (`int` / `float` / `str`) et lire une entrée (`input`) ;

☐ **spécifier** (docstring) et **tester** (`assert`) une fonction ;

☐ justifier qu'une boucle `while` se termine (variant).

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 544

1. Après `a = 3`, `b = a + 1`, `a = b * 2`, que valent `a` et `b` ?
2. Quels entiers parcourt `range(2, 8, 2)` ?
3. Que valent `17 // 5`, `17 % 5` et `17 / 5` ?
4. Une fonction `f` se termine par `print(x)` au lieu de `return x`. Que vaut `y = f(5)` ?
5. Répéter jusqu'à ce que le solde double : `for` ou `while` ? Que faut-il pour être sûr qu'elle s'arrête ?

# DÉFI ADVENT OF CODE

## DÉFI The Tyranny of the Rocket Equation

*le carburant de la fusée — division entière, fonctions, boucle while*

Jour 1

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p01-defi-aoc-2019-01` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2019/day/1`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Créer un compte sur le site est **facultatif** (connexion avec GitHub, Google…). Avec un compte, le lien *get your puzzle input* affiche **vos** données : tout sélectionner, copier, puis coller dans un fichier texte vide enregistré sous le nom `input.txt`, **dans le même dossier** que votre programme Python. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Une fusée est faite d'une centaine de modules, et chacun a besoin d'une quantité de carburant calculée à partir de sa masse. On veut le carburant total à embarquer. Le calcul pour un module est simple, mais le répéter pour cent modules (et, en partie 2, plusieurs fois pour chacun) devient vite pénible et source d'erreurs : un programme s'en charge.

**Ce qu'on réinvestit.** Les notions de ce chapitre : une **fonction** qui fait le calcul pour un module (écrite une fois, appelée cent fois), la **division entière** `//` qui arrondit à l'unité inférieure, et la boucle `while` pour répéter un calcul tant qu'il donne une quantité positive, avec son **variant** pour être sûr qu'elle s'arrête.

**La démarche de la fiche.**

1. Comprendre la formule sur des questions courtes et l'appliquer à la main à trois modules.

2. Choisir la bonne division en Python, puis écrire la fonction `carburant` pour un module.
3. Lire le fichier et additionner les carburants de tous les modules : c'est la réponse de la partie 1.
4. Pour la partie 2, réutiliser `carburant` dans une boucle `while`, puis vérifier que cette boucle s'arrête.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `fuel` | carburant | `mass` | masse |
| `module` | module (élément de la fusée) | `required` | nécessaire |
| `to divide by` | diviser par | `to round down` | arrondir à l'unité inférieure |
| `to subtract` | soustraire | `to add together` | additionner |

### Résumé de l'énoncé (partie 1)

**Contexte.** Le Père Noël est bloqué au bord du système solaire, et les elfes préparent une fusée pour aller le chercher. La fusée est faite de modules, et chaque module a besoin de carburant pour décoller, en quantité qui dépend de sa masse. Avant le décollage, il faut savoir combien de carburant embarquer en tout.
**Les données.** Le fichier liste les masses des modules : chaque ligne contient **un seul entier positif**, la masse d'un module. Il y a une ligne par module, et l'ordre des lignes n'a pas d'importance.
**La règle.**

- Pour un module, on divise sa masse par 3 et on **arrondit à l'entier inférieur**.
- On retire 2 au résultat : c'est le carburant de ce module. Autrement dit, c'est le quotient de la division entière par 3, moins 2.
- On fait ce calcul séparément pour chaque module, puis on additionne les résultats.

**La réponse attendue.** La somme des carburants de tous les modules. Par exemple, pour deux modules de masses 12 et 21, les carburants sont $12//3-2=2$ et $21//3-2=5$ : la réponse serait 7.

### Exercice 1 *Vérifier sa compréhension*

**But.** S'assurer que l'arrondi « à l'entier inférieur » est bien compris, y compris pour de petites masses.
**À quoi ça sert ?** C'est le seul point délicat de la formule : un mauvais arrondi donne une réponse fausse, même si tout le reste du programme est juste. La dernière question prépare la partie 2, où l'on rencontrera des résultats nuls ou négatifs.

Questions pièges, à traiter sur le cahier :

1. Quel carburant pour une masse de 9 ? de 10 ? de 11 ? Pourquoi trouve-t-on le même résultat ?
2. Pour une masse de 2000, quelle différence entre arrondir à l'entier inférieur et arrondir à l'entier le plus proche ?
3. Que donne la formule pour une masse de 6 ou de 7 ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Appliquer la formule à la main à trois modules inventés, puis faire le total.
**À quoi ça sert ?** Ces trois valeurs serviront de tests : la fonction `carburant` devra les retrouver, et le total donne la réponse attendue du programme complet sur cet exemple.

Calculer le carburant nécessaire pour des modules (inventés) de masses `30`, `90` et `2000`, puis le total. Vérification : on doit obtenir `8`, `28`, `664`, soit un total de `700`.

## 2 Programmer la partie 1

### Exercice 3 *Division entière*

**But.** Repérer, parmi trois façons de diviser en Python, celle qui arrondit à l'unité inférieure.
**À quoi ça sert ?** C'est l'opérateur à utiliser dans la fonction `carburant`. Avec la mauvaise division, on obtiendrait un nombre à virgule, et toutes les réponses seraient fausses.

En Python, comparer `2000 / 3`, `2000 // 3` et `int(2000 / 3)`. Lequel correspond à « diviser puis arrondir à l'unité inférieure » pour un nombre positif ?

### Exercice 4 *Une fonction pour un module*

**But.** Écrire la fonction qui calcule le carburant d'un seul module et la tester avec des `assert`.
**À quoi ça sert ?** Elle sera appelée pour chaque ligne du fichier dans la partie 1, puis réutilisée, sans modification, dans la boucle de la partie 2. Les `assert` garantissent qu'elle est juste avant qu'on s'en serve.

Écrire une fonction `carburant(masse)` qui renvoie le carburant d'un module. La tester avec des `assert` sur les trois masses de l'exemple, par exemple `assert carburant(30) == 8`.▷ coup de pouce

#### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans le fichier (par exemple avec `for ligne in fichier:`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

### Exercice 5 *Le total*

**But.** Lire les masses dans le fichier et additionner les carburants de tous les modules.
**À quoi ça sert ?** Le résultat affiché sur vos données est la réponse de la partie 1. Le motif de l'accumulateur (un total qui part de 0 et grossit à chaque tour) servira encore dans la partie 2.

#### Outil Python : lire un fichier ligne par ligne (`open`)

`open("input.txt")` ouvre le fichier ; une boucle `for` en parcourt les lignes une à une, sous forme de chaînes. Chaque ligne se termine par un retour à la ligne invisible, que `ligne.strip()` retire. Ces lignes sont fournies :

```
fichier = open("input.txt")       # ouvre le fichier
for ligne in fichier:             # une ligne a chaque tour de boucle
    ligne = ligne.strip()         # retire le retour a la ligne final
    masse = int(ligne)            # la chaine "1969" devient l'entier 1969
    print(masse)
fichier.close()                   # ferme le fichier
```

Recopier ce programme et l'exécuter sur le fichier `exemple.txt` contenant les trois masses inventées `30`, `90` et `2000` (une par ligne), en remplaçant `"input.txt"` par `"exemple.txt"`. Le modifier ensuite pour calculer la somme des carburants de tous les modules, avec le motif de l'accumulateur : sur l'exemple, on doit trouver `700`. Revenir enfin à `"input.txt"` pour obtenir la réponse. ▷ coup de pouce

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

**Résumé de la partie 2 (à lire après avoir validé la partie 1)**

**Ce qui change.** On avait oublié un détail : le carburant lui-même pèse, et il faut donc du carburant pour le transporter ! Et ce carburant supplémentaire pèse à son tour…
**La nouvelle règle.**

- Pour un module, on calcule son carburant avec la formule de la partie 1.
- On calcule ensuite le carburant de ce carburant, avec la même formule, puis le carburant de ce nouveau carburant, et ainsi de suite.
- On s'arrête dès qu'on obtient une valeur nulle ou négative : elle compte pour 0.
- Le carburant du module est la somme de toutes ces quantités. Ce calcul se fait **module par module**.

**La réponse attendue.** La somme, sur tous les modules, de ces carburants. Par exemple, pour un module de masse 60 : on trouve 18, puis 4 (carburant de 18), puis une valeur négative (carburant de 4) qui arrête le calcul ; le carburant de ce module est $18 + 4 = 22$.

### Exercice 6 *Du carburant pour le carburant*

**But.** Écrire la fonction qui applique la nouvelle règle à un module, en répétant le calcul tant qu'il donne une quantité positive.
**À quoi ça sert ?** En remplaçant `carburant` par `carburant_total` dans le programme du total, on obtient la réponse de la partie 2. Comme on ne sait pas à l'avance combien de fois répéter le calcul, c'est le rôle de la boucle `while`.

Écrire une fonction `carburant_total(masse)` qui applique la nouvelle règle à un module. Avec les masses de l'exemple, on doit trouver `8`, `35` et `980`, soit un total de `1023`. (Le détail à la main du cas de la masse `90` est demandé à l'exercice suivant.) ▷ coup de pouce ▷ début de solution

### Exercice 7 *Vérifier la boucle*

**But.** Suivre à la main l'exécution de la boucle sur un exemple, puis justifier qu'elle s'arrête toujours.
**À quoi ça sert ?** Un tableau de valeurs permet de trouver une erreur dans une boucle. Justifier l'arrêt (la quantité diminue à chaque tour : c'est le variant) garantit que le programme ne tournera pas indéfiniment sur les vraies données.

Pour la masse `90`, écrire le tableau des valeurs successives de `ajout` et `total` à chaque tour de boucle. Combien de tours fait la boucle ? Pourquoi est-on sûr qu'elle s'arrête ?

## 4 Approfondissement

★★★ **Exercice 8** *Approfondissement 1 (pour aller plus loin, notion de Terminale) : une fonction qui s'appelle elle-même*

**But.** Réécrire `carburant_total` sans boucle, avec une fonction qui s'appelle elle-même.
**À quoi ça sert ?** La règle de la partie 2 se décrit naturellement « en fonction d'elle-même ». Cette découverte prépare la récursivité, étudiée en Terminale, et donne une seconde méthode pour vérifier la réponse.

Une fonction peut s'appeler elle-même : on parle de fonction *récursive*. C'est une notion de Terminale, proposée ici pour les curieux.

1. Expliquer pourquoi, pour une masse $m$ : carburant total de $m$ = carburant de $m$ + carburant total *de ce carburant*, **lorsque** le carburant de $m$ est strictement positif (sinon le carburant total vaut 0).

2. Écrire une fonction `carburant_total_rec(masse)`, sans boucle, qui s'appelle elle-même sur la quantité de carburant qu'elle vient de calculer. ▷ coup de pouce ▷ début de solution

3. Écrire la suite des appels pour la masse 90 et vérifier qu'on retrouve `35`. Pourquoi est-on sûr que les appels s'arrêtent ?

### Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 – piste Une seule ligne suffit dans la fonction : `return` suivi du calcul, avec l'opérateur `//`. ↑énoncé

Ex. 5 – piste Une variable `total` qui vaut 0 avant la boucle ; à chaque tour, on lui ajoute `carburant(masse)`. On affiche `total` après la boucle. ↑énoncé

Ex. 6 – piste On ne sait pas à l'avance combien de fois il faut recalculer : c'est le cas typique d'une boucle `while`. Que faut-il faire d'une quantité nulle ou négative ? ↑énoncé

Ex. 6 – début de solution `total = 0` et `ajout = carburant(masse)` ; `while ajout > 0:` on ajoute `ajout` à `total`, puis on remplace `ajout` par `carburant(ajout)`. Bien réutiliser la fonction de la partie 1. ↑énoncé

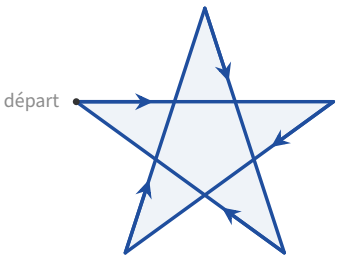
Ex. 8 q. 2 – piste Si le carburant d'une masse est nul ou négatif, la réponse est 0 et on s'arrête : c'est le *cas de base* (la condition d'arrêt). Sinon, la réponse est ce carburant, plus le carburant total de ce carburant. ↑énoncé

Ex. 8 q. 2 – début de solution `c = carburant(masse)` ; `if c <= 0: return 0` ; sinon `return c + carburant_total_rec(c)`. ↑énoncé

# Complément — Dessiner avec la tortue

```
import turtle as t

for branche in range(5):
    t.forward(100)
    t.right(144)
```




5 tours de boucle, 5 côtés

DANS CE CHAPITRE


- Activités préparatoires . . . . . 67
- Cours . . . . . 69
- Exercices . . . . . 74
- TP et projets . . . . . 80
- Corrigés . . . . . 545

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 Le robot dessinateur

▶ corrigé p. 545

*guider un camarade avec trois ordres, avant de guider une tortue*

Durée : 25 min · Sans ordinateur, par deux, crayon et cahier à petits carreaux

### Consignes

- Crayon et cahier (ou feuille) à petits carreaux : tous les tracés et toutes les réponses se font sur le cahier.
- L'un est le **programmeur**, l'autre le **robot**. Le robot tient le crayon posé sur un point du quadrillage, **tourné** dans une direction (la flèche). Il ne comprend que trois ordres :
  - **AVANCER n** : avancer tout droit de `n` carreaux **en traçant** ;
  - **GAUCHE** : tourner d'un quart de tour vers **sa** gauche, sur place, sans tracer ;
  - **DROITE** : tourner d'un quart de tour vers **sa** droite, sur place, sans tracer.
- Le robot exécute les ordres **un par un, dans l'ordre**, sans rien deviner et sans poser de question. Gauche et droite sont celles du robot : celles qu'on aurait en marchant sur le trait, dans le sens de la flèche.

### Exercice 1 *Exécuter un programme*

Chacun joue le robot, sur son cahier. Le robot part du point, tourné vers la flèche (reproduire ce point de départ sur le quadrillage du cahier).

**AVANCER 2**
**DROITE**
**AVANCER 2**
**DROITE**
**AVANCER 2**
**GAUCHE**
**AVANCER 2**
**GAUCHE**
**AVANCER 2**



1. Tracer le dessin obtenu. Que représente-t-il ?
2. Combien d'ordres ce programme contient-il ? Combien tracent un trait ?
3. Que dessinerait-on en échangeant **GAUCHE** et **DROITE** partout ?

### Exercice 2 *Le programmeur et le robot*

Les deux cartes sont à la fin de l'activité. Le programmeur regarde la carte A **sans la montrer** au robot, écrit sur son cahier le programme qui fait tracer la figure, puis le lit au robot, qui trace sur son cahier en partant d'un point, tourné vers le haut. On compare avec la carte. Puis on échange les rôles avec la carte B.

4. Écrire le programme de la carte A. Combien contient-il d'ordres ?

**5.** Écrire le programme de la carte B. Combien contient-il d'ordres ?

**6.** Le robot a-t-il tracé exactement la figure ? Sinon, l'erreur venait-elle du programme ou du robot ?

**7.** En relisant le programme B, que remarque-t-on ?

### Exercice 3 *Répéter*

On autorise un nouvel ordre : **RÉPÉTER k FOIS :** suivi d'un bloc d'ordres, que le robot exécute `k` fois de suite.

**8.** Écrire, **sans RÉPÉTER**, le programme d'un carré de côté 3. Puis l'écrire **avec RÉPÉTER**.

**9.** Réécrire le programme de la carte B avec **RÉPÉTER**.

**10.** Le robot part tourné vers le haut et exécute : **RÉPÉTER 4 FOIS : AVANCER 1, DROITE, AVANCER 1, DROITE, AVANCER 1, GAUCHE, AVANCER 1, GAUCHE**.
Tracer le résultat sur le cahier. Combien d'ordres aurait-il fallu écrire sans **RÉPÉTER** ?

### Exercice 4 *Faire le tour*

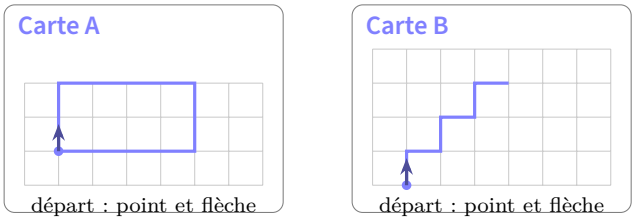
**11.** Après le carré de la question 8, le robot est revenu à son point de départ, tourné dans la même direction. De combien de degrés a-t-il tourné **en tout** ?

**12.** Un robot plus habile sait tourner de n'importe quel angle. Pour tracer un triangle à trois côtés égaux et revenir tourné comme au départ, il tourne trois fois du même angle. Lequel ? Et pour un hexagone (six côtés égaux) ?

### Exercice 5 *Ce qu'on a découvert*

**13.** Recopier et compléter sur le cahier : « Un programme est une suite d'. . . . . exécutées . . . . . , une par une : c'est une . . . . . Le robot ne devine rien : une seule erreur dans le programme et le dessin est . . . . . Pour ne pas recopier plusieurs fois les mêmes ordres, on utilise une . . . . . Pour faire le tour d'une figure à `n` coins égaux, on tourne à chaque coin de . . . . . degrés. »

**Cartes de l'exercice 2** (le robot ne doit pas les voir)



# COURS

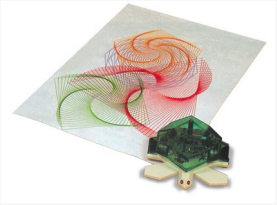
**Idée** une « tortue » se déplace sur l'écran en traçant un trait derrière elle. En lui donnant des ordres simples (avance, tourne), on dessine des figures.

**Prérequis** le chapitre *Les bases de la programmation Python* (boucles et fonctions). Le complément est ouvert par l'activité débranchée qui précède. La **partie 1** utilise des instructions en séquence puis la boucle `for` ; la **partie 2**, les fonctions.

**Pourquoi** `turtle` rend **visibles** les boucles et les fonctions : on *voit* la répétition et la factorisation. C'est un formidable terrain d'entraînement.

**Pour les curieux** — *Un peu d'histoire* (facultatif)

La « tortue » vient du langage **Logo**, créé en 1967 par **Seymour Papert** et Wally Feurzeig. À l'origine, c'était un vrai petit robot en forme de tortue qui traçait au sol ! Papert, mathématicien du MIT qui avait travaillé à Genève avec le psychologue Jean Piaget, en tire une idée forte : on apprend le mieux en **fabriquant** des choses. Dans les années 1980, des robots tortues comme celui ci-contre entrent dans les écoles : munis d'un feutre, ils dessinent sur une grande feuille posée par terre les figures programmées en Logo. Python a hérité de cette tortue, devenue virtuelle.



*Un robot tortue de sol et ses dessins*

## Partie 1 — Des instructions en séquence, puis la boucle `for`

### I Prendre la tortue en main

Tout programme commence par **importer** le module, et se termine par `turtle.done()` (qui garde la fenêtre ouverte). Entre les deux, les ordres sont exécutés **dans l'ordre**, de haut en bas, un par un : c'est une **séquence** d'instructions.

```
1 import turtle
2
3 turtle.forward(100)    # avance de 100 pas (pixels)
4 turtle.left(90)        # tourne de 90 degres vers la gauche
5 turtle.forward(100)
6
7 turtle.done()          # garde la fenetre ouverte
```

On peut **raccourcir** les noms en donnant un surnom au module, ce qui allège le code :

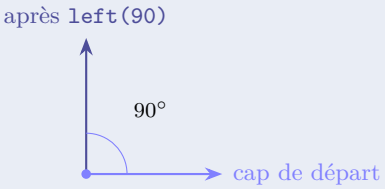
```
1 import turtle as t
2
3 t.forward(100)
4 t.left(90)
5 t.forward(100)
6 t.done()
```

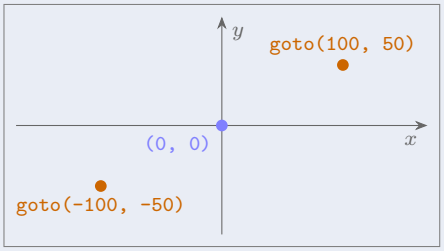
## A. Les ordres essentiels

| Ordre | Effet |
|---|---|
| `forward(d)` / `fd(d)` | avance de `d` pas |
| `backward(d)` / `bk(d)` | recule de `d` pas |
| `left(a)` / `lt(a)` | tourne de `a` degrés vers la gauche |
| `right(a)` / `rt(a)` | tourne de `a` degrés vers la droite |
| `penup()` / `pendown()` | lever / baisser le stylo (se déplacer sans tracer) |
| `goto(x, y)` | aller au point de coordonnées `(x, y)` |
| `color("red")` | couleur du trait |
| `pensize(n)` | épaisseur du trait |
| `speed(n)` | vitesse (`0` = très rapide, `1` = lent) |
| `circle(r)` | trace un cercle de rayon `r` |

**Remarque**

La tortue a une **position** et une **direction** (le « cap »). `left`/`right` ne font que changer le cap ; le trait n'apparaît qu'avec `forward`/`backward`. Les angles sont en degrés.




*la fenêtre : l'origine est au centre*

► **Exercices d'application :** exercices 1 **et** 2 (le carré et le rectangle, d'abord ordre par ordre)

# II Répéter : la boucle `for`

Dessiner un carré « à la main », c'est quatre fois la même chose :

```
1  import turtle as t
2  t.forward(100)
3  t.left(90)
4  t.forward(100)
5  t.left(90)
6  t.forward(100)
7  t.left(90)
8  t.forward(100)
9  t.left(90)
10 t.done()
```

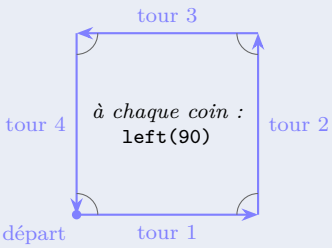
… mais **répéter**, c'est exactement le rôle d'une boucle `for`. Le même carré, en bien plus court :

▷ **Je prédis.** Quelle figure ? La tracer à main levée (point de départ, sens de rotation), puis comparer au dessin ci-dessous.

```
1  import turtle as t
2  for cote in range(4):
3      t.forward(100)
4      t.left(90)
5  t.done()
```

### Règle 1 — Écrire une boucle `for`

`for cote in range(4):` répète **4 fois** les lignes qui suivent et qui sont **décalées** vers la droite (4 espaces) : c'est le **corps** de la boucle. Ne pas oublier le **deux-points** en fin de ligne. La ligne `t.done()`, non décalée, n'est exécutée qu'une fois, après la boucle. (`range` a été présenté au chapitre *Les bases de la programmation Python*.)




*carré : 4 tours de boucle, 4 fois 90° = 360°*

### Règle 2 — Le secret des polygones réguliers

Pour fermer un polygone régulier à `n` côtés, la tortue doit avoir tourné en tout de 360°. À chaque sommet, elle tourne donc de $\frac{360}{n}$ degrés.

Un hexagone (6 côtés, on tourne de $360/6 = 60°$) devient immédiat ; l'étoile, dessinée à côté, demande deux tours complets (720°) : ce sera un exercice.

▷ **Je prédis.** Quelle figure ? La tracer à main levée (point de départ, sens de rotation), puis comparer au dessin ci-dessous.

```
1 import turtle as t
2 for cote in range(6):    # hexagone
3     t.forward(80)
4     t.left(60)
5 t.done()
```



*hexagone* *étoile à 5 branches*

► **Exercices d'application :** exercices 3 à 9 (escalier, polygones, étoile, rosace de cercles, spirale, défis) et 15 et 16 (prédire une figure ; trouver l'angle)

## III Colorier une forme

Pour remplir une figure, on encadre le tracé par `begin_fill()` et `end_fill()`.

```
1 import turtle as t
2 t.color("orange")
3 t.begin_fill()
4 for i in range(3):
5     t.forward(120)
6     t.left(120)
7 t.end_fill()
8 t.done()
```

1

# Partie 2 — Les fonctions

## IV Factoriser : les fonctions

Quand on veut réutiliser un dessin, on en fait une **fonction** avec un **paramètre** pour le régler.

```
import turtle as t

def carre(cote):
    for i in range(4):
        t.forward(cote)
        t.left(90)

carre(50)
carre(100)      # on reutilise, avec une autre taille
t.done()
```

Encore mieux : une fonction pour **n'importe quel** polygone régulier.

```
def polygone(n, cote):
    for i in range(n):
        t.forward(cote)
        t.left(360 / n)

polygone(3, 100)    # triangle
polygone(8, 60)     # octogone
```

► **Exercices d'application :** exercices 10 (un polygone au choix) et 12 (trois carrés colorés)

## V Le plus beau : les rosaces

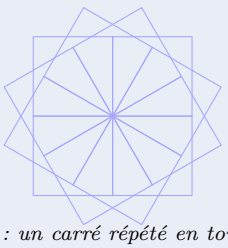
Une **rosace** s'obtient en répétant une même figure tout en **tournant** un peu à chaque fois. Ici, 12 carrés, pivotés de $360/12 = 30°$ :

▷ **Je prédis.** Quelle figure ? La tracer à main levée (point de départ, sens de rotation), puis comparer au dessin ci-dessous.

```
import turtle as t

def carre(cote):
    for i in range(4):
        t.forward(cote)
        t.left(90)

for k in range(12):      # 12 carres...
    carre(100)
    t.left(30)           # ...pivotes de 30 degres
t.done()
```



*rosace : un carré répété en tournant*

**Remarque**
On tient là toute la puissance du complément : une **fonction** (le motif), une **boucle** (la répétition) et un petit **tour** à chaque étape suffisent à produire des figures très riches. À vous de jouer dans les exercices !

► **Exercices d'application :** exercices 11 (rosace de triangles), 13 (le damier) et 14 (la cible)

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)
Ce complément met en images les boucles et les fonctions du chapitre *Les bases de la programmation Python*, et `turtle` est une **bibliothèque**, au sens vu dans ce même chapitre (section *Les fonctions*). En Terminale, la tortue tracera des fractales au chapitre *La récursivité*.

## Mémo

| **Pour...** | **On utilise...** |
|---|---|
| avancer / reculer | `forward(d)` / `backward(d)` |
| tourner | `left(a)` / `right(a)` |
| répéter un motif | une boucle `for` |
| polygone régulier à `n` côtés | tourner de `360 / n` à chaque sommet |
| réutiliser un dessin | une **fonction** avec paramètre(s) |
| remplir | `begin_fill()` ... `end_fill()` |
| se déplacer sans tracer | `penup()` ... `pendown()` |

# EXERCICES

## Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et exécuter* sur machine. Commencez toujours par `import turtle as t` et terminez par `t.done()`.
- **Partie 1** : des ordres en séquence, puis la boucle `for`. **Partie 2** : avec des fonctions.
- **Réflexes** : une figure qui se répète → une boucle `for` ; un motif à réutiliser → une **fonction** (partie 2) ; un polygone à `n` côtés → on tourne de `360 / n`.

## Partie 1 — Séquence d'instructions et boucle `for` ↑cours

### Premiers tracés ↑cours

★ **Exercice 1** *Le carré* ▶ corrigé p. 546 • essentiel

▶ Dessiner un carré de côté `100` (4 côtés, on tourne de `90` à chaque fois) : d'abord en écrivant les ordres **à la suite**, l'un sous l'autre (combien en faut-il ?), puis à l'aide d'une boucle `for`.

▷ coup de pouce



*résultat attendu*

★ **Exercice 2** *Le rectangle* ▶ corrigé p. 547

▶ Dessiner un rectangle de `200` de large sur `100` de haut. *(Attention : les quatre côtés ne sont pas tous égaux, mais on tourne toujours de* `90`*.)*

▷ coup de pouce

★ **Exercice 3** *L'escalier* ▶ corrigé p. 547 • essentiel

▶ Dessiner un escalier de **5 marches** : une marche est un déplacement horizontal suivi d'un déplacement vertical. Utiliser une boucle qui répète 5 fois « avancer, tourner, avancer, tourner ».

▷ coup de pouce



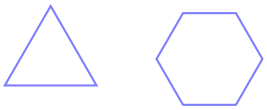
*résultat attendu*

### Les polygones réguliers ↑cours

★★ **Exercice 4** *Triangle et hexagone* ▶ corrigé p. 547

▶ Dessiner un **triangle équilatéral** (3 côtés), puis un **hexagone** (6 côtés). Dans chaque cas, de combien de degrés faut-il tourner à chaque sommet ?

▷ coup de pouce



*triangle* *hexagone*

★★ **Exercice 5** *L'étoile à cinq branches* ▶ corrigé p. 547

▶ Dessiner une étoile à 5 branches. ▷ coup de pouce

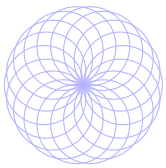


*résultat attendu*

## Répéter en tournant ↑ cours

★★ **Exercice 6** *Rosace de cercles* ▶ corrigé p. 548

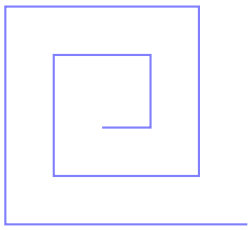
▶ La tortue sait tracer un cercle avec `t.circle(rayon)`. Dessiner une jolie « fleur » en répétant **18 cercles** de rayon `80`, en tournant un peu entre deux cercles. De combien faut-il tourner à chaque fois ? ▷ coup de pouce



*résultat attendu*

★★ **Exercice 7** *La spirale carrée* ▶ corrigé p. 548

▶ Dessiner une spirale : on tourne toujours de `90` degrés, mais la longueur **augmente** à chaque nouveau segment (par exemple `10, 20, 30, ...`). Faire une vingtaine de segments. ▷ coup de pouce

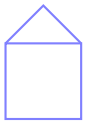


*une spirale carrée*

## Défis ↑ cours

★★★ **Exercice 8** *La maison* ▶ corrigé p. 548 ≈ 15 min

▶ Dessiner une maison : un **carré** pour la façade, surmonté d'un **triangle** pour le toit. *(Il faudra bien positionner la tortue et gérer les angles du toit.)* ▷ coup de pouce ▷ début de solution



*résultat attendu*

1

★★ **Exercice 9** *Un triangle selon un assistant d'IA* ▶ corrigé p. 548

Un élève demande à un assistant d'IA : « Écris un programme Python avec la tortue qui dessine un triangle équilatéral de côté 100. » Voici la réponse obtenue.

```
import turtle as t
for i in range(3):
    t.forward(100)
    t.left(60)
t.done()
```

« La boucle répète trois fois : avancer de 100, puis tourner à gauche de 60 degrés, puisque les angles d'un triangle équilatéral mesurent 60 degrés. Après les trois tours, la tortue est revenue à son point de départ et le triangle est fermé. »

1. La réponse est-elle correcte ? Calculer de combien de degrés la tortue a tourné en tout à la fin du programme, puis ▶ exécuter le programme.
2. Localiser et corriger l'erreur.
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce

## Partie 2 — Avec des fonctions ↑ cours

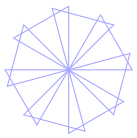
### Factoriser : les fonctions ↑ cours

★★ **Exercice 10** *Un polygone au choix* ▶ corrigé p. 549 • essentiel

▶ Écrire une fonction `polygone_perimetre(n, perimetre)` qui dessine un polygone régulier à `n` côtés dont le **périmètre** total vaut `perimetre`. L'utiliser pour dessiner, avec le même périmètre `360`, un triangle, un carré et un hexagone : quelle longueur de côté chacun a-t-il ? ▷ coup de pouce

★★ **Exercice 11** *Rosace de triangles* ▶ corrigé p. 549

▶ Écrire une fonction `triangle(cote)` qui dessine un triangle équilatéral, puis l'utiliser pour dessiner une rosace formée de **8 triangles**, tous partant du même point. De combien de degrés faut-il pivoter entre deux triangles pour faire exactement un tour complet ? ▷ coup de pouce



*résultat attendu*

### Couleurs et déplacements ↑ cours

★★ **Exercice 12** *Trois carrés colorés* ▶ corrigé p. 550 • essentiel

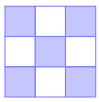
▶ Dessiner **trois carrés** côte à côte, remplis de trois couleurs différentes (par exemple rouge, vert, bleu), en écrivant une fonction `carre_plein(cote, couleur)` que l'on appelle trois fois.

▷ coup de pouce

## Défis ↑cours

★★★ **Exercice 13** *Le damier* ▶ corrigé p. 550 ≈ 30 min

▶ À l'aide de **deux boucles imbriquées**, dessiner une grille de $3 \times 3$ **carrés** (9 cases). Pour un vrai défi, remplir une case sur deux en noir, comme un damier. ▷ coup de pouce ▷ début de solution

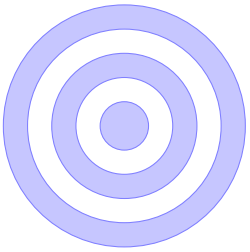


*résultat attendu*

★★★ **Exercice 14** *La cible* ▶ corrigé p. 551 ≈ 20 min

▶ On veut dessiner une cible de tir à l'arc : **cinq disques** de même centre `(0, 0)`, de rayons 100, 80, 60, 40 et 20, remplis alternativement en rouge et en blanc. *(On précise que* `t.circle(r)` *trace un cercle de rayon* `r` *dont le centre se trouve à* `r` *pas sur la* ***gauche*** *de la tortue.)*

1. La tortue est tournée vers la droite, comme au départ. En quel point doit-elle se trouver pour que `t.circle(100)` trace un cercle de centre `(0, 0)` ? Et pour `t.circle(r)` ?
2. Écrire une fonction `disque(r, couleur)` qui trace un disque de rayon `r`, de centre `(0, 0)`, rempli de la couleur `couleur`.
3. À l'aide d'une boucle `for` et de `disque`, dessiner la cible. Pourquoi faut-il commencer par le **plus grand** disque ? ▷ coup de pouce



*résultat attendu*

## Entraînement ↑cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

★ **Exercice 15** *Prédire la figure* ▶ corrigé p. 551 • essentiel

Pour chaque programme (la tortue part de `(0, 0)`, tournée vers la droite), dessiner **à main levée, sans machine**, la figure obtenue, en indiquant le point de départ. ▶ Exécuter ensuite pour vérifier. ▷ coup de pouce

a)

```
for i in range(5):
    t.forward(80)
    t.left(72)
```

b)

```
for i in range(4):
    t.forward(100)
    t.right(90)
```

c)

```
for i in range(2):
    t.forward(100)
    t.left(90)
    t.forward(50)
    t.left(90)
```

d)

```
for i in range(3):
    t.forward(100)
    t.left(90)
```

1

★ **Exercice 16** IA IA IA *Trouver l'angle* ▶ corrigé p. 551 • essentiel

▶ Pour chacun des polygones réguliers suivants, donner l'angle dont la tortue doit tourner à chaque sommet, puis écrire la boucle qui le dessine avec des côtés de `60` : a) un pentagone (5 côtés) ; b) un octogone (8 côtés) ; c) un ennéagone (9 côtés) ; d) un décagone (10 côtés). ▷ coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Écrire d'abord les ordres pour *un seul* côté. Quel groupe d'ordres se répète, et combien de fois ? C'est lui qui va dans la boucle, décalé de 4 espaces. ↑énoncé

Ex. 2 – piste Les côtés vont par paires : un grand côté puis un petit. Quel groupe d'ordres se répète, et combien de fois ? ↑énoncé

Ex. 3 – piste Après avoir monté une contremarche, la tortue regarde vers le haut : de quel côté doit-elle tourner pour repartir à l'horizontale ? Dessiner une marche sur papier en suivant la tortue. ↑énoncé

Ex. 4 – piste Pour le carré, la tortue tourne 4 fois de `90` : `360` degrés en tout pour revenir à son orientation de départ. Le même total vaut-il pour le triangle ? Attention : l'angle dont tourne la tortue n'est pas l'angle intérieur du triangle. ↑énoncé

Ex. 5 – piste Suivre du doigt le tracé de l'étoile : la tortue fait **deux** tours complets sur elle-même avant de revenir à son orientation de départ. Combien de degrés en tout, répartis sur combien de sommets ? ↑énoncé

Ex. 6 – piste Même raisonnement que pour les polygones : après les 18 cercles, la tortue doit avoir fait exactement un tour complet sur elle-même. ↑énoncé

Ex. 7 – piste La longueur doit changer à chaque passage dans la boucle : utiliser la variable de boucle `i` dans le calcul de la longueur (que vaut `10 * i` pour `i` allant de `1` à `20` ?). ↑énoncé

Ex. 8 – piste Tracer d'abord le carré, puis se demander où se trouve la tortue et dans quelle direction elle regarde. Le toit est un triangle équilatéral posé sur le côté du haut : de combien tourner à chaque sommet ? ↑énoncé

Ex. 8 – début de solution
```
for i in range(4):  ...  (la façade)
    t.left(90)
    t.forward(100)  # la tortue est en haut à gauche
    t.right(30) ...
```
↑énoncé

Ex. 9 q. 3 – piste Pour refermer une figure, de combien de degrés la tortue doit-elle avoir tourné en tout ? Comparer avec `3 * 60`. ↑énoncé

Ex. 10 – piste Partir de la fonction `polygone(n, cote)` du cours : la longueur d'un côté n'est plus donnée, il faut la calculer à partir de `perimetre` et de `n`. ↑énoncé

Ex. 11 – piste Écrire d'abord `triangle(cote)` et vérifier qu'elle ramène la tortue à son point et à son orientation de départ. La rosace n'est alors qu'une boucle : un triangle, puis on tourne. ↑énoncé

Ex. 12 – piste Relire le tableau des commandes du cours : comment passer d'un carré au suivant **sans tracer de trait** ? Comment **remplir** une figure (voir l'exemple coloré du cours) ? ↑énoncé

Ex. 13 – piste Une boucle pour les lignes, une boucle pour les colonnes : la case de la ligne `ligne` et de la colonne `colonne` commence au point `(colonne * cote, ligne * cote)`. Pour le damier : à quelle condition sur `ligne + colonne` une case est-elle noire ? ↑énoncé

Ex. 13 – début de solution
```
for ligne in range(3):
    for colonne in range(3):
        t.penup()
        t.goto(colonne * cote, ligne * cote)
        ...
```
↑énoncé

**Ex. 14 q. 3 – piste** Le rayon du disque numéro `i` (pour `i` allant de `0` à `4`) vaut `100 - 20 * i`. La couleur dépend de la parité de `i`, comme les cases du damier. ↑énoncé

**Ex. 15 – piste** Suivre la tortue avec un crayon : à chaque `forward`, tracer le segment ; à chaque `left` ou `right`, faire pivoter le crayon. Compter de combien de degrés elle a tourné en tout. ↑énoncé

**Ex. 16 – piste** Après le dernier côté, la tortue doit avoir fait exactement un tour complet sur elle-même : combien de degrés, à partager entre combien de sommets ? ↑énoncé

1

# TP ET PROJETS

## PROJET La tortue écrivain

**Fichiers à télécharger :** `maththibaud.github.io/informatique-en-autonomie/f/p01b-projet-tortue-ecrivain` — **archive .zip**

**Remarque** — *Le but*

Le programme demande une **phrase** à l'utilisateur, puis la tortue l'**écrit** dans la fenêtre en **dessinant chaque lettre**, trait après trait.
Chaque lettre est une **fonction** (`lettre_A`, `lettre_B`…). Toutes obéissent au **même contrat** : elles commencent et finissent au même endroit de leur case. C'est ce contrat qui permet d'enchaîner les lettres sans jamais se perdre.
Vous avancez par **étapes** ; chacun va aussi loin qu'il le peut. Un programme qui écrit proprement dix lettres vaut mieux qu'un alphabet complet qui déraille.



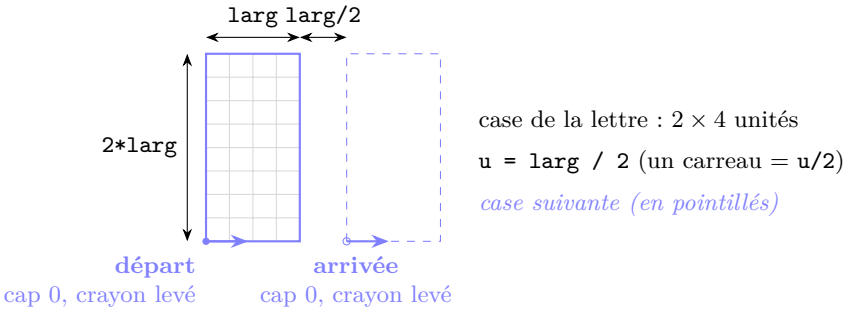

*Ce que l'on vise (étape 5).*

**Mode d'emploi**

- Fichier à télécharger (lien ci-dessus) : `projet_ecriture_depart.py` (le contrat, la fonction de test `verifier`, les outils de l'étape 3 et la lettre `L`). Renommez-le `ecriture_NOM.py`. Laissez `t.speed(0)` au début : sinon, vous passerez la séance à regarder la tortue dessiner.
- Paliers : **Étapes 1–2** le minimum attendu **Étapes 3–4** le cœur du projet **Étapes 5–6** pour aller plus loin.
- Travaillez d'abord **sur le cahier à petits carreaux** (une case de lettre = 4 carreaux de large sur 8 de haut, un carreau = une demi-unité ; voir le schéma du contrat), puis codez, puis **testez** chaque lettre avant de passer à la suivante. Le journal de bord de chaque lettre se tient aussi sur le cahier.
- **À l'oral**, le professeur désigne une lettre de votre programme : vous devez dire, ligne par ligne, **où est la tortue et où elle regarde**.
- Usage de l'IA : aucune pour les étapes 1 et 2 (c'est là que l'on apprend à « être la tortue ») ; en appui ensuite (déboguer, comprendre une erreur), mais chaque lettre est dessinée et expliquée par vous. Voir la charte d'usage de l'IA, au début du livre.

## Le contrat d'une lettre

On note `larg` la **largeur** des lettres (en pixels) : une lettre tient dans un rectangle de largeur `larg` et de hauteur `2 * larg`. On découpe ce rectangle en un quadrillage d'unité `u = larg / 2` : une lettre fait **2 unités de large** et **4 unités de haut**, et on laisse **1 unité** d'espace avant la lettre suivante.




### Le contrat de toutes les lettres

- **Départ** : la tortue est au **coin bas-gauche** de la case, elle regarde **vers la droite** (cap 0), crayon **levé**.
- **Pendant** : la lettre ne sort pas de sa case (2 × 4 unités).
- **Arrivée** : la tortue est au **coin bas-gauche de la case suivante** (`3u` plus à droite, à la même hauteur), cap 0, crayon levé.

En informatique, le départ s'appelle une **précondition** et l'arrivée une **postcondition** : c'est la **spécification** de la fonction.

### Interdits

`goto`, `setposition`/`setpos`, `setx`, `sety`, `home`, `setheading`, `teleport` sont **interdits**. On ne se déplace qu'avec `forward`, `backward`, `left`, `right`, `circle`, `penup`, `pendown`. Une lettre qui utilise un ordre interdit ne compte pas.
*Pourquoi ?* Avec `goto`, on n'a jamais besoin de savoir où l'on est. Ici, à chaque ligne du programme, **vous devez savoir où est la tortue et où elle regarde**.

## Étape 1 — Être la tortue : les lettres « bâton »

On commence par les lettres faites de traits **horizontaux et verticaux**. Voici le `L`, entièrement commenté :

```
def lettre_L(larg):
    """Dessine un L de largeur larg (contrat respecte)."""
    haut = 2 * larg    # hauteur de la lettre
    ecart = larg / 2   # espace avant la lettre suivante
    t.left(90)         # je regarde vers le haut
    t.forward(haut)    # je monte en haut de la case (crayon leve)
    t.pendown()
    t.backward(haut)   # je redescends en tracant la barre verticale
    t.right(90)        # je regarde de nouveau vers la droite
    t.forward(larg)    # barre du bas
    t.penup()
    t.forward(ecart)   # je rejoins la case suivante
```

1

### Remarque

On évite d'appeler une variable `l` : dans le code, on la confond trop facilement avec le chiffre `1` ; d'où `larg`. Le fichier de départ contient aussi la plus simple des « lettres », l'espace :

```
def espace(larg):
    """Une case vide."""
    t.penup()
    t.forward(larg + larg / 2)
```

### Exemple — *Le journal de bord de la tortue*

Pour écrire une lettre, on tient à la main son **journal de bord** : après chaque ordre, la position, le cap et le crayon. Pour le `L` (départ en (0,0)) :

| ordre | position | cap | crayon |
|---|---|---|---|
| (départ) | (0,0) | 0 | levé |
| `left(90)` | (0,0) | 90 | levé |
| `forward(haut)` | (0, `2*larg`) | 90 | levé |
| `pendown()` | (0, `2*larg`) | 90 | baissé |

| ordre | position | cap | crayon |
|---|---|---|---|
| `backward(haut)` | (0,0) | 90 | baissé |
| `right(90)` | (0,0) | 0 | baissé |
| `forward(larg)` | (`larg`, 0) | 0 | baissé |
| `penup()` | (`larg`, 0) | 0 | levé |
| `forward(ecart)` | (`1.5*larg`, 0) | 0 | levé ✓ |

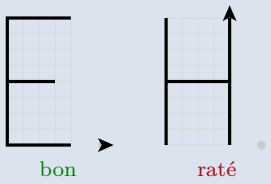
La dernière ligne est exactement l'**arrivée** du contrat.

1. Sur le cahier à petits carreaux, dessiner le `E` dans sa case, puis écrire son journal de bord (colonnes : ordre, position, cap, crayon, comme ci-dessus) **avant** de le programmer.
2. ▶ Programmer `lettre_E`, puis `lettre_F`, `lettre_H`, `lettre_I`, `lettre_T`.
3. ▶ Tester chaque lettre (voir l'encadré ci-dessous) **avant** de passer à la suivante.

### Tester une lettre

Le fichier de départ fournit deux tests ; vous n'avez pas à les écrire.

**Le test visuel** `essayer(lettre_E)` dessine en gris le quadrillage de la case et le point d'arrivée attendu, puis la lettre, puis un **tampon** de la tortue (`t.stamp()`). Le contrat est respecté si la flèche est **sur le point gris** et pointe **vers la droite**. On voit aussi tout de suite si la lettre déborde de sa case.




**Le test automatique** `verifier(lettre_E)` dessine la lettre puis **contrôle** l'arrivée (position, cap, crayon) avec des `assert` : s'il ne se passe rien, le contrat est respecté ; sinon, Python s'arrête avec un message (`AssertionError: mauvaise position d'arrivee`). C'est le **seul** endroit où l'on a le droit de demander sa position à la tortue.

### Remarque — *Pourquoi* `verifier` *tolère-t-il un demi-pixel d'écart ?*

Les calculs sur les nombres à virgule sont **approchés** (`0.1 + 0.2` ne vaut pas exactement `0.3` en Python), et `circle` trace en réalité de petits segments. Une lettre correcte peut donc arriver en `75.00000001` au lieu de `75` : on ne compare jamais deux nombres à virgule avec `==`, on vérifie qu'ils sont **assez proches**.

**Point d'étape ✓** En enchaînant `lettre_H(40)`, `lettre_E(40)`, `lettre_L(40)`, `lettre_L(40)`, la tortue écrit `HELL`, lettres alignées et régulièrement espacées — sans aucun `goto`.

## Étape 2 — Écrire un mot, puis une phrase

Il reste à faire le lien entre un **caractère** (`"E"`) et la **fonction** qui le dessine (`lettre_E`).

```
def ecrire(phrase, larg):
    """Ecrit la phrase caractere par caractere, en lettres de largeur larg."""
    for c in phrase.upper():          # on ecrit tout en majuscules (pour l'instant)
        if c == "E":
            lettre_E(larg)
        elif c == "L":
            lettre_L(larg)
        elif c == " ":
            espace(larg)
        # ... une branche par lettre programmee ...
        else:
            inconnu(larg)             # caractere que l'on ne sait pas dessiner

phrase = input("Votre phrase : ")
ecrire(phrase, 30)
```

4. ▶ Compléter `ecrire` avec toutes vos lettres.
5. ▶ Écrire `inconnu(larg)` : elle dessine un petit **rectangle vide** (c'est ce que font les vraies polices quand un caractère manque ; les typographes l'appellent le « tofu »). Elle doit, elle aussi, respecter le contrat !
6. Pourquoi le contrat est-il indispensable pour que `ecrire` fonctionne ? Que se passerait-il si `lettre_H` finissait en haut de sa case ?

**Remarque** — *Notation : `phrase.upper()`*
`ch.upper()` renvoie la chaîne `ch` écrite en majuscules ; plus loin, `c.islower()` dira si le caractère `c` est une minuscule. Cette écriture « avec un point » s'appelle une **méthode** : le chapitre *Les types construits* en présentera d'autres.

**Remarque** — *Deux pièges*

- `input` attend la phrase dans la **console**, pas dans la fenêtre de la tortue : cliquez dans la console pour taper. Autre possibilité, une petite fenêtre de saisie : `phrase = t.textinput("La tortue", "Votre phrase :")`.
- `"é".upper()` vaut `"É"`, une lettre que vous n'avez pas (encore) dessinée : les accents sortent en « tofu » jusqu'à l'étape 6. Ce n'est pas un bug !

**Remarque** — *Variante : la police de la classe*
On se répartit l'alphabet : chaque binôme programme quelques lettres, puis on rassemble toutes les fonctions dans un seul fichier. Si **tout le monde** a respecté le contrat, `ecrire` fonctionne avec des lettres écrites par des élèves différents, sans rien retoucher. C'est tout l'intérêt d'une **spécification** : chacun peut utiliser le travail des autres sans savoir comment il est fait. Pour contrôler 26 lettres d'un coup, le test automatique `verifier` devient précieux.

**Point d'étape ✓** Le programme demande une phrase et l'écrit sur une ligne. Les lettres non encore programmées apparaissent en « tofu ».

## Étape 3 — Les diagonales

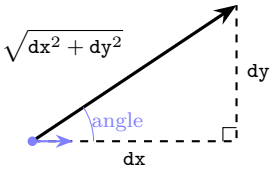
Pour le `A`, le `N` ou le `Z`, il faut tracer en **biais**. Il faut alors connaître la **longueur** du trait (théorème de Pythagore) et l'**angle** dont tourner. Le fichier de départ fournit deux outils qui font ce calcul :

```
def trait(dx, dy):
    """Trace un segment de dx vers la droite et dy vers le haut.
    Le cap doit etre 0 avant ; il est de nouveau 0 apres."""
    angle = math.degrees(math.atan2(dy, dx))    # l'angle du segment
    t.pendown()
    t.left(angle)
    t.forward(math.sqrt(dx ** 2 + dy ** 2))     # Pythagore
    t.right(angle)

# saut(dx, dy) : pareil, mais crayon leve
```

Pour aller de `dx` vers la droite et `dy` vers le haut, la tortue suit l'**hypoténuse** d'un triangle rectangle :

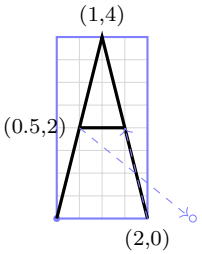
- sa **longueur** vient du théorème de Pythagore : $\sqrt{\texttt{dx}^2 + \texttt{dy}^2}$ ;
- l'**angle** dont il faut tourner est donné par `math.atan2(dy, dx)` (en radians, que `math.degrees` convertit en degrés). Vous n'avez pas à savoir le calculer : retenez seulement **ce qu'il représente**.




Après le trait, `t.right(angle)` annule le virage : la tortue regarde de nouveau vers la droite.

> **Remarque**
> `trait` et `saut` sont des déplacements **relatifs** : ils disent *de combien* bouger, pas *où* aller. Ce ne sont donc pas des `goto` déguisés : vous devez toujours savoir d'où vous partez. Elles ont leur propre petit contrat : **cap 0 avant, cap 0 après**. **Attention** : si la tortue ne regarde pas vers la droite au moment de l'appel (par exemple juste après un `circle`), le trait part dans une mauvaise direction.

```
def lettre_A(larg):
    u = larg / 2              # une unite du quadrillage
    trait(u, 4 * u)           # jambe gauche : -> (u, 4u)
    trait(u, -4 * u)          # jambe droite : -> (2u, 0)
    saut(-u / 2, 2 * u)       # -> (1.5u, 2u)
    trait(-u, 0)              # la barre : -> (0.5u, 2u)
    saut(2.5 * u, -2 * u)     # case suivante : -> (3u, 0)
```




On repère les points **en unités** sur le quadrillage : c'est beaucoup plus simple que de raisonner en pixels, et la lettre s'adapte toute seule à la taille `larg`.

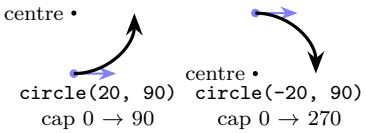
**7.** Dans `trait`, à quoi sert la dernière ligne `t.right(angle)` ? Que se passerait-il sans elle pour la lettre suivante ?

**8.** ▶ Programmer `K`, `M`, `N`, `V`, `W`, `X`, `Y`, `Z`. Réécrire éventuellement vos lettres de l'étape 1 avec `trait` et `saut` (comparer la longueur du code).

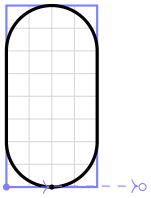
## Étape 4 — Les arrondis

`t.circle(r, angle)` trace un **arc** de cercle de rayon `r` ; la tortue tourne de `angle` degrés en chemin.

- `r > 0` : le centre est à **gauche** de la tortue, elle tourne vers la gauche ;
- `r < 0` : le centre est à **droite**, elle tourne vers la droite ;
- **attention** : après `circle`, le **cap a changé** ! Notez-le en commentaire à chaque fois ;
- avant un `trait` ou un `saut`, revenir au **cap 0** (`left`/`right`).




```
def lettre_O(larg):
    u = larg / 2
    saut(u, 0)              # milieu du bas, cap 0
    t.pendown()
    t.circle(u, 90)         # cap 90
    t.forward(2 * u)        # cote droit
    t.circle(u, 180)        # cap 270
    t.forward(2 * u)        # cote gauche
    t.circle(u, 90)         # cap 0 : retour au milieu du bas
    saut(2 * u, 0)          # case suivante
```



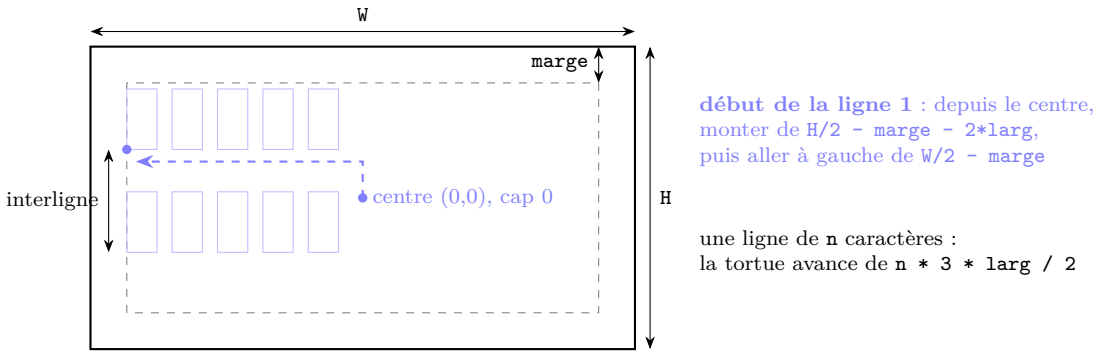
**9.** Écrire sur le cahier le journal de bord du `O` (position et cap après chaque ligne).

**10.** ▶ Programmer `C`, `D`, `U`, `J`, `P`, `B`, `R`, `G`, `Q`, et enfin `S` (deux arcs de sens contraires).

**Point d'étape ✓** Les 26 lettres respectent le contrat (`essayer` ou `verifier`). La tortue écrit n'importe quelle phrase en majuscules sur une ligne.

## Étape 5 — La mise en page

Au lancement, la tortue est au **centre** de la fenêtre, cap 0. C'est le seul point que l'on connaît « gratuitement » : tout le reste se calcule à partir de là. La taille de la fenêtre s'obtient avec `t.window_width()` (largeur `W`) et `t.window_height()` (hauteur `H`).




**11. Partir du bon endroit.** ▶ Écrire une fonction qui amène la tortue, depuis le centre, au début de la première ligne (en haut à gauche, en laissant une marge), et qui la laisse avec cap 0 et crayon levé.

**12. Aller à la ligne.** Si une ligne contient `n` caractères, de combien la tortue a-t-elle avancé ?
▶ En déduire le code de `retour_ligne` :

1

```
def retour_ligne(n, larg):
    """Depart : fin d'une ligne de n caracteres, cap 0, crayon leve.
    Arrivee : debut de la ligne suivante, cap 0, crayon leve."""
    t.penup()
    t.backward( ..... )     # (a) je reviens au debut de la ligne
    t.right(90)
    t.forward( ..... )      # (b) je descends d'un interligne (3 * larg ?)
    t.left(90)
```

13. **Adapter la taille.** La largeur utile est `L = W - 2 * marge`. Pour que `n` caractères tiennent sur une ligne, il faut $\texttt{n} \times \frac{3\,\texttt{larg}}{2} \leqslant \texttt{L}$, c'est-à-dire $\texttt{larg} \leqslant \frac{2\,\texttt{L}}{3\,\texttt{n}}$. ▶ Calculer ainsi la plus grande largeur `larg` possible ; si elle devient trop petite (moins de 10 pixels, par exemple), passer sur plusieurs lignes.

14. **Défi** ▶ Couper la phrase **entre les mots** (`phrase.split()`), jamais au milieu d'un mot, puis **centrer** chaque ligne. `phrase.split()` *renvoie une* ***liste*** *de mots : une liste permet de stocker plusieurs éléments à la suite (on y reviendra en détail au chapitre* Les types construits*).*

## Étape 6 — Le style (à la carte)

Choisissez vos améliorations ; chacune doit continuer à respecter le contrat.

- **Épaisseur et couleur** : `t.pensize(larg / 10)` (épaisseur proportionnelle à la taille), `t.color(...)`. Pour un tracé instantané : `t.tracer(0)` au début et `t.update()` à la fin.
- **Majuscules et minuscules** : voir « Majuscules et minuscules » ci-dessous.
- **Lettres plus belles** : empattements, italique… voir « Des empattements » ci-dessous.
- **Accents et ponctuation** : dessiner `e`, puis revenir en arrière pour poser l'accent ; `. , ! ? '` ; les chiffres.
- **Défi Écriture liée** : les lettres s'enchaînent sans lever le crayon. Il faut alors **changer le contrat** (où commence et où finit chaque lettre ?) et l'écrire dans les docstrings.

### Majuscules et minuscules

Jusqu'ici, `ecrire` passait tout en majuscules avec `phrase.upper()`. Pour respecter la casse, on retire le `.upper()` et on confie chaque caractère à une fonction `caractere(c, larg)` qui décide comment le dessiner. Pour y voir clair, on commence par sortir la longue chaîne de `if`/`elif` de `ecrire` et par la ranger dans sa propre fonction, `dessiner(c, larg)`. Deux idées, ensuite, que l'on peut combiner.

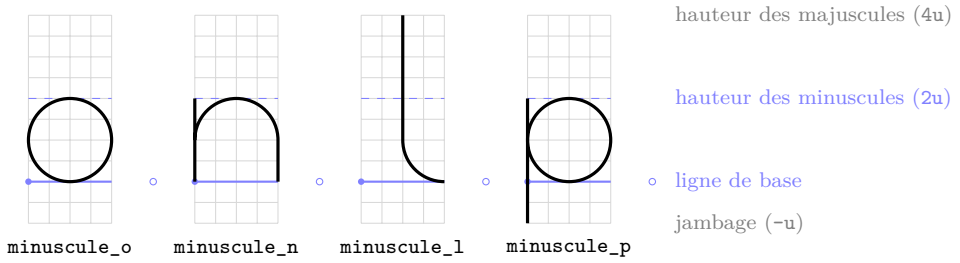
**Idée 1 — la petite capitale (simple).** Une minuscule, c'est la majuscule **en plus petit**. Mais une lettre de taille `petit` n'avance que de `3 * petit / 2` : on **complète** le déplacement pour que la case garde sa largeur normale, sinon les lettres se chevauchent et le contrat n'est plus respecté.

```
def dessiner(c, larg):
    """Dessine le caractere c en taille larg (tofu s'il est inconnu)."""
    if c == "A":                        # la chaine de if/elif de l'etape 2,
        lettre_A(larg)                  # sortie de ecrire et rangee
    elif c == "B":                      # dans sa propre fonction
        lettre_B(larg)
    # ... une branche par lettre ...
    elif c == " ":
        espace(larg)
    else:
        inconnu(larg)
```

```
12
13 def caractere(c, larg):
14     """Majuscule en taille larg ; minuscule = la majuscule en plus petit."""
15     if c.islower():
16         petit = 0.65 * larg
17         dessiner(c.upper(), petit)       # la case avance de 3 * petit / 2...
18         t.forward(3 * (larg - petit) / 2)   # ... jusqu'a 3 * larg / 2
19     else:
20         dessiner(c, larg)
```

`c.islower()` vaut `True` pour une lettre minuscule ; `"Tortue"` s'écrit alors TORTUE, comme dans les titres de journaux.

**Idée 2 — la vraie minuscule (ambitieuse).** On dessine une autre forme, avec trois repères sur le quadrillage : la **ligne de base** (`y = 0`), la **hauteur des minuscules** (`2u = larg`, la moitié d'une majuscule) et le **jambage** qui descend sous la ligne (jusqu'à `-u`, comme dans le `p`) ou qui monte jusqu'à `4u` (comme dans le `l`).




```
def minuscule_o(larg):
    u = larg / 2
    saut(u, 0)          # milieu du bas
    t.pendown()
    t.circle(u)         # tour complet, cap 0
    saut(2 * u, 0)

def minuscule_l(larg):
    u = larg / 2
    saut(u, 4 * u)      # -> (u, 4u)
    trait(0, -3 * u)    # -> (u, u)
    t.right(90)         # cap 270
    t.circle(u, 90)     # -> (2u, 0), cap 0
    saut(u, 0)
```

```
def minuscule_n(larg):
    u = larg / 2
    trait(0, 2 * u)     # -> (0, 2u)
    saut(0, -u)         # -> (0, u)
    t.left(90)          # cap 90
    t.pendown()
    t.circle(-u, 180)   # arche : -> (2u, u)
    t.forward(u)        # -> (2u, 0), cap 270
    t.penup()
    t.left(90)          # cap 0
    saut(u, 0)

def minuscule_p(larg):
    u = larg / 2
    saut(0, 2 * u)
    trait(0, -3 * u)    # jambage : -> (0, -u)
    saut(0, 2 * u)      # -> (0, u)
    t.left(90)          # cap 90
    t.pendown()
    t.circle(-u)        # la panse
    t.penup()
    t.right(90)         # cap 0
    saut(3 * u, -u)
```

On ajoute en tête de `caractere` une branche par vraie minuscule, qui appelle sa fonction ; `caractere` choisit ainsi la vraie minuscule si elle existe, sinon la petite capitale.

```
1  def caractere(c, larg):
2      if c == "o":                      # une vraie minuscule existe
3          minuscule_o(larg)
4      elif c == "n":
5          minuscule_n(larg)
6      elif c == "l":
7          minuscule_l(larg)
8      elif c == "p":
9          minuscule_p(larg)
10     elif c.islower():                 # sinon : petite capitale
11         petit = 0.65 * larg
12         dessiner(c.upper(), petit)
```

```
13          t.forward(3 * (larg - petit) / 2)
14      else:                                   # majuscule, chiffre, ponctuation...
15          dessiner(c, larg)
```

Le jambage du `p` **sort de la case** : on **adapte le contrat**. Le départ et l'arrivée restent sur la ligne de base, mais la case descend jusqu'à `-u` (pensez à augmenter l'interligne). Écrivez ce nouveau contrat dans les docstrings.

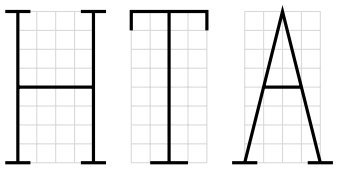
### Des empattements

Les **empattements** sont les petits traits qui terminent les jambes des lettres. Le texte que vous lisez en ce moment en a (police « à empattements », comme le Times) ; les noms de fonctions en police machine aussi. Les polices qui n'en ont pas (Arial, Helvetica…) sont dites « *sans serif* », de l'anglais *serif*, empattement.

On écrit **un seul outil**, puis on l'appelle à chaque extrémité d'une jambe :

```
def empattement(largeur):
    """Petit trait centre sur la tortue ;
    retour au point de depart, cap 0, crayon leve."""
    saut(-largeur / 2, 0)
    trait(largeur, 0)
    saut(-largeur / 2, 0)

def lettre_H_serif(larg):
    u = larg / 2
    empattement(2 * u / 3)  # pied gauche
    trait(0, 4 * u)
    empattement(2 * u / 3)  # tete gauche
    saut(0, -2 * u)
    trait(2 * u, 0)         # barre
    saut(0, 2 * u)
    empattement(2 * u / 3)  # tete droite
    trait(0, -4 * u)
    empattement(2 * u / 3)  # pied droit
    saut(u, 0)
```



*H, T et A à empattements*

```
def lettre_T_serif(larg):
    u = larg / 2
    saut(0, 3.5 * u)
    trait(0, u / 2)      # crochet gauche
    trait(2 * u, 0)      # barre du haut
    trait(0, -u / 2)     # crochet droit
    saut(-u, u / 2)      # -> (u, 4u)
    trait(0, -4 * u)     # -> (u, 0)
    empattement(u)       # pied
    saut(2 * u, 0)
```

```
def lettre_A_serif(larg):
    u = larg / 2
    empattement(2 * u / 3)  # pied gauche
    trait(u, 4 * u)
    trait(u, -4 * u)
    empattement(2 * u / 3)  # pied droit
    saut(-u / 2, 2 * u)
    trait(-u, 0)
    saut(2.5 * u, -2 * u)
```

**Remarque** — *Pourquoi `2 * u / 3` ?*
Un empattement dépasse de la case de la moitié de sa largeur, donc ici de `u/3`, dans l'espace entre deux lettres (qui mesure `u`). Avec des empattements plus larges que `u`, ceux de deux lettres voisines se toucheraient. Le départ et l'arrivée, eux, ne changent pas : le contrat est respecté, et `empattement` a le sien (« je reviens où j'étais »).

**Remarque** — *Encore une idée : l'italique en une ligne*
Pour pencher les lettres, il suffit de modifier `trait` et `saut` : on décale chaque déplacement vers la droite proportionnellement à sa montée, en ajoutant en première ligne `dx = dx + 0.25 * dy`. Toutes les lettres écrites avec `trait` et `saut` se penchent d'un coup, et elles respectent toujours le contrat : une lettre redescend autant qu'elle monte, donc les décalages s'annulent à l'arrivée. *Limite* : les arcs de `circle` ne se penchent pas, et une lettre qui en contient peut ne plus arriver au bon endroit : vérifiez-la.

**Erreurs fréquentes**

- appeler `trait` ou `saut` alors que la tortue ne regarde pas vers la droite (juste après un `circle`, un `left`…) ;
- oublier `pendown()` (rien ne se dessine), ou oublier `penup()` à la fin (un trait parasite relie la lettre à la suivante) ;
- se tromper de signe dans `circle` : `r > 0` tourne à gauche, `r < 0` tourne à droite ;
- écrire les longueurs **en pixels** (`forward(50)`) au lieu de les écrire en fonction de `larg` (`forward(larg)`) : la lettre est juste en taille 50 et fausse dans toutes les autres. Testez avec deux tailles !
- programmer une lettre sans l'avoir dessinée sur le quadrillage : on se perd au bout de trois lignes ;
- utiliser `goto` « juste pour une lettre » : elle ne compte pas.

**Remarque** — *Un peu d'histoire : des lettres faites de traits*

Dans les années 1960, les ordinateurs dessinaient sur des **traceurs** : un vrai stylo, déplacé par des moteurs, qui se lève et se baisse — exactement comme notre tortue. Pour y écrire du texte, il fallait décrire chaque lettre comme une **suite de traits**. En 1967, **Allen V. Hershey**, chercheur dans un laboratoire de la marine américaine, publie ainsi des centaines de caractères (latins, grecs, symboles mathématiques…), chacun défini par une suite de points sur une petite grille : votre quadrillage de conception, en somme. Les **polices Hershey** servent encore aujourd'hui pour graver ou découper des lettres avec des machines à commande numérique.

Les polices de vos écrans fonctionnent autrement : elles décrivent le **contour** des lettres par des courbes, que l'ordinateur remplit. Et quand un caractère manque, elles affichent le fameux rectangle vide, le « tofu ». Google a d'ailleurs baptisé sa famille de polices couvrant presque toutes les écritures du monde **Noto**, pour « *no more tofu* ».

## Grille d'évaluation

| Critère |
|---|
| **Contrat** : toutes les lettres le respectent (`essayer` ou `verifier`) ; aucun ordre interdit |
| **Alphabet** : lettres bâton ; diagonales ; arrondis |
| **Écrire une phrase** : `input`, espace, caractère inconnu, `ecrire` |
| **Mise en page** : départ calculé, retour à la ligne, taille adaptée (centrage en bonus) |
| **Style** : au moins deux améliorations de l'étape 6 |
| **Code et oral** : une fonction par lettre, docstrings, commentaires de position ; vous savez dérouler n'importe quelle lettre |

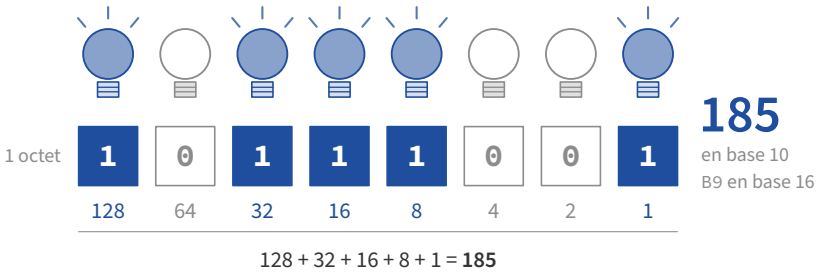
**Remarque** — *Ce que le projet fait travailler*

**Décomposer** un gros problème en petites fonctions ; **spécifier** chacune par un contrat (précondition, postcondition) ; **tester** (à l'œil avec `essayer`, automatiquement avec des `assert`) ; **réutiliser** (`ecrire` ne connaît rien au dessin des lettres, elle se contente d'appeler les fonctions en leur faisant confiance).

2

CHAPITRE 2

# Le binaire et l'écriture des nombres




DANS CE CHAPITRE


- Activités préparatoires . . . 91
- Cours . . . 93
- Exercices . . . 108
- TP et projets . . . 116
- Fiche de révision . . . 129
- Défi Advent of Code . . . 132
- Corrigés . . . 552

# ACTIVITÉS PRÉPARATOIRES



## ACTIVITÉ 1 Les cartes à points

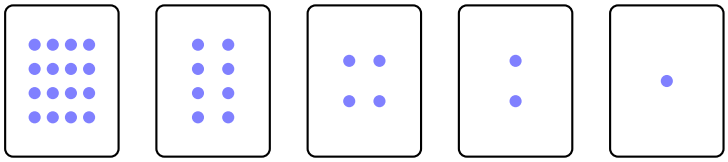
▶ corrigé p. 552

*compter avec des cartes qu'on montre ou qu'on cache*

Durée : 25 min · Par binômes, sans machine

### Consignes

- Les réponses s'écrivent sur le cahier. Matériel : 5 cartes portant 1, 2, 4, 8 et 16 points (au dos : rien), posées **dans cet ordre, de droite à gauche**.
- Règle du jeu : une carte est soit **montrée** (on compte ses points), soit **cachée** (elle ne compte pas). Le nombre représenté est le **total des points visibles**.
- L'un **annonce** un nombre, l'autre **retourne** les cartes pour le représenter ; on échange les rôles à chaque question.



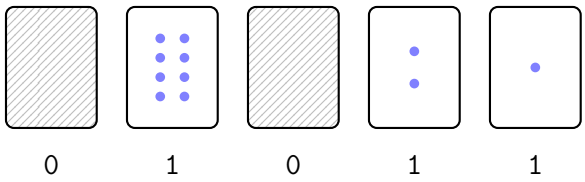

### Exercice 1 *Observer les cartes*

**1.** Comment passe-t-on du nombre de points d'une carte à celui de la carte placée à sa gauche ?

**2.** Si l'on ajoutait une sixième carte à gauche, combien de points porterait-elle ?

### Exercice 2 *Représenter des nombres*

Ci-dessous, la configuration **cachée, montrée, cachée, montrée, montrée** représente $8 + 2 + 1 = 11$. On la note `01011` : `1` pour une carte montrée, `0` pour une carte cachée.




**3.** Représenter avec les cartes, puis noter sur le cahier avec des `0` et des `1`, les nombres 5, 9, 21 et 30.

**4.** Dans l'autre sens, quels nombres représentent `10010`, `00111` et `11111` ?

**5.** Quel est le **plus petit** nombre qu'on peut représenter ? le **plus grand** ?

**6.** Peut-on représenter **tous** les nombres entre ces deux-là ? Un même nombre peut-il s'obtenir de deux façons différentes ? Essayer avec 6, 13, 27.

**7.** Combien de configurations différentes (montrée/cachée) existe-t-il pour 5 cartes ? Comparer avec la question 5.

### Exercice 3 *Compter, ajouter une carte*

**8.** Compter de 0 à 8 en retournant les cartes, et noter chaque configuration sur le cahier, en continuant la suite : 0 = `00000`, 1 = `00001`, 2 = … jusqu'à 8. Que fait la carte « 1 point » à chaque étape ? Que se passe-t-il quand on passe de 7 à 8 ?

**9.** On ajoute la sixième carte (question 2). Quel est maintenant le plus grand nombre représentable ? Et avec 8 cartes ?

**10.** Remarquer : le plus grand nombre avec 5 cartes, plus 1, donne… quoi ?

### Exercice 4 *Des cartes aux interrupteurs*

Dans un ordinateur, il n'y a pas de cartes, mais des milliards de minuscules **interrupteurs** (les transistors) : le courant passe, ou ne passe pas.

**11.** Une rangée de 5 lampes affiche : éteinte, allumée, allumée, éteinte, allumée. En la lisant comme les cartes, quel nombre transmet-elle ?

**12.** Pourquoi est-il plus sûr de n'utiliser que **deux** états (allumé / éteint) plutôt que dix niveaux de luminosité différents ?

**13.** **Mettre des mots.** Recopier et compléter sur le cahier : « Chaque carte n'a que …… états possibles. Les points des cartes sont les puissances de …… : 1, 2, 4, 8, 16… Avec $n$ cartes, on représente …… nombres différents, de 0 à …… Écrire un nombre avec des `0` et des `1`, c'est l'écrire en …… »

# COURS



**Programme (BO)** *« Écriture d'un entier positif dans une base $b \geqslant 2$ ; passer de la représentation d'une base dans une autre (bases 2, 10 et 16 privilégiées). Représentation binaire d'un entier relatif (complément à 2). Valeurs booléennes : 0, 1. Opérateurs booléens :* `and`, `or`, `not`. *Expressions booléennes. Représentation approximative des réels : notion de nombre flottant. Représentation d'un texte en machine : encodages ASCII, ISO-8859-1, Unicode. »*

**Prérequis** les bases de Python (opérateurs `//` et `%`, boucles, opérateurs `and`, `or`, `not`).
→ fiches de révision : ch. 1 p. 59

**Objectifs** comprendre *comment* une machine stocke un nombre ; *convertir* entre bases 2, 10 et 16 ; dresser une *table de vérité* et comprendre comment des *portes logiques* additionnent ; coder un entier *relatif* (complément à 2) ; comprendre *pourquoi* les nombres à virgule sont *approchés* ; comprendre comment un *texte* est codé (ASCII, Unicode) et *pourquoi* un même fichier peut s'afficher en charabia.

**Situation de départ**

1. Pourquoi un octet ne peut-il pas dépasser 255 ?
2. Pourquoi Python affiche-t-il `0.30000000000000004` quand on lui demande `0.1 + 0.2` ?
3. Pourquoi un texte s'affiche-t-il parfois avec « cafÃ© » à la place de « café » ?

## I Pourquoi le binaire ? De quoi parle-t-on dans la machine

### A. Deux états, et rien d'autre

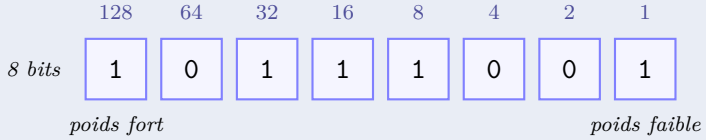
Au cœur d'un ordinateur, il n'y a ni chiffres, ni lettres, ni images : il y a des milliards de minuscules interrupteurs électroniques, les **transistors**. Chacun ne sait faire qu'une chose : **laisser passer le courant, ou non**. Deux états, deux seulement.

Pourquoi deux et pas dix ? Parce que distinguer *deux* niveaux (« il y a du courant » / « il n'y en a pas ») est **fiable** : même si la tension varie un peu à cause du bruit électrique, on sait encore de quel côté on est. Distinguer dix niveaux de tension différents serait bien plus fragile. Le choix de deux états est donc un choix de **robustesse**.

**Définition 1 — Bit et octet**

Un **bit** (de l'anglais *binary digit*, « chiffre binaire ») est la plus petite information : un `0` ou un `1`. On regroupe les bits par paquets de huit : un paquet de 8 bits s'appelle un **octet** (*byte* en anglais).

↻ **Retour sur l'activité.** La rangée de lampes allumées ou éteintes de l'activité « Les cartes à points » transmettait déjà des bits, et la question des dix niveaux de luminosité était celle de la robustesse : deux états se distinguent sans erreur.



*Un octet : 8 cases valant chacune* `0` *ou* `1`*. Ici, on lira bientôt le nombre* `185`*.*

## B. Tout est nombre, donc tout est bits

2

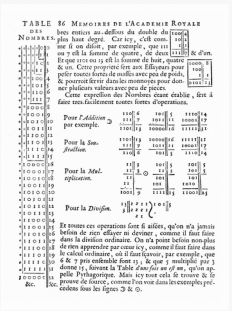
Une fois qu'on sait coder des *nombres* avec des `0` et des `1`, on sait tout coder : un texte (chaque caractère reçoit un numéro), une image (chaque pixel, trois nombres de couleur), un son (des milliers de mesures par seconde)… C'est l'idée fondatrice de l'informatique : **une même machine, manipulant des bits, peut tout représenter**. Ce chapitre s'attaque à la brique de base : **les nombres**.

**Pour les curieux** — *Un peu d'histoire* (facultatif)

Le système binaire est décrit dès **1703** par le mathématicien et philosophe allemand *Gottfried Wilhelm Leibniz*, dans un mémoire présenté à l'Académie royale des sciences de Paris, l'*Explication de l'arithmétique binaire* : on y voit, ci-contre, la table des premiers entiers écrits avec des `0` et des `1`, et les additions, soustractions et multiplications posées en binaire. Leibniz admire la simplicité de ces calculs, sans imaginer encore de machine pour les faire. C'est *Claude Shannon* qui, en **1937**, montre que l'algèbre des deux valeurs « vrai/faux » de *George Boole* permet de **concevoir des circuits électriques** : le pont était jeté entre la logique et la machine.



*Leibniz vers 1695 (portrait par C. B. Francke)*



*L'arithmétique binaire de Leibniz (1703)*

**Pour les curieux** — *Une blague d'informaticien* (facultatif)

La blague classique des informaticiens : « *Il existe 10 sortes de gens : ceux qui comprennent le binaire, et les autres.* » (Ici `10` se lit « deux »…)

# II Les systèmes de numération : la notion de base

## A. Le décimal, qu'on croit connaître

Nous comptons en **base 10** : dix chiffres (`0` à `9`), et dès qu'un rang est « plein », on passe au rang suivant. La position d'un chiffre lui donne un **poids**, qui est une puissance de 10 :

$$185 = 1 \times 10^2 + 8 \times 10^1 + 5 \times 10^0.$$

Le chiffre le plus à droite est au rang `0`. **Décomposer un nombre en puissances de sa base**, voilà toute l'affaire.

## B. Une base quelconque

**Définition 2 — Écriture en base $b$**

En base $b$, on dispose de $b$ chiffres (de `0` à $b-1$) et chaque rang pèse une puissance de $b$. Un nombre écrit $c_{n-1}\,c_{n-2}\cdots c_1\,c_0$ en base $b$ vaut

$$c_{n-1}\,b^{n-1} + c_{n-2}\,b^{n-2} + \cdots + c_1\,b^1 + c_0\,b^0.$$

La **base 2** (binaire) n'utilise que deux chiffres, `0` et `1`, et chaque rang pèse une puissance de 2. Pour lever toute ambiguïté, on note la base en indice : $185_{10}$, $\texttt{10111001}_2$.

**Propriété 1 — Combien de valeurs sur $n$ bits ?**

Avec **1** bit on code 2 valeurs (`0`, `1`) ; avec **2** bits, 4 (`00`, `01`, `10`, `11`) ; avec **3** bits, 8… Chaque bit ajouté **double** le nombre de possibilités : sur $n$ bits, on code $2^n$ valeurs, soit les entiers de 0 à $2^n - 1$.



↻ **Retour sur l'activité.** Les cartes à points portaient les poids de la base 2 : 1, 2, 4, 8, 16, chacune valant le double de sa voisine de droite. Une carte montrée était un `1`, une carte cachée un `0`, et les $2^5 = 32$ configurations des cinq cartes donnaient tous les nombres de 0 à $31 = 2^5 - 1$.

Un **octet** (8 bits) code donc $2^8 = 256$ valeurs : les entiers de `0` à `255`. C'est pourquoi ce nombre revient partout en informatique.

► **Exercices d'application :** exercice 1 (compter les possibilités)

# III Coder un entier positif en binaire

## A. Du binaire vers le décimal (le plus facile)

On applique directement la définition : on additionne les poids des rangs qui portent un `1`.

**Exemple** — *Lire un octet*

$$\texttt{10111001}_2 = 1{\cdot}128 + 0{\cdot}64 + 1{\cdot}32 + 1{\cdot}16 + 1{\cdot}8 + 0{\cdot}4 + 0{\cdot}2 + 1{\cdot}1 = 185_{10}.$$

## B. Du décimal vers le binaire : les divisions successives

**Règle 1 — Méthode des divisions par 2**

On divise le nombre par 2 en notant le **reste** (`0` ou `1`) ; on recommence avec le quotient, jusqu'à obtenir un quotient nul. Le résultat se lit en prenant les restes **de bas en haut**.

**Exemple** — *Convertir 185 en binaire*

$$\begin{aligned}
185 &= 2 \times 92 + \mathbf{1} && \text{(premier reste, lu en dernier : bit de poids faible)}\\
92 &= 2 \times 46 + \mathbf{0}\\
46 &= 2 \times 23 + \mathbf{0}\\
23 &= 2 \times 11 + \mathbf{1}\\
11 &= 2 \times 5 + \mathbf{1}\\
5 &= 2 \times 2 + \mathbf{1}\\
2 &= 2 \times 1 + \mathbf{0}\\
1 &= 2 \times 0 + \mathbf{1} && \text{(dernier reste, lu en premier : bit de poids fort)}
\end{aligned}$$

En lisant les restes de bas en haut : $185_{10} = \texttt{10111001}_2$. *(On retrouve bien l'octet du début du cours.)*

**Remarque** — ► *En pratique (console Python).*
Python parle binaire couramment. Un littéral binaire s'écrit avec le préfixe `0b`. À tester :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> 0b10111001      # ecrire un nombre en binaire
185
>>> bin(185)        # decimal -> binaire (chaine)
'0b10111001'
```

```
>>> int("10111001", 2)   # binaire -> decimal
185
```

La méthode des divisions par 2 est *exactement* l'algorithme que vous savez déjà écrire avec une boucle `while` (restes `n % 2`, réduction `n // 2`).

2

### C. Additionner en binaire

On pose l'addition comme en base 10, rang par rang **de droite à gauche**, en propageant les retenues. Il suffit de connaître quatre cas : $0+0=0$ ; $0+1=1$ ; $1+1=10_2$ (on pose `0`, on retient `1`) ; $1+1+1=11_2$ (on pose `1`, on retient `1`).

**Exemple** — *Poser* $13+11$ *en binaire*

| | | | | |
|---|---|---|---|---|
| retenues | 1 | 1 | 1 | 1 |
| | | 1 | 1 | 0 | 1 |
| + | | 1 | 0 | 1 | 1 |
| | 1 | 1 | 0 | 0 | 0 |

$11000_2 = 16+8 = 24 = 13+11.$

Une retenue qui sort à gauche ajoute un rang : la somme de deux nombres de 4 bits peut demander 5 bits. Cette addition, que le processeur effectue sans cesse, resservira plus loin pour coder les entiers négatifs (complément à deux).

↻ **Retour sur l'activité.** En comptant de 7 à 8 avec les cartes, les trois cartes de droite se cachaient et la carte « 8 points » se montrait : c'est l'addition $00111_2 + 1 = 01000_2$, dont la retenue se propage de rang en rang.

► **Exercices d'application :** exercices 2 à 6 (convertir binaire/décimal ; les programmer)

## IV Logique booléenne : des portes logiques à l'addition

Un transistor ne sait que laisser passer le courant, ou non. Pour qu'une machine *calcule*, on assemble les transistors en **portes logiques** : de petits circuits qui reçoivent un ou deux bits et en produisent un. C'est l'idée de Shannon évoquée plus haut : les deux valeurs d'un bit sont aussi les deux valeurs « vrai / faux » de la logique de Boole.

### A. Les opérateurs `et`, `ou`, `non`

**Définition 3 — Valeur booléenne, opérateur booléen**

Une **valeur booléenne** ne prend que deux valeurs : `1` (vrai) ou `0` (faux). Un **opérateur booléen** combine des valeurs booléennes et renvoie une valeur booléenne :

- **non** $a$ vaut `1` si $a$ vaut `0`, et `0` si $a$ vaut `1` : il **inverse** ;
- $a$ **et** $b$ vaut `1` **seulement si** $a$ et $b$ valent `1` tous les deux ;
- $a$ **ou** $b$ vaut `1` dès qu'**au moins un** des deux vaut `1`.

On résume le comportement d'un opérateur par sa **table de vérité** : une ligne par combinaison possible des entrées, avec la valeur obtenue en sortie.

| $a$ | non $a$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

| $a$ | $b$ | $a$ et $b$ | $a$ ou $b$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Ce sont exactement les opérateurs `not`, `and` et `or` de Python, rencontrés au chapitre *Les bases de la programmation Python* pour combiner des conditions, avec `True` à la place de `1` et `False` à la place de `0`. Dans un processeur, chacun est réalisé par une **porte logique** de quelques transistors, que les schémas électroniques dessinent ainsi :




*Les portes NON, ET, OU et OU exclusif (symboles américains, les plus répandus).*

## B. Le ou exclusif (`xor`)

Le **ou** de la logique est *inclusif* : 1 ou 1 vaut 1. Dans la langue courante, « fromage ou dessert » signifie plutôt « l'un ou l'autre, *mais pas les deux* » : c'est le **ou exclusif**.

**Définition 4 — Ou exclusif**

*a* **xor** *b* (*exclusive or*, « ou exclusif ») vaut `1` lorsque **exactement un** des deux bits vaut `1`, c'est-à-dire lorsque *a* et *b* sont **différents**.

| *a* | *b* | *a* ou *b* | *a* xor *b* |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 |

*Le ou et le ou exclusif ne diffèrent que sur la dernière ligne.*

**Remarque** — ▶ *En pratique (console Python).*
Python n'a pas de mot-clé `xor`. Mais puisque « ou exclusif » signifie « différents », sur deux booléens il suffit d'écrire `a != b` :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> True != False      # exactement un des deux est vrai
True
>>> True != True       # les deux sont vrais
False
```

## C. Dresser la table de vérité d'une expression

Une **expression booléenne** combine des variables à l'aide de ces opérateurs, par exemple « (*a* ou *b*) et non(*a* et *b*) ». Comme en calcul, les parenthèses indiquent ce que l'on évalue en premier.

**Règle 2 — Dresser la table de vérité d'une expression**

1. Prévoir **une ligne par combinaison des entrées** : avec $n$ variables, il y a $2^n$ lignes — les $2^n$ valeurs codables sur $n$ bits. On les énumère sans en oublier **en comptant en binaire** : `00`, `01`, `10`, `11`.
2. Prévoir **une colonne par étape du calcul**, des parenthèses les plus intérieures jusqu'à l'expression complète.
3. Remplir **colonne par colonne**, à l'aide des tables de **et**, **ou**, **non**.

**Exemple** — *Table de $E = (a$ ou $b)$ et non$(a$ et $b)$*

| $a$ | $b$ | $a$ ou $b$ | $a$ et $b$ | non$(a$ et $b)$ | $E$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 |

La dernière colonne vaut 0, 1, 1, 0 : c'est exactement la table du **xor**. Deux expressions qui ont la même table de vérité sont dites **équivalentes** : on vient de fabriquer un ou exclusif avec seulement des portes et, ou, non.

## D. Du calcul logique à l'addition : le demi-additionneur

Revenons à l'addition posée plus haut (« Additionner en binaire »). Dans la colonne de droite, on additionne deux bits $a$ et $b$. Le résultat tient sur **deux bits** : un bit de **somme** $s$, posé dans la colonne, et une **retenue** $r$, reportée dans la colonne de gauche.

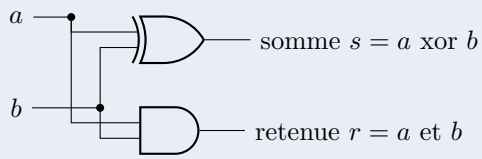
| $a$ | $b$ | $a+b$ | retenue $r$ | somme $s$ |
|---|---|---|---|---|
| 0 | 0 | $0 = 00_2$ | 0 | 0 |
| 0 | 1 | $1 = 01_2$ | 0 | 1 |
| 1 | 0 | $1 = 01_2$ | 0 | 1 |
| 1 | 1 | $2 = 10_2$ | 1 | 0 |

**Propriété 2 — Le demi-additionneur**

La colonne de la somme est la table du **xor**, celle de la retenue est la table du **et** :

$$s = a \text{ xor } b \qquad\qquad r = a \text{ et } b.$$

Le circuit formé d'une porte **xor** et d'une porte **et** s'appelle un **demi-additionneur** : il additionne deux bits.




**Pour les curieux** — *Pourquoi « demi » ?* (facultatif)
Dans les autres colonnes de l'addition posée, il faut aussi ajouter la **retenue venue de la droite** : trois bits et non deux (c'est le cas $1 + 1 + 1 = 11_2$ vu plus haut). On assemble pour cela deux demi-additionneurs en un **additionneur complet**, puis on place un additionneur complet par colonne : le processeur additionne ainsi des nombres de 64 bits avec, en tout et pour tout, des portes logiques (on retrouvera ce circuit dans l'unité arithmétique et logique, au chapitre *Architecture des ordinateurs et systèmes d'exploitation*). Le TP « Programmer l'arithmétique binaire » fait construire cette chaîne en Python, porte après porte.

► **Exercices d'application :** exercices 7 à 9 (opérateurs booléens, table de vérité d'une expression, demi-additionneur)

## V L'hexadécimal : le binaire en plus court

Écrire de longues suites de `0` et de `1` est pénible et source d'erreurs. On utilise donc très souvent la **base 16** (hexadécimal). Son intérêt : **un chiffre hexadécimal représente exactement 4 bits**, donc un octet tient en *deux* chiffres.

Il faut 16 chiffres : après `0`…`9`, on emprunte des lettres.

| déc | bin | hex | déc | bin | hex | déc | bin | hex | déc | bin | hex |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0000 | 0 | 4 | 0100 | 4 | 8 | 1000 | 8 | 12 | 1100 | C |
| 1 | 0001 | 1 | 5 | 0101 | 5 | 9 | 1001 | 9 | 13 | 1101 | D |
| 2 | 0010 | 2 | 6 | 0110 | 6 | 10 | 1010 | A | 14 | 1110 | E |
| 3 | 0011 | 3 | 7 | 0111 | 7 | 11 | 1011 | B | 15 | 1111 | F |

**Règle 3 — Binaire ↔ hexadécimal : regrouper par 4**

On **regroupe les bits par paquets de 4**, en partant de la droite (on complète par des `0` à gauche si besoin), et on remplace chaque paquet par son chiffre hexadécimal. Et réciproquement.

**Exemple**

$$\texttt{11011001}_2 = \underbrace{\texttt{1101}}_{\texttt{D}}\ \underbrace{\texttt{1001}}_{\texttt{9}} = \texttt{D9}_{16}, \qquad \texttt{7F}_{16} = \underbrace{\texttt{0111}}_{\texttt{7}}\ \underbrace{\texttt{1111}}_{\texttt{F}} = \texttt{01111111}_2.$$

Pour passer du **décimal** à l'hexadécimal, on divise par 16 (au lieu de 2) ; pour revenir, on multiplie par les puissances de 16 :

$$\texttt{3E8}_{16} = 3\cdot 16^2 + 14\cdot 16^1 + 8\cdot 16^0 = 768 + 224 + 8 = 1000_{10} \quad (\texttt{E} \text{ vaut } 14).$$

**Remarque** — ▶ *En pratique (console Python).*

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> hex(217)          # decimal -> hexadecimal
'0xd9'
>>> 0x7F              # ecrire un nombre en hexadecimal
127
>>> int("3E8", 16)
1000
```

On croise l'hexadécimal partout : les **couleurs** du Web (`#FF8000` = rouge `FF`, vert `80`, bleu `00`) et les **adresses mémoire** en sont deux exemples quotidiens.

▶ **Exercices d'application :** exercices 10 à 13 et 26 (l'hexadécimal ; changer de base)

## VI Les entiers relatifs : le complément à deux

Jusqu'ici, tous nos nombres étaient positifs. Comment coder $-12$ avec des bits, sans inventer de signe « moins » ?

## A. La fausse bonne idée : un bit de signe

On pourrait réserver le bit de poids fort au signe (`0` pour $+$, `1` pour $-$). Mais cette idée a un défaut rédhibitoire : elle crée **deux zéros**, un « $+0$ » (`00000000`) et un « $-0$ » (`10000000`), ce qui complique tous les calculs. On lui préfère une méthode plus astucieuse.

## B. Le complément à deux

**Règle 4 — Coder un entier négatif sur $n$ bits**

Pour coder $-x$ (avec $x > 0$) sur $n$ bits :

1. écrire $x$ en binaire sur $n$ bits ;
2. **inverser** tous les bits (`0`↔`1`) ;
3. **ajouter 1** au résultat.

**Exemple** — *Coder $-12$ sur 8 bits*

$$12 = \texttt{00001100} \xrightarrow{\text{on inverse}} \texttt{11110011} \xrightarrow{+1} \texttt{11110100}.$$

Donc $-12$ s'écrit `11110100` sur 8 bits.

Pourquoi ça marche ? Parce qu'avec cette convention, l'addition binaire ordinaire donne le bon résultat, **sans traitement spécial** : $12 + (-12)$ doit valoir 0.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| $12$ | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| $-12$ | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| $=$ | ~~1~~ | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

(la retenue déborde des 8 bits)

La retenue finale « déborde » au-delà des 8 bits : il reste `00000000`, soit 0. Tout fonctionne.

**Règle 5 — Lire le signe, et les bornes**

Avec le complément à deux sur $n$ bits :

- le **bit de poids fort** donne le signe : `0` ⇒ positif, `1` ⇒ négatif ;
- on code les entiers de $-2^{n-1}$ à $+2^{n-1} - 1$.

Sur 8 bits : de $-128$ à $+127$. Sur 16 bits : de $-32768$ à $+32767$.

**Règle 6 — Décoder un entier en complément à deux sur $n$ bits**

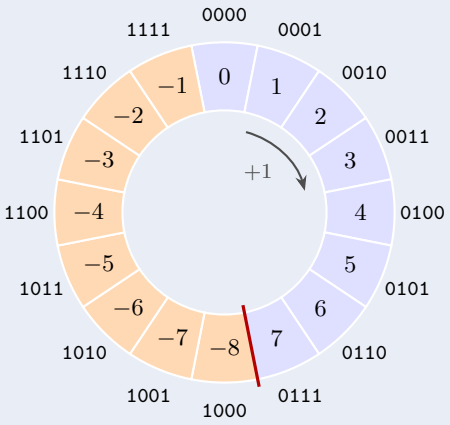
- Si le **bit de poids fort** vaut `0`, le nombre est positif : on le lit directement (somme des poids).
- S'il vaut `1`, le nombre est **négatif** : **inverser** tous les bits puis **ajouter 1** donne sa **valeur absolue**.

**Exemple** — *Décoder* `11111010` *sur 8 bits*
Le bit de poids fort vaut `1` : le nombre est négatif.

$$\texttt{11111010} \xrightarrow{\text{on inverse}} \texttt{00000101} \xrightarrow{+1} \texttt{00000110} = 6.$$

Donc `11111010` code $-6$. À l'inverse, `00101101` commence par `0` : il code directement $32+8+4+1=45$.

**Le cadran du complément à deux sur 4 bits.** On range les 16 mots de 4 bits en cercle, comme les heures d'une horloge. Ajouter 1, c'est avancer d'un cran dans le sens des aiguilles d'une montre : de `1111` ($-1$) on passe à `0000` (0), la retenue « déborde » comme plus haut. Les mots qui commencent par `0` (à droite) sont les positifs 0 à 7 ; ceux qui commencent par `1` (à gauche) sont les négatifs $-8$ à $-1$. Seul piège : en bas, de `0111` ($+7$) à `1000` ($-8$), on passe brutalement du plus grand au plus petit : c'est le **dépassement de capacité**.



dépassement : $7+1$ donne $-8$

**Remarque** — ▶ *En pratique (console Python).*
Les entiers de Python sont **de taille arbitraire** : ils grandissent tant qu'il reste de la mémoire (essayez `2 ** 200` !). Le complément à deux sur un nombre fixe de bits reste néanmoins la réalité du *matériel* et de nombreux langages, où un entier trop grand « déborde ».

▶ **Exercices d'application :** exercices 14 à 17 et 27 (entiers relatifs : complément à deux)

# VII Les nombres à virgule : les flottants

## A. Écrire une fraction en binaire

Après la virgule, les rangs pèsent des puissances **négatives** de 2 : $2^{-1}=0{,}5$, puis $2^{-2}=0{,}25$, etc. Pour convertir une partie fractionnaire, on **multiplie par 2** de façon répétée et on relève la partie entière (`0` ou `1`) à chaque étape.

**Exemple** — *Convertir* 0,1875

$$\begin{aligned} 0{,}1875 \times 2 &= \mathbf{0},375 \\ 0{,}375 \times 2 &= \mathbf{0},75 \\ 0{,}75 \times 2 &= \mathbf{1},5 \qquad \text{(on garde 0,5)} \\ 0{,}5 \times 2 &= \mathbf{1},0 \qquad \text{(reste 0 : on s'arrête)} \end{aligned}$$

On lit les parties entières **de haut en bas** : $0{,}1875_{10} = \texttt{0,0011}_2$. Donc $5{,}1875_{10} = \texttt{101,0011}_2$.

## B. Le piège fondamental : $0{,}1$ n'est pas exact

Reprenons la méthode sur 0,1 :

$$0{,}1 \times 2 = \mathbf{0}{,}2 \to \mathbf{0}{,}4 \to \mathbf{0}{,}8 \to \mathbf{1}{,}6 \to \mathbf{1}{,}2 \to \mathbf{0}{,}4 \to \cdots$$

Le motif se **répète sans fin** : $0{,}1_{10} = \texttt{0,0001100110011}\ldots_2$. Autrement dit, 0,1 **n'a pas d'écriture binaire finie**, exactement comme $\frac{1}{3} = 0{,}333\ldots$ n'a pas d'écriture décimale finie.

**Règle 7 — Les flottants sont approchés**

Une machine ne dispose que d'un nombre **fini** de bits (souvent 64) pour stocker un nombre à virgule, appelé **flottant** (*float*). Elle doit donc **arrondir** les écritures infinies : la valeur stockée est le plus souvent une **approximation**. On ne teste **jamais** l'égalité de deux flottants.

**Remarque** — ▶ *En pratique (console Python).*
La démonstration tient en une ligne :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Ce n'est **pas un bug** de Python (le résultat avait été constaté au chapitre *Les bases de la programmation Python*) : c'est la conséquence directe du fait que 0,1 et 0,2 ne sont pas représentables exactement en binaire. Pour comparer deux flottants, on teste si leur écart est *minuscule* : `abs(a - b) < 1e-9`, où `abs` donne la valeur absolue et `1e-9` désigne $10^{-9}$.

## C. Pour aller plus loin : la norme IEEE 754

*(Culture générale : le codage précis des flottants n'est pas exigible au programme de première.)*
Comme en base 10 on écrit $6{,}02 \times 10^{23}$, on met un flottant binaire sous la forme $1{,}\ldots \times 2^{e}$ (un seul 1 avant la virgule). La norme **IEEE 754** (1985) range alors, sur 32 bits (« simple précision ») :




- **1 bit de signe** (`0` positif, `1` négatif) ;
- **8 bits d'exposant**, stocké *décalé* de +127 (pour éviter un signe séparé) ;
- **23 bits de mantisse** (les décimales après le `1,`).

C'est la limitation de la mantisse à 23 (ou 52) bits qui *force* l'arrondi vu plus haut.

▶ **Exercices d'application :** exercices 18 à 20 (les nombres à virgule)

## VIII Combien de bits pour un nombre ?

Le programme demande de savoir **estimer la taille** d'un nombre en mémoire.

**Propriété 3 — Nombre de bits**

Un entier $N \geqslant 1$ s'écrit en binaire sur $k$ bits, où $k$ est le **plus petit** entier tel que $2^k > N$ (autrement dit $2^{k-1} \leqslant N < 2^k$). Repères utiles : $2^{10} = 1024 \approx 10^3$, donc 10 bits couvrent $\approx$ mille valeurs, 20 bits $\approx$ un million, 30 bits $\approx$ un milliard.

**Exemple**

$2^6 = 64 \leqslant 100 < 128 = 2^7$ : l'entier 100 s'écrit sur 7 bits ($100 = 1100100_2$).

**Somme et produit.** Si $a$ et $b$ s'écrivent sur $k$ bits au plus, alors $a < 2^k$ et $b < 2^k$. Donc $a + b < 2^k + 2^k = 2^{k+1}$ : la somme tient sur $k+1$ bits au plus (un bit de plus, à cause de la retenue). Et $a \times b < 2^k \times 2^k = 2^{2k}$ : le produit tient sur $2k$ bits au plus.

► **Exercices d'application :** exercice 28 (combien de bits ?)

**Pour les curieux** — *Les tailles « rondes » de l'informatique* (facultatif)

C'est ce comptage qui explique les tailles « rondes » de l'informatique : un octet → 256 valeurs ; 32 bits →≈ 4 milliards (la limite des « anciennes » adresses mémoire) ; 64 bits → un nombre astronomique, largement suffisant aujourd'hui.

## IX Coder le texte : de l'ASCII à Unicode

Nous savons coder des nombres. Or, comme annoncé au début du chapitre, **une machine ne sait *que* manipuler des nombres**. Coder un texte revient donc à une idée simple : **donner un numéro à chaque caractère**. C'est ce numéro, écrit en binaire, qui est réellement stocké.

### A. L'ASCII : la première table

**Définition 5 — Code ASCII**

L'**ASCII** (*American Standard Code for Information Interchange*, 1963) est une table qui associe à chaque caractère un numéro de `0` à `127`. Chaque caractère tient donc sur **7 bits** ($2^7 = 128$ valeurs).

| car. | code | car. | code | car. | code | car. | code |
|---|---|---|---|---|---|---|---|
| espace | 32 | `0` | 48 | `A` | 65 | `a` | 97 |
| `!` | 33 | `1` | 49 | `B` | 66 | `b` | 98 |
| `?` | 63 | `9` | 57 | `Z` | 90 | `z` | 122 |

*Extrait de la table ASCII (codes en décimal).*

Deux régularités bien pratiques : les chiffres, les majuscules et les minuscules se suivent *dans l'ordre* ; et une minuscule vaut toujours sa majuscule + 32 (`A`=65, `a`=97).

**À retenir — `ord` et `chr` en Python**

En Python, `ord` donne le code d'un caractère, et `chr` fait l'inverse.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> ord("A")        # caractere -> code
65
>>> chr(97)         # code -> caractere
'a'
>>> ord("A") == ord("a") - 32
True
```

Le `A` majuscule vaut donc $65 = 1000001_2$ : voilà ce que contient réellement l'octet qui stocke la lettre `A`.

## B. Trop petit : de l'ASCII à Unicode

L'ASCII a été conçu pour l'anglais : il ignore les **accents** (`é`, `à`, `ç`), sans parler du grec, du russe, du chinois ou des *émojis*. On a d'abord bricolé des tables sur **8 bits** (256 valeurs), comme l'**ISO-8859-1** (dite « Latin-1 »), qui ajoute les caractères d'Europe de l'Ouest. Mais 256 places ne suffisent pas au monde entier, et chaque région avait *sa* table incompatible : le même octet ne désignait pas la même lettre d'un pays à l'autre.

**Définition 6 — Unicode et UTF-8**

**Unicode** est un *catalogue universel* qui attribue un numéro unique, appelé **point de code** (noté `U+...`), à *tous* les caractères de *toutes* les écritures (plus de 150 000 aujourd'hui). **UTF-8** est la façon la plus répandue d'*écrire ces points de code en octets* : il utilise **1 octet** pour les caractères ASCII, et **2 à 4 octets** pour les autres.

UTF-8 a une qualité décisive : pour un caractère ASCII, il produit *exactement* l'octet ASCII. Un vieux fichier anglais reste donc valide, et c'est pourquoi UTF-8 s'est imposé partout (le Web est aujourd'hui à plus de 98 % en UTF-8).

**Remarque** — ▶ *En pratique (console Python).*
On **encode** un texte (caractères → octets) et on **décode** (octets → caractères).

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> "e".encode("utf-8")     # ASCII : 1 octet
b'e'
>>> "é".encode("utf-8")     # hors ASCII : 2 octets
b'\xc3\xa9'
>>> len("café")                    # 4 caracteres
4
>>> len("café".encode("utf-8"))    # mais 5 octets (le e accent en prend 2)
5
```

On mesure ici la différence entre **nombre de caractères** et **taille en octets** : elles ne coïncident que si tout le texte est en ASCII.

## C. Le charabia (*mojibake*) : quand on se trompe d'encodage

Un fichier texte n'est qu'une suite d'octets : **rien** n'y indique de façon absolue quel encodage a servi à l'écrire. Si on *écrit* un fichier en UTF-8 mais qu'un logiciel le *relit* comme du Latin-1, chaque octet est mal interprété et le texte s'affiche en charabia.

**Exemple** — *D'où vient le fameux « Ã© »*
Le caractère `é` s'écrit sur *deux* octets en UTF-8 (`C3 A9`). Relu comme du Latin-1 (un octet = un caractère), `C3` devient `Ã` et `A9` devient `©`. En Python, `"café".encode("utf-8").decode("latin-1")` renvoie donc `'cafÃ©'` : c'est *exactement* le « cafÃ© » qu'on voit parfois sur une page Web mal configurée. *Convertir* un fichier d'un encodage à l'autre, c'est le relire correctement puis le réécrire : `octets.decode("utf-8").encode("latin-1")`.

**Remarque**
Un caractère peut prendre jusqu'à **4 octets** en UTF-8 : c'est le cas des émojis. Ainsi le visage souriant (point de code `U+1F600`) est un *seul* caractère — `ord` vaut `128512` — mais occupe **4 octets** en mémoire.

► **Exercices d'application :** exercices 21 à 25 et 29 (coder le texte : ASCII, Unicode ; convertir d'un encodage à l'autre)

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** Les conversions reposent sur `//`, `%` et la boucle `while` des *Bases de la programmation Python*, où l'on avait constaté sans l'expliquer que `0.1 + 0.2` ne vaut pas `0.3`.
- **Ce qu'il prépare.** Transistors et portes logiques formeront le cœur du processeur (*Architecture des ordinateurs et systèmes d'exploitation*) ; l'adresse d'une machine sur Internet (*Le Web : réseaux et interactions homme-machine*) sera faite de quatre octets ; UTF-8 reviendra avec les fichiers CSV (*Les données en tables*) et les pages HTML.
- **En Terminale.** La *Cryptographie* chiffrera un message en calculant sur les numéros de ses lettres (chiffre de César).

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Bit / octet | 1 bit = `0` ou `1` ; 1 octet = 8 bits = $2^8 = 256$ valeurs |
| Base $b$ | chaque rang pèse une puissance de $b$ ; $n$ bits codent $2^n$ valeurs |
| Bin. → déc. | additionner les poids $(128, 64, 32, \ldots)$ des rangs à `1` |
| Déc. → bin. | divisions successives par 2, restes lus **de bas en haut** |
| Logique booléenne | tables de **et**, **ou**, **non**, **xor** ; $n$ variables → $2^n$ lignes |
| Demi-additionneur | somme = $a$ xor $b$ ; retenue = $a$ et $b$ |
| Hexadécimal | 1 chiffre hexa = 4 bits ; on regroupe les bits par 4 |
| Relatif : coder | **complément à 2** : inverser les bits, puis +1 ; de $-2^{n-1}$ à $2^{n-1}-1$ sur $n$ bits |
| Relatif : décoder | bit de poids fort `1` ⇒ négatif ; inverser puis +1 donne la valeur absolue |
| Flottant | écriture souvent **approchée** ; jamais de `==` entre *float* |
| Texte (ASCII) | un **numéro** par caractère ; ASCII = 7 bits (128 car.) ; `ord`/`chr` |
| Unicode / UTF-8 | Unicode = catalogue universel (`U+`) ; UTF-8 = 1 à 4 octets, compatible ASCII |
| Charabia | *mojibake* : **mauvais encodage** à la lecture ⇒ « cafÃ© » ; `encode`/`decode` |
| Pourquoi le binaire | deux états = fiabilité ; tout (texte, image, son) se ramène à des bits |

### Retour à la situation de départ

1. Un octet compte 8 bits, et chaque bit ajouté double le nombre de valeurs : $2^8 = 256$ valeurs, les entiers de 0 à $2^8 - 1 = 255$ (§ II).
2. 0,1 n'a pas d'écriture binaire finie ; sur un nombre fini de bits, la machine n'en stocke qu'une valeur **approchée** (un **flottant**), et les petites erreurs s'ajoutent (§ VII).
3. Le texte a été écrit en **UTF-8**, où « é » occupe deux octets (`C3 A9`), puis relu en **Latin-1**, un octet par caractère : `C3` devient « Ã » et `A9` devient « © » (§ IX).

## Erreurs fréquentes

- **Se tromper dans le poids des bits.** Chaque bit vaut une **puissance de 2** ; le bit de **droite** vaut $2^0 = 1$. *Le réflexe :* écrire les poids … 8 4 2 1 au-dessus.
- **Confondre binaire et hexadécimal.** La base 16 utilise `0`–`9` puis `A`–`F` ; **un** chiffre hexa = **quatre** bits.
- **Tester l'égalité de deux flottants.** L'écriture est souvent **approchée** ($0{,}1 + 0{,}2 \neq 0{,}3$). *Le réflexe :* ne jamais faire `==` sur des *float*.
- **Oublier le nombre de bits.** Sur `n` bits, on code $2^n$ valeurs (de 0 à $2^n - 1$) : au-delà, **débordement**.
- **Confondre ou et ou exclusif.** 1 ou 1 vaut 1, mais 1 xor 1 vaut 0. *Le réflexe :* xor = « exactement un des deux », c'est-à-dire « différents ».
- **Divisions successives mal menées.** On lit les restes **de bas en haut** pour obtenir l'écriture binaire.
- **Confondre caractère et octet.** En UTF-8 un caractère accentué ou un émoji occupe **plusieurs octets** : `len(texte)` (caractères) ≠ taille en octets.
- **Croire qu'un fichier « connaît » son encodage.** Un fichier n'est qu'une suite d'octets ; lu avec le **mauvais encodage**, il donne du charabia (« cafÃ© »).

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

- ☐ convertir **binaire** → **décimal** (somme des poids) ;
- ☐ convertir **décimal** → **binaire** (divisions successives par 2) ;
- ☐ convertir avec l'**hexadécimal** (paquets de 4 bits) ;
- ☐ donner le nombre de valeurs codables sur `n` bits ($2^n$) ;
- ☐ évaluer le **nombre de bits** d'un entier, d'une somme, d'un produit ;
- ☐ **additionner** deux nombres en binaire ;
- ☐ dresser la **table de vérité** d'une expression booléenne (**et**, **ou**, **non**, **xor**) et expliquer le **demi-additionneur** ;
- ☐ coder un **entier relatif** (complément à deux) ;
- ☐ expliquer pourquoi un **flottant** est souvent **approché** ;
- ☐ expliquer comment un **texte** est codé (**ASCII**, `ord`/`chr`) ;
- ☐ distinguer **ASCII**, **Latin-1** et **Unicode/UTF-8**, et expliquer le **charabia** ;
- ☐ **convertir** un texte d'un encodage à l'autre (décoder, puis réencoder).

# EXERCICES



## Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- [ à la main ] à traiter **à la main** (sur le cahier) ; [ sur machine ] à faire ou **vérifier en console** Python.
- Rappels : `bin(n)`, `hex(n)`, `int("...", 2)`, `int("...", 16)`, littéraux `0b...` et `0x...`.

## Bases et conversions ↑cours

★ **Exercice 1** *Compter les possibilités* ▶ corrigé p. 553
[ à la main ]

1. Combien de valeurs différentes peut-on coder sur `4` bits ? sur `8` bits ? sur `n` bits ?
2. Quel est le plus grand entier positif codable sur un octet (8 bits) ? ▷ coup de pouce

★ **Exercice 2** *Du binaire vers le décimal* ▶ corrigé p. 553 • essentiel
[ à la main ] Convertir en base 10 : $1011_2$ ; $11001_2$ ; $10011101_2$. ▷ coup de pouce

★ **Exercice 3** *Du décimal vers le binaire* ▶ corrigé p. 553 • essentiel
[ à la main ] Convertir en base 2 (méthode des divisions par 2) : 13 ; 42 ; 200. ▷ coup de pouce

★ **Exercice 4** *Vérifier à la machine* ▶ corrigé p. 553
[ sur machine ] Vérifier les réponses des exercices 2 et 3 en console. ▷ coup de pouce

```
>>> bin(200)
>>> int("11001", 2)
```

## Programmer les conversions ↑cours

★★ **Exercice 5** *Décimal vers binaire* ▶ corrigé p. 554
[ sur machine ] Écrire une fonction `decimal_vers_binaire(n)` qui renvoie l'écriture binaire d'un entier `n` $\geqslant 0$ sous forme de **chaîne**, à l'aide d'une boucle `while`. Traiter le cas `n = 0`. ▷ coup de pouce
▷ début de solution

```
>>> decimal_vers_binaire(13)
'1101'
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 6** *Binaire vers décimal* ▶ corrigé p. 554
[ sur machine ] Écrire une fonction `binaire_vers_decimal(ch)` qui prend une chaîne de `0` et de `1` et renvoie l'entier correspondant, en parcourant la chaîne avec une boucle `for`. ▷ coup de pouce
▷ début de solution

```
>>> binaire_vers_decimal("10111001")
185
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Logique booléenne et portes logiques ↑cours

★ **Exercice 7** *Évaluer des expressions booléennes* ▶ corrigé p. 554 • essentiel
[ à la main ]

1. On prend $a = 1$ et $b = 0$. Donner la valeur de : non $a$ ; $a$ et $b$ ; $a$ ou $b$ ; $a$ xor $b$ ; non($a$ ou $b$).
2. Même question avec $a = 1$ et $b = 1$.
3. [ sur machine ] Vérifier en console, en écrivant `True` pour `1`, `False` pour `0` et `a != b` pour $a$ xor $b$ : ▷ coup de pouce

```
>>> not True
>>> True and False
>>> True or False
>>> True != False
>>> not (True or False)
```

★★ **Exercice 8** *Dresser la table d'une expression* ▶ corrigé p. 555 • essentiel
[ à la main ]

1. Dresser la table de vérité de non($a$ et $b$), puis celle de (non $a$) ou (non $b$). Que constate-t-on ? Comment qualifie-t-on ces deux expressions ?
2. Dresser la table de vérité de ($a$ et $b$) ou $c$. Combien de lignes faut-il ? Pour combien d'entre elles l'expression vaut-elle `1` ? ▷ coup de pouce

★★ **Exercice 9** *Le demi-additionneur* ▶ corrigé p. 555
[ à la main ]

1. Recopier et compléter la table de l'addition de deux bits $a$ et $b$, en écrivant le résultat sur deux bits : la retenue $r$ (à gauche) et la somme $s$ (à droite).

| $a$ | $b$ | retenue $r$ | somme $s$ |
|---|---|---|---|
| 0 | 0 | | |
| 0 | 1 | | |
| 1 | 0 | | |
| 1 | 1 | | |

2. Quelle porte logique donne la colonne $r$ ? Laquelle donne la colonne $s$ ? Dessiner le circuit du demi-additionneur.
3. Dans un circuit qui additionne deux nombres de 4 bits, seule la colonne de droite peut être confiée à un demi-additionneur. Pourquoi ? Le constater en posant `0110` + `0011` : quelle colonne doit additionner trois bits ? Que manque-t-il au demi-additionneur ? ▷ coup de pouce

## L'hexadécimal ↑cours

★★ **Exercice 10** *Binaire et hexadécimal* ▶ corrigé p. 556 • essentiel
[ à la main ] En regroupant les bits par paquets de 4 : convertir $\texttt{11011010}_2$ en hexadécimal, et $\texttt{3F}_{16}$ en binaire. ▷ coup de pouce

★★ **Exercice 11** *Décimal et hexadécimal* ▶ corrigé p. 556
[ à la main ] Convertir 255 et 3000 en hexadécimal ; convertir $\texttt{2A}_{16}$ et $\texttt{FF}_{16}$ en décimal. ▷ coup de pouce

★★ **Exercice 12** *Une couleur du Web* ▶ corrigé p. 556

[sur machine] Sur le Web, une couleur s'écrit `#RRGGBB` en hexadécimal (rouge, vert, bleu, chacun sur un octet). Retrouver les trois composantes (en base 10) de la couleur `#FF8000`.

```
>>> int("FF", 16)
>>> int("80", 16)
>>> int("00", 16)
```

À votre avis, quelle *couleur* obtient-on (rouge « à fond », un peu de vert, pas de bleu) ? ▷ coup de pouce

★★★ **Exercice 13** *Changer de base* ▶ corrigé p. 556 ≈ 20 min

[sur machine] Écrire une fonction `decimal_vers_base(n, b)` qui renvoie, sous forme de chaîne, l'écriture d'un entier `n` $\geqslant 0$ en base `b`, pour $2 \leqslant b \leqslant 16$. Les chiffres utilisés sont ceux de la chaîne `"0123456789ABCDEF"`. ▷ coup de pouce ▷ début de solution

```
>>> decimal_vers_base(255, 16)
'FF'
>>> decimal_vers_base(13, 2)
'1101'
>>> decimal_vers_base(100, 8)
'144'
```

Vérifier les résultats avec `hex`, `bin` et `int("144", 8)`.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Les entiers relatifs (complément à deux) ↑ cours

★★ **Exercice 14** *Coder et décoder* ▶ corrigé p. 557 • essentiel

[à la main]

1. Donner la représentation de $-20$ sur `8` bits (complément à deux).
2. On lit `11111011` (8 bits, complément à deux). Est-ce un nombre positif ou négatif ? Quelle est sa valeur ? ▷ coup de pouce

★★ **Exercice 15** *Le complément à deux selon un assistant d'IA* ▶ corrigé p. 557

Un élève demande à un assistant d'IA : « Comment écrit-on $-5$ sur 8 bits en complément à deux ? » Voici la réponse obtenue.

« On écrit d'abord 5 sur 8 bits : `00000101`. Le complément à deux consiste à inverser tous les bits, ce qui donne `11111010`. Le bit de poids fort vaut `1`, ce qui confirme que le nombre est négatif. Donc $-5$ s'écrit `11111010` sur 8 bits. Avec cette convention, on représente sur 8 bits les entiers de $-128$ à $+127$. »

1. [à la main] La réponse est-elle correcte ? Vérifier en posant l'addition $5 + (-5)$ sur 8 bits avec l'écriture proposée : que doit-on trouver ?
2. Localiser et corriger l'erreur.
3. [sur machine] Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ? ▷ coup de pouce

★★★ **Exercice 16** *Débordement* ▶ corrigé p. 557 ≈ 10 min

[à la main] [sur machine]

1. Quelles sont les valeurs minimale et maximale d'un entier relatif codé sur `8` bits ? sur `16` bits ?
2. En console, calculer `2 ** 64`. Pourquoi Python y arrive-t-il alors qu'un entier « sur 64 bits fixes » déborderait ? ▷ coup de pouce ▷ début de solution

★★★ **Exercice 17** *Le cas étrange de $-128$* ▶ corrigé p. 557 ≈ 15 min

[ à la main ]

1. Coder $-128$ sur `8` bits en complément à deux (on écrira d'abord 128 en binaire sur 8 bits).
2. Appliquer la méthode « inverser puis ajouter 1 » à l'écriture obtenue, comme pour calculer son opposé. Que constate-t-on ?
3. Expliquer ce résultat à l'aide des valeurs extrêmes codables sur 8 bits.
4. Sur 8 bits, combien d'entiers **strictement négatifs** peut-on coder ? Combien d'entiers **positifs ou nuls** ? Pourquoi les bornes ne sont-elles pas symétriques ? ▷ coup de pouce

▷ début de solution

## Les nombres à virgule ↑ cours

★★ **Exercice 18** *Fractions en binaire* ▶ corrigé p. 557

[ à la main ]

1. Convertir en binaire : 0,75 et 6,25.
2. Convertir en base 10 : $\texttt{101,101}_2$.
3. Appliquer la méthode des multiplications par 2 à $\frac{1}{3}$ (on part de $\frac{1}{3}$, puis $\frac{2}{3}\ldots$) : écrire les premiers chiffres après la virgule. Que remarque-t-on ? Que doit faire une machine qui veut stocker $\frac{1}{3}$ ? ▷ coup de pouce

★ **Exercice 19** *La trahison des flottants* ▶ corrigé p. 558 • essentiel

[ sur machine ] En console, taper `0.1 + 0.2`, puis `0.1 + 0.2 == 0.3`. Expliquer en une phrase ce que l'on observe. Proposer une manière **correcte** de tester si deux flottants sont « égaux ». ▷ coup de pouce

★★ **Exercice 20** *Additionner dix fois un dixième* ▶ corrigé p. 558

[ sur machine ] En console, calculer la somme de dix fois `0.1` :

```
total = 0
for i in range(10):
    total = total + 0.1
print(total)
```

Quel résultat obtient-on ? Est-il exactement égal à `1.0` ? Relier cette observation au cours. ▷ coup de pouce

## Coder le texte : ASCII et Unicode ↑ cours

★ **Exercice 21** *Lire la table ASCII* ▶ corrigé p. 558

[ à la main ] On donne les codes ASCII : `A`=65, `Z`=90, `a`=97, `0`=48.

1. Quel est le code de `C` ? de `c` ? *(On rappelle : minuscule = majuscule + 32.)*
2. Quel caractère se trouve 25 places avant `Z` dans la table ? Quel caractère a pour code 57 ?
3. Écrire le code du caractère `A` (65) en binaire sur 7 bits. ▷ coup de pouce

★ **Exercice 22** `ord` *et* `chr` ▶ corrigé p. 558

[ sur machine ] En console, prévoir *puis vérifier* : ▷ coup de pouce

```
ord("H")
chr(105)
chr(ord("B") + 32)
ord("i") - ord("H")
```

★★ **Exercice 23** *Caractères et octets* ▶ corrigé p. 558 • essentiel

[sur machine] Soit le mot `"élève"`.

1. Combien a-t-il de **caractères** ? Vérifier avec `len("élève")`.
2. Combien occupe-t-il d'**octets** en UTF-8 ? Vérifier avec `len("élève".encode("utf-8"))`.
3. Expliquer l'écart entre les deux nombres. ▷ coup de pouce

★★ **Exercice 24** *D'où vient le charabia* ▶ corrigé p. 558

[à la main] Une page Web affiche `cafÃ©` au lieu de `café`.

1. Sachant que `é` s'écrit sur deux octets en UTF-8, expliquer en une ou deux phrases comment un tel affichage se produit. ▷ coup de pouce
2. [sur machine] Vérifier avec `"café".encode("utf-8").decode("latin-1")`.

★★ **Exercice 25** *Programmer : coder un mot* ▶ corrigé p. 559

[sur machine] Écrire une fonction `afficher_codes(mot)` qui affiche, ligne par ligne, chaque caractère de `mot` suivi de son code ASCII.

*Exemple :* `afficher_codes("Hi")` affiche `H 72` puis, à la ligne suivante, `i 105`. ▷ coup de pouce

## Entraînement ↑ cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

★ **Exercice 26** *Décimal, binaire, hexadécimal* ▶ corrigé p. 559 • essentiel

[à la main] Recopier et compléter le tableau : sur chaque ligne, une seule écriture est donnée ; trouver les deux autres (le binaire sur 8 bits). Pour passer du décimal à l'hexadécimal, faire les divisions successives par 16. ▷ coup de pouce

| | décimal | binaire (8 bits) | hexadécimal |
|---|---|---|---|
| a) | 75 | | |
| b) | | 10110110 | |
| c) | | | C8 |
| d) | | | 3D |

[sur machine] Vérifier avec `bin`, `hex` et `int("...", 16)`.

★ **Exercice 27** *Complément à deux sur 8 bits* ▶ corrigé p. 559 • essentiel

[à la main]

1. Coder sur 8 bits, en complément à deux : a) $-1$ ; b) $-37$ ; c) $-100$ ; d) 77.
2. Décoder ces mots de 8 bits écrits en complément à deux : a) `11111110` ; b) `11100111` ; c) `01011010` ; d) `10000001`.
3. [sur machine] Vérifier les codages de la question 1 avec `bin(256 - 37)`, etc. ▷ coup de pouce

★ **Exercice 28** *Combien de bits ?* ▶ corrigé p. 559 • essentiel

[à la main]

1. Combien de bits faut-il pour écrire en binaire l'entier 500 ? l'entier 70 000 ? Justifier en encadrant chaque nombre entre deux puissances de 2.
2. Deux entiers positifs $a$ et $b$ s'écrivent chacun sur **12 bits** au plus, c'est-à-dire $a < 2^{12}$ et $b < 2^{12}$. Combien de bits faut-il, au plus, pour écrire leur **somme** $a+b$ ? leur **produit** $a \times b$ ? Justifier.
3. [sur machine] Vérifier avec $a = b = 4095$, le plus grand entier de 12 bits : calculer `a + b` et `a * b`, puis compter les bits avec `len(bin(...)) - 2` (le `- 2` retire le préfixe `0b`). ▷ coup de pouce

★★ **Exercice 29** *Convertir un texte d'un encodage à l'autre* ▶ corrigé p. 559

[sur machine] Un ancien logiciel a enregistré le mot `"café"` en **Latin-1** (ISO-8859-1). On simule ce fichier en console :

```
octets = "café".encode("latin-1")
```

1. Afficher `octets` et `len(octets)`. Combien d'octets occupe ce mot en Latin-1 ? Quel octet (en hexadécimal) code le `é` ?
2. On veut **convertir** ce texte en UTF-8. Exécuter `texte = octets.decode("latin-1")`, puis `nouveaux = texte.encode("utf-8")`. Combien d'octets maintenant ? Pourquoi un de plus ?
3. Que se passe-t-il si l'on écrit directement `octets.decode("utf-8")` ? En déduire la méthode pour convertir un fichier texte d'un encodage à un autre.
4. *(Si votre éditeur le permet.)* Enregistrer un fichier texte contenant `Noël à Monaco` avec l'encodage *ISO-8859-1* (menu d'enregistrement ou de l'encodage), puis le rouvrir en *UTF-8*. Qu'observe-t-on ? Comment le convertir proprement ? ▷ coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

**Ex. 1 q. 2 – piste** Énumérer à la main les valeurs possibles sur 1 bit, puis sur 2 bits, puis sur 3 bits : que devient leur nombre chaque fois qu'on ajoute un bit ? Pour la question 2, attention : la première valeur est 0. ↑énoncé

**Ex. 2 – piste** Écrire au-dessus de chaque bit son poids, en partant de la **droite** : 1, 2, 4, 8… Puis additionner les poids des bits qui valent `1`. ↑énoncé

**Ex. 3 – piste** Disposer les divisions comme dans l'exemple de 185 du cours, une ligne par division, jusqu'à un quotient nul. Dans quel sens lire ensuite la colonne des restes ? ↑énoncé

**Ex. 4 – piste** Le préfixe `0b` affiché par `bin` signale simplement une écriture binaire ; le second argument de `int` indique la base dans laquelle la chaîne est écrite. ↑énoncé

**Ex. 5 – piste** Refaire à la main les divisions par 2 de 13 : à chaque étape, quel reste note-t-on, et de quel côté de la chaîne déjà écrite faut-il le placer ? ↑énoncé

**Ex. 5 – début de solution**

```
resultat = ""
while n > 0:
    resultat = str(n % 2) + resultat
    ...
```

↑énoncé

**Ex. 6 – piste** Lire les bits de gauche à droite en gardant un accumulateur `n`. Quand on ajoute un bit à droite d'un nombre binaire, sa valeur est multipliée par combien ? ↑énoncé

**Ex. 6 – début de solution**

```
n = 0
for c in ch:
    n = n * 2 + int(c)   (puis renvoyer n)
```

↑énoncé

**Ex. 7 q. 3 – piste** Relire la table de vérité de chaque opérateur dans le cours. Pour non($a$ ou $b$), calculer d'abord la parenthèse. Le xor vaut `1` quand les deux bits sont différents. ↑

**Ex. 8 q. 2 – piste** Une ligne par combinaison des entrées : avec $n$ variables, $2^n$ lignes, énumérées en comptant en binaire (`000`, `001`, `010`…). Ajouter une colonne par étape du calcul, en commençant par les parenthèses. ↑énoncé

**Ex. 9 q. 3 – piste** Dans la colonne de droite, on n'ajoute que deux bits. Dans la colonne suivante, combien de bits faut-il ajouter ? D'où vient le bit supplémentaire ? ↑énoncé

**Ex. 10 – piste** Un paquet de 4 bits code une valeur de 0 à 15, donc exactement un chiffre hexadécimal. Par quel côté commencer les paquets ? Dans l'autre sens, chaque chiffre hexadécimal redonne 4 bits. ↑énoncé

Ex. 11 – piste Même méthode qu'en binaire, avec la base 16 : divisions successives par 16 dans un sens, somme des poids (1, 16, 256…) dans l'autre. Ne pas oublier que `A` vaut 10 et `F` vaut 15. ↑énoncé

Ex. 12 – piste Découper `FF8000` en trois paquets de deux chiffres hexadécimaux : un octet par composante. Que vaut chaque octet sur l'échelle de 0 à 255 ? ↑énoncé

Ex. 13 – piste Généraliser `decimal_vers_binaire` : que devient la division par 2 ? Le reste est un nombre entre 0 et $b-1$ : comment obtenir le *caractère* correspondant à partir de la chaîne `"0123456789ABCDEF"` ? ↑énoncé

Ex. 13 – début de solution
```
chiffres = "0123456789ABCDEF"
resultat = ""
while n > 0:
    resultat = chiffres[n % b] + resultat

    ...
```
↑énoncé

Ex. 14 q. 2 – piste Pour coder : écrire 20 sur 8 bits, inverser, ajouter 1. Pour décoder : le bit de poids fort donne le signe ; la même opération « inverser puis ajouter 1 » redonne la valeur absolue. ↑énoncé

Ex. 15 q. 3 – piste Un nombre plus son opposé doit donner 0 : poser l'addition sur 8 bits en ignorant la retenue qui sort. En Python, `bin(256 - 5)` donne l'écriture sur 8 bits de −5. ↑

Ex. 16 q. 2 – piste Sur 8 bits, le bit de poids fort indique le signe : quel est le plus grand nombre qui commence par `0` ? Et quel est le code du plus petit négatif ? Généraliser à 16 bits avec des puissances de 2. ↑énoncé

Ex. 16 q. 2 – début de solution Sur 8 bits, le plus grand positif est `01111111`, soit $2^7-1$. Écrire de même le plus petit négatif à l'aide d'une puissance de 2, puis remplacer 7 par 15. ↑énoncé

Ex. 17 q. 4 – piste Pour la question 2, l'opposé de −128 serait +128 : ce nombre fait-il partie des valeurs codables sur 8 bits en complément à deux ? ↑énoncé

Ex. 17 q. 4 – début de solution 128 = `10000000` ; inversion : `01111111` ; ajout de 1 : … Pour la question 4, ne pas oublier que 0 occupe l'un des codes qui commencent par `0`. ↑énoncé

Ex. 18 q. 3 – piste Traiter séparément la partie entière (méthode habituelle) et la partie après la virgule : la multiplier par 2, noter le chiffre avant la virgule, recommencer. Après la virgule, les poids sont $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$… Pour $\frac{1}{3}$, calculer avec des fractions plutôt qu'avec des décimaux. ↑énoncé

Ex. 19 – piste Les nombres `0.1` et `0.2` ont-ils une écriture binaire finie ? Plutôt que l'égalité exacte, regarder si l'écart entre les deux nombres est très petit (la fonction `abs` donne la valeur absolue). ↑énoncé

Ex. 20 – piste Le nombre `0.1` a-t-il une écriture binaire finie ? Que se passe-t-il pour une petite erreur répétée dix fois ? ↑énoncé

Ex. 21 q. 3 – piste Les lettres majuscules ont des codes consécutifs : `B` suit `A`, et ainsi de suite ; de même pour les chiffres à partir de `0`=48. Pour la question 3, décomposer 65 en puissances de 2. ↑énoncé

Ex. 22 – piste `ord` donne le code d'un caractère, `chr` le caractère d'un code. Retrouver les codes à partir de `A`=65 et `a`=97 (exercice précédent). ↑énoncé

Ex. 23 q. 3 – piste Quelles lettres du mot ne font pas partie de l'ASCII ? Combien d'octets UTF-8 utilise-t-il pour une lettre accentuée ? ↑énoncé

Ex. 24 q. 1 – piste Combien de caractères bizarres remplacent le `é` ? En Latin-1, combien d'octets occupe chaque caractère ? Le fichier a été écrit dans un encodage et lu dans un autre. ↑énoncé

Ex. 25 – piste Parcourir les caractères du mot avec une boucle `for` et, à chaque tour, afficher le caractère et le résultat de `ord`. ↑énoncé

**Ex. 26 – piste** Le binaire sert de relais : un chiffre hexadécimal correspond à un paquet de 4 bits. Du décimal vers l'hexadécimal : diviser par 16 et lire les restes de bas en haut (un reste de 11 s'écrit `B`). ↑énoncé

**Ex. 27 q. 3 – piste** Commencer par regarder le signe. Un positif s'écrit directement en binaire sur 8 bits ; pour un négatif, appliquer « inverser, puis ajouter 1 ». Au décodage, le bit de poids fort donne le signe. ↑énoncé

**Ex. 28 q. 3 – piste** Un entier $N$ s'écrit sur $k$ bits, où $k$ est le plus petit entier tel que $2^k > N$. Pour la somme : $a + b < 2^{12} + 2^{12}$, que l'on écrit comme une seule puissance de 2. Pour le produit : $2^{12} \times 2^{12} = 2^{?}$. ↑énoncé

**Ex. 29 q. 4 – piste** `decode` transforme des octets en caractères, `encode` des caractères en octets. Pour convertir, il faut d'abord lire les octets avec **leur** encodage d'origine, puis réécrire les caractères obtenus dans le nouvel encodage. Combien d'octets UTF-8 pour une lettre accentuée ? ↑énoncé

# TP ET PROJETS



## TP Programmer l'arithmétique binaire ▶ corrigé p. 560

*des convertisseurs à l'additionneur et au multiplicateur*

**Fichiers à télécharger :** `maththibaud.github.io/informatique-en-autonomie/f/p02-tp-arithmetique-binaire` — **archive .zip**

### Mode d'emploi

- Objectif : **reprogrammer ce que vous savez faire à la main** (conversions, complément à deux), puis construire l'addition et la multiplication **à partir de simples portes logiques**, comme dans un processeur.
- Tous les nombres binaires sont des **chaînes** de `"0"` et de `"1"`, écrites comme d'habitude (bit de poids fort à gauche) : `"1011"` représente 11.
- **Interdit** : `bin`, `int(..., 2)`, `format`… Ils ne servent qu'à *vérifier* vos résultats.
- Les questions [ à la main ] se traitent sur le cahier ; les questions [ sur machine ] dans le fichier Python.
- Partez du fichier `activite_arithmetique_binaire_depart.py` : chaque partie a sa fonction de tests (`tester_A()` pour la partie 1, `tester_B()` pour la partie 2, et ainsi de suite jusqu'à `tester_G()`) à appeler dès que la partie est terminée.
- Niveau : ★ échauffement ★★ classique ★★★ défi.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

**Remarque**

**Boîte à outils sur les chaînes.** Avec `s = "1011"` :

- `len(s)` vaut `4` ; `s[0]` vaut `"1"` (premier caractère) ; `s[len(s) - 1]` vaut `"1"` (dernier) ;
- `"0" + s` vaut `"01011"` (concaténation) ; `"0" * 3` vaut `"000"` (répétition) ;
- on construit une nouvelle chaîne **caractère par caractère** : partir de `r = ""`, puis ajouter à chaque tour de boucle un caractère à droite (`r = r + c`) ou à gauche (`r = c + r`) ;
- `for c in s:` parcourt les caractères de gauche à droite ; `for i in range(len(s) - 1, -1, -1):` parcourt les *indices* de droite à gauche (utile pour les retenues) ;
- `str(1)` vaut `"1"` et `int("1")` vaut `1`.

## 1 Les convertisseurs

★ **Exercice 1** *Du binaire vers le décimal*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

[sur machine] Écrire `binaire_vers_decimal(s)`. Méthode de Horner : on lit les bits de gauche à droite ; à chaque bit `c`, on fait `n = 2 * n + int(c)`.

[à la main] Avant de coder, dérouler cette méthode à la main sur `"1011"` (valeurs successives de `n`).

```
>>> binaire_vers_decimal("1011")
11
```

★ **Exercice 2** *Du décimal vers le binaire*

[sur machine] Écrire `decimal_vers_binaire(n)` par divisions successives par 2 : chaque reste `n % 2` est un bit, **ajouté à gauche** de la chaîne déjà construite. Attention au cas `n = 0`, qui doit renvoyer `"0"`.

```
>>> decimal_vers_binaire(13)
'1101'
```

★ **Exercice 3** *Compléter à gauche*

[sur machine] Écrire `completer(s, nb_bits)` qui ajoute des `"0"` à gauche de `s` jusqu'à atteindre `nb_bits` caractères (et ne change rien si `s` est déjà assez long).

```
>>> completer("101", 8)
'00000101'
```

★★ **Exercice 4** *Du binaire vers l'hexadécimal*

[sur machine] Écrire `binaire_vers_hexa(s)` en trois étapes :

(1) ajouter des `"0"` à gauche de `s` jusqu'à ce que sa longueur soit un multiple de 4 ;
(2) parcourir `s` caractère par caractère, en ajoutant chaque caractère à une chaîne `paquet` ;
(3) dès que `paquet` compte 4 caractères, le convertir en un chiffre de `"0123456789ABCDEF"` (avec `binaire_vers_decimal`), ajouter ce chiffre au résultat, puis vider `paquet`.

```
>>> binaire_vers_hexa("11111010")
'FA'
```

## 2 Le complément à deux

★ **Exercice 5** *Inverser les bits*

[sur machine] Écrire `inverser(s)` qui remplace chaque `"0"` par `"1"` et réciproquement.

★★ **Exercice 6** *Ajouter 1 : la retenue qui se propage*

[à la main] Poser à la main `0111 + 1` puis `1111 + 1` sur 4 bits. Recopier et compléter la règle : « en partant de la droite, tant que le bit vaut `1`, on le remplace par … et la retenue … ; au premier `0` rencontré, on le remplace par … et la retenue … ».

[sur machine] Écrire `incrementer(s)` qui ajoute 1 à `s` **sur le même nombre de bits** (la retenue qui sort à gauche est perdue). On parcourt les indices de droite à gauche avec un booléen `retenue`, qui vaut `True` au départ.

```
>>> incrementer("0111")
'1000'
>>> incrementer("1111")
'0000'
```

*Gardez cette fonction précieusement : c'est exactement ce que fait un registre de la machine à billes Turing Tumble (atelier associé).*

★★ **Exercice 7** *Coder un entier relatif*

[sur machine] Écrire `complement_a_deux(n, nb_bits)` :

- si `n` $\geqslant 0$ : écrire `n` en binaire et compléter à `nb_bits` ;
- sinon : écrire $|$`n`$|$ sur `nb_bits`, **inverser**, puis **ajouter 1** (réutiliser vos fonctions !).

```
>>> complement_a_deux(-5, 8)
'11111011'
```

★★ **Exercice 8** *Décoder un entier relatif*

[sur machine] Écrire `decoder_c2(s)`. Si le bit de poids fort est `0`, c'est un entier positif ordinaire. Sinon, la valeur est celle de `s` lu comme un entier positif, **moins** $2^n$, où $n$ est le nombre de bits.

[à la main] Vérifier la règle à la main sur `"11111011"` : $251 - 256 = \ldots$

```
>>> decoder_c2("11111011")
-5
```

## 3 Les portes logiques

Les opérateurs **et**, **ou**, **non**, **xor**, leurs tables de vérité et les symboles des portes ont été vus en cours (section « Logique booléenne : des portes logiques à l'addition ») : il s'agit maintenant de les **programmer**. Dans cette partie, un bit est une chaîne d'un caractère : `"0"` ou `"1"`.

★ **Exercice 9** *Les trois portes de base*

[sur machine] Écrire `non(a)`, `et(a, b)` et `ou(a, b)` avec des `if`, en suivant leurs tables de vérité (cours). Chacune renvoie `"0"` ou `"1"`.

★★ **Exercice 10** *Construire le OU exclusif*

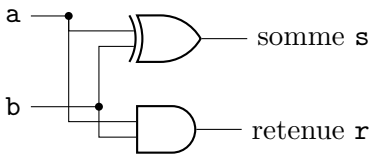
Rappel du cours : le OU exclusif (*xor*) vaut `"1"` lorsque **exactement un** des deux bits vaut `"1"`.

1. [à la main] Dresser la table de vérité de l'expression ($a$ et non $b$) ou (non $a$ et $b$), avec la méthode du cours (une colonne par étape). Vérifier qu'elle coïncide avec celle du xor.
2. [sur machine] Écrire `xor(a, b)` **sans aucun** `if`, en combinant seulement `non`, `et` et `ou`. Indice : « `a` et pas `b`, ou bien pas `a` et `b` ».
3. [sur machine] Afficher sa table de vérité avec deux boucles imbriquées `for a in "01":` et `for b in "01":`.

## 4 Les additionneurs

★★ **Exercice 11** *Le demi-additionneur*

Le cours l'a établi : en additionnant deux bits, la **somme** est donnée par une porte **xor** et la **retenue** par une porte **et**. On programme maintenant ce circuit, en réutilisant les portes de la partie précédente.

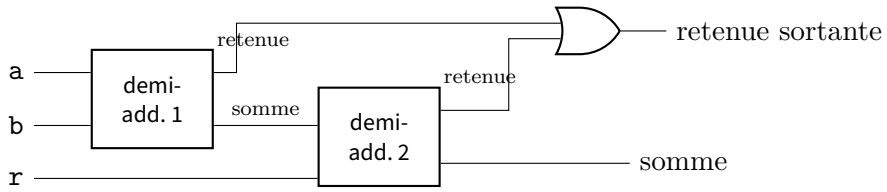



[sur machine] Écrire `demi_additionneur(a, b)` qui renvoie la chaîne `retenue + somme`. C'est donc exactement $a + b$ écrit en binaire sur deux bits.

```
>>> demi_additionneur("1", "1")
'10'
```

★★ **Exercice 12** *L'additionneur complet*

Pour additionner des nombres de plusieurs bits, chaque colonne reçoit **trois** bits : `a`, `b` et la retenue `r` venue de la colonne de droite. L'additionneur complet se construit avec **deux demi-additionneurs** et une porte OU :




[sur machine] Écrire `additionneur_complet(a, b, r)` qui renvoie `retenue + somme` de $a+b+r$, en appelant deux fois `demi_additionneur`. Si `d1 = demi_additionneur(a, b)`, alors `d1[0]` est la retenue et `d1[1]` la somme.

```
>>> additionneur_complet("1", "0", "1")
'10'
```

★★ **Exercice 13** *L'additionneur $n$ bits*

En chaînant $n$ additionneurs complets (la retenue sortante de l'un entre dans le suivant, à gauche), on additionne deux nombres de $n$ bits. C'est l'**additionneur à propagation de retenue** des processeurs.

[sur machine] Écrire `additionner(s1, s2)` (deux chaînes de même longueur $n$). Le résultat fait $n+1$ bits : la dernière retenue est placée à gauche.

```
>>> additionner("0101", "0110")      # 5 + 6
'01011'
```

[à la main] Combien d'appels à `additionneur_complet` faut-il pour additionner deux nombres de 64 bits ?

## 5 La soustraction

★★ **Exercice 14** *Soustraire, c'est additionner l'opposé*

En complément à deux, $a - b = a + (-b)$, et $-b$ s'obtient en inversant les bits de $b$ puis en ajoutant 1. Le processeur n'a donc **pas besoin d'un circuit de soustraction** !

[sur machine] La somme renvoyée par `additionner` a $n+1$ bits ; pour garder $n$ bits, il faut ignorer la retenue finale, c'est-à-dire le premier caractère. Écrire d'abord `sans_premier(s)`, qui renvoie la chaîne `s` privée de son premier caractère : on recopie un à un les caractères d'indices `1` à `len(s) - 1`.

```
>>> sans_premier("10110")
'0110'
```

[sur machine] Écrire ensuite `soustraire(s1, s2)` en une ligne, avec `additionner`, `inverser`, `incrementer` et `sans_premier`.

```
>>> soustraire("0010", "0111")      # 2 - 7
'1011'
>>> decoder_c2("1011")
-5
```

★★★ **Exercice 15** *Détecter le dépassement*

Sur 4 bits, le complément à deux représente les entiers de $-8$ à 7.

1. [ à la main ] Calculer `0111 + 0001` sur 4 bits et décoder le résultat. Que s'est-il passé ?
2. [ à la main ] Montrer que la somme de deux nombres de signes *contraires* ne peut jamais dépasser.
3. [ sur machine ] En déduire `depassement(s1, s2)` qui renvoie `True` si `s1` et `s2` ont le même bit de signe et que leur somme (sur $n$ bits) a un bit de signe différent.

## 6 La multiplication

★ **Exercice 16** *Multiplier par 2, 4, 8…*

[ à la main ] Que devient l'écriture binaire d'un nombre quand on le multiplie par 2 ? par $2^k$ ? (Comparer avec la multiplication par 10 en décimal.)

[ sur machine ] Écrire `decaler(s, k)` qui renvoie `s` multiplié par $2^k$.

★★★ **Exercice 17** *Le multiplicateur*

[ à la main ] Poser la multiplication `1101 × 1011` comme à l'école primaire. Constater que chaque ligne intermédiaire est soit `0`, soit `1101` décalé : en binaire, **on n'a jamais besoin des tables de multiplication !**

[ sur machine ] Écrire `multiplier(s1, s2)` (entiers positifs) :

- le résultat tient sur $n =$ `len(s1) + len(s2)` bits : partir de `"0" * n` ;
- parcourir les bits de `s2` de droite à gauche (rang `k` $= 0, 1, 2\ldots$) ;
- si le bit vaut `"1"`, compléter `decaler(s1, k)` à $n$ bits, l'additionner au résultat avec `additionner`, puis retirer le bit de gauche de la somme avec `sans_premier` pour revenir à $n$ bits.

```
>>> multiplier("1101", "1011")      # 13 * 11
'10001111'
```

## 7 Comme la machine à billes

★★ **Exercice 18** *Additionner avec des billes*

Dans la machine Turing Tumble, un registre de bits reçoit des billes : **chaque bille ajoute 1**.

1. [ sur machine ] Écrire `ajouter_billes(registre, nb_billes)` qui appelle `incrementer` autant de fois qu'il y a de billes. Vérifier que `ajouter_billes("0101", 6)` donne `"1011"`.
2. Pour calculer $A + B$ sur 32 bits, combien d'appels à `incrementer` faut-il dans le pire cas ? Et combien d'appels à `additionneur_complet` avec `additionner` ? Conclure : pourquoi les processeurs n'additionnent-ils pas « bille par bille » ?

★★ **Exercice 19** *Le convertisseur d'un assistant d'IA*

Un élève demande à un assistant d'IA une fonction de conversion. Voici la réponse :
« Voici la fonction, testée sur plusieurs valeurs :

```
1 def decimal_vers_binaire(n):
2     if n == 0:
3         return "0"
4     s = ""
5     while n > 0:
6         s = s + str(n % 2)
7         n = n // 2
8     return s
9
```

```
10 assert decimal_vers_binaire(0) == "0"
11 assert decimal_vers_binaire(5) == "101"
12 assert decimal_vers_binaire(7) == "111"
13 assert decimal_vers_binaire(9) == "1001"
```

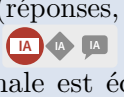
Les quatre tests passent, la fonction est donc correcte. »

1. [ sur machine ] Les tests passent-ils vraiment ? Essayer `decimal_vers_binaire(6)`. La fonction est-elle correcte ?
2. Localiser et corriger l'erreur.
3. Qu'ont en commun `"101"`, `"111"` et `"1001"` ? Pourquoi ces tests étaient-ils incapables de détecter l'erreur ? Proposer un test qui l'aurait détectée. Que penser de la phrase « les tests passent, la fonction est donc correcte » ?

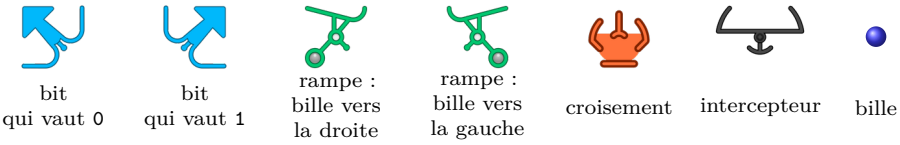
## ATELIER Turing Tumble : calculer avec des billes

▶ corrigé p. 564

### Mode d'emploi

- **Turing Tumble** est un ordinateur *mécanique* : des billes tombent sur des pièces qui les aiguillent. Pas d'électricité, mais les mêmes idées que dans un processeur.
- Un groupe manipule le **plateau** ; les autres utilisent le **simulateur en ligne**, puis on tourne. Simulateur : `jessecrossen.github.io/ttsim`.
- Les numéros de puzzles renvoient au livret du jeu. Les dessins reproduisent le plateau : **picots** (petits points) et **emplacements** en quinconce (où l'on clipse les pièces). Les pièces sont dessinées d'après le simulateur.
- Les questions [ à la main ] se traitent sur le cahier (réponses, tables, schémas de montages).
- Usage de l'IA, indiqué sur chaque exercice :  aucune ;  en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ;  l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Le matériel en deux minutes



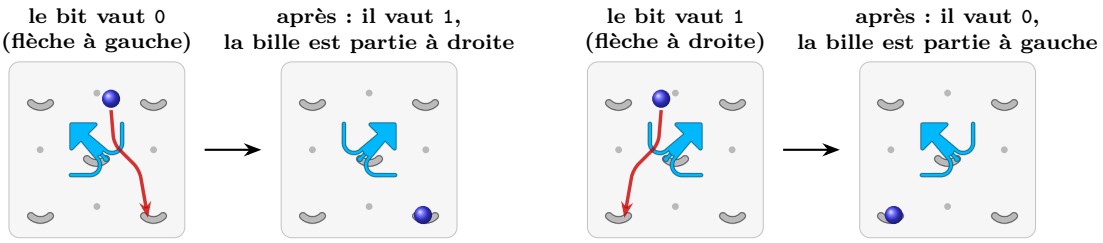

La rampe est une barre inclinée terminée par un crochet : la bille roule jusqu'au crochet, la rampe bascule et lâche la bille de ce côté, puis le **contrepoids** (le rond) la remet en place. Le bit, lui, n'a pas de contrepoids : il **reste** dans sa nouvelle position.

| **Pièce** | **Rôle** | **Équivalent électronique** |
|---|---|---|
| Rampe (verte) | envoie la bille toujours du même côté | un fil |
| Croisement (orange) | deux chemins qui se croisent sans se mélanger | deux fils qui se croisent |
| **Bit** (bleu) | pointe à gauche ou à droite ; **change de sens** à chaque bille | un transistor / une bascule mémoire |
| Intercepteur (noir) | arrête la bille : la machine s'arrête | fin du programme |
| Bit à engrenage + engrenages | comme un bit, mais fait basculer les bits reliés | des bits « câblés » ensemble |
| Leviers du bas | une bille qui arrive à gauche lâche une bille bleue, à droite une bille rouge | l'horloge : une instruction après l'autre |

## 2 Le bit : une mémoire d'un chiffre binaire

**Définition 7 — Convention du jeu**

Un bit qui pointe **à droite** vaut `1` ; un bit qui pointe **à gauche** vaut `0`. Quand une bille le traverse, elle tombe du côté **opposé** à la flèche, puis le bit **bascule**.




★ **Exercice 1** *Table du bit*

[à la main] Recopier et compléter sur le cahier le tableau suivant.

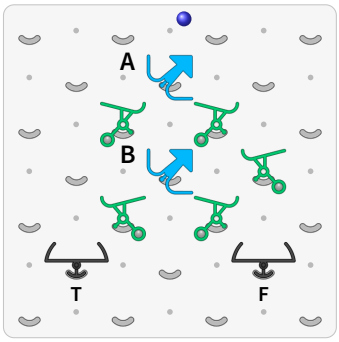
| Valeur avant | Côté où tombe la bille | Valeur après |
|---|---|---|
| 0 | ... | ... |
| 1 | ... | ... |

Quelle opération le bit réalise-t-il sur sa propre valeur ? Quelle porte logique du TP d'arithmétique binaire lui correspond ?

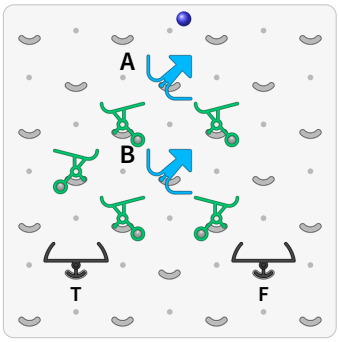
## 3 Des portes logiques avec des billes

Un bit peut servir d'**entrée** : on le règle à la main avant de lancer la bille (`1` = vrai, `0` = faux). La bille teste les bits l'un après l'autre et finit dans l'intercepteur **T** (vrai, *true*) ou **F** (faux, *false*). Voici deux montages, testés dans le simulateur, qui ne diffèrent que par l'orientation de quelques rampes.

**Montage 1**




**Montage 2**




### ★★ Exercice 2 — *Deux portes à reconnaître*

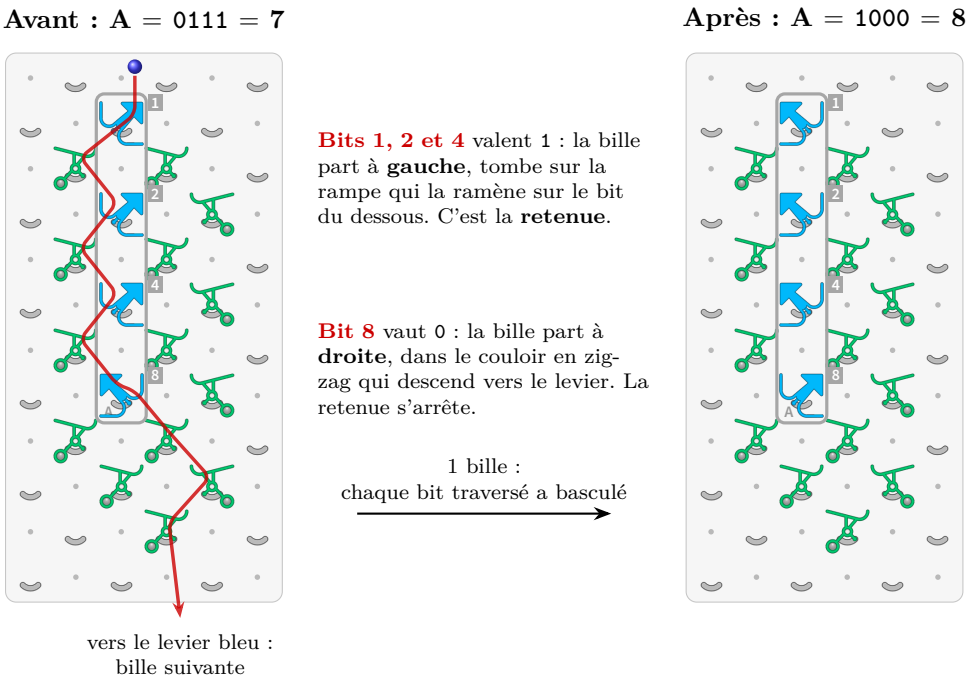
1. [ à la main ] Pour chaque montage, suivre la bille dans les quatre cas ($A, B \in \{0, 1\}$) et dresser la table de vérité (colonnes A, B, intercepteur atteint). Quelle porte logique chaque montage réalise-t-il ?
2. Dans le montage 1, que se passe-t-il quand A vaut `0` : le bit B est-il seulement « lu » ? Faire le lien avec l'évaluation paresseuse de `and` en Python (`a and b` n'évalue pas `b` quand `a` est faux).
3. Après le passage de la bille, les bits traversés ont basculé. Que faut-il faire avant de tester le cas suivant ?
4. [ plateau ou simulateur ] Le puzzle **18** (« Entanglement ») demande justement une porte ET : le monter et vérifier.

### ★★ Exercice 3 — *NON, puis OU exclusif*

1. [ à la main ] Avec **un seul** bit A, deux rampes et les deux intercepteurs, dessiner sur le cahier un montage qui réalise la porte NON (T si A vaut `0`, F sinon).
2. *Défi* (★★★) : le OU exclusif doit envoyer vers T quand *exactement un* des deux bits vaut `1`. Pourquoi ne suffit-il pas de modifier les rampes du montage 1 ? Proposer une solution avec **deux** bits B et B' réglés à la même valeur. Quelle pièce du jeu permettrait de les relier pour qu'on n'ait qu'une seule entrée B à régler ?

## 4 Le registre : un compteur binaire

Voici le montage du puzzle **21** (« Quantum Number ») : quatre bits **empilés en colonne** forment le registre A, entourés de rampes. Le bit du **haut** vaut 1, le suivant 2, puis 4, puis 8. La valeur du registre est la somme des poids des bits qui pointent à droite.

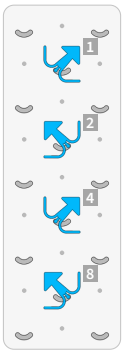


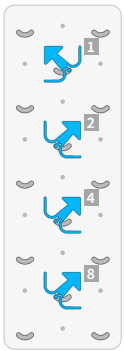

**Propriété 4 — Un registre compte en binaire**

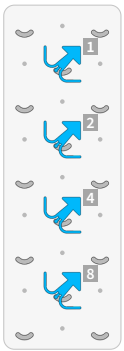
Sur un bit qui vaut `1`, la bille le remet à `0` et **continue** vers le bit suivant : c'est la **retenue**. Sur un bit qui vaut `0`, la bille le met à `1` et **sort** : la retenue s'arrête. Chaque bille ajoute donc **1** au registre : c'est exactement la fonction `incrementer` du TP d'arithmétique binaire.

★ **Exercice 4** IA IA IA *Lire un registre*

[ à la main ] Donner la valeur de chaque registre (le bit du haut vaut 1).

**a.** 

**b.** 

**c.** 

★ **Exercice 5** IA IA IA *Pair ou impair ? La moitié ?*

[ à la main ] Sans calculer la valeur du registre :

1. Comment savoir, d'un seul coup d'œil, si le nombre stocké est pair ?
2. On cache le bit du haut et on lit les trois autres bits comme un nombre de 3 bits (le bit 2 devient le bit des unités, etc.). Que trouve-t-on pour les registres a., b. et c. ci-dessus ? Quelle opération vient-on de faire ?

★★ **Exercice 6** *Faire tourner la machine dans sa tête*

1. [ à la main ] Le registre vaut `0101` (5). On lâche 6 billes. Écrire la valeur du registre après chaque bille et, pour chaque bille, le nombre de bits qu'elle traverse.
2. [ plateau ou simulateur ] Monter le puzzle 21 et vérifier. Astuce du livret : bloquer le levier gauche avec le doigt pour arrêter la machine, puis comparer le registre et le nombre de billes tombées.
3. On part de `0000` et on lâche 16 billes. Quelles billes traversent les **quatre** bits ? Que vaut le registre juste après chacune d'elles ?

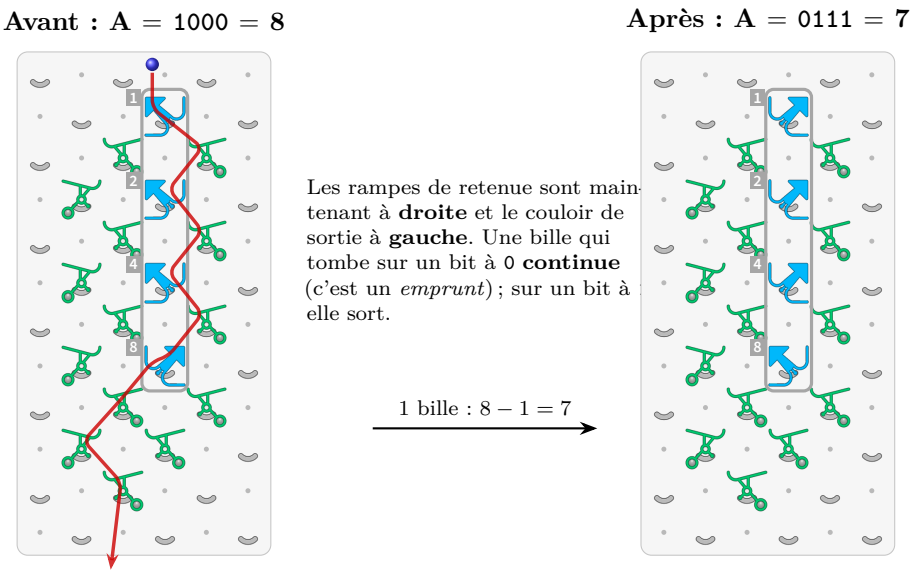
## 5 Additionner, soustraire

★★ **Exercice 7** *L'addition « bille par bille »*

1. [ plateau ou simulateur ] Régler le registre sur $A = 5$, puis lâcher $B = 6$ billes. Lire le résultat.
2. Avec un registre assez grand (8 bits), combien de billes faut-il pour calculer $5 + 200$ ? Et $200 + 5$ ? Quelle version est la plus rapide ?
3. **Pour aller plus loin** (puzzle **37**) : la machine additionne deux *registres* (un de 3 bits valant 5 et un de 4 bits valant 6) et range la somme 11 dans le second. Observer comment les bits à engrenage « lisent » le premier registre.

★★ **Exercice 8** *Le décompteur*

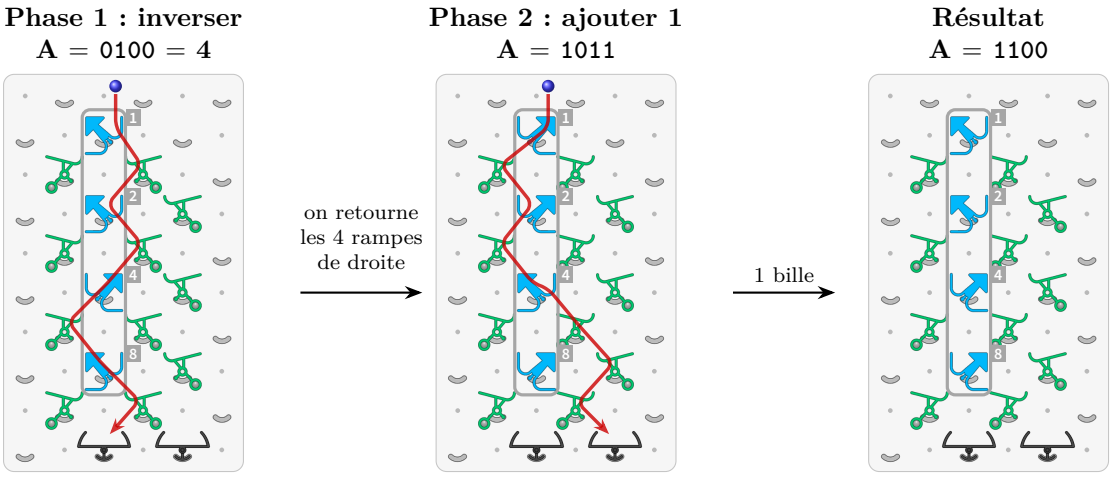
En retournant les rampes de l'autre côté du registre, il **décompte** : chaque bille retire 1 (puzzle **22**, « Depletion »).




1. [ à la main ] Comparer avec le compteur : pourquoi suffit-il de changer les rampes de côté pour passer de « +1 » à « −1 » ? (Penser à la règle de l'emprunt quand on pose `1000` − 1.)
2. [ plateau ou simulateur ] Puzzles **23** et **24** : laisser passer exactement 4, puis 9 billes, et intercepter la suivante. Quelle valeur de départ faut-il donner au registre ?
3. [ à la main ] Le registre vaut `0000` et on retire encore 1. Que vaut-il ? Lu en **complément à deux** sur 4 bits, que représente ce résultat ? Conclure : le complément à deux n'est pas une convention arbitraire, c'est ce que fait naturellement un compteur qui passe sous zéro.

## 6 L'opposé d'un nombre : inverser, puis ajouter 1

En complément à deux, $-A$ s'obtient en **inversant** tous les bits de $A$, puis en **ajoutant 1**. La machine sait faire les deux, avec le *même* registre : il suffit de changer l'orientation des quatre rampes situées juste à droite des bits. Ces montages ont été testés dans le simulateur.




### ★★ Exercice 9 *Calculer $-4$ avec deux billes*

1. [ à la main ] Phase 1 : dans le montage « inverser », les deux côtés de chaque bit mènent au bit du dessous. Combien de fois chaque bit est-il traversé ? Pourquoi obtient-on bien l'inverse de A ?
2. Phase 2 : quel montage déjà vu reconnaît-on ? Pourquoi a-t-on placé des intercepteurs en bas ?
3. [ à la main ] Décoder le résultat `1100` en complément à deux sur 4 bits. A-t-on bien obtenu $-4$ ?
4. Entre les deux phases, les *données* (les bits) n'ont pas bougé : seules les rampes ont changé. À quoi correspondent les rampes dans un ordinateur : aux données ou au programme ?

### ★★★ Exercice 10 *Des cas surprenants*

1. [ plateau ou simulateur ] Calculer $-5$ (A = `0101`). Vérifier avec `complement_a_deux(-5, 4)` du TP d'arithmétique binaire.
2. [ à la main ] Que donne la machine pour A = `0000` ? Dans quel intercepteur finit la bille de la phase 2 ? Pourquoi le résultat reste-t-il juste ?
3. [ à la main ] Et pour A = `1000`, c'est-à-dire $-8$ ? Expliquer le résultat étonnant en revenant à la plage des entiers représentables sur 4 bits.

> **Remarque**
> Le puzzle **27** (« Reflection ») demande d'inverser neuf bits quelle que soit leur position de départ, et les puzzles **28** à **30** introduisent la **bascule** (*latch*) : un bit à engrenage qui, une fois basculé, le reste, quelles que soient les billes suivantes. C'est une **mémoire** qui retient un événement, comme l'indicateur de dépassement du puzzle 30. Dans un processeur, les mémoires sont fabriquées de la même façon, à partir de portes logiques rebouclées sur elles-mêmes.

## 7 Le dépassement

★★ **Exercice 11** *Quand le registre déborde*

1. [ à la main ] Un registre de 4 bits vaut `1111`. On lâche une bille. Que vaut-il ensuite ? Où est passée la retenue ?
2. [ plateau ou simulateur ] Puzzle **30** (« Overflow ») : un registre de 3 bits compte les billes ; un bit à engrenage `OV` doit basculer, et *rester* basculé, dès qu'on dépasse 7. À quoi sert ce bit ? Faire le lien avec la fonction `depassement` du TP d'arithmétique binaire.

## 8 Multiplier

★★ **Exercice 12** *Compter de 2 en 2*

1. [ à la main ] Si les billes entraient dans le registre directement par le bit de poids 2 (et non par le bit du haut), de combien chaque bille augmenterait-elle le registre ? Et par le bit de poids 4 ?
2. [ plateau ou simulateur ] *Défi* : modifier le compteur du puzzle 21 pour qu'il compte de 2 en 2. Que vaut le registre après $B$ billes ? Faire le lien avec `decaler` : multiplier par 2, c'est décaler d'un rang.

★★★ **Exercice 13** *Le multiplicateur mécanique*

1. [ plateau ou simulateur ] Puzzle **48** : multiplier un registre par 3. Proposer une idée à partir de ce qui précède (un registre qui décompte $A$ fois, et pour chaque décompte …).
2. [ plateau ou simulateur ] Puzzle **56** : multiplier deux nombres de 2 bits. Le livret range le produit dans un registre de 3 bits, donc *modulo 8* : pourquoi ce modulo ? Combien de bits faudrait-il pour ne jamais perdre d'information ? (Voir l'exercice *Le multiplicateur* du TP d'arithmétique binaire.)
3. [ au-delà du programme ] Une machine est dite « Turing-complète » si, avec autant de mémoire qu'il le faut, elle peut effectuer n'importe quel calcul qu'effectue un ordinateur (la notion sera précisée en Terminale, au chapitre *Calculabilité*). Le plateau ne mesure que quelques dizaines de cases et la boîte contient un nombre limité de pièces. Pourquoi, malgré tout, peut-on dire que Turing Tumble est « Turing-complet » ?

## 9 Bilan

| **Turing Tumble** | **Processeur** | **TP arithmétique (Python)** |
|---|---|---|
| un bit bleu | un bit de mémoire (bascule) | un caractère `"0"`/`"1"` |
| un registre de bits | un registre du processeur | une chaîne `"0101"` |
| une bille qui traverse le registre | un incrément (`+1`) | `incrementer(s)` |
| la bille qui continue | la retenue | le booléen `retenue` |
| rampes retournées (décompteur) | un décrément (`-1`) | `soustraire(s, "0001")` |
| `1111` + 1 bille → `0000` | le dépassement (*overflow*) | `depassement(s1, s2)` |
| entrer par le bit de poids 2 | le décalage ($\times$ 2) | `decaler(s, 1)` |
| bits A, B testés, intercepteurs T/F | une porte logique | `et(a, b)`, `ou(a, b)` |
| montage « inverser » | NON sur chaque bit | `inverser(s)` |
| inverser, puis 1 bille | l'opposé $-A$ | `complement_a_deux(-n, 4)` |

# FICHE DE RÉVISION



## L'essentiel

- **Binaire** (base 2) : deux chiffres `0` et `1`. Chaque bit vaut une **puissance de 2**.
- **Hexadécimal** (base 16) : `0`–`9` puis `A`=10… `F`=15. **Un** chiffre hexa = **quatre** bits.
- Sur `n` bits : $2^n$ valeurs, de 0 à $2^n - 1$.
- **Flottant** : écriture souvent **approchée** ⇒ jamais de `==` entre deux flottants.

## Binaire → décimal (somme des poids)

| poids | 8 | 4 | 2 | 1 |
|---|---|---|---|---|
| bits | 1 | 0 | 1 | 1 |

$1011_2 = 8 + 0 + 2 + 1 = 11$. *En code :*

```
def binaire_vers_decimal(ch):
    n = 0
    for c in ch:                # de gauche a droite
        n = n * 2 + int(c)
    return n                    # binaire_vers_decimal("10110") -> 22
```

## Décimal → binaire (divisions successives)

On divise par 2, on note les restes, on les lit **de bas en haut** :

$$11 = 2 \times 5 + \mathbf{1}\,; \quad 5 = 2 \times 2 + \mathbf{1}\,; \quad 2 = 2 \times 1 + \mathbf{0}\,; \quad 1 = 2 \times 0 + \mathbf{1} \;\Rightarrow\; 1011_2.$$

## Logique booléenne et demi-additionneur

| $a$ | $b$ | non $a$ | $a$ et $b$ | $a$ ou $b$ | $a$ xor $b$ |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 | 0 |

- **xor** = « exactement un des deux » = « différents » (en Python, sur des booléens : `a != b`).
- **Table d'une expression** : $n$ variables ⇒ $2^n$ lignes (on compte en binaire), une colonne par étape du calcul.
- **Demi-additionneur** (addition de deux bits) : somme $s = a$ xor $b$, retenue $r = a$ et $b$.

## À connaître aussi

**Nombre de bits** : $N$ s'écrit sur $k$ bits si $2^{k-1} \leqslant N < 2^k$ ; une somme de deux nombres de $k$ bits tient sur $k+1$ bits, un produit sur $2k$. **Addition binaire** ($1 + 1 = 10$, on retient), **hexadécimal** par paquets de 4 bits, **entiers relatifs** en **complément à deux**, et le fait que **tout** (texte, image, son) se ramène à des bits.

### Coder le texte

- Chaque caractère a un **numéro** ; en Python `ord("A")`=65 et `chr(97)`=`"a"`.
- **ASCII** : 7 bits, 128 caractères (anglais, pas d'accents). **ISO-8859-1** (Latin-1) : 8 bits, 256 caractères.
- **Unicode** = catalogue universel (points de code `U+`) ; **UTF-8** = 1 à 4 octets, **compatible ASCII**.
- **Caractère** ≠ **octet** : `"é"` = 1 caractère mais **2 octets** en UTF-8.
- **Charabia** (*mojibake*) : lire un fichier avec le **mauvais encodage** ⇒ « cafÃ© ».
- **Convertir** un texte : décoder les octets avec **leur** encodage d'origine, puis réencoder les caractères dans le nouvel encodage.

### Erreurs à éviter

Se tromper de poids (le bit de droite vaut 1) ; confondre base 2 et base 16 ; confondre ou et ou exclusif (1 ou 1 vaut 1, 1 xor 1 vaut 0) ; tester `==` sur des flottants ; oublier la limite : sur `n` bits, débordement au-delà de $2^n - 1$.

### Savoir-faire à maîtriser — je sais…

- ☐ convertir **binaire** → **décimal** (somme des poids) ;
- ☐ convertir **décimal** → **binaire** (divisions successives par 2) ;
- ☐ convertir avec l'**hexadécimal** (paquets de 4 bits) ;
- ☐ donner le nombre de valeurs codables sur `n` bits ($2^n$) ;
- ☐ évaluer le **nombre de bits** d'un entier, d'une somme, d'un produit ;
- ☐ **additionner** deux nombres en binaire ;
- ☐ dresser la **table de vérité** d'une expression booléenne (**et**, **ou**, **non**, **xor**) et expliquer le **demi-additionneur** ;
- ☐ coder un **entier relatif** (complément à deux) ;
- ☐ expliquer pourquoi un **flottant** est souvent **approché** ;
- ☐ expliquer comment un **texte** est codé (**ASCII**, `ord`/`chr`) ;
- ☐ distinguer **ASCII**, **Latin-1** et **Unicode/UTF-8**, et expliquer le **charabia** ;
- ☐ **convertir** un texte d'un encodage à l'autre (décoder, puis réencoder).

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 567

1. Convertir $110101_2$ en décimal, puis $110101_2$ en hexadécimal.
2. Combien de valeurs sur 10 bits ? Quel est le plus grand entier positif codable ?
3. Dresser la table de vérité de $a$ et (non $b$). Pour quelle(s) ligne(s) vaut-elle `1` ?
4. Coder $-6$ sur 8 bits en complément à deux.
5. Quel entier code `11110100` sur 8 bits en complément à deux ?
6. Pourquoi 0,5 est-il stocké exactement par la machine, mais pas 0,1 ?
7. Combien de bits pour écrire 1000 ? Et, au plus, pour le produit de deux nombres de 10 bits ?
8. Le mot `"où"` : combien de caractères, combien d'octets en UTF-8 ?

**Et avant ?** — *questions sur les chapitres précédents.* ► réponses p. 567

1. Que valent `45 // 2` et `45 % 2` ? (ch. 1)
2. Combien de tours fait une boucle `while n > 0` dont le corps est `n = n // 10`, si `n` vaut `2026` au départ ? (ch. 1)

### S'entraîner sur plusieurs chapitres


- Problème de synthèse n° 1 — Le binaire avec des entiers, p. 510
- Problème de synthèse n° 2 — Le bit de parité, p. 511
- Problème de synthèse n° 3 — Des entiers relatifs sur 8 bits, p. 511

# DÉFI ADVENT OF CODE



## DÉFI Dive !

*piloter le sous-marin — lire des commandes, variables d'état*

Jour 2

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p02-defi-aoc-2021-02` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2021/day/2`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (facultatif), le lien *get your puzzle input* donne **vos** données, à enregistrer sous `input.txt` à côté du programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Un sous-marin suit un trajet programmé de mille commandes environ : avancer, plonger, remonter. On veut savoir où il se trouvera à la fin du trajet. Suivre mille commandes à la main serait long et l'erreur quasi certaine : un programme le fait en un instant.

**Ce qu'on réinvestit.** Les bases du chapitre 1 : lire une ligne et accéder à ses caractères par leurs **indices**, convertir un caractère en entier avec `int`, choisir l'action avec `if`/`elif`, et tenir à jour quelques **variables** qui décrivent l'état du sous-marin. L'approfondissement 2 réinvestit en plus l'écriture d'un nombre en base 10, chiffre par chiffre, vue dans ce chapitre.

**La démarche de la fiche.**

1. Comprendre l'effet de chaque commande, puis suivre à la main un petit trajet dans un tableau.
2. Apprendre à reconnaître une commande et à lire son nombre à partir d'une ligne du fichier.
3. Écrire la fonction qui suit tout le trajet : elle donne la réponse de la partie 1.

4. Pour la partie 2, reprendre la même fonction avec une troisième variable et de nouveaux effets pour les commandes.



## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `forward` | en avant | `depth` | profondeur |
| `horizontal position` | position horizontale | `command` | commande |
| `to increase by` | augmenter de | `to decrease by` | diminuer de |
| `planned course` | trajet prévu | `to multiply` | multiplier |

### Résumé de l'énoncé (partie 1)

**Contexte.** Vous pilotez maintenant le sous-marin. Son ordinateur de bord contient un trajet déjà programmé : une liste de commandes qui le font avancer, plonger ou remonter. Avant de lancer le trajet, on veut savoir où il va mener le sous-marin.
**Les données.** Chaque ligne du fichier est une commande, exécutée dans l'ordre : un mot (`forward`, `down` ou `up`), une espace, puis un entier $X$, par exemple `down 4`.
**La règle.**

- On suit deux grandeurs, qui valent 0 au départ : la position horizontale et la profondeur.
- `forward X` fait avancer : la position horizontale augmente de $X$.
- `down X` fait plonger : la profondeur augmente de $X$.
- `up X` fait remonter : la profondeur *diminue* de $X$ (c'est une profondeur, donc remonter la réduit).

**La réponse attendue.** Le produit position horizontale × profondeur à la fin du trajet. Par exemple, après `forward 5`, `down 2`, `up 1`, `forward 3`, la position horizontale vaut 8 et la profondeur 1 : la réponse serait $8 \times 1 = 8$.

### Exercice 1 *Vérifier sa compréhension*

**But.** Réfléchir à l'effet des commandes sur la réponse et à la façon de lire une ligne.
**À quoi ça sert ?** La première question montre l'effet d'une variable restée à 0 sur un produit ; la deuxième prépare la partie 2, où l'ordre des commandes compte ; la troisième annonce la méthode de lecture d'une ligne utilisée dans le programme.

Questions pièges, à traiter sur le cahier :

1. Quelle est la réponse pour un trajet qui ne contient aucune commande `forward` ?
2. Si l'on mélange l'ordre des lignes, la réponse de la partie 1 change-t-elle ?
3. Pour la ligne `"down 7"`, que valent `ligne[0]` et `ligne[-1]` ? De quel type est `ligne[-1]` ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Suivre un petit trajet inventé dans un tableau, commande après commande.
**À quoi ça sert ?** Le tableau montre exactement ce que le programme devra faire : mettre à jour deux variables à chaque ligne. Sa dernière ligne donne un résultat connu, qui servira à tester la fonction `trajet`.

On considère le trajet (inventé) suivant :

```
forward 3
down 4
forward 6
up 1
down 5
forward 2
up 2
```

Faire un tableau avec une ligne par commande et deux colonnes : position horizontale et profondeur. Vérification : on doit arriver à `11` et `6`, soit la réponse `66`.

## 2 Programmer la partie 1

### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans le fichier (avec `for ligne in fichier:`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
1 ligne = "1721\n"
2 print(ligne.strip())                  # 1721
3 print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

### Exercice 3 *Lire une commande*

**But.** Extraire d'une ligne du fichier la lettre qui identifie la commande et le nombre $X$.
**À quoi ça sert ?** Ce sont les deux seules informations dont le programme a besoin pour chaque ligne : la lettre choisit l'action, le nombre dit de combien bouger. Cette lecture sera utilisée dans les deux parties.

Dans les fichiers de données de ce défi, le nombre $X$ n'a qu'**un seul chiffre** (le vérifier en parcourant votre fichier). On peut donc lire une commande sans la découper : sa première lettre suffit à reconnaître la commande, et son dernier caractère est le nombre.

```
1 ligne = "forward 3\n"          # une ligne telle qu'on la lit dans le fichier
2 ligne = ligne.strip()          # "forward 3"
3 print(ligne[0])                # f
4 print(int(ligne[-1]))          # 3 (un entier)
```

Recopier et exécuter ce code. Pourquoi faut-il `strip()` avant de lire `ligne[-1]` ? Les trois commandes ont-elles bien des premières lettres différentes ?

### Exercice 4 *Suivre le trajet*

**But.** Écrire la fonction qui lit tout le fichier, applique chaque commande et renvoie le produit final.
**À quoi ça sert ?** Elle donne la réponse de la partie 1. Sa structure (lecture, deux variables d'état, un choix selon la lettre) sera reprise telle quelle dans la partie 2, avec une variable de plus.

Écrire une fonction `trajet(nom_fichier)` qui lit le fichier ligne par ligne (`open`, boucle `for`, `strip`), met à jour deux variables `horizontal` et `profondeur`, et renvoie leur produit. Tester sur l'exemple, puis sur vos données. ▷ coup de pouce ▷ début de solution

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

**Résumé de la partie 2 (à lire après avoir validé la partie 1)**

**Ce qui change.** En relisant le manuel du sous-marin, on découvre que les commandes ne signifient pas tout à fait ce qu'on croyait. Une troisième grandeur entre en jeu, la *visée* (*aim*), qui vaut 0 au départ et indique l'inclinaison du sous-marin : plus elle est grande, plus le sous-marin descend quand il avance.
**La nouvelle règle.**

- `down X` augmente la visée de $X$, sans toucher à la profondeur.
- `up X` diminue la visée de $X$, sans toucher à la profondeur.
- `forward X` augmente la position horizontale de $X$ **et** augmente la profondeur de visée $\times X$.

**La réponse attendue.** Le même produit position horizontale × profondeur à la fin du trajet. Avec le même trajet inventé `forward 5`, `down 2`, `up 1`, `forward 3` : la visée vaut 0 au premier `forward`, puis 1 au second, qui fait donc descendre de $1 \times 3 = 3$. On arrive à la position 8 et à la profondeur 3 : la réponse serait 24.

### Exercice 5 *Avec une troisième variable*

**But.** Refaire le trajet de l'exemple avec la visée, puis adapter la fonction de la partie 1 à la nouvelle règle.
**À quoi ça sert ?** Le tableau fait à la main donne un résultat connu (`440`) pour tester `trajet2`, qui donne la réponse de la partie 2. On voit qu'une nouvelle règle se traduit souvent par une variable d'état de plus.

Refaire le tableau du trajet inventé de l'exercice « À la main » (7 commandes) avec une colonne de plus (la visée) : on doit arriver à `11` et `40`, soit la réponse `440`. Écrire enfin la fonction `trajet2(nom_fichier)`. ▷ coup de pouce ▷ début de solution

## 4 Approfondissement

★★ **Exercice 6** *Approfondissement 1 : un trajet contrôlé*

**But.** Ajouter une nouvelle commande et signaler clairement les commandes inconnues, avec le numéro de la ligne fautive.
**À quoi ça sert ?** Un vrai programme doit réagir proprement à des données imprévues au lieu de donner un résultat faux sans prévenir. Le numéro de ligne permet de retrouver tout de suite l'erreur dans le fichier.

1. Ajouter une commande inventée `back X` : la position horizontale *diminue* de `X` dans une fonction `trajet_controle(nom_fichier)` qui reprend la partie 1.

2. Faire en sorte qu'une commande inconnue affiche un message d'erreur clair avec le numéro de la ligne fautive (la première ligne porte le numéro 1), sans arrêter le programme. ▷ coup de pouce

3. On ajoute à la fin du trajet inventé de la fiche (7 lignes) les deux lignes `back 4` et `jump 3`. Prévoir à la main le message affiché et le résultat, puis vérifier avec le programme.

## ★★ Exercice 7 — *Approfondissement 2 : des nombres à plusieurs chiffres*

**But.** Lire un nombre de plusieurs chiffres, caractère par caractère, sans découper la ligne.
**À quoi ça sert ?** La méthode de la fiche ne marche que si $X$ a un seul chiffre ; celle-ci lève cette limite. Elle reprend la reconstitution d'un nombre en base 10 vue dans ce chapitre, et s'adapte aux autres bases.

Et si $X$ pouvait avoir plusieurs chiffres, comme dans `forward 125` ? On lit alors les chiffres un par un, de gauche à droite, comme lorsqu'on reconstitue la valeur d'un nombre écrit en base 10.

1. On part de `nombre = 0`. À chaque chiffre $c$ lu, on remplace `nombre` par $10 \times$ `nombre` $+ c$. Écrire les valeurs successives de `nombre` pour les chiffres de `125`.

2. Écrire une fonction `valeur_longue(ligne)` qui renvoie le nombre écrit après l'espace, quel que soit son nombre de chiffres. ▷ coup de pouce ▷ début de solution

3. Comment adapter cette méthode à un nombre écrit en base 2 ?

### Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 – piste Les deux variables valent 0 avant la boucle ; dans la boucle, on calcule `lettre` et `x`, puis un `if` / `elif` / `elif` sur la lettre. ↑ énoncé

Ex. 4 – début de solution `lettre = ligne[0]` et `x = int(ligne[-1])` après le `strip()` ; puis `if lettre == "f": horizontal += x`, `elif lettre == "d": ...` ↑ énoncé

Ex. 5 – piste Les mots ne changent pas, mais leur effet, si : deux des commandes ne modifient plus la profondeur directement. ↑ énoncé

Ex. 5 – début de solution Trois variables à 0 : `horizontal`, `profondeur`, `visee`. La commande `forward` modifie maintenant **deux** variables, dont l'une avec une multiplication par `visee`. ↑ énoncé

Ex. 6 q. 2 – piste Un compteur `numero`, augmenté de 1 au début de chaque tour de boucle, et un dernier `else` pour les mots inconnus. ↑ énoncé

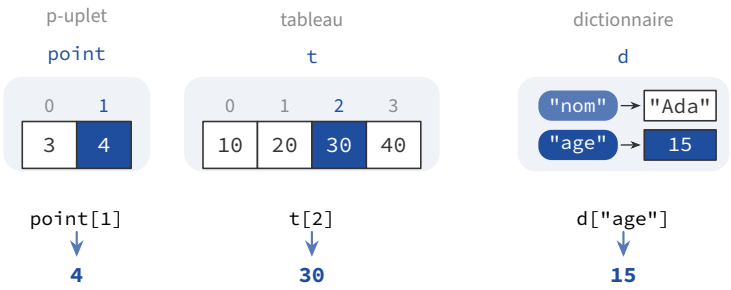
Ex. 7 q. 2 – piste Parcourir les caractères de la ligne avec une variable booléenne `apres_espace`, qui passe à `True` quand on rencontre l'espace. ↑ énoncé

Ex. 7 q. 2 – début de solution Dans la boucle : `if apres_espace:` on met à jour `nombre` avec `int(c)` ; `elif c == " ":` on passe `apres_espace` à `True`. ↑ énoncé

3

CHAPITRE 3

# Les types construits




DANS CE CHAPITRE


- Activités préparatoires . . . 138
- Cours . . . 140
- Exercices . . . 154
- TP et projets . . . 165
- Fiche de révision . . . 170
- Défi Advent of Code . . . 172
- Corrigés . . . 568

# ACTIVITÉS PRÉPARATOIRES



## ACTIVITÉ 1 Des données en vrac

▶ corrigé p. 568

*ranger, retrouver, modifier : trois façons de regrouper*

Durée : 25 min · Par binômes, sans machine

### Consignes

- Matériel : le cahier (toutes les réponses et tous les dessins s'y font), un crayon, quelques papillons adhésifs (*post-it*).
- Jusqu'ici, une variable rangeait **une seule** valeur. Voici trois « tas » de données qui vont ensemble : il s'agit de décider **comment les ranger** pour bien s'en servir ensuite.

| **Tas 1 : une carte d'identité** | **Tas 2 : une liste de courses** | **Tas 3 : les casiers de la classe** |
|---|---|---|
| « Durand », « Léa », 2009, « Monaco »<br>(nom, prénom, année de naissance, ville de naissance) | pain, lait, pommes, beurre<br>(écrits au fil de la semaine) | Léa : casier 112<br>Hugo : casier 87<br>Inès : casier 245 |

### Exercice 1 *Retrouver une information*

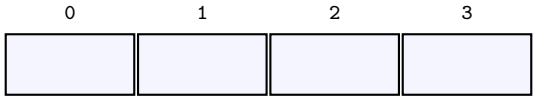
Recopier sur le cahier le tableau ci-dessous : il sert pour les questions 1 à 3.

1. Pour retrouver une information dans chaque tas, que dit-on le plus naturellement : « la troisième » (une **position**) ou « celle de Hugo » (une **étiquette**) ? Compléter la 1re ligne du tableau.
2. Chaque tas va-t-il **changer** après sa création ? Si oui, donner un exemple de changement ; si non, expliquer pourquoi il *ne doit pas* changer. Compléter la 2e ligne.
3. L'**ordre** des éléments compte-t-il ? (Que se passe-t-il si on échange deux éléments ?) Compléter la 3e ligne.

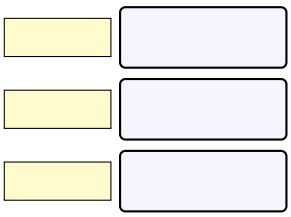
| | **Tas 1 (identité)** | **Tas 2 (courses)** | **Tas 3 (casiers)** |
|---|---|---|---|
| on retrouve par… | | | |
| va changer ? | | | |
| l'ordre compte ? | | | |

**Exercice 2** *Des rangées et des tiroirs*

On propose deux façons de ranger : une **rangée de cases numérotées** à partir de 0, ou des **tiroirs étiquetés**.




*une rangée de cases numérotées*



*des tiroirs étiquetés*

4. Reproduire les deux dessins sur le cahier, puis ranger le tas 2 dans la rangée de cases et le tas 3 dans les tiroirs. Que lit-on dans la case numéro 2 ?

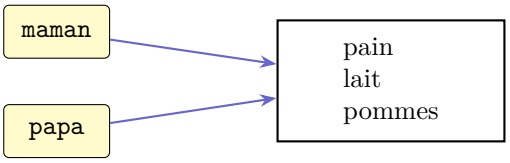
5. On ajoute « œufs » à la liste de courses, puis on raye « lait ». Que devient la rangée ? Les numéros des cases changent-ils ?

6. Hugo change de casier : il prend le 90. Qu'efface-t-on sur le dessin ? Peut-on avoir deux tiroirs étiquetés « Hugo » ? Que faudrait-il faire si deux élèves s'appelaient Léa ?

7. La carte d'identité se range aussi dans une rangée de 4 cases. Mais personne ne doit pouvoir écrire « 2010 » à la place de « 2009 » par erreur. Comment le représenter sur le dessin ?

**Exercice 3** *Une étiquette, ou deux ?*

En Python, une variable est plutôt une **étiquette** (un post-it) collée sur une feuille, avec une **flèche** vers cette feuille. Le soir, Maman écrit la liste de courses et y colle l'étiquette `maman`. Papa dit « c'est aussi ma liste » et colle l'étiquette `papa` **sur la même feuille**.




8. Papa barre « pain » et écrit « brioche » à la place. Que lit Maman sur *sa* liste ?

9. **Prédire, puis vérifier.** Voici la même histoire en Python. Écrire sur le cahier ce qu'affiche la dernière ligne *avant* de l'exécuter (en classe, le professeur l'exécute au tableau ; seul, l'exécuter dans une console Python), puis noter à côté le résultat obtenu.

```
1 maman = ["pain", "lait", "pommes"]
2 papa = maman
3 papa[0] = "brioche"
4 print(maman)
```

10. Que faudrait-il faire, avec les feuilles de papier, pour que Papa puisse modifier sa liste **sans** toucher à celle de Maman ? Dessiner la nouvelle situation sur le cahier (étiquettes, flèches, feuilles).

**Exercice 4** *Mettre des mots*

11. Recopier et compléter : « Pour choisir comment regrouper des données, on se pose deux questions : est-ce qu'on retrouve un élément par sa ..... ou par une ..... ? Est-ce que la collection va ..... après sa création ? »

12. Recopier et compléter : « Une variable n'est pas une boîte qui contient la valeur : c'est une ..... qui ..... vers un objet. Deux variables peuvent désigner ..... objet. »

# COURS



**Programme (BO)** *« Types construits »* : *p-uplets*, *p-uplets nommés*, *tableaux indexés* (listes) et *tableaux de tableaux*, *dictionnaires* (tableaux associatifs). Capacités attendues : écrire une fonction renvoyant un p-uplet ; lire et *modifier* les éléments d'un tableau par leur index ; construire un tableau *par compréhension* ; *itérer* sur les éléments d'un tableau ; représenter des matrices par des tableaux de tableaux ; itérer sur les éléments d'un dictionnaire.

**Prérequis** les bases de Python : variables, types simples (`int`, `float`, `bool`, `str`), boucles `for`/`while`, fonctions. → fiches de révision : ch. 1 p. 59

**Objectifs** savoir **regrouper** des données dans le bon type construit, y **accéder**, les **parcourir** et, quand c'est possible, les **modifier** — en comprenant *ce qui se passe en mémoire*. Ce chapitre est le socle des chapitres suivants (algorithmique, tris, données en tables…).

**Situation de départ**

1. Comment un programme peut-il garder ensemble les 30 notes d'une classe, sans créer 30 variables ?
2. Comment retrouver directement le numéro de casier de Hugo, sans chercher à quelle place il se trouve dans une liste ?
3. Après `copie = notes`, on change `copie[0]` : pourquoi `notes` a-t-elle changé aussi ? Comment faire une vraie copie ?

**Remarque** — *Deux fils conducteurs, à garder en tête tout le chapitre*
**(1) Le fil rouge.** Une variable n'est pas une boîte qui *contient* une valeur : c'est une **étiquette** (un nom) qui **pointe vers un objet** rangé en mémoire. Ce seul dessin — des noms, des flèches, des objets — expliquera *tout* le chapitre : la copie, le partage accidentel de données, le piège des matrices, et jusqu'aux clés de dictionnaire. On le rappellera par un encadré **« Fil rouge »**.
**(2) Le programme est restreint**, mais ces types sont **partout** : on va donc souvent plus loin. Chaque dépassement est signalé par le badge [ au-delà du programme ] — utile, fréquent, mais non exigible au **baccalauréat**.

## I Le point de départ : une variable est une flèche

Jusqu'ici, une variable rangeait **une** valeur simple : un nombre, un booléen, une chaîne. C'est insuffisant pour stocker les 500 notes d'une classe ou un annuaire. On a besoin de variables dont la valeur **regroupe plusieurs valeurs**.

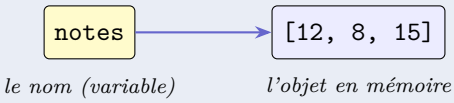
**Définition 1 — Type construit**

Un **type construit** (ou *composé*) est un type dont chaque valeur **regroupe plusieurs valeurs** en un seul objet. Par opposition, `int`, `float`, `bool` et `str` sont considérés ici comme des types **simples**.

Avant tout, installons le modèle mental qui va tout éclairer.

**Règle 1 — Le modèle mémoire : nom ⟶ objet**

En Python, l'instruction `notes = [12, 8, 15]` fait *deux* choses : elle crée l'objet `[12, 8, 15]` quelque part en mémoire, puis fait **pointer** le nom `notes` vers cet objet. La variable **ne contient pas** l'objet : elle le **désigne**.




De cette image découle une distinction **fondamentale** : peut-on modifier l'objet vers lequel on pointe ?

**Définition 2 — Mutable / immuable**

Un objet est **mutable** (modifiable) si l'on peut changer son contenu *sans en créer un nouveau* : c'est le cas des tableaux (`list`) et des dictionnaires (`dict`). Il est **immuable** sinon : c'est le cas des nombres, des chaînes (`str`) et des **p-uplets** (`tuple`).

**Remarque** — *La chaîne : une collection que vous connaissez déjà*
Le type `str` annonce déjà les séquences : ses caractères sont **ordonnés** et indexés à partir de 0 ; on peut aussi en extraire des **tranches** (nouveau : `ch[3:7]` ci-dessous, détaillé avec les tableaux). Ces réflexes vont resservir tels quels.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> ch = "Bonjour"
>>> ch[0]        # premier caractere (indice 0)
'B'
>>> ch[3:7]      # tranche : indices 3 a 6
'jour'
>>> len(ch)      # longueur
7
```

## II Deux questions, trois types

Plutôt que d'apprendre trois types par cœur, apprenons à **choisir**. Devant une collection de données, on se pose **deux questions** :

**Q1.** Comment vais-je **retrouver** un élément : par sa *position* (un indice 0,1,2…) ou par une *étiquette* que je choisis (une *clé*) ?

**Q2.** Cette collection va-t-elle **changer** après sa création (*mutable*), ou est-elle **figée** (*immuable*) ?

Les réponses désignent le type à utiliser — c'est la **carte de décision** du chapitre :

| | accès | modifiable ? | à utiliser pour… |
|---|---|---|---|
| `tuple` (p-uplet) | par indice | **non** (immuable) | des données figées, plusieurs valeurs renvoyées |
| `list` (tableau) | par indice | **oui** (mutable) | une collection ordonnée qui évolue |
| `dict` (dictionnaire) | par **clé** | **oui** (mutable) | associer une valeur à une étiquette |

↻ **Retour sur l'activité.** Ce sont les deux questions du tableau rempli dans l'activité « Des données en vrac » : la carte d'identité, figée et lue par position, sera un p-uplet ; la liste de courses, qui change, un tableau ; les casiers, retrouvés par le prénom, un dictionnaire.

On étudie maintenant les trois, dans cet ordre : le plus simple (figé) d'abord, puis le mutable, puis l'accès par clé.

# III Le p-uplet (`tuple`) : une collection figée

**Définition 3 — p-uplet**

Un **p-uplet** (type `tuple`) est une suite **ordonnée** de $p$ éléments, de types éventuellement différents, que l'on **ne peut pas modifier** après création (immuable).



## A. Création : c'est la virgule qui compte

On sépare les éléments par des **virgules** ; les parenthèses sont facultatives mais recommandées.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> point = (3, 4)             # un couple
>>> t = 1, 2, 3, "bonjour"     # parentheses sous-entendues
>>> t
(1, 2, 3, 'bonjour')
>>> vide = ()                  # p-uplet vide
>>> singleton = (3,)           # UN element : la virgule est OBLIGATOIRE
>>> type((3)), type((3,))      # sans virgule, (3) est juste l'entier 3
(<class 'int'>, <class 'tuple'>)
```

## B. Accès : indice, tranche, `len`, `in`

Comme pour les chaînes : indices à partir de 0, indices négatifs depuis la fin, tranches.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> t = (10, 20, 30, 40, 50)
>>> t[0], t[-1]        # premier, dernier
(10, 50)
>>> t[1:4]             # indices 1, 2, 3 (4 exclu)
(20, 30, 40)
>>> len(t)
5
>>> 20 in t            # test d'appartenance
True
```

## C. Immuable : toute modification est refusée

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> t = (1, 2, 3)
>>> t[0] = 99
Traceback (most recent call last):
  ...
TypeError: 'tuple' object does not support item assignment
```

**Remarque** — *À quoi sert un type qu'on ne peut pas modifier ?*
À **garantir** qu'une donnée ne bougera pas par erreur : coordonnées d'un point, date, couleur RVB. Et — on le verra au §VI — seul un objet immuable peut servir de **clé** de dictionnaire.

## D. Affectation multiple : déballer un p-uplet

Le p-uplet permet d'**affecter plusieurs variables en une seule fois** : on « déballe » ses composantes dans autant de variables.

```
>>> a, b, c = 1, 2, 3      # a vaut 1, b vaut 2, c vaut 3
```

### Règle 2 — Échanger deux variables en une ligne

Au chapitre *Les bases de la programmation Python*, échanger `a` et `b` demandait une variable temporaire. L'affectation multiple le fait en une ligne :

```
>>> a, b = 1, 2
>>> a, b = b, a           # a vaut 2, b vaut 1
```

Python **évalue d'abord tout le membre de droite** : il construit le p-uplet `(b, a)`, c'est-à-dire `(2, 1)`, avec les valeurs *du moment* ; puis il le déballe dans `a` et `b`. Aucune valeur n'est perdue. La même écriture permet d'échanger deux cases d'un tableau (§ IV), `t[i], t[j] = t[j], t[i]` : on la retrouvera au chapitre *Les algorithmes de tri*, et on la rencontre dans des sujets de bac. Cette facilité est propre à Python : dans la plupart des langages, la variable temporaire reste nécessaire.

C'est exactement ainsi qu'on récupère les différentes valeurs renvoyées par une fonction (paragraphe suivant).

## E. Une fonction qui renvoie un p-uplet (capacité attendue)

L'usage numéro un : renvoyer **plusieurs résultats** d'un coup, récupérés par affectation multiple.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 def division(a, b):
2     """Renvoie (quotient, reste) de la division entiere de a par b."""
3     return a // b, a % b
4
5 q, r = division(25, 7)      # q vaut 3, r vaut 4
```

**Exemple** — *Renvoyer le plus petit et le plus grand*

```
1 def ordonner(a, b):
2     """Renvoie le couple (plus petit, plus grand) des nombres a et b."""
3     if a <= b:
4         return a, b
5     else:
6         return b, a
7
8 petit, grand = ordonner(9, 4)   # petit = 4, grand = 9
```

▷ **Je prédis.** Que renvoient `ordonner(5, 5)` et `ordonner(-3, -8)` ?

▷ **Je vérifie.** `(5, 5)` : avec `<=`, l'égalité passe par le premier cas ; puis `(-8, -3)` : `-8` est le plus petit.

► **Exercices d'application :** exercices 1 à 5 (les p-uplets)

# IV Le tableau (`list`) : ordonné et modifiable

Un **tableau** ressemble à un p-uplet… mais il est **mutable**. C'est là que le fil rouge devient indispensable.

## A. Création

3

**Définition 4 — Tableau**

Un **tableau** (type `list`) est une suite **ordonnée** et **modifiable** d'éléments, entre **crochets**.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> notes = [12, 8, 15, 9]
>>> vide = []
>>> singleton = [3]             # un element : PAS de virgule (contrairement au tuple)
>>> [0] * 5                     # repetition : cinq zeros
[0, 0, 0, 0, 0]
```

## B. Lire et modifier par indice (capacité attendue)

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> notes = [12, 8, 15, 9]
>>> notes[1]           # lecture
8
>>> notes[1] = 20      # ECRITURE : on remplace l'element d'indice 1
>>> notes
[12, 20, 15, 9]
>>> notes[0:2]         # tranche
[12, 20]
```

**Règle 3 — Indices négatifs et tranches**

- **Indice négatif** : on compte depuis la fin. `t[-1]` est le dernier élément, `t[-2]` l'avant-dernier ; `t[-k]` désigne `t[len(t) - k]`.
- **Tranche** `t[a:b]` : un *nouveau* tableau, formé des éléments d'indices `a` à `b - 1` (`b` est exclu, comme dans `range`). Si `a` est omis, on part du début ; si `b` est omis, on va jusqu'à la fin. *(Les tranches sont très utiles, mais leur connaissance n'est pas exigible au baccalauréat.)*
- Les deux se combinent : `t[:-1]` est le tableau privé de son dernier élément, c'est-à-dire `t[0:len(t) - 1]`.

Tout cela vaut aussi pour les chaînes et les p-uplets.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> notes = [12, 20, 15, 9]
>>> notes[-1]          # dernier element
9
>>> notes[-2]          # avant-dernier
15
>>> notes[1:3]         # indices 1 et 2 (3 exclu)
[20, 15]
>>> notes[:2]          # depuis le debut
[12, 20]
>>> notes[2:]          # jusqu'a la fin
```

```
[15, 9]
>>> notes[:-1]          # tout sauf le dernier
[12, 20, 15]
```

## C. Parcourir un tableau (capacité attendue)

**(1) Par élément**, quand seules les valeurs comptent :

```
somme = 0                 # INITIALISER
for note in notes:        # PARCOURIR : note prend chaque valeur
    somme = somme + note  # METTRE A JOUR
```

▷ **Je prédis.** Avec `notes = [12, 20, 15, 9]`, que vaut `somme` après la boucle ?
▷ **Je vérifie.** 56 : `somme` passe par 12, 32, 47, puis 56.

**(2) Par indice**, quand on a besoin de la position ou qu'on veut *modifier* :

```
for i in range(len(notes)):
    notes[i] = notes[i] + 1      # +1 point a chaque note
```

**Remarque** — *Pourquoi la méthode 1 ne modifie pas le tableau*
Dans `for note in notes`, la variable `note` est un *nouveau* nom qui pointe tour à tour vers chaque élément. Lui réaffecter `note = note + 1` le fait pointer ailleurs, **sans toucher** au tableau. Pour modifier, il faut l'indice (méthode 2). Ces deux parcours deviendront un patron général (somme, recherche, maximum) au chapitre *Algorithmique : le parcours séquentiel*.

## D. Construction par compréhension (capacité attendue)

Écriture concise, à **maîtriser absolument** : elle fabrique un tableau à partir d'un itérable.

**Règle 4 — Syntaxe de la compréhension**

Deux formes (une par ligne), chaque couleur repérant une partie :

forme simple : `[ expression(x) for x in itérable ]`

avec filtre : `[ expression(x) for x in itérable if condition(x) ]`

expression = ce que l'on range  variable = parcourt l'itérable  itérable = ce que l'on parcourt
condition = filtre (facultatif)

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> [2 * n for n in range(5)]            # les 5 premiers pairs
[0, 2, 4, 6, 8]
>>> [x * x for x in [1, 2, 3, 4]]        # les carres
[1, 4, 9, 16]
>>> [n for n in range(20) if n % 3 == 0]  # avec filtre
[0, 3, 6, 9, 12, 15, 18]
```

**Pour les curieux** — *Un premier pas vers la programmation fonctionnelle* (facultatif)
Une compréhension est une **expression** qui *construit un nouveau tableau* sans modifier quoi que ce soit d'autre : c'est un élément de **programmation fonctionnelle**, un paradigme (une manière de programmer) que l'on étudiera en **Terminale**.

**Exemple** — *La compréhension condense une boucle à accumulateur*

Les deux codes produisent `[1, 4, 9, 16, 25]` :

```
carres = []                          # INITIALISER : version longue, tableau vide
for k in range(1, 6):                # PARCOURIR
    carres.append(k * k)             # METTRE A JOUR

carres = [k * k for k in range(1, 6)]  # version par comprehension
```

## E. Méthodes des tableaux [ au-delà du programme ]

Un tableau étant mutable, il possède des **méthodes** (appel avec un point). Deux d'entre elles suffisent cette année, et reviennent dans les sujets de bac.

### Règle 5 — Ajouter et retirer en fin de tableau

Un tableau `t` dispose des méthodes suivantes :

| Appel | Effet |
|---|---|
| `t.append(x)` | ajoute `x` à la **fin** |
| `t.pop()` | retire et **renvoie** le dernier élément |
| `t.pop(i)` | retire et renvoie l'élément d'indice `i` |

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> t = [3, 1, 2]
>>> t.append(5)       # -> [3, 1, 2, 5]
>>> t.pop()           # renvoie 5 ; t -> [3, 1, 2]
5
>>> t.pop(0)          # renvoie 3 ; t -> [1, 2]
3
```

### Règle 6 — Modifier sur place *vs* renvoyer une copie

`append` modifie le tableau et renvoie `None` : n'écrivez **jamais** `t = t.append(5)` (vous perdriez tout). Pour obtenir un *nouveau* tableau sans modifier l'original, on le construit, par exemple par **concaténation** (`+`).

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> t = [3, 1, 2]
>>> t + [5]           # NOUVEAU tableau
[3, 1, 2, 5]
>>> [0] + t           # ajout en tete, NOUVEAU tableau
[0, 3, 1, 2]
>>> t                 # original intact
[3, 1, 2]
```

### Règle 7 — Méthode : tout le reste s'écrit avec une boucle

Compter les occurrences d'une valeur, chercher sa position, retirer une valeur, inverser ou trier un tableau : on ne cherche pas une méthode toute faite, on **écrit le parcours** (exercices « Chercher et compter » et « Renverser sans `reverse` »). Ces algorithmes sont au cœur des chapitres *Algorithmique : le parcours séquentiel* et *Les tris*.

## F. Le fil rouge à l'œuvre : références, alias et copie

Voici le point le plus important — et le plus piégeux — du chapitre. Reprenons le dessin du §I.

**Règle 8 — Affecter un tableau ne le copie pas !**

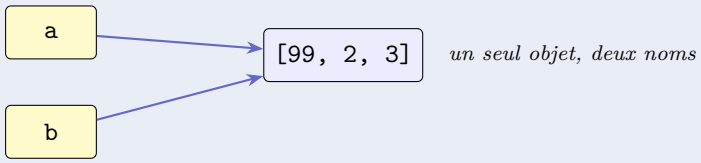
`b = a` ne crée **aucun** nouvel objet : il fait simplement pointer `b` vers **le même** tableau que `a`. On dit que `a` et `b` sont des **alias**. Modifier via l'un se voit via l'autre.



↻ **Retour sur l'activité.** C'est la liste de courses de l'activité : Papa et Maman avaient collé deux étiquettes sur la même feuille, et `print(maman)` affichait `['brioche', 'lait', 'pommes']`. Pour que Papa ait sa propre liste, il fallait recopier la feuille : c'est la vraie copie présentée ci-dessous.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> a = [1, 2, 3]
>>> b = a           # b pointe vers le MEME objet
>>> b[0] = 99
>>> a               # a a change aussi !
[99, 2, 3]
```




Pour une **vraie copie** (objet indépendant), on *reconstruit* un tableau :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> a = [1, 2, 3]
>>> b = [x for x in a]   # (ou a[:]) : NOUVEL objet
>>> b[0] = 99
>>> a               # intact
[1, 2, 3]
```

**Remarque** — *Copie d'un tableau de tableaux*
Une copie simple ne duplique que le premier niveau : pour un tableau *de tableaux*, les sous-tableaux restent partagés. Pour tout dédoubler, il faut aussi recopier **chaque** sous-tableau (copie *profonde*) : c'est l'objet de l'exemple suivant.

**Exemple** — *Copier un tableau de tableaux avec deux compréhensions emboîtées*
Construire une copie indépendante de `grille = [[1, 2], [3, 4]]` à l'aide de **deux** compréhensions emboîtées, puis vérifier que modifier la copie ne touche pas l'original.

▶ **Correction.** La compréhension *intérieure* recrée chaque ligne, la compréhension *extérieure* crée un nouveau tableau de ces lignes : aucun tableau n'est partagé.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> grille = [[1, 2], [3, 4]]
>>> copie = [[x for x in ligne] for ligne in grille]
>>> copie[0][0] = 99
>>> copie
[[99, 2], [3, 4]]
>>> grille          # original intact
[[1, 2], [3, 4]]
```

À comparer avec une copie simple `[ligne for ligne in grille]` : la même modification aurait changé `grille` en `[[99, 2], [3, 4]]`, car les lignes seraient restées partagées.

► **Exercices d'application :** exercices 6 à 16, 35 et 37 (les tableaux ; les compréhensions)

## V Tableaux de tableaux : les matrices (capacité attendue)

Un tableau dont les éléments sont des tableaux **de même longueur** représente une **matrice** : un quadrillage de $n$ lignes et $p$ colonnes.

```
M = [[1, 2, 3],
     [4, 5, 6]]        # 2 lignes, 3 colonnes
```

### Règle 9 — Accès à une case : `M[i][j]`

`M[i]` est la ligne d'indice `i` (un tableau) ; `M[i][j]` est la case ligne `i`, colonne `j`. Le nombre de lignes est `len(M)`, de colonnes `len(M[0])`.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> M[0]        # premiere ligne
[1, 2, 3]
>>> M[1][2]     # ligne 1, colonne 2
6
```

On parcourt une matrice avec **deux boucles imbriquées** :

```
total = 0                       # INITIALISER
for i in range(len(M)):         # PARCOURIR : les lignes
    for j in range(len(M[0])):  # PARCOURIR : les colonnes
        total = total + M[i][j] # METTRE A JOUR : somme de toutes les cases
```

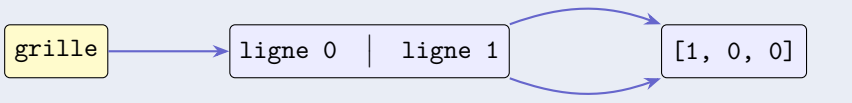
▷ **Je prédis.** Que vaut `total` à la fin, pour la matrice `M` ci-dessus ?
▷ **Je vérifie.** 21, soit $1+2+3+4+5+6$ : les deux boucles passent une fois par chacune des six cases.

### A. Le fil rouge tranche le débat : `[[0]*p]*n` est un piège

Comment créer une grille de zéros ? On serait tenté d'écrire `[[0] * p] * n`, par exemple `[[0] * 3] * 2`. **Erreur.** Le modèle mémoire l'explique d'un coup : l'opérateur `*` ne *copie pas* la ligne, il **répète la même référence**. Les deux lignes sont alors **un seul et même tableau** — exactement l'alias du §IV.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

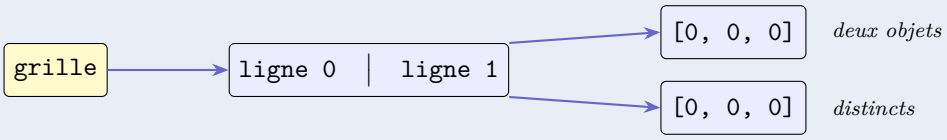
```
>>> grille = [[0] * 3] * 2    # PIEGE
>>> grille[0][0] = 1
>>> grille
[[1, 0, 0], [1, 0, 0]]        # la 2e ligne a change aussi !
```




*les 2 lignes pointent vers le MÊME tableau*

La **bonne** façon : une **compréhension imbriquée**, qui crée à chaque tour une ligne *distincte*.

```
n, p = 2, 3
grille = [[0 for j in range(p)] for i in range(n)]   # 2 lignes INDEPENDANTES
```




**Exemple** — *Une image est une matrice*

Une image en niveaux de gris est une matrice d'entiers de 0 (noir) à 255 (blanc). La traiter, c'est parcourir cette matrice :

```
# Negatif : chaque pixel p devient 255 - p
for i in range(len(image)):                # PARCOURIR : les lignes
    for j in range(len(image[0])):         # PARCOURIR : les colonnes
        image[i][j] = 255 - image[i][j]    # METTRE A JOUR
```

▷ **Je prédis.** Que devient un pixel noir (`0`) ? un pixel blanc ? un pixel de valeur `100` ?

▷ **Je vérifie.** `255` (blanc), `0` (noir) et `155` : le clair devient sombre et inversement.

En couleurs, chaque pixel est lui-même un p-uplet `(rouge, vert, bleu)`.

▶ **Exercices d'application :** exercices 17 à 21 (les matrices)

## VI Le dictionnaire (dict) : l'accès par clé

Dans un tableau, l'étiquette d'accès est **imposée** : un indice $0, 1, 2, \ldots$. Dans un dictionnaire, on la **choisit** : c'est la **clé**.

**Définition 5 — Dictionnaire**

Un **dictionnaire** (type `dict`) est un ensemble de couples **clé** → **valeur**. Les clés sont **uniques** et **immuables** (souvent `str`, `int` ou `tuple`). On l'écrit entre **accolades**, chaque couple sous la forme `clé: valeur`.

↻ **Retour sur l'activité.** Les tiroirs étiquetés de l'activité formaient un dictionnaire : le prénom est la clé, le numéro de casier la valeur. Deux tiroirs « Hugo » étaient impossibles, comme deux clés égales dans un dictionnaire.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> age = {"Alice": 17, "Bob": 16, "Chloe": 17}
>>> age["Alice"]       # acces par la cle, non par un indice
17
>>> vide = {}          # dictionnaire vide
```

**Remarque** — *Tableau associatif = enregistrement*

Le nom savant est **tableau associatif**. C'est l'outil idéal pour une *fiche* : `{"nom": "Ada", "annee": 1815, "domaine": "informatique"}`. On parle aussi de **p-uplet nommé** : comme dans un p-uplet, on regroupe plusieurs valeurs, mais chacune est repérée par un **nom de champ** (`"nom"`, `"annee"`…) plutôt que par une position. En Python, les p-uplets nommés sont écrits avec des dictionnaires. Au chapitre *Les données en tables*, une table entière sera une liste de telles fiches.

## A. Ajouter, modifier, supprimer

Différence majeure avec le tableau : on crée une **nouvelle clé** par simple affectation (impossible avec un indice de tableau, qui doit déjà exister).

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> age = {"Alice": 17, "Bob": 16}
>>> age["David"] = 15      # nouvelle cle : creee
>>> age["Bob"] = 17        # cle existante : valeur remplacee
>>> del age["Alice"]       # suppression du couple
>>> len(age)               # nombre de couples
2
```

### Règle 10 — Le test `in` porte sur les clés

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> "Bob" in age     # une cle ?
True
>>> 17 in age        # NON : in ne regarde pas les valeurs
False
```

Tester `cle in d` avant `d[cle]` évite l'erreur `KeyError` sur une clé absente.

## B. Itérer sur un dictionnaire (capacité attendue)

Une boucle `for` sur un dictionnaire parcourt ses **clés**. Trois méthodes précisent le parcours.

### Règle 11 — `keys`, `values`, `items`

`d.keys()` : les clés. `d.values()` : les valeurs. `d.items()` : les couples `(clé, valeur)`.

```
1  age = {"Alice": 17, "Bob": 16, "Chloe": 17}
2
3  for prenom in age:              # parcourt les cles (defaut)
4      print(prenom, "a", age[prenom], "ans")
5
6  for prenom, a in age.items():   # couples : tres pratique
7      print(prenom, "a", a, "ans")
8
9  total = 0                       # INITIALISER
10 for a in age.values():          # PARCOURIR : les valeurs
11     total = total + a           # METTRE A JOUR
12 moyenne = total / len(age)      # moyenne des ages
```

▷ **Je prédis.** Que vaut `moyenne` à la fin ?

▷ **Je vérifie.** `50 / 3`, soit environ `16.67` : `total` vaut $17 + 16 + 17 = 50$ et le dictionnaire contient 3 couples.

**Exemple** — *Compter des occurrences : le cas d'école du dictionnaire*

Clé = la lettre, valeur = son compteur.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1  def compter(mot):
2      compteur = {}                                  # INITIALISER : dictionnaire vide
3      for lettre in mot:                             # PARCOURIR
4          if lettre in compteur:                     # TESTER : lettre deja vue ?
5              compteur[lettre] = compteur[lettre] + 1  # METTRE A JOUR
6          else:
7              compteur[lettre] = 1                   # METTRE A JOUR : premiere rencontre
8      return compteur                                # RENVOYER
```

```
9
10 compter("abracadabra")                    # {'a': 5, 'b': 2, 'r': 2, 'c': 1,
                                             # 'd': 1}
```

## C. Construction par compréhension [ au-delà du programme ]

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> {n: n * n for n in range(1, 5)}
{1: 1, 2: 4, 3: 9, 4: 16}
```

**Exemple** — *Chiffre de César via un dictionnaire de correspondance*
Décalage de 3 rangs ; `ord`/`chr` convertissent lettre ↔ code (`'A'` ↔ 65).

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 decalage = {chr(65 + i): chr(65 + (i + 3) % 26) for i in range(26)}
2 # decalage['A'] = 'D' , decalage['X'] = 'A' , ...
3 code = ""                          # INITIALISER
4 for c in "BONJOUR":                # PARCOURIR
5     code = code + decalage[c]      # METTRE A JOUR : code vaut "ERQMRXU"
```

## D. Le fil rouge, une dernière fois : pourquoi la clé est-elle immuable ?

**Règle 12 — Une clé de dictionnaire doit être immuable**

Un tableau ne peut pas être une clé ; un p-uplet d'éléments immuables, si (`(12, "lundi")` convient, mais pas `(1, [2])`, qui contient un tableau). Le modèle mémoire l'explique : le dictionnaire range chaque valeur *à une place calculée à partir de la clé*. Si la clé pouvait **changer** après coup (comme un tableau mutable), cette place ne correspondrait plus à rien et la valeur serait **introuvable**. D'où l'exigence d'une clé **figée**.

**Pour les curieux** — *Copier un dictionnaire* (facultatif)
Comme pour les tableaux, `d2 = d` crée un **alias**, pas une copie. Pour obtenir un nouveau dictionnaire contenant les mêmes couples clé-valeur, on le reconstruit par compréhension : `d2 = {cle: d[cle] for cle in d}`.

**Exemple** — *Clés composées avec des p-uplets*
Comme un p-uplet est immuable, il fait une clé parfaite pour une grille creuse ou un planning. Hôtel, clé `(chambre, jour)` → client :

```
1 reservations = {}
2 reservations[(12, "lundi")] = "Alice"
3 reservations[(12, "mardi")] = "Bob"
4 # "Chambre 12 le mardi ?"  ->  reservations[(12, "mardi")]  vaut "Bob"
```

▶ **Exercices d'application :** exercices 22 à 30 et 36 (les dictionnaires ; le fil rouge alias/copie)

## VII Un peu d'histoire, et pour aller plus loin

**Pour les curieux** — *Aux origines du tableau associatif* (facultatif)
Accéder à une donnée par une **clé** plutôt que par une position repose sur le **hachage** (*hashing*), inventé en **1953** chez IBM par **Hans Peter Luhn** : une fonction transforme la clé en une adresse mémoire, rendant l'accès quasi instantané — et exigeant, on l'a vu, une clé *figée* (« hachable »). Les **tableaux**, eux, sont aussi anciens que les premiers ordinateurs : ranger des valeurs dans des cases mémoire *contiguës* et numérotées est au cœur de l'architecture de **von Neumann** (1945).

**Pour les curieux** — *Ces types sont partout* (facultatif)
Le format **JSON**, qui fait circuler la quasi-totalité des données du Web, n'est que des tableaux et des dictionnaires emboîtés. En calcul scientifique, la bibliothèque **NumPy** fournit de vrais tableaux à $n$ dimensions, bien plus rapides que les `list` sur de gros volumes.

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** Les chaînes et les boucles des *Bases de la programmation Python* ; le modèle provisoire du casier y est remplacé par celui de l'étiquette et de la flèche.
- **Ce qu'il prépare.** Les tableaux seront la matière première de l'année (**parcours**, **tris**, **dichotomie**) ; une **table de données** sera une liste de dictionnaires.
- **En Terminale.** *Structures linéaires* et *Programmation objet* présenteront d'autres façons de ranger des données ; les dictionnaires représenteront les graphes, et *Diviser pour régner* fera tourner une image stockée dans une matrice.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| p-uplet `( )` | ordonné, **immuable** ; sert à renvoyer plusieurs valeurs, à faire une clé |
| tableau `[ ]` | ordonné, **mutable** ; lecture *et* modification par indice |
| dictionnaire `{}` | couples **clé** → **valeur** ; `in` teste les clés ; `keys/values/items` |
| Les 2 questions | accès par indice ou par **clé** ? **modifiable** ou figé ? |
| Compréhension | `[expr(x) for x in ... if ...]` : construit un tableau en une ligne |
| Fil rouge (référence) | une variable est une **flèche** vers un objet |
| Alias vs copie | `b = a` → même objet ; `[x for x in a]` / `a[:]` → vraie copie |
| Piège des matrices | `[[0]*p]*n` partage une seule ligne ; utiliser une **compréhension** |
| Clé de dictionnaire | doit être **immuable** : p-uplet (d'éléments immuables) oui, tableau non |

### Retour à la situation de départ

1. Dans un **tableau** (`list`) : `notes = [12, 8, 15, ...]` ; chaque note se lit et se modifie par son **indice**, `notes[i]`, à partir de 0 (§ IV).
2. Avec un **dictionnaire** : `casiers = {"Léa": 112, "Hugo": 87}`, puis `casiers["Hugo"]` donne `87` ; on accède par la **clé**, non par une position (§ VI).
3. `copie = notes` ne crée aucun objet : les deux noms sont des **alias** du même tableau. Pour une vraie copie, on reconstruit le tableau : `copie = [x for x in notes]` (§ IV).

## Erreurs fréquentes

- **Croire que `b = a` copie la liste.** On crée une **deuxième étiquette** sur la **même** liste : modifier `b` modifie `a`. *Le réflexe :* pour copier, une compréhension `[x for x in a]` (ou `a[:]`).
- **Le piège `[[0]*p]*n`.** Les `n` lignes sont la **même** liste partagée. *Le réflexe :* `[[0]*p for i in range(n)]`.
- **Vouloir modifier un p-uplet.** Un `tuple` est **immuable** ; une liste est mutable.
- **Clé de dictionnaire mutable.** Une clé doit être **immuable** (un p-uplet d'éléments immuables oui, une liste non).
- **Confondre indice et clé.** On accède à une liste par **indice** $(0, 1, 2\ldots)$, à un dictionnaire par **clé**.
- **Modifier une liste pendant qu'on la parcourt** (ajout / suppression) : résultats imprévisibles.

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

☐ créer et manipuler **p-uplets**, **listes** (tableaux) et **dictionnaires** ;
☐ accéder par **indice** / par **clé**, et **parcourir** une structure ;
☐ construire une **matrice** (tableau de tableaux) **sans** le piège du partage ;
☐ écrire une **compréhension** de liste ;
☐ comprendre **mutabilité** et **aliasing** (modèle mémoire) et **copier** proprement ;
☐ **choisir** la bonne structure (indice / clé, mutable / immuable).

# EXERCICES



## Mode d'emploi

- Étoiles, ● essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et tester* en Python. Le symbole `>>>` figure une saisie dans la console. Les exercices de **lecture de trace** sont à faire **sans machine** d'abord, puis à vérifier.
- **Réflexe fil rouge** : dès qu'on partage ou copie des données *mutables* (tableau, dictionnaire), dessinez les **noms**, les **objets** et les **flèches**.
- Le badge [ au-delà du programme ] signale un point hors programme (utile, non exigible).

## P-uplets (`tuple`) ↑ cours

★ **Exercice 1** *Lecture de trace* ▶ corrigé p. 569 ● essentiel

Sans machine, donner la valeur de chaque expression. `t = (5, 10, 15, 20, 25)`

**a)** `t[0]` **b)** `t[-1]` **c)** `t[1:3]` **d)** `len(t)` **e)** `15 in t` **f)** `t[-2]`

▷ coup de pouce

★ **Exercice 2** *Vrai ou faux ?* ▶ corrigé p. 569

Justifier en une ligne (ou par un contre-exemple).

1. `(7)` est un p-uplet à un élément.
2. On peut écrire `t[0] = 3` pour changer le premier élément d'un p-uplet.
3. Un p-uplet peut contenir des éléments de types différents.
4. `()` est un p-uplet vide valide.

▷ coup de pouce

★★ **Exercice 3** *Une fonction, plusieurs résultats* ▶ corrigé p. 570

▶ Écrire une fonction `somme_moyenne(t)` qui renvoie le **p-uplet** (somme, moyenne) des éléments d'un p-uplet de nombres non vide (par exemple `(12, 15, 9)`). On récupérera le résultat par `s, m = somme_moyenne(...)`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 4** *Conversion de durée* ▶ corrigé p. 570

▶ Écrire `convertir(n)` qui, à partir d'un nombre de secondes `n`, renvoie le p-uplet `(heures, minutes, secondes)`.

```
>>> convertir(3661)
(1, 1, 1)
```

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 5** *Permutation circulaire* ▶ corrigé p. 570

Soit `a, b, c = 1, 2, 3`.

1. Écrire les instructions (avec une **variable temporaire**) qui font « tourner » les valeurs de sorte qu'ensuite `a` vaille `2`, `b` vaille `3` et `c` vaille `1`. Vérifier en console.
2. Obtenir le même résultat **en une seule ligne**, sans variable temporaire, avec l'affectation multiple. Pourquoi aucune valeur n'est-elle perdue ?

▷ coup de pouce

## Tableaux (`list`) : accès, modification, parcours ↑ cours

★ **Exercice 6** *Lecture de trace* ▶ corrigé p. 570 • essentiel

Sans machine, donner l'état du tableau après chaque ligne.

```
1 t = [4, 8, 15, 16, 23]
2 t[0] = 42
3 t[-1] = 0
4 x = t[1:3]
5 n = len(t)
```

Que valent `t`, `x` et `n` à la fin ?

▷ coup de pouce

★ **Exercice 7** *Fabriquer des tableaux* ▶ corrigé p. 570

▶ Sans les écrire à la main, produire :

**a)** le tableau des entiers de `0` à `9` ;
**b)** le tableau `[10, 20, 30, 40, 50]` ;
**c)** un tableau de `8` cases valant toutes `0`.

▷ coup de pouce

★★ **Exercice 8** *Parcours : les grands classiques* ▶ corrigé p. 571 • essentiel

▶ Pour un tableau `t` de nombres non vide, écrire les fonctions suivantes **sans** utiliser `sum`, `max` ni `min` :

1. `somme(t)` : la somme des éléments ;
2. `maximum(t)` : le plus grand élément ;
3. `moyenne(t)` : la moyenne ;
4. `nb_positifs(t)` : le nombre d'éléments strictement positifs.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 9** *Modifier sur place* ▶ corrigé p. 571

▶ Écrire des fonctions qui **modifient** le tableau reçu (sans en renvoyer un nouveau) :

1. `ajouter_un(t)` : ajoute `1` à chaque élément ;
2. `plafonner(t, m)` : remplace par `m` tout élément supérieur à `m`.

▷ coup de pouce

★★ **Exercice 10** *Chercher et compter* ▶ corrigé p. 571

▶ Écrire sans utiliser les méthodes `count`/`index` :

1. `occurrences(t, x)` : le nombre de fois où `x` apparaît dans `t` ;
2. `est_present(t, x)` : `True` si `x` est dans `t`, sinon `False` ;
3. `premier_indice(t, x)` : l'indice de la première occurrence de `x`, ou `-1` s'il est absent.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 11** *Renverser sans* `reverse` ▶ corrigé p. 572

▶ Écrire `renverser(t)` qui **renvoie un nouveau** tableau contenant les éléments de `t` en ordre inverse, sans utiliser la méthode `reverse` ni la tranche `[::-1]`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★ **Exercice 12** *Ajouter et retirer [ au-delà du programme ]* ▶ corrigé p. 572

▶ Partant de `t = [3, 1, 4, 1, 5]`, donner une instruction pour chaque étape (dans l'ordre), et le contenu de `t` après chacune :

**a)** ajouter `9` à la fin (méthode `append`) ;
**b)** ajouter `0` en tête, par concaténation ;
**c)** retirer (et récupérer) le dernier élément ;
**d)** retirer (et récupérer) l'élément d'indice `2` ;
**e)** ne garder que les trois premiers éléments, à l'aide d'une tranche.

▷ coup de pouce

## Construction par compréhension ↑ cours

★ **Exercice 13** *Lecture de trace* ▶ corrigé p. 572

Sans machine, donner le tableau produit.

**a)** `[n + 1 for n in range(5)]`
**b)** `[2 ** k for k in range(5)]`
**c)** `[c for c in "python" if c in "aeiouy"]`
**d)** `[n for n in range(30) if n % 5 == 0]`

▷ coup de pouce

★★ **Exercice 14** *Écrire des compréhensions* ▶ corrigé p. 572 • essentiel

Donner **en une ligne** (compréhension) le tableau :

**1.** des cubes de `0` à `6` ;
**2.** des multiples de `3` strictement inférieurs à `40` ;
**3.** des longueurs des mots de `["chat", "python", "nsi"]` (on pourra utiliser `len`) ;
**4.** des entiers de `1` à `50` qui sont des carrés parfaits.

▷ coup de pouce

★★ **Exercice 15** *De la boucle à la compréhension* ▶ corrigé p. 573

Réécrire chacun de ces codes en **une** compréhension.

```
# (a)                                  # (b)
r = []                                 r = []
for x in [5, -2, 7, -8, 3]:            for mot in ["le", "chat", "dort"]:
    if x > 0:                              r.append(mot[0])
        r.append(x)
```

▷ coup de pouce

★★ **Exercice 16** *Les voyelles d'une phrase* ▶ corrigé p. 573

▶ À l'aide d'une compréhension, écrire `voyelles(phrase)` qui renvoie le tableau des voyelles de `phrase` (minuscules ou majuscules), dans l'ordre. Puis en déduire `nb_voyelles(phrase)`.

```
>>> voyelles("Bonjour NSI")
['o', 'o', 'u', 'I']
```

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Matrices (tableaux de tableaux) ↑ cours

★ **Exercice 17** *Se repérer dans une matrice* ▶ corrigé p. 573

On pose `M = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`.

**a)** Que valent `M[0]`, `M[2][1]`, `M[1][0]` ?
**b)** Que valent `len(M)` et `len(M[0])` ? Que représentent-ils ?
**c)** Écrire l'instruction qui met la case centrale à `0`.

▷ coup de pouce

★★ **Exercice 18** *Parcourir une matrice* ▶ corrigé p. 573 • essentiel

▶ Pour une matrice `M` de nombres (`n` lignes, `p` colonnes), écrire :

1. `somme_matrice(M)` : la somme de toutes les cases ;
2. `maximum_matrice(M)` : la plus grande case ;
3. `somme_ligne(M, i)` : la somme de la ligne d'indice `i`.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 19** *Créer une grille — et le piège* ▶ corrigé p. 574

1. **Lecture de trace (sans machine).** On exécute :

```
1 g = [[0] * 3] * 2
2 g[0][0] = 1
```

Donner la valeur de `g`. Est-ce le résultat attendu ? **Expliquer** avec un schéma du fil rouge (noms, objets, flèches).

2. ▶ Écrire `grille_zeros(n, p)` qui renvoie *correctement* une matrice de `n` lignes et `p` colonnes remplie de `0`, en utilisant une compréhension. Vérifier que modifier une case n'en modifie qu'une.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 20** *Transposée* ▶ corrigé p. 574 ≈ 15 min

▶ Écrire `transposee(M)` qui renvoie la **transposée** d'une matrice `n`×`p` (les lignes deviennent les colonnes), sans modifier `M`.

```
>>> transposee([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]
```

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 21** *Traitement d'image* ▶ corrigé p. 575 ≈ 20 min

Une image en niveaux de gris est une matrice d'entiers de `0` à `255`.

1. ▶ `negatif(image)` : **modifie** l'image en remplaçant chaque pixel `p` par `255 - p`.
2. ▶ `miroir_gauche_droite(image)` : renvoie une **nouvelle** image, miroir gauche-droite, c'est-à-dire symétrique par rapport à un axe vertical (chaque ligne est renversée).

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Dictionnaires (`dict`) ↑cours

★ **Exercice 22** *Lecture de trace* ▶ corrigé p. 575 • essentiel

On part de `d = {"a": 1, "b": 2}`. Donner l'état de `d` (ou la valeur) après chaque ligne, dans l'ordre.

```
d["c"] = 3
d["a"] = 10
del d["b"]
x = "c" in d
y = 3 in d
z = len(d)
```

▷ coup de pouce

★★ **Exercice 23** *Parcourir un dictionnaire* ▶ corrigé p. 575

On dispose de `notes = {"Alice": 14, "Bob": 9, "Chloe": 17}`.

1. ▶ Écrire `moyenne(notes)` qui renvoie la moyenne des valeurs.
2. ▶ Écrire `admis(notes, seuil)` qui renvoie le tableau des prénoms dont la note est $\geqslant$ `seuil`.
3. ▶ Écrire `meilleur(notes)` qui renvoie le prénom ayant la meilleure note.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★ **Exercice 24** *Compter des votes : code à trous* ▶ corrigé p. 576

Le tableau `votes` contient le prénom choisi par chaque votant. Recopier et compléter les pointillés pour que la fonction renvoie un dictionnaire associant à chaque prénom son nombre de voix.

```
def compter_votes(votes):
    resultat = ......
    for prenom in ......:
        if prenom ...... resultat:
            resultat[prenom] = ......
        else:
            resultat[prenom] = ......
    return resultat
```

```
>>> compter_votes(["Ada", "Léo", "Ada"])
{'Ada': 2, 'Léo': 1}
```

▷ coup de pouce

★★ **Exercice 25** *Comptage d'occurrences* ▶ corrigé p. 576 • essentiel

▶ Écrire `compter_lettres(mot)` qui renvoie un dictionnaire associant à chaque lettre de `mot` son nombre d'occurrences.

```
>>> compter_lettres("banane")
{'b': 1, 'a': 2, 'n': 2, 'e': 1}
```

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 26** *Inverser un dictionnaire* ▶ corrigé p. 576

▶ Écrire `inverser(d)` qui échange clés et valeurs (on suppose les valeurs uniques).

```
>>> inverser({"a": 1, "b": 2})
{1: 'a', 2: 'b'}
```

**Question :** pourquoi cela ne marcherait-il pas si deux clés partageaient la même valeur ?

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 27** *Un relevé de notes à clés composées* ▶ corrigé p. 576 ≈ 15 min

On enregistre les notes d'une classe dans un dictionnaire dont la clé est le p-uplet `(élève, matière)`.

```
releve = {("Ada", "NSI"): 18, ("Ada", "Maths"): 15, ("Léo", "NSI"): 12}
```

1. Donner l'instruction qui lit la note d'Ada en Maths.
2. ▶ Écrire `moyenne_eleve(releve, eleve)` qui renvoie la moyenne d'un élève sur toutes ses matières.
3. **Pourquoi** la clé est-elle un p-uplet et non un tableau ? Répondre en une phrase, avec le vocabulaire du fil rouge du chapitre.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Le fil rouge à l'épreuve : références, alias, copie ↑ cours

★★ **Exercice 28** *Alias ou copie ? (lecture de trace)* ▶ corrigé p. 577 • essentiel

Sans machine, prédire l'affichage. Pour chaque cas, dire s'il y a **un** ou **deux** objets en mémoire.

```
# (a)                    # (b)
a = [1, 2, 3]            a = [1, 2, 3]
b = a                    b = [x for x in a]
b.append(4)              b.append(4)
print(a, b)              print(a, b)
```

▷ coup de pouce

★★ **Exercice 29** *Effet de bord d'une fonction* ▶ corrigé p. 577

Sans machine, prédire l'affichage et l'**expliquer**.

```
def ajoute_zero(t):
    t.append(0)

notes = [12, 15]
ajoute_zero(notes)
print(notes)
```

**Question :** comment récrire `ajoute_zero` pour qu'elle *ne modifie pas* le tableau reçu mais en **renvoie** un nouveau ?

▷ coup de pouce

★★★ **Exercice 30** *Le piège en dictionnaire* ▶ corrigé p. 577 ≈ 10 min

Sans machine, prédire l'affichage.

```
modele = {"vie": 100, "sac": []}
joueur1 = modele
joueur2 = {cle: modele[cle] for cle in modele}
joueur1["vie"] = 50
joueur2["sac"].append("clef")
print(modele["vie"], modele["sac"])
```

[ au-delà du programme ] ▷ coup de pouce ▷ début de solution

## Synthèse : petits problèmes ↑ cours

★★ **Exercice 31** *Carnet de notes* ▶ corrigé p. 577

Un carnet associe à chaque élève le **tableau** de ses notes :

```
carnet = {"Alice": [12, 15, 9], "Bob": [8, 14]}
```

1. ▶ `ajouter_note(carnet, eleve, note)` : ajoute une note à un élève (créer l'élève s'il est absent).
2. ▶ `moyenne_eleve(carnet, eleve)` : la moyenne d'un élève.
3. ▶ `moyennes(carnet)` : un dictionnaire prénom → moyenne.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 32** *Panier d'une boutique* ▶ corrigé p. 578 ≈ 20 min

Un panier est un dictionnaire `produit` → `quantité`, et `prix` un dictionnaire `produit` → `prix unitaire`.

```
panier = {"pomme": 3, "pain": 2}
prix   = {"pomme": 0.5, "pain": 1.2, "lait": 0.9}
```

1. ▶ `total(panier, prix)` : le montant total du panier.
2. ▶ `ajouter(panier, produit, q)` : ajoute `q` unités (cumule si le produit y est déjà).
3. ▶ `produit_plus_gros_sous_total(panier, prix)` : le nom du produit du panier dont le *sous-total* est le plus élevé.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 33** *Vers le bac — le jeu du démineur (grille)* ▶ corrigé p. 578 ≈ 20 min

▶ On représente un champ de mines par une matrice de `0` et de `1` (`1` = mine). Écrire `compter_voisines(champ, i, j)` qui renvoie le nombre de mines dans les (au plus `8`) cases voisines de la case `(i, j)`. *Attention aux bords : ne pas sortir de la grille.*

```
>>> champ = [[0, 1, 0], [1, 0, 0], [0, 0, 1]]
>>> compter_voisines(champ, 1, 1)
3
```

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 34** *Une copie selon un assistant d'IA* ▶ corrigé p. 579

Un élève demande à un assistant d'IA : « Écris une fonction Python qui renvoie un nouveau tableau contenant le double de chaque note, sans modifier le tableau d'origine. » Voici la réponse obtenue.

```
def doubler(t):
    copie = t                # copie du tableau d'origine
    for i in range(len(copie)):
        copie[i] = copie[i] * 2
    return copie
```

« On commence par copier le tableau dans `copie`, puis on double chaque case de la copie et on la renvoie : le tableau `t` passé en argument n'est pas touché. »

On exécute ensuite :

```
notes = [12, 15, 9]
doubles = doubler(notes)
print(doubles, notes)
```

1. La réponse est-elle correcte ? Prédire l'affichage (avec le dessin noms / flèches / objets), puis ▶ vérifier.
2. Localiser et corriger l'erreur.
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce



## Entraînement ↑ cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

★ **Exercice 35** IA IA IA *Indices et tranches* ▶ corrigé p. 579 • essentiel

Sans machine, donner `t[0]`, `t[-1]`, `t[1:3]`, `t[len(t) - 2]` et `len(t)` pour chacune des valeurs suivantes de `t`, puis ▶ vérifier dans la console : a) `[7, 3, 9, 1, 5]` ; b) `("lun", "mar", "mer", "jeu")` ; c) `"Monaco"`.

▷ coup de pouce

★ **Exercice 36** IA IA IA *Dictionnaires : prédire l'état* ▶ corrigé p. 579 • essentiel

Sans machine, donner l'état du dictionnaire après chaque ligne, puis la valeur des dernières expressions. ▶ Vérifier ensuite.

a)

```
stock = {"pomme": 4, "poire": 0}
stock["kiwi"] = 7
stock["pomme"] = stock["pomme"] + 2
del stock["poire"]
len(stock)
7 in stock
```

b)

```
ages = {"Ada": 16, "Léo": 15}
ages["Zoé"] = 17
ages["Ada"] = 17
"Léo" in ages
15 in ages
len(ages)
```

▷ coup de pouce

★ **Exercice 37** IA IA IA *Compréhensions : prédire le tableau* ▶ corrigé p. 579 • essentiel

Sans machine, donner le tableau construit par chaque compréhension, puis ▶ vérifier.

a) `[3 * k for k in range(4)]`
b) `[len(m) for m in ["a", "bb", "ccc"]]`
c) `[x for x in [4, -1, 0, 7] if x > 0]`
d) `[c + c for c in "nsi"]`
e) `[k for k in range(10) if k % 2 == 1]`

▷ coup de pouce

## Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Les indices commencent à `0` ; un indice négatif compte à partir de la fin (`-1` désigne le dernier élément). Dans une tranche `t[a:b]`, l'indice `b` est exclu. ↑ énoncé

Ex. 2 – piste Relire la sous-section « Création : c'est la virgule qui compte » du cours, puis tester chaque affirmation dans la console (par exemple `type((7))`). ↑ énoncé

Ex. 3 – piste Deux étapes : un accumulateur pour la somme, puis la moyenne à partir de la somme et de `len(t)`. Comment une fonction renvoie-t-elle deux valeurs à la fois ? ↑ énoncé

Ex. 4 – piste Combien de secondes dans une heure ? dans une minute ? Penser au quotient entier `//` et au reste `%`. ↑ énoncé

Ex. 5 – piste Question 1 : quelle valeur serait perdue si l'on écrivait d'abord `a = b` ? C'est elle qu'il faut mettre de côté. Question 2 : relire l'échange `a, b = b, a` du cours ; que calcule Python en premier ? ↑ énoncé

**Ex. 6 – piste** Écrire le contenu de `t` après chaque ligne. La tranche `x` est prise *après* les deux modifications, et `t[1:3]` s'arrête avant l'indice `3`. ↑énoncé

**Ex. 7 – piste** Pour a) et b), une compréhension sur un `range` bien choisi (avec un pas pour b). Pour c), penser à la répétition `*`, déjà vue avec les chaînes. ↑énoncé

**Ex. 8 – piste** Somme et comptage : un accumulateur initialisé *avant* la boucle. Maximum : avec quelle valeur initialiser le « meilleur jusqu'ici », sachant que le tableau est non vide ? ↑

**Ex. 8 – début de solution**
```
def maximum(t):
    m = t[0]
    for x in t:
```
puis comparer `x` à `m`. ↑énoncé

**Ex. 9 – piste** Pour modifier une case, il faut connaître sa position : parcourir par **indice** (`for i in range(len(t))`) plutôt que par élément. ↑énoncé

**Ex. 10 – piste** Pour `premier_indice`, parcourir par indice et sortir avec `return` dès qu'on trouve `x`. Que renvoyer si la boucle se termine sans avoir rien trouvé ? ↑énoncé

**Ex. 11 – piste** Partir d'un tableau vide et le remplir avec `append` : dans quel ordre faut-il parcourir les indices de `t` ? ↑énoncé

**Ex. 12 – piste** Les méthodes `append` et `pop` sont présentées dans la sous-section « Méthodes des tableaux » du cours. Recalculer les indices après chaque étape : le tableau change à chaque ligne. ↑énoncé

**Ex. 13 – piste** Lire la compréhension comme une boucle : pour chaque valeur de la variable, si la condition est vraie, on ajoute l'expression au tableau. Écrire d'abord la liste des valeurs prises par la variable. ↑énoncé

**Ex. 14 – piste** Forme générale : `[expression for variable in ... if condition]`. Pour la 4, plutôt que de tester chaque entier, peut-on fabriquer directement les carrés ? Jusqu'où faire varier la variable ? ↑énoncé

**Ex. 15 – piste** Repérer dans chaque boucle trois morceaux : ce qui est ajouté (dans `append`), la ligne `for`, et la condition éventuelle. Dans quel ordre les placer entre crochets ? ↑énoncé

**Ex. 16 – piste** Le filtre porte sur chaque caractère : est-il dans `"aeiouyAEIOUY"` ? Pour `nb_voyelles`, réutiliser `voyelles`. ↑énoncé

**Ex. 17 – piste** `M[i]` est la ligne d'indice `i` (elle-même un tableau) ; `M[i][j]` est la case d'indice `j` de cette ligne. Lignes et colonnes sont numérotées à partir de `0`. ↑énoncé

**Ex. 18 – piste** Deux boucles imbriquées : l'une sur les lignes, l'autre sur les cases de la ligne. Pour le maximum, initialiser avec une case qui existe. Pour `somme_ligne`, une seule boucle suffit. ↑énoncé

**Ex. 18 – début de solution**
```
def somme_matrice(M):
    s = 0
    for ligne in M:
        for x in ligne:
```
↑énoncé

**Ex. 19 – piste** Question 1 : `* 2` recopie-t-il la ligne, ou seulement la flèche vers la ligne ? Question 2 : chaque ligne doit être un objet neuf ; quelle construction réévalue `[0] * p` pour chaque ligne ? ↑énoncé

**Ex. 20 – piste** Si `M` a `n` lignes et `p` colonnes, combien de lignes et de colonnes a sa transposée ? Comparer la case `(i, j)` de la transposée avec une case de `M`. ↑énoncé

**Ex. 20 – début de solution**
```
n = len(M)
p = len(M[0])
```
puis construire `p` lignes de `n` cases chacune, par compréhension ou avec deux boucles. ↑énoncé

**Ex. 21 – piste** 1 : on modifie sur place, donc on parcourt par indices (ligne `i`, colonne `j`). 2 : chaque ligne se renverse comme dans l'exercice « Renverser sans `reverse` » ; on construit l'image résultat ligne par ligne. ↑énoncé

Ex. 21 – début de solution
```
def negatif(image):
    for i in range(len(image)):
        for j in range(len(image[i])):
```
↑énoncé

Ex. 22 – piste Affecter une clé absente l'ajoute ; affecter une clé présente remplace sa valeur. Le mot-clé `in` teste-t-il les clés ou les valeurs ? ↑énoncé

Ex. 23 – piste `for prenom in notes:` parcourt les **clés** ; la note est `notes[prenom]`. Pour `meilleur`, il faut retenir le *prénom* du meilleur jusqu'ici, pas seulement sa note. ↑énoncé

Ex. 24 – piste Le résultat part vide. Pour chaque prénom : s'il est déjà une clé, on augmente son compte ; sinon, on le crée. Avec quelle valeur ? ↑énoncé

Ex. 25 – piste Même schéma que l'exercice précédent : qu'est-ce qui joue ici le rôle de clé ? Que faire quand on rencontre une lettre pour la première fois ? ↑énoncé

Ex. 26 – piste Remplir un dictionnaire neuf en parcourant les clés de `d` : que devient la clé, que devient la valeur ? Pour la question : que se passe-t-il quand on affecte deux fois la même clé ? ↑énoncé

Ex. 27 – piste Question 2 : parcourir les clés `(nom, matiere)` du relevé et ne garder que celles de l'élève cherché ; il faut une somme *et* un compteur. Question 3 : relire la règle du cours sur les clés de dictionnaire. ↑énoncé

Ex. 27 – début de solution
```
total = 0
n = 0
for (nom, matiere) in releve:
    if nom == eleve:
```
↑énoncé

Ex. 28 – piste Dessiner, pour chaque cas, les noms et les flèches après la 2e ligne : la compréhension `[x for x in a]` fabrique-t-elle un nouvel objet ? ↑énoncé

Ex. 29 – piste Pendant l'appel, le paramètre `t` et la variable `notes` sont-ils deux objets, ou deux étiquettes sur le même objet ? Pour la question, commencer par fabriquer une copie. ↑

Ex. 30 – piste La compréhension copie le premier niveau seulement : la valeur associée à `"sac"` est-elle recopiée, ou seulement la flèche vers ce tableau ? ↑énoncé

Ex. 30 – début de solution Dessiner trois noms `modele`, `joueur1`, `joueur2` : `joueur1` pointe vers le même dictionnaire que `modele` ; `joueur2` pointe vers un nouveau dictionnaire. Suivre ensuite chaque flèche lors des deux modifications. ↑énoncé

Ex. 31 – piste Question 1 : tester `eleve in carnet` avant d'ajouter. Question 3 : réutiliser `moyenne_eleve` pour chaque clé du carnet. ↑énoncé

Ex. 31 – début de solution
```
def ajouter_note(carnet, eleve, note):
    if eleve not in carnet:
        carnet[eleve] = []
```
↑énoncé

Ex. 32 – piste Sous-total d'un produit = quantité × prix unitaire. Question 3 : c'est une recherche de maximum, mais on retient le *nom* du produit en plus de son sous-total. ↑énoncé

Ex. 32 – début de solution
```
def total(panier, prix):
    s = 0
    for produit in panier:
```
↑énoncé

Ex. 33 – piste Une voisine de `(i, j)` s'écrit `(i + di, j + dj)` avec `di` et `dj` pris dans `-1, 0, 1`. Quel couple `(di, dj)` faut-il écarter ? Quand une case `(a, b)` est-elle dans la grille ? ↑

Ex. 33 – début de solution
```
for di in [-1, 0, 1]:
    for dj in [-1, 0, 1]:
        a, b = i + di, j + dj
```
↑énoncé

Ex. 34 – piste Dessiner les noms `t` et `copie` juste après la première ligne de la fonction : combien d'objets tableau y a-t-il ? ↑énoncé

Ex. 35 – piste Écrire les indices sous chaque élément : `0`, `1`, `2`… puis, en partant de la fin, `-1`, `-2`… Une tranche d'un tableau est un tableau, celle d'un p-uplet un p-uplet, celle d'une chaîne une chaîne. ↑énoncé

Ex. 36 – piste Une affectation `d[cle] = valeur` crée la clé si elle est absente et remplace sa valeur sinon. `in` et `len` portent sur les clés. ↑énoncé

Ex. 37 – piste Écrire d'abord les valeurs successives de la variable, puis barrer celles qui ne vérifient pas la condition, et enfin calculer l'expression pour chacune des valeurs restantes. ↑énoncé

# TP ET PROJETS



## PROJET Sudoku : vérifier une grille

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p03-projet-sudoku` — **archive .zip**

*Un projet de l'année : une vraie application des* types construits.

| | |
|---|---|
| **Prérequis** | boucles `for`, `if`, fonctions, tableaux (type `list`), matrices (`grille[i][j]`). |
| **Fichiers** | `sudoku_grilles.py` (les grilles de test, à télécharger) et `sudoku.py` (à compléter — vous y écrivez tout votre code). |
| **Ce que vous allez faire** | représenter et manipuler une **matrice** (la grille $9 \times 9$) à l'aide d'un **tableau de tableaux** et de l'accès `grille[i][j]` ; parcourir une ligne, une colonne, un bloc ; écrire des fonctions avec une **spécification** claire et les **tester** avec des `assert`. |
| **IA** | Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre. L'exercice 1 (lire dans la grille), geste de base des matrices, se fait sans IA. |

### 1 Le jeu et sa représentation

Une grille de Sudoku fait 9 cases sur 9. Il faut la remplir avec les chiffres de **1 à 9** en respectant trois règles.

**Les règles du Sudoku**

1. chaque **ligne** contient les 9 chiffres sans répétition ;
2. chaque **colonne** contient les 9 chiffres sans répétition ;
3. chacun des neuf **blocs** $3 \times 3$ contient les 9 chiffres sans répétition.

Le but n'est **pas** de jouer, mais d'écrire un programme qui sait dire si une grille respecte les règles.

**Définition 6 — Représentation en Python**

La grille est une **matrice** $9 \times 9$. On la représente par un **tableau de 9 tableaux** (type `list`), chaque sous-tableau étant une ligne ; une case vide vaut `0`.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 | | | 7 | | | | |
| 6 | | | 1 | 9 | 5 | | | |
| | 9 | 8 | | | | | 6 | |
| 8 | | | | 6 | | | | 3 |
| 4 | | | 8 | | 3 | | | 1 |
| 7 | | | | 2 | | | | 6 |
| | 6 | | | | | 2 | 8 | |
| | | | 4 | 1 | 9 | | | 5 |
| | | | | 8 | | | 7 | 9 |

```
grille = [
  [5,3,0, 0,7,0, 0,0,0],
  [6,0,0, 1,9,5, 0,0,0],
  ...
]
```

- `grille[0]` est la **première ligne** : `[5, 3, 0, 0, 7, 0, 0, 0, 0]`.
- `grille[0][1]` est la case **ligne 0, colonne 1**, ici `3`.
- **Attention** : les numéros de ligne et de colonne vont de `0` à `8`, pas de 1 à 9.

Ouvrez `sudoku.py` : tout votre travail s'y fait. Il commence par `from sudoku_grilles import FACILE, COMPLETE_OK, DOUBLON_LIGNE, DOUBLON_COLONNE`. Lancez-le à tout moment : les tests en bas du fichier vous disent ce qui marche.

## 2 Lire dans la grille

**Exercice 1** *Lire une ligne, une colonne et un bloc*

On écrit trois fonctions ; chacune **renvoie un tableau de 9 valeurs**.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

**1.** `valeurs_ligne(grille, i)` renvoie le tableau des 9 valeurs de la ligne numéro `i`. C'est très court : la ligne `i` est *exactement* `grille[i]`.

*Exemple :* sur `FACILE`, `valeurs_ligne(FACILE, 0)` renvoie `[5, 3, 0, 0, 7, 0, 0, 0, 0]`.

```
def valeurs_ligne(grille, i):
    # A COMPLETER (une seule ligne : renvoyer la ligne i)
    ...
```

**2.** `valeurs_colonne(grille, j)` renvoie le tableau des 9 valeurs de la colonne d'indice `j` : il faut aller chercher `grille[0][j]`, `grille[1][j]`, …, `grille[8][j]`. C'est exactement une **compréhension** (vue dans le cours) : on parcourt `i` de `0` à `8` et on collecte `grille[i][j]`.

*Exemple :* `valeurs_colonne(FACILE, 0)` renvoie `[5, 6, 0, 8, 4, 7, 0, 0, 0]`.

```
def valeurs_colonne(grille, j):
    # A COMPLETER : renvoyer, PAR COMPREHENSION, la liste
    # des grille[i][j] pour i allant de 0 a 8
    return [...]
```

**3.** `valeurs_bloc(grille, i, j)` renvoie les 9 valeurs du bloc $3 \times 3$ qui **contient** la case `(i, j)`. On calcule d'abord le coin en haut à gauche du bloc, puis on parcourt ses 3 lignes et ses 3 colonnes.

> **Remarque**
> **Pourquoi `(i // 3) * 3` ?** `//` est la division entière : pour `i = 0, 1, 2` on obtient `0` ; pour `3, 4, 5` on obtient `3` ; pour `6, 7, 8` on obtient `6`. C'est exactement le début de chaque bande de blocs.

*Exemple :* `valeurs_bloc(FACILE, 0, 0)` renvoie `[5, 3, 0, 6, 0, 0, 0, 9, 8]` (le bloc en haut à gauche).

```python
def valeurs_bloc(grille, i, j):
    debut_i = (i // 3) * 3
    debut_j = (j // 3) * 3
    valeurs = []
    # A COMPLETER : parcourir les 3 lignes (de debut_i a debut_i+2)
    # et les 3 colonnes (de debut_j a debut_j+2), et ajouter chaque valeur
    ...
    return valeurs
```

**À tester** avec des `assert` (si tout est correct, *rien* ne s'affiche) :

```python
assert valeurs_ligne(FACILE, 0)    == [5, 3, 0, 0, 7, 0, 0, 0, 0]
assert valeurs_colonne(FACILE, 0)  == [5, 6, 0, 8, 4, 7, 0, 0, 0]
assert valeurs_bloc(FACILE, 0, 0)  == [5, 3, 0, 6, 0, 0, 0, 9, 8]
```

## 3 Peut-on poser un chiffre ?

**Exercice 2** *A-t-on le droit d'écrire* `v` *en* `(i, j)` *?*

Écrire la fonction `placement_valide(grille, i, j, v)`. Elle renvoie `True` si écrire le chiffre `v` dans la case `(i, j)` **ne crée aucun doublon** — ni dans la ligne, ni dans la colonne, ni dans le bloc — et `False` sinon. On suppose que `v` est entre 1 et 9 et que la case est vide. *Astuce : réutiliser les fonctions de lecture ; en Python,* `v in liste` *teste si* `v` *est dans la liste.*

*Exemples :* sur `FACILE`, on **peut** écrire 4 en `(0, 2)` (aucun conflit) ; on **ne peut pas** y écrire 5 (il y a déjà un 5 sur la ligne 0) ni 8 (il y a déjà un 8 dans le bloc).

```python
def placement_valide(grille, i, j, v):
    # A COMPLETER : renvoyer False si v est deja dans la ligne i,
    # dans la colonne j, ou dans le bloc de (i, j) ; True sinon
    ...
```

**À tester** :

```python
assert placement_valide(FACILE, 0, 2, 4) == True    # OK
assert placement_valide(FACILE, 0, 2, 5) == False   # 5 deja sur la ligne
assert placement_valide(FACILE, 0, 2, 8) == False   # 8 deja dans le bloc
```

## 4 Vérifier une grille complète

**Exercice 3** *Contrôler une grille entière*

**1.** `grille_complete(grille)` renvoie `True` si **aucune** case ne vaut `0` (la grille est entièrement remplie), et `False` sinon.

```python
def grille_complete(grille):
    # A COMPLETER : False s'il reste une case egale a 0, True sinon
    ...
```

**2.** `grille_valide(grille)` renvoie `True` si une grille **déjà remplie** respecte les trois règles (aucun doublon dans une ligne, une colonne ou un bloc). *Astuce : pour chaque case, mettre sa valeur de côté dans une variable, remplacer la case par* `0`*, tester* `placement_valide`*, puis remettre la valeur.*

```python
def grille_valide(grille):
    # A COMPLETER : True si aucun doublon dans une ligne, une colonne
    # ou un bloc ; False sinon
    ...
```

*Exemple :* la grille `DOUBLON_LIGNE` contient deux `3` au début de la première ligne : elle est **complète mais fausse**, et sert justement à vérifier que votre `grille_valide` détecte bien l'erreur.

**À tester** :

```python
assert grille_complete(FACILE)        == False
assert grille_complete(COMPLETE_OK)   == True
assert grille_valide(COMPLETE_OK)     == True
assert grille_valide(DOUBLON_LIGNE)   == False
assert grille_valide(DOUBLON_COLONNE) == False
```



## 5 Pour aller plus loin (si vous avez fini)

- Écrire `cases_vides(grille)` qui renvoie le nombre de cases encore à remplir.
- Écrire `coups_possibles(grille, i, j)` : la liste des chiffres qu'on *pourrait* écrire dans une case vide. Une case avec **un seul** coup possible se remplit toute seule !
- Avec ça, on peut déjà résoudre les Sudoku faciles « à la logique », sans deviner.

## 6 Bonus : créer une grille complète

Jusqu'ici on *vérifie* des grilles. On peut aussi en **fabriquer** une, déjà remplie, **sans aucune recherche** : on part d'une grille « patron » donnée par une formule, puis on la brouille par des transformations qui gardent une grille valide.

**Définition 7 — La grille patron**

La formule `(3*(i%3) + i//3 + j) % 9 + 1` remplit une grille $9 \times 9$ **déjà correcte**. (On l'admet ; on peut le vérifier avec votre `grille_valide`.)

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 |
| 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 |
| 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 1 |
| 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 | 4 |
| 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 3 | 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 |
| 6 | 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 |
| 9 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

$\longrightarrow$

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 2 | 6 | 5 | 7 | 1 | 9 | 8 | 4 | 3 |
| 3 | 4 | 8 | 2 | 6 | 5 | 9 | 1 | 7 |
| 7 | 1 | 9 | 3 | 4 | 8 | 5 | 6 | 2 |
| 9 | 3 | 4 | 8 | 2 | 6 | 1 | 7 | 5 |
| 5 | 7 | 1 | 9 | 3 | 4 | 6 | 2 | 8 |
| 8 | 2 | 6 | 5 | 7 | 1 | 4 | 3 | 9 |
| 1 | 9 | 3 | 4 | 8 | 2 | 7 | 5 | 6 |
| 4 | 8 | 2 | 6 | 5 | 7 | 3 | 9 | 1 |
| 6 | 5 | 7 | 1 | 9 | 3 | 2 | 8 | 4 |

*À gauche le patron, à droite une grille obtenue après brouillage. Les deux sont valides.*

Trois transformations gardent une grille valide (à vous de comprendre *pourquoi*) :

- **renommer les chiffres** : choisir une permutation de 1..9 (par ex. $1 \to 3$, $2 \to 1 \ldots$) et remplacer partout ;
- **échanger deux lignes d'une *même* bande** de 3 (ou échanger deux bandes entières) ;
- idem pour les **colonnes** ; ou **transposer** (échanger lignes et colonnes).

**Exercice 4** *Le générateur de grilles*

Le fichier `sudoku.py` fournit déjà trois fonctions :

- `grille_patron()`, qui renvoie la grille patron ;
- `transposer(grille)`, qui échange lignes et colonnes ;
- `generer_grille_complete()`, qui enchaîne : patron, renommage des chiffres, mélange des lignes, transposition, mélange des lignes (donc des colonnes), transposition.

Compléter `renommer_chiffres` (remplacer chaque chiffre selon une permutation choisie au hasard) et `melanger_lignes` (ré-agencer les lignes, bande par bande).

```
def renommer_chiffres(grille):
    nouveaux = [i for i in range(1, 10)]
    random.shuffle(nouveaux)          # une permutation au hasard
    # A COMPLETER : construire le dictionnaire {1:nouveaux[0], ...}
    # puis renvoyer une NOUVELLE grille dans laquelle chaque valeur est remplacee
    ...

def melanger_lignes(grille):
    bandes = [0, 1, 2]
    random.shuffle(bandes)
    resultat = []
    for b in bandes:
        lignes = [b*3, b*3 + 1, b*3 + 2]
        random.shuffle(lignes)
        # A COMPLETER : ajouter grille[indice] a resultat pour chaque indice
        ...
    return resultat
```

**À tester** : la grille produite doit toujours être valide.

```
assert grille_valide(generer_grille_complete()) == True
```

> **Remarque**
> **Pourquoi ça marche ?** Échanger deux lignes d'une même bande ne change ni le contenu des colonnes, ni celui des blocs (les 3 lignes restent dans les mêmes blocs). Renommer les chiffres ne crée aucun doublon là où il n'y en avait pas. La grille reste donc valide — on a juste changé son apparence.

## Bilan

- Une matrice se représente par un **tableau de tableaux**, accès `grille[i][j]`.
- Un même chiffre est interdit deux fois sur une ligne, une colonne ou un bloc.
- Un programme, ça se **spécifie** (ce qu'il attend, ce qu'il renvoie) puis ça se **teste** : réussir les tests ne prouve pas qu'il est juste, mais échouer prouve qu'il est faux.

> **Remarque** — *Lien avec la suite*
> **En Terminale**, on fera écrire au programme la solution *lui-même*, en **essayant** des chiffres et en **revenant en arrière** quand il se trompe : c'est le *backtracking* (retour sur trace), une application de la **récursivité**.

# FICHE DE RÉVISION



## Choisir la bonne structure (deux questions)

| Structure | Accès | Mutable ? | Exemple |
|---|---|---|---|
| p-uplet (`tuple`) | par indice | non | `(48.8, 2.3)` |
| tableau (`list`) | par indice | oui | `[12, 15, 9]` |
| dictionnaire (`dict`) | par clé | oui | `{"Ada": 20}` |

**Q1 :** j'accède par *position* (indice) ou par *étiquette* (clé) ? **Q2 :** ai-je besoin de *modifier* (mutable) ?

## Manipulations de base

```
t = [12, 15, 9]          # tableau
t[0]                     # 12   (acces par indice)
t.append(20)             # ajoute a la fin
notes = {"Ada": 20, "Tom": 12}
notes["Ada"]             # 20   (acces par cle)
for cle in notes:        # parcours des cles
    print(cle, notes[cle])
carres = [k*k for k in range(5)]   # comprehension -> [0,1,4,9,16]
```

## Le modèle mémoire (aliasing)

Une variable **pointe** vers un objet (elle le désigne, ne le contient pas). `b = a` fait pointer `b` vers le **même** tableau : `a` et `b` sont des **alias**.

```
>>> a = [1, 2, 3]
>>> b = a          # PAS une copie
>>> b.append(4)
>>> a
[1, 2, 3, 4]       # a a change aussi !
```

Pour une vraie **copie** (nouvel objet) : `b = [x for x in a]` (ou `a[:]`).

## Le piège des matrices

`[[0]*p]*n` partage **une seule** ligne. La bonne construction :

```
grille = [[0 for j in range(p)] for i in range(n)]   # n lignes independantes
```

## Erreurs à éviter

- `b = a` n'est pas une copie ;
- `[[0]*p]*n` (lignes partagées) ;
- modifier un p-uplet (immuable) ;
- clé de dictionnaire mutable (liste interdite).

## Savoir-faire à maîtriser — je sais…

- ☐ créer et manipuler **p-uplets**, **listes** (tableaux) et **dictionnaires** ;
- ☐ accéder par **indice** / par **clé**, et **parcourir** une structure ;
- ☐ construire une **matrice** (tableau de tableaux) **sans** le piège du partage ;
- ☐ écrire une **compréhension** de liste ;
- ☐ comprendre **mutabilité** et **aliasing** (modèle mémoire) et **copier** proprement ;
- ☐ **choisir** la bonne structure (indice / clé, mutable / immuable).

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 580

1. Quel type construit choisir pour associer une note à chaque prénom ?
2. Après `a = [1, 2]`, `b = [x for x in a]` puis `b.append(3)`, que vaut `a` ?
3. Quel tableau produit `[k % 3 for k in range(5)]` ?
4. Pourquoi `M = [[0]*3]*2` est-il un piège ?
5. Pour `M = [[1, 2], [3, 4], [5, 6]]`, que valent `len(M)`, `len(M[0])` et `M[2][1]` ?

**Et avant ?** — *questions sur les chapitres précédents.* ▶ réponses p. 580

1. Écrire 13 en binaire. (ch. 2)
2. Pourquoi `"5" + 3` provoque-t-il une erreur ? (ch. 1)

# DÉFI ADVENT OF CODE



## DÉFI Binary Diagnostic

*le diagnostic du sous-marin — représentation binaire des entiers*

Jour 3

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p03-defi-aoc-2021-03` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2021/day/3`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Le fichier contient un millier de nombres écrits en binaire, tous de même longueur. Il faut les étudier **colonne par colonne** (quel bit est le plus fréquent à chaque position ?) pour fabriquer deux nouveaux nombres binaires, puis les convertir en base 10. Compter les `1` d'une colonne sur mille lignes, et recommencer pour chaque colonne : c'est un travail pour un programme.

**Ce qu'on réinvestit.** L'**écriture binaire des entiers**, vue au chapitre précédent : lire un nombre bit par bit et le convertir en base 10 avec les puissances de 2. Les **tableaux** de ce chapitre : on garde la liste des lignes sous forme de chaînes, ce qui permet d'accéder au bit d'indice `k` de chaque nombre, et en partie 2 on construit des listes de plus en plus courtes par filtrage.

**La démarche de la fiche.**

1. Comprendre la règle avec des questions courtes, puis la faire à la main sur sept nombres dont on connaît la réponse.

2. Lire le fichier en gardant chaque ligne sous forme de chaîne.
3. Écrire une fonction qui compte les `1` d'une colonne, puis construire $\gamma$ et $\varepsilon$ et les convertir : réponse de la partie 1.
4. Pour la partie 2, réutiliser ce comptage pour filtrer la liste colonne après colonne.

## 1 Lire l'énoncé



Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `diagnostic report` | rapport de diagnostic | `binary number` | nombre en binaire |
| `bit` | chiffre binaire (0 ou 1) | `position` | position, rang |
| `most common` | le plus fréquent | `least common` | le moins fréquent |
| `rate` | taux | `power consumption` | consommation électrique |
| `rating` | valeur, note | `keep / discard` | garder / écarter |

### Résumé de l'énoncé (partie 1)

**Contexte.** Le sous-marin fait des bruits inquiétants, et l'on demande son rapport de diagnostic. Ce rapport ne se lit pas nombre par nombre : on l'étudie colonne par colonne pour en déduire deux nombres, $\gamma$ (*gamma*) et $\varepsilon$ (*epsilon*), qui permettent de calculer la consommation électrique du sous-marin.
**Les données.** Chaque ligne du fichier est un nombre écrit en binaire, c'est-à-dire une suite de `0` et de `1`, comme `00110`. Toutes les lignes ont la même longueur ; on peut donc les écrire les unes sous les autres et lire des **colonnes** : la colonne $k$ réunit les bits en position $k$ de tous les nombres.
**La règle.**

- Le nombre $\gamma$ se construit bit par bit : son bit en position $k$ est le bit **le plus fréquent** dans la colonne $k$ du rapport.
- Le nombre $\varepsilon$ se construit de même avec le bit **le moins fréquent** de chaque colonne. Autrement dit, $\varepsilon$ s'écrit avec les bits contraires de ceux de $\gamma$.
- On convertit ensuite $\gamma$ et $\varepsilon$ de la base 2 vers la base 10.

**La réponse attendue.** Le produit des valeurs **en base 10** de $\gamma$ et $\varepsilon$. Par exemple, pour les trois nombres `110`, `100` et `011`, les bits les plus fréquents des trois colonnes sont `1`, `1` et `0` : $\gamma$ s'écrit `110` (soit 6) et $\varepsilon$ s'écrit `001` (soit 1), donc la réponse serait $6 \times 1 = 6$.

### Exercice 1 *Comprendre ce qui est demandé*

**But.** Vérifier sur des cas particuliers qu'on a bien compris la construction de $\gamma$ et $\varepsilon$, et la façon de lire les données.
**À quoi ça sert ?** La deuxième question, en particulier, décide de la façon de stocker les lignes dans le programme : une mauvaise décision ici fausse tout le reste sans message d'erreur.

Après lecture du résumé (et de l'énoncé), vérifier sa compréhension sur ces cas particuliers :

1. Avec un nombre impair de lignes, une colonne peut-elle avoir autant de `0` que de `1` ?
2. Pourquoi ne faut-il pas convertir `"00110"` en entier dès la lecture si l'on veut garder les colonnes ?
3. Les nombres $\gamma$ et $\varepsilon$ ont-ils toujours des bits opposés ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Construire $\gamma$ et $\varepsilon$ à la main sur sept nombres, puis les convertir en base 10.
**À quoi ça sert ?** On obtient un exemple dont on **connaît la réponse** (220), qui servira à tester chaque fonction du programme ; la conversion à la main est aussi celle que le programme devra faire.

On considère le fichier (inventé) suivant :

```
10110
01100
11101
10011
00110
11100
10101
```

Pour chaque colonne, compter les `1` et les `0`. Écrire $\gamma$ et $\varepsilon$ en binaire, puis les convertir en base 10 à la main (poids $16, 8, 4, 2, 1$). Vérification : on doit obtenir 20 et 11, et la réponse `220`.

## 2 Programmer la partie 1

### Exercice 3 *Lire le fichier*

**But.** Ranger les lignes du fichier dans une liste `nombres`, sans les convertir.
**À quoi ça sert ?** Toutes les questions suivantes travaillent sur cette liste. La garder sous forme de chaînes permet de lire directement le bit d'indice `k` de chaque nombre, et de ne pas perdre les zéros de tête.

#### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans le fichier (avec `for ligne in fichier:`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
1 ligne = "1721\n"
2 print(ligne.strip())                  # 1721
3 print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Rappel pour lire un fichier ligne par ligne :

```
1 fichier = open("input.txt", encoding="utf-8")
2 for ligne in fichier:
3     ligne = ligne.strip()   # enleve le retour a la ligne final
4 fichier.close()
```

Construire la liste `nombres` des lignes, **gardées sous forme de chaînes** : on a besoin d'accéder à chaque bit. ▷ coup de pouce

### Exercice 4 *Compter une colonne*

**But.** Écrire une fonction qui compte combien de nombres ont un `1` dans une colonne donnée.
**À quoi ça sert ?** Connaissant le nombre de `1` d'une colonne, on en déduit le nombre de `0` et donc le bit le plus fréquent : c'est l'outil qui sert à construire $\gamma$ et $\varepsilon$, puis à chaque étape de la partie 2.

Écrire une fonction `nb_uns(nombres, k)` qui renvoie le nombre de lignes dont le bit d'indice `k` vaut `"1"`. Sur l'exemple inventé de la fiche, `nb_uns(nombres, 0)` vaut `5`. ▷ coup de pouce

### Outil Python : conversions binaires (`int(s, 2)`, `bin`)

Python sait convertir directement entre l'écriture binaire et un entier. `int(s, 2)` lit la chaîne `s` comme un nombre écrit en base 2 ; `bin(n)` donne l'écriture binaire de `n`, précédée de `0b`.

```
print(int("10100", 2))      # 20 : la chaine est lue en base 2
print(int("00110", 2))      # 6 : les zeros de tete ne genent pas
print(bin(20))              # 0b10100 : ecriture binaire, prefixe 0b
print(bin(20)[2:])          # 10100 : sans le prefixe
```

**Intérêt.** Ces fonctions servent à **vérifier** votre propre conversion. Écrire la conversion soi-même (méthode du cours) reste l'objectif de l'exercice.
*À essayer.* Que renvoient `int("11111", 2)` et `bin(11)` ?

### Exercice 5 — *Gamma, epsilon et conversion*

**But.** Construire les chaînes `gamma` et `epsilon` colonne par colonne, les convertir en base 10 et calculer la réponse de la partie 1.
**À quoi ça sert ?** C'est l'aboutissement de la partie 1. La fonction de conversion `binaire_vers_entier` resservira telle quelle pour les deux valeurs de la partie 2.

Construire les chaînes `gamma` et `epsilon` colonne par colonne, puis calculer la réponse. Pour convertir, écrire d'abord votre propre fonction `binaire_vers_entier(s)` ; vérifier ensuite avec la fonction toute faite `int(s, 2)`. ▷ coup de pouce ▷ début de solution

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

### Résumé de la partie 2 (à lire après avoir validé la partie 1)

**Ce qui change.** On vérifie maintenant le système de survie du sous-marin. Il faut encore deux valeurs, mais elles ne se construisent plus bit par bit : chacune est l'un des nombres du rapport, trouvé par **élimination**.
**La nouvelle règle.**

- On part de la liste complète et l'on examine les colonnes de gauche à droite.
- Pour la première valeur, à chaque colonne, on ne garde que les nombres dont le bit vaut le bit **le plus fréquent** de cette colonne ; en cas d'égalité, on garde ceux qui ont un `1`.
- Pour la seconde valeur, on garde au contraire ceux dont le bit vaut le bit **le moins fréquent** ; en cas d'égalité, on garde ceux qui ont un `0`.
- Le bit le plus ou le moins fréquent se calcule **sur les nombres restants**, pas sur la liste de départ.
- On s'arrête dès qu'il ne reste qu'un nombre : c'est la valeur cherchée.

**La réponse attendue.** Le produit des deux valeurs en base 10. Par exemple, avec `110`, `100` et `011` : pour la première valeur, la colonne 0 fait garder `110` et `100`, puis la colonne 1 est à égalité, on garde `110` (soit 6) ; pour la seconde, la colonne 0 ne laisse que `011` (soit 3). La réponse serait $6 \times 3 = 18$.

### Exercice 6 *Filtrer la liste*

**But.** Écrire une fonction qui ne garde que les nombres ayant un bit donné dans une colonne, puis l'appliquer colonne après colonne jusqu'à ce qu'il ne reste qu'un nombre.
**À quoi ça sert ?** Le filtrage, répété, donne chacune des deux valeurs de la partie 2. On réutilise la fonction de comptage de la partie 1 (sur la liste restante) et la fonction de conversion pour obtenir la réponse.

Écrire une fonction `filtrer(nombres, k, bit)` qui renvoie la nouvelle liste des seuls nombres dont le bit d'indice `k` vaut `bit`. Puis écrire une fonction qui applique le procédé de la partie 2 colonne après colonne jusqu'à ce qu'il ne reste qu'un nombre. Lire très attentivement la règle en cas d'**égalité** : elle n'est pas la même pour les deux valeurs. Sur l'exemple inventé de la fiche, on doit trouver `22` et `6`, donc la réponse `132`. ▷ coup de pouce ▷ début de solution

## 4 Approfondissement

### ★★★ Exercice 7 *Approfondissement 1 : sans chaînes*

**But.** Refaire la partie 1 en manipulant directement des entiers, sans passer par des chaînes de caractères.
**À quoi ça sert ?** On retrouve un bit par des divisions entières et des restes : c'est le lien entre l'écriture binaire et l'arithmétique du chapitre précédent, et une autre façon, plus économe, de traiter les mêmes données.

1. À la main : avec 5 bits, quel est le bit de poids $2^2$ de $22 = 10110_2$ ? Calculer `(22 // 4) % 2` et comparer.
2. Refaire la partie 1 en convertissant chaque ligne en entier dès la lecture, et en obtenant le bit de poids $2^p$ par des calculs. ▷ coup de pouce
3. Comment obtenir `epsilon` à partir de `gamma` par une seule soustraction ? ▷ début de solution

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

**Ex. 3 – piste** Si l'on convertissait tout de suite en entier, la ligne `"00110"` deviendrait `110` et l'on perdrait les zéros de tête. ↑ énoncé

**Ex. 4 – piste** Un compteur et une boucle sur la liste ; le bit d'indice `k` de la chaîne `s` est `s[k]`. Attention : on compare à la **chaîne** `"1"`, pas à l'entier `1`. ↑ énoncé

**Ex. 5 – piste** Le bit le plus fréquent est `"1"` si le nombre de uns dépasse la moitié du nombre de lignes. Pour la conversion, on peut ajouter les puissances de 2 comme dans le cours, ou utiliser cette astuce plus courte : partir de `n = 0` puis, pour chaque bit de gauche à droite, `n = 2 * n + int(bit)`. ↑ énoncé

**Ex. 5 – début de solution** `gamma = ""` puis, pour `k` de `0` à `len(nombres[0]) - 1` : comparer `nb_uns(nombres, k)` à `len(nombres) - nb_uns(nombres, k)` et ajouter `"1"` ou `"0"` à `gamma` (l'inverse pour `epsilon`). ↑ énoncé

**Ex. 6 – piste** Le bit majoritaire se recalcule à chaque étape sur la liste **restante**, pas sur la liste de départ. ↑ énoncé

**Ex. 6 – début de solution** Une boucle `while len(liste) > 1:` avec un indice `k` qui augmente de 1 à chaque tour ; à chaque tour, compter les uns de la colonne `k` dans `liste`, choisir le bit à garder, puis `liste = filtrer(liste, k, bit_garde)`. ↑

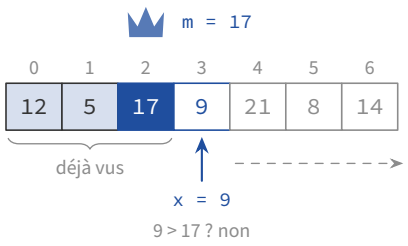
**Ex. 7 q. 2 – piste** Le bit de poids $2^p$ de `n` est `(n // 2**p) % 2`. Pour `gamma`, ajouter $2^p$ quand ce bit est majoritaire. ↑ énoncé

**Ex. 7 q. 3 – début de solution** Avec $m$ bits, `gamma + epsilon` s'écrit avec $m$ chiffres `1`, c'est-à-dire $2^m - 1$. ↑ énoncé

4

CHAPITRE 4

# Algorithmique : le parcours séquentiel




```
def maximum(t):
    m = t[0]
    for x in t:
        if x > m:
            m = x
    return m
```

DANS CE CHAPITRE


- Activités préparatoires . . . . . 178
- Cours . . . . . 180
- Exercices . . . . . 188
- TP et projets . . . . . 196
- Fiche de révision . . . . . 202
- Défi Advent of Code . . . . . 204
- Corrigés . . . . . 581

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 La carte la plus haute

▶ corrigé p. 581

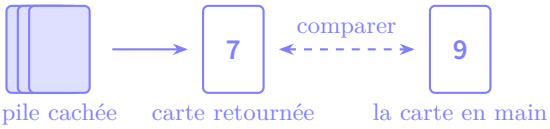
*trouver un maximum en ne retenant qu'une seule carte*

Durée : 20 min · Sans ordinateur, par deux



### Consignes

- Matériel : un paquet de cartes (ou des petits papiers numérotés). Les réponses aux questions s'écrivent sur le cahier.
- Le **meneur** mélange une dizaine de cartes, les pose en pile face cachée, puis les retourne **une par une**. Une carte retournée est aussitôt défaussée, sauf si le chercheur décide de la prendre en main (il défausse alors celle qu'il tenait) : une carte défaussée ne peut plus être revue.
- Le **chercheur** n'a le droit de garder **qu'une seule carte en main**, et ne prend aucune note. Quand la pile est vide, il doit annoncer la carte la plus haute du paquet.
- Valeurs : as = 1, valet = 11, dame = 12, roi = 13. On échange les rôles à chaque partie.




### Exercice 1 *Jouer*

1. Jouer deux parties de 10 cartes (une fois chercheur, une fois meneur). Le chercheur a-t-il trouvé la bonne carte à chaque fois ?
2. Que fait le chercheur avec la **toute première** carte retournée ? Pourquoi ?
3. Écrire la règle du chercheur sous la forme : « à chaque nouvelle carte, si … alors … ; sinon … ».
4. Le meneur a retourné 9 cartes sur 10 et le chercheur tient un roi. Le chercheur peut-il annoncer « c'est le roi » sans regarder la dernière carte ? Et s'il tient un 10 ? Peut-on, en général, s'arrêter avant la fin de la pile ?

### Exercice 2 *Compter le travail*

Une **comparaison**, c'est regarder la carte retournée et la carte en main pour décider laquelle garder.

5. Combien de comparaisons le chercheur a-t-il faites pour 10 cartes ? Combien en ferait-il pour 52 cartes ? Pour $n$ cartes ?
6. Le meneur cherche à faire **changer** de carte le chercheur le plus souvent possible. Dans quel ordre doit-il ranger la pile ? Et pour qu'il ne change **jamais** de carte ? Le nombre de comparaisons change-t-il ?

**7.** Un camarade propose une astuce : « au départ, je fais comme si je tenais une carte de valeur 0 ». On rejoue avec des petits papiers portant des **températures** : $-3$, $-8$, $-1$, $-5$, $-6$. Qu'annonce le camarade ? A-t-il raison ? Quelle est la bonne façon de commencer ?

**8.** Nouvelle mission : annoncer **combien** de cartes rouges contient la pile, toujours sans rien noter. Que faut-il retenir pendant la partie, et comment le faire évoluer à chaque carte ?

## Exercice 3 *Ce qu'on a découvert*

**9.** Recopier et compléter sur le cahier, avec vos mots.

- On examine les cartes …
- À tout moment de la partie, la carte en main est …
- Pour $n$ cartes, il faut … comparaisons : si le paquet double, le travail …
- Pour trouver la plus haute carte ou pour compter les rouges, on fait la même chose : …

# COURS

**Programme (BO)** *« Algorithmes sur les tableaux »* : *parcours séquentiel* d'un tableau ; *recherche* d'un élément ; calcul de la *moyenne*, du *minimum*, du *maximum* ; *notion de coût* d'un algorithme (coût *linéaire*).

**Prérequis** les boucles `for`/`while`, les fonctions, et les **tableaux** (chapitre *Les types construits* : indices, `len`, parcours). → fiches de révision : ch. 1 p. 59, ch. 3 p. 170

**Objectifs** reconnaître et écrire le **patron** du parcours séquentiel ; le décliner (somme, moyenne, comptage, recherche, min/max) ; **évaluer et comparer** le coût d'un algorithme.



### Situation de départ

1. Pour savoir si un jour de l'année a atteint 40 °C, faut-il forcément lire tous les relevés de température ? Et pour trouver la température la plus basse ?
2. Si l'on double le nombre de relevés, le programme met-il toujours deux fois plus de temps ?
3. Comment être certain qu'une fonction qui cherche le plus grand nombre d'une liste répond juste pour *toutes* les listes, et pas seulement pour celles qu'on a essayées ?

**Remarque** — *Le fil conducteur : un patron, et sa facture*
Presque tous les algorithmes de ce chapitre suivent **le même schéma** : parcourir un tableau du début à la fin en tenant à jour une information. On apprend d'abord ce **patron unique**, puis on se pose *la* question de l'algorithmicien : **combien ça coûte ?** On verra qu'un même calcul, selon l'endroit où on le place, peut être rapide (*linéaire*) ou lent (*quadratique*). Les points qui dépassent le programme sont signalés par le badge [ au-delà du programme ].

## I Le parcours séquentiel : un patron unique

### Définition 1 — Parcours séquentiel

**Parcourir** un tableau, c'est examiner ses éléments **un par un**, du premier au dernier. On dit aussi *parcours linéaire*.

La plupart des traitements suivent le même **squelette**, dit motif de l'**accumulateur** : on prépare un résultat, on le met à jour à chaque élément, on le renvoie.

### Règle 1 — Le patron du parcours

```
1 def traitement(t):
2     resultat = ...            # INITIALISER
3     for element in t:         # PARCOURIR
4         resultat = ...        # METTRE A JOUR
5     return resultat           # RENVOYER
```

Écrire un algorithme de parcours, c'est simplement **choisir** l'initialisation et la mise à jour.

**Méthode — Les étiquettes d'étapes**

Dans les exemples et les corrigés, un commentaire en **majuscules** nomme le **rôle** de la ligne. Ce sont toujours les mêmes mots, d'un exercice à l'autre : `INITIALISER`, `PARCOURIR`, `TESTER`, `METTRE A JOUR`, `RENVOYER` ; plus tard `VERIFIER` (une précondition) et, pour la récursivité, `CAS DE BASE`, `APPEL RECURSIF`. Devant un nouveau problème, on se demande d'abord : *que faut-il initialiser ? que met-on à jour ? que renvoie-t-on ?* Le code vient ensuite.

**Méthode — Par élément ou par indice ?**

On parcourt **par élément** (`for x in t`) quand seules les valeurs comptent ; **par indice** (`for i in range(len(t))`) quand on a besoin de la position (ou pour *modifier* le tableau). Les deux réalisent le même parcours.

► **Exercices d'application :** exercices 1 **et** 2 (le patron du parcours)

## II Décliner le patron : somme, moyenne, comptages

**Exemple** — *La somme, puis la moyenne*

Même squelette : on accumule dans `s`, initialisé à `0`.

```
def somme(t):
    s = 0                       # INITIALISER : element neutre de l'addition
    for x in t:                 # PARCOURIR
        s = s + x               # METTRE A JOUR
    return s                    # RENVOYER

def moyenne(t):                 # t suppose NON vide
    return somme(t) / len(t)
```

▷ **Je prédis.** Que renvoient `somme([4, -1, 2])` et `moyenne([4, -1, 2])` ?

▷ **Je vérifie.** `5`, puis `5 / 3`, soit environ `1.67` (un flottant, même si les éléments sont des entiers).

**Exemple** — *Compter : on accumule un compteur*

```
def nb_positifs(t):
    c = 0                       # INITIALISER
    for x in t:                 # PARCOURIR
        if x > 0:               # TESTER : la mise a jour est conditionnelle
            c = c + 1           # METTRE A JOUR
    return c                    # RENVOYER
```

▷ **Je prédis.** Que renvoie `nb_positifs([3, 0, -2, 5])` ?

▷ **Je vérifie.** `2` : `0` n'est pas strictement positif.

**À retenir — Le bon élément neutre**

On initialise l'accumulateur avec l'**élément neutre** de l'opération : `0` pour une somme ou un compteur, `1` pour un produit, `""` pour concaténer du texte, `[]` pour construire un tableau.

► **Exercices d'application :** exercices 3 **et** 4 (comptages, moyenne pondérée)

## III Chercher un élément : la recherche séquentielle

Chercher, c'est parcourir jusqu'à trouver. Deux questions distinctes : **l'élément est-il présent ?** et **où est-il ?**

### Règle 2 — Recherche séquentielle

On parcourt le tableau ; dès qu'on trouve, on peut **s'arrêter** (`return`). Si le parcours se termine sans succès, l'élément est absent.



```python
def est_present(t, x):
    for element in t:          # PARCOURIR
        if element == x:       # TESTER
            return True        # RENVOYER : trouve, inutile de continuer
    return False               # RENVOYER : parcours fini sans succes

def indice(t, x):
    """Indice de la premiere occurrence de x, ou -1 si absent."""
    for i in range(len(t)):    # PARCOURIR
        if t[i] == x:          # TESTER
            return i           # RENVOYER
    return -1                  # RENVOYER : absent
```

▷ **Je prédis.** Que renvoient `indice([5, 8, 5], 5)` et `indice([5, 8, 5], 2)` ?
▷ **Je vérifie.** `0` (la *première* occurrence : on sort dès qu'on trouve), puis `-1` (absent).

**Remarque** — *Absent : `-1` ou `None`*
Pour signaler un élément absent, `indice` renvoie `-1`, qui ne peut pas être un indice obtenu par le parcours. Beaucoup d'énoncés, au bac notamment, préfèrent renvoyer `None` (« rien ») : les deux conventions existent, on applique **celle de l'énoncé**.

**Remarque** — *« Sortir tôt » : un réflexe utile*
Renvoyer dès qu'on a trouvé évite de parcourir inutilement la fin du tableau. C'est correct *et* souvent plus rapide — on y revient au §V (meilleur/pire cas).

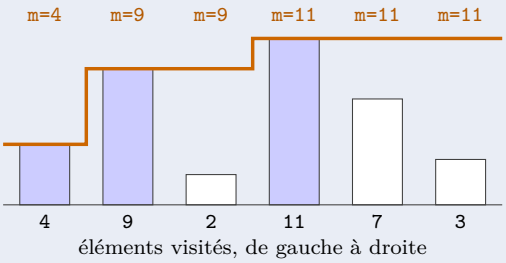
► **Exercices d'application :** exercices 5 à 7 et 26 (recherche séquentielle)

## IV Le plus grand, le plus petit : l'invariant du champion

Pas de raccourci ici : pour connaître le maximum, il faut avoir **tout** regardé. On garde un « champion provisoire », mis à jour dès qu'on rencontre mieux.

```python
def maximum(t):                # t suppose NON vide
    m = t[0]                   # INITIALISER : le premier element
    for x in t:                # PARCOURIR
        if x > m:              # TESTER
            m = x              # METTRE A JOUR : nouveau champion
    return m                   # RENVOYER
```

Déroulons `maximum([4, 9, 2, 11, 7, 3])`. Les barres sont les éléments, dans l'ordre de visite ; le trait **orange** est la valeur de `m` *après* chaque tour. Il ne fait que **monter**, et seulement quand un nouveau champion (barre colorée) apparaît.




### Règle 3 — Un invariant de boucle

À **chaque** tour, la variable `m` contient le maximum des éléments **déjà vus**. Cette propriété, vraie au départ et préservée à chaque tour, s'appelle un **invariant de boucle** : elle *garantit* qu'à la fin, `m` est bien le maximum de *tout* le tableau. C'est ainsi qu'on **prouve** qu'un algorithme est correct.

↻ **Retour sur l'activité.** La carte en main du chercheur, dans l'activité « La carte la plus haute », était ce champion : à tout moment, c'était la plus haute des cartes déjà retournées. Impossible de s'arrêter avant la fin de la pile : la dernière carte pouvait être plus haute.

**Remarque** — *Pourquoi initialiser avec `t[0]` et non `0` ?*
Initialiser à `0` donnerait un résultat faux sur un tableau de nombres tous négatifs (le maximum de `[-3, -7]` est `-3`, pas `0`). On part donc d'un **vrai** élément du tableau. Pour le minimum, on remplace `>` par `<`.

↻ **Retour sur l'activité.** Les températures $-3$, $-8$, $-1$, $-5$, $-6$ de l'activité le montraient : en partant d'une carte imaginaire de valeur 0, le camarade annonçait 0, une valeur absente du paquet, au lieu de $-1$.

**Exemple** — *Renvoyer aussi la position*
Souvent on veut l'*indice* du maximum, pas seulement sa valeur :

```
1 def indice_du_max(t):
2     i_max = 0                 # INITIALISER
3     for i in range(len(t)):   # PARCOURIR
4         if t[i] > t[i_max]:   # TESTER
5             i_max = i         # METTRE A JOUR
6     return i_max              # RENVOYER
```

▷ **Je prédis.** Que renvoie `indice_du_max([4, 9, 2, 9])` ?
▷ **Je vérifie.** `1` : le test `>` est strict, le second `9` ne détrône pas le premier.

## A. Prouver qu'une propriété est un invariant

Dire qu'une propriété est un invariant, cela ne se devine pas : cela se **démontre**, toujours de la même façon, en **deux étapes**.

### Règle 4 — Les deux étapes d'une preuve d'invariant

Pour prouver qu'une propriété $P$ est un **invariant** d'une boucle :

1. **Initialisation** : $P$ est vraie *avant* le premier tour (juste après les initialisations).
2. **Hérédité** (ou *conservation*) : *si* $P$ est vraie avant un tour quelconque, *alors* elle est encore vraie après ce tour.

Si ces deux points tiennent, $P$ est vraie à **chaque** tour. En ajoutant qu'à la fin la boucle a parcouru *tout* le tableau, on obtient la **correction** de l'algorithme.

**Exemple** — *Preuve pour le champion du maximum*
Propriété $P$ : « `m` contient le maximum de `t[0]` et des éléments **déjà visités** par la boucle ».

- **Initialisation** : avant le premier tour, la boucle n'a encore visité aucun élément et `m = t[0]` : `m` est bien le maximum de `t[0]` seul. $P$ est vraie.
- **Hérédité** : supposons $P$ vraie. À la visite de `x`, soit `x <= m` et `m` ne change pas, soit `x > m` et `m` devient `x`. Dans les deux cas, `m` est le maximum des éléments visités *avec* `x` : $P$ reste vraie.

À la fin, tous les éléments ont été visités : `m` est le maximum de *tout* le tableau. □

**Exemple** — *Un second invariant : la somme*
Pour `somme` (§II), propriété $P$ : « `s` contient la somme des éléments déjà visités ».

- **Initialisation** : avant le premier tour, aucun élément n'est visité et `s = 0` : la somme de rien vaut `0`. $P$ est vraie.
- **Hérédité** : si `s` est la somme des éléments déjà vus, alors après `s = s + x` elle devient cette somme *plus* `x`, c'est-à-dire la somme des éléments vus *avec* `x`. $P$ reste vraie.

À la fin, `s` est la somme du tableau entier. □

**Méthode — Bien choisir l'invariant : « déjà visités »**

La clé est presque toujours la même formulation : « la variable contient le résultat **pour les éléments déjà visités** ». C'est cette petite phrase qui rend l'hérédité facile à écrire. On retrouvera les invariants au chapitre *Spécifier et mettre au point ses programmes* (les vérifier par un `assert`), puis pour justifier les tris.

► **Exercices d'application :** exercices 8 à 13 (minimum, mot le plus long, position du champion, amplitude, moyenne sans la pire note, deuxième plus grand), 25 (dérouler la recherche du maximum) et 14 à 16 (invariants)

## V Combien ça coûte ?

Deux algorithmes peuvent tous deux être *corrects*, mais l'un être bien plus **rapide** que l'autre. Pour comparer sans dépendre de la machine, on **compte les opérations** en fonction de la taille `n` du tableau.

**Définition 2 — Coût, coût linéaire**

Le **coût** d'un algorithme est le nombre d'opérations élémentaires qu'il effectue, exprimé en fonction de la taille `n` des données. Un parcours simple fait un nombre d'opérations **proportionnel à `n`** : on dit que son coût est **linéaire**.

↻ **Retour sur l'activité.** Pour 10 cartes, le chercheur faisait 9 comparaisons, pour 52 cartes 51, et $n-1$ pour $n$ cartes, quel que soit l'ordre de la pile : si le paquet double, le travail double. C'est un coût linéaire.

Toutes les fonctions vues jusqu'ici (`somme`, `moyenne`, `maximum`, `est_present`…) font **un** parcours : leur coût est **linéaire**. Si l'on double `n`, le temps double.

**À retenir — Meilleur cas, pire cas : l'exemple de la recherche**

La recherche séquentielle ne coûte pas toujours pareil :

- **meilleur cas** : l'élément est en tête → `1` comparaison ;
- **pire cas** : l'élément est absent (ou en dernier) → `n` comparaisons.

Le pire cas reste **linéaire** : c'est lui qu'on retient pour caractériser un algorithme. Si le tableau est **trié**, on fera beaucoup mieux : c'est la recherche dichotomique, que l'on verra au chapitre *La recherche dichotomique*.



## A. Le piège quadratique : moyenne *vs* variance

Voici le cœur du chapitre. On note $x_0, x_1, \ldots, x_{n-1}$ les $n$ valeurs du tableau. Leur **moyenne** est

$$m = \frac{x_0 + x_1 + \cdots + x_{n-1}}{n}.$$

La **variance** [ au-delà du programme ] mesure ensuite leur dispersion autour de cette moyenne : c'est la moyenne des carrés des écarts à $m$,

$$V = \frac{(x_0 - m)^2 + (x_1 - m)^2 + \cdots + (x_{n-1} - m)^2}{n}.$$

**Version naïve** — on traduit la formule mot à mot :

```
def variance_naive(t):
    n = len(t)
    s = 0                                # INITIALISER
    for x in t:                          # PARCOURIR
        s = s + (x - moyenne(t)) ** 2    # METTRE A JOUR ; DANGER : moyenne(t) A CHAQUE tour
    return s / n                         # RENVOYER
```

Cette version est **correcte**… mais lente. À chacun des `n` tours de boucle, elle rappelle `moyenne(t)`, qui fait elle-même un parcours de `n` éléments. Total : `n` × `n` $= n^2$ opérations. On dit que son coût est **quadratique**.

**Version efficace** — on calcule la moyenne **une seule fois**, *avant* la boucle :

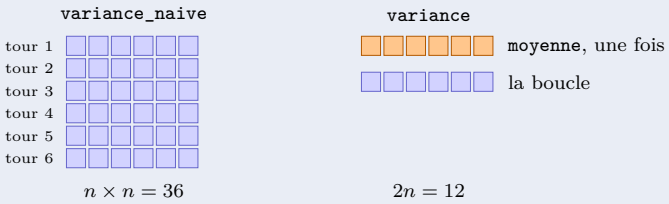
```
def variance(t):
    n = len(t)
    m = moyenne(t)                # calculee UNE fois : n operations
    s = 0                         # INITIALISER
    for x in t:                   # PARCOURIR
        s = s + (x - m) ** 2      # METTRE A JOUR : chaque tour est immediat
    return s / n                  # RENVOYER
```

▷ **Je prédis.** Que renvoie `variance([1, 3])` ? Et `variance([7, 7, 7])` ?
▷ **Je vérifie.** `1.0` : la moyenne vaut `2`, les écarts `-1` et `1` ont pour carrés `1` et `1`, dont la moyenne est `1.0`. Puis `0.0` : des valeurs toutes égales ne sont pas dispersées.

Ici, un parcours pour la moyenne, un parcours pour la somme : environ `2n` opérations, donc un coût **linéaire**. Le résultat est **identique**, mais regardons la facture :

| taille `n` | `moyenne` ($\approx n$) | `variance_naive` ($\approx n^2$) | `variance` ($\approx 2n$) |
|---|---|---|---|
| 10 | 10 | 100 | 20 |
| 100 | 100 | 10 000 | 200 |
| 1 000 | 1 000 | 1 000 000 | 2 000 |

Chaque case représente une opération, pour un tableau de $n = 6$ éléments. Dans `variance_naive`, *chaque* tour de boucle (une ligne) refait tout le parcours de `moyenne` : on remplit un **carré** de $6 \times 6 = 36$ cases. Dans `variance`, deux parcours *l'un après l'autre* : $2 \times 6 = 12$ cases seulement.




Ces ordres de grandeur se vérifient au chronomètre (TP proposé en fin de chapitre) : quand $n$ double, le temps de `variance_naive` est multiplié par 4, celui de `variance` par 2.

4

**Règle 5 — La leçon**

*Où* l'on place un calcul change tout. Un calcul qui ne dépend pas de la boucle doit être **sorti** de la boucle. Placer un parcours *à l'intérieur* d'un autre parcours fait passer le coût de **linéaire** ($n$) à **quadratique** ($n^2$) — pour $n = 1000$, on passe de quelques milliers à un **million** d'opérations.

**Pour les curieux** — *Reconnaître le quadratique* (facultatif)

Deux boucles **imbriquées** sur le même tableau (ou un appel de parcours *dans* une boucle) sont le signe d'un coût quadratique. En notation savante, on écrit un coût linéaire $O(n)$ et un coût quadratique $O(n^2)$ (notation « grand O », que vous retrouverez en terminale).

► **Exercices d'application :** exercices 17 à 21 (le coût : linéaire ou quadratique ; min et max en une passe, deux tableaux)

## VI Un peu d'histoire

**Pour les curieux** — *Du mot « algorithme » à l'analyse de leur coût* (facultatif)

Le mot **algorithme** vient du savant persan **al-Khwârizmî** (IX$^{e}$ siècle), dont le nom latinisé donna *algorismus*. Mais *mesurer* le coût d'un algorithme est bien plus récent : la notation « grand O » est due aux mathématiciens **Paul Bachmann** (1894) et **Edmund Landau**, et c'est **Donald Knuth** qui, à partir des années 1960, en fait l'outil central de l'*analyse des algorithmes* dans son œuvre monumentale *The Art of Computer Programming*. Choisir le bon algorithme, et pas seulement un algorithme correct, est depuis au cœur de l'informatique : sur des milliards de données, la différence entre linéaire et quadratique, c'est la différence entre une seconde et… des années.

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** Le patron du parcours généralise l'**accumulateur** des *Bases de la programmation Python* et s'applique aux tableaux des *Types construits*.
- **Ce qu'il prépare.** Il resservira partout : boucle interne du **tri par sélection**, point de comparaison pour la **dichotomie**, filtres sur les **données en tables**, distances des $k$ **plus proches voisins**.
- **En Terminale.** Le même patron parcourra les listes chaînées, les arbres et les graphes ; la **programmation dynamique** poussera à l'extrême la leçon de la variance : ne jamais recalculer.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Le patron | init → mise à jour à chaque élément → renvoi |
| Élément neutre | `0` (somme, compteur), `1` (produit), `""`, `[]` |
| Parcourir | par élément (`for x in t`) ou par indice (`for i in range...`) |
| Recherche séquentielle | s'arrête dès qu'on trouve ; `-1` ou `None` si absent |
| Minimum / maximum | garder un **champion** ; initialiser avec `t[0]`, jamais `0` |
| Invariant de boucle | propriété vraie à chaque tour : *prouve* que le résultat est correct |
| Coût linéaire | un parcours : $\approx n$ opérations (recherche : pire cas = absent) |
| Coût quadratique | un parcours *dans* un parcours : $\approx n^2$ (piège de la variance !) |
| Le réflexe | **sortir de la boucle** ce qui n'en dépend pas |

### Retour à la situation de départ

1. Pour 40 °C, non : la **recherche séquentielle** s'arrête dès qu'elle trouve (§ III) ; seul le pire cas, aucun jour à 40 °C, oblige à tout lire (§ V). Pour la plus basse, oui : on garde un **champion** jusqu'au bout, car la dernière valeur peut être la plus basse (§ IV).
2. Pour un parcours simple, oui : son coût est **linéaire**. Mais un parcours placé *dans* un autre (la variance naïve) a un coût **quadratique** : le temps est multiplié par 4 (§ V).
3. On prouve un **invariant de boucle** : « `m` contient le maximum des éléments déjà visités », vrai au départ (initialisation) et conservé à chaque tour (hérédité) ; à la fin, tout a été visité (§ IV).

## Erreurs fréquentes

- **Mal initialiser le maximum.** Partir de `0` échoue si toutes les valeurs sont négatives. *Le réflexe :* initialiser au **premier élément** du tableau.
- **Confondre indice et valeur.** `t[i]` est la *valeur*, `i` est la *position*.
- **Se tromper de bornes.** `range(n)` va de `0` à `n-1` ; le dernier indice est `len(t)-1`.
- **Recalculer dans la boucle ce qui est constant.** Appeler `moyenne(t)` *à l'intérieur* d'un parcours rend le coût **quadratique** (piège de la variance). *Le réflexe :* sortir de la boucle ce qui n'en dépend pas.
- **Ne pas s'arrêter dès qu'on a trouvé.** Dans une recherche, `return` dès la première occurrence.
- **Oublier le cas du tableau vide** (ou d'une valeur absente).

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais...*

☐ écrire un **parcours séquentiel** (somme, moyenne, comptage) ;
☐ écrire une **recherche séquentielle** (présence, indice) et m'arrêter dès qu'on trouve ;
☐ écrire un **min / max** avec l'**invariant du champion** et la bonne initialisation ;
☐ énoncer et **prouver un invariant** de boucle (initialisation + hérédité) ;
☐ estimer le **coût** (linéaire / quadratique) et éviter les recalculs inutiles.

# EXERCICES

## Mode d'emploi

- Étoiles, ● essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et tester* en Python. Le symbole `>>>` figure une saisie dans la console.
- **Sauf mention contraire**, on n'utilise **pas** les fonctions toutes faites `sum`, `max`, `min` : le but est d'écrire le **parcours** soi-même.
- **Réflexe coût** : pour chaque fonction, se demander *combien de fois* on parcourt le tableau.



## Le patron du parcours ↑ cours

★ **Exercice 1** *Lecture de trace* ▶ corrigé p. 582 ● essentiel

Sans machine, dire ce que renvoie `mystere([4, 1, 7, 2])`.

```
1 def mystere(t):
2     r = 0
3     for x in t:
4         if x > r:
5             r = x
6     return r
```

Que calcule cette fonction en général ? Renvoie-t-elle un résultat correct si tous les éléments sont négatifs ? Expliquer. ▷ coup de pouce

★ **Exercice 2** *Reconnaître le patron* ▶ corrigé p. 582

▶ En suivant le patron du cours (initialisation, mise à jour, renvoi), écrire :

1. `produit(t)` : le produit de tous les éléments ;
2. `concatener(mots)` : la chaîne obtenue en collant tous les mots d'un tableau de chaînes.

▷ coup de pouce

## Décliner : comptages, moyennes ↑ cours

★ **Exercice 3** *Compter dans un intervalle : code à trous* ▶ corrigé p. 582 ● essentiel

La fonction `nb_entre(t, a, b)` doit renvoyer le nombre d'éléments du tableau `t` compris entre `a` et `b` (bornes incluses). Recopier et compléter les pointillés en suivant le patron du parcours (initialisation, mise à jour, renvoi).

```
1 def nb_entre(t, a, b):
2     c = ......
3     for x in ......:
4         if ...... and ......:
5             c = ......
6     return ......
```

```
>>> nb_entre([12, 5, 18, 9, 15], 10, 15)
2
```

▷ coup de pouce

★★ **Exercice 4** *Taux de réussite et moyenne pondérée* ▶ corrigé p. 583 • essentiel

▶ Les notes d'un élève sont rangées dans un tableau `notes` **non vide** ; le tableau `coefs`, de même longueur, donne leurs coefficients (`coefs[i]` est le coefficient de `notes[i]`). Écrire :

1. `taux_reussite(notes)` : le pourcentage de notes supérieures ou égales à `10` ;
2. `moyenne_ponderee(notes, coefs)` : la somme des produits `notes[i] * coefs[i]`, divisée par la somme des coefficients.

```
>>> taux_reussite([12, 8, 15, 9])
50.0
>>> moyenne_ponderee([12, 8, 15], [2, 1, 1])
11.75
```

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Recherche séquentielle ↑cours

★ **Exercice 5** *Un multiple dans le tableau* ▶ corrigé p. 583 • essentiel

▶ Écrire `a_un_multiple(t, k)` qui renvoie `True` si au moins un élément de `t` est un multiple de `k`, `False` sinon. La fonction doit **s'arrêter** dès qu'un multiple est trouvé. Combien de tours de boucle effectue `a_un_multiple([7, 10, 3, 20], 5)` ? Et `a_un_multiple([7, 10, 3, 20], 6)` ?

▷ coup de pouce

★★ **Exercice 6** *Où est-il ?* ▶ corrigé p. 583

▶ Écrire :

1. `premiere_voyelle(mot)` : l'indice de la **première** voyelle (`a`, `e`, `i`, `o`, `u`, `y`) de la chaîne `mot`, ou `-1` s'il n'y en a pas ;
2. `dernier_indice(t, x)` : l'indice de la **dernière** occurrence de `x` dans le tableau `t`, ou `-1` si `x` est absent.

```
>>> premiere_voyelle("schtroumpf")
5
>>> dernier_indice([4, 1, 4, 2], 4)
2
```

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 7** *Recherche sous condition* ▶ corrigé p. 584

▶ Écrire `premier_negatif(t)` qui renvoie le premier élément strictement négatif rencontré, ou `None` s'il n'y en a pas.

```
>>> premier_negatif([4, 7, -2, 9, -5])
-2
```

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Minimum, maximum et champion ↑ cours

★ **Exercice 8** *Le plus petit* ▶ corrigé p. 584 • essentiel

▶ Écrire `minimum(t)` (tableau non vide), **sans `min`**. Préciser l'**invariant** de votre boucle. Pourquoi serait-il faux d'initialiser le champion à `0` ? (Penser au tableau `[3, 8, 5]`.) ▷ coup de pouce

★ **Exercice 9** *Le mot le plus long : lignes à remettre dans l'ordre* ▶ corrigé p. 585

Les lignes de la fonction `mot_le_plus_long(mots)` (tableau de chaînes **non vide**) ont été mélangées et leur indentation a été perdue. Les recopier **dans le bon ordre** et **correctement indentées**, puis ▶ tester sur `["chat", "girafe", "ours", "souris"]`. Pourquoi obtient-on `"girafe"` et non `"souris"`, qui a pourtant la même longueur ?

```
return champion
champion = mot
def mot_le_plus_long(mots):
if len(mot) > len(champion):
champion = mots[0]
for mot in mots:
```

▷ coup de pouce

★★ **Exercice 10** *La position du champion* ▶ corrigé p. 585

▶ Écrire `indice_du_min(t)` qui renvoie l'indice du plus petit élément d'un tableau **non vide** (le premier en cas d'égalité). ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 11** *Amplitude* ▶ corrigé p. 585

▶ Écrire `amplitude(t)` qui renvoie la différence entre le plus grand et le plus petit élément d'un tableau non vide. Combien de parcours votre solution effectue-t-elle ? ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 12** *Moyenne de fin de trimestre* ▶ corrigé p. 585

▶ Les notes d'un élève sont dans un tableau. Écrire `moyenne_sans_pire(t)` qui calcule la moyenne **après avoir retiré la plus mauvaise note**. On supposera au moins deux notes.

```
>>> moyenne_sans_pire([8, 12, 15, 5])
11.666666666666666
```

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 13** *Le deuxième plus grand* ▶ corrigé p. 586 ≈ 20 min

▶ Écrire `deuxieme_max(t)` qui renvoie le deuxième plus grand élément *distinct* d'un tableau (on suppose au moins deux valeurs distinctes), en **un seul** parcours.

```
>>> deuxieme_max([3, 9, 9, 7, 2])
7
```

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Invariants de boucle ↑cours

★ **Exercice 14** *Que contient la variable ?* ▶ corrigé p. 586 • essentiel

On considère la fonction suivante :

```
1 def f(t):
2     c = 0
3     for x in t:
4         if x % 2 == 0:
5             c = c + 1
6     return c
```

1. Énoncer l'**invariant** de la boucle : que contient `c` à chaque tour ?
2. En déduire ce que calcule la fonction.

▷ coup de pouce

★★ **Exercice 15** *Énoncer un invariant* ▶ corrigé p. 586

On reprend la fonction `produit(t)` (initialisation `p = 1`, puis `p = p * x` à chaque tour).

1. Énoncer l'**invariant** de la boucle (ce que contient `p` à chaque tour).
2. Expliquer en une phrase pourquoi cet invariant **garantit** que le résultat final est correct.

▷ coup de pouce

★★★ **Exercice 16** *Une preuve d'invariant en deux étapes* ▶ corrigé p. 586 ≈ 15 min

On considère `nb_positifs(t)`, qui compte les éléments strictement positifs (initialisation `c = 0` ; à chaque tour, `c = c + 1` si `x > 0`). Soit la propriété

$P$ : « `c` contient le nombre d'éléments strictement positifs **déjà visités** ».

En suivant la méthode du cours, **démontrer** que $P$ est un invariant :

1. **Initialisation** : montrer que $P$ est vraie avant le premier tour.
2. **Hérédité** : en supposant $P$ vraie avant un tour, montrer qu'elle l'est encore après (distinguer les cas `x > 0` et `x <= 0`).
3. **Conclusion** : expliquer pourquoi la fonction renvoie bien le nombre d'éléments **strictement** positifs du tableau entier.

▷ coup de pouce ▷ début de solution

## Le coût : linéaire ou quadratique ? ↑cours

★ **Exercice 17** *Compter les tours* ▶ corrigé p. 587 • essentiel

Pour chaque fonction, dire combien de tours de boucle sont effectués pour un tableau de `n` éléments, et si le coût est **linéaire** ou **quadratique**.

```
1 # (a)                          # (b)
2 def f(t):                      def g(t):
3     s = 0                          c = 0
4     for x in t:                    for i in range(len(t)):
5         s = s + x                      for j in range(len(t)):
6     return s                               if t[i] == t[j]:
7                                                c = c + 1
8                                    return c
```

▷ coup de pouce

★★ **Exercice 18** *Min et max en une seule passe* ▶ corrigé p. 587

▶ Écrire `mini_maxi(t)` qui renvoie le p-uplet `(min, max)` en **un seul** parcours du tableau. Comparer au coût de deux parcours séparés. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 19** *Détecter un doublon* ▶ corrigé p. 587

▶ Écrire `a_un_doublon(t)` qui renvoie `True` si (au moins) deux éléments de `t` sont égaux. Quel est le coût de votre solution (en nombre de comparaisons dans le pire cas) ? [ au-delà du programme ]

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 20** *Deux tableaux* ▶ corrigé p. 588 ≈ 15 min

▶ Écrire `element_commun(t1, t2)` qui renvoie `True` si les deux tableaux ont (au moins) un élément en commun. Combien de fois, dans le pire cas, compare-t-on deux éléments si `t1` et `t2` ont chacun `n` éléments ? Quel est le coût ? [ au-delà du programme ]

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 21** *Moyenne, variance — et le piège du coût* ▶ corrigé p. 588 ≈ 30 min

C'est **l'exercice central** du chapitre. La variance [ au-delà du programme ] mesure la dispersion : $V = \dfrac{(x_0 - m)^2 + (x_1 - m)^2 + \cdots + (x_{n-1} - m)^2}{n}$, où $m$ est la moyenne.

1. ▶ Écrire `moyenne(t)` (un parcours).
2. ▶ Écrire `variance_naive(t)` en **appelant `moyenne(t)` à l'intérieur** de la boucle. La fonction est-elle correcte ?
3. **Combien de fois** au total `variance_naive` additionne-t-elle des nombres pour un tableau de `n` éléments ? En déduire son coût.
4. ▶ Écrire `variance(t)` qui calcule la moyenne **une seule fois**, avant la boucle. Quel est son coût ?
5. Vérifier que les deux versions donnent le **même** résultat sur `[2, 4, 4, 4, 5, 5, 7, 9]` (la variance vaut `4.0`). Laquelle utiliseriez-vous sur un million de valeurs, et pourquoi ?

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Synthèse : petits problèmes ↑ cours

★★ **Exercice 22** *Relevé météo* ▶ corrigé p. 589

▶ Un tableau `temperatures` donne la température de chaque jour du mois.

1. `jour_le_plus_chaud(temperatures)` : l'**indice** (le numéro du jour, à partir de 0) du maximum (le premier en cas d'égalité).
2. `nb_jours_gel(temperatures)` : le nombre de jours où la température a été strictement négative.
3. `plus_longue_serie_chaude(temperatures, seuil)` : la longueur de la plus longue série *consécutive* de jours dont la température est **strictement supérieure** à `seuil`. [ au-delà du programme ]

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 23** *Vers le bac — contrôle de saisie* ▶ corrigé p. 589

▶ Un tableau `notes` est censé ne contenir que des entiers entre 0 et 20.

1. `toutes_valides(notes)` : `True` si toutes les notes sont dans `[0 ; 20]`, `False` dès qu'une note sort de l'intervalle (s'arrêter tôt).
2. `indices_invalides(notes)` : le tableau des indices des notes hors intervalle.

▷ coup de pouce

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

### ★★ Exercice 24 — *Une recherche selon un assistant d'IA* ▶ corrigé p. 590

Un élève demande à un assistant d'IA : « Écris une fonction Python qui dit si une valeur `x` est présente dans un tableau `t`. » Voici la réponse obtenue.

```
def est_present(t, x):
    """Renvoie True si x est dans t, False sinon."""
    for element in t:
        if element == x:
            return True
        else:
            return False
```

« La boucle parcourt le tableau : dès qu'un élément est égal à `x`, la fonction renvoie `True`, sinon elle renvoie `False`. Le coût est linéaire dans le pire des cas. »

1. La réponse est-elle correcte ? Faire la trace de `est_present([3, 8, 5], 5)`.
2. Localiser et corriger l'erreur.
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce

## Entraînement ↑ cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

### ★ Exercice 25 — *Dérouler la recherche du maximum* ▶ corrigé p. 590 • essentiel

On considère la fonction du cours :

```
def maximum(t):
    m = t[0]
    for x in t:
        if x > m:
            m = x
    return m
```

Sans machine, donner la valeur de `m` après chaque tour de boucle, puis le résultat, pour : a) `[5, 2, 8, 1, 8]` ; b) `[-4, -9, -2, -7]` ; c) `[7]`.

▷ coup de pouce

### ★ Exercice 26 — *Recherche séquentielle : combien de tours ?* ▶ corrigé p. 590 • essentiel

Avec la fonction `est_present(t, x)` du cours (on renvoie `True` dès qu'on trouve, `False` après la boucle), donner sans machine le résultat **et** le nombre de tours de boucle effectués pour chaque appel : a) `est_present([4, 7, 1, 9], 1)` ; b) `est_present([4, 7, 1, 9], 5)` ; c) `est_present([], 3)` ; d) `est_present([2, 2, 2], 2)`. Lequel de ces appels illustre le pire cas ?

▷ coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

**Ex. 1 – piste** Faire un tableau de trace : une ligne par tour, avec `x` et `r`. Refaire ensuite la trace avec `[-3, -7]` : la condition `x > r` est-elle vraie une seule fois ? ↑ énoncé

**Ex. 2 – piste** Seules l'initialisation et la mise à jour changent. Par quelle valeur commencer pour qu'elle ne modifie pas le résultat : pour un produit ? pour une chaîne que l'on allonge ? ↑

**Ex. 3 – piste** C'est un compteur : quelle valeur avant la boucle, et de combien augmente-t-il ? « Compris entre `a` et `b` » se traduit par deux comparaisons qui doivent être vraies *toutes les deux*. ↑ énoncé

**Ex. 4 – piste** Question 1 : un compteur, puis une division par le nombre de notes. Question 2 : deux accumulateurs mis à jour dans la *même* boucle ; il faut lire à la fois `notes[i]` et `coefs[i]`, donc parcourir par indice. ↑énoncé

**Ex. 4 – début de solution**
```
def moyenne_ponderee(notes, coefs):
    total = 0
    total_coefs = 0
    for i in range(len(notes)):
```
↑énoncé

**Ex. 5 – piste** `x` est un multiple de `k` quand `x % k == 0`. Où placer `return False` : dans la boucle ou après ? ↑énoncé

**Ex. 6 – piste** Première voyelle : parcourir par indice, tester `mot[i] in "aeiouy"` et s'arrêter dès qu'on trouve. Dernière occurrence : soit parcourir tout le tableau en mémorisant le dernier indice rencontré, soit parcourir les indices à l'envers. ↑énoncé

**Ex. 7 – piste** Même structure que `a_un_multiple` : on renvoie l'élément dès que la condition est vraie. Que renvoyer si la boucle se termine sans succès ? ↑énoncé

**Ex. 8 – piste** Reprendre le patron du champion du cours en changeant le sens de la comparaison. Pour l'invariant, compléter : « à chaque tour, `m` est … des éléments déjà visités ». ↑énoncé

**Ex. 9 – piste** La ligne `def` vient en premier. Qu'est-ce qui doit être fait une seule fois avant la boucle, à chaque tour (et seulement sous condition), une seule fois après ? ↑énoncé

**Ex. 10 – piste** Retenir l'*indice* du champion, pas sa valeur : parcourir par indice et comparer `t[i]` à la valeur du champion. Faut-il `<` ou `<=` pour garder le premier en cas d'égalité ? ↑énoncé

**Ex. 11 – piste** Il faut deux « champions » : le plus grand et le plus petit. Peut-on les mettre à jour dans la même boucle ? ↑énoncé

**Ex. 12 – piste** Faut-il vraiment retirer la note du tableau ? Une fois la pire note enlevée, que deviennent la somme des notes et leur nombre ? ↑énoncé

**Ex. 13 – piste** Garder deux champions : `premier` (le plus grand vu) et `second`. Quand un élément bat `premier`, que devient l'ancien `premier` ? Et que faire d'un élément égal à `premier` ? ↑

**Ex. 13 – début de solution**
```
premier = None
second = None
for x in t:
```
puis distinguer deux cas : `x` bat `premier` ; `x` se place entre `second` et `premier`. ↑énoncé

**Ex. 14 – piste** Faire la trace sur un petit tableau, par exemple `[3, 4, 7, 8]` : après chaque tour, que compte `c` parmi les éléments déjà visités ? ↑énoncé

**Ex. 15 – piste** S'inspirer de l'invariant du champion : « `p` contient … des éléments **déjà visités** ». Que sont les éléments déjà visités à la sortie de la boucle ? ↑énoncé

**Ex. 16 – piste** Initialisation : avant le premier tour, combien d'éléments ont été visités, et combien de positifs parmi eux ? Hérédité : partir de « `c` = nombre de positifs déjà visités » et regarder l'effet du nouvel élément `x`. ↑énoncé

**Ex. 16 – début de solution** Hérédité, cas `x > 0` : le nombre de positifs déjà visités augmente de `1`, et l'instruction `c = c + 1` fait de même. Rédiger ensuite le cas `x <= 0`, puis la conclusion. ↑énoncé

**Ex. 17 – piste** Combien de tours fait une boucle `for` sur un tableau de `n` éléments ? Dans (b), la boucle intérieure est exécutée en entier à *chaque* tour de la boucle extérieure. ↑énoncé

**Ex. 18 – piste** Deux champions initialisés avec `t[0]`, et deux tests dans le même tour de boucle. Combien de tours font deux parcours séparés ? ↑énoncé

**Ex. 19 – piste** Comparer chaque élément à ceux qui sont *après* lui : deux boucles imbriquées sur les indices. Attention à ne jamais comparer un élément à lui-même. ↑énoncé

**Ex. 20 – piste** Pour chaque élément de `t1`, chercher s'il est dans `t2` : une recherche séquentielle à l'intérieur d'une autre boucle. Le pire cas : aucun élément commun. ↑énoncé

Ex. 20 – début de solution
```
for x in t1:
    for y in t2:
        if x == y:
```
↑énoncé

Ex. 21 – piste Question 3 : `moyenne(t)` fait `n` additions, et `variance_naive` l'appelle à chaque tour de sa boucle. Combien de tours fait cette boucle ? ↑énoncé

Ex. 21 – début de solution
```
def variance(t):
    m = moyenne(t)
    s = 0
    for x in t:
```
↑énoncé

Ex. 22 – piste 1 : c'est la position du champion. 2 : un compteur. 3 : deux variables, la longueur de la série *en cours* et la meilleure longueur vue ; que devient la série en cours un jour plus froid ? ↑énoncé

Ex. 22 – début de solution
```
courante = 0
record = 0
for temp in temperatures:
    if temp > seuil:
```
↑énoncé

Ex. 23 – piste 1 : renvoyer `False` dès qu'une note sort de l'intervalle, et `True` seulement après la boucle. 2 : partir d'un tableau vide et y ajouter des *indices*, donc parcourir par indice. ↑

Ex. 24 – piste Faire la trace pas à pas : au premier tour, `element` vaut `3`. Lequel des deux `return` est exécuté ? La boucle continue-t-elle ? ↑énoncé

Ex. 25 – piste Un tableau de trace à deux colonnes, `x` et `m`, une ligne par tour. Avant le premier tour, `m` vaut déjà `t[0]`. `m` ne change que si `x` est *strictement* plus grand. ↑énoncé

Ex. 26 – piste Compter les éléments examinés : la boucle s'arrête au premier élément égal à `x`. Si `x` est absent, combien d'éléments faut-il examiner avant de répondre ? ↑énoncé

# TP ET PROJETS

## TP Un an de météo à la loupe

▶ corrigé p. 590

*parcours, invariant… et le coût mesuré au chronomètre*



**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p04-tp-meteo-annee` — **archive .zip**

**Durée** 2 h à 2 h 30 (une à deux séances), en binôme, sur machine.

**Prérequis** tout le chapitre *Algorithmique : le parcours séquentiel* (patron, recherche séquentielle, champion, invariant, coût).

**Fichiers** `tp_parcours_meteo.csv` (365 jours de relevés, à télécharger) et `tp_parcours_depart.py` (à compléter : tout votre code s'y écrit). Les deux fichiers doivent être dans le **même dossier**.

### But du TP

Le fichier `tp_parcours_meteo.csv` contient un an de relevés quotidiens d'une station météo du bord de mer (données **simulées** à partir de moyennes climatiques réalistes) : date, température minimale, température maximale, hauteur de pluie. Vous allez :

1. écrire, **uniquement avec des parcours**, le programme qui imprime le **bulletin climatique** de l'année ;
2. vérifier **au chronomètre** ce que le cours affirme : une variance « naïve » est **quadratique**, la bonne est **linéaire** — et tracer les deux courbes ;
3. (défi) lisser la courbe des températures par une **moyenne glissante**, sans tomber dans le même piège.

**Produit final** : un programme qui affiche le bulletin, un graphique « linéaire contre quadratique » et un court bilan rédigé.

### Consignes

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme. ▶ *À programmer* dans `tp_parcours_depart.py`.
- **Règle du TP** : pas de `sum`, `max` ni `min` ; chaque résultat s'obtient par un **parcours** écrit par vous.
- En bas du fichier, le programme principal contient des lignes en commentaire (`#`) : on les **décommente au fur et à mesure**. Des tests `assert` sont fournis ; s'ils passent, un message s'affiche. Dans le fichier, les parties 2, 3 et 4 du TP s'appellent « Partie B », « Partie C » et « Partie D ».
- Le compte rendu (réponses, tableaux complétés, bilan) se fait sur le cahier (ou dans un fichier texte).
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ;

l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Prise en main

★ **Exercice 1** *Ouvrir les données*

Ouvrir `tp_parcours_meteo.csv` avec un éditeur de texte (pas un tableur). Ses premières lignes sont :

```
date,tmin,tmax,pluie
2025-01-01,7.4,10.6,40.1
2025-01-02,8.2,12.8,0.0
```

La fonction `charger_releves`, fournie, lit ce fichier et renvoie **quatre tableaux de même longueur** : `dates`, `tmin`, `tmax`, `pluie`. Le jour numéro `i` est décrit par `dates[i]`, `tmin[i]`, `tmax[i]` et `pluie[i]`. Le fichier fournit aussi `mois_de(date)`, qui extrait le numéro du mois d'une date.

1. ▶ Exécuter le fichier de départ. Combien de jours sont chargés ?
2. Dans la console, afficher `dates[31]`, `tmax[31]` et `pluie[31]`. Quel jour est-ce ?
3. Quel est l'indice du 31 décembre ? Que renvoie `mois_de("2025-07-14")` ?
4. Pourquoi doit-on parcourir **par indice** (et non par élément) pour relier une température à sa date ?

## 2 Le bulletin climatique

Chaque fonction de cette partie est **un seul parcours**. Avant d'écrire, se demander : *quelle initialisation ? quelle mise à jour ? que renvoie-t-on ?*

★ **Exercice 2** *Accumuler et compter*

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Écrire `somme(t)` puis `moyenne(t)` (cette dernière en une ligne, en appelant `somme`).
2. ▶ Écrire `nb_au_moins(t, seuil)` qui compte les éléments de `t` **supérieurs ou égaux** à `seuil`.

```
>>> nb_au_moins([3, 8, 5, 8], 5)
3
```

3. Avec ces fonctions, calculer : la moyenne des maximales, le cumul de pluie de l'année, le nombre de **jours de pluie** (au moins 1 mm, convention des météorologues), de **jours de chaleur** (maximale $\geq 30\,°C$) et de **nuits tropicales** (minimale $\geq 20\,°C$).

★★ **Exercice 3** *Des champions datés*

On ne veut plus seulement une valeur record, mais **la date** du record : il faut donc garder l'**indice** du champion.

1. ▶ Écrire `indice_min(t)`, l'indice du plus petit élément (en cas d'égalité, le premier). En déduire la date et la température de la **nuit la plus froide**.
2. ▶ L'**amplitude** d'un jour est `tmax[i] - tmin[i]`. Écrire `indice_ecart_max(tmin, tmax)` qui renvoie l'indice du jour de plus grande amplitude. *Le champion n'est plus une case d'un tableau, mais une quantité calculée à partir de deux tableaux.*

```
>>> indice_ecart_max([10, 12, 9], [15, 20, 13])
1
```

3. **Sans machine.** Énoncer l'**invariant de boucle** de `indice_ecart_max` (« à chaque tour, l'indice retenu, noté ici `i_best`, est… »), puis le **prouver** en deux étapes (initialisation, hérédité). ▷ coup de pouce

## ★★ Exercice 4 *Chercher la première fois*



1. ▶ Écrire `premier_jour_au_moins(t, seuil)` qui renvoie l'indice du **premier** jour où `t[i] >= seuil`, ou `-1` s'il n'y en a pas. La fonction doit **s'arrêter dès qu'elle a trouvé**.
2. Quelle est la date du premier jour à 30 °C ? Y a-t-il eu un jour à 35 °C ?
3. Combien de comparaisons fait votre fonction pour le seuil 30 ? pour le seuil 35 ? Lequel de ces deux appels réalise le **pire cas** ?

## ★★ Exercice 5 *Mois par mois*

1. ▶ Écrire `moyenne_mois(dates, t, mois)` qui calcule la moyenne des `t[i]` pour les seuls jours du mois demandé (de 1 à 12). *Deux accumulateurs : une somme et un compteur ; la mise à jour est conditionnelle.*

```
>>> moyenne_mois(["2025-01-31", "2025-02-01", "2025-02-02"], [5, 8, 10], 2)
9.0
```

2. ▶ Construire le tableau `moyennes` des 12 moyennes mensuelles des maximales, puis trouver, par un champion, le **mois le plus chaud**.
3. Pour obtenir les 12 moyennes, combien de fois au total parcourt-on le tableau des 365 jours ? Proposer (sans forcément la coder) une méthode en **un seul** parcours. [ au-delà du programme ] ▷ coup de pouce

## ★ Exercice 6 *Imprimer le bulletin*

▶ Compléter la fonction `bulletin`, puis décommenter `tester_partie_B()` et `bulletin(...)`. Recopier sur le cahier le tableau ci-dessous et le compléter avec vos résultats :

| Indicateur | Valeur (avec la date si besoin) |
|---|---|
| Moyenne des maximales | |
| Cumul de pluie / jours de pluie | |
| Jours de chaleur / nuits tropicales | |
| Nuit la plus froide | |
| Plus grande amplitude | |
| Premier jour à 30 °C | |
| Mois le plus chaud | |

## 3 Le piège quadratique, chronomètre en main

Le cours (§ V) affirme que `variance_naive`, qui rappelle `moyenne(t)` *dans* sa boucle, coûte de l'ordre de $n^2$ opérations, et que `variance`, qui calcule la moyenne une seule fois, en coûte de l'ordre de $2n$. Les deux fonctions sont **fournies** dans le fichier : on va **mesurer**.

★ **Exercice 7** *Même résultat ?*

▶ Calculer `variance_naive(tmax)` et `variance(tmax)`. Les résultats sont-ils égaux ?

L'**écart-type** est la racine carrée de la variance (`v ** 0.5`) : il donne l'écart « typique » à la moyenne, en degrés. Quelle valeur obtient-on pour `tmax` ?

★★ **Exercice 8** *Construire un chronomètre*

Le module `time` fournit `time.perf_counter()`, qui renvoie l'heure d'une horloge très précise, en secondes. La **différence** entre deux lectures donne une durée.

1. ▶ Écrire `chronometre(f, t)` qui renvoie la durée, en secondes, de l'appel `f(t)`. *En Python, une fonction peut être passée en paramètre :* `chronometre(variance, tmax)` *chronomètre* `variance(tmax)`. ▷ coup de pouce
2. ▶ Compléter `mesures(tailles)` : pour chaque taille `n`, elle fabrique un tableau aléatoire de `n` valeurs (fonction `tableau_aleatoire`, fournie), chronomètre les deux variances sur **ce même tableau** et range les durées dans `temps_naif` et `temps_lin`.
3. ▶ Lancer les mesures pour `tailles = [250, 500, 1000, 2000, 4000]` puis recopier sur le cahier et compléter le tableau suivant (arrondir à 3 chiffres significatifs) :

| `n` | `variance_naive` (s) | rapport au `n` précédent | `variance` (s) | rapport au `n` précédent |
|---|---|---|---|---|
| 250 | | — | | — |
| 500 | | | | |
| 1 000 | | | | |
| 2 000 | | | | |
| 4 000 | | | | |

★★ **Exercice 9** *Lire les mesures*

1. Quand `n` **double**, par combien environ est multiplié le temps de `variance_naive` ? celui de `variance` ? Relier chaque rapport au coût annoncé par le cours.
2. ▶ Décommenter `tracer(...)`. Décrire l'allure des deux courbes. Pourquoi celle de `variance` paraît-elle « collée » à l'axe ?
3. **Prédire sans exécuter.** En partant de la mesure pour $n = 4\,000$, estimer la durée de `variance_naive` pour $n = 64\,000$ (quatre doublements), puis pour $n = 1\,000\,000$ (ordre de grandeur). Faire de même pour `variance`. *Ne pas lancer* `variance_naive` *sur un million de valeurs…*
4. ▶ Vérifier la prédiction pour `variance` sur 1 000 000 valeurs. Quel est l'écart avec l'estimation ?

> **Remarque** — *Pourquoi des temps qui « tremblent » ?*
> Deux mesures identiques ne donnent jamais exactement la même durée : l'ordinateur fait autre chose en même temps. C'est l'**ordre de grandeur** et le **rapport** quand `n` double qui comptent, pas la milliseconde. On peut lisser en répétant chaque mesure plusieurs fois.

## 4 Défi : la moyenne glissante

La courbe des maximales est « en dents de scie ». Pour voir la tendance, les météorologues utilisent la **moyenne glissante sur $k$ jours** : pour chaque jour `i` (à partir du $k$-ième), la moyenne des $k$ jours qui se terminent au jour `i`, c'est-à-dire de `t[i-k+1]`, …, `t[i]`.

```
>>> glissante([1, 2, 3, 4, 5], 3)     # (1+2+3)/3, (2+3+4)/3, (3+4+5)/3
[2.0, 3.0, 4.0]
```

### ★★ Exercice 10 *Version directe*

▶ Écrire `glissante_naive(t, k)` qui, pour chaque jour `i` de `k-1` à `len(t)-1`, recalcule la somme des $k$ valeurs de la fenêtre avec une boucle. Combien d'additions fait-elle pour un tableau de $n$ valeurs ? Que devient ce coût si l'on choisit $k = n/2$ ?

### ★★★ Exercice 11 *Faire glisser la fenêtre*

Passer de la fenêtre du jour `i-1` à celle du jour `i`, c'est **ajouter** `t[i]` et **retirer** `t[i-k]` : inutile de tout recompter.

1. ▶ Écrire `glissante(t, k)` qui calcule la première somme une fois, puis la met à jour en une opération par jour. ▷ coup de pouce Vérifier qu'elle renvoie le même tableau que `glissante_naive` sur `tmax` avec $k = 7$ (comparer les valeurs à $10^{-9}$ près : les calculs à virgule ne tombent pas toujours juste).
2. Quel est son coût ? En quoi est-ce **la même leçon** que pour la variance ?
3. ▶ Pour tracer sur un même graphique `tmax` et sa moyenne glissante, recopier la fonction suivante (elle utilise, comme `tracer`, le module de tracé `matplotlib`). La moyenne glissante commence au jour `k - 1` : c'est pourquoi elle est tracée contre `range(k - 1, len(t))`.

```
def tracer_glissante(t, k):
    """Trace t et sa moyenne glissante sur k jours."""
    import matplotlib.pyplot as plt
    plt.plot(range(len(t)), t, label="maximales")
    plt.plot(range(k - 1, len(t)), glissante(t, k), label=f"moyenne sur {k} jours")
    plt.legend()
    plt.show()
```

Appeler `tracer_glissante(tmax, 7)`, puis `tracer_glissante(tmax, 30)`. Que montre la courbe lissée sur 30 jours ?

## Bilan à rédiger

Sur le cahier, en cinq à huit lignes, et en vous appuyant sur **vos** mesures :

- quel patron unique se cache derrière toutes les fonctions du bulletin, et quel invariant garantit les résultats « champions » ;
- ce que signifient concrètement « coût linéaire » et « coût quadratique » quand on double la taille des données ;
- la règle à retenir pour écrire un parcours efficace.

**Coups de pouce** *après avoir vraiment cherché : la* ***piste*** *d'abord, le* ***début de solution*** *si besoin.*

Ex. 3 q. 3 – piste Reprendre la preuve du champion du maximum dans le cours : remplacer « valeur » par « amplitude du jour » et « `m` » par « le jour `i_best` ». ↑énoncé

Ex. 5 q. 3 – piste Douze sommes et douze compteurs, rangés dans deux tableaux indexés par le numéro du mois. ↑énoncé

Ex. 8 q. 1 – piste Lire l'horloge dans une variable `debut`, appeler `f(t)`, relire l'horloge, renvoyer la différence. ↑énoncé

Ex. 11 q. 1 – piste `s = s + t[i] - t[i - k]`, pour `i` allant de `k` à `len(t) - 1` ; la première moyenne est celle de `t[0]` à `t[k-1]`. ↑énoncé

# FICHE DE RÉVISION



## L'essentiel — le patron de parcours

On **initialise** un accumulateur, on **parcourt** le tableau, on **met à jour** à chaque élément, on **renvoie**. C'est le même squelette pour la somme, la moyenne, le comptage, la recherche, le min / max.

```
def somme(t):
    s = 0                   # initialisation
    for x in t:             # parcours
        s = s + x           # mise a jour
    return s
```

## Min / max : l'invariant du champion

```
def maximum(t):
    m = t[0]                # champion = 1er element (JAMAIS 0 !)
    for x in t:
        if x > m:
            m = x           # invariant : m = max des elements deja vus
    return m
```

**Recherche** (`est_present`, `indice`) : parcourir et **`return` dès** qu'on trouve ; sinon, à la fin, `False`, ou `-1` / `None` pour un indice (suivez l'énoncé).

## Prouver un invariant (2 étapes)

**Initialisation** : la propriété est vraie *avant* la boucle. **Hérédité** : si elle est vraie au début d'un tour, elle l'est encore à la fin. ⇒ vraie à la sortie.

## Le coût — et le piège de la variance

Un parcours simple est **linéaire** (coût proportionnel à `n`). Un parcours *dans* un parcours est **quadratique** (`n` × `n`). Calculer **`moyenne(t)` à l'intérieur** d'une boucle la recalcule `n` fois (`variance_naive`). *Le réflexe :* calculer la moyenne **une seule fois**, avant la boucle.

## Erreurs à éviter

- initialiser le max à `0` (échoue sur des négatifs) ;
- confondre indice `i` et valeur `t[i]` ;
- recalculer dans la boucle ce qui est constant ;
- ne pas s'arrêter dès qu'on a trouvé.

## Savoir-faire à maîtriser — je sais…

☐ écrire un **parcours séquentiel** (somme, moyenne, comptage) ;

☐ écrire une **recherche séquentielle** (présence, indice) et m'arrêter dès qu'on trouve ;

☐ écrire un **min / max** avec l'**invariant du champion** et la bonne initialisation ;

☐ énoncer et **prouver un invariant** de boucle (initialisation + hérédité) ;

☐ estimer le **coût** (linéaire / quadratique) et éviter les recalculs inutiles.

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 595

1. Qu'est-ce qu'un invariant de boucle ?
2. Que renvoie `nb_positifs([3, -1, 4, 0])` (compteur : `c = c + 1` quand `x > 0`) ?
3. Quel est le coût d'une recherche séquentielle dans le pire cas, sur `n` éléments ?
4. Que renvoie la fonction `maximum` si l'on initialise `m = 0` et qu'on l'appelle sur `[-4, -9, -2]` ?
5. Variance : où calculer la moyenne ?

**Et avant ?** — *questions sur les chapitres précédents.* ▶ réponses p. 595

1. Pour `t = [5, 8, 2]`, que valent `t[-1]` et `t[1:]` ? (ch. 3)
2. Combien de valeurs différentes peut-on coder sur 8 bits ? (ch. 2)
3. Que vaut `total` après `total = 0` puis une boucle `for i in range(4)` de corps `total = total + i` ? (ch. 1)



**S'entraîner sur plusieurs chapitres**

- Problème de synthèse n° 4 — Le bulletin de notes, p. 511
- Problème de synthèse n° 5 — Une image en noir et blanc, p. 512

# DÉFI ADVENT OF CODE

## DÉFI Sonar Sweep

*le sondeur du sous-marin — comparer à l'élément précédent*

Jour 1



**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p04-defi-aoc-2021-01` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2021/day/1`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (facultatif), le lien *get your puzzle input* donne **vos** données, à enregistrer sous `input.txt` à côté du programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** On dispose d'une longue liste de profondeurs mesurées les unes après les autres (plus de deux mille dans les vraies données). On veut savoir combien de fois la profondeur augmente d'une mesure à la suivante. À la main, sur quelques nombres, c'est immédiat ; sur deux mille, il faut un programme.
**Ce qu'on réinvestit.** Le parcours d'un tableau **par indices**, vu dans ce chapitre : avec l'indice `i`, on accède à la fois à l'élément `mesures[i]` et à son voisin `mesures[i - 1]`, ce qu'une boucle `for x in mesures` ne permet pas directement. On utilise aussi un compteur.
**La démarche de la fiche.**

1. Comprendre la règle avec des questions courtes, puis l'appliquer à la main sur un petit exemple dont on connaît la réponse.
2. Lire le fichier de données et le ranger dans une liste d'entiers.
3. Écrire une fonction qui compte les augmentations : elle donne la réponse de la partie 1.
4. Réutiliser cette même fonction, sans la modifier, pour la partie 2.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `depth` | profondeur | `sea floor` | fond de la mer |
| `measurement` | mesure | `to increase` | augmenter |
| `to decrease` | diminuer | `previous` | précédent |
| `report` | relevé, rapport | `N/A` | sans objet |

### Résumé de l'énoncé (partie 1)

**Contexte.** Les clés du traîneau sont tombées à la mer, et vous voilà dans un sous-marin pour les récupérer. Pendant que le sous-marin avance, son sonar mesure la profondeur du fond marin, point après point. Avant de plonger plus loin, on veut savoir à quel point le fond descend : est-ce qu'il s'enfonce souvent, ou est-ce qu'il reste plat ?
**Les données.** Le fichier est le relevé du sonar. Chaque ligne contient **un seul entier** : une profondeur, en mètres. Les lignes sont dans l'ordre où les mesures ont été faites : la ligne 2 a été mesurée juste après la ligne 1, et ainsi de suite. Plus le nombre est grand, plus le fond est profond.
**La règle.**

- On compare chaque mesure à la mesure **juste avant elle** (et seulement à celle-là).
- Si elle est **strictement** plus grande, le fond s'est enfoncé : on compte une augmentation.
- Si elle est plus petite ou égale, on ne compte rien.
- La première mesure n'a pas de mesure avant elle : elle ne peut pas compter.

**La réponse attendue.** Le nombre total d'augmentations dans tout le fichier. Par exemple, pour les quatre mesures 10, 12, 12, 15, il y a deux augmentations (de 10 à 12, puis de 12 à 15) : la réponse serait 2.

### Exercice 1 *Vérifier sa compréhension*

**But.** S'assurer qu'on a bien compris la règle, en particulier ses cas limites, avant d'écrire la moindre ligne de code.
**À quoi ça sert ?** Chacune de ces questions correspond à une erreur fréquente dans le programme : une boucle qui fait une comparaison de trop ou de moins, une égalité comptée par erreur.

À traiter sur le cahier :

1. Pour un fichier de 5 mesures, combien de comparaisons faut-il faire ?
2. Que vaut la réponse si toutes les mesures sont égales ? Si elles sont rangées par ordre décroissant ?
3. Une mesure plus grande que la *toute première* mesure du fichier, mais plus petite que celle juste avant elle, est-elle comptée ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Appliquer la règle à la main sur dix mesures.
**À quoi ça sert ?** On obtient un petit exemple dont on **connaît la réponse**. Plus loin, on testera le programme sur cet exemple avant de le lancer sur les vraies données : si le programme ne trouve pas 5 ici, inutile d'aller plus loin.

On considère le relevé (inventé) suivant, écrit ici sur une seule ligne pour gagner de la place (dans le fichier, il y a un nombre par ligne) :

```
150  152  149  155  160  160  158  165  170  168
```

Écrire sous chaque nombre (sauf le premier) « + » s'il augmente par rapport au précédent, « − » s'il diminue, « = » s'il ne change pas. Vérification : la réponse attendue est `5`.

## 2 Programmer la partie 1

### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans le fichier (avec `for ligne in fichier:`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

### Exercice 3 *Lire le fichier dans une liste*

**But.** Transformer le fichier texte en une liste Python d'entiers, `mesures`.
**À quoi ça sert ?** Le fichier contient du texte (des chaînes de caractères comme `"152\n"`), qu'on ne peut pas comparer correctement : il faut des entiers. Une fois la liste construite, tout le reste du défi travaille sur elle, sans relire le fichier.

Compléter ce programme pour qu'il construise la liste `mesures` des entiers du fichier :

```
mesures = []
fichier = open("input.txt")
for ligne in fichier:
    ...          # convertir la ligne en entier et l'ajouter a la liste
fichier.close()
print(len(mesures), mesures[:5])
```

Tester d'abord avec un fichier `exemple.txt` contenant l'exemple ci-dessus (un nombre par ligne).

▷ coup de pouce

### Exercice 4 *Compter les augmentations*

**But.** Écrire la fonction qui applique la règle à toute la liste.
**À quoi ça sert ?** C'est le cœur du défi : appelée sur la liste lue dans le fichier, elle donne directement la réponse de la partie 1. On l'écrit sous forme de **fonction** (qui prend une liste et renvoie un nombre) pour pouvoir la réutiliser telle quelle dans la partie 2.

Écrire une fonction `nb_augmentations(mesures)` qui renvoie le nombre de mesures plus grandes que la précédente. Tester sur l'exemple (`5`), puis sur vos données.

▷ coup de pouce
▷ début de solution

### Exercice 5 *Erreur classique*

**But.** Comprendre pourquoi la boucle doit commencer à l'indice 1.
**À quoi ça sert ?** Les erreurs d'indices (un élément oublié, un élément de trop, une erreur `IndexError`) sont les plus fréquentes dans les parcours de tableaux ; savoir les repérer fera gagner beaucoup de temps dans les défis suivants.

Que se passerait-il si la boucle était `for i in range(len(mesures) - 1)` en comparant `mesures[i]` et `mesures[i + 1]` ? Et avec `for i in range(len(mesures))` ? Tester pour vérifier.

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

**Résumé de la partie 2 (à lire après avoir validé la partie 1)**

**Ce qui change.** Une mesure isolée peut être faussée par le bruit du sonar : un poisson qui passe, une vague. Pour avoir une idée plus fiable du relief, on ne compare plus les mesures une à une, mais des **groupes de trois mesures consécutives**.
**La nouvelle règle.**

- On calcule la somme des mesures 1, 2 et 3 ; puis celle des mesures 2, 3 et 4 ; puis 3, 4 et 5, et ainsi de suite. Les groupes se **chevauchent** : chaque nouveau groupe perd la plus ancienne mesure et gagne la suivante.
- On s'arrête quand il ne reste plus trois mesures pour former un groupe complet.
- On compte, comme dans la partie 1, combien de fois une somme est strictement plus grande que la somme juste avant elle.

**La réponse attendue.** Ce nombre d'augmentations entre sommes consécutives.

### Exercice 6 *Des sommes de trois mesures*

**But.** Fabriquer la liste des sommes de trois mesures consécutives, puis compter ses augmentations.
**À quoi ça sert ?** La partie 2 pose la même question que la partie 1, mais sur une autre liste. Plutôt que de tout réécrire, on construit cette nouvelle liste et on lui applique la fonction `nb_augmentations` déjà écrite : c'est tout l'intérêt d'avoir écrit une fonction.

Écrire une fonction `sommes_par_trois(mesures)` qui renvoie la liste des sommes de trois mesures consécutives, puis réutiliser `nb_augmentations` sur cette nouvelle liste. Sur l'exemple inventé de la fiche, on doit trouver `7`. ▷ coup de pouce ▷ début de solution

## 4 Approfondissement

★★★ **Exercice 7** *Approfondissement 1 : sans calculer les sommes*

**But.** Trouver une façon plus courte de résoudre la partie 2, en raisonnant avant de programmer.
**À quoi ça sert ?** En observant ce que deux sommes voisines ont en commun, on remplace une addition de trois nombres par une seule comparaison. Simplifier un calcul par un raisonnement est une démarche qui reviendra souvent en informatique.

1. Comparer la somme qui commence à `i` et celle qui commence à `i + 1` : quels termes ont-elles en commun ? Quelle comparaison suffit pour savoir laquelle est la plus grande ? ▷ coup de pouce
2. Sur l'exemple inventé de la fiche, faire les comparaisons trouvées à la main et retrouver `7`.
3. Écrire une fonction `nb_augmentations_fenetres(mesures)` qui résout la partie 2 sans calculer aucune somme. ▷ début de solution
4. Généraliser : écrire `nb_augmentations_taille(mesures, taille)` pour des groupes de `taille` mesures. Que donne-t-elle avec `taille = 1` ?

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste `ligne.strip()` enlève le retour à la ligne, `int(...)` convertit en entier, `mesures.append(...)` ajoute à la liste. ↑énoncé

Ex. 4 – piste Parcourir les **indices** avec `for i in range(1, len(mesures))` : l'élément précédent de `mesures[i]` est `mesures[i - 1]`. ↑énoncé

Ex. 4 – début de solution Un compteur `nb = 0` avant la boucle ; dans la boucle, `if mesures[i] > mesures[i - 1]:` alors `nb += 1`. Pourquoi la boucle commence-t-elle à 1 et non à 0 ? ↑énoncé

Ex. 6 – piste Si la liste a $n$ éléments, combien y a-t-il de groupes de trois éléments consécutifs ? La somme qui commence à l'indice `i` est `mesures[i] + mesures[i + 1] + mesures[i + 2]`. ↑énoncé

Ex. 6 – début de solution `for i in range(len(mesures) - 2):` et ajouter la somme ci-dessus à une liste `sommes`. Ensuite : `nb_augmentations(sommes)`. ↑énoncé

Ex. 7 q. 1 – piste Les deux sommes partagent deux mesures sur trois. Il suffit de comparer les deux mesures qui ne sont pas communes. ↑énoncé

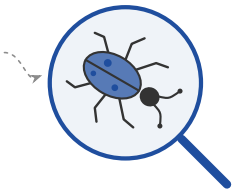
Ex. 7 q. 3 – début de solution La somme commençant à `i - 2` est plus grande que celle commençant à `i - 3` exactement quand `mesures[i] > mesures[i - 3]` ; `i` va de 3 à `len(mesures) - 1`. ↑énoncé

5

CHAPITRE 5

# Spécifier et mettre au point ses programmes

```
assert indice_maximum([2, 3, 1]) == 1
assert indice_maximum([]) is None
AssertionError  la fonction a renvoyé 0
assert indice_maximum([3, 1, 3, 7]) == 3
assert indice_maximum([8, 3, 3, 7]) == 0
```



DANS CE CHAPITRE


- Activités préparatoires . . . . . 210
- Cours . . . . . 212
- Exercices . . . . . 224
- TP et projets . . . . . 233
- Fiche de révision . . . . . 239
- Défi Advent of Code . . . . . 242
- Corrigés . . . . . 596

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 La chasse aux bugs

▶ corrigé p. 596

*faire échouer un programme qui « marche presque »*

Durée : 20 min · Sans ordinateur, par deux, crayon en main

### Consignes

- Les réponses et les tableaux s'écrivent sur le cahier.
- Un programmeur pressé a écrit deux fonctions. Il affirme : « je les ai essayées, elles marchent ». Vous êtes **testeurs** : votre mission est de les **faire échouer**.
- Un **cas** se note en trois colonnes : ce qu'on donne à la fonction, ce qu'elle *devrait* renvoyer d'après la commande du client, ce qu'elle renvoie *vraiment* (obtenu en exécutant le code à la main).

### Exercice 1 *La mention au bac*

La commande du client : « La fonction reçoit une note sur 20. Elle renvoie `"Tres bien"` à partir de 16, `"Bien"` à partir de 14, `"Assez bien"` à partir de 12, `"Admis"` à partir de 10 et `"Refuse"` en dessous de 10. »

```
1  def mention(note):
2      if note >= 16:
3          return "Tres bien"
4      elif note >= 14:
5          return "Bien"
6      elif note > 12:
7          return "Assez bien"
8      elif note >= 10:
9          return "Admis"
10     else:
11         return "Refuse"
```

**1.** Recopier sur le cahier le tableau suivant, en prévoyant une ligne par cas essayé.

| note donnée | résultat attendu | résultat obtenu |
|---|---|---|
| ... | ... | ... |

Le programmeur a essayé les notes `17`, `15` et `8` : remplir les trois premières lignes du tableau. Ses essais prouvent-ils que la fonction est correcte ?

**2.** Chercher une note **entre 0 et 20** pour laquelle la fonction se trompe, et l'ajouter au tableau. Quelle ligne du programme est en cause ?

**3.** Si vous deviez n'essayer que cinq ou six notes, lesquelles choisiriez-vous pour avoir le plus de chances de débusquer ce genre d'erreur ? Pourquoi celles-là ?

**4.** Que renvoient `mention(23)` et `mention(-5)` ? Est-ce la faute du programme ou de celui qui l'appelle ? Que vaudrait-il mieux que le programme fasse ?

**Exercice 2** *La moyenne d'une classe*

La commande du client : « La fonction reçoit le tableau des notes d'une classe et renvoie leur moyenne. »

```
1 def moyenne(notes):
2     total = 0
3     for i in range(1, len(notes)):
4         total = total + notes[i]
5     return total / len(notes)
```

**5.** Le programmeur a essayé `moyenne([0, 10, 20])` et `moyenne([0, 0])`. Recopier sur le cahier le tableau suivant, exécuter ces deux appels à la main et le remplir.

| notes données | résultat attendu | résultat obtenu |
|---|---|---|
| ... | ... | ... |

**6.** Trouver une liste de notes pour laquelle la fonction se trompe, et l'ajouter comme nouvelle ligne du tableau de cas. Expliquer l'erreur en une phrase.

**7.** Pourquoi les deux essais du programmeur n'avaient-ils rien révélé ?

**8.** Que se passe-t-il lors de l'appel `moyenne([])` ? Que devrait prévoir la commande du client ?

**Exercice 3** *Ce qu'on a découvert*

**9.** Recopier et compléter sur le cahier avec vos mots.

- Un programme qui donne le bon résultat sur quelques essais ...
- Les cas qui font le plus souvent échouer un programme sont ...
- Pour savoir si un résultat est juste, il faut d'abord savoir précisément ...
- Refaire tous ces essais à la main après chaque correction est long : on aimerait ...

# COURS

**Programme (BO)** *Langages et programmation.* **Spécification** : prototyper une fonction, décrire les **préconditions** sur les arguments et les **postconditions** sur les résultats (des assertions peuvent les garantir). **Mise au point** : utiliser des **jeux de tests** (le succès d'un jeu de tests ne garantit pas la correction). **Diversité des langages** : repérer traits communs et particuliers d'un langage. **Bibliothèques** : utiliser une documentation.

**Prérequis** écrire des **fonctions**, avec la première approche de la docstring, de `assert` et des jeux de tests (chapitre *Les bases de la programmation Python*) ; **boucles** `for`/`while` et **conditions** ; **tableaux** (listes) ; la notion d'**invariant de boucle** (vue au chapitre *Algorithmique : le parcours séquentiel*). → fiches de révision : ch. 1 p. 59

**Objectifs** **spécifier** une fonction (rôle, pré/postconditions, docstring) ; la **documenter** sans excès ; adopter une **programmation défensive** (`assert`, valeur sentinelle) ; écrire un **bon jeu de tests** et comprendre ses limites ; **déboguer** par la trace et l'affichage ; formaliser un **invariant de boucle** ; situer Python parmi la **diversité des langages** (compilé ou interprété, typage statique ou dynamique).

## Situation de départ

1. Un programme a donné le bon résultat sur les trois exemples essayés : peut-on en conclure qu'il est juste ?
2. Que doit faire une fonction qui calcule une moyenne si on lui donne une liste vide ?
3. Un programme plante, ou donne un résultat faux : par où commencer pour trouver l'erreur ?

**Remarque** — *Le fil conducteur : peut-on faire confiance à un programme ?*
Un programme dit *comment* calculer un résultat — il ne dit pas *ce qu'il* calcule, ni ne *garantit* que le résultat est correct. Or on confie à des programmes le pilotage d'un avion, le dossier médical d'un patient, le calcul d'une paie. Écrire du code qui « a l'air de marcher » ne suffit donc pas : il faut **dire précisément ce que fait chaque fonction** (la spécifier), **se protéger** des mauvais usages, et **tester** pour attraper les erreurs. Ce chapitre rassemble les **bonnes habitudes du programmeur** — à prendre dès maintenant et à rejouer dans tous les chapitres suivants.
Ces habitudes ne partent pas de zéro : comme amorcé au chapitre *Les bases de la programmation Python*, vous écrivez déjà des **docstrings**, vous posez un `assert` sur une précondition et vous accompagnez vos fonctions d'un petit **jeu de tests** (les « trois réflexes à prendre dès aujourd'hui »), en sachant qu'un test réussi ne prouve pas qu'une fonction est juste. Ce chapitre reprend ces réflexes là où nous les avions laissés pour les **approfondir** : spécification complète, programmation défensive, choix méthodique des tests, débogage, invariant de boucle.

# I Que fait ce programme ?

Considérons cette fonction.

```
def maximum(t):
    m = 0                    # INITIALISER
    for i in range(len(t)):  # PARCOURIR
        if t[i] > t[m]:      # TESTER
            m = i            # METTRE A JOUR
    return m                 # RENVOYER
```

▷ **Je prédis.** Que renvoie `maximum([4, 9, 2])` ?

▷ **Je vérifie.** `1`, et non `9` : la fonction renvoie la *position* du plus grand élément, pas sa valeur. Le nom est trompeur.

On comprend chaque instruction, mais que *calcule*-t-elle ? Le nom suggère « le maximum ». En l'essayant, on découvre qu'elle renvoie en réalité l'**indice** du maximum. On aurait gagné du temps avec un nom explicite — `indice_maximum` — et une variable `ind_max` plutôt que `m`. Et surtout : que se passe-t-il sur un tableau **vide** ? La fonction renvoie `0`, un indice qui n'existe pas.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> t = []
>>> i = maximum(t)
>>> t[i]
IndexError: list index out of range
```

L'erreur ne se déclenche pas dans la fonction, mais **plus loin**, quand on utilise son résultat — c'est le pire des cas, car le coupable est difficile à retrouver.

**Définition 1 — Spécifier une fonction**

**Spécifier** une fonction, c'est décrire son **contrat**, sans dire comment elle s'y prend :

- son **prototype** : son nom, ses paramètres, ce qu'elle renvoie ;
- ses **préconditions** : ce qui doit être vrai *sur les arguments* pour qu'on ait le droit de l'appeler ;
- ses **postconditions** : ce qui est garanti *sur le résultat* si les préconditions sont respectées.

**Exemple** — *Une spécification de* `indice_maximum`

*Prototype :* `indice_maximum(t)` renvoie un entier ou `None`. *Précondition :* `t` est une liste de nombres. *Postcondition :* renvoie l'indice du plus grand élément (le premier en cas d'égalité), ou `None` si `t` est vide.

# II Documenter ses programmes

Une précondition écrite en commentaire n'est visible que par qui lit le code. Python permet d'attacher à une fonction une **chaîne de documentation** (*docstring*) : une chaîne entre triples guillemets, placée juste après le `def`.

```
def indice_maximum(t):
    """Renvoie l'indice du maximum du tableau t,
    ou None si t est vide."""
    ...
```

On la consulte avec `help` sans voir le code :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> help(indice_maximum)
indice_maximum(t)
    Renvoie l'indice du maximum du tableau t,
    ou None si t est vide.
```

**Règle 1 — Une docstring pour chaque fonction**

Toute fonction que vous écrivez doit avoir une docstring qui donne au moins son **rôle**, et s'il y a lieu sa **précondition** et sa **postcondition**. C'est la première pièce de la spécification, et elle voyage avec la fonction.

### Du bon usage des commentaires

5

Il ne faut pas commenter à outrance. Ce commentaire est inutile :

```
x = x + 1  # incrémenter x
```

Le plus souvent, de **bons noms** de variables remplacent avantageusement un commentaire : deux coordonnées nommées `x` et `y` n'ont pas besoin d'être expliquées. « Un bon code se suffit à lui-même. » On garde un commentaire uniquement pour ce qui ne se déduit *pas* de la lecture :

```
n = 25  # nombre de nombres premiers plus petits que 100
```

▶ **Exercices d'application :** exercices 1 et 2 (spécifier et documenter)

## III Programmation défensive

Pour éviter qu'un mauvais argument produise un résultat incohérent, on peut **tester la précondition à l'entrée** de la fonction. L'outil de choix est l'instruction `assert` : elle vérifie une condition et **interrompt** le programme avec un message si elle est fausse.

```
def indice_maximum(t):
    """Renvoie l'indice du maximum de t. t est supposé non vide."""
    assert len(t) > 0, "le tableau est vide"   # VERIFIER : la precondition
    ...
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> indice_maximum([])
AssertionError: le tableau est vide
```

C'est de la **programmation défensive** : on échoue *tôt*, *clairement*, à l'endroit du vrai problème.

**Règle 2 — Deux stratégies défensives**

Face à un argument interdit, deux attitudes, aussi bonnes l'une que l'autre selon le contexte :

- **échouer** avec `assert` : celui qui appelle doit garantir la précondition ;
- **renvoyer une valeur sentinelle** qui ne peut pas être confondue avec un résultat valide — en Python, `None`. Celui qui appelle teste alors le résultat avant de l'utiliser.

↺ **Retour sur l'activité.** Dans l'activité « La chasse aux bugs », `mention(23)` renvoyait `"Tres bien"` et `moyenne([])` provoquait une division par zéro : les préconditions « note entre 0 et 20 » et « tableau non vide » n'étaient ni écrites, ni vérifiées.

```
r = indice_maximum(t)
if r is not None:
    print("le maximum est", t[r])
```

Pour tester la valeur `None`, on écrit `r is None` (ou `r is not None`) plutôt que `r == None`.

**Remarque** — *Deux conventions pour « absent » :* `None` *ou* `-1`
On rencontre les deux, y compris au bac : `None` (ici) ou `-1` ; **suivez la convention de l'énoncé**. Avec `-1`, attention : `t[-1]` est un indice *valide* (le dernier élément), donc testez `r != -1` avant d'utiliser `t[r]`.

► **Exercices d'application :** exercices 3 à 5 (programmation défensive)

## IV Tester ses programmes

Même bien spécifiée, une fonction peut contenir une erreur de programmation. Pour les débusquer, on la **teste** : on l'applique à des cas concrets et on vérifie qu'elle produit le résultat attendu. Plutôt que de vérifier à l'œil dans la console, on confie la vérification à Python avec des `assert` placés **dans le fichier**, à la suite de la fonction. On teste ici la version de `indice_maximum` qui respecte la spécification du §I (elle renvoie `None` sur un tableau vide), et non la version défensive du §III, qui échouerait sur `[]` avec son `assert`.

```
assert indice_maximum([2, 3, 1]) == 1
assert indice_maximum([]) is None
assert indice_maximum([3, 1, 3, 7]) == 3
assert indice_maximum([8, 3, 3, 7]) == 0      # max au début
assert indice_maximum([-3, -1, -3, -7]) == 1 # tous négatifs
```

Si l'un échoue, on corrige, puis on **relance tout** : corriger une erreur en introduit parfois une autre. Écrits une fois, ces tests protègent la fonction pour toutes ses versions futures.

**Règle 3 — Le succès des tests ne prouve pas la correction**

Passer tous ses tests **ne garantit pas** qu'un programme est correct : il peut échouer sur un cas non testé. On ne peut pas tout tester (les entiers, les tableaux sont en nombre infini). L'objectif est un **bon** jeu de tests, qui couvre tous les *comportements* du programme.

↻ **Retour sur l'activité.** Les essais du programmeur de l'activité (17, 15 et 8) passaient tous, et pourtant `mention(12)` renvoyait `"Admis"` au lieu de `"Assez bien"`.

**Propriété 1 — Ingrédients d'un bon jeu de tests**

- chaque **cas** de la spécification a son test (le tableau vide en particulier) ;
- une fonction **booléenne** : un test qui renvoie `True`, un qui renvoie `False` ;
- les **valeurs limites** : 0, un nombre négatif, le premier et le dernier indice ;
- les **cas particuliers** : maximum au début ou à la fin, égalités, doublons.

↻ **Retour sur l'activité.** Les cinq ou six notes à choisir dans l'activité étaient les valeurs limites 10, 12, 14 et 16, avec 0 et 20. Et la moyenne qui oubliait `notes[0]` ne se trahissait ni sur `[0, 10, 20]` ni sur `[0, 0]`, dont le premier élément vaut 0 : il manquait un cas où il n'est pas nul.

**Remarque** — *Réponses multiples*
Quand plusieurs résultats sont corrects, on ne peut pas comparer à *une* valeur. Si la spécification ne tranche pas (elle ne dit pas « le premier en cas d'égalité »), l'indice du maximum de `[3, 1, 3]` peut être 0 ou 2 : on teste alors l'appartenance à l'ensemble des bonnes réponses.

```
m = indice_maximum([3, 1, 3])
assert m == 0 or m == 2
```

## Écrire les appels attendus avant le code : les doctests

On peut écrire les tests **dans la docstring** elle-même, sous la forme d'une session de console : une ligne `>>>` avec l'appel, puis la ligne du résultat attendu. Ces exemples, appelés **doctests**, s'écrivent **avant le code** : ils disent, par l'exemple, ce que la fonction doit faire.

```
def indice_maximum(t):
    """Renvoie l'indice du maximum du tableau t (le premier en cas
    d'egalite), ou None si t est vide.

    >>> indice_maximum([2, 3, 1])
    1
    >>> indice_maximum([]) is None
    True
    >>> indice_maximum([-3, -1, -3, -1])
    1
    """
    if len(t) == 0:
        return None
    ind_max = 0                         # INITIALISER
    for i in range(len(t)):             # PARCOURIR
        if t[i] > t[ind_max]:           # TESTER
            ind_max = i                 # METTRE A JOUR
    return ind_max                      # RENVOYER

import doctest
doctest.testmod()       # rejoue tous les exemples des docstrings du fichier
```

Si tous les exemples donnent le résultat écrit, `doctest.testmod()` n'affiche **rien**.

▷ **Je prédis.** Et si l'on écrit par erreur `t[i] >= t[ind_max]` ?

Le troisième exemple échoue, et `testmod()` affiche l'appel, le résultat attendu (*Expected*) et le résultat obtenu (*Got*) :

```
**********************************************************************
File "maximum.py", line 9, in __main__.indice_maximum
Failed example:
    indice_maximum([-3, -1, -3, -1])
Expected:
    1
Got:
    3
**********************************************************************
1 items had failures:
   1 of   3 in __main__.indice_maximum
***Test Failed*** 1 failures.
```

### Méthode — Trois appels, avant le code

Les trois exemples ci-dessus sont les **trois appels attendus** à écrire avant de coder : un cas **normal** (`[2, 3, 1]`), un cas **limite** (le tableau vide), un cas **piège** (des égalités et des nombres négatifs). Sous forme d'`assert` ou de doctests, ils servent d'abord de spécification, puis de tests. À l'**épreuve pratique** du bac, les sujets donnent souvent les exemples d'appels exactement sous cette forme (`>>>` puis le résultat attendu) : les recopier dans la docstring permet de vérifier son code en une commande.

### Tests aléatoires et oracle [ au-delà du programme ]

Pour aller plus loin, on peut **générer des tests au hasard**. Encore faut-il savoir juger le résultat sans connaître la réponse : on confie ce verdict à une fonction **oracle**, en général plus simple à écrire que le programme testé. Exemple : `doublon(t)` renvoie un élément présent au moins deux fois ; l'oracle `verifie_doublon(r, t)` recompte les occurrences de `r`.

```
for n in range(2, 30):
    for k in range(20):
        t = tableau_avec_doublon(n)
        assert verifie_doublon(doublon(t), t)
```

► **Exercices d'application :** exercices 6 à 11 et 28 (tester ses programmes ; exercice 10 : les tests d'abord, en doctests)

## V Corriger les erreurs (déboguer)

Un test qui échoue prouve qu'il y *a* une erreur, mais ne dit pas **où**. Commence une enquête. Prenons une fonction censée tester si un tableau est croissant.

```
def est_croissant(t):
    i = len(t) - 1
    while i >= 0:
        if t[i] <= t[i + 1]:    # bug 1 : i+1 hors du tableau
            return True         # bug 2 : conclut trop tôt
        else:
            return False
        i -= 1
```

Sur `[1, 2, 3, 4]`, on obtient `IndexError: list index out of range`. **Idée n° 1 : dérouler l'exécution.** On voit que `i` vaut 3 et qu'on compare `t[3]` et `t[4]` : l'indice 4 n'existe pas. On corrige en comparant `t[i-1]` et `t[i]`.

**Idée n° 2 : ajouter des affichages.** La fonction répond alors `True` sur `[1, 3, 2, 4]` (faux !). Un `print` au début de la boucle montre qu'**un seul tour** a lieu : le `return True` conclut dès la première paire en ordre. Il faut ne renvoyer `True` qu'*après* avoir tout parcouru. Après correction et retrait des affichages :

```
def est_croissant(t):
    """Renvoie True si les elements de t sont ranges par ordre croissant."""
    i = len(t) - 1           # INITIALISER : on part de la fin
    while i > 0:             # PARCOURIR : de droite a gauche
        if t[i - 1] > t[i]:  # TESTER : une descente ?
            return False     # RENVOYER : inutile d'aller plus loin
        i -= 1               # METTRE A JOUR
    return True              # RENVOYER : aucune descente

assert est_croissant([1, 2, 3, 4]) == True
assert est_croissant([1, 3, 2, 4]) == False
assert est_croissant([]) == True
```

**Règle 4 — La boîte à outils du débogage**

- **dérouler** l'exécution (mentalement, sur papier, ou avec le pas-à-pas de l'éditeur) au voisinage du problème ;
- ajouter des **affichages** temporaires : quels blocs s'exécutent, combien de fois, quelles valeurs prennent les variables ;
- une fois corrigé, **retirer les affichages** mais **garder les tests**.

► **Exercices d'application :** exercices 12 à 14 et 27 (corriger, déboguer)

## VI Invariant de boucle

Vous avez rencontré cette notion avec l'invariant du champion (chapitre *Algorithmique : le parcours séquentiel*) ; on l'applique ici à une boucle `while`. Pour se convaincre qu'une boucle est correcte, on cherche une propriété **vraie avant la boucle et préservée à chaque tour** : un **invariant de boucle**. Vraie au départ et maintenue, elle est donc encore vraie *à la sortie* — c'est ce qui prouve le résultat. Prenons la division euclidienne par soustractions successives.

```
def division_euclidienne(a, b):    # a >= 0, b > 0
    q = 0                          # INITIALISER
    r = a                          # INITIALISER
    while r >= b:                  # PARCOURIR : tant qu'on peut retirer b
        # invariant : 0 <= r  et   a == q * b + r
        q += 1                     # METTRE A JOUR
        r -= b                     # METTRE A JOUR
    return q, r                    # RENVOYER
```

▷ **Je prédis.** Que renvoie `division_euclidienne(17, 5)` ? Et `division_euclidienne(3, 5)` ?
▷ **Je vérifie.** `(3, 2)` après trois tours (`r` vaut 17, 12, 7, puis `2 < 5`) ; puis `(0, 3)` sans aucun tour, car `3 >= 5` est faux dès le départ.

On veut $\texttt{a} = q \times \texttt{b} + \texttt{r}$ avec $0 \leqslant \texttt{r} < \texttt{b}$. La condition de boucle donne $\texttt{r} < \texttt{b}$ à la sortie. Restent les deux invariants :

- $0 \leqslant \texttt{r}$ : on part de $\texttt{r} = \texttt{a} \geqslant 0$ et on ne retranche `b` que si $\texttt{r} \geqslant \texttt{b}$ ;
- $\texttt{a} = q \times \texttt{b} + \texttt{r}$ : vrai au départ ($q = 0$, $\texttt{r} = \texttt{a}$), et préservé car $(q{+}1) \times \texttt{b} + (\texttt{r}{-}\texttt{b}) = q \times \texttt{b} + \texttt{r}$.

Le grand intérêt pratique : en cas de doute, on transforme l'invariant en `assert` et Python le **vérifie à chaque tour** pendant la mise au point.

```
while r >= b:
    assert 0 <= r              # VERIFIER : invariant
    assert a == q * b + r      # VERIFIER : invariant
    q += 1
    r -= b
```

> **Remarque** — *Et la terminaison ?*
> L'invariant prouve que le résultat est juste *si* la boucle s'arrête. Pour une boucle `while`, il faut aussi un **variant** (chapitre *Les bases de la programmation Python*, § V.B) : ici `r` est un entier, positif ou nul (premier invariant), qui diminue strictement à chaque tour (on lui retranche `b > 0`). La boucle se termine donc.

▶ **Exercices d'application :** exercices 15 à 19 (docstring de `division_euclidienne` ; invariant de boucle ; un meilleur nom : docstring, invariant et tests)

## VII Diversité et unité des langages

Python n'est qu'un langage parmi des centaines réellement utilisés. Pourquoi tant de langages ? Parce qu'un langage est un **outil** : chacun fait des **choix** (vitesse, sûreté, simplicité, proximité avec la machine) qui le rendent bon pour certaines tâches, moins pour d'autres. Retenons d'abord la distinction de fond : un **algorithme** est une méthode, indépendante de toute machine ; un **programme** en est la *traduction* dans un langage précis — comme une même idée se dit en français ou en anglais.

Tous les langages partagent un même **corpus de constructions élémentaires** : séquences d'instructions, affectation, conditionnelles, boucles (bornées et non bornées), appels de fonction. C'est l'**unité** des langages : qui sait programmer dans l'un retrouve vite ses repères dans un autre.

**Exemple** — *Le même calcul dans trois langages*
Somme des dix premiers entiers, en Python, puis en C, puis en JavaScript — repérez le *commun* (une boucle, une accumulation) et le *particulier* (types déclarés, accolades, points-virgules).

```python
s = 0
for i in range(1, 11):
    s = s + i
```

```c
// C
int s = 0;
for (int i = 1; i <= 10; i++)
    s = s + i;
```

```javascript
// JavaScript
let s = 0;
for (let i = 1; i <= 10; i++)
    s = s + i;
```

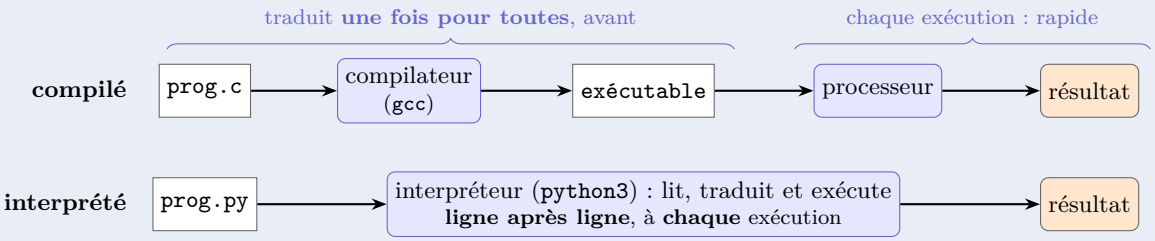
Les langages diffèrent aussi par leur **style** (leur *paradigme*) : **impératif** (une suite d'ordres qui modifient des variables — c'est ainsi qu'on programme en Python cette année), mais aussi **fonctionnel** (des fonctions qui transforment des valeurs, sans modifier l'existant), **objet** (des objets qui regroupent données et fonctions), **logique** (on énonce des faits et des règles, la machine *déduit* les réponses), **événementiel** (on réagit à un clic, une touche, comme en JavaScript). Un même langage peut en mélanger plusieurs : Python est à la fois impératif, fonctionnel et objet.

## Compiler ou interpréter

Un processeur ne comprend que le **langage machine** (des 0 et des 1 ; le chapitre *Architecture des ordinateurs et systèmes d'exploitation* y reviendra). Il faut donc **traduire** notre programme.

**Définition 2 — Compilation et interprétation**

- Un langage **compilé** est entièrement traduit *une fois pour toutes*, avant l'exécution, par un **compilateur**. On obtient un fichier **exécutable**, que l'on lance ensuite directement.
- Un langage **interprété** est traduit *au fur et à mesure*, pendant l'exécution, par un **interpréteur**. Il faut l'interpréteur à chaque exécution.




**Exemple**
**C** est compilé : la commande `gcc prog.c` produit un exécutable que l'on lance. **Python** est interprété : `python3 prog.py` lit et exécute le fichier à la volée.

**Règle 5 — Ce que chaque choix apporte**

- **Compilé** : exécution **rapide** (la traduction est déjà faite) et erreurs de langage repérées *avant* de lancer le programme ; mais il faut recompiler à chaque modification, et l'exécutable ne fonctionne que sur le type de machine visé.
- **Interprété** : **souple** et rapide à mettre au point (on teste immédiatement dans la console), le *même* fichier tourne sur toute machine qui a l'interpréteur ; mais l'exécution est **plus lente**, et certaines erreurs n'apparaissent qu'*au moment* où la ligne fautive est atteinte.

**Pour les curieux** — *La réalité est plus nuancée* (facultatif)
Beaucoup de langages, dont Python et Java, sont d'abord traduits en un langage intermédiaire (le *bytecode*), lui-même exécuté par une **machine virtuelle** : un compromis entre les deux mondes. On dira simplement, en Première, que Python est interprété et C compilé.

## Typage statique ou dynamique

Une valeur a une **nature** : entier, texte, booléen… C'est son **type**. Les langages diffèrent par le *moment* où le type est vérifié.

### Définition 3 — Typage statique et dynamique

- **Typage statique** : le type de chaque variable est fixé et vérifié *avant* l'exécution. Additionner un nombre et un texte est refusé **sans même lancer** le programme.
- **Typage dynamique** : le type est connu *pendant* l'exécution, quand la ligne est atteinte ; la même erreur ne se manifeste qu'à ce moment-là.

**Exemple**
En C (statique), on **écrit** le type : `int n = 5;` déclare un entier. En OCaml (statique), on ne l'écrit presque jamais, mais le langage le **devine** : dans `let n = 5`, OCaml sait que `n` est un entier (c'est l'*inférence de types*). En Python (dynamique), `n = 5` crée simplement un nom qui désigne la valeur `5` ; plus loin, `n` peut désigner une chaîne.

C'est pourquoi, en Python, une erreur de type peut dormir longtemps dans un programme :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1 def etiquette(age):
2     if age >= 18:
3         return "majeur"
4     return "mineur, " + age + " ans"   # erreur de type !
5
6 print(etiquette(20))   # affiche majeur : la ligne 4 n'est jamais atteinte
7 print(etiquette(15))   # TypeError : on ne concatene pas un texte et un entier
```

Seul un test passant par la ligne 4 révèle l'erreur (correction : `str(age)`) — raison de plus pour écrire un **bon jeu de tests** qui couvre *tous* les cas.

### Règle 6 — Le compromis sûreté / souplesse

Le typage statique **attrape tôt** beaucoup d'erreurs, au prix d'un peu plus de rigueur à l'écriture. Le typage dynamique est **plus léger**, mais reporte la découverte de ces erreurs à l'exécution. Ni l'un ni l'autre n'est « meilleur » : ce sont des priorités différentes.

### Quel langage pour quoi ?

Chaque grand domaine a ses langages de prédilection, pour des raisons historiques autant que techniques.

| Domaine | Langages fréquents |
|---|---|
| Systèmes, embarqué, jeux vidéo | C, C++, Rust |
| Web (côté navigateur) | JavaScript |
| Science des données, IA, enseignement | Python |
| Applications mobiles | Kotlin (Android), Swift (iPhone) |
| Interroger des bases de données | SQL |
| Recherche, preuve de programmes | OCaml, Haskell |

Aucun langage n'est « le meilleur » : chacun est un **compromis** adapté à certaines tâches.

**Remarque** — *Langages qui ne programment pas*
Attention : **HTML** (qui servira à décrire une page au chapitre *Le Web : réseaux et interactions homme-machine*) est un langage *formalisé*, mais **pas** un langage de programmation : on n'y écrit pas d'algorithme. **SQL** (interroger une base de données, en Terminale) est un langage de **requêtes**, pas un langage de programmation *généraliste* : on y décrit le résultat voulu plutôt qu'un algorithme pour l'obtenir.

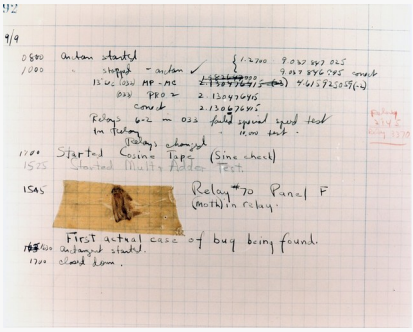
### Modularité et bibliothèques

On ne réécrit pas tout à chaque fois. En regroupant des fonctions utiles dans des **modules** réutilisables, on constitue des **bibliothèques** (`math`, `random`, `turtle`…). Pour s'en servir, pas besoin de connaître leur code : il suffit de lire leur **documentation**, qui joue le rôle de la spécification vue plus haut. La boucle est bouclée : *bien spécifier ses propres fonctions, c'est déjà écrire la documentation d'une future bibliothèque.*

► **Exercices d'application :** exercices 20 à 26 (diversité des langages)

## VIII Un peu d'histoire

**Pour les curieux** — *Des bugs qui ont fait l'histoire* (facultatif)
Le **9 septembre 1947**, à Harvard, l'équipe du calculateur électromécanique **Mark II**, dont fait partie **Grace Hopper**, cherche pourquoi la machine se trompe. Coupable : un **papillon de nuit** coincé dans un relais. On le colle dans le cahier de bord avec la mention « *First actual case of bug being found* » — « premier cas *réel* de bogue trouvé », preuve que le mot *bug* (« insecte ») désignait déjà un défaut chez les ingénieurs. Celle qui a rendu l'anecdote célèbre, Grace Hopper (1906–1992), mathématicienne et officière de la marine américaine, est surtout une pionnière des langages : elle écrit en 1952 l'un des premiers **compilateurs** et défend l'idée, jugée farfelue à l'époque, de programmer avec des **mots** plutôt qu'avec des nombres — idée qui mènera au langage Cobol (1959), et dont Python est l'héritier lointain.




*Le cahier de bord du Mark II (9 septembre 1947), avec le papillon collé*



*Grace Hopper en 1984*

Un demi-siècle plus tard, un bug coûte beaucoup plus cher qu'un papillon. Le **4 juin 1996**, la fusée **Ariane 5** explose moins de 40 secondes après son premier décollage, en Guyane. En cause, un programme de navigation **réutilisé d'Ariane 4** : il convertissait une vitesse en un entier trop petit pour contenir les valeurs, bien plus grandes, de la nouvelle fusée. Le programme était correct… pour Ariane 4. Sa **précondition** (« la vitesse reste petite ») n'avait été ni vérifiée par un `assert`, ni **testée** dans les conditions de vol d'Ariane 5. Environ 370 millions de dollars partent en fumée — sans aucune victime heureusement.

Le chercheur néerlandais **Edsger Dijkstra** (1930–2002) avait prévenu dès 1970 : « *tester un programme peut montrer la présence de bugs, jamais leur absence* ». C'est exactement la règle de ce chapitre : les tests sont indispensables, mais seul un **raisonnement** (spécification, invariant de boucle) peut garantir qu'un programme est correct.

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** Docstring, `assert` et jeux de tests amorcés dans les *Bases de la programmation Python* ; l'invariant du champion du *Parcours séquentiel*.
- **Ce qu'il prépare.** Ces réflexes serviront aussitôt pour les **tris** et la **dichotomie** (spécification, tests, invariants).
- **En Terminale.** *La récursivité* se spécifie et se teste exactement comme ici ; *Programmation objet et paradigmes* comparera les styles de programmation ; *Calculabilité* prouvera qu'aucun programme ne décide l'arrêt de tous les autres.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Spécifier | prototype + **précondition** (arguments) + **postcondition** (résultat) |
| Documenter | une **docstring** par fonction ; des noms parlants ; commenter l'inattendu |
| Se défendre | `assert` sur la précondition *ou* valeur sentinelle `None` |
| Tester | un **jeu de tests** en `assert` (vide, limites, booléens, doublons) |
| Se méfier | tests verts ≠ programme correct (Dijkstra) |
| Déboguer | dérouler + `print` pour localiser ; garder les tests, retirer les affichages |
| Invariant de boucle | vrai au départ et préservé à chaque tour ⇒ vrai à la sortie |
| Unité des langages | mêmes briques : séquence, affectation, condition, boucle, fonction |
| Compilé / interprété | traduit une fois, rapide (C) / traduit à la volée, souple (Python) |
| Typage | **statique** : vérifié avant l'exécution (C) ; **dynamique** : pendant (Python) |
| Bibliothèques | on les utilise en lisant leur **documentation** |

### Retour à la situation de départ

1. Non : le succès des tests ne prouve pas la correction (§ IV). On vise un **bon jeu de tests** (cas de la spécification, valeurs limites, cas particuliers), et seul un raisonnement, comme un **invariant de boucle**, garantit le résultat (§ VI).
2. C'est à la **spécification** de le dire (§ I) : soit une précondition « liste non vide » vérifiée par un `assert`, qui échoue tôt et clairement, soit une **valeur sentinelle** `None` que l'appelant teste (§ III).
3. On **déroule** l'exécution au voisinage du problème et on ajoute des **affichages** temporaires des variables ; une fois l'erreur corrigée, on retire les affichages et on garde les tests (§ V).

## Erreurs fréquentes

- **Oublier de tester « absent ».** Avec `-1`, `t[-1]` est valide ! *Le réflexe :* suivre la convention de l'énoncé.
- **Confondre précondition et postcondition.** Précondition : avant l'appel, sur les arguments ; postcondition : après, sur le résultat.
- **Oublier le tableau vide** (ou 0, ou un nombre négatif) dans le jeu de tests.
- **Conclure `return True` dès le premier élément conforme**, sans finir le parcours.
- **Croire un programme correct parce que les tests passent.** Un cas non testé peut échouer (Ariane 5).
- **Laisser des `print` de débogage** dans la version finale — mais *garder* les tests.
- **Dire que C est interprété** ou que Python est compilé ; appeler HTML un « langage de programmation », ou SQL un langage de programmation *généraliste*.

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

☐ **spécifier** une fonction : prototype, **précondition** sur les arguments, **postcondition** sur le résultat, dans une **docstring** ;

☐ documenter un programme sans excès (noms parlants, commentaires utiles) ;

☐ protéger une fonction par un `assert` sur sa précondition, ou par une **valeur sentinelle** ;

☐ écrire un **jeu de tests** qui couvre les cas limites (tableau vide, extrémités, absent…) et expliquer pourquoi des tests réussis ne prouvent pas qu'un programme est correct ;

☐ **déboguer** : dérouler le programme à la main, placer des affichages (`print`) pour localiser l'erreur, puis les retirer en gardant les tests ;

☐ énoncer un **invariant de boucle** et vérifier qu'il est vrai au départ et préservé à chaque tour ;

☐ repérer les traits communs et particuliers de plusieurs langages (compilé ou interprété, typage statique ou dynamique) et utiliser la **documentation** d'une bibliothèque.

# EXERCICES

## Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et tester* en Python. `>>>` figure une saisie dans la console.
- Réflexe de tout le chapitre : **une docstring, une précondition, des tests** pour chaque fonction.



## Spécifier et documenter ↑ cours

★ **Exercice 1** *Précondition ou postcondition ?* ▶ corrigé p. 597 • essentiel

Pour chaque affirmation sur une fonction `racine_carree(x)`, dire s'il s'agit d'une **précondition** ou d'une **postcondition**.

a) `x` doit être positif ou nul ;
b) le résultat élevé au carré redonne `x` ;
c) le résultat est positif ou nul.

▷ coup de pouce

★ **Exercice 2** *Prototyper puis documenter* ▶ corrigé p. 597

Écrire l'en-tête (`def...`) **et** la docstring d'une fonction `puissance(x, n)` qui calcule `x` à la puissance `n`, pour `n` entier positif ou nul. On n'écrit *pas* le corps. ▷ coup de pouce

## Programmation défensive ↑ cours

★ **Exercice 3** *Poser un garde-fou* ▶ corrigé p. 597 • essentiel

Ajouter à la fonction `puissance(x, n)` une ligne `assert` qui garantit sa précondition, avec un message clair. Que se passe-t-il si on appelle `puissance(2, -3)` ? ▷ coup de pouce

★★ **Exercice 4** `assert` *ou* `None` *?* ▶ corrigé p. 597

On écrit `premier_element(t)` qui renvoie le premier élément d'un tableau.

a) Écrire une version **défensive avec `assert`** (précondition : `t` non vide).
b) Écrire une version qui **renvoie `None`** sur un tableau vide.
c) Pour chaque version, écrire le code qui **appelle** la fonction en respectant sa stratégie.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 5** *Le piège de* `-1` ▶ corrigé p. 598

Une fonction `indice_de(v, t)` renvoie l'indice de `v` dans `t`, et `-1` si `v` est absente (convention courante, présente dans des sujets de bac).

a) Sur un exemple, montrer ce qui se passe si l'appelant écrit `t[indice_de(v, t)]` sans vérifier le résultat.
b) Écrire ce code appelant correctement.
c) Quelle autre convention évite ce piège ?

▷ coup de pouce

## Tester ses programmes ↑cours

★ **Exercice 6** *Tests pour* `puissance` ▶ corrigé p. 598 • essentiel

Écrire un bon jeu de tests (au moins quatre `assert`) pour `puissance(x, n)`. On pensera au **cas limite** `n = 0`. ▷ coup de pouce

★★ **Exercice 7** *Trouver le contre-exemple* ▶ corrigé p. 598 • essentiel

On prétend que la fonction suivante teste l'appartenance de `v` au tableau `t`.

```
def appartient(v, t):
    i = 0
    while i < len(t) - 1 and t[i] != v:
        i = i + 1
    return i < len(t)
```

Donner un jeu de tests, dont un `assert` qui **échoue** et prouve que la fonction est incorrecte. Où est l'erreur ? ▷ coup de pouce

★★ **Exercice 8** *Deux erreurs pour le prix d'une* ▶ corrigé p. 599

Même énoncé pour cette autre version.

```
def appartient(v, t):
    for i in range(len(t)):
        if t[i] == v:
            trouvee = True
        else:
            trouvee = False
    return trouvee
```

Donner des tests montrant **deux** raisons distinctes pour lesquelles elle est fausse. ▷ coup de pouce

★★ **Exercice 9** *Un bon jeu de tests* ▶ corrigé p. 599

Une fonction `est_pair(n)` renvoie un booléen. En s'appuyant sur la propriété « Ingrédients d'un bon jeu de tests » du cours, lister les **cas** qu'un bon jeu de tests doit couvrir, puis écrire les `assert` correspondants. ▷ coup de pouce

★★ **Exercice 10** *Les tests d'abord* ▶ corrigé p. 599

On veut une fonction `nb_occurrences(c, mot)` qui renvoie le nombre de fois que le caractère `c` apparaît dans la chaîne `mot`. On écrit les tests **avant** le code.

a) Écrire l'en-tête et la docstring de `nb_occurrences`, avec **trois doctests** (une ligne `>>>` avec l'appel, puis le résultat attendu) : un cas **normal**, un cas **limite**, un cas **piège**.

b) ▶ Écrire ensuite le corps de la fonction, sans utiliser de fonction toute faite, et vérifier les exemples avec `doctest.testmod()`.

c) ▶ Initialiser volontairement le compteur à `1`. Que montre `testmod()` ?

▷ coup de pouce

★★★ **Exercice 11** *Des tests qui démasquent* ▶ corrigé p. 600 ≈ 20 min

La fonction `est_trie(t)` doit renvoyer `True` si les éléments du tableau `t` sont rangés dans l'ordre croissant au sens large (`[2, 2, 5]` est trié), `False` sinon. Trois élèves en proposent chacun une version.

```
def est_trie_1(t):
    for i in range(len(t) - 2):
        if t[i] > t[i + 1]:
            return False
    return True

def est_trie_2(t):
    for i in range(len(t) - 1):
        if t[i] >= t[i + 1]:
            return False
    return True
```

```
12
13 def est_trie_3(t):
14     for i in range(len(t) - 1):
15         if t[i] > t[i + 1]:
16             return False
17         else:
18             return True
19     return True
```

a) Sans machine, trouver pour chaque version un tableau sur lequel elle répond faux.
b) ▶ Écrire un jeu de tests **le plus court possible** (des `assert` sur `est_trie`) qu'une version correcte passe. Le rejouer ensuite en remplaçant `est_trie` par chacune des trois versions : **chacune** doit échouer sur au moins un `assert`. Combien d'`assert` suffisent ?
c) Les tests `[]`, `[5]`, `[1, 2, 3]` et `[3, 1, 2]` auraient-ils détecté une de ces erreurs ? Qu'en conclure sur le choix des tests ?

▷ coup de pouce ▷ début de solution

## Corriger les erreurs (déboguer) ↑ cours

★★ **Exercice 12** *La chasse au bug* ▶ corrigé p. 600 • essentiel

La fonction doit renvoyer le plus petit élément d'un tableau non vide.

```
1 def minimum(t):
2     m = 0
3     for x in t:
4         if x < m:
5             m = x
6     return m
```

a) Trouver un tableau sur lequel elle renvoie un résultat **faux**.
b) Expliquer l'erreur (dérouler l'exécution), puis **corriger** la fonction.
c) Ajouter une docstring, une précondition défensive et des tests.

▷ coup de pouce

★★ **Exercice 13** *Débogage guidé par l'affichage* ▶ corrigé p. 600

La fonction doit renvoyer le nombre d'éléments **strictement positifs** d'un tableau.

```
1 def nb_positifs(t):
2     nb = 0
3     for x in t:
4         if x > 0:
5             nb = 1
6     return nb
```

Elle renvoie `1` pour `[3, 8, -2, 5]` alors qu'on attend `3`. En imaginant un `print(nb)` dans la boucle, expliquer ce qu'on observerait, identifier l'erreur et la corriger. ▷ coup de pouce

★★ **Exercice 14** *Une fonction spécifiée par un assistant d'IA* ▶ corrigé p. 601

Un élève demande à un assistant d'IA : « Écris une fonction `nb_chiffres(n)` qui renvoie le nombre de chiffres d'un entier positif, avec sa docstring et un jeu de tests. » Voici la réponse obtenue.

```
1 def nb_chiffres(n):
2     """Renvoie le nombre de chiffres de l'entier n.
3     Precondition : n >= 0. Par convention nb_chiffres(0) vaut 1."""
4     compteur = 0
5     while n > 0:
6         n = n // 10
7         compteur = compteur + 1
8     return compteur
```

```
9
10 assert nb_chiffres(2024) == 4
11 assert nb_chiffres(7) == 1
12 assert nb_chiffres(0) == 1
```

« À chaque tour, on retire le dernier chiffre par division entière et on compte ; la boucle s'arrête quand il ne reste plus rien. Les trois tests couvrent un cas général, un cas à un chiffre et le cas limite `0` : ils passent tous. »

a) La réponse est-elle correcte ? Exécuter le fichier tel quel (ou faire la trace de `nb_chiffres(0)`).
b) Localiser et corriger l'erreur, sans changer la spécification.
c) Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce



## Invariant de boucle ↑ cours

★ **Exercice 15** *Une docstring pour* `division_euclidienne` ▶ corrigé p. 601

Donner une chaîne de documentation (rôle, précondition, postcondition) à la fonction `division_euclidienne` vue dans le cours. ▷ coup de pouce

★★ **Exercice 16** *Énoncer un invariant* ▶ corrigé p. 601 • essentiel

Donner un invariant de boucle pour cette fonction, puis prouver qu'à la sortie il donne le résultat attendu.

```
1 def puissance(x, n):
2     r = 1
3     for i in range(n):
4         r = r * x
5     return r
```

▷ coup de pouce

★★ **Exercice 17** *Un meilleur nom* ▶ corrigé p. 601

Cette fonction « marche », mais se lit mal.

```
1 def f(t):
2     s = 0
3     for i in range(len(t)):
4         s = s + t[i]
5     return s
```

Lui donner : (a) un **nom** explicite, (b) une **docstring**, (c) un **invariant** de boucle, (d) trois **tests**.

▷ coup de pouce

★★★ **Exercice 18** *Invariant et* `assert` ▶ corrigé p. 602 ≈ 15 min

On compte les éléments pairs d'un tableau d'entiers.

```
1 def nb_pairs(t):
2     c = 0
3     for i in range(len(t)):
4         if t[i] % 2 == 0:
5             c = c + 1
6     return c
```

(a) Énoncer l'invariant reliant `c` et `i`. (b) L'écrire sous forme d'un `assert` placé au bon endroit dans la boucle. (c) En quoi cela aide-t-il à la mise au point ? ▷ coup de pouce ▷ début de solution

★★★ **Exercice 19** *Une division qui se trompe parfois* ▶ corrigé p. 602 ≈ 20 min

Cette fonction doit renvoyer le couple `(q, r)` du quotient et du reste de la division euclidienne de `a` par `b`, pour `a` entier positif ou nul et `b` entier strictement positif.

```
def division(a, b):
    q = 0
    r = a
    while r > b:
        r = r - b
        q = q + 1
    return q, r
```

a) Écrire sa docstring : rôle, précondition, postcondition (deux relations entre `a`, `b`, `q` et `r`).

b) `division(17, 5)` renvoie `(3, 2)`, ce qui est juste. Trouver un appel pour lequel le résultat est faux.

c) ▶ Ajouter un `assert` de l'invariant `a == q * b + r` au début du corps de la boucle, et un `assert` de la postcondition juste avant le `return`. Lequel échoue sur votre appel ? Qu'en déduire : l'erreur est-elle dans le corps de la boucle ou dans sa condition ?

d) Corriger la fonction et ajouter l'`assert` de la précondition. Que se passerait-il, sans lui, pour `division(5, 0)` ?

▷ coup de pouce ▷ début de solution

## Diversité des langages et bibliothèques ↑ cours

★ **Exercice 20** *Compilé ou interprété ?* ▶ corrigé p. 603 • essentiel

Pour chacune des affirmations, indiquer si elle décrit plutôt un langage **compilé** ou **interprété**, et justifier en une phrase.

a) « Le même fichier de code tourne sur ma machine et sur celle de mon voisin, sans rien retraduire. »

b) « On obtient un fichier exécutable rapide, mais il faut tout retraduire à chaque modification. »

c) « Une erreur de langage se manifeste seulement quand le programme atteint la ligne fautive. »

d) « Les erreurs de langage sont signalées avant même de lancer le programme. »

▷ coup de pouce

★★ **Exercice 21** *Statique ou dynamique ?* ▶ corrigé p. 603

a) Pour chacune de ces trois lignes, dire si le typage est **statique** ou **dynamique**, et si le type est **écrit** ou **deviné** (inféré) : (C) `int n = 5;` (Python) `n = 5` (OCaml) `let n = 5`

b) On considère la fonction Python suivante.

```
def prix_ttc(prix, code):
    if code == "normal":
        return prix * 1.2
    return "prix reduit : " + prix * 1.055
```

Les appels `prix_ttc(10, "normal")` et `prix_ttc(50, "normal")` fonctionnent. Peut-on en conclure que la fonction est correcte ? Quel appel révèle l'erreur, et quel message Python affiche-t-il ? Un langage à typage statique aurait-il laissé passer cette erreur ?

▷ coup de pouce

★ **Exercice 22** *Le bon outil* ▶ corrigé p. 603

Associer chaque besoin au langage le plus naturel (une seule réponse par ligne) : **Python**, **C**, **JavaScript**, **SQL**.

a) Rendre une page web interactive dans le navigateur.
b) Interroger une base de données pour retrouver des élèves.
c) Programmer un tout petit objet embarqué où chaque microseconde compte.
d) Analyser rapidement un jeu de données pour un projet de science.

▷ coup de pouce

★★ **Exercice 23** *Vrai ou faux, et pourquoi* ▶ corrigé p. 603

Dire si chaque affirmation est vraie ou fausse, en corrigeant les fausses.

a) « Il existe un langage meilleur que tous les autres. »
b) « Un algorithme dépend du langage dans lequel on l'écrit. »
c) « Python peut se programmer dans plusieurs paradigmes. »
d) « Un langage compilé est en général plus lent à l'exécution qu'un langage interprété. »

▷ coup de pouce

★ **Exercice 24** *Langage de programmation ou pas ?* ▶ corrigé p. 603

Pour chacun, dire si c'est un **langage de programmation généraliste** (qui permet d'écrire n'importe quel algorithme) : Python, HTML, SQL, C, JavaScript. Justifier en une phrase. Pour les autres, dire à quoi ils servent. ▷ coup de pouce

★★ **Exercice 25** *Lire un programme dans un langage inconnu* ▶ corrigé p. 603

Voici une fonction dans un langage que vous ne connaissez pas.

```
fonction double(x)
    retourne x * 2
fin
```

Repérer : (a) la **construction élémentaire** commune avec Python (appel, valeur de retour) ; (b) ce qui **diffère** de la syntaxe Python. Réécrire cette fonction en Python. ▷ coup de pouce

★★ **Exercice 26** *Lire une documentation* ▶ corrigé p. 603

La documentation du module `random` indique : `randint(a, b)` *« renvoie un entier aléatoire $N$ tel que $a \leqslant N \leqslant b$ »*.

a) Quelle est la **précondition** implicite sur `a` et `b` ?
b) Écrire un appel qui simule le lancer d'un dé à six faces.
c) A-t-on besoin de connaître le *code* de `randint` pour l'utiliser ?

▷ coup de pouce

## Entraînement ↑cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

★ **Exercice 27** *Lire un message d'erreur* ▶ corrigé p. 604 • essentiel

Chaque programme s'arrête sur une erreur ; on donne la ligne où Python s'arrête et la **dernière ligne** du message. Pour chacun : nommer le type d'erreur, expliquer sa cause, puis ▶ corriger le programme (ou dire quelle précondition aurait dû être vérifiée).

a) Arrêt sur la ligne `total = total + notes[i]` :

```
notes = [12, 15, 9]
total = 0
for i in range(len(notes) + 1):
    total = total + notes[i]
print(total)
```

```
IndexError: list index out of range
```

b) Arrêt sur la ligne `return s / len(t)`, lors de l'appel `print(moyenne([]))` :

```
def moyenne(t):
    s = 0
    for x in t:
        s = s + x
    return s / len(t)

print(moyenne([]))
```

```
ZeroDivisionError: division by zero
```

c) Arrêt sur la ligne `print(stock["kiwi"])` :

▷ coup de pouce

```
1 stock = {"pomme": 4, "poire": 0}
2 print(stock["kiwi"])
```

```
KeyError: 'kiwi'
```

★ **Exercice 28** *Un test qui échoue* ▶ corrigé p. 604 • essentiel

Chaque fonction ne respecte pas sa spécification. Pour chacune, écrire un `assert` qui **échoue**, en choisissant un cas limite, puis ▶ le vérifier et corriger la fonction.

a) `est_majeur(age)` doit renvoyer `True` si `age >= 18` ; son corps est `return age > 18`.

b) `somme_jusqua(n)` doit renvoyer $1 + 2 + \cdots + n$ :

```
1 def somme_jusqua(n):
2     s = 0
3     for i in range(n):
4         s = s + i
5     return s
```

c) `est_croissant(a, b, c)` doit renvoyer `True` si $a \leqslant b \leqslant c$ ; son corps est `return a < b and b < c`.

▷ coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Une précondition porte sur ce que l'on *donne* à la fonction (ses arguments) ; une postcondition sur ce qu'elle *renvoie*. Pour chaque phrase, de quoi parle-t-on ? ↑énoncé

Ex. 2 – piste Suivre le modèle de docstring du cours : une phrase pour le rôle, puis « Précondition : » et « Postcondition : ». La docstring se place juste sous la ligne `def`, entre triples guillemets. ↑énoncé

Ex. 3 – piste La précondition de `puissance` porte sur `n`. La syntaxe est `assert condition, "message"`, où la condition énonce ce qui doit être *vrai*. Que vaut-elle pour `n = -3` ? ↑

Ex. 4 – piste c) Avec la version `assert`, l'appelant doit vérifier quelque chose *avant* l'appel ; avec la version `None`, il vérifie *après* l'appel, sur le résultat. ↑énoncé

Ex. 5 – piste Que vaut `t[-1]` en Python ? Imaginer un appelant pressé qui écrit `t[indice_de(v, t)]` sans rien vérifier. ↑énoncé

**Ex. 6 – piste** Un test s'écrit `assert appel == resultat_attendu`, le résultat attendu étant calculé à la main. Varier les cas : `n = 0`, `n = 1`, un cas « normal », une base particulière comme `1`. ↑énoncé

**Ex. 7 – piste** Varier les situations : valeur en tête, en dernière position, absente. Dans chaque cas, que vaut `i` à la sortie de la boucle, et que renvoie alors `i < len(t)` ? ↑énoncé

**Ex. 8 – piste** Que se passe-t-il si `t` est vide ? Et si l'élément cherché est présent, mais pas en dernière position ? ↑énoncé

**Ex. 9 – piste** Une fonction booléenne a deux réponses possibles : les tester toutes les deux. Penser aussi aux valeurs limites : 0 et les nombres négatifs. ↑énoncé

**Ex. 10 – piste** Cas limite : la chaîne vide. Cas piège : un caractère en majuscule et en minuscule dans le même mot (`"A"` et `"a"` sont-ils le même caractère ?). Le résultat attendu s'écrit sur la ligne qui suit l'appel, sans `>>>`. ↑énoncé

**Ex. 11 – piste** Écrire d'abord une version correcte, puis comparer chaque version avec elle, ligne à ligne. Quel tableau oblige le programme à passer par la ligne qui diffère ? ↑énoncé

**Ex. 11 – début de solution** Version 1 : sur un tableau de 3 éléments, quelles paires d'indices sont réellement comparées ? Version 2 : que répond-elle sur deux éléments égaux ? Version 3 : combien de paires examine-t-elle avant de répondre ? ↑

**Ex. 12 – piste** Essayer un tableau dont tous les éléments sont strictement positifs. Avec quelle valeur faut-il initialiser le « meilleur jusqu'ici » ? ↑énoncé

**Ex. 13 – piste** Écrire la suite des valeurs que `print(nb)` afficherait sur `[3, 8, -2, 5]`. À chaque élément positif, de combien `nb` devrait-il évoluer ? ↑énoncé

**Ex. 14 – piste** Dérouler à la main `nb_chiffres(0)` : combien de tours fait la boucle `while n > 0` ? Que vaut alors `compteur` ? ↑énoncé

**Ex. 15 – piste** Rôle : que renvoie la fonction ? Précondition : quelles valeurs de `a` et de `b` sont permises (penser à la division par zéro) ? Postcondition : quelles relations relient `a`, `b`, `q` et `r` ? ↑

**Ex. 16 – piste** Que vaut `r` au début du tour d'indice `i` ? Le vérifier pour `i = 0, 1, 2`, puis se demander combien de tours ont été faits à la sortie. ↑énoncé

**Ex. 17 – piste** Commencer par l'essayer sur `[1, 2, 3]` : que calcule-t-elle ? Pour l'invariant : que contient `s` au début du tour d'indice `i` ? Pour les tests, penser au tableau vide. ↑énoncé

**Ex. 18 – piste** Au début du tour d'indice `i`, `c` compte les éléments pairs parmi quels éléments ? Comment calculer ce nombre *autrement*, pour le comparer à `c` ? ↑énoncé

**Ex. 18 – début de solution** Écrire d'abord une petite fonction de vérification `nb_pairs_debut(t, i)` qui renvoie le nombre d'éléments pairs parmi ceux d'indices `0` à `i - 1`. L'`assert` compare `c` à `nb_pairs_debut(t, i)` ; il doit être exécuté à chaque tour, *avant* la mise à jour de `c`. ↑énoncé

**Ex. 19 – piste** Penser aux cas limites : que se passe-t-il au moment où `r` devient exactement égal à `b` ? ↑énoncé

**Ex. 19 – début de solution** Postcondition : `a == q * b + r`, `0 <= r` et `r < b`. Sur l'appel trouvé, l'invariant reste vrai à chaque tour : ce n'est donc pas le corps de la boucle qui est en cause. ↑énoncé

**Ex. 20 – piste** Un compilateur traduit tout le programme une fois, avant l'exécution ; un interpréteur traduit et exécute au fur et à mesure. Pour chaque phrase, *quand* la traduction a-t-elle lieu ? ↑énoncé

**Ex. 21 – piste** a) Le type est-il écrit dans la ligne ? Est-il vérifié avant ou pendant l'exécution ? b) Quelle ligne de la fonction n'est exécutée par aucun des deux appels ? ↑énoncé

**Ex. 22 – piste** Lequel de ces langages s'exécute dans le navigateur ? Lequel interroge des tables ? Lequel est très proche de la machine ? Le dernier reste pour la science des données. ↑

**Ex. 23 – piste** Pour chaque affirmation, chercher un exemple ou un contre-exemple dans la partie « Diversité et unité des langages » du cours. ↑énoncé

Ex. 24 – piste Un langage de programmation permet d'écrire des algorithmes : variables, conditions, boucles. Peut-on écrire une boucle en HTML ? ↑énoncé

Ex. 25 – piste Repérer le mot qui joue le rôle de `def`, celui qui joue le rôle de `return`, et la façon dont la fin du bloc est marquée. ↑énoncé

Ex. 26 – piste a) Que deviendrait la phrase de la documentation si `a` était plus grand que `b` ? b) Quelles sont les valeurs possibles d'un dé ? ↑énoncé

Ex. 27 – piste Le type d'erreur (avant le deux-points) dit ce qui s'est passé ; la ligne indiquée dit où. Pour a), lister les valeurs prises par `i` et les indices qui existent dans `notes`. ↑énoncé

Ex. 28 – piste Une erreur se cache souvent à la frontière : la valeur exacte d'un seuil, deux valeurs égales, la dernière valeur d'un `range`. Calculer à la main le résultat attendu, puis celui de la fonction. ↑énoncé

# TP ET PROJETS

## TP Le bulletin bogué ▶ corrigé p. 604

*spécifier, tester, déboguer et sécuriser un vrai module*

Durée : 2 à 3 h (deux séances) · Sur machine, par deux

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p05-tp-bulletin-bogue` — **archive .zip**



### But du TP

Un programmeur pressé a écrit le module qui calcule les bulletins de notes du lycée. Il affirme : « j'ai testé, tout passe ». Pourtant, le bulletin qu'il affiche est faux de la première à la dernière ligne. Votre mission : **spécifier** ses sept fonctions, écrire des **jeux de tests** qui démasquent les bugs, **déboguer**, **sécuriser** le code par la programmation défensive, **prouver** une boucle par un invariant et un variant — puis comparer avec un autre langage. Produit final : un fichier `bulletin.py` correct, documenté et testé, et un **rapport de bugs**.

### Consignes

- Fichier à télécharger (lien ci-dessus) : `tp_mise_au_point_depart.py`. Le **copier** sous le nom `bulletin.py` et ne travailler que sur la copie.
- Les réponses aux questions (sans ▶ ) s'écrivent sur le cahier.
- ▶ signale ce qui est *à programmer et tester*. Garder **tous** les tests écrits au fil du TP en bas du fichier, et relancer le fichier après chaque modification.
- Si le programme se fige, c'est une **boucle infinie** : l'interrompre avec `Ctrl + C` et le noter.
- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Prise en main : un bulletin qui a l'air juste

Voici les sept fonctions du fichier fourni (commentaires d'origine compris). Les notes sont sur 20 et rangées dans l'ordre chronologique des devoirs.

```
def moyenne(notes):
    # moyenne des notes
    somme = 0
    for i in range(1, len(notes)):
        somme = somme + notes[i]
    return somme / len(notes)

def moyenne_ponderee(notes, coefs):
    # moyenne en tenant compte des coefficients
```

```
    somme = 0
    for i in range(len(notes)):
        somme = somme + notes[i] * coefs[i]
    return somme / len(notes)

def arrondi_demi(x):
    # arrondi au demi-point
    return int(x * 2) / 2

def nb_admis(notes):
    # combien de notes sont au moins egales a 10
    nb = 0
    for x in notes:
        if x > 10:
            nb = nb + 1
    return nb

def rang(moyennes, m):
    # rang de la moyenne m parmi les moyennes de la classe
    r = 0
    for x in moyennes:
        if x >= m:
            r = r + 1
    return r

def nb_progressions(notes):
    # combien de fois une note est meilleure que la precedente
    nb = 0
    for i in range(len(notes)):
        if notes[i] > notes[i - 1]:
            nb = nb + 1
    return nb

def premiere_sous(notes, seuil):
    # indice de la premiere note strictement inferieure au seuil
    i = 0
    while i < len(notes):
        if notes[i] < seuil:
            return i
        i = i + 1
    return None
```

Le fichier se termine par les tests du programmeur et par l'affichage du bulletin de Lina :

```
assert moyenne([0, 10, 20]) == 10
assert moyenne_ponderee([12, 14], [1, 1]) == 13
assert arrondi_demi(12.5) == 12.5
assert nb_admis([8, 12, 15]) == 2
assert rang([12, 15, 9], 12) == 2
assert nb_progressions([8, 10, 12]) == 2
assert premiere_sous([5, 12, 14], 10) == 0
print("Tous les tests passent.")

bulletin("Lina", [15, 9, 12, 10], [1, 2, 1, 2], [13.5, 11.0, 15.0, 11.0, 8.5, 9.5])
```

★ **Exercice 1** *Le bulletin de Lina, à la main*

1. ▶ Lancer le fichier. Recopier ce qu'il affiche.
2. Lina a eu 15 (coef. 1), 9 (coef. 2), 12 (coef. 1), 10 (coef. 2). Calculer **à la main** :
   a) sa moyenne pondérée, puis cette moyenne arrondie au demi-point le plus proche ;
   b) le nombre de ses notes qui valent au moins 10 ;
   c) le nombre de fois où elle a progressé d'un devoir au suivant ;

d) son rang, sachant que les moyennes de la classe (dont la sienne) sont `13.5, 11.0, 15.0, 11.0, 8.5, 9.5` (les ex aequo partagent le même rang).

3. Comparer avec l'affichage. Quelles lignes du bulletin sont fausses ? Quel résultat est même *impossible*, quelles que soient les notes ?
4. « Tous les tests passent. » Que prouve cette phrase ? Que ne prouve-t-elle pas ?

## 2 Spécifier : écrire le contrat

Le cahier des charges du lycée décrit ce que chaque fonction *doit* faire.

**Cahier des charges**

- `moyenne(notes)` : la moyenne des notes ; `None` si le tableau est vide.
- `moyenne_ponderee(notes, coefs)` : somme des `note` × `coef` divisée par la **somme des coefficients** ; autant de coefficients que de notes, tous strictement positifs ; au moins une note.
- `arrondi_demi(x)` : pour un nombre positif `x`, le demi-point **le plus proche** (`12.2` donne `12.0`, `12.8` donne `13.0`) ; à égale distance, on arrondit vers le haut (`12.25` donne `12.5`).
- `nb_admis(notes)` : le nombre de notes **supérieures ou égales** à 10.
- `rang(moyennes, m)` : `m` est l'une des moyennes de la classe ; son rang vaut 1 + le nombre de moyennes **strictement** supérieures (deux ex aequo ont le même rang).
- `nb_progressions(notes)` : le nombre de devoirs dont la note est strictement supérieure à celle du devoir **précédent** (le premier devoir n'a pas de précédent).
- `premiere_sous(notes, seuil)` : l'indice de la première note strictement inférieure à `seuil`, ou `None` s'il n'y en a pas.
- Dans toutes les fonctions, une note est un nombre entre 0 et 20.

★★ **Exercice 2** *Des docstrings à la place des commentaires*

1. Pour chacune des sept fonctions, remplacer le commentaire par une **docstring** qui donne son rôle, ses **préconditions** (sur les arguments) et sa **postcondition** (sur le résultat). Vérifier le résultat avec `help(rang)` dans la console.
2. Le nom `nb_admis` est-il bien choisi pour « nombre de notes d'au moins 10 » ? Proposer mieux, sans le changer dans le fichier (le reste du logiciel l'utilise).
3. Pour `rang`, donner une postcondition qui encadre le résultat entre deux valeurs.

## 3 Tester pour démasquer

★★ **Exercice 3** *Un vrai jeu de tests*

1. Pour chaque fonction, expliquer en une phrase pourquoi le test du programmeur est **trop faible** (par exemple : quel ingrédient d'un bon jeu de tests (propriété du cours) lui manque-t-il ?).
2. ▶ Écrire pour chaque fonction un jeu de tests d'au moins **trois** `assert` : un cas normal, un cas limite, un cas particulier tiré du cahier des charges. **Ne rien corriger pour l'instant** : le but est de *faire échouer* le code.
3. Mettre en commentaire chaque test qui échoue, en le faisant précéder de `# ECHOUE`, pour que les tests suivants s'exécutent. Recopier sur le cahier (ou en commentaire dans `bulletin.py`) le tableau ci-dessous et le compléter (un bug par fonction, et un de plus : le cas du tableau vide pour `moyenne`).

▷ coup de pouce

| Fonction | Test qui échoue | Attendu | Obtenu |
|---|---|---|---|
| `moyenne` | | | |
| `moyenne` (vide) | | | |
| `moyenne_ponderee` | | | |
| `arrondi_demi` | | | |
| `nb_admis` | | | |
| `rang` | | | |
| `nb_progressions` | | | |
| `premiere_sous` | | | |

5

★★ **Exercice 4** *Un bug sans message d'erreur*

L'appel `nb_progressions([12, 8, 10])` renvoie `2` au lieu de `1`, **sans aucune erreur**.

1. Pour `i = 0`, quelles cases sont comparées ? Pourquoi Python ne signale-t-il pas d'`IndexError` ?
2. Pourquoi le test du programmeur, `[8, 10, 12]`, ne pouvait-il pas voir ce bug ?

## 4 Déboguer et corriger

★★ **Exercice 5** *Enquêter avec des affichages*

1. ▶ Dans `premiere_sous`, ajouter `print("i =", i)` comme première ligne du corps de la boucle, puis lancer `premiere_sous([12, 14, 5], 10)`. Qu'observe-t-on ? Quelle est la cause ?
2. ▶ Dans `moyenne_ponderee`, afficher `somme` après la boucle et la valeur par laquelle on divise, pour l'appel `moyenne_ponderee([10, 16], [1, 2])`. Quelle est la bonne valeur du diviseur ?
3. Dérouler à la main `rang([15, 12, 15], 15)` : valeurs successives de `x` et de `r`. Pourquoi le résultat est-il juste quand il n'y a pas d'ex aequo ?

★★ **Exercice 6** *Tout corriger, tout relancer*

1. ▶ Corriger les sept fonctions, **une à la fois**, en relançant **tous** les tests après chaque correction. Retirer les affichages de débogage, garder les tests (et retirer les `# ECHOUE`).
2. Pour `arrondi_demi`, on peut ajouter `0.5` quelque part avant `int` : où exactement, et pourquoi cela transforme-t-il une troncature en arrondi ? Vérifier sur `12.2`, `12.25` et `12.8`.
3. Relancer le bulletin de Lina et le comparer à votre calcul de l'exercice 1.
4. Rédiger le **rapport de bugs** (sur le cahier ou en commentaire en tête de `bulletin.py`) : pour chaque fonction, le *symptôme* (le test qui échoue), la *cause* (la ligne fautive) et la *correction*.

▷ coup de pouce

## 5 Programmation défensive

★★ **Exercice 7** *Des garde-fous*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Écrire `notes_valides(notes)` qui renvoie `True` si toutes les notes sont entre 0 et 20, `False` sinon. La tester (penser à `0`, `20`, `-1`, `20.5` et au tableau vide).
2. ▶ Ajouter des `assert` avec message pour **toutes** les préconditions du cahier des charges : notes valides, même longueur pour `notes` et `coefs`, coefficients strictement positifs, `x` positif, `m` présente dans `moyennes`… Vérifier que `moyenne_ponderee([12, 25], [1, 1])` s'arrête avec un message clair.
3. Pour `moyenne`, le cahier des charges impose la valeur **sentinelle** `None` pour le tableau vide, et non un `assert`. Quel est alors le devoir de celui qui appelle `moyenne` ?
4. ▶ Ajouter au bulletin une dernière ligne `Premiere note sous la moyenne : devoir numero 2` (le numéro est l'indice + 1), affichée **seulement** si `premiere_sous(notes, 10)` ne renvoie pas `None`.

## 6 Prouver une boucle : invariant et variant

★★★ **Exercice 8** *La boucle de* `premiere_sous`*, version corrigée*

1. Compléter l'invariant : « au début de chaque tour, les notes d'indices `0` à `i - 1` sont toutes … ». Recopier et compléter cette phrase.
2. Montrer qu'il est vrai avant le premier tour, puis qu'il reste vrai d'un tour au suivant.
3. Si la fonction renvoie `None`, que vaut `i` à la sortie de la boucle ? En déduire, grâce à l'invariant, que la postcondition est respectée.
4. ▶ Traduire l'invariant en `assert` placé en tête du corps de la boucle (une petite boucle `for` sur les indices déjà vus), et relancer les tests.
5. Donner un **variant** qui prouve que la boucle corrigée se termine. Pourquoi la version d'origine n'en avait-elle pas ? ▷ coup de pouce

★★ **Exercice 9** *Un invariant pour un compteur*

Énoncer l'invariant de la boucle `for` de `nb_admis` corrigée (que vaut `nb` après avoir parcouru une partie des notes ?), et en déduire que la fonction renvoie le bon résultat.

## 7 Défi : au-delà de Python

★★★ **Exercice 10** *Le même calcul en C*

Le lycée voisin calcule ses moyennes avec ce programme en langage C.

```c
float moyenne(int notes[], int n) {
    int somme = 0;
    for (int i = 0; i < n; i++) {
        somme = somme + notes[i];
    }
    return somme / n;
}
```

1. Repérer ce qui est **commun** avec Python (constructions) et ce qui est **particulier** au C (syntaxe, déclarations).
2. Le typage du C est-il statique ou dynamique ? Qu'est-ce qui le montre dans le code ? Que se passerait-il, et *quand*, si l'on écrivait `somme = somme + "a";` ?

3. En C, la division de deux `int` est une division **entière** (comme `//` en Python). Que renvoie ce programme pour les notes `12` et `13` ? Quel test aurait démasqué le bug ? La correction consiste à écrire `(float) somme / n` : que fait, d'après vous, `(float)` ?
4. Le programme C est **compilé**, `bulletin.py` est **interprété**. Le compilateur aurait-il signalé le bug de la question 3 ? Et le bug d'indentation de `premiere_sous` ?

★★★ **Exercice 11** *Mille tests au hasard*

[ au-delà du programme ] On ne connaît pas à l'avance le bon arrondi d'un nombre tiré au hasard, mais on sait le **vérifier** : le résultat doit être un multiple de `0.5`, à au plus `0.25` de `x`.

1. ▶ Écrire l'**oracle** `verifie_arrondi(r, x)`, qui renvoie `True` si `r` est un arrondi acceptable de `x`. (`r` est un multiple de `0.5` si `r * 2` est un entier, c'est-à-dire si `int(r * 2) == r * 2`.)
2. ▶ Après `import random`, tirer 1 000 nombres au hasard, chacun par `x = random.randint(0, 2000) / 100`, et vérifier `arrondi_demi(x)` avec l'oracle, par un `assert`.
3. ▶ Avec la version d'origine (copiée sous un autre nom), l'`assert` arrêterait le programme au premier échec : le remplacer par un **compteur** des tirages où l'oracle renvoie `False`. Combien de tirages sur 1 000 la démasquent ?

## Bilan à rédiger

Sur le cahier, en une demi-page :

1. Les sept tests du programmeur passaient alors que les sept fonctions étaient fausses. Quelle règle du cours cette situation illustre-t-elle ? Quel point commun avaient ses tests ?
2. Parmi les outils du TP (docstring, jeu de tests, affichages, `assert` de précondition, invariant, variant), lequel vous a été le plus utile, et pour quel bug ?
3. Vos tests passent maintenant tous. Le module est-il pour autant *certainement* correct ? Justifier.

## Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 q. 3 – piste Penser aux valeurs *égales* à une frontière (une note de 10, deux moyennes égales, un `x` à égale distance), aux tableaux vides, à un tableau d'un seul élément, et à une première valeur qui joue un rôle particulier (la première note, le premier devoir). ↑

Ex. 6 q. 4 – piste Pour `rang`, deux changements : la valeur initiale de `r` et la comparaison. Pour `nb_progressions`, c'est le `range` qui doit changer, pas la comparaison. ↑énoncé

Ex. 8 q. 5 – piste Avant le premier tour, `i` vaut `0` : de quelles notes parle l'invariant ? Pour le variant, chercher une quantité entière, positive, qui diminue de 1 à chaque tour. ↑énoncé

# FICHE DE RÉVISION



## L'idée maîtresse

Un programme dit *comment* calculer, pas *ce qu'il* calcule ni s'il est *correct*. Bonnes habitudes : **spécifier**, **documenter**, **se défendre**, **tester**, **déboguer**, **raisonner** (invariants).

## Spécifier une fonction

- **prototype** : nom, paramètres, ce qui est renvoyé ;
- **précondition** : vraie *sur les arguments*, avant l'appel ;
- **postcondition** : garantie *sur le résultat*, si la précondition est respectée.

## Documenter

Une **docstring** par fonction (entre `"""..."""`, juste après `def`), lue par `help()`. Des **noms parlants** valent mieux qu'un commentaire ; ne commenter que l'inattendu.

```
def indice_maximum(t):
    """Renvoie l'indice du max de t, ou None si t est vide."""
    ...
```

## Programmation défensive

Deux stratégies (aussi bonnes selon le contexte) :

```
assert len(t) > 0, "le tableau est vide"   # 1) echouer tot et clairement
if len(t) == 0:                            # 2) renvoyer la valeur
    return None                            #    sentinelle None
```

« Absent » : `None` ou `-1` selon l'énoncé. Avec `-1`, **tester** avant d'utiliser : `t[-1]` est un indice valide.

## Tester

Des `assert` dans le fichier, après la fonction. Un **bon** jeu de tests couvre : chaque **cas** de la spéc, le **tableau vide**, les **valeurs limites** (0, négatifs), les deux réponses d'un **booléen**, les **doublons**.

```
assert indice_maximum([2, 3, 1]) == 1
assert indice_maximum([]) is None
```

## Déboguer

Un test qui échoue dit *qu'*il y a un bug, pas *où*. On **déroule** l'exécution et on ajoute des `print` temporaires (quels blocs, combien de fois, quelles valeurs). Après correction : retirer les affichages, **garder les tests**.

## Invariant de boucle

Propriété **vraie au départ** et **préservée à chaque tour** ⇒ vraie **à la sortie**. Se vérifie avec un `assert` pendant la mise au point.

```
while r >= b:
    assert a == q * b + r   # invariant (division_euclidienne)
    q += 1
    r -= b
```

### Diversité des langages

Un **algorithme** est indépendant du langage ; un **programme** le traduit. Mêmes **briques** partout (séquence, affectation, condition, boucle, fonction). Les langages diffèrent par :

- l'**exécution** : **compilé** (C : traduit une fois en exécutable, rapide) ou **interprété** (Python : traduit à la volée, souple) ;
- le **typage** : **statique** (vérifié avant l'exécution : C, OCaml) ou **dynamique** (pendant : Python — une erreur de type n'apparaît que si la ligne est atteinte) ;
- le **style** (paradigme) : impératif, fonctionnel, objet, logique, événementiel.

**HTML** = langage formalisé mais **pas** de programmation ; **SQL** = langage de requêtes, pas un langage de programmation généraliste. On réutilise du code via des **bibliothèques** : lire leur **documentation** suffit.



### Repères historiques

1947 : l'équipe du Mark II, dont fait partie Grace Hopper (pionnière des compilateurs), trouve un papillon de nuit dans un relais, le « premier vrai bug ». 1996 : Ariane 5 explose, un programme réutilisé d'Ariane 4 sans vérifier sa précondition. Dijkstra : « tester montre la présence de bugs, jamais leur absence ».

### Erreurs à éviter

- confondre précondition (arguments) et postcondition (résultat) ;
- utiliser le résultat d'une recherche (`-1` ou `None`) sans tester le cas « absent » ;
- `return` **dans** la boucle qui conclut au 1er tour ;
- croire un programme correct *parce que* les tests passent ;
- dire que C est interprété, ou appeler HTML un langage de programmation (SQL : un langage de requêtes).

### Savoir-faire à maîtriser — je sais…

☐ **spécifier** une fonction : prototype, **précondition** sur les arguments, **postcondition** sur le résultat, dans une **docstring** ;

☐ documenter un programme sans excès (noms parlants, commentaires utiles) ;

☐ protéger une fonction par un `assert` sur sa précondition, ou par une **valeur sentinelle** ;

☐ écrire un **jeu de tests** qui couvre les cas limites (tableau vide, extrémités, absent…) et expliquer pourquoi des tests réussis ne prouvent pas qu'un programme est correct ;

☐ **déboguer** : dérouler le programme à la main, placer des affichages (`print`) pour localiser l'erreur, puis les retirer en gardant les tests ;

☐ énoncer un **invariant de boucle** et vérifier qu'il est vrai au départ et préservé à chaque tour ;

☐ repérer les traits communs et particuliers de plusieurs langages (compilé ou interprété, typage statique ou dynamique) et utiliser la **documentation** d'une bibliothèque.

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 610

1. « Le résultat renvoyé est positif ou nul » : précondition ou postcondition ?
2. Que se passe-t-il à l'exécution de `assert len(t) > 0, "vide"` si `t = []` ?
3. Que renvoie un `minimum(t)` qui initialise `m = 0`, appelé sur `[3, 8, 5]` ?
4. Tous mes tests passent : mon programme est-il correct ?
5. Une recherche renvoie `-1` pour « absent » (convention de l'énoncé). Quel piège éviter avant d'utiliser `t[r]` ?

**Et avant ?** — *questions sur les chapitres précédents.* ▶ réponses p. 610

1. Avec quelle valeur initialiser un accumulateur qui calcule un produit ? (ch. 4)
2. Coder $-1$ sur 8 bits en complément à deux. (ch. 2)

# DÉFI ADVENT OF CODE

## DÉFI Trebuchet ?!

*l'étalonnage du trébuchet — chiffres dans une chaîne*

Jour 1

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p05-defi-aoc-2023-01` — **archive .zip**



### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2023/day/1`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Chaque ligne du fichier mélange des lettres et des chiffres ; il faut en extraire le premier et le dernier chiffre, former un nombre à deux chiffres, puis additionner ces nombres sur un millier de lignes. Chaque ligne est facile à traiter à la main, mais il y en a beaucoup trop, et la partie 2 ajoute des chiffres écrits en toutes lettres.
**Ce qu'on réinvestit.** La démarche de ce chapitre : **tester** chaque fonction sur des cas limites choisis exprès (une ligne à un seul chiffre, des mots qui se chevauchent…) avant de faire confiance au résultat. C'est indispensable ici, car la partie 2 cache un piège qui donne une réponse fausse sans aucun message d'erreur. On utilise aussi les chaînes, leurs tranches et la concaténation.
**La démarche de la fiche.**

1. Comprendre la règle avec des questions courtes, puis traiter à la main un petit fichier dont on connaît la réponse.
2. Lire le fichier et observer à quoi ressemblent les lignes.

3. Écrire une fonction qui extrait les chiffres d'une ligne, puis une fonction qui calcule la valeur d'une ligne : la somme donne la partie 1.
4. Pour la partie 2, modifier seulement la fonction qui extrait les chiffres, en la testant sur des lignes piégées.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `calibration value` | valeur d'étalonnage | `digit` | chiffre |
| `combine` | réunir, accoler | `two-digit number` | nombre à deux chiffres |
| `sum` | somme | `spelled out` | écrit en toutes lettres |
| `amended` | corrigé, modifié | `letters` | lettres |



### Résumé de l'énoncé (partie 1)

**Contexte.** Les elfes s'apprêtent à vous envoyer dans le ciel avec un trébuchet, mais son document de réglage a été « décoré » par un jeune elfe : à chaque ligne, les nombres utiles sont noyés au milieu de lettres et d'autres chiffres. Avant le lancement, il faut retrouver la valeur de réglage cachée dans chaque ligne.
**Les données.** Chaque ligne du fichier est une chaîne qui mêle des lettres minuscules et des chiffres, par exemple `x3yz7k`. Chaque ligne contient au moins un chiffre.
**La règle.**

- Dans une ligne, on repère le **premier** chiffre (le plus à gauche) et le **dernier** chiffre (le plus à droite) ; les chiffres du milieu ne servent pas.
- La valeur de la ligne est le nombre à deux chiffres obtenu en écrivant le premier chiffre puis le dernier. On accole les deux chiffres, on ne les additionne pas.
- S'il n'y a qu'un chiffre, il sert à la fois de premier et de dernier.

**La réponse attendue.** La somme des valeurs de toutes les lignes. Par exemple, la ligne `a4b7c2` vaut 42 (premier chiffre 4, dernier chiffre 2 ; le 7 ne sert pas) ; un fichier formé de cette seule ligne donnerait la réponse 42.

**Exercice 1** *Comprendre ce qui est demandé*

**But.** Vérifier sur des cas particuliers qu'on a bien compris comment se calcule la valeur d'une ligne.
**À quoi ça sert ?** Ces cas (un seul chiffre, des chiffres à accoler et non à additionner) deviendront les premiers tests de la fonction `valeur`. La dernière question annonce la façon de former le nombre en Python.

Après lecture du résumé (et de l'énoncé), vérifier sa compréhension sur ces cas particuliers :

1. Que vaut la ligne `a7b` ? et `k0z5` ?
2. Une ligne `x45y` vaut-elle 45, 4 ou 9 ?
3. Quel type de données a un caractère comme `"7"` ? Que donne `"7" + "2"` ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Calculer à la main la valeur de quatre lignes et leur somme.
**À quoi ça sert ?** On obtient un exemple dont on **connaît la réponse** (205) : le programme devra la retrouver avant d'être lancé sur les vraies données.

On considère le fichier (inventé) suivant :

```
x3yz7k
ab5cd
9
q12w34e
```

Donner la valeur de chaque ligne. Vérification : on doit obtenir `205`. Attention : la valeur de `q12w34e` n'est pas $12 + 34$.



## 2 Programmer la partie 1

### Exercice 3 *Lire le fichier*

**But.** Lire le fichier ligne par ligne et regarder quelques lignes de vos données.
**À quoi ça sert ?** Observer les vraies données avant de programmer évite les mauvaises surprises (longueur des lignes, caractères présents) et permet de vérifier que la lecture fonctionne.

#### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Rappel pour lire un fichier ligne par ligne :

```
fichier = open("input.txt", encoding="utf-8")
for ligne in fichier:
    ligne = ligne.strip()   # enlève le retour à la ligne final
fichier.close()
```

Afficher les 5 premières lignes de vos données pour voir à quoi elles ressemblent.

#### Méthode : reconnaître un chiffre, repérer un mot avec une tranche

Un caractère `c` est un chiffre lorsqu'il fait partie de la chaîne `"0123456789"` : le test `c in "0123456789"` vaut alors `True`. La comparaison `ligne[i:i + len(mot)] == mot` dit si `mot` apparaît dans `ligne` à partir de l'indice `i`.

```
print("4" in "0123456789", "q" in "0123456789")   # True False
ligne = "abcone2"
print(ligne[3:6] == "one")              # True : "one" commence a l'indice 3
print(ligne[2:5] == "one")              # False
```

**Intérêt.** Avec `len(mot)`, la tranche a toujours la bonne longueur, quel que soit le mot cherché ; en faisant varier `i`, on teste toutes les positions possibles de la ligne.
*À essayer.* Avec `ligne = "eightwo"`, que valent `ligne[0:5] == "eight"` et `ligne[4:7] == "two"` ? Les deux mots partagent-ils une lettre ?

### Exercice 4 *Les chiffres d'une ligne*

**But.** Écrire une fonction qui renvoie la liste des chiffres d'une ligne, dans l'ordre.
**À quoi ça sert ?** Une fois cette liste obtenue, le premier et le dernier chiffre sont simplement son premier et son dernier élément. C'est aussi la seule fonction qu'il faudra modifier dans la partie 2.

Écrire une fonction `chiffres(ligne)` qui renvoie la liste des chiffres de la ligne, dans l'ordre, sous forme de chaînes. Tester : `chiffres("q12w34e")` doit valoir `["1", "2", "3", "4"]`.

▷ coup de pouce

### Exercice 5 *Valeur d'une ligne et total*

**But.** Calculer la valeur d'une ligne à partir de ses chiffres, puis la somme sur tout le fichier.
**À quoi ça sert ?** La somme obtenue est la réponse de la partie 1. Comme `valeur` s'appuie sur `chiffres`, il suffira d'améliorer `chiffres` pour obtenir la partie 2 sans toucher au reste.

Écrire une fonction `valeur(ligne)` puis calculer la somme sur tout le fichier. Vérifier `205` sur l'exemple.

▷ coup de pouce ▷ début de solution



## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

**Résumé de la partie 2 (à lire après avoir validé la partie 1)**

**Ce qui change.** On s'aperçoit que certains chiffres étaient écrits en toutes lettres, en anglais, au milieu des autres lettres : ils comptaient eux aussi.
**La nouvelle règle.**

- Les mots `one`, `two`, `three`, `four`, `five`, `six`, `seven`, `eight`, `nine` comptent maintenant comme les chiffres 1 à 9.
- Le premier et le dernier chiffre d'une ligne peuvent donc être écrits avec un chiffre ou en lettres.
- La valeur d'une ligne se calcule comme avant, en accolant le premier et le dernier chiffre.

**La réponse attendue.** La somme des nouvelles valeurs de toutes les lignes. Par exemple, la ligne `xtwo3sevenx` vaut 27 : son premier chiffre est `two` et son dernier est `seven`.

### Exercice 6 *Des chiffres en toutes lettres*

**But.** Améliorer la fonction `chiffres` pour qu'elle reconnaisse aussi les chiffres écrits en toutes lettres, puis recalculer la somme.
**À quoi ça sert ?** La fonction `valeur` et le calcul du total restent inchangés : seule l'extraction des chiffres change. Les lignes de test proposées, construites exprès, permettent de détecter le piège avant de lancer le programme sur les vraies données.

Modifier la fonction `chiffres(ligne)` pour qu'elle reconnaisse aussi les chiffres écrits en toutes lettres, comme le demande la partie 2. Sur le fichier (inventé) ci-dessous, on doit obtenir `215`.

```
twone4x
7pqrsixteen
zoneight
nine87seven
```

**Piège** : deux mots peuvent partager des lettres (regarder `twone` et `zoneight`). Une méthode qui remplace les mots par des chiffres dans la chaîne risque de détruire l'un des deux mots : vérifier votre programme sur ces lignes.

▷ coup de pouce ▷ début de solution

## 4 Approfondissement

★★★ **Exercice 7** *Approfondissement 1 : chercher par les deux bouts*

**But.** Trouver le premier et le dernier chiffre sans construire la liste de tous les chiffres.
**À quoi ça sert ?** On ne fait que le travail nécessaire : une recherche depuis le début, une depuis la fin. Cette version est plus rapide et, en prime, elle élimine complètement le problème des mots qui se chevauchent.

On n'a besoin que du premier et du dernier chiffre.

1. À la main, sur `zoneight` : en partant de la fin, à quel indice trouve-t-on pour la première fois un chiffre ou le début d'un mot ?

2. Écrire une fonction `chiffre_en(ligne, i)` qui renvoie le chiffre (sous forme de chaîne) qui commence à l'indice `i`, ou `""` s'il n'y en a pas.

3. En déduire une version qui parcourt la ligne depuis le début jusqu'au premier chiffre trouvé, puis depuis la fin jusqu'au dernier, sans construire la liste complète. Pourquoi le chevauchement de mots ne pose-t-il alors plus aucun problème ? ▷ coup de pouce

### Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 – piste Le test `c in "0123456789"` vaut `True` si le caractère `c` est un chiffre. ↑énoncé

Ex. 5 – piste Le premier élément d'une liste `L` est `L[0]`, le dernier `L[-1]` (ou `L[len(L) - 1]`). ↑énoncé

Ex. 5 – début de solution Accoler deux **chaînes** avec `+` puis convertir : `int(L[0] + L[-1])`. Si l'on additionnait des entiers, on obtiendrait une somme et non un nombre à deux chiffres. ↑énoncé

Ex. 6 – piste Ranger les neuf mots anglais dans une liste `mots`, dans l'ordre, de sorte que `mots[0]` soit le mot de 1. Le chiffre correspondant au mot d'indice `k` est alors `str(k + 1)`. ↑énoncé

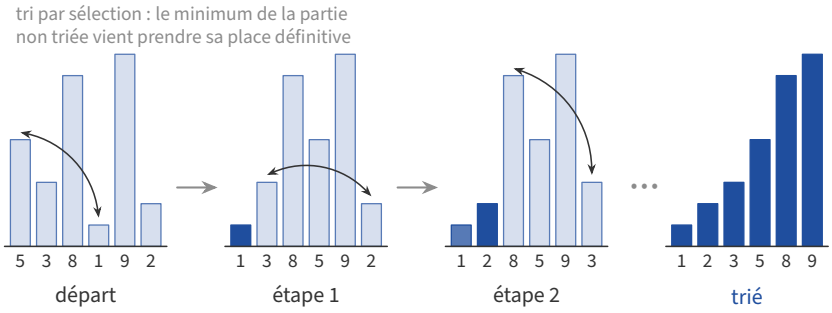
Ex. 6 – début de solution Plutôt que de modifier la ligne, la parcourir indice par indice : à chaque indice `i`, regarder si le caractère est un chiffre, puis, pour chaque mot de la liste, si la ligne contient ce mot **à partir de** l'indice `i` (une tranche de la bonne longueur : `ligne[i:i + len(mot)]`). ↑énoncé

Ex. 7 q. 3 – piste Deux boucles `while` : l'une avec un indice qui monte depuis `0`, l'autre avec un indice qui descend depuis `len(ligne) - 1`. Chacune s'arrête dès que `chiffre_en` renvoie autre chose que `""`. ↑énoncé

6

CHAPITRE 6

# Les algorithmes de tri




DANS CE CHAPITRE


- Activités préparatoires . . . 248
- Cours . . . 250
- Exercices . . . 259
- TP et projets . . . 267
- Fiche de révision . . . 272
- Défi Advent of Code . . . 274
- Corrigés . . . 611

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 Trier sans tricher

▶ corrigé p. 611

*ranger 8 cartes en comptant les comparaisons*

Durée : 20 min · Par binômes, sans machine, réponses sur le cahier

### Consignes

- Matériel : 8 cartes numérotées, posées **face cachée**.
- L'un est le **trieur**, l'autre l'**arbitre**. L'arbitre mélange les cartes et les aligne face cachée sur la table, aux positions 0 à 7.
- Le trieur **ne voit jamais** les valeurs. Il n'a droit qu'à deux actions :
  - **comparer** : il désigne deux cartes ; l'arbitre les regarde en cachette et dit laquelle est la plus petite. L'arbitre fait **un trait** à chaque comparaison ;
  - **déplacer** : il échange deux cartes, ou glisse une carte à un autre endroit de la ligne (gratuit, sans regarder).
- Objectif : ranger les cartes de la plus petite (à gauche) à la plus grande (à droite), avec **le moins de comparaisons possible**. À la fin, on retourne les cartes pour vérifier.
- *Variante :* 8 boîtes opaques de masses différentes et une balance à deux plateaux ; comparer, c'est peser deux boîtes l'une contre l'autre.

### Exercice 1 *Chacun sa méthode*

1. Le trieur range les cartes avec la méthode de son choix. Combien de comparaisons l'arbitre a-t-il comptées ? Le rangement est-il réussi ?
2. Décrire en deux ou trois phrases la méthode utilisée.
3. On échange les rôles et l'arbitre remélange. Le nouveau trieur essaie une **autre** méthode. Nombre de comparaisons ?
4. Relever les résultats des autres binômes. Quel est le plus petit nombre de comparaisons de la classe ? Une même méthode donne-t-elle toujours le même nombre ?

### Exercice 2 *Deux méthodes à l'essai*

**Méthode 1 — « le plus petit d'abord ».** On garde une carte « championne » et on la compare tour à tour à toutes les autres (si l'autre est plus petite, elle devient la championne) : on obtient la plus petite des 8, qu'on place en position 0. On recommence avec les 7 cartes restantes, et ainsi de suite.

5. Combien faut-il de comparaisons pour trouver la plus petite des 8 cartes ? la plus petite des 7 restantes ? Combien en tout ? Vérifier en appliquant la méthode.
6. Ce total dépend-il de l'ordre dans lequel l'arbitre a posé les cartes ?

**Méthode 2 — « comme une main de cartes ».** On prend les cartes une par une, de gauche à droite. Chaque nouvelle carte est comparée à ses voisines de gauche, de la plus proche à la plus lointaine, et glissée à sa place parmi les cartes déjà rangées : on s'arrête dès qu'on rencontre une carte plus petite qu'elle (ou qu'on arrive au bout).

**7.** L'arbitre pose les cartes **déjà dans l'ordre**. Nombre de comparaisons avec la méthode 2 ?

**8.** L'arbitre pose les cartes **dans l'ordre inverse**. Nombre de comparaisons ?

**9.** Les cartes sont *presque* rangées (seules deux voisines sont inversées). Quelle méthode choisir ? Pourquoi ?

### Exercice 3 *Ce qu'on a découvert*

**10.** Avec la méthode 1 et **16** cartes, combien faudrait-il de comparaisons ? Environ combien de fois plus qu'avec 8 cartes ?

**11.** **Ce qu'on a découvert.** Recopier et compléter sur le cahier : « Trier, c'est réorganiser les éléments pour qu'ils soient …, en n'utilisant que deux opérations : … deux éléments et les …. Pour savoir si une méthode est rapide, on compte … »

# COURS

**Programme (BO)** *« Algorithmes sur les tableaux »* : *tri par insertion*, *tri par sélection* ; *coût* d'un tri (coût *quadratique*).

**Prérequis** les boucles `for`/`while`, les tableaux (indices, `len`), et surtout le chapitre **« Algorithmique : le parcours séquentiel »** : patron du parcours, invariant de boucle, coûts linéaire et quadratique. → fiches de révision : ch. 4 p. 202

**Objectifs** comprendre *pourquoi* on trie ; connaître et savoir écrire **deux** algorithmes de tri (sélection, insertion) ; **tracer** leur exécution ; **évaluer leur coût** et les comparer.



**Situation de départ**

1. Pourquoi prendre la peine de ranger une liste dans l'ordre, si c'est seulement pour y chercher ensuite ?
2. Pour ranger un paquet de cartes, vaut-il mieux chercher à chaque fois la plus petite carte restante, ou glisser chaque nouvelle carte à sa place dans la main ?
3. Ranger 1 000 fois plus de cartes demande-t-il 1 000 fois plus de travail ?

**Remarque** — *Une histoire de rangement — et de vitesse*
Un annuaire de dix millions de noms qui ne serait pas trié serait *inutilisable* : on ne retrouverait jamais personne. Trier, c'est **préparer les données** pour qu'on puisse ensuite y chercher, compter, comparer… à toute vitesse. C'est l'une des opérations les plus exécutées au monde : chaque tri de colonne dans un tableur, chaque classement de résultats d'un moteur de recherche, chaque « trier par date » de vos messages en déclenche un. Dans ce chapitre, on ouvre le capot. On y retrouvera notre question fétiche : **combien ça coûte ?** Les points qui dépassent le programme sont signalés par le badge [ au-delà du programme ].

## I Pourquoi trier ?

Trier un tableau, c'est ranger ses éléments **dans l'ordre** (croissant, le plus souvent). Cela paraît anodin, mais un tableau trié devient une donnée bien plus **puissante** :

- **chercher devient fulgurant** : dans une liste triée on pourra faire une *recherche dichotomique* (chapitre suivant, *La recherche dichotomique*), incomparablement plus rapide qu'un parcours ;
- **le plus petit / le plus grand** sont aux deux bouts ; la **médiane** est au milieu ;
- les **doublons** se retrouvent *côte à côte*, donc faciles à détecter ;
- **fusionner** deux listes déjà triées, ou **afficher** un classement, devient immédiat.

**Règle 1 — L'idée-force**

On ne trie pas pour le plaisir de ranger : on trie **une fois** pour ensuite chercher, comparer et calculer **beaucoup plus vite**. Le tri est un *investissement*.

### Définition 1 — Le problème du tri

**Trier** un tableau `t`, c'est réorganiser ses éléments de sorte que

$$\texttt{t[0]} \leq \texttt{t[1]} \leq \texttt{t[2]} \leq \cdots \leq \texttt{t[n-1]}.$$

On travaillera **en place** : on réarrange le tableau donné, sans en créer un nouveau. Les seules opérations autorisées sont **comparer** deux éléments et les **échanger** (ou les déplacer).

### À retenir — Deux opérations à compter

Pour mesurer le coût d'un tri, on compte deux choses : le nombre de **comparaisons** (`t[j] < t[i]` ?) et le nombre d'**échanges**. C'est là que se joue la rapidité de l'algorithme.

### Règle 2 — Échanger : le piège de la variable perdue

Pour échanger le contenu de deux cases `t[i]` et `t[j]`, on **ne peut pas** écrire naïvement :

```
t[i] = t[j]     # PERDU : l'ancienne valeur de t[i] est ecrasee...
t[j] = t[i]     # ... donc ici, t[j] recopie la NOUVELLE valeur : rate !
```

▷ **Je prédis.** Avec `t = [4, 7]`, `i = 0` et `j = 1`, que contient `t` après ces deux lignes ?
▷ **Je vérifie.** `[7, 7]` : la valeur `4` a disparu, écrasée dès la première ligne.

Il faut d'abord **mettre de côté** l'une des deux valeurs dans une **variable temporaire** :

```
temp = t[i]     # on sauvegarde t[i]
t[i] = t[j]     # t[i] recoit t[j]
t[j] = temp     # t[j] recoit l'ancienne valeur de t[i]
```

C'est l'image des deux verres, l'un d'eau, l'autre de jus : pour intervertir leurs contenus, il faut un **troisième verre vide**. Cette variable temporaire est nécessaire dans la plupart des langages ; c'est la version que nous garderons.

**Remarque** — *En Python : l'échange en une ligne*
L'affectation multiple du chapitre *Les types construits* permet aussi d'écrire `t[i], t[j] = t[j], t[i]` : Python évalue d'abord le p-uplet de droite (les deux *anciennes* valeurs), puis le déballe dans les deux cases. Aucune valeur n'est perdue. On rencontre cette forme dans des sujets de bac (par exemple le tri par insertion d'Amérique du Nord 2026, jour 2, exercice 2) : il faut savoir la lire.

# II Le tri par sélection

**L'idée** est celle d'un joueur qui range sa main en la posant sur la table : *je cherche la plus petite carte de tout le paquet, je la mets en première position ; puis la plus petite du reste, en deuxième position ; et ainsi de suite.*

**Règle 3 — Principe du tri par sélection**

Pour chaque position `i` de gauche à droite : on **cherche le minimum** de la partie non encore triée (les cases d'indice `i` à la fin), puis on l'**échange** avec la case `i`. À la fin de l'étape, la case `i` contient sa valeur définitive.

↻ **Retour sur l'activité.** La méthode 1 de l'activité « Trier sans tricher », « le plus petit d'abord », était déjà le tri par sélection : la carte « championne » est le minimum de la partie non encore triée.



```
1  def tri_selection(t):
2      """Trie le tableau t en place, dans l'ordre croissant."""
3      n = len(t)
4      for i in range(n):                  # PARCOURIR : chaque position i
5          # 1. chercher l'indice du minimum dans la portion t[i:]
6          i_min = i                       # INITIALISER : champion provisoire
7          for j in range(i + 1, n):       # PARCOURIR : la partie non triee
8              if t[j] < t[i_min]:         # TESTER
9                  i_min = j               # METTRE A JOUR : nouveau champion
10         # 2. placer ce minimum en position i (echange via variable temporaire)
11         temp = t[i]                     # METTRE A JOUR : l'echange
12         t[i] = t[i_min]
13         t[i_min] = temp
```

▷ **Je prédis.** Pour un tableau de 4 éléments, combien de fois le test `t[j] < t[i_min]` est-il exécuté ? Ce nombre dépend-il de l'ordre des éléments ?

▷ **Je vérifie.** `6` fois $(3+2+1+0)$, et toujours `6` : les deux boucles `for` vont toujours jusqu'au bout, même si le tableau est déjà trié.

**Remarque** — *On reconnaît deux vieilles connaissances*

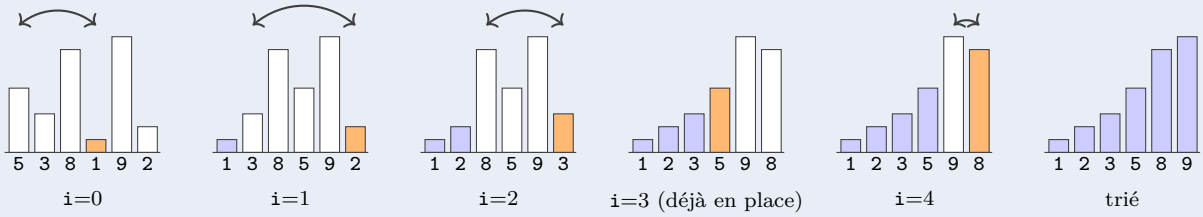
La boucle interne est *exactement* la recherche du **minimum** du chapitre *Algorithmique : le parcours séquentiel* (l'invariant du champion : `i_min` est l'indice du plus petit élément déjà vu). Ce minimum est ensuite « sélectionné » et rangé — d'où le nom.

## A. Dérouler l'algorithme

Traçons le tri de `[5, 3, 8, 1, 9, 2]`. Le tableau est montré **au début de chaque étape** : la zone `grisée` est **définitivement triée**, et on repère en `orange` le **minimum de la zone restante**, qu'on amène à sa place par un échange.

| étape | tableau | | | | | | action |
|---|---|---|---|---|---|---|---|
| i=0 | 5 | 3 | 8 | 1 | 9 | 2 | min = 1 (case 3) → échange avec la case 0 |
| i=1 | 1 | 3 | 8 | 5 | 9 | 2 | min = 2 (case 5) → échange avec la case 1 |
| i=2 | 1 | 2 | 8 | 5 | 9 | 3 | min = 3 (case 5) → échange avec la case 2 |
| i=3 | 1 | 2 | 3 | 5 | 9 | 8 | min = 5 : déjà en place |
| i=4 | 1 | 2 | 3 | 5 | 9 | 8 | min = 8 (case 5) → échange avec la case 4 |
| arrivée | 1 | 2 | 3 | 5 | 8 | 9 | tableau trié |

La même trace en **histogramme** : chaque barre a la hauteur de sa valeur. On voit la zone triée ( colorée ) grandir d'une barre à chaque étape, en partant de la gauche, et le minimum ( orange ) venir prendre sa place par un échange (flèche).




**Règle 4 — L'invariant du tri par sélection**

Juste avant de traiter la case `i`, la partie `t[0..i-1]` est **triée** *et* contient les `i` plus petits éléments du tableau, **à leur place définitive**. C'est cet invariant qui garantit qu'à la fin le tableau entier est trié. Il se démontre en deux étapes, comme au chapitre *Algorithmique : le parcours séquentiel*, et peut être vérifié par un `assert` pendant la mise au point, comme au chapitre *Spécifier et mettre au point ses programmes*.

Démonstration : (L'invariant du tri par sélection)
Notons $P$ la propriété : « `t[0..i-1]` est triée et contient les `i` plus petits éléments du tableau ».

- **Initialisation** (`i` $= 0$) : la partie `t[0..-1]` est *vide* ; une partie vide est triée et contient bien les 0 plus petits éléments. $P$ est vraie.
- **Hérédité** : supposons $P$ vraie au début de l'étape `i`. Comme `t[0..i-1]` contient les `i` plus petits éléments, chaque élément de la partie restante `t[i..n-1]` est supérieur ou égal à chacun d'eux. La boucle interne trouve le minimum de cette partie restante (invariant du champion) : c'est donc le $(\texttt{i}+1)^{\text{e}}$ plus petit élément du tableau, et il est supérieur ou égal à `t[i-1]`. L'échange le place en case `i` et ne touche qu'aux cases `i` et `i_min`, toutes deux hors de `t[0..i-1]`. Au début de l'étape suivante, `t[0..i]` est triée et contient les `i` $+1$ plus petits éléments : $P$ reste vraie.

Après la dernière étape (`i` $=$ `n` $-1$), $P$ porte sur `t[0..n-1]` : le tableau entier est trié. □

**Règle 5 — Terminaison du tri par sélection**

Le tri par sélection ne contient que deux boucles `for`, donc **bornées** : la boucle externe fait `n` tours, la boucle interne au plus `n` $-1$ à chacun d'eux. Le tri se termine toujours ; nul besoin de variant.

► **Exercices d'application :** exercices 1 à 5 et 22 (dérouler la sélection ; programmer le tri par sélection)

## III Le tri par insertion

**L'idée** est celle du joueur qui ramasse ses cartes *une par une* et garde sa main toujours triée : *je prends la carte suivante, et je la glisse à la bonne place parmi celles que je tiens déjà.* C'est le tri le plus « naturel » pour un humain.

### Règle 6 — Principe du tri par insertion

On parcourt le tableau de la gauche vers la droite. À chaque étape, la partie de gauche est *déjà triée* ; on prend l'élément suivant et on l'**insère** à sa place dans cette partie triée, en **décalant** vers la droite tous les éléments plus grands que lui.

↻ **Retour sur l'activité.** La méthode 2 de l'activité, « comme une main de cartes », était le tri par insertion : chaque carte, comparée à ses voisines de gauche, était glissée à sa place parmi les cartes déjà rangées.

```
1  def tri_insertion(t):
2      """Trie le tableau t en place, dans l'ordre croissant."""
3      n = len(t)
4      for i in range(1, n):              # PARCOURIR
5          x = t[i]                       # INITIALISER : l'element a inserer dans la partie triee
6          j = i - 1                      # INITIALISER
7          # decaler vers la droite tant que l'on rencontre plus grand que x
8          while j >= 0 and t[j] > x:     # PARCOURIR vers la gauche, tant que plus grand que x
9              t[j + 1] = t[j]            # METTRE A JOUR : decalage a droite
10             j = j - 1                  # METTRE A JOUR
11         t[j + 1] = x                   # METTRE A JOUR : on depose x dans le "trou" cree
```

▷ **Je prédis.** Combien de fois la ligne `t[j + 1] = t[j]` est-elle exécutée pour `[1, 2, 3, 4]` ? Et pour `[4, 3, 2, 1]` ?
▷ **Je vérifie.** `0` fois pour le tableau déjà trié (le `while` s'arrête aussitôt à chaque étape), `6` fois ($1+2+3$) pour le tableau à l'envers : contrairement à la sélection, l'insertion profite d'un tableau déjà rangé.

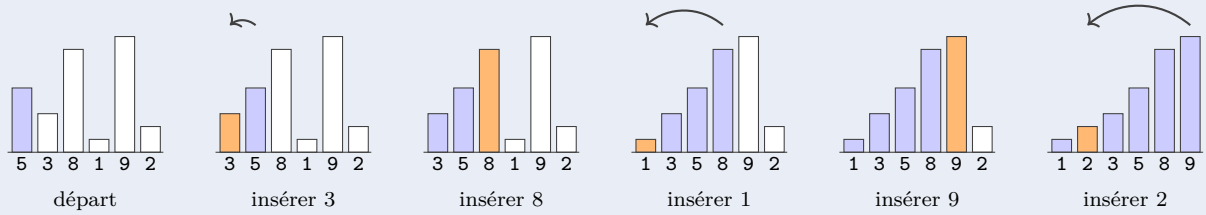
**Remarque** — *Le rôle du* `while`
La condition d'arrêt `t[j] > x` fait reculer `j` *seulement* tant qu'il y a des éléments plus grands à décaler. Dès qu'on trouve un élément *plus petit ou égal* (ou que `j` devient $-1$ : la condition `j >= 0` est alors fausse), on s'arrête : la place de `x` est trouvée. On range `x` juste après, en `t[j+1]`.

## A. Dérouler l'algorithme

Traçons le tri de `[5, 3, 8, 1, 9, 2]`. Le tableau est montré **après chaque insertion** : la zone `grisée` est **triée mais provisoire** (elle bougera encore), et l'élément `orange` est celui qu'on **vient d'insérer** à sa place.

| étape | tableau | | | | | | action |
|---|---|---|---|---|---|---|---|
| départ | 5 | 3 | 8 | 1 | 9 | 2 | la case 0 est triée à elle seule |
| insérer 3 | 3 | 5 | 8 | 1 | 9 | 2 | 3 s'insère avant 5 |
| insérer 8 | 3 | 5 | 8 | 1 | 9 | 2 | 8 est déjà bien placé |
| insérer 1 | 1 | 3 | 5 | 8 | 9 | 2 | 1 remonte tout au début |
| insérer 9 | 1 | 3 | 5 | 8 | 9 | 2 | 9 est déjà bien placé |
| insérer 2 | 1 | 2 | 3 | 5 | 8 | 9 | 2 se glisse en 2e position |

En **histogramme**, après chaque insertion : la zone triée ( colorée ) grandit d'une barre, mais ses barres *changent de place* ; la barre orange vient d'être glissée à sa place (la flèche part de sa case d'origine), et les barres plus grandes qu'elle ont été décalées d'un cran vers la droite.




**Règle 7 — L'invariant du tri par insertion**

Au début de l'étape `i`, la partie `t[0..i-1]` est formée des éléments qui étaient **initialement** en `t[0..i-1]`, rangés dans l'ordre **croissant**. Ce ne sont pas forcément les `i` plus petits du tableau : ils pourront être « poussés » plus loin par les insertions suivantes. Après le dernier tour ($\texttt{i} = \texttt{n} - 1$), la propriété vaut pour `t[0..n-1]` : le tableau entier est trié.

**Règle 8 — Terminaison du tri par insertion**

La boucle `for` est bornée. La boucle `while`, elle, ne l'est pas : il faut un **variant** (chapitre *Les bases de la programmation Python*, § V.B). La quantité `j` convient : c'est un entier, positif ou nul tant que la boucle tourne (la condition impose `j >= 0`), qui diminue de 1 à chaque tour (`j = j - 1`). La boucle `while` s'arrête donc après au plus `i` tours, et le tri se termine.

**À retenir — Sélection ou insertion : deux philosophies**

Le tri par **sélection** place, à chaque étape, un élément à sa position *définitive* (il ne bougera plus). Le tri par **insertion** garde une partie gauche toujours triée, mais qui *grandit et se réorganise* au fil de l'eau. Même résultat, chemins différents.

► **Exercices d'application :** exercices 6 à 10 et 23 (dérouler l'insertion, comparer les deux tris ; programmer le tri par insertion) ; exercice 24 (invariant et terminaison des deux tris)

## IV Combien ça coûte ?

Reprenons notre réflexe : on compte les opérations en fonction de la taille `n` du tableau. Or nos deux tris ont la même forme : **une boucle dans une boucle**. Le chapitre *Algorithmique : le parcours séquentiel* nous a appris à reconnaître ce motif…

**Règle 9 — Nos deux tris (sélection, insertion) sont quadratiques**

La sélection comme l'insertion consistent à parcourir le tableau *à l'intérieur* d'un autre parcours. Le nombre de comparaisons est de l'ordre de $n^2$ : leur coût est **quadratique**. Si l'on double le nombre d'éléments, le temps de tri est multiplié par **quatre**.

Démonstration : (Le compte exact pour la sélection)
Pour placer la case 0, on compare le minimum candidat aux $n-1$ éléments suivants. Pour la case 1, aux $n-2$ suivants. … Pour l'avant-dernière, à 1 seul. Le nombre total de comparaisons vaut

$$(n-1) + (n-2) + \cdots + 2 + 1 = \frac{n(n-1)}{2},$$

une quantité proportionnelle à $n^2$ : c'est bien un coût **quadratique**. Et ce, *quel que soit* le tableau de départ. □

Démonstration : (Le pire cas pour l'insertion)
Le pire cas est le tableau trié *à l'envers* : chaque élément est plus petit que tous ceux qui le précèdent. L'élément d'indice $i$ recule alors de $i$ cases : $i$ comparaisons et $i$ décalages (la boucle s'arrête quand `j` vaut $-1$). Au total,

$$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}$$

comparaisons, et autant de décalages : là encore, un coût **quadratique**. □

### A. Le meilleur atout de l'insertion

Les deux tris sont quadratiques *dans le pire des cas*. Mais l'insertion a une qualité que la sélection n'a pas :

**Règle 10 — Le cas favorable de l'insertion**

Si le tableau est **déjà trié** (ou presque), la boucle `while` du tri par insertion ne fait *aucun* (ou très peu de) décalage : elle s'arrête tout de suite, ou presque. Le tri se réduit alors à un **seul parcours** : son coût devient **linéaire** ! Le tri par sélection, lui, refait bêtement tout son travail : il est quadratique *même sur un tableau déjà trié*.

↻ **Retour sur l'activité.** Dans l'activité, la méthode 1 faisait toujours $7 + 6 + \cdots + 1 = 28$ comparaisons pour 8 cartes, quel que soit l'ordre de départ ; la méthode 2 n'en faisait que 7 sur des cartes déjà rangées, mais 28 sur des cartes dans l'ordre inverse.

| Tri | meilleur cas | pire cas | échanges |
|---|---|---|---|
| Sélection | $\approx n^2$ (toujours) | $\approx n^2$ | peu ($\approx n$) |
| Insertion | $\approx n$ (déjà trié !) | $\approx n^2$ | beaucoup (décalages) |

**Pour les curieux** — *Concrètement, ce que « quadratique » veut dire* (facultatif)
Pour $n = 1\,000$ éléments, un tri quadratique fait de l'ordre de 500 000 comparaisons : instantané. Mais pour $n = 1\,000\,000$, il en faudrait de l'ordre de $5 \times 10^{11}$ : **des minutes, voire des heures**. C'est pourquoi les vrais logiciels utilisent des tris plus malins (coût $n \log_2 n$), que vous découvrirez en terminale.

► **Exercices d'application :** exercices 11 à 13 (le coût d'un tri)

## V D'autres tris existent

Sélection et insertion ne sont que deux membres d'une grande famille.

- Le **tri à bulles** fait « remonter » les grandes valeurs vers la fin par échanges de voisins successifs. Simple à comprendre, mais rarement efficace — vous le programmerez en exercice guidé.
- Le **tri fusion** et le **tri rapide** atteignent un coût $n \log_2 n$, bien meilleur : ce sont eux (et leurs cousins) qui trient « pour de vrai » dans les logiciels. [au-delà du programme]
- En Python, il suffit d'écrire `t.sort()` (tri en place) ou `sorted(t)` (nouveau tableau trié). Derrière ces mots se cache **Timsort**, un algorithme hybride très rapide inventé en 2002 par Tim Peters, aujourd'hui utilisé aussi par Java et Android. [au-delà du programme]

**Pour les curieux** — *Danser un algorithme* (facultatif)
Pour *voir* un tri prendre vie, cherchez les vidéos « **AlgoRythmics** » : l'université Sapientia de Transylvanie (à Târgu Mureş, en Roumanie) y met en scène des **danses folkloriques** qui exécutent, pas à pas, le tri par sélection, par insertion, à bulles ou le tri rapide. C'est étrangement hypnotique — et bien plus parlant qu'un long discours.

**Pour les curieux** — *Le pire tri du monde, pour rire* (facultatif)
Le **bogosort** (« tri stupide ») consiste à… mélanger le tableau au hasard, puis vérifier s'il est trié ; sinon, recommencer. Sur un jeu de 52 cartes, il faudrait en moyenne de l'ordre de $8 \times 10^{67}$ tirages (autant que d'ordres possibles pour les 52 cartes), bien plus qu'il n'y a d'atomes dans toute la Terre (environ $10^{50}$). À ne *jamais* utiliser — mais il rappelle avec humour qu'un algorithme *correct* peut être catastrophiquement *lent*.

► **Exercices d'application :** exercice 14 (le tri à bulles)



## VI Un peu d'histoire

**Pour les curieux** — *Trier, aux origines de l'informatique* (facultatif)

Trier des données mécaniquement est *antérieur* aux ordinateurs : dès le recensement américain de **1890**, les machines à cartes perforées de **Herman Hollerith** (dont l'entreprise deviendra IBM) dépouillaient des millions de fiches : à chaque carte lue, le couvercle d'un casier s'ouvrait et l'opérateur y rangeait la carte. Dès **1901**, des trieuses automatiques faisaient tomber chaque carte dans un casier selon la position d'un trou. En **1945**, dans l'un des tout premiers programmes de l'histoire, **John von Neumann** écrit la routine de *fusion*, cœur du tri fusion, pour la machine EDVAC : le tri fut ainsi l'un des premiers problèmes que l'on chercha à faire résoudre par une machine à programme enregistré. Depuis, il reste un terrain d'étude central : c'est sur des problèmes simples à énoncer comme « ranger dans l'ordre » que l'informatique a forgé sa science du **coût** des algorithmes.



*Une trieuse de cartes perforées Hollerith*



*L'EDVAC, en service à partir de 1951*

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** Le tri par sélection n'est qu'une suite de recherches du minimum (*Algorithmique : le parcours séquentiel*), justifiée par un **invariant de boucle** (*Spécifier et mettre au point ses programmes*).
- **Ce qu'il prépare.** Trier rendra possible la **recherche dichotomique** ; trier une table selon une colonne servira dans *Les données en tables*, *Les algorithmes gloutons* et *Les k plus proches voisins*.
- **En Terminale.** *Diviser pour régner* présentera le **tri fusion**, en $n \log_2 n$, bien plus rapide que nos tris quadratiques sur de grands tableaux.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Trier | ranger dans l'ordre croissant, **en place** (comparaisons + échanges) |
| Pourquoi trier | chercher/comparer vite ensuite (dichotomie, médiane, doublons…) |
| Tri par sélection | chercher le **min** du reste, le placer ; position définitive à chaque étape |
| Tri par insertion | **insérer** chaque élément dans la partie gauche déjà triée (décalages) |
| Invariant (sélection) | `t[0..i-1]` triée, formée des `i` plus petits (place définitive) |
| Invariant (insertion) | `t[0..i-1]` : ses éléments de départ, rangés dans l'ordre |
| Terminaison | boucles `for` bornées ; variant `j` pour le `while` de l'insertion |
| Échanger | `temp = t[i]` ; `t[i] = t[j]` ; `t[j] = temp` (Python : `t[i], t[j] = t[j], t[i]`) |
| Coût des deux tris | **quadratique** ($\approx n^2$) : une boucle dans une boucle |
| Atout de l'insertion | **linéaire** si le tableau est déjà (presque) trié |
| En pratique | `t.sort()` / `sorted(t)` : Python utilise Timsort ($n \log_2 n$) |

### Retour à la situation de départ

6

1. Trier est un **investissement** : une fois le tableau trié, chercher devient fulgurant (recherche dichotomique), le minimum et le maximum sont aux deux bouts, les doublons côte à côte (§ I).
2. Ce sont le **tri par sélection** et le **tri par insertion** (§ II et III). Tous deux sont quadratiques dans le pire cas, mais l'insertion devient **linéaire** sur une main déjà presque rangée, alors que la sélection fait toujours $n(n-1)/2$ comparaisons (§ IV).
3. Non : le coût est **quadratique**, de l'ordre de $n^2$ comparaisons. Pour 1 000 fois plus de cartes, il faut environ un million de fois plus de comparaisons (§ IV).

## Erreurs fréquentes

- **Échanger deux valeurs sans variable temporaire.** `t[i] = t[j]` puis `t[j] = t[i]` **perd** la première valeur. *Le réflexe :* `temp = t[i]` ; `t[i] = t[j]` ; `t[j] = temp` (ou, en Python, `t[i], t[j] = t[j], t[i]`).
- **Se tromper de bornes.** Le dernier indice est `len(t)-1` ; attention aux `range` du tri.
- **Perdre de vue l'élément « en cours de tri ».** À chaque passe, on place **un** élément (le minimum choisi / l'élément inséré).
- **Tri par insertion : mal décaler.** On décale vers la droite les éléments **plus grands** avant de poser l'élément à sa place.
- **Croire qu'un tri quadratique passe partout.** Sur de grands tableaux, $n^2$ devient énorme. *Le réflexe :* en pratique, `sort()` / `sorted()` (Timsort, $n \log_2 n$).

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

- ☐ **dérouler** à la main un tri par sélection et un tri par insertion ;
- ☐ écrire le **tri par sélection** (recherche de l'indice du minimum, puis échange) ;
- ☐ écrire le **tri par insertion** ;
- ☐ **échanger** deux valeurs avec une variable temporaire ;
- ☐ énoncer l'**invariant** de chaque tri et justifier sa **terminaison** ;
- ☐ justifier le **coût quadratique** ($\approx n^2$, soit $\frac{n(n-1)}{2}$ comparaisons) ;
- ☐ reconnaître l'**atout de l'insertion** sur un tableau presque trié ;
- ☐ savoir qu'en pratique on utilise `sort()` / `sorted()`.

# EXERCICES

**Mode d'emploi**

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et tester* en Python. Le symbole `>>>` figure une saisie dans la console.
- **Sauf mention contraire**, on n'utilise **pas** `sort`, `sorted`, `min` ni `max` : le but est d'écrire le **tri** soi-même. On trie **en place** (le tableau donné est modifié).
- **Réflexe coût** : pour chaque tri, se demander combien de *comparaisons* il effectue.

## Le tri par sélection ↑ cours



★ **Exercice 1** *Trace du tri par sélection* ▶ corrigé p. 612 • essentiel

Sans machine, on trie `[7, 2, 5, 3, 8, 1]` par **sélection**. Sur le cahier, dresser un tableau de trace avec une ligne par étape `i` : l'état du tableau **au début** de l'étape, puis le minimum sélectionné dans la partie non triée (valeur et case). Terminer par une ligne « arrivée » qui donne le tableau trié. ▷ coup de pouce

★ **Exercice 2** *L'ingrédient : trouver le minimum* ▶ corrigé p. 612

▶ Écrire `indice_min(t, debut)` qui renvoie l'indice du plus petit élément de la portion `t[debut:]` (des indices `debut` jusqu'à la fin). Cette fonction est le cœur du tri par sélection. ▷ coup de pouce

```
>>> indice_min([9, 4, 7, 2, 8], 1)
3
```

★ **Exercice 3** *Remettre le tri par sélection dans l'ordre* ▶ corrigé p. 612

Voici les lignes de la fonction `tri_selection` du cours (recherche du minimum écrite en entier), **mélangées** et **sans indentation**. Les remettre dans le bon ordre et les indenter correctement, puis vérifier sur machine que `[5, 3, 8, 1]` devient `[1, 3, 5, 8]`. ▷ coup de pouce

```
t[i_min] = temp
for j in range(i + 1, n):
i_min = j
def tri_selection(t):
temp = t[i]
for i in range(n):
if t[j] < t[i_min]:
n = len(t)
t[i] = t[i_min]
i_min = i
```

★★ **Exercice 4** *Le tri par sélection* ▶ corrigé p. 613 • essentiel

▶ En utilisant la fonction `indice_min` de l'exercice « L'ingrédient : trouver le minimum », écrire `tri_selection(t)` qui trie le tableau `t` **en place** dans l'ordre croissant. Tester sur plusieurs exemples, dont un tableau vide et un tableau déjà trié. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 5** *Dans l'autre sens* ▶ corrigé p. 613

▶ Écrire `tri_selection_decroissant(t)` qui trie `t` du plus grand au plus petit. *Par rapport au tri par sélection du cours (où la recherche du minimum est écrite en entier), un seul caractère change : lequel ?* ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Le tri par insertion ↑ cours

★ **Exercice 6** *Trace du tri par insertion* ▶ corrigé p. 614 • essentiel

Même tableau `[7, 2, 5, 3, 8, 1]`, trié cette fois par **insertion**. Donner l'état du tableau **après chaque insertion** (5 étapes), en soulignant la partie déjà triée. ▷ coup de pouce

★ **Exercice 7** *Compléter le tri par insertion* ▶ corrigé p. 614

Compléter les cinq trous `.....` de cette fonction de tri par insertion, puis la tester sur `[4, 2, 5, 1]`. ▷ coup de pouce

```
def tri_insertion(t):
    n = len(t)
    for i in range(1, .....):
        x = t[i]                # l'element a inserer
        j = .....
        while j >= 0 and ..... :
            t[j + 1] = .....    # decaler vers la droite
            j = j - 1
        t[.....] = x            # poser x dans le trou
```

★★ **Exercice 8** *Qui a fait quoi ?* ▶ corrigé p. 614

On a photographié un tableau *en cours* de tri, juste après une étape :

`[1, 2, 3, 8, 5, 9, 4, 7]`

On sait que le tri est soit une sélection, soit une insertion, et qu'on trie de gauche à droite. En observant la partie `[1, 2, 3]`, peut-on être sûr de laquelle il s'agit ? Justifier. ▷ coup de pouce

★★ **Exercice 9** *Le tri par insertion* ▶ corrigé p. 614 • essentiel

▶ Écrire `tri_insertion(t)` qui trie `t` **en place** par insertion. Vérifier sur `[5, 3, 8, 1, 9, 2]` qu'on obtient `[1, 2, 3, 5, 8, 9]`. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 10** *Compter les décalages* ▶ corrigé p. 615

Modifier `tri_insertion` en `tri_insertion_compte(t)` qui renvoie le **nombre total de décalages** effectués (les affectations `t[j+1] = t[j]`). ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. Combien de décalages sur `[1, 2, 3, 4, 5]` (déjà trié) ?
2. Combien sur `[5, 4, 3, 2, 1]` (trié à l'envers) ? Que remarque-t-on ?

## Le coût d'un tri ↑cours

★ **Exercice 11** *Linéaire ou quadratique ?* ▶ corrigé p. 615 • essentiel

Pour chacune des affirmations, répondre par vrai ou faux et justifier.

1. Le tri par sélection fait le même nombre de comparaisons, que le tableau soit trié ou non.
2. Le tri par insertion est plus rapide sur un tableau déjà trié que sur un tableau quelconque de même taille.
3. Doubler la taille d'un tableau multiplie par 2 le temps d'un tri quadratique. ▷ coup de pouce

★★ **Exercice 12** *Compter les comparaisons de la sélection* ▶ corrigé p. 615

▶ Écrire `nb_comparaisons_selection(t)` qui trie `t` par sélection **et** renvoie le nombre de comparaisons `t[j] < t[i_min]` effectuées.

1. Combien vaut ce nombre pour un tableau de 5 éléments ? de 10 ?
2. Vérifier expérimentalement la formule $\frac{n(n-1)}{2}$ vue en cours. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 13** *Vérifier qu'un tableau est trié* ▶ corrigé p. 616 • essentiel

▶ Écrire `est_trie(t)` qui renvoie `True` si `t` est trié dans l'ordre croissant, `False` sinon (s'arrêter dès qu'on trouve une « descente »). Quel est son coût ? À quoi peut servir cette fonction pour *tester* vos tris ? ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Exercice guidé : le tri à bulles ↑cours

★★ **Exercice 14** *Construire le tri à bulles* ▶ corrigé p. 616

Le **tri à bulles** compare les éléments *voisins* et les échange s'ils sont dans le désordre. À force de passages, les grandes valeurs « remontent » comme des bulles vers la fin du tableau.

1. ▶ **Un passage.** Écrire `un_passage(t)` qui parcourt `t` de gauche à droite et, pour chaque couple de voisins `t[j]` et `t[j+1]`, les échange si `t[j] > t[j+1]`. Appliquer `un_passage` à `[5, 3, 8, 1, 9, 2]` : quel élément se retrouve à coup sûr en dernière position ? ▷ coup de pouce
2. **Pourquoi « à sa place » ?** Expliquer en une phrase pourquoi, après un passage complet, le **plus grand** élément est forcément tout à droite. ▷ coup de pouce
3. ▶ **Le tri complet.** En répétant les passages, écrire `tri_bulle(t)`. Combien de passages suffisent pour un tableau de `n` éléments ? *(Remarque : le i-ème passage n'a plus besoin d'aller jusqu'au bout, puisque la fin est déjà triée.)* ▷ début de solution
4. ▶ **Le drapeau malin.** [ au-delà du programme ] Si un passage n'effectue **aucun** échange, c'est que le tableau est déjà trié : inutile de continuer. Écrire `tri_bulle_optimise(t)` qui s'arrête dans ce cas, à l'aide d'une variable booléenne `echange`. Sur un tableau déjà trié, combien de passages fait-elle ? ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Synthèse : vers le bac et au-delà ↑ cours

★★ **Exercice 15** *Classement d'une compétition* ▶ corrigé p. 617

Chaque concurrent est décrit par un p-uplet `(nom, temps)` et le tableau `resultats` les contient dans le désordre.

1. ▶ Adapter le tri par sélection en `classer(resultats)` qui trie le tableau **en place** par **temps croissant** (le plus rapide en premier), puis le renvoie (pour l'affichage ci-dessous). *On compare désormais les temps* `resultats[j][1]`. ▷ coup de pouce
2. ▶ Écrire `podium(resultats)` qui renvoie les **trois** premiers noms. ▷ coup de pouce

```
>>> classer([("Ana", 53), ("Bob", 47), ("Cid", 61)])
[('Bob', 47), ('Ana', 53), ('Cid', 61)]
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 16** *La médiane* ▶ corrigé p. 618

▶ La **médiane** d'une série de nombres est la valeur du milieu quand on les range dans l'ordre. Écrire `mediane(t)` qui trie une **copie** de `t` (pour ne pas modifier l'original — utiliser une compréhension `[x for x in t]`) puis renvoie l'élément central (on suppose `len(t)` **impair**). En quoi trier rend-il ce calcul facile ? ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 17** *Chasse aux doublons* ▶ corrigé p. 618 ≈ 15 min

▶ Écrire `a_un_doublon(t)` qui renvoie `True` si deux éléments de `t` sont égaux. **Stratégie maligne** : trier une copie du tableau, puis se contenter d'un **seul** parcours pour comparer chaque élément à son *voisin*. Une fois le tri fait, quel est le coût du reste de l'algorithme ? ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 18** *Presque trié* ▶ corrigé p. 618 ≈ 20 min

On dit qu'un tableau est *presque trié* si chaque élément est à au plus une case de sa position finale. On donne à trier un tel tableau : `[2, 1, 4, 3, 6, 5]`.

1. Combien de *comparaisons* (et de décalages) le tri par insertion effectue-t-il ? Le comparer au tri par sélection. ▷ coup de pouce ▷ début de solution
2. En déduire lequel des deux tris on préférerait pour des données *déjà presque en ordre* (par exemple une liste triée dans laquelle on vient d'ajouter quelques éléments). [ au-delà du programme ]

★★★ **Exercice 19** *Les $k$ plus petits* ▶ corrigé p. 618 ≈ 20 min

▶ Pour afficher les $k$ meilleurs temps d'une course de $n$ coureurs, inutile de tout trier.

```
>>> k_plus_petits([53, 47, 61, 50, 49, 58], 3)
[47, 49, 50]
```

1. Écrire `k_plus_petits(t, k)` qui renvoie un tableau contenant, dans l'ordre croissant, les `k` plus petits éléments de `t` (on suppose `k <= len(t)`). On s'interdit de trier le tableau en entier : on n'effectue que les `k` premières étapes d'un tri par sélection, sur une copie de `t`.
2. Combien de comparaisons sont effectuées pour $k = 3$ et $n = 100$ ? Comparer avec les $\frac{n(n-1)}{2}$ d'un tri complet.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 20** *Trier des mots par longueur* ▶ corrigé p. 619 ≈ 30 min

On veut ranger des mots du plus court au plus long, **sans** changer l'ordre de départ des mots de même longueur.

```
>>> mots = ["pomme", "kiwi", "figue", "noix", "ananas"]
>>> tri_longueur(mots)
>>> mots
['kiwi', 'noix', 'pomme', 'figue', 'ananas']
```

1. ▶ Adapter le tri par insertion en `tri_longueur(mots)`, qui compare les **longueurs** des mots. Vérifier qu'on obtient bien la sortie ci-dessus.
2. Dans la condition du `while`, que se passerait-il si l'on remplaçait `>` par `>=` ? Le vérifier sur l'exemple et expliquer.
3. **Sans machine** : le tri par sélection respecte-t-il, lui aussi, l'ordre de départ des mots de même longueur ? Chercher un exemple de trois mots qui tranche la question.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 21** *Un tri selon un assistant d'IA* ▶ corrigé p. 619

Un élève demande à un assistant d'IA : « Écris le tri par sélection en Python, en place. » Voici la réponse obtenue.

```
1  def tri_selection(t):
2      n = len(t)
3      for i in range(n):
4          i_min = i
5          for j in range(i + 1, n):
6              if t[j] < t[i_min]:
7                  i_min = j
8          t[i] = t[i_min]        # on place le minimum en position i
```

« À chaque étape `i`, la boucle interne trouve l'indice du minimum de la partie non triée, puis ce minimum est placé en position `i`. Après `n` étapes, le tableau est trié. Le coût est quadratique. »

1. La réponse est-elle correcte ? Dérouler `tri_selection(t)` à la main sur `t = [5, 2, 4, 1]`, en écrivant le tableau après chaque étape `i`.
2. Localiser et corriger l'erreur.
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce

## Entraînement ↑ cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

★ **Exercice 22** *Dérouler le tri par sélection* ▶ corrigé p. 620 • essentiel

Sans machine, trier par **sélection** chacun des tableaux suivants : à chaque étape, donner le minimum sélectionné et l'état du tableau après l'échange. a) `[4, 1, 3, 2]` ; b) `[9, 7, 5, 3]` ; c) `[3, 1, 3, 2]`.

▷ coup de pouce

★ **Exercice 23** *Dérouler le tri par insertion* ▶ corrigé p. 620 • essentiel

Sans machine, trier par **insertion** chacun des tableaux suivants : donner l'état du tableau après chaque insertion, et compter le nombre total de décalages. a) `[4, 1, 3, 2]` ; b) `[1, 2, 4, 3]` ; c) `[9, 7, 5, 3]`.

▷ coup de pouce

## Prouver un tri ↑cours

★★ **Exercice 24** IA IA IA *Invariant et terminaison* ▶ corrigé p. 620 ≈ 20 min

Sans machine. On trie le tableau `t = [6, 4, 9, 2, 7]`.

1. **Sélection.** Donner l'état du tableau au **début** des étapes `i = 1`, 2, 3 et 4. Vérifier, à chaque fois, l'invariant du cours : « `t[0..i-1]` est triée et contient les `i` plus petits éléments du tableau ».
2. **Insertion.** Même travail avec le tri par insertion et son invariant : « `t[0..i-1]` est formée des éléments initialement en `t[0..i-1]`, rangés dans l'ordre croissant ».
3. Au début de l'étape `i = 3` du tri par insertion, la partie `t[0..2]` contient-elle les trois plus petits éléments du tableau ? Pourquoi cela ne contredit-il pas l'invariant de l'insertion ? Lequel des deux invariants permet d'affirmer qu'un élément placé ne bougera plus ?
4. Expliquer pourquoi le tri par sélection se **termine** toujours.
5. Dans le tri par insertion, justifier à l'aide d'un **variant** que la boucle `while` se termine. Combien de tours fait-elle, au plus, à l'étape `i` ? Combien en fait-elle réellement à chaque étape sur `t` ?

▷ coup de pouce

6

### Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Reprendre la présentation de « Dérouler l'algorithme » du cours : à l'étape `i`, on cherche le minimum parmi les cases `i` à la fin seulement, puis on l'échange avec la case `i`. ↑énoncé

Ex. 2 – piste C'est la recherche de la position du champion (chapitre sur le parcours séquentiel), mais en partant de la case `debut` au lieu de la case `0`. ↑énoncé

Ex. 3 – piste Deux temps à chaque étape `i` : chercher l'indice du minimum (une boucle sur `j`), *puis* échanger avec une variable temporaire. Les lignes de l'échange viennent-elles dans la boucle sur `j` ou après ? ↑énoncé

Ex. 4 – piste À l'étape `i`, où faut-il chercher le minimum ? La fonction `indice_min` répond exactement à cette question ; il reste à échanger deux cases (variable temporaire). ↑énoncé

Ex. 4 – début de solution
```
def tri_selection(t):
    for i in range(len(t)):
        i_min = indice_min(t, i)
        ... (échange de t[i] et t[i_min])
```
↑énoncé

Ex. 5 – piste Pour ranger du plus grand au plus petit, quel élément de la partie non triée faut-il amener en position `i` à chaque étape ? ↑énoncé

Ex. 6 – piste À chaque étape, on prend le premier élément de la partie non triée et on le glisse à sa place dans la partie triée de gauche, comme une carte dans une main déjà rangée. ↑énoncé

Ex. 7 – piste `j` part de la case juste à gauche de `i`. On décale tant que l'élément examiné est plus grand que `x`. Quand la boucle s'arrête, `x` se pose juste à droite de la case `j`. ↑énoncé

Ex. 8 – piste Dans chacun des deux tris, la partie triée de gauche contient-elle forcément les plus petits éléments du tableau ? Le tableau de départ est-il connu ? ↑énoncé

Ex. 9 – piste Mettre de côté l'élément à insérer dans une variable `x`, faire reculer un indice `j` tant qu'on rencontre plus grand que `x`, sans oublier de poser `x` à la fin. ↑énoncé

Ex. 9 – début de solution
```
def tri_insertion(t):
    for i in range(1, len(t)):
        x = t[i]
        j = i - 1
```
↑énoncé

Ex. 10 – piste Un compteur initialisé à `0` avant les boucles ; où se trouve, dans le code, l'instruction qui réalise un décalage ? ↑énoncé

**Ex. 11 q. 3 – piste** Pour la 3 : si le temps est proportionnel à $n^2$, que devient $n^2$ quand on remplace $n$ par $2n$ ? ↑énoncé

**Ex. 12 q. 2 – piste** Repartir du tri par sélection du cours, où la recherche du minimum est écrite en entier : la comparaison à compter y apparaît une seule fois. Un compteur augmenté de 1 à chaque fois qu'elle est évaluée suffit. ↑énoncé

**Ex. 13 – piste** Comparer chaque élément à son voisin de droite : quel est le dernier indice `i` pour lequel `t[i + 1]` existe ? ↑énoncé

**Ex. 14 q. 1 – piste** Dernier couple de voisins : `t[len(t) - 2]` et `t[len(t) - 1]`. Jusqu'où doit donc aller `j` ? ↑énoncé

**Ex. 14 q. 2 – piste** Suivre le plus grand élément dès que le parcours l'atteint : que lui arrive-t-il à chacune des comparaisons suivantes ? ↑énoncé

**Ex. 14 q. 3 – début de solution**
```
def tri_bulle(t):
    n = len(t)
    for i in range(n - 1):
        for j in range(n - 1 - i):
```
↑énoncé

**Ex. 14 q. 4 – piste** À quel moment remettre `echange` à `False` ? À quel moment la passer à `True` ? Où la tester ? ↑énoncé

**Ex. 15 q. 1 – piste** Recopier le tri par sélection du cours : seule la ligne de comparaison change, l'échange déplace des p-uplets entiers. ↑énoncé

**Ex. 15 q. 2 – piste** Une fois le tableau classé, où sont les trois plus rapides ? Dans un p-uplet `(nom, temps)`, à quel indice est le nom ? ↑énoncé

**Ex. 16 – piste** Dans un tableau trié de taille impaire, à quel indice se trouve l'élément central ? Essayer avec 3, puis 5 éléments, et relier cet indice à `len(t)`. ↑énoncé

**Ex. 17 – piste** Après le tri, où se trouvent deux valeurs égales l'une par rapport à l'autre ? Il suffit alors de comparer chaque case à la suivante. ↑énoncé

**Ex. 17 – début de solution**
```
copie = [x for x in t]
tri_selection(copie)
for i in range(len(copie) - 1):
    ...
```
↑énoncé

**Ex. 18 q. 1 – piste** Dérouler l'insertion à la main : de combien de cases chaque élément inséré recule-t-il ? On peut aussi vérifier avec `tri_insertion_compte`. ↑énoncé

**Ex. 18 q. 1 – début de solution** Insérer `1` : une comparaison et un décalage (le `2`) ; insérer `4` : une comparaison, aucun décalage ; insérer `3` : . . . (ne pas oublier la comparaison qui arrête la boucle). Pour la sélection, le nombre de comparaisons ne dépend que de $n$ : formule $\frac{n(n-1)}{2}$. ↑énoncé

**Ex. 19 – piste** Quelle propriété du tri par sélection garantit qu'après `k` étapes, les `k` premières cases contiennent déjà les `k` plus petits éléments, à leur place définitive ? ↑énoncé

**Ex. 19 – début de solution**
```
copie = [x for x in t]
for i in range(k):
    i_min = indice_min(copie, i)
    ... (échange)    puis renvoyer copie[0:k].
```
↑énoncé

**Ex. 20 – piste** Question 1 : seule la comparaison change, on compare `len(mots[j])` et `len(x)`. Question 2 : dérouler l'insertion de `"noix"` derrière `"kiwi"`. Question 3 : un échange peut faire « sauter » un mot par-dessus un autre de même longueur. ↑énoncé

**Ex. 20 – début de solution** Question 3 : essayer la sélection sur `["abc", "xyz", "a"]`, étape par étape. ↑énoncé

**Ex. 21 q. 3 – piste** Après chaque étape `i`, vérifier que toutes les valeurs de départ sont encore présentes dans le tableau. ↑énoncé

**Ex. 22 – piste** À l'étape `i`, ne regarder que les cases `i` à la fin. Si le minimum est déjà en case `i`, l'échange ne change rien. Pour un tableau de 4 cases, 3 étapes suffisent. ↑énoncé

Ex. 23 – piste L'élément à insérer recule tant que son voisin de gauche est plus grand que lui : chaque élément qui lui laisse sa place compte pour un décalage. ↑énoncé

Ex. 24 – piste Question 3 : comparer `t[0..2]` à la liste des trois plus petites valeurs `2`, `4`, `6`. Question 4 : quelles sortes de boucles contient le tri par sélection ? Question 5 : quelle variable diminue à chaque tour du `while`, et quelle condition l'empêche de devenir négative pendant la boucle ? ↑énoncé

# TP ET PROJETS

## TP Le banc d'essai des tris

▶ corrigé p. 621

*compter, chronométrer, comparer : sélection, insertion et* `sorted`

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p06-tp-banc-essai-tris` — **archive .zip**

**Durée** 2 h à 2 h 30 (une à deux séances), en binôme, sur machine.
**Prérequis** le cours *Les algorithmes de tri* (sélection, insertion, invariants, coût) et la notion de coût du chapitre *Algorithmique : le parcours séquentiel*.
**Fichier** `tp_tris_depart.py` (à compléter : tout votre code s'y écrit).



### But du TP

Le cours affirme que les tris par sélection et par insertion sont **quadratiques**, que l'insertion devient **linéaire** sur un tableau déjà trié, et que `sorted` est bien plus rapide. Vous allez construire un **banc d'essai** pour le vérifier vous-mêmes :

1. des tris **instrumentés** qui comptent leurs comparaisons et leurs mouvements ;
2. des **jeux d'essai** (listes aléatoires, triées, inversées, presque triées) ;
3. des **mesures** : nombres d'opérations, puis temps au chronomètre, tracés sur un graphique ;
4. une **visualisation** du tri pas à pas, et (défi) un **classement en direct**.

**Produit final** : un programme qui imprime le tableau comparatif des deux tris, un graphique des temps, et un bilan rédigé qui confirme — ou nuance — le cours.

### Consignes

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme. ▶ *À programmer* dans `tp_tris_depart.py`.
- **Règle du TP** : on écrit les tris soi-même ; `sort()` et `sorted()` ne servent que de **référence** (pour vérifier et pour la course de vitesse).
- En bas du fichier, le programme principal contient des lignes en commentaire (`#`) : on les **décommente au fur et à mesure**. Des tests `assert` sont fournis.
- Le compte rendu (tableaux, réponses, bilan) se fait sur le cahier (ou dans un fichier texte).
- Usage de l'IA, indiqué sur chaque exercice : IA aucune ; IA en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; IA l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Les jeux d'essai

★ **Exercice 1** *Fabriquer des listes*

Le fichier fournit `aleatoire(n)` (`n` entiers au hasard entre 0 et 1000), `est_trie(t)` et `presque_triee(n, k)` (une liste triée dans laquelle on a échangé `k` paires de voisins).

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Exécuter le fichier, puis afficher dans la console `aleatoire(10)` et `presque_triee(10, 2)`.
2. ▶ Écrire `triee(n)`, qui renvoie `[0, 1, ..., n-1]`, et `inversee(n)`, qui renvoie `[n, n-1, ..., 1]`, chacune en une ligne, avec une compréhension sur `range(...)`.

```
>>> triee(4), inversee(4)
([0, 1, 2, 3], [4, 3, 2, 1])
```

3. Pourquoi aura-t-on besoin de listes triées, inversées et presque triées pour *tester* un tri, et pas seulement de listes au hasard ?

## 2 Des tris qui comptent leur travail

On **instrumente** les deux tris du cours : ils trient le tableau **en place** comme d'habitude, et **renvoient en plus** un couple de compteurs.

★★ **Exercice 2** *Le tri par sélection instrumenté*

▶ Écrire `tri_selection_compte(t)` qui trie `t` par sélection et renvoie le couple `(comparaisons, echanges)` :

- `comparaisons` compte les comparaisons `t[j] < t[i_min]` ;
- `echanges` compte les échanges **réellement utiles** : on n'échange que si `i_min != i`.

```
>>> t = [5, 3, 8, 1, 9, 2]
>>> tri_selection_compte(t)
(15, 4)
>>> t
[1, 2, 3, 5, 8, 9]
```

▷ coup de pouce

★★ **Exercice 3** *Le tri par insertion instrumenté*

▶ Écrire `tri_insertion_compte(t)` qui trie `t` par insertion et renvoie `(comparaisons, decalages)` : une **comparaison** est chaque test `t[j] > x` effectué, un **décalage** chaque affectation `t[j + 1] = t[j]`.

```
>>> tri_insertion_compte([5, 3, 8, 1, 9, 2])
(11, 8)
>>> tri_insertion_compte([1, 2, 3, 4, 5])
(4, 0)
```

▷ coup de pouce

Décommenter `tester_partie_A()` (la « partie A » du fichier correspond à cette section) : les tests trient des listes de plusieurs tailles et de trois sortes, et comparent à `sorted`. Ils doivent tous passer avant de continuer.

## 3 Le banc d'essai : compter

★ **Exercice 4** *Lancer le banc*

▶ Décommenter `banc_essai([100, 200, 400])` puis recopier sur le cahier le tableau suivant et le compléter avec les résultats.

| `n` | liste | sélection : comparaisons | sélection : échanges | insertion : comparaisons | insertion : décalages |
|---|---|---|---|---|---|
| 100 | aléatoire | | | | |
| | triée | | | | |
| | inversée | | | | |
| 200 | aléatoire | | | | |
| | triée | | | | |
| | inversée | | | | |
| 400 | aléatoire | | | | |
| | triée | | | | |
| | inversée | | | | |

★★ **Exercice 5** *Faire parler les nombres*

1. Les comparaisons de la sélection dépendent-elles de la sorte de liste ? Vérifier qu'elles valent $\frac{n(n-1)}{2}$.
2. Quand `n` double, par combien sont multipliées les comparaisons de la sélection ? celles de l'insertion sur une liste aléatoire ? Quel coût cela confirme-t-il ?
3. Sur une liste **triée**, combien de comparaisons fait l'insertion ? Exprimer ce nombre en fonction de `n`. C'est le « meilleur atout » du cours : quel est alors son coût ?
4. Sur une liste **inversée**, comparer les deux tris. Sur une liste **aléatoire**, l'insertion fait environ *deux fois moins* de comparaisons que la sélection : proposer une explication (de combien de cases recule, en moyenne, un élément inséré ?).
5. La sélection fait au plus `n - 1` échanges, l'insertion des milliers de décalages. Dans quelle situation pourrait-on préférer la sélection ? *(Penser à des objets lourds à déplacer.)* [ au-delà du programme ]
6. **Sur une liste inversée**, la sélection ne fait que `n/2` échanges. Dérouler-la sur `[6, 5, 4, 3, 2, 1]` pour comprendre pourquoi.

## 4 Le chronomètre

★★ **Exercice 6** *La course*

1. ▶ Écrire `chronometre(tri, t)` qui renvoie la durée, en secondes, du tri d'une **copie** de `t` par la fonction `tri` (mesurée avec `time.perf_counter()`). Pourquoi est-il indispensable de trier une copie ?
▷ coup de pouce
2. ▶ La fonction `course(tailles)`, fournie, chronomètre la sélection, l'insertion et `sorted` sur une **même** liste aléatoire pour chaque taille. La lancer pour `[500, 1000, 2000, 4000]` puis recopier sur le cahier et compléter le tableau suivant (3 chiffres significatifs) :

| `n` | sélection (s) | insertion (s) | `sorted` (s) | sélection / `sorted` |
|---|---|---|---|---|
| 500 | | | | |
| 1 000 | | | | |
| 2 000 | | | | |
| 4 000 | | | | |

★★ **Exercice 7** *Lire les temps*

1. Quand `n` double, par combien est multiplié le temps de chacun des deux tris ? et celui de `sorted` ?
2. ▶ Décommenter `tracer(...)` et décrire les trois courbes.
3. Sur liste aléatoire, l'insertion faisait deux fois moins de comparaisons… mais est-elle deux fois plus rapide ? Proposer une explication (que fait-elle d'autre que comparer ?).
4. ▶ Chronométrer les deux tris sur `triee(4000)`. Quel est le rapport entre les deux temps ? Est-ce cohérent avec l'exercice 5 ?
5. ▶ Compter les comparaisons de l'insertion sur `presque_triee(1000, k)` pour `k` valant 0, 5, 20, 100. Que constate-t-on ? Pour quelles données réelles l'insertion est-elle donc un excellent choix ?
6. **Prédire.** À partir de la mesure pour $n = 4\,000$, estimer le temps de la sélection pour trier 1 000 000 de valeurs. Comparer à la phrase du cours : « des minutes, voire des heures ».

## 5 Voir le tri travailler

★★ **Exercice 8** *Des barres dans la console*

1. ▶ Écrire `afficher_barres(t, i_actif)` qui affiche chaque valeur de `t` sur une ligne, suivie d'autant de `#` que sa valeur ; la ligne d'indice `i_actif` est marquée d'une flèche (`-1` : aucune flèche). *Rappel :* `"#" * 5` *vaut* `"#####"`.
2. ▶ Écrire `tri_insertion_visu(t, pause)` : le tri par insertion, qui affiche le tableau **après chaque insertion**, en marquant l'élément qu'on vient de placer, puis attend `pause` secondes (`time.sleep(pause)`). Voici un **extrait** de l'affichage attendu sur `[5, 3, 8, 1, 9, 2]` (l'état de départ, puis les étapes 1 et 3 ; l'étape 2, où 8 reste à sa place, n'est pas reproduite) :

```
5 #####
3 ###
8 ########
1 #
9 #########
2 ##
```

```
Étape 1 : on insère 3
3 ###  <--
5 #####
8 ########
1 #
9 #########
2 ##
```

```
Étape 3 : on insère 1
1 #  <--
3 ###
5 #####
8 ########
9 #########
2 ##
```

3. Sur l'affichage, repérer l'**invariant** du tri par insertion : quelle partie du tableau est triée après l'étape $i$ ?
4. ▶ [ au-delà du programme ] Écrire de même `tri_selection_visu`. Regarder les deux animations sur une liste de 12 entiers au hasard entre 1 et 30 (`[random.randint(1, 30) for k in range(12)]`) : en quoi « voit-on » que la sélection place chaque élément à sa place *définitive*, et pas l'insertion ?

## 6 Défi : le classement en direct

Lors d'une course, les temps des coureurs arrivent **un par un**, et le tableau d'affichage doit toujours montrer un classement **trié**. Deux stratégies :

- **retrier** tout le classement par sélection à chaque arrivée (fonction `direct_en_retriant`, fournie) ;
- **insérer** le nouveau temps à sa place dans le classement déjà trié — exactement *une* étape du tri par insertion.

### ★★★ Exercice 9 *Insérer à sa place*

1. ▶ Écrire `inserer(classement, x)` : `classement` est trié ; la fonction y ajoute `x` à la fin (`append`), le fait reculer à sa place par décalages, et renvoie le nombre de comparaisons effectuées.

```
>>> c = [47, 50, 53, 61]
>>> inserer(c, 52)
3
>>> c
[47, 50, 52, 53, 61]
```

2. ▶ Écrire `direct_par_insertion(arrivees)` qui part d'un classement vide, insère les temps un par un, et renvoie le couple `(classement, total des comparaisons)`.
3. ▶ Comparer les deux stratégies sur `aleatoire(300)` : vérifier que les classements obtenus sont identiques et relever les deux totaux de comparaisons.
4. Expliquer l'écart. Combien coûte, au pire, l'arrivée d'un coureur dans chaque stratégie quand $k$ coureurs sont déjà classés ?

### Bilan à rédiger

Sur le cahier, en huit à dix lignes, **en citant vos mesures** :

- ce que veut dire « quadratique » quand on double la taille d'une liste ;
- dans quels cas l'insertion est bien meilleure que la sélection, et pourquoi ;
- pourquoi, en pratique, on utilise `sort()` / `sorted()` ;
- une limite de vos mesures (ce qu'un chronomètre ne mesure pas bien).

### Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 2 – piste Partir du tri par sélection du cours. Deux compteurs initialisés à `0` avant les boucles ; la comparaison à compter est dans la boucle interne, l'échange juste après elle. ↑énoncé

Ex. 3 – piste Garder la boucle `while j >= 0 and t[j] > x` du cours et compter une comparaison à chaque tour. Attention : quand la boucle s'arrête alors que `j >= 0`, c'est que le test `t[j] > x` a été fait (et était faux) : il faut aussi le compter. ↑énoncé

Ex. 6 q. 1 – piste `copie = [x for x in t]`, lire l'horloge, appeler `tri(copie)`, relire l'horloge. Que se passerait-il si la sélection triait `t` lui-même, puis que l'insertion recevait ce même `t` ? ↑énoncé

# FICHE DE RÉVISION

## Échanger deux valeurs (le bon réflexe)

```
temp = t[i]          # on met de cote
t[i] = t[j]
t[j] = temp          # sans temp, on perd une valeur !
```

En Python, l'affectation multiple échange aussi en une ligne : `t[i], t[j] = t[j], t[i]` (forme rencontrée dans des sujets de bac).

## Tri par sélection

À chaque passe, on cherche le **minimum** de la partie non triée et on le place au début.

```
def tri_selection(t):
    n = len(t)
    for i in range(n):
        i_min = i          # minimum de t[i:]
        for j in range(i + 1, n):
            if t[j] < t[i_min]:
                i_min = j
        temp = t[i]        # echange
        t[i] = t[i_min]
        t[i_min] = temp
```

## Tri par insertion

Comme un joueur qui range ses cartes : on prend l'élément suivant et on le **glisse à sa place** dans la partie de gauche déjà triée, en décalant vers la droite les éléments plus grands.

```
def tri_insertion(t):
    n = len(t)
    for i in range(1, n):
        x = t[i]
        j = i - 1
        while j >= 0 and t[j] > x:
            t[j + 1] = t[j]  # decaler
            j = j - 1
        t[j + 1] = x         # poser x
```

## Le coût

Les deux tris comparent (au pire) **tous les éléments deux à deux** : coût **quadratique**, $\frac{n(n-1)}{2} \approx \frac{n^2}{2}$, de l'ordre de $n^2$. **Atout de l'insertion** : **linéaire** si le tableau est déjà (presque) trié. En pratique, on utilise `t.sort()` / `sorted(t)` (**Timsort**, $n \log_2 n$).

## Invariants et terminaison

**Sélection** : au début de l'étape `i`, `t[0..i-1]` est triée et contient les `i` plus petits éléments, à leur place définitive. **Insertion** : au début de l'étape `i`, `t[0..i-1]` contient les éléments initiaux de `t[0..i-1]`, rangés dans l'ordre croissant. **Terminaison** : la sélection n'a que des boucles `for` (bornées) ; dans l'insertion, `j` est un variant du `while` (entier $\geqslant 0$ dans la boucle, qui diminue de 1 à chaque tour).

## Erreurs à éviter

- échanger sans variable temporaire (valeur perdue) ;
- se tromper de bornes (dernier indice `len(t)-1`) ;
- insertion : oublier de décaler avant de poser ;
- croire un tri quadratique adapté aux grands tableaux.

## Savoir-faire à maîtriser — je sais…

- ☐ **dérouler** à la main un tri par sélection et un tri par insertion ;
- ☐ écrire le **tri par sélection** (recherche de l'indice du minimum, puis échange) ;
- ☐ écrire le **tri par insertion** ;
- ☐ **échanger** deux valeurs avec une variable temporaire ;
- ☐ énoncer l'**invariant** de chaque tri et justifier sa **terminaison** ;
- ☐ justifier le **coût quadratique** ($\approx n^2$, soit $\frac{n(n-1)}{2}$ comparaisons) ;
- ☐ reconnaître l'**atout de l'insertion** sur un tableau presque trié ;

☐ savoir qu'en pratique on utilise `sort()` / `sorted()`.

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 625

1. Quel tri place, à chaque étape, un élément à sa position **définitive** ?
2. État de `[4, 1, 3, 2]` après les **deux** premières étapes du tri par insertion ?
3. Combien de comparaisons fait le tri par sélection sur 10 éléments ?
4. Pourquoi `t[i] = t[j]` puis `t[j] = t[i]` n'échange-t-il pas ?
5. Données déjà presque triées : sélection ou insertion ?

**Et avant ?** — *questions sur les chapitres précédents.* ▶ réponses p. 625

1. Un jeu de tests doit-il contenir le tableau vide ? Pourquoi ? (ch. 5)
2. Quel tableau produit `[x for x in [4, 7, 2, 9] if x > 3]` ? (ch. 3)

# DÉFI ADVENT OF CODE

## DÉFI Historian Hysteria

*deux listes à réconcilier — trier, distances, compter*

Jour 1

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p06-defi-aoc-2024-01` — **archive .zip**



### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2024/day/1`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (facultatif), le lien *get your puzzle input* donne **vos** données, à enregistrer sous `input.txt` à côté du programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Deux listes de mille numéros chacune devraient être identiques, mais ne le sont pas. On veut mesurer à quel point elles diffèrent (partie 1), puis à quel point elles se ressemblent (partie 2). Trier deux listes de mille nombres et les comparer à la main est impensable : il faut un programme.
**Ce qu'on réinvestit.** Le **tri d'un tableau** vu dans ce chapitre (par insertion ou par sélection) : une fois les deux listes triées, le plus petit nombre de gauche se trouve en face du plus petit de droite, et ainsi de suite, ce qui est exactement l'association demandée. On construit aussi des listes avec `append` et on compte des apparitions avec une boucle.
**La démarche de la fiche.**

1. Comprendre l'association des nombres, puis calculer la réponse à la main sur un petit fichier.
2. Lire le fichier en deux listes, une par colonne.
3. Trier les deux listes et additionner les écarts : c'est la réponse de la partie 1.
4. Pour la partie 2, compter les apparitions de chaque nombre de gauche dans la liste de

droite.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `list` | liste | `side by side` | côte à côte |
| `left / right` | gauche / droite | `to pair up` | associer par paires |
| `smallest` | le plus petit | `second-smallest` | le deuxième plus petit |
| `how far apart` | à quelle distance (l'écart) | `to add up` | additionner |
| `to reconcile` | mettre en accord | `location ID` | identifiant de lieu |

### Résumé de l'énoncé (partie 1)

**Contexte.** Les historiens du pôle Nord cherchent leur chef, introuvable. Dans son bureau, ils ont dressé deux listes de lieux à visiter, chaque lieu étant désigné par un numéro. Mais les deux listes ne concordent pas : avant de les mettre en accord, il faut mesurer à quel point elles diffèrent.
**Les données.** Le fichier présente les deux listes côte à côte. Chaque ligne contient deux entiers séparés par plusieurs espaces : le premier appartient à la liste de **gauche**, le second à la liste de **droite**. Les deux nombres d'une même ligne n'ont aucun lien entre eux.
**La règle.**

- On ne compare pas les nombres d'une même ligne : on trie chaque liste dans l'ordre croissant.
- On associe le plus petit nombre de gauche au plus petit de droite, le deuxième plus petit au deuxième plus petit, etc.
- L'écart d'une paire est la distance entre ses deux nombres, c'est-à-dire la valeur absolue de leur différence (on ne se soucie pas de savoir lequel est le plus grand).

**La réponse attendue.** La somme des écarts de toutes les paires. Par exemple, avec les deux lignes `4 9` et `1 3`, la liste de gauche triée est 1, 4 et celle de droite 3, 9 ; les écarts sont 2 et 5 : la réponse serait 7.

### Exercice 1 *Vérifier sa compréhension*

**But.** Vérifier qu'on a compris comment les nombres sont associés et comment se mesure un écart.
**À quoi ça sert ?** La première question montre que c'est l'ordre, et non le contenu, qui change d'une liste à l'autre ; la deuxième justifie l'emploi de la valeur absolue ; la troisième assure qu'on peut associer les nombres deux à deux sans en oublier.

Questions pièges, à traiter sur le cahier :

1. Si les deux listes contiennent exactement les mêmes nombres, dans un autre ordre, quelle est la réponse ?
2. L'écart entre 3 et 7 est-il le même que l'écart entre 7 et 3 ?
3. Pourquoi les deux listes ont-elles forcément la même longueur ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Trier à la main les deux colonnes d'un petit fichier inventé et calculer la somme des écarts.
**À quoi ça sert ?** On voit concrètement pourquoi il faut trier avant d'associer, et l'on obtient une valeur connue (4) pour tester `distance_totale`.

On considère le fichier (inventé) suivant :

```
5   2
2   5
7   4
1   2
2   8
6   2
```

Écrire les deux listes triées l'une sous l'autre, puis les écarts. Vérification : on doit obtenir `4`.

## 2 Programmer la partie 1



### Exercice 3 *Lire les deux colonnes*

**But.** Lire le fichier et ranger les nombres de gauche et de droite dans deux listes d'entiers.
**À quoi ça sert ?** Les deux parties travaillent sur ces deux listes, chacune séparément : il faut donc d'abord séparer les colonnes, puisque les nombres d'une même ligne n'ont pas de lien.

#### Outil Python : découper une ligne (`split`)

`ligne.split()` découpe la chaîne `ligne` sur les espaces et renvoie la **liste** des morceaux (des chaînes) ; les espaces multiples ne gênent pas. Avec un argument, `ligne.split(", ")` découpe sur ce séparateur.

```
1 morceaux = "5   2".split()
2 print(morceaux)              # ['5', '2'] : deux chaines
3 print(int(morceaux[0]))      # 5 : converti en entier
4 print("a, b, c".split(", ")) # ['a', 'b', 'c']
```

Écrire une fonction `lire(nom_fichier)` qui renvoie les deux listes `gauche` et `droite`. Sur l'exemple : `[5, 2, 7, 1, 2, 6]` et `[2, 5, 4, 2, 8, 2]`. ▷ coup de pouce ▷ début de solution

#### Outil Python : la valeur absolue (`abs`)

La fonction `abs`, déjà rencontrée au chapitre *Le binaire et l'écriture des nombres*, renvoie la valeur absolue d'un nombre : le nombre sans son signe. Pour deux nombres `x` et `y`, `abs(x - y)` est la distance entre eux, quel que soit le plus grand.

```
1 print(abs(-4))      # 4
2 print(abs(4))       # 4
3 print(abs(3 - 7))   # 4
4 print(abs(7 - 3))   # 4 : meme distance
```

**Intérêt.** Inutile d'écrire un `if` pour savoir quel nombre est le plus grand. Question : que vaut `abs(0)` ? `abs(-2.5)` ?

### Exercice 4 *La distance totale*

**But.** Trier les deux listes, associer leurs éléments rang par rang et additionner les écarts.
**À quoi ça sert ?** Appliquée aux listes lues dans vos données, cette fonction donne la réponse de la partie 1. Le tri du chapitre y fait l'essentiel du travail.

Écrire une fonction `distance_totale(gauche, droite)` qui renvoie la réponse de la partie 1. Tester sur l'exemple, puis sur vos données. ▷ coup de pouce ▷ début de solution

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

**Résumé de la partie 2 (à lire après avoir validé la partie 1)**

**Ce qui change.** Les historiens remarquent que beaucoup de numéros se retrouvent dans les deux listes : on ne mesure plus la différence, mais la **ressemblance** des deux listes, d'une autre façon. Le tri ne joue plus aucun rôle.
**La nouvelle règle.**

- Pour chaque nombre de la liste de gauche, on compte combien de fois il apparaît dans la liste de droite (ce compte peut valoir 0).
- On multiplie ce nombre par son compte.
- Un nombre présent deux fois à gauche est traité deux fois.

**La réponse attendue.** La somme de ces produits. Par exemple, avec à gauche 3, 5, 3 et à droite 3, 3, 1 : chaque 3 de gauche apparaît deux fois à droite et rapporte $3 \times 2 = 6$, le 5 n'y apparaît pas et rapporte 0 ; la réponse serait 12.

### Exercice 5 *Compter les apparitions*

**But.** Compter combien de fois un nombre apparaît dans une liste, puis utiliser ce comptage pour la partie 2.
**À quoi ça sert ?** La réponse de la partie 2 est une somme de produits « nombre × nombre d'apparitions ». Compter les comparaisons faites prépare la question de l'approfondissement : cette méthode est-elle assez rapide ?

1. Écrire une fonction `nb_apparitions(x, liste)` qui compte, avec une boucle, combien de fois `x` apparaît dans `liste`. Combien de comparaisons fait `nb_apparitions(2, [2, 5, 4, 2, 8, 2])` ?
2. Utiliser cette fonction pour répondre à la partie 2. Sur l'exemple inventé de la fiche, on doit trouver `17`.

▷ coup de pouce ▷ début de solution

## 4 Approfondissement

★★★ **Exercice 6** *Approfondissement 1 : compter une fois pour toutes*

**But.** Estimer le coût de la méthode de la partie 2, puis l'accélérer avec un dictionnaire.
**À quoi ça sert ?** Compter les apparitions dans un dictionnaire, en un seul parcours, évite de reparcourir mille fois la liste de droite. C'est la méthode à retenir quand les listes deviennent grandes.

1. Avec 1000 nombres par liste, combien de comparaisons fait la solution de la partie 2 qui utilise `nb_apparitions` ?
2. Sur l'exemple, écrire à la main le dictionnaire `compte` qui associe à chaque nombre de la liste de droite son nombre d'apparitions.

3. Écrire une fonction `similarite_dico(gauche, droite)` qui construit ce dictionnaire en un parcours de la liste de droite, puis répond en un seul parcours de la liste de gauche. Quel est son coût ? ▷ coup de pouce ▷ début de solution

## Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

**Ex. 3 – piste** Même schéma que d'habitude (`open`, boucle sur les lignes). `"5 2".split()` renvoie `["5", "2"]` : les espaces multiples ne gênent pas. ↑énoncé

**Ex. 3 – début de solution** `morceaux = ligne.split()` puis `gauche.append(int(morceaux[0]))` et de même pour `droite`. Une fonction peut renvoyer deux valeurs : `return gauche, droite`, récupérées par `g, d = lire("exemple.txt")`. ↑énoncé

**Ex. 4 – piste** Écrire d'abord une fonction `trier(t)` (tri par insertion ou par sélection du chapitre) et trier les deux listes, puis parcourir les indices : l'écart entre deux nombres `x` et `y` est `abs(x - y)`. ↑énoncé

**Ex. 4 – début de solution** Pour ne pas modifier les listes lues, trier des copies : `g_tri = [x for x in gauche]`, puis `trier(g_tri)`. Ensuite, un accumulateur `total` et une boucle sur `range(len(g_tri))`. ↑énoncé

**Ex. 5 – piste** Relire précisément ce que l'énoncé multiplie : quel nombre, et par quoi ? Faut-il trier pour cette partie ? ↑énoncé

**Ex. 5 – début de solution** Pour chaque `x` de la liste de gauche, on ajoute au total `x * nb_apparitions(x, droite)`. ↑énoncé

**Ex. 6 q. 3 – piste** Pour chaque `y` de la liste de droite : si `y` est déjà une clé, augmenter sa valeur de 1 ; sinon, créer la clé avec la valeur 1. ↑énoncé

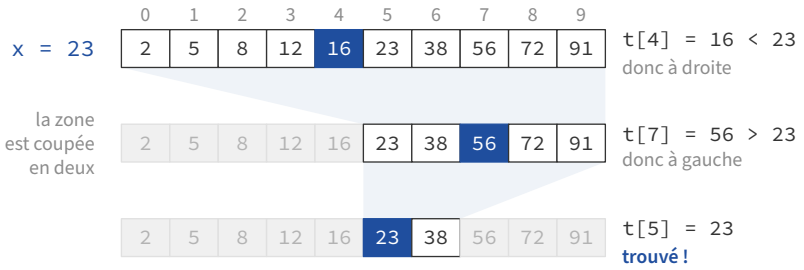
**Ex. 6 q. 3 – début de solution** Ensuite, pour chaque `x` de gauche : si `x in compte`, ajouter `x * compte[x]` au total ; sinon, rien à ajouter. Sans ce test, `compte[x]` provoquerait une erreur pour un nombre absent de la liste de droite. ↑

7

CHAPITRE 7

# La recherche dichotomique






## DANS CE CHAPITRE


- Activités préparatoires . . . . . 280
- Cours . . . . . 282
- Exercices . . . . . 288
- TP et projets . . . . . 294
- Fiche de révision . . . . . 299
- Défi Advent of Code . . . . . 301
- Corrigés . . . . . 626

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 Le nombre mystère

▶ corrigé p. 626

*trouver un nombre en un minimum de coups*

Durée : 20 min · Par binômes, sans machine, réponses sur le cahier

### Consignes

- Une feuille de brouillon pour le maître du jeu.
- Le **maître du jeu** choisit un nombre secret et l'écrit, caché, sur son brouillon. Le **joueur** fait des propositions ; à chaque proposition, le maître du jeu répond seulement « **plus grand** », « **plus petit** » ou « **gagné** ».
- On compte les **coups** : chaque proposition compte pour un coup, y compris la dernière.



### Exercice 1 *Entre 1 et 100*

1. Jouer quatre parties avec un secret entre 1 et 100, en échangeant les rôles à chaque partie. Noter sur le cahier, pour chaque partie, le nombre secret et le nombre de coups.
2. Décrire en une ou deux phrases la stratégie du joueur qui a utilisé le moins de coups.

### Exercice 2 *La meilleure stratégie*

3. Quelle est la meilleure **première** proposition ? Pourquoi ?
4. Avec cette première proposition, combien de nombres possibles reste-t-il, au pire, après la réponse ?
5. En continuant ainsi, écrire la suite du nombre de possibilités restantes, au pire : $100 \to 50 \to \ldots$ jusqu'à 1. Combien de coups faut-il donc, au pire, pour être sûr de gagner ?
6. **Le maître du jeu malin.** Nouvelle règle : le maître du jeu n'écrit plus son secret. Il répond à chaque fois de façon à faire durer la partie le plus longtemps possible, mais sans jamais contredire ses réponses précédentes. Jouer deux parties, le joueur appliquant la meilleure stratégie. Noter le nombre de coups de chaque partie. Le maître du jeu malin peut-il dépasser le nombre trouvé à la question 5 ?

### Exercice 3 *Entre 1 et 1000*

7. **Avant de jouer**, prédire le nombre de coups nécessaires, au pire, avec la meilleure stratégie.
8. Jouer contre le maître du jeu malin, avec un secret entre 1 et 1000. Combien de coups a-t-il fallu ? La prédiction était-elle juste ?
9. Recopier et compléter sur le cahier le tableau suivant, en raisonnant (sans jouer) pour 200, 2 000 et 1 000 000.

| Nombres possibles | 100 | 200 | 1 000 | 2 000 | 1 000 000 |
|---|---|---|---|---|---|
| Coups au pire | | | | | |

Quand on **double** le nombre de possibilités, combien de coups faut-il en plus ? Et quand on le multiplie par 1 000 ?

**Exercice 4** *Ce qu'on a découvert*

**10.** **Ce qu'on a découvert.** Recopier et compléter sur le cahier : « La meilleure stratégie consiste à proposer le … des nombres encore possibles : chaque coup … le nombre de possibilités par … Avec $k$ coups, on peut départager jusqu'à $2^k - 1$ nombres : comme $2^{10} - 1 = \ldots$, il suffit de … coups pour 1 000 nombres. »

**11.** Cette stratégie marcherait-elle si le maître du jeu ne répondait que « gagné » ou « perdu » ? Qu'est-ce qui la rend possible ?

# COURS

**Programme (BO)** *« Recherche dichotomique dans un tableau trié »* : *montrer la terminaison de la recherche dichotomique à l'aide d'un variant de boucle* ; des assertions peuvent être utilisées ; la preuve de la correction peut être présentée par le professeur.

**Prérequis** la boucle `while`, les tableaux (indices, `len`) ; la **recherche séquentielle** et la **notion de coût** (chapitre *Algorithmique : le parcours séquentiel*) ; le fait qu'un tableau puisse être **trié** (chapitre *Les algorithmes de tri*). → fiches de révision : ch. 4 p. 202, ch. 6 p. 272

**Objectifs** comprendre et écrire la recherche dichotomique ; **dérouler** son exécution ; prouver qu'elle **se termine** (variant de boucle) ; établir son **coût logarithmique** et mesurer à quel point il est petit.

## Situation de départ

1. Pour trouver un mot dans un dictionnaire papier de 1 000 pages, pourquoi personne ne lit les pages une à une depuis le début ?
2. Combien d'essais faut-il, au pire, pour trouver un nombre pensé entre 1 et 1 000 000, si l'on vous répond seulement « plus grand » ou « plus petit » ?
3. Si le mot cherché n'est pas dans le dictionnaire, est-on sûr que la recherche finira par s'arrêter ?

**Remarque** — *Le jeu qui cache un algorithme*

« Je pense à un nombre entre 1 et 100. À chaque essai, je vous dis *plus petit* ou *plus grand*. » Personne ne propose 1, puis 2, puis 3… Tout le monde propose **50**, puis coupe encore en deux. En 7 essais au maximum, c'est gagné. Ce réflexe si naturel *est* un algorithme fondamental : la **recherche dichotomique** (du grec *dikho-tomia*, « couper en deux »). Vous l'avez d'ailleurs déjà programmé sans le savoir : au chapitre *Les bases de la programmation Python*, le défi « L'ordinateur devine » du TP « Le nombre mystère » appliquait exactement cette stratégie du milieu. Ce chapitre ne la redécouvre donc pas : il lui donne son **nom**, l'étend à la recherche dans un **tableau trié**, **prouve** qu'elle se termine toujours et **calcule** son coût. Nous allons voir qu'il transforme une recherche dans un million de valeurs en… **vingt** comparaisons. Les points qui dépassent le programme sont signalés par le badge [ au-delà du programme ].

## I L'idée : couper en deux, encore et encore

Au chapitre *Algorithmique : le parcours séquentiel*, nous avons cherché un élément en parcourant *tout* le tableau : dans le pire des cas (élément absent), il faut `n` comparaisons. Coût **linéaire**. Peut-on faire mieux ?

### Règle 1 — Le déclic

On ne peut pas faire mieux qu'un parcours… **sauf si le tableau est trié**. Dans un tableau trié, comparer l'élément cherché à celui du **milieu** permet d'**éliminer la moitié** du tableau d'un seul coup.

Cherchons `x` dans un tableau trié. On regarde l'élément du milieu `m` :

- si `m == x` : **trouvé** ;

- si `x > m` : `x` ne peut être que dans la **moitié droite** (tout ce qui est à gauche est plus petit) ;
- si `x < m` : `x` ne peut être que dans la **moitié gauche**.

On recommence sur la moitié retenue, qui est deux fois plus petite. Et ainsi de suite, jusqu'à trouver `x`… ou jusqu'à ce qu'il ne reste plus rien à explorer.

**Définition 1 — Recherche dichotomique**

Rechercher `x` par **dichotomie** dans un tableau **trié**, c'est restreindre la zone de recherche à un intervalle `[deb ; fin]` que l'on **divise par deux** à chaque étape, en comparant `x` à l'élément du milieu.

↻ **Retour sur l'activité.** Dans l'activité « Le nombre mystère », la meilleure stratégie consistait à proposer le milieu des nombres encore possibles (50 pour commencer). Elle n'était possible que parce que le maître du jeu répondait « plus grand » ou « plus petit », et pas seulement « perdu » : dans un tableau, c'est le tri qui permet ce genre de réponse.

## II L'algorithme

On repère la zone de recherche par deux indices : `deb` (début) et `fin`. Le milieu est `mil = (deb + fin) // 2` (division **entière**). Tant que la zone n'est pas vide (`deb <= fin`), on teste son milieu.

```
1  def recherche_dichotomique(t, x):
2      """Renvoie True si x est dans le tableau TRIE t, False sinon."""
3      deb = 0                          # INITIALISER : premier indice
4      fin = len(t) - 1                 # INITIALISER : dernier indice
5      while deb <= fin:                # PARCOURIR : tant que la zone n'est pas vide
6          mil = (deb + fin) // 2       # indice du milieu
7          if t[mil] == x:              # TESTER
8              return True              # RENVOYER : trouve !
9          elif x > t[mil]:             # TESTER
10             deb = mil + 1            # METTRE A JOUR : on garde la moitie DROITE
11         else:
12             fin = mil - 1            # METTRE A JOUR : on garde la moitie GAUCHE
13     return False                     # RENVOYER : zone vide, x est absent
```

▷ **Je prédis.** Avec `t = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]`, combien de tours de boucle font `recherche_dichotomique(t, 91)` et `recherche_dichotomique(t, 4)` ?
▷ **Je vérifie.** `4` tours pour `91` (milieux d'indices 4, 7, 8, puis 9 : trouvé), `3` tours pour `4` (milieux 4, 1, 0, puis `deb = 1 > fin = 0` : `False`). Un absent ne coûte pas forcément plus qu'un présent.

**Remarque** — *Les deux façons de sortir de la boucle*
On sort **par le `return True`** dès qu'on tombe sur `x` ; ou bien la boucle s'arrête quand `deb > fin`, c'est-à-dire quand la zone de recherche est devenue **vide** : `x` est alors absent. Les frontières `deb` et `fin` se sont « croisées » sans jamais se rencontrer sur `x`.

**Remarque** — *Version qui renvoie la position*
Pour connaître *où* se trouve `x`, il suffit de renvoyer `mil` au lieu de `True`, et `-1` au lieu de `False`.

### A. Dérouler l'algorithme

Cherchons `x = 23` dans `t = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` (indices 0 à 9). La zone courante `[deb ; fin]` est `bleutée`, le milieu testé est en `orange`, ce qui est éliminé est grisé.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| départ | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 56 | 72 | 91 | `mil`=4, $16 < 23$ : à droite |
| `deb`=5 | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 56 | 72 | 91 | `mil`=7, $56 > 23$ : à gauche |
| `fin`=6 | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 56 | 72 | 91 | `mil`=5, $23 = 23$ : **trouvé !** |

En **3** comparaisons seulement, sur 10 éléments. Un parcours séquentiel en aurait fait 6 (23 est en 6e position).

► **Exercices d'application :** exercices 1 à 3 et 18 (dérouler la recherche ; le rôle du tri) et 4 à 8 (écrire l'algorithme)

## III L'algorithme se termine-t-il ?

Avec une boucle `while`, une question capitale se pose : est-on sûr de ne pas tourner **indéfiniment** ? Ici, oui, et on peut le **prouver**.

**Règle 2 — Un variant de boucle**

Considérons la quantité `fin - deb`, la « largeur » de la zone de recherche. À **chaque** tour, on remplace `deb` par `mil + 1`, ou `fin` par `mil - 1` : dans les deux cas, cette largeur **diminue strictement**. Tant que la boucle tourne (`deb <= fin`), cette quantité entière est **positive ou nulle** ; or un entier positif qui décroît strictement ne peut pas le faire éternellement : tôt ou tard `deb > fin`, et la boucle s'arrête. Une telle grandeur est appelée un **variant de boucle** (notion déjà rencontrée au chapitre *Les bases de la programmation Python*).

Démonstration : (La terminaison)
La largeur `fin - deb` est un entier. À chaque tour de boucle qui ne renvoie pas `True`, elle diminue d'au moins 1 (la moitié testée est *écartée*, milieu compris). Une suite d'entiers strictement décroissante finit forcément par franchir 0 : on atteint `deb > fin`, la condition de la boucle devient fausse, l'algorithme se termine en renvoyant `False`. □

**Pour les curieux** — *Pourquoi la recherche est correcte* (facultatif)
On peut aussi prouver que la réponse est **juste**, grâce à un **invariant** : « si `x` est dans `t`, alors il se trouve dans la zone `t[deb..fin]` ». C'est vrai au départ (la zone est le tableau entier). À chaque tour, on n'écarte que des cases qui ne peuvent pas contenir `x` : si `x > t[mil]`, toutes les cases de `deb` à `mil` contiennent des valeurs $\leqslant$ `t[mil] < x`, puisque le tableau est trié (même raisonnement à gauche). L'invariant reste donc vrai. Quand la boucle s'arrête sans avoir trouvé `x`, la zone est vide : `x` n'est pas dans `t`, et renvoyer `False` est correct.

**Remarque** — *Variant / invariant : ne pas confondre*
Un **invariant** (chapitre *Algorithmique : le parcours séquentiel*) est une propriété qui reste *vraie* à chaque tour et prouve que le résultat est **correct**. Un **variant** est une quantité qui *décroît* à chaque tour et prouve que l'algorithme **se termine**. Correction *et* terminaison : les deux garanties d'un bon algorithme.

► **Exercices d'application :** exercice 9 (la terminaison)

## IV Combien ça coûte ? Le coût logarithmique

À chaque tour, la zone de recherche est **divisée par deux**. La vraie question est donc : *combien de fois peut-on diviser* `n` *par 2 avant d'arriver à 1 ?*

$$n \to \frac{n}{2} \to \frac{n}{4} \to \frac{n}{8} \to \cdots \to 1.$$

### Règle 3 — Coût logarithmique

La recherche dichotomique sur un tableau de taille `n` fait **au plus** $\lfloor \log_2 n \rfloor + 1$ **tours**, soit le nombre de divisions par 2 pour arriver à 1, **plus un** (le dernier tour examine la seule case restante). Par exemple, pour $n = 16$ : $16 \to 8 \to 4 \to 2 \to 1$ fait 4 divisions, donc au plus **5 tours**. Ce nombre est de l'ordre de $\log_2(n)$ : on dit que son coût est **logarithmique**. C'est *radicalement* plus petit que `n`.

↻ **Retour sur l'activité.** La suite $100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$ de l'activité donnait 7 coups au pire entre 1 et 100 ; même le maître du jeu malin ne pouvait pas dépasser ce nombre. Le tableau final montrait déjà l'effet du logarithme : un coup de plus quand on double les possibilités, dix de plus quand on les multiplie par 1 000.

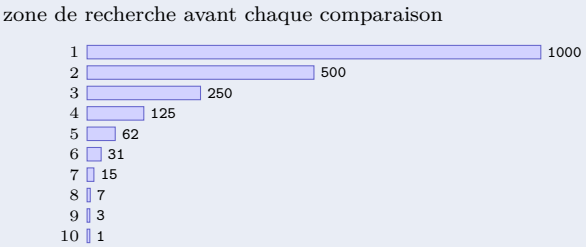
**Pour les curieux** — *Le logarithme en deux mots* (facultatif)
Diviser `n` par 2 un nombre $a$ de fois pour tomber sur 1 revient à écrire $n = 2^a$. Ce nombre $a$ d'étapes se note $a = \log_2(n)$ : c'est la **réciproque** de la puissance de 2.

Retenez surtout l'ordre de grandeur : **doubler la taille du tableau n'ajoute qu'*une seule* étape** (un tour de plus, au plus).

▷ **Je prédis.** Au plus combien de tours pour un tableau trié de `1000` éléments ? Et de `2000` ?
▷ **Je vérifie.** `10` : $1000 \to 500 \to 250 \to 125 \to 62 \to 31 \to 15 \to 7 \to 3 \to 1$ fait 9 divisions, plus le dernier tour. Pour `2000`, une division de plus : `11`.

Le tableau ci-dessous donne le nombre *maximal* de comparaisons. Le contraste avec la recherche séquentielle est saisissant. À droite, pour `n = 1000`, la largeur de la zone de recherche *avant* chacune des 10 comparaisons, dans le pire des cas (élément absent, plus grand que tous) : chaque barre est la **moitié** de la précédente.

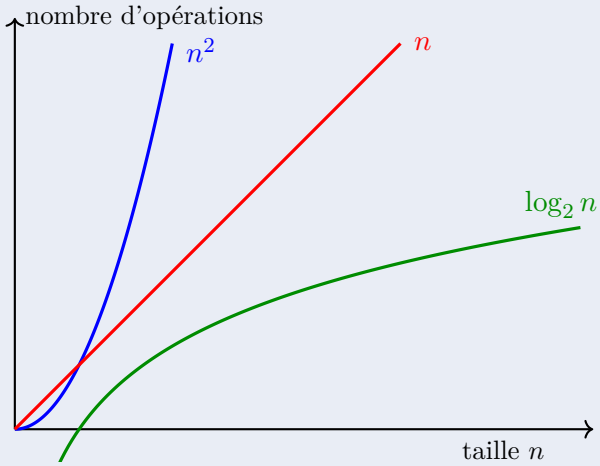
| taille `n` | séquentielle ($\approx n$) | dichotomique ($\approx \log_2 n$) |
|---|---|---|
| 100 | 100 | 7 |
| 1 000 | 1 000 | 10 |
| 1 000 000 | 1 000 000 | 20 |
| 1 000 000 000 | 1 000 000 000 | 30 |




Chercher dans l'annuaire du monde entier (8 milliards de noms triés) ne demanderait que **33 comparaisons**. Voilà pourquoi on trie.

### A. Comparer les trois coûts d'un coup d'œil

Voici les trois coûts rencontrés cette année : **linéaire** ($n$), **quadratique** ($n^2$) et **logarithmique** ($\log_2 n$). Plus une courbe reste *basse*, plus l'algorithme est *efficace* quand `n` grandit.




**Règle 4 — La hiérarchie à retenir**

Pour de grandes données : $\log_2 n$ (dichotomie) $\ll n$ (parcours) $\ll n^2$ (tris naïfs). Le logarithme écrase tout : c'est le coût rêvé.

**Remarque** — *Le prix à payer : un tableau trié*
La dichotomie **exige** un tableau trié. Si le tableau ne l'est pas, il faut d'abord le trier (coût quadratique avec les tris par sélection ou par insertion du chapitre *Les algorithmes de tri*, ou $n \log_2 n$ avec un bon tri). Cela ne vaut le coup que si l'on fait **beaucoup** de recherches : on trie *une* fois, on cherche *souvent*, très vite.

► **Exercices d'application :** exercices 10 à 12 et 19 (le coût logarithmique)

## V Un peu d'histoire et de culture

**Pour les curieux** — *Un algorithme simple… et pourtant redoutable à écrire* (facultatif)
La recherche dichotomique est décrite dès **1946**, mais la première version publiée *sans bug* pour un tableau quelconque n'apparaît qu'en **1962** ! Le grand informaticien **Donald Knuth** note que, si l'idée est limpide, « les détails » (les $\pm 1$ sur `deb` et `fin`, la condition d'arrêt) font trébucher la plupart des programmeurs. Plus fort encore : la recherche dichotomique de la bibliothèque standard de **Java** a contenu pendant **neuf ans** un bug (un calcul de milieu qui pouvait déborder sur de très grands tableaux), découvert seulement en 2006. La morale : un algorithme « évident » mérite d'être écrit, testé et *prouvé* avec soin — exactement ce que nous avons fait avec le variant de boucle.

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** La recherche séquentielle et la notion de coût (*Algorithmique : le parcours séquentiel*), le tableau trié (*Les algorithmes de tri*), le **variant** (*Les bases de la programmation Python*).
- **Un écho au binaire.** Il faut environ $\log_2 n$ bits pour écrire $n$ (*Le binaire et l'écriture des nombres*), comme environ $\log_2 n$ étapes pour isoler une valeur parmi $n$.
- **En Terminale.** La dichotomie sera l'exemple type de *Diviser pour régner* et s'écrira sous

forme récursive (*La récursivité*) ; l'**arbre binaire de recherche** (*Les arbres*) l'inscrira dans la structure même des données.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Condition d'emploi | le tableau doit être **trié** |
| Le principe | comparer au **milieu**, éliminer une **moitié**, recommencer |
| Les indices | `deb`, `fin` ; milieu `mil = (deb + fin) // 2` |
| Aller à droite / gauche | `x > t[mil]` → `deb = mil+1` ; sinon `fin = mil-1` |
| Condition d'arrêt | `deb > fin` : zone vide → élément absent |
| Terminaison | `fin - deb` décroît strictement : c'est un **variant** de boucle |
| Coût | **logarithmique** $\approx \log_2 n$ (doubler `n` → +1 étape) |
| Ordre de grandeur | 1 000 000 éléments → **20** comparaisons seulement |
| Variant vs invariant | variant → *termine* ; invariant → *correct* |

### Retour à la situation de départ

1. Parce que les mots sont rangés dans l'ordre : en ouvrant au milieu et en comparant, on élimine la moitié du dictionnaire d'un coup. C'est la **recherche dichotomique**, qui exige un tableau **trié** (§ I et II).
2. 20 au plus : le coût est **logarithmique**, de l'ordre de $\log_2 n$, et doubler le nombre de possibilités n'ajoute qu'un essai (§ IV).
3. Oui : la largeur `fin - deb` de la zone de recherche est un **variant**. Elle diminue strictement à chaque tour, jusqu'à `deb > fin` : l'élément est alors déclaré absent (§ III).

## Erreurs fréquentes

- **Chercher par dichotomie dans un tableau non trié.** L'algorithme **ne fonctionne pas** : le tri est une **hypothèse** indispensable.
- **Mal mettre à jour `deb` / `fin` / `mil`.** Oublier le $\pm 1$ ⇒ **boucle infinie**. *Le réflexe :* `mil = (deb + fin) // 2`, puis `deb = mil + 1` ou `fin = mil - 1`.
- **Oublier le cas « absent ».** Quand `deb > fin`, l'élément n'est pas là : renvoyer `False` / `-1`.
- **Confondre variant et invariant.** Le **variant** (`fin - deb`) prouve la **terminaison** ; l'invariant prouve la **correction**.
- **Croire la recherche séquentielle aussi rapide.** Elle est **linéaire** ; la dichotomie est **logarithmique**.

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

☐ **dérouler** une recherche dichotomique et compter les tours ;
☐ écrire la **recherche dichotomique** (indices `deb` / `fin` / `mil`) ;
☐ justifier la **terminaison** par un **variant** (`fin - deb` décroît) ;
☐ justifier le **coût logarithmique** (on divise par deux à chaque tour) ;
☐ reconnaître l'**hypothèse** indispensable : le tableau est **trié**.

# EXERCICES

## Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et tester* en Python. Le symbole `>>>` figure une saisie dans la console.
- **Rappel :** la recherche dichotomique ne s'applique qu'à un tableau **trié**. Le milieu se calcule avec `(deb + fin) // 2`.
- Dans cette feuille, on compte les **tours de boucle**, c'est-à-dire les valeurs `t[mil]` examinées : c'est ce que le cours appelle le nombre de « comparaisons ».

## Dérouler la recherche à la main ↑cours

On utilise le tableau trié

`t = [3, 8, 11, 16, 19, 24, 30, 33, 38, 47, 52]` (indices 0 à 10).

★ **Exercice 1** *Un élément présent* ▶ corrigé p. 627 • essentiel

On cherche `x = 33` par dichotomie. Recopier et compléter un tableau donnant, à chaque tour : `deb`, `fin`, `mil`, la valeur `t[mil]` et la décision (à gauche / à droite / trouvé). Combien de tours ont été nécessaires ? ▷ coup de pouce

★ **Exercice 2** *Un élément absent* ▶ corrigé p. 627 • essentiel

Même travail pour `x = 20`. Où et pourquoi la recherche s'arrête-t-elle ? Que renvoie l'algorithme ? ▷ coup de pouce

★★ **Exercice 3** *Le rôle du tri* ▶ corrigé p. 627

On applique *malgré tout* l'algorithme au tableau **non trié** `[9, 7, 5, 3, 1]` pour y chercher `x = 9`.

1. Dérouler l'algorithme. Que renvoie-t-il ? ▷ coup de pouce
2. `9` est pourtant bien dans le tableau. Expliquer, avec cet exemple, pourquoi le tri est **indispensable**.

## Écrire l'algorithme ↑cours

★ **Exercice 4** *Compléter la recherche dichotomique* ▶ corrigé p. 628 • essentiel

Compléter les six trous `.....` de cette fonction, puis la tester sur le tableau `t` donné plus haut (avant l'exercice 1) avec `x = 24` (présent) et `x = 10` (absent). ▷ coup de pouce

```
def recherche_dichotomique(t, x):
    deb = 0
    fin = .....
    while ..... :               # tant que la zone n'est pas vide
        mil = .....
        if t[mil] == x:
            return True
        elif x > t[mil]:
            deb = .....         # on garde la moitie droite
        else:
            fin = .....         # on garde la moitie gauche
    return .....
```

★★ **Exercice 5** *La recherche dichotomique* ▶ corrigé p. 628

▶ Écrire `recherche_dichotomique(t, x)` qui renvoie `True` si `x` est présent dans le tableau trié `t`, `False` sinon. Tester sur `t = [3, 8, 11, 16, 19, 24, 30, 33, 38, 47, 52]` avec `x = 38` (présent) puis `x = 25` (absent). ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 6** *Renvoyer la position* ▶ corrigé p. 628

▶ Écrire `indice_dichotomique(t, x)` qui renvoie l'**indice** de `x` dans `t`, ou `-1` si `x` est absent.

```
>>> indice_dichotomique([3, 8, 11, 16, 19], 16)
3
```

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 7** *Compter les tours* ▶ corrigé p. 629

▶ Écrire `nb_tours(t, x)` qui effectue la recherche dichotomique de `x` et renvoie le **nombre de tours de boucle** exécutés. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. Combien de valeurs sont examinées pour chercher 7 dans `[0, 1, 1, 2, 3, 5, 8, 13, 21]` ?
2. Donner un exemple (tableau + valeur cherchée) où l'on examine **exactement** 4 valeurs.

▷ coup de pouce

★★★ **Exercice 8** *La première occurrence* ▶ corrigé p. 629 ≈ 20 min

Dans un tableau trié qui contient des **doublons**, `indice_dichotomique` renvoie une occurrence de `x`, mais pas forcément la première : `indice_dichotomique([2, 4, 4, 4, 7, 9], 4)` renvoie 2.

1. ▶ Écrire `premiere_occurrence(t, x)` qui renvoie l'indice de la **première** occurrence de `x` dans le tableau trié `t`, ou `-1` si `x` est absent, en gardant un coût **logarithmique**.
2. Un camarade propose plutôt : trouver une occurrence par dichotomie, puis reculer case par case tant que la case de gauche vaut encore `x`. Pourquoi son coût n'est-il plus logarithmique dans le pire cas ? Donner un tableau qui le montre.

```
>>> premiere_occurrence([2, 4, 4, 4, 7, 9], 4)
1
>>> premiere_occurrence([2, 4, 4, 4, 7, 9], 5)
-1
```

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## La terminaison ↑ cours

★★ **Exercice 9** *Pourquoi ça s'arrête* ▶ corrigé p. 630 • essentiel

La recherche dichotomique utilise une boucle `while` : il faut s'assurer qu'elle ne tourne pas indéfiniment.

1. Montrer que la quantité `fin - deb` **diminue strictement** à chaque tour où l'on n'a pas trouvé `x` (examiner les deux cas : « à droite » et « à gauche »). ▷ coup de pouce
2. En déduire que l'algorithme se termine toujours. Comment appelle-t-on une telle quantité ?

## Le coût logarithmique ↑cours

★ **Exercice 10** *Combien d'étapes ?* ▶ corrigé p. 630 • essentiel

Sans machine, donner le nombre *maximal* de comparaisons d'une recherche dichotomique pour un tableau trié de :

**1.** 8 éléments ; **2.** 16 éléments ; **3.** 1 000 éléments (donner un ordre de grandeur).

▷ coup de pouce

★★ **Exercice 11** *Le plus petit* `k` *tel que* $2^k > n$ ▶ corrigé p. 630

▶ Écrire `nb_tours_max(n)` qui renvoie le plus petit entier `k` tel que `2**k > n` — c'est le nombre maximal de valeurs examinées par la dichotomie dans un tableau de taille `n`. *On utilisera une boucle qui double une puissance de 2 tant qu'elle ne dépasse pas* `n`.

```
>>> nb_tours_max(100)
7
```

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 12** *Le pire cas, en vrai* ▶ corrigé p. 631

▶ Le pire cas de la dichotomie est atteint quand l'élément est **absent**. En cherchant la valeur `1` dans un tableau ne contenant que des `0` (par exemple `[0] * n`), afficher le nombre de tours pour `n = 100, 1000, 10000, 100000, 1000000`. Que constate-t-on quand `n` est multiplié par 10 ? Comparer à la recherche séquentielle.

▷ coup de pouce

## Le jeu du nombre mystère ↑cours

★ **Exercice 13** *Deviner en un minimum d'essais* ▶ corrigé p. 631

On joue à deviner un nombre entre `1` et `100` : à chaque proposition, on répond *plus petit* ou *plus grand*.

**1.** Quelle est la meilleure première proposition ? Pourquoi ?

**2.** Combien d'essais suffisent, dans le pire des cas, en jouant de façon **optimale** ? Relier ce nombre à la dichotomie.

▷ coup de pouce

★★★ **Exercice 14** *L'ordinateur devine : la preuve* ▶ corrigé p. 631 ≈ 30 min

Au chapitre *Les bases de la programmation Python*, le défi « L'ordinateur devine » du TP « Le nombre mystère » vous a fait programmer un ordinateur qui devine un nombre en proposant toujours le **milieu**. On reprend ici ce programme, non pour le réécrire, mais pour l'**analyser** avec les outils du cours (la fonction `comparer` du TP est remplacée par des comparaisons directes).

```
1  def essais_ordinateur(secret, maxi):
2      """Simule la strategie du milieu ; renvoie le nombre d'essais.
3      Precondition : 1 <= secret <= maxi."""
4      mini = 1
5      essais = 0
6      trouve = False
7      while not trouve:
8          proposition = (mini + maxi) // 2
9          essais = essais + 1
10         if proposition < secret:
11             mini = proposition + 1
12         elif proposition > secret:
13             maxi = proposition - 1
14         else:
15             trouve = True
16     return essais
```

1. Comparer avec `recherche_dichotomique` du cours : quel rôle jouent `mini`, `maxi` et `proposition` ? Dans quel « tableau trié » l'ordinateur cherche-t-il ?
2. **Terminaison.** Montrer que `maxi - mini` diminue strictement à chaque essai raté. Grâce à la précondition, le secret reste toujours entre `mini` et `maxi` (c'est un invariant). En déduire que la boucle s'arrête. Comment appelle-t-on la quantité `maxi - mini` ?
3. ▶ Appeler `essais_ordinateur(150, 100)`, qui **ne respecte pas** la précondition (interrompre le programme au besoin). Expliquer ce qui se passe en suivant les valeurs de `mini` et `maxi`. Quelle condition de boucle, utilisée dans le cours, aurait évité ce problème ?
4. **Coût.** Sans exécuter le programme, donner le nombre d'essais dans le pire des cas pour `maxi` égal à 100, 1000 et un million, en justifiant avec le raisonnement du cours. Combien d'essais faudrait-il, dans le pire des cas, à un ordinateur « naïf » qui propose 1, puis 2, puis 3… ?

▷ coup de pouce ▷ début de solution

★★★ **Exercice 15** *La racine carrée entière* ▶ corrigé p. 632 ≈ 20 min

▶ La dichotomie ne sert pas qu'à chercher dans un tableau : elle permet de chercher un **nombre** dans un intervalle. Écrire `racine_entiere(n)` qui renvoie le plus grand entier `r` tel que `r * r <= n` (`n` entier positif), par dichotomie sur les entiers de `0` à `n`, sans utiliser `**` ni le module `math`.

```
>>> racine_entiere(50)
7
>>> racine_entiere(49)
7
```

Combien de tours de boucle environ pour `n = 1000000` ? Comparer avec l'essai de tous les entiers `0, 1, 2...` un par un.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 16** *Le mot dans le dictionnaire* ▶ corrigé p. 632

Un dictionnaire de langue (le livre !) est une liste de mots triée par ordre alphabétique. Ici, on la représente par un **tableau** trié de chaînes (et non par un dictionnaire Python). En Python, `"chat" < "chien"` compare deux chaînes dans l'ordre du dictionnaire.

1. ▶ Adapter `recherche_dichotomique` en `est_dans_dico(dico, mot)` qui cherche un **mot** dans un tableau de mots trié.

   ▷ coup de pouce

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.
2. Sur un dictionnaire de 100 000 mots, combien de comparaisons faut-il au maximum pour savoir si un mot y figure ?

▷ coup de pouce

★★ **Exercice 17** *La dichotomie selon un assistant d'IA* ▶ corrigé p. 633

Un élève demande à un assistant d'IA : « Écris une fonction Python rapide qui dit si une valeur est dans un tableau. » Voici la réponse obtenue.

```
def contient(t, x):
    deb = 0
    fin = len(t) - 1
    while deb <= fin:
        mil = (deb + fin) // 2
        if t[mil] == x:
            return True
        elif x > t[mil]:
            deb = mil + 1
        else:
            fin = mil - 1
    return False
```

« C'est une recherche dichotomique : à chaque tour, la zone de recherche est divisée par deux, d'où un coût logarithmique. Elle fonctionne sur n'importe quel tableau, trié ou non, puisqu'on compare `x` à l'élément du milieu jusqu'à le trouver. »

1. La réponse est-elle correcte ? Dérouler `contient([3, 9, 1, 7], 1)` à la main (valeurs de `deb`, `fin` et `mil` à chaque tour). ▷ coup de pouce
2. Localiser l'erreur (dans le code ou dans l'explication) et la corriger.
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

## Entraînement ↑ cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

★ **Exercice 18** *Dérouler la dichotomie* ▶ corrigé p. 633 • essentiel

On utilise le tableau trié `t = [2, 5, 7, 10, 13, 18, 21, 26, 30]` (indices 0 à 8). Pour chaque valeur cherchée, donner à chaque tour `deb`, `fin`, `mil`, `t[mil]` et la décision, puis le résultat et le nombre de tours : a) `x = 21` ; b) `x = 2` ; c) `x = 14` ; d) `x = 30`. ▷ coup de pouce

★ **Exercice 19** *Combien de valeurs examinées au plus ?* ▶ corrigé p. 633 • essentiel

Sans machine, donner le nombre maximal de valeurs examinées par une recherche dichotomique dans un tableau trié de : a) 7 éléments ; b) 15 éléments ; c) 50 éléments ; d) 200 éléments ; e) un million d'éléments. ▷ coup de pouce

## Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Au départ, `deb = 0` et `fin = 10`. Le milieu est `(deb + fin) // 2`. Si `x` est plus grand que `t[mil]`, on garde la moitié droite : `deb` devient `mil + 1`. ↑ énoncé

Ex. 2 – piste Continuer les tours tant que `deb <= fin`. Que valent `deb` et `fin` à la fin ? Entre quelles valeurs du tableau 20 aurait-il dû se trouver ? ↑ énoncé

Ex. 3 q. 1 – piste Au premier tour, comparer `9` à `t[mil]` : de quel côté l'algorithme poursuit-il sa recherche ? Est-ce de ce côté que se trouve `9` ? ↑ énoncé

Ex. 4 – piste `fin` est le dernier *indice* du tableau. La zone `deb...fin` est vide quand `deb` dépasse `fin`. Comme `t[mil]` vient d'être examiné, on l'exclut de la nouvelle zone. ↑ énoncé

Ex. 5 – piste Deux indices `deb` et `fin` délimitent la zone où `x` peut encore se trouver. Que faire tant que cette zone n'est pas vide ? Que conclure quand elle l'est devenue ? ↑ énoncé

Ex. 5 – début de solution

```
deb = 0
fin = len(t) - 1
```

```
while deb <= fin:
    mil = (deb + fin) // 2
```
↑énoncé

**Ex. 6 – piste** Partir de `recherche_dichotomique` : que faut-il renvoyer à la place de `True` quand `t[mil] == x` ? et à la place de `False` ? ↑énoncé

**Ex. 7 – piste** Un compteur initialisé à `0`, augmenté de 1 au début de chaque tour ; attention, il y a deux sorties possibles (trouvé, absent) et il faut le renvoyer dans les deux. ↑énoncé

**Ex. 7 q. 2 – piste** Avec combien d'éléments, au minimum, une recherche peut-elle durer 4 tours ? Chercher une valeur placée tout au bout, ou plus grande que toutes. ↑énoncé

**Ex. 8 – piste** Quand on tombe sur `x` en `mil`, est-ce forcément la première occurrence ? De quel côté peut se cacher une occurrence plus à gauche ? ↑énoncé

**Ex. 8 – début de solution** Quand `t[mil] == x` : mémoriser `mil` dans une variable `resultat` (initialisée à `-1`), puis **continuer** la recherche à gauche avec `fin = mil - 1` au lieu de s'arrêter. ↑énoncé

**Ex. 9 q. 1 – piste** Dans le cas « à droite », `deb` devient `mil + 1`. Or `mil` est compris entre `deb` et `fin` : comparer la nouvelle valeur de `deb` à l'ancienne. Même raisonnement pour `fin`. ↑énoncé

**Ex. 10 – piste** Combien de fois faut-il diviser la taille par 2 pour arriver à 1 ? ↑énoncé

**Ex. 11 – piste** Garder deux variables : `k` et la puissance $2^k$ correspondante, que l'on double à chaque tour. Vérifier à la main avec `n = 1` puis `n = 2`. ↑énoncé

**Ex. 12 – piste** Réutiliser la fonction `nb_tours` de l'exercice « Compter les tours », appelée dans une boucle `for` sur la liste des tailles. ↑énoncé

**Ex. 13 q. 2 – piste** Quelle proposition élimine le plus de nombres, quelle que soit la réponse ? Combien de nombres restent possibles après chaque essai, dans le pire des cas ? ↑énoncé

**Ex. 14 – piste** Question 2 : examiner séparément les cas « trop petit » et « trop grand », comme pour `fin - deb` dans le cours. Question 3 : afficher `mini` et `maxi` à chaque tour avec un `print` ; que vaut `proposition` quand `mini` dépasse `maxi` ? ↑énoncé

**Ex. 14 – début de solution** Question 4 : le nombre de candidats est au moins divisé par deux à chaque essai raté ; chercher la plus petite puissance de 2 qui dépasse `maxi` (ou utiliser `nb_tours_max`). ↑énoncé

**Ex. 15 – piste** Si `mil * mil <= n`, `mil` convient, mais la réponse est peut-être plus grande : de quel côté continuer ? Et si `mil * mil > n` ? ↑énoncé

**Ex. 15 – début de solution** Même schéma que « La première occurrence » : `deb = 0`, `fin = n`, `r = 0` ; dans la boucle, quand `mil * mil <= n`, on mémorise `r = mil` puis `deb = mil + 1`. ↑énoncé

**Ex. 16 q. 1 – piste** Les opérateurs `<`, `>` et `==` comparent aussi des chaînes : que reste-t-il à changer dans le code ? ↑énoncé

**Ex. 16 q. 2 – piste** Chercher la plus petite puissance de 2 qui dépasse 100 000 (ou utiliser `nb_tours_max`). ↑énoncé

**Ex. 17 q. 1 – piste** Noter à chaque tour `deb`, `fin`, `mil` et `t[mil]`. La case qui contient `1` est-elle examinée à un moment ? ↑énoncé

**Ex. 18 – piste** Même tableau de trace que dans les exercices 1 et 2. Au premier tour, `mil` vaut `(0 + 8) // 2`. Pour une valeur absente, s'arrêter quand `deb` dépasse `fin`. ↑énoncé

**Ex. 19 – piste** C'est le plus petit entier $k$ tel que $2^k > n$ (exercice « Le plus petit `k` »). Écrire la suite des puissances de 2 : $1, 2, 4, 8, 16 \ldots$ ↑énoncé

# TP ET PROJETS

## TP L'annuaire et le devin

▶ corrigé p. 633

*la recherche dichotomique en vrai : jouer, chercher, mesurer, compléter*

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p07-tp-annuaire-devin` — **archive .zip**

**Durée** 2 h à 2 h 30 (une à deux séances), en binôme, sur machine.

**Prérequis** le cours *La recherche dichotomique* (algorithme, variant, coût logarithmique) ; la recherche séquentielle et la notion de coût (chapitre *Algorithmique : le parcours séquentiel*).

**Fichiers** `tp_dichotomie_annuaire.py` (fabrique l'annuaire, fourni, à ne pas modifier) et `tp_dichotomie_depart.py` (à compléter). Les deux fichiers doivent être dans le **même dossier**.



### But du TP

On va mettre la dichotomie au travail sur deux vrais problèmes :

1. un **devin** : un programme qui trouve le nombre auquel *vous* pensez, et qui démasque les tricheurs ;
2. un **annuaire** (fictif) de 16 000 abonnés : retrouver un numéro par recherche séquentielle puis par dichotomie, **compter** et **chronométrer** les deux, et tracer la courbe du coût logarithmique ;
3. une **autocomplétion** : taper le début d'un nom et obtenir les premières suggestions, instantanément.

**Produit final** : le devin, un annuaire interrogeable avec autocomplétion, un graphique « mesuré contre $\log_2 n$ » et un bilan rédigé.

### Consignes

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme. ▶ *À programmer* dans `tp_dichotomie_depart.py`.
- On écrit les recherches soi-même : pas de `in`, `index` ni `bisect` (sauf quand une question le demande, pour comparer).
- En bas du fichier, le programme principal contient des lignes en commentaire (`#`) : on les **décommente au fur et à mesure**. Des tests `assert` sont fournis.
- Le compte rendu (tableaux, réponses, bilan) se fait sur le cahier (ou dans un fichier texte).
- Usage de l'IA, indiqué sur chaque exercice : IA aucune ; IA en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; IA l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Le devin

Cette fois, c'est **vous** qui pensez à un nombre et **l'ordinateur** qui cherche. À chaque proposition, vous répondez au clavier `+` (votre nombre est plus grand), `-` (plus petit) ou `=` (trouvé).

★★ **Exercice 1** *Programmer le devin*

1. ▶ Écrire `devin(bas, haut)` qui cherche par dichotomie un nombre entre `bas` et `haut`, en posant ses questions avec `input`. Elle affiche le nombre d'essais quand elle a trouvé et le **renvoie**.

```
>>> devin(1, 100)
Je propose 50 (+, - ou =) ? +
Je propose 75 (+, - ou =) ? -
Je propose 62 (+, - ou =) ? =
Trouvé en 3 essais !
3
```

▷ coup de pouce

2. ▶ Jouer cinq parties entre 1 et 100. Quel est le nombre maximal d'essais observé ? Pourquoi ne dépasse-t-on jamais 7 ?
3. ▶ Jouer entre 1 et 1 000 000 (penser à un nombre à six chiffres). Combien d'essais ? Combien, au pire ?

★★ **Exercice 2** *Démasquer le tricheur*

Un joueur malhonnête répond au hasard. Que fait votre programme ?

1. Entre 1 et 10, répondre `+`, `-`, `+`, `-`… Que se passe-t-il ? Quelles valeurs prennent `bas` et `haut` à la fin ?
2. ▶ Modifier `devin` : si la zone de recherche devient **vide**, elle affiche « Impossible : vos réponses se contredisent. Tricheur ! » et renvoie `-1`.
3. Expliquer pourquoi la zone vide **prouve** la tricherie. *(Que contient la zone* `[bas ; haut]` *à chaque tour si le joueur est honnête ? C'est un invariant.)*
4. Justifier, avec le **variant** du cours, que le devin s'arrête toujours, même face à un tricheur.

## 2 L'annuaire

★ **Exercice 3** *Prise en main*

La fonction `charger_annuaire()`, fournie, renvoie deux tableaux de même longueur : `noms` (des chaînes `"NOM Prénom"`, triées par ordre alphabétique) et `numeros` : `numeros[i]` est le numéro de `noms[i]`. *Tous ces abonnés et numéros sont inventés.*

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Afficher la longueur de l'annuaire, son premier et son dernier abonné.
2. ▶ Écrire `est_trie(t)` (s'arrêter dès qu'on trouve deux voisins dans le désordre) et vérifier que `noms` est trié. Pourquoi est-ce indispensable pour la suite ?
3. En Python, deux chaînes se comparent caractère par caractère, selon leur code ; l'espace passe avant les lettres et les majuscules avant les minuscules. Dans l'annuaire, `"LE GALL Adam"` est-il avant ou après `"LEBLANC Adam"` ? Vérifier.

★★ **Exercice 4** *Deux façons de trouver un numéro*

Chaque fonction renvoie un **couple** : le numéro trouvé (ou `None` si le nom est absent) et le **nombre de comparaisons** effectuées, chaque tour de boucle comptant pour une comparaison.

1. ▶ Écrire `numero_sequentiel(noms, numeros, nom)` : parcours du début à la fin, arrêt dès qu'on trouve.
2. ▶ Écrire `numero_dicho(noms, numeros, nom)` : recherche dichotomique, qui renvoie `numeros[mil]` quand `noms[mil] == nom`.

```
>>> numero_dicho(["A", "B", "C"], ["1", "2", "3"], "B")
('2', 1)
```

▷ coup de pouce

3. ▶ Décommenter `tester_partie_B(noms, numeros)` : tous les tests doivent passer.

## 3 Mesurer

★ **Exercice 5** *Quelques recherches à la loupe*

▶ Recopier sur le cahier le tableau suivant et le compléter avec le nombre de comparaisons renvoyé par chaque fonction.

| nom cherché | séquentielle | dichotomique |
|---|---|---|
| `"ADAM Adam"` (le premier) | | |
| `"DUPONT Alice"` | | |
| `"MARTIN Hugo"` | | |
| `"VINCENT Zoe"` (le dernier) | | |
| `"ZORRO Zoe"` (absent) | | |

1. Dans quel cas la recherche séquentielle gagne-t-elle ? Est-ce un cas fréquent ?
2. Quel nom la dichotomie trouve-t-elle en **une seule** comparaison ?

★★ **Exercice 6** *En moyenne, et au chronomètre*

1. ▶ La fonction `moyenne_comparaisons`, fournie, cherche `nb_essais` noms tirés au hasard dans l'annuaire et renvoie le nombre moyen de comparaisons. La lancer avec 1000 essais pour chaque méthode et noter les deux moyennes. Expliquer la valeur obtenue pour la séquentielle (où se trouve, en moyenne, un nom tiré au hasard ?).
2. ▶ Écrire `chrono_recherches(noms, numeros, recherche, cibles)` qui renvoie la durée totale, mesurée avec `time.perf_counter()`, de la recherche de **chacun** des noms de la liste `cibles`.
3. ▶ Fabriquer une liste de 1000 noms tirés au hasard dans l'annuaire, puis chronométrer les deux méthodes **sur la même liste**. Noter les deux durées et leur rapport.
4. ▶ Chronométrer aussi `nom in noms` pour les mêmes 1000 noms. L'opérateur `in` parcourt la liste élément par élément, mais il est programmé en langage C, bien plus rapide que Python. Le classement des trois méthodes vous surprend-il ? Que se passerait-il avec un annuaire de 100 millions d'abonnés ?

★★ **Exercice 7** *La courbe du logarithme*

1. ▶ La fonction `courbe_moyennes`, fournie, mesure le nombre moyen de comparaisons de la dichotomie sur les `n` premiers abonnés (`noms[0:n]`, qui reste trié). La lancer pour `tailles = [1000, 2000, 4000, 8000, 16000]` puis recopier et compléter sur le cahier le tableau suivant :

| **taille `n`** | 1 000 | 2 000 | 4 000 | 8 000 | 16 000 |
|---|---|---|---|---|---|
| **comparaisons (moyenne)** | | | | | |
| $\log_2 n$ (`log2`, fournie) | | | | | |

2. De combien augmente le nombre moyen de comparaisons quand la taille **double** ? Comparer à $\log_2 n$. Quelle phrase du cours retrouve-t-on ?

3. ▶ Décommenter `tracer(...)` et décrire les deux courbes.

4. **Prédire.** Combien de comparaisons faudrait-il, au plus, pour chercher un nom dans un annuaire de 8 milliards de personnes ? et par recherche séquentielle ?

## 4 L'autocomplétion

On tape `DUP` dans la barre de recherche : l'annuaire doit proposer **aussitôt** les premiers noms qui commencent par `DUP`. Dans un tableau trié, tous ces noms sont **côte à côte** : il suffit de trouver où commence le bloc.

★★★ **Exercice 8** *Où commence le bloc ?*

1. ▶ Écrire `premier_indice_au_moins(noms, debut_nom)` qui renvoie le **plus petit** indice `i` tel que `noms[i] >= debut_nom` (et `len(noms)` s'il n'y en a aucun), par dichotomie. *On ne s'arrête plus quand on « trouve » : on mémorise le candidat* `mil` *et l'on continue à chercher plus à gauche.*

```
>>> premier_indice_au_moins(["A", "C", "E"], "B")
1
>>> premier_indice_au_moins(["A", "C", "E"], "Z")
3
```

▷ coup de pouce

2. Pourquoi le premier nom qui commence par `"DUP"` est-il exactement le premier nom `>= "DUP"` ?

3. ▶ Écrire `suggestions(noms, prefixe, k)` qui renvoie au plus `k` noms commençant par `prefixe`, dans l'ordre. *La chaîne* `s` *commence par* `p` *lorsque la tranche* `s[:len(p)]` *est égale à* `p`.

```
>>> suggestions(noms, "DUP", 3)
['DUPONT Adam', 'DUPONT Adele', 'DUPONT Agathe']
```

4. ▶ Écrire `nb_commencant_par(noms, prefixe)`, puis décommenter `tester_partie_D(noms)`. Combien d'abonnés commencent par `"MAR"` ? par `"LE"` ?

5. Combien de comparaisons coûte `suggestions(noms, prefixe, k)` au plus, en fonction de $n$ et de $k$ ? Pourquoi l'autocomplétion reste-t-elle instantanée sur un annuaire géant ?

## 5 Bonus : la dichotomie sur les nombres à virgule

★★★ **Exercice 9** *Encadrer une racine carrée*

On cherche $\sqrt{a}$ pour un nombre $a \geq 1$ : elle est entre 0 et $a$. On coupe l'intervalle `[bas ; haut]` en deux et l'on garde la moitié qui contient la racine, **tant que** sa largeur dépasse la précision voulue. Cette fois, on ne cherche pas dans un tableau : on cherche un **nombre** dans un intervalle.

1. ▶ Écrire `racine(a, precision)` qui renvoie le milieu du dernier intervalle et le nombre de tours effectués (`mil = (bas + haut) / 2`, division *décimale*). Comparer `racine(2, 0.000001)` à `2 ** 0.5`. ▷ coup de pouce
2. Combien de tours pour $a = 2$ puis $a = 1\,000\,000$, à la précision $10^{-6}$ ? Expliquer ces nombres : par combien la largeur de l'intervalle est-elle divisée à chaque tour ? Combien de divisions faut-il pour passer d'une largeur $a$ à une largeur $10^{-6}$ ?

### Bilan à rédiger

Sur le cahier, en huit à dix lignes, **en citant vos mesures** :

- la condition indispensable pour utiliser la dichotomie, et ce qu'elle rapporte (comparaisons, temps) par rapport à la recherche séquentielle ;
- ce que signifie « coût logarithmique » quand on double la taille des données ;
- une situation où la dichotomie ne cherche pas dans un tableau (devin, racine) ;
- ce que les mesures au chronomètre ajoutent (ou pas) au simple comptage des comparaisons.

### Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 q. 1 – piste C'est la recherche dichotomique du cours : `bas` et `haut` jouent le rôle de `deb` et `fin`, la proposition celui de `mil`, et la réponse au clavier remplace la comparaison avec `t[mil]`. ↑ énoncé

Ex. 4 q. 2 – piste Reprendre la recherche dichotomique du cours. Les opérateurs `==`, `<` et `>` comparent des chaînes comme des nombres. Le compteur augmente de 1 au début de chaque tour, et on le renvoie dans les deux sorties (trouvé, absent). ↑ énoncé

Ex. 8 q. 1 – piste Une variable `res`, initialisée à `len(noms)`. Si `noms[mil] >= debut_nom`, `mil` est un candidat : `res = mil`, puis `fin = mil - 1`. Sinon, `deb = mil + 1`. On renvoie `res` quand la zone est vide. ↑ énoncé

Ex. 9 q. 1 – piste Si `mil * mil <= a`, la racine est dans `[mil ; haut]` : on fait `bas = mil`. Sinon, `haut = mil`. Ici, pas de `+ 1` ni de `- 1` : ce ne sont plus des indices. ↑ énoncé

# FICHE DE RÉVISION

## L'idée — le jeu du nombre mystère

« Je pense à un nombre entre 1 et 100. » On propose toujours le **milieu**, puis on **coupe en deux** l'intervalle restant. En 7 essais au maximum, c'est gagné : c'est la **recherche dichotomique**. *Elle exige un tableau* ***trié****.*

## L'algorithme

```
def recherche_dichotomique(t, x):
    deb = 0
    fin = len(t) - 1
    while deb <= fin:              # tant que la zone n'est pas vide
        mil = (deb + fin) // 2
        if t[mil] == x:
            return True
        elif x > t[mil]:
            deb = mil + 1          # on garde la moitie droite
        else:
            fin = mil - 1          # on garde la moitie gauche
    return False                   # zone vide (deb > fin) : absent
```

## Terminaison et coût

**Terminaison** : `fin - deb` (le **variant**) décroît strictement à chaque tour ⇒ la boucle s'arrête. **Coût logarithmique** : on **divise par deux** le nombre de candidats à chaque tour, d'où au plus $\lfloor \log_2 n \rfloor + 1$ tours (nombre de divisions par 2 pour arriver à 1, plus un : 5 tours pour $n = 16$).

| taille `n` | 100 | 1 000 | 1 000 000 | 1 000 000 000 |
|---|---|---|---|---|
| séquentielle ($\approx n$) | 100 | 1 000 | 1 000 000 | 1 000 000 000 |
| dichotomique ($\approx \log_2 n$) | 7 | 10 | 20 | 30 |

**Version « position »** : renvoyer `mil` au lieu de `True`, et `-1` (ou `None` : suivez l'énoncé) au lieu de `False`.

## Erreurs à éviter

- tableau **non trié** ;
- `deb` / `fin` / `mil` mal mis à jour (boucle infinie) ;
- oublier le cas « absent » (`deb > fin`) ;
- confondre variant (termine) et invariant (correct).

## Savoir-faire à maîtriser — je sais…

☐ **dérouler** une recherche dichotomique et compter les tours ;

☐ écrire la **recherche dichotomique** (indices `deb` / `fin` / `mil`) ;

☐ justifier la **terminaison** par un **variant** (`fin - deb` décroît) ;

☐ justifier le **coût logarithmique** (on divise par deux à chaque tour) ;

☐ reconnaître l'**hypothèse** indispensable : le tableau est **trié**.

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 638

1. Quelle condition le tableau doit-il remplir pour une recherche dichotomique ?
2. Avec `deb = 2` et `fin = 9`, que vaut `mil` ?
3. Nombre maximal de tours pour un tableau trié de 1 000 éléments ?
4. Tant que la boucle `while deb <= fin` tourne, quel est le signe de `fin - deb` ? Pourquoi est-ce utile ?
5. Pourquoi écrit-on `mil = (deb + fin) // 2` et non `(deb + fin) / 2` ?

**Et avant ?** — *questions sur les chapitres précédents.* ► réponses p. 638

1. Quel est le coût du tri par insertion sur un tableau déjà trié ? (ch. 6)
2. Combien de comparaisons fait une recherche séquentielle dans le pire cas, sur 1 000 éléments ? (ch. 4)
3. Que vaut `n != 0 and 10 / n > 1` si `n` vaut `0` ? Pourquoi n'y a-t-il pas d'erreur ? (ch. 1)

# DÉFI ADVENT OF CODE

## DÉFI High-Entropy Passphrases

*les phrases de passe — doublons et anagrammes*

Jour 4

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p07-defi-aoc-2017-04` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2017/day/4`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Le fichier contient plusieurs centaines de « phrases de passe », des suites de mots. Il faut compter celles qui ne contiennent aucun mot en double (partie 1), puis celles qui ne contiennent aucune paire de mots formés des mêmes lettres (partie 2). Comparer chaque mot à tous les autres, ligne après ligne, est un travail long et sans intérêt à la main : c'est le rôle du programme.

**Ce qu'on réinvestit.** Le **tri** du chapitre précédent : pour savoir si deux mots sont formés des mêmes lettres, on range les lettres de chacun dans l'ordre alphabétique, et il suffit alors de comparer les résultats avec `==`. L'approfondissement 1 fait le lien avec ce chapitre : dans une liste triée, les doublons se retrouvent côte à côte, et un élément se retrouve vite par **recherche dichotomique**.

**La démarche de la fiche.**

1. Comprendre la règle sur des cas particuliers, puis l'appliquer à la main sur cinq lignes.
2. Lire le fichier et découper chaque ligne en liste de mots.

3. Écrire une fonction qui dit si une liste ne contient aucun doublon : on obtient la partie 1.
4. Pour la partie 2, calculer la « signature » triée de chaque mot, puis réutiliser la même fonction sur la liste des signatures.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `passphrase` | phrase de passe | `valid` | valide |
| `duplicate` | en double | `separated by spaces` | séparés par des espaces |
| `word` | mot | `policy` | règle, politique |
| `anagram` | anagramme | `rearrange` | réarranger, permuter |

### Résumé de l'énoncé (partie 1)

**Contexte.** Un nouveau système informatique protège ses comptes par des « phrases de passe » plutôt que par des mots de passe. Pour qu'une phrase soit assez sûre, elle doit respecter une règle de construction. On dispose d'une liste de phrases de passe et l'on veut savoir combien d'entre elles sont acceptables.
**Les données.** Le fichier contient une phrase de passe par ligne. Une phrase est formée de plusieurs mots en lettres minuscules, séparés par des espaces. Le nombre de mots varie d'une ligne à l'autre.
**La règle.**

- Une phrase est valide si elle ne contient **jamais deux fois le même mot**.
- Il faut que les deux mots soient exactement identiques pour faire un doublon : une simple ressemblance (un mot contenu dans un autre, les mêmes lettres dans un autre ordre) ne rend pas la phrase invalide.
- Une phrase d'un seul mot est donc toujours valide.

**La réponse attendue.** Le nombre de lignes valides du fichier. Par exemple, la ligne `vert bleu vert` est invalide (`vert` deux fois), alors que `vert verte` est valide.

### Exercice 1 *Comprendre ce qui est demandé*

**But.** Vérifier sa compréhension de la règle sur des cas particuliers.
**À quoi ça sert ?** Il faut savoir exactement ce qui est un doublon et ce qui n'en est pas avant d'écrire le test en Python. La troisième question prépare déjà la partie 2.

Après lecture du résumé (et de l'énoncé), vérifier sa compréhension sur ces cas particuliers :

1. Les mots `abc` et `abcd` sont-ils considérés comme un doublon ?
2. Une ligne d'un seul mot est-elle valide ?
3. (Pour plus tard.) Les mots `ab` et `aab` utilisent les mêmes lettres : sont-ils des anagrammes ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Décider à la main quelles lignes d'un petit fichier sont valides.
**À quoi ça sert ?** On obtient un exemple dont on connaît la réponse (3) pour tester le programme ; ce même fichier resservira pour la partie 2.

On considère le fichier (inventé) suivant :

```
le chat et le chien
un deux trois
ab ba abc
mer rame arme
sol los sol
```

Indiquer pour chaque ligne si elle est valide. Vérification : on doit obtenir `3`. Les mots `ab` et `ba` sont-ils des doublons au sens de la partie 1 ?

## 2 Programmer la partie 1

### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

### Exercice 3 *Lire et découper*

**But.** Transformer chaque ligne du fichier en une liste de mots.
**À quoi ça sert ?** La règle porte sur les mots, pas sur les caractères : c'est cette liste qu'on examinera pour chercher des doublons.

Rappel pour lire un fichier ligne par ligne :

```
fichier = open("input.txt", encoding="utf-8")
for ligne in fichier:
    ligne = ligne.strip()   # enleve le retour a la ligne final
fichier.close()
```

Pour chaque ligne, construire la liste de ses mots et l'afficher. ▷ coup de pouce

### Exercice 4 *Tous distincts ?*

**But.** Écrire une fonction qui dit si une liste ne contient aucun doublon, puis compter les lignes valides.
**À quoi ça sert ?** C'est la réponse de la partie 1. La fonction, écrite pour une liste quelconque, sera réutilisée sans modification dans la partie 2.

Écrire une fonction `tous_distincts(mots)` qui renvoie `True` si la liste `mots` ne contient aucun doublon. Tester sur les lignes de l'exemple, puis compter les lignes valides de vos données.

▷ coup de pouce ▷ début de solution

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

### Résumé de la partie 2 (à lire après avoir validé la partie 1)

**Ce qui change.** Le service de sécurité durcit la règle : un mot ne doit pas pouvoir s'obtenir en mélangeant les lettres d'un autre mot de la même ligne.
**La nouvelle règle.**

- Une phrase est valide si aucun de ses mots n'est une **anagramme** d'un autre de ses mots.
- Deux mots sont anagrammes s'ils ont les mêmes lettres, chacune en même nombre, dans un ordre éventuellement différent.
- Deux mots identiques sont aussi anagrammes : une ligne invalide dans la partie 1 reste invalide.

**La réponse attendue.** Le nombre de lignes valides avec cette nouvelle règle. Par exemple, `pot top` est maintenant invalide, mais `pot pots` reste valide (un `s` de plus).



### Méthode : ranger les lettres d'un mot dans l'ordre alphabétique

Une chaîne ne se modifie pas. On passe donc par une **liste** de caractères, que l'on trie avec le tri par insertion du chapitre précédent (les caractères se comparent avec `<` et `>`, dans l'ordre alphabétique pour les minuscules), puis on recolle les caractères un par un dans une chaîne.

```
1  def tri_insertion(t):
2      n = len(t)
3      for i in range(1, n):
4          x = t[i]
5          j = i - 1
6          while j >= 0 and t[j] > x:
7              t[j + 1] = t[j]
8              j = j - 1
9          t[j + 1] = x
10
11 lettres = [c for c in "rame"]   # liste des caracteres
12 tri_insertion(lettres)
13 print(lettres)                  # ['a', 'e', 'm', 'r']
14 mot_trie = ""
15 for c in lettres:               # on recolle les caracteres
16     mot_trie += c
17 print(mot_trie)                 # aemr
```

**Intérêt.** Une chaîne se compare directement à une autre avec `==` ; une liste comparée à une chaîne donne toujours `False`. Recoller la liste évite ce piège.
*À essayer.* Que vaut `["a", "e", "m", "r"] == "aemr"` ? Que donne la même méthode sur le mot `"banane"` ?

### Exercice 5 *Reconnaître deux anagrammes*

**But.** Écrire une fonction qui range les lettres d'un mot dans l'ordre alphabétique.
**À quoi ça sert ?** Deux anagrammes donnent la même chaîne triée : la question « ces mots sont-ils anagrammes ? » devient la question « ces chaînes sont-elles égales ? », à laquelle on sait déjà répondre.

Écrire une fonction `signature(mot)` qui renvoie la chaîne formée des lettres du mot rangées dans l'ordre alphabétique. Par exemple, `signature("rame")` doit valoir `"aemr"`. Expliquer pourquoi deux mots sont anagrammes l'un de l'autre si et seulement s'ils ont la même signature. ▷ coup de pouce

### Exercice 6 *La partie 2*

**But.** Répondre à la partie 2 en combinant les deux fonctions déjà écrites.
**À quoi ça sert ?** En remplaçant chaque mot par sa signature, une anagramme devient un doublon : la fonction de la partie 1 s'applique alors telle quelle. C'est l'intérêt d'avoir écrit des fonctions réutilisables.

En réutilisant `signature` et `tous_distincts`, répondre à la nouvelle question. Sur le petit exemple inventé de l'exercice 2, on doit trouver `1`. ▷ coup de pouce

## 4 Approfondissement

★★ **Exercice 7** *Approfondissement 1 : trier pour rapprocher les doublons*

**But.** Détecter les doublons d'une liste en la triant d'abord, puis en ne comparant que des voisins.
**À quoi ça sert ?** On compare le coût de deux méthodes pour le même problème, et l'on fait le lien avec la recherche dichotomique de ce chapitre, qui exige elle aussi une liste triée.

Une autre idée pour `tous_distincts` réutilise le **tri**.

1. À la main : trier les mots de la ligne `le chat et le chien`. Où se trouvent les deux `le` dans la liste triée ?
2. Expliquer pourquoi, dans une liste triée, il suffit de comparer chaque élément à son **voisin** pour savoir s'il y a un doublon.
3. Écrire `tous_distincts_tri(mots)` avec une seule boucle et vérifier qu'elle donne la même réponse que la partie 1. ▷ coup de pouce ▷ début de solution
4. Pour une ligne de $k$ mots, comparer le nombre de comparaisons de la double boucle et de cette méthode, au pire (le tri par insertion fait au pire $\frac{k(k-1)}{2}$ comparaisons). Le gain serait-il le même avec un tri qui ne fait qu'environ $k \log_2 k$ comparaisons (on en verra un en Terminale) ?
5. **Lien avec la dichotomie.** Une fois la liste de mots triée, comment savoir si un mot donné y figure sans la parcourir en entier ? Combien de comparaisons faut-il au plus pour une liste de 1 000 mots ? ▷ coup de pouce

### Outil Python : codes des caractères (`ord`, `chr`)

Chaque caractère a un numéro (son code). `ord` donne le code d'un caractère, `chr` fait l'inverse. Les lettres minuscules ont des codes consécutifs.

```
print(ord("a"))              # 97 : code du caractere
print(ord("c") - ord("a"))   # 2 : rang de "c" dans l'alphabet (a -> 0)
print(chr(98))               # b : operation inverse
compteurs = [0] * 26
compteurs[ord("c") - ord("a")] += 1    # on compte un "c"
```

**Intérêt.** `ord(c) - ord("a")` transforme une lettre en un indice de 0 à 25 : on peut alors compter les lettres dans une simple liste de 26 entiers.
*À essayer.* Que vaut `ord("z") - ord("a")` ? Et `chr(ord("a") + 3)` ?

★★ **Exercice 8** *Approfondissement 2 : une signature sans tri*

**But.** Remplacer la signature triée par le tableau des effectifs des 26 lettres.
**À quoi ça sert ?** Cette signature se calcule en un seul parcours du mot, sans tri ; on vérifie qu'elle donne la même réponse à la partie 2, ce qui confirme les deux méthodes.

1. À la main : donner les effectifs des lettres `a`, `e`, `m`, `r` dans `rame` et dans `arme`. Que constate-t-on ?
2. Écrire `signature26(mot)` qui renvoie la liste des 26 effectifs (nombre de `a`, de `b`…) du mot, puis refaire la partie 2 avec cette signature. Obtient-on la même réponse ? ▷ coup de pouce

▷ début de solution

**Outil Python : les ensembles (`set`)**

Un **ensemble** (`set`) est une collection **sans doublon** et **sans ordre** : pas d'indice, on peut seulement ajouter un élément, en retirer un, et demander s'il est présent. On peut y ranger des nombres, des chaînes ou des p-uplets comme `(0, 0)`. Cet encadré sert aussi pour les défis des chapitres suivants.

```
vus = set()                 # ensemble vide (attention : {} est un dictionnaire vide)
vus.add("a")                # ajout
vus.add("b")
vus.add("a")                # deja present : rien ne change
print(len(vus))             # 2
print("a" in vus)           # True : test d'appartenance
print(len(set("nsin")))     # 3 : les caracteres distincts de "nsin"
a = {1, 2, 3}               # ensemble ecrit directement
b = {2, 3, 4}
print(a | b)                # reunion : {1, 2, 3, 4}
print(a & b)                # intersection (elements communs) : {2, 3}
```

**Intérêt.** Avec une liste, `x in liste` compare `x` aux éléments un par un : jusqu'à $n$ comparaisons. Un ensemble range chaque élément à un endroit calculé à partir de sa valeur (comme les clés d'un dictionnaire) : Python va directement voir à cet endroit. Le test `x in ensemble` prend un temps qui ne dépend pas du nombre d'éléments.
*À essayer.* Que vaut `len(set(["le", "chat", "le"]))` ?

★★ **Exercice 9** *Approfondissement 3 : tous distincts en une ligne*

**But.** Réécrire `tous_distincts` à l'aide d'un ensemble.
**À quoi ça sert ?** Un ensemble supprime automatiquement les doublons : comparer les longueurs suffit. On découvre un outil qui servira dans les défis des chapitres suivants.

Un ensemble ne garde qu'un exemplaire de chaque élément. En déduire une version de `tous_distincts(mots)` en une seule ligne, et la tester sur l'exemple. ▷ coup de pouce

**Coups de pouce** *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste `ligne.split()` (sans argument) découpe la chaîne sur les espaces et renvoie une liste de chaînes. ↑ énoncé

Ex. 4 – piste Deux boucles imbriquées sur les indices `i` et `j > i` : dès que `mots[i] == mots[j]`, la liste contient un doublon. ↑ énoncé

Ex. 4 – début de solution `for i in range(len(mots)):` puis `for j in range(i + 1, len(mots)):` ; renvoyer `False` dès que `mots[i] == mots[j]`, et `True` après les deux boucles. ↑ énoncé

Ex. 5 – piste Reprendre la méthode de l'encadré : liste des caractères, tri par insertion, puis une boucle qui recolle les caractères dans une chaîne vide au départ. ↑ énoncé

Ex. 6 – piste Pour une ligne, construire la liste des signatures de ses mots (boucle et `append`), puis appliquer la fonction de la partie 1 à cette nouvelle liste. ↑énoncé

Ex. 7 q. 3 – piste Travailler sur une copie `t = [m for m in mots]`, triée avec `tri_insertion(t)`, pour ne pas modifier la liste de départ. ↑énoncé

Ex. 7 q. 3 – début de solution Comparer `t[i]` et `t[i + 1]` pour `i` de `0` à `len(t) - 2` ; attention à ne pas sortir de la liste. ↑énoncé

Ex. 7 q. 5 – piste Les chaînes se comparent avec `<` dans l'ordre alphabétique : la recherche dichotomique du cours s'applique telle quelle. ↑énoncé

Ex. 8 q. 2 – piste Partir de `[0] * 26` et ajouter 1 à la case d'indice `ord(lettre) - ord("a")` pour chaque lettre. ↑énoncé

Ex. 8 q. 2 – début de solution Deux listes se comparent directement avec `==` : la fonction `tous_distincts` marche telle quelle sur une liste de signatures-listes. ↑

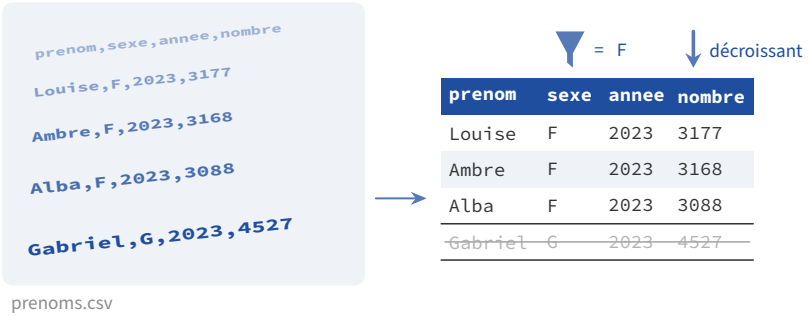
Ex. 9 – piste Comparer la longueur de la liste et celle de l'ensemble de ses éléments. ↑énoncé

8

CHAPITRE 8

# Les données en tables






prenoms.csv

## DANS CE CHAPITRE


- Activités préparatoires . . . . . 309
- Cours . . . . . 311
- Exercices . . . . . 323
- TP et projets . . . . . 331
- Fiche de révision . . . . . 342
- Défi Advent of Code . . . . . 344
- Corrigés . . . . . 639

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 Ouvrir un fichier CSV

▶ corrigé p. 639

*ce que contient vraiment un fichier de données*

Durée : 20 min · Seul ou en binôme, sur un poste

**Fichiers à télécharger :** `maththibaud.github.io/informatique-en-autonomie/f/p08-activite-fichiers-csv` — **archive .zip**

### Consignes

- Aucun programme à écrire ; les réponses se notent sur le cahier.
- Fichiers à télécharger (lien ci-dessus) : `prenoms.csv` (les prénoms les plus donnés en France en 2023 et 2025, d'après l'INSEE), `communes.csv` et `departements.csv`.
- Outils : un **éditeur de texte** (Bloc-notes, TextEdit, l'éditeur de Python…) et un **tableur** (LibreOffice Calc ou autre).

**Exercice 1** *Dans un éditeur de texte*

Ouvrir `prenoms.csv` avec l'**éditeur de texte**.

1. Combien le fichier compte-t-il de lignes ? En quoi la première ligne est-elle différente de toutes les autres ?
2. Quel caractère sépare les valeurs sur une ligne ?
3. Que signifie, à votre avis, la ligne `Jade,F,2025,2925` ? Donner le sens de chacune des quatre valeurs.

**Exercice 2** *Dans un tableur*

Ouvrir maintenant `prenoms.csv` avec le **tableur**. Une fenêtre d'import demande le séparateur : choisir la **virgule**.

4. Combien y a-t-il de colonnes ? de lignes de données (sans compter la première) ?
5. Rouvrir le fichier en choisissant cette fois le **point-virgule** comme séparateur. Que se passe-t-il ? Pourquoi ?
6. Ouvrir `communes.csv` dans le tableur (séparateur : virgule). Comment s'affiche le code `dep` de Nice ? Et dans l'éditeur de texte ? Pourquoi cette différence peut-elle poser problème ?

**Exercice 3** *Chercher une information à la main*

Répondre **sans formule ni tri automatique**, en lisant les lignes une à une (dans l'éditeur ou dans le tableur).

7. Quel est le prénom de garçon le plus donné en 2025, et combien de fois ? Quelles lignes avez-vous dû regarder ? Lesquelles avez-vous ignorées ?
8. Combien de filles ont reçu le prénom Louise en 2023 et en 2025 réunis (un seul total) ?

**9.** Dans quelle région se trouve Menton ? Quels fichiers a-t-il fallu consulter, et grâce à quelle colonne ? Peut-on répondre à la même question pour Ajaccio ?

## Exercice 4 *Ce qu'on a découvert*

**10. Ce qu'on a découvert.** Recopier la phrase sur le cahier en complétant les trous **(a)** à **(e)** avec les mots : *texte*, *en-tête*, *enregistrement*, *séparateur*, *colonne*.

« Un fichier CSV est un simple fichier … **(a)**. Sa première ligne, l'… **(b)**, donne le nom de chaque … **(c)**. Chaque ligne suivante est un … **(d)** : elle décrit un individu. Sur une ligne, les valeurs sont séparées par un … **(e)**, qu'il faut connaître pour bien lire le fichier. »

# COURS

**Programme (BO)** *« Données en table »* : *indexation* d'une table ; *recherche* de lignes répondant à un critère ; *tri* d'une table selon les valeurs d'un descripteur ; *fusion* de deux tables ayant un descripteur commun. *Importer* une table depuis un fichier au format **CSV**.

**Prérequis** les **listes** et les **dictionnaires** (chapitre *Les types construits*) ; le **patron du parcours** et la **notion de coût** (chapitre *Algorithmique : le parcours séquentiel*) ; les boucles `for` et les fonctions. → fiches de révision : ch. 3 p. 170, ch. 4 p. 202

**Objectifs** représenter une table par une **liste de dictionnaires** ; l'**importer** depuis un CSV ; **rechercher / filtrer** selon un critère ; **calculer** sur une colonne (moyenne, min, max, comptage) ; **trier** selon un descripteur ; **fusionner** deux tables.

## Situation de départ

1. Comment un programme peut-il retrouver, parmi les 700 000 lignes d'un fichier de prénoms, celles des filles nées en 2023 ?
2. Pourquoi Python affiche-t-il `'31773168'` quand on additionne les nombres `3177` et `3168` lus dans un fichier de données ?
3. Une ville est décrite dans un fichier, sa région dans un autre : comment réunir les deux informations ?

**Remarque** — *Le fil conducteur : une table, c'est un classeur de fiches*
Tout ce chapitre repose sur **une seule image**. Une table de données, c'est un **classeur** : chaque **fiche** décrit un individu (un prénom, une commune, un film…) en renseignant les mêmes **rubriques**. En Python, une fiche sera un **dictionnaire** `{rubrique : valeur}`, et le classeur entier une **liste** de ces fiches. Dès qu'on a cette image, tout le reste n'est que du **parcours** : traiter la table, c'est parcourir la liste des fiches — exactement le patron du chapitre *Algorithmique : le parcours séquentiel*. Les points qui dépassent le programme portent le badge [ au-delà du programme ].

**Remarque** — *Un projet à la clé*
Ce chapitre débouche sur l'un des **projets de l'année** : *analyser un vrai fichier de données*. On y filtre, trie et croise le fichier des **prénoms donnés en France** (données publiques de l'INSEE) — une véritable application de ce qu'on apprend ici : c'est le projet proposé en fin de chapitre « Les prénoms de l'INSEE ».

# I Descripteurs, enregistrements, format CSV

**Définition 1 — Table, descripteur, enregistrement**

Une **table de données** organise l'information en lignes et colonnes.

- un **descripteur** (ou *attribut*) est le nom d'une colonne : ce qu'on renseigne pour chacun ;
- un **enregistrement** (une *fiche*, une *ligne*) donne **une valeur pour chaque descripteur**.

Un enregistrement est donc un ***p*-uplet nommé** : une collection de couples `descripteur : valeur`.

Voici une petite table : les prénoms de filles les plus donnés en France en 2023 (chiffres INSEE non arrondis, éditions antérieures à 2026).

| prenom | sexe | annee | nombre |
|---|---|---|---|
| Louise | F | 2023 | 3177 |
| Ambre | F | 2023 | 3168 |
| Alba | F | 2023 | 3088 |
| Jade | F | 2023 | 2891 |

Ici les **descripteurs** sont `prenom`, `sexe`, `annee`, `nombre`. La ligne de Louise est **un enregistrement** ; sa valeur pour le descripteur `nombre` est `3177`.

**Définition 2 — Le format CSV**

Un fichier **CSV** (*Comma-Separated Values*, « valeurs séparées par des virgules ») est un simple fichier **texte** qui représente une table :

- la **première ligne** donne les descripteurs (l'*en-tête*) ;
- **chaque ligne suivante** est un enregistrement ;
- sur une ligne, les valeurs sont séparées par un **séparateur** (souvent la virgule `,` ou le point-virgule `;`).

↻ **Retour sur l'activité.** Dans l'activité « Ouvrir un fichier CSV », l'éditeur de texte montrait exactement cela : une première ligne à part (l'en-tête), puis un enregistrement par ligne, comme `Jade,F,2025,2925`. Et choisir le point-virgule comme séparateur dans le tableur mettait chaque ligne entière dans une seule colonne.

Le contenu du fichier `prenoms.csv` commence ainsi (c'est du texte, on peut l'ouvrir avec un simple éditeur) :

```
prenom,sexe,annee,nombre
Louise,F,2023,3177
Ambre,F,2023,3168
Alba,F,2023,3088
```

**Pour les curieux** — *Pourquoi un simple fichier texte, et pas un tableur ?* (facultatif)
Un fichier CSV n'est *que* du texte. C'est en apparence pauvre, mais c'est une force :

- **universel et pérenne** : n'importe quel logiciel (tableur, éditeur, programme Python) le lit, sans dépendre d'une marque ni d'une version ;
- **léger** : environ 16 Mo pour les 700 000 lignes du fichier des prénoms (4 Mo compressé) ;

- **sans limite cachée** : un fichier texte peut avoir **autant de lignes qu'on veut**. Un tableur, non.

Ce dernier point n'est pas théorique. En **octobre 2020**, l'agence de santé publique anglaise (*Public Health England*) a « perdu » **15 841** résultats de tests COVID-19 : les laboratoires transmettaient leurs données dans un fichier tableur au vieux format `.xls`, **limité à 65 536 lignes**. Une fois cette limite atteinte, les nouveaux cas n'étaient pas signalés par une erreur… ils étaient **silencieusement ignorés**. Résultat : près de **48 000** personnes ayant été en contact avec un cas positif n'ont pas été prévenues à temps. Un simple fichier CSV, lui, n'aurait rien plafonné. *La leçon : le bon format de données peut avoir des conséquences très réelles.*

**Remarque** — *Deux pièges très concrets : le séparateur et l'encodage*
**(1) Virgule ou point-virgule ?** En France, la virgule sert déjà de séparateur *décimal* (`3,14`). Pour éviter la confusion, les fichiers « à la française » utilisent le **point-virgule** `;` comme séparateur de colonnes. Il faut donc **toujours vérifier** quel séparateur est utilisé avant de traiter un fichier.
**(2) L'encodage.** Un fichier texte range chaque caractère selon un **encodage**. Le standard aujourd'hui est **UTF-8** (il connaît les accents, les emojis…). Si on ouvre un fichier avec le mauvais encodage, `é` peut devenir `Ã©`. On précisera donc `encoding="utf-8"` à l'ouverture.

**Exemple** — *Une vraie table publique : le fichier des prénoms de l'INSEE*
L'État publie des milliers de jeux de données **librement réutilisables** sur `data.gouv.fr`. Le plus célèbre en NSI est le **Fichier des prénoms** de l'INSEE : tous les prénoms donnés en France **depuis 1900**. Dans l'édition parue en juillet 2026, son en-tête réel est :

```
sexe;prenom;periode;valeur;rang
```

Le séparateur est le **point-virgule** ; `sexe` vaut `1` (garçon) ou `2` (fille) ; `periode` est l'année de naissance, `valeur` le nombre de naissances (désormais **arrondi au multiple de 5**) et `rang` le classement du prénom cette année-là. Les éditions précédentes utilisaient un autre en-tête (`sexe;preusuel;annais;nombre`) : un même jeu de données peut **changer de format**, il faut toujours lire l'en-tête avant de programmer. Le fichier compte plus de **700 000 enregistrements** pour environ 16 Mo (4 Mo une fois compressé) : le texte reste **compact**. Pour ce cours, on travaillera sur un **extrait allégé** (`prenoms.csv`, séparateur virgule, en-têtes plus lisibles, sexe `F`/`G`, prénoms écrits **sans accents**) : il contient le top 10 de chaque sexe en 2023, avec les chiffres non arrondis des éditions précédentes, et en 2025, avec les chiffres de l'édition 2025, arrondis au multiple de 5.

► **Exercices d'application :** exercice 1 (vocabulaire d'un fichier CSV)

## II Représenter une table : la liste de dictionnaires

Puisqu'un enregistrement associe une valeur à chaque descripteur, l'objet Python qui lui ressemble le plus est le **dictionnaire** (chapitre *Les types construits*). Toute la table devient alors une **liste de dictionnaires**.

**Règle 1 — Table = liste de dictionnaires**

- **un enregistrement** → un dictionnaire `{descripteur : valeur}` ;
- **la table** → une `liste` de ces dictionnaires.

```
1 prenoms = [
2     {"prenom": "Louise", "sexe": "F", "annee": "2023", "nombre": "3177"},
```

```
    {"prenom": "Ambre",  "sexe": "F", "annee": "2023", "nombre": "3168"},
    {"prenom": "Gabriel","sexe": "G", "annee": "2023", "nombre": "4527"},
]
```

*la table (vue « tableur »)*

| prenom | sexe | annee | nombre |
|---|---|---|---|
| Louise | F | 2023 | 3177 |
| Ambre | F | 2023 | 3168 |
| Gabriel | G | 2023 | 4527 |

*la liste* `prenoms` *: une fiche = un dictionnaire*

```
0 {"prenom": "Louise", "sexe": "F", "annee": "2023", "nombre": "3177"}
1 {"prenom": "Ambre", "sexe": "F", "annee": "2023", "nombre": "3168"}
2 {"prenom": "Gabriel", "sexe": "G", "annee": "2023", "nombre": "4527"}
```

*Chaque ligne devient un dictionnaire ; les en-têtes de colonnes deviennent ses clés ; l'indice de la fiche dans la liste est en couleur.*

On accède alors à une valeur en deux temps : d'abord la fiche (par son **indice** dans la liste), puis la rubrique (par son **descripteur**).

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> prenoms[0]["prenom"]
'Louise'
>>> prenoms[2]["nombre"]
'4527'
```

**Remarque** — *Pourquoi des dictionnaires, et pas des listes de listes ?*
On *pourrait* coder chaque fiche par une liste `["Louise","F","2023","3177"]`. Mais il faudrait alors se souvenir que `nombre` est la case `3`… et écrire `ligne[3]`, illisible et fragile (si on ajoute une colonne, tout décale). Avec un dictionnaire, on écrit `ligne["nombre"]` : **on nomme ce qu'on manipule**. C'est exactement le dictionnaire-*fiche* (l'enregistrement) annoncé au chapitre *Les types construits*.

▶ **Exercices d'application :** exercices 2 à 4 (représenter une table : liste de dictionnaires)

## III Importer une table depuis un CSV

Saisir la table à la main serait absurde pour 700 000 lignes. La bibliothèque standard `csv` sait lire un fichier et en faire **directement** une liste de dictionnaires, grâce à `csv.DictReader` qui utilise la ligne d'en-tête comme clés.

### Règle 2 — Le patron d'importation d'un CSV

```
import csv

def charger(fichier):
    with open(fichier, encoding="utf-8", newline="") as f:
        lecteur = csv.DictReader(f)
        table = [ligne for ligne in lecteur]
    return table
```

- `open(...)` ouvre le fichier ; `with` le **referme** tout seul à la fin ;
- `csv.DictReader` lit l'**en-tête**, puis transforme chaque ligne en dictionnaire ; si le séparateur est le point-virgule, on le précise : `csv.DictReader(f, delimiter=";")` ; pour un fichier **tabulé** (colonnes séparées par des tabulations, souvent d'extension `.tsv` ou `.txt`), on écrit `delimiter="\t"` ;
- la compréhension `[ligne for ligne in lecteur]` range toutes les fiches dans une liste.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> prenoms = charger("prenoms.csv")
>>> len(prenoms)
40
>>> prenoms[0]
{'prenom': 'Louise', 'sexe': 'F', 'annee': '2023', 'nombre': '3177'}
```

**Remarque** — *Le piège n° 1 : tout est du texte !*
`csv` lit un fichier **texte** : **toutes** les valeurs sont des **chaînes de caractères** (`str`), **même les nombres**. Ainsi `prenoms[0]["nombre"]` vaut la chaîne `"3177"`, pas l'entier `3177`.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> prenoms[0]["nombre"] + prenoms[1]["nombre"]
'31773168'          # concatenation de chaines, pas une addition !
```

**Le réflexe** : dès qu'on veut *calculer* sur une colonne numérique, on **convertit** avec `int(...)` ou `float(...)`.

↻ **Retour sur l'activité.** Le tableur de l'activité, lui, devinait les types : il affichait `6` au lieu du code `06` de Nice. Python garde le texte tel quel, et c'est à vous de convertir ce qui doit l'être : un code de département n'est pas un nombre.

► **Exercices d'application :** exercices 5 à 8 (séparateur, encodage ; importer ; le piège du texte)

8

## IV Rechercher et filtrer : le parcours revient

Sélectionner les lignes qui vérifient un critère, c'est **parcourir** la table et **garder** celles qui conviennent dans une nouvelle liste. C'est le patron du parcours, avec un accumulateur qui est… une liste.

**Règle 3 — Filtrer selon un critère**

```
def selection(table, sexe, annee):
    resultat = []                                          # INITIALISER :
                                                           # l'accumulateur est une liste
    for ligne in table:                                    # PARCOURIR
        if ligne["sexe"] == sexe and ligne["annee"] == annee:  # TESTER
            resultat.append(ligne)                         # METTRE A JOUR
    return resultat                                        # RENVOYER
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> filles23 = selection(prenoms, "F", "2023")
>>> [ligne["prenom"] for ligne in filles23]
['Louise', 'Ambre', 'Alba', 'Jade', 'Emma', 'Rose',
 'Alma', 'Alice', 'Romy', 'Anna']
```

**Remarque** — *La même chose en une ligne : la compréhension*
Ce filtrage s'écrit aussi avec une **compréhension de liste** (vue au chapitre *Les types construits*), qui dit littéralement « garde `ligne` pour chaque `ligne` qui vérifie le critère » :

```
filles23 = [ligne for ligne in prenoms
            if ligne["sexe"] == "F" and ligne["annee"] == "2023"]
```

Deux critères se combinent avec `and` (les deux conditions) ou `or` (au moins une).

Pour trouver **un** enregistrement précis (le *premier* qui convient), on applique le réflexe « sortir tôt » de la recherche séquentielle : on renvoie dès qu'on a trouvé.

```python
def chercher(table, prenom):
    for ligne in table:           # PARCOURIR
        if ligne["prenom"] == prenom:   # TESTER
            return ligne          # RENVOYER : trouve, on s'arrete tout de suite
    return None                   # RENVOYER : parcours fini sans succes
```

▷ **Je prédis.** Dans l'extrait, `Louise` figure en 2023 puis en 2025. Que renvoie `chercher(prenoms, "Louise")` ?
▷ **Je vérifie.** La fiche de **2023** (`"nombre": "3177"`) : c'est la première rencontrée, et la fonction s'arrête aussitôt ; la fiche de 2025 n'est jamais examinée. Pour un autre prénom absent de l'extrait, elle renverrait `None`.

## A. Nettoyer une table : doublons et cohérence

Une table réelle contient presque toujours des **erreurs de saisie** : une même fiche recopiée deux fois (un **doublon**), ou une valeur impossible (un nombre d'attributions négatif, une année dans le futur). Avant de calculer, on les recherche : ce sont encore des **parcours**.

### Règle 4 — Repérer les doublons, tester la cohérence

- Un **test de cohérence** vérifie qu'une valeur est possible : on *filtre* les fiches qui ne le respectent pas.
- Pour les **doublons**, on choisit les descripteurs qui identifient une fiche (ici prénom, sexe et année) et on mémorise au fur et à mesure les identifiants déjà rencontrés.

```python
def incoherentes(table):
    """Fiches dont le nombre n'est pas strictement positif."""
    return [ligne for ligne in table if int(ligne["nombre"]) <= 0]

def doublons(table):
    """Fiches dont (prenom, sexe, annee) a deja ete rencontre plus haut."""
    deja_vus = []                 # INITIALISER : identifiants deja rencontres
    resultat = []                 # INITIALISER
    for ligne in table:           # PARCOURIR
        cle = (ligne["prenom"], ligne["sexe"], ligne["annee"])
        if cle in deja_vus:       # TESTER
            resultat.append(ligne)    # METTRE A JOUR : c'est un doublon
        else:
            deja_vus.append(cle)  # METTRE A JOUR : premiere rencontre
    return resultat               # RENVOYER
```

▷ **Je prédis.** Une table contient trois fois la même fiche, et rien d'autre. Que renvoie `doublons` ?
▷ **Je vérifie.** Une liste de **deux** fiches : la première copie est mémorisée dans `deja_vus`, seules les deux suivantes sont des doublons.

Que faire des fiches repérées ? Les écarter, ou les corriger si l'on connaît la bonne valeur : c'est le **nettoyage** de la table, que le TP proposé en fin de chapitre pratique sur de vrais relevés météo.

► **Exercices d'application :** exercices 9 à 12, 28 (rechercher et filtrer) et 30 (doublons et cohérence)

## V Calculer sur une colonne : moyenne, min, max, comptage

Une fois la table (ou un sous-ensemble filtré) en main, calculer sur une colonne, c'est **re-parcourir** en tenant à jour une information : un total, un compteur, un champion. On retrouve mot pour mot le chapitre *Algorithmique : le parcours séquentiel* — avec, en plus, la **conversion** `int(...)`.

**Exemple** — *Compter, faire une moyenne*

```
def compter(table, sexe):           # combien de fiches pour ce sexe ?
    n = 0                           # INITIALISER
    for ligne in table:             # PARCOURIR
        if ligne["sexe"] == sexe:   # TESTER
            n += 1                  # METTRE A JOUR
    return n                        # RENVOYER

def nombre_moyen(table):            # attribution moyenne sur la table
    total = 0                       # INITIALISER
    for ligne in table:             # PARCOURIR
        total = total + int(ligne["nombre"])   # METTRE A JOUR : convertir !
    return total / len(table)       # RENVOYER
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> compter(prenoms, "G")
20
>>> nombre_moyen(filles23)
2628.7
```

**Règle 5 — L'invariant du champion sur une table**

Pour le **maximum** (ou le minimum) d'une colonne, on garde la **fiche championne**, initialisée à la première, et on la remplace dès qu'on rencontre mieux.

```
def plus_donne(table):
    champion = table[0]                                  # INITIALISER : la premiere
                                                         # fiche
    for ligne in table:                                  # PARCOURIR
        if int(ligne["nombre"]) > int(champion["nombre"]):  # TESTER
            champion = ligne                             # METTRE A JOUR
    return champion["prenom"], int(champion["nombre"])   # RENVOYER
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> plus_donne(filles23)
('Louise', 3177)
>>> plus_donne(prenoms)          # toutes lignes, 2023 et 2025 confondues
('Gabriel', 4625)
```

**Remarque** — *Le réflexe coût*

Chacune de ces fonctions parcourt la table **une seule fois** : le coût est **linéaire** en le nombre de lignes. Attention au piège de la variance vu au chapitre *Algorithmique : le parcours séquentiel* : ne **jamais** rappeler `nombre_moyen(table)` *à l'intérieur* d'une boucle sur la table (le coût deviendrait *quadratique*).

► **Exercices d'application :** exercices 13 à 16 (calculer sur une colonne)

## VI Trier une table selon un descripteur

Trier une table, c'est ranger ses fiches selon les valeurs d'**un** descripteur. Inutile de réécrire un tri par sélection ou par insertion (chapitre *Les algorithmes de tri*) : la fonction `sorted` le fait, à condition de lui dire **sur quoi** comparer, via le paramètre `key`.

### Règle 6 — Trier avec `sorted` et une clé

```
1 def trier_par_nombre(table):
2     return sorted(table, key=lambda ligne: int(ligne["nombre"]),
3                   reverse=True)
```

- `key` indique la valeur à comparer pour chaque fiche. La petite expression `lambda ligne: ...` est une fonction anonyme qui, à une fiche, associe sa clé de tri ;
- `reverse=True` trie du plus grand au plus petit ;
- `sorted` renvoie une **nouvelle** liste et **ne modifie pas** l'original.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> top = trier_par_nombre(selection(prenoms, "F", "2025"))
>>> [(l["prenom"], l["nombre"]) for l in top][:3]
[('Louise', '3070'), ('Jade', '2925'), ('Ambre', '2805')]
```

**Remarque** — *Clé numérique ou clé texte*

La clé change complètement le tri. `int(ligne["nombre"])` trie par **valeur numérique** ; `ligne["prenom"]` trie par **ordre alphabétique** :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> [l["prenom"] for l in sorted(filles23, key=lambda l: l["prenom"])][:3]
['Alba', 'Alice', 'Alma']
```

Si on oublie `int`, `"3070"` et `"900"` seraient comparés *comme des mots*, caractère par caractère : comme `'3' < '9'`, la chaîne `"900"` est jugée plus grande que `"3070"`. Dans l'ordre croissant, `"3070"` passerait donc *avant* `"900"` ; avec `reverse=True`, `"900"` passerait avant `"3070"`. Encore le piège n° 1 !

► **Exercices d'application :** exercices 17 à 19 et 29 (trier une table)

## VII Fusionner deux tables : la jointure

L'information est souvent **répartie** sur plusieurs tables reliées par un descripteur commun. Les rassembler s'appelle une **fusion** (ou *jointure*).

### Définition 3 — Fusion sur un descripteur commun

Fusionner deux tables partageant un descripteur, c'est produire une table où chaque enregistrement de la première est **complété** par les informations de la seconde **ayant la même valeur** pour ce descripteur.

↻ **Retour sur l'activité.** Pour trouver la région de Menton dans l'activité, vous avez fait une jointure à la main : passer de `communes.csv` à `departements.csv` grâce à la colonne `dep`. Ajaccio, de code `2A` absent du second fichier, restait sans réponse.

Prenons une table de communes et une table de départements, reliées par le code `dep` :

communes

| nom | dep |
|---|---|
| Nice | 06 |
| Menton | 06 |
| Lyon | 69 |

même dep

departements

| dep | departement | region |
|---|---|---|
| 06 | Alpes-Maritimes | Provence-Alpes-Cote d'Azur |
| 69 | Rhone | Auvergne-Rhone-Alpes |

## Règle 7 — Le patron de la jointure

Pour chaque fiche de gauche, on cherche dans la table de droite **celle qui a la même clé**, et on **combine** les deux dictionnaires.

```
def jointure(gauche, droite, cle):
    resultat = []                                  # INITIALISER
    for lg in gauche:                              # PARCOURIR : la table de gauche
        for ld in droite:                          # PARCOURIR : la table de droite
            if lg[cle] == ld[cle]:                 # TESTER : meme valeur de descripteur
                fusion = {c: lg[c] for c in lg}    # copie de la fiche de gauche
                for c in ld:
                    fusion[c] = ld[c]              # on ajoute les colonnes de droite
                resultat.append(fusion)            # METTRE A JOUR
    return resultat                                # RENVOYER
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> villes = jointure(communes, departements, "dep")
>>> villes[0]["nom"], villes[0]["region"]
('Nice', "Provence-Alpes-Cote d'Azur")
```

La table obtenue réunit les colonnes des deux tables, une ligne par commune :

| nom | dep | departement | region |
|---|---|---|---|
| Nice | 06 | Alpes-Maritimes | Provence-Alpes-Cote d'Azur |
| Menton | 06 | Alpes-Maritimes | Provence-Alpes-Cote d'Azur |
| Lyon | 69 | Rhone | Auvergne-Rhone-Alpes |

de communes — ajouté depuis departements

**Remarque** — *Une fiche sans correspondance disparaît*
Si une commune a un code `dep` **absent** de la table des départements (par exemple `2A`, la Corse-du-Sud, non listée ici), **aucune** ligne ne sera produite pour elle : elle *tombe* de la fusion. C'est logique — on n'a pas d'information à lui rattacher — mais il faut y penser. **Réflexe coût** : cette jointure examine chaque paire (gauche, droite) : son coût est *quadratique* [au-delà du programme].

**Remarque** — *Le domaine de valeurs*
Le **domaine de valeurs** d'un descripteur est l'ensemble des valeurs qu'il peut prendre (par exemple, pour `dep` : les codes de département, écrits en texte `"06"`, `"2A"`...). Pour fusionner, les deux descripteurs doivent prendre leurs valeurs dans le **même domaine** : un code `"06"` (texte) ne sera jamais égal à l'entier `6`, et un code de département ne se compare pas à un *nom* de département. Avant une jointure, on vérifie donc que les deux colonnes s'écrivent de la même façon.

► **Exercices d'application :** exercices 20 à 22 (fusionner : la jointure)

## VIII Pour aller plus vite : la bibliothèque `pandas` [ au-delà du programme ]

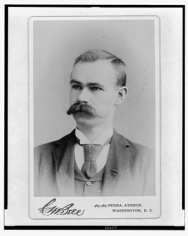
Tout ce chapitre s'écrit **sans aucune bibliothèque**, pour *comprendre* ce qu'on fait. Dans la vraie vie, les scientifiques des données utilisent `pandas`, qui fait tenir chaque opération en une ligne. **Ce n'est pas au programme**, mais il est bon de savoir que ça existe.

```
import pandas
t = pandas.read_csv("prenoms.csv")            # importer (types devines !)
t.loc[(t["sexe"] == "F") & (t["annee"] == 2023), ["prenom", "nombre"]]
t["nombre"].mean()                            # moyenne d'une colonne
t.sort_values(by="nombre", ascending=False)   # trier
pandas.merge(communes, departements)          # fusionner
```

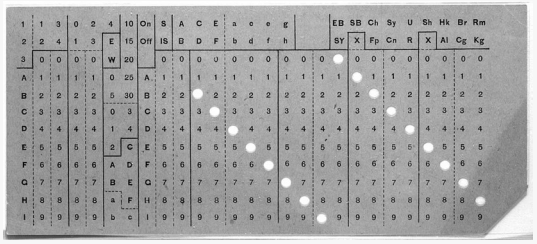
**Pour les curieux** — *Le prix de la magie* (facultatif)
`pandas` devine tout seul que `nombre` est un entier (plus besoin de `int`). C'est confortable, mais c'est une **boîte noire** : au bac comme dans ce cours, on veut savoir écrire soi-même le parcours, le filtre, le tri et la jointure. `pandas` viendra *après*.

## IX Un peu d'histoire

**Pour les curieux** — *Des cartes perforées à l'*open data (facultatif)
En **1890**, recenser les 63 millions d'Américains à la main menaçait de prendre plus de dix ans. **Herman Hollerith** (1860–1929) invente une machine qui lit des **cartes perforées** — une carte par personne, un trou par caractéristique : c'est déjà « une fiche par individu, les mêmes rubriques pour tous », l'idée même d'une table. Grâce à elle, un premier total de la population est annoncé quelques semaines seulement après le recensement, et le dépouillement complet ne prend que quelques années au lieu de la décennie redoutée. Hollerith fonde en 1896 une société qui, par fusions, deviendra en **1924** une certaine… **IBM**.



*Herman Hollerith*



*Une carte perforée Hollerith : chaque position de trou code une réponse*

En **1970**, **Edgar Codd** pose la théorie des **bases de données relationnelles** (des tables reliées par des descripteurs communs — notre jointure !), qu'on étudiera en Terminale. Le format **CSV**, lui, traîne dans les ordinateurs depuis les années 1970 pour sa simplicité désarmante. Et depuis **2011**, l'État français ouvre ses données sur `data.gouv.fr` : le *parcours de tables* n'est plus un exercice, c'est un pouvoir citoyen. (Petit vertige : une fois compressé, le fichier des prénoms tout entier, plus de 700 000 lignes, ne pèse que quelques mégaoctets, à peu près autant qu'*une seule* photo de votre téléphone.)

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** La liste de dictionnaires (*Les types construits*) ; filtrer, compter, garder la fiche championne (*Algorithmique : le parcours séquentiel*) ; le tri (*Les algorithmes de tri*) ; le charabia « Ã© », expliqué dans *Le binaire et l'écriture des nombres*.
- **Ce qu'il prépare.** Le jeu de données des *k plus proches voisins* sera exactement une telle table : une fiche par individu, des descripteurs, plus une étiquette.
- **En Terminale.** Avec *Bases de données et langage SQL*, on n'écrira plus le parcours : on *décrira* les lignes voulues (`SELECT`… `WHERE`…), jointures comprises.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Table | liste de dictionnaires ; 1 fiche = 1 dictionnaire `{descripteur:valeur}` |
| Descripteur | nom d'une colonne (une *clé* des dictionnaires) |
| CSV | fichier *texte* ; 1re ligne = en-tête ; attention au **séparateur** (`,` ou `;`) |
| Importer | `csv.DictReader` → liste de dictionnaires ; `encoding="utf-8"` |
| Piège n° 1 | tout est `str` : `int(ligne["nombre"])` avant de **calculer** ou **trier** |
| Rechercher | **parcourir** et `append` ceux qui vérifient le critère (ou compréhension) |
| Nettoyer | repérer les **doublons** (identifiants déjà vus) et les valeurs **incohérentes** (filtre) |
| Calculer | moyenne / min / max / comptage = le **patron du parcours**, coût *linéaire* |
| Trier | `sorted(table, key=lambda l: l["..."] , reverse=...)` ; ne modifie pas l'original |
| Fusionner | jointure sur un descripteur commun, de même **domaine de valeurs** ; les fiches sans correspondance *disparaissent* |

### Retour à la situation de départ

1. On importe le fichier en **liste de dictionnaires** (§ III), puis on le **parcourt** en gardant les fiches dont le sexe vaut `"F"` et l'année `"2023"` : c'est un **filtre**, de coût linéaire (§ IV).
2. Tout ce qu'on lit dans un CSV est du **texte** : `"3177" + "3168"` **concatène** deux chaînes. Il faut convertir avec `int(...)` avant de calculer (§ III).
3. Par une **jointure** sur le descripteur commun `dep` : chaque commune est complétée par la fiche du département de même code ; une commune sans correspondance disparaît (§ VII).

## Erreurs fréquentes

- **Oublier que tout est du texte.** `ligne["nombre"]` est une *chaîne*. `"3177"+"3168"` concatène. *Le réflexe :* `int(...)` avant tout calcul ou tri numérique.
- **Se tromper de séparateur.** Un fichier « à la française » utilise `;`. Symptôme : une seule colonne géante. *Solution :* `csv.DictReader(f, delimiter=";")`.
- **Mauvais encodage.** Des `Ã©` à la place des `é` : préciser `encoding="utf-8"`.
- **Confondre indice de ligne et descripteur.** `table[0]` est *une fiche* ; `table[0]["prenom"]` est *une valeur*.
- **Trier « comme des mots ».** Sans `int`, `"900"` est jugé plus grand que `"3070"` : dans un classement décroissant, il passe devant.
- **Oublier les fiches sans correspondance** dans une jointure.

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

- ☐ reconnaître descripteurs et enregistrements, lire un fichier CSV brut ;
- ☐ **importer** un CSV en liste de dictionnaires avec `csv.DictReader` ;
- ☐ **filtrer** selon un ou plusieurs critères (boucle ou compréhension) ;
- ☐ repérer les **doublons** et les valeurs **incohérentes** d'une table ;
- ☐ **calculer** moyenne / min / max / comptage sur une colonne, sans oublier `int(...)` ;
- ☐ **trier** une table selon un descripteur avec `sorted` et `key` ;
- ☐ **fusionner** deux tables sur un descripteur commun.

# EXERCICES

## Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et tester* en Python. Le symbole `>>>` figure une saisie dans la console.
- On travaille sur le fichier `prenoms.csv` (extrait du *Fichier des prénoms* de l'INSEE, `data.gouv.fr`) : le **top 10** des filles et celui des garçons en 2023 et en 2025, soit 40 fiches ; descripteurs `prenom`, `sexe` (`F`/`G`), `annee` (2023 ou 2025), `nombre`. Et sur `communes.csv` / `departements.csv`.
- **Réflexe n° 1** : les valeurs lues dans un CSV sont des **chaînes**. `int(...)` avant tout *calcul* ou *tri* numérique.

On suppose disponible dans tous les exercices la fonction d'importation du cours :

```
import csv
def charger(fichier):
    with open(fichier, encoding="utf-8", newline="") as f:
        return [ligne for ligne in csv.DictReader(f)]

prenoms = charger("prenoms.csv")
```



### Lire une table, lire un CSV ↑cours

★ **Exercice 1** IA IA IA *Vocabulaire* ▶ corrigé p. 640 • essentiel

On donne les quatre premières lignes d'un fichier CSV public (jeux de la médiathèque) :

```
titre;editeur;duree;age_min
Les Aventuriers du Rail;Days of Wonder;45;8
Dixit;Libellud;30;8
7 Wonders;Repos Prod;30;10
```

1. Quel est le **séparateur** ? Combien y a-t-il de **descripteurs** ? d'**enregistrements** dans cet extrait ?
2. Citer les descripteurs. Donner la valeur du descripteur `duree` pour l'enregistrement `Dixit`.
3. Ce fichier est-il « à la française » ou « anglo-saxon » ? Justifier.

▷ coup de pouce

### Représenter une table en Python ↑cours

★ **Exercice 2** IA IA IA *Une table à la main* ▶ corrigé p. 640 • essentiel

Écrire la variable `jeux` représentant les **trois** lignes de l'exercice 1 sous forme d'une **liste de dictionnaires**.

▷ coup de pouce

★ **Exercice 3** IA IA IA *Accès* ▶ corrigé p. 640▶

Avec la variable `jeux` précédente, que renvoient ces expressions ?

```
>>> jeux[1]["titre"]
>>> jeux[0]["age_min"]
>>> len(jeux)
```

Écrire une expression qui donne l'éditeur de `7 Wonders`.

▷ coup de pouce

★★ **Exercice 4** *Descripteurs automatiques* ▶ corrigé p. 640▶

Écrire `descripteurs(table)` qui renvoie la liste des noms de colonnes d'une table (on suppose la table non vide). ▷ coup de pouce

```
>>> descripteurs(prenoms)
['prenom', 'sexe', 'annee', 'nombre']
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Importer, et le piège du texte ↑ cours

★ **Exercice 5** *Un fichier mal formé* ▶ corrigé p. 641

Un camarade ouvre avec `csv.DictReader` le fichier des jeux de la médiathèque (exercice « Vocabulaire ») et obtient **une seule colonne** géante. Quelle est l'erreur la plus probable ? Comment la corriger dans l'appel à `csv.DictReader` ? ▷ coup de pouce

★★ **Exercice 6** *Encodage* ▶ corrigé p. 641

En ouvrant un fichier, l'élève voit `RaphaÃ«l` au lieu de `Raphaël`. Quel réglage a été oublié ? Écrire la ligne `open(...)` corrigée. ▷ coup de pouce

★ **Exercice 7** *Lecture de trace* ▶ corrigé p. 641

Sans machine, dire ce que renvoie `mystere(prenoms)`.

```
1 def mystere(table):
2     r = 0
3     for ligne in table:
4         if ligne["sexe"] == "F":
5             r = r + 1
6     return r
```

Que calcule cette fonction, en une phrase ? ▷ coup de pouce

★ **Exercice 8** *Le piège du texte* ▶ corrigé p. 641▶ • essentiel

Prédire, *puis* vérifier, la valeur de chaque expression :

```
>>> prenoms[0]["nombre"]
>>> prenoms[0]["nombre"] + prenoms[1]["nombre"]
>>> int(prenoms[0]["nombre"]) + int(prenoms[1]["nombre"])
```

Expliquer la différence entre les deux dernières lignes. ▷ coup de pouce

## Rechercher et filtrer ↑ cours

★ **Exercice 9** *Filtrer* ▶ corrigé p. 641▶ • essentiel

Écrire `selection(table, sexe, annee)` qui renvoie la liste des fiches correspondant à ce `sexe` et cette `annee`. Vérifier :

```
>>> filles23 = selection(prenoms, "F", "2023")
>>> [l["prenom"] for l in filles23]
['Louise', 'Ambre', 'Alba', 'Jade', 'Emma', 'Rose',
 'Alma', 'Alice', 'Romy', 'Anna']
```

▷ coup de pouce

★★ **Exercice 10** *En compréhension* ▶ corrigé p. 641▶

Réécrire `selection` en **une seule ligne** avec une compréhension de liste. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 11** *Rechercher une fiche* ▶ corrigé p. 642▶ • essentiel

Écrire `chercher(table, prenom, annee)` qui renvoie **la** fiche de ce prénom pour cette année, ou `None` si elle n'existe pas. On s'arrêtera dès qu'on l'a trouvée. ▷ coup de pouce

```
>>> chercher(prenoms, "Jade", "2025")["nombre"]
'2925'
>>> print(chercher(prenoms, "Kevin", "2025"))
None
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 12** *Deux critères* ▶ corrigé p. 642▶

Écrire un appel (ou une compréhension) qui donne la liste des prénoms de **garçons de 2025** attribués **plus de 3000** fois. *(Attention au réflexe n° 1 du mode d'emploi.)* ▷ coup de pouce

## Calculer sur une colonne ↑ cours

★ **Exercice 13** *Compter* ▶ corrigé p. 642▶

Écrire `compter(table, sexe)` qui renvoie le nombre de *fiches* pour ce sexe. Combien pour `"G"` ? ▷ coup de pouce

★★ **Exercice 14** *Total et moyenne* ▶ corrigé p. 642▶ • essentiel

1. Écrire `total(table, sexe, annee)` qui renvoie la **somme** des `nombre` des fiches correspondantes.

▷ coup de pouce

2. En déduire combien de garçons nés en 2023 ont reçu l'un des 10 prénoms de garçons du fichier.
3. Écrire `nombre_moyen(table)` (moyenne des `nombre`). Que vaut-elle sur `filles23` ?

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 15** *Champion et lanterne rouge* ▶ corrigé p. 643▶

1. Écrire `plus_donne(table)` qui renvoie le couple (prénom, nombre), le nombre étant un **entier**, de la fiche au plus grand `nombre` (invariant du champion). ▷ coup de pouce
2. Écrire `moins_donne(table)` sur le même modèle.
3. Tester sur `prenoms` entier. *Pourquoi faut-il convertir avec* `int` *pour que ce soit correct ?*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 16** *Coût* ▶ corrigé p. 643 ≈ 10 min

On veut, pour *chaque* fiche, afficher son `nombre` **et** la moyenne de la table. Un élève écrit :

```
1 for ligne in table:
2     print(ligne["nombre"], nombre_moyen(table))   # <-- ?
```

Pourquoi le coût devient-il **quadratique** ? Comment le rendre **linéaire** ? (Rappel du chapitre *Algorithmique : le parcours séquentiel.*) ▷ coup de pouce ▷ début de solution

## Trier une table ↑cours

★★ **Exercice 17** *Trier par nombre* ▶ corrigé p. 644▶ • essentiel

Écrire `trier_par_nombre(table)` qui renvoie la table triée du plus donné au moins donné. Afficher les **3 premiers** prénoms de garçons de 2025. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 18** *Alphabétique* ▶ corrigé p. 644▶

Trier `filles23` par **ordre alphabétique** des prénoms. Quelle est la seule ligne à changer par rapport à l'exercice précédent ? ▷ coup de pouce

★★ **Exercice 19** *Le tri qui ment* ▶ corrigé p. 644▶

Trier `filles23` avec `key=lambda l: l["nombre"]` (sans `int`) et `reverse=True`. Le résultat est-il le bon classement ? Est-ce **garanti** ? **Expliquer** précisément pourquoi, si un prénom n'avait été donné que 900 fois, la chaîne `"900"` passerait avant `"3070"` dans ce tri avec `reverse=True`. ▷ coup de pouce

## Fusionner deux tables ↑cours

★ **Exercice 20** *Remettre la jointure dans l'ordre* ▶ corrigé p. 644

Voici les lignes de la fonction `jointure` du cours, **mélangées** et **sans indentation**. Les remettre dans le bon ordre et les indenter correctement. Pour chaque ligne, dire en quelques mots son rôle.

```
resultat.append(fusion)
for ld in droite:
def jointure(gauche, droite, cle):
fusion[c] = ld[c]
return resultat
if lg[cle] == ld[cle]:
resultat = []
fusion = {c: lg[c] for c in lg}
for lg in gauche:
for c in ld:
```

▷ coup de pouce

★★ **Exercice 21** *Jointure* ▶ corrigé p. 645▶ • essentiel

On charge `communes = charger("communes.csv")` et `departements = charger("departements.csv")`, reliées par le descripteur `dep`.

1. Écrire `jointure(gauche, droite, cle)` (patron du cours).
2. Vérifier que la fiche de `Nice` obtient bien sa `region`.
3. Le fichier `communes.csv` contient `Ajaccio` (code `2A`), *absent* de `departements.csv`. Que devient Ajaccio après la jointure ? Pourquoi ? ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 22** *Compter par groupe* ▶ corrigé p. 645▶

À l'aide de la jointure précédente, écrire un code qui affiche, pour la **région** `"Provence-Alpes-Cote d'Azur"` (écrite sans accent dans `departements.csv`), le nombre de communes du fichier qui s'y trouvent. ▷ coup de pouce

## Mini-projets de synthèse (données réelles) ↑cours

★★★ **Exercice 23** *Le palmarès à la loupe* ▶ corrigé p. 645▶ ≈20 min

Écrire un programme qui, à partir de `prenoms.csv`, affiche pour l'année **2025** :

1. le prénom de fille et le prénom de garçon les plus donnés ;
2. la moyenne des attributions chez les filles, puis chez les garçons ;
3. le **top 3** des garçons, dans l'ordre.

▷ coup de pouce ▷ début de solution

★★★ **Exercice 24** *Qui monte, qui descend ?* ▶ corrigé p. 646▶ ≈20 min

On veut comparer 2023 et 2025 chez les **garçons**.

1. Écrire `rang(table, sexe, annee)` qui renvoie un *dictionnaire* `{prenom : rang}` (rang 1 = le plus donné), obtenu en triant puis en numérotant.
2. En comparant les rangs 2023 et 2025, déterminer un prénom qui a **gagné** des places et un qui en a **perdu**. *(Sur les vraies données, Noah passe $6^e \to 2^e$ et* `Mael`*, écrit sans accent dans le fichier, $5^e \to 10^e$.)*

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 25** *Entrants et sortants* ▶ corrigé p. 646▶ ≈15 min

On note `noms2023` et `noms2025` les listes des prénoms du top 10 des filles en 2023 et en 2025. À l'aide de **tests d'appartenance** (`in` / `not in`, par exemple dans une compréhension), déterminer : les prénoms présents **en 2023 et en 2025**, ceux qui en sont **sortis** (en 2023 mais plus en 2025), ceux qui y sont **entrés** (en 2025 mais pas en 2023). *(Résultat attendu :* `'Anna'` *sort,* `'Adele'` *entre — les prénoms sont écrits sans accents dans le fichier.)* ▷ coup de pouce ▷ début de solution

★★★ **Exercice 26** *Écrire un CSV [au-delà du programme]* ▶ corrigé p. 646▶ ≈20 min

*Bonus, au-delà du programme.* À l'aide de `csv.DictWriter`, écrire une fonction `exporter(table, fichier)` qui **sauvegarde** une table (par exemple le résultat d'un filtre) dans un nouveau fichier CSV, puis vérifier en le rechargeant avec `charger`. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 27** *Une moyenne selon un assistant d'IA* ▶ corrigé p. 647▶

Un élève demande à un assistant d'IA : « J'ai chargé `prenoms.csv` avec `csv.DictReader` dans une liste de dictionnaires `prenoms`. Écris une fonction qui calcule le nombre moyen d'attributions (colonne `nombre`). » Voici la réponse obtenue.

```
def moyenne_nombre(table):
    total = 0
    for ligne in table:
        total = total + ligne["nombre"]
    return total / len(table)
```

« On parcourt la table en accumulant la colonne `nombre` dans `total`, puis on divise par le nombre de lignes. Comme `DictReader` a déjà lu les valeurs numériques, il n'y a rien à convertir : `moyenne_nombre(prenoms)` renvoie directement la moyenne. »

1. La réponse est-elle correcte ? Exécuter `moyenne_nombre(prenoms)` et noter ce qui se passe. ▷ coup de pouce
2. Localiser et corriger l'erreur.
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

## Entraînement ↑cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

Dans les deux exercices, on utilise la petite table suivante, lue dans un CSV (les valeurs sont donc des chaînes) :

```
eleves = [{"nom": "Ada", "classe": "1A", "note": "15"},
          {"nom": "Léo", "classe": "1B", "note": "9"},
          {"nom": "Zoé", "classe": "1A", "note": "12"}]
```

★ **Exercice 28** *Lire et filtrer une table* ▶ corrigé p. 647 • essentiel

Sans machine, donner la valeur de chaque expression, puis ▶ vérifier.

a) `eleves[2]["nom"]`
b) `eleves[0]["note"] + eleves[1]["note"]`
c) `int(eleves[0]["note"]) + int(eleves[1]["note"])`
d) `[e["nom"] for e in eleves if e["classe"] == "1A"]`
e) `[e["nom"] for e in eleves if int(e["note"]) < 10]`

▷ coup de pouce

★ **Exercice 29** *Trier une table : prédire l'ordre* ▶ corrigé p. 647 • essentiel

Sans machine, donner la liste des noms dans l'ordre obtenu par chaque tri, puis ▶ vérifier.

a) `sorted(eleves, key=lambda e: e["nom"])`
b) `sorted(eleves, key=lambda e: int(e["note"]))`
c) `sorted(eleves, key=lambda e: int(e["note"]), reverse=True)`
d) `sorted(eleves, key=lambda e: e["note"])`

Pourquoi les résultats de b) et d) diffèrent-ils ?

▷ coup de pouce

## Nettoyer une table ↑cours

★★ **Exercice 30** *Doublons et notes impossibles* ▶ corrigé p. 647▶

Un professeur a saisi les notes d'un devoir dans un CSV, avec quelques erreurs :

```
notes = [{"nom": "Ada", "classe": "1A", "note": "15"},
         {"nom": "Léo", "classe": "1B", "note": "9"},
         {"nom": "Ada", "classe": "1A", "note": "15"},
         {"nom": "Zoé", "classe": "1A", "note": "23"},
         {"nom": "Tom", "classe": "1B", "note": "-2"},
         {"nom": "Ada", "classe": "1B", "note": "11"}]
```

On considère qu'un élève est identifié par son `nom` **et** sa `classe`.

1. Écrire `notes_impossibles(table)` qui renvoie la liste des **noms** des fiches dont la note n'est pas comprise entre 0 et 20. Que renvoie-t-elle sur `notes` ?
2. Écrire `doublons(table)` qui renvoie la liste des fiches dont le couple (`nom`, `classe`) a déjà été rencontré plus haut dans la table. Combien de doublons dans `notes` ? Pourquoi la dernière fiche n'en est-elle pas un ?
3. Écrire `nettoyer(table)` qui renvoie une **nouvelle** table sans doublon et sans note impossible. Calculer la moyenne des notes avant et après nettoyage.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste La première ligne contient les descripteurs (les noms de colonnes) ; chaque ligne suivante est un enregistrement. Quel caractère sépare les valeurs d'une même ligne ? ↑énoncé

Ex. 2 – piste Une fiche par enregistrement : un dictionnaire dont les clés sont les descripteurs. Comme `csv.DictReader`, écrire les valeurs sous forme de chaînes. ↑énoncé

Ex. 3 – piste On choisit d'abord la fiche par son indice dans la liste (à partir de `0`), puis la valeur par le nom du descripteur. ↑énoncé

Ex. 4 – piste Les noms de colonnes sont les clés de n'importe quelle fiche, par exemple de la première, `table[0]`. Une boucle `for` sur un dictionnaire parcourt ses clés : il reste à les ranger dans une liste, par exemple avec une compréhension. ↑énoncé

Ex. 5 – piste Par défaut, `csv.DictReader` sépare les valeurs par une virgule. Quel séparateur utilisent les fichiers « à la française » ? Le paramètre s'appelle `delimiter`. ↑énoncé

Ex. 6 – piste Quel paramètre de `open` indique comment lire les caractères accentués ? Quel est l'encodage standard aujourd'hui ? ↑énoncé

Ex. 7 – piste C'est le patron du compteur : `r` augmente de 1 pour chaque fiche qui vérifie la condition. Combien de fiches de filles contient `prenoms` (deux années, un top 10 par année) ? ↑énoncé

Ex. 8 – piste Quel est le type de `prenoms[0]["nombre"]` ? Que fait `+` entre deux chaînes ? ↑énoncé

Ex. 9 – piste Partir d'une liste vide et y ajouter (`append`) chaque fiche qui vérifie les deux conditions. Attention : l'année est lue comme une chaîne, d'où `"2023"` entre guillemets. ↑énoncé

Ex. 10 – piste Une compréhension avec filtre a la forme `[expression for element in liste if condition]` : quelle condition reprendre de `selection` ? ↑énoncé

Ex. 11 – piste C'est une recherche séquentielle : `return` dès que les deux critères sont vérifiés. À quel endroit placer le `return None` ? ↑énoncé

Ex. 12 – piste Trois conditions reliées par `and` (sexe, année, nombre). Sur quel type doit porter la comparaison avec `3000` ? ↑énoncé

Ex. 13 – piste Même patron que `mystere` dans l'exercice « Lecture de trace » ; seule la valeur comparée change. ↑énoncé

Ex. 14 q. 1 – piste Patron de l'accumulateur : un total initialisé à `0`, augmenté pour chaque fiche retenue. On peut réutiliser `selection`. ↑énoncé

Ex. 14 q. 3 – piste La moyenne, c'est la somme des valeurs divisée par le nombre de fiches, `len(table)`. ↑énoncé

Ex. 15 q. 1 – piste Prendre la première fiche comme championne provisoire, puis la remplacer dès qu'une fiche a un `nombre` plus grand (comparer des entiers). Renvoyer à la fin un p-uplet construit à partir de la championne. ↑énoncé

Ex. 16 – piste Combien de fiches `nombre_moyen` parcourt-elle à chaque appel ? Combien de fois est-elle appelée ? La moyenne change-t-elle d'un tour à l'autre ? ↑énoncé

Ex. 16 – début de solution Calculer la moyenne **une seule fois**, avant la boucle : `m = nombre_moyen(table)`, puis utiliser `m` dans la boucle. ↑énoncé

Ex. 17 – piste `sorted` avec `key` (sur quoi comparer ?) et `reverse=True` ; pour la clé, penser au réflexe n° 1. Pour les 3 premiers : filtrer avec `selection`, trier, puis une tranche `[:3]`. ↑

Ex. 18 – piste Seule la clé du tri change : sur quel descripteur trier ? Faut-il convertir ? ↑énoncé

Ex. 19 – piste Deux chaînes se comparent caractère par caractère, de gauche à droite, comme les mots d'un dictionnaire : comparer d'abord le premier caractère de chacune. ↑énoncé

Ex. 20 – piste Deux boucles imbriquées (gauche, puis droite) et un test d'égalité sur la clé. Quand les clés sont égales : copier la fiche de gauche, y ajouter les colonnes de droite, ranger le résultat. ↑énoncé

Ex. 21 q. 3 – piste Pour la fiche d'Ajaccio, la condition `lg[cle] == ld[cle]` est-elle vraie pour au moins une fiche de droite ? Que se passe-t-il alors dans la boucle ? ↑énoncé

Ex. 22 – piste Parcourir le résultat de la jointure avec un compteur, comme dans l'exercice « Compter ». ↑énoncé

Ex. 23 – piste Inutile de tout réécrire : chaque question combine des fonctions déjà écrites (`selection`, `plus_donne`, `nombre_moyen`, `trier_par_nombre`). ↑énoncé

Ex. 23 – début de solution
```
filles25 = selection(prenoms, "F", "2025")
garcons25 = selection(prenoms, "G", "2025")
print(plus_donne(filles25), plus_donne(garcons25))
```
↑énoncé

Ex. 24 – piste Après le tri décroissant, la fiche d'indice `i` a le rang `i + 1`. Pour comparer, parcourir les prénoms de 2023 et regarder s'ils ont aussi un rang en 2025 (test `in` sur les clés). ↑

Ex. 24 – début de solution
```
lignes = trier_par_nombre(selection(table, sexe, annee))
resultat = {}
for i in range(len(lignes)):
    ...
```
↑énoncé

Ex. 25 – piste Construire d'abord les deux listes de prénoms (par compréhension avec filtre), puis, pour chaque question, une compréhension qui garde les prénoms d'une liste selon qu'ils sont `in` ou `not in` l'autre. ↑énoncé

Ex. 25 – début de solution
```
noms2023 = [l["prenom"] for l in selection(prenoms, "F",
"2023")]
sortis = [p for p in noms2023 if p not in noms2025]
```
↑énoncé

Ex. 26 – piste Ouvrir le fichier en écriture (`"w"`) ; un `csv.DictWriter` a besoin de la liste des noms de colonnes (paramètre `fieldnames`) : la fonction `descripteurs` la donne. ↑énoncé

Ex. 26 – début de solution
```
with open(fichier, "w", encoding="utf-8", newline="") as f:
    ecrivain = csv.DictWriter(f, fieldnames=descripteurs(
table))
    ecrivain.writeheader()
```
↑énoncé

Ex. 27 q. 1 – piste Quel est le type de `ligne["nombre"]` ? Relire le réflexe n° 1 du mode d'emploi. ↑

Ex. 28 – piste D'abord la fiche (indice dans la liste), puis la valeur (nom du descripteur). Entre deux chaînes, `+` colle les textes. ↑énoncé

Ex. 29 – piste La clé dit sur quoi comparer les fiches. Sans `int`, on compare des chaînes caractère par caractère, comme des mots du dictionnaire : comparer d'abord les premiers caractères de `"15"`, `"9"` et `"12"`. ↑énoncé

Ex. 30 – piste Pour les doublons, garder une liste `deja_vus` des couples `(nom, classe)` rencontrés, comme dans le cours ; une fiche est un doublon si son couple est déjà dans cette liste. ↑

# TP ET PROJETS

## TP La météo de la Riviera ▶ corrigé p. 648

*importer, nettoyer, interroger, croiser et exporter une vraie table*

Durée : 2 à 3 h (deux séances) · Seul ou en binôme, sur machine

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p08-tp-meteo-riviera` — **archive .zip**

### But du TP

Un réseau de six stations météo, de Monaco à Gréolières, a transmis ses relevés quotidiens du mois d'août. Comme toutes les données réelles, le fichier reçu est **imparfait** : valeurs manquantes, lignes en double, capteur défectueux, station inconnue… Vous allez écrire en Python, **sans bibliothèque spécialisée**, la chaîne complète d'un traitement de données : **importer** le CSV, le **contrôler** et le **nettoyer**, l'**interroger** (filtres, moyennes, records), le **trier**, le **croiser** avec la table des stations, et **produire** un fichier CSV de bilan lisible dans un tableur.

### Consignes

- Un compte rendu court est demandé à la fin (partie 7).
- Fichiers à télécharger (lien ci-dessus), à placer **dans le même dossier** : `releves_aout.csv`, `stations.csv` et `tp_donnees_tables_depart.py` (le programme à compléter : tout votre code s'y écrit).
- Les données sont **fictives mais réalistes** (août 2025). Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- ▶ *À programmer et tester.* En bas du programme, des tests vérifient votre travail : à chaque lancement, chaque partie affiche `[OK]` ou `[A FAIRE]` (au début, tout est `[A FAIRE]` : c'est normal).
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Prise en main : importer la table

★ **Exercice 1** *Regarder le fichier avant de programmer*

Ouvrir `releves_aout.csv` dans un **éditeur de texte** (pas dans le tableur). En voici le début :

```
station;date;tmin;tmax;pluie
MON;2025-08-01;23,1;29,0;0,0
MON;2025-08-02;22,2;27,2;0,0
```

1. Quels sont les **descripteurs** ? Quel est le **séparateur** ? Pourquoi n'est-ce pas la virgule ?

2. Que représentent `tmin`, `tmax` et `pluie` (unités probables) ? Sous quel format les dates sont-elles écrites ?
3. Ouvrir aussi `stations.csv`. Quel descripteur est **commun** aux deux fichiers ?

★ **Exercice 2** *Charger les deux tables*▶

La fonction `charger(fichier, separateur)` est donnée : c'est celle du cours, avec en plus le paramètre `delimiter` de `csv.DictReader`.

1. Charger les deux fichiers dans des variables `releves` et `stations`. Afficher le nombre de lignes de chaque table, puis `releves[0]`.
2. Essayer `charger("releves_aout.csv", ",")`. Que contient alors `releves[0]` ? Expliquer.
3. De quel type est `releves[0]["tmax"]` ? Pourquoi `float(releves[0]["tmax"])` provoque-t-il une erreur ?

★ **Exercice 3** *Convertir un nombre « à la française »*▶

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Compléter `en_nombre(texte)` qui convertit une valeur lue dans le fichier en `float` (`"23,4"` donne `23.4`) et renvoie `None` si la chaîne est vide (valeur manquante). ▷ coup de pouce

```
>>> en_nombre("23,4")
23.4
>>> en_nombre("") is None
True
```

## 2 Contrôle qualité : repérer les lignes douteuses

Avant de calculer quoi que ce soit, un analyste **vérifie** ses données. Une seule valeur absurde suffit à fausser une moyenne ou un record.

★ **Exercice 4** *Les valeurs manquantes*▶

Compléter `est_complete(ligne)`, qui renvoie `True` si `tmin`, `tmax` et `pluie` sont toutes renseignées. Afficher ensuite, par une **compréhension**, la station et la date des lignes incomplètes. Combien y en a-t-il ?

★★ **Exercice 5** *Les doublons*▶

Une même station ne doit envoyer qu'**un** relevé par jour : la clé d'une ligne est le couple `(station, date)`.

1. Compléter `compter_par_cle(table)`, qui renvoie un **dictionnaire** associant à chaque clé `(station, date)` le nombre de lignes qui la portent. ▷ coup de pouce
2. En déduire `doublons(table)`, la liste des clés présentes plus d'une fois. Quelles sont-elles ?
3. Afficher les lignes de chacun de ces doublons. Sont-ils tous **identiques** ? Lequel pose un vrai problème ?

★★ **Exercice 6** *Les valeurs incohérentes*▶

Compléter `est_coherente(ligne)` : elle renvoie `True` si la ligne est complète, si `tmin` $\leqslant$ `tmax`, et si les deux températures sont comprises entre $-30$ et 50 degrés. Lister les lignes complètes mais incohérentes et proposer, pour chacune, une **explication** plausible de l'erreur.

★★ **Exercice 7** *Nettoyer la table*▶

Compléter `nettoyer(table)`, qui construit une **nouvelle** table :

- ne garde que les lignes complètes et cohérentes ;
- ne garde, pour chaque clé `(station, date)`, que la **première** ligne rencontrée (un dictionnaire `deja_vus` retient les clés déjà gardées) ;
- range dans chaque nouvelle fiche des **nombres** (`float`) et non plus des chaînes.

Combien de lignes reste-t-il ? Combien ont été écartées ? **Tout le reste du TP utilise la table nettoyée**, notée `propre`.

▷ coup de pouce

## 3 Interroger la table : filtres et calculs

★ **Exercice 8** *Une station à la loupe*▶

Compléter `releves_de(table, station)` (filtre par compréhension) et `moyenne(table, descripteur)`. Pour Monaco (`"MON"`), afficher le nombre de relevés et la moyenne des températures maximales, affichée au dixième avec une f-string : `f"{x:.1f}"`.

★★ **Exercice 9** *Records et comptages*▶

1. Compléter `jour_le_plus_chaud(table)`, qui renvoie le triplet `(station, date, tmax)` de la ligne où `tmax` est maximale (invariant du champion). Quel est le record du mois, toutes stations confondues ? Et celui de Monaco ?
2. Compléter `compter_si(table, descripteur, seuil)`, qui compte les lignes dont la valeur est $\geqslant$ `seuil`. Une **nuit tropicale** est une nuit où la température ne descend pas sous **20 degrés** : combien Monaco en a-t-elle connu ? Et Gréolières ?
3. Un **jour de pluie** est un jour avec au moins **1 mm**. Pour chaque station, afficher le cumul de pluie du mois et le nombre de jours de pluie (une boucle sur la table `stations`).

★★ **Exercice 10** *L'épisode orageux*▶

On veut maintenant regrouper les relevés **par date**, toutes stations confondues.

1. Écrire `pluie_par_jour(table)` qui renvoie un **dictionnaire** associant à chaque date le cumul de pluie de toutes les stations ce jour-là. Combien de clés contient-il ?
2. Quel jour le réseau a-t-il été le plus arrosé, et combien de millimètres sont tombés au total ? Quel autre jour le suit de près ?
3. Combien de jours du mois n'a-t-il plu **nulle part** ?

▷ coup de pouce

## 4 Trier

★★ **Exercice 11** *Palmarès*▶

1. Avec `sorted` et une clé, afficher les **cinq** journées les plus chaudes du mois (station, date, `tmax`). Que remarquez-vous ?
2. Construire la liste des couples `(nom de la station, cumul de pluie)` et la trier du plus arrosé au plus sec.
3. Les dates sont écrites `AAAA-MM-JJ`. Comparer `sorted(["2025-08-15", "2025-09-02", "2025-08-20"])` et `sorted(["15/08/2025", "02/09/2025", "20/08/2025"])`. Pourquoi les informaticiens préfèrent-ils le premier format ?

## 5 Croiser : la jointure avec la table des stations

★★ **Exercice 12** *Littoral ou arrière-pays ?*▶

La fonction `jointure` du cours est fournie dans le programme.

1. Calculer la jointure de `propre` et de `stations` sur le descripteur `"station"`. Combien de lignes obtient-on ? Comparer avec `len(propre)` : quelles lignes ont **disparu**, et pourquoi ? Comment un analyste devrait-il réagir ?
2. Grâce au descripteur `littoral` apporté par la jointure, comparer la moyenne des `tmin` des stations du littoral et celle des stations de l'arrière-pays. Même question pour les `tmax`.
3. Calculer, pour les deux groupes, l'**écart moyen** entre `tmax` et `tmin`. Proposer une explication (la mer, l'altitude).
4. La station `VIL` est celle de **Villefranche-sur-Mer** (altitude 10 m, sur le littoral). Ajouter sa fiche à la table `stations` avec `append` (un dictionnaire ayant les mêmes descripteurs que les autres fiches), puis refaire la jointure : combien de lignes obtient-on maintenant ?

## 6 Produire : exporter un CSV de bilan

★★ **Exercice 13** *Le fichier* `bilan_stations.csv`▶

1. Compléter `en_texte(nombre)`, réciproque de `en_nombre`, qui écrit le nombre avec **un** chiffre après la virgule : `en_texte(23.4)` vaut `"23,4"` et `en_texte(31)` vaut `"31,0"`. ▷ coup de pouce
2. Construire une table `bilan` contenant **une fiche par station**, avec les descripteurs `station`, `nom`, `tmin_moy`, `tmax_moy`, `pluie_totale`, `nuits_tropicales` (moyennes et cumul arrondis au dixième, écrits avec `en_texte`).
3. Compléter la fonction `exporter` (une ligne manque : écrire chaque fiche avec `ecrivain.writerow`) et produire `bilan_stations.csv`, séparateur `;`.
4. Vérifier le résultat de deux façons : en le rechargeant avec `charger`, puis en l'ouvrant dans le **tableur**. Les nombres sont-ils bien reconnus ?

> **Remarque** — *Pourquoi écrire* `23,4` *et pas* `23.4` *?*
> Un tableur réglé en français attend une virgule décimale ; avec `23.4`, il risque de lire du texte, voire une date. Produire un fichier, c'est penser à **celui qui va le relire**.

## 7 Pour aller plus loin, et compte rendu

★★★ **Exercice 14** *La plus longue vague de chaleur*▶

Écrire `plus_longue_serie(table, seuil)` qui renvoie le plus grand nombre de **jours consécutifs** où `tmax` ⩾ `seuil`, pour une table ne contenant qu'une station. Quelle station a connu la plus longue série de jours à 30 degrés ou plus ? *Attention : la table doit être parcourue dans l'ordre des dates, et il peut manquer des jours (relevés écartés au nettoyage). Un jour manquant* ***interrompt*** *la série, comme un jour sous le seuil.* ▷ coup de pouce

★★★ **Exercice 15** *[au-delà du programme] La même chose avec* `pandas`▶

*Si la bibliothèque* `pandas` *est installée.* Les scientifiques des données écrivent ce TP en quelques lignes :

```
import pandas
t = pandas.read_csv("releves_aout.csv", sep=";", decimal=",")
t = t.dropna()                                          # lignes incompletes
t = t[(t["tmin"] <= t["tmax"]) & (t["tmax"] <= 50)]     # lignes incoherentes
t = t.drop_duplicates(subset=["station", "date"])       # doublons
print(t.groupby("station")["tmax"].mean().round(1))
```

Exécuter ce code et comparer avec vos moyennes. Une station apparaît ici alors qu'elle avait disparu de votre première jointure (exercice 12, question 1) : laquelle, et pourquoi ? Qu'est-ce que votre propre code vous a appris que ces cinq lignes cachent ?

**Exercice 16** *Compte rendu*

Rédiger, en quelques lignes, un compte rendu (dans un fichier texte placé à côté du programme, ou sur le cahier) :

1. combien de lignes le fichier contenait, combien ont été écartées, et pour quelles raisons ;
2. trois résultats chiffrés tirés de votre bilan (un record, une comparaison littoral/arrière-pays, un classement) ;
3. une phrase sur ce qui se serait passé si l'on avait calculé les records **sans** nettoyer la table.

## Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Construire une nouvelle chaîne en parcourant le texte caractère par caractère : on recopie chaque caractère, sauf la virgule, qu'on remplace par un point. Il ne reste plus qu'à appliquer `float`. ↑énoncé

Ex. 5 q. 1 – piste C'est le patron du comptage avec un dictionnaire : si la clé est déjà présente, on ajoute 1 ; sinon on la crée avec la valeur 1. ↑énoncé

Ex. 7 – piste Pour chaque ligne : si elle est cohérente et que sa clé n'est pas dans `deja_vus`, on enregistre la clé, puis on ajoute à `propre` un nouveau dictionnaire dont les trois valeurs numériques passent par `en_nombre`. ↑énoncé

Ex. 10 – piste Même patron que `compter_par_cle` : la clé est la date, et au lieu d'ajouter 1, on ajoute la pluie de la ligne. Pour le jour le plus arrosé, appliquer l'invariant du champion aux clés du dictionnaire. ↑énoncé

Ex. 13 q. 1 – piste La f-string `f"{nombre:.1f}"` écrit le nombre au dixième, avec un point ; il reste à remplacer ce point par une virgule, caractère par caractère, comme dans `en_nombre`. ↑énoncé

Ex. 14 – piste Trier d'abord les relevés par date ; le numéro du jour est `int(ligne["date"][8:10])`. Garder deux compteurs : la longueur de la série en cours et la meilleure longueur vue jusque-là. La série en cours repart à 0 sous le seuil, et à 1 si le jour précédent manque. ↑

# PROJET Les prénoms de l'INSEE

*filtrer, trier, regrouper et croiser un vrai fichier de données publiques*

Durée : 2 à 3 séances · En binôme, sur machine

**Fichiers à télécharger :** `maththibaud.github.io/informatique-en-autonomie/f/p08-projet-prenoms-insee` — **archive .zip**

## But du projet

Depuis 1900, l'INSEE compte, année après année, les prénoms donnés aux enfants nés en France, et publie ces données en accès libre. Vous allez explorer ce fichier **réel** avec Python, **sans bibliothèque spécialisée** : l'**importer** et le **nettoyer**, l'**interroger** (popularité d'un prénom, palmarès d'une année), **regrouper** ses lignes avec un dictionnaire, **trier**, **croiser** deux tables et **tracer** des courbes. Pour finir, chaque binôme pose **sa propre question** aux données, la traite et présente sa réponse à la classe.

## Consignes

- La restitution (partie 8) s'ajoute à ces séances. Les parties 1 à 7 sont guidées, la partie 8 est libre.
- Fichiers à télécharger (lien ci-dessus), à placer **dans le même dossier** : `prenoms_insee_extrait.csv` (les données), `projet_prenoms_evenements.csv` (une petite table pour la partie 6) et `projet_prenoms_insee_depart.py` (le programme à compléter : tout votre code s'y écrit).
- ▶ *À programmer.* À chaque exécution, le programme teste vos fonctions et affiche, pour chaque partie, `OK` ou `ECHEC`. Au début, tout est en échec : c'est normal. Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## Les données

**Source** : INSEE, *Fichier des prénoms*, édition 2025 (parue en juillet 2026), fichier national : naissances en France de 1900 à 2025. Données publiées sous **licence ouverte** (Etalab 2.0) : on peut les copier, les transformer et les rediffuser librement, à condition de **citer la source**. Page de téléchargement : `www.insee.fr/fr/statistiques/8595130`.
Le fichier complet compte près de 725 000 lignes (16 Mo). L'extrait fourni a été fabriqué par un petit programme (`projet_prenoms_extraire.py`) qui garde le **même format** mais seulement les naissances de **1950 à 2025** et les lignes dont l'effectif atteint **au moins 100**.

# 1 Découvrir le fichier

★ **Exercice 1** *Regarder avant de programmer*

Ouvrir `prenoms_insee_extrait.csv` dans un **éditeur de texte**. En voici le début :

```
sexe;prenom;periode;valeur;rang
1;GABRIEL;2025;4625;1
1;NOAH;2025;3465;2
1;LÉO;2025;3420;3
```

1. Quels sont les descripteurs ? Quel est le séparateur ? Comment le sexe est-il codé (chercher `LOUISE`) ?
2. Comment les prénoms sont-ils écrits ? Observer le dernier chiffre des nombres de la colonne `valeur` : quels chiffres rencontre-t-on ? D'après la documentation de l'INSEE, les effectifs sont **arrondis au multiple de 5 le plus proche** : pourquoi, à votre avis ?

3. Le cours rappelle l'ancien en-tête `sexe;preusuel;annais;nombre`, celui des **éditions précédentes** du fichier. Quelle leçon en tirer pour un programme qui lit des données publiques ?
4. Le prénom Léo n'apparaît pas en 1980 dans l'extrait. Peut-on en conclure qu'aucun Léo n'est né en 1980 ?

## 2 Importer et nettoyer

★ **Exercice 2** *Charger la table*▶

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Écrire `charger(fichier, separateur)` : c'est la fonction du cours, avec en plus le paramètre `delimiter` de `csv.DictReader`. Charger l'extrait dans une variable `brut`. Combien de lignes contient-il ? Afficher `brut[0]` : de quel type sont les valeurs ?

★★ **Exercice 3** *Une fiche propre*▶

Pour travailler confortablement, on transforme chaque ligne brute en une **nouvelle** fiche, plus lisible. Compléter `convertir(ligne)` :

```
>>> convertir({"sexe": "1", "prenom": "LÉO", "periode": "2025", "valeur": "3420", "rang":
    "3"})
{'sexe': 'G', 'prenom': 'Léo', 'annee': 2025, 'nombre': 3420, 'rang': 3}
```

Pour le prénom, compléter d'abord `prenom_lisible(prenom)`, qui garde la majuscule en **début de mot** (premier caractère, ou caractère qui suit un tiret) et met les autres lettres en minuscules : `prenom_lisible("JEAN-PIERRE")` vaut `"Jean-Pierre"`. Elle s'appuie sur la fonction `en_minuscule`, fournie dans le fichier de départ : dans la table Unicode, la minuscule d'une lettre majuscule, accentuée ou non (de `A` à `Z` et de `À` à `Ü`), est rangée **32** places plus loin.

```
1 def en_minuscule(car):
2     if "A" <= car <= "Z" or "À" <= car <= "Ü":
3         return chr(ord(car) + 32)      # majuscule -> minuscule
4     return car                         # pas une majuscule : inchange
```

▷ coup de pouce

★★ **Exercice 4** *Un programme robuste*▶

Les éditions précédentes du fichier contenaient des lignes qui ne décrivent pas un vrai prénom une année connue : le prénom `_PRENOMS_RARES` (le cumul des prénoms trop rares, regroupés pour qu'on ne puisse pas reconnaître les personnes) et l'année `XXXX` (année de naissance inconnue). L'extrait n'en contient pas, mais votre programme doit pouvoir lire *aussi* ces anciennes éditions.

1. Compléter `est_valide(ligne)`, qui reçoit une ligne **brute** (avant conversion) et renvoie `False` si `ligne["prenom"]` commence par `_` ou si `ligne["periode"]` n'est pas écrite uniquement avec des chiffres.

▷ coup de pouce

2. Un camarade propose plutôt le test `"RARES" in ligne["prenom"]`. Or le fichier complet contient la ligne `1;RARES;2025;15;2053`. Que décrit-elle ? Pourquoi ce test serait-il une erreur ?
3. Compléter `nettoyer(table)`, qui renvoie la table des fiches converties, pour les seules lignes valides. **Toute la suite utilise cette table nettoyée**, notée `table`.

## 3 Interroger la table

★ **Exercice 5** *Combien de Louise ?*▶

Compléter `nombre_de(table, prenom, sexe, annee)`, qui renvoie le nombre de naissances, ou `0` si la table ne contient pas de ligne correspondante. Combien de Louise sont nées en 1950, en 1975, en 2000, en 2025 ? Que signifie exactement le `0` renvoyé pour 1975 ?

★★ **Exercice 6** *La carrière d'un prénom*▶

1. Compléter `evolution(table, prenom, sexe)`, qui renvoie un **dictionnaire** `{annee: nombre}`. En combien d'années le prénom Kevin figure-t-il dans l'extrait ? Quelle est la première ?
2. Compléter `annee_record(table, prenom, sexe)`, qui renvoie le couple `(annee, nombre)` de l'année la plus forte (invariant du champion sur les clés du dictionnaire). Donner l'année record de Marie, de Kevin, de Léo et de Jade. Pourquoi le record de Marie est-il à prendre avec précaution ?

★★ **Exercice 7** *Le top 10 d'une année*▶

Compléter `top(table, annee, sexe, n)`, qui renvoie la liste des `n` couples `(prenom, nombre)` les plus donnés. Afficher le top 10 de 2025 pour les filles et pour les garçons, puis celui de 1950. Vérifier votre classement grâce au descripteur `rang`.

8

## 4 Regrouper avec un dictionnaire

★★ **Exercice 8** *Totaliser sur plusieurs années*▶

Compléter `total_par_prenom(table, sexe, debut, fin)`, qui renvoie un dictionnaire associant à chaque prénom le total de ses naissances de `debut` à `fin` (inclus), puis `champion(dico)`, qui renvoie le couple `(cle, valeur)` de la plus grande valeur. Quel prénom de fille a été le plus donné de 2020 à 2025, et combien de fois ? ▷ coup de pouce

★★ **Exercice 9** *Le prénom de chaque décennie*▶

Compléter `plus_donne_par_decennie(table, sexe)`, qui renvoie un dictionnaire `{1950: (prenom, total), 1960: ..., 2020: ...}`. Présenter les résultats des filles et des garçons dans un tableau. Pourquoi la décennie 2020 n'est-elle pas comparable aux autres ? ▷ coup de pouce

★★★ **Exercice 10** *Des prénoms de plus en plus variés ?*▶

Compléter `diversite(table, annee, sexe)`, qui renvoie le nombre de prénoms différents de l'année, le total des naissances correspondantes et la **part du top 10** dans ce total (en pourcentage, à afficher au dixième avec `f"{part:.1f}"`). Comparer 1950, 1975, 2000 et 2025, pour chaque sexe, puis rédiger deux phrases de conclusion. Le total obtenu pour 2025 est-il le nombre de bébés nés en France cette année-là ?

## 5 Trier

★★ **Exercice 11** *Le palmarès de 1950 à 2025*▶

Compléter `palmares(totaux, n)`, qui transforme le dictionnaire `totaux` en une liste de couples `(prenom, total)`, la trie avec `sorted` et une clé, et renvoie les `n` premiers. Afficher le palmarès des 10 prénoms les plus donnés de 1950 à 2025, pour chaque sexe. Combien de ces prénoms figurent dans un top 10 de 2025 ?

★★★ **Exercice 12** *Les ex aequo*▶

1. En 2025, deux prénoms de filles du top 20 ont exactement la même valeur. Lesquels ? Quel rang l'INSEE donne-t-il à chacun ?
2. Compléter `classement(table, annee, sexe)`, qui renvoie la liste des prénoms de l'année du plus donné au moins donné et, **à égalité**, dans l'ordre alphabétique. ▷ coup de pouce
3. Votre classement et celui de l'INSEE diffèrent-ils sur ces deux prénoms ? Pourquoi l'INSEE peut-il les départager, et pas vous ?

## 6 Croiser deux tables

★★ **Exercice 13** *Qui monte, qui descend en dix ans ?*▶

On veut comparer 2015 et 2025 par une **jointure** sur le descripteur `prenom` (la fonction `jointure` du cours est fournie).

1. Pourquoi ne peut-on pas joindre directement les fiches de 2015 et celles de 2025 ? Que deviendrait le descripteur `nombre` dans la fiche fusionnée ?
2. Compléter `colonne_annee(table, annee, sexe)`, qui renvoie des fiches réduites à deux descripteurs, comme `{"prenom": "Louise", "n2015": 4545}`.
3. Compléter `montees(table, sexe, a1, a2)` (voir sa spécification dans le programme). Combien de prénoms de filles figurent à la fois en 2015 et en 2025 ? Quels sont les 5 plus fortes hausses et les 5 plus fortes baisses, chez les filles puis chez les garçons ?
4. Les prénoms donnés en 2025 mais absents en 2015 ont **disparu** de la jointure. Compléter `nouveaux(table, sexe, a1, a2)` pour les retrouver. Combien y en a-t-il chez les garçons ? Expliquer la présence simultanée d'`Elio` parmi les hausses et d'`Élio` parmi les nouveaux.

★★★ **Exercice 14** *L'effet d'un film, d'une série, d'un champion*▶

On entend souvent qu'un film, une série ou un sportif lance la mode d'un prénom. Pour le vérifier, on dispose d'une petite table qui associe cinq prénoms à un événement et à son année (descripteurs `prenom`, `annee_evt`, `evenement`) : c'est le fichier `projet_prenoms_evenements.csv`.

1. Charger cette table, convertir `annee_evt` en entier, puis calculer sa jointure avec `table` sur le descripteur `prenom`. Combien de lignes obtient-on ? Que contient chacune ?
2. Compléter `avant_apres(fusion, prenom)` (voir sa spécification) et dresser, pour les cinq prénoms, le tableau : événement, nombre l'année d'avant, maximum dans les années qui suivent.
3. Pour quels prénoms les données appuient-elles l'idée d'un « effet » ? Pour lequel la hausse avait-elle commencé **bien avant** l'événement ? (Regarder `evolution(table, prenom, sexe)` sur les années précédentes ; la partie 7 permettra de le voir sur une courbe.) Un tableau de chiffres suffit-il à prouver qu'un événement est la *cause* d'une mode ?

## 7 Tracer des courbes

★★ **Exercice 15** *La courbe d'un prénom*▶

La fonction `tracer` est presque complète : elle utilise la bibliothèque `matplotlib`, où `plt.plot(abscisses, ordonnees)` trace une courbe à partir de deux listes de même longueur.

1. Compléter la ligne `nombres = ...` : une liste qui contient, pour chaque année de `annees`, le nombre de naissances, ou `0` si l'année est absente du dictionnaire `evo`.
2. Tracer les courbes de Marie, Léa et Kevin, puis celles des cinq prénoms de la table des événements. Les courbes confirment-elles vos conclusions de l'exercice précédent ?
3. Le fichier `courbes.png` produit servira pour la restitution.

## 8 Votre question, votre réponse

★★★ **Exercice 16** *Une question posée aux données*▶

Chaque binôme formule **une question précise** à laquelle le fichier permet de répondre, la soumet au professeur, puis la traite en **réutilisant** les fonctions déjà écrites. Quelques pistes (on peut en choisir une autre) :

- les **prénoms composés** (`Marie-...`, `Jean-...`) : quelle part des naissances, décennie par décennie ?
- les prénoms **mixtes**, donnés aux filles *et* aux garçons la même année : lesquels, et depuis quand ?
- la **longueur moyenne** des prénoms (pondérée par le nombre de naissances) a-t-elle changé ?
- les **variantes d'orthographe** (Kevin, Kévin ; Eric, Éric ; Elio, Élio) : laquelle l'emporte, et quand ?
- la « **durée de vie** » d'une mode : pendant combien d'années un prénom reste-t-il au-dessus de la moitié de son record ?
- les prénoms de la **génération de vos parents** : que sont devenus les dix prénoms préférés de 1985 ?

Le binôme rend son programme (commenté) et prépare une **restitution** au choix : un **court oral** de 3 minutes devant la classe, ou une **affiche** A3. Dans les deux cas, on y trouve : la question ; la méthode (quelles fonctions, quels filtres, quels regroupements) ; le résultat, chiffré, avec au moins un tableau ou une courbe ; **une limite** des données qui pourrait fausser la réponse.

| Critères d'évaluation | Observations |
|---|---|
| Parties 2 à 7 : fonctions écrites, tests `OK`, réponses aux questions | |
| Question précise, à laquelle le fichier permet vraiment de répondre | |
| Traitement : fonctions réutilisées, code lisible, commenté et testé | |
| Résultat chiffré, présenté par un tableau ou une courbe lisible | |
| Regard critique : une limite des données identifiée (seuil de 100, arrondi à 5, orthographes…) | |
| Restitution claire, dans le temps imparti ; chaque membre du binôme sait expliquer le code | |

## 9 Pour aller plus loin

★★★ **Exercice 17** *Le fichier complet*▶

Télécharger le fichier national complet sur la page de l'INSEE (`www.insee.fr/fr/statistiques/8595130`, lien « Fichier national », archive `prenoms-2025-nat_csv.zip`), le décompresser, et relancer votre programme sur `prenoms-2025-nat.csv`. Combien de temps prend le chargement ? et la partie 6 (pensez au coût de la jointure) ? Les naissances de Léo en 1980, de Lilou en 1997 ou d'Arya en 2011 apparaissent-elles maintenant ? Quels résultats de votre projet changent ?

★★★ **Exercice 18** *[ au-delà du programme ] Et avec* `pandas` *?*▶

*Si la bibliothèque* `pandas` *est installée.* Exécuter le code suivant et retrouver deux résultats de votre projet. Qu'a dû deviner `pandas` que votre fonction `convertir` faisait explicitement ?

```
import pandas
t = pandas.read_csv("prenoms_insee_extrait.csv", sep=";")
filles25 = t[(t["periode"] == 2025) & (t["sexe"] == 2)]
print(filles25.sort_values("valeur", ascending=False).head(3))
print(t[t["sexe"] == 1].groupby("prenom")["valeur"].sum().nlargest(3))
```

## Coups de pouce *après avoir vraiment cherché : la* ***piste*** *d'abord, le* ***début de solution*** *si besoin.*

**Ex. 3 – piste** Construire le résultat caractère par caractère, avec une variable booléenne `debut_de_mot`, vraie au départ ; après chaque caractère, elle devient vraie si ce caractère est un tiret, fausse sinon. ↑énoncé

**Ex. 4 q. 1 – piste** La tranche `"_PRENOMS"[:1]` vaut `"_"` (le premier caractère). Pour l'année, parcourir ses caractères : un caractère `car` est un chiffre lorsque `"0" <= car <= "9"` ; dès qu'on en trouve un qui n'en est pas un, la ligne n'est pas valide. ↑énoncé

**Ex. 8 – piste** C'est le patron du comptage avec un dictionnaire, mais au lieu d'ajouter 1, on ajoute le nombre de la fiche. ↑énoncé

**Ex. 9 – piste** Une boucle `for debut in range(1950, 2030, 10)`, et dans la boucle, un appel à `champion` sur le résultat de `total_par_prenom`. ↑énoncé

**Ex. 12 q. 2 – piste** Trier **deux fois** : d'abord par prénom, puis par nombre (décroissant). `sorted` ne change jamais l'ordre de deux fiches qui ont la même clé : à nombre égal, l'ordre alphabétique obtenu au premier tri est donc conservé. ↑énoncé

# FICHE DE RÉVISION

### L'essentiel — une table = une liste de dictionnaires

Une **table** range l'information en lignes et colonnes. Un **descripteur** est le nom d'une colonne ; un **enregistrement** (une fiche, une ligne) donne une valeur pour chaque descripteur. En Python : **1 fiche = 1 dictionnaire**, **la table = une liste** de ces dictionnaires. Accès en deux temps : la fiche par son **indice**, la valeur par son **descripteur** (`table[2]["nombre"]` : 3e fiche, colonne `nombre`).

### Importer un CSV — et LE piège

Un **CSV** est un fichier *texte* : 1re ligne = en-tête (les descripteurs), lignes suivantes = enregistrements, valeurs séparées par un **séparateur** (`,` ou, « à la française », `;`).

```
import csv
def charger(fichier):
    with open(fichier, encoding="utf-8", newline="") as f:
        lecteur = csv.DictReader(f)    # delimiter=";" si besoin
        table = [ligne for ligne in lecteur]
    return table
```

**Piège n° 1 :** tout ce qu'on lit est une **chaîne** (`"3177"`), même les nombres. On **convertit** avec `int(...)` / `float(...)` avant tout *calcul* ou *tri* numérique.

### Rechercher / filtrer — c'est un parcours

On parcourt et on **garde** les fiches qui vérifient le critère (accumulateur = une liste).

```
def selection(table, sexe, annee):
    resultat = []
    for ligne in table:
        if ligne["sexe"] == sexe and ligne["annee"] == annee:
            resultat.append(ligne)
    return resultat
```

En compréhension : `[ligne for ligne in table if ligne["sexe"] == "F"]`. Pour trouver **une** fiche (`chercher`) : parcourir et `return` **dès** qu'on l'a trouvée (`None` sinon).

### Calculer sur une colonne — comptage, moyenne, min/max

Le patron du parcours… avec `int(...)`. Le **max** garde la *fiche championne*, initialisée à la première.

```
def plus_donne(table):
    champion = table[0]
    for ligne in table:
        if int(ligne["nombre"]) > int(champion["nombre"]):
            champion = ligne
    return champion["prenom"], int(champion["nombre"])
```

Chaque fonction parcourt la table **une fois** ⇒ coût **linéaire**.

### Trier selon un descripteur

```
sorted(table, key=lambda ligne: int(ligne["nombre"]), reverse=True)
sorted(table, key=lambda ligne: ligne["prenom"])
```

Par valeur décroissante, puis par ordre alphabétique : `key` dit **sur quoi** comparer ; `sorted` renvoie une **nouvelle** liste. **Sans `int`**, on trie « comme des mots » : `"900"` passe devant `"3070"` !

### Fusionner deux tables — la jointure

Pour chaque fiche de gauche, chercher celle de droite qui a **la même clé**, et combiner les deux dictionnaires. Une fiche **sans correspondance disparaît**.

```
def jointure(gauche, droite, cle):
    resultat = []
    for lg in gauche:
        for ld in droite:
            if lg[cle] == ld[cle]:
                fusion = {c: lg[c] for c in lg}
                for c in ld:
                    fusion[c] = ld[c]
                resultat.append(fusion)
    return resultat
```

### Erreurs à éviter

Calculer ou trier sans `int` ; se tromper de séparateur (`delimiter=";"`) ou d'encodage (`encoding="utf-8"`) ; confondre `table[0]` (une fiche) et `table[0]["prenom"]` (une valeur) ; oublier les fiches perdues par une jointure.

### Savoir-faire à maîtriser — je sais…

- ☐ reconnaître descripteurs et enregistrements, lire un fichier CSV brut ;
- ☐ **importer** un CSV en liste de dictionnaires avec `csv.DictReader` ;
- ☐ **filtrer** selon un ou plusieurs critères (boucle ou compréhension) ;
- ☐ repérer les **doublons** et les valeurs **incohérentes** d'une table ;
- ☐ **calculer** moyenne / min / max / comptage sur une colonne, sans oublier `int(...)` ;
- ☐ **trier** une table selon un descripteur avec `sorted` et `key` ;
- ☐ **fusionner** deux tables sur un descripteur commun.

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 654

1. Dans `prenom,sexe,annee,nombre`, que sont ces quatre mots ?
2. Que vaut `table[0]["nombre"] + table[1]["nombre"]` si ces valeurs sont `"3177"` et `"3168"` ?
3. Tout le CSV tient dans une seule colonne. Cause probable ?
4. Trier une table par `nombre` décroissant : quelle instruction ?
5. Jointure `communes` / `departements` : que devient une commune de code `2A` absent à droite ?

**Et avant ?** — *questions sur les chapitres précédents.* ▶ réponses p. 654

1. Combien de tours fait au plus une recherche dichotomique dans un tableau trié d'un million d'éléments ? (ch. 7)
2. Qu'est-ce qu'une précondition ? (ch. 5)
3. Pourquoi un fichier UTF-8 relu en Latin-1 affiche-t-il « Ã© » au lieu de « é » ? (ch. 2)

**S'entraîner sur plusieurs chapitres**

- Problème de synthèse n° 6 — Le classement d'un cross, p. 512
- Problème de synthèse n° 7 — Les doublons d'une liste d'inscriptions, p. 513

# DÉFI ADVENT OF CODE

## DÉFI Cube Conundrum

*les cubes de couleur — découpage imbriqué de chaînes*

Jour 2

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p08-defi-aoc-2023-02` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2023/day/2`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Chaque ligne du fichier raconte une partie d'un jeu de cubes de couleur : son numéro, puis plusieurs tirages. Il faut d'abord trouver les parties compatibles avec un sac donné (partie 1), puis le plus petit sac possible pour chaque partie (partie 2). Une centaine de lignes, chacune avec plusieurs tirages : le calcul à la main serait long et source d'erreurs.
**Ce qu'on réinvestit.** La lecture d'un fichier et le **découpage de lignes en champs avec `split`**, vus dans les défis précédents, avec ici **plusieurs niveaux de séparateurs** : les deux-points séparent le numéro du reste, le point-virgule sépare les tirages, la virgule sépare les couleurs d'un tirage, l'espace sépare le nombre de la couleur. C'est le travail typique des données en tables : extraire des champs d'un texte structuré. La partie 2 reprend en plus le calcul d'un maximum.
**La démarche de la fiche.**

1. Comprendre la règle sur des cas particuliers, puis la faire à la main sur quatre parties.
2. Découper une ligne étage par étage, en affichant le résultat de chaque `split`.

3. Écrire une fonction qui dit si une partie est possible, puis additionner les numéros : on obtient la partie 1.
4. Pour la partie 2, garder le même découpage et calculer, pour chaque couleur, le plus grand nombre de cubes vus.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `cube` | cube | `bag` | sac |
| `game` | partie | `reveal` | montrer, révéler |
| `handful` | poignée | `semicolon` | point-virgule |
| `possible` | possible | `fewest` | le moins possible |
| `power` | puissance (ici : produit) | `set` | ensemble, lot |

### Résumé de l'énoncé (partie 1)

**Contexte.** Sur une île, un elfe vous propose un jeu. Il a mis dans un sac des cubes rouges, verts et bleus, sans dire combien. À chaque partie, il sort plusieurs poignées de cubes, les montre, puis les remet dans le sac. On veut savoir quelles parties auraient pu se dérouler avec un contenu de sac donné.
**Les données.** Chaque ligne du fichier raconte une partie et a la forme `Game n:` suivi des tirages, séparés par des points-virgules `;`. Un tirage est une liste de morceaux séparés par des virgules, chaque morceau ayant la forme `3 blue` : un nombre, une espace, une couleur (`red`, `green` ou `blue`). Une couleur peut être absente d'un tirage, et l'ordre des couleurs varie.
**La règle.**

- L'énoncé fixe le contenu du sac : **12 cubes rouges, 13 verts et 14 bleus**.
- Une partie est **possible** avec ce sac si aucun tirage ne montre plus de cubes d'une couleur que le sac n'en contient (autant, c'est permis).
- Les cubes sont remis dans le sac entre deux tirages : on compare chaque tirage séparément, sans additionner les tirages d'une même partie.
- Il suffit d'un seul morceau qui dépasse pour que la partie soit impossible.

**La réponse attendue.** La somme des numéros des parties possibles. Par exemple, avec un sac (inventé) de 5 cubes de chaque couleur, `Game 7: 2 red, 4 blue; 5 green` est possible, mais pas `Game 8: 1 red; 6 blue` : si le fichier ne contenait que ces deux parties, la réponse serait 7.

**Exercice 1** *Comprendre ce qui est demandé*

**But.** Vérifier sa compréhension de la règle sur des cas particuliers.
**À quoi ça sert ?** Chaque question correspond à une erreur possible : oublier qu'une couleur peut manquer, additionner les tirages d'une partie, refuser un tirage qui atteint exactement la limite.

Après lecture du résumé (et de l'énoncé), vérifier sa compréhension sur ces cas particuliers :

1. Une couleur peut-elle être absente d'un tirage ? Qu'est-ce que cela change ?
2. Si une partie montre 7 rouges dans un tirage puis 7 rouges dans un autre, faut-il additionner ?
3. Un tirage avec exactement autant de cubes que le sac en contient est-il permis ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Décider à la main quelles parties d'un petit fichier sont possibles.
**À quoi ça sert ?** On obtient une réponse connue (`6`) pour tester le programme ; ce petit fichier resservira pour la partie 2.

On considère le fichier (inventé) suivant, avec ce même sac (12 rouges, 13 verts, 14 bleus) :

```
Game 1: 4 red, 2 green; 1 blue, 13 red
Game 2: 3 blue; 5 green, 2 red; 1 red, 1 green, 2 blue
Game 3: 14 green, 3 red; 10 blue
Game 4: 6 blue, 6 red, 6 green
```

Quelles parties sont possibles ? Vérification : on doit obtenir `6`.

## 2 Programmer la partie 1

### Exercice 3 *Découper une ligne, étage par étage*

**But.** Extraire d'une ligne le numéro de la partie, puis chaque nombre de cubes avec sa couleur.
**À quoi ça sert ?** C'est l'étape la plus délicate du défi : une fois chaque morceau réduit à un entier et une couleur, les deux parties se résolvent avec de simples comparaisons. Procéder étage par étage, en affichant chaque résultat, permet de repérer immédiatement un séparateur mal choisi.

#### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Rappel pour lire un fichier ligne par ligne :

```
fichier = open("input.txt", encoding="utf-8")
for ligne in fichier:
    ligne = ligne.strip()   # enlève le retour à la ligne final
fichier.close()
```

Sur la ligne `"Game 2: 3 blue; 5 green, 2 red"`, obtenir successivement : le numéro `2` (un entier) ; la liste des tirages `["3 blue", "5 green, 2 red"]` ; pour chaque tirage, la liste de ses morceaux ; pour chaque morceau, le nombre (entier) et la couleur. ▷ coup de pouce ▷ début de solution

### Exercice 4 *Une partie est-elle possible ?*

**But.** Écrire le test « cette partie est possible avec le sac de l'énoncé », puis additionner les numéros des parties possibles.
**À quoi ça sert ?** C'est la réponse de la partie 1 : le découpage de la question précédente fournit les nombres, cette fonction les compare aux limites du sac.

Écrire une fonction `est_possible(tirages)` qui reçoit la partie droite de la ligne (après les deux-points) et renvoie `True` si aucun morceau ne dépasse le contenu du sac. Puis calculer la réponse : `6` sur l'exemple. ▷ coup de pouce

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

**Résumé de la partie 2 (à lire après avoir validé la partie 1)**

**Ce qui change.** On ne compare plus les parties à un sac donné : l'elfe se demande maintenant combien de cubes il fallait au minimum dans le sac pour chaque partie.
**La nouvelle règle.**

- Pour chaque partie, le plus petit sac qui la rend possible contient, pour chaque couleur, le plus grand nombre de cubes de cette couleur vus dans un même tirage de cette partie.
- La « puissance » d'une partie est le produit de ces trois nombres (rouges × verts × bleus).
- Toutes les parties comptent, plus seulement celles qui étaient possibles dans la partie 1.

**La réponse attendue.** La somme des puissances de toutes les parties. Par exemple, pour `Game 7: 2 red, 4 blue; 5 green`, il faut au moins 2 rouges, 5 verts et 4 bleus : la puissance est $2 \times 5 \times 4 = 40$.

### Exercice 5 *Le minimum nécessaire*

**But.** Calculer, pour chaque partie, le plus grand nombre de cubes de chaque couleur, puis la somme des puissances.
**À quoi ça sert ?** C'est la réponse de la partie 2. Le découpage écrit pour la partie 1 se réutilise tel quel : seul le traitement des nombres change (un maximum au lieu d'une comparaison à une limite).

Pour chaque partie, calculer le plus grand nombre de cubes rouges vus dans un même tirage, de même pour les verts et les bleus, puis répondre à la nouvelle question. Sur le petit exemple inventé de l'exercice 2, on doit trouver `692`. ▷ coup de pouce ▷ début de solution

## 4 Approfondissement

### ★★ Exercice 6 *Approfondissement 1 : une seule fonction*

**But.** Résoudre les deux parties avec une seule fonction qui calcule les maxima d'une partie.
**À quoi ça sert ?** On remarque que la partie 1 se déduit de la partie 2 : regrouper le travail commun dans une fonction évite d'écrire deux fois le même parcours.

1. Écrire une fonction `maxima(tirages)` qui renvoie le triplet `(rouge, vert, bleu)` des maxima d'une partie (si ce n'est déjà fait en partie 2), puis réécrire la partie 1 en n'utilisant que cette fonction. ▷ coup de pouce
2. À la main : donner les maxima de `Game 3` de l'exemple. Pourquoi la partie est-elle impossible ?

### ★★ Exercice 7 *Approfondissement 2 : un dictionnaire de couleurs*

**But.** Ranger les trois maxima dans un dictionnaire indexé par les couleurs.
**À quoi ça sert ?** La couleur lue dans le fichier sert directement de clé : le programme devient plus court et s'adapte sans effort à une couleur supplémentaire.

Remplacer les trois variables `rouge`, `vert`, `bleu` par un dictionnaire `plus_grand` dont les clés sont les noms des couleurs. Quel avantage si l'on ajoutait une quatrième couleur ? ▷ coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Quatre `split` emboîtés : sur `": "`, puis sur `"; "`, puis sur `", "`, enfin sur l'espace. Afficher le résultat de chaque étape avant de passer à la suivante. ↑énoncé

Ex. 3 – début de solution `entete, tirages = ligne.split(": ")` ; le numéro est `int(entete.split()[1])`. Puis `for tirage in tirages.split("; "):` et, dedans, `for morceau in tirage.split(", "):` avec `nombre, couleur = morceau.split()`. ↑énoncé

Ex. 4 – piste Une fonction `maximum_autorise(couleur)` qui renvoie la limite correspondant à la couleur (trois `if`) rend le test très court : il suffit qu'un seul morceau dépasse pour que la partie soit impossible. ↑énoncé

Ex. 5 – piste Trois variables `rouge`, `vert`, `bleu` initialisées à 0 au début de chaque partie, et mises à jour avec un `if` à chaque morceau. C'est un calcul de maximum (patron du champion), comme au chapitre *Algorithmique : le parcours séquentiel*. ↑énoncé

Ex. 5 – début de solution Sur l'exemple, les maxima de `Game 4` sont `6, 6, 6`, ce qui donne `216`. Ajouter le produit des trois maxima à un total. ↑énoncé

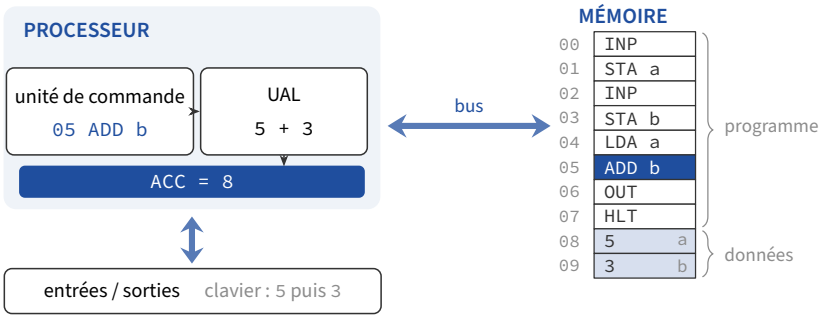
Ex. 6 q. 1 – piste Une partie est possible si et seulement si chacun de ses trois maxima est inférieur ou égal au contenu du sac. ↑énoncé

Ex. 7 – piste Initialiser `plus_grand = {"red": 0, "green": 0, "blue": 0}` ; la couleur lue dans le morceau sert directement de clé, plus besoin de `if`. ↑énoncé

9

CHAPITRE 9

# Architecture des ordinateurs et systèmes d'exploitation






## DANS CE CHAPITRE


- Activités préparatoires . . . . . 350
- Cours . . . . . 352
- Exercices . . . . . 365
- TP et projets . . . . . 373
- Fiche de révision . . . . . 378
- Défi Advent of Code . . . . . 380
- Corrigés . . . . . 655

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 Perdu dans le terminal

▶ corrigé p. 655

*explorer un vrai Linux, sans souris*

Durée : 20 min · Par deux, sur un ordinateur

### Consignes

- Ni installation, ni compte à créer. Les réponses se notent sur le cahier.
- Ouvrir `https://bellard.org/jslinux/` et cliquer sur *click here* sur la ligne **x86 – Alpine Linux 3.12.0 – Console**. Attendre l'invite `localhost:~#`, puis cliquer dans la fenêtre noire.
- On tape une commande, puis la touche **Entrée**. Les espaces comptent (on écrit `cd ..` avec une espace, jamais `cd..`), les majuscules aussi.
- Ce Linux tourne *dans votre navigateur* : vous ne pouvez rien casser. En cas de doute, rechargez la page : tout revient à l'état de départ.
- Le but n'est pas d'apprendre des commandes par cœur, mais de **deviner ce qu'elles font** en observant ce qu'elles affichent.



### Exercice 1 *Où suis-je ?*

1. Taper `pwd`. Qu'affiche le terminal ? Est-ce un nom de fichier, un nom de dossier, ou autre chose ?
2. Taper `ls`. Combien d'éléments sont affichés ? Recopier leurs noms.
3. Taper `cat readme.txt`. Que fait, d'après vous, la commande `cat` ?

### Exercice 2 *Se déplacer*

4. Taper `cd /`, puis `pwd`, puis `ls`. Le dossier `/` contient-il le dossier où vous étiez au départ ? Pourquoi, à votre avis, appelle-t-on `/` la **racine** ?
5. Taper `cd root`, puis `pwd`. Taper ensuite `cd ..`, puis `pwd`. Que désigne `..` ?
6. Depuis n'importe où, taper `cd` tout seul, puis `pwd`. Où arrive-t-on ?
7. Depuis votre point de départ, on peut atteindre `bin` par `cd /bin` ou par `cd ../bin`. Essayer les deux. Quelle est la différence d'écriture entre ces deux *chemins* ?

### Exercice 3 *Créer*

8. Revenir au point de départ. `mkdir` veut dire *make directory*. Créer un dossier `nsi`, entrer dedans, puis y créer un fichier vide avec `touch essai.txt`. Écrire les commandes tapées, dans l'ordre.
9. Dessiner sur le cahier l'arbre des dossiers et fichiers à partir de `/root`, tel que l'affiche `ls`.

**Exercice 4** *Le fichier caché*

**10.** Revenir au point de départ et taper `ls -a`. Quels noms nouveaux apparaissent ? Qu'ont-ils en commun ?

**11.** L'un de ces dossiers cachés contient un dossier `firefox`, qui contient lui-même un fichier `profiles.ini`. Le retrouver (les dossiers s'affichent en bleu), puis l'afficher. Écrire le **chemin complet** de ce fichier, depuis la racine.

**Exercice 5** *Ce qu'on a découvert*

**12.** Recopier sur le cahier et compléter avec vos mots.

- `pwd` affiche ... ; `ls` ...
- `cd` ... ; `..` désigne ...
- un chemin qui commence par `/` part ... ; sinon, il part ...
- un nom qui commence par un point est ...

# COURS

**Programme (BO)** *Modèle d'architecture séquentielle (von Neumann)* : distinguer les rôles et les caractéristiques des différents constituants d'une machine ; *dérouler* l'exécution d'une séquence d'instructions simples du type langage machine ; architectures monoprocesseur et multiprocesseur ; les circuits combinatoires réalisent des fonctions booléennes. *Systèmes d'exploitation* : identifier les fonctions d'un système d'exploitation ; utiliser les commandes de base en ligne de commande ; gérer les droits et permissions d'accès aux fichiers ; systèmes libres et propriétaires. *Périphériques d'entrée et de sortie* : identifier le rôle des *capteurs* et des *actionneurs*. (La partie « réseau » de la même rubrique est traitée au chapitre *Le Web : réseaux et interactions homme-machine*.)

**Prérequis** le **binaire** et le codage de l'information (chapitre *Le binaire et l'écriture des nombres*) ; savoir ce qu'est un **programme** et une **variable**. → fiches de révision : ch. 2 p. 129

**Objectifs** comprendre **comment** une machine exécute un programme ; nommer les **composants** d'un ordinateur et leur rôle ; **dérouler** un petit programme en langage machine ; savoir ce que fait un **système d'exploitation** ; **se déplacer et manipuler** des fichiers en ligne de commande sous Linux ; **lire et modifier** les droits d'un fichier.



## Situation de départ

1. Où se trouve un programme pendant qu'il s'exécute, et qui lit ses instructions une à une ?
2. Comment plusieurs applications ouvertes en même temps se partagent-elles un seul ordinateur sans se gêner ?
3. Sur un ordinateur partagé, qu'est-ce qui empêche un autre utilisateur de lire vos fichiers ?

**Remarque** — *Le fil conducteur : un mille-feuille de couches*

Un ordinateur qui affiche cette page fait, à la base, circuler des **électrons** dans des interrupteurs microscopiques. Comment passe-t-on de l'électron au clic de souris ? Grâce à un empilement de **couches d'abstraction** : chaque couche ne parle qu'à celle juste en dessous, et **cache sa complexité** à celle du dessus.

| |
|---|
| L'**utilisateur** (clic, commande) |
| les **applications** (navigateur, Python…) |
| le **système d'exploitation** (Linux, Windows, macOS) |
| le **langage machine** (instructions du processeur) |
| le **processeur** et la **mémoire** (von Neumann) |
| les **portes logiques**, faites de **transistors** |
| les **électrons** dans le silicium |

Ce chapitre gravit ce mille-feuille : on part de la brique du bas (**le transistor**), on monte au **matériel** (von Neumann) puis au **langage machine**, et on termine par la couche qui donne le plus de confort : le **système d'exploitation**. Les points qui dépassent le programme portent

le badge [ au-delà du programme ].

## I La brique de base : le transistor

Tout, dans un ordinateur, se ramène à deux états : le courant **passe** ou **ne passe pas**, qu'on note **1** et **0**. L'objet qui réalise physiquement ce choix est le **transistor**.

**Définition 1 — Le transistor**

Un **transistor** est un minuscule **interrupteur commandé électriquement** : une troisième borne (la « commande ») décide si le courant passe ou non entre les deux autres. C'est un interrupteur *sans pièce mécanique*, qui bascule des **milliards de fois par seconde**.

En combinant quelques transistors, on fabrique des **portes logiques** qui calculent sur les 0 et les 1. Rappel du chapitre *Le binaire et l'écriture des nombres* : les portes **ET**, **OU**, **NON** (et le **OU exclusif**) réalisent les opérateurs booléens du même nom, décrits par leurs **tables de vérité**, et une porte OU exclusif associée à une porte ET forme déjà un **demi-additionneur**, qui additionne deux bits. En combinant ainsi des portes, on fabrique des circuits qui **additionnent**, **comparent**, **mémorisent**… et de proche en proche, tout un **processeur**. Le transistor est donc la **brique élémentaire** à partir de laquelle tout est construit.

**À retenir — Quelques ordres de grandeur**

Il faut avoir en tête des *ordres de grandeur*, pas des chiffres exacts :

- un transistor mesure aujourd'hui de quelques **nanomètres** à quelques dizaines de nanomètres ($1\ \text{nm} = 10^{-9}$ m, soit environ 5 atomes de silicium côte à côte) ; les « 3 nm » annoncés par les fabricants sont un nom commercial plus qu'une vraie mesure ;
- un processeur de smartphone ou d'ordinateur contient **des milliards à des dizaines de milliards** de transistors ($10^9$ à $10^{10}$) ;
- il exécute quelques **milliards d'opérations par seconde** : sa **fréquence** est de l'ordre du **gigahertz** ($1\ \text{GHz} = 10^9$ battements par seconde).

**Exemple** — *La loi de Moore*

En **1965**, Gordon **Moore** (cofondateur d'Intel) remarque que le nombre de transistors qu'on sait graver sur une puce **double chaque année** ; en **1975**, il révise ce rythme : un doublement **environ tous les deux ans**. Cette observation, la **loi de Moore**, s'est vérifiée pendant un demi-siècle : de **2 300** transistors sur le premier microprocesseur (l'Intel 4004, en 1971) à **plusieurs dizaines de milliards** aujourd'hui. Ce n'est pas une loi physique mais une **tendance**, qui **ralentit** désormais : on approche de la taille de l'atome, en dessous de laquelle un interrupteur n'a plus de sens.

## II L'architecture de von Neumann

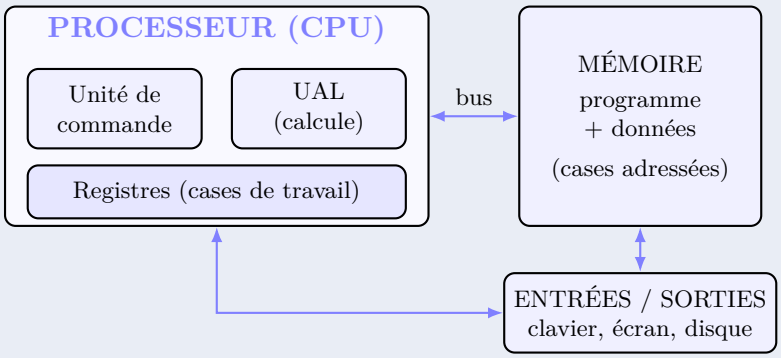
Comment organiser ces milliards d'interrupteurs pour **exécuter un programme** ? La réponse, proposée en **1945**, est le **modèle de von Neumann**. **Quasiment tous** les ordinateurs, du smartphone au super-calculateur, le suivent encore.

### Définition 2 — Les quatre unités du modèle de von Neumann

Une machine de von Neumann est faite de quatre grands organes :

- l'**unité de commande** : elle *lit* les instructions du programme, les *décode* et *ordonne* leur exécution (le « chef ») ;
- l'**unité arithmétique et logique** (**UAL**) : elle *calcule* (additions, comparaisons, ET/OU logiques…) ;
- la **mémoire** : elle *range* à la fois le **programme** et les **données**, dans des cases numérotées par une **adresse** ;
- les **entrées-sorties** (E/S) : elles font *communiquer* la machine avec le monde (clavier, écran, disque, réseau…).

L'unité de commande et l'UAL, réunies avec quelques cases de travail ultra-rapides, forment le **processeur** (ou **CPU**, *Central Processing Unit*).




Les organes sont reliés par des **bus** : des faisceaux de fils qui transportent les données, les adresses et les commandes.

### Règle 1 — L'idée géniale : le programme est une donnée

Dans le modèle de von Neumann, le **programme est rangé dans la même mémoire que les données**, sous forme de nombres. Un ordinateur ne « connaît » donc pas ses programmes à l'avance : il en **lit** un depuis la mémoire, exactement comme il lirait un nombre. C'est ce qui rend une machine **universelle** : changer de programme, c'est juste changer les nombres en mémoire — pas recâbler la machine.

**Remarque** — *Un ou plusieurs processeurs*
Le modèle de von Neumann décrit une machine **monoprocesseur** : un seul processeur exécute les instructions *l'une après l'autre*. Les ordinateurs et les smartphones actuels sont **multiprocesseurs** : leur puce réunit plusieurs **cœurs** (4, 8 ou davantage), chacun capable d'exécuter son propre programme. Plusieurs calculs avancent alors **réellement en même temps**, au lieu de se partager à tour de rôle un processeur unique ; mais un programme n'en profite que s'il est découpé en tâches indépendantes.

**À retenir — La hiérarchie des mémoires**

Toutes les mémoires ne se valent pas. Plus une mémoire est **rapide**, plus elle est **petite et chère** :

- les **registres**, dans le processeur : minuscules (quelques dizaines de cases) mais instantanés ;
- la **mémoire vive** (**RAM**) : quelques giga-octets, rapide, mais **volatile** (tout s'efface à l'extinction) ;
- le **disque** (SSD, disque dur) : énorme et **permanent**, mais bien plus lent.

Éteindre l'ordinateur sans enregistrer perd le travail resté en RAM : voilà pourquoi « enregistrer », c'est **recopier de la RAM vers le disque**.

## A. Les entrées-sorties : capteurs et actionneurs (capacité attendue)

L'unité d'**entrées-sorties** est la porte par laquelle la machine **échange avec le monde physique**. On distingue deux rôles complémentaires.

**Définition 3 — Capteur et actionneur**

Un **capteur** *mesure* une grandeur du monde physique et la transforme en une valeur numérique *entrant* dans la machine (c'est une **entrée**). Un **actionneur** fait l'inverse : il reçoit un ordre numérique et *agit* sur le monde physique (c'est une **sortie**).




*La chaîne capteur → programme → actionneur, au cœur de tout objet connecté.*

**Exemple** — *Un thermostat*

Un radiateur connecté *lit* la température de la pièce (**capteur**), la compare à la consigne (**programme**), puis *allume ou coupe* le chauffage (**actionneur**). De même, un smartphone regorge de capteurs (écran tactile, accéléromètre, luminosité, GPS) et d'actionneurs (haut-parleur, vibreur, dalle de l'écran).

**Remarque**

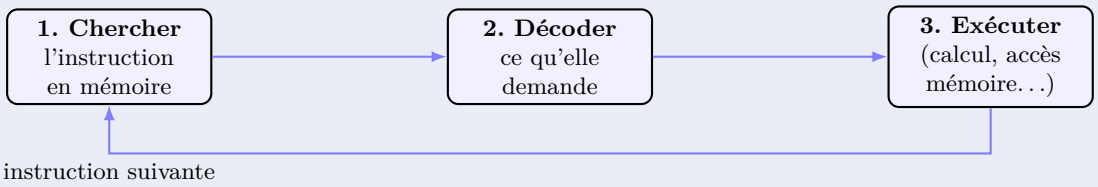
Clavier, souris et écran sont les entrées-sorties les plus familières ; mais dès qu'un programme pilote un **objet** (robot, domotique, voiture), c'est toujours la même boucle : **capter** le monde, **décider**, **agir**. On y reviendra avec les *objets connectés*.

► **Exercices d'application :** exercices 1 à 6 (transistor, von Neumann, mémoires, capteurs/actionneurs)

# III Le processeur au travail : le langage machine

## A. Le cycle d'exécution

Le processeur ne fait qu'**une** chose, mais des milliards de fois par seconde : répéter le **cycle** suivant.




À chaque tour, l'unité de commande utilise un registre spécial, le **compteur de programme** (ou *compteur ordinal*), qui contient l'**adresse de la prochaine instruction**. Normalement il avance d'un cran à chaque cycle ; une instruction de **saut** peut le forcer à une autre adresse — c'est ainsi qu'on réalise les **tests** et les **boucles**.

## B. Langage machine et assembleur

**Définition 4 — Langage machine, assembleur**

Le processeur n'obéit qu'à un jeu réduit d'instructions très simples (charger une case, additionner, comparer, sauter…), codées en **binaire** : c'est le **langage machine**. Illisible pour un humain, on l'écrit avec des **abréviations** : c'est l'**assembleur**. Une instruction assembleur = une instruction machine, juste plus lisible.

**Remarque** — *La grande pyramide des langages*

Notre `a = b + c` en Python ne « tourne » pas tel quel : un autre programme le **traduit** en une suite d'instructions machine. Plus on descend, plus c'est proche de la machine et loin de l'humain :

$$\text{Python (très lisible)} \longrightarrow \text{assembleur} \longrightarrow \text{langage machine (des 0 et des 1).}$$

C'est encore le mille-feuille : chaque niveau cache la complexité du niveau du dessous.

## C. Une machine-modèle : le *Little Man Computer*

Pour *dérouler* un vrai programme machine sans se noyer, on utilise une machine-modèle classique, le **Little Man Computer** (LMC). Elle est **von Neumann** en miniature :

- une **mémoire** de 100 cases numérotées de `00` à `99` (qui contiennent **indistinctement** programme *et* données) ;
- un seul registre de calcul, l'**accumulateur** (noté **ACC**) ;
- un **compteur de programme** qui pointe la prochaine instruction.

| Instr. | Effet | En français |
|---|---|---|
| `INP` | lit une entrée dans ACC | « demande un nombre » |
| `OUT` | affiche ACC | « affiche le résultat » |
| `LDA n` | ACC ← contenu de la case `n` | « charge » (*LoaD*) |
| `STA n` | case `n` ← ACC | « range » (*STore*) |
| `ADD n` | ACC ← ACC + contenu de la case `n` | « ajoute » |
| `SUB n` | ACC ← ACC − contenu de la case `n` | « soustrais » |
| `BRA n` | saute à l'instruction en case `n` | saut *toujours* |
| `BRZ n` | saute à `n` *si* ACC $= 0$ | saut *si zéro* |
| `BRP n` | saute à `n` *si* ACC $\geq 0$ | saut *si positif* |
| `HLT` | arrête la machine | stop |
| `DAT` | réserve une case pour une donnée | déclare une variable |

**Exemple** — *Dérouler un programme (capacité attendue)*

Voici un programme LMC qui lit deux nombres et affiche leur **somme**. À gauche l'**adresse** de la case, à droite l'instruction qui y est rangée. `a` et `b` sont des étiquettes désignant les cases de données `08` et `09`.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
00   INP          lire le 1er nombre
01   STA a        le ranger dans a
02   INP          lire le 2e nombre
03   STA b        le ranger dans b
04   LDA a        ACC <- a
05   ADD b        ACC <- a + b
06   OUT          afficher
07   HLT
08   a  DAT
09   b  DAT
```

**Déroulé pour les entrées 5 puis 3** (on suit l'accumulateur et les cases) :

| Adr. | Instr. | ACC | a | b |
|---|---|---|---|---|
| 00 | INP | 5 | – | – |
| 01 | STA a | 5 | 5 | – |
| 02 | INP | 3 | 5 | – |
| 03 | STA b | 3 | 5 | 3 |
| 04 | LDA a | 5 | 5 | 3 |
| 05 | ADD b | 8 | 5 | 3 |
| 06 | OUT | 8 | 5 | 3 |

La sortie est **8**. *Dérouler*, c'est exactement cela : suivre ligne à ligne l'état de l'accumulateur et des cases.

**Remarque** — *Deux compléments d'écriture*

`un DAT 1` réserve une case nommée `un` et y range d'avance la valeur `1` (sans valeur, la case vaut `0`). Une étiquette (comme `a` et `b` ci-dessus) peut aussi être placée devant une **instruction** : `BRA boucle` saute alors à l'instruction marquée `boucle`, sans avoir à connaître son adresse.

**Remarque** — *Sur machine*

Des **simulateurs en ligne**, comme `peterhigginson.co.uk/lmc/`, exécutent ce processeur pas à pas dans le navigateur : on **voit** l'accumulateur, le compteur de programme et les cases changer. Écrire et tester ses propres programmes LMC (tests, boucles, multiplication…) est l'objet de la partie A du **TP** proposé en fin de chapitre.

► **Exercices d'application :** exercices 7 à 11 (le langage machine, le LMC)

## IV Le système d'exploitation : le chef d'orchestre

Écrire chaque programme en langage machine, en gérant soi-même la mémoire, le disque et l'écran, serait un cauchemar. Une couche s'en charge à notre place : le **système d'exploitation**.

**Définition 5 — Système d'exploitation (OS)**

Un **système d'exploitation** (*Operating System*, OS) est l'ensemble de programmes qui **gèrent le matériel** et le **mettent à disposition** des applications et de l'utilisateur. Il joue les **intermédiaires** entre le matériel (compliqué, varié) et les logiciels (qui veulent juste « lire un fichier » ou « afficher une fenêtre »).

**Règle 2 — Les grandes fonctions d'un OS**

- **gérer le processeur** : décider quel programme s'exécute, et quand (plusieurs semblent tourner « en même temps ») ;
- **gérer la mémoire** : attribuer à chaque programme sa part de RAM, empêcher qu'ils s'écrasent l'un l'autre ;
- **gérer les fichiers** : organiser le disque en dossiers et fichiers ;
- **gérer les périphériques** : parler au clavier, à l'écran, à l'imprimante, au réseau, via des *pilotes* ;
- **gérer les utilisateurs** : comptes, mots de passe, **droits d'accès**.

Toujours le même mot-clé : **abstraction**. L'OS offre des actions simples (« ouvrir un fichier ») et cache la complexité du matériel dessous.

**À retenir — Les grandes familles de systèmes d'exploitation**

Trois lignées se partagent le monde : **Windows** (Microsoft), **macOS** (Apple) et surtout la famille **UNIX/Linux**. Cette dernière, **invisible** au grand public, est pourtant la plus répandue : elle fait tourner l'immense majorité des **serveurs** du Web, la plupart des **smartphones** (Android est bâti sur un noyau Linux) et les objets connectés. Windows et macOS sont des logiciels **propriétaires** (code source fermé) ; Linux est **libre** : chacun peut l'étudier, le modifier et le partager. *Attention* : « libre » ne veut pas dire « gratuit » — l'anglais *free* entretient la confusion.

► **Exercices d'application :** exercices 12 à 14 (le système d'exploitation)

## V Parler à l'OS en Linux : la ligne de commande

L'interface graphique (fenêtres, souris) est confortable, mais la **ligne de commande** (le **terminal**) reste indispensable : plus rapide, automatisable, et incontournable sur les serveurs (souvent sans écran). On travaille ici sous **Linux** : pour essayer les commandes, il suffit d'un terminal Linux en ligne, comme **JSLinux** (`bellard.org/jslinux`), déjà utilisé dans l'activité de découverte (on y est l'utilisateur `root`, de répertoire personnel `/root`).

**À retenir — L'invite de commande**

Le terminal affiche une **invite** qui rappelle qui vous êtes et où vous êtes, puis attend votre commande :

```
eleve@pc-nsi:~/Documents$ ls
```

Ici l'utilisateur est `eleve`, la machine `pc-nsi`, et le répertoire courant est `~/Documents`. Le **tilde** `~` désigne votre **répertoire personnel** (`/home/eleve`).

↻ **Retour sur l'activité.** L'invite `localhost:~#` de l'activité « Perdu dans le terminal » se lisait de la même façon : la machine s'appelait `localhost`, et l'on se trouvait dans son répertoire personnel `~`, c'est-à-dire `/root`, puisqu'on y était l'administrateur `root`.

## A. L'arborescence des fichiers

Sous Linux, les fichiers sont organisés en **arbre**, dont la base est la **racine**, notée `/`.

```
/                          la racine
|-- home/
|   |-- eleve/             votre repertoire personnel  (~)
|       |-- Documents/
|       |   |-- cours.txt
|       |   |-- tp_os.txt
|       |-- Images/
|           |-- logo.png
|-- etc/
    |-- hosts
```

**Définition 6 — Chemin absolu, chemin relatif**

Un **chemin** désigne l'emplacement d'un fichier.

- un chemin **absolu** part de la racine et commence donc par `/` : `/home/eleve/Documents/cours.txt` ;
- un chemin **relatif** part du **répertoire courant** et ne commence *pas* par `/`.

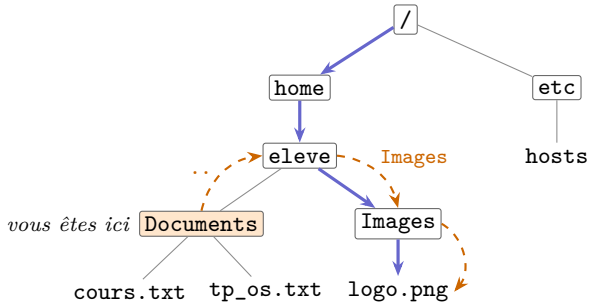
Deux raccourcis universels : `.` désigne le répertoire **courant**, et `..` le répertoire **parent** (un cran au-dessus).

↻ **Retour sur l'activité.** Depuis `/root`, l'activité faisait atteindre `bin` de deux façons : `cd /bin` suivait un chemin absolu, parti de la racine `/` ; `cd ../bin` un chemin relatif, qui remontait d'abord d'un cran avec `..`

**Exemple** — *Se repérer*

Depuis `/home/eleve/Documents`, pour atteindre `logo.png` :

- chemin absolu : `/home/eleve/Images/logo.png` (**trait plein** : on part de la racine) ;
- chemin relatif : `../Images/logo.png` (**tirets** : on remonte d'un cran avec `..`, puis on descend dans `Images`).




▷ **Je prédis.** Depuis `/home/eleve/Documents`, on tape `cd ../..` puis `pwd`. Qu'affiche `pwd` ?
▷ **Je vérifie.** `/home` : chaque `..` remonte d'un cran, de `Documents` à `eleve`, puis de `eleve` à `home`.

## B. Les commandes de base (capacité attendue)

| Commande | Rôle | Exemple |
|---|---|---|
| `pwd` | afficher le répertoire courant | `pwd` |
| `ls` | lister le contenu d'un répertoire | `ls Documents` |
| `ls -l` | lister en *détail* (droits, taille…) | `ls -l` |
| `cd` | changer de répertoire | `cd ../Images` |
| `mkdir` | créer un répertoire | `mkdir Projets` |
| `touch` | créer un fichier vide | `touch note.txt` |
| `cat` | afficher le contenu d'un fichier | `cat cours.txt` |
| `cp` | copier un fichier | `cp a.txt sauve/` |
| `mv` | déplacer ou renommer | `mv a.txt b.txt` |
| `rm` | supprimer un fichier | `rm note.txt` |
| `rm -r` | supprimer un répertoire et son contenu | `rm -r Projets` |
| `man` | afficher le manuel d'une commande | `man ls` |

Ces commandes se combinent : par exemple, depuis `~/Documents`, `mkdir archives` puis `cp cours.txt archives/` rangent une copie de `cours.txt` dans un nouveau dossier `archives`. Les manipulations (construire une arborescence, copier, déplacer, supprimer, tester les droits avec deux utilisateurs) se font dans la partie B du **TP** proposé en fin de chapitre.

**Remarque** — *Deux réflexes de survie*
**(1) `rm` ne pardonne pas** : il supprime **définitivement**, sans corbeille. Un `rm -r` lancé au mauvais endroit peut effacer tout un dossier. On réfléchit avant de valider (option `rm -i` pour demander confirmation).
**(2) Linux est sensible à la casse** : `Documents` et `documents` sont **deux** répertoires différents. La touche **Tab** complète automatiquement les noms : on l'utilise sans modération, elle évite les fautes de frappe.

► **Exercices d'application :** exercices 15 à 19 et 27 (la ligne de commande Linux)

## VI Droits et permissions

Linux est **multi-utilisateur** : plusieurs comptes partagent la machine, chacun avec ses fichiers. Il faut donc contrôler **qui** a le droit de faire **quoi**. Un compte particulier, l'**administrateur** (`root`), a **tous** les droits.

### Règle 3 — Trois droits, trois catégories

Chaque fichier ou répertoire porte **trois droits** :

`r` = lecture (*read*), `w` = écriture (*write*), `x` = exécution (*execute*),

accordés séparément à **trois catégories** d'utilisateurs :

`u` = le propriétaire (*user*), `g` = le groupe, `o` = les autres (*others*).

**Exemple** — *Lire une ligne de* `ls -l`

```
eleve@pc-nsi:~$ ls -l
-rw-r--r--  1 eleve  eleve   58  8 sep 09:12 cours.txt
drwxr-xr-x  2 eleve  eleve 4096  8 sep 09:14 Projets
```

Décryptons la première chaîne `-rw-r--r--`, lue **par groupes de trois** après le premier caractère :

| type | propriétaire (u) | groupe (g) | autres (o) |
|---|---|---|---|
| `-` (fichier) | `rw-` | `r--` | `r--` |

*Traduction* : c'est un fichier ; le propriétaire peut le **lire et écrire** (`rw-`), le groupe et les autres peuvent seulement le **lire** (`r--`). Le `d` initial de la 2e ligne indique un **répertoire** ; pour un dossier, `x` signifie « on peut le **traverser** ».

### Règle 4 — Modifier les droits : `chmod`

La commande `chmod` change les permissions, avec la grammaire `chmod [ugoa][+-=][rwx] fichier` :

```
chmod o-r cours.txt      retire aux autres le droit de lecture
chmod g+w cours.txt      donne au groupe le droit d'ecriture
chmod u+x script.sh      rend le fichier executable par son proprietaire
```

(`a` = « tout le monde » = `u`, `g` et `o` à la fois ; `=` réinitialise.)

### Règle 5 — Les droits en chiffres

Chaque droit est **présent ou absent** : c'est un **bit**. On donne à chaque droit une valeur, `r` = 4, `w` = 2, `x` = 1, et chaque catégorie s'écrit avec **un chiffre**, la somme de ses droits présents (de 0 à 7). Trois chiffres, dans l'ordre **propriétaire, groupe, autres**, décrivent donc tous les droits d'un fichier ; `chmod` les accepte et fixe alors **tous** les droits d'un coup.

**Exemple** — `chmod 754 fichier`

| | propriétaire (u) | groupe (g) | autres (o) |
|---|---|---|---|
| chiffre | $7 = 4+2+1$ | $5 = 4+1$ | 4 |
| en binaire | 111 | 101 | 100 |
| droits | `rwx` | `r-x` | `r--` |

Après `chmod 754 fichier`, `ls -l` affiche `-rwxr-xr--`. On reconnaît l'écriture binaire du chapitre **Le binaire et l'écriture des nombres** : `r`, `w`, `x` sont les bits de poids 4, 2 et 1, donc un chiffre ne dépasse jamais $111_2 = 7$. Codes courants : `755` (`rwxr-xr-x`, un script que tous peuvent lancer), `644` (`rw-r--r--`, un document lisible par tous), `700` (`rwx------`, privé).

▷ **Je prédis.** Le fichier `cours.txt` est en `-rw-r--r--`. Qu'affiche `ls -l` pour lui après `chmod 640 cours.txt` ? Quelle commande `chmod` avec des lettres aurait le même effet ?
▷ **Je vérifie.** `-rw-r-----` : $6 = 4+2$ donne `rw-`, 4 donne `r--` et 0 donne `---`. Seuls les autres ont perdu un droit, la lecture : c'est l'effet de `chmod o-r cours.txt`.

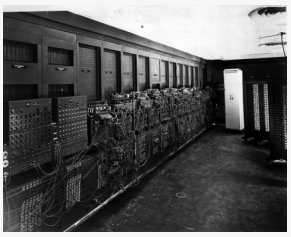
**Remarque** — *Pourquoi c'est un enjeu de sécurité*
Les droits ne sont pas un détail d'administration : ils **protègent** vos données des autres comptes, et limitent les dégâts qu'un programme malveillant peut causer. Un fichier de mots de passe lisible par « les autres », c'est une fuite ; un fichier téléchargé rendu `x` sans réfléchir, c'est un risque. `root`, qui a tous les droits, est aussi le compte le plus dangereux : on ne l'utilise que lorsque c'est nécessaire.

► **Exercices d'application :** exercices 20 à 23 et 26 (droits et permissions)



## VII Un peu d'histoire

**Pour les curieux** — *De l'ampoule au smartphone : la grande accélération* (facultatif)



*L'ENIAC et ses armoires de tubes à vide*

**1945** — Dans un rapport resté célèbre (le *First Draft of a Report on the EDVAC*), le mathématicien **John von Neumann** (1903–1957) décrit une machine où **programme et données partagent la même mémoire**. L'idée n'est pas de lui seul : elle doit beaucoup à **Alan Turing** (sa « machine universelle » de 1936) et aux ingénieurs **Eckert** et **Mauchly**, bâtisseurs de l'ENIAC. Mais c'est ce rapport qui diffuse le modèle — au point qu'il porte, un peu injustement, son seul nom.

**1947** — Aux **Bell Labs**, **Bardeen**, **Brattain** et **Shockley** inventent le **transistor** (prix Nobel 1956). Fini les tubes à vide fragiles et brûlants des premiers ordinateurs — l'**ENIAC** (1945), premier grand calculateur électronique, en alignait près de 18 000 dans une salle entière, pour 30 tonnes — : la brique moderne est née.

**1965** — **Gordon Moore** énonce sa loi. La miniaturisation s'emballe : l'Intel 4004 (1971) tient 2 300 transistors ; une puce actuelle en aligne des dizaines de milliards.

**1969** — Aux Bell Labs encore, **Ken Thompson** et **Dennis Ritchie** créent **UNIX**, un système élégant et portable qui inspirera presque tous les suivants (et donnera naissance au langage C).

**1991** — Un étudiant finlandais, **Linus Torvalds**, écrit « pour le plaisir » un noyau libre inspiré d'UNIX : **Linux**. Trois décennies plus tard, ce projet d'étudiant fait tourner le Web, Android et les super-calculateurs. *Le mille-feuille de couches sur lequel vous lisez cette page a*

*mis moins d'un siècle à se construire.*

**Pour les curieux** — *Margaret Hamilton et l'ordinateur d'Apollo 11* (facultatif)

Dans les années 1960, au laboratoire d'instrumentation du MIT, l'Américaine **Margaret Hamilton** dirige l'équipe qui écrit le **logiciel de vol** des missions Apollo. L'ordinateur de bord, l'*Apollo Guidance Computer*, est minuscule à nos yeux : **2 048 mots** de mémoire vive (16 bits chacun, soit 4 ko), **36 864 mots** de mémoire morte, littéralement *tissée* à la main avec des fils passant ou non dans de petits anneaux magnétiques, et une horloge à **2 MHz**, environ mille fois moins qu'un processeur actuel.

Le **20 juillet 1969**, pendant la descente vers la Lune, l'écran affiche les alarmes `1202` puis `1201` : un radar inutile à cet instant envoie des données en continu, et le processeur a **plus de travail qu'il ne peut en faire**. Le logiciel avait été conçu pour résister à une telle surcharge : il partage le temps du processeur entre les tâches **par ordre de priorité**, abandonne les moins urgentes et garde le guidage du module lunaire. L'alunissage se poursuit. C'est exactement le premier rôle d'un système d'exploitation vu dans ce chapitre : **gérer le processeur**. En 2016, Margaret Hamilton reçoit la médaille présidentielle de la Liberté pour ce logiciel.

*Sources :* F. O'Brien, *The Apollo Guidance Computer : Architecture and Operation*, Springer, 2010 ; NASA, « Margaret Hamilton », nasa.gov.

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** Les 0 et les 1 du chapitre *Le binaire et l'écriture des nombres* sont les états des transistors ; les sauts `BRZ` et `BRP` du *Little Man Computer* réalisent les `if` et les `while` des *Bases de la programmation Python* ; la traduction en langage machine distingue langages compilés et interprétés (*Spécifier et mettre au point ses programmes*).
- **Ce qu'il prépare.** Les serveurs du chapitre suivant, *Le Web : réseaux et interactions homme-machine*, seront le plus souvent des machines Linux, administrées en ligne de commande ; le système d'exploitation y découpera les données en **paquets** et les **encapsulera**, couche après couche, avant de les confier à la carte réseau.
- **En Terminale.** *Processus et systèmes d'exploitation* détaillera le partage du processeur entre les programmes en cours, *Systèmes sur puce* réunira processeur, mémoire et entrées-sorties sur une seule puce.



## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Transistor | interrupteur commandé → 0/1 ; brique de toutes les portes logiques |
| Ordres de grandeur | transistor ∼ nm ; milliards par puce ; fréquence ∼ GHz ; loi de Moore |
| von Neumann | unité de commande + UAL (= CPU) + mémoire + entrées-sorties |
| Idée clé | **programme et données dans la même mémoire** → machine universelle |
| Mémoires | registres < RAM (volatile) < disque (permanent) : vitesse ↓, taille ↑ |
| Cycle CPU | chercher → décoder → exécuter, en boucle |
| Langage machine | instructions simples en binaire ; assembleur = version lisible |
| Dérouler | suivre l'accumulateur et les cases, instruction par instruction |
| Entrées-sorties | **capteur** = mesure (entrée) ; **actionneur** = agit (sortie) ; boucle capter/décider/agir |
| Système d'exploitation | gère processeur, mémoire, fichiers, périphériques, utilisateurs |
| Ligne de commande | `pwd ls cd mkdir touch cat cp mv rm man` ; `.` courant, `..` parent |
| Chemins | absolu (commence par `/`) vs relatif (part du répertoire courant) |
| Droits | `rwx` pour `u/g/o` ; lire `ls -l` ; modifier avec `chmod` ; en chiffres `r`= 4, `w`= 2, `x`= 1 (`754`) |

### Retour à la situation de départ

1. Dans la **mémoire**, avec les données : c'est le modèle de **von Neumann** (§ II). L'**unité de commande** du processeur le lit instruction par instruction, en répétant le cycle chercher → décoder → exécuter (§ III).
2. C'est le travail du **système d'exploitation** : il décide quel programme utilise le processeur, et quand, et attribue à chacun sa part de mémoire sans qu'ils s'écrasent (§ IV).
3. Les **droits d'accès** : chaque fichier porte des droits `r`, `w`, `x` pour le propriétaire, le groupe et les autres ; `chmod o-r` retire aux autres le droit de lecture. Seul l'administrateur `root` garde tous les droits (§ VI).

## Erreurs fréquentes

- **Confondre mémoire vive et disque.** La RAM est *volatile* : ce qui n'est pas enregistré sur le disque est perdu à l'extinction.
- **Croire que le processeur « comprend » Python.** Il n'exécute que du *langage machine* ; l'interpréteur Python traduit le programme, au fur et à mesure, en instructions machine.
- **Oublier le `/` initial** d'un chemin absolu, ou en mettre un à un chemin relatif.
- **Se tromper de casse** : `Documents` ≠ `documents` sous Linux.
- **Lancer `rm -r` sans réfléchir** : suppression *définitive*, sans corbeille.
- **Lire les droits d'un bloc** : ils se lisent *par groupes de trois* (`u`, puis `g`, puis `o`).

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

☐ nommer les composants du modèle de von Neumann et leur rôle ;

☐ **distinguer** un **capteur** (entrée : mesure) d'un **actionneur** (sortie : action) et citer des exemples ;

☐ citer des ordres de grandeur (taille, nombre de transistors, fréquence) et énoncer la loi de Moore ;

☐ **dérouler** une petite séquence d'instructions en langage machine (accumulateur et cases) ;

☐ expliquer ce qu'est un système d'exploitation et citer ses grandes fonctions ;

☐ me **repérer** dans une arborescence (chemins absolu et relatif, `.` et `..`) ;

☐ **utiliser** les commandes de base en ligne de commande sous Linux ;

☐ **lire** une ligne de `ls -l` et **modifier** des droits avec `chmod`, en lettres (`g+w`) ou en chiffres (`754`).

# EXERCICES

**Mode d'emploi**

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À taper et tester* sur un simulateur en ligne (`peterhigginson.co.uk/lmc/`).
- Rappel du **Little Man Computer** : mémoire de cases `00`–`99`, un accumulateur **ACC**, instructions `INP OUT LDA STA ADD SUB BRA BRZ BRP HLT DAT`.
- Les commandes Linux sont à essayer dans un **terminal Linux en ligne**, comme JSLinux (`bellard.org/jslinux`), utilisé dans l'activité de découverte.

Sauf mention contraire, les exercices de ligne de commande utilisent l'**arborescence de référence** suivante, et vous démarrez depuis `/home/eleve`.

```
/
|-- home/
|   |-- eleve/                ~  = repertoire de depart
|       |-- Documents/
|       |   |-- cours_nsi.txt
|       |   |-- tp_os.txt
|       |-- Images/
|       |   |-- logo.png
|       |-- Projets/          vide au depart
|       |-- notes.txt
|-- etc/
    |-- hosts
```

## Architecture matérielle et modèle de von Neumann ↑cours

★ **Exercice 1** IA *Chaque chose à sa place* ▶ corrigé p. 656

Associer chaque composant à son rôle.

| | |
|---|---|
| (1) unité de commande | (a) fait les calculs (additions, comparaisons…) |
| (2) UAL | (b) range programme et données dans des cases adressées |
| (3) mémoire | (c) lit, décode et ordonne l'exécution des instructions |
| (4) entrées-sorties | (d) font communiquer la machine avec le monde extérieur |

▷ coup de pouce

★ **Exercice 2** IA *Vrai ou faux* ▶ corrigé p. 656 • essentiel

Pour chaque affirmation, répondre par vrai ou faux et **justifier** en une phrase.

a) Le processeur (CPU) contient l'unité de commande et l'UAL.
b) La mémoire vive (RAM) conserve les données même quand l'ordinateur est éteint.
c) Dans le modèle de von Neumann, le programme est rangé dans la mémoire, comme les données.
d) Un registre est plus grand mais plus lent que le disque dur. ▷ coup de pouce

★ **Exercice 3** *Ordres de grandeur* ▶ corrigé p. 656

Recopier chaque grandeur en la reliant à l'ordre de grandeur qui convient : *quelques nanomètres à quelques dizaines de nanomètres* / *des milliards à des dizaines de milliards* / *quelques gigahertz.*

a) taille d'un transistor gravé aujourd'hui ;
b) nombre de transistors dans un processeur moderne ;
c) fréquence (nombre de cycles par seconde) d'un processeur.

▷ coup de pouce

★★ **Exercice 4** *La loi de Moore* ▶ corrigé p. 657

L'Intel 4004 (1971) comptait environ **2 300** transistors. La loi de Moore affirme que ce nombre **double tous les deux ans**.

a) Combien de transistors, environ, la loi prévoit-elle après 10 ans (soit 5 doublements) ?
b) La loi de Moore est-elle une loi de la physique ? Pourquoi finit-elle par ralentir ?

▷ coup de pouce

★★ **Exercice 5** *RAM ou disque ?* ▶ corrigé p. 657 • essentiel

On enregistre un document puis on éteint brutalement l'ordinateur (coupure de courant).

a) Le travail *enregistré* est-il perdu ? Où se trouve-t-il ?
b) Le travail *non enregistré* est-il perdu ? Dans quelle mémoire se trouvait-il ?
c) Expliquer, avec les mots « volatile » et « permanent », ce que fait le bouton « Enregistrer ».

▷ coup de pouce

★ **Exercice 6** *Capteur ou actionneur ?* ▶ corrigé p. 657 • essentiel

Pour chaque élément, dire s'il s'agit d'un **capteur** (entrée : il *mesure*) ou d'un **actionneur** (sortie : il *agit*) : *a)* un thermomètre ; *b)* un moteur de volet roulant ; *c)* un micro ; *d)* une LED ; *e)* l'écran tactile (les deux ? préciser) ; *f)* un haut-parleur.

Puis décrire, pour un **arrosage automatique de plante**, la boucle *capter* → *décider* → *agir* en nommant le capteur et l'actionneur.

▷ coup de pouce

## Le processeur au travail : langage machine (LMC) ↑ cours

★★ **Exercice 7** *Dérouler pas à pas* ▶ corrigé p. 657 • essentiel

On donne le programme LMC suivant.

```
00   INP
01   STA 08
02   ADD 08
03   ADD 08
04   OUT
05   HLT
08       DAT
```

a) Dérouler l'exécution pour l'entrée **4**, en remplissant un tableau à colonnes `Adresse`, `Instruction`, `ACC`, `case 08`.
b) Quelle valeur est affichée par `OUT` ?
c) En une phrase : que calcule ce programme à partir du nombre lu ?

▷ coup de pouce

★★ **Exercice 8** *Le plus grand des deux* ▶ corrigé p. 657

On donne ce programme (les cases `11` et `12` sont nommées `a` et `b`).

```
00   INP
01   STA a
02   INP
03   STA b
04   SUB a        ACC <- b - a
05   BRP 08       si ACC >= 0, aller en 08
06   LDA a
07   BRA 09
08   LDA b
09   OUT
10   HLT
11   a  DAT
12   b  DAT
```

a) Dérouler pour les entrées **7** puis **2**. Quelle est la sortie ?
b) Dérouler pour les entrées **3** puis **8**. Quelle est la sortie ?
c) Que calcule ce programme ? À quoi sert l'instruction `BRP` ? ▷ coup de pouce

★★ **Exercice 9** ▶ *À vous d'écrire* ▶ corrigé p. 657

Écrire un programme LMC qui lit **trois** nombres et affiche leur **somme**. Le tester sur le simulateur avec les entrées 10, 20, 30 (on doit lire 60). ▷ coup de pouce

★★★ **Exercice 10** ▶ *Le plus petit* ▶ corrigé p. 658 ≈ 15 min

En s'inspirant du programme « le plus grand des deux », écrire un programme LMC qui lit deux nombres et affiche le **plus petit**. Le tester (entrées 7 et 2 → 2 ; entrées 3 et 8 → 3). ▷ coup de pouce ▷ début de solution

★★★ **Exercice 11** ▶ *Une boucle : la somme des entiers* ▶ corrigé p. 658 ≈ 20 min

On donne ce programme utilisant des **étiquettes** (le simulateur les accepte).

```
        INP
        STA n
        LDA zero
        STA somme
        LDA n
        STA i
boucle  LDA i
        BRZ fin          si i = 0, on sort
        LDA somme
        ADD i
        STA somme        somme <- somme + i
        LDA i
        SUB un
        STA i            i <- i - 1
        BRA boucle
fin     LDA somme
        OUT
        HLT
zero    DAT 0
un      DAT 1
n       DAT
i       DAT
somme   DAT
```

a) Dérouler pour l'entrée **3** en suivant `i` et `somme` à chaque passage de `boucle`. Quelle est la sortie ?
b) Que calcule ce programme en fonction de l'entrée $n$ ?
c) Quel rôle jouent `BRZ` et `BRA` pour réaliser la boucle ? ▷ coup de pouce ▷ début de solution

## Le système d'exploitation ↑ cours

★ **Exercice 12** *À quoi sert l'OS ?* ▶ corrigé p. 658 • essentiel

Pour chacune des situations, indiquer **quelle grande fonction** du système d'exploitation est en jeu (gérer le processeur / la mémoire / les fichiers / les périphériques / les utilisateurs).

a) deux applications semblent s'exécuter « en même temps » ;
b) on branche une imprimante et elle fonctionne sans rien réinstaller ;
c) un fichier de Léa n'est pas accessible au compte de Tom ;
d) chaque programme dispose de sa part de RAM sans écraser les autres ;
e) on retrouve ses documents rangés en dossiers. ▷ coup de pouce

★ **Exercice 13** *Familles de systèmes* ▶ corrigé p. 658

Répondre brièvement.

a) Citer trois grandes familles de systèmes d'exploitation.
b) Sur quel noyau le système Android des smartphones est-il bâti ?
c) « Un logiciel libre est forcément gratuit. » Vrai ou faux ? Expliquer. ▷ coup de pouce



★★ **Exercice 14** *L'abstraction* ▶ corrigé p. 659

Quand une application veut « ouvrir un fichier », elle ne s'adresse pas directement au disque dur : elle passe par le système d'exploitation. En reliant cela au **mille-feuille de couches** du cours, expliquer en quelques lignes l'intérêt de cette *abstraction* (penser à ce qui se passerait si chaque application devait connaître le modèle exact du disque). ▷ coup de pouce

## Ligne de commande Linux ↑ cours

★ **Exercice 15** *Se repérer* ▶ corrigé p. 659 • essentiel

En utilisant l'arborescence de référence :

a) Quelle commande affiche le répertoire courant ?
b) Donner le **chemin absolu** du fichier `logo.png`.
c) Vous êtes dans `/home/eleve/Documents`. Donner le chemin **relatif** pour atteindre `logo.png`.
▷ coup de pouce

★★ **Exercice 16** *Naviguer* ▶ corrigé p. 659

Vous démarrez dans `/home/eleve`. Écrire la commande demandée (chemin relatif sauf mention contraire).

a) se déplacer dans `Documents` ;
b) depuis `Documents`, revenir dans `/home/eleve` *sans taper le chemin complet* ;
c) aller directement dans `/etc` en **chemin absolu** ;
d) lister le contenu de `Images` *sans s'y déplacer*. ▷ coup de pouce

★★ **Exercice 17** *Créer, copier, déplacer* ▶ corrigé p. 659

Depuis `/home/eleve`, écrire les commandes pour :

a) créer un répertoire `Rendus` dans `Projets` ;
b) créer un fichier vide `brouillon.txt` dans `Projets/Rendus` ;
c) afficher le contenu de `Documents/cours_nsi.txt` ;
d) copier `tp_os.txt` de `Documents` vers `Projets/Rendus`, puis renommer la copie en `tp_final.txt` ;
e) déplacer `notes.txt` de `/home/eleve` vers `Documents`.

▷ coup de pouce

★★ **Exercice 18** *Corriger les erreurs* ▶ corrigé p. 659

Chaque commande contient une erreur. L'identifier et proposer la correction.

a) `cd /home/eleve/Documents/cours_nsi.txt` (on veut aller *dans* `Documents`) ;
b) `rm Images` (on veut supprimer le répertoire `Images` et son contenu) ;
c) vous êtes dans `/home/eleve/Images` et tapez `cat cours_nsi.txt` (le fichier est dans `Documents`).

▷ coup de pouce

★ **Exercice 19** *Reconstituer une séquence* ▶ corrigé p. 659

Remettre ces commandes dans le bon ordre pour : créer un dossier `TP`, y créer un fichier `index.txt`, puis afficher son contenu.

```
A. cat TP/index.txt     B. mkdir TP     C. touch TP/index.txt
```

▷ coup de pouce

## Droits et permissions ↑ cours

★★ **Exercice 20** *Lire les droits* ▶ corrigé p. 659 • essentiel

La commande `ls -l` affiche :

```
-rwxr-xr--  1 eleve  nsi   240  8 sep 10:02 script.sh
drwxr-x---  2 eleve  nsi  4096  8 sep 10:05 secret
```

a) Pour `script.sh` : s'agit-il d'un fichier ou d'un répertoire ? Quels droits a le **propriétaire** ? le **groupe** ? les **autres** ?
b) Un membre du groupe `nsi` peut-il *exécuter* `script.sh` ? Le *modifier* ?
c) Que signifie le `d` au début de la ligne de `secret` ? Les « autres » peuvent-ils entrer dans ce répertoire ?

▷ coup de pouce

★★ **Exercice 21** *Modifier les droits* ▶ corrigé p. 660

Écrire la commande `chmod` correspondante.

a) retirer aux « autres » le droit de lecture sur `cours_nsi.txt` ;
b) ajouter au « groupe » le droit d'écriture sur `cours_nsi.txt` ;
c) rendre `script.sh` exécutable par son propriétaire ;
d) donner à tout le monde le droit de lecture sur `logo.png`.

▷ coup de pouce

★★ **Exercice 22** *Les droits en chiffres* ▶ corrigé p. 660

a) Écrire en trois chiffres les droits `rwxr-xr-x`, `rw-r--r--`, `rwx------` et `r--r-----`.
b) Quels droits (en lettres) donnent `chmod 640`, `chmod 751` et `chmod 600` ?
c) Écrire la commande `chmod` en chiffres qui donne sur `script.sh` tous les droits au propriétaire, la lecture et l'exécution au groupe, et aucun droit aux autres.
d) Pourquoi chaque chiffre est-il compris entre 0 et 7 ? Combien de réglages différents des droits d'un fichier existe-t-il en tout ? ▷ coup de pouce

★★ **Exercice 23** *Le compte* `root` ▶ corrigé p. 660

a) Qui est l'utilisateur `root` et que peut-il faire de plus que les autres ?
b) Pourquoi est-il déconseillé de travailler en permanence en tant que `root` ?
c) En quoi les droits d'accès participent-ils à la **sécurité** d'une machine partagée ? ▷ coup de pouce

## Mini-problème de synthèse ↑ cours

★★★ **Exercice 24** *Du transistor au fichier* ▶ corrigé p. 660 ≈ 20 min

Léo, sur son environnement Linux, écrit un petit programme, l'enregistre, puis le rend exécutable.

a) Léo tape le code dans un éditeur : dans quelle mémoire se trouve son texte tant qu'il n'a pas enregistré ? Que risque une coupure de courant ?
b) Il enregistre dans `/home/leo/Projets/jeu.py`. Donner la commande qui, depuis `/home/leo`, affiche le contenu de ce fichier avec un **chemin relatif**.
c) Il ajoute le droit d'exécution pour lui-même (le propriétaire), sans toucher aux autres droits. Écrire la commande `chmod`.
d) Quand il lance son programme Python, celui-ci n'est pas exécuté tel quel par le processeur. Expliquer, avec les mots *traduction*, *langage machine* et *cycle chercher-décoder-exécuter*, ce qui se passe « en dessous ».

▷ coup de pouce ▷ début de solution

★★ **Exercice 25** *Le modèle de von Neumann selon un assistant d'IA* ▶ corrigé p. 660

Un élève demande à un assistant d'IA : « Explique-moi le modèle de von Neumann. » Voici la réponse obtenue.

« Le modèle de von Neumann, proposé en 1945, décrit l'organisation de presque tous les ordinateurs actuels. Il comporte quatre unités : l'unité de commande, qui lit et décode les instructions ; l'unité arithmétique et logique (UAL), qui effectue les calculs ; la mémoire, faite de cases numérotées par une adresse ; et les dispositifs d'entrée-sortie, qui font le lien avec l'extérieur (clavier, écran, disque). L'unité de commande et l'UAL, avec quelques registres très rapides, forment le processeur, relié aux autres organes par des bus. Le processeur répète sans cesse le cycle chercher-décoder-exécuter. Le point essentiel du modèle est que les instructions du programme sont rangées dans une mémoire séparée de celle des données, ce qui évite que le processeur confonde une instruction et une donnée. »

a) La réponse est-elle correcte ? Confronter chaque phrase au cours.
b) Localiser et corriger l'erreur.
c) Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ? ▷ coup de pouce

## Entraînement ↑cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

### ★ Exercice 26 *Lire et fixer des droits* ▶corrigé p. 661 • essentiel

1. Pour chacun des droits suivants d'un fichier `f`, dire ce que peuvent faire le propriétaire, le groupe et les autres, puis donner la commande `chmod` en chiffres qui les fixe : a) `rwxr-x---` ; b) `rw-rw-r--` ; c) `r--------` ; d) `rwxr-xr-x`.
2. Le fichier `notes.txt` a les droits `rw-r--r--`. Donner la commande `chmod` (avec des lettres) qui : a) retire aux autres le droit de lecture ; b) donne au groupe le droit d'écriture ; c) donne à tout le monde le droit d'exécution.

▷coup de pouce

### ★ Exercice 27 *Chemins absolus et relatifs* ▶corrigé p. 661 • essentiel

Avec l'arborescence de référence, donner le **chemin absolu** de chaque élément, puis son **chemin relatif** depuis le répertoire courant indiqué : a) `notes.txt`, depuis `/home/eleve/Documents` ; b) `hosts`, depuis `/home/eleve` ; c) `cours_nsi.txt`, depuis `/home/eleve/Images` ; d) `logo.png`, depuis la racine `/`.

▷coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

**Ex. 1 – piste** Revoir le schéma des quatre unités du modèle de von Neumann dans le cours : laquelle « commande », laquelle « calcule », laquelle « range », laquelle fait le lien avec l'extérieur ? ↑

**Ex. 2 q. 4 – piste** Pour b) : que devient la RAM quand on coupe le courant ? Pour d) : classer registre, RAM et disque de la mémoire la plus rapide (et la plus petite) à la plus lente. ↑énoncé

**Ex. 3 q. 3 – piste** Une grandeur en mètres, une en nombre d'objets, une en cycles par seconde (hertz) : chaque ordre de grandeur proposé n'a qu'une unité possible. ↑énoncé

**Ex. 4 q. 2 – piste** a) Doubler 5 fois, c'est multiplier par $2 \times 2 \times 2 \times 2 \times 2$. b) Comparer la taille d'un transistor actuel à celle d'un atome. ↑énoncé

**Ex. 5 q. 3 – piste** Pour chaque mémoire (RAM, disque), se demander ce qu'elle devient quand le courant est coupé. ↑énoncé

**Ex. 6 – piste** Pour chaque élément, se demander s'il fait *entrer* une information dans la machine (il mesure) ou s'il produit un effet dans le monde (il agit). Pour la plante : que faut-il mesurer, et quel appareil faut-il commander ? ↑énoncé

**Ex. 7 q. 3 – piste** Une ligne du tableau par instruction exécutée. `ADD 08` ajoute à ACC le contenu de la case 08, sans modifier cette case. ↑énoncé

**Ex. 8 q. 3 – piste** Calculer ACC juste après `SUB a` : c'est son signe qui décide si l'on saute en 08 ou non. ↑énoncé

**Ex. 9 – piste** Il n'y a qu'un accumulateur : chaque nombre lu doit être rangé dans une case (réservée par `DAT`) avant d'en lire un autre. ↑énoncé

**Ex. 10 – piste** Dans « le plus grand des deux », repérer quelle case est chargée quand ACC $\geqslant 0$ et quelle case l'est sinon : que faut-il changer ? ↑énoncé

**Ex. 10 – début de solution** Le début ne change pas :

```
00 INP
01 STA a
02 INP
03 STA b
04 SUB a (ACC ⩾ 0 signifie b ⩾ a).
```

↑énoncé

**Ex. 11 q. 3 – piste** Faire un tableau à deux colonnes `i` et `somme`, avec une ligne à chaque passage par l'étiquette `boucle`. ↑énoncé

Ex. 11 q. 3 – début de solution Au premier passage par `boucle` : `i` vaut 3 et `somme` vaut 0. Après le corps de la boucle, `somme` vaut 3 et `i` vaut 2, puis `BRA` renvoie à `boucle`. ↑énoncé

Ex. 12 q. 5 – piste Pour chaque situation, se demander quelle *ressource* est partagée ou organisée : le temps de calcul, la mémoire vive, le disque, un appareil branché, ou les comptes ? ↑

Ex. 13 q. 3 – piste Relire la partie du cours sur le système d'exploitation et sur le logiciel libre : le mot « libre » parle-t-il du prix ou de ce qu'on a le droit de faire avec le code ? ↑énoncé

Ex. 14 – piste Imaginer qu'on remplace le disque dur par un modèle d'une autre marque : qui faut-il modifier, chaque application ou seulement l'OS ? ↑énoncé

Ex. 15 q. 3 – piste Un chemin absolu part de la racine `/` ; un chemin relatif part du répertoire courant, et `..` fait remonter d'un cran dans l'arbre. ↑énoncé

Ex. 16 q. 4 – piste Revoir les raccourcis `.` et `..`, et la différence entre un chemin qui commence par `/` et un chemin qui n'en commence pas. ↑énoncé

Ex. 17 q. 5 – piste On reste dans `/home/eleve` : chaque commande peut viser un fichier plus bas dans l'arbre avec un chemin relatif comme `Projets/Rendus`. Revoir le tableau des commandes du cours. ↑énoncé

Ex. 18 q. 3 – piste Pour chaque commande, se demander : le dernier mot désigne-t-il un fichier ou un répertoire ? Et depuis quel répertoire la commande est-elle lancée ? ↑énoncé

Ex. 19 – piste Peut-on créer un fichier dans un dossier qui n'existe pas encore ? Peut-on afficher un fichier qui n'existe pas encore ? ↑énoncé

Ex. 20 q. 3 – piste Découper la chaîne : un caractère pour le type, puis trois blocs de trois caractères, dans l'ordre propriétaire, groupe, autres. ↑énoncé

Ex. 21 q. 4 – piste Une commande `chmod` se lit : *pour qui* (`u`, `g`, `o` ou `a`), *ajouter ou retirer* (`+` ou `-`), *quel droit* (`r`, `w`, `x`), puis le fichier. ↑énoncé

Ex. 22 q. 4 – piste Lire chaque catégorie comme un nombre binaire de 3 bits : `r`, `w`, `x` sont les bits de poids 4, 2 et 1 (`-` vaut 0). ↑énoncé

Ex. 23 q. 3 – piste Imaginer la même faute de frappe dans une commande `rm -r`, tapée par un utilisateur ordinaire puis par `root` : quels fichiers peuvent être effacés dans chaque cas ? ↑énoncé

Ex. 24 – piste Chaque question reprend une partie du cours : a) les mémoires, b) les chemins relatifs, c) `chmod`, d) le cycle du processeur. ↑énoncé

Ex. 24 – début de solution a) Avant l'enregistrement, le texte n'existe qu'en mémoire vive. b) Le chemin relatif part de `/home/leo` : il ne commence donc pas par `/`. d) Python est d'abord traduit ; le processeur répète ensuite son cycle sur des instructions… ↑énoncé

Ex. 25 q. 3 – piste Relire l'idée clé du modèle dans le cours : où sont rangés les instructions du programme et les données ? ↑énoncé

Ex. 26 – piste Trois blocs de trois caractères, dans l'ordre propriétaire, groupe, autres. En chiffres : `r` vaut 4, `w` vaut 2, `x` vaut 1, et l'on additionne dans chaque bloc. En lettres : pour qui (`u`, `g`, `o`, `a`), `+` ou `-`, quel droit. ↑énoncé

Ex. 27 – piste Le chemin absolu se lit de la racine jusqu'à l'élément. Pour le chemin relatif, remonter avec `..` jusqu'au premier répertoire commun, puis redescendre. ↑énoncé

# TP ET PROJETS

## TP Sous le capot ▶ corrigé p. 661

*programmer un processeur-jouet, puis piloter un vrai Linux en ligne de commande*

Durée : 2 à 3 h (une séance par partie) · En binôme, sur machine

### But du TP

Deux couches du « mille-feuille » du cours, à pratiquer sur machine. **Partie A** : écrire de vrais programmes pour le **Little Man Computer** (LMC), le processeur-modèle du cours, et les regarder s'exécuter instruction par instruction — calculs, tests, boucles. **Partie B** : dans un vrai Linux qui tourne dans le navigateur, construire une arborescence, manipuler des fichiers, puis mettre à l'épreuve les **droits d'accès** avec deux utilisateurs. Produits finaux : une collection de programmes LMC qui marchent, et un **tableau « qui peut faire quoi »** établi par l'expérience.

### Consignes

- Rien à installer. Gardez sous les yeux le jeu d'instructions LMC et le tableau des commandes du cours.
- Partie A : `peterhigginson.co.uk/lmc/`. On tape le programme dans la zone de gauche, on clique sur **Submit** (ou **Assemble into RAM**), puis **Run** (ou **Step** pour avancer d'une instruction). Quand la machine attend une entrée, on tape le nombre dans la case **Input** puis **Entrée**. Les boutons << et >> règlent la vitesse.
- Partie B : `bellard.org/jslinux/`, ligne **x86 – Alpine Linux 3.12.0 – Console** (comme dans l'activité « Perdu dans le terminal »). Recharger la page remet tout à zéro : notez vos commandes au fur et à mesure.
- ▶ *À faire sur machine.* Pour chaque programme écrit, recopier le code final et les tests effectués (sur le cahier ou dans un fichier texte). Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Partie A — Programmer le Little Man Computer

★ **Exercice 1** *Prise en main : la somme, vue de l'intérieur*▶

Taper le programme du cours (avec des étiquettes, le simulateur choisit lui-même les adresses), l'assembler, puis l'exécuter **pas à pas** avec les entrées 5 puis 3.

```
      INP
      STA a
      INP
      STA b
      LDA a
      ADD b
      OUT
      HLT
a     DAT
b     DAT
```

1. Après l'assemblage, quels nombres apparaissent dans les cases 00 à 09 de la mémoire (RAM) ? Les recopier.
2. En comparant au programme, retrouver le **code** de chaque instruction : `INP` → …, `STA` → 3…, `LDA` → …, `ADD` → …, `OUT` → …, `HLT` → … Que signifient les deux derniers chiffres ?
3. Pendant l'exécution pas à pas, observer le **compteur de programme** : de combien avance-t-il à chaque instruction ? Quelle est sa valeur quand la machine s'arrête ?
4. Quelle idée du modèle de von Neumann cette mémoire illustre-t-elle ?

★ **Exercice 2** *Le programme mystère*

Voici le contenu des cases 00 à 09 d'une mémoire, **sans** le programme assembleur : `901 309 901 109 309 109 902 000 000 000`.

On sait aussi que `SUB` a pour code 2…, `BRA` 6…, `BRZ` 7… et `BRP` 8….

1. « Désassembler » : réécrire ce programme en assembleur, une instruction par case de 00 à 07 (les cases 08 et 09 sont des données, à 000 au départ).
2. Dérouler à la main pour les entrées 5 puis 3. Que calcule ce programme ?
3. ▶ Vérifier de deux façons : taper votre version assembleur dans le simulateur et contrôler qu'après l'assemblage la RAM contient exactement les nombres de l'énoncé ; puis l'exécuter avec 5 et 3.

★★ **Exercice 3** *L'écart entre deux nombres*▶

Écrire un programme qui lit deux nombres $a$ et $b$ et affiche leur **écart** (le plus grand moins le plus petit) : 2 et 7 donnent 5 ; 7 et 2 donnent 5 aussi. Le tester sur ces deux cas, puis avec deux nombres égaux.

▷ coup de pouce

★★ **Exercice 4** *Le compte à rebours*▶

Écrire un programme qui lit un nombre $n$ puis affiche $n$, $n-1$, …, 1, 0 et s'arrête. Le tester avec 3 (sorties : 3, 2, 1, 0), puis avec 0.

1. Quelle instruction permet de **revenir en arrière** dans le programme ? Laquelle permet d'en **sortir** ?
2. Il faut retirer 1 à chaque tour : où trouver ce 1 ? (Indice : `DAT` peut recevoir une valeur initiale, par exemple `un DAT 1`.)
3. Que se passerait-il si l'on oubliait le test d'arrêt ? *Ne pas essayer longtemps : bouton* ***Stop***.

▷ coup de pouce

★★ **Exercice 5** *La multiplication sans multiplier*▶

Le LMC ne sait pas multiplier. Mais $a \times b$, c'est $a + a + \cdots + a$ ($b$ fois).

1. Écrire un programme qui lit $a$ puis $b$ et affiche $a \times b$, en ajoutant $a$ à un résultat `res` (initialisé à 0) et en décrémentant $b$ à chaque tour, jusqu'à ce que $b$ vaille 0. Le tester : $6 \times 7$, $5 \times 0$, $0 \times 5$.
2. Dans la version suggérée par le coup de pouce, le corps de la boucle compte 8 instructions (comptez celles de la vôtre). Combien d'instructions environ la machine exécute-t-elle pour $3 \times 200$ ? Et pour $200 \times 3$ ? Comment rendre le programme plus rapide dans le premier cas ?

▷ coup de pouce

★★ **Exercice 6** *Le plus grand de trois*▶

L'exercice 8 (« Le plus grand des deux ») de la rubrique Exercices de ce chapitre donne un programme qui affiche le plus grand de *deux* nombres. Écrire un programme qui lit **trois** nombres et affiche le plus grand. Le tester avec au moins quatre jeux d'entrées bien choisis (le maximum en 1re, 2e, 3e position ; des nombres égaux).

▷ coup de pouce

★★★ **Exercice 7** *Division euclidienne*▶

Écrire un programme qui lit $a$ puis $b$ (avec $b > 0$) et affiche le **quotient** puis le **reste** de la division euclidienne de $a$ par $b$, en retirant $b$ à $a$ autant de fois que possible. Tester : 17 et 5 (sorties 3 puis 2) ; 15 et 5 ; 4 et 9. Que se passe-t-il si $b = 0$ ? Quel est l'équivalent Python de ce programme (avec une boucle `while`) ?

## 2 Partie B — Piloter Linux en ligne de commande

Dans JSLinux, vous êtes connecté en tant qu'administrateur : l'invite se termine par `#` (et non par `$`). Rappels pratiques : la touche **Tab** complète les noms ; la flèche **haut** rappelle les commandes précédentes ; `history` les liste toutes. La commande `man` n'est pas installée ici : *`commande`* `--help` affiche l'aide d'une commande (par exemple `ls --help`).

★ **Exercice 8** *Où suis-je, qui suis-je ?*▶

1. Taper `whoami`, `pwd`, puis `ls -l`. Qui êtes-vous ? Où êtes-vous ? Quel est le chemin absolu de votre répertoire personnel ?
2. Pour `readme.txt`, écrire les droits du propriétaire, du groupe et des autres. Qui en est le propriétaire ?

★★ **Exercice 9** *Construire l'arborescence du club*▶

On veut obtenir, dans le répertoire personnel, l'arborescence suivante :

```
club
|-- archives
|-- docs
|   |-- annonce.txt
|-- projets
    |-- jeu
    |-- site
```

1. Écrire, **avant** de les taper, les commandes qui créent les répertoires (depuis le répertoire personnel, en entrant dans `club`).
2. Pour créer un fichier qui contient du texte, on peut **rediriger** l'affichage d'`echo` vers un fichier : `echo 'Reunion jeudi 13h' > docs/annonce.txt`. Le faire, puis afficher le contenu du fichier.

3. Vérifier le résultat avec la commande `tree` (qui dessine l'arbre), puis avec `ls -R` (qui liste le contenu de chaque répertoire, l'un après l'autre). *Si le terminal répond* `tree: not found`, *ce n'est pas votre erreur : la commande n'est pas installée,* `ls -R` *suffit.*

★★ **Exercice 10** *Un script et le droit* `x`▶

1. Dans `club`, créer un petit programme : `echo 'echo Bonjour le club' > bonjour.sh`. Afficher ses droits avec `ls -l`.
2. Le lancer avec `./bonjour.sh`. Quel message obtient-on ? Pourquoi ?
3. Le lancer avec `sh bonjour.sh`. Cette fois ça marche : quel programme est réellement exécuté, et de quel droit sur `bonjour.sh` a-t-il besoin ?
4. Rendre le fichier exécutable par son propriétaire, vérifier avec `ls -l` (qu'est-ce qui a changé ?), et relancer `./bonjour.sh`.

★★ **Exercice 11** *Copier, déplacer, renommer, supprimer*▶

Depuis `club`, **prévoir** sur le cahier l'arbre obtenu après la suite d'actions ci-dessous, **puis** écrire et exécuter les commandes, et comparer avec `tree` (ou `ls -R` si `tree` n'est pas installée).

1. copier `annonce.txt` dans `archives` ;
2. renommer cette copie en `annonce_sept.txt` ;
3. déplacer `bonjour.sh` dans `projets/jeu`, puis le lancer depuis `club` sans changer de répertoire ;
4. supprimer le répertoire `projets/site` et tout son contenu. Pourquoi faut-il être particulièrement prudent avec cette commande ?

9

★★ **Exercice 12** *L'administrateur n'a pas de limites*▶

Créer `secret.txt` contenant une ligne de texte, puis lui retirer **tous** les droits : `chmod 000 secret.txt`. Vérifier avec `ls -l`, puis essayer `cat secret.txt`. Que constate-t-on ? Expliquer, et en déduire pourquoi le cours conseille de ne pas travailler en permanence avec le compte `root`.

★★★ **Exercice 13** *Mini-scénario : le dossier partagé*▶

Pour voir les droits à l'œuvre, il faut un **deuxième utilisateur**. En tant que `root`, taper :

```
adduser -D lea
mkdir /tmp/partage
cd /tmp/partage
echo 'Sortie au musee le 12' > annonce.txt
echo 'Lea 15, Tom 12' > notes.txt
chmod o-r notes.txt
ls -l
```

(`adduser -D lea` crée un compte `lea` sans mot de passe, possible seulement pour l'administrateur.)

**1. Prévoir, puis vérifier.** Recopier le tableau et le compléter *avant* de passer sur le compte de Léa avec `su lea` (l'invite se termine alors par `$`). Puis taper chaque commande et noter le résultat réel. On revient au compte `root` avec `exit`.

| commande tapée par `lea`, dans `/tmp/partage` | prévision | résultat |
|---|---|---|
| `cat annonce.txt` | | |
| `cat notes.txt` | | |
| `echo 'Lea vient' >> annonce.txt` *(ajoute une ligne)* | | |
| `touch essai.txt` | | |
| `cat /root/club/docs/annonce.txt` | | |
| `cd ; touch essai.txt` *(dans son répertoire personnel)* | | |

**2. Changer les règles.** Revenu en `root`, écrire les commandes `chmod` qui : (a) retirent aux « autres » le droit de lire et de traverser `/root/club` ; (b) permettent aux « autres » de modifier `annonce.txt` ; (c) permettent au groupe et aux autres de lire `notes.txt`. Repasser sur le compte de Léa et refaire les cinq premières lignes du tableau.

**3. L'énigme.** Léa peut maintenant *modifier* `annonce.txt`. Peut-elle le *supprimer* avec `rm` ? Essayer. En regardant les droits du répertoire `/tmp/partage` (`ls -ld /tmp/partage`), proposer une explication.

▷ coup de pouce

## 3 Bilan

**Exercice 14**  *Ce que j'ai appris*

1. Dans le LMC, comment réalise-t-on un `if` ? une boucle `while` ? Citer les instructions utilisées.
2. Pourquoi le simulateur affiche-t-il des nombres dans la mémoire, et pas des mots comme `ADD` ?
3. Recopier sur le cahier et compléter le tableau « qui peut faire quoi » à partir de vos expériences :

| pour… | lire un fichier | modifier un fichier | créer/supprimer dans un répertoire |
|---|---|---|---|
| il faut le droit… | | | |
| sur… | | | |
| `root` est-il concerné ? | | | |

### Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Calculer d'abord `b - a` : si le résultat est positif ou nul (`BRP`), c'est la réponse ; sinon, recalculer dans l'autre sens `a - b`. ↑énoncé

Ex. 4 – piste Une boucle en LMC : une étiquette sur la première instruction du corps (`boucle OUT`), un test qui saute vers `fin` quand l'accumulateur vaut 0 (`BRZ fin`), et un saut inconditionnel `BRA boucle` en bas du corps. ↑énoncé

Ex. 5 – piste Cases à prévoir : `a DAT`, `b DAT`, `res DAT 0`, `un DAT 1`. Le corps de boucle : charger `b` ; s'il vaut 0, aller à `fin` ; sinon `b` ← `b` − 1, puis `res` ← `res` + `a`. ↑énoncé

Ex. 6 – piste Appliquer l'invariant du champion : ranger le premier nombre dans une case `max` ; pour chacun des deux autres, calculer `nombre - max` ; si c'est positif ou nul (`BRP`), ce nombre devient le nouveau `max`. ↑énoncé

Ex. 13 – piste Pour un répertoire, `r` permet d'en lister le contenu, `x` d'y entrer ou de le traverser, et `w` d'y créer ou d'y supprimer des fichiers. ↑énoncé

# FICHE DE RÉVISION

## L'idée maîtresse — un mille-feuille de couches

De l'**électron** au **clic**, tout est empilé : transistors → portes logiques → processeur + mémoire (von Neumann) → langage machine → système d'exploitation → applications → utilisateur. Chaque couche **cache la complexité** de celle du dessous.

## Le transistor et les ordres de grandeur

Le **transistor** est un **interrupteur commandé** (→ 0/1) : la brique de toutes les portes logiques, donc du processeur. À retenir : taille ∼ **quelques nm** ; **plusieurs milliards** par puce ; fréquence ∼ **quelques GHz**. **Loi de Moore** : le nombre de transistors par puce double environ tous les **2 ans** (une *tendance*, pas une loi physique ; elle ralentit).

## Le modèle de von Neumann

Quatre unités : **unité de commande** (lit/décode/ordonne) + **UAL** (calcule) = le **processeur (CPU)** ; **mémoire** (range *programme et données*) ; **entrées-sorties**. Reliées par des **bus**.
**Idée clé :** le **programme est une donnée** (rangé en mémoire comme des nombres) ⇒ machine **universelle**.
**Mémoires** (vitesse ↓, taille ↑) : **registres** < **RAM** (rapide, *volatile*) < **disque** (lent, *permanent*). « Enregistrer » = recopier de la RAM vers le disque.

## Entrées-sorties : capteurs et actionneurs

**Capteur** = *mesure* une grandeur qui **entre** (thermomètre, micro, bouton, GPS). **Actionneur** = reçoit un ordre et **agit** sur le monde (moteur, écran, LED, haut-parleur). Boucle d'un objet connecté : **capter** → **décider** → **agir**.

## Le processeur : langage machine

Le processeur répète le **cycle** : **chercher** → **décoder** → **exécuter**. Il n'obéit qu'au **langage machine** (binaire) ; l'**assembleur** en est la version lisible. Un programme Python ne « tourne » pas tel quel : l'**interpréteur** le **traduit**, au fur et à mesure, en instructions machine.
**Little Man Computer** (machine-modèle) : cases `00`–`99`, un accumulateur **ACC**. *Dérouler* = suivre ACC et les cases, ligne par ligne.

```
INP          lire dans ACC          LDA n / STA n    charger / ranger la case n
OUT          afficher ACC           ADD n / SUB n    ACC +/- case n
BRA n        saut toujours          BRZ n / BRP n    saut si ACC=0 / si ACC>=0
HLT          stop                   DAT              reserve une donnee
```

## Le système d'exploitation (OS)

Il **gère le matériel** et l'offre simplement aux programmes. Ses grandes fonctions : gérer le **processeur**, la **mémoire**, les **fichiers**, les **périphériques**, les **utilisateurs**. Mot-clé : **abstraction**. Familles : **Windows**, **macOS**, **Linux/UNIX** (Android = noyau Linux). *Libre* (Linux) ≠ *gratuit*.

## Ligne de commande Linux

**Arborescence** : racine `/` ; `~` = répertoire personnel. **Chemin absolu** (commence par `/`) vs **relatif** (part du répertoire courant) ; `.` = courant, `..` = parent.

```
pwd  ou je suis          cd  changer de repertoire   cp  copier
ls / ls -l  lister    mkdir  creer un dossier     mv  deplacer / renommer
cat  afficher           touch  creer un fichier     rm / rm -r  supprimer
```

**Attention :** `rm` (`rm -r` : un dossier) supprime *définitivement* ; Linux est *sensible à la casse* (`Doc` ≠ `doc`).

### Droits et permissions

Trois droits `r` (lire), `w` (écrire), `x` (exécuter), pour trois catégories `u` (propriétaire), `g` (groupe), `o` (autres). On lit `ls -l` **par groupes de trois** :

`-rw-r--r--` → fichier ; `u`=`rw-`, `g`=`r--`, `o`=`r--` (`d` au début = répertoire)

Modifier : `chmod [ugoa][+-=][rwx] fichier` (ex. `chmod g+w f.txt`). **En chiffres** : `r` = 4, `w` = 2, `x` = 1 ; un chiffre (la somme) par catégorie `u`, `g`, `o` : `chmod 754 f.txt` donne `rwxr-xr--`. `root` = administrateur (tous les droits).

### Savoir-faire à maîtriser — je sais…

- ☐ nommer les composants du modèle de von Neumann et leur rôle ;
- ☐ **distinguer** un **capteur** (entrée : mesure) d'un **actionneur** (sortie : action) et citer des exemples ;
- ☐ citer des ordres de grandeur (taille, nombre de transistors, fréquence) et énoncer la loi de Moore ;
- ☐ **dérouler** une petite séquence d'instructions en langage machine (accumulateur et cases) ;
- ☐ expliquer ce qu'est un système d'exploitation et citer ses grandes fonctions ;
- ☐ me **repérer** dans une arborescence (chemins absolu et relatif, `.` et `..`) ;
- ☐ **utiliser** les commandes de base en ligne de commande sous Linux ;
- ☐ **lire** une ligne de `ls -l` et **modifier** des droits avec `chmod`, en lettres (`g+w`) ou en chiffres (`754`).

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 667

1. Quelles unités du modèle de von Neumann forment le processeur (CPU) ?
2. Programme LMC `INP`, `STA 05`, `ADD 05`, `OUT`, `HLT` : qu'affiche-t-il pour l'entrée 7 ?
3. Coupure de courant : le travail non enregistré est-il perdu ? Pourquoi ?
4. Dans `/home/eleve/Images`, `cat cours.txt` échoue (le fichier est dans `Documents`). Commande correcte ?
5. Fichier `-rw-rw-r--` : que peuvent faire les autres ? Commande pour retirer au groupe le droit d'écriture ?

**Et avant ?** — *questions sur les chapitres précédents.* ▶ réponses p. 667

1. De quel type sont les valeurs lues dans un CSV avec `csv.DictReader` ? (ch. 8)
2. État de `[5, 2, 4, 1]` après la première étape du tri par sélection ? (ch. 6)
3. Comment parcourir les couples (clé, valeur) d'un dictionnaire `d` ? (ch. 3)

# DÉFI ADVENT OF CODE

## DÉFI Cathode-Ray Tube

*l'écran du communicateur — simuler un petit processeur*

Jour 10

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p09-defi-aoc-2022-10` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2022/day/10`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Un petit processeur exécute un programme de plus d'une centaine d'instructions. Certaines durent un cycle d'horloge, d'autres deux, et elles modifient la valeur d'un registre. Il faut connaître la valeur de ce registre à des cycles précis (partie 1), puis à chaque cycle pour dessiner une image (partie 2). Suivre cela à la main sur des centaines de cycles serait long et source d'erreurs : on le fait simuler par un programme.
**Ce qu'on réinvestit.** Le fonctionnement d'un processeur vu dans ce chapitre : une **horloge** qui cadence le travail, des **cycles**, un **registre** qui garde une valeur modifiée par les instructions. Côté programmation : une liste qui garde l'historique du registre, et le lien entre numéro de cycle et indice dans cette liste.
**La démarche de la fiche.**

1. Comprendre la règle avec des questions courtes, puis remplir à la main un tableau cycle par cycle sur un petit programme.

2. Lire le programme et séparer chaque instruction de son nombre.
3. Construire la liste des valeurs du registre pendant chaque cycle, puis en déduire la réponse de la partie 1.
4. Pour la partie 2, réutiliser cette même liste pour décider, cycle par cycle, si chaque pixel de l'écran est allumé.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `clock circuit` | horloge (circuit) | `cycle` | cycle (top d'horloge) |
| `register` | registre | `instruction` | instruction |
| `take two cycles` | durer deux cycles | `during` | pendant |
| `signal strength` | intensité du signal | `CRT` | écran cathodique |
| `pixel` | pixel | `sprite` | sprite, motif affiché |

### Résumé de l'énoncé (partie 1)

**Contexte.** L'écran de votre appareil de communication est cassé, et pour le remplacer il faut comprendre le petit processeur qui le pilote. Ce processeur est cadencé par une horloge et possède un seul registre, `X`. On veut suivre la valeur de `X` au fil des cycles d'horloge, pour mesurer un signal à certains moments précis.
**Les données.** Le fichier est le programme exécuté par le processeur, une instruction par ligne. Une ligne est soit `noop`, soit `addx` suivi d'un espace et d'un entier, parfois négatif (par exemple `addx -2`).
**La règle.**

- `X` vaut **1** au départ ; les cycles sont numérotés à partir de 1, et les instructions s'exécutent l'une après l'autre.
- `noop` dure 1 cycle et ne fait rien.
- `addx n` dure 2 cycles ; `X` augmente de `n` seulement **à la fin** de ces deux cycles : pendant ces deux cycles, `X` garde son ancienne valeur.
- L'intensité du signal pendant le cycle $c$ vaut $c\times$ (valeur de `X` **pendant** ce cycle).

**La réponse attendue.** La somme des intensités pendant les cycles 20, 60, 100, 140, 180 et 220. Par exemple, avec le programme `addx 3` puis `noop`, `X` vaut 1 pendant les cycles 1 et 2, puis 4 pendant le cycle 3 : l'intensité pendant le cycle 3 serait $3 \times 4 = 12$.

### Exercice 1 *Comprendre ce qui est demandé*

**But.** Vérifier sur des cas particuliers qu'on a bien compris le décalage entre le moment où une instruction commence et celui où `X` change.
**À quoi ça sert ?** La distinction entre « pendant » et « après » un cycle est le piège principal du défi : une erreur ici décale toutes les valeurs, et la réponse est fausse sans aucun message d'erreur.

Après lecture du résumé (et de l'énoncé), vérifier sa compréhension sur ces cas particuliers :

1. Pendant les deux cycles d'un `addx 3`, `X` a-t-il déjà augmenté ?
2. Si l'on calcule `X` *après* le cycle 20 au lieu de *pendant*, quelle erreur commet-on ?
3. Combien de cycles dure un programme de 5 `noop` et 5 `addx` ?

### Lien avec le cours : l'architecture de von Neumann

Ce petit processeur ressemble à celui du chapitre sur l'architecture : une **horloge** cadence le travail, le programme est une suite d'instructions exécutées l'une après l'autre, et un **registre** (ici `X`) garde une valeur que les instructions modifient. Certaines instructions demandent plus de cycles que d'autres, comme dans un vrai processeur.

#### Exercice 2 *À la main, sur un petit exemple*

**But.** Suivre la valeur de `X` cycle par cycle sur un programme de six instructions.
**À quoi ça sert ?** Le tableau obtenu est exactement la liste que le programme devra construire à la question suivante : on dispose ainsi d'un résultat attendu pour le tester.

On considère le programme (inventé) suivant :

```
noop
addx 4
addx -2
noop
addx 5
addx -1
```

Remplir un tableau à deux lignes : numéro du cycle (de 1 à 10) et valeur de `X` **pendant** ce cycle. Vérification : pendant le cycle 4, `X` vaut 5 ; pendant le cycle 9, il vaut 8 ; à la fin du programme, il vaut 7. Que vaudrait l'intensité du signal pendant le cycle 6 ? (On doit trouver 18.)

## 2 Programmer la partie 1

#### Exercice 3 *Lire le programme*

**But.** Ranger les instructions dans une liste `programme` et savoir séparer le nom de l'instruction de son nombre.
**À quoi ça sert ?** Le programme du processeur devient une liste que la simulation parcourra instruction par instruction ; il faut pouvoir récupérer l'entier de chaque `addx`, y compris s'il est négatif.

### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
1 ligne = "1721\n"
2 print(ligne.strip())                  # 1721
3 print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Rappel pour lire un fichier ligne par ligne :

```
1 fichier = open("input.txt", encoding="utf-8")
2 for ligne in fichier:
3     ligne = ligne.strip()   # enlève le retour à la ligne final
4 fichier.close()
```

Construire la liste `programme` des lignes. Pour chaque ligne, savoir distinguer `noop` de `addx` et récupérer l'entier (éventuellement négatif) qui suit `addx`. ▷ coup de pouce

### Exercice 4 *L'historique du registre*

**But.** Construire la liste des valeurs de `X` pendant chaque cycle.
**À quoi ça sert ?** C'est la question centrale : une fois cette liste construite, il suffit de la consulter pour la partie 1 comme pour la partie 2, sans plus jamais se soucier de la durée des instructions.

Écrire une fonction `valeurs_pendant(programme)` qui renvoie la liste des valeurs de `X` pendant chaque cycle : l'élément d'indice `0` correspond au cycle 1. Sur l'exemple, on doit obtenir `[1, 1, 1, 5, 5, 3, 3, 3, 8, 8]`. ▷ coup de pouce ▷ début de solution

### Exercice 5 *L'intensité du signal*

**But.** Calculer la réponse de la partie 1 à partir de l'historique.
**À quoi ça sert ?** On apprend à passer d'un numéro de cycle (qui commence à 1) à un indice de liste (qui commence à 0) ; le même décalage servira pour placer les pixels dans la partie 2.

Avec cette liste, calculer la réponse de la partie 1. Attention au décalage entre numéro de cycle et indice dans la liste. ▷ coup de pouce

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

**Résumé de la partie 2 (à lire après avoir validé la partie 1)**

**Ce qui change.** On découvre que `X` commande en fait la position d'un motif lumineux sur l'écran, et que l'écran se dessine pendant que le programme s'exécute.
**La nouvelle règle.**

- L'écran fait 40 pixels de large sur 6 lignes. Il se dessine un pixel par cycle, de gauche à droite, puis ligne par ligne : le cycle 1 dessine la colonne 0 de la première ligne, le cycle 40 la colonne 39, et le cycle 41 recommence au début de la ligne suivante.
- Le motif fait 3 pixels de large et il est centré sur la colonne `X` : il occupe les colonnes `X - 1`, `X` et `X + 1`.
- Le pixel dessiné pendant un cycle est allumé si sa colonne est l'une des trois colonnes du motif pendant ce cycle ; sinon il reste éteint.

**La réponse attendue.** Les 8 lettres majuscules qui apparaissent sur l'écran une fois dessiné. Par exemple, si `X` vaut 1 pendant le cycle 1, le motif occupe les colonnes 0, 1 et 2, et le pixel de la colonne 0 est allumé.

### Exercice 6 *Dessiner l'écran*

**But.** Calculer, pour chaque cycle, si le pixel dessiné est allumé, et afficher l'écran ligne par ligne pour lire les lettres.
**À quoi ça sert ?** L'historique de la partie 1 donne déjà la valeur de `X` pendant chaque cycle : il ne reste qu'à calculer la colonne du pixel et à la comparer à `X`. Afficher l'image est la seule façon de lire la réponse.

Lire attentivement comment l'écran est balayé (combien de pixels par ligne, combien de lignes) et à quelle condition un pixel est allumé. Construire l'image ligne par ligne sous forme de chaînes de caractères (par exemple `"#"` pour allumé et `"."` pour éteint), l'afficher, et lire les lettres majuscules qui apparaissent : c'est la réponse. ▷ coup de pouce ▷ début de solution

## 4 Approfondissement

★★★ **Exercice 7** *Approfondissement 1 : un vrai petit processeur*

**But.** Ajouter un second registre et deux nouvelles instructions, puis adapter la simulation.
**À quoi ça sert ?** Un vrai processeur a plusieurs registres et beaucoup d'instructions de durées différentes : on voit comment rendre la simulation facile à étendre plutôt que d'ajouter un cas particulier à chaque nouvelle instruction.

On ajoute un second registre `Y`, qui vaut 0 au départ, et deux instructions inventées : `addy n` (2 cycles, ajoute `n` à `Y` à la fin) et `copy` (1 cycle, copie `X` dans `Y` **à la fin** de ce cycle).

1. À la main : écrire les valeurs de `(X, Y)` pendant chaque cycle du programme `addx 4`, `copy`, `addy 3`, `noop`.
2. Écrire `historique(programme)` qui renvoie la liste des couples `(X, Y)` pendant chaque cycle, et comparer avec la question 1. ▷ coup de pouce
3. Quelle structure de données serait pratique pour associer à chaque instruction sa durée ?

### Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste `ligne.split()` donne `["addx", "-2"]` ; `int("-2")` vaut bien `-2`. ↑énoncé

Ex. 4 – piste Pour `noop`, ajouter une fois la valeur actuelle de `X` à la liste ; pour `addx`, l'ajouter **deux** fois, et seulement ensuite modifier `X`. ↑énoncé

Ex. 4 – début de solution `X = 1`, `valeurs = []` ; `for ligne in programme:` si c'est `noop`, `valeurs.append(X)` ; sinon deux `append(X)` puis `X += int(...)`. ↑énoncé

Ex. 5 – piste La valeur pendant le cycle `c` est `valeurs[c - 1]`. Les cycles demandés se suivent de 40 en 40 : `range(20, 221, 40)`. ↑énoncé

Ex. 6 – piste Réutiliser la liste de la partie 1 : pour le cycle numéro `c`, la colonne du pixel dessiné est `(c - 1) % 40`. Comparer cette colonne à la valeur de `X` pendant ce cycle. ↑énoncé

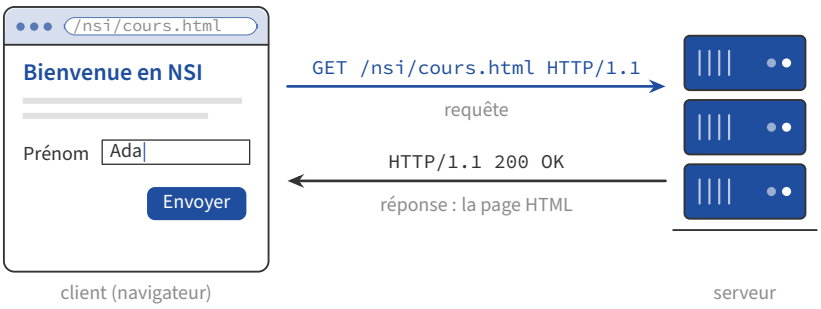
Ex. 6 – début de solution Le pixel est allumé si l'écart entre la colonne et `X` vaut au plus 1 : `abs(colonne - X) <= 1`. Ajouter le caractère à une chaîne `ligne_ecran` et l'afficher (puis la vider) tous les 40 cycles. ↑énoncé

Ex. 7 q. 2 – piste Une fonction `duree(nom)` qui renvoie le nombre de cycles d'une instruction (avec des `if`) permet d'écrire une seule boucle d'ajout pour toutes les instructions. ↑énoncé

10

CHAPITRE 10

# Le Web : réseaux et interactions homme-machine




DANS CE CHAPITRE


- Activités préparatoires . . . 386
- Cours . . . 388
- Exercices . . . 406
- TP et projets . . . 416
- Fiche de révision . . . 436
- Défi Advent of Code . . . 438
- Corrigés . . . 668

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 Le réseau qui perd des papiers

▶ corrigé p. 668

*un jeu de rôle pour ne perdre aucun message*

Durée : 30 min · Par groupes de trois, sans machine

### Consignes

- Matériel : une vingtaine de petits papiers, un crayon chacun ; les réponses se notent sur le cahier.
- Trois rôles. L'**émetteur** doit transmettre le mot `BALLON`, **une lettre par papier**. Le **récepteur**, assis dos à l'émetteur, écrit les lettres reçues dans l'ordre d'arrivée. Le **réseau** porte les papiers de l'un à l'autre, sans parler.
- Le réseau suit en secret sa **feuille de route** (encadré en fin d'énoncé, que **seul** le joueur « réseau » consulte) : elle lui dit quels papiers **perdre** (il les garde dans sa poche). Sur Internet aussi, des paquets se perdent (câble saturé, routeur en panne).
- Interdit de parler ou de se retourner : on ne communique **que par papiers**. On change de rôles à chaque manche.

### Exercice 1 *Manche 1 : on envoie, c'est tout*

L'émetteur envoie les six lettres, une par une, sans attendre.

1. Quel mot le récepteur a-t-il écrit ?
2. Le récepteur peut-il savoir qu'il lui manque une lettre ? L'émetteur, lui, sait-il que sa lettre n'est pas arrivée ?
3. Proposer une règle (une seule phrase) pour que l'émetteur **sache** si chaque papier est bien arrivé.

### Exercice 2 *Manche 2 : on répond « reçu »*

Nouvelle règle : à chaque lettre reçue, le récepteur renvoie un papier « **reçu** ». L'émetteur n'envoie la lettre suivante qu'après avoir reçu ce papier. S'il n'a rien reçu au bout de **10 secondes** (compter dans sa tête), il **renvoie** la même lettre.

4. Quel mot le récepteur a-t-il écrit ?
5. Que s'est-il passé ? (Ouvrir la feuille de route du réseau seulement maintenant.)
6. Le récepteur a-t-il pu deviner, au moment où il l'a reçu, que l'un des papiers était un **renvoi** et non une nouvelle lettre ? Pourquoi le mot `BALLON` rend-il la chose impossible ?
7. Une lettre perdue est désormais renvoyée. Mais quel nouveau problème le papier « reçu » a-t-il fait apparaître ?

**Exercice 3** *Manche 3 : on numérote, mais avec un seul chiffre*

Pour reconnaître un renvoi, on écrit un **numéro** sur chaque papier. Premier réflexe : 1, 2, 3, 4… Mais sur un vrai réseau, chaque chiffre écrit sur un paquet prend de la place. On essaie donc avec **un seul chiffre**, `0` ou `1`.

Règles : les lettres sont marquées tour à tour `0`, `1`, `0`, `1`…; un renvoi garde **le même** chiffre. Le récepteur note à chaque fois le chiffre qu'il **attend** (au début : `0`). S'il reçoit un papier portant le chiffre qu'il vient **déjà** de recevoir, il ne recopie pas la lettre, mais renvoie quand même « reçu ».

**8.** Recopier sur le cahier et compléter le tableau des papiers envoyés par l'émetteur pendant la manche (une case par envoi, renvois compris ; il peut rester des cases vides) :

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| lettre | `B` | | | | | | | | |
| chiffre | `0` | | | | | | | | |

**9.** Quel mot le récepteur a-t-il écrit ? Est-il correct ?

**10.** Pourquoi le récepteur doit-il renvoyer « reçu » même pour un papier qu'il jette ? (Penser à ce que ferait l'émetteur sinon.)

**11.** Pourquoi un seul chiffre suffit-il ? (Combien de papiers différents l'émetteur peut-il avoir « en route » à un instant donné ?)

**Exercice 4** *Mettre des mots*

**12.** Recopier sur le cahier en complétant les trous **(a)** à **(d)** : « Pour être sûr qu'un paquet est arrivé, le récepteur renvoie un … **(a)**. Sans réponse au bout d'un certain … **(b)**, l'émetteur … **(c)** le même paquet. Pour ne pas confondre un renvoi avec un nouveau paquet, on marque chaque paquet d'un … **(d)** qui vaut alternativement `0` et `1`. »

**13.** Comment appelle-t-on un ensemble de règles que tous les participants doivent respecter pour communiquer ?

**Feuille de route du réseau — réservée au joueur « réseau »**

- **Manche 1** : perdre le **3e** papier confié (la première lettre `L`).
- **Manche 2** : transmettre toutes les lettres, mais perdre le **3e** papier « reçu » (celui de la première lettre `L`).
- **Manche 3** : perdre le **2e** papier de lettre (le premier `A`) et le **4e** papier « reçu » transporté.

# COURS

**Programme (BO)** *Interactions entre l'homme et la machine sur le Web* : identifier les **composants graphiques** d'une page et les **événements** qu'ils traitent (description en HTML, comportement en JavaScript) ; analyser et modifier les méthodes exécutées lors d'un **clic** ; distinguer ce qui est exécuté sur le **client** ou sur le **serveur**, et dans quel ordre ; distinguer ce qui est **mémorisé dans le client** et retransmis au serveur ; reconnaître **quand et pourquoi la transmission est chiffrée** ; analyser un **formulaire** simple, distinguer les transmissions par `GET` et `POST`. *Architectures matérielles (réseau)* : intérêt du découpage des données en **paquets** et de leur **encapsulation** ; protocole du **bit alterné** ; **simuler ou mettre en œuvre un réseau** ; rôle des constituants du **réseau local de l'établissement**.

**Prérequis** le chapitre *Architecture des ordinateurs et systèmes d'exploitation* (client/serveur = deux machines en réseau) ; savoir écrire une fonction. → fiches de révision : ch. 9 p. 378

**Objectifs** comprendre **ce qui se passe** entre un clic et l'affichage d'une page ; **lire une URL** ; savoir ce qu'est un **protocole**, une **adresse IP**, un **paquet** ; distinguer **client** et **serveur** ; écrire une page **HTML/CSS** et la rendre **interactive** en JavaScript ; envoyer des données par un **formulaire** (`GET`/`POST`).

## Situation de départ

1. Que se passe-t-il entre le clic sur un lien et l'affichage de la page ?
2. Si un morceau d'une vidéo se perd en route, comment l'ordinateur qui l'a envoyé peut-il le savoir ?
3. Comment un site marchand se souvient-il de votre panier d'une page à l'autre ?

**Remarque** — *Le fil conducteur : le voyage d'un clic*
Vous tapez une adresse, vous appuyez sur *Entrée*, et en une fraction de seconde une page s'affiche. Que s'est-il passé ? Un **voyage** : votre navigateur (le **client**) a trouvé la bonne machine sur le réseau (via son **adresse IP**), lui a envoyé une **requête HTTP** découpée en **paquets** qui ont traversé des **routeurs**, le **serveur** a renvoyé des fichiers **HTML/CSS/JavaScript**, et le navigateur les a **assemblés** en une page avec laquelle vous **interagissez**. Ce chapitre suit ce voyage, de bout en bout. Les points qui dépassent le programme portent le badge [ au-delà du programme ].

# I Internet et le Web : ne pas confondre

**Définition 1 — Internet et le Web**

**Internet** est le réseau mondial : l'*infrastructure* de câbles, fibres, antennes et machines qui relient les ordinateurs entre eux. **Le Web** (*World Wide Web*) est l'un des services qui circulent sur Internet : l'ensemble des pages reliées par des liens hypertextes, consultées avec un navigateur.

Internet transporte bien d'autres choses que le Web : le courriel, les messageries, les jeux en ligne, le streaming… *Confondre les deux, c'est confondre la route (Internet) et l'un des véhicules qui roulent dessus (le Web).*

# II Le réseau : comment les machines se parlent

**Définition 2 — Protocole**

Un **protocole** est un **ensemble de règles** précises que deux machines respectent pour communiquer. Comme deux personnes doivent parler la même langue et respecter des tours de parole, deux ordinateurs doivent suivre le même protocole pour se comprendre.

↻ **Retour sur l'activité.** La dernière question de l'activité « Le réseau qui perd des papiers » y répondait : les règles des manches 2 et 3, que l'émetteur et le récepteur devaient respecter tous les deux, formaient un protocole.

**Définition 3 — Adresse IP**

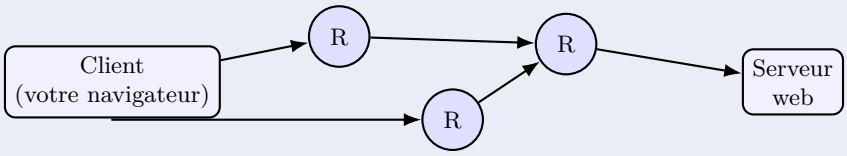
Pour être jointe sur le réseau, chaque machine possède une **adresse IP** (*Internet Protocol*), son « numéro » sur ce réseau. En version IPv4, c'est une suite de quatre nombres de 0 à 255, par exemple `193.51.208.14`. C'est l'équivalent d'une **adresse postale** : sans elle, impossible de livrer un message à la bonne machine.

**Remarque** — *Adresses privées et adresse publique*
Une adresse **publique** (comme `193.51.208.14`) est unique sur tout Internet. Les machines d'un réseau local (maison, lycée) reçoivent le plus souvent une adresse **privée**, du type `192.168.x.x` ou `10.x.x.x`, unique seulement dans ce réseau : des milliers de maisons utilisent `192.168.1.10` ! Quand un paquet sort vers Internet, la box (le routeur d'accès) remplace l'adresse privée par sa propre adresse publique, puis redistribue la réponse à la bonne machine : c'est la **traduction d'adresses** (NAT).



**Règle 1 — On voyage en paquets**

Un message (une page, un mail, une vidéo) n'est pas envoyé d'un bloc : il est découpé en petits morceaux, les **paquets**. Chaque paquet porte l'adresse IP de départ et d'arrivée, voyage indépendamment à travers le réseau, et les paquets sont réassemblés à l'arrivée. Si un paquet se perd, il est renvoyé.




*les paquets traversent des **routeurs** (R) et peuvent prendre des chemins différents*

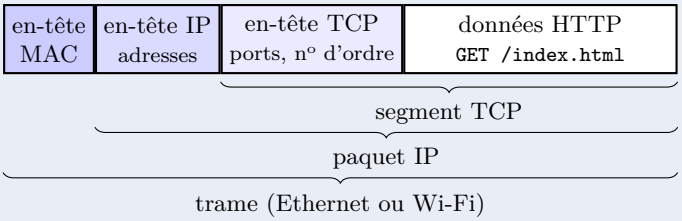
**Règle 2 — L'encapsulation : des enveloppes dans des enveloppes**

Pour voyager, les données traversent plusieurs **couches** de protocoles. À l'envoi, chaque couche ajoute devant ce qu'elle reçoit un **en-tête** contenant les informations dont *elle* a besoin, comme on glisse une lettre dans une enveloppe, puis cette enveloppe dans une autre : c'est l'**encapsulation**.

- **Application** (HTTP) : la requête elle-même, par exemple `GET /index.html` ;
- **Transport** (TCP) : les **numéros de port** (le service visé : 80 pour HTTP, 443 pour

HTTPS) et un **numéro d'ordre** qui permet de remettre les morceaux dans l'ordre et d'en accuser réception ; on obtient un **segment** ;
- **Réseau** (IP) : les **adresses IP** source et destination ; on obtient un **paquet** ;
- **Liaison** (Ethernet, Wi-Fi) : les adresses **MAC** (adresse physique de la carte réseau) de la machine émettrice et de la prochaine machine du trajet ; on obtient une **trame**.

À l'arrivée, chaque couche retire « son » en-tête et passe le reste à la couche du dessus (*désencapsulation*). L'intérêt : chaque couche fait son travail **sans connaître les autres**. HTTP ne sait pas si le paquet passe par un câble, le Wi-Fi ou la fibre ; un routeur n'ouvre l'enveloppe que jusqu'à l'en-tête IP, dont il a besoin pour aiguiller.




### Définition 4 — Routeur et commutateur

Un **routeur** est une machine qui aiguille les paquets : il lit l'adresse IP de destination et choisit vers quel voisin renvoyer le paquet pour le rapprocher du but (comme un centre de tri postal). Un **commutateur** (*switch*) relie les machines d'un *même* réseau local (une salle, une maison). L'ensemble des routeurs interconnectés forme Internet.

### À retenir — Le réseau du lycée

Le réseau local d'un établissement assemble quelques constituants, chacun avec un rôle précis :

- des **commutateurs** (*switchs*), un par salle ou par étage, qui relient par câble les postes d'un même réseau local ;
- des **points d'accès Wi-Fi**, qui relient sans fil les portables et tablettes à ce même réseau ;
- des **serveurs** internes : fichiers des élèves, comptes et mots de passe, impression, logiciels ;
- un **pare-feu**, qui filtre les échanges entre le lycée et l'extérieur (il bloque ce qui n'est pas autorisé) ;
- un **routeur d'accès** (la « box » de l'établissement), qui relie le réseau local à Internet et traduit les adresses privées en adresse publique.

► **Exercices d'application :** activité « Simuler un réseau », à faire sur le simulateur Filius : construire deux réseaux reliés par un routeur, observer l'encapsulation, reconnaître les constituants d'un réseau d'établissement

### À retenir — Trouver l'adresse : le DNS

On ne retient pas des suites de chiffres : on tape `www.lycee.mc`, pas `193.51.208.14`. Le **DNS** (*Domain Name System*) est l'**annuaire** du réseau : il **traduit un nom** de domaine en **adresse IP**. Première étape du voyage : le navigateur demande au DNS « quelle est l'IP de `www.lycee.mc` ? ».

**Définition 5 — Client-serveur et pair-à-pair**

Deux façons d'organiser les échanges :

- **client-serveur** : des machines clientes *demandent* des ressources à une machine serveur qui les *fournit* (le modèle du Web) ;
- **pair-à-pair** (*peer-to-peer*, P2P) : chaque machine est à la fois cliente *et* serveur, et échange directement avec les autres, sans passer par un serveur central (partage de fichiers, certaines messageries).

## A. Ne perdre aucun paquet : l'accusé de réception et le bit alterné

Un paquet perdu est *renvoyé* : encore faut-il que l'émetteur **sache** qu'il est perdu. Le récepteur doit donc **répondre**.

### Définition 6 — Accusé de réception

Après avoir reçu un paquet, le récepteur renvoie un petit message de confirmation, l'**accusé de réception** (*ACK*, de *acknowledgement*). Tant que l'émetteur n'a pas reçu l'accusé au bout d'un certain délai, il considère le paquet perdu et le **renvoie**.

↻ **Retour sur l'activité.** Le papier « reçu » de la manche 2 était un accusé de réception, et les 10 secondes comptées par l'émetteur le délai au-delà duquel il renvoyait sa lettre.

Si c'est l'**accusé** qui se perd, l'émetteur renvoie un paquet déjà reçu : le récepteur doit pouvoir **reconnaître ce doublon**.

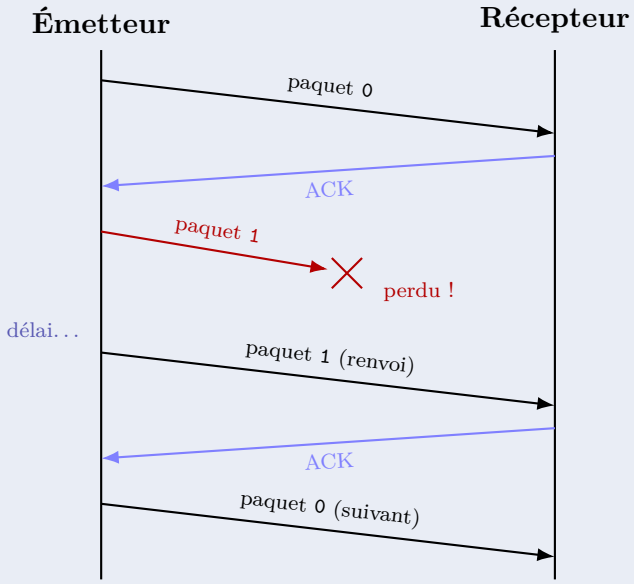
### Règle 3 — Le protocole du bit alterné

On étiquette chaque paquet avec un **seul bit**, qui vaut alternativement `0`, `1`, `0`, `1`… L'accusé renvoie le numéro du paquet attendu ensuite. Les règles :

- l'émetteur n'envoie le paquet suivant (avec le bit changé) que lorsqu'il a reçu l'accusé du paquet courant ;
- sans accusé au bout du délai, il renvoie le même paquet, avec le *même* bit ;
- si le récepteur reçoit un paquet portant le bit qu'il a *déjà* traité, c'est un **doublon** : il le jette mais renvoie quand même l'accusé.

Un seul bit suffit à distinguer « le nouveau paquet » de « le même, renvoyé ».

↻ **Retour sur l'activité.** Dans la manche 2, un papier « reçu » perdu faisait écrire `BALLLON` au récepteur, incapable de reconnaître le `L` renvoyé. Dans la manche 3, le chiffre `0` ou `1` de chaque papier lui permettait de jeter ce doublon : c'était le protocole du bit alterné.




*Le paquet `1` est perdu ; faute d'accusé, l'émetteur le renvoie à l'identique. Le bit permet au récepteur de ne pas confondre un renvoi avec un nouveau paquet.*

**Remarque**
Ce mécanisme d'accusés et de renvois est au cœur du protocole **TCP**, qui garantit sur Internet qu'aucune donnée n'est perdue ni dupliquée ; le bit alterné en est la version la plus simple.

► **Exercices d'application :** exercices 1 à 5 (réseaux : Internet/Web, paquets, bit alterné), 29 (encapsulation) et 30 (le réseau d'un lycée)

## III Le modèle client-serveur et l'URL

Sur le Web, votre navigateur est un **client** : il demande des pages à des **serveurs web**, des machines allumées en permanence dont le rôle est de *servir* des ressources. Pour désigner la ressource voulue, on utilise une **URL**.

**Définition 7 — URL — *Uniform Resource Locator***

Une URL est l'**adresse d'une ressource** sur le Web. On la décompose :

`https://www.lycee.mc/nsi/cours.html`
protocole — nom de domaine — chemin

- `https` : le **protocole** utilisé (ici la version *sécurisée* de HTTP) ;
- `www.lycee.mc` : le **nom de domaine** du serveur (traduit en IP par le DNS) ;
- `/nsi/cours.html` : le **chemin** de la ressource sur le serveur.

**À retenir — Reconnaître une page sécurisée**

Quand l'URL commence par `https` (et non `http`), la communication est **chiffrée** : personne ne peut lire au passage ce qui circule. **Réflexe** : avant de saisir un mot de passe ou des coordonnées bancaires, **vérifier le `https`**. Le « s » veut dire *secure*. Attention : le `https` (le cadenas) prouve que la liaison est chiffrée, **pas** que le site est honnête ; un site d'hameçonnage peut lui aussi être en `https`, il faut donc aussi vérifier le nom de domaine. On étudiera le chiffrement en Terminale.

**Pour les curieux** — *Le port* (facultatif)
L'adresse IP désigne la **machine** ; le **port** (un numéro) désigne le **service** visé sur cette machine. Le Web utilise par convention le port **80** en `http` et le port **443** en `https`. C'est pourquoi on ne les écrit presque jamais dans l'URL : ils sont **sous-entendus**.

## IV HTTP : la langue du Web

Une fois le serveur trouvé, client et serveur dialoguent selon le protocole **HTTP** (*HyperText Transfer Protocol*) : le client envoie une **requête**, le serveur renvoie une **réponse**.

### Règle 4 — Requête et réponse HTTP

Une requête contient notamment une méthode et le chemin de la ressource :

```
GET /nsi/cours.html HTTP/1.1
```

La réponse commence par un **code de statut**, suivi d'**en-têtes** (des métadonnées : type et taille du contenu, date…), puis d'une ligne vide et enfin du **corps** (la ressource elle-même) :

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1024
...
<!doctype html> ...le code HTML de la page...
```

### Définition 8 — Les deux méthodes à connaître : `GET` et `POST`

- `GET` : **demander** une ressource (le cas le plus courant : afficher une page). Les éventuels paramètres sont visibles dans l'URL.
- `POST` : **envoyer** des données au serveur pour qu'il les traite (typiquement, les données d'un formulaire). Les données ne sont *pas* dans l'URL.

### À retenir — Quelques codes de statut

Le code résume ce qui s'est passé : **200** « OK, voici la ressource » ; **304** « pas modifiée, garde ta version en *cache* » (évite de re-télécharger un fichier inchangé) ; **404** « ressource introuvable » (la fameuse page d'erreur) ; **403** « accès interdit » ; **500** « erreur du serveur ». Les codes qui commencent par **2** sont des succès, par **3** des redirections ou du cache, par **4** des erreurs du client, par **5** des erreurs du serveur.

### Règle 5 — Une page = souvent des dizaines de requêtes

Afficher une page « simple » ne demande presque jamais une seule ressource. Le navigateur reçoit d'abord le HTML, le lit, et y découvre des références vers d'autres fichiers (feuilles `.css`, scripts `.js`, images, polices…) : pour chacun il envoie **une nouvelle requête HTTP**. Une page riche déclenche ainsi facilement plusieurs dizaines d'échanges requête/réponse avant d'être complète.

### Méthode — Observer les échanges : les outils de développement

Tous les navigateurs offrent des **outils de développement** (touche `F12`, onglet **Réseau**). On y voit **en direct**, pour chaque page, la **liste des requêtes**, leur **méthode** (`GET`…), leur **code de statut** (200, 304, 404…), leur **taille** et leur **temps** de chargement. C'est le meilleur moyen de *voir* concrètement le protocole HTTP à l'œuvre.

► **Exercices d'application :** exercices 6 à 9 et 27 (URL et HTTP)

## V HTML : structurer la page

La ressource renvoyée est le plus souvent un fichier **HTML** (*HyperText Markup Language*). Ce n'est **pas** un langage de programmation : c'est un langage de **balisage** qui décrit la **structure** du contenu (titres, paragraphes, images, liens…).

**Définition 9 — Balises**

Le contenu est encadré par des **balises** : une balise ouvrante `<p>` et une balise fermante `</p>`. Certaines portent des **attributs**.

```html
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Ma page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Bienvenue en NSI</h1>
    <p>Ceci est un <strong>paragraphe</strong> important.</p>
    <a href="https://eduscol.education.fr">Un lien hypertexte</a>
    <img src="logo.png" alt="logo du lycee">
  </body>
</html>
```

- `<head>` : les informations *sur* la page (titre, encodage, feuille de style) ; `<body>` : le contenu *visible* ;
- `<h1>` un titre, `<p>` un paragraphe, `<strong>` du texte important, `<img>` une image ;
- `<a href="...">` crée un **lien hypertexte** : c'est lui qui relie les pages entre elles et fait du Web une **toile**.

**Remarque** — *D'autres balises courantes*

Au-delà de l'exemple : les **titres** `<h1>` à `<h6>` (du plus au moins important) ; les **listes** `<ul>` (à puces) ou `<ol>` (numérotée), dont chaque élément est un `<li>` ; `<br>` un retour à la ligne ; `<table>` un tableau ; et les **conteneurs** `<div>` (en bloc) et `<span>` (en ligne) qui **regroupent** des éléments pour les mettre en forme ou les manipuler. Un **commentaire**, ignoré par le navigateur, s'écrit `<!-- ... -->`.

**Remarque** — *Les balises « auto-fermantes »*

Certaines balises n'entourent aucun contenu et n'ont **pas** de balise fermante : `<img>`, `<br>`, `<meta>`, `<input>`, `<link>`. On écrit `<img src="logo.png">` et jamais `<img>...</img>`.

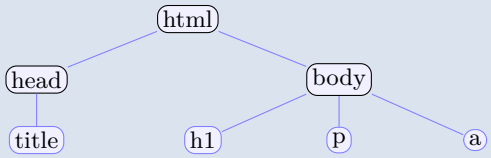
**Définition 10 — Les attributs `id` et `class`**

On peut « étiqueter » une balise pour la **retrouver** ensuite (en CSS ou en JavaScript) :

- `id="menu"` : un identifiant **unique** dans la page (une seule balise peut le porter) ;
- `class="bouton"` : une étiquette **partagée**, que plusieurs balises peuvent avoir en commun (pour les traiter ensemble).

**À retenir — Une page est un arbre : le DOM**

Les balises s'**emboîtent** sans se croiser : le document forme un **arbre**, appelé **DOM** (*Document Object Model*). C'est cet arbre que le navigateur construit en mémoire, et que le JavaScript pourra modifier.




► **Exercices d'application :** exercices 10 à 13 (balises ; lire, corriger, créer du HTML)

## VI CSS : mettre en forme

Le **CSS** (*Cascading Style Sheets*) décrit la **présentation** : couleurs, tailles, polices, dispositions. **Principe fondamental : séparer le fond (HTML) de la forme (CSS).** Le même contenu peut ainsi changer d'apparence sans qu'on touche au HTML.

**Règle 6 — Une règle CSS**

Une règle vise un **sélecteur** (quelles balises) et lui applique des **propriétés**, chacune sous la forme `propriété: valeur;` :

```
h1      { color: darkblue; text-align: center; }   /* toutes les balises h1 */
p       { font-size: 14px; line-height: 1.5; }     /* tous les paragraphes  */
#menu   { background-color: gold; }        /* l'unique balise id="menu"     */
.bouton { color: white; padding: 8px; }    /* toutes les balises class="bouton"
    */
```

**Définition 11 — Les trois sélecteurs à connaître**

- par **nom de balise** (`h1`, `p`) : vise *toutes* les balises de ce type ;
- par **identifiant**, préfixe `#` (`#menu`) : vise l'*unique* balise `id="menu"` ;
- par **classe**, préfixe `.` (`.bouton`) : vise *toutes* les balises `class="bouton"`.

**Remarque** — *Trois façons de brancher le CSS*
Le plus propre : une **feuille externe** `style.css` reliée par `<link rel="stylesheet" href="style.css">` (réutilisable sur tout le site). On peut aussi écrire le CSS **dans** le `<head>`, entre `<style>...</style>`, ou **en ligne** sur une balise via l'attribut `style="..."` — à réserver aux petits ajustements.

**Remarque** — *Quelques propriétés utiles*
`color` (couleur du texte), `background-color` (fond), `font-size`, `font-family`, `text-align` ; et pour l'espacement : `margin` (marge *extérieure*), `padding` (marge *intérieure*), `border` (bordure), `width`/`height` (dimensions). Les tailles s'expriment en `px` (pixels), `%` ou `em`.

**Pour les curieux** — *Pourquoi « cascade » ?* (facultatif)
Plusieurs règles peuvent viser la même balise. En cas de **conflit**, la plus **spécifique** l'emporte (un `#id` bat une `.classe`, qui bat un nom de balise) ; à spécificité égale, la **dernière** écrite gagne. Les styles « ruissellent » ainsi du général vers le particulier — d'où le nom *Cascading Style Sheets*.

### A. Disposer les éléments : Flexbox [ au-delà du programme ]

Par défaut, les blocs (`<div>`, `<p>`, `<li>`…) s'empilent **verticalement**, l'un sous l'autre. Pour les aligner **horizontalement** et les répartir proprement, on transforme leur **conteneur** en « boîte flexible » avec la propriété `display: flex`.

```
.barre {
    display: flex;              /* les enfants se placent en ligne */
    gap: 10px;                  /* espace entre eux */
    justify-content: center;    /* repartition horizontale */
    align-items: center;        /* alignement vertical */
}
```

Appliquée à un bloc `<div class="barre">...</div>`, cette règle aligne **tous ses enfants directs** sur une même ligne. `justify-content` règle la **répartition horizontale** (`center`, `space-between`, `flex-start`…), `align-items` l'**alignement vertical**, et `gap` l'espace entre les éléments. C'est l'outil standard pour construire une **barre de navigation** ou disposer des cartes côte à côte.




*un conteneur* `display:flex` + `justify-content:space-between`



► **Exercices d'application :** exercices 14 à 16 (CSS : relier une feuille, sélecteurs `id`/`class`, aligner avec Flexbox)

## VII JavaScript : rendre la page interactive

HTML structure, CSS met en forme… mais tout cela est **figé**. Le **JavaScript** (JS) est un vrai **langage de programmation**, exécuté **dans le navigateur** (côté **client**), qui rend la page **vivante** : réagir à un clic, modifier le contenu, valider un formulaire…

**Règle 7 — Les trois langages du Web**

**HTML** = le *fond* (structure) ; **CSS** = la *forme* (présentation) ; **JavaScript** = le *comportement* (interaction). Trois rôles distincts, une seule page.

**Remarque** — *Un peu de syntaxe JavaScript*
On **déclare** une variable avec `let` (ou `const` si sa valeur ne change jamais) ; une **fonction** s'écrit `function nom() { ...}` ; les instructions se terminent par `;` et un commentaire par `//`. C'est proche du Python, mais avec des **accolades** `{ }` à la place de l'indentation.

## A. Répondre à un événement

Un **événement** est quelque chose qui **se produit** dans la page, le plus souvent une action de l'utilisateur : un **clic** (`click`), le **survol** de la souris (`mouseover`), une **touche** du clavier (`keydown`), la **modification** d'un champ (`change`), l'**envoi** d'un formulaire (`submit`), le **chargement** de la page (`load`)… On y réagit en exécutant une fonction JavaScript.

```html
<p id="monPara">Voici une page qui ne fait pas grand-chose.</p>
<button onclick="changer()">Cliquer ici</button>
<script src="script.js"></script>
```

**Remarque** — *Deux manières de brancher un événement*
Rapide : un **attribut** `on...` directement dans le HTML, comme `<button onclick="changer()">`. Plus propre (le comportement reste séparé de la structure) : depuis le JavaScript, avec `addEventListener`. Le bouton porte alors un identifiant, et plus d'attribut `onclick` :

```html
<button id="monBouton">Cliquer ici</button>
```

```js
let bouton = document.querySelector("#monBouton");
bouton.addEventListener("click", changer);   // sans les parentheses
```

## B. Modifier le document (le DOM)

Dans `script.js`, on **sélectionne** un élément puis on le **modifie** :

```js
function changer() {
    let maBalise = document.querySelector("#monPara");
    maBalise.innerHTML = "Bravo ! Le contenu vient de changer.";
}
```

- `document.querySelector("#monPara")` retrouve dans le DOM la balise d'`id monPara` ;
- `.innerHTML = "..."` **remplace son contenu**. Au clic, le texte du paragraphe change — **sans recharger la page**.

**Définition 12 — Sélectionner et modifier dans le DOM**

**Sélectionner** un élément :

- `document.getElementById("monPara")` : par son `id` ;
- `document.querySelector("#monPara")` : par un *sélecteur CSS* (le premier qui correspond) ;
- `document.querySelectorAll(".bouton")` : *tous* ceux qui correspondent.

Puis le **modifier** :

- `.textContent` ou `.innerHTML` : changer le contenu ;
- `.style.color`, `.style.background`… : changer la forme ;
- `.classList.add("actif")` / `.classList.remove("actif")` : ajouter/retirer une classe CSS ;
- `.value` : lire (ou écrire) le contenu d'un champ de formulaire.

**Exemple** — *Réagir à une saisie*

Une zone de texte, un bouton, et un paragraphe qui salue l'utilisateur :

```html
<input type="text" id="nom">
<button onclick="saluer()">Bonjour</button>
<p id="message"></p>
```

```javascript
function saluer() {
    let nom = document.querySelector("#nom").value;   // lire le champ
    document.querySelector("#message").textContent = "Bonjour " + nom + " !";
}
```

▷ **Je prédis.** On tape `Léa` dans le champ, puis on clique **deux fois** sur le bouton. Que contient le paragraphe ?

▷ **Je vérifie.** « Bonjour Léa ! », une seule fois : `textContent = ...` *remplace* le contenu, il ne l'ajoute pas à la suite.

Rien n'est envoyé au serveur : tout se passe **dans le navigateur**, instantanément.

**À retenir — Côté client, côté serveur**

Le **JavaScript** est le langage de l'interaction **côté client** : il s'exécute **sur votre machine**, **après** réception de la page, pour les réactions immédiates (afficher/masquer, vérifier qu'un champ n'est pas vide…). D'autres traitements se font **côté serveur**, sur la machine distante, dans un **autre langage** — par exemple **PHP**, ou Python : consulter une base de données, vérifier un mot de passe, générer une page personnalisée. *Savoir qui fait quoi* est essentiel : ce qui se passe côté client est visible et modifiable par l'utilisateur, donc jamais sûr pour des vérifications sensibles.

10

**Remarque** — *Faut-il apprendre PHP ? (mise au point)*

Pour **l'interaction** dans la page, le programme s'en tient au **JavaScript** — il n'y a pas d'autre langage à connaître de ce côté. **PHP** n'est cité ici que comme *exemple* de langage **côté serveur**, pour bien opposer « ce que fait le client » et « ce que fait le serveur ». Le programme précise d'ailleurs qu'il ne s'agit **pas** de développer une expertise dans ces langages (PHP comme JavaScript) : on doit savoir *distinguer les rôles*, pas coder un serveur en PHP. C'est en Terminale, avec les **bases de données**, que le « côté serveur » sera approfondi.

## C. Assembler les trois langages : une page complète

Voici une page **autonome** qui réunit tout : la **structure** (HTML), la **mise en forme** avec Flexbox (CSS) et l'**interactivité** (JavaScript). L'utilisateur tape un article, clique sur *Ajouter*, et l'article s'ajoute à une liste — **sans jamais recharger la page**.

```html
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Ma liste</title>
  <style>
    body      { font-family: sans-serif; margin: 20px; }
    .ligne    { display: flex; gap: 8px; }      /* champ + bouton en ligne */
    #liste li { color: darkblue; }
  </style>
</head>
<body>
  <h1>Ma liste de courses</h1>
```

```
  <div class="ligne">
    <input type="text" id="article" placeholder="Un article...">
    <button onclick="ajouter()">Ajouter</button>
  </div>
  <ul id="liste"></ul>

  <script>
    function ajouter() {
        let champ = document.querySelector("#article");
        let texte = champ.value;                // lire la saisie
        if (texte !== "") {                     // ignorer une saisie vide
            let liste = document.querySelector("#liste");
            liste.innerHTML += "<li>" + texte + "</li>";   // ajouter un <li>
            champ.value = "";                   // vider le champ
        }
    }
  </script>
</body>
</html>
```

▷ **Je prédis.** On tape `pain`, on clique sur *Ajouter*, puis on clique encore une fois sans rien taper. Que contient la liste ?
▷ **Je vérifie.** Un seul article, « pain » : après le premier clic le champ a été vidé, et au second clic la saisie vide est ignorée par le test `texte !== ""`.

- le `<style>` met en forme et **aligne** le champ et le bouton avec **Flexbox** ;
- au clic, `ajouter()` **lit** le champ (`.value`), **teste** qu'il n'est pas vide, **ajoute** un `<li>` à la liste, puis **vide** le champ ;
- les trois langages coopèrent, et tout se déroule **côté client**.

► **Exercices d'application :** exercices 17 à 21 (JavaScript ; client ou serveur) et 31 (programmer une interface d'après un cahier des charges)

## VIII Les formulaires : envoyer des données

Jusqu'ici le client *recevait*. Un **formulaire** lui permet d'**envoyer** des données au serveur (identifiants, recherche, message…).

```
<form action="/connexion" method="post">
    <label for="pseudo">Pseudo :</label>
    <input type="text" name="pseudo" id="pseudo">
    <input type="password" name="mdp">
    <button type="submit">Se connecter</button>
</form>
```

▷ **Je prédis.** Si l'on remplaçait `method="post"` par `method="get"`, que montrerait la barre d'adresse après l'envoi, pour le pseudo `lea` et le mot de passe `1234` ?
▷ **Je vérifie.** `.../connexion?pseudo=lea&mdp=1234` : le mot de passe apparaîtrait en clair dans l'URL (et resterait dans l'historique). C'est pourquoi ce formulaire utilise `post`.

- `action` : l'**adresse** sur le serveur qui recevra les données ;
- `method` : **comment** les envoyer — `get` (données dans l'URL) ou `post` (données dans le corps de la requête) ;
- chaque `<input>` a un `name` : c'est l'**étiquette** sous laquelle sa valeur arrive au serveur.

**Remarque** — *Quelques types de champs*
L'attribut `type` d'un `<input>` choisit la nature du champ : `text` (texte visible), `password` (masqué), `radio` (un seul choix parmi plusieurs), `checkbox` (cases à cocher, plusieurs choix possibles), `email`, `number`, `date`... Un `<input type="submit">` (ou un `<button type="submit">`) **envoie** le formulaire vers l'`action`.

**Règle 8 — `GET` ou `POST` pour un formulaire ?**

- `GET` : les données apparaissent dans l'URL après un `?`, sous la forme `clef=valeur` séparées par `&` (`.../recherche?q=chat&tri=date`). Pratique pour une **recherche** qu'on peut partager ou mettre en favori ; à éviter pour un mot de passe (visible et enregistré dans l'historique).
- `POST` : les données ne sont pas dans l'URL. À utiliser dès qu'on envoie quelque chose de sensible ou qui **modifie** l'état du serveur (connexion, publication...).

**Pour les curieux** — *Le web dynamique* (facultatif)
Quand le serveur **fabrique** la page en fonction de la requête (heure, utilisateur connecté, résultats d'une recherche), on parle de **web dynamique**. Un programme s'exécute alors **côté serveur**, dans un langage comme PHP, Python (*Flask*), etc., pour **générer** le HTML renvoyé. C'est ce qui distingue un site vivant d'un simple fichier figé.

► **Exercices d'application :** exercices 22, 23 et 28 (lire et écrire un formulaire)

## A. Les cookies : ce que le navigateur retient

Le protocole HTTP est **sans mémoire** : chaque requête est traitée seule, et le serveur ne sait pas, a priori, que deux requêtes viennent du même visiteur. Pour s'en souvenir (rester connecté, garder un panier, retenir une langue), le serveur demande au navigateur de garder une petite information : un **cookie**.



**Règle 9 — Le cookie : mémorisé dans le client, retransmis au serveur**

- Le serveur envoie le cookie dans un en-tête de sa réponse, `Set-Cookie`, sous la forme `nom=valeur`.
- Le navigateur (le client) le **mémorise**, rangé avec le nom du site qui l'a déposé.
- À chaque nouvelle requête vers ce même site, le navigateur le **retransmet automatiquement** dans l'en-tête `Cookie`. Il ne l'envoie jamais à un autre site.

```
HTTP/1.1 200 OK                          <- reponse du serveur
Set-Cookie: langue=fr; Max-Age=2592000

GET /accueil.html HTTP/1.1               <- requete suivante du navigateur
Cookie: langue=fr
```

La durée est fixée par le serveur : `Max-Age=2592000` (en secondes) garde le cookie 30 jours ; sans durée, il est effacé à la fermeture du navigateur.

**Remarque** — *Usages et vie privée*
Les cookies servent à rester connecté (le cookie contient un identifiant de session), à garder un panier ou des préférences. Mais des cookies déposés par des régies publicitaires présentes sur de nombreux sites permettent aussi de **suivre** un internaute d'un site à l'autre. Les sites doivent donc demander l'accord de l'utilisateur pour les cookies qui ne sont pas indispensables : c'est le rôle des bandeaux « Accepter / Refuser ». On peut consulter et effacer ses cookies dans le navigateur (`F12`, onglet *Application* ou *Stockage*). Enfin, un cookie est stocké **chez le client** : l'utilisateur peut le lire et le modifier, donc le serveur ne doit jamais lui faire une confiance aveugle.

► **Exercices d'application :** exercice 24 (cookies)

## IX Un peu d'histoire

**Pour les curieux** — *D'ARPANET au CERN* (facultatif)
**Fin des années 1960**, la recherche militaire américaine crée **ARPANET**, l'ancêtre d'Internet. Dans les années 1970, **Vinton Cerf** et **Robert Kahn** conçoivent **TCP/IP**, la famille de protocoles qui permet à des réseaux différents de communiquer : Internet devient possible. En **1983** apparaît le **DNS**, l'annuaire des noms.

**1989–1991**, au **CERN** (le laboratoire européen de physique, près de Genève), **Tim Berners-Lee** invente le **World Wide Web** pour que les chercheurs partagent leurs documents : il crée d'un coup le **HTML**, les **URL** et le protocole **HTTP**. Le premier site est en ligne en 1991. En **1993**, le navigateur *Mosaic* popularise le Web auprès du grand public, et en 1995 **Brendan Eich** écrit **JavaScript** en dix jours. *Trente ans plus tard, ce système imaginé pour quelques physiciens relie la planète entière.*

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** Les adresses IP (quatre **octets**) et `utf-8` viennent du chapitre *Le binaire et l'écriture des nombres* ; le DNS agit comme un **dictionnaire** (*Les types construits*) ; les serveurs sont des machines comme celles d'*Architecture des ordinateurs et systèmes d'exploitation*.
- **En Terminale.** *Réseaux* détaillera comment les routeurs choisissent le chemin des paquets ; le DOM deviendra un **arbre** (*Les arbres*) et le `https` s'expliquera en *Cryptographie*.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Internet vs Web | Internet = le réseau ; le Web = un service (pages + liens) dessus |
| Protocole | ensemble de règles pour communiquer (ex. HTTP, TCP/IP) |
| Adresse IP | « numéro » d'une machine sur le réseau ; publique (unique sur Internet) ou privée (`192.168.x.x`) |
| Paquets | message découpé, routé indépendamment, réassemblé à l'arrivée |
| Encapsulation | chaque couche ajoute son en-tête : HTTP ⊂ TCP (ports) ⊂ IP (adresses) ⊂ trame (MAC) |
| Réseau local | commutateurs, Wi-Fi, serveurs, pare-feu, routeur d'accès (box) |
| Routeur | aiguille les paquets vers la destination ; le DNS traduit nom → IP |
| Client / serveur | le client *demande*, le serveur *fournit* ; P2P = chacun les deux |
| Fiabilité (bit alterné) | **accusé de réception** + renvoi si perte ; un bit `0/1` évite les **doublons** |
| URL | `protocole://domaine/chemin` ; `https` = sécurisé (chiffré) |
| HTTP | requête (`GET`/`POST`) → réponse (code `200`, `304`, `404`…) |
| Une page | souvent des *dizaines* de requêtes (HTML, CSS, JS, images…) |
| Outils dev | `F12`, onglet *Réseau* : voir les requêtes, méthodes, codes, tailles |
| HTML | *fond* : structure en balises ; `id` (unique) / `class` (partagée) ; arbre (DOM) |
| CSS | *forme* : sélecteurs `balise` / `#id` / `.classe` + `{ propriété: valeur; }` |
| Flexbox | `display:flex` sur le *conteneur* : aligne les enfants en ligne (`gap`, `justify-content`) |
| JavaScript | *comportement* : événement (`click`…) → `querySelector` + `.value`/DOM, côté client |
| Formulaire | `<form action method>` ; `POST` pour envoyer/du sensible |
| Cookie | `Set-Cookie` : *mémorisé* par le client, *renvoyé* au serveur à chaque requête vers ce site |

### Retour à la situation de départ

1. Le navigateur (le **client**) obtient du **DNS** l'adresse IP du serveur, puis lui envoie une **requête HTTP** `GET`, découpée en **paquets** aiguillés par des routeurs (§ II à IV). Le serveur répond (code `200`) avec le HTML, et le navigateur demande ensuite les feuilles CSS, les scripts et les images avant d'afficher la page (§ IV).
2. Le récepteur renvoie un **accusé de réception** ; sans accusé au bout d'un délai, l'émetteur renvoie le paquet, et un **bit alterné** `0/1` permet au récepteur de reconnaître un doublon (§ II).
3. Par un **cookie** : le serveur demande au navigateur de mémoriser une petite information (`Set-Cookie`), que le navigateur lui renvoie à chaque requête vers ce site (§ VIII).

## Erreurs fréquentes

- **Confondre Internet et le Web** : l'un est le réseau, l'autre un service qui roule dessus.
- **Croire que HTML est un langage de programmation** : c'est un langage de *balisage* (structure).
- **Oublier de fermer une balise** (`</p>`) ou les emboîter en les croisant.
- **Mélanger les rôles** HTML / CSS / JS (fond / forme / comportement).
- **Confondre `id` et `class`** : un `id` est *unique* dans la page, une `class` est *partagée* ; en CSS, `#` pour l'id, `.` pour la classe.
- **Envoyer un mot de passe en `GET`** : il finit visible dans l'URL et l'historique.
- **Croire qu'une vérification côté client (JS) est sûre** : l'utilisateur peut la contourner ; le serveur doit re-vérifier.
- **Croire qu'afficher une page ne demande qu'une seule requête** : il en faut souvent des dizaines (CSS, JS, images, polices…).
- **Mettre `display:flex` sur un enfant** au lieu du **conteneur** : c'est le conteneur qui devient flexible, ses enfants s'alignent.

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

- ☐ expliquer la différence entre Internet et le Web, et ce qu'est un protocole ;
- ☐ dire à quoi servent une adresse IP, un paquet, un routeur, le DNS ;
- ☐ expliquer l'**encapsulation** (ce que chaque couche ajoute) et le rôle des constituants d'un **réseau local** ;
- ☐ **dérouler** le protocole du **bit alterné** (accusé de réception, renvoi, doublon) sur un scénario de perte ;
- ☐ **décomposer** une URL et reconnaître une page sécurisée ;
- ☐ distinguer ce qui relève du **client** et du **serveur**, et `GET` de `POST` ;
- ☐ **observer** les requêtes HTTP échangées avec les **outils de développement** (`F12`, onglet Réseau) ;
- ☐ écrire et analyser une page **HTML** simple (balises, `id`/`class`) et sa mise en forme **CSS** (sélecteurs `balise`, `#id`, `.classe`) ;
- ☐ **disposer** des éléments côte à côte avec **Flexbox** (`display:flex`) [ au-delà du programme ] ;
- ☐ rendre une page interactive : réagir à un **événement**, **sélectionner** un élément (`querySelector`), **lire un champ** (`.value`) et **modifier le DOM** ;
- ☐ assembler HTML + CSS + JavaScript en une **petite page interactive complète**, qui répond à un **cahier des charges** donné ;
- ☐ écrire un **formulaire** et choisir sa méthode d'envoi ;
- ☐ distinguer ce qu'un **cookie** fait **mémoriser par le client** et ce qui est **retransmis au serveur**.

# EXERCICES

## Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À taper et tester* dans un navigateur (fichier `.html` ouvert localement, ou un éditeur en ligne).
- Rôles à ne pas confondre : **HTML** = le fond (structure) ; **CSS** = la forme ; **JavaScript** = le comportement.

## Réseaux : comment les machines se parlent ↑cours

★ **Exercice 1** *Internet ou Web ?* ▶ corrigé p. 669 • essentiel

Pour chaque phrase, dire si elle concerne **Internet** (le réseau) ou **le Web** (le service).

a) consulter une page de cours avec des liens hypertextes ;
b) envoyer un courriel ;
c) l'ensemble des câbles, fibres et routeurs qui relient les machines ;
d) cliquer d'article en article sur une encyclopédie en ligne.

▷ coup de pouce

★ **Exercice 2** *Le bon mot* ▶ corrigé p. 669

Compléter avec : *protocole* / *adresse IP* / *paquet* / *routeur* / *DNS*.

a) Le … traduit un nom de domaine en ….
b) Un message est découpé en … avant de circuler.
c) Un … aiguille ces morceaux vers leur destination.
d) HTTP est un … : un ensemble de règles de communication.

▷ coup de pouce

★ **Exercice 3** *Client-serveur ou pair-à-pair ?* ▶ corrigé p. 669 • essentiel

a) Décrire, avec les mots *demande* et *fournit*, le modèle client-serveur.
b) Dans un logiciel de partage de fichiers en *pair-à-pair*, quel(s) rôle(s) joue votre machine ?
c) Quand vous consultez un site web, votre ordinateur est-il plutôt client ou serveur ? Pourquoi ?

▷ coup de pouce

★★ **Exercice 4** *Le voyage d'un paquet* ▶ corrigé p. 669

Un paquet part de votre machine (IP `192.168.1.10`) vers un serveur (IP `193.51.208.14`).

a) Quelles adresses le paquet doit-il porter pour arriver *et* pour qu'on puisse lui répondre ?
b) Deux paquets d'un même message peuvent-ils suivre des chemins différents ? Que se passe-t-il si l'un se perd ?

▷ coup de pouce

★★ **Exercice 5** *Dérouler le protocole du bit alterné* ▶ corrigé p. 669 • essentiel

Alice envoie à Bob quatre paquets, étiquetés successivement `0`, `1`, `0`, `1`. Après chaque paquet reçu, Bob renvoie un accusé de réception (ACK). Sans ACK au bout du délai, Alice renvoie le *même* paquet.

a) Rappeler à quoi sert le bit `0/1` qui étiquette chaque paquet.
b) **Scénario 1.** Tout arrive normalement. Combien de paquets et combien d'accusés circulent en tout ?
c) **Scénario 2.** Le **2e paquet** (étiquette `1`) se perd en route. Décrire, étape par étape, ce que fait Alice, puis Bob. Combien de fois le 2e paquet est-il finalement envoyé ?
d) **Scénario 3.** Cette fois le 2e paquet arrive bien, mais c'est son **accusé qui se perd**. Alice renvoie donc le 2e paquet (étiquette `1`). Comment Bob comprend-il que c'est un **doublon**, et que fait-il alors ? ▷ coup de pouce

## URL et protocole HTTP ↑ cours

★ **Exercice 6** *Décomposer une URL* ▶ corrigé p. 670 • essentiel

On donne l'URL `https://www.lycee.mc/nsi/web/cours.html`

a) Quel est le **protocole** ? La communication est-elle sécurisée ? À quoi le voit-on ?
b) Quel est le **nom de domaine** du serveur ?
c) Quel est le **chemin** de la ressource sur le serveur ? ▷ coup de pouce

★ **Exercice 7** *Les codes de réponse HTTP* ▶ corrigé p. 670

Associer chaque code à la situation correspondante : `200` / `304` / `403` / `404` / `500`.

a) la page demandée existe et le serveur l'envoie ;
b) l'adresse contient une faute de frappe : la page n'existe pas ;
c) le programme du serveur a planté pendant la préparation de la page ;
d) la page existe, mais vous n'avez pas le droit de la consulter ;
e) l'image n'a pas changé depuis votre dernière visite : le navigateur garde sa copie. ▷ coup de pouce

★★ **Exercice 8** *Lire un échange HTTP* ▶ corrigé p. 670

```
1 GET /nsi/web/cours.html HTTP/1.1
2
3 HTTP/1.1 404 Not Found
4 Content-Type: text/html
```

a) Qui envoie la première ligne ? Qui envoie les deux lignes suivantes ?
b) Quelle est la **méthode** de la requête ?
c) Que signifie le code **404** ? La ressource a-t-elle été trouvée ? Que renverrait le serveur en cas de succès ? ▷ coup de pouce

★★ **Exercice 9** `GET` *ou* `POST` *?* ▶ corrigé p. 670

Pour chaque situation, choisir la méthode la plus adaptée et **justifier**.

a) afficher la page d'accueil d'un site ;
b) envoyer un identifiant et un mot de passe pour se connecter ;
c) lancer une recherche « chat » qu'on aimerait pouvoir mettre en favori. ▷ coup de pouce

## HTML et CSS ↑ cours

★ **Exercice 10** *À chaque balise son rôle* ▶ corrigé p. 670

Associer chaque balise à son rôle : `<h1>` / `<p>` / `<a>` / `<img>` / `<ul>` / `<li>`.

a) un lien hypertexte ;
b) un élément d'une liste ;
c) un titre principal ;
d) une image ;
e) un paragraphe ;
f) une liste à puces.

Laquelle de ces balises s'écrit **sans** balise fermante ? ▷ coup de pouce

★ **Exercice 11** *Lire du HTML* ▶ corrigé p. 670 • essentiel

```
1 <body>
2   <h1>Recettes</h1>
3   <p>Voici ma recette <strong>preferee</strong>.</p>
4   <a href="dessert.html">Voir les desserts</a>
5 </body>
```

a) Nommer la balise du titre, celle du paragraphe, celle du lien.
b) Vers quelle page mène le lien ? Est-ce un chemin relatif ou absolu ?
c) Dessiner l'**arbre** (DOM) de ce fragment. ▷ coup de pouce

★★ **Exercice 12** *Corriger le HTML* ▶ corrigé p. 671

Ce fragment contient deux erreurs de balises. Les repérer et les corriger.

```
1 <p>Un paragraphe <strong>en gras</p></strong>
2 <h2>Un sous-titre
```

▷ coup de pouce

★★ **Exercice 13** ▶ *Ma première page* ▶ corrigé p. 671

Écrire une page HTML complète (avec `<!doctype html>`, `<head>` et `<body>`) affichant un titre « Mon site NSI », un paragraphe de présentation, et un lien vers `eduscol.education.fr`. L'ouvrir dans un navigateur. ▷ coup de pouce

★★ **Exercice 14** *Relier le CSS* ▶ corrigé p. 671 • essentiel

On donne le CSS :

```
1 h1 { color: darkblue; text-align: center; }
2 p  { font-size: 18px; }
3 #avertissement { color: red; }
```

a) Quelle sera l'apparence des titres `<h1>` ?
b) Quel élément de la page sera affiché en rouge ? (Sa balise peut-elle se déduire du CSS seul ?)
c) On écrit ces règles dans un fichier `.css` à part plutôt que dans le HTML. Quel principe cela illustre-t-il, et quel en est l'intérêt pour un site de plusieurs pages ? ▷ coup de pouce

★★ **Exercice 15** `id` *ou* `class` *?* ▶ corrigé p. 671

On donne ce fragment :

```
1 <h1 id="titre">Menu</h1>
2 <p class="plat">Pizza</p>
3 <p class="plat">Salade</p>
```

a) Pourquoi `titre` est-il un `id` et `plat` une `class` ? (penser à *unique / partagé*)
b) Écrire le sélecteur CSS qui met **le titre** en bleu.
c) Écrire le sélecteur CSS qui met **les deux plats** en vert, *d'un seul coup.* ▷ coup de pouce

★★ **Exercice 16** ▶ *Aligner avec Flexbox* ▶ corrigé p. 671

[ au-delà du programme ] Écrire une page contenant une barre de navigation : un `<div class="barre">` qui contient trois liens (`<a>`) « Accueil », « Cours », « Contact ».

a) Dans le CSS, écrire la règle `.barre` qui aligne les trois liens **horizontalement** avec un espace de `15px` entre eux.
b) Les répartir ensuite avec `justify-content: space-between`. Que change cette valeur ?
c) *Piège :* faut-il mettre `display:flex` sur le `<div>` ou sur les `<a>` ? Pourquoi ? ▷ coup de pouce

## JavaScript : rendre la page vivante ↑ cours

★ **Exercice 17** *Prévoir l'effet* ▶ corrigé p. 672 • essentiel

```
1 <p id="message">Bonjour</p>
2 <button onclick="saluer()">Clique</button>
```

```
1 function saluer() {
2     let e = document.querySelector("#message");
3     e.innerHTML = "Bonsoir !";
4 }
```

a) Que se passe-t-il quand on clique sur le bouton ?
b) À quel **événement** réagit-on ? Quel rôle joue `querySelector` ?
c) Ce traitement s'exécute-t-il côté client ou côté serveur ? ▷ coup de pouce

★★★ **Exercice 18** ▶ *Un compteur de clics* ▶ corrigé p. 672 ≈ 15 min

Sur une page contenant `<p id="score">0</p>` et un bouton, écrire une fonction JavaScript qui, à chaque clic, **augmente de 1** le nombre affiché. ▷ coup de pouce ▷ début de solution

★★ **Exercice 19** ▶ *Additionner deux nombres* ▶ corrigé p. 672

Une page contient deux champs et un bouton :

```
1 <input type="number" id="a">
2 <input type="number" id="b">
3 <button id="calc">Additionner</button>
4 <p id="resultat"></p>
```

a) Écrire la fonction `additionner()` qui **lit** les deux champs avec `.value`, en fait la somme (`parseInt(...)`) et l'affiche dans le paragraphe `#resultat`.
b) Au lieu de `onclick` dans le HTML, **brancher** le bouton depuis le JavaScript avec `addEventListener`.
c) Ce calcul a-t-il besoin du serveur ? ▷ coup de pouce ▷ début de solution

★★★ **Exercice 20** ▶ *Mini-projet : un thème clair/sombre* ▶ corrigé p. 672 ≈ 30 min

Construire une page **complète** (HTML + CSS + JS dans un seul fichier) avec un titre, un paragraphe, et un bouton « Changer de thème ».

a) En CSS, définir une classe `.sombre` qui met le fond en noir et le texte en blanc (`background` et `color`).

b) En JavaScript, au clic sur le bouton, **ajouter ou retirer** cette classe sur le `<body>`, avec l'instruction :

```
document.body.classList.toggle("sombre")
```

c) *Bonus :* placer le bouton et un second bouton « À propos » dans une barre `display:flex` alignée à droite (`justify-content: flex-end`). ▷ coup de pouce ▷ début de solution

★★ **Exercice 21** *Client ou serveur ?* ▶ corrigé p. 673

a) Pour chaque tâche, dire si elle est plutôt exécutée **côté client** (dans le navigateur, en JavaScript) ou **côté serveur** (sur la machine distante) :

1. afficher ou masquer un menu quand on clique dessus ;
2. changer la couleur d'un bouton au survol de la souris ;
3. vérifier qu'un champ contient bien un « @ » *avant* d'envoyer un formulaire ;
4. vérifier qu'un mot de passe est correct pour connecter l'utilisateur ;
5. consulter une base de données pour afficher les résultats d'une recherche ;
6. enregistrer définitivement une commande d'achat.

b) Un site vérifie **en JavaScript** qu'un code promo est valide avant d'accorder une réduction. Un élève affirme : « puisque c'est vérifié, le prix payé est forcément juste. » Expliquer **pourquoi le serveur doit refaire la vérification** de son côté. ▷ coup de pouce

c) Le traitement côté serveur s'écrit dans un *autre* langage que celui de l'interaction dans la page. Citer un exemple de langage côté serveur, et le langage utilisé côté client.



## Les formulaires ↑ cours

★★ **Exercice 22** *Lire un formulaire* ▶ corrigé p. 673 • essentiel

```
<form action="/inscription" method="post">
    <input type="text" name="prenom">
    <input type="email" name="courriel">
    <button type="submit">Envoyer</button>
</form>
```

a) Vers quelle adresse les données sont-elles envoyées ? Avec quelle méthode ?

b) Sous quels noms les deux valeurs arriveront-elles au serveur ?

c) Avec cette méthode, les données apparaissent-elles dans l'URL ? ▷ coup de pouce

★★★ **Exercice 23** ▶ *Un formulaire de recherche* ▶ corrigé p. 673 ≈ 20 min

Le site `https://www.recettes.mc` veut un formulaire de recherche, envoyé à l'adresse `/recherche`.

a) Écrire ce formulaire : un champ texte nommé `q` (le mot cherché), un champ texte nommé `tri`, et un bouton d'envoi. Choisir la méthode, sachant qu'on veut pouvoir mettre une recherche en favori.

b) Un visiteur tape « tarte » dans `q` et « prix » dans `tri`, puis envoie. Écrire l'URL complète demandée au serveur.

c) On lit dans l'historique : `https://www.recettes.mc/recherche?q=soupe&tri=date`. Qu'avait-on saisi dans chaque champ ?
d) Le site ajoute un formulaire de connexion avec un mot de passe. Peut-on reprendre la même méthode ? Pourquoi ?

▷ coup de pouce ▷ début de solution

★★ **Exercice 24** *Ce que retient le navigateur : les cookies* ▶ corrigé p. 674

À la première visite d'un internaute, le serveur `https://www.recettes.mc` répond :

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Set-Cookie: theme=sombre; Max-Age=604800
Set-Cookie: panier=tarte-citron
```

a) Ces deux informations sont-elles mémorisées dans le client ou sur le serveur ?
b) L'internaute clique ensuite sur un lien vers la page `/desserts.html` du même site. Écrire la première ligne de la requête envoyée, puis l'en-tête qui retransmet les cookies.
c) Combien de temps chaque cookie est-il conservé ?
d) L'internaute se rend ensuite sur `https://www.meteo.mc`. Le navigateur envoie-t-il ces cookies à ce site ?
e) Un autre site marchand range le prix à payer dans un cookie `prix=45`, et son serveur facture le prix qu'il lit dans ce cookie. Pourquoi est-ce une mauvaise idée ?

▷ coup de pouce ▷ début de solution

★★ **Exercice 25** *Le voyage d'un clic (synthèse)* ▶ corrigé p. 674

Remettre dans l'ordre les étapes qui vont de la saisie d'une URL à l'affichage de la page.

1. le navigateur affiche la page en assemblant HTML, CSS et JavaScript ;
2. le serveur renvoie une réponse HTTP contenant le HTML ;
3. le DNS traduit le nom de domaine en adresse IP ;
4. le navigateur envoie une requête HTTP `GET` au serveur ;
5. les paquets traversent des routeurs jusqu'au serveur.

▷ coup de pouce

★★ **Exercice 26** `GET` *et* `POST` *selon un assistant d'IA* ▶ corrigé p. 674

Un élève demande à un assistant d'IA : « Quelle est la différence entre les méthodes `GET` et `POST` ? » Voici la réponse obtenue.

« Ce sont les deux méthodes principales du protocole HTTP. `GET` sert à **demander** une ressource, par exemple afficher une page : c'est la méthode utilisée quand on tape une URL dans le navigateur. `POST` sert à **envoyer** des données au serveur pour qu'il les traite, typiquement celles d'un formulaire. Dans les deux cas, le navigateur (le client) envoie une requête et le serveur répond avec un code de statut (`200`, `404`, etc.). La différence concrète pour un formulaire : avec `GET`, les données sont placées dans le corps de la requête et n'apparaissent pas dans l'URL ; avec `POST`, elles sont ajoutées à la fin de l'URL après un `?`. »

a) La réponse est-elle correcte ? Vérifier avec le cours, puis en soumettant un formulaire `method="get"` (par exemple une recherche sur un moteur de recherche) et en observant l'URL obtenue.
b) Localiser et corriger l'erreur.
c) Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce

## Entraînement ↑cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

### ★ Exercice 27 *Lire des URL* ▶ corrigé p. 674 • essentiel

Pour chaque URL, donner le **protocole**, le **nom de domaine**, le **chemin** et, s'il y en a, les **paramètres** (noms et valeurs).

a) `http://www.lycee.mc/index.html`
b) `https://fr.wikipedia.org/wiki/Alan_Turing`
c) `https://www.recettes.mc/recherche?q=soupe&tri=date`

▷ coup de pouce

### ★ Exercice 28 *Formulaires : `GET`, `POST` et l'URL* ▶ corrigé p. 674 • essentiel

Les formulaires ci-dessous sont sur le site `https://www.site.mc`. Pour chacun, l'utilisateur remplit les champs comme indiqué, puis envoie. Donner la méthode, l'URL demandée au serveur, et dire si les valeurs saisies sont visibles dans cette URL.

a) on saisit « Nice » :

```html
<form action="/meteo" method="get">
  <input type="text" name="ville">
  <button type="submit">OK</button>
</form>
```

b) on saisit « ada » et « 1815 » :

```html
<form action="/connexion" method="post">
  <input type="text" name="login">
  <input type="password" name="mdp">
  <button type="submit">OK</button>
</form>
```

c) on saisit « velo » et « 100 » :

```html
<form action="/annonces" method="get">
  <input type="text" name="cat">
  <input type="number" name="prix_max">
  <button type="submit">OK</button>
</form>
```

d) Dans l'autre sens : après l'envoi d'un formulaire, la barre d'adresse affiche

`https://www.site.mc/films?genre=comedie&annee=2024`

Quelles étaient la méthode, l'adresse `action`, les noms des champs et les valeurs saisies ?

▷ coup de pouce

## Réseau : encapsulation et réseau local ↑cours

### ★★ Exercice 29 *Des enveloppes dans des enveloppes* ▶ corrigé p. 675

Le portable d'Inès (adresse IP `192.168.1.23`) demande la page `/horaires.html` au serveur web `51.75.20.30`, en `https` (port `443`). Sa carte Wi-Fi a l'adresse MAC `3C:22:FB:10:4A:01` ; la box, première étape du trajet, a l'adresse MAC `F4:CA:E5:81:02:9B`.

a) Dessiner la trame envoyée par le portable sous forme de quatre cases emboîtées (comme le schéma de l'encapsulation du cours) et écrire dans chaque case les informations qu'elle contient : données HTTP, en-tête TCP, en-tête IP, en-tête MAC.
b) Un routeur, au milieu du trajet, reçoit cette trame. Quelle information doit-il lire pour décider où envoyer le paquet ? Dans quel en-tête la trouve-t-il ?
c) Au-delà de la box, d'un routeur à l'autre, l'en-tête MAC change à chaque saut, mais pas les adresses IP. Pourquoi ? (Qu'arrive-t-il à l'adresse source en passant par la box ?)
d) Inès passe du Wi-Fi au réseau 4G de son téléphone. Le navigateur et le serveur doivent-ils changer leur façon d'écrire la requête HTTP ? Quel avantage de l'encapsulation cela illustre-t-il ?

▷ coup de pouce

★ **Exercice 30** *Le réseau d'un lycée* ▶ corrigé p. 675

Voici les constituants d'un réseau d'établissement : un **commutateur** par salle, des **points d'accès Wi-Fi**, un **serveur** de fichiers et de comptes, un **pare-feu**, un **routeur d'accès** relié à Internet.

a) Associer à chaque constituant l'un de ces rôles : (1) relier sans fil les tablettes au réseau ; (2) relier par câble les postes d'une même salle ; (3) filtrer les échanges entre le lycée et l'extérieur ; (4) relier le réseau du lycée à Internet ; (5) stocker les documents des élèves et vérifier les mots de passe.
b) Un poste de la salle NSI ouvre `fr.wikipedia.org`. Citer, dans l'ordre, les constituants du lycée traversés par la requête avant qu'elle n'atteigne Internet.
c) Les postes du lycée ont des adresses `10.0.x.x`. Le serveur de Wikipédia voit-il cette adresse ? Expliquer.
d) Un site de jeux en ligne est inaccessible depuis le lycée, alors qu'il fonctionne à la maison. Quel constituant en est très probablement la cause ?

▷ coup de pouce

## Programmer une interface d'après un cahier des charges ↑ cours

★★★ **Exercice 31** ▶ *Une inscription à l'atelier robotique* ▶ corrigé p. 675 ≈ 40 min

Le club du lycée veut une petite page pour s'inscrire à l'atelier robotique. Programmer cette **interface homme-machine** (IHM) en respectant le **cahier des charges** suivant :

**C1.** La page affiche le titre « Inscription à l'atelier robotique ».
**C2.** Elle contient deux champs de saisie de texte : le **prénom** (`id="prenom"`) et la **classe** (`id="classe"`), chacun avec un texte indicatif (`placeholder`).
**C3.** Un bouton « S'inscrire » appelle, au clic, la fonction `inscrire()`.
**C4.** Si l'un des deux champs est vide, le paragraphe `id="message"` affiche « Remplir les deux champs. » en **rouge**, et rien d'autre ne change.
**C5.** Sinon, le message affiche, en **vert**, « Inscription enregistrée : » suivi du prénom ; une ligne « prénom (classe) », par exemple « Léa (1G2) », s'ajoute à la liste `<ul id="inscrits">` ; les deux champs sont vidés.
**C6.** Tout se passe sans recharger la page.

Rappel de syntaxe : comme en Python, mais avec des accolades, on écrit `if (condition) { ... } else { ... }` ; l'égalité se teste avec `===`, la différence avec `!==` (comme dans le cours), et le « ou » logique s'écrit `||`.

a) Lister les **composants graphiques** de cette interface et, pour chacun, l'**événement** qu'il traite (s'il en traite un).
b) Écrire le code HTML du corps de la page (exigences C1 à C3).
c) Écrire la fonction JavaScript `inscrire()` (exigences C4 à C6).
d) Proposer **trois essais** qui vérifient les exigences C4 et C5 (ce qu'on saisit, ce qu'on doit observer), puis les réaliser dans le navigateur.
e) On recharge la page : la liste des inscrits est de nouveau vide. Pourquoi ? Que faudrait-il ajouter pour que le professeur retrouve toutes les inscriptions ?

▷ coup de pouce ▷ début de solution

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 q. 4 – piste Internet est le réseau physique qui relie les machines ; le Web est *un* service qui l'utilise, fait de pages reliées par des liens hypertextes. Le courriel est un autre service. ↑énoncé

Ex. 2 q. 4 – piste Le mot « adresse IP » complète la première phrase ; pour les autres, revoir le vocabulaire de la section sur le réseau du cours. ↑énoncé

Ex. 3 q. 3 – piste Le client est celui qui demande, le serveur celui qui fournit. En pair-à-pair, une même machine peut-elle faire les deux ? ↑énoncé

Ex. 4 q. 2 – piste Penser à une lettre postale : quelles adresses écrit-on sur l'enveloppe pour qu'elle arrive, et pour qu'on puisse y répondre ? ↑énoncé

Ex. 5 q. 4 – piste Dessiner deux colonnes, Alice et Bob, et une flèche par message (paquet ou accusé), en notant l'étiquette `0` ou `1` de chaque paquet. ↑énoncé

Ex. 6 q. 3 – piste Une URL se lit de gauche à droite : le protocole avant `://`, puis le nom du serveur jusqu'au premier `/`, puis le chemin. ↑énoncé

Ex. 7 q. 5 – piste Les codes en `2..` signalent un succès, en `3..` une redirection ou une copie déjà à jour, en `4..` une erreur du client, en `5..` une erreur du serveur. ↑énoncé

Ex. 8 q. 3 – piste Une requête commence par une méthode ; une réponse commence par la version de HTTP, suivie d'un code. ↑énoncé

Ex. 9 q. 3 – piste Pour chaque cas, se demander : les données peuvent-elles être visibles (et mémorisées) dans l'URL, ou doivent-elles rester cachées ? ↑énoncé

Ex. 10 – piste `h` comme *heading* (titre), `p` comme paragraphe, `a` comme ancre, `ul` comme liste non ordonnée, `li` comme élément de liste. Quelle balise n'entoure aucun texte ? ↑énoncé

Ex. 11 q. 3 – piste Dans l'arbre, chaque balise est un nœud ; les balises placées *entre* l'ouverture et la fermeture d'une autre en sont les enfants. ↑énoncé

Ex. 12 – piste Vérifier que chaque balise ouverte est fermée, et que les balises s'emboîtent sans se croiser : la dernière ouverte est la première fermée. ↑énoncé

Ex. 13 – piste Repartir du squelette de page complète du cours. Un lien vers un autre site demande une adresse complète, qui commence par `https://`, dans `href`. ↑énoncé

Ex. 14 q. 3 – piste Le symbole `#` devant un nom désigne l'élément qui porte cet `id`. Pour c), relire les rôles des trois langages dans le mode d'emploi. ↑énoncé

Ex. 15 q. 3 – piste Revoir les trois sortes de sélecteurs du cours : par balise, par `id`, par classe, et le symbole qui précède chacun. ↑énoncé

Ex. 16 q. 3 – piste Partir de l'exemple Flexbox du cours : quelles propriétés y figurent, et dans quelle règle sont-elles écrites ? ↑énoncé

Ex. 17 q. 3 – piste Repérer dans le HTML l'attribut qui relie le bouton à la fonction, puis lire la fonction ligne par ligne : quelle balise est sélectionnée, et que devient son contenu ? ↑

Ex. 18 – piste Trois étapes : lire le nombre affiché, lui ajouter 1, réécrire le résultat dans le paragraphe. Le texte lu est une chaîne : `parseInt(e.innerHTML)` le transforme en entier. ↑énoncé

Ex. 18 – début de solution
```
function compter() {
    let e = document.querySelector("#score");
    let n = parseInt(e.innerHTML);

    ...
```
↑énoncé

Ex. 19 q. 3 – piste `.value` donne le texte saisi dans un champ : le convertir avant d'additionner, sinon `+` colle les deux textes. Pour b), revoir la forme `bouton.addEventListener("click", ...)` du cours. ↑énoncé

Ex. 19 q. 3 – début de solution
```
function additionner() {
    let a = parseInt(document.querySelector("#a").value);
    let b = ...
```
↑énoncé

Ex. 20 q. 3 – piste Construire dans l'ordre : la page HTML, puis la règle `.sombre` dans un `<style>` du `<head>`, puis une fonction appelée au clic. ↑énoncé

Ex. 20 q. 3 – début de solution
```
<button onclick="changer()">Changer de thème</button>
function changer() {

    ...
```
↑énoncé

Ex. 21 q. 2 – piste Qui peut voir et modifier le JavaScript d'une page ? ↑énoncé

Ex. 22 q. 3 – piste Les attributs `action`, `method` et `name` répondent chacun à l'une des questions. ↑énoncé

Ex. 23 – piste Revoir dans le cours la forme d'une URL avec `GET` : un `?`, puis des paires `clef=valeur` séparées par `&`. ↑énoncé

Ex. 23 – début de solution
```
<form action="/recherche" method="...">
    <input type="text" name="q">

    ...
```
↑énoncé

Ex. 24 – piste `Max-Age` est une durée en secondes ; une journée compte $24 \times 3600$ secondes. ↑énoncé

Ex. 24 – début de solution Où le cookie est-il rangé, et qui peut donc le lire et le modifier ? ↑énoncé

Ex. 25 q. 5 – piste Avant de pouvoir contacter le serveur, le navigateur doit connaître son adresse IP. ↑

Ex. 26 q. 3 – piste Confronter chaque phrase à la partie « Les formulaires » du cours, en particulier ce qui est dit de l'URL. ↑énoncé

Ex. 27 – piste Lire de gauche à droite : avant `://`, le protocole ; jusqu'au premier `/`, le domaine ; puis le chemin, jusqu'à un éventuel `?`. Après le `?`, des paires `nom=valeur` séparées par `&`. ↑énoncé

Ex. 28 – piste Avec `get`, l'URL est l'adresse `action`, suivie d'un `?` puis des paires `name=valeur` séparées par `&`. Avec `post`, l'URL est seulement l'adresse `action` : les valeurs voyagent dans le corps de la requête. ↑énoncé

Ex. 29 – piste Chaque couche n'ajoute que ce dont elle a besoin : l'application, la requête ; le transport, les ports ; le réseau, les adresses IP ; la liaison, les adresses MAC du « saut » en cours. Revoir la règle « L'encapsulation » du cours. ↑énoncé

Ex. 30 – piste Revoir l'encadré « Le réseau du lycée » et la remarque sur les adresses privées et publiques. ↑énoncé

Ex. 31 – piste Repartir de la page « Ma liste de courses » du cours : un champ, un bouton, une liste `<ul>` que l'on complète avec `innerHTML`. Il y a ici deux champs à lire, et la couleur du message se change avec `.style.color`. ↑énoncé

Ex. 31 – début de solution
```
function inscrire() {
    let prenom = document.querySelector("#prenom").value;
    let classe = ...
    let message = document.querySelector("#message");
    if (prenom === "" || classe === "") {

        ...
```
↑énoncé

# TP ET PROJETS

## TP 1 Mon premier site, pas à pas

▶ corrigé p. 676

*le site du Club Astro : structurer, mettre en forme, rendre interactif*

Durée : 2 h (une séance) · Seul, sur machine

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p10-tp-site-web-guide` — archive .zip

### But du TP

Construire, à partir d'un squelette fourni, le petit site du club d'astronomie du lycée : deux pages reliées par une barre de navigation, mises en forme par une feuille de style commune, puis rendues interactives en JavaScript (un compteur de « j'aime », une inscription qui vérifie ce qu'on a saisi, un mode nuit). Les trois parties suivent les trois premiers jalons du **projet « Créer un site web interactif »** : ce sont exactement les gestes que vous referez ensuite, seul, sur votre propre site.

### Consignes

- Fichiers à télécharger (lien ci-dessus) : le dossier `monsite/`, qui contient `index.html` (la page d'accueil), `style.css` (la feuille de style) et `script.js` (le programme JavaScript). Ne pas renommer ces fichiers.
- Outils : un éditeur de texte (pas un traitement de texte) et un navigateur. Après chaque modification, **enregistrer** le fichier, puis **actualiser** la page dans le navigateur (touche `F5`).
- Les commentaires `TODO` du squelette indiquent où écrire. Chaque partie commence par le rappel des outils utiles, avec le paragraphe du cours où les retrouver. Les réponses aux questions (sans ▶ ) s'écrivent sur le cahier.
- ▶ *À écrire et tester dans le navigateur.* Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 La structure HTML (jalon J1)

Environ 40 minutes. On écrit le **fond** de la page : les titres, les textes, la liste, le tableau et les liens.

**Outils de la partie 1 — les balises (cours, § V « HTML : structurer la page »)**

- `<head>` contient les informations *sur* la page (`<title>`, le titre de l'onglet) ; `<body>` contient ce qui s'affiche.
- `<h1>`, `<h2>` : titres ; `<p>` : paragraphe ; `<strong>` : texte important ; `<ul>` : liste à puces, dont chaque élément est un `<li>`.
- `<a href="...">texte</a>` : lien hypertexte. Pour une page du même dossier, le nom du fichier suffit (`infos.html` : c'est un chemin **relatif**, comme au chapitre précédent) ; pour un autre site, on écrit l'adresse complète, qui commence par `https://`.
- `<div>` : conteneur qui regroupe des éléments ; attributs `id="..."` (identifiant *unique* dans la page) et `class="..."` (étiquette *partagée*).
- Tableau : `<table>` contient des lignes `<tr>` ; chaque ligne contient des cases `<th>` (case de titre) ou `<td>` (case ordinaire). Le squelette en donne un modèle.
- `<!-- ... -->` : commentaire, ignoré par le navigateur. Les consignes `TODO` du squelette en sont.

Chaque balise ouverte se referme, et les balises s'emboîtent sans se croiser (sauf les balises auto-fermantes comme `<meta>`, `<link>`, `<input>`).

### ★ Exercice 1 — *Découvrir le squelette*

1. Ouvrir `index.html` dans le navigateur (double-clic), puis dans l'éditeur de texte. Quels textes le navigateur affiche-t-il ? Pourquoi les lignes `TODO` n'apparaissent-elles pas ?
2. Combien de blocs `<div class="carte">` la page contient-elle ? Pourquoi est-ce une `class` et pas un `id` ?
3. Repérer le tableau du planning : combien de lignes compte-t-il pour l'instant ? Combien de cases par ligne ?

### ★ Exercice 2 — *Titres, annonce et présentation*▶

1. Remplacer le titre de l'onglet `Mon site` par `Club Astro - Accueil`. Où ce titre s'affiche-t-il ?
2. Écrire le titre principal `<h1>` « Club Astro ».
3. Juste en dessous, écrire un paragraphe d'identifiant `annonce` qui contient : « Prochaine nuit d'observation : vendredi à 21 h, sur le toit du lycée. »
4. Dans la carte « Qui sommes-nous ? », écrire un paragraphe de deux ou trois phrases qui présente le club ; mettre quelques mots en valeur avec `<strong>`.
5. Dans la carte « Ce que l'on observe », écrire une liste à puces d'au moins trois objets du ciel.

### ★ Exercice 3 — *Compléter le planning*▶

Ajouter au tableau, sur le modèle de la ligne du lundi, une ligne « Mercredi, 14 h, Fabriquer une lunette » et une ligne « Vendredi, 21 h, Nuit d'observation ». Pourquoi la première ligne utilise-t-elle des `<th>` et les autres des `<td>` ?

### ★★ Exercice 4 — *La barre de navigation et une deuxième page*▶

1. À l'endroit indiqué, sous l'annonce, écrire un bloc `<div class="barre">` qui contient trois liens : « Accueil » vers `index.html`, « Infos pratiques » vers `infos.html`, « En savoir plus » vers `https://fr.wikipedia.org/wiki/Astronomie`. ▷ coup de pouce
2. Lesquels de ces liens utilisent un chemin relatif ? Lequel une adresse complète ? Pourquoi le lien « Infos pratiques » ne mène-t-il encore nulle part ?
3. Créer la page `infos.html` : dans l'éditeur, enregistrer une copie de `index.html` sous ce nom, **dans le même dossier**. Dans cette copie, changer le titre de l'onglet en `Club Astro -`

`Infos pratiques` ; garder le `<h1>`, la barre et la ligne `<script ...>` de la fin ; remplacer l'annonce et toutes les cartes par une seule carte « Infos pratiques » (lieu du rendez-vous, matériel à apporter).

4. Vérifier qu'on passe d'une page à l'autre, dans les deux sens, avec la barre.

**Point d'étape ✓** La page d'accueil affiche son titre, l'annonce, la barre, la présentation, la liste et un planning de quatre lignes ; la barre permet de passer d'une page à l'autre. C'est le jalon J1 du projet : des pages qui existent et sont reliées.

## 2 La mise en forme CSS (jalon J2)

Environ 35 minutes. On écrit maintenant la **forme**, dans un fichier à part : le HTML ne change presque plus.

**Outils de la partie 2 — les règles CSS (cours, § VI « CSS : mettre en forme »)**

- Relier la feuille de style : `<link rel="stylesheet" href="style.css">`, dans le `<head>` de **chaque** page.
- Une règle : `sélecteur { propriété: valeur; }`. Trois sélecteurs : par nom de balise (`h1`), par classe (`.barre`), par identifiant (`#annonce`).
- Propriétés : `color`, `background-color`, `font-family`, `font-size`, `text-align`, `margin` (marge extérieure), `padding` (marge intérieure), `border` (par exemple `1px solid gray`). Tailles en `px` ; couleurs par leur nom anglais (`gold`, `white`, `midnightblue`, `aliceblue`…).
- Flexbox (§ VI A) : `display: flex` sur le **conteneur** aligne ses enfants en ligne ; `gap` règle l'espace entre eux, `justify-content` leur répartition horizontale, `align-items` leur alignement vertical.



★ **Exercice 5** *Relier la feuille de style*▶

1. Ouvrir `style.css`. Une règle y est fournie comme modèle :

```
.carte {
  border: 2px solid gray;
  padding: 10px 20px;
  margin-bottom: 15px;
}
```

   Quelles balises vise-t-elle ? Que fait chacune de ses trois propriétés ?
2. Relier `style.css` à `index.html`, à l'endroit indiqué. Actualiser : qu'est-ce qui change ?
3. Relier aussi `style.css` à `infos.html`. Quel est l'intérêt d'une seule feuille de style pour tout le site ?

★★ **Exercice 6** *Mettre en forme la page*▶

Dans `style.css`, à la place des commentaires `TODO J2`, écrire les règles suivantes. Actualiser après chacune pour en voir l'effet.

1. Pour `body` : une police sans empattement (`sans-serif`), une couleur de fond claire, une marge de `20px`. ▷ coup de pouce
2. Pour `h1` : une couleur de texte, et le texte centré.
3. Pour l'annonce : un fond doré, le texte centré, une marge intérieure de `8px`. Pourquoi faut-il ici un `#` et non un point ?

4. Pour `th` : un fond foncé et un texte blanc ; pour `td` : une bordure fine et une marge intérieure de `5px`.
5. Changer la couleur de fond dans la règle de `body`, puis ouvrir `infos.html`. Que constate-t-on ?

★★ **Exercice 7** *La barre de navigation avec Flexbox*▶

1. Écrire la règle de la classe `barre` : ses liens en ligne, séparés de `10px`, sur un fond foncé, avec une marge intérieure de `10px`. ▷ coup de pouce
2. Essayer successivement `justify-content: center`, puis `space-between`, puis `flex-end`. Décrire l'effet de chacune, et garder celle que vous préférez.
3. Les liens bleus se lisent mal sur un fond foncé : écrire une règle qui met **tous** les liens en blanc. Quel sélecteur faut-il ?

**Point d'étape ✓** Les deux pages utilisent le même `style.css`, la barre est en ligne, et vous avez écrit les trois sortes de sélecteurs : balise, classe, identifiant. C'est le jalon J2 du projet : le style s'applique, le fond et la forme sont séparés.

## 3 Les interactions JavaScript (jalon J3)

Environ 45 minutes. On écrit enfin le **comportement**, dans `script.js`. La ligne `<script src="script.js"></script>` est déjà à la fin du `<body>` : le programme est chargé après les balises qu'il utilise.

### Outils de la partie 3 — JavaScript (cours, § VII « JavaScript : rendre la page interactive »)

- Une fonction : `function aimer() { ... }` ; une variable : `let n = 0;` ; chaque instruction finit par `;` et un commentaire commence par `//`.
- Réagir à un clic : attribut `onclick="aimer()"` sur le bouton, ou, depuis le JavaScript, `bouton.addEventListener("click", aimer);` (sans parenthèses après `aimer`).
- Sélectionner un élément : `document.querySelector("#nbJaime")` (ou `document.getElementById("nbJaime")`) ; le `<body>` s'obtient directement par `document.body`.
- Modifier : `.textContent` (le contenu), `.style.color` (la couleur du texte), `.classList.toggle("nuit")` (ajoute la classe si elle est absente, la retire sinon), `.value` (le texte d'un champ). `parseInt(texte)` convertit un texte en entier (exercice 18 de la feuille).
- `"Merci " + nom` colle deux textes ; un nombre collé à un texte devient du texte.
- Si rien ne se passe : touche `F12`, onglet **Console** ; le message d'erreur indique le fichier et la ligne.

La condition s'écrit comme en Python, avec des parenthèses autour du test et des accolades autour de chaque bloc ; on compare avec `===` (égal), `!==` (différent), `<`, `<=`, `>`, `>=`.

```
if (nom === "") {
    ...
} else if (places === 0) {
    ...
} else {
    ...
}
```

```
if nom == "":
    ...
elif places == 0:
    ...
else:
    ...
```

### ★ Exercice 8 — *Le bouton « J'aime »*

1. Dans la carte « Vous aimez le club ? », écrire un bouton « J'aime » qui appelle `aimer()` au clic, puis le paragraphe :

```
<p>Nombre de j'aime : <span id="nbJaime">0</span></p>
```

2. Dans `script.js`, écrire la fonction `aimer()` : sélectionner le `<span>`, lire le nombre affiché, lui ajouter 1, réécrire le résultat. ▷ coup de pouce
3. Dans `style.css`, écrire une règle qui affiche ce nombre en plus grand (`font-size`).
4. Cliquer cinq fois, puis actualiser la page. Que devient le compteur ? Expliquer pourquoi.

### ★★ Exercice 9 — *Valider l'inscription*

1. Dans la carte « S'inscrire à la nuit d'observation », écrire, dans cet ordre : un paragraphe « Places restantes : » suivi de `<span id="places">3</span>` ; un champ de texte d'identifiant `nom` ; un bouton « S'inscrire » qui appelle `inscrire()` au clic ; un paragraphe vide d'identifiant `reponse`. ▷ coup de pouce
2. Écrire la fonction `inscrire()` : si le champ est vide, afficher en rouge, dans le paragraphe `reponse`, « Veuillez écrire votre nom. » ; sinon, afficher en vert un message qui reprend le nom saisi, par exemple « Merci Léa, à vendredi ! ». Tester les deux cas. ▷ coup de pouce
3. Le club n'a que trois places. Améliorer `inscrire()` : à chaque inscription acceptée, le nombre de places affiché diminue de 1 ; quand il vaut 0, la fonction affiche en rouge « Désolé Tom, c'est complet ! » (avec le nom saisi) au lieu d'inscrire. Tester avec quatre inscriptions de suite. ▷ début de solution
4. Après une inscription acceptée, vider le champ (comme dans la page complète de la fin du § VII).
5. Un visiteur peut-il contourner cette limite de trois places ? Où faudrait-il faire la vérification pour qu'elle soit sûre ?

### ★★ Exercice 10 — *Le mode nuit*

Les astronomes s'éclairent en rouge : cette lumière n'éblouit pas des yeux habitués à l'obscurité. On ajoute au site un mode nuit, fond noir et texte rouge.

1. Dans `style.css`, écrire une règle pour la classe `nuit` : fond noir, texte rouge (`red`, ou `tomato`, plus doux).
2. Dans la barre, **sur les deux pages**, ajouter le bouton `<button id="boutonNuit">Mode nuit</button>`, sans attribut `onclick`.
3. Dans `script.js`, écrire une fonction `basculerNuit()` qui ajoute la classe `nuit` au `<body>` si elle est absente, et la retire sinon. Puis, en dehors de toute fonction, brancher le bouton `boutonNuit` sur cette fonction avec `addEventListener`. ▷ coup de pouce
4. Passer en mode nuit, puis cliquer sur « Infos pratiques ». Le mode nuit est-il conservé ? Quel mécanisme du cours permet à un site de retenir une préférence d'une page à l'autre ?

**Point d'étape ✓** Le compteur avance, l'inscription refuse un champ vide puis un club complet, le mode nuit fonctionne sur les deux pages. Votre fonction `inscrire` lit une saisie, utilise des variables et une condition, et modifie la page : c'est la « fonction algorithmique » demandée par le projet. C'est le jalon J3.

### Outil du bonus — la boucle `while` en JavaScript

Comme en Python, avec des parenthèses autour de la condition et des accolades autour du bloc répété :

```
let i = 10;
while (i > 0) {
    ...
    i = i - 1;
}
```

```
i = 10
while i > 0:
    ...
    i = i - 1
```

★★★ **Exercice 11** *Bonus : le compte à rebours* ▶

1. Ajouter à la page d'accueil une carte « Le décollage » avec un bouton « Lancer le compte à rebours » et un paragraphe vide d'identifiant `decollage`.
2. Écrire une fonction `compteARebours()` qui construit, avec une boucle `while`, le texte `10... 9... 8... 7... 6... 5... 4... 3... 2... 1... Décollage !`, puis l'affiche dans le paragraphe.

▷ coup de pouce

3. Le texte apparaît d'un seul coup, et non seconde après seconde. Pourquoi ?

## Vers le projet

- Ce site n'est qu'un exemple : dans le projet, le thème, les pages et les interactions sont les vôtres. Vous pouvez repartir de ce dossier, en le réorganisant selon l'**arborescence imposée** : la feuille de style va dans `styles/`, le programme dans `scripts/`, les autres pages dans `contenus/`. Les chemins changent alors : depuis `index.html`, on écrit `styles/style.css` ; depuis une page de `contenus/`, `../styles/style.css` (on remonte d'un cran avec `..`, comme dans le terminal).
- Le projet demande aussi des images (`<img src="..." alt="...">`, cours § V), un formulaire avec un attribut `method` (cours § VIII, jalon J4) et au moins quatre interactions : ce TP en compte déjà trois.

## Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 q. 1 – piste Reprendre la forme du lien de l'exemple du cours (§ V) : la destination entre les guillemets de `href`, le texte cliquable entre les deux balises. ↑énoncé

Ex. 6 q. 1 – piste Chaque règle a la forme de la règle `.carte` fournie : le sélecteur, puis, entre accolades, une propriété par ligne, terminée par un point-virgule. Le premier sélecteur s'écrit simplement `body`. ↑énoncé

Ex. 7 q. 1 – piste Revoir l'exemple `.barre` du cours (§ VI A) : `display: flex` et `gap` s'écrivent dans la règle du conteneur, pas dans celle des liens. ↑énoncé

Ex. 8 q. 2 – piste C'est l'exercice 18 de la feuille : trois étapes, lire, ajouter 1, réécrire. Le texte lu est une chaîne : `parseInt` la transforme en entier. ↑énoncé

Ex. 9 q. 1 – piste L'exemple « Réagir à une saisie » du cours (§ VII B) contient déjà un champ, un bouton et un paragraphe vide. ↑énoncé

Ex. 9 q. 2 – piste Lire le champ avec `.value` dans une variable `nom`, puis tester si elle est égale à la chaîne vide `""`. La couleur se règle avec `reponse.style.color = "red";`. ↑énoncé

Ex. 9 q. 3 – début de solution

```
let zonePlaces = document.querySelector("#places");
let places = parseInt(zonePlaces.textContent);
if (nom === "") { ... }
```

```
else if (places === 0) { ... }
else { zonePlaces.textContent = places - 1; ... }
```
↑énoncé

Ex. 10 q. 3 – piste L'exercice 20 de la feuille fait la même chose avec une classe `sombre`. Pour le branchement, revoir la remarque « Deux manières de brancher un événement » du cours (§ VII A). ↑énoncé

Ex. 11 q. 2 – piste Partir d'une variable `texte` qui vaut la chaîne vide `""` ; à chaque tour, lui coller le nombre suivi de `"... "` ; après la boucle, lui coller `"Décollage !"`. ↑énoncé

## PROJET Créer un site web interactif

> **Remarque** — *De quoi s'agit-il ?*
> Vous allez concevoir, **seul**, un petit site web **interactif** en HTML, CSS et JavaScript, sur le thème de votre choix. Ce projet est le **couronnement** du chapitre sur le Web : il réunit tout ce que vous avez appris (structure, mise en forme, interactions, formulaire).
>
> **Ce qui est noté n'est pas seulement le site fini**, mais surtout ce que *vous* savez faire et expliquer : votre **soutenance orale**, un **contrôle individuel sur machine**, et votre **carnet de bord**. Lisez bien la partie « Comment vous serez évalué ».

### 1 Le cahier des charges

**Outils :** un simple **éditeur de texte** (et un navigateur pour tester). **Pas de framework** (React, Bootstrap…), pas de générateur de site : uniquement du **HTML, CSS et JavaScript** écrits par vous.

Votre site doit comporter :

- **3 à 4 pages** reliées par une **navigation claire** (un menu présent sur chaque page) ;
- des **liens hypertextes**, avec au moins un lien vers un autre site (**adresse complète**, en `https://`) et un **chemin relatif** vers une de vos pages ;
- des **images** (libres de droit, sources citées) et au moins un **tableau** ;
- une **feuille de style CSS séparée** : le fond (HTML) et la forme (CSS) dans des fichiers distincts ; elle utilise des sélecteurs de **balise**, de **classe** et d'**identifiant**, et au moins une disposition avec **Flexbox** (vue dans le TP guidé) ;
- au moins **quatre interactions JavaScript** réagissant à des **événements** (clic, survol…) et modifiant la page (le DOM) ;
- **au moins une fonction JavaScript « algorithmique »** que vous avez écrite : elle doit contenir une **variable** et une **condition** ou une **boucle** (un mini-quiz qui compte les points, un compteur avec un seuil, une vérification de formulaire…) — pas seulement un `innerHTML = "..."` ;
- un **formulaire** (avec un attribut `method`).

### Aide-mémoire JavaScript : boucle et hasard (en plus du cours)

Le cours montre la condition `if (...) { ...}`. Pour une boucle ou un tirage au hasard :

- `for (let i = 1; i <= 10; i++) { ...}` répète le bloc pour `i` allant de 1 à 10 (comme `for i in range(1, 11):` en Python) ;
- `while (n > 0) { ...}` répète tant que la condition est vraie ;
- `Math.floor(Math.random() * 6) + 1` donne un entier au hasard entre 1 et 6 (comme `randint(1, 6)`).

### Arborescence imposée

Tout le site tient dans un dossier `monsite/` organisé ainsi :

| | |
|---|---|
| `monsite/index.html` | la page d'accueil (nom **obligatoire**) |
| `monsite/contenus/` | les autres pages `.html` |
| `monsite/styles/` | les feuilles de style `.css` |
| `monsite/scripts/` | les fichiers JavaScript `.js` |
| `monsite/medias/` | les images et autres médias |

## 2 Choisir un thème

Choisissez **un** thème dans le menu ci-dessous (ou proposez le vôtre, à faire valider). Chaque thème suggère des interactions ; à vous de les enrichir.

| Thème | Interactions JavaScript possibles |
|---|---|
| Site *utile* (convertisseur, minuteur, *to-do*) | calcul en direct, ajout/suppression d'éléments, compte à rebours |
| Mini-*quiz* sur un sujet | score qui se met à jour, correction, message selon le résultat |
| Mini-*jeu* (devine le nombre, pierre-feuille-ciseaux) | tirage aléatoire, test de la réponse, compteur de parties |
| *Portfolio* / page personnelle | galerie d'images, mode clair/sombre, afficher/masquer des sections |
| Site d'un *club* ou d'une *association* | formulaire d'adhésion, agenda interactif, bouton « j'aime » |

*Contrainte de bon sens :* un thème **correct et respectueux**, sans contenu personnel sensible (le site pourra être montré en classe).

## 3 L'intelligence artificielle : mode d'emploi

### L'IA est autorisée — mais déclarée, et elle ne pense pas à votre place

Vous **pouvez** utiliser une IA (comme le web, pour chercher une technique ou débloquer un bug). **Mais** :

- vous devez **comprendre et pouvoir expliquer chaque ligne** de votre site. *Ce que vous ne savez pas expliquer à l'oral ne comptera pas* — pire, cela se retournera contre vous ;
- vous **déclarez** votre usage de l'IA dans votre **carnet de bord** : quels prompts, ce que vous avez **gardé, modifié ou rejeté**, et **pourquoi** ;

- une IA se **trompe** souvent : à vous de repérer et corriger. Une suggestion d'IA n'est pas une vérité.

En résumé :  IA **en appui** pendant le projet (chercher, déboguer, vérifier ; le code final est écrit et compris par vous) ;  **sans IA** pour le contrôle sur machine et la soutenance. Voir la charte d'usage de l'IA, au début du livre.
**L'essentiel de votre note vient de ce que *vous* savez faire seul** (voir plus bas). Un site parfait que vous ne savez pas expliquer vaut moins qu'un site modeste que vous maîtrisez.

## 4 Le déroulé : les jalons

Le projet se fait **en classe**, par étapes. À la fin de chaque séance, un point rapide avec le professeur valide votre avancée (et nourrit votre carnet de bord).

| Jalon | Objectif de la séance | Validé si… |
|---|---|---|
| J1 | structure HTML des pages + navigation | les pages existent et sont reliées |
| J2 | mise en forme CSS (fichier séparé) | le style s'applique, fond/forme séparés |
| J3 | interactions JavaScript | au moins 4 interactions fonctionnent |
| J4 | formulaire, finitions, carnet de bord | le cahier des charges est complet |

## 5 Comment vous serez évalué

L'évaluation récompense d'abord votre **compréhension**, pas seulement le résultat. Elle porte sur quatre éléments :

- la **soutenance orale** (individuelle) : expliquer votre code *et modifier une interaction en direct* ;
- un **contrôle sur machine, sans IA** : refaire seul, en classe, les gestes du projet ;
- le **carnet de bord** (dont l'usage de l'IA) : sérieux de la démarche et transparence ;
- **le site** (cahier des charges, code, navigation) : le produit fini et sa qualité.

**Remarque** — *Deux points d'attention*
**La soutenance** (5 à 10 min) : vous présentez le site, on vous demande d'**expliquer** une fonction, de **prévoir** l'effet d'une modification, et d'**ajouter une petite interaction devant le professeur**.
**Le contrôle sans IA** : en classe, sans internet ni IA, à partir d'une page fournie, vous ajouterez quelques interactions et corrigerez un bug. C'est la preuve que *vous* savez faire.

## 6 Le rendu

- Vérifiez que le site s'ouvre en double-cliquant sur `index.html`, et que **toute** la navigation fonctionne.
- **Compressez** le dossier `monsite/` (fichier `.zip`) et remettez-le selon les consignes de votre professeur, avant la date indiquée.
- Joignez votre **carnet de bord** complété.
- Citez vos **sources** (images, techniques) et n'utilisez que des médias **libres de droit**.

## 7 Critères d'évaluation

La grille détaille, critère par critère, les quatre éléments de l'évaluation présentés plus haut. Les critères du site reprennent les jalons J1 à J4 : le TP qui précède (« Mon premier site, pas à pas ») vous a fait franchir les trois premiers sur un exemple ; il s'agit maintenant de refaire ces gestes sur votre propre site.

| Critère | Ce qui est attendu |
|---|---|
| **Le site** | |
| Structure et navigation (J1) | 3 à 4 pages aux balises bien fermées et bien emboîtées ; un menu sur chaque page ; au moins un chemin relatif et un lien en adresse complète (`https://`) ; l'arborescence imposée respectée |
| Mise en forme (J2) | une feuille CSS séparée, reliée à chaque page ; des sélecteurs de balise, de classe et d'identifiant ; une disposition avec Flexbox |
| Interactions (J3) | au moins quatre interactions qui réagissent à un événement et modifient la page ; une fonction écrite par vous, avec une variable et une condition ou une boucle |
| Contenus et formulaire (J4) | des images libres de droit, sources citées ; un tableau ; un formulaire avec un attribut `method` ; le site s'ouvre depuis `index.html` |
| **Soutenance orale** | |
| Démonstration | le site fonctionne et respecte le cahier des charges |
| HTML et CSS | expliquer la structure des pages et la séparation du fond et de la forme |
| Une fonction JavaScript | l'expliquer ligne par ligne : l'événement, la sélection dans la page, la logique (condition ou boucle) |
| Modification en direct | réaliser devant le professeur la petite modification demandée |
| Recul | justifier un choix ; parler honnêtement de son usage de l'IA |
| **Contrôle sur machine, sans IA** | |
| Ajouter des interactions | sur une page fournie : brancher un événement, sélectionner un élément, modifier la page |
| Corriger un bug | repérer l'erreur (console de `F12`, relecture) et la corriger |
| **Carnet de bord** | |
| Suivi des séances | une entrée par séance : ce qui a été fait, les difficultés, les solutions trouvées |
| Usage de l'IA | chaque usage déclaré (la demande, la réponse, ce qui a été gardé, modifié ou rejeté, et pourquoi) ; le bilan final |

## ACTIVITÉ 2 Simuler un réseau

▶ corrigé p. 683

*du réseau local à Internet, sur le simulateur Filius*

Durée : 55 min · Par binômes, sur le simulateur gratuit et libre Filius

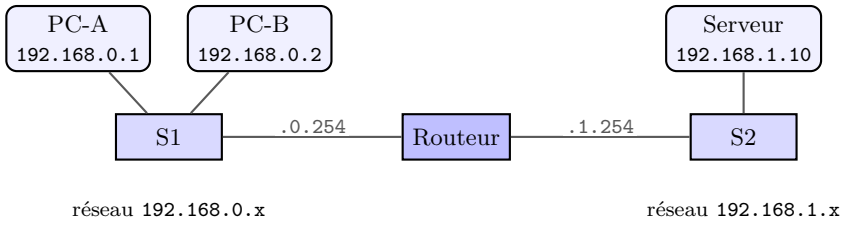
### Consignes

- Sur le simulateur gratuit et libre **Filius** (en français). Les réponses se notent sur le cahier.
- Les questions marquées ▶ se font sur le simulateur. Sans machine, les autres se traitent sur les schémas de l'énoncé.
- Rappels de seconde : un **commutateur** (*switch*) relie les machines d'un même réseau local ; un **routeur** relie plusieurs réseaux ; la **passerelle** d'une machine est l'adresse du routeur de son réseau, à qui elle confie les paquets destinés à un autre réseau. Filius demande aussi un **masque** : recopier `255.255.255.0`, sans chercher à l'expliquer.

**Prise en main de Filius.** Le mode **construction** (le marteau) sert à poser les composants (Ordinateur, Switch, Routeur) et à les relier avec le Câble ; un double-clic sur une machine ouvre sa configuration (adresse IP, masque, passerelle). Le mode **simulation** (la flèche verte) lance les échanges : un double-clic sur une machine permet d'y installer des logiciels (Ligne de commande, Serveur web, Navigateur web) ; un clic droit, « Afficher les échanges de données », montre les messages qui passent.

**Exercice 1** *Deux réseaux reliés par un routeur*

▶ Construire le réseau suivant : deux réseaux locaux, chacun avec son switch, reliés par un routeur à deux interfaces.



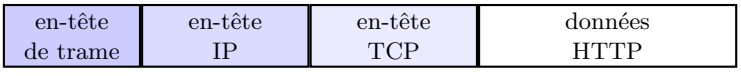

Interfaces du routeur : `192.168.0.254` côté S1, `192.168.1.254` côté S2 ; masque `255.255.255.0` partout. Pour l'instant, **ne pas remplir** les passerelles.

1. ▶ En simulation, installer la Ligne de commande sur PC-A et taper `ping 192.168.0.2`. Le `ping` réussit-il ? Quels composants le message traverse-t-il ? Le routeur intervient-il ?
2. ▶ Taper `ping 192.168.1.10`. Que se passe-t-il ? Expliquer pourquoi, sans passerelle, PC-A ne peut pas joindre le serveur.
3. ▶ Régler la passerelle de PC-A et de PC-B sur `192.168.0.254`, celle du serveur sur `192.168.1.254`, puis relancer le `ping`. Décrire le trajet complet du message.
4. ▶ Taper `traceroute 192.168.1.10` sur PC-A. Combien d'étapes sont affichées ? Que représente l'étape intermédiaire ?

**Exercice 2** *Ouvrir les enveloppes : l'encapsulation*

▶ Sur le serveur, installer le logiciel **Serveur web** et le démarrer ; sur PC-A, installer le **Navigateur web** et ouvrir l'adresse `http://192.168.1.10`.

5. ▶ La page d'accueil du serveur s'affiche-t-elle sur PC-A ? Quel protocole, vu dans le chapitre, le navigateur a-t-il utilisé pour la demander ?
6. ▶ Ouvrir « Afficher les échanges de données » sur PC-A. Pour chaque ligne, Filius indique le protocole et la couche concernée. Relever les protocoles qui apparaissent pendant le chargement de la page. Retrouver la ligne de la requête `GET` : quelles sont ses adresses IP source et destination ?
7. Un message HTTP voyage « emballé » dans plusieurs en-têtes. Recopier et compléter le schéma ci-dessous avec : *adresses IP source et destination*, *ports (80 pour le serveur web)*, *adresses MAC*, *requête* `GET /`.

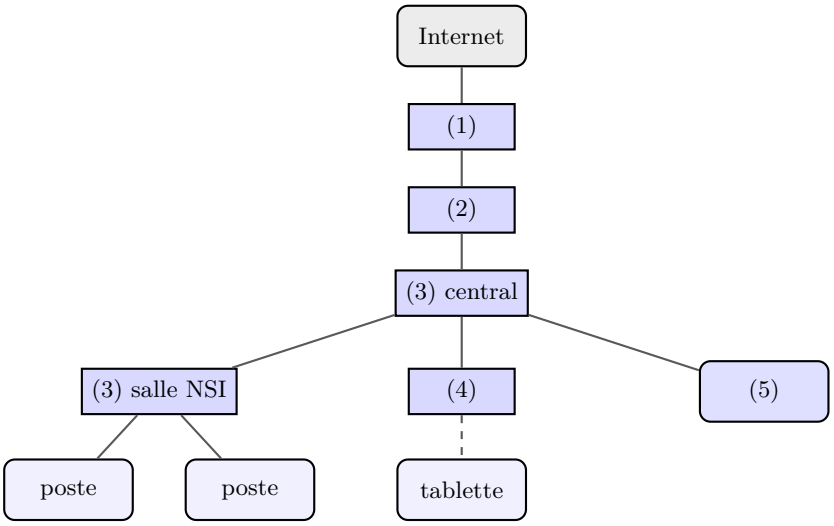



8. Un routeur reçoit ce message au milieu du trajet. Jusqu'à quel en-tête a-t-il besoin de « l'ouvrir » pour faire son travail ? Et un switch, qui ne regarde que les adresses de trame ?

**9.** Si PC-A était relié en Wi-Fi au lieu d'un câble, quelle enveloppe changerait ? Le navigateur et le serveur devraient-ils écrire la requête autrement ? Quel est donc l'intérêt de découper le travail en couches qui s'emboîtent ?

**Exercice 3** *Le réseau d'un établissement*

Le schéma suivant représente un réseau d'établissement typique ; celui du lycée lui ressemble (on peut demander au professeur ce qui diffère).




**10.** Identifier les constituants numérotés de (1) à (5) parmi : *point d'accès Wi-Fi*, *commutateur*, *pare-feu*, *routeur d'accès (box)*, *serveur de fichiers et de comptes*. Donner le rôle de chacun en une phrase.

**11.** Quels constituants la requête d'un poste de la salle NSI vers `fr.wikipedia.org` traverse-t-elle avant d'atteindre Internet ? Et celle de la tablette ?

**12.** Les postes du lycée ont des adresses comme `10.0.3.17`. Pourquoi Wikipédia ne voit-il jamais cette adresse ? Quel constituant intervient ?

**13.** Un élève enregistre son travail « dans son dossier du lycée » depuis un poste de la salle NSI. Ce fichier sort-il sur Internet ? Quel chemin suit-il ?



## TP 2 Côté serveur

▶ corrigé p. 684

*un formulaire avec Flask, et ce qui circule sur le réseau*

Durée : 2 h 30 à 3 h (une séance longue, ou deux séances avec une coupure après la partie 3)

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p10-tp-flask` — **archive .zip**

### But du TP

Jusqu'ici, vos pages étaient des fichiers **figés** : le navigateur les ouvrait, et tout ce qui « bougeait » était exécuté par lui, **côté client**. Vous allez maintenant passer de l'autre côté : écrire en Python, avec la bibliothèque **Flask**, le **serveur** qui fabrique les pages et reçoit les formulaires (cours, § VIII, « Le web dynamique »). Le client et le serveur seront sur la **même machine**, la vôtre. Vous traiterez un formulaire envoyé en `POST`, puis le même en `GET`, vous écrirez une page de connexion (avec des comptes **fictifs**), et vous regarderez enfin **ce qui circule réellement** entre le navigateur et le serveur quand on envoie un mot de passe.

### Consignes

- En binôme, sur machine. Basthon ne convient pas : il faut Spyder (ou un autre Python installé sur la machine) avec Flask, et un navigateur.
- Fichiers à télécharger (lien ci-dessus) : le dossier `tp_flask/`, qui contient `tp_flask_depart.py` (tout le code Python s'y écrit) et le dossier `templates/` (les modèles de pages HTML, à compléter). Ne renommer ni les fichiers ni le dossier `templates/`, et les laisser l'un à côté de l'autre.
- Les commentaires `A COMPLETER` et `TODO` indiquent où écrire. Chaque partie commence par le rappel des outils utiles. Les réponses aux questions (sans ▶ ) s'écrivent sur le cahier.
- **Aucune vraie donnée** : on n'écrit dans ces pages que des prénoms, des messages et les mots de passe de test fournis, jamais un mot de passe personnel.
- ▶ *À écrire et tester sur machine.* Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

### Lancer et arrêter le serveur dans Spyder

1. Ouvrir `tp_flask_depart.py` dans Spyder et l'exécuter (`F5`). La console affiche `Running on http://127.0.0.1:8080` et ne rend plus la main : c'est normal, le programme est devenu un serveur qui **attend des requêtes**.
2. Dans le navigateur, ouvrir l'adresse `http://localhost:8080`.
3. **Arrêter** le serveur : bouton carré rouge (« Interrompre le noyau ») en haut de la console, ou clic dans la console puis `Ctrl+C`.
4. Après **chaque modification du fichier Python** : arrêter le serveur, enregistrer, relancer (`F5`). Une modification d'un modèle de `templates/` ne demande qu'une actualisation de la page.

**En cas de problème.** `Address already in use` (port déjà utilisé) : un ancien serveur tourne encore ; menu *Consoles* → *Redémarrer le noyau*, puis `F5`. `TemplateNotFound` : le dossier `templates/` n'est pas à côté du fichier `.py`, ou son nom est mal écrit. `ModuleNotFoundError: No module named 'flask'` : prévenir le professeur.

## 1 Un serveur sur ma machine

Environ 25 minutes.

**Outils de la partie 1 — le serveur Flask minimal (cours, § III et IV)**

```python
from flask import Flask
app = Flask(__name__)            # l'application : c'est elle qui repond aux requetes

@app.route("/")                  # la "route" : le chemin demande dans l'URL
def accueil():                   # la fonction appelee quand ce chemin est demande
    return "<h1>Bonjour</h1>"    # le texte renvoye : le corps de la reponse HTTP

app.run(port=8080)               # demarre le serveur, qui attend sur le port 8080
```

`localhost` (ou `127.0.0.1`) désigne **la machine elle-même** : le navigateur s'adresse au serveur qui tourne sur son propre ordinateur. Le `8080` est le **port** du service (cours, § III).

### ★ Exercice 1 *Premier contact*▶

1. Lancer `tp_flask_depart.py`, puis ouvrir `http://localhost:8080`. Qui joue ici le rôle du client ? du serveur ? Sur quelle(s) machine(s) ?
2. Afficher le code source de la page reçue (`Ctrl+U`). Le navigateur a-t-il reçu du Python ? Qu'a-t-il reçu exactement ?
3. Regarder la console de Spyder : une ligne est apparue. Y retrouver la **méthode** de la requête, le **chemin** demandé et le **code de statut** de la réponse (cours, § IV).
4. Demander l'adresse `http://localhost:8080/bidule`. Quel code de statut lit-on dans la console ? Que signifie-t-il ?
5. Arrêter le serveur, puis actualiser la page dans le navigateur. Que se passe-t-il ? Relancer le serveur.
6. Pourquoi faut-il écrire `:8080` dans cette adresse, alors qu'on n'écrit jamais de numéro de port pour aller sur un site ordinaire ?

### ★ Exercice 2 *Une deuxième route*▶

1. À l'endroit indiqué, écrire une route `/apropos` qui renvoie un titre « À propos », une phrase qui présente le site, et un lien vers la page d'accueil. ▷ coup de pouce
2. Tester `http://localhost:8080/apropos`, puis le lien. Combien de requêtes la console a-t-elle enregistrées ?
3. Que se passe-t-il si l'on oublie de relancer le serveur après avoir écrit la route ?

**Point d'étape ✓** Les deux chemins `/` et `/apropos` répondent (code 200), un chemin inconnu donne 404, et vous savez arrêter et relancer le serveur.

## 2 Des pages fabriquées par le serveur

Environ 35 minutes. Écrire tout le HTML dans des chaînes Python devient vite illisible : on le range dans des **modèles** (*templates*), des fichiers HTML du dossier `templates/`, où le serveur glisse des valeurs calculées au moment de la requête.

**Outils de la partie 2 — les modèles**

- Dans le Python : `return render_template("heure.html", heure=h)` renvoie la page `templates/heure.html`, en lui transmettant une variable `heure` qui vaut `h`. On peut en transmettre plusieurs, séparées par des virgules.
- Dans le modèle : `{{ heure }}` est **remplacé** par la valeur de la variable `heure`, sur le serveur, avant l'envoi.
- L'heure : `maintenant = datetime.now()`, puis `maintenant.strftime("%H:%M:%S")` donne une chaîne comme `"14:05:32"` et `maintenant.strftime("%d/%m/%Y")` une chaîne comme `"08/11/2026"`.

★ **Exercice 3** *Un modèle pour l'accueil*▶

1. Ouvrir `templates/accueil.html` dans l'éditeur de Spyder (ou un éditeur de texte). Compléter le titre de l'onglet, puis écrire un titre `<h1>` et un paragraphe de présentation à l'endroit indiqué.
2. Modifier la route `/` pour qu'elle renvoie ce modèle au lieu de la chaîne. Tester.
3. Cliquer sur le lien « L'heure du serveur ». Que se passe-t-il, et pourquoi ?

★★ **Exercice 4** *L'heure du serveur*▶

1. Écrire la route `/heure` : elle calcule la date et l'heure du moment et les transmet au modèle `heure.html`. Compléter ce modèle pour qu'il affiche une phrase comme « Nous sommes le 08/11/2026 et il est 14 :05 :32 ». ▷ coup de pouce
2. Actualiser la page plusieurs fois. Qu'observe-t-on ? En quoi cette page diffère-t-elle d'un fichier HTML ordinaire ?
3. Afficher le code source de la page (`Ctrl+U`). Les accolades y figurent-elles ? Qui les a remplacées, et à quel moment ?
4. Si ce serveur était installé à Tokyo et consulté depuis Monaco, quelle heure la page afficherait-elle ? Et si l'heure était calculée par un programme JavaScript de la page ? Relier la réponse à la distinction client / serveur du cours.

★★ **Exercice 5** *Un compteur de visites*▶

Le fichier contient déjà le dictionnaire `compteur = {"visites": 0}`, écrit en dehors de toute fonction : il existe tant que le serveur tourne.

1. Modifier la route `/` : à chaque appel, elle augmente de 1 le nombre de visites, puis transmet ce nombre au modèle. Compléter `accueil.html` pour qu'il affiche « Vous êtes le visiteur numéro … ». ▷ coup de pouce
2. Actualiser plusieurs fois, puis ouvrir la même adresse dans un **autre navigateur** (ou une fenêtre de navigation privée). Le compteur repart-il de 1 ? Où ce nombre est-il donc stocké ?
3. Arrêter puis relancer le serveur. Que devient le compteur ? Pourquoi ?
4. Dans le TP « Mon premier site », le compteur de « j'aime » revenait à 0 à chaque actualisation. Expliquer la différence entre les deux compteurs.

**Point d'étape ✓** L'accueil vient d'un modèle et compte ses visiteurs ; la page `/heure` change à chaque actualisation. Vous avez vu que le navigateur ne reçoit que du HTML, fabriqué sur le serveur au moment de la requête.

## 3 Un formulaire, en POST puis en GET

Environ 40 minutes.

**Outils de la partie 3 — recevoir un formulaire (cours, § VIII)**

- Côté client : `<form action="/bonjour" method="post">` envoie les champs vers le chemin `/bonjour` ; chaque champ `<input type="text" name="prenom">` arrive au serveur sous l'étiquette donnée par son `name`.
- Côté serveur, la route doit accepter la méthode : `@app.route("/bonjour", methods=["POST"])`. Sans `methods`, une route n'accepte que `GET`.
- Lire une donnée reçue en `POST` : `request.form["prenom"]` ; reçue en `GET` : `request.args["prenom"]`. Le résultat est toujours une **chaîne**.

### ★★ Exercice 6 — *Envoyer en* POST▶

1. Écrire la route `/formulaire`, qui renvoie le modèle `formulaire.html`. Dans ce modèle, la balise `<form>` est fournie : écrire à l'intérieur un champ texte `prenom`, un champ texte `classe` (chacun précédé de son `<label>`) et un bouton d'envoi.
2. Écrire la route `/bonjour` : elle accepte la méthode `POST`, lit le prénom et la classe reçus et les transmet au modèle `bonjour.html`, à compléter pour qu'il affiche « Bonjour Léa ! » et la classe. ▷ coup de pouce
3. Envoyer le formulaire. Quelle adresse lit-on dans la barre d'adresse ? Y voit-on le prénom ? Où se trouvent donc les données dans la requête ?
4. Sur la page de réponse, actualiser (`F5`). Que demande le navigateur ? Pourquoi ?
5. Taper directement l'adresse `http://localhost:8080/bonjour` dans la barre d'adresse. Quel code de statut obtient-on ? Avec quelle méthode le navigateur envoie-t-il la requête quand on tape une adresse ?

### ★★ Exercice 7 — *Le même en* GET▶

1. Dans `templates/`, enregistrer une copie de `formulaire.html` sous le nom `formulaire_get.html`. Dans la copie, changer le titre, la méthode (`get`) et la destination (`/bonjour_get`).
2. Écrire la route `/formulaire_get`, qui renvoie ce nouveau modèle, et la route `/bonjour_get`, qui fait le même travail que `/bonjour` mais pour un formulaire envoyé en `GET`. Elle réutilise le modèle `bonjour.html`.
3. Envoyer ce formulaire avec le prénom `Léa` et votre classe. Recopier l'adresse obtenue. Repérer le `?`, les paires `clef=valeur` et le séparateur `&`. Comment le `é` a-t-il été écrit ? [ au-delà du programme ]
4. Modifier le prénom **directement dans la barre d'adresse**, puis valider. Le formulaire a-t-il servi ? Qu'en conclure sur les données qu'un serveur reçoit ?
5. Regarder la console de Spyder : où retrouve-t-on le prénom ? Et pour l'envoi en `POST` de l'exercice précédent ?
6. Ouvrir `http://localhost:8080/bonjour_get`, sans rien après. Que se passe-t-il ? Quelle ligne du programme est en cause ?
7. Ouvrir l'historique du navigateur (`Ctrl+H`). Lequel des deux envois y a laissé le prénom ? Pour un moteur de recherche, quelle méthode choisir ? Pour un mot de passe ?

**Point d'étape ✓** Les deux formulaires mènent à la même page « Bonjour », l'un par `POST`, l'autre par `GET` ; en `GET`, les données sont dans l'URL, dans l'historique et dans le journal du serveur.

## 4 Défi : une page de connexion

Environ 30 minutes. Le fichier contient deux comptes **fictifs** :

```
COMPTES_TEST = {"alice": "tournesol42", "bob": "pivoine7"}
```

Un dictionnaire dont les clefs sont les identifiants et les valeurs les mots de passe. (Un vrai site ne garde jamais les mots de passe en clair : il n'en conserve qu'une empreinte, dont on reparlera en Terminale.)

### ★★ Exercice 8 *Vérifier des identifiants*

La fonction `verifier_identifiants(comptes, identifiant, mot_de_passe)` doit renvoyer `True` si `identifiant` est une clef du dictionnaire `comptes` et que le mot de passe associé est `mot_de_passe`, et `False` dans tous les autres cas. Elle se teste sans le serveur : c'est du Python ordinaire.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. Dans `tester_connexion`, sous l'`assert` fourni, écrire vos trois `assert` (le résultat attendu d'un appel qui doit renvoyer `False` s'écrit `assert not ...`).
2. Écrire la fonction, décommenter `tester_connexion()` dans le programme principal et exécuter : les tests doivent passer avant le démarrage du serveur.

▷ coup de pouce

### ★★★ Exercice 9 *La page de connexion*

1. Compléter `templates/connexion.html` : un formulaire envoyé en `POST` vers `/verifier`, avec un champ `identifiant`, un champ `mot_de_passe` de type `password` et un bouton « Se connecter ».
2. Écrire la route `/connexion`, qui affiche ce formulaire, et la route `/verifier`, qui lit les deux champs, appelle `verifier_identifiants` et transmet au modèle `resultat.html` (à compléter) un message : « Bienvenue, alice : vous êtes connecté(e). » ou « Identifiant ou mot de passe incorrect. ». ▷ début de solution
3. Tester avec les deux comptes, avec un mauvais mot de passe, avec un identifiant inconnu, avec des champs vides.
4. Que change le type `password` pour la personne qui tape ? Protège-t-il le mot de passe pendant son **voyage** vers le serveur ? (Gardez votre réponse : la partie suivante va la vérifier.)

**Point d'étape ✓** La connexion accepte `alice` et `bob` avec leur mot de passe, et refuse tout le reste ; `tester_connexion` passe.

## 5 Ce qui circule sur le réseau

Environ 35 minutes. Quand le formulaire de connexion part, que contient **exactement** la requête ? On va la regarder, avec l'un des deux outils ci-dessous : **Wireshark** s'il est installé sur la machine (exercice 10), sinon les **outils de développement** du navigateur (exercice 11). L'exercice 12 est commun.

**Règle du jeu**

On n'observe que **son propre trafic**, sur **sa machine**, avec des mots de passe **de test**. Intercepter les communications des autres est **interdit par la loi** et par la charte informatique du lycée. Ici, le client et le serveur sont sur le même ordinateur : la requête passe par l'**interface de bouclage** (*loopback*), un réseau « interne » qui ne sort jamais de la machine.

**Pour le professeur — à préparer avant la séance**

**Voie A** : installer **Wireshark** (gratuit, `https://www.wireshark.org`), ce qui demande les droits d'administrateur. Sous Windows, garder l'installation de **Npcap** proposée et cocher son option de capture du trafic de bouclage (*loopback*) ; sous macOS, accepter le composant *ChmodBPF*, qui permet de capturer sans être administrateur. Vérifier une fois que l'interface de bouclage apparaît. **Voie B** : rien à installer, tout navigateur récent convient. **Flask** est fourni avec Anaconda : vérifier dans la console de Spyder que `import flask` ne produit pas d'erreur, et que le port `8080` est libre.

★★ **Exercice 10** *Voie A : avec Wireshark*▶

Wireshark est un **analyseur de réseau** : il enregistre une copie de chaque paquet qui passe par une interface réseau et en décode les couches (cours, § II, encapsulation).

1. Le serveur Flask étant lancé, ouvrir Wireshark. Dans la liste des interfaces, double-cliquer sur l'interface de bouclage : `Loopback: lo0` (macOS), `Adapter for loopback traffic capture` (Windows) ou `Loopback: lo` (Linux). La capture démarre.
2. Dans la barre de filtre, en haut, taper `http` puis `Entrée` : seuls les échanges HTTP restent affichés.
3. Dans le navigateur, se connecter avec `alice` et `tournesol42`. Revenir dans Wireshark et arrêter la capture (carré rouge).
4. Repérer la ligne `POST /verifier`. La sélectionner : le panneau du milieu détaille ses couches. Retrouver l'adresse IP source et destination, les ports, puis, tout en bas, la partie `HTML Form URL Encoded`. Qu'y lit-on ?
5. Clic droit sur la ligne, *Suivre* → *Flux TCP* (*Follow* → *TCP Stream*). Recopier la première ligne de la requête, l'en-tête `Content-Type`, la dernière ligne de la requête, et la première ligne de la réponse.
6. Refaire une capture en envoyant le formulaire de l'exercice 7 (`GET`). Où se trouvent cette fois les données dans la requête ?

★★ **Exercice 11** *Voie B : sans installation, les outils de développement*▶

1. Sur la page de connexion, ouvrir les outils de développement (`F12`), onglet **Réseau**, puis se connecter avec `alice` et `tournesol42`.
2. Cliquer sur la requête `verifier`. Dans l'onglet **En-têtes**, relever la méthode, l'adresse demandée, le code de statut et l'adresse du serveur (avec son port), puis l'en-tête de requête `Content-Type`.
3. Ouvrir l'onglet qui montre ce que le navigateur a envoyé : **Requête** (Firefox) ou **Charge utile** (Chrome, Edge). Qu'y lit-on ? Afficher la version brute (bouton *Brut* ou *Afficher la source*) et la recopier.
4. Refaire l'expérience avec le formulaire de l'exercice 7 (`GET`). Où se trouvent cette fois les données ?
5. Ces outils montrent ce que le navigateur *envoie*, juste avant que la requête ne parte sur le réseau : ils montreraient la même chose sur un site en `https`. Quel argument permet pourtant d'affirmer qu'ici le mot de passe a *voyagé* tel quel, lisible ? ▷ coup de pouce

★★ **Exercice 12** *Conclure : et avec* `https` *?*

1. Revenir à la réponse de l'exercice 9, question 4 : était-elle juste ?
2. Si ce site était sur Internet, en `http`, par quels équipements la requête de connexion passerait-elle entre un ordinateur connecté au Wi-Fi d'un café et le serveur (cours, § II) ? Lesquels pourraient lire le mot de passe ?
3. Le cours dit qu'un routeur n'ouvre l'enveloppe « que jusqu'à l'en-tête IP ». Est-ce une protection pour le mot de passe ? Pourquoi ?
4. Avec `https`, qu'est-ce qui change dans ce que verrait Wireshark ? Qu'est-ce qui resterait visible ?
5. L'envoi en `POST` suffit-il à protéger un mot de passe ? Et `https` avec un envoi en `GET` ? Rédiger en deux ou trois phrases la règle à retenir pour un formulaire de connexion.

**Point d'étape ✓** Vous avez vu le mot de passe dans le corps de la requête `POST`, et le prénom dans l'URL de la requête `GET`. Sans `https`, l'un comme l'autre circule en clair.

## 6 Bonus : un livre d'or

★★★ **Exercice 13** *Le livre d'or*▶

La liste `livre`, en dehors de toute fonction, contient les messages publiés, chacun sous la forme d'un dictionnaire `{"pseudo": ..., "texte": ...}`. Le modèle `livre.html` fournit déjà la boucle qui affiche tous les messages.

1. Écrire trois `assert` dans `tester_livre`, puis la fonction `ajouter_message(livre, pseudo, texte)` décrite par sa documentation. Décommenter `tester_livre()` et exécuter.
2. Compléter `livre.html` avec un formulaire (pseudo, texte, bouton), envoyé en `POST` vers `/livre/ecrire`. Écrire la route `/livre`, qui affiche le modèle avec la liste des messages et une variable `erreur` vide, et la route `/livre/ecrire`, qui appelle `ajouter_message` puis affiche le même modèle, avec un message d'erreur si l'ajout a été refusé.
3. Publier deux messages depuis deux navigateurs différents : chacun voit-il les messages de l'autre ? Pourquoi ?
4. Juste après avoir publié, actualiser la page (`F5`) et accepter le renvoi. Que se passe-t-il ? Pourquoi ?
5. Publier le message `<b>coucou</b>`. Comment s'affiche-t-il ? Regarder le code source de la page (`Ctrl+U`) : qu'est devenu le `<` ? Pourquoi est-il prudent que le serveur procède ainsi avec un texte écrit par un visiteur ? [ au-delà du programme ]



### Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 2 q. 1 – piste Copier les trois lignes de la route `/` et changer trois choses : le chemin entre les guillemets de `@app.route`, le nom de la fonction, le texte renvoyé. Le lien vers l'accueil a pour destination `/`. ↑énoncé

Ex. 4 q. 1 – piste Deux variables à transmettre, par exemple `date` et `heure` : `render_template("heure.html", date=..., heure=...)`. Dans le modèle, chacune s'écrit entre doubles accolades, à l'endroit où sa valeur doit apparaître. ↑énoncé

Ex. 5 q. 1 – piste `compteur["visites"]` se lit et se modifie comme n'importe quelle valeur de dictionnaire. La mise à jour se fait *avant* le `return`. ↑énoncé

Ex. 6 q. 2 – piste Trois lignes dans la fonction : deux pour lire les champs avec `request.form`, une pour le `return render_template(...)` avec deux variables, comme à l'exercice 4. ↑énoncé

Ex. 8 q. 2 – piste Que fait `comptes[identifiant]` si `identifiant` n'est pas une clef ? Tester d'abord sa présence avec `in`. ↑énoncé

Ex. 9 q. 2 – début de solution
```
identifiant = request.form["identifiant"]
   mot_de_passe = request.form["mot_de_passe"]
   if verifier_identifiants(COMPTES_TEST, identifiant,
mot_de_passe):
      message = f"Bienvenue, {identifiant} : vous êtes
connecté(e)."
   else: ...
```
↑énoncé

Ex. 11 q. 5 – piste Le programme Python a lu le prénom (exercice 6) et le mot de passe (exercice 9) directement, sans aucune clef de déchiffrement : sous quelle forme les octets lui sont-ils donc parvenus ? ↑énoncé

# FICHE DE RÉVISION

## L'idée maîtresse — le voyage d'un clic

Entre l'URL tapée et la page affichée : le **DNS** traduit le nom en **adresse IP** → le navigateur (**client**) envoie une **requête HTTP** → les **paquets** traversent des **routeurs** → le **serveur** renvoie du **HTML/CSS/JS** → le navigateur **affiche** et l'utilisateur **interagit**.

## Réseaux

**Internet** = le réseau (l'infrastructure) ; **le Web** = un service (pages + liens) qui roule dessus. **Protocole** = règles de communication (HTTP, TCP/IP). **Adresse IP** = numéro unique d'une machine (ex. `193.51.208.14`). Un message voyage en **paquets** (routés indépendamment, réassemblés, renvoyés si perdus). **Routeur** = aiguille les paquets ; **DNS** = annuaire nom → IP. **Client-serveur** (le client demande, le serveur fournit) vs **pair-à-pair** (chacun les deux). **Bit alterné** : pour ne rien perdre, le récepteur renvoie un **accusé de réception** ; sans accusé, l'émetteur **renvoie** le paquet ; un bit `0/1` sur chaque paquet permet de repérer et jeter les **doublons** (idée de base de TCP).

## URL et HTTP

```
https://www.lycee.mc/nsi/cours.html
protocole   domaine        chemin
```

`https` = liaison chiffrée (pas forcément site honnête) : à vérifier avant de saisir mot de passe / coordonnées. **HTTP** : requête (`GET` demander / `POST` envoyer) → réponse avec un **code** : **200** OK, **404** introuvable, **403** interdit, **500** erreur serveur.

## Les trois langages du Web

**HTML** = le *fond* (structure, balises) ; **CSS** = la *forme* (présentation) ; **JavaScript** = le *comportement* (interaction). Le document HTML forme un **arbre** : le **DOM**.

```
<!doctype html>
<html lang="fr">
  <head> <meta charset="utf-8"> <title>...</title>
         <link rel="stylesheet" href="style.css"> </head>
  <body> <h1>Titre</h1> <p>Texte <a href="p2.html">lien</a></p> </body>
</html>
```

Balise ouvrante `<p>` / fermante `</p>`, à **emboîter sans croiser**. `<a href>` = lien hypertexte.

## CSS — séparer le fond de la forme

```
h1 { color: darkblue; }           /* balise    */
#menu { background-color: gold; } /* id="menu" */
.bouton { color: white; }         /* class="bouton" */
```

## JavaScript — événement + DOM (côté client)

```
<p id="monPara">Bonjour</p>
<button onclick="changer()">Cliquer ici</button>
```

```
function changer() {
    let maBalise = document.querySelector("#monPara");
    maBalise.innerHTML = "Bonsoir";
}
```

Événement (`onclick`, `onmouseover`…) → fonction ; `querySelector` sélectionne, `innerHTML` modifie. **S'exécute côté client**.

### Formulaires et cookies

```
<form action="/connexion" method="post">
  <input type="text" name="pseudo">
  <button type="submit">Envoyer</button>
</form>
```

`action` = où vont les données ; `method` = `get` (dans l'URL) ou `post` (pas dans l'URL, pour le sensible) ; chaque `name` = l'étiquette côté serveur. **Cookie** : envoyé par le serveur (`Set-Cookie`), **mémorisé** par le client, **retransmis** au même site à chaque requête ; modifiable par l'utilisateur.

### Erreurs à éviter

Confondre Internet (réseau) et Web (service) ; appeler HTML un langage de programmation ; croiser ou oublier de fermer des balises ; mélanger les rôles HTML / CSS / JS ; envoyer un mot de passe en `GET` ; croire sûre une vérification faite côté client (le serveur doit re-vérifier).

### Savoir-faire à maîtriser — je sais…

- ☐ expliquer la différence entre Internet et le Web, et ce qu'est un protocole ;
- ☐ dire à quoi servent une adresse IP, un paquet, un routeur, le DNS ;
- ☐ expliquer l'**encapsulation** (ce que chaque couche ajoute) et le rôle des constituants d'un **réseau local** ;
- ☐ **dérouler** le protocole du **bit alterné** (accusé de réception, renvoi, doublon) sur un scénario de perte ;
- ☐ **décomposer** une URL et reconnaître une page sécurisée ;
- ☐ distinguer ce qui relève du **client** et du **serveur**, et `GET` de `POST` ;
- ☐ **observer** les requêtes HTTP échangées avec les **outils de développement** (`F12`, onglet Réseau) ;
- ☐ écrire et analyser une page **HTML** simple (balises, `id`/`class`) et sa mise en forme **CSS** (sélecteurs `balise`, `#id`, `.classe`) ;
- ☐ **disposer** des éléments côte à côte avec **Flexbox** (`display:flex`) [au-delà du programme] ;
- ☐ rendre une page interactive : réagir à un **événement**, **sélectionner** un élément (`querySelector`), **lire un champ** (`.value`) et **modifier le DOM** ;
- ☐ assembler HTML + CSS + JavaScript en une **petite page interactive complète**, qui répond à un **cahier des charges** donné ;
- ☐ écrire un **formulaire** et choisir sa méthode d'envoi ;
- ☐ distinguer ce qu'un **cookie** fait **mémoriser par le client** et ce qui est **retransmis au serveur**.

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 690

1. Internet et le Web : lequel est le réseau, lequel est un service ?
2. Le navigateur connaît `www.lycee.mc` : qui lui donne l'adresse IP du serveur ?
3. Code de réponse HTTP pour une page introuvable ? Pour « tout va bien » ?
4. Que devient `<p id="monPara">Bonjour</p>` après un clic qui appelle `changer()` (partie JavaScript ci-dessus) ?
5. Formulaire de connexion avec mot de passe : `get` ou `post` ? Pourquoi ?

**Et avant ?** — *questions sur les chapitres précédents.* ▶ réponses p. 690

1. Dans un chemin, que désignent `..` et `~` ? (ch. 9)
2. Tableau trié deux fois plus grand : combien de tours de plus, au plus, pour une recherche dichotomique ? (ch. 7)
3. Une recherche renvoie l'indice de la première occurrence. Que renvoie-t-elle pour `7` dans `[4, 7, 7]` ? (ch. 4)

# DÉFI ADVENT OF CODE

## DÉFI Tuning Trouble

*le signal radio — fenêtre glissante sur une chaîne*

Jour 6

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p10-defi-aoc-2022-06` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2022/day/6`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Le fichier contient un seul très long flux de lettres (plusieurs milliers). Il faut trouver le premier endroit où 4 lettres consécutives sont toutes différentes, puis (partie 2) le premier endroit où 14 lettres consécutives le sont. Sur quelques dizaines de lettres, on y arrive à l'œil ; sur plusieurs milliers, il faut un programme.
**Ce qu'on réinvestit.** Le lien avec ce chapitre sur les réseaux : un appareil qui reçoit des données doit savoir **où commence un paquet** dans le flux qui lui arrive, et les protocoles prévoient pour cela des **marqueurs**, des motifs reconnaissables. Côté programmation, on utilise une **fenêtre glissante** : une tranche de taille fixe que l'on décale d'un caractère à la fois le long de la chaîne, avec les tranches et le test `in`.
**La démarche de la fiche.**

1. Comprendre la règle (en particulier comment on compte la position), puis la faire à la main sur deux petits flux.
2. Lire le fichier, qui ne contient qu'une ligne.

3. Écrire une fonction qui dit si une chaîne a tous ses caractères différents, puis faire glisser la fenêtre : on obtient la partie 1.
4. Pour la partie 2, ajouter un paramètre `taille` à la fonction : seul l'appel change.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `datastream` | flux de données | `buffer` | tampon, mémoire de réception |
| `marker` | marqueur | `start-of-packet` | début de paquet |
| `all different` | tous différents | `characters processed` | caractères traités |
| `received` | reçu | `start-of-message` | début de message |

### Résumé de l'énoncé (partie 1)

**Contexte.** Un elfe vous confie un petit appareil de communication, qui doit d'abord se caler sur le signal radio des elfes. Le signal arrive comme une suite de lettres reçues l'une après l'autre. Pour comprendre ce signal, l'appareil doit repérer où commence un paquet de données : ce début est signalé par un groupe de lettres consécutives toutes différentes.
**Les données.** Le fichier contient tout le flux reçu sur **une seule ligne** : une longue suite de lettres minuscules, sans espace. Le premier caractère de la ligne est le premier reçu.
**La règle.**

- On lit les caractères de gauche à droite, un par un.
- Après chaque caractère lu, on regarde les **4 derniers caractères lus** (le caractère qu'on vient de lire et les trois précédents).
- Dès que ces 4 caractères sont tous différents, on a trouvé le marqueur de début de paquet, et l'on s'arrête.
- On ne peut pas répondre avant d'avoir lu au moins 4 caractères.

**La réponse attendue.** Le nombre de caractères lus depuis le début jusqu'au dernier caractère du marqueur inclus, positions comptées à partir de 1. Par exemple, pour le flux `xxyxzwq`, les 4 derniers caractères lus sont tous différents pour la première fois après la lecture du sixième caractère (`yxzw`) : la réponse serait 6.

**Exercice 1** *Comprendre ce qui est demandé*

**But.** Vérifier sa compréhension de la règle, surtout de la façon dont on compte la position.
**À quoi ça sert ?** La difficulté de ce défi n'est pas la recherche elle-même, mais le décalage d'indices : Python compte à partir de 0, la réponse compte à partir de 1, et une tranche s'arrête juste avant sa borne de fin. Ces questions évitent une réponse fausse d'une ou trois unités.

Après lecture du résumé (et de l'énoncé), vérifier sa compréhension sur ces cas particuliers :

1. Peut-on obtenir une réponse plus petite que 4 ?
2. Pour `abcd...`, quelle est la réponse ? et pour `aaaabcd` ?
3. Si la fenêtre `flux[i:i+4]` convient, faut-il répondre `i`, `i + 4` ou `i + 3` ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Trouver à la main le marqueur de deux petits flux, en écrivant les fenêtres successives.
**À quoi ça sert ?** On voit concrètement la fenêtre « glisser » d'un caractère à la fois, ce que le programme fera ; et l'on obtient deux réponses connues (`7` et `10`) pour le tester.

Pour chacun des flux (inventés) suivants, trouver la réponse de la partie 1 :

```
aabacbdeffgh
nsinsinsiabcdefghijklmnop
```

Vérification : on doit obtenir `7` pour le premier, `10` pour le second. Écrire les fenêtres de 4 caractères examinées l'une après l'autre pour le premier flux.

## 2 Programmer la partie 1

### Exercice 3 *Lire le fichier*

**But.** Lire tout le flux dans une seule chaîne `flux`.
**À quoi ça sert ?** Le fichier n'a qu'une ligne : on n'a pas besoin de boucle de lecture, mais il faut retirer le retour à la ligne final, qui serait compté comme un caractère du flux.

#### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Le fichier ne contient qu'une ligne : on le lit d'un coup. `fichier.read()` renvoie *tout* le contenu du fichier en une seule chaîne (au lieu d'une liste de lignes comme `readlines()`).

```
fichier = open("input.txt", encoding="utf-8")
flux = fichier.read().strip()   # toute la ligne, sans le retour final
fichier.close()
```

Afficher la longueur de `flux` et ses 10 premiers caractères (`flux[0:10]`).

### Exercice 4 *Des caractères tous différents ?*

**But.** Écrire une fonction qui dit si une chaîne courte n'a aucun caractère en double.
**À quoi ça sert ?** C'est le test appliqué à chaque fenêtre : la recherche du marqueur consiste à appeler cette fonction sur des fenêtres successives jusqu'à ce qu'elle renvoie `True`.

Écrire une fonction `tous_differents(morceau)` qui renvoie `True` si la chaîne `morceau` ne contient pas deux fois le même caractère. Tester avec `"acbd"` et `"abac"`. ▷ coup de pouce

### Exercice 5 *Faire glisser la fenêtre*

**But.** Parcourir le flux avec une fenêtre de 4 caractères et renvoyer la position du premier marqueur.
**À quoi ça sert ?** C'est la réponse de la partie 1. La fenêtre glissante est une technique qu'on retrouve souvent : moyennes sur plusieurs jours, recherche d'un motif dans un texte, etc.

Écrire une fonction `position_marqueur(flux)` qui renvoie la réponse de la partie 1. Vérifier les deux valeurs de l'exemple, puis lancer sur vos données. ▷ coup de pouce ▷ début de solution

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

### Résumé de la partie 2 (à lire après avoir validé la partie 1)

**Ce qui change.** L'appareil doit aussi repérer le début des messages, signalé par un marqueur plus long que celui des paquets.
**La nouvelle règle.**

- Même lecture qu'à la partie 1, mais on regarde les **14** derniers caractères lus au lieu des 4 derniers.
- On s'arrête dès que ces 14 caractères sont tous différents.

**La réponse attendue.** Le nombre de caractères lus jusqu'au dernier caractère de ce premier groupe de 14, positions comptées à partir de 1 comme dans la partie 1.

### Exercice 6 *Généraliser*

**But.** Rendre la taille de la fenêtre réglable par un paramètre.
**À quoi ça sert ?** Une seule fonction répond alors aux deux parties : la partie 2 ne demande que de changer l'appel. C'est l'intérêt de ne pas écrire de valeur « en dur » dans une fonction.

Ajouter un paramètre : `position_marqueur(flux, taille)`. La partie 1 correspond à `taille = 4`. Répondre à la partie 2. Sur le second flux de l'exemple, on doit trouver 23. ▷ coup de pouce

## 4 Approfondissement

### Rappel : les ensembles (`set`)

Les ensembles sont présentés dans l'encadré « Outil Python : les ensembles » du défi *High-Entropy Passphrases* (chapitre 7). L'essentiel : `set()` crée un ensemble vide, `e.add(x)` ajoute `x` (sans doublon), `len(e)` compte les éléments, et le test `x in e` est quasi instantané, quelle que soit la taille de `e`. Ici, on construit l'ensemble des caractères d'une chaîne en les ajoutant un à un dans une boucle : les doublons disparaissent d'eux-mêmes.

```
1 caracteres = set()
2 for c in "abac":
3     caracteres.add(c)
```

*À essayer.* Que vaut alors `len(caracteres)` ? Comparer avec `len("abac")`.

### ★★ Exercice 7 *Approfondissement 1 : tous différents avec un ensemble*

**But.** Compter le travail fait par `tous_differents`, puis l'écrire avec un ensemble.
**À quoi ça sert ?** Le nombre de comparaisons grandit vite avec la taille de la fenêtre (14 au lieu de 4) ; l'ensemble évite de comparer chaque caractère à tous les autres.

1. Pour une fenêtre de taille $k$, combien de comparaisons de caractères `tous_differents` fait-elle au pire ? Calculer pour $k = 4$ et $k = 14$. ▷ coup de pouce
2. Écrire `tous_differents_ens(morceau)` avec un ensemble, et vérifier qu'elle donne les mêmes réponses. ▷ coup de pouce

### Outil Python : codes des caractères (`ord`, `chr`)

Chaque caractère a un numéro (son code). `ord` donne le code d'un caractère, `chr` fait l'inverse. Les lettres minuscules ont des codes consécutifs.

```
print(ord("a"))              # 97 : code du caractere
print(ord("c") - ord("a"))   # 2 : rang de "c" dans l'alphabet (a -> 0)
print(chr(98))               # b : operation inverse
compteurs = [0] * 26
compteurs[ord("c") - ord("a")] += 1    # on compte un "c"
```

**Intérêt.** `ord(c) - ord("a")` transforme une lettre en un indice de 0 à 25 : on peut alors compter les lettres dans une simple liste de 26 entiers.
*À essayer.* Que vaut `ord("z") - ord("a")` ? Et `chr(ord("a") + 3)` ?

★★★ **Exercice 8** *Approfondissement 2 : ne pas tout recompter*

**But.** Mettre à jour les compteurs de lettres quand la fenêtre glisse, au lieu de tout réexaminer.
**À quoi ça sert ?** On obtient un programme dont la durée ne dépend presque plus de la taille de la fenêtre : c'est la version efficace de la fenêtre glissante.

Quand la fenêtre glisse d'un cran, un seul caractère entre et un seul sort : les autres ne changent pas.

1. À la main, sur `aabacbdeffgh` avec $k = 4$ : écrire, pour chaque glissement, le caractère qui entre et celui qui sort.
2. Écrire `position_rapide(flux, taille)` qui tient à jour une liste de 26 compteurs (un par lettre) et le nombre de lettres différentes présentes dans la fenêtre. Vérifier qu'elle redonne les réponses des parties 1 et 2. ▷ coup de pouce ▷ début de solution
3. Combien d'opérations pour un flux de $n$ caractères ? Cela dépend-il encore de $k$ ?



## Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 – piste Pour chaque indice `i`, regarder si `morceau[i]` apparaît dans la suite de la chaîne, `morceau[i+1:]`, avec l'opérateur `in`. ↑énoncé

Ex. 5 – piste La fenêtre qui se termine juste avant l'indice `fin` est la tranche `flux[fin-4:fin]`. Le premier `fin` possible est 4. ↑énoncé

Ex. 5 – début de solution Une boucle `while` qui avance `fin` de 1 tant que la fenêtre `flux[fin-4:fin]` n'est pas formée de caractères tous différents. À la sortie, `fin` est exactement le nombre de caractères traités. ↑énoncé

Ex. 6 – piste Remplacer chaque 4 écrit « en dur » par `taille`. Si la fonction était bien écrite, la partie 2 ne demande que de changer l'appel. ↑énoncé

Ex. 7 q. 1 – piste Le premier caractère est comparé aux $k-1$ suivants, le deuxième aux $k-2$ suivants, etc. ↑énoncé

Ex. 7 q. 2 – piste Parcourir la fenêtre en mettant chaque caractère dans un ensemble : les caractères sont tous différents si et seulement si, à la fin, l'ensemble a la même taille que la fenêtre. ↑énoncé

Ex. 8 q. 2 – piste Quand un caractère entre, ajouter 1 à son compteur ; si ce compteur passe de 0 à 1, une lettre différente de plus est présente. ↑énoncé

Ex. 8 q. 2 – début de solution Quand un caractère sort (celui d'indice `i - taille`), retirer 1 à son compteur ; s'il retombe à 0, une lettre différente de moins. On a trouvé le marqueur quand le nombre de lettres différentes vaut `taille`. ↑énoncé

11

CHAPITRE 11

# Les algorithmes gloutons

```
pieces = [200, 100, 50, 20, 10, 5, 2, 1]

def rendre(somme):
    rendu = []
    for p in pieces:
        while somme >= p:
            rendu.append(p)
            somme = somme - p
    return rendu

>>> rendre(38)
[20, 10, 5, 2, 1]
```




DANS CE CHAPITRE


- Activités préparatoires . . . 444
- Cours . . . 446
- Exercices . . . 453
- TP et projets . . . 458
- Fiche de révision . . . 464
- Défi Advent of Code . . . 466
- Corrigés . . . 691

# ACTIVITÉS PRÉPARATOIRES

## ACTIVITÉ 1 La caisse du marchand

▶ corrigé p. 691

*rendre la monnaie avec le moins de pièces possible*

Durée : 20 min · Par deux, sans ordinateur, réponses sur le cahier

### Consignes

- Matériel : de vraies pièces et de vrais billets, ou les jetons à découper en fin d'activité (ou simplement leurs valeurs écrites sur des bouts de papier).
- L'un joue le **marchand** et rend la monnaie ; l'autre **vérifie** (la somme est-elle exacte ? peut-on faire avec moins ?). On échange les rôles à chaque somme.
- Le but : rendre **exactement** la somme, avec le **moins possible** de pièces et de billets. On dispose d'autant de pièces et de billets de chaque valeur qu'on veut.

### Exercice 1 *Rendre la monnaie en euros*

Valeurs disponibles : **1, 2, 5, 10, 20 et 50 €**.

1. Rendre successivement **8 €**, **13 €**, **27 €** et **36 €**. Pour chaque somme, noter sur le cahier les pièces et billets rendus, et leur nombre.
2. Le vérificateur a-t-il trouvé, pour l'une des sommes, une façon de faire avec moins de pièces ? Laquelle ?
3. Pour 36 €, quelle valeur avez-vous posée **en premier** ? Pourquoi celle-là ? Écrire, en une phrase, la **règle** que vous avez suivie pour choisir chaque pièce.

### Exercice 2 *Appliquer la règle sans réfléchir*

4. Appliquer votre règle, sans chercher d'autre solution, pour **49 €**, puis pour **88 €**. Combien de pièces et billets faut-il ?
5. En appliquant la règle, vous est-il arrivé de **reprendre** une pièce déjà posée pour en mettre une autre ?

### Exercice 3 *Au pays de Nsiland*

À Nsiland, les pièces valent **1, 5 et 12 nsis**. Les pièces de 12 sont les plus grosses.

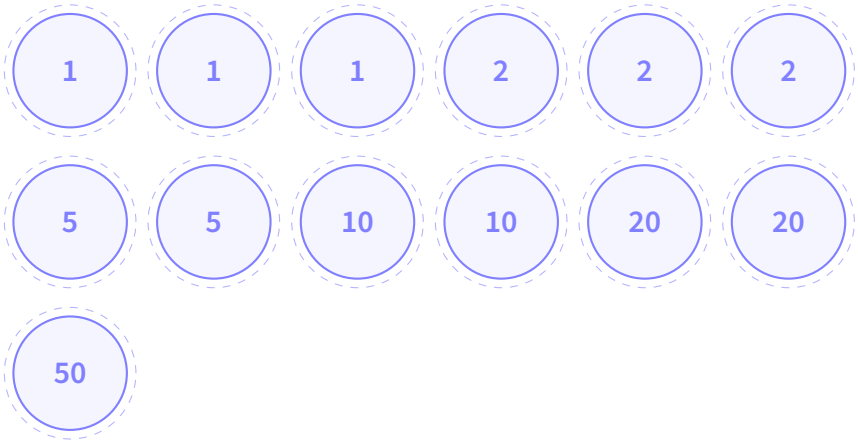
6. Appliquer votre règle pour rendre **15 nsis**. Quelles pièces ? Combien ?
7. Le vérificateur cherche une meilleure solution. Existe-t-elle ? Laquelle ?
8. Trouver une autre somme, entre 16 et 20 nsis, pour laquelle votre règle ne donne pas le moins de pièces possible.

### Exercice 4 *Ce qu'on a découvert*

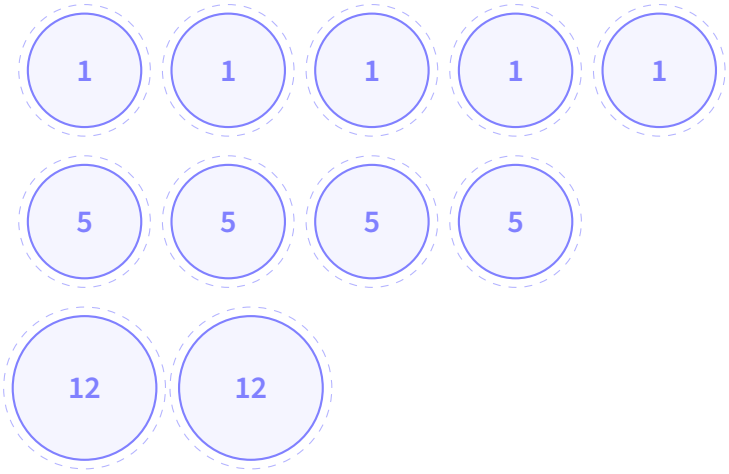
9. Recopier et compléter sur le cahier, avec vos mots.
   - À chaque étape, la règle consiste à prendre …
   - Une fois une pièce posée, on …
   - Avec les euros, la règle donne …
   - Avec les pièces de Nsiland, la règle donne …

**Jetons à découper** (si l'on n'a pas de vraies pièces)

Euros




Nsiland (nsis)



# COURS

**Programme (BO)** *Algorithmique* : résoudre un **problème d'optimisation** à l'aide d'un **algorithme glouton** (exemple emblématique : le *rendu de monnaie*).

**Prérequis** les boucles `for`/`while`, les listes, écrire une fonction ; la **notion de coût** (chapitre *Algorithmique : le parcours séquentiel*) ; l'**invariant du champion** (garder le meilleur vu jusque-là). → fiches de révision : ch. 4 p. 202

**Objectifs** reconnaître un **problème d'optimisation** ; comprendre et appliquer la **stratégie gloutonne** ; la **programmer** sur le rendu de monnaie ; savoir qu'un algorithme glouton est **rapide mais pas toujours optimal** (voire pas toujours une solution).

---

**Situation de départ**

1. Un commerçant doit rendre 48 € avec le moins de pièces et de billets possible : quelle méthode simple peut-il suivre ?
2. Prendre à chaque étape ce qui paraît le meilleur sur le moment donne-t-il toujours le meilleur résultat final ?
3. Pourquoi un ordinateur ne peut-il pas simplement essayer tous les trajets possibles pour trouver la tournée la plus courte d'un livreur qui passe par 20 villes ?

**Remarque** — *Le fil conducteur : le meilleur choix, ici et maintenant*
Un algorithme glouton (*greedy* en anglais) construit une solution par une **suite de choix**, en prenant à chaque étape ce qui **paraît le meilleur sur le moment**, **sans jamais revenir en arrière**. C'est la stratégie de celui qui, à un buffet, remplit son assiette en attrapant à chaque fois le plat le plus appétissant devant lui. C'est **simple** et **rapide** — mais, on le verra, cette myopie a un prix : le résultat n'est **pas toujours le meilleur possible**. Les points qui dépassent le programme portent le badge [ au-delà du programme ].



## I Les problèmes d'optimisation

**Définition 1 — Problème d'optimisation**

Un **problème d'optimisation** demande, parmi de **nombreuses** solutions valides, d'en trouver une **la meilleure possible** selon un critère. Il faut donc :

- un ensemble de **solutions valides** (celles qui respectent les contraintes) ;
- une **mesure de qualité** (une quantité à *minimiser* ou *maximiser*).

Une **solution optimale** est une meilleure solution ; une **solution approchée** est une bonne solution, pas forcément la meilleure.

**Exemple** — *Trois problèmes d'optimisation*

- **rendu de monnaie** : rendre une somme avec le *moins de pièces possible* ;
- **sac à dos** : remplir un sac de capacité limitée en emportant le *plus de valeur possible* ;
- **voyageur de commerce** : visiter des villes et revenir au départ en parcourant le *moins de kilomètres possible*.

**Pour les curieux** — *Pourquoi ne pas tout essayer ? L'explosion combinatoire* (facultatif)

On pourrait **énumérer toutes** les solutions et garder la meilleure. Mais leur nombre **explose** : pour le voyageur, il y a déjà 181 440 circuits différents pour 10 villes, environ 240 millions pour 13, et plus de 60 *millions de milliards* pour 20 (en partant d'une ville fixée, on peut visiter les $n-1$ autres dans $(n-1)\times(n-2)\times\cdots\times 1$ ordres, et chaque circuit est compté deux fois, une fois dans chaque sens). Même un ordinateur ne peut pas les parcourir. Le tableau ci-dessous donne la taille $n$ traitable en environ un milliard d'opérations selon le coût de l'algorithme :

| Coût | $n$ traitable ($\approx 10^9$ opérations) |
|---|---|
| $n$ (linéaire) | 1 000 000 000 |
| $n^2$ (quadratique) | environ 31 600 |
| $2^n$ (exponentiel) | environ 30 |
| $n!$ (factoriel) | 12 |

Face à ces problèmes, on renonce souvent à *la* meilleure solution pour une **bonne** solution obtenue **vite**. C'est là qu'intervient la méthode gloutonne.

► **Exercices d'application :** exercice 1 (problèmes d'optimisation)

## II La stratégie gloutonne

### Règle 1 — Le principe glouton

On construit la solution **morceau par morceau**. À chaque étape :

1. parmi les choix encore **possibles** (faisables), on prend celui qui **optimise le critère localement** (le « meilleur sur le moment ») ;
2. on **ne remet jamais en cause** un choix déjà fait ;
3. on s'arrête quand la solution est complète.

↻ **Retour sur l'activité.** Dans l'activité « La caisse du marchand », vous n'avez jamais eu à reprendre une pièce déjà posée : c'est le point 2 du principe.

Pour appliquer la méthode à un problème, il faut donc décider :

- quels sont les **candidats** (les morceaux qu'on ajoute) ;
- le **critère de choix glouton** (comment on classe « le meilleur sur le moment ») ;
- la **contrainte de faisabilité** (ce qui reste possible).

### À retenir — La force et la faiblesse, en une phrase

Ne faire qu'**un** choix à chaque étape (au lieu de tout explorer) rend l'algorithme **très rapide**. Mais ce choix est **myope** : le meilleur coup immédiat peut mener à une solution globale médiocre. *Rapide n'est pas optimal.*

► **Exercices d'application :** exercice 2 (le principe glouton)

## III L'exemple phare : le rendu de monnaie

Un commerçant doit rendre une somme avec le **moins de pièces et billets possible**. On dispose des coupures en euros (on oublie les centimes) : 1, 2, 5, 10, 20, 50, 100, 200.

### Règle 2 — La stratégie gloutonne pour le rendu de monnaie

Tant qu'il reste une somme à rendre, on choisit **la plus grande coupure qui ne dépasse pas** la somme restante (le choix qui fait *décroître le plus vite* ce qui reste à rendre).

↻ **Retour sur l'activité.** C'est la règle que vous avez formulée dans l'activité : pour 36 €, on pose d'abord le billet de 20, puis 10, 5 et 1, soit 4 pièces et billets.

**Exemple** — *Rendre 9 €*
Somme = 9 : la plus grande coupure ≤ 9 est **5** (reste 4) ; puis **2** (reste 2) ; puis **2** (reste 0). Résultat : $5 + 2 + 2$, soit **3 pièces**. On peut vérifier qu'on ne fait pas mieux : c'est **optimal**.

### Règle 3 — Le rendu de monnaie glouton en Python

```
1  pieces = [200, 100, 50, 20, 10, 5, 2, 1]   # de la plus grande a la plus petite
2
3  def rendre(somme):
4      rendu = []                  # INITIALISER
5      for p in pieces:            # PARCOURIR : de la plus grande a la plus petite
6          while somme >= p:       # TESTER : la coupure "tient" encore
7              rendu.append(p)     # METTRE A JOUR
8              somme = somme - p   # METTRE A JOUR
9      return rendu                # RENVOYER
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> rendre(9)
[5, 2, 2]
>>> rendre(48)
[20, 20, 5, 2, 1]
>>> len(rendre(48))
5
```

▷ **Je prédis.** On range par erreur les coupures dans l'ordre **croissant** : `pieces = [1, 2, 5, 10, 20, 50, 100, 200]`. Que renvoie alors `rendre(9)` ?
▷ **Je vérifie.** `[1, 1, 1, 1, 1, 1, 1, 1, 1]` : la pièce de 1 « tient » neuf fois et vide la somme avant que les autres coupures soient essayées. Le résultat est juste (9 € rendus), mais avec 9 pièces au lieu de 3 : l'ordre des candidats fait partie de l'algorithme.

**Remarque** — *Le réflexe coût*
On parcourt les coupures de la plus grande à la plus petite **une seule fois**, et pour chacune on retire autant de fois que possible : l'algorithme est **très rapide**. On peut d'ailleurs remplacer la boucle `while` par une division : le nombre de coupures `p` est `somme // p`, et il reste `somme % p`.

**Propriété 1 — Les euros forment un système « canonique »**

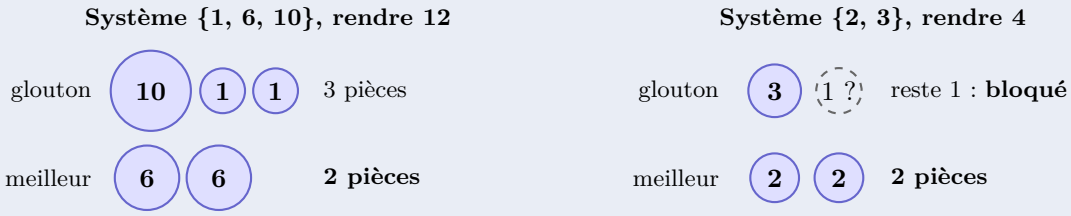
Avec le système des euros, l'algorithme glouton donne **toujours** la solution **optimale**. On dit que ce système est **canonique**. *(Ce n'est pas une évidence : cela se démontre, coupure par coupure.)*

**Remarque** — *Attention : ce n'est pas toujours le cas ! Deux contre-exemples*
Le glouton n'est optimal **que** pour certains systèmes de pièces.

- **Système {1, 6, 10}, rendre 12.** Le glouton prend 10, puis 1, puis 1 : **3 pièces**. Mais $6+6$ ne fait que **2 pièces** ! Le glouton donne une solution *correcte mais pas optimale.*
- **Système {2, 3}, rendre 4.** Le glouton prend 3… il reste 1, impossible à rendre : le glouton **échoue**, alors que $2+2$ marchait ! Ici il ne trouve *aucune* solution pourtant existante.

**Leçon :** un algorithme glouton doit toujours être *justifié* sur le problème précis ; on ne peut pas supposer qu'il est optimal.

↻ **Retour sur l'activité.** Les pièces de Nsiland (1, 5 et 12 nsis) de l'activité formaient un autre contre-exemple : pour 15 nsis, la règle donnait $12+1+1+1$, soit 4 pièces, au lieu de $5+5+5$ ; il en allait de même pour 16 et pour 20 nsis.




► **Exercices d'application :** exercices 3 (les ingrédients du rendu de monnaie) et 4 à 9, 15 (le rendu de monnaie)

## IV Un deuxième exemple : le sac à dos

On dispose d'objets ayant chacun une **valeur** et un **poids**, et d'un sac de **capacité** limitée. On veut emporter le **plus de valeur possible** sans dépasser la capacité.

**Exemple** — *Un petit sac à dos*
Capacité du sac : **5 kg**. Objets disponibles :

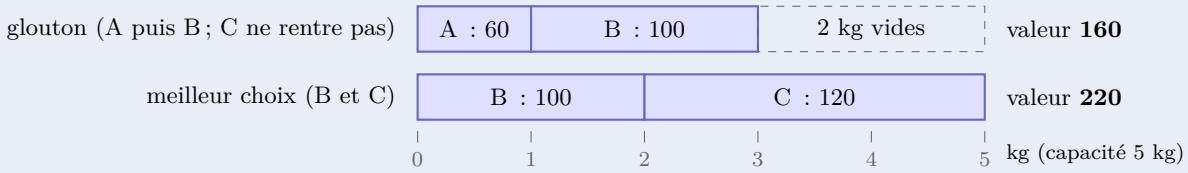
| Objet | Valeur | Poids | Valeur / poids |
|---|---|---|---|
| A | 60 | 1 kg | 60 |
| B | 100 | 2 kg | 50 |
| C | 120 | 3 kg | 40 |

Quel **critère glouton** choisir ? Prendre d'abord l'objet le plus *utile au kilo*, c'est-à-dire le meilleur rapport **valeur / poids**. Ordre : A (60), B (50), C (40) — on **trie** donc les objets par rapport décroissant (chapitre *Les algorithmes de tri*).

- on prend **A** (1 kg, valeur 60) ; il reste 4 kg ;
- on prend **B** (2 kg, valeur 100) ; il reste 2 kg ;
- **C** pèse 3 kg : il ne rentre pas. On s'arrête.

Butin : A + B, poids 3 kg, **valeur 160**.

**Remarque** — *Encore une fois : rapide, mais pas garanti optimal*
Ici le meilleur choix réel était **B + C** (5 kg, valeur **220**) : le glouton, en prenant A « le plus rentable au kilo » d'abord, est passé à côté. Le glouton par rapport valeur/poids est **optimal si l'on peut couper les objets** (sac à dos « fractionnaire »), mais **pas** quand on doit prendre chaque objet en entier ou pas du tout. *Le même piège que le rendu de monnaie.*

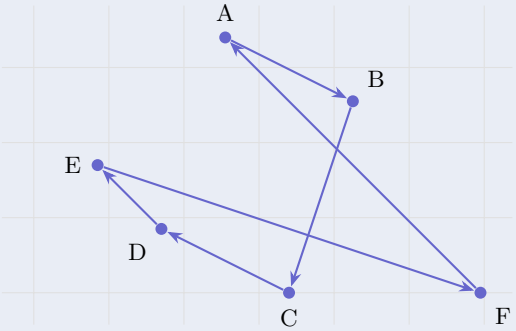



► **Exercices d'application :** exercices 10, 11 et 16 (le sac à dos)

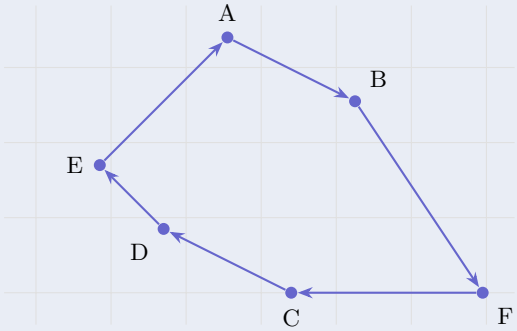
## V Quand le glouton se trompe : le voyageur de commerce

Le **voyageur de commerce** doit visiter toutes les villes d'une liste et revenir à son point de départ en parcourant le **moins de kilomètres**. La stratégie gloutonne naturelle : **aller à chaque fois vers la ville non visitée la plus proche**.

**Remarque** — *Une bonne solution, pas la meilleure*
Ce glouton « du plus proche voisin » donne **très vite** un itinéraire correct, souvent bon, mais **rarement optimal** : en fonçant toujours vers le plus proche, on peut laisser une ville isolée qu'il faudra rejoindre par un long détour final. C'est le cas typique où le glouton sert d'**approximation** rapide, faute de pouvoir tout essayer.




glouton « plus proche voisin » :
`A B C D E F A`, longueur ≈ 21,0




meilleure tournée :
`A B F C D E A`, longueur ≈ 15,3

Parti de A, le glouton enchaîne les sauts courts (B, C, D, E)… en oubliant F, isolée au sud-est. Il doit alors traverser toute la carte pour aller la chercher, puis revenir en A : deux longues traversées. La meilleure tournée passe par F « en chemin », juste après B. (Longueurs à vol d'oiseau, un carreau = une unité.)

**Pour les curieux** — *Un problème « difficile » et une question à un million de dollars* (facultatif)
Le voyageur de commerce fait partie des problèmes dits **NP-complets** : on **sait vérifier** vite qu'un itinéraire donné est court, mais personne ne **sait trouver** la solution optimale rapidement quand il y a beaucoup de villes. Savoir si un algorithme rapide existe pour *tous* ces problèmes est la fameuse question **« P = NP ? »**, non résolue à ce jour : un prix d'**un million de dollars** récompense qui la tranchera. En attendant, pour ces problèmes, les algorithmes gloutons (et d'autres heuristiques) sont précieux : ils donnent une bonne réponse en un temps raisonnable.

► **Exercices d'application :** exercice 12 (le plus proche voisin)

## VI Un peu d'histoire

**Pour les curieux** — *Des télécommunications à la conjecture du siècle* (facultatif)

Les stratégies « au plus pressé » sont étudiées dès les années 1950, bien avant d'avoir un nom : l'expression « algorithme glouton » (*greedy algorithm*) ne se répand qu'autour de 1970 (on l'attribue souvent au mathématicien Jack Edmonds, en 1971). En **1952**, **David Huffman**, étudiant au MIT, invente un algorithme glouton de **compression** de données (le codage de Huffman) qui sert encore aujourd'hui. En **1956–1957**, **Joseph Kruskal** et **Robert Prim** publient des algorithmes gloutons pour relier des points au moindre coût (les « arbres couvrants », utiles pour les réseaux électriques et de télécommunications). Le glouton est donc né avec des applications très concrètes.



*Stephen Cook en 1968*

Le revers de la médaille — les problèmes que le glouton ne sait qu'*approcher* — a donné naissance en **1971** (**Stephen Cook**) à la théorie des problèmes **NP-complets**, et à la question **P = NP**, l'un des sept « problèmes du millénaire » dotés d'un prix d'un million de dollars. Le mathématicien américain **Stephen Cook**, alors jeune professeur (ci-contre en 1968, à Berkeley), montre en 1971 qu'un problème de logique (le problème SAT) est « NP-complet » : savoir le résoudre vite permettrait de résoudre vite tous les problèmes d'une très vaste famille. Dès 1972, Richard Karp en exhibe 21 autres, dont le sac à dos ; on en connaît aujourd'hui des milliers, dont le voyageur de commerce, qui sont en fait *le même* problème déguisé : savoir en résoudre un vite permettrait de les résoudre tous. Cette découverte vaut à Cook le prix Turing en 1982 (Karp le recevra en 1985). *Ainsi, un chapitre de Première touche à l'une des plus grandes questions ouvertes de l'informatique.*

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise.** Les candidats sont triés (*Les algorithmes de tri*) ; à chaque étape, on garde le meilleur, comme le champion du *Parcours séquentiel* ; les $2^n$ choix d'une exploration complète sont les $2^n$ nombres de $n$ bits (*Le binaire et l'écriture des nombres*).
- **En Terminale.** *La programmation dynamique* résoudra de façon exacte les problèmes où le glouton échoue (pièces $\{1, 6, 10\}$, sac à dos), et l'algorithme de Dijkstra (*Les graphes*) sera un glouton prouvé optimal.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Problème d'optimisation | trouver, parmi beaucoup de solutions valides, la *meilleure* (min ou max) |
| Solution optimale / approchée | la meilleure / une bonne, pas forcément la meilleure |
| Explosion combinatoire | tout essayer est impossible (millions, milliards… de solutions) |
| Stratégie gloutonne | suite de choix ; à chaque étape le *meilleur choix local* ; jamais revenir en arrière |
| Ingrédients | candidats + critère de choix + contrainte de faisabilité |
| Force / faiblesse | **rapide et simple** ; mais **pas toujours optimal** (ni même une solution) |
| Rendu de monnaie | prendre la plus grande coupure ≤ somme restante |
| Système canonique | système où le glouton est *toujours* optimal (ex. les euros) |
| Contre-exemples | {1,6,10} rendre 12 (sous-optimal) ; {2,3} rendre 4 (échec) |

### Retour à la situation de départ

1. La **stratégie gloutonne** : prendre à chaque fois la plus grande coupure qui ne dépasse pas la somme restante, soit $20+20+5+2+1$ (5 pièces et billets). Les euros formant un système **canonique**, c'est optimal (§ III).
2. Non. Avec les pièces $\{1, 6, 10\}$, rendre 12 donne $10+1+1$ au lieu de $6+6$, et avec $\{2, 3\}$, rendre 4 échoue (§ III) ; pour le sac à dos et le voyageur de commerce aussi, le glouton donne une solution seulement **approchée** (§ IV et V).
3. Le nombre de circuits **explose** : plus de 60 millions de milliards pour 20 villes, trop pour un ordinateur (§ I). On se contente d'une bonne solution obtenue vite, par exemple avec le glouton du plus proche voisin (§ V).

## Erreurs fréquentes

- **Croire que le glouton est toujours optimal.** Il faut le *justifier* pour chaque problème.
- **Oublier de trier / parcourir les candidats dans le bon ordre** (coupures de la plus grande à la plus petite ; objets par valeur/poids décroissant).
- **Confondre « pas optimal » et « pas de solution ».** Le glouton peut donner une solution non optimale ($\{1,6,10\}$) ou carrément échouer ($\{2,3\}$).
- **Vouloir revenir en arrière** : un algorithme glouton, par définition, ne revient jamais sur un choix.
- **Confondre valeur et poids** dans le sac à dos : le critère glouton est le *rapport* valeur/poids.

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

☐ reconnaître un problème d'optimisation et dire ce qu'on minimise ou maximise ;
☐ énoncer le principe glouton (choix local, sans retour en arrière) et ses ingrédients ;
☐ **dérouler** et **programmer** le rendu de monnaie glouton ;
☐ donner un **contre-exemple** où le glouton n'est pas optimal (ou échoue) ;
☐ appliquer un critère glouton à un autre problème (sac à dos, plus proche voisin) ;
☐ expliquer pourquoi le glouton est *rapide* mais *pas toujours* optimal.

# EXERCICES

## Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et tester* en Python. Le symbole >>> figure une saisie dans la console.
- Coupures en euros disponibles (on oublie les centimes) : 1, 2, 5, 10, 20, 50, 100, 200.

## Problèmes d'optimisation et principe glouton ↑ cours

★ **Exercice 1** *Qu'optimise-t-on ?* ▶ corrigé p. 692 • essentiel

Pour chaque problème, dire ce qu'on cherche à **minimiser** ou à **maximiser**.

a) rendre une somme avec des pièces ;
b) remplir un sac à dos de capacité limitée ;
c) choisir un trajet passant par plusieurs villes ;
d) répartir des fichiers sur des CD sans dépasser leur capacité.

▷ coup de pouce

★ **Exercice 2** *Vrai ou faux* ▶ corrigé p. 692

Répondre et justifier en une phrase.

a) Un algorithme glouton fait, à chaque étape, le meilleur choix local.
b) Un algorithme glouton revient en arrière s'il s'aperçoit qu'il s'est trompé.
c) Un algorithme glouton donne toujours la solution optimale.
d) Un algorithme glouton est en général rapide.

▷ coup de pouce

★★ **Exercice 3** *Les ingrédients* ▶ corrigé p. 692

Pour le rendu de monnaie glouton, identifier : (a) les **candidats**, (b) le **critère de choix glouton**, (c) la **contrainte de faisabilité**.

▷ coup de pouce

## Rendu de monnaie ↑ cours

★ **Exercice 4** *Dérouler à la main* ▶ corrigé p. 692 • essentiel

Avec les coupures en euros, appliquer la stratégie gloutonne et donner la liste des coupures rendues, puis leur nombre :

a) 8 € b) 27 € c) 63 € d) 199 €

▷ coup de pouce

★★ **Exercice 5** ▶ *La fonction* `rendre` ▶ corrigé p. 692

On donne la fonction du cours :

```
1 pieces = [200, 100, 50, 20, 10, 5, 2, 1]
2 def rendre(somme):
3     rendu = []
4     for p in pieces:
5         while somme >= p:
6             rendu.append(p)
7             somme = somme - p
8     return rendu
```

a) Tester `rendre(48)` et `rendre(88)`, puis utiliser la fonction pour vérifier vos réponses de l'exercice précédent (8, 27, 63 et 199 €).
b) Écrire une fonction `nb_pieces(somme)` qui renvoie le **nombre** de coupures rendues (sans construire la liste).
c) [ au-delà du programme ] Réécrire la boucle `while` à l'aide d'une division : le nombre de coupures `p` vaut `somme // p`, et il reste `somme % p`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 6** *Un système qui piège le glouton* ▶ corrigé p. 693 • essentiel

Dans un pays imaginaire, les pièces valent **1, 5 et 8**. On rend la somme **10**.

a) Quelle solution donne l'algorithme glouton (plus grande pièce d'abord) ? Combien de pièces ?
b) Existe-t-il une meilleure solution ? Laquelle ?
c) Que peut-on conclure sur l'optimalité du glouton pour ce système ? ▷ coup de pouce

★★ **Exercice 7** *Quand le glouton échoue carrément* ▶ corrigé p. 693

Les pièces valent **4 et 5**. On veut rendre **8**.

a) Dérouler l'algorithme glouton. Que se passe-t-il ?
b) Une solution existe-t-elle pourtant ? Laquelle ?
c) Expliquer la différence entre « le glouton n'est pas optimal » et « le glouton échoue ».

▷ coup de pouce

★ **Exercice 8** *Le rendu de monnaie en désordre* ▶ corrigé p. 693

Les lignes de la fonction `rendre_general(systeme, somme)`, qui applique le rendu glouton pour un système de pièces quelconque (rangé de la plus grande à la plus petite), ont été mélangées et ont perdu leur indentation.

```
return rendu
somme = somme - p
def rendre_general(systeme, somme):
while somme >= p:
rendu = []
rendu.append(p)
for p in systeme:
```

a) Remettre les lignes dans l'ordre et les indenter correctement.
b) Que renvoie `rendre_general([4, 3, 1], 6)` ? Est-ce le moins de pièces possible ?

▷ coup de pouce

★★★ **Exercice 9** ▶ *Tester si un système est canonique* ▶ corrigé p. 693 ≈ 20 min

Un système est **canonique** si le glouton y est toujours optimal. On ne teste ici que les sommes de 1 à 30.

a) Écrire `rendre_sys(systeme, somme)` qui applique le glouton pour un système de pièces quelconque (rangé de la plus grande à la plus petite pièce) et renvoie le nombre de pièces (ou `None` si le glouton n'aboutit pas).
b) On suppose disponible une fonction `optimal(systeme, somme)` qui renvoie le *vrai* nombre minimal de pièces (on ne l'écrit pas ici : une version pour trois pièces est programmée dans le TP du chapitre). Écrire une fonction `tester(systeme)` qui compare, pour toutes les sommes de 1 à 30, `rendre_sys` et `optimal`, et affiche la première somme où le glouton se trompe. Sans `optimal`, on ne peut pas l'exécuter : prévoir à la main ce qu'elle afficherait pour le système `[10, 6, 1]`. ▷ coup de pouce ▷ début de solution

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Le sac à dos ↑ cours

★★ **Exercice 10** *Remplir le sac* ▶ corrigé p. 694 • essentiel

Sac de capacité **4 kg**. Objets : A (valeur 30, 2 kg), B (valeur 20, 2 kg), C (valeur 24, 3 kg).

a) Calculer le rapport **valeur / poids** de chaque objet et les classer du meilleur au moins bon.
b) Appliquer le glouton (meilleur rapport d'abord, tant que ça rentre). Quels objets emporte-t-on, pour quel poids et quelle valeur ?
c) En essayant toutes les combinaisons qui tiennent dans 4 kg, vérifier qu'on ne fait pas mieux. Le glouton était-il optimal ici ? ▷ coup de pouce

★★ **Exercice 11** *Un sac à dos où le glouton se trompe* ▶ corrigé p. 694

Sac de capacité **6 kg**. Objets : A (valeur 30, 1 kg), B (valeur 60, 3 kg), C (valeur 72, 4 kg).

a) Appliquer le glouton par rapport valeur/poids. Quelle valeur obtient-on ?
b) Quelle est la meilleure combinaison possible ? De combien le glouton se trompe-t-il ?

▷ coup de pouce

## Le plus proche voisin (voyageur) ↑ cours

★★ **Exercice 12** *Toujours vers le plus proche* ▶ corrigé p. 694

Quatre villes, distances (en km) :

| | A | B | C | D |
|---|---|---|---|---|
| A | — | 10 | 15 | 20 |
| B | 10 | — | 35 | 25 |
| C | 15 | 35 | — | 30 |
| D | 20 | 25 | 30 | — |

On part de **A**, on visite toutes les villes et on revient à A, en allant à chaque fois vers la ville **non visitée la plus proche**.

a) Dérouler l'algorithme glouton : donner l'itinéraire et la distance totale.
b) L'itinéraire A–C–B–D–A mesure-t-il plus ou moins que celui du glouton ? Que peut-on dire de la qualité de la solution gloutonne *ici* (et est-ce garanti en général) ? ▷ coup de pouce

## Mini-problème de synthèse ↑ cours

★★★ **Exercice 13** *Le distributeur de billets* ▶ corrigé p. 694 ≈ 15 min

Un distributeur ne contient que des billets de **50, 20 et 10** euros. On retire une somme (multiple de 10).

a) Appliquer le glouton pour 90 €, puis pour 80 €. Combien de billets à chaque fois ?
b) Ce système {10, 20, 50} est-il canonique pour les multiples de 10 jusqu'à 100 ? (tester à la main ou en réutilisant `rendre_sys`).
c) Le distributeur n'a plus de billets de 20. Pour 80 €, le glouton avec {10, 50} donne-t-il encore la solution qui utilise le moins de billets ? ▷ coup de pouce ▷ début de solution

★★ **Exercice 14** *Le rendu de monnaie selon un assistant d'IA* ▶ corrigé p. 694

Un élève demande à un assistant d'IA : « Écris une fonction Python qui rend une somme avec le moins de pièces possible. » Voici la réponse obtenue.

```
def rendre(somme, pieces):
    """pieces : liste des coupures, triee par ordre decroissant"""
    rendu = []
    for p in pieces:
        while somme >= p:
            rendu.append(p)
            somme = somme - p
    return rendu
```

« Cette fonction est gloutonne : à chaque étape, elle prend la plus grande pièce possible. Elle renvoie toujours le nombre minimal de pièces, quel que soit le système de pièces fourni, puisqu'elle utilise à chaque fois la coupure la plus grande. »

a) La réponse est-elle correcte ? Vérifier à la main avec le système {4, 3, 1} et la somme 6.
b) Localiser l'erreur (dans le code ou dans l'explication) et la corriger.
c) Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce

## Entraînement ↑ cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

★ **Exercice 15** *Le glouton dans d'autres systèmes* ▶ corrigé p. 695 • essentiel

Pour chaque système de pièces et chaque somme, appliquer le rendu glouton, donner les pièces rendues et leur nombre, puis dire s'il existe une solution avec moins de pièces.

a) système {1, 2, 5, 10}, somme 14 ;
b) système {1, 4, 6}, somme 8 ;
c) système {1, 5, 12}, somme 15 ;
d) système {1, 10, 25}, somme 30.

▷ coup de pouce

★ **Exercice 16** *Le sac à dos glouton* ▶ corrigé p. 695 • essentiel

Pour chaque sac, calculer le rapport valeur/poids des objets, appliquer le glouton (meilleur rapport d'abord, tant que l'objet rentre), puis comparer à la meilleure combinaison possible.

a) capacité 10 kg ; objets A (valeur 40, 5 kg), B (valeur 30, 4 kg), C (valeur 12, 3 kg) ;
b) capacité 6 kg ; objets A (valeur 30, 4 kg), B (valeur 18, 3 kg), C (valeur 18, 3 kg).

▷ coup de pouce

## Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 q. 4 – piste Pour chaque problème, chercher la quantité qui mesure la qualité d'une solution (un nombre de pièces, une valeur, une distance…) : veut-on qu'elle soit la plus petite ou la plus grande possible ? ↑ énoncé

Ex. 2 q. 4 – piste Relire la définition de la stratégie gloutonne dans le cours : à chaque étape, combien de choix fait-on, et peut-on les remettre en cause ensuite ? Pour c), penser à un contre-exemple. ↑ énoncé

Ex. 3 – piste Relire les trois ingrédients du cours et se demander, pour chacun, ce qu'il devient quand on rend une somme : qu'ajoute-t-on à la solution, comment le choisit-on, qu'est-ce qui interdit un choix ? ↑ énoncé

Ex. 4 – piste À chaque étape, prendre la plus grande coupure qui ne dépasse pas la somme restante, puis recalculer ce qui reste. La même coupure peut servir plusieurs fois. ↑énoncé

Ex. 5 q. 3 – piste Pour b), garder la structure de `rendre`, mais remplacer la liste `rendu` par un compteur qui part de 0. ↑énoncé

Ex. 6 q. 3 – piste Pour b), chercher une solution qui n'utilise pas la pièce de 8. ↑énoncé

Ex. 7 q. 3 – piste Suivre la somme restante après chaque pièce prise : que vaut-elle quand plus aucune pièce ne convient ? ↑énoncé

Ex. 8 q. 2 – piste C'est la fonction `rendre` de l'exercice 5 (« La fonction `rendre` »), avec le système en paramètre. Quelle ligne doit précéder la boucle, laquelle vient tout à la fin ? ↑énoncé

Ex. 9 q. 2 – piste a) Partir de `rendre_general` : compter les pièces au lieu de les ranger dans une liste ; à la fin, si la somme restante n'est pas nulle, le glouton n'a pas abouti. b) Une boucle sur les sommes de 1 à 30, et une comparaison à chaque tour. ↑énoncé

Ex. 9 q. 2 – début de solution
```
def rendre_sys(systeme, somme):
    n = 0
    for p in systeme:
        ...
```
↑énoncé

Ex. 10 q. 3 – piste Pour c), lister toutes les combinaisons possibles (un objet seul, deux objets…) et éliminer celles qui dépassent 4 kg. ↑énoncé

Ex. 11 q. 2 – piste Après avoir pris les objets de meilleur rapport, regarder la place qui reste : l'objet suivant y tient-il ? Pour b), comparer toutes les combinaisons qui tiennent dans 6 kg. ↑énoncé

Ex. 12 q. 2 – piste À chaque étape, ne lire que la ligne de la ville où l'on se trouve, en barrant les villes déjà visitées. Ne pas oublier le retour à A. ↑énoncé

Ex. 13 q. 3 – piste Pour b), comparer, pour chaque somme 10, 20, …, 100, le nombre de billets du glouton à la meilleure solution trouvée à la main. ↑énoncé

Ex. 13 q. 3 – début de solution a) Pour 90 €, le glouton prend 50, puis 20 et encore 20 : 3 billets. c) Le glouton commence par 50 : que reste-t-il à rendre, et avec quels billets ? ↑énoncé

Ex. 14 q. 3 – piste Comparer le code à celui du cours. S'il est identique, l'erreur est dans une phrase de l'explication : laquelle est contredite par l'exercice « Un système qui piège le glouton » ? ↑énoncé

Ex. 15 – piste Le glouton commence toujours par la plus grande pièce qui ne dépasse pas la somme restante. Pour chercher mieux, essayer de ne pas prendre cette plus grande pièce. ↑énoncé

Ex. 16 – piste Après chaque objet pris, noter la place qui reste. Un objet qui ne rentre pas est simplement sauté. Pour la meilleure combinaison, lister toutes celles qui respectent la capacité. ↑énoncé

# TP ET PROJETS

## TP Le glouton à l'épreuve ▶ corrigé p. 695

*monnaie, sac à dos, tournée sur la Riviera : programmer, comparer à l'optimum, mesurer*

Durée : 2 à 3 h (deux séances) · Seul ou en binôme, sur machine

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p11-tp-glouton-epreuve` — **archive .zip**

### But du TP

Le cours affirme qu'un algorithme glouton est **rapide** mais **pas toujours optimal**. On va le **vérifier soi-même**, chiffres à l'appui, sur trois problèmes d'optimisation. Pour chacun : on programme le glouton, on calcule la **vraie** meilleure solution par **force brute** (en essayant tout, ce qui n'est possible que sur de petites tailles), puis on **mesure** l'écart entre les deux — et le temps que coûte la force brute. Le produit final est un **tableau de bilan** rempli avec vos propres mesures.

### Consignes

- Fichier à télécharger (lien ci-dessus) : `tp_gloutons_depart.py` (tout votre code s'y écrit).
- ▶ *À programmer et tester.* En bas du fichier, `test_partie1()`, `test_partie2()` et `test_partie3()` vérifient vos fonctions : à chaque lancement, chaque partie affiche `[OK]` ou `[A FAIRE]` (au début, tout est `[A FAIRE]` : c'est normal).
- Notez au fur et à mesure vos résultats et vos réponses sur le cahier : ils servent au bilan final (partie 4). Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône ⧉ en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Usage de l'IA, indiqué sur chaque exercice : IA aucune ; IA en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; IA l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

## 1 Rendu de monnaie : canonique ou pas ?

Un système de pièces est **canonique** si le glouton y rend *toujours* le nombre minimal de pièces. Les euros le sont ; le système $\{1, 6, 10\}$ ne l'est pas. Comment le savoir pour un système quelconque ? En comparant, somme par somme, le glouton à la **vraie** solution optimale.

★ **Exercice 1** IA *Le glouton fourni* ▶

La fonction `rendu_glouton(systeme, somme)` est donnée : c'est celle du cours, mais elle renvoie le **nombre** de pièces, ou `None` quand il reste une somme impossible à rendre.

1. Prévoir à la main, puis vérifier : `rendu_glouton([10, 6, 1], 12)` et `rendu_glouton([3, 2], 4)`.

2. Pourquoi le test `if somme != 0` est-il nécessaire en fin de fonction ?

★★ **Exercice 2** *La vraie solution, par force brute*▶

Pour un système de **trois** pièces `[p1, p2, p3]`, on peut essayer *toutes* les façons de rendre la somme : $a$ pièces `p1` ($a$ de 0 à `somme // p1`), puis $b$ pièces `p2`, avec $b$ de 0 jusqu'au nombre maximal de pièces `p2` qui tiennent dans ce qui reste ; le reste doit alors se faire **exactement** en pièces `p3`.

1. Pour `[10, 6, 1]` et la somme 12, recopier et compléter le tableau des essais :

| $a$ (pièces de 10) | $b$ (pièces de 6) | reste (en pièces de 1) | nombre de pièces |
|---|---|---|---|
| 0 | 0 | 12 | 12 |
| 0 | 1 | … | … |
| 0 | 2 | … | … |
| 1 | 0 | … | … |

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

2. Compléter `optimal3(systeme, somme)`, qui renvoie le plus petit nombre de pièces trouvé, ou `None` si aucun essai ne tombe juste. ▷ coup de pouce

★★ **Exercice 3** *Chercher un contre-exemple*▶

Compléter `premier_contre_exemple(systeme, smax)`, qui renvoie la plus petite somme de 1 à `smax` où le glouton ne donne pas l'optimum (ou `None`). Puis recopier sur le cahier et compléter le tableau suivant (`smax = 100`) :

| système | {1,2,5} | {1,6,10} | {1,3,4} | {1,5,7} | {1,10,25} | {1,10,20} | {5,10,25} |
|---|---|---|---|---|---|---|---|
| contre-exemple | | | | | | | |
| pièces (glouton ; optimum) | | | | | | | |

Pour chaque contre-exemple trouvé, noter le nombre de pièces du glouton puis celui de l'optimum, par exemple « 3 ; 2 ». S'il n'y en a pas, écrire « aucun » dans la ligne « contre-exemple ». *Attention : la fonction attend le système trié du plus grand au plus petit, par exemple* `[10, 6, 1]`.

★★ **Exercice 4** *La chasse aux systèmes non canoniques*▶

On examine **tous** les systèmes $\{1, a, b\}$ avec $2 \leqslant a < b \leqslant 20$.

1. Écrire une double boucle qui compte combien de ces systèmes il y a, et combien ne sont **pas** canoniques (en testant les sommes de 1 à 100). Un système « pris au hasard » a-t-il des chances d'être canonique ?
2. Pour chaque système non canonique, comparer le plus petit contre-exemple à $a + b$. Que constate-t-on ? *(On admet que ce constat est un théorème : pour trois pièces, il suffit de tester les sommes inférieures à $a + b$.)*

★★★ **Exercice 5** *Une règle cachée, et deux histoires vraies*▶

1. Écrivons $b = q \times a + r$ (division euclidienne : `q, r = b // a, b % a`). En affichant `a`, `b`, `q`, `r` pour les systèmes canoniques et non canoniques, chercher une règle qui permet de prévoir, *sans rien essayer*, si $\{1, a, b\}$ est canonique. La vérifier sur les 171 systèmes.
2. Aux États-Unis, on rend la monnaie avec des pièces de 1, 5, 10 et 25 cents. Si la pièce de 5 cents disparaissait, le système $\{1, 10, 25\}$ serait-il encore canonique ? (Appliquer la règle de la question 1, et comparer avec le tableau de l'exercice « Chercher un contre-exemple ».)
3. Avant 1971, le Royaume-Uni comptait en *pence* avec des pièces de 1, 3, 6, 12, 24 et 30 pence. Avec `rendu_glouton`, rendre 48 pence. Trouver à la main une solution en moins de pièces. Ce système était-il canonique ?

▷ coup de pouce

## 2 Le sac à dos du randonneur

Pour un bivouac dans le Mercantour, Léa peut porter **10 kg**. Chaque objet a un poids et un **intérêt** (en points, selon Léa). Elle veut un sac d'intérêt total **maximal**, sans dépasser 10 kg.

| objet | tente | duvet | réchaud | nourriture | trousse | gourde | photo | jumelles | doudoune | livre |
|---|---|---|---|---|---|---|---|---|---|---|
| poids (kg) | 4 | 2 | 2 | 3 | 1 | 2 | 1 | 1 | 2 | 1 |
| intérêt | 75 | 60 | 30 | 70 | 40 | 55 | 25 | 10 | 45 | 5 |

Dans le programme, un objet est un triplet `(nom, poids, intérêt)` ; la liste `OBJETS` et la constante `CAPACITE` sont fournies.

★ **Exercice 6** *Trois critères gloutons*

Le glouton prend les objets **un par un dans un certain ordre**, et met chaque objet dans le sac s'il rentre encore. Trois ordres sont naturels : par intérêt **décroissant**, par poids **croissant**, par rapport intérêt/poids **décroissant**.

1. Calculer à la main le rapport intérêt/poids de chaque objet et les classer.
2. Dérouler à la main le glouton « par rapport » : quels objets sont pris, pour quel poids et quel intérêt ?

★★ **Exercice 7** *Programmer le glouton*▶

1. Compléter `rapport(objet)`, puis `sac_glouton(objets, capacite, critere, decroissant)`. Le paramètre `critere` est une **fonction** (`interet`, `poids` ou `rapport`) qui sert de clé de tri : `sorted(objets, key=critere, reverse=decroissant)`.
2. Lancer les trois gloutons sur `OBJETS`. Noter pour chacun les objets pris et l'intérêt total. Votre déroulé à la main était-il juste ?

★★ **Exercice 8** *L'optimum, en comptant en binaire*▶

Avec $n$ objets, il y a $2^n$ sacs possibles (chaque objet est pris ou non). On les numérote de 0 à $2^n - 1$ : écrit en **binaire** sur $n$ bits, le numéro $k$ dit quels objets on prend (bit $i$ égal à 1 : l'objet $i$ est dans le sac). Par exemple, avec 3 objets, $k = 5$, qui s'écrit `101` en binaire, désigne le sac « objet 0 et objet 2 ».

1. Avec 3 objets, quels sacs désignent $k = 3$ et $k = 6$ ? Combien de sacs pour les 10 objets de Léa ?
2. Compléter `sac_optimal(objets, capacite)` : pour chaque $k$, construire le sac, et le garder s'il ne dépasse pas la capacité et bat le meilleur sac trouvé jusque-là. ▷ coup de pouce
3. Quel est le sac optimal ? Lequel des trois gloutons l'a trouvé ? De combien de points (et de quel pourcentage) chacun s'en écarte-t-il ? Le choix fait par l'optimum vous semble-t-il raisonnable pour un bivouac ? Qu'en conclure sur le *modèle* ?

★★ **Exercice 9** *Mesurer sur 200 sacs au hasard*▶

Une instance ne prouve rien : on va mesurer sur **200 instances** de 10 objets tirées au hasard (fonction `objets_au_hasard` fournie), avec une capacité de 15 kg. Écrire une boucle qui, pour chaque instance, calcule l'optimum et le résultat des trois gloutons, puis affiche pour chaque critère : le **nombre de fois** où il trouve l'optimum, et l'**écart moyen** en pourcentage, $\frac{\text{optimum} - \text{glouton}}{\text{optimum}} \times 100$. Écrire `random.seed(2026)` avant la boucle pour que tout le monde obtienne les mêmes instances. ▷ coup de pouce

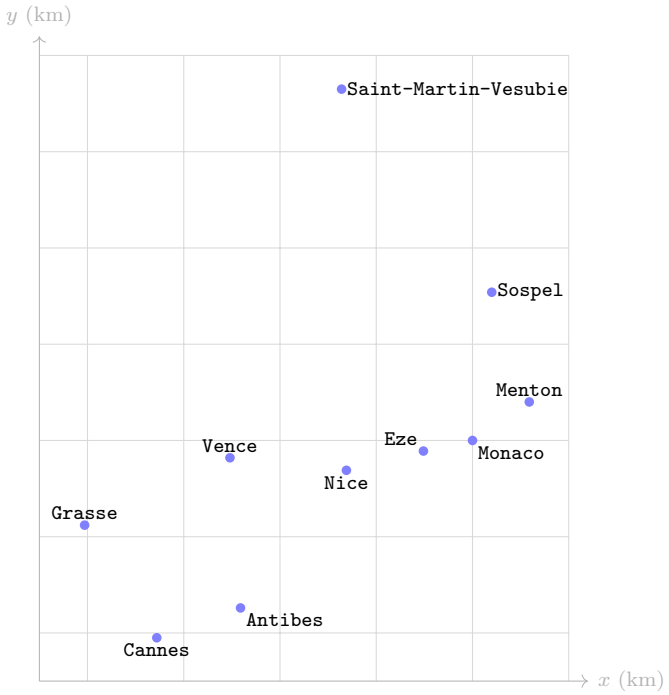
★★ **Exercice 10** *Le prix de la force brute*▶

Avec `time.time()` (appelé avant et après), mesurer la durée de `sac_optimal` pour $n = 10, 12, 14, 16, 18, 20$ objets (capacité : `3 * n // 2`). Par combien le temps est-il multiplié quand on ajoute **deux** objets ? En déduire une estimation pour 30 objets, puis pour 40. Combien de temps prend le glouton pour 40 objets ?

## 3 Une tournée sur la Riviera

Un livreur part de Monaco, doit passer une fois dans chacune des neuf autres villes ci-dessous, puis **revenir** à Monaco, en parcourant le moins de kilomètres possible (distances à vol d'oiseau). Les coordonnées, en km, sont rangées dans le dictionnaire `VILLES` ; la fonction `distance(v1, v2)` est fournie.

| ville | $x$ | $y$ |
|---|---|---|
| `Monaco` | 0,0 | 0,0 |
| `Menton` | 5,9 | 4,0 |
| `Sospel` | 2,0 | 15,4 |
| `Eze` | −5,1 | −1,1 |
| `Nice` | −13,1 | −3,1 |
| `Vence` | −25,2 | −1,8 |
| `Grasse` | −40,3 | −8,8 |
| `Cannes` | −32,8 | −20,5 |
| `Antibes` | −24,1 | −17,4 |
| `Saint-Martin-Vesubie` | −13,6 | 36,5 |




★ **Exercice 11** *Longueur d'une tournée*▶

Compléter `longueur(tournee)`, qui additionne les distances entre villes consécutives **sans oublier le retour** de la dernière ville à la première. Vérifier : `longueur(["Monaco", "Menton", "Sospel"])` vaut environ 34,7 km.

★★ **Exercice 12** *Le plus proche voisin*▶

1. Compléter `plus_proche_voisin(depart, villes)` : depuis la ville où l'on se trouve, aller vers la ville **non visitée la plus proche** (invariant du champion), la retirer de `a_visiter`, recommencer.
2. Afficher la tournée obtenue depuis Monaco et sa longueur. La dessiner sur le cahier, sur un croquis à main levée de la carte. Quelle étape paraît maladroite ?

★★ **Exercice 13** *L'optimum, et ce qu'il coûte*▶

1. Compléter la fonction `tournee_optimale`. La fonction `tous_les_ordres(autres)`, **fournie** dans le fichier, renvoie la liste de **tous les ordres** possibles des 9 autres villes (elle insère chaque ville, l'une après l'autre, à toutes les positions possibles). Combien y en a-t-il ? Mesurer la durée du calcul.
2. Afficher la tournée optimale et sa longueur ; la dessiner sur le même croquis, d'une autre couleur. Calculer l'écart du glouton en pourcentage.
3. Pour 15 villes, il y aurait $14 \times 13 \times \cdots \times 2 \times 1$ ordres à essayer. Estimer la durée du calcul sur votre machine.

★★ **Exercice 14** *Et si l'on partait d'ailleurs ?*▶

Une tournée revient à son point de départ : on peut la commencer dans n'importe quelle ville. Lancer le plus proche voisin depuis **chacune** des dix villes et garder la tournée la plus courte. Quelle longueur obtient-on ? Quel est le nouvel écart à l'optimum ? Combien de fois a-t-on appelé `plus_proche_voisin`, comparé aux ordres essayés par la force brute ?

★★★ **Exercice 15** *Décroiser la tournée*▶

Sur votre croquis, la tournée gloutonne se **croise**-t-elle ? Un circuit qui se croise n'est jamais optimal : en **retournant** le morceau de tournée situé entre les deux segments qui se croisent, on raccourcit. Écrire `ameliorer(tournee)` qui, tant que c'est possible, cherche deux positions $i < j$ (avec $i \geqslant 1$ pour garder la ville de départ) telles que retourner `tournee[i:j+1]` raccourcisse la tournée, et effectue ce retournement. L'appliquer à la tournée gloutonne depuis Monaco : quelle longueur obtient-on ? ▷ coup de pouce *Les longueurs sont des flottants, calculés avec de petites erreurs d'arrondi : ne compter comme amélioration qu'un gain d'au moins 1 mètre, c'est-à-dire tester* `longueur(essai) < longueur(tournee) - 0.001`.

**Repère historique — le voyageur de commerce, de 49 villes à 85 900 points**

**1954** — À la RAND Corporation, **George Dantzig**, **Ray Fulkerson** et **Selmer Johnson** trouvent la tournée **optimale** de **49 villes** des États-Unis. Ils n'essaient pas tous les ordres (il y en a plus de $10^{60}$) : ils construisent une tournée, puis **prouvent** qu'aucune autre n'est plus courte.

**1958** — **G. A. Croes**, ingénieur chez Shell, publie la méthode de l'exercice « Décroiser la tournée » : retourner un morceau de tournée tant que cela raccourcit le trajet. On l'appelle aujourd'hui **2-opt** ; elle reste l'une des améliorations les plus utilisées.

**2006** — Le logiciel **Concorde** (Applegate, Bixby, Chvátal et Cook) résout exactement un problème de **85 900 points**, issu du dessin d'une puce électronique, au prix de plus de 136 années de calcul processeur réparties sur de nombreuses machines. Pour des millions de points (livraisons, perçage de circuits), on se contente encore de bonnes tournées approchées, obtenues par un glouton puis améliorées par décroisement.

*Sources :* G. Dantzig, D. R. Fulkerson, S. M. Johnson, *Operations Research*, vol. 2, 1954 ; G. A. Croes, *Operations Research*, vol. 6, 1958 ; W. J. Cook, *In Pursuit of the Traveling Salesman*, Princeton University Press, 2012.

## 4 Bilan

**Exercice 16** *Le tableau de bilan*

Recopier sur le cahier et remplir avec **vos** mesures.

| problème | glouton | optimum | écart | durée de la force brute |
|---|---|---|---|---|
| monnaie {1,6,10}, somme 12 | | | | — |
| sac de Léa (meilleur critère) | | | | |
| 200 sacs au hasard (critère « rapport ») | optimum trouvé … fois | | moyen : | — |
| tournée depuis Monaco | | | | |
| tournée, meilleur départ | | | | — |

Puis répondre sur le cahier, en quelques lignes : (1) dans quel cas le glouton est-il **garanti** optimal ? (2) Quand la force brute devient-elle impossible, et pourquoi ? (3) Dans quelle situation réelle préféreriez-vous une solution gloutonne à une solution optimale ?

### Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 2 q. 2 – piste Dans la boucle sur `b` : calculer `reste = somme - a * p1 - b * p2` ; si `reste % p3 == 0`, l'essai réussit avec `a + b + reste // p3` pièces : c'est un candidat pour le minimum (invariant du champion, avec `None` au départ). ↑énoncé

Ex. 5 – piste Comparer, pour chaque système, le reste `r` à `a - q`. Commencer par les systèmes où `r` vaut 0. ↑énoncé

Ex. 8 q. 2 – piste `(k // 2 ** i) % 2` est le bit numéro `i` de `k` (le chiffre des unités en binaire est le bit 0). ↑énoncé

Ex. 9 – piste Deux dictionnaires indexés par le nom du critère (`"interet"`, `"poids"`, `"rapport"`) : l'un compte les succès, l'autre cumule les écarts ; on divise par 200 à la fin. ↑énoncé

Ex. 15 – piste La compréhension `[tournee[j - k] for k in range(j - i + 1)]` donne les villes des positions `j`, `j - 1`, …, `i` : c'est le morceau retourné. La tournée essayée est alors `tournee[:i] + retourne + tournee[j + 1:]`. Une variable `amelioration` (booléenne) dit s'il faut refaire un passage. ↑énoncé

# FICHE DE RÉVISION

## L'idée maîtresse

Un algorithme **glouton** construit une solution par une **suite de choix** : à chaque étape, le **meilleur choix local**, **sans jamais revenir en arrière**. C'est **rapide et simple**, mais **pas toujours optimal** (parfois même pas de solution du tout).

## Problème d'optimisation

Trouver, parmi de **nombreuses** solutions valides, la **meilleure** selon un critère (à *minimiser* ou *maximiser*). Tout énumérer est souvent **impossible** (explosion combinatoire : millions, milliards de solutions).

## Les ingrédients d'un glouton

- les **candidats** (les morceaux qu'on ajoute) ;
- le **critère de choix** glouton (« le meilleur sur le moment ») ;
- la **contrainte de faisabilité** (ce qui reste possible).

## Le rendu de monnaie (exemple phare)

Critère : prendre la **plus grande coupure** $\leq$ somme restante.

```
pieces = [200, 100, 50, 20, 10, 5, 2, 1]   # de la plus grande a la plus petite
def rendre(somme):
    rendu = []
    for p in pieces:
        while somme >= p:
            rendu.append(p)
            somme = somme - p
    return rendu
```

**Système canonique** : système où le glouton est *toujours* optimal (ex. les **euros**). Ce n'est pas toujours le cas !

- $\{1, 6, 10\}$, rendre 12 : glouton $10 + 1 + 1$ (3), mieux $6 + 6$ (2) $\Rightarrow$ **non optimal** ;
- $\{2, 3\}$, rendre 4 : glouton prend 3, reste 1 $\Rightarrow$ **échoue**, alors que $2 + 2$ marchait.

*« Pas optimal »* (une solution, pas la meilleure) $\neq$ *« échoue »* (aucune solution trouvée).

## Le sac à dos

Maximiser la valeur sans dépasser la capacité. Critère glouton : meilleur **rapport valeur/poids** d'abord, tant que ça rentre. **Pas toujours optimal** quand les objets sont indivisibles : prendre le plus « rentable » d'abord peut gaspiller la place restante.

## Le voyageur de commerce

Visiter toutes les villes au moindre kilométrage. Glouton « **plus proche voisin** » : aller à chaque fois vers la ville non visitée la plus proche. Rapide, souvent bon, **rarement optimal** sur beaucoup de villes. Problème *difficile* (NP-complet) : d'où l'intérêt des solutions **approchées**.

## Erreurs à éviter

- croire que le glouton est **toujours optimal** (il faut le justifier) ;
- parcourir les candidats dans le mauvais ordre (coupures décroissantes ; objets par valeur/-poids) ;
- confondre « pas optimal » et « échoue » ;
- vouloir revenir en arrière (un glouton ne le fait jamais).

### Savoir-faire à maîtriser — je sais…

- ☐ reconnaître un problème d'optimisation et dire ce qu'on minimise ou maximise ;
- ☐ énoncer le principe glouton (choix local, sans retour en arrière) et ses ingrédients ;
- ☐ **dérouler** et **programmer** le rendu de monnaie glouton ;
- ☐ donner un **contre-exemple** où le glouton n'est pas optimal (ou échoue) ;
- ☐ appliquer un critère glouton à un autre problème (sac à dos, plus proche voisin) ;
- ☐ expliquer pourquoi le glouton est *rapide* mais *pas toujours* optimal.

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 700

1. Quelles sont les deux idées clés d'un algorithme glouton ?
2. Que renvoie `rendre(37)` avec les euros ?
3. Pièces $\{1, 3, 4\}$, somme 6 : solution gloutonne ? meilleure solution ?
4. Sac à dos : selon quel critère le glouton classe-t-il les objets ?
5. Un glouton a trouvé une solution : est-elle forcément la meilleure ?

**Et avant ?** — *questions sur les chapitres précédents.* ► réponses p. 700

1. Dans le protocole du bit alterné, à quoi sert le bit `0/1` ? (ch. 10)
2. Comment trier une table (liste de dictionnaires) dans l'ordre alphabétique du descripteur `"nom"` ? (ch. 8)
3. Python est-il un langage compilé ou interprété ? (ch. 5)

# DÉFI ADVENT OF CODE

## DÉFI Lobby

*les batteries de l'escalator — un algorithme glouton*

Jour 3

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p11-defi-aoc-2025-03` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2025/day/3`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** Chaque ligne du fichier est une suite de chiffres. Il faut y choisir quelques chiffres, sans changer leur ordre, pour former le plus grand nombre possible : deux chiffres dans la partie 1, douze dans la partie 2, sur deux cents lignes de cent chiffres. Essayer tous les choix est envisageable pour deux chiffres, mais totalement impossible pour douze : il faut une méthode plus intelligente.

**Ce qu'on réinvestit.** Les **algorithmes gloutons** de ce chapitre : on construit le nombre chiffre après chiffre, de gauche à droite, en prenant à chaque fois le meilleur chiffre possible, sans revenir en arrière. Comme un glouton ne donne pas toujours la meilleure solution, il faudra aussi **justifier** qu'il est correct ici, et le vérifier en le comparant à une recherche exhaustive.

**La démarche de la fiche.**

1. Comprendre la règle avec des questions courtes, puis chercher à la main le meilleur nombre sur quatre lignes.

2. Résoudre la partie 1 en essayant tous les couples de chiffres : une méthode lente mais sûre.
3. Écrire une version gloutonne et vérifier qu'elle donne les mêmes résultats.
4. Pour la partie 2, constater que l'essai de tous les choix est impossible, puis généraliser le glouton à $k$ chiffres.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `battery` | pile, batterie | `bank` | rangée, groupe |
| `joltage rating` | tension indiquée (1 à 9) | `turn on` | allumer, activer |
| `exactly two` | exactement deux | `rearrange` | réordonner |
| `largest possible` | le plus grand possible | `total output` | total produit |

### Résumé de l'énoncé (partie 1)

**Contexte.** Dans le hall d'un immeuble, les ascenseurs sont en panne, et même l'escalier mécanique est à l'arrêt faute de courant. Des batteries de secours peuvent l'alimenter. Elles sont rangées en groupes, et chaque batterie porte une tension de 1 à 9. Il faut tirer le maximum de chaque groupe.
**Les données.** Chaque ligne du fichier décrit un groupe de batteries : c'est une suite de chiffres de 1 à 9, sans espace, par exemple `4815`. Chaque chiffre est la tension d'une batterie, et les chiffres sont dans l'ordre où les batteries sont rangées.
**La règle.**

- Dans chaque groupe, on allume **exactement deux** batteries.
- Le groupe produit le nombre à deux chiffres formé par leurs tensions, **dans l'ordre de la ligne** : la batterie la plus à gauche donne le chiffre des dizaines. On ne peut pas réordonner les batteries.
- Pour chaque groupe, on cherche le plus grand nombre que l'on peut produire ainsi.

**La réponse attendue.** La somme de ces maxima sur toutes les lignes. Par exemple, dans le groupe `4815`, on peut former 48, 41, 45, 81, 85 ou 15 : le maximum est 85.

**Exercice 1** *Comprendre ce qui est demandé*

**But.** Vérifier sur des cas particuliers qu'on a bien compris la contrainte d'ordre et la façon de choisir les deux chiffres.
**À quoi ça sert ?** Ces trois cas contiennent déjà l'idée de la méthode gloutonne : le chiffre des dizaines ne peut pas être le dernier de la ligne, et en cas d'égalité il faut bien choisir lequel prendre.

Après lecture du résumé (et de l'énoncé), vérifier sa compréhension sur ces cas particuliers :

1. Dans la ligne `19`, peut-on obtenir `91` ?
2. Si le plus grand chiffre de la ligne est le dernier caractère, peut-il servir de chiffre des dizaines ?
3. Dans `9399`, quels chiffres faut-il choisir ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Trouver à la main le meilleur nombre de deux chiffres pour quatre lignes, et leur somme.
**À quoi ça sert ?** On obtient un exemple dont on **connaît la réponse** (394), qui servira à tester les deux méthodes de la partie 1. En cherchant, on découvre aussi comment on procède naturellement : c'est souvent un glouton.

On considère le fichier (inventé) suivant :

```
372819465123987
111119111181111
564738291564738
999888777666555
```

Pour chaque ligne, trouver le plus grand nombre de deux chiffres possible. Vérification : on doit obtenir `394`. Pour la troisième ligne, pourquoi ne peut-on pas obtenir `99` ?

## 2 Programmer la partie 1

### Exercice 3 *Lire et tester toutes les paires*

**But.** Lire le fichier et résoudre la partie 1 en essayant tous les couples de chiffres possibles.
**À quoi ça sert ?** Cette méthode exhaustive est simple et sûre : elle donne la réponse de la partie 1 et servira de référence pour vérifier la méthode gloutonne, plus rapide, de la question suivante.

#### Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Rappel pour lire un fichier ligne par ligne :

```
fichier = open("input.txt", encoding="utf-8")
for ligne in fichier:
    ligne = ligne.strip()   # enlève le retour à la ligne final
fichier.close()
```

Écrire une fonction `meilleur_par_paires(ligne)` qui essaie tous les couples d'indices `i < j` et renvoie le plus grand nombre `int(ligne[i] + ligne[j])`. Calculer la réponse (`394` sur l'exemple).

▷ coup de pouce

#### Méthode : position du plus grand caractère

Les chiffres se comparent très bien sous forme de caractères : `"9" > "8"` vaut `True`. Pour trouver l'indice de la **première** apparition du plus grand caractère de `ligne` entre les indices `debut` (inclus) et `fin` (exclu), on parcourt cette zone en mémorisant la position du meilleur caractère vu jusque-là.

```
def position_max(ligne, debut, fin):
    """Indice de la premiere apparition du plus grand caractere
    parmi ligne[debut], ..., ligne[fin - 1]."""
    p = debut
    for i in range(debut + 1, fin):
        if ligne[i] > ligne[p]:      # strict : en cas d'egalite, on garde la premiere
```

```
            p = i
    return p


ligne = "38193"
print(position_max(ligne, 0, 5))   # 3 : le 9 est a l'indice 3
print(position_max(ligne, 0, 3))   # 1 : le 8, parmi "381"
```

**Intérêt.** Une seule boucle donne à la fois le plus grand chiffre (`ligne[p]`) et sa position `p` ; les bornes `debut` et `fin` permettent de chercher dans une partie de la ligne seulement, sans la recopier.
*À essayer.* Que renvoient `position_max("9919", 0, 4)` et `position_max("9919", 1, 3)` ? Que renverrait le premier appel si l'on écrivait `>=` au lieu de `>` ?

### Exercice 4 *Une idée gloutonne*

**But.** Écrire une version qui choisit d'abord le meilleur chiffre des dizaines, puis le meilleur chiffre des unités après lui.
**À quoi ça sert ?** Deux parcours au lieu de tous les couples : c'est cette idée, et non la recherche exhaustive, qui pourra être généralisée à douze chiffres dans la partie 2.

Écrire une seconde fonction `meilleur_glouton(ligne)` qui ne fait que deux parcours : choisir d'abord le meilleur chiffre des dizaines, puis le meilleur chiffre des unités *après* lui. Vérifier qu'elle donne les mêmes résultats que la précédente. ▷ coup de pouce ▷ début de solution

**Lien avec le cours : algorithmes gloutons**

Un algorithme **glouton** construit une solution en faisant, à chaque étape, le choix qui semble le meilleur sur le moment, sans jamais revenir en arrière (comme le rendu de monnaie du cours). Il est rapide, mais il ne donne pas toujours la meilleure solution : il faut le **justifier**. Ici, pourquoi est-il correct ? Deux nombres de même longueur se comparent d'abord par leur premier chiffre ; le premier chiffre choisi doit donc être le plus grand *parmi ceux qui laissent assez de chiffres après eux*. Si ce maximum apparaît plusieurs fois, prendre sa *première* apparition laisse le plus de choix pour la suite. Après ce choix, il reste le même problème, plus petit, sur la fin de la ligne.

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

**Résumé de la partie 2 (à lire après avoir validé la partie 1)**

**Ce qui change.** Deux batteries par groupe ne suffisent pas à faire repartir l'escalier : il faut en allumer beaucoup plus.
**La nouvelle règle.**

- Dans chaque groupe, on allume maintenant **exactement 12** batteries.
- Elles forment un nombre de 12 chiffres, toujours dans l'ordre de la ligne, sans réordonner les batteries.
- Pour chaque groupe, on cherche le plus grand nombre de 12 chiffres que l'on peut produire ainsi.

**La réponse attendue.** La somme de ces maxima sur toutes les lignes. Par exemple, pour un groupe de 13 batteries, cela revient à en laisser une seule éteinte : avec `1234567891234`, on éteint le premier `1` et l'on obtient 234567891234.

### Exercice 5 *Combien de possibilités ?*

**But.** Estimer le nombre de choix possibles dans la partie 2.
**À quoi ça sert ?** On constate que la méthode exhaustive de la partie 1 est ici inutilisable : c'est ce qui justifie de généraliser la méthode gloutonne.

Dans la partie 2, on ne choisit plus deux chiffres. Regarder la longueur des lignes de vos données. Pour une ligne de 100 chiffres, essayer toutes les possibilités est-il raisonnable ? (Pour 12 chiffres choisis parmi 100, il y a plus de $10^{15}$ possibilités.)

### Exercice 6 *Le glouton généralisé*

**But.** Écrire une fonction qui choisit `k` chiffres de gauche à droite, avec le même principe glouton qu'en partie 1.
**À quoi ça sert ?** Avec `k = 12`, elle donne la réponse de la partie 2 ; avec `k = 2`, elle doit redonner celle de la partie 1, ce qui permet de la tester.

Écrire une fonction `meilleur(ligne, k)` qui choisit `k` chiffres, de gauche à droite, avec le même principe glouton. La partie 1 correspond à `k = 2`. Sur l'exemple inventé de la fiche, avec `k = 12`, la première ligne donne `819465123987` et le total vaut `2676756647502`. ▷ coup de pouce

▷ début de solution

## 4 Approfondissement

### Outil Python : le hasard (`random`)

Le module `random` tire des nombres au hasard ; `random.randint(a, b)` renvoie un entier entre `a` et `b` inclus.

```
1 import random
2
3 print(random.randint(1, 9))          # un entier au hasard entre 1 et 9 (inclus)
```

**Intérêt.** Pour tester un algorithme rapide, on le compare à une recherche exhaustive lente mais sûre, sur beaucoup de petits cas tirés au hasard.
*À essayer.* Écrire une boucle qui construit une chaîne de 15 chiffres, chacun tiré au hasard entre 1 et 9.



### ★★★ Exercice 7 *Approfondissement 1 : convaincre et tester*

**But.** Vérifier sur de nombreux exemples tirés au hasard que le glouton donne toujours le même résultat qu'une recherche exhaustive, puis justifier pourquoi.
**À quoi ça sert ?** Un algorithme glouton n'est pas toujours correct : on apprend ici à s'en convaincre de deux façons complémentaires, par des tests automatiques et par un raisonnement.

1. Écrire une fonction `exhaustif(ligne, k)` qui essaie tous les choix de `k` chiffres pris dans l'ordre de la ligne et renvoie le plus grand nombre obtenu. ▷ coup de pouce
2. Comparer `meilleur(ligne, k)` et `exhaustif(ligne, k)` sur 300 lignes de 15 chiffres tirées au hasard, avec `k` lui aussi tiré au hasard entre 1 et 15 (`random.randint(1, 15)`). Combien de désaccords ?
3. Rédiger en quelques lignes la justification du glouton pour `k` quelconque.
4. Que se passerait-il si l'on prenait la *dernière* apparition du maximum ? Tester la ligne `"9919"` avec `k = 3`. ▷ coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Deux boucles imbriquées, la seconde commençant à `i + 1`, et un calcul de maximum. ↑

Ex. 4 – piste Le chiffre des dizaines compte dix fois plus que celui des unités : il faut le prendre le plus grand possible. Mais il ne peut pas être le dernier caractère de la ligne (il faut laisser une place pour les unités). ↑énoncé

Ex. 4 – début de solution Avec la fonction `position_max` de l'encadré : `p = position_max(ligne, 0, len(ligne) - 1)` donne le chiffre des dizaines (le dernier caractère est exclu), puis `q = position_max(ligne, p + 1, len(ligne))` celui des unités. Le résultat est `int(ligne[p] + ligne[q])`. ↑énoncé

Ex. 6 – piste Quand il reste `r` chiffres à choisir, le prochain chiffre doit laisser au moins `r - 1` chiffres après lui : on le cherche donc entre l'indice `debut` (juste après le chiffre choisi précédemment) et l'indice `len(ligne) - r` inclus. ↑énoncé

Ex. 6 – début de solution Une boucle `for r in range(k, 0, -1):` ; la recherche se fait avec `p = position_max(ligne, debut, len(ligne) - r + 1)` ; on ajoute `ligne[p]` à une chaîne `resultat`, puis on avance `debut` juste après : `debut = p + 1`. ↑énoncé

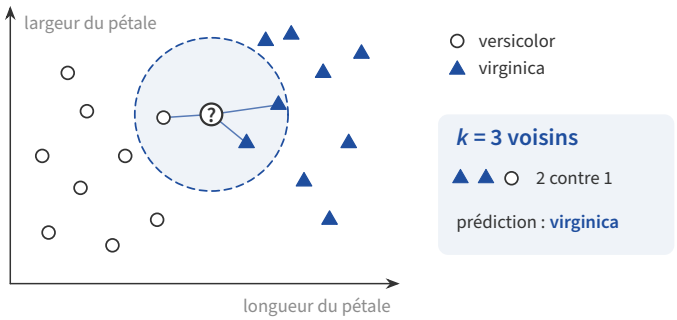
Ex. 7 q. 1 – piste Construire la liste de tous les débuts de choix : partir de `[""]` puis, pour chaque caractère `c` de la ligne, remplacer chaque début `s` par deux débuts, `s` (on ne prend pas `c`) et `s + c` (on le prend, si `s` a moins de `k` caractères). À la fin, garder le plus grand des choix de longueur `k`. ↑énoncé

Ex. 7 q. 4 – piste Prendre une apparition plus tardive laisse moins de chiffres après elle. ↑énoncé

12

CHAPITRE 12

# Les $k$ plus proches voisins






DANS CE CHAPITRE


- Activités préparatoires . . . . . 473
- Cours . . . . . 475
- Exercices . . . . . 485
- TP et projets . . . . . 491
- Fiche de révision . . . . . 503
- Défi Advent of Code . . . . . 505
- Corrigés . . . . . 702

# ACTIVITÉS PRÉPARATOIRES

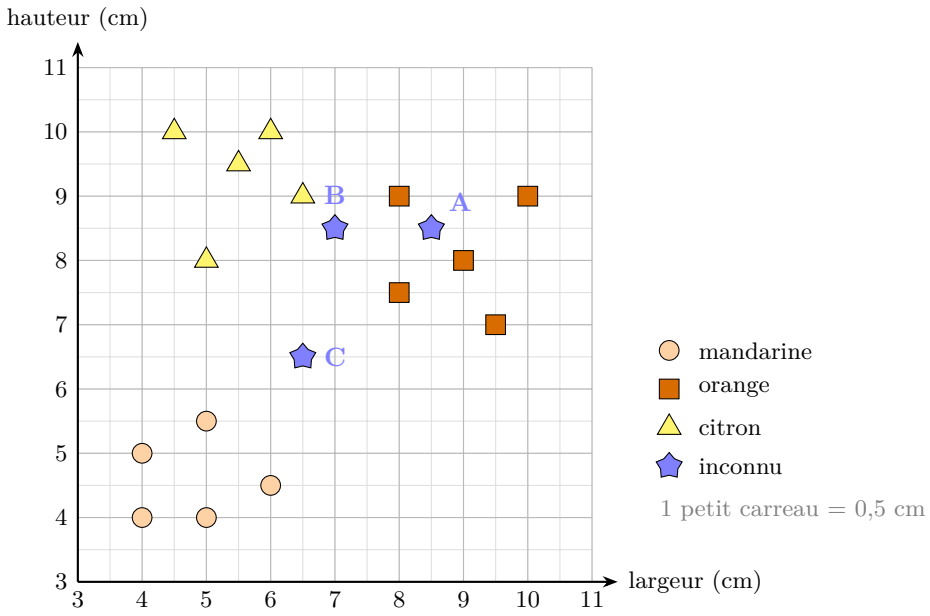
## ACTIVITÉ 1 Quel est ce fruit ? ▶ corrigé p. 702

*classer un fruit inconnu d'après ceux qu'on connaît déjà*

Durée : 25 min · Par binômes, sans machine

### Consignes

- Matériel : le graphique ci-dessous, le cahier pour les réponses, une calculatrice.
- Un primeur a mesuré 15 fruits dont il connaît la sorte : des **mandarines** (○), des **oranges** (□) et des **citrons** (△). Pour chacun, il a noté la **largeur** et la **hauteur** (en cm), et l'a placé comme un point sur le graphique.
- Trois nouveaux fruits, **A**, **B** et **C** (⋆), ont été mesurés, mais leur étiquette est tombée. Il faut deviner leur sorte **uniquement** à partir du graphique.




### Exercice 1 *Lire le nuage de points*

1. Décrire en une phrase où se trouvent les mandarines, les oranges, les citrons. Qu'est-ce qui distingue un citron d'une orange ? une orange d'une mandarine ?
2. D'après vous, quelle est la sorte du fruit **A** ? Comment l'avez-vous décidé ?

### Exercice 2 *Demander aux voisins*

3. Règle 1 : « *un fruit inconnu est de la même sorte que le fruit connu le plus proche* ». Quel est le fruit connu le plus proche de **B** ? Quelle sorte prédit la règle 1 ?
4. Règle 3 (le numéro de la règle est le nombre de voisins consultés) : « *on regarde les 3 fruits connus les plus proches, et on prend la sorte majoritaire* ». Repérer ces 3 fruits autour de B (noter leurs coordonnées). Quelle sorte prédit la règle 3 ?

**5.** Même question avec les **5** fruits les plus proches de B.

**6.** Les règles ne sont pas d'accord ! Laquelle vous semble la plus fiable ? Que risque-t-on si l'on ne regarde qu'**un** voisin ? Et si l'on regardait les 15 fruits ?

## Exercice 3 *Le fruit qui fâche*

**7.** À l'œil, quel est le fruit connu le plus proche de **C** ? Comparer avec le binôme voisin.

**8.** Pour trancher sans se disputer, il faut **mesurer** « être proche ». Proposer une façon de mesurer l'écart entre deux points du graphique.

**9.** Entre C et la mandarine de coordonnées (5 ; 5,5), compter le décalage horizontal et le décalage vertical (en cm). Faire de même entre C et l'orange (8 ; 7,5). Qu'observe-t-on ?

**10.** Avec ces deux décalages, comment obtenir la longueur du segment qui relie les deux points, « à vol d'oiseau » ? (Penser à un triangle rectangle.) Calculer cette longueur pour la mandarine et pour l'orange.

**11.** Le plus proche voisin ne permet pas de décider pour C. Que proposer ? Essayer la règle 3 : quelle sorte obtient-on ?

## Exercice 4 *Mettre des mots*

**12.** Recopier sur le cahier en complétant les trous **(a)** à **(e)** : « Pour deviner la sorte d'un fruit inconnu, on cherche parmi les fruits … **(a)** ceux qui sont les plus … **(b)** de lui, et on prend la sorte … **(c)** parmi eux. Il faut choisir … **(d)** on regarde de voisins, et une façon de mesurer la … **(e)** entre deux fruits. »

**13.** Un ordinateur ne « voit » pas le graphique. De quoi a-t-il besoin pour appliquer la même méthode ?

# COURS

**Programme (BO)** *Algorithmique* : écrire un algorithme de **classification** par la méthode des $k$ **plus proches voisins** ($k$-NN).

**Prérequis** **listes** et **dictionnaires** ; **patron de parcours** et **invariant du champion** ; **tri** (`sorted` et clé) ; **données en tables** (un individu = une fiche de descripteurs).
→ fiches de révision : ch. 3 p. 170, ch. 4 p. 202, ch. 6 p. 272, ch. 8 p. 342

**Objectifs** comprendre la **classification supervisée** ; définir une **distance** entre deux individus ; écrire l'algorithme des $k$ **plus proches voisins** ; comprendre la **frontière de décision** et **choisir** $k$ ; **évaluer** un classifieur (entraînement / test) ; connaître ses **forces et limites** ; l'appliquer à la **reconnaissance d'images**.

---

**Situation de départ**

1. Comment un ordinateur peut-il reconnaître un chiffre écrit à la main, sans qu'on lui ait écrit une règle pour chaque chiffre ?
2. Peut-on mesurer à quel point deux fleurs « se ressemblent » ?
3. Pour deviner l'espèce d'une fleur, vaut-il mieux consulter la seule fleur connue qui lui ressemble le plus, ou plusieurs ? Combien ?

**Remarque** — *Le fil conducteur : « qui se ressemble s'assemble »*
Comment un ordinateur reconnaît-il un chiffre écrit à la main, recommande-t-il un film, ou devine-t-il l'espèce d'une fleur ? L'idée des $k$-NN est d'une simplicité désarmante : pour classer un individu **inconnu**, on regarde les individus **déjà connus qui lui ressemblent le plus**, et on lui attribue l'**étiquette majoritaire** parmi eux. « Dis-moi qui sont tes voisins, je te dirai qui tu es. » Tout le chapitre consiste à rendre cette phrase **précise** et **programmable** — jusqu'à **reconnaître des chiffres et des lettres manuscrits** en fin de chapitre. Les points qui dépassent le programme portent le badge [ au-delà du programme ].



## I Apprendre à une machine à reconnaître

Reconnaître un visage, filtrer un courriel indésirable, diagnostiquer une maladie sur une radio, suggérer une chanson : derrière ces prouesses se cache souvent la même idée, l'**apprentissage automatique** (*machine learning*). Plutôt que de *programmer* explicitement toutes les règles, on **montre des exemples** à la machine et on la laisse **généraliser**.

**Définition 1 — Classification supervisée**

On dispose d'un ensemble d'**exemples déjà étiquetés** : pour chacun, on connaît ses **descripteurs** (des mesures) *et* sa **classe** (son étiquette). On reçoit ensuite un **nouvel individu** dont on connaît les descripteurs mais **pas** la classe, et on veut la **prédire**. C'est de l'**apprentissage supervisé** : la machine apprend « sous la supervision » d'exemples corrigés.

↺ **Retour sur l'activité.** Dans l'activité « Quel est ce fruit ? », le primeur disposait ainsi de 15 fruits déjà étiquetés (mandarine, orange ou citron), décrits par deux descripteurs, la largeur et la hauteur ; il fallait prédire la sorte de A, B et C.

> **Pour les curieux** — *Supervisé, non supervisé* (facultatif)
> Si les exemples sont **étiquetés** (on connaît la bonne réponse), on parle d'apprentissage **supervisé** — c'est le cas du $k$-NN. Quand aucune étiquette n'est fournie et que la machine doit *regrouper* seule les individus qui se ressemblent, on parle d'apprentissage **non supervisé** (*clustering*).

Le $k$-NN est sans doute l'algorithme d'apprentissage supervisé **le plus simple** qui soit — et pourtant redoutablement efficace sur beaucoup de problèmes. C'est une excellente porte d'entrée dans le monde de l'intelligence artificielle.

## II Le jeu de données : l'exemple des iris

En apprentissage, tout part d'un **jeu de données** (*dataset*) : un tableau où chaque **ligne** est un individu et chaque **colonne** un descripteur, plus une colonne d'**étiquette**. On retrouve exactement les *données en tables* (chapitre *Les données en tables*) : un individu = une fiche.

**Exemple** — *Le célèbre jeu de données Iris*

Le jeu **Iris** (botanique, 1936) décrit **150 fleurs** d'iris réparties en **3 espèces** (*setosa*, *versicolor*, *virginica*), par **4 mesures** : longueur et largeur du **sépale**, longueur et largeur du **pétale** (en cm). C'est le jeu de données le plus célèbre de l'histoire de la classification. Les espèces *versicolor* et *virginica* se ressemblent beaucoup à l'œil nu (photos ci-dessous) : ce sont les **mesures** qui permettent de les distinguer. En voici quelques lignes (pétales seulement) :

| long. pétale | larg. pétale | espèce |
|---|---|---|
| 1.4 | 0.2 | setosa |
| 1.5 | 0.2 | setosa |
| 4.5 | 1.5 | versicolor |
| 4.9 | 1.5 | versicolor |
| 5.5 | 2.1 | virginica |
| 5.8 | 2.2 | virginica |



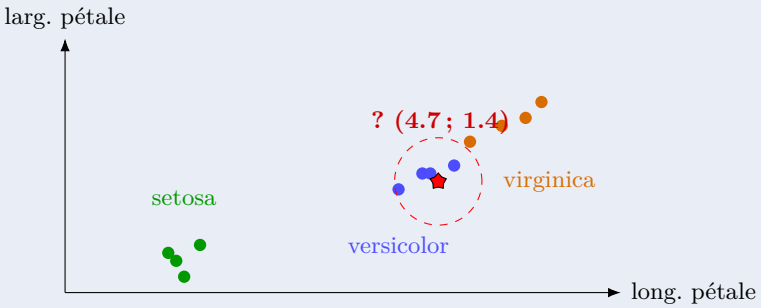
*Iris versicolor*



*Iris virginica*

**Remarque** — *Un individu = un point*

Chaque descripteur est une **coordonnée**. Avec 2 descripteurs, un individu est un **point du plan** ; avec $p$ descripteurs, un point d'un espace à $p$ dimensions. **Classer**, c'est regarder **où** tombe le nouveau point par rapport aux points déjà étiquetés. On se limite souvent à 2 descripteurs pour pouvoir **dessiner** — ici la longueur et la largeur du pétale, qui séparent déjà très bien les trois espèces.



## III Mesurer la ressemblance : la distance

Pour dire qui « ressemble » à qui, il faut mesurer une **distance**. En 2D, c'est la distance à vol d'oiseau, donnée par le théorème de Pythagore.

### Définition 2 — Distance euclidienne

Entre deux individus $A = (a_1, \dots, a_p)$ et $B = (b_1, \dots, b_p)$ décrits par $p$ descripteurs,

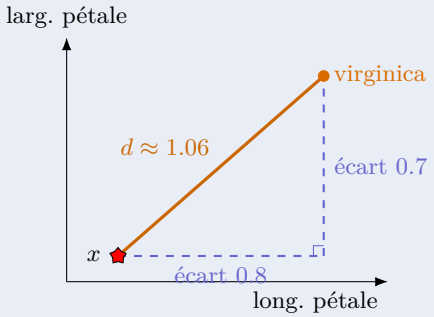
$$d(A,B) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_p - b_p)^2}\,.$$

C'est la généralisation de Pythagore : la racine de la **somme des carrés des écarts**, descripteur par descripteur.

↻ **Retour sur l'activité.** Pour le fruit C de l'activité, la mandarine (5 ; 5,5) et l'orange (8 ; 7,5) donnaient les mêmes écarts, 1,5 cm et 1 cm, donc la même distance $\sqrt{1{,}5^2 + 1^2} \approx 1{,}80$ cm : le plus proche voisin ne pouvait pas trancher.

**Sur l'exemple des iris** (2 descripteurs), entre la fleur inconnue $x = (4.7; 1.4)$ et la *virginica* $(5.5; 2.1)$ : les écarts sont $5.5 - 4.7 = 0.8$ (longueur) et $2.1 - 1.4 = 0.7$ (largeur). Ce sont les deux côtés de l'angle droit, et la distance est l'hypoténuse :

$$d^2 = 0.8^2 + 0.7^2 = 0.64 + 0.49 = 1.13$$
$$\text{donc} \quad d = \sqrt{1.13} \approx 1.06.$$




### Règle 1 — La distance en Python

```
1  def distance(A, B):
2      """distance euclidienne entre deux listes de meme longueur"""
3      somme = 0                                  # INITIALISER
4      for i in range(len(A)):                    # PARCOURIR : descripteur par descripteur
5          somme = somme + (A[i] - B[i]) ** 2     # METTRE A JOUR : carre de l'ecart
6      return somme ** 0.5                        # RENVOYER : la racine
```

▷ **Je prédis.** Que renvoient `distance([1, 2], [4, 6])` et `distance([4, 6], [1, 2])` ?
▷ **Je vérifie.** `5.0` les deux fois : $3^2 + 4^2 = 25$, dont la racine est 5. C'est un flottant (`** 0.5` donne toujours un flottant), et l'ordre des points ne compte pas : les écarts négatifs disparaissent au carré.

**Remarque** — *Comparer sans la racine carrée*
Pour **comparer** des distances, la racine est **inutile** : $\sqrt{u} < \sqrt{v}$ exactement quand $u < v$. On peut donc trier sur la **somme des carrés** (plus rapide) et ne garder `**0.5` que si l'on veut la vraie valeur. *On note $d^2$ cette « distance au carré ».*

**Pour les curieux** — *Il existe d'autres distances* (facultatif)
La distance euclidienne n'est pas la seule. La distance de **Manhattan** (somme des écarts en *valeur absolue*, comme un taxi qui suit les rues) est parfois préférée. Le choix de la distance fait partie de la conception du classifieur.

► **Exercices d'application :** exercices 1, 2 et 16 (vocabulaire, calculer des distances)

# IV L'algorithme des $k$ plus proches voisins

## Règle 2 — La méthode des $k$-NN

Pour classer un nouvel individu $x$ :

1. calculer la **distance** de $x$ à **chaque** exemple connu ;
2. retenir les $k$ **exemples les plus proches** (ses $k$ « voisins ») ;
3. lui attribuer la **classe majoritaire** parmi ces $k$ voisins (le vote).

**Exemple** — *Classer la fleur $x = (4.7; 1.4)$, en détail*
On calcule la distance au carré de $x$ à chacun des 6 exemples :

| exemple | espèce | calcul de $d^2$ | $d^2$ |
|---|---|---|---|
| $(1.4\,;\,0.2)$ | setosa | $3.3^2 + 1.2^2$ | 12.33 |
| $(1.5\,;\,0.2)$ | setosa | $3.2^2 + 1.2^2$ | 11.68 |
| $(4.5\,;\,1.5)$ | versicolor | $0.2^2 + 0.1^2$ | **0.05** |
| $(4.9\,;\,1.5)$ | versicolor | $0.2^2 + 0.1^2$ | **0.05** |
| $(5.5\,;\,2.1)$ | virginica | $0.8^2 + 0.7^2$ | **1.13** |
| $(5.8\,;\,2.2)$ | virginica | $1.1^2 + 0.8^2$ | 1.85 |

**1-NN** ($k = 1$) : le plus proche est une *versicolor* ($d^2 = 0.05$) → **versicolor**. **3-NN** ($k = 3$) : les 3 plus proches sont deux *versicolor* $(0.05\,;\,0.05)$ et une *virginica* (1.13) → vote 2 contre 1 → **versicolor**.

On assemble le tout en Python. On réutilise le **tri par clé** (chapitres *Les algorithmes de tri* et *Les données en tables*) pour classer les distances, et un **dictionnaire d'occurrences** (chapitre *Les types construits*) pour le vote.

## Règle 3 — Les $k$ plus proches, puis le vote

```
1  def k_plus_proches(exemples, x, k):
2      """exemples : liste de (descripteurs, classe). Renvoie les classes des k voisins."""
3      dists = []                                                    # INITIALISER
4      for descripteurs, classe in exemples:                         # PARCOURIR
5          dists.append((distance(x, descripteurs), classe))         # METTRE A JOUR
6      dists.sort(key=lambda couple: couple[0])                      # tri par distance
                                                                     # croissante
7      return [classe for (d, classe) in dists[:k]]                  # RENVOYER : les k
                                                                     # premiers
8
9  def vote(classes):
10     compte = {}                                                   # INITIALISER
11     for c in classes:                                             # PARCOURIR
12         if c in compte:                                           # TESTER
13             compte[c] += 1                                        # METTRE A JOUR :
                                                                     # occurrences
14         else:
15             compte[c] = 1                                         # METTRE A JOUR
16     meilleure = None                                              # INITIALISER : le
                                                                     # champion
17     for c in compte:                                              # PARCOURIR
18         if meilleure is None or compte[c] > compte[meilleure]:    # TESTER
19             meilleure = c                                         # METTRE A JOUR
20     return meilleure                                              # RENVOYER
```

```
21
22 def classer(exemples, x, k):
23     return vote(k_plus_proches(exemples, x, k))
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> classer(iris, [4.7, 1.4], 3)
'versicolor'
```

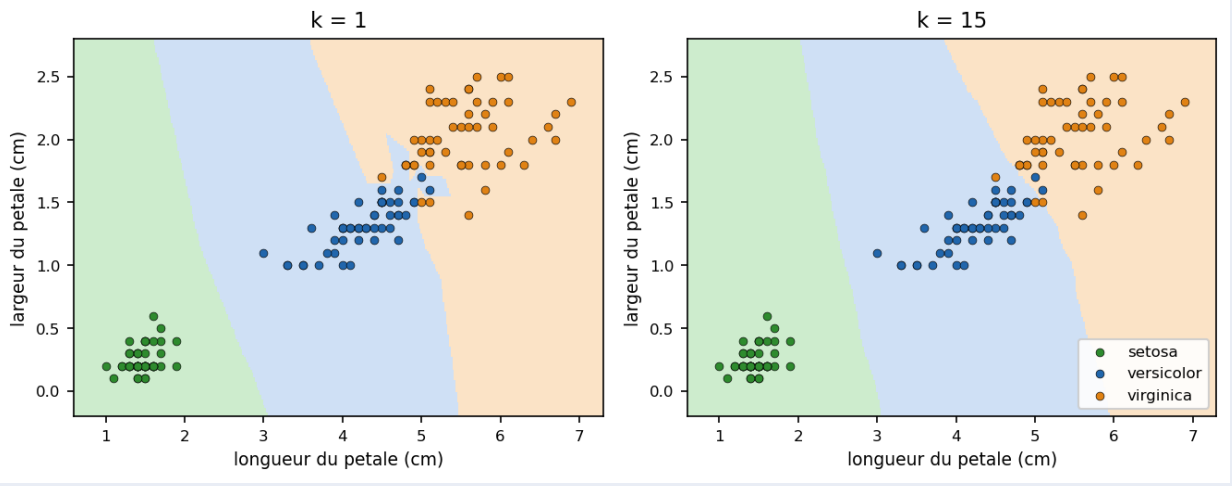
▷ **Je prédis.** Que renvoie `vote(["setosa", "virginica", "virginica", "setosa"])` ?
▷ **Je vérifie.** `'setosa'` : égalité 2 contre 2, et le test `>` est strict, donc le champion reste la première classe rencontrée. Le vote ne départage pas vraiment : d'où l'intérêt d'un $k$ impair.

**Remarque** — *Une méthode « paresseuse »*
Le $k$-NN n'a **rien à apprendre à l'avance** : il ne construit aucun modèle, il se contente de **garder tous les exemples** et de comparer au moment de la question. On dit qu'il est **paresseux** (*lazy*). Simple… mais on verra que cela a un coût.

► **Exercices d'application :** exercices 3 à 7 et 17 (voter, dérouler puis programmer le k-NN)

## V La frontière de décision et le choix de $k$

Imaginons qu'on **colorie** tout le plan selon la classe que le $k$-NN prédirait en chaque point : on obtient des **régions** colorées, séparées par une **frontière de décision**. Cette frontière change avec $k$.




*Jeu Iris (pétales). À gauche $k = 1$ : la frontière est très **découpée** et forme même un petit « îlot » autour d'un point isolé. À droite $k = 15$ : la frontière est **lisse** et ignore ces détails.*

**Règle 4 — L'effet de $k$**

- $k$ **trop petit** ($k = 1$) : on copie le voisin le plus proche. La frontière **colle** à chaque exemple, y compris aux points **aberrants** ou bruités : on *surapprend*, on est très sensible au bruit.
- $k$ **trop grand** : on interroge des voisins de plus en plus *lointains*, la frontière se **lisse** à l'excès ; à l'extrême ($k$ = nombre d'exemples), on répond **toujours** la classe la plus fréquente, quel que soit l'individu.
- $k$ **impair** (pour 2 classes) : évite les **égalités** de vote.

Le bon $k$ est un **compromis** : ni trop petit (frontière qui épouse le bruit), ni trop grand (frontière qui efface l'information locale).

↻ **Retour sur l'activité.** Pour le fruit B de l'activité, un voisin désignait un citron, trois voisins une orange, cinq voisins de nouveau un citron : la réponse dépend bien de $k$.

► **Exercices d'application :** exercices 8 **et** 9 (l'effet de $k$, départager une égalité)

## VI Comment sait-on que ça marche ? Entraînement et test

Un classifieur peut sembler parfait… sur les exemples qu'il connaît déjà. Ce qui compte, c'est sa performance sur des individus **jamais vus**.

### Règle 5 — Séparer entraînement et test

On coupe le jeu de données en deux :

- un **jeu d'entraînement** : les exemples que le $k$-NN utilise pour voter ;
- un **jeu de test** : d'autres exemples, **étiquetés** eux aussi, mais dont on *cache* l'étiquette pour vérifier la prédiction.

Le **taux de réussite** (la proportion de bonnes réponses sur le jeu de test) mesure la vraie qualité.

```
1 def taux_reussite(train, test, k):
2     bons = 0                                          # INITIALISER
3     for descripteurs, vraie in test:                  # PARCOURIR
4         if classer(train, descripteurs, k) == vraie:  # TESTER : bonne prediction ?
5             bons += 1                                 # METTRE A JOUR
6     return bons / len(test)                           # RENVOYER : une proportion
```

▷ **Je prédis.** Que renvoie `taux_reussite(iris, iris, 1)` (test sur l'entraînement lui-même) ?
▷ **Je vérifie.** `1.0` : chaque exemple est à distance 0 de lui-même, donc son propre plus proche voisin.

**Remarque** — *Évaluer sur l'entraînement serait tricher*
Mesurer la qualité sur les exemples d'entraînement est trompeur : avec $k = 1$, chaque exemple est son propre plus proche voisin, donc on aurait **100 %**… sans rien prouver ! Seul le jeu de **test** dit si le modèle **généralise**. Tester plusieurs valeurs de $k$ et garder la meilleure sur le jeu de test est la façon habituelle de **choisir** $k$.

**Pour les curieux** — *La matrice de confusion* (facultatif)
Pour comprendre *où* un classifieur se trompe, on dresse une **matrice de confusion** : un tableau qui croise la *vraie* classe et la classe *prédite*. On y lit d'un coup d'œil les confusions fréquentes (par exemple « des 4 pris pour des 9 »). C'est l'outil de base pour diagnostiquer un modèle.

## VII Deux pièges : l'échelle et la dimension

### Règle 6 — Normaliser des descripteurs d'échelles différentes

Si un descripteur va de 0 à 1 et un autre de 0 à 1000, le second **écrase** le premier dans la distance. Exemple : classer des personnes par leur **âge** (0–100) et leur **revenu** (0–5000). Un écart de 10 ans (énorme) pèse $10^2 = 100$, un écart de 200 € (minime) pèse $200^2 = 40\,000$ : le revenu **domine** tout. **Solution** : *normaliser*, c'est-à-dire ramener chaque descripteur à la même échelle (par exemple entre 0 et 1) pour qu'ils comptent **équitablement**.

**Pour les curieux** — *La malédiction de la dimension* (facultatif)
Quand le nombre de descripteurs devient très grand, un phénomène contre-intuitif apparaît : tous les points finissent par être *à peu près aussi loin* les uns des autres, et la notion de « plus proche voisin » perd de son sens. C'est la **malédiction de la dimension**. Le $k$-NN reste néanmoins très efficace tant que les descripteurs sont peu nombreux et bien choisis.

► **Exercices d'application :** exercices 10 **et** 11 (le piège de l'échelle ; le coût)

## VIII Forces et limites

| Forces | Limites |
|---|---|
| très **simple** à comprendre et à programmer | **lent** à la prédiction : compare à *tous* les exemples |
| **aucun apprentissage** préalable (paresseux) | doit **garder toutes** les données en mémoire |
| s'adapte à **n'importe quel** nombre de classes | sensible à l'**échelle** des descripteurs (normaliser) |
| donne de **bons résultats** sur beaucoup de problèmes | sensible aux descripteurs **inutiles** et au choix de $k$ |

**Remarque** — *Le réflexe coût*
Pour classer *un seul* individu, on calcule sa distance à **tous** les exemples : le coût est **linéaire** en le nombre d'exemples. Avec des milliers d'exemples et des centaines de descripteurs (une image = des centaines de pixels !), cela devient **lent**. C'est le prix de la « paresse » : rien n'est préparé à l'avance, tout est recalculé à chaque question.

## IX Vers la reconnaissance d'images (OCR)

Le plus beau : une **image** n'est qu'un **tableau de pixels**, donc… une **liste de nombres** ! Une petite image de $14 \times 14$ pixels devient une liste de 196 nombres (0 = blanc, 1 = encre). Deux images se **ressemblent** si elles diffèrent sur **peu** de pixels : c'est exactement notre distance (avec des 0/1, la distance au carré *compte les pixels différents* !).

### Règle 7 — Reconnaître un chiffre ou une lettre manuscrite

On dispose d'une **base d'exemples** étiquetés : des milliers d'images de chiffres (**MNIST**) ou de lettres (**EMNIST**). Pour reconnaître une image inconnue, on lui applique le $k$-NN : ses $k$ images les plus proches votent pour un chiffre (ou une lettre). C'est ce que vous ferez dans le **TP** proposé en fin de chapitre — sur de *vraies* écritures manuscrites.

**Pour les curieux** — *C'est ainsi qu'on a longtemps lu les codes postaux* (facultatif)
La lecture automatique des **codes postaux** et des **montants sur les chèques** a d'abord reposé sur des méthodes de ce genre. Le $k$-NN sur MNIST atteint déjà, sans réglage sophistiqué, **plus de 95 %** de bonnes réponses — de quoi trier bien des lettres.

► **Exercices d'application :** exercices 12 **et** 13 (applications : recommander un poste, reconnaître une image)

## X Pour aller plus loin [ au-delà du programme ]

- **$k$-NN pondéré** : au lieu d'un vote « une voix par voisin », on donne *plus de poids* aux voisins les plus proches (par exemple un poids en $1/d$). Un voisin très proche compte alors davantage qu'un voisin lointain.
- **Régression** : au lieu de voter une classe, on peut *moyenner* une valeur numérique des $k$ voisins (prédire un prix, une note…).
- **Accélérations** : pour éviter de comparer à *tous* les exemples, on utilise des structures de données (arbres $k$-d) qui trouvent les voisins plus vite.

## XI Un peu d'histoire

**Pour les curieux** — *Des statisticiens de 1951 à l'intelligence artificielle* (facultatif)
La **règle du plus proche voisin** est formalisée dès **1951** par deux statisticiens américains, **Evelyn Fix** et **Joseph Hodges**, dans un rapport pour l'armée de l'air. En **1967**, **Thomas Cover** et **Peter Hart** en donnent l'analyse théorique et démontrent une propriété étonnante : quand les exemples sont très nombreux, la règle du plus proche voisin ne se trompe **jamais plus de deux fois** plus que la *meilleure* méthode imaginable.

Le jeu de données **Iris**, lui, vient du statisticien **Ronald Fisher** (1936), à partir de mesures du botaniste Edgar Anderson : c'est le jeu de données le plus célèbre de la classification. Les images de chiffres manuscrits **MNIST**, popularisées par le Français **Yann LeCun** (pionnier de l'IA, prix Turing 2018) dans les années 1990, sont devenues le « bonjour le monde » de la reconnaissance d'images. Aujourd'hui, le $k$-NN reste utilisé dans les **systèmes de recommandation** (« les personnes qui vous ressemblent ont aimé… ») et comme **référence** à battre pour tout nouveau modèle. *Une idée de 1951 fait partie des tout premiers pas de l'intelligence artificielle moderne.*

**Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)

- **Des données déjà rencontrées.** Le jeu de données est une table (*Les données en tables*) ; les voix se comptent dans un dictionnaire (*Les types construits*) ; une image n'est qu'un tableau de pixels codés par des nombres (*Le binaire et l'écriture des nombres*).
- **Des algorithmes déjà rencontrés.** La distance est un accumulateur et le vote garde un champion (*Algorithmique : le parcours séquentiel*) ; les voisins s'obtiennent par un tri (*Les algorithmes de tri*) ; séparer entraînement et test rejoint la leçon de *Spécifier et mettre au point ses programmes* : un résultat ne vaut que vérifié sur des cas bien choisis.
- **En Terminale.** Les *arbres* (l'arbre binaire de recherche ; au-delà du programme, les arbres $k$-d) permettront de retrouver des voisins sans tout parcourir.

## Bilan — la carte mémoire

| Notion | À retenir |
|---|---|
| Apprentissage supervisé | apprendre à étiqueter à partir d'exemples *déjà étiquetés* |
| Jeu de données | un tableau : lignes = individus, colonnes = descripteurs + étiquette |
| Individu | un point : ses descripteurs sont ses coordonnées |
| $k$-NN | distance à tous les exemples → $k$ plus proches → vote majoritaire |
| Distance euclidienne | $\sqrt{\sum(a_i - b_i)^2}$ ; pour comparer, la racine est inutile ($d^2$) |
| Frontière de décision | petit $k$ : découpée (surapprentissage) ; grand $k$ : lisse |
| Choix de $k$ | petit mais pas trop ; impair (2 classes) ; le **tester** |
| Entraînement / test | évaluer sur des exemples *jamais vus* ; taux de réussite |
| Normalisation | ramener les descripteurs à la même échelle |
| Coût | linéaire en le nombre d'exemples *à chaque* prédiction (paresseux) |
| Image → vecteur | une image = une liste de pixels → OCR par $k$-NN |

### Retour à la situation de départ

1. On lui montre des milliers d'images de chiffres **déjà étiquetées** (§ I). Une image étant une liste de pixels, une image inconnue reçoit l'**étiquette majoritaire** parmi ses $k$ plus proches voisins (§ IV et IX).
2. Oui : chaque fleur devient un **point** dont les coordonnées sont ses mesures (ses **descripteurs**), et la **distance euclidienne**, généralisation de Pythagore, mesure leur ressemblance (§ II et III).
3. Plusieurs : avec $k = 1$, on copie le plus proche, même s'il est aberrant ; avec $k$ trop grand, on répond toujours la classe la plus fréquente (§ V). Le bon $k$, impair s'il y a deux classes, se choisit en le testant sur un **jeu de test** (§ VI).

## Erreurs fréquentes

- **Oublier le vote** : les $k$-NN renvoient $k$ classes, il faut en tirer la *majoritaire*.
- **Prendre $k$ pair** avec 2 classes → risque d'**égalité**.
- **Confondre distance et écart d'un seul descripteur** : la distance combine *tous* les descripteurs.
- **Ne pas normaliser** des descripteurs d'échelles très différentes.
- **Évaluer sur l'entraînement** : trompeur ($k = 1$ donnerait 100 %). On évalue sur le *test*.
- **Croire que le $k$-NN « apprend »** : il ne fait que *mémoriser* et comparer à la demande.

## Savoir-faire à maîtriser

*À la fin du chapitre, je sais…*

☐ expliquer ce qu'est une classification supervisée et un jeu de données ;

☐ calculer une **distance euclidienne** entre deux individus ;

☐ dérouler le $k$-NN à la main et **programmer** distance / $k$ plus proches / vote ;

☐ lire une **frontière de décision** et discuter le **choix de $k$** ;

☐ **évaluer** un classifieur sur un jeu de test (taux de réussite) ;

☐ citer les **limites** (coût, échelle) et expliquer comment une **image** devient classable par $k$-NN.

# EXERCICES

## Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- ▶ *À programmer et tester* en Python. Rappel : pour *comparer* des distances, on peut garder la **somme des carrés** (sans racine).

## Classification et distance ↑ cours

★ **Exercice 1** *Le vocabulaire* ▶ corrigé p. 703

Répondre par une phrase.

a) Qu'est-ce qu'une classification **supervisée** ?
b) Dans le $k$-NN, que représentent les **descripteurs** d'un individu ? son **étiquette** ?
c) Résumer la méthode des $k$ plus proches voisins en une phrase. ▷ coup de pouce

★ **Exercice 2** *Calculer des distances* ▶ corrigé p. 703 • essentiel

a) Distance euclidienne entre $A = (3; 4)$ et $B = (0; 0)$.
b) Distance entre $A = (1; 2; 2)$ et $B = (4; 2; 6)$ (trois descripteurs).
c) Entre $A = (4.7; 1.4)$ et $B = (4.5; 1.5)$, donner la distance **au carré**, puis la distance.

▷ coup de pouce

★ **Exercice 3** *Qui gagne le vote ?* ▶ corrigé p. 703 • essentiel

On a déjà calculé la distance au carré entre un fruit inconnu $x$ et six fruits étiquetés.

| **fruit** | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $d^2$ à $x$ | 2 | 9 | 1 | 5 | 3 | 12 |
| étiquette | pomme | poire | poire | pomme | pomme | poire |

a) Ranger les six fruits du plus proche au plus lointain de $x$.
b) Quelle étiquette prédit le 1-NN ? le 3-NN ? le 5-NN ? ▷ coup de pouce

★★ **Exercice 4** *Dérouler le k-NN à la main (iris)* ▶ corrigé p. 703 • essentiel

On dispose des exemples suivants (longueur ; largeur de pétale → espèce) :

| **long.** | **larg.** | **espèce** |
|---|---|---|
| 1.4 | 0.2 | setosa |
| 1.5 | 0.2 | setosa |
| 4.5 | 1.5 | versicolor |
| 4.9 | 1.5 | versicolor |
| 5.5 | 2.1 | virginica |
| 5.8 | 2.2 | virginica |

On veut classer la fleur $x = (5.2; 1.9)$.

a) Calculer la distance **au carré** de $x$ à chacun des 6 exemples.
b) Quelle espèce prédit le 1-NN (le plus proche voisin) ?
c) Quelle espèce prédit le 3-NN (vote des 3 plus proches) ? ▷ coup de pouce

## Programmer le $k$-NN ↑cours

★★ **Exercice 5** ▶ *Les briques de base* ▶corrigé p. 704

Écrire (ou compléter) les fonctions Python :

a) `distance2(a, b)` : la somme des carrés des écarts entre deux listes de même longueur, c'est-à-dire la distance **au carré** $d^2$ du cours (sans racine : pour comparer des distances, elle suffit) ;
b) `vote(etiquettes)` : l'étiquette la plus fréquente d'une liste (avec un dictionnaire d'occurrences).

▷coup de pouce

▷Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★ **Exercice 6** ▶ *Le plus proche voisin, à trous* ▶corrigé p. 704

La fonction suivante renvoie l'étiquette de l'exemple le plus proche de `x` (c'est le 1-NN). Elle utilise la fonction `distance2` de l'exercice précédent.

```
1 def plus_proche(exemples, x):
2     dists = []
3     for descripteurs, etiquette in exemples:
4         dists.append((..................., etiquette))
5     dists.sort(key=lambda couple: ..........)
6     return dists[.....][1]
```

a) Compléter les trois trous.
b) Que contient la liste `dists` après le tri ? Que désigne `dists[0][1]` ?
c) Tester avec les six iris de l'exercice 4 et `x = [4.7, 1.4]` : on doit obtenir `'versicolor'`.

▷coup de pouce

★★ **Exercice 7** ▶ *Classer* ▶corrigé p. 704

On représente les exemples par une liste de couples `(descripteurs, etiquette)`. Écrire :

a) `k_plus_proches(exemples, x, k)` : la liste des étiquettes des $k$ exemples les plus proches de `x` (trier selon `distance2`) ;
b) `classer(exemples, x, k)` qui renvoie l'étiquette prédite pour `x`.

Tester sur la table des iris de l'exercice 4 avec `x = [4.7, 1.4]` et $k = 3$. ▷coup de pouce ▷début de solution

▷Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

## Choisir $k$, comprendre les limites ↑cours

★★ **Exercice 8** *L'effet de $k$* ▶corrigé p. 705 • essentiel

a) Pourquoi $k = 1$ est-il très sensible à un exemple **aberrant** (mal étiqueté) ?
b) Que prédit le $k$-NN si l'on prend $k$ égal au **nombre total** d'exemples ? Le résultat dépend-il encore de $x$ ?
c) Pour un problème à **deux** classes, pourquoi préfère-t-on un $k$ **impair** ? ▷coup de pouce

★★★ **Exercice 9** *Départager une égalité* ▶ corrigé p. 705 ≈ 30 min

Le vote peut aboutir à une **égalité**. Une règle courante : en cas d'égalité, on **retire le voisin le plus lointain** et on revote, jusqu'à ce qu'une seule classe arrive en tête.

a) Avec les six iris de l'exercice 4 et $x = (4.7; 1.4)$, pour quelles valeurs de $k$ le vote est-il à égalité ?
b) Écrire `vote_departage(voisins)`, où `voisins` est la liste des étiquettes des voisins, rangées du plus proche au plus lointain, qui applique cette règle.
c) Tester : `vote_departage(["A", "B", "B", "A", "C"])` doit renvoyer `"B"`. Expliquer pourquoi ce n'est pas l'étiquette du voisin le plus proche.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 10** *Le piège de l'échelle* ▶ corrigé p. 706

On classe des personnes par leur **âge** (de 0 à 100) et leur **revenu mensuel** (de 0 à 5000). On utilise la distance euclidienne brute.

a) Entre deux personnes, quel descripteur **domine** la distance ? Pourquoi ?
b) Que faut-il faire avant d'appliquer le $k$-NN pour que les deux descripteurs comptent *équitablement* ?

▷ coup de pouce

★ **Exercice 11** *Le coût* ▶ corrigé p. 706

Une base contient 10 000 exemples, chacun décrit par 196 pixels.

a) Pour classer **une** image, à combien d'exemples doit-on calculer la distance ?
b) Pourquoi dit-on que le $k$-NN est un algorithme « paresseux » ? Quel est le coût par prédiction (linéaire, quadratique…) en fonction du nombre d'exemples ? ▷ coup de pouce

## Applications ↑ cours

★★ **Exercice 12** *Recommander un poste (football)* ▶ corrigé p. 706

On décrit des joueurs par leur **taille** (cm) et leur **poids** (kg), avec leur **poste** :

| taille | poids | poste |
|---|---|---|
| 180 | 75 | milieu |
| 178 | 72 | milieu |
| 192 | 88 | défenseur |
| 195 | 90 | défenseur |
| 170 | 65 | attaquant |
| 173 | 68 | attaquant |

Un nouveau joueur mesure 176 cm pour 70 kg.

a) Calculer la distance au carré aux 6 joueurs.
b) Quel poste recommande le 1-NN ? le 3-NN ?
c) Ici taille et poids ont des ordres de grandeur *proches* : faut-il normaliser ? (Contraster avec l'exercice « Le piège de l'échelle » : âge et revenu.) ▷ coup de pouce

★★ **Exercice 13** *Reconnaître une image $3 \times 3$ (sans machine)* ▶ corrigé p. 706

Une image $3 \times 3$ est une liste de 9 pixels (0 blanc, 1 encre), lus ligne par ligne. On a deux **modèles** étiquetés :

$$T = \begin{smallmatrix} 1&1&1\\0&1&0\\0&1&0 \end{smallmatrix} \quad (\text{« T »}) \qquad\qquad L = \begin{smallmatrix} 1&0&0\\1&0&0\\1&1&1 \end{smallmatrix} \quad (\text{« L »})$$

On reçoit l'image inconnue $N = \begin{smallmatrix} 1&1&1\\0&1&0\\0&1&1 \end{smallmatrix}$.

a) Écrire $T$, $L$ et $N$ comme des listes de 9 nombres.
b) Avec des pixels 0/1, la distance au carré compte le **nombre de pixels différents**. Calculer $d^2(N,T)$ et $d^2(N,L)$.
c) Que prédit le 1-NN ? Cela vous semble-t-il correct visuellement ? ▷ coup de pouce

## Mini-problème de synthèse ↑ cours

★★★ **Exercice 14** *Évaluer la qualité* ▶ corrigé p. 706 ≈ 20 min

On dispose d'exemples **d'entraînement** (étiquetés) et d'exemples **de test** (étiquetés eux aussi, mais on *cache* l'étiquette pour évaluer).

a) ▶ Écrire `taux_reussite(train, test, k)` qui renvoie la proportion d'exemples de `test` correctement classés par le $k$-NN entraîné sur `train`.
b) Pourquoi mesure-t-on la qualité sur des exemples **de test** et non sur ceux d'entraînement ?
c) On teste $k = 1, 3, 5, 7$ et on garde le $k$ qui donne le meilleur taux sur le jeu de test. Pourquoi ne fixe-t-on pas $k$ une fois pour toutes, sans tester ? Que risque-t-on avec un $k$ trop petit ? trop grand ? ▷ coup de pouce ▷ début de solution

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 15** *Un $k$-NN selon un assistant d'IA* ▶ corrigé p. 707

Un élève demande à un assistant d'IA : « Écris en Python une fonction `classer(exemples, x, k)` des $k$ plus proches voisins, où `exemples` est une liste de couples `(descripteurs, classe)`. » Voici la réponse obtenue (`distance` est la fonction du cours).

```
1  def classer(exemples, x, k):
2      dists = []
3      for descripteurs, classe in exemples:
4          dists.append((classe, distance(x, descripteurs)))
5      dists.sort()                  # du plus proche au plus lointain
6      compte = {}
7      for classe, d in dists[:k]:   # les k voisins
8          if classe in compte:
9              compte[classe] += 1
10         else:
11             compte[classe] = 1
12     meilleure = None
13     for c in compte:
14         if meilleure is None or compte[c] > compte[meilleure]:
15             meilleure = c
16     return meilleure
```

« On calcule la distance de `x` à chaque exemple, on trie, on garde les $k$ premiers et on renvoie leur classe majoritaire. »

a) La réponse est-elle correcte ? ▶ Vérifier sur les six iris du cours avec $x = (4.7\,;\,1.4)$ et $k = 3$ (le cours donne *versicolor*).
b) Localiser et corriger l'erreur.
c) Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ? ▷ coup de pouce

## Entraînement ↑cours

*Exercices courts pour s'entraîner aux gestes de base, chacun avec plusieurs données. Ils font tous partie du parcours essentiel.*

★ **Exercice 16** *Distances au carré* ▶ corrigé p. 707 • essentiel

Calculer la distance au carré $d^2$ entre $A$ et $B$, puis la distance $d$ (valeur approchée si besoin) :
a) $A = (2;5)$, $B = (6;2)$ ; b) $A = (0;3)$, $B = (2;0)$ ; c) $A = (1;2;3)$, $B = (3;1;0)$ ;
d) $A = (1.5;0.4)$, $B = (1.2;0.2)$. ▷ coup de pouce

★ **Exercice 17** *Le vote des $k$ plus proches voisins* ▶ corrigé p. 707 • essentiel

On donne la distance au carré entre un individu inconnu $x$ et des exemples étiquetés. Pour chaque série, donner la prédiction du $k$-NN pour $k = 1$, $k = 3$ et $k = 5$.

a) $d^2$ : 4 (rouge), 1 (bleu), 9 (rouge), 2 (rouge), 6 (bleu) ;
b) $d^2$ : 3 (chat), 7 (chien), 1 (chien), 5 (chat), 2 (chat), 8 (chien). Que se passe-t-il pour $k = 6$ ?

▷ coup de pouce

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 q. 3 – piste « Supervisée » : la machine apprend à partir d'exemples dont on connaît déjà la bonne réponse. Pour c), trois verbes suffisent : mesurer, garder, voter. ↑énoncé

Ex. 2 q. 3 – piste Pour chaque descripteur, calculer l'écart, puis son carré ; additionner ces carrés : c'est la distance au carré. La distance est sa racine carrée. ↑énoncé

Ex. 3 q. 2 – piste Il suffit de comparer les $d^2$ : ranger les fruits par $d^2$ croissant, puis, pour chaque valeur de $k$, compter les étiquettes parmi les $k$ premiers seulement. ↑énoncé

Ex. 4 q. 3 – piste Pour chaque exemple : carré de l'écart des longueurs, plus carré de l'écart des largeurs. Ranger ensuite les six résultats dans l'ordre croissant avant de voter. ↑énoncé

Ex. 5 q. 2 – piste `distance2` : un accumulateur et une boucle sur les indices. `vote` : compter d'abord chaque étiquette dans un dictionnaire, puis chercher l'étiquette de compte maximal (invariant du champion). ↑énoncé

Ex. 6 q. 3 – piste Chaque couple rangé dans `dists` est `(distance, etiquette)`. Sur quel élément du couple faut-il trier ? Après le tri, où se trouve le plus proche ? ↑énoncé

Ex. 7 – piste Ranger dans une liste des couples `(distance2(x, descripteurs), etiquette)`, puis trier cette liste en disant à `sort` sur quoi comparer : avec `key=lambda couple: couple[0]`, on trie selon le premier élément de chaque couple, la distance. ↑énoncé

Ex. 7 – début de solution
```
def k_plus_proches(exemples, x, k):
    dists = []
    for descripteurs, etiquette in exemples:
        ...
```
↑énoncé

Ex. 8 q. 3 – piste Pour b), imaginer que tous les exemples votent : la position de `x` change-t-elle encore le résultat ? Pour c), essayer un vote à 4 voisins entre deux classes. ↑énoncé

Ex. 9 – piste Écrire d'abord une fonction qui compte les voix dans un dictionnaire. Ensuite, tant que plusieurs classes ont le nombre maximal de voix, recommencer sur la liste privée de son dernier élément. ↑énoncé

Ex. 9 – début de solution
```
def vote_departage(voisins):
    n = len(voisins)
    while n > 0:
        compte = compter(voisins[:n])
        ...
```
↑énoncé

Ex. 10 q. 2 – piste Comparer un écart typique de 10 ans et un écart typique de 200 € : que valent leurs carrés ? ↑énoncé

Ex. 11 q. 2 – piste Relire l'algorithme : avant de pouvoir voter, de quels exemples faut-il connaître la distance ? Le $k$-NN prépare-t-il quelque chose avant qu'on lui demande de classer ? ↑

Ex. 12 q. 3 – piste Même méthode que pour les iris : un tableau des six distances au carré, rangées dans l'ordre croissant. ↑énoncé

Ex. 13 q. 3 – piste Comparer les deux listes case par case et compter les cases où elles diffèrent. ↑énoncé

Ex. 14 q. 3 – piste Pour a), parcourir les couples de `test`, prédire l'étiquette de chacun avec `classer(train, ..., k)`, compter les bonnes réponses, puis diviser par le nombre d'exemples de test. ↑énoncé

Ex. 14 q. 3 – début de solution
```
def taux_reussite(train, test, k):
    bons = 0
    for descripteurs, etiquette in test:
        ...
```
↑énoncé

Ex. 15 q. 3 – piste Dérouler le code sur les iris en écrivant le contenu de `dists` avant et après le tri. ↑

Ex. 16 – piste Un écart par descripteur, que l'on élève au carré ; on additionne. L'ordre ne compte pas : $(2-6)^2 = (6-2)^2$. ↑énoncé

Ex. 17 – piste Toujours commencer par ranger les exemples du plus proche au plus lointain. Le vote ne porte que sur les $k$ premiers de ce classement. ↑énoncé

# TP ET PROJETS

## TP Reconnaître des chiffres et des lettres manuscrits

▶ corrigé p. 708

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p12-tp-ocr` — **archive .zip**

*Un projet de l'année : une vraie application de ce qu'on apprend en cours.*

**Remarque** — *Où l'on va*
Vous allez faire **reconnaître par la machine** de vrais chiffres et de vraies lettres **écrits à la main**. On avance en **deux temps** :

1. d'abord, on comprend l'algorithme des $k$ plus proches voisins sur un **tout petit exemple** de 4 points (qu'on peut vérifier à la main) ;
2. ensuite, on l'applique à des **images** — car une image n'est qu'une longue liste de nombres.

À la toute fin, vous ferez reconnaître **votre propre** lettre, dessinée à la main. Suivez les **points d'étape** : ils indiquent ce que vous devez obtenir pour être sûr d'être sur la bonne voie.

### Avant de commencer : préparer le terrain

1. Créez un dossier de travail. Placez-y les **quatre fichiers de données** à télécharger (lien ci-dessus) : `chiffres_train.csv`, `chiffres_test.csv`, `lettres_train.csv`, `lettres_test.csv`.
2. Dans le **même dossier**, placez le fichier fourni `ocr.py` : c'est la **trame** du programme (fonctions à trous, fonctions fournies et tests). C'est vous qui allez la compléter, morceau par morceau. En bas du fichier, des tests affichent `[OK]` ou `[A FAIRE]` pour chaque étape (au début, tout est `[A FAIRE]` : c'est normal) ; vos essais s'écrivent tout à la fin, sous les tests.
3. À chaque étape, on ajoute un bout de code, puis on le **teste** tout de suite.
4. Usage de l'IA : aucune dans la partie 1 (les fonctions `distance`, `k_plus_proches`, `vote` et `classer` sont le cœur de l'algorithme : vous les écrivez seul) ; en appui ensuite (comprendre un message d'erreur, analyser les résultats), chaque ligne de `ocr.py` étant écrite et comprise par vous. Voir la charte d'usage de l'IA, au début du livre.

**Remarque** — *Source des données*
Les images de **chiffres** proviennent de la base **MNIST** (Y. LeCun, C. Cortes, C. J. C. Burges), diffusée sous licence **Creative Commons BY-SA 3.0** ; les images de **lettres** proviennent de la base **EMNIST** (G. Cohen, S. Afshar, J. Tapson, A. van Schaik, 2017), issue de la *NIST Special Database 19*. Pour ce TP, elles ont été réduites à $14 \times 14$ pixels en noir et blanc ; les fichiers ainsi obtenus sont diffusés sous la même licence (CC BY-SA 3.0).

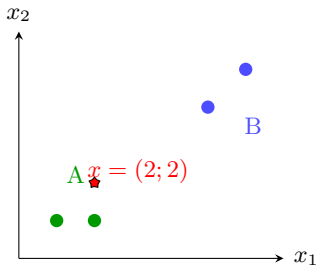
# 1 Comprendre le $k$-NN sur 4 points

*Ici, pas encore d'images : on apprend la mécanique de l'algorithme sur un exemple minuscule. Toute cette partie se fait* ***sans IA****.*

## 1.1 Le problème

On a **4 exemples** déjà étiquetés, décrits par deux nombres $(x_1 ; x_2)$, et rangés en deux classes `A` et `B` :

| $x_1$ | $x_2$ | classe |
|---|---|---|
| 1 | 1 | A |
| 2 | 1 | A |
| 5 | 4 | B |
| 6 | 5 | B |




On veut deviner la classe du nouveau point $x = (2;2)$ (l'étoile rouge). À l'œil, il est clairement du côté des `A`… vérifions-le par le calcul.

## 1.2 À la main d'abord

La **distance au carré** entre deux points $(a_1 ; a_2)$ et $(b_1 ; b_2)$ vaut $(a_1 - b_1)^2 + (a_2 - b_2)^2$.

**1.** Recopier sur le cahier et compléter le tableau des distances au carré de $x = (2;2)$ aux 4 exemples :

| exemple | classe | calcul | $d^2$ |
|---|---|---|---|
| (1 ;1) | A | $1^2 + 1^2$ | … |
| (2 ;1) | A | $0^2 + 1^2$ | … |
| (5 ;4) | B | $3^2 + 2^2$ | … |
| (6 ;5) | B | $4^2 + 3^2$ | … |

**2.** Quel est le **plus proche** voisin ? En déduire la prédiction du 1-NN.

**3.** Quels sont les **3 plus proches** ? Faire voter leur classe : que prédit le 3-NN ?

## 1.3 La distance, en Python

Dans `ocr.py`, on représente un point par une **liste** (ex. `[2, 2]`). Compléter la fonction qui calcule la distance au carré. *Attention :* dans `ocr.py`, cette fonction s'appelle `distance`, mais, contrairement à la fonction `distance` du cours, elle renvoie la distance **au carré** $d^2$, sans racine : pour *comparer* des distances, c'est suffisant (et plus rapide).

```
1  def distance(a, b):
2      """somme des ecarts au carre, coordonnee par coordonnee"""
3      s = 0
4      for i in range(len(a)):
5          s = s + ..............      # (a) l'ecart au carre sur la coordonnee i
6      return s
```

▶ Compléter le trou **(a)**, puis **tester** :

```
>>> distance([1, 1], [2, 2])
2
```

**Point d'étape ✓** `distance([1, 1], [2, 2])` doit renvoyer **2** (car $1^2 + 1^2 = 2$). Si vous obtenez autre chose, revoyez le trou (a).

### 1.4 Trouver les $k$ plus proches voisins

On range les exemples dans une liste de couples `(descripteurs, classe)` :

```
exemples = [([1,1], "A"), ([2,1], "A"), ([5,4], "B"), ([6,5], "B")]
```

La fonction suivante calcule la distance de `x` à chaque exemple, trie du plus proche au plus lointain, et renvoie les classes des `k` premiers :

```
def k_plus_proches(exemples, x, k):
    dists = []
    for descripteurs, classe in exemples:
        dists.append(( ..... , classe))   # (b) distance de x a cet exemple
    dists.sort(key=lambda couple: ........)          # (c) tri par la distance
    return [classe for (d, classe) in dists[:k]]     # les k plus proches
```

▶ Compléter **(b)** et **(c)** (la clé de tri d'un couple est son premier élément, la distance), puis tester :

```
>>> k_plus_proches(exemples, [2, 2], 3)
['A', 'A', 'B']
```

**Point d'étape ✓** Vous devez obtenir `['A', 'A', 'B']` : les 3 plus proches de $(2;2)$ sont les deux `A` puis un `B`. C'est exactement ce que vous aviez trouvé à la main en 1.2 !

### 1.5 Le vote majoritaire

Il reste à choisir la classe **la plus fréquente** parmi ces voisins. On compte les occurrences dans un **dictionnaire** :

```
def vote(classes):
    compte = {}
    for c in classes:
        if c in compte:
            ..............          # (d) ajouter 1 au compteur de c
        else:
            compte[c] = 1           # premiere voix pour c
    meilleure = None
    for c in compte:
        if meilleure is None or compte[c] > compte[meilleure]:
            meilleure = c
    return meilleure
```

▶ Compléter **(d)** (indice : si `c` est déjà une clé de `compte`, `compte[c]` est son compteur actuel). Tester :

```
>>> vote(["A", "B", "A"])
'A'
```

### 1.6 Assembler : `classer`

Dans `ocr.py`, écrire `classer` : elle tient en une ligne, puisqu'elle enchaîne les deux fonctions précédentes. Essayer avant de lire la solution ci-dessous.

```
def classer(exemples, x, k):
    return vote(k_plus_proches(exemples, x, k))
```

▶ Tester :

```
>>> classer(exemples, [2, 2], 3)
'A'
```

**Point d'étape ✓** Vous venez d'écrire un classifieur $k$-NN complet ! Il tient en trois petites fonctions : `distance`, `k_plus_proches`, `vote`. La suite ne change *rien* à ces fonctions : on va juste leur donner des **images** au lieu de points.

## 2 Une image est une liste de nombres

### 2.1 Charger les données

La fonction de chargement est fournie dans `ocr.py` (elle réutilise vos compétences « données en tables ») : chaque ligne d'un CSV donne une **étiquette** puis les **pixels**.

```
import csv
TAILLE = 14

def charger(fichier):
    donnees = []
    with open(fichier, encoding="utf-8", newline="") as f:
        lecteur = csv.reader(f)
        next(lecteur)                          # sauter la ligne d'en-tete
        for ligne in lecteur:
            etiquette = ligne[0]
            pixels = [int(v) for v in ligne[1:]]
            donnees.append((pixels, etiquette))
    return donnees
```

▶ Charger les chiffres et vérifier la taille :

```
>>> train = charger("chiffres_train.csv")
>>> test = charger("chiffres_test.csv")
>>> len(train), len(test)
(3000, 500)
```

**Point d'étape ✓** Vous devez obtenir **3000** exemples d'entraînement et **500** de test. Si Python dit `FileNotFoundError`, c'est que les CSV ne sont pas dans le même dossier que `ocr.py`.

### 2.2 Voir ce que la machine voit

Chaque image est déjà, exactement comme en partie 1, un **couple (pixels, étiquette)** — sauf que `pixels` contient maintenant **196 nombres** (des 0 et des 1). La fonction suivante, fournie dans `ocr.py`, dessine une image en « art ASCII » :

```
def afficher(pixels):
    for l in range(TAILLE):
        ligne = ""
        for c in range(TAILLE):
            if pixels[l * TAILLE + c] == 1:
                ligne = ligne + "#"      # encre
            else:
                ligne = ligne + "."      # blanc
        print(ligne)
```

▶ Afficher la première image d'entraînement et son étiquette :

```
pixels, etiquette = train[0]
afficher(pixels)
print("etiquette :", etiquette)
```

Vous devez voir apparaître un chiffre manuscrit :

```
..............
..............
....######....
...########...
...#.....###..
..........##..
..........##..
..##.....###..
..#####..##...
```

```
..#..######...
..#########...
..######......
..............
..............
etiquette : 2
```

### 2.3 Pourquoi 196 nombres ?

1. L'image fait 14 pixels de large sur 14 de haut. Combien de pixels en tout ? Vérifier avec `len(pixels)`.
2. Que valent les nombres de la liste ? Que représentent un `1` et un `0` ?

### 2.4 La distance mesure la ressemblance

Reprenons *la même* fonction `distance` qu'en partie 1 : elle marche aussi bien sur des listes de 196 nombres ! Deux images se ressemblent si elles diffèrent sur **peu** de pixels.

1. ▶ Parcourir `train` pour ranger dans `zero_a` et `zero_b` les *pixels* des deux premières images d'étiquette `"0"`, et dans `un` ceux de la première image d'étiquette `"1"` (attention : les étiquettes sont des chaînes de caractères, `"0"` et non `0`). Puis comparer :

```
>>> distance(zero_a, zero_b)     # deux "0" entre eux
39
>>> distance(zero_a, un)         # un "0" et un "1"
51
```

2. Laquelle des deux distances est la plus petite ? Expliquer pourquoi c'est **rassurant** pour reconnaître les chiffres.

**Point d'étape ✓** Deux « 0 » sont *plus proches* l'un de l'autre qu'un « 0 » et un « 1 ». C'est toute l'idée : le bon chiffre est celui des voisins qui **ressemblent** le plus.

## 3 Reconnaître les chiffres

### 3.1 Classer une image

▶ Prendre une image de **test** (que le programme n'a jamais vue), l'afficher, et demander sa classe au $k$-NN avec $k = 3$ :

```
1 pixels, vraie = test[0]
2 afficher(pixels)
3 print("vraie :", vraie, "  predite :", classer(train, pixels, 3))
```

**Point d'étape ✓** La classe **prédite** doit coïncider avec la **vraie** étiquette. La machine vient de reconnaître un chiffre qu'elle n'avait jamais vu, uniquement en le comparant à ses 3 voisins les plus proches.

### 3.2 Sur plusieurs images

▶ Classer les **10** premières images de test et compter les bonnes réponses :

```
1 bons = 0
2 for pixels, vraie in test[:10]:
3     if classer(train, pixels, 3) == vraie:
4         bons = bons + 1
5 print(bons, "bonnes reponses sur 10")
```

### 3.3 Le taux de réussite

▶ Écrire une fonction qui calcule la **proportion** de bonnes réponses. *Attention :* classer beaucoup d'images est **lent** (le $k$-NN recompare tout à chaque fois) ; on se limite donc aux `limite` premières images de test.

```
def taux_reussite(exemples, tests, k, limite=100):
    tests = tests[:limite]
    bons = 0
    for pixels, vraie in tests:
        if ..... :       # (e) la prediction est-elle correcte ?
            bons = bons + 1
    return bons / len(tests)
```

L'écriture `limite=100` donne au paramètre `limite` une **valeur par défaut** : si l'on appelle la fonction sans le préciser, il vaut 100. La tranche `tests[:limite]` garde les `limite` premiers tests.

▶ Compléter **(e)** et lancer (cela prend quelques dizaines de secondes) :

```
>>> taux_reussite(train, test, 3, 100)
0.97
```

**Point d'étape ✓** Vous devez obtenir environ **0.92 à 0.97**, c'est-à-dire **plus de 9 chiffres sur 10** correctement reconnus — avec quelques lignes de Python et *sans* aucune bibliothèque d'intelligence artificielle !

### 3.4 L'influence de $k$

▶ Mesurer le taux (100 premiers tests) pour plusieurs valeurs de $k$, puis recopier et compléter le tableau sur le cahier :

| $k$ | 1 | 3 | 5 | 7 |
|---|---|---|---|---|
| taux | | | | |

Le taux change-t-il beaucoup ? Essayer aussi un $k$ **énorme** (`k = 500`) : que se passe-t-il, et pourquoi ?

## 4 Aller plus loin

### 4.1 Regarder les erreurs

▶ Afficher les images **mal classées** parmi les 100 premières :

```
for pixels, vraie in test[:100]:
    predite = classer(train, pixels, 3)
    if predite != vraie:
        afficher(pixels)
        print("vraie :", vraie, "  predite :", predite, "\n")
```

Choisir une erreur et l'expliquer : le chiffre est-il mal écrit ? ressemble-t-il vraiment à celui prédit ? Ces erreurs vous semblent-elles **compréhensibles** ?

### 4.2 Et les lettres ?

▶ Refaire les étapes 3.3 et 3.4 avec les **lettres** (prendre plutôt $k = 5$) :

```
trainL = charger("lettres_train.csv")
testL = charger("lettres_test.csv")
print(taux_reussite(trainL, testL, 5, 100))
```

1. Le taux est-il meilleur ou moins bon que pour les chiffres ? Proposer **deux** raisons (combien de classes pour les chiffres ? pour les lettres ?).
2. Quelles **confusions** de lettres vous semblent inévitables (O/Q, I/L, U/V…) ?

## 5 Défi — *dessine ta propre lettre*

L'outil suivant, fourni dans `ocr.py`, transforme une grille dessinée à la main en image (`'#'` = encre) :

```
def grille_vers_vecteur(grille):
    pixels = []
    for ligne in grille:
        for c in ligne:
            if c in "#1X*":
                pixels.append(1)      # encre
            else:
                pixels.append(0)      # blanc
    return pixels
```

▶ Dessiner votre lettre dans une grille de **14 lignes de 14 caractères**, puis la faire reconnaître :

```
ma_lettre = [
    "..............",
    "..............",
    "..#########...",
    "..#########...",
    "......###.....",
    "......###.....",
    "......###.....",
    "......###.....",
    "......###.....",
    "......###.....",
    "......###.....",
    "..............",
    "..............",
]
x = grille_vers_vecteur(ma_lettre)
afficher(x)
print("reconnue comme :", classer(trainL, x, 5))
```

1. Cette grille dessine un `T`. Est-il reconnu ? Dessinez ensuite **vos initiales** et testez.
2. Les lettres arrondies (`O`, `C`) dessinées de façon anguleuse sont souvent mal reconnues. Et si votre lettre est **mal centrée** ou trop **fine** ? Expliquer, en pensant que la distance compare *pixel par pixel, à la même position.*

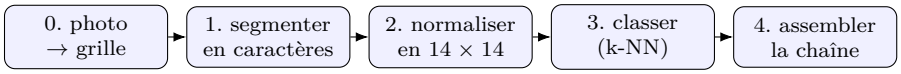
### Bilan

En une poignée de fonctions, sans aucune bibliothèque d'« intelligence artificielle », vous avez construit un **reconnaisseur d'écriture manuscrite**. C'est exactement l'idée qui a longtemps servi à lire les **codes postaux** et les **chèques**. À retenir : les fonctions `distance`, `k_plus_proches` et `vote` sont **les mêmes** pour 4 points ou pour des images — seule change la **longueur** des listes. Et la limite du $k$-NN : il doit **tout garder** en mémoire et **tout recomparer** à chaque image (c'est lent sur de très grandes bases).

# PROJET Lire une plaque d'immatriculation

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p12-projet-plaque` — archive .zip

**Remarque** — *Le but*
Vous savez faire reconnaître *un* caractère par le $k$-NN (TP précédent). Ici, on construit la **chaîne complète** qui lit une **plaque entière**, depuis une **photo** jusqu'au **texte**. On **suppose** qu'on dispose d'une photo ne montrant **que la plaque** (rien autour), bien nette.




On avance étape par étape, en testant à chaque fois. On réutilise votre fichier `ocr.py` du TP OCR (fonctions `charger`, `classer`, `afficher`) ; on complète le fichier fourni `plaque.py`, placé dans le même dossier : c'est la **trame** du projet (fonctions à trous, fonctions fournies et tests qui affichent `[OK]` ou `[A FAIRE]` à chaque lancement). Autres fichiers fournis : `plaque_photo.png` (une photo de plaque), `caracteres_train.csv` (caractères imprimés), `image_vers_grille.py` (outil). Vos essais de `ocr.py` doivent rester sous la ligne `if __name__ == '__main__':` de la fin : sinon, ils seraient relancés à chaque `import`.

**Remarque** — *Usage de l'IA*
**Sans IA** : la binarisation à la main (0.1) et les trous **(a)**, **(b)**, **(c)** à compléter — ce sont les gestes à savoir refaire seul. **En appui** pour le reste (comprendre un message d'erreur, analyser le seuil et les limites) : ce que vous écrivez, vous devez pouvoir l'expliquer. Voir la charte d'usage de l'IA, au début du livre.

## Étape 0 — De la photo à la grille (binarisation)

Une image en **niveaux de gris** est une grille de **nombres** de **0** (noir) à **255** (blanc). **Binariser**, c'est décider pour chaque pixel s'il est de l'**encre** ou du **fond**, avec un **seuil** : *pixel sombre (valeur < seuil)* → *encre (1)*, sinon fond (0).

### 0.1 Binariser (à la main puis en Python)

Voici une petite image 6 × 6 en niveaux de gris (un « T ») :

| | | | | | |
|---|---|---|---|---|---|
| 20 | 20 | 20 | 20 | 20 | 20 |
| 220 | 220 | 30 | 30 | 220 | 220 |
| 220 | 220 | 30 | 30 | 220 | 220 |
| 220 | 220 | 30 | 30 | 220 | 220 |
| 220 | 220 | 30 | 30 | 220 | 220 |
| 220 | 220 | 220 | 220 | 220 | 220 |

1. **À la main**, binariser avec le **seuil 100** (1 si valeur < 100). Quelle lettre apparaît ?
2. ▶ Compléter `binariser` (trou **(a)**) dans `plaque.py` :

```
def binariser(gris, seuil=100):
    """gris : grille de nombres 0-255. Renvoie une grille de 0/1."""
    grille = []
```

```
    for ligne in gris:
        nouvelle = []
        for valeur in ligne:
            if ..............:          # (a) le pixel est-il sombre ?
                nouvelle.append(1)       # encre
            else:
                nouvelle.append(0)       # fond
        grille.append(nouvelle)
    return grille
```

### 0.2 Sur une vraie image (avec Pillow)

Pour ouvrir un fichier `.png`, on utilise la bibliothèque **Pillow**, via l'outil fourni :

```
from image_vers_grille import charger_image_gris, enregistrer_grille
# charger_image_gris(fichier) -> grille de niveaux de gris (0 a 255)
```

▶ Charger la photo, regarder deux pixels, puis binariser et afficher :

```
gris = charger_image_gris("plaque_photo.png")
print("un pixel de FOND :", gris[2][2], "  un pixel de TEXTE :", gris[20][15])
grille = binariser(gris, 100)
afficher_grille(grille)          # afficher_grille : voir l'etape 1 ci-dessous
```

**Point d'étape ✓** Le fond (clair) donne une grande valeur (194), le texte (sombre) une petite (45 pour ce pixel ; selon les pixels, le texte va de 0 à environ 140, car les bords des caractères sont gris). Après `binariser`, on voit apparaître la plaque en `'#'/'.'`.

## Étape 1 — Segmenter par projection

On dispose maintenant d'une grille de 0/1. Pour la découper en caractères : **entre deux caractères, il y a une colonne sans encre**. On compte donc l'encre de chaque colonne (la « **projection** ») ; les caractères sont les groupes de colonnes où ce total est $> 0$.

```
def afficher_grille(grille):
    for ligne in grille:
        texte = ""
        for v in ligne:
            if v == 1:
                texte = texte + "#"      # encre
            else:
                texte = texte + "."      # fond
        print(texte)

def segmenter(grille):
    """Renvoie la liste des (debut, fin) de colonnes de chaque caractere."""
    hauteur = len(grille)
    largeur = len(grille[0])
    encre = []
    for x in range(largeur):
        total = 0
        for y in range(hauteur):
            total = total + ..............     # (b) le pixel (y, x)
        encre.append(total)
    segments = []
    x = 0
    while x < largeur:
        if encre[x] > 0:
            debut = x
            while x < largeur and encre[x] > 0:
                x = x + 1
            if x - debut >= 3:          # ignorer les traits trop fins (bruit)
```

```
                segments.append((debut, x))
            else:
                x = x + 1
    return segments
```

▶ Compléter **(b)**, puis :

```
>>> len(segmenter(binariser(charger_image_gris("plaque_photo.png"), 100)))
7
```

**Point d'étape ✓** On doit trouver **7 segments** pour une plaque de 7 caractères.

## Étape 2 — Normaliser en $14 \times 14$

Chaque caractère découpé a une taille quelconque : il faut le ramener au **format d'entraînement** ($14 \times 14$). On donne les deux fonctions :

```
def decouper(grille, debut, fin):
    """Extrait le caractere entre les colonnes debut et fin, recadre en hauteur."""
    colonnes = []
    for ligne in grille:
        colonnes.append(ligne[debut:fin])
    haut = 0
    while 1 not in colonnes[haut]:       # premiere ligne avec de l'encre
        haut = haut + 1
    bas = len(colonnes) - 1
    while 1 not in colonnes[bas]:        # derniere ligne avec de l'encre
        bas = bas - 1
    return colonnes[haut:bas + 1]

def redimensionner(sous_grille, N=14):
    """Ramene la sous-grille a N x N (moyenne par bloc) -> liste de N*N pixels."""
    h = len(sous_grille)
    w = len(sous_grille[0])
    vecteur = []
    for i in range(N):
        for j in range(N):
            # le bloc de la sous-grille qui devient le pixel (i, j)
            y0 = i * h // N
            y1 = max(y0 + 1, (i + 1) * h // N)
            x0 = j * w // N
            x1 = max(x0 + 1, (j + 1) * w // N)
            total = 0
            for y in range(y0, y1):
                for x in range(x0, x1):
                    total = total + sous_grille[y][x]
            aire = (y1 - y0) * (x1 - x0)
            if 2 * total >= aire:        # majorite d'encre
                vecteur.append(1)
            else:
                vecteur.append(0)
    return vecteur
```

▶ Découper et redimensionner le **premier** caractère, puis l'afficher avec `afficher` (de `ocr.py`). Reconnaissez-vous un « A » de $14 \times 14$ ? Pourquoi doit-on absolument obtenir 196 pixels ? (indice : la fonction `distance` compare des listes…)

## Étapes 3 et 4 — Classer et assembler

Une plaque est écrite en **police imprimée**. On entraîne donc le $k$-NN sur des caractères **imprimés** (chiffres et lettres, plusieurs polices), rassemblés dans le fichier `caracteres_train.csv`.

```
from ocr import charger, classer, afficher

def lire_plaque(fichier_image, exemples, seuil=100, k=3):
    grille = binariser(charger_image_gris(fichier_image), seuil)
    plaque = ""
    for debut, fin in segmenter(grille):
        caractere = redimensionner(decouper(grille, debut, fin))
        plaque = plaque + ..............     # (c) la classe predite pour ce caractere
    return plaque
```

▶ Compléter **(c)** (utiliser `classer(exemples, caractere, k)`), puis :

```
>>> imprime = charger("caracteres_train.csv")
>>> lire_plaque("plaque_photo.png", imprime)
'AB123CD'
```

**Point d'étape ✓** Le programme lit **AB123CD** : parti d'une simple photo, votre k-NN a lu la plaque de bout en bout !

### L'expérience qui fait comprendre : le bon jeu d'entraînement

Les caractères **manuscrits** sont ceux du TP OCR (bases MNIST et EMNIST, licence CC BY-SA 3.0, voir la source citée dans le TP).

▶ Relire la **même** plaque avec un entraînement **manuscrit** :

```
manuscrit = charger("lettres_train.csv") + charger("chiffres_train.csv")
print(lire_plaque("plaque_photo.png", manuscrit))
```

Le résultat est du charabia (p. ex. `DM1DJCM`). **Pourquoi** le même programme échoue-t-il avec un autre entraînement ? Quelle **règle générale** de l'apprentissage cela illustre-t-il ?

## Bien choisir le seuil

Sur `plaque_photo.png`, un pixel de **fond** vaut 194, un pixel de **texte** entre 0 et environ 140 (les bords des caractères sont gris). ▶ Refaire la lecture en changeant le seuil, puis recopier et compléter le tableau sur le cahier :

| seuil | résultat |
|---|---|
| 60 | … |
| 100 | … |
| 250 | … |

1. Avec un seuil **trop bas** (60), une partie du texte (les pixels gris des bords) n'est plus captée : que devient la lecture ?
2. Avec un seuil **trop haut** (250), même le fond (194) devient encre : que se passe-t-il à la segmentation ?
3. En déduire : entre quelles valeurs choisir le seuil ?

## Et sur une vraie photo ? Les limites

Ici tout était **facile** car la photo ne montrait *que* la plaque, nette, sur fond uni. En vrai, un lecteur de plaques doit en plus :

- **trouver** la plaque dans une photo pleine de décor (détection) ;
- gérer l'**angle**, la **lumière**, les ombres, le **flou** ;
- séparer des caractères **collés** ou **penchés** (la projection ne suffit plus) ;
- éviter les **confusions** `O`/`0`, `I`/`1`, `B`/`8` — c'est pourquoi les plaques françaises n'utilisent ni `I`, ni `O`, ni `U`.

Ces étapes relèvent de la **vision par ordinateur** et dépassent le $k$-NN. Mais le cœur — *reconnaître un caractère en le comparant à des exemples* — reste exactement ce que vous avez programmé.

## Bilan

Lire une plaque, c'est enchaîner : **binariser** (photo → grille, avec un bon seuil) → **segmenter** (projection) → **normaliser** ($14 \times 14$) → **classer** (k-NN) → **assembler**. Deux leçons : (1) le **prétraitement** (binarisation, segmentation, normalisation) est souvent plus délicat que la reconnaissance elle-même ; (2) le **jeu d'entraînement doit ressembler** à ce qu'on veut lire — un k-NN manuscrit ne lit pas une plaque imprimée.

# FICHE DE RÉVISION

## L'idée maîtresse — « qui se ressemble s'assemble »

Pour classer un individu inconnu : on calcule sa **distance** à tous les exemples *déjà étiquetés*, on garde les $k$ **plus proches**, et on lui donne leur **classe majoritaire** (vote). C'est un **apprentissage supervisé** : on apprend à partir d'exemples corrigés.

## Jeu de données et distance

Un **jeu de données** : lignes = individus, colonnes = descripteurs + étiquette. Un individu = un **point** (ses descripteurs sont ses coordonnées).

**Distance euclidienne** : $d(A,B) = \sqrt{(a_1-b_1)^2 + \cdots + (a_p-b_p)^2}$ (Pythagore généralisé). Pour *comparer*, la racine est inutile : on peut trier sur la **distance au carré** $d^2$. Avec des pixels 0/1, $d^2$ **compte les pixels différents**.

## L'algorithme

```
def distance(A, B):
    somme = 0
    for i in range(len(A)):
        somme = somme + (A[i] - B[i]) ** 2
    return somme ** 0.5

def k_plus_proches(exemples, x, k):  # (descripteurs, classe)
    dists = []
    for descripteurs, classe in exemples:
        dists.append((distance(x, descripteurs), classe))
    dists.sort(key=lambda couple: couple[0])  # distance croissante
    return [classe for (d, classe) in dists[:k]]

def vote(classes):
    compte = {}
    for c in classes:
        if c in compte:
            compte[c] += 1   # occurrences
        else:
            compte[c] = 1
    meilleure = None
    for c in compte:                        # champion du vote
        if meilleure is None or compte[c] > compte[meilleure]:
            meilleure = c
    return meilleure

def classer(exemples, x, k):
    return vote(k_plus_proches(exemples, x, k))
```

## Choisir $k$ et la frontière de décision

$k$ **trop petit** ($k = 1$) : frontière très *découpée*, colle au bruit (surapprentissage). $k$ **trop grand** : frontière *lisse*, efface l'information locale (à l'extrême, toujours la classe majoritaire). $k$ **impair** pour 2 classes (éviter les égalités). On **teste** plusieurs $k$ et on garde le meilleur.

## Évaluer : entraînement / test

On sépare **entraînement** (les exemples qui votent) et **test** (exemples étiquetés dont on cache l'étiquette). Le **taux de réussite** sur le test mesure la vraie qualité. *Évaluer sur l'entraînement est trompeur* ($k = 1 \Rightarrow 100\,\%$).

### Pièges

- **Échelle** : un descripteur de grande plage écrase les autres ⇒ **normaliser** (même échelle).
- **Coût** : le $k$-NN compare à *tous* les exemples ⇒ **lent** (« paresseux », rien appris à l'avance), coût **linéaire** en le nombre d'exemples.

### Application : la reconnaissance d'images (OCR)

Une image = une **liste de pixels** (0/1). Deux images se ressemblent si elles diffèrent sur peu de pixels. Sur MNIST (chiffres) / EMNIST (lettres), le $k$-NN reconnaît de **vrais manuscrits** (>95 % sur les chiffres).

### Erreurs à éviter

Oublier le **vote** (les $k$ voisins donnent $k$ classes) ; $k$ **pair** avec 2 classes ; ne pas **normaliser** ; évaluer sur l'**entraînement** ; croire que le $k$-NN « apprend » (il mémorise et compare).

### Savoir-faire à maîtriser — je sais…

- ☐ expliquer ce qu'est une classification supervisée et un jeu de données ;
- ☐ calculer une **distance euclidienne** entre deux individus ;
- ☐ dérouler le $k$-NN à la main et **programmer** distance / $k$ plus proches / vote ;
- ☐ lire une **frontière de décision** et discuter le **choix de** $k$ ;
- ☐ **évaluer** un classifieur sur un jeu de test (taux de réussite) ;
- ☐ citer les **limites** (coût, échelle) et expliquer comment une **image** devient classable par $k$-NN.

**Se tester** — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 710

1. Le $k$-NN est-il un apprentissage supervisé ? Pourquoi ?
2. Distance au carré entre $(1;2)$ et $(4;6)$ ?
3. Voisins rangés du plus proche au plus lointain : chat, chien, chien, chat, chat. Prédiction du 1-NN ? du 3-NN ? du 5-NN ?
4. Deux classes, $k = 4$ : quel risque ? Que choisir à la place ?
5. Âge (0–100) et revenu (0–5000) : que faire avant de calculer les distances ?

**Et avant ?** — *questions sur les chapitres précédents.* ▶ réponses p. 710

1. Pièces {1, 6, 10}, rendre 12 : solution gloutonne ? meilleure solution ? (ch. 11)
2. Fichier `-rw-r--r--` : qui peut le modifier ? (ch. 9)
3. Coût du tri par sélection sur `n` éléments ? (ch. 6)

### S'entraîner sur plusieurs chapitres

- Problème de synthèse n° 8 — Les droits d'un fichier, en binaire, p. 514
- Problème de synthèse n° 9 — Un conseil de film, p. 514
- Problème de synthèse n° 10 — La soirée cinéma, p. 515

# DÉFI ADVENT OF CODE

DÉFI

## Rope Bridge

*le pont de corde — simulation et généralisation*

Jour 9

**Fichiers à télécharger :**
`maththibaud.github.io/informatique-en-autonomie/f/p12-defi-aoc-2022-09` — **archive .zip**

### Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2022/day/9`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

### Le défi en bref

**Le problème.** On simule une corde sur une grille : on tire sa tête selon deux mille mouvements, et sa queue suit en restant collée à elle. Il faut compter les cases différentes par lesquelles passe la queue. Comme chaque mouvement se décompose en plusieurs pas et que la queue doit être recalculée après chacun, seul un programme peut suivre la corde jusqu'au bout.
**Ce qu'on réinvestit.** Une **distance entre deux cases**, comme dans l'algorithme des $k$ plus proches voisins de ce chapitre : ici, deux cases « se touchent » quand le plus grand écart entre leurs coordonnées vaut au plus 1, et c'est ce calcul qui décide si la queue doit bouger. On utilise aussi des p-uplets pour les positions, des fonctions, et en partie 2 une liste de positions.
**La démarche de la fiche.**

1. Comprendre la règle avec des questions courtes, puis simuler à la main quelques mouvements dont on connaît la réponse.
2. Lire les mouvements du fichier.

3. Écrire une fonction qui fait suivre la queue, puis simuler tous les pas en notant les cases visitées : réponse de la partie 1.
4. Pour la partie 2, représenter une corde de dix nœuds par une liste et réutiliser la même fonction pour chaque nœud.

## 1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

| | | | |
|---|---|---|---|
| `rope` | corde | `knot` | nœud |
| `head / tail` | tête / queue | `touching` | en contact, qui se touchent |
| `adjacent` | voisin (y compris en diagonale) | `overlapping` | superposés |
| `step` | pas | `visit at least once` | visiter au moins une fois |

### Résumé de l'énoncé (partie 1)

**Contexte.** Vous traversez un vieux pont de corde, et pour ne pas regarder en bas vous imaginez une corde qui se déplace sur une grille. La corde a deux bouts, la tête et la queue, qui restent toujours proches l'un de l'autre : quand on tire la tête, la queue finit par suivre. On s'intéresse au chemin suivi par la queue.
**Les données.** Chaque ligne du fichier est un mouvement de la tête, de la forme `D n` : une lettre qui donne la direction (`U` vers le haut, `D` vers le bas, `L` vers la gauche, `R` vers la droite), un espace, puis le nombre `n` de cases à parcourir. Par exemple, `R 4` fait avancer la tête de quatre cases vers la droite.
**La règle.**

- Tête et queue partent de la même case.
- Chaque mouvement `D n` se fait **une case à la fois**, en `n` pas.
- Après chaque pas de la tête, si la queue la touche encore (même case ou case voisine, diagonales comprises), elle ne bouge pas.
- Sinon, la queue avance d'une case vers la tête : en ligne droite si elles sont sur la même ligne ou la même colonne, en diagonale sinon. Autrement dit, la queue rattrape la tête en restant collée à elle.

**La réponse attendue.** Le nombre de cases différentes visitées par la queue, départ compris. Par exemple, avec le seul mouvement `R 3`, la tête arrive trois cases à droite ; la queue reste sur place au premier pas, puis avance d'une case à chacun des deux pas suivants : elle a visité 3 cases, la réponse serait 3.

### Exercice 1 *Comprendre ce qui est demandé*

**But.** Vérifier sur des cas particuliers qu'on a bien compris quand et comment la queue se déplace.
**À quoi ça sert ?** La première question est le cas le plus délicat (le déplacement en diagonale) ; la deuxième évite l'erreur qui consiste à déplacer la tête de plusieurs cases d'un coup.

Après lecture du résumé (et de l'énoncé), vérifier sa compréhension sur ces cas particuliers :

1. La tête est en $(2, 1)$ et la queue en $(0, 0)$ : où va la queue ?
2. Pourquoi faut-il traiter `R 4` comme quatre déplacements d'une case ?
3. La queue peut-elle se retrouver sur la même case que la tête ?

### Exercice 2 *À la main, sur un petit exemple*

**But.** Simuler pas à pas cinq mouvements sur papier quadrillé et compter les cases visitées par la queue.
**À quoi ça sert ?** On obtient un exemple dont on **connaît la réponse** (7), qui permettra de tester la simulation. Le faire à la main montre aussi l'ordre des opérations : d'abord la tête, ensuite la queue.

Sur une feuille quadrillée, simuler pas à pas les mouvements (inventés) suivants, la tête et la queue partant de la même case :

```
R 3
U 2
L 4
D 1
R 2
```

Marquer chaque case par laquelle passe la queue. Vérification : on doit trouver `7` cases.

## 2 Programmer la partie 1

### Exercice 3 *Lire les mouvements*

**But.** Transformer chaque ligne du fichier en un couple `(direction, nombre_de_pas)`.
**À quoi ça sert ?** La simulation parcourra cette liste de couples ; le nombre de pas doit être un entier pour pouvoir servir dans une boucle.

**Outil Python : `strip` (nettoyer une ligne lue)**

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

**Intérêt.** Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Rappel pour lire un fichier ligne par ligne :

```
fichier = open("input.txt", encoding="utf-8")
for ligne in fichier:
    ligne = ligne.strip()   # enlève le retour à la ligne final
fichier.close()
```

Construire la liste des mouvements sous forme de couples `(direction, nombre_de_pas)`, par exemple `("R", 3)`. ▷ coup de pouce

### Exercice 4 *Faire suivre un nœud*

**But.** Écrire la fonction `suivre`, qui calcule la nouvelle position de la queue à partir de celle de la tête.
**À quoi ça sert ?** C'est le cœur du défi : la règle de la queue est entièrement dans cette fonction. Elle resservira telle quelle dans la partie 2, où chaque nœud suit celui qui le précède.

Écrire une fonction `suivre(xt, yt, xq, yq)` qui reçoit la position de la tête et celle de la queue, et renvoie la nouvelle position de la queue. Tester les trois situations : en contact, à deux cases en ligne droite, en diagonale « lointaine ». ▷ coup de pouce ▷ début de solution

### Rappel : les ensembles (`set`)

Les ensembles sont présentés dans l'encadré « Outil Python : les ensembles » du défi *High-Entropy Passphrases* (chapitre 7). L'essentiel : `set()` crée un ensemble vide, `e.add(x)` ajoute `x` (sans doublon), `len(e)` compte les éléments, et le test `x in e` est quasi instantané, quelle que soit la taille de `e`. Un ensemble peut contenir des p-uplets comme `(0, 0)`.
*À essayer.* Que vaut `len({(0, 0), (0, 0), (2, 1)})` ?

### Exercice 5 *Simuler*

**But.** Enchaîner tous les pas de la tête, faire suivre la queue après chacun, et noter les cases visitées par la queue.
**À quoi ça sert ?** Le nombre de cases notées est la réponse de la partie 1. La boucle écrite ici sera la base de la simulation de la partie 2.

Simuler tous les mouvements **un pas à la fois** : à chaque pas, déplacer la tête d'une case, faire suivre la queue, et noter la case de la queue dans une liste sans doublon (ou un ensemble, voir l'encadré). Vérifier `7` sur l'exemple. ▷ coup de pouce

## 3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

### Résumé de la partie 2 (à lire après avoir validé la partie 1)

**Ce qui change.** Le pont tremble et la corde imaginaire s'allonge : elle n'a plus deux bouts seulement, mais **10 nœuds** alignés les uns derrière les autres.
**La nouvelle règle.**

- Le premier nœud est la tête ; il se déplace comme dans la partie 1, selon les mouvements du fichier, une case à la fois.
- Chacun des 9 autres nœuds suit le nœud qui le précède, avec exactement la même règle que la queue de la partie 1.
- Tous les nœuds partent de la même case.

**La réponse attendue.** Le nombre de cases différentes visitées par le **dernier** nœud, départ compris. Par exemple, avec le seul mouvement `R 3`, le mouvement s'amortit le long de la corde avant d'arriver au bout : le dernier nœud ne bouge pas, et la réponse serait 1.

### Exercice 6 *Une corde plus longue*

**But.** Représenter la corde par une liste de positions et faire suivre chaque nœud par le suivant.
**À quoi ça sert ?** On ne réécrit pas la règle : la fonction `suivre` de la partie 1 s'applique à chaque paire de nœuds voisins. Avec une liste de 2 nœuds, on doit d'ailleurs retrouver la réponse de la partie 1, ce qui fournit un test gratuit.

Généraliser : la corde est maintenant représentée par une **liste de positions** `noeuds`, `noeuds[0]` étant la tête. Répondre à la nouvelle question. Sur l'exemple inventé de la fiche, la réponse de la partie 2 est `1` ; sur les mouvements `U 6`, `R 12`, `D 9`, `L 15`, on doit trouver `39` pour la partie 1 et `11` pour la partie 2. ▷ coup de pouce ▷ début de solution

## 4 Approfondissement

★★ **Exercice 7** *Approfondissement 1 : un ensemble de cases visitées*

**But.** Ranger les cases visitées dans un ensemble plutôt que dans une liste, et estimer le gain de temps.
**À quoi ça sert ?** Avec des milliers de cases visitées, le test d'appartenance sur une liste devient coûteux : on mesure concrètement l'intérêt de choisir la bonne structure de données.

1. Réécrire la simulation de la partie 2 en rangeant les cases visitées dans un ensemble plutôt que dans une liste. Quel test devient inutile ? ▷ coup de pouce
2. Si la queue visite environ 6 000 cases, combien de comparaisons le test `in` sur une liste coûte-t-il au total, à peu près ? Et avec un ensemble ?

★★★ **Exercice 8** *Approfondissement 2 : voir la corde*

**But.** Afficher la corde sur une petite grille après chaque mouvement, puis expliquer un comportement propre à la partie 2.
**À quoi ça sert ?** Visualiser une simulation est un excellent moyen de vérifier qu'elle se comporte comme prévu ; on comprend aussi pourquoi une corde longue présente des situations qui n'existent pas avec deux nœuds.

1. Écrire une fonction qui affiche la corde sur une petite grille (le numéro de chaque nœud, des points ailleurs) et l'utiliser après chaque ligne de mouvements de l'exemple. ▷ coup de pouce
2. Pourquoi, dans la partie 2, un nœud peut-il se retrouver à deux cases en diagonale de son prédécesseur, ce qui n'arrive jamais avec deux nœuds ? ▷ coup de pouce

### Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste `d, n = ligne.split()` puis `int(n)`. ↑énoncé

Ex. 4 – piste La queue doit bouger seulement si `abs(xt - xq) > 1 or abs(yt - yq) > 1`. Dans ce cas, elle se rapproche de la tête d'**au plus une case** dans chaque direction. ↑énoncé

Ex. 4 – début de solution Écrire une petite fonction `signe(n)` qui renvoie `1`, `-1` ou `0`. Si la queue doit bouger : `xq += signe(xt - xq)` et `yq += signe(yt - yq)`. Ce seul calcul couvre la ligne droite **et** la diagonale. ↑énoncé

Ex. 5 – piste Pour `("U", 2)`, on fait deux tours de boucle, chacun déplaçant la tête d'une seule case. Décider du sens du repère (par exemple `y` augmente vers le haut) et s'y tenir. ↑énoncé

Ex. 6 – piste Chaque nœud suit celui qui le précède, exactement avec la fonction `suivre` de la partie 1 : le nœud d'indice `i` joue le rôle de la queue, celui d'indice `i - 1` le rôle de la tête. ↑

Ex. 6 – début de solution À chaque pas : déplacer `noeuds[0]`, puis `for i in range(1, len(noeuds)): noeuds[i] = suivre(...)` en utilisant `noeuds[i - 1]`. La case à noter est celle du **dernier** nœud, `noeuds[-1]`. Avec une liste de 2 nœuds, on doit retrouver la partie 1. ↑énoncé

Ex. 7 q. 1 – piste `visitees = {(0, 0)}` puis `visitees.add(noeuds[-1])` à chaque pas. ↑énoncé

Ex. 8 q. 1 – piste Parcourir les lignes de haut en bas (`y` décroissant) ; pour chaque case, chercher si un nœud s'y trouve. ↑énoncé

Ex. 8 q. 2 – piste Dans la partie 1, la tête ne bouge jamais en diagonale. Dans la partie 2, le nœud 1 peut bouger en diagonale et entraîner le nœud 2 de façon nouvelle. ↑énoncé

# Problèmes de synthèse

**Mode d'emploi**

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 39.
- Chaque problème **croise** des notions de plusieurs chapitres : en programmation, on ne se demande jamais « est-ce un exercice sur les boucles ou sur le binaire ? ».
- Sous chaque titre, la ligne Chapitres croisés indique ce qu'il faut avoir vu. Un problème peut être repris plus tard dans l'année, pour réviser.

## Après les chapitres 1 et 2 (Les bases de la programmation Python ; Le binaire et l'écriture des nombres)

**Remarque**

Dans cette section, on travaille uniquement avec des **entiers** : pas de listes ni de chaînes de caractères. Les deux outils clés sont `n % 2` (le dernier bit de `n`) et `n // 2` (`n` privé de son dernier bit), comme dans la méthode des divisions successives.

★★ **Exercice 1** IA IA IA *Le binaire avec des entiers* ▶ corrigé p. 711 ≈ 45 min Chapitres croisés : *Les bases de la programmation Python (boucle* `while`*, fonctions) ; Le binaire et l'écriture des nombres*

On veut faire écrire par Python l'écriture binaire d'un entier, **sans chaîne de caractères** : on la représente par un entier « qui ne contient que des 0 et des 1 ». Par exemple, l'écriture binaire de 13 est représentée par l'entier `1101` (mille cent un).

1. Écrire à la main l'écriture binaire de 13 et de 185 par la méthode des divisions successives. Dans quel ordre obtient-on les bits ?
2. ▶ Écrire une fonction `en_binaire(n)` qui renvoie cet entier pour un entier `n` positif ou nul (`en_binaire(13)` renvoie `1101` et `en_binaire(0)` renvoie `0`). On pourra utiliser une variable `puissance` qui vaut successivement 1, 10, 100… ▷ coup de pouce
3. ▶ Écrire la fonction réciproque `en_decimal(b)` : `en_decimal(1101)` renvoie `13`. À l'aide d'une boucle, vérifier que `en_decimal(en_binaire(n))` redonne bien `n` pour tous les entiers `n` de 0 à 300.
4. Justifier que la boucle de `en_binaire` se termine : quel **variant** peut-on proposer ?
5. ▶ Écrire une fonction `nb_bits(n)` qui renvoie le nombre de bits nécessaires pour écrire l'entier $n \geqslant 1$. Combien de bits faut-il pour 255 ? pour 256 ? Faire le lien avec le cours.

★★ **Exercice 2** *Le bit de parité* ▶ corrigé p. 712 ≈ 30 min Chapitres croisés : *Les bases de la programmation Python (accumulateur, booléens) ; Le binaire et l'écriture des nombres*

Lors d'une transmission, un bit peut être modifié par une perturbation. Pour détecter l'erreur, l'émetteur ajoute à droite du message un **bit de parité**, choisi pour que le nombre total de bits à 1 soit **pair**.

1. ▶ Écrire une fonction `nb_uns(n)` qui renvoie le nombre de bits égaux à 1 dans l'écriture binaire de `n` (`nb_uns(13)` renvoie 3).
2. ▶ En déduire une fonction `bit_parite(n)` qui renvoie le bit de parité à ajouter à `n` : `0` si `n` a un nombre pair de bits à 1, `1` sinon.
3. Ajouter un bit $b$ à droite de l'écriture binaire de $n$ revient à calculer $2n+b$. Justifier, puis écrire une fonction `emettre(n)` qui renvoie le nombre réellement envoyé. Que vaut `emettre(13)`, en décimal et en binaire ?
4. ▶ Écrire une fonction `verifier(m)` qui renvoie `True` si le nombre reçu `m` a un nombre pair de bits à 1, et `False` sinon.
5. Le nombre émis (bit de parité compris) est `11011`. On reçoit `11111`, puis, lors d'un autre envoi, `11101`. Que répond `verifier` dans chaque cas ? Quelle est la limite de ce procédé ?

▷ coup de pouce

★★★ **Exercice 3** *Des entiers relatifs sur 8 bits* ▶ corrigé p. 712 ≈ 45 min Chapitres croisés : *Le binaire et l'écriture des nombres (complément à deux) ; Les bases de la programmation Python*

Sur 8 bits, on code les entiers de $-128$ à 127 en complément à deux. Le code d'un entier est un nombre de 0 à 255 (l'octet qu'on range en mémoire).

1. Coder $-12$ sur 8 bits avec la méthode du cours (inverser les bits, ajouter 1), puis donner la valeur décimale de l'octet obtenu.
2. Pour $a$ compris entre 0 et 255, inverser les 8 bits de $a$ donne $255 - a$. Justifier. En déduire que le code de $-x$ (avec $0 < x \leqslant 128$) est $256 - x$. Vérifier sur $-12$.
3. ▶ Écrire une fonction `code_8bits(x)` qui renvoie le code de l'entier `x` (avec $-128 \leqslant x \leqslant 127$), et la fonction réciproque `valeur(v)` qui renvoie l'entier représenté par un octet `v`. Tester avec une boucle que `valeur(code_8bits(x))` redonne `x` pour tous les entiers de $-128$ à 127.
4. La machine additionne deux octets puis ne garde que 8 bits, c'est-à-dire le reste de la division par 256. ▶ Écrire une fonction `addition_8bits(a, b)` qui simule ce calcul pour deux entiers relatifs. Que renvoie-t-elle pour $12 + (-12)$ ? pour $100 + 50$ ? pour $127 + 1$ ? Expliquer ces deux derniers résultats.

▷ coup de pouce

## Après les chapitres 3 et 4 (Les types construits ; Algorithmique : le parcours séquentiel)

★★ **Exercice 4** *Le bulletin de notes* ▶ corrigé p. 713 ≈ 30 min Chapitres croisés : *Les types construits (dictionnaires, tableaux, p-uplets) ; Algorithmique : le parcours séquentiel*

Les notes d'une petite classe sont rangées dans un dictionnaire qui associe à chaque prénom le tableau de ses notes.

```
1 notes = {"Léa": [12, 15, 9], "Noé": [8, 11, 14, 10], "Zoé": [17, 13]}
```

1. Que valent `notes["Noé"]`, `len(notes)` et `notes["Léa"][2]` ? Écrire l'instruction qui ajoute la note 16 à Zoé.

2. ▶ Écrire une fonction `moyenne(t)` qui renvoie la moyenne d'un tableau de nombres non vide, puis une fonction `moyennes(notes)` qui renvoie un **nouveau** dictionnaire associant à chaque prénom sa moyenne.

3. ▶ Écrire une fonction `meilleur(notes)` qui renvoie le p-uplet `(prénom, moyenne)` de l'élève qui a la meilleure moyenne. ▷ coup de pouce

4. ▶ Écrire une fonction `nb_notes_sous(notes, seuil)` qui compte, tous élèves confondus, les notes strictement inférieures à `seuil`. Combien de fois la comparaison `note < seuil` est-elle exécutée pour une classe de 30 élèves ayant chacun 8 notes ?

★★★ **Exercice 5** *Une image en noir et blanc* ▶ corrigé p. 713 ≈ 45 min Chapitres croisés : *Les types construits (tableaux de tableaux) ; Algorithmique : le parcours séquentiel ; Le binaire et l'écriture des nombres*

Une image de 8 × 8 pixels en noir et blanc est représentée par un tableau de 8 lignes, chaque ligne étant un tableau de 8 entiers : `1` pour un pixel noir, `0` pour un pixel blanc.

```
image = [[0, 0, 1, 1, 1, 1, 0, 0],
         [0, 1, 0, 0, 0, 0, 1, 0],
         [1, 0, 1, 0, 0, 1, 0, 1],
         [1, 0, 0, 0, 0, 0, 0, 1],
         [1, 0, 1, 0, 0, 1, 0, 1],
         [1, 0, 0, 1, 1, 0, 0, 1],
         [0, 1, 0, 0, 0, 0, 1, 0],
         [0, 0, 1, 1, 1, 1, 0, 0]]
```

1. Dessiner l'image sur un quadrillage de 8 × 8 cases. Que vaut `image[5][3]` ?

2. ▶ Écrire une fonction `nb_noirs(image)` qui compte les pixels noirs, à l'aide de deux boucles imbriquées.

3. ▶ Écrire, **par compréhension**, une fonction `negatif(image)` qui renvoie une **nouvelle** image où le noir et le blanc sont échangés, sans modifier `image`.

4. Chaque ligne, lue de gauche à droite, est l'écriture binaire d'un entier de 0 à 255 : la ligne entière tient donc dans un seul **octet**. Quel entier représente la première ligne ? ▶ Écrire une fonction `ligne_en_octet(ligne)`, puis une fonction `compresser(image)` qui renvoie le tableau des 8 octets de l'image. ▷ coup de pouce

5. ▶ Écrire la fonction réciproque `decompresser(octets)`, qui reconstruit l'image à partir des 8 octets, et vérifier que `decompresser(compresser(image)) == image`. Combien d'entiers faut-il stocker avant et après compression ?

## Après les chapitres 5 à 8 (Spécifier et mettre au point ses programmes ; Les algorithmes de tri ; La recherche dichotomique ; Les données en tables)

★★ **Exercice 6** *Le classement d'un cross* ▶ corrigé p. 714 ≈ 60 min Chapitres croisés : *Spécifier et mettre au point ses programmes ; Les algorithmes de tri ; La recherche dichotomique ; Les données en tables ; Algorithmique : le parcours séquentiel*

Les résultats d'un cross du lycée sont rangés dans une table : une liste de dictionnaires, un par coureur, avec son temps en secondes.

```
coureurs = [{"nom": "Inès", "classe": "1G1", "temps": 754},
            {"nom": "Hugo", "classe": "1G2", "temps": 712},
            {"nom": "Léna", "classe": "1G1", "temps": 803},
            {"nom": "Tom", "classe": "1G3", "temps": 698},
            {"nom": "Maël", "classe": "1G2", "temps": 745},
            {"nom": "Sara", "classe": "1G3", "temps": 731}]
```

1. On veut une fonction `plus_rapide(table)` qui renvoie le nom du coureur le plus rapide. Écrire sa **spécification** : prototype, docstring, précondition sur la table, postcondition sur le résultat.

2. ▶ Écrire cette fonction, en vérifiant la précondition par un `assert`. Écrire ensuite un **jeu de tests** d'au moins trois `assert`, dont une table d'un seul coureur et une table où le plus rapide est le dernier. Si tous les tests passent, la fonction est-elle forcément juste ?

3. ▶ Adapter le **tri par insertion** du cours pour écrire `trier_par_temps(table)`, qui trie la table **en place** par temps croissant. Donner l'ordre d'arrivée. ▷ coup de pouce

4. Au début du tour de boucle d'indice `i`, que peut-on affirmer des fiches d'indices 0 à `i - 1` ? Comment s'appelle une telle propriété ?

5. Après le tri, on construit **par compréhension** le tableau `temps` des temps, dans l'ordre. ▶ Adapter la recherche dichotomique du cours en une fonction `position(t, x)` qui renvoie un indice de `x` dans le tableau trié `t`, ou `-1` si `x` est absent. Quelle est la place ($1^{er}$, $2^{e}$…) du coureur qui a mis 745 secondes ? Combien de tours de boucle a-t-il fallu ? Même question pour 760 secondes.

★★★ **Exercice 7** *Les doublons d'une liste d'inscriptions* ▶ corrigé p. 715 ≈ 45 min Chapitres croisés : *Spécifier et mettre au point ses programmes (tests, débogage) ; Les algorithmes de tri (coût) ; Algorithmique : le parcours séquentiel ; Les types construits*

Les élèves s'inscrivent à un atelier avec leur identifiant. Certains se sont inscrits plusieurs fois.

```
inscrits = ["lea.m", "tom.b", "ines.k", "hugo.p",
            "tom.b", "sara.d", "tom.b", "ines.k"]
```

1. ▶ Écrire une fonction `a_un_doublon(t)` qui renvoie `True` si deux cases du tableau `t` contiennent la même valeur, en comparant chaque case à toutes les suivantes. Combien de comparaisons fait-elle, au pire, pour un tableau de 1000 identifiants ?

2. Dans un tableau **trié**, deux valeurs égales sont forcément voisines. Un élève propose :

```
def a_un_doublon_trie(t):
    """t est trie ; renvoie True si deux cases sont egales."""
    for i in range(len(t)):
        if t[i] == t[i + 1]:
            return True
    return False
```

   Que se passe-t-il pour `a_un_doublon_trie(["a", "b", "c"])` ? Et pour `["a", "a", "c"]` ? Pourquoi ce second test ne suffisait-il pas à repérer l'erreur ? Corriger la fonction. ▷ coup de pouce

3. ▶ Écrire un jeu de tests de la fonction corrigée qui couvre les cas limites : tableau vide, un seul élément, doublon au début, doublon à la fin, aucun doublon.

4. On trie d'abord une copie avec `sorted`, puis on appelle `a_un_doublon_trie`. Sachant que `sorted` fait de l'ordre de $n \log_2 n$ comparaisons, comparer le coût des deux méthodes pour $n = 1000$ (on prendra $\log_2 1000 \approx 10$).

5. ▶ Écrire une fonction `doublons(t)` qui, pour un tableau `t` trié, renvoie la liste des valeurs présentes au moins deux fois, chacune **une seule fois**. Que renvoie `doublons(sorted(inscrits))` ? ▷ coup de pouce

## Après les chapitres 9 à 12 (Architecture des ordinateurs et systèmes d'exploitation ; Le Web : réseaux et interactions homme-machine ; Les algorithmes gloutons ; Les $k$ plus proches voisins)

★★ **Exercice 8** *Les droits d'un fichier, en binaire* ▶ corrigé p. 716 ≈ 45 min Chapitres croisés : *Architecture des ordinateurs et systèmes d'exploitation (droits) ; Le binaire et l'écriture des nombres ; Les types construits (chaînes)*

La commande `ls -l` affiche les droits d'un fichier sur 10 caractères, par exemple `-rwxr-x---` : le type, puis trois groupes de trois droits. On utilise l'écriture des droits en chiffres vue dans le cours *Architecture des ordinateurs et systèmes d'exploitation* (règle « Les droits en chiffres » : `r` = 4, `w` = 2, `x` = 1, un chiffre par catégorie) : `-rwxr-x---` s'écrit `750`.

1. Pour un fichier affiché `-rw-r-----` : qui peut le lire ? le modifier ? Écrire ses droits en trois chiffres.

2. ▶ Écrire une fonction `triplet_en_chiffre(triplet)` qui reçoit une chaîne de trois caractères comme `"r-x"` et renvoie le chiffre correspondant.

▷ coup de pouce

3. ▶ En déduire une fonction `code_droits(ligne)` qui reçoit les 10 caractères affichés par `ls -l` et renvoie la chaîne des trois chiffres (`code_droits("-rwxr-x---")` renvoie `"750"`). Que renvoie-t-elle pour `drwxr-xr-x` ?

4. ▶ Écrire la fonction réciproque `droits(code)` : `droits("754")` renvoie `"rwxr-xr--"`. Vérifier, avec trois boucles imbriquées, que `code_droits("-" + droits(code))` redonne `code` pour les 512 codes possibles.

▷ coup de pouce

5. Pourquoi y a-t-il exactement 512 réglages différents des droits d'un fichier ? Pourquoi chaque chiffre ne dépasse-t-il jamais 7 ?

★★ **Exercice 9** *Un conseil de film* ▶ corrigé p. 717 ≈ 45 min Chapitres croisés : *Le Web : réseaux et interactions homme-machine (formulaire, URL, cookies) ; Les données en tables ; Les k plus proches voisins*

Le site `https://www.cine.mc` demande au visiteur de noter de 0 à 10 son goût pour l'action, l'humour et l'émotion, puis lui conseille des films.

```
<form action="/conseil" method="get">
    <input type="number" name="action">
    <input type="number" name="humour">
    <input type="number" name="emotion">
    <button type="submit">Me conseiller</button>
</form>
```

1. Un visiteur saisit 8, 2 et 6. Écrire l'URL demandée au serveur.

2. ▶ Le serveur doit retrouver les réponses dans l'URL. La méthode `split` découpe une chaîne selon un séparateur : `"a=1&b=2".split("&")` renvoie `["a=1", "b=2"]`. Écrire une fonction `lire_reponses(url)` qui renvoie le dictionnaire `{"action": 8, "humour": 2, "emotion": 6}` (avec des entiers).

▷ coup de pouce

3. Le site connaît ces films, notés selon les mêmes critères :

```
films = [
    {"titre": "Les Pirates du lagon", "genre": "aventure",
     "action": 9, "humour": 4, "emotion": 3},
    {"titre": "Mission Mistral", "genre": "aventure",
```

```
     "action": 8, "humour": 1, "emotion": 5},
    {"titre": "Le Dîner raté", "genre": "comédie",
     "action": 1, "humour": 9, "emotion": 3},
    {"titre": "Coloc en folie", "genre": "comédie",
     "action": 3, "humour": 8, "emotion": 2},
    {"titre": "La Lettre", "genre": "drame",
     "action": 1, "humour": 2, "emotion": 9},
    {"titre": "Après l'orage", "genre": "drame",
     "action": 4, "humour": 1, "emotion": 8},
    {"titre": "Course contre la nuit", "genre": "aventure",
     "action": 9, "humour": 2, "emotion": 7}]
```

Calculer à la main la distance au carré entre le profil $(8; 2; 6)$ et les films *Mission Mistral* et *Après l'orage*.

4. ▶ En reprenant les fonctions du cours, écrire `conseils(films, profil, k)`, qui renvoie les titres des `k` films les plus proches du profil, et `genre_conseille(films, profil, k)`, qui renvoie le genre majoritaire parmi eux. Que conseille-t-on au visiteur pour $k = 3$ ?

5. Un second visiteur a le profil $(3; 4; 7)$. Quel genre lui conseille-t-on pour $k = 3$ ? Avec $k = 4$, que se passe-t-il lors du vote, et que renvoie la fonction `vote` du cours ?

6. Pour ne pas redemander les notes, le serveur répond avec l'en-tête `Set-Cookie: profil=8-2-6; Max-Age=2592000`. Où ce profil est-il mémorisé ? Que contient la requête quand le visiteur revient sur le site, deux semaines plus tard ? Est-ce grave si le visiteur modifie ce cookie ?

★★★ **Exercice 10** *La soirée cinéma* ▶ corrigé p. 718 ≈ 45 min Chapitres croisés : *Les algorithmes gloutons ; Les algorithmes de tri ; Le binaire et l'écriture des nombres ; Les types construits*

Une soirée dure au plus 240 minutes. Chaque film est un p-uplet `(titre, durée en minutes, points d'envie)`, et l'on veut le plus de points possible sans dépasser 240 minutes.

```
soiree = [("Les Pirates du lagon", 60, 6), ("Le Dîner raté", 120, 9),
          ("La Lettre", 70, 5), ("Coloc en folie", 85, 6),
          ("Mission Mistral", 90, 8)]
```

1. **Glouton n° 1** : on range les films par points décroissants, puis on prend chaque film qui tient encore dans la soirée. Dérouler à la main : quels films, combien de points ?

2. **Glouton n° 2** : même méthode, mais en rangeant les films par **points par minute** décroissants. Dérouler à la main.

3. ▶ Écrire `soiree_gloutonne(duree_max, ordre)`, qui reçoit les films déjà rangés dans l'ordre du critère et renvoie la liste des titres choisis. Vérifier les deux réponses précédentes.

▷ coup de pouce

4. ▶ Une soirée est un choix de films : chaque film est pris ou non, c'est un bit. Les $2^5 = 32$ soirées possibles correspondent donc aux entiers de 0 à 31 écrits en binaire. Écrire `meilleure_soiree(duree_max, films)` qui les essaie toutes et renvoie le couple (liste des titres, total de points) de la meilleure soirée. Les deux gloutons étaient-ils optimaux ?

▷ coup de pouce

5. Avec 40 films, combien de soirées la force brute devrait-elle essayer ? Pourquoi utilise-t-on quand même des algorithmes gloutons ?

## Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 q. 2 – piste À chaque tour : on récupère le dernier bit `n % 2`, on le place à la bonne position en le multipliant par `puissance`, puis on passe au bit suivant (`n` devient `n // 2`, `puissance` est multipliée par 10). ↑énoncé

Ex. 2 q. 5 – piste Compter les bits modifiés dans chacun des deux messages reçus. Un nombre pair de bits à 1 peut-il le rester quand on change deux bits ? ↑énoncé

Ex. 3 q. 4 – piste On code `a` et `b`, on additionne les deux codes, on prend le reste modulo 256 (on « oublie » la retenue qui déborde), puis on revient à un entier relatif avec `valeur`. ↑

Ex. 4 q. 3 – piste C'est la recherche d'un maximum du cours : on garde le meilleur trouvé jusqu'ici (son prénom *et* sa moyenne) et on le remplace dès qu'on trouve mieux. ↑énoncé

Ex. 5 q. 4 – piste En lisant les bits de gauche à droite, on peut construire le nombre au fur et à mesure : à chaque nouveau bit, la valeur déjà obtenue est multipliée par 2 (tous ses bits se décalent d'un cran vers la gauche), puis on ajoute le nouveau bit. ↑énoncé

Ex. 6 q. 3 – piste On déplace des fiches entières, mais on compare leurs temps : la condition du `while` devient `table[j]["temps"] > fiche["temps"]`. ↑énoncé

Ex. 7 q. 2 – piste Pour le dernier indice `i`, que vaut `i + 1` ? Quel est le dernier indice valide d'un tableau ? ↑énoncé

Ex. 7 q. 5 – piste Parcourir les indices à partir de 1 et comparer `t[i]` à `t[i - 1]`. Avant d'ajouter une valeur, vérifier qu'elle n'est pas déjà la dernière de la liste résultat. ↑énoncé

Ex. 8 q. 2 – piste Comme pour lire un nombre binaire de gauche à droite : on part de `v = 0` et, pour chaque caractère, `v` devient `2 * v` plus le bit (0 pour `-`, 1 sinon). ↑énoncé

Ex. 8 q. 4 – piste Pour un chiffre `d` : le bit de poids 4 est `d // 4`, celui de poids 2 est `(d // 2) % 2`, celui de poids 1 est `d % 2`. ↑énoncé

Ex. 9 q. 2 – piste Trois découpages successifs : selon `"?"` (on garde la partie de droite), selon `"&"`, puis chaque paire selon `"="`. ↑énoncé

Ex. 10 q. 3 – piste Comme au TP des algorithmes gloutons, écrire une fonction `points(f)` qui renvoie `f[2]`, le nombre de points du film `f` ; l'ordre se construit alors avec `sorted(soiree, key=points, reverse=True)`. Pour le second critère, une fonction `points_par_minute(f)` renvoie `f[2] / f[1]`. ↑énoncé

Ex. 10 q. 4 – piste Pour l'entier `code`, le film d'indice `i` est pris si le bit de poids $2^i$ vaut 1, c'est-à-dire si `(code // 2 ** i) % 2 == 1`. ↑énoncé

II

DEUXIÈME PARTIE

# Corrigés

CORRIGÉS DU CHAPITRE 1

# 1 Les bases de la programmation Python

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 L'ordinateur humain ↑énoncé p. 18

**Exercice 1** *Des boîtes et une flèche*

▶ **Solution.** **1.** `a` : 7, puis 2 (le 7 est gommé) ; `b` : 10.

**2.** Affichage : `2 10`.

**3.** Non : à la ligne 2, on a calculé `a + 3` avec la valeur *de ce moment-là* (7) et rangé le **résultat** 10 dans `b`. La boîte `b` contient un nombre, pas la formule « `a + 3` » : modifier `a` ensuite ne change pas `b`.

**4.** Affichage : `6`.

**5.** On commence par la **droite** : on calcule `n + 1` avec le contenu actuel de `n` ($5+1=6$), puis on range le résultat dans la boîte de **gauche**, `n`. Le signe `=` ne veut pas dire « est égal à » mais « **reçoit** ».

**Exercice 2** *Échanger deux boîtes*

▶ **Solution.** **6.** `a` : 1 puis 2 ; `b` : 2 puis 2. Affichage : `2 2`. L'échange a échoué.

**7.** La valeur `1` a été **gommée** par `a = b` : elle est perdue avant d'avoir pu être rangée dans `b`. Correction avec une troisième boîte :

```
1 a = 1
2 b = 2
3 c = a      # on met la valeur de a a l'abri
4 a = b
5 b = c
6 print(a, b)   # 2 1
```

Image à donner : pour échanger le contenu de deux verres, il faut un troisième verre vide.

**Exercice 3** *Répéter sans réécrire*

▶ **Solution.** **8.**

| | avant la boucle | tour 1 | tour 2 | tour 3 | tour 4 |
|---|---|---|---|---|---|
| `i` | — | 1 | 2 | 3 | 4 |
| `total` | 0 | 1 | 3 | 6 | 10 |

Affichage : `10`.

**9.** `print(total)` : une seule fois (elle n'est pas décalée, elle est exécutée après la boucle). `total = total + i` : quatre fois, une par tour.

**10.** La ligne fait alors partie de la boucle : elle est exécutée à chaque tour. Affichage : `1`, `3`, `6`, `10` (un nombre par ligne). Le décalage (l'**indentation**) change le sens du programme.

**11.** Remplacer `range(1, 5)` par `range(1, 101)` : la borne de droite n'est pas atteinte. Le programme affiche alors `5050`.

**Exercice 4** *Ce qu'on a découvert*

▶ **Solution. 12.** Réponse possible :

- un nom comme `total` désigne une boîte de la mémoire qui contient une valeur : c'est une **variable** ;
- pour exécuter `nom = calcul`, l'ordinateur calcule d'abord la partie droite avec les valeurs actuelles, puis range le résultat dans la boîte de gauche, en effaçant l'ancienne valeur : c'est l'**affectation** ;
- les lignes décalées sous un `for` sont répétées, une fois par valeur de `i` : c'est une **boucle**.

# CORRIGÉS DES EXERCICES

**Remarque**
Plusieurs écritures sont souvent possibles : on donne **une** version simple.

## Variables, affectation, types

**Exercice 1** *Lecture de trace* ↑énoncé p. 39

▶ **Solution.** On suit chaque affectation (on évalue la droite, on range à gauche) :

| instruction | x | y |
|---|---|---|
| `x = 5` | 5 | — |
| `y = 3` | 5 | 3 |
| `x = x + y` | 8 | 3 |
| `y = x - y` | 8 | 5 |
| `x = x - y` | 3 | 5 |

L'affichage est donc `3 5`. Ce programme **échange** les contenus de `x` et `y` *sans* variable temporaire, en n'utilisant que des additions et des soustractions.

**Exercice 2** *Échanger deux variables* ↑énoncé p. 39

▶ **Solution. 1.** `temp` reçoit `"maths"`, puis `soir` reçoit `"maths"` (la valeur `"NSI"` est perdue), puis `matin` reçoit `temp`, c'est-à-dire `"maths"` : les deux variables valent `"maths"`. L'erreur : la variable temporaire a mis à l'abri `matin`, mais c'est `soir` qui est écrasée en premier ; et `matin` ne reçoit jamais l'ancienne valeur de `soir`.

**2.**

```
matin = "maths"
soir = "NSI"
temp = matin      # on met a l'abri la valeur de matin
matin = soir      # matin recoit "NSI"
soir = temp       # soir recoit l'ancienne valeur de matin
print(matin, soir)    # NSI maths
```

La variable temporaire met à l'abri la valeur que la première affectation va écraser (ici celle de `matin`).

▷ **Erreur fréquente.** écrire seulement `matin = soir` puis `soir = matin` : après la première ligne, `matin` vaut déjà `"NSI"`, et les deux variables finissent à `"NSI"`.

### Exercice 3 *Convertisseur* ↑énoncé p. 39

**▶ Solution.** On suit le gabarit du cours : le calcul s'écrit avec le paramètre et on le renvoie. Ajouter 20 % revient à multiplier par $1 + \frac{20}{100} = 1{,}20$.

```
def celsius_en_fahrenheit(c):
    return 9 / 5 * c + 32

def prix_ttc(ht):
    return ht * 1.20
```

On teste dans la console :

```
>>> celsius_en_fahrenheit(100)
212.0
>>> celsius_en_fahrenheit(0)
32.0
>>> prix_ttc(50)
60.0
```

### Exercice 4 *Rectangle : périmètre et aire* ↑énoncé p. 40

**▶ Solution.** À la main, pour 3 sur 4 : périmètre $3 + 4 + 3 + 4 = 2 \times (3 + 4) = 14$ et aire $3 \times 4 = 12$ ; on écrit les mêmes opérations avec les paramètres.

```
def perimetre(largeur, hauteur):
    return 2 * (largeur + hauteur)

def aire(largeur, hauteur):
    return largeur * hauteur
```

On teste dans la console :

```
>>> perimetre(3, 4)
14
>>> aire(3, 4)
12
```

### Exercice 5 *Le bon usage de `//` et `%`* ↑énoncé p. 40

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert dizaines(347) == 4       # cas normal
assert dizaines(5) == 0         # cas limite : un seul chiffre
assert dizaines(305) == 0       # cas piege : le chiffre des dizaines est 0
```

`n % 10` isole le dernier chiffre ; `n // 10` « décale » le nombre d'un cran vers la droite.

```
def unites(n):
    return n % 10

def dizaines(n):
    return (n // 10) % 10

def centaines(n):
    return (n // 100) % 10
```

On teste dans la console :

```
>>> unites(347)
7
>>> dizaines(347)
4
>>> centaines(347)
3
>>> unites(60)
0
>>> dizaines(60)
```

```
6
>>> centaines(60)
0
>>> unites(7)
7
>>> dizaines(7)
0
>>> centaines(7)
0
```

## Les conditionnelles

**Exercice 6** *Pair ou impair* ↑énoncé p. 40

**▶ Solution.** Un entier est pair quand le reste de sa division par 2 est nul (test de divisibilité du cours) : la condition est `n % 2 == 0`.

```
1 def est_pair(n):
2     return n % 2 == 0
```

On teste dans la console :

```
>>> est_pair(4)
True
>>> est_pair(7)
False
>>> est_pair(0)
True
```

Remarquer qu'on *renvoie directement* le booléen `n % 2 == 0` : inutile d'écrire `if ...: return True else: return False`.

▷ **Erreur fréquente.** écrire `n % 2 = 0` : le signe `=` est l'affectation, la comparaison s'écrit `==`. Python répond par une `SyntaxError`.

**Exercice 7** *Le plus grand des deux* ↑énoncé p. 40

**▶ Solution.** Deux cas suffisent : si `a > b`, on renvoie `a` ; sinon (`b` plus grand ou égal), on renvoie `b`.

```
1 def maximum2(a, b):
2     if a > b:
3         return a
4     return b        # couvre aussi le cas a == b
```

On teste dans la console :

```
>>> maximum2(3, 5)
5
>>> maximum2(5, 3)
5
>>> maximum2(4, 4)
4
```

▷ **Erreur fréquente.** écrire `if a > b: return a` puis `if b > a: return b` : pour `a == b`, aucune condition n'est vraie et la fonction renvoie `None`. Le test d'égalité `maximum2(4, 4)` démasque l'oubli.

**Exercice 8** *Le signe d'un nombre* ↑énoncé p. 40

**▶ Solution.** Trois cas qui s'excluent : `if` pour le premier, `elif` pour le deuxième, `else` pour « tout le reste » (ici `x` nul).

```
def signe(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0
```

On teste dans la console :

```
>>> signe(12)
1
>>> signe(-3)
-1
>>> signe(0)
0
```

**Exercice 9** *Mention au bac* ↑énoncé p. 40

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert mention(13) == "Assez bien"      # cas normal
assert mention(16) == "Tres bien"       # cas limite : le seuil est inclus
assert mention(9.5) == "Refuse"         # cas piege : une note a virgule sous 10
```

On teste les seuils du plus haut au plus bas avec `if`/`elif`, et le `else` final recueille les notes inférieures à 10.

```
def mention(note):
    if note >= 16:
        return "Tres bien"
    elif note >= 14:
        return "Bien"
    elif note >= 12:
        return "Assez bien"
    elif note >= 10:
        return "Admis"
    else:
        return "Refuse"
```

**1.** L'ordre va du plus exigeant au moins exigeant, car Python exécute le *premier* `elif` vrai. Si on testait `note >= 10` en premier, une note de `18` renverrait « Admis » : toutes les bonnes notes seraient mal classées.

**2.** `mention(25)` renvoie « Tres bien » (car $25 \geqslant 16$) : la fonction accepte sans broncher une note impossible. C'est justement ce qu'une *précondition* permettra d'interdire (section *Les fonctions* du cours).

**Exercice 10** *Année bissextile* ↑énoncé p. 41

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert est_bissextile(2024) == True     # cas normal
assert est_bissextile(2400) == True     # cas limite : divisible par 400
assert est_bissextile(2100) == False    # cas piege : divisible par 4 et par 100
```

On traduit la phrase mot à mot : « divisible par 400 » donne `annee % 400 == 0`, « *ou* divisible par 4 *et* pas par 100 » donne `or (... and ...)`. Un seul booléen suffit :

```
def est_bissextile(annee):
    return annee % 400 == 0 or (annee % 4 == 0 and annee % 100 != 0)
```

On teste dans la console :

```
>>> est_bissextile(2000)
True
>>> est_bissextile(1900)
False
>>> est_bissextile(2024)
True
>>> est_bissextile(2023)
False
```

**Exercice 11** *Le plus grand des trois* ↑énoncé p. 41

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert maximum3(3, 7, 5) == 7        # cas normal
assert maximum3(4, 4, 4) == 4        # cas limite : trois valeurs egales
assert maximum3(-2, -8, -5) == -2    # cas piege : que des negatifs
```

On garde un « champion provisoire » que l'on met à jour :

```
def maximum3(a, b, c):
    m = a
    if b > m:
        m = b
    if c > m:
        m = c
    return m
```

On teste dans la console :

```
>>> maximum3(3, 9, 5)
9
>>> maximum3(3, 5, 9)
9
>>> maximum3(4, 4, 4)
4
```

**Exercice 12** *Quel type de triangle ?* ↑énoncé p. 41

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert type_triangle(3, 4, 5) == "scalene"        # cas normal
assert type_triangle(2, 2, 2) == "equilateral"    # cas limite : trois cotes egaux
assert type_triangle(5, 3, 5) == "isocele"        # cas piege : a et c egaux, pas a et b
```

On teste l'égalité des trois côtés, puis d'exactement deux (grâce à l'ordre : si les trois étaient égaux, on serait déjà sorti).

```
def type_triangle(a, b, c):
    if a == b and b == c:
        return "equilateral"
    elif a == b or b == c or a == c:
        return "isocele"
    else:
        return "scalene"
```

On teste dans la console :

```
>>> type_triangle(3, 3, 3)
'equilateral'
>>> type_triangle(3, 3, 5)
'isocele'
>>> type_triangle(3, 4, 5)
'scalene'
```

**Exercice 13** *Réduction par paliers* ↑énoncé p. 41

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert prix_reduit(200) == 190.0      # cas normal
assert prix_reduit(100) == 95.0       # cas limite : 100 est deja au palier de 5 %
assert prix_reduit(300) == 270.0      # cas piege : 300 est au palier de 10 %
```

On choisit d'abord le taux avec `if/elif/else` (bornes : `< 100`, puis `< 300`, donc `300` est au palier de 10 %), puis on calcule une seule fois le prix réduit.

```
def prix_reduit(montant):
    if montant < 100:
        taux = 0
    elif montant < 300:
        taux = 5
    else:
        taux = 10
    return montant * (1 - taux / 100)
```

On teste dans la console :

```
>>> prix_reduit(50)
50.0
>>> prix_reduit(200)
190.0
>>> prix_reduit(300)
270.0
```

## Les boucles bornées (`for`)

**Exercice 14** *Remettre l'accumulateur dans l'ordre* ↑énoncé p. 41

**▶ Solution.** **1.** On initialise l'accumulateur *avant* la boucle, on le met à jour *dans* la boucle (un cran d'indentation de plus), on le renvoie *après* la boucle :

```
def somme_entre(a, b):
    total = 0                  # INITIALISER
    for i in range(a, b + 1):  # PARCOURIR
        total = total + i      # METTRE A JOUR
    return total               # RENVOYER
```

On teste dans la console :

```
>>> somme_entre(3, 6)    # 3 + 4 + 5 + 6
18
```

**2.** `somme_entre(3, 6)` renvoie bien `18`.

**3.** L'accumulateur serait remis à `0` à chaque tour : à la fin, `total` ne contiendrait que la dernière valeur ajoutée, `b`. Ici, la fonction renverrait `6` au lieu de `18`.

**Exercice 15** *Sommes* ↑énoncé p. 42

**▶ Solution.** Motif de l'accumulateur : `total` vaut `0` avant la boucle, on lui ajoute `i` (puis `i * i`) à chaque tour, et on le renvoie après la boucle. Pour aller jusqu'à `n` inclus, on écrit `range(1, n + 1)`.

```
def somme(n):
    total = 0                  # INITIALISER
    for i in range(1, n + 1):  # PARCOURIR
        total = total + i      # METTRE A JOUR
    return total               # RENVOYER

def somme_carres(n):
    total = 0                  # INITIALISER
    for i in range(1, n + 1):  # PARCOURIR
        total = total + i * i  # METTRE A JOUR
    return total               # RENVOYER
```

On teste dans la console :

```
>>> somme(100)
5050
>>> somme_carres(3)    # 1 + 4 + 9
14
```

▷ **Erreur fréquente.** écrire `range(1, n)` : la borne `n` est exclue, et `somme(100)` renverrait `4950` au lieu de `5050`.

### Exercice 16 *Table de multiplication* ↑énoncé p. 42

**▶ Solution.** La variable `i` doit prendre les valeurs 1 à 10, d'où `range(1, 11)` (borne exclue). La fonction **affiche** et ne renvoie rien : on l'appelle et on contrôle l'affichage à l'écran.

```
def table(n):
    for i in range(1, 11):
        print(n, "x", i, "=", n * i)

table(7)    # affiche 7 x 1 = 7, puis 7 x 2 = 14, ... jusqu'a 7 x 10 = 70
```

### Exercice 17 *Construire une chaîne* ↑énoncé p. 42

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert ligne_etoiles(3) == "***"     # cas normal
assert ligne_etoiles(0) == ""        # cas limite : aucune etoile
assert ligne_etoiles(1) == "*"       # cas piege : un seul tour de boucle
```

On part d'une chaîne vide et on ajoute une étoile à chaque tour (accumulateur de texte, exactement comme l'accumulateur numérique ci-dessus).

```
def ligne_etoiles(n):
    chaine = ""                # INITIALISER : la chaine vide
    for i in range(n):         # PARCOURIR
        chaine = chaine + "*"  # METTRE A JOUR
    return chaine              # RENVOYER
```

On teste dans la console :

```
>>> ligne_etoiles(5)
'*****'
>>> ligne_etoiles(0)
''
```

### Exercice 18 *Factorielle* ↑énoncé p. 42

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert factorielle(5) == 120     # cas normal
assert factorielle(1) == 1       # cas limite : un seul coureur
assert factorielle(0) == 1       # cas piege : 0! vaut 1, pas 0
```

L'accumulateur d'un *produit* s'initialise à `1`. La boucle `range(2, n+1)` ne fait aucun tour si `n` vaut `0` ou `1`, ce qui donne bien `1`.

```
def factorielle(n):
    produit = 1                # INITIALISER : element neutre du produit
    for i in range(2, n + 1):  # PARCOURIR
        produit = produit * i  # METTRE A JOUR
    return produit             # RENVOYER
```

On teste dans la console :

```
>>> factorielle(0)
1
>>> factorielle(1)
1
>>> factorielle(5)
120
```

Avec 5 coureurs, il y a donc `120` ordres d'arrivée possibles ; avec 10 coureurs, déjà `factorielle(10)` = `3628800`.

▷ **Erreur fréquente.** initialiser `produit = 0`, par habitude de la somme : tous les produits valent alors `0`.

### Exercice 19 *Compter les voyelles* ↑énoncé p. 42

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert compter_voyelles("bonjour") == 3    # cas normal
assert compter_voyelles("") == 0           # cas limite : mot vide
assert compter_voyelles("rythme") == 2     # cas piege : le y compte
```

Motif du **compteur** : `n` part de `0`, on parcourt le mot lettre par lettre et on ajoute `1` seulement quand la lettre est une voyelle.

```
def compter_voyelles(mot):
    n = 0                              # INITIALISER
    for lettre in mot:                 # PARCOURIR
        if lettre in "aeiouyAEIOUY":   # TESTER
            n = n + 1                  # METTRE A JOUR
    return n                           # RENVOYER
```

On teste dans la console :

```
>>> compter_voyelles("bonjour")
3
>>> compter_voyelles("")
0
```

▷ **Erreur fréquente.** indenter `return n` au niveau du `if` (dans la boucle) : la fonction s'arrête dès la première lettre et `compter_voyelles("bonjour")` renvoie `0`.

### Exercice 20 *Diviseurs et nombres parfaits* ↑énoncé p. 43

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert somme_diviseurs(6) == 12    # cas normal : 1 + 2 + 3 + 6
assert somme_diviseurs(1) == 1     # cas limite : 1 n'a qu'un diviseur
assert somme_diviseurs(7) == 8     # cas piege : n lui-meme compte (1 + 7)
```

Accumulateur sous condition : on essaie tous les `d` de `1` à `n` et on n'ajoute que ceux qui vérifient `n % d == 0`. `est_parfait` réutilise ensuite `somme_diviseurs`.

```
def somme_diviseurs(n):
    total = 0                  # INITIALISER
    for d in range(1, n + 1):  # PARCOURIR
        if n % d == 0:         # TESTER : d divise-t-il n ?
            total = total + d  # METTRE A JOUR
    return total               # RENVOYER

def est_parfait(n):
    return somme_diviseurs(n) == 2 * n

for k in range(1, 100):
    if est_parfait(k):
        print(k)               # affiche 6 puis 28
```

On teste dans la console :

```
>>> somme_diviseurs(6)
12
>>> est_parfait(28)
True
>>> est_parfait(12)
False
```

Les nombres parfaits inférieurs à 100 sont `6` et `28`.

**Exercice 21** *Compter les occurrences d'une lettre* ↑énoncé p. 43

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert occurrences("a", "banana") == 3    # cas normal
assert occurrences("a", "") == 0          # cas limite : mot vide
assert occurrences("A", "banana") == 0    # cas piege : "A" et "a" sont differents
```

Même compteur que « Compter les voyelles » ; seule la condition change : `lettre == caractere`.

```
def occurrences(caractere, mot):
    n = 0                       # INITIALISER
    for lettre in mot:          # PARCOURIR
        if lettre == caractere: # TESTER
            n = n + 1           # METTRE A JOUR
    return n                    # RENVOYER
```

On teste dans la console :

```
>>> occurrences("a", "banana")
3
>>> occurrences("z", "banana")
0
```

**Exercice 22** *Puissance sans* `**` ↑énoncé p. 43

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert puissance(2, 10) == 1024    # cas normal
assert puissance(5, 0) == 1        # cas limite : exposant nul
assert puissance(-2, 3) == -8      # cas piege : base negative
```

L'accumulateur d'un produit s'initialise à `1`. La boucle fait `n` tours ; si `n` vaut `0`, elle n'en fait aucun et renvoie `1`.

```
def puissance(a, n):
    resultat = 1                # INITIALISER : a puissance 0
    for i in range(n):          # PARCOURIR : n tours
        resultat = resultat * a # METTRE A JOUR
    return resultat             # RENVOYER
```

On teste dans la console :

```
>>> puissance(2, 10)
1024
>>> puissance(5, 0)
1
>>> puissance(3, 3)
27
```

**Exercice 23** *FizzBuzz* ↑énoncé p. 43

**▶ Solution.** Le piège : il faut tester le cas « multiple de 3 **et** de 5 » (c'est-à-dire multiple de 15) **en premier**, sinon il serait capté par `% 3` et ne donnerait jamais « FizzBuzz ».

```
def fizzbuzz(n):
    for i in range(1, n + 1):
        if i % 15 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)

fizzbuzz(15)    # la derniere ligne affichee doit etre FizzBuzz
```

**Exercice 24** *Un triangle d'étoiles* ↑énoncé p. 43

▶ **Solution.** `"*" * i` construit une chaîne de `i` étoiles ; on affiche une ligne par tour de boucle.

```
def triangle(n):
    for i in range(1, n + 1):
        print("*" * i)

triangle(3)    # affiche *, puis **, puis ***
```

Variante sans `"*" * i`, en réutilisant `ligne_etoiles` : `print(ligne_etoiles(i))` à chaque tour.

## Les boucles non bornées (`while`)

**Exercice 25** *Compte à rebours* ↑énoncé p. 44

▶ **Solution.**

```
def compte_a_rebours(n):
    while n > 0:   # PARCOURIR : de n jusqu'a 1
        print(n)
        n = n - 1  # METTRE A JOUR : le variant diminue
```

**Terminaison.** La variable `n` est un **variant** : elle est positive tant qu'on entre dans la boucle (`n > 0`) et diminue de `1` à chaque tour. Une suite d'entiers positifs qui décroît strictement atteint forcément `0`, ce qui rend la condition fausse : la boucle se termine.

▷ **Erreur fréquente.** oublier `n = n - 1` : `n` ne change plus, la condition reste vraie et la boucle est infinie (`Ctrl + C` pour l'arrêter).

**Exercice 26** *Décortiquer un entier* ↑énoncé p. 44

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert nombre_de_chiffres(347) == 3    # cas normal
assert nombre_de_chiffres(0) == 1      # cas limite : 0 s'ecrit avec un chiffre
assert nombre_de_chiffres(100) == 3    # cas piege : les zeros comptent
```

On ne sait pas d'avance combien de chiffres a `n` : boucle `while n > 0`. À chaque tour, on traite le dernier chiffre (`n % 10`), puis on le retire (`n = n // 10`).

```
def nombre_de_chiffres(n):
    if n == 0:
        return 1                # RENVOYER : 0 s'ecrit avec un chiffre
    compteur = 0                # INITIALISER
    while n > 0:                # PARCOURIR : chiffre par chiffre
        n = n // 10             # METTRE A JOUR : on retire le dernier chiffre
        compteur = compteur + 1 # METTRE A JOUR
    return compteur             # RENVOYER

def somme_chiffres(n):
    total = 0                   # INITIALISER
    while n > 0:                # PARCOURIR
        total = total + n % 10  # METTRE A JOUR
        n = n // 10             # METTRE A JOUR
    return total                # RENVOYER
```

On teste dans la console :

```
>>> nombre_de_chiffres(347)
3
>>> nombre_de_chiffres(0)
1
>>> somme_chiffres(347)
14
```

Dans les deux fonctions, `n` est un **variant** : c'est un entier positif, et l'affectation `n = n // 10` le fait décroître strictement à chaque tour (car `n` $\geqslant 1$ dans la boucle), jusqu'à `0`.

▷ **Erreur fréquente.** oublier le cas `n = 0` : la boucle ne fait aucun tour et `nombre_de_chiffres(0)` renverrait `0` au lieu de `1`.

### Exercice 27 *Racine entière* ↑énoncé p. 44

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert racine_entiere(15) == 3    # cas normal
assert racine_entiere(0) == 0     # cas limite : aucune dalle
assert racine_entiere(16) == 4    # cas piege : carre parfait, 4 * 4 = 16 convient
```

On part de `k = 0` et on augmente `k` tant que le carré du *suivant* ne dépasse pas `n` ; en sortie, `k` convient et `k + 1` ne convient plus.

```
def racine_entiere(n):
    k = 0                          # INITIALISER
    while (k + 1) * (k + 1) <= n:  # TESTER : k + 1 convient-il encore ?
        k = k + 1                  # METTRE A JOUR
    return k                       # RENVOYER
```

On teste dans la console :

```
>>> racine_entiere(15)
3
>>> racine_entiere(16)
4
>>> racine_entiere(0)
0
```

Avec 15 dalles, la plus grande terrasse carrée fait $3 \times 3$ (9 dalles) ; avec 16 dalles, $4 \times 4$. **Terminaison.** La quantité `n - k*k` décroît strictement (`k` augmente de `1` à chaque tour) et reste positive tant qu'on boucle : la boucle s'arrête donc.

### Exercice 28 *Dépasser un seuil* ↑énoncé p. 44

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert seuil(1000) == 45    # cas normal
assert seuil(0) == 1        # cas limite : des le premier jour
assert seuil(10) == 5       # cas piege : 1 + 2 + 3 + 4 = 10 ne depasse pas 10
```

Deux variables avancent ensemble (le jour `n` et le contenu `total`) ; on continue *tant que* le seuil n'est pas dépassé, c'est-à-dire tant que `total <= s`.

```
def seuil(s):
    n = 0                  # INITIALISER : le jour
    total = 0              # INITIALISER : le contenu de la tirelire
    while total <= s:      # PARCOURIR : jour apres jour
        n = n + 1          # METTRE A JOUR
        total = total + n  # METTRE A JOUR
    return n               # RENVOYER
```

On teste dans la console :

```
>>> seuil(1000)    # 1+...+44 = 990, 1+...+45 = 1035
45
```

Le total `total` est le contenu de la tirelire et `n` le numéro du jour : Léa dépasse 1000 € le 45e jour. **Terminaison.** La quantité `s - total` décroît strictement à chaque tour (on ajoute au moins `1` à `total`) : elle finit par devenir négative, ce qui arrête la boucle.

**Exercice 29** *PGCD (algorithme d'Euclide)* ↑énoncé p. 44

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert pgcd(12, 18) == 6     # cas normal
assert pgcd(7, 7) == 7       # cas limite : a et b egaux, aucun tour
assert pgcd(17, 5) == 1      # cas piege : aucun diviseur commun sauf 1
```

```
def pgcd(a, b):
    while a != b:       # TESTER : on continue tant qu'ils different
        if a > b:
            a = a - b   # METTRE A JOUR
        else:
            b = b - a   # METTRE A JOUR
    return a            # RENVOYER
```

On teste dans la console :

```
>>> pgcd(12, 18)
6
>>> pgcd(7, 7)
7
>>> pgcd(1071, 462)
21
```

**Terminaison.** La quantité `a + b` est un variant : à chaque tour on retranche à l'un des deux une valeur strictement positive, donc `a + b` décroît strictement tout en restant $\geqslant 2$. La boucle ne peut donc pas tourner indéfiniment.

**Exercice 30** *Racine numérique* ↑énoncé p. 45

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert racine_numerique(347) == 5    # cas normal
assert racine_numerique(7) == 7      # cas limite : deja un seul chiffre
assert racine_numerique(19) == 1     # cas piege : 19 -> 10 -> 1, deux tours
```

Tant que `n` a plus d'un chiffre, on le remplace par la somme de ses chiffres.

```
def somme_chiffres(n):          # deja ecrite plus haut
    total = 0
    while n > 0:
        total = total + n % 10
        n = n // 10
    return total

def racine_numerique(n):
    while n >= 10:              # TESTER : plus d'un chiffre ?
        n = somme_chiffres(n)   # METTRE A JOUR
    return n                    # RENVOYER
```

On teste dans la console :

```
>>> racine_numerique(347)
5
>>> racine_numerique(9)
9
>>> racine_numerique(12345)
6
```

**Terminaison.** Dès que `n` a au moins deux chiffres, la somme de ses chiffres est *strictement plus petite* que `n` : la valeur de `n` décroît strictement à chaque tour tout en restant positive, jusqu'à passer sous 10.

## Fonctions : spécifier et tester

### Exercice 31 *Nombres premiers* ↑énoncé p. 45

**▶ Solution.** On écarte d'abord les entiers inférieurs à 2, puis on cherche un diviseur entre 2 et `n - 1`. Le jeu de tests contient des cas normaux et les cas limites 0, 1, 2 demandés.

```
def est_premier(n):
    if n < 2:
        return False
    for d in range(2, n):       # PARCOURIR : les diviseurs possibles
        if n % d == 0:          # TESTER
            return False        # RENVOYER : un diviseur trouve, on sort
    return True                 # RENVOYER : aucun diviseur

assert est_premier(2) == True   # cas limite
assert est_premier(7) == True
assert est_premier(97) == True
assert est_premier(0) == False  # cas limite
assert est_premier(1) == False  # cas limite
assert est_premier(9) == False
```

Dès qu'un diviseur est trouvé, `return False` sort immédiatement de la fonction : inutile de continuer.

▷ **Erreur fréquente.** ajouter `else: return True` dans la boucle : la fonction répond dès le premier diviseur essayé (`d = 2`), et `est_premier(9)` renverrait `True`. On ne peut conclure « premier » qu'*après* la boucle.

### Exercice 32 *Palindrome* ↑énoncé p. 45

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert est_palindrome("kayak") == True    # cas normal
assert est_palindrome("") == True         # cas limite : mot vide
assert est_palindrome("abca") == False    # cas piege : memes bords, mais pas le reste
```

On compare les caractères par paires, en partant des deux bords et en se rapprochant du centre avec deux indices `i` et `j`. La quantité `j - i` est un variant : elle diminue de 2 à chaque tour.

```
def est_palindrome(mot):
    i = 0                       # INITIALISER
    j = len(mot) - 1            # INITIALISER
    while i < j:                # PARCOURIR : des deux bords vers le centre
        if mot[i] != mot[j]:    # TESTER
            return False        # RENVOYER
        i = i + 1               # METTRE A JOUR
        j = j - 1               # METTRE A JOUR
    return True                 # RENVOYER

assert est_palindrome("radar") == True
assert est_palindrome("bonjour") == False
assert est_palindrome("") == True  # cas limites
assert est_palindrome("a") == True
```

### Exercice 33 *Jouer avec les bords d'un mot* ↑énoncé p. 45

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert meme_bord("radar") == True     # cas normal
assert meme_bord("a") == True         # cas limite : une seule lettre
assert meme_bord("Radar") == False    # cas piege : "R" et "r" sont differents
```

Aucune boucle : on compare directement la première lettre (`mot[0]`) et la dernière (`mot[-1]`), après avoir écarté le mot vide, qui n'a pas de première lettre.

```python
def meme_bord(mot):
    if mot == "":
        return False          # cas limite : pas de premiere lettre a comparer
    return mot[0] == mot[-1]

def memes_bords(mot1, mot2):
    if mot1 == "" or mot2 == "":
        return False          # cas limite : pas de premiere lettre a comparer
    return mot1[0] == mot2[0] and mot1[-1] == mot2[-1]

assert meme_bord("radar") == True
assert meme_bord("bonjour") == False
assert memes_bords("chat", "chocolat") == True
assert memes_bords("chat", "chien") == False
assert memes_bords("", "chat") == False
assert meme_bord("") == False
```

*Mot vide :* sans précaution, `mot[0]` provoquerait une `IndexError`. Renvoyer `False` est un **choix de spécification** (« pas de lettre, donc pas de bords égaux ») ; on le garde le même dans les deux fonctions et on l'écrit dans la docstring. Le choix inverse (`True`) se défend aussi, pourvu qu'il soit annoncé.

### Exercice 34 *Le bug qui passe les tests* ↑énoncé p. 46

**▶ Solution. 1.** La fonction renvoie `a * b` au lieu de `a + b`. Or les deux tests choisis tombent sur des cas où produit et somme coïncident : $2 \times 2 = 2 + 2 = 4$ et $0 \times 0 = 0 + 0 = 0$. Les `assert` passent donc, alors que la fonction est fausse.

**2.** Un test qui les sépare démasque le bug, par exemple :

```python
assert addition(1, 3) == 4    # echoue : 1 * 3 = 3, pas 4
```

**3.** Réussir un jeu de tests prouve seulement que la fonction est correcte **sur les cas testés**, jamais qu'elle l'est *en général*. Un jeu de tests mal choisi peut laisser passer un programme faux : c'est exactement le sens de la phrase du programme officiel. D'où l'importance de tester des cas **variés** (normaux, limites, particuliers).

## Exercices de synthèse

### Exercice 35 *Les horaires du bus* ↑énoncé p. 46

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```python
assert prochain_bus(400) == 402      # cas normal
assert prochain_bus(390) == 390      # cas limite : un bus part pile a cet instant
assert prochain_bus(1255) == -1      # cas piege : avant 21 h, mais apres le dernier bus
```

On part du premier bus (390) et on passe au suivant (`+ 12`) tant qu'il part avant l'instant `t`. En sortie de boucle, il reste à vérifier que ce bus part encore avant 21 h 00, soit $21 \times 60 = 1260$.

```python
def en_minutes(h, m):
    return h * 60 + m

def prochain_bus(t):
    depart = 390                 # INITIALISER : 6 h 30
    while depart < t:            # PARCOURIR : bus apres bus
        depart = depart + 12     # METTRE A JOUR : bus suivant
    if depart > 1260:            # TESTER : apres 21 h 00, plus de bus
        return -1
    return depart                # RENVOYER

def attente(h, m):
    t = en_minutes(h, m)
    depart = prochain_bus(t)
    if depart == -1:
        return -1
```

```
    return depart - t

assert en_minutes(6, 30) == 390
assert prochain_bus(400) == 402
assert prochain_bus(0) == 390    # avant le premier bus
assert prochain_bus(402) == 402  # un bus part pile a cet instant
assert attente(8, 0) == 6
assert attente(22, 15) == -1
```

**4.** Les départs ont lieu à $390 + 12k$. Comme $1260 - 390 = 870 = 72 \times 12 + 6$, le dernier départ est $390 + 72 \times 12 = 1254$ minutes, et `1254 // 60` vaut 20, `1254 % 60` vaut 54 : le dernier bus part à **20 h 54**. (On peut le vérifier : `prochain_bus(1249)` renvoie 1254 et `prochain_bus(1255)` renvoie `-1`.)

**Exercice 36** *Un mot de passe robuste* ↑énoncé p. 46

**► Solution.** Même motif que « Compter les voyelles » : un compteur par catégorie de caractères, puis les quatre règles reliées par `and`.

```
def compter_chiffres(mdp):
    n = 0                                # INITIALISER
    for c in mdp:                        # PARCOURIR
        if c in "0123456789":            # TESTER
            n = n + 1                    # METTRE A JOUR
    return n                             # RENVOYER

def est_robuste(mdp):
    majuscules = 0                       # INITIALISER
    speciaux = 0                         # INITIALISER
    for c in mdp:                        # PARCOURIR
        if c in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
            majuscules = majuscules + 1  # METTRE A JOUR
        if c in "!?@#$%&*":
            speciaux = speciaux + 1      # METTRE A JOUR
    return (len(mdp) >= 8 and majuscules >= 1
            and compter_chiffres(mdp) >= 1 and speciaux >= 1)

assert compter_chiffres("Monaco98") == 2
assert compter_chiffres("") == 0
assert est_robuste("Monaco98!") == True
assert est_robuste("Mon98!") == False      # trop court
assert est_robuste("monaco98!") == False   # pas de majuscule
assert est_robuste("Monacooo!") == False   # pas de chiffre
assert est_robuste("Monaco98") == False    # pas de caractere special
```

Chaque mot de passe refusé n'enfreint qu'**une seule** règle : si un test échoue, on sait immédiatement quelle condition est mal écrite.

**Exercice 37** *Nombres premiers jumeaux* ↑énoncé p. 47

**► Solution.** On réutilise `est_premier`. La condition `p + 2 < n` garantit que les *deux* nombres du couple sont strictement inférieurs à `n`.

```
def premiers_jumeaux(n):
    for p in range(2, n):
        if p + 2 < n and est_premier(p) and est_premier(p + 2):
            print(p, "et", p + 2)

premiers_jumeaux(20)    # affiche 3 et 5, 5 et 7, 11 et 13, 17 et 19
```

**Exercice 38** *Une somme selon un assistant d'IA* ↑énoncé p. 47

**▶ Solution. 1.** Non. Trace de `somme_impairs(5)` : `range(1, 5, 2)` donne `1` puis `3`, et s'arrête **avant** `5` (la borne est exclue). `somme` vaut `0`, puis `1`, puis `4` : la fonction renvoie **4**, et non `9` comme l'annonce pourtant l'assistant.

```
>>> somme_impairs(5)
4
```

**2.** L'erreur est la borne du `range` : `range(1, n, 2)` exclut `n`. Il faut `range(1, n + 1, 2)` :

```
def somme_impairs(n):
    somme = 0                       # INITIALISER
    for i in range(1, n + 1, 2):    # PARCOURIR : 1, 3, 5... jusqu'a n inclus
        somme = somme + i           # METTRE A JOUR
    return somme                    # RENVOYER

print(somme_impairs(5))             # 9
print(somme_impairs(6))             # 9  (1 + 3 + 5)
```

**3.** Exécuter l'exemple que l'assistant donne lui-même : il annonce `9` pour `somme_impairs(5)`, la console affiche `4`. La réponse se contredit ; un seul appel suffisait. Le cas où `n` est lui-même impair est justement le cas limite à tester.

**Exercice 39** *Euclide en accéléré* ↑énoncé p. 47

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert pgcd_rapide(1071, 462) == 21    # cas normal
assert pgcd_rapide(12, 12) == 12       # cas limite : a et b egaux
assert pgcd_rapide(12, 18) == 6        # cas piege : a < b, le premier tour echange
```

**1.** On ajoute un compteur à la fonction de l'exercice « PGCD » :

```
def pgcd(a, b):
    """Renvoie le PGCD de a et b (entiers strictement positifs)."""
    tours = 0
    while a != b:
        if a > b:
            a = a - b
        else:
            b = b - a
        tours = tours + 1
    print(tours, "tours")
    return a

print(pgcd(1000, 3))    # 335 tours, puis 1
```

Il faut **335 tours** : 333 soustractions de 3 pour descendre de 1000 à 1, puis deux tours pour amener `b` de 3 à 1.

**2.** Retirer 3 autant de fois que possible à 1000, c'est faire la division euclidienne de 1000 par 3 : ce qui reste à la fin est le **reste** `1000 % 3`, qui vaut `1`.

**3.** On calcule d'abord le reste dans `r`, sinon l'ancienne valeur de `a` serait perdue :

```
def pgcd_rapide(a, b):
    """Renvoie le PGCD de a et b (entiers strictement positifs)."""
    tours = 0                   # INITIALISER
    while b != 0:               # TESTER
        r = a % b               # METTRE A JOUR : le reste d'abord
        a = b                   # METTRE A JOUR
        b = r                   # METTRE A JOUR
        tours = tours + 1       # METTRE A JOUR
    print(tours, "tours")
    return a                    # RENVOYER

print(pgcd_rapide(1000, 3))     # 2 tours, puis 1
```

```
print(pgcd_rapide(12, 18))      # 3 tours, puis 6
print(pgcd_rapide(1071, 462))   # 3 tours, puis 21
```

Pour `pgcd_rapide(1000, 3)`, **2 tours** suffisent au lieu de 335. Si `a < b`, le premier tour échange simplement les deux nombres (`12 % 18` vaut `12`).

**4.** La quantité `b` est un entier positif ou nul, et elle **diminue strictement** à chaque tour, car le reste de la division par `b` est toujours strictement inférieur à `b`. Une telle quantité ne peut pas diminuer indéfiniment : elle finit par valoir `0`, et la boucle s'arrête.

## Entraînement

### Exercice 40 *Lire un entier au clavier* ↑énoncé p. 48

**▶ Solution.** `input` renvoie une chaîne : on la convertit avec `int` avant de calculer.

```
age = int(input("Votre age ? "))
print("Dans 10 ans, vous aurez", age + 10, "ans")

prix = int(input("Prix d'un article (en euros) ? "))
print("Trois articles coutent", 3 * prix, "euros")

minutes = int(input("Nombre de minutes ? "))
print(minutes // 60, "h", minutes % 60, "min")    # 135 donne 2 h 15 min
```

Sans `int`, `age` est la chaîne `"16"` et `age + 10` provoque :

```
TypeError: can only concatenate str (not "int") to str
```

▷ **Erreur fréquente.** oublier `int` : on ne peut pas additionner un `str` et un `int`. Et avec `3 * prix` sans conversion, Python ne signale rien mais répète la chaîne : `3 * "4"` donne `"444"`.

### Exercice 41 *Longueur, premier et dernier caractère* ↑énoncé p. 48

**▶ Solution. 1.** a) `6`, `'M'`, `'o'` ; b) `3`, `'N'`, `'I'` ; c) `1`, `'z'`, `'z'` (premier et dernier caractère sont le même) ; d) `12` (l'espace compte), `'G'`, `'r'`.

**2.** Les positions vont de `0` à `len(mot) - 1` : la position `len(mot)` n'existe pas (`IndexError: string index out of range`).

**3.**

```
def dernier(mot):
    return mot[len(mot) - 1]
```

### Exercice 42 *Prédire l'affichage, puis exécuter :* `for` ↑énoncé p. 48

**▶ Solution.** (un nombre par ligne à l'écran) a) `0 3 6 9` : `i` prend les valeurs `0` à `3`. b) `2 6 10` : de 2 en 4 ; `14` dépasserait la borne exclue `12`. c) `1 3 6 10` : le `print` est *dans* la boucle, on voit le total après chaque tour. d) `ISN` : chaque lettre est placée *devant* le mot déjà construit, ce qui le renverse ; un seul affichage, après la boucle.

▷ **Erreur fréquente.** ne pas regarder l'indentation du `print` : en c), il est dans la boucle (quatre affichages) ; en d), il est après (un seul).

### Exercice 43 *Prédire l'affichage, puis exécuter :* `while` ↑énoncé p. 49

**▶ Solution.** a) `32` : `n` vaut 2, 4, 8, 16, puis 32, qui rend la condition fausse ; un seul affichage, après la boucle. b) `10 7 4 1` : après `1`, `n` vaut `-2` et la boucle s'arrête. c) seulement `fin` : la condition `5 < 5` est fausse dès le départ, la boucle ne fait aucun tour. d) `1 6` : `x` vaut 50, 25, 12, 6, 3, 1, soit 6 tours.

▷ **Erreur fréquente.** croire qu'une boucle `while` fait toujours au moins un tour : la condition est testée *avant* le premier tour (cas c).

### Exercice 44 *Spécifier et tester une fonction* ↑énoncé p. 49

**▶ Solution.** La docstring dit le rôle, ce que l'on attend des arguments (précondition) et ce que l'on garantit (postcondition) ; l'`assert` traduit la précondition ; les tests visent des cas normaux et un cas limite.

```
def prix_billet(age):
    """Renvoie le prix du billet (en euros) pour une personne de cet age.
    Precondition : age est un entier positif ou nul.
    Postcondition : renvoie 5 si age < 12, et 9 sinon."""
    assert age >= 0, "l'age doit etre positif"
    if age < 12:
        return 5
    return 9

assert prix_billet(8) == 5
assert prix_billet(30) == 9
assert prix_billet(12) == 9     # cas limite : le seuil de la condition

def aire_carre(cote):
    """Renvoie l'aire d'un carre de cote donne.
    Precondition : cote >= 0.
    Postcondition : le resultat est positif ou nul."""
    assert cote >= 0, "le cote doit etre positif"
    return cote * cote

assert aire_carre(3) == 9
assert aire_carre(2.5) == 6.25
assert aire_carre(0) == 0       # cas limite
```

▷ **Erreur fréquente.** ne tester que des valeurs « faciles » (8 et 30) : c'est au seuil `12` qu'une erreur `<` au lieu de `<=` (ou l'inverse) se cache.

### Exercice 45 *Corriger d'après le message d'erreur (1)* ↑énoncé p. 49

**▶ Solution. 1.** Une `IndentationError` (erreur d'indentation). Le message cite la ligne 4 (le `if`) et la ligne 5, où se trouve l'erreur : « un bloc décalé était attendu après l'instruction `if` de la ligne 4 ». La ligne `n = n + 1` n'est pas décalée sous le `if`.

**2.** On décale `n = n + 1` de 4 espaces de plus (8 en tout) ; le programme affiche alors `3`.

**3.** Non : comme une erreur de syntaxe, une erreur d'indentation est détectée en *lisant* le fichier, avant toute exécution (il n'y a d'ailleurs pas de ligne `Traceback`). Aucune ligne n'a été exécutée, même pas le `print` final.

▷ **Erreur fréquente.** décaler la ligne d'un nombre d'espaces quelconque : chaque niveau de bloc ajoute exactement 4 espaces, et toutes les lignes d'un même bloc doivent être alignées.

### Exercice 46 *Corriger d'après le message d'erreur (2)* ↑énoncé p. 49

**▶ Solution. 1.** Une `NameError` : le nom `totale` n'existe pas. Elle se produit à la ligne 3, dans la fonction `moyenne` (dernier bloc avant la dernière ligne).

**2.** La ligne 5 est l'*appel* de la fonction : le message remonte le chemin suivi, de l'appel (ligne 5) jusqu'à la ligne fautive, à l'intérieur de la fonction (ligne 3).

**3.** La variable s'appelle `total` (ligne 2) : on écrit `return total / 3`. Le programme affiche alors `12.0`.

▷ **Erreur fréquente.** corriger la ligne 5, la première citée : la ligne fautive est celle du *dernier* bloc, juste au-dessus du type d'erreur.

# CORRIGÉS DES TP



## TP Le nombre mystère, édition de luxe

↑énoncé p. 53

**Ancrage BO (1re)** constructions élémentaires (séquence, affectation, conditionnelles, boucles bornées et non bornées, appels de fonction) ; spécification (docstring, précondition par `assert`) ; jeux de tests ; terminaison par un variant.

**Code** tout le code ci-dessous forme **un seul fichier** `nombre_mystere.py`.

**Contrainte** aucune liste ni aucun tuple (non vus à ce stade) : les affichages multiples sont des `print` successifs, et chaque fonction renvoie **une seule** valeur (d'où `borne_max` et `essais_autorises` séparées).

### 1 Prise en main : comparer et contrôler

**Exercice 1** IA IA IA *La réponse de l'ordinateur*

▶ Corrigé.

```
import random

def comparer(proposition, secret):
    """Renvoie "trop petit", "trop grand" ou "gagne"."""
    if proposition < secret:
        return "trop petit"
    elif proposition > secret:
        return "trop grand"
    else:
        return "gagne"
```

Une réponse *renvoyée* est **réutilisable** (`partie`, puis `essais_ordinateur` s'en servent) et **testable** par `assert` ; une réponse affichée est perdue pour le programme.

**Exercice 2** IA IA IA *Une proposition dans les clous*

▶ Corrigé.

```
def est_dans_intervalle(n, mini, maxi):
    """Renvoie True si mini <= n <= maxi (bornes incluses), False sinon."""
    return n >= mini and n <= maxi

assert est_dans_intervalle(57, 1, 100) == True
assert est_dans_intervalle(1, 1, 100) == True      # borne basse
assert est_dans_intervalle(100, 1, 100) == True    # borne haute
assert est_dans_intervalle(0, 1, 100) == False     # juste en dessous
assert est_dans_intervalle(101, 1, 100) == False   # juste au-dessus
```

Les cas limites indispensables sont les **deux bornes** (`1`, `100`) et leurs **voisins extérieurs** (`0`, `101`) : ce sont eux qui distinguent `>=` de `>`. On accepte aussi la version avec `if ...: return True else: return False`.

## 2 Le cœur du jeu : la boucle de partie

### Exercice 3 *Une première partie*

▶ Corrigé.

```
secret = random.randint(1, 100)
essais = 0
reponse = ""
while reponse != "gagne":
    proposition = int(input("Votre proposition : "))
    essais = essais + 1
    reponse = comparer(proposition, secret)
    if reponse != "gagne":
        print(reponse)
print("Bravo, trouve en", essais, "essai(s) !")
```

**1.** On ne sait pas *à l'avance* combien d'essais seront nécessaires : c'est le cas typique du `while`.
**2.** `input` renvoie toujours une **chaîne** (`"50"`) ; il faut `int(input(...))`, sinon la comparaison avec un entier provoque une `TypeError`.
**3.** Non : aucune quantité ne décroît forcément, un joueur peut proposer `1` indéfiniment. La boucle ne s'arrête que si le joueur finit par trouver. *Initialiser* `reponse = ""` permet d'entrer une première fois dans la boucle.

### Exercice 4 *Un nombre d'essais limité*

▶ Corrigé.

```
def partie(secret, maxi, essais_max):
    """Joue une partie. Renvoie le nombre d'essais utilises
    si le joueur gagne, 0 s'il perd."""
    essais = 0
    trouve = False
    while not trouve and essais < essais_max:
        proposition = int(input("Votre proposition : "))
        if not est_dans_intervalle(proposition, 1, maxi):
            print("Hors limites ! Entre 1 et", maxi, "(essai non compte).")
        else:
            essais = essais + 1
            reponse = comparer(proposition, secret)
            if reponse == "gagne":
                trouve = True
                print("Bravo, trouve en", essais, "essai(s) !")
            else:
                print(reponse, "-", indice(proposition, secret),
                      "- il reste", essais_max - essais, "essai(s)")
    if trouve:
        return essais
    print("Perdu ! Le nombre mystere etait", secret)
    return 0
```

(Version finale, avec l'indice de l'exercice 5 ; à ce stade, la ligne `print(reponse)` suffit.)

**3.** Variant : `essais_max - essais`, entier positif ou nul, qui diminue de 1 à chaque proposition comptée ; la boucle s'arrête au plus tard quand il atteint 0. Si le joueur tape `0` sans arrêt, `essais` ne bouge plus : il n'y a plus de variant et la boucle peut tourner sans fin. Ce n'est pas grave ici (c'est le joueur qui le décide, et `Ctrl + C` l'interrompt), mais il faut savoir le dire : la terminaison n'est garantie *que* si les saisies sont valides.
**4.** `partie` dépend de ce que tape l'utilisateur (`input`) et communique par `print` : un `assert` ne peut ni répondre à sa place ni lire l'écran. D'où la règle : **séparer le calcul** (petites fonctions qui renvoient une valeur, testables) **de l'interaction** (fonctions courtes qui ne font qu'enchaîner `input`, `print` et les appels).

## 3 Indices et score

**Exercice 5** *Chaud ou froid ?*

▶ **Corrigé.**

```
def ecart(a, b):
    """Renvoie la distance entre a et b (positive ou nulle)."""
    if a >= b:
        return a - b
    else:
        return b - a

def indice(proposition, secret):
    """Renvoie "brulant" (ecart <= 3), "chaud" (ecart <= 10) ou "froid"."""
    d = ecart(proposition, secret)
    if d <= 3:
        return "brulant"
    elif d <= 10:
        return "chaud"
    else:
        return "froid"
```

Les six tests donnés ne vérifient la frontière qu'*au-dessus* pour `"brulant"` et *en dessous* pour `"chaud"`/`"froid"`. Tests de frontière à ajouter, des deux côtés du secret :

```
assert indice(39, 42) == "brulant"   # ecart 3, sous le secret
assert indice(38, 42) == "chaud"     # ecart 4
assert indice(52, 42) == "chaud"     # ecart 10, au-dessus
assert indice(53, 42) == "froid"     # ecart 11
```

Erreur fréquente détectée ainsi : `d = proposition - secret` (sans `ecart`), négatif sous le secret, qui donne toujours `"brulant"`.

**Exercice 6** *Compter les points*

▶ **Corrigé.** **1.** 2 essais sur 7 : $10 + 10 \times 5 = 60$ points ; 7 essais sur 7 : $10 + 10 \times 0 = 10$ points.

```
def points(essais, essais_max):
    """Renvoie les points d'une partie.
    Precondition : 0 <= essais <= essais_max (0 signifie : perdu)."""
    assert essais >= 0 and essais <= essais_max, "nombre d'essais impossible"
    if essais == 0:
        return 0
    elif essais == 1:
        return 100
    else:
        return 10 + 10 * (essais_max - essais)

assert points(0, 7) == 0       # perdu
assert points(1, 7) == 100     # du premier coup
assert points(2, 7) == 60
assert points(7, 7) == 10      # gagne au dernier essai
```

L'ordre des tests compte : si l'on teste `essais == 1` *après* la formule générale, le cas « premier coup » n'est jamais atteint.

## 4 Niveaux et menu

**Exercice 7** *Trois niveaux de difficulté*

▶ Corrigé.

```python
def afficher_menu():
    """Affiche les niveaux, un par ligne."""
    print("1 : facile    (entre 1 et 10,   4 essais)")
    print("2 : moyen     (entre 1 et 100,  7 essais)")
    print("3 : difficile (entre 1 et 1000, 10 essais)")
    print("4 : l'ordinateur devine (entre 1 et 100)")   # ajout de l'exercice 12

def borne_max(niveau):
    """Renvoie la plus grande valeur possible du secret.
    Precondition : niveau vaut 1, 2 ou 3."""
    assert niveau >= 1 and niveau <= 3, "niveau inconnu"
    if niveau == 1:
        return 10
    elif niveau == 2:
        return 100
    else:
        return 1000

def essais_autorises(niveau):
    """Renvoie le nombre d'essais du niveau.
    Precondition : niveau vaut 1, 2 ou 3."""
    assert niveau >= 1 and niveau <= 3, "niveau inconnu"
    if niveau == 1:
        return 4
    elif niveau == 2:
        return 7
    else:
        return 10
```

**3.** `borne_max(4)` s'arrête net sur `AssertionError: niveau inconnu`. Sans l'`assert`, le `else` renverrait `1000` : un résultat faux, silencieux, découvert bien plus tard (ou jamais). Mieux vaut échouer tôt et clairement.

**Exercice 8** *Un menu à l'épreuve des fautes de frappe*

▶ Corrigé.

```python
def demander_niveau():
    """Affiche le menu et redemande jusqu'a un choix valide (entier renvoye)."""
    afficher_menu()
    choix = input("Votre niveau (1, 2, 3 ou 4) : ")
    while choix != "1" and choix != "2" and choix != "3" and choix != "4":
        print("Choix impossible.")
        choix = input("Votre niveau (1, 2, 3 ou 4) : ")
    return int(choix)
```

(Sans l'exercice 12, la condition ne porte que sur `"1"`, `"2"`, `"3"`.) `int("deux")` provoque une `ValueError` qui arrête le programme : on vérifie donc la chaîne *avant* de la convertir. Piège classique : écrire `or` au lieu de `and` dans la condition (elle est alors toujours vraie : boucle infinie).

## 5 Le jeu complet

**Exercice 9** *Rejouer et faire le bilan*

▶ Corrigé.

```
def jouer():
    """Enchaine les parties et affiche le bilan."""
    nb_parties = 0
    nb_gagnees = 0
    total = 0
    encore = "o"
    while encore == "o":
        niveau = demander_niveau()
        if niveau == 4:                    # ajout de l'exercice 12
            ordinateur_devine(100)
        else:
            maxi = borne_max(niveau)
            essais_max = essais_autorises(niveau)
            secret = random.randint(1, maxi)
            essais = partie(secret, maxi, essais_max)
            nb_parties = nb_parties + 1
            if essais > 0:
                nb_gagnees = nb_gagnees + 1
            gain = points(essais, essais_max)
            total = total + gain
            print("Points gagnes :", gain, "- total :", total)
        encore = input("Rejouer ? (o/n) ")
    print("Bilan :", nb_parties, "partie(s),", nb_gagnees, "gagnee(s),",
          total, "points.")

# ... les tests assert ...
jouer()
```

Exemple de partie (secret `42` puis `3` ; le menu, affiché avant chaque choix de niveau, n'est pas reproduit) :

```
Votre niveau (1, 2, 3 ou 4) : 5
Choix impossible.
Votre niveau (1, 2, 3 ou 4) : 2
Votre proposition : 50
trop grand - chaud - il reste 6 essai(s)
Votre proposition : 0
Hors limites ! Entre 1 et 100 (essai non compte).
Votre proposition : 25
trop petit - froid - il reste 5 essai(s)
Votre proposition : 37
trop petit - chaud - il reste 4 essai(s)
Votre proposition : 40
trop petit - brulant - il reste 3 essai(s)
Votre proposition : 42
Bravo, trouve en 5 essai(s) !
Points gagnes : 30 - total : 30
Rejouer ? (o/n) o
   (niveau 1 : 5, 8, 9, 10 -> Perdu ! Le nombre mystere etait 3)
Points gagnes : 0 - total : 30
Rejouer ? (o/n) n
Bilan : 2 partie(s), 1 gagnee(s), 30 points.
```

**1.** Séquence : le corps de `jouer` ; affectations : `total = total + gain` ; conditionnelles : `comparer`, `indice`, `points` ; boucles non bornées : `partie`, `demander_niveau`, `jouer` ; appels de fonction : partout dans `jouer`. La boucle bornée sert dans `pire_cas` (parcourir *tous* les secrets).
**2.** Compteurs : `nb_parties`, `nb_gagnees` ; accumulateur : `total`. Tous initialisés à `0` **avant** la

boucle : placés dedans, ils seraient remis à zéro à chaque partie (erreur la plus fréquente de l'exercice).

## 6 Défi : l'ordinateur devine

**Exercice 10** *Jouer la stratégie à la main*

▶ Corrigé.

| Essai | mini | maxi | proposition | réponse |
|---|---|---|---|---|
| 1 | 1 | 100 | 50 | trop grand |
| 2 | 1 | 49 | 25 | trop petit |
| 3 | 26 | 49 | 37 | gagné |

**Exercice 11** *Le pire des cas*

▶ Corrigé.

```python
def essais_ordinateur(secret, maxi):
    """Simule la strategie du milieu ; renvoie le nombre d'essais.
    Precondition : 1 <= secret <= maxi."""
    assert secret >= 1 and secret <= maxi
    mini = 1
    essais = 0
    trouve = False
    while not trouve:
        proposition = (mini + maxi) // 2
        essais = essais + 1
        reponse = comparer(proposition, secret)
        if reponse == "trop petit":
            mini = proposition + 1
        elif reponse == "trop grand":
            maxi = proposition - 1
        else:
            trouve = True
    return essais

def pire_cas(maxi):
    """Renvoie le plus grand nombre d'essais sur tous les secrets de 1 a maxi."""
    pire = 0
    for secret in range(1, maxi + 1):
        e = essais_ordinateur(secret, maxi)
        if e > pire:
            pire = e
    return pire

def nb_moities(n):
    """Nombre de divisions entieres par 2 pour passer de n a 0."""
    k = 0
    while n > 0:
        n = n // 2
        k = k + 1
    return k

assert pire_cas(10) == 4 and pire_cas(100) == 7 and pire_cas(1000) == 10
for m in range(1, 201):
    assert pire_cas(m) == nb_moities(m)
```

**2.** `pire_cas(10)`, `pire_cas(100)`, `pire_cas(1000)` valent **4, 7 et 10** : exactement les essais autorisés. Chaque niveau est donc « juste » : un joueur qui applique la bonne stratégie gagne *toujours*, de justesse dans le pire des cas. (Entre 1 et 100, la moyenne est de 5,8 essais ; 37 secrets

sur 100 demandent les 7 essais.)

**3.** Les 200 `assert` passent. `nb_moities(1000000)` vaut **20** : 20 essais suffisent pour un nombre entre 1 et un million.

*Terminaison de* `essais_ordinateur` : variant `maxi - mini + 1` (le nombre de candidats), qui diminue strictement à chaque essai raté ; comme le secret reste toujours entre `mini` et `maxi`, on finit par le proposer.

**Exercice 12** *À vous de penser, à lui de deviner*

▶ **Corrigé.**

```
def ordinateur_devine(maxi):
    """Le joueur pense a un nombre entre 1 et maxi ; l'ordinateur le devine."""
    mini = 1
    essais = 0
    fini = False
    while not fini:
        if mini > maxi:
            print("Tricheur ! Aucun nombre ne convient.")
            fini = True
        else:
            proposition = (mini + maxi) // 2
            essais = essais + 1
            reponse = input("Je propose " + str(proposition) + " (+, - ou =) : ")
            if reponse == "+":
                mini = proposition + 1
            elif reponse == "-":
                maxi = proposition - 1
            elif reponse == "=":
                print("Trouve en", essais, "essai(s) !")
                fini = True
            else:
                print("Repondre par +, - ou =.")
                essais = essais - 1
```

Avec la séquence `+ - + =`, on obtient l'affichage de l'énoncé ; répondre `+` à chaque fois affiche `Tricheur !` juste après la 7e proposition (100). Pour l'ajouter au jeu, on modifie `afficher_menu`, `demander_niveau` et `jouer` (voir plus haut) ; `comparer`, `indice`, `points`, `partie`, `borne_max` et `essais_autorises` restent **intactes** — c'est tout l'intérêt du découpage en fonctions. Attention : `borne_max(4)` déclencherait l'`assert`, d'où le `if niveau == 4` placé *avant* son appel.

## Bilan attendu

- Testées par `assert` : `comparer`, `est_dans_intervalle`, `ecart`, `indice`, `points`, `borne_max`, `essais_autorises`, `essais_ordinateur`, `pire_cas`, `nb_moities`. Non testables ainsi : `partie`, `afficher_menu`, `demander_niveau`, `ordinateur_devine`, `jouer` (elles lisent le clavier ou affichent) : on les essaie à la main.
- Test révélateur d'une erreur : réponse personnelle (exemples typiques : `assert points(1, 7) == 100` qui échoue quand le cas « premier coup » est placé après la formule générale ; `assert indice(39, 42) == "brulant"` qui révèle un écart négatif).
- Pourquoi 7 essais suffisent : chaque essai raté élimine la proposition et **au moins la moitié** des candidats restants ; de 100 candidats on passe à au plus 50, 25, 12, 6, 3, 1 : après 6 essais ratés il reste au plus un nombre, que le 7e essai trouve.

## Réponses des questions de la fiche de révision

↑ questions p. 60

### Se tester

1. `a` vaut `8`, `b` vaut `4` (§ II.B)
2. `2, 4, 6` (la borne `8` est exclue) (§ IV.A)
3. `3`, `2` et `3.4` (un flottant) (§ II.C)
4. `None` : la fonction affiche mais ne renvoie rien (§ VI.A)
5. `while` (nombre de tours inconnu) ; un variant : un entier positif qui décroît strictement (§ V.B)

# Complément — Dessiner avec la tortue

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 Le robot dessinateur ↑énoncé p. 67

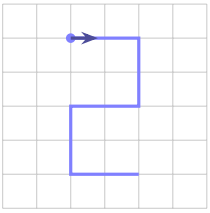
**Exercice 1** *Exécuter un programme*

▶ Solution.

**1.** Le robot trace le **chiffre 2**.

**2.** 9 ordres, dont 5 **AVANCER** qui tracent ; les 4 virages ne tracent rien.

**3.** Chaque virage change de sens : la figure devient son symétrique par rapport à la première ligne horizontale, un **5**, qui monte au lieu de descendre et sort du quadrillage par le haut. Une seule inversion gauche/droite suffit à tout changer.



**Exercice 2** *Le programmeur et le robot*

▶ Solution.

**4. Carte A** (7 ordres) : **AVANCER 2, DROITE, AVANCER 4, DROITE, AVANCER 2, DROITE, AVANCER 4.**

**5. Carte B** (11 ordres) : **AVANCER 1, DROITE, AVANCER 1, GAUCHE, AVANCER 1, DROITE, AVANCER 1, GAUCHE, AVANCER 1, DROITE, AVANCER 1.**

D'autres programmes sont justes (par exemple en tournant à gauche au départ pour la carte A, ou en ajoutant un dernier virage pour revenir tourné vers le haut) : la figure seule fait foi.

**6.** Réponses variables. Les erreurs viennent le plus souvent du **programme** (un virage oublié, gauche et droite confondues, une longueur fausse) ; le robot, lui, n'a qu'à obéir — c'est tout l'intérêt de la règle « sans rien deviner ».

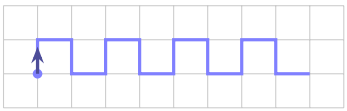
**7.** Le même groupe de quatre ordres **AVANCER 1, DROITE, AVANCER 1, GAUCHE** revient trois fois : on a envie de dire « trois fois la même chose ».

**Exercice 3** *Répéter*

▶ Solution. **8.** Sans répétition (8 ordres) : **AVANCER 3, GAUCHE, AVANCER 3, GAUCHE, AVANCER 3, GAUCHE, AVANCER 3, GAUCHE** (le dernier virage est facultatif : il remet le robot dans sa direction de départ). Avec : **RÉPÉTER 4 FOIS : AVANCER 3, GAUCHE.**

**9. RÉPÉTER 3 FOIS : AVANCER 1, DROITE, AVANCER 1, GAUCHE** — le dernier **GAUCHE**, en trop par rapport au programme de la question 5, ne trace rien.

**10.** Une frise de **4 créneaux** ; sans **RÉPÉTER**, il faudrait $4 \times 8 = 32$ ordres, contre une ligne de répétition et 8 ordres.



**Exercice 4** *Faire le tour*

▶ **Solution.** **11.** Quatre quarts de tour, soit $4 \times 90 = \mathbf{360}$ degrés : un tour complet. C'est vrai pour *toute* figure fermée parcourue en revenant dans la direction de départ (rectangle de la carte A compris, avec son dernier virage).

**12.** Trois virages égaux pour un tour complet : $360/3 = \mathbf{120}$ degrés (et non 60, l'angle *intérieur* du triangle : en tournant de 60 degrés, le robot trace… un hexagone). Hexagone : $360/6 = \mathbf{60}$ degrés.

**Exercice 5** *Ce qu'on a découvert*

▶ **Solution.** **13.** « Un programme est une suite d'**instructions** exécutées **dans l'ordre**, une par une : c'est une **séquence**. Le robot ne devine rien : une seule erreur dans le programme et le dessin est **faux**. Pour ne pas recopier plusieurs fois les mêmes ordres, on utilise une **boucle** (une répétition). Pour faire le tour d'une figure à `n` coins égaux, on tourne à chaque coin de `360 / n` degrés. »

# CORRIGÉS DES EXERCICES

> **Remarque**
> Plusieurs écritures sont possibles ; on en donne une simple. Chaque programme suppose `import turtle as t` au début et `t.done()` à la fin. On rappelle la règle d'or : un polygone régulier à `n` côtés se ferme en tournant de `360 / n` à chaque sommet.

## Partie 1 — Séquence d'instructions et boucle `for`

### Premiers tracés

**Exercice 1** *Le carré* ↑énoncé p. 74

▶ **Solution.** **En séquence** : 8 ordres (4 fois « avancer, tourner »).

```
1  import turtle as t
2  t.forward(100)
3  t.left(90)
4  t.forward(100)
5  t.left(90)
6  t.forward(100)
7  t.left(90)
8  t.forward(100)
9  t.left(90)
10 t.done()
```

**Avec une boucle** : on répète 4 fois le couple « avancer, tourner ».

```
1  import turtle as t
2  for i in range(4):
3      t.forward(100)
4      t.left(90)
5  t.done()
```

▷ **Erreur fréquente.** ne pas décaler `t.left(90)` : il sort de la boucle, la tortue avance quatre fois tout droit (un seul trait de 400) et ne tourne qu'à la fin.

**Exercice 2** *Le rectangle* ↑énoncé p. 74

**▶ Solution.** Deux longueurs et deux largeurs : on répète **deux fois** le couple « long côté / court côté ».

```
import turtle as t
for i in range(2):
    t.forward(200)   # longueur
    t.left(90)
    t.forward(100)   # largeur
    t.left(90)
t.done()
```

**Exercice 3** *L'escalier* ↑énoncé p. 74

**▶ Solution.** Chaque marche = un pas horizontal, un pas vertical. Après être monté, on se **remet** à l'horizontale (`right(90)`).

```
import turtle as t
for i in range(5):
    t.forward(40)   # horizontal
    t.left(90)
    t.forward(40)   # vertical
    t.right(90)     # on repart a l'horizontale
t.done()
```

▷ **Erreur fréquente.** tourner deux fois à gauche : la tortue revient vers son point de départ et repasse cinq fois sur le même petit carré.

## Les polygones réguliers

**Exercice 4** *Triangle et hexagone* ↑énoncé p. 74

**▶ Solution.** Pour refermer la figure, la tortue tourne de 360 degrés en tout, partagés entre les sommets. Triangle : $360/3 = \mathbf{120}$ degrés. Hexagone : $360/6 = \mathbf{60}$ degrés.

```
import turtle as t
for i in range(3):      # triangle equilateral
    t.forward(120)
    t.left(120)
for i in range(6):      # hexagone
    t.forward(80)
    t.left(60)
t.done()
```

▷ **Erreur fréquente.** tourner de `60` pour le triangle : c'est l'angle *intérieur* ; la tortue, elle, tourne de l'angle extérieur, `120`. Avec `60`, on obtient la moitié d'un hexagone.

**Exercice 5** *L'étoile à cinq branches* ↑énoncé p. 75

**▶ Solution.** En traçant l'étoile, la tortue fait **deux** tours complets, soit 720 degrés, répartis sur 5 pointes : $720/5 = \mathbf{144}$ degrés. En tournant ainsi, on « saute » un sommet sur deux, ce qui trace l'étoile.

```
import turtle as t
for i in range(5):
    t.forward(200)
    t.left(144)
t.done()
```

## Répéter en tournant

**Exercice 6** *Rosace de cercles* ↑énoncé p. 75

▶ **Solution.** 18 cercles répartis sur un tour complet : on tourne de 360/18 = **20** degrés entre chaque.

```
import turtle as t
for k in range(18):
    t.circle(80)
    t.left(20)
t.done()
```

**Exercice 7** *La spirale carrée* ↑énoncé p. 75

▶ **Solution.** On garde l'angle à 90 degrés, mais on **augmente** la longueur à chaque nouveau segment (à chaque passage dans la boucle).

```
import turtle as t
cote = 10              # INITIALISER : avant la boucle
for i in range(20):    # PARCOURIR : 20 segments
    t.forward(cote)
    t.left(90)
    cote = cote + 10   # METTRE A JOUR : le cote grandit
t.done()
```

▷ **Erreur fréquente.** écrire `cote = 10` *dans* la boucle : la longueur est remise à 10 à chaque tour, et la tortue repasse sur un petit carré.

*Autre méthode :* au tour numéro `i`, le côté mesure `10 * i` ; on peut donc faire varier `i` de 1 à 20 sans variable `cote` : `for i in range(1, 21):` puis `t.forward(10 * i)` et `t.left(90)`.

## Défis

**Exercice 8** *La maison* ↑énoncé p. 75

▶ **Solution.** On trace d'abord la façade (un carré). La tortue revient alors en bas à gauche ; on monte au coin haut-gauche, puis on pose un toit triangulaire (triangle équilatéral, angles de 120 degrés).

```
import turtle as t
# La facade (carre de cote 100)
for i in range(4):
    t.forward(100)
    t.left(90)
# On monte au coin haut-gauche pour poser le toit
t.left(90)
t.forward(100)
t.right(30)          # on vise le sommet du toit
# Le toit (triangle equilateral)
for i in range(3):
    t.forward(100)
    t.right(120)
t.done()
```

**Exercice 9** *Un triangle selon un assistant d'IA* ↑énoncé p. 76

▶ **Solution.** **1.** Non. En trois tours, la tortue tourne de $3 \times 60 = 180°$ seulement, alors que, pour fermer un polygone, elle doit avoir tourné de 360° en tout. À l'exécution, on obtient trois côtés d'un hexagone (une figure ouverte) : la tortue finit à environ (100 ; 173) du point de départ, cap à 180°, et rien n'est fermé. **2.** L'erreur est l'angle `left(60)`. L'angle *intérieur* du triangle vaut bien 60°, mais la tortue tourne de l'angle **extérieur**, $360/3 = 120°$ (règle du cours : `360 / n` à chaque sommet).

```
import turtle as t
for i in range(3):
    t.forward(100)
    t.left(120)
t.done()
```

**3.** Faire le calcul « nombre de côtés × angle = 360 » avant de lancer : $3 \times 60 \neq 360$. Ou, plus simplement, exécuter le programme et regarder le dessin : ici, l'erreur se voit à l'écran en une seconde.

## Partie 2 — Avec des fonctions

### Factoriser : les fonctions

**Exercice 10** *Un polygone au choix* ↑énoncé p. 76

**▶ Solution.** Le côté vaut `perimetre / n` et l'angle `360 / n` : la même fonction dessine tous les polygones réguliers.

```
import turtle as t
def polygone_perimetre(n, perimetre):
    cote = perimetre / n
    for i in range(n):
        t.forward(cote)
        t.left(360 / n)

polygone_perimetre(3, 360)   # triangle : cotes de 120
polygone_perimetre(4, 360)   # carre : cotes de 90
polygone_perimetre(6, 360)   # hexagone : cotes de 60
t.done()
```

Avec le même périmètre 360, le triangle a des côtés de **120**, le carré de **90** et l'hexagone de **60**.

▷ **Erreur fréquente.** laisser `t.left(90)`, recopié du carré : seul l'appel `polygone_perimetre(4, ...)` se referme. L'angle doit dépendre du paramètre : `360 / n`.

**Exercice 11** *Rosace de triangles* ↑énoncé p. 76

**▶ Solution.** On écrit une fonction `triangle` (trois fois : avancer, tourner de `120`), puis on la répète 8 fois en pivotant de `45` degrés, car $8 \times 45 = 360$.

```
import turtle as t
def triangle(cote):
    for i in range(3):
        t.forward(cote)
        t.left(120)

for k in range(8):
    triangle(100)
    t.left(45)
t.done()
```

## Couleurs et déplacements

**Exercice 12** *Trois carrés colorés* ↑énoncé p. 76

**▶ Solution.** On fabrique une fonction « carré plein » (couleur + `begin_fill`/`end_fill`), et on déplace la tortue *sans tracer* entre deux carrés grâce à `penup`/`goto`/`pendown`.

```
import turtle as t
def carre_plein(cote, couleur):
    t.color(couleur)
    t.begin_fill()
    for i in range(4):
        t.forward(cote)
        t.left(90)
    t.end_fill()

t.penup()
t.goto(-150, 0)
t.pendown()
carre_plein(80, "red")
t.penup()
t.goto(-50, 0)
t.pendown()
carre_plein(80, "green")
t.penup()
t.goto(50, 0)
t.pendown()
carre_plein(80, "blue")
t.done()
```

▷ **Erreur fréquente.** oublier `t.penup()` avant `t.goto(...)` : la tortue trace un trait entre deux carrés.

## Défi

**Exercice 13** *Le damier* ↑énoncé p. 77

**▶ Solution.** Deux boucles imbriquées balayent lignes et colonnes, et on se place sur chaque case avec `goto`. Chaque case est un carré ; on ne **remplit** qu'une case sur deux, selon la parité de `ligne + colonne`.

```
import turtle as t
def carre(cote):
    for i in range(4):
        t.forward(cote)
        t.left(90)

cote = 40
t.color("black")
for ligne in range(3):
    for colonne in range(3):
        t.penup()
        t.goto(colonne * cote, ligne * cote)
        t.pendown()
        if (ligne + colonne) % 2 == 0:   # case noire : carre rempli
            t.begin_fill()
            carre(cote)
            t.end_fill()
        else:                            # case blanche : contour seul
            carre(cote)
t.done()
```

**Exercice 14** *La cible* ↑énoncé p. 77

▶ **Solution.** **1.** Tournée vers la droite, la tortue a sa gauche vers le haut : le centre du cercle est `r` pas **au-dessus** d'elle. Pour un centre en `(0, 0)`, elle doit partir de `(0, -100)` pour `t.circle(100)`, et de `(0, -r)` pour `t.circle(r)`.

**2. et 3.** Un cercle complet ramène la tortue à son point et à son orientation de départ : elle reste tournée vers la droite pour le disque suivant. On trace du plus grand au plus petit, car chaque disque rempli **recouvre** ce qui était dessiné dessous : en commençant par le plus petit, le disque de rayon `100`, tracé en dernier, cacherait tous les autres.

```
import turtle as t
def disque(r, couleur):
    """Trace un disque de rayon r, de centre (0, 0), rempli de couleur."""
    t.penup()
    t.goto(0, -r)
    t.pendown()
    t.color(couleur)
    t.begin_fill()
    t.circle(r)
    t.end_fill()

for i in range(5):
    if i % 2 == 0:
        disque(100 - 20 * i, "red")
    else:
        disque(100 - 20 * i, "white")
t.done()
```

## Entraînement

**Exercice 15** *Prédire la figure* ↑énoncé p. 77

▶ **Solution.** a) Un **pentagone** régulier de côté 80, au-dessus du point de départ : $5 \times 72 = 360$, la figure se referme. b) Un **carré** de côté 100, tracé *sous* le point de départ : avec `right`, la tortue tourne dans le sens des aiguilles d'une montre. c) Un **rectangle** de 100 sur 50 (deux paires « grand côté, petit côté »). d) Une figure **ouverte** : trois côtés d'un carré ; la tortue n'a tourné que de $3 \times 90 = 270$ degrés, le côté gauche manque.

▷ **Erreur fréquente.** confondre `left` et `right` : la figure b) est la même que le carré de l'exercice 1, mais retournée vers le bas.

**Exercice 16** *Trouver l'angle* ↑énoncé p. 78

▶ **Solution.** L'angle vaut `360 / n` : a) $360/5 = 72$ ; b) $360/8 = 45$ ; c) $360/9 = 40$ ; d) $360/10 = 36$. Par exemple, pour le pentagone :

```
for i in range(5):
    t.forward(60)
    t.left(72)
```

Pour les autres, on remplace `5` et `72` par `8` et `45`, `9` et `40`, `10` et `36`.

CORRIGÉS DU CHAPITRE 2

# 2 Le binaire et l'écriture des nombres

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 Les cartes à points ↑énoncé p. 91

**Exercice 1** *Observer les cartes*

▶ **Solution.** **1.** Chaque carte porte **le double** de la carte à sa droite.

**2.** $2 \times 16 = 32$ points.

**Exercice 2** *Représenter des nombres*

▶ **Solution.** **3.** $5 = 4 + 1 =$ `00101` ; $9 = 8 + 1 =$ `01001` ; $21 = 16 + 4 + 1 =$ `10101` ; $30 = 16 + 8 + 4 + 2 =$ `11110`.

**4.** `10010` $= 16 + 2 = 18$ ; `00111` $= 4 + 2 + 1 = 7$ ; `11111` $= 16 + 8 + 4 + 2 + 1 = 31$.

**5.** Le plus petit : 0 (toutes les cartes cachées) ; le plus grand : 31 (toutes montrées).

**6.** Oui, tous les nombres de 0 à 31 s'obtiennent, et d'**une seule** façon : $6 =$ `00110`, $13 =$ `01101`, $27 =$ `11011`. Méthode qui émerge souvent : on part de la plus grosse carte, on la montre si elle « ne dépasse pas », et on continue avec ce qui reste.

**7.** Chaque carte a 2 états : $2 \times 2 \times 2 \times 2 \times 2 = 32$ configurations. C'est exactement le nombre d'entiers de 0 à 31 : chaque configuration correspond à un nombre et un seul.

**Exercice 3** *Compter, ajouter une carte*

▶ **Solution.** **8.** $2 =$ `00010`, $3 =$ `00011`, $4 =$ `00100`, $5 =$ `00101`, $6 =$ `00110`, $7 =$ `00111`, $8 =$ `01000`. La carte « 1 point » se retourne **à chaque étape**. De 7 à 8, les trois cartes de droite se cachent toutes et la carte « 8 » apparaît : c'est une **retenue**, comme $999 + 1 = 1000$ en décimal.

**9.** Avec 6 cartes : $31 + 32 = 63$. Avec 8 cartes : $1 + 2 + \cdots + 128 = 255$.

**10.** $31 + 1 = 32$ : la carte suivante. Avec $n$ cartes, le plus grand nombre est $2^n - 1$.

**Exercice 4** *Des cartes aux interrupteurs*

▶ **Solution.** **11.** `01101` $= 8 + 4 + 1 = 13$.

**12.** Avec deux états, il suffit de savoir s'il y a du courant ou non : même si la tension varie un peu (bruit, usure, température), on ne se trompe pas. Dix niveaux proches seraient facilement confondus.

**13.** « Chaque carte ne peut valoir que **deux** états. Les points des cartes sont les puissances de **2**. Avec $n$ cartes, on représente $\mathbf{2^n}$ nombres différents, de 0 à $\mathbf{2^n - 1}$. Écrire un nombre avec des `0` et des `1`, c'est l'écrire en **binaire** (base 2). »

# CORRIGÉS DES EXERCICES



> **Remarque**
> Les conversions à la main sont détaillées ; on peut toujours les vérifier avec `bin`, `hex` et `int`.

## Bases et conversions

**Exercice 1** *Compter les possibilités* ↑énoncé p. 108

**▶ Solution.** Chaque bit ajouté double le nombre de possibilités (2 sur 1 bit, 4 sur 2 bits, 8 sur 3 bits…). **1.** Sur `4` bits : $2^4 = 16$ valeurs ; sur `8` bits : $2^8 = 256$ ; sur `n` bits : $2^n$. **2.** Le plus grand entier sur un octet est $2^8 - 1 = 255$ (soit `11111111`).

▷ **Erreur fréquente.** répondre 256 : il y a bien 256 valeurs, mais elles vont de 0 à 255.

**Exercice 2** *Du binaire vers le décimal* ↑énoncé p. 108

**▶ Solution.** On additionne les poids des rangs à `1` :

- $\texttt{1011}_2 = 8 + 2 + 1 = 11$ ;
- $\texttt{11001}_2 = 16 + 8 + 1 = 25$ ;
- $\texttt{10011101}_2 = 128 + 16 + 8 + 4 + 1 = 157$.

▷ **Erreur fréquente.** attribuer les poids en partant de la gauche : c'est le bit de **droite** qui pèse 1. Le bit de gauche de `11001` pèse 16, pas 1.

**Exercice 3** *Du décimal vers le binaire* ↑énoncé p. 108

**▶ Solution.** Divisions successives par 2 jusqu'à un quotient nul, restes lus **de bas en haut** :

$$\begin{aligned} 13 &= 2\times 6 + \mathbf{1} \\ 6 &= 2\times 3 + \mathbf{0} \\ 3 &= 2\times 1 + \mathbf{1} \\ 1 &= 2\times 0 + \mathbf{1} \end{aligned} \qquad \begin{aligned} 42 &= 2\times 21 + \mathbf{0} \\ 21 &= 2\times 10 + \mathbf{1} \\ 10 &= 2\times 5 + \mathbf{0} \\ 5 &= 2\times 2 + \mathbf{1} \\ 2 &= 2\times 1 + \mathbf{0} \\ 1 &= 2\times 0 + \mathbf{1} \end{aligned} \qquad \begin{aligned} 200 &= 2\times 100 + \mathbf{0} \\ 100 &= 2\times 50 + \mathbf{0} \\ 50 &= 2\times 25 + \mathbf{0} \\ 25 &= 2\times 12 + \mathbf{1} \\ 12 &= 2\times 6 + \mathbf{0} \\ 6 &= 2\times 3 + \mathbf{0} \\ 3 &= 2\times 1 + \mathbf{1} \\ 1 &= 2\times 0 + \mathbf{1} \end{aligned}$$

D'où $13 = \texttt{1101}_2$, $42 = \texttt{101010}_2$ et $200 = \texttt{11001000}_2$. On peut contrôler en sens inverse : $32 + 8 + 2 = 42$ et $128 + 64 + 8 = 200$.

▷ **Erreur fréquente.** lire les restes de haut en bas : on obtient l'écriture à l'envers (13 donnerait `1011`, qui vaut 11). Le premier reste obtenu est le bit de poids faible, écrit à droite.

**Exercice 4** *Vérifier à la machine* ↑énoncé p. 108

**▶ Solution.** `bin` donne l'écriture binaire (précédée de `0b`) ; `int(..., 2)` lit une chaîne écrite en base 2 :

```
>>> bin(200)
'0b11001000'
>>> int("11001", 2)
25
```

## Programmer les conversions

**Exercice 5** *Décimal vers binaire* ↑énoncé p. 108

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert decimal_vers_binaire(13) == "1101"    # cas normal
assert decimal_vers_binaire(0) == "0"        # cas limite : zero
assert decimal_vers_binaire(8) == "1000"     # cas piege : des zeros a droite
```

C'est la méthode des divisions par 2, avec accumulation des restes en tête de chaîne.

```
def decimal_vers_binaire(n):
    if n == 0:
        return "0"                       # RENVOYER : cas particulier
    resultat = ""                        # INITIALISER
    while n > 0:                         # PARCOURIR : division apres division
        resultat = str(n % 2) + resultat # METTRE A JOUR : on place le reste devant
        n = n // 2                       # METTRE A JOUR
    return resultat                      # RENVOYER

assert decimal_vers_binaire(13) == "1101"
assert decimal_vers_binaire(200) == "11001000"
assert decimal_vers_binaire(0) == "0"
```

**Exercice 6** *Binaire vers décimal* ↑énoncé p. 108

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert binaire_vers_decimal("1011") == 11    # cas normal
assert binaire_vers_decimal("0") == 0        # cas limite : un seul bit nul
assert binaire_vers_decimal("0011") == 3     # cas piege : des zeros en tete
```

Le schéma d'Horner : à chaque chiffre, on double l'accumulateur et on ajoute le bit.

```
def binaire_vers_decimal(ch):
    n = 0                    # INITIALISER
    for c in ch:             # PARCOURIR : bits de gauche a droite
        n = n * 2 + int(c)   # METTRE A JOUR
    return n                 # RENVOYER

assert binaire_vers_decimal("1011") == 11
assert binaire_vers_decimal("10111001") == 185
```

## Logique booléenne et portes logiques

**Exercice 7** *Évaluer des opérateurs booléens* ↑énoncé p. 109

▶ **Solution.** **1.** Avec $a = 1$ et $b = 0$ : non $a = 0$ ; $a$ et $b = 0$ ; $a$ ou $b = 1$ ; $a$ xor $b = 1$ (les deux bits sont différents) ; non($a$ ou $b$) = non $1 = 0$.

**2.** Avec $a = 1$ et $b = 1$ : non $a = 0$ ; $a$ et $b = 1$ ; $a$ ou $b = 1$ ; $a$ xor $b = 0$ (les deux bits sont égaux : c'est la ligne où ou et xor diffèrent) ; non($a$ ou $b$) $= 0$.

**3.**

```
>>> not True
False
>>> True and False
False
>>> True or False
True
>>> True != False
True
>>> not (True or False)
False
```

**Exercice 8** *Dresser la table d'une expression* ↑énoncé p. 109

**▶ Solution.** **1.** Une ligne par combinaison de $a$ et $b$ (4 lignes), une colonne par étape :

| $a$ | $b$ | $a$ et $b$ | non($a$ et $b$) | non $a$ | non $b$ | (non $a$) ou (non $b$) |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 0 | 0 |

Les colonnes de non($a$ et $b$) et de (non $a$) ou (non $b$) sont identiques (1, 1, 1, 0) : les deux expressions ont la même table de vérité, elles sont **équivalentes**. « Il est faux que $a$ et $b$ soient tous deux vrais » revient à dire « l'un des deux au moins est faux ».

**2.** Trois variables : $2^3 = 8$ lignes.

| $a$ | $b$ | $c$ | $a$ et $b$ | ($a$ et $b$) ou $c$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |

L'expression vaut 1 pour **5** lignes sur 8 : les quatre où $c = 1$, plus la ligne $a = b = 1$, $c = 0$.

▷ **Erreur fréquente.** oublier des combinaisons : avec 3 variables, il faut $2^3 = 8$ lignes. Les énumérer en comptant en binaire (000, 001… 111) garantit de n'en oublier aucune.

**Exercice 9** *Le demi-additionneur* ↑énoncé p. 109

**▶ Solution.** **1.** $0 + 0 = 00_2$, $0 + 1 = 1 + 0 = 01_2$, $1 + 1 = 10_2$ :

| $a$ | $b$ | $r$ | $s$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

**2.** La colonne $r$ (0, 0, 0, 1) est la table du **et** : $r = a$ et $b$. La colonne $s$ (0, 1, 1, 0) est la table du **xor** : $s = a$ xor $b$. Le circuit relie les deux entrées $a$ et $b$ à la fois à une porte xor (sortie : la somme) et à une porte et (sortie : la retenue), comme sur le schéma du cours.

**3.** La colonne de droite ne reçoit jamais de retenue : on n'y ajoute que **deux** bits, un demi-additionneur suffit. Toute autre colonne peut recevoir la retenue de sa voisine de droite. Dans $0110 + 0011$, la deuxième colonne ($1 + 1 = 10_2$) produit une retenue : la troisième colonne doit additionner **trois** bits (1, 0 et la retenue 1). Il manque donc au demi-additionneur une entrée pour la **retenue venue de la droite** ; c'est le rôle de l'additionneur complet, construit avec deux demi-additionneurs (voir le TP « Programmer l'arithmétique binaire »). Résultat : $0110 + 0011 = 1001$, soit $6 + 3 = 9$.

## L'hexadécimal

**Exercice 10** *Binaire et hexadécimal* ↑énoncé p. 109

▶ **Solution.** On regroupe par 4 bits :

$$\texttt{11011010}_2 = \underbrace{\texttt{1101}}_{\texttt{D}}\ \underbrace{\texttt{1010}}_{\texttt{A}} = \texttt{DA}_{16}, \qquad \texttt{3F}_{16} = \underbrace{\texttt{0011}}_{\texttt{3}}\ \underbrace{\texttt{1111}}_{\texttt{F}} = \texttt{00111111}_2.$$

▷ **Erreur fréquente.** faire les paquets en partant de la gauche quand le nombre de bits n'est pas un multiple de 4 : `1111001` se découpe en `(0)111 1001`, soit `79`, et non `1111 001`.

**Exercice 11** *Décimal et hexadécimal* ↑énoncé p. 109

▶ **Solution.** Du décimal vers l'hexadécimal : divisions successives par 16, restes lus de bas en haut (un reste de 10 à 15 s'écrit `A` à `F`).

$$\begin{aligned} 255 &= 16\times 15 + \mathbf{15}\ (\texttt{F}) \\ 15 &= 16\times 0 + \mathbf{15}\ (\texttt{F}) \end{aligned} \qquad \begin{aligned} 3000 &= 16\times 187 + \mathbf{8} \\ 187 &= 16\times 11 + \mathbf{11}\ (\texttt{B}) \\ 11 &= 16\times 0 + \mathbf{11}\ (\texttt{B}) \end{aligned}$$

D'où $255 = \texttt{FF}_{16}$ et $3000 = \texttt{BB8}_{16}$ (contrôle : $11{\cdot}256 + 11{\cdot}16 + 8 = 2816 + 176 + 8 = 3000$).

De l'hexadécimal vers le décimal, on additionne les chiffres multipliés par leur poids : $\texttt{2A}_{16} = 2{\cdot}16 + 10 = 42$ ; $\texttt{FF}_{16} = 15{\cdot}16 + 15 = 255$.

▷ **Erreur fréquente.** recopier un reste à deux chiffres tel quel : 3000 n'est pas « `11118` ». Chaque reste, de 0 à 15, donne **un seul** chiffre hexadécimal : 11 s'écrit `B`.

**Exercice 12** *Une couleur du Web* ↑énoncé p. 110

▶ **Solution.** On découpe `FF8000` en trois octets, `FF`, `80` et `00` (rouge, vert, bleu), et l'on convertit chacun en décimal : $\texttt{80}_{16} = 8 \times 16 = 128$.

```
>>> int("FF", 16)
255
>>> int("80", 16)
128
>>> int("00", 16)
0
```

Rouge à fond (255), vert à moitié (128), pas de bleu : on obtient un **orange**.

**Exercice 13** *Changer de base* ↑énoncé p. 110

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
1 assert decimal_vers_base(255, 16) == "FF"    # cas normal
2 assert decimal_vers_base(0, 2) == "0"        # cas limite : zero
3 assert decimal_vers_base(10, 16) == "A"      # cas piege : un chiffre qui est une lettre
```

Même algorithme que pour le binaire, en divisant par `b` au lieu de 2. Le reste `n % b`, compris entre 0 et $b-1$, sert d'indice dans la chaîne des chiffres.

```
1  def decimal_vers_base(n, b):
2      chiffres = "0123456789ABCDEF"
3      if n == 0:
4          return "0"                          # RENVOYER : cas particulier
5      resultat = ""                           # INITIALISER
6      while n > 0:                            # PARCOURIR
7          resultat = chiffres[n % b] + resultat  # METTRE A JOUR
8          n = n // b                          # METTRE A JOUR
9      return resultat                         # RENVOYER
10
11 assert decimal_vers_base(255, 16) == "FF"
12 assert decimal_vers_base(13, 2) == "1101"
13 assert decimal_vers_base(100, 8) == "144"
14 assert decimal_vers_base(0, 5) == "0"
```

Vérification : `hex(255)` affiche `'0xff'`, `bin(13)` affiche `'0b1101'` et `int("144", 8)` vaut `100`. La boucle se termine car `n` décroît strictement (`n // b < n` dès que $b \geqslant 2$).

## Les entiers relatifs (complément à deux)

**Exercice 14** *Coder et décoder* ↑énoncé p. 110

▶ **Solution.** **1.** Coder $-20$ sur 8 bits : $20 =$ `00010100` ; on inverse : `11101011` ; on ajoute 1 : `11101100`. Donc $-20 =$ `11101100`.

**2.** `11111011` commence par `1` : c'est un **négatif**. Pour trouver sa valeur, on inverse (`00000100`) et on ajoute 1 (`00000101` $= 5$) : la valeur est donc $-5$.

▷ **Erreur fréquente.** inverser 20 écrit sans les zéros de gauche (`10100`) : il faut d'abord l'écrire sur **8** bits, `00010100`, car ces zéros deviennent des `1` à l'inversion.

**Exercice 15** *Le complément à deux selon un assistant d'IA* ↑énoncé p. 110

▶ **Solution.** **1.** Non. Posons $5+(-5)$ avec l'écriture proposée : `00000101`$+$`11111010` $=$ `11111111`, et non `00000000` : ce n'est pas l'opposé de 5. D'ailleurs `11111010` est l'écriture de $-6$ (l'addition avec `00000110` donne bien `1 00000000`, où la retenue déborde des 8 bits). **2.** L'erreur est dans l'étape « inverser tous les bits » : le complément à deux, c'est inverser **puis ajouter 1** ; l'assistant a oublié le $+1$. $5 =$ `00000101` $\to$ inversion `11111010` $\to +1$ : **`11111011`**. Vérification : `00000101` $+$ `11111011` $=$ `1 00000000`, il reste 0 sur 8 bits. Le reste de la réponse (signe lu sur le bit de poids fort, bornes $-128$ à $+127$) est juste. **3.** Additionner le nombre et son opposé supposé : on doit trouver 0 sur 8 bits. Sur machine, on peut aussi utiliser le fait que, sur $n$ bits, $-x$ se code comme l'entier positif $2^n - x$ : `bin(256 - 5)` affiche `'0b11111011'`, ce qui contredit `11111010`.

**Exercice 16** *Débordement* ↑énoncé p. 110

▶ **Solution.** **1.** Sur 8 bits : de $-128$ à $+127$. Sur 16 bits : de $-32768$ à $+32767$.

**2.** `2 ** 64` vaut `18446744073709551616` : Python le calcule car ses entiers sont de **taille arbitraire** (ils grandissent avec la mémoire). Sur « 64 bits fixes » (comme dans le matériel ou d'autres langages), cette valeur **dépasse** le plus grand entier codable : elle « déborde ».

**Exercice 17** *Le cas étrange de $-128$* ↑énoncé p. 111

▶ **Solution.** **1.** 128 $=$ `10000000` ; on inverse : `01111111` ; on ajoute 1 : `10000000`. Donc $-128$ se code `10000000`.

**2.** On inverse `10000000` : `01111111` ; on ajoute 1 : `10000000`. On **retombe sur le même code** : l'« opposé » de $-128$ calculé ainsi est encore $-128$.

**3.** L'opposé de $-128$ est $+128$, qui n'est **pas codable** sur 8 bits : le plus grand positif est $+127$ (`01111111`). Le calcul déborde et redonne $-128$.

**4.** Les codes qui commencent par `1` sont les $2^7 = 128$ négatifs, de $-128$ à $-1$. Les codes qui commencent par `0` sont aussi au nombre de 128, mais l'un d'eux est 0 : il ne reste que 127 positifs stricts, de 1 à 127. D'où l'asymétrie des bornes $-128$ et $+127$.

## Les nombres à virgule

**Exercice 18** *Fractions en binaire* ↑énoncé p. 111

▶ **Solution.** **1.** $0{,}75 =$ `0,11`$_2$ (car $0{,}75 \times 2 = 1{,}5$ puis $0{,}5 \times 2 = 1{,}0$) ; $6{,}25 =$ `110,01`$_2$ (car $6 =$ `110` et $0{,}25 =$ `0,01`).

**2.** `101,101`$_2 = 4 + 1 + \frac{1}{2} + \frac{1}{8} = 5{,}625$.

**3.** $\frac{1}{3} \times 2 = \mathbf{0} + \frac{2}{3}$ ; $\frac{2}{3} \times 2 = \mathbf{1} + \frac{1}{3}$ ; puis on retrouve $\frac{1}{3}$ : les chiffres `0`, `1` se répètent sans fin, $\frac{1}{3} =$ `0,010101`$\ldots_2$. Comme 0,1 (cours), $\frac{1}{3}$ n'a pas d'écriture binaire finie : une machine n'en stocke qu'une valeur **arrondie** sur un nombre fini de bits.

▷ **Erreur fréquente.** lire les chiffres après la virgule de bas en haut, comme les restes de la partie entière : ceux-là se lisent dans l'**ordre** où on les obtient.

**Exercice 19** *La trahison des flottants* ↑énoncé p. 111

**▶ Solution.**

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

`0,1` et `0,2` n'ayant pas d'écriture binaire finie, ils sont stockés de façon **approchée** : la somme n'est pas exactement `0.3`. Pour comparer deux flottants, on teste si leur écart est minuscule :

```
1 abs((0.1 + 0.2) - 0.3) < 1e-9      # True
```

**Exercice 20** *Additionner dix fois un dixième* ↑énoncé p. 111

**▶ Solution.** On obtient `0.9999999999999999`, et non `1.0`. Chaque `0.1` étant déjà approché, les petites erreurs s'**accumulent** à chaque addition. C'est la même cause qu'à l'exercice précédent. Les flottants sont le plus souvent des approximations : seuls les nombres qui ont une écriture binaire finie (0,5 ; 0,75 ; 6,25…) sont stockés exactement, et 0,1 n'en fait pas partie.

## Coder le texte : ASCII et Unicode

**Exercice 21** *Lire la table ASCII* ↑énoncé p. 111

**▶ Solution.**

1. `C` $= 65 + 2 = 67$ ; `c` $= 67 + 32 = 99$.
2. 25 places avant `Z` : code $90 - 25 = 65$, c'est celui de `A` ; le code 57 est celui de `9` (les chiffres vont de `0`=48 à `9`=57).
3. $65 = 64 + 1 =$ `1000001`$_2$ (sur 7 bits).

**Exercice 22** `ord` *et* `chr` ↑énoncé p. 111

**▶ Solution.** `H` est la 8$^{\text{e}}$ lettre : $65 + 7 = 72$ ; `i` vaut $97 + 8 = 105$ ; ajouter 32 au code d'une majuscule donne sa minuscule.

```
>>> ord("H")
72
>>> chr(105)
'i'
>>> chr(ord("B") + 32)
'b'
>>> ord("i") - ord("H")
33
```

**Exercice 23** *Caractères et octets* ↑énoncé p. 112

**▶ Solution.**

1. `"élève"` a **5 caractères** : `len("élève")` renvoie `5`.
2. Il occupe **7 octets** en UTF-8 : `len("élève".encode("utf-8"))` renvoie 7.
3. Les lettres `é` et `è` ne sont pas dans l'ASCII : chacune prend **2 octets** en UTF-8. On a donc 3 lettres ASCII (1 octet) + 2 lettres accentuées (2 octets) $= 3 + 4 = 7$ octets, pour seulement 5 caractères.

**Exercice 24** *D'où vient le charabia* ↑énoncé p. 112

**▶ Solution.**

1. Le fichier a été **écrit en UTF-8** (où `é` = deux octets `C3 A9`) mais **relu comme du Latin-1**, où chaque octet est un caractère : `C3` devient `Ã` et `A9` devient `©`, d'où `cafÃ©`. Le fichier n'indique pas lui-même son encodage.
2. `"café".encode("utf-8").decode("latin-1")` renvoie `'cafÃ©'`, ce qui reproduit exactement le charabia observé.

**Exercice 25** *Programmer : coder un mot* ↑énoncé p. 112

▶ **Solution.** On parcourt les caractères et on affiche chacun avec son code `ord`.

```
def afficher_codes(mot):
    for c in mot:
        print(c, ord(c))

afficher_codes("Hi")   # affiche  H 72  puis  i 105
```

## Entraînement

**Exercice 26** *Décimal, binaire, hexadécimal* ↑énoncé p. 112

▶ **Solution.** a) $75 = 16\times4+11$ et $4 = 16\times0+4$ : $75 = \texttt{4B}_{16} = \texttt{0100 1011}_2$. b) $\texttt{1011 0110}_2 = \texttt{B6}_{16}$, et $128+32+16+4+2 = 182$. c) $\texttt{C8}_{16} = \texttt{1100 1000}_2$, et $12\times16+8 = 200$. d) $\texttt{3D}_{16} = \texttt{0011 1101}_2$, et $3\times16+13 = 61$.

▷ **Erreur fréquente.** oublier les zéros de gauche d'un paquet : `3` s'écrit `0011` sur 4 bits ; sans eux, l'écriture binaire de `3D` serait fausse.

**Exercice 27** *Complément à deux sur 8 bits* ↑énoncé p. 112

▶ **Solution.** **1.** a) $1 = \texttt{00000001} \to \texttt{11111110} \to \texttt{11111111}$ ; b) $37 = \texttt{00100101} \to \texttt{11011010} \to \texttt{11011011}$ ; c) $100 = \texttt{01100100} \to \texttt{10011011} \to \texttt{10011100}$ ; d) 77 est positif : on l'écrit directement, `01001101`.

**2.** a) bit de poids fort `1` : on inverse (`00000001`), on ajoute 1 (`00000010` $= 2$) : $-2$. b) `00011000` puis `00011001` $= 25$ : $-25$. c) bit de poids fort `0` : positif, $64+16+8+2 = 90$. d) `01111110` puis `01111111` $= 127$ : $-127$.

**3.** `bin(256 - 1)`, `bin(256 - 37)`, `bin(256 - 100)` affichent `'0b11111111'`, `'0b11011011'`, `'0b10011100'`.

▷ **Erreur fréquente.** appliquer « inverser, ajouter 1 » à un nombre positif comme 77, ou à un mot qui commence par `0` comme `01011010` : la méthode ne concerne que les négatifs.

**Exercice 28** *Combien de bits ?* ↑énoncé p. 112

▶ **Solution.** **1.** $2^8 = 256 \leqslant 500 < 512 = 2^9$ : il faut **9 bits** ($500 = \texttt{111110100}_2$). $2^{16} = 65\,536 \leqslant 70\,000 < 131\,072 = 2^{17}$ : il faut **17 bits**.

**2.** Somme : $a+b < 2^{12}+2^{12} = 2\times2^{12} = 2^{13}$, donc $a+b$ s'écrit sur **13 bits** au plus (un de plus, à cause de la retenue finale). Produit : $a\times b < 2^{12}\times2^{12} = 2^{24}$, donc **24 bits** au plus (on additionne les nombres de bits).

**3.** `4095 + 4095` vaut `8190` et `len(bin(8190)) - 2` vaut `13` ; `4095 * 4095` vaut `16769025` et `len(bin(16769025)) - 2` vaut `24` (en effet $2^{23} \leqslant 16\,769\,025 < 2^{24} = 16\,777\,216$). Les bornes de la question 2 sont donc atteintes.

▷ **Erreur fréquente.** croire qu'un produit de deux nombres de 12 bits tient sur 12 ou 13 bits : multiplier les nombres revient à *additionner* les nombres de bits.

**Exercice 29** *Convertir un texte d'un encodage à l'autre* ↑énoncé p. 113

▶ **Solution.** **1.** `octets` s'affiche `b'caf\xe9'` et `len(octets)` vaut `4` : en Latin-1, chaque caractère occupe **un octet**, et `é` est l'octet `E9` (233).

**2.** `texte` vaut `'café'` (les octets ont été lus avec leur bon encodage) ; `nouveaux` s'affiche `b'caf\xc3\xa9'` : **5 octets**, car en UTF-8 le `é` s'écrit sur deux octets (`C3 A9`), les trois lettres ASCII restant sur un octet.

**3.** Python lève une erreur `UnicodeDecodeError` : l'octet `E9` seul n'est pas une suite UTF-8 valide. Pour convertir un fichier, on le **décode avec son encodage d'origine** (ici Latin-1), puis on **réencode** les caractères obtenus dans le nouvel encodage (ici UTF-8). Décoder avec le mauvais encodage donne une erreur ou du charabia (exercice 24).

**4.** Le fichier rouvert en UTF-8 affiche des caractères de remplacement (souvent un losange contenant un point d'interrogation, ou un simple `?`) à la place de `ë` et `à` : leurs octets Latin-1 ne forment pas de l'UTF-8 valide. Pour le convertir, on le rouvre en précisant l'encodage *ISO-8859-1*, puis on l'enregistre en *UTF-8*.



# CORRIGÉS DES TP

## TP Programmer l'arithmétique binaire

↑ énoncé p. 116

### 1 Les convertisseurs

**Exercice 1** *Du binaire vers le décimal*

▶ **Solution.** À la main sur `"1011"` : `n` vaut successivement $0 \to 1 \to 2 \to 5 \to 11$.

```
def binaire_vers_decimal(s):
    n = 0
    for c in s:
        n = 2 * n + int(c)
    return n
```

**Exercice 2** *Du décimal vers le binaire*

▶ **Solution.** Le reste est ajouté **à gauche** : le premier reste obtenu est le bit des unités.

```
def decimal_vers_binaire(n):
    if n == 0:
        return "0"
    s = ""
    while n > 0:
        s = str(n % 2) + s
        n = n // 2
    return s
```

**Exercice 3** *Compléter à gauche*

▶ **Solution.**

```
def completer(s, nb_bits):
    while len(s) < nb_bits:
        s = "0" + s
    return s
```

**Exercice 4** *Du binaire vers l'hexadécimal*

▶ **Solution.** Les caractères sont accumulés dans `paquet` ; chaque paquet complet de 4 bits est converti par `binaire_vers_decimal`, qui donne l'indice du chiffre dans `chiffres`.

```
def binaire_vers_hexa(s):
    chiffres = "0123456789ABCDEF"
    while len(s) % 4 != 0:
        s = "0" + s
    h = ""
    paquet = ""
    for c in s:
        paquet = paquet + c
        if len(paquet) == 4:
```

```
            h = h + chiffres[binaire_vers_decimal(paquet)]
            paquet = ""
    return h
```

## 2 Le complément à deux

**Exercice 5** *Inverser les bits*

▶ **Solution.**

```
def inverser(s):
    r = ""
    for c in s:
        if c == "0":
            r = r + "1"
        else:
            r = r + "0"
    return r
```

**Exercice 6** *Ajouter 1 : la retenue qui se propage*

▶ **Solution.** `0111` + 1 = `1000` et `1111` + 1 = `(1)0000`. Règle : tant que le bit vaut `1`, on le remplace par `0` et la retenue **continue** ; au premier `0`, on le remplace par `1` et la retenue **s'arrête** ; les bits suivants sont recopiés.

```
def incrementer(s):
    r = ""
    retenue = True
    for i in range(len(s) - 1, -1, -1):
        if retenue and s[i] == "1":
            r = "0" + r        # 1 + 1 = 10 : on pose 0, la retenue continue
        elif retenue and s[i] == "0":
            r = "1" + r        # 0 + 1 = 1 : la retenue s'arrete
            retenue = False
        else:
            r = s[i] + r       # plus de retenue : on recopie
    return r
```

**Exercice 7** *Coder un entier relatif*

▶ **Solution.**

```
def complement_a_deux(n, nb_bits):
    if n >= 0:
        return completer(decimal_vers_binaire(n), nb_bits)
    return incrementer(inverser(completer(decimal_vers_binaire(-n), nb_bits)))
```

Remarque : pour $-128$ sur 8 bits, on obtient bien `10000000`.

**Exercice 8** *Décoder un entier relatif*

▶ **Solution.** $11111011_2 = 251$ et $251 - 256 = -5$.

```
def decoder_c2(s):
    if s[0] == "0":
        return binaire_vers_decimal(s)
    return binaire_vers_decimal(s) - 2 ** len(s)
```

## 3 Les portes logiques

**Exercice 9** *Les trois portes de base*

▶ **Solution.**

```
def non(a):
    if a == "0":
        return "1"
    return "0"

def et(a, b):
    if a == "1" and b == "1":
        return "1"
    return "0"

def ou(a, b):
    if a == "1" or b == "1":
        return "1"
    return "0"
```

**Exercice 10** *Construire le OU exclusif*

▶ **Solution.** **1.** Pour $(a,b) = (0,0),(0,1),(1,0),(1,1)$ : $a$ et non $b$ vaut $0,0,1,0$ ; non $a$ et $b$ vaut $0,1,0,0$ ; leur ou vaut donc $0,1,1,0$, exactement la table du xor : $(0,0) \mapsto 0$, $(0,1) \mapsto 1$, $(1,0) \mapsto 1$, $(1,1) \mapsto 0$.

**2. et 3.**

```
def xor(a, b):
    return ou(et(a, non(b)), et(non(a), b))

for a in "01":
    for b in "01":
        print(a, b, xor(a, b))
```

## 4 Les additionneurs

**Exercice 11** *Le demi-additionneur*

▶ **Solution.** Comme vu en cours : $0+0=$ `00`, $0+1=$ `01`, $1+0=$ `01`, $1+1=$ `10`. La retenue (bit de gauche) est la table du **ET** ; la somme (bit de droite) est celle du **OU exclusif**.

```
def demi_additionneur(a, b):
    return et(a, b) + xor(a, b)
```

**Exercice 12** *L'additionneur complet*

▶ **Solution.** Les deux retenues intermédiaires ne peuvent pas valoir `1` en même temps. Un OU suffit donc pour obtenir la retenue sortante.

```
def additionneur_complet(a, b, r):
    d1 = demi_additionneur(a, b)
    d2 = demi_additionneur(d1[1], r)
    return ou(d1[0], d2[0]) + d2[1]
```

**Exercice 13** *L'additionneur $n$ bits*

▶ **Solution.**

```
def additionner(s1, s2):
    r = ""
    retenue = "0"
    for i in range(len(s1) - 1, -1, -1):
        res = additionneur_complet(s1[i], s2[i], retenue)
        r = res[1] + r
        retenue = res[0]
    return retenue + r
```

Pour deux nombres de 64 bits : **64** appels, un par colonne.

## 5 La soustraction

**Exercice 14** *Soustraire, c'est additionner l'opposé*

▶ **Solution.**

```
def sans_premier(s):
    r = ""
    for i in range(1, len(s)):
        r = r + s[i]
    return r

def soustraire(s1, s2):
    return sans_premier(additionner(s1, incrementer(inverser(s2))))
```

**Exercice 15** *Détecter le dépassement*

▶ **Solution.** **1.** $0111 + 0001 = 1000$, qui se décode en $-8$ : on a obtenu $7 + 1 = -8$. Le vrai résultat, 8, sort de la plage $[-8\,;\,7]$ et « déborde » sur le bit de signe.

**2.** Si $x \geqslant 0$ et $y < 0$, alors $y \leqslant x + y < x$ : la somme est comprise entre deux nombres représentables, donc elle est représentable.

**3.**

```
def depassement(s1, s2):
    somme = sans_premier(additionner(s1, s2))
    return s1[0] == s2[0] and somme[0] != s1[0]
```

## 6 La multiplication

**Exercice 16** *Multiplier par 2, 4, 8…*

▶ **Solution.** Multiplier par 2 revient à ajouter un 0 à droite, tous les bits montant d'un rang. Multiplier par $2^k$ revient à ajouter $k$ zéros, exactement comme $\times 10$ en décimal.

```
def decaler(s, k):
    return s + "0" * k
```

**Exercice 17** *Le multiplicateur*

▶ **Solution.** Lignes intermédiaires de `1101 × 1011` : `1101`, `11010`, `000000`, `1101000`. Leur somme vaut `10001111`, soit $13 \times 11 = 143$.

```
def multiplier(s1, s2):
    n = len(s1) + len(s2)
    resultat = "0" * n
    k = 0
    for i in range(len(s2) - 1, -1, -1):
        if s2[i] == "1":
            partiel = completer(decaler(s1, k), n)
```

```
            resultat = sans_premier(additionner(resultat, partiel))
        k = k + 1
    return resultat
```

Le produit de deux nombres de $p$ et $q$ bits est strictement inférieur à $2^{p+q}$ : il tient toujours sur $p+q$ bits, et la retenue retirée par `sans_premier` vaut toujours `0`.

## 7 Comme la machine à billes

**Exercice 18** *Additionner avec des billes*

▶ **Solution. 1.**

```
def ajouter_billes(registre, nb_billes):
    for i in range(nb_billes):
        registre = incrementer(registre)
    return registre
```

**2.** Sur 32 bits, $B$ peut valoir jusqu'à $2^{32}-1 \approx 4{,}3$ milliards : autant d'appels à `incrementer`. `additionner` n'utilise que **32** additionneurs complets, quel que soit $B$. Le coût bille par bille est proportionnel à la *valeur* de $B$, alors que celui de l'additionneur est proportionnel à son *nombre de bits*.

**Exercice 19** *Le convertisseur d'un assistant d'IA*

▶ **Solution. 1.** Les quatre tests passent bien, mais `decimal_vers_binaire(6)` renvoie `"011"` au lieu de `"110"`. La fonction est fausse.

**2.** Les bits sont ajoutés *à droite* alors que le premier reste est le bit des unités. Il faut écrire `s = str(n % 2) + s`.

**3.** `"101"`, `"111"` et `"1001"` (et `"0"`) sont des **palindromes** : écrits à l'envers, ils ne changent pas. Des tests qui ne portent que sur des palindromes ne peuvent pas détecter une chaîne retournée. N'importe quel nombre non palindrome, par exemple 6, 13 ou 2, l'aurait révélé. Le succès d'un jeu de tests ne garantit pas la correction d'un programme : il faut *choisir* des tests capables de faire échouer les erreurs plausibles.

# ATELIER Turing Tumble : calculer avec des billes

↑ énoncé p. 121

**Remarque**

Conventions (celles du guide officiel de Turing Tumble) : un bit vaut 1 quand il pointe à droite, et le bit du haut d'un registre est celui des unités. Les montages des portes ET/OU, de l'inverseur et de « inverser puis +1 » ont été conçus pour l'atelier et se vérifient dans le simulateur (tous les cas des portes ; `0101` → `1010` ; `1011` → `1100`).

## 2 Le bit : une mémoire d'un chiffre binaire

**Exercice 1** *Table du bit*

▶ **Solution.** Valeur `0` : la bille tombe à **droite**, le bit passe à `1`. Valeur `1` : la bille tombe à **gauche**, le bit passe à `0`. Le bit **inverse** sa valeur à chaque passage : c'est la porte **NON** appliquée à sa propre mémoire.

## 3 Des portes logiques avec des billes

**Exercice 2** *Deux portes à reconnaître*

▶ **Solution.** **1.** Montage 1 : seul le cas $A = B = 1$ mène à T (A à `1` : la bille part à gauche, la rampe la ramène sur B ; B à `1` : à gauche, vers T). Dans les trois autres cas, elle finit dans F : c'est la porte **ET**. Montage 2 : seul $A = B = 0$ mène à F. Si A vaut `1`, la bille part directement vers T ; sinon elle est envoyée sur B. C'est la porte **OU**.

**2.** Si A vaut `0`, la bille part à droite et n'atteint jamais B : B n'est ni lu ni basculé. Le résultat est déjà connu (faux), exactement comme `a and b` en Python, qui n'évalue pas `b` quand `a` est faux. Le montage 2 fait de même pour `a or b` quand `a` est vrai.

**3.** Remettre les bits traversés dans leur position de départ (ou dans la position du cas suivant) : l'entrée est « consommée » par le passage de la bille.

**4.** Montage du puzzle 18 (sur le plateau ou le simulateur) : la bille n'atteint l'intercepteur « vrai » que si les deux bits d'entrée valent `1` (même logique que le montage 1).

**Exercice 3** *NON, puis OU exclusif*

▶ **Solution.** **1.** Un seul bit A : s'il vaut `0`, la bille tombe à droite, donc une rampe vers la droite mène à T ; s'il vaut `1`, elle tombe à gauche, et une rampe vers la gauche mène à F.

**2.** Dans les deux branches issues de A, il faut connaître B. Or, dans le montage 1, la branche « A = `0` » ne passe pas par B. On ne peut pas non plus faire converger les deux branches sur le même bit B : après B, on ne saurait plus d'où venait la bille. Solution : A = `1` mène à un bit B (B = `1` → F, B = `0` → T) ; A = `0` mène à un second bit B' (B' = `1` → T, B' = `0` → F). Pour n'avoir qu'une entrée, on relie B et B' par des **bits à engrenage** : ils basculent ensemble et ont toujours la même valeur.

## 4 Le registre : un compteur binaire

**Exercice 4** *Lire un registre*

▶ **Solution.** **a.** $1 + 4 = 5$ (`0101`) ; **b.** $2 + 4 + 8 = 14$ (`1110`) ; **c.** 15 (`1111`).

**Exercice 5** *Pair ou impair ? La moitié ?*

▶ **Solution.** **1.** Le nombre est pair si et seulement si le bit du haut (celui des unités) pointe à gauche. **2.** a. `0101` (5) → `010` = 2 ; b. `1110` (14) → `111` = 7 ; c. `1111` (15) → `111` = 7. On a fait la **division entière par 2** (`n // 2`) ; le bit caché est le reste (`n % 2`). C'est le décalage vers la droite, l'inverse de `decaler`.

**Exercice 6** *Faire tourner la machine dans sa tête*

▶ **Solution.** **1.**

| bille | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| registre après | `0110` | `0111` | `1000` | `1001` | `1010` | `1011` |
| bits traversés | 2 | 1 | 4 | 1 | 2 | 1 |

On trouve bien $5 + 6 = 11$. Une bille traverse tous les bits à `1` de droite, plus le premier `0`.

**2.** Sur le plateau (puzzle 21), après 6 billes le registre affiche `1011` : on retrouve le tableau ci-dessus. En bloquant le levier, le nombre de billes tombées est égal à la valeur du registre (départ à `0000`).

**3.** La 8e bille (`0111` → `1000`) et la 16e (`1111` → `0000`). Pour la 16e, la bille ressort sous le bit de poids 8 : c'est la retenue qui sort du registre, elle est perdue.

## 5 Additionner, soustraire

**Exercice 7** *L'addition « bille par bille »*

▶ **Solution.** **1.** `1011`, soit 11. **2.** $5 + 200$ : 200 billes ; $200 + 5$ : 5 billes. Mieux vaut mettre le grand nombre dans le registre, car le coût est proportionnel au nombre de billes, donc à la *valeur* ajoutée. Un additionneur électronique, lui, coûte autant d'étapes que de bits (TP d'arithmétique binaire, exercice *Additionner avec des billes*). **3.** Observation : chaque bit à engrenage du premier registre, quand il vaut `1`, aiguille vers le second registre autant de billes que son poids ; le second registre, qui est un compteur, fait le reste : $5 + 6 = 11 =$ `1011`.

**Exercice 8** *Le décompteur*

▶ **Solution.** **1.** `1000` $-\ 1$ : les `0` de droite deviennent `1` (l'emprunt se propage) jusqu'au premier `1`, qui devient `0`. C'est la règle de l'incrément, `0` et `1` échangés. Chaque bit traversé bascule toujours de la même façon ; seul change le bit sur lequel la bille doit *continuer* : ceux à `0`, d'où elle tombe à droite. Les rampes de retenue passent donc à droite et le couloir de sortie à gauche. **2.** Départ à 4 pour le puzzle 23, à 9 pour le puzzle 24 : la bille suivante trouve le registre à 0 et est dirigée vers l'intercepteur. **3.** `0000` $-\ 1 =$ `1111`, qui vaut $-1$ en complément à deux sur 4 bits. Le complément à deux est simplement ce qu'on obtient en continuant de compter à rebours sous zéro, modulo $2^4$.

## 6 L'opposé d'un nombre : inverser, puis ajouter 1

**Exercice 9** *Calculer $-4$ avec deux billes*

▶ **Solution.** **1.** Chaque bit est traversé **exactement une fois** : qu'il envoie la bille à gauche ou à droite, une rampe la ramène sur le bit du dessous. Chaque bit bascule donc une fois, ce qui donne l'inverse. **2.** C'est le compteur du puzzle 21 (une bille ajoute 1). Sans intercepteurs, la bille atteindrait le levier et lâcherait une nouvelle bille, qui recommencerait (inverser encore, ou ajouter encore 1). **3.** $\texttt{1100}_2 = 12$ et $12 - 16 = -4$ : c'est bien $-4$. **4.** Les rampes jouent le rôle du **programme** (les instructions), les bits celui des **données**. On a reprogrammé la machine sans toucher aux données.

**Exercice 10** *Des cas surprenants*

▶ **Solution.** **1.** `0101` → `1010` → `1011`, et `complement_a_deux(-5, 4)` renvoie bien `"1011"`. **2.** `0000` → `1111`, puis la bille de la phase 2 traverse les quatre bits (tous à `1`), les remet à `0` et finit dans l'intercepteur de **gauche**, celui de la retenue qui sort. Résultat `0000` : $-0 = 0$, la retenue perdue ne gêne pas (calcul modulo 16). **3.** `1000` → `0111` → `1000` : la machine affirme que $-(-8) = -8$ ! Sur 4 bits, la plage est $[-8\,;\,7]$ : $+8$ n'est pas représentable, c'est un **dépassement**. La fonction `complement_a_deux` a le même comportement.

## 7 Le dépassement

**Exercice 11** *Quand le registre déborde*

▶ **Solution.** **1.** `0000` : la retenue est sortie du registre et s'est perdue. La machine calcule *modulo 16*. **2.** Le bit `OV` est un **indicateur de dépassement**, comme le drapeau *overflow* d'un processeur. Il garde la trace du fait que le résultat affiché est faux, rôle que joue `depassement` dans le TP d'arithmétique binaire.

## 8 Multiplier

**Exercice 12** *Compter de 2 en 2*

**► Solution.** **1.** +2 par le bit de poids 2, +4 par le bit de poids 4 (on saute le ou les premiers rangs). **2.** Après $B$ billes, le registre vaut $A + 2B$ (modulo 16). Entrer un rang plus bas revient à multiplier la contribution de chaque bille par 2, comme `decaler(s, 1)`.

**Exercice 13** *Le multiplicateur mécanique*

**► Solution.** **1.** Idée attendue : un registre $A$ décompte ; à chaque décompte, on ajoute 3 au registre résultat. On peut par exemple envoyer une bille par le bit des unités et une autre par le bit de poids 2 ($1 + 2 = 3$). On répète jusqu'à ce que $A$ vaille 0 : c'est la multiplication vue comme une addition répétée. **2.** $3 \times 3 = 9 =$ `1001` demande 4 bits. Avec un registre de 3 bits, la retenue de poids 8 est perdue : on obtient le résultat modulo 8. Un produit de deux nombres de 2 bits tient toujours sur $2 + 2 = 4$ bits. **3.** « Turing-complet » signifie qu'avec un plateau et des pièces *illimités*, la machine pourrait effectuer n'importe quel calcul qu'effectue un ordinateur : la limite vient de la taille du plateau, pas du principe (des passionnés l'ont établi en montrant qu'un plateau assez grand pourrait imiter un autre modèle de calcul universel). Un vrai ordinateur est d'ailleurs lui aussi limité par sa mémoire.

## Réponses des questions de la fiche de révision

↑ questions p. 130

### Se tester

**1.** $32 + 16 + 4 + 1 = 53$ ; paquets `0011 0101` : $35_{16}$ (§ III.A et § V)
**2.** $2^{10} = 1024$ valeurs ; le plus grand est 1023 (§ II.B)
**3.** Seulement pour $a = 1$, $b = 0$ (table : `0`, `0`, `1`, `0`) (§ IV.C)
**4.** $6 =$ `00000110`, inversé `11111001`, +1 : `11111010` (§ VI.B)
**5.** Bit de poids fort `1` : négatif ; inversé `00001011`, +1 : `00001100` $= 12$, donc $-12$ (§ VI.B)
**6.** $0{,}5 =$ `0,1`$_2$ a une écriture binaire finie ; $0{,}1 =$ `0,000110011...`$_2$ se répète sans fin, il est arrondi (§ VII.B)
**7.** $2^9 = 512 \leqslant 1000 < 1024 = 2^{10}$ : 10 bits ; produit : $2 \times 10 = 20$ bits au plus (§ VIII)
**8.** 2 caractères ; 3 octets (`ù` hors ASCII prend 2 octets) (§ IX.B)

### Et avant ?

**1.** 22 et `1` : le quotient entier et le reste (ch. 1, § II.C)
**2.** 4 tours : `n` vaut 2026, 202, 20, 2, puis 0 (ch. 1, § V)

CORRIGÉS DU CHAPITRE 3

# Les types construits

3

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 Des données en vrac

↑ énoncé p. 138

**Exercice 1** *Retrouver une information*

▶ **Solution.** **1–3.** Réponses attendues (les justifications comptent plus que les mots) :

| | **Tas 1 (identité)** | **Tas 2 (courses)** | **Tas 3 (casiers)** |
|---|---|---|---|
| on retrouve par… | la position (le 1er = nom, le 3e = année…) | la position (ou on parcourt toute la liste) | l'étiquette (le prénom) |
| va changer ? | **non** : l'année de naissance ne change pas, et ne *doit* pas changer | **oui** : on ajoute, on raye | **oui** : un élève arrive, change de casier |
| l'ordre compte ? | oui : échanger nom et prénom change le sens | peu (mais on peut vouloir l'ordre du magasin) | non : on cherche par prénom |

**Exercice 2** *Des rangées et des tiroirs*

▶ **Solution.** **4.** Cases 0 à 3 : pain, lait, pommes, beurre. La case numéro 2 contient **pommes** (et non lait : on numérote à partir de 0, convention déjà vue avec les chaînes et `range`).

**5.** On ajoute une case à la fin (« œufs », case 4) ; rayer « lait » fait **glisser** pommes, beurre et œufs d'un cran vers la gauche : leurs numéros changent (pommes passe de 2 à 1). La rangée **s'allonge ou raccourcit** : elle est modifiable.

**6.** On efface 87 dans le tiroir « Hugo » et on écrit 90 ; l'étiquette ne bouge pas. Deux tiroirs « Hugo » : non, on ne saurait plus lequel ouvrir — une étiquette doit être **unique**. Deux Léa : il faut une étiquette qui les distingue (« Léa D. » et « Léa M. », ou un numéro d'élève).

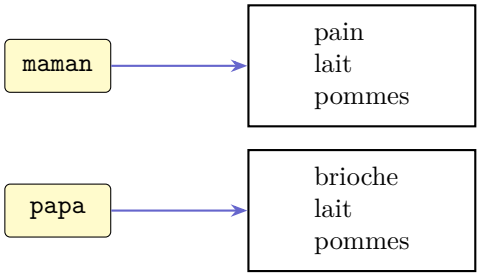
**7.** Un cadenas, une rangée « plastifiée » ou « sous verre » : on peut lire les cases, pas y écrire. Toute idée de « figé » convient.

**Exercice 3** *Une étiquette, ou deux ?*

▶ **Solution.** **8.** Maman lit **brioche, lait, pommes** : il n'y a qu'**une** feuille, avec deux étiquettes dessus.

**9.** Python affiche :
`['brioche', 'lait', 'pommes']`. Beaucoup d'élèves prédisent `['pain', 'lait', 'pommes']`, car ils pensent que `papa = maman` « copie » la liste : c'est exactement la confusion que l'activité doit faire apparaître.

**10.** Il faut **photocopier** (ou recopier) la feuille et coller l'étiquette `papa` sur la copie : deux feuilles, une flèche chacune. En Python, on fabriquera cette copie avec une compréhension (que l'on verra dans le cours) : `papa = [x for x in maman]` ; Maman garde alors `['pain', 'lait', 'pommes']`.




**Exercice 4** *Mettre des mots*

▶ **Solution.** **11.** « . . .par sa **position** (un indice) ou par une **étiquette** (une clé) ? Est-ce que la collection va **changer** (être modifiée) après sa création ? »

**12.** « . . .c'est une **étiquette** (un nom) qui **pointe** vers un objet. Deux variables peuvent désigner **le même** objet. »

# CORRIGÉS DES EXERCICES

> **Remarque**
> Plusieurs écritures sont souvent possibles : on donne **une** version simple.

## P-uplets (`tuple`)

**Exercice 1** *Lecture de trace* ↑énoncé p. 154

▶ **Solution.** Les indices vont de `0` à `4` (ou de `-5` à `-1` en partant de la fin) ; la tranche `t[1:3]` prend les indices `1` et `2`. **a)** `5` **b)** `25` **c)** `(10, 15)` **d)** `5` **e)** `True` **f)** `20`.

▷ **Erreur fréquente.** inclure l'élément d'indice `3` dans `t[1:3]` : comme pour `range`, la borne de droite est exclue.

**Exercice 2** *Vrai ou faux ?* ↑énoncé p. 154

▶ **Solution.** **1.** Faux : `(7)` vaut l'entier `7` ; le singleton s'écrit `(7,)`. **2.** Faux : un p-uplet est immuable, l'affectation lève une `TypeError`. **3.** Vrai : `(1, "deux", 3.0)` est valide. **4.** Vrai : `()` est le p-uplet vide.

▷ **Erreur fréquente.** croire que les parenthèses font le p-uplet : c'est la virgule. `(7)` est un simple entier entre parenthèses.

**Exercice 3** *Une fonction, plusieurs résultats* ↑énoncé p. 154

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert somme_moyenne((12, 15, 9)) == (36, 12.0)  # cas normal
assert somme_moyenne((7,)) == (7, 7.0)           # cas limite : un seul element
assert somme_moyenne((1, 2)) == (3, 1.5)         # cas piege : la moyenne n'est pas entiere
```

Un accumulateur donne la somme ; la moyenne s'en déduit. Pour renvoyer deux valeurs, on renvoie le p-uplet `s, s / len(t)`, que l'appelant déballe avec `s, m = ...`.

```
def somme_moyenne(t):
    s = 0                          # INITIALISER
    for x in t:                    # PARCOURIR
        s = s + x                  # METTRE A JOUR
    return s, s / len(t)           # RENVOYER : le p-uplet (somme, moyenne)

s, m = somme_moyenne((12, 15, 9))  # s vaut 36, m vaut 12.0
```

**Exercice 4** *Conversion de durée* ↑énoncé p. 154

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert convertir(3661) == (1, 1, 1)    # cas normal
assert convertir(0) == (0, 0, 0)       # cas limite : aucune seconde
assert convertir(3599) == (0, 59, 59)  # cas piege : juste avant une heure
```

Une heure fait 3600 secondes : le quotient `n // 3600` donne les heures ; dans le reste `n % 3600`, le quotient par 60 donne les minutes, et le reste par 60 les secondes.

```
def convertir(n):
    heures = n // 3600
    minutes = (n % 3600) // 60
    secondes = n % 60
    return heures, minutes, secondes
```

**Exercice 5** *Permutation circulaire* ↑énoncé p. 155

**▶ Solution.** **1.** On sauvegarde `a` dans une variable temporaire, puis on décale les valeurs :

```
temp = a          # temp vaut 1 (l'ancien a)
a = b             # a vaut 2
b = c             # b vaut 3
c = temp          # c vaut 1
```

**2.** En une ligne :

```
a, b, c = b, c, a    # a vaut 2, b vaut 3, c vaut 1
```

Python évalue d'abord tout le membre de droite : il construit le p-uplet `(2, 3, 1)` avec les valeurs *du moment*, puis le déballe dans `a`, `b`, `c`. Aucune affectation n'a lieu avant que les trois anciennes valeurs aient été lues : rien n'est perdu.

## Tableaux (`list`) : accès, modification, parcours

**Exercice 6** *Lecture de trace* ↑énoncé p. 155

**▶ Solution.** Après exécution : `t = [42, 8, 15, 16, 0]`, `x = [8, 15]` (la tranche `t[1:3]`, prise *après* les deux modifications : `t[1]` et `t[2]` n'ont pas changé), `n = 5`.

**Exercice 7** *Fabriquer des tableaux* ↑énoncé p. 155

**▶ Solution.** Compréhension sur un `range` (de 10 à 50 inclus, avec un pas de 10, d'où la borne `51`), et répétition `*` pour le tableau de zéros.

```
a = [i for i in range(10)]            # [0, 1, ..., 9]
b = [i for i in range(10, 51, 10)]    # [10, 20, 30, 40, 50]
c = [0] * 8                           # huit zeros
```

**Exercice 8** *Parcours : les grands classiques* ↑énoncé p. 155

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert maximum([3, 9, 4]) == 9      # cas normal
assert maximum([5]) == 5            # cas limite : un seul element
assert maximum([-7, -2, -9]) == -2  # cas piege : que des negatifs
```

Somme et comptage : accumulateur initialisé à `0` avant la boucle. Maximum : on part d'un élément qui existe, `t[0]`, et on le remplace dès qu'on trouve plus grand. La moyenne réutilise `somme`.

```
def somme(t):
    s = 0               # INITIALISER
    for x in t:         # PARCOURIR
        s = s + x       # METTRE A JOUR
    return s            # RENVOYER

def maximum(t):
    m = t[0]            # INITIALISER
    for x in t:         # PARCOURIR
        if x > m:       # TESTER
            m = x       # METTRE A JOUR
    return m            # RENVOYER

def moyenne(t):
    return somme(t) / len(t)

def nb_positifs(t):
    c = 0               # INITIALISER
    for x in t:         # PARCOURIR
        if x > 0:       # TESTER
            c = c + 1   # METTRE A JOUR
    return c            # RENVOYER
```

▷ **Erreur fréquente.** initialiser le maximum à `0` : pour `[-5, -2, -9]`, la fonction renverrait `0`, qui n'est même pas dans le tableau.

**Exercice 9** *Modifier sur place* ↑énoncé p. 155

**▶ Solution.** On parcourt par **indice** (indispensable pour modifier).

```
def ajouter_un(t):
    for i in range(len(t)):  # PARCOURIR
        t[i] = t[i] + 1      # METTRE A JOUR

def plafonner(t, m):
    for i in range(len(t)):  # PARCOURIR
        if t[i] > m:         # TESTER
            t[i] = m         # METTRE A JOUR
```

▷ **Erreur fréquente.** écrire `for x in t:` puis `x = x + 1` : on modifie la variable `x`, pas la case du tableau, qui reste inchangé.

**Exercice 10** *Chercher et compter* ↑énoncé p. 155

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert premier_indice([4, 7, 1], 7) == 1  # cas normal
assert premier_indice([], 3) == -1        # cas limite : tableau vide
assert premier_indice([2, 5, 2], 2) == 0  # cas piege : la PREMIERE occurrence
```

Compteur pour `occurrences` ; pour les deux recherches, on sort par `return` dès que `x` est trouvé, et l'on ne conclut à l'absence qu'*après* la boucle.

```
def occurrences(t, x):
    c = 0                   # INITIALISER
    for v in t:             # PARCOURIR
```

```
        if v == x:            # TESTER
            c = c + 1         # METTRE A JOUR
    return c                  # RENVOYER

def est_present(t, x):
    for v in t:               # PARCOURIR
        if v == x:            # TESTER
            return True       # RENVOYER
    return False              # RENVOYER

def premier_indice(t, x):
    for i in range(len(t)):   # PARCOURIR
        if t[i] == x:         # TESTER
            return i          # RENVOYER
    return -1                 # RENVOYER : absent
```

▷ **Erreur fréquente.** ajouter `else: return False` dans la boucle de `est_present` : la fonction conclut dès le premier élément examiné.

**Exercice 11** *Renverser sans* `reverse` ↑énoncé p. 156

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert renverser([1, 2, 3]) == [3, 2, 1]          # cas normal
assert renverser([]) == []                        # cas limite : tableau vide
assert renverser([[1, 2], [3]]) == [[3], [1, 2]]  # cas piege : sous-tableaux intacts
```

On construit un nouveau tableau en parcourant `t` de la fin vers le début.

```
def renverser(t):
    r = []                                # INITIALISER
    for i in range(len(t) - 1, -1, -1):   # PARCOURIR : de la fin vers le debut
        r.append(t[i])                    # METTRE A JOUR
    return r                              # RENVOYER
```

*Autre méthode :* une fois la compréhension vue, le même parcours des indices à l'envers s'écrit en une ligne : `return [t[i] for i in range(len(t) - 1, -1, -1)]`.

**Exercice 12** *Ajouter et retirer [au-delà du programme]* ↑énoncé p. 156

▶ **Solution.** Avec `t = [3, 1, 4, 1, 5]` :

```
t.append(9)      # a) [3, 1, 4, 1, 5, 9]
t = [0] + t      # b) [0, 3, 1, 4, 1, 5, 9] (nouveau tableau)
x = t.pop()      # c) x vaut 9 ; t -> [0, 3, 1, 4, 1, 5]
y = t.pop(2)     # d) y vaut 1 ; t -> [0, 3, 4, 1, 5]
t = t[:3]        # e) [0, 3, 4]
```

## Construction par compréhension

**Exercice 13** *Lecture de trace* ↑énoncé p. 156

▶ **Solution.** On écrit les valeurs de la variable, on garde celles qui vérifient la condition, on calcule l'expression. **a)** `[1, 2, 3, 4, 5]` **b)** `[1, 2, 4, 8, 16]` **c)** `['y', 'o']` **d)** `[0, 5, 10, 15, 20, 25]`.

**Exercice 14** *Écrire des compréhensions* ↑énoncé p. 156

▶ **Solution.**

```
[k ** 3 for k in range(7)]                # 1. cubes de 0 a 6
[n for n in range(40) if n % 3 == 0]      # 2. multiples de 3 < 40
[len(mot) for mot in ["chat", "python", "nsi"]]   # 3. longueurs
[k * k for k in range(1, 8)]              # 4. carres parfaits de 1 a 50
```

*Pour 4, on fabrique directement les carrés plutôt que de tester chaque entier :* $7 \times 7 = 49 \leqslant 50$ *mais* $8 \times 8 = 64 > 50$, *donc* `k` *va de* `1` *à* `7`. *Si l'on ne veut pas chercher la borne, on filtre :* `[k * k for k in range(1, 51) if k * k <= 50]`.

**Exercice 15** *De la boucle à la compréhension* ↑énoncé p. 156

**► Solution.** Ce qui est ajouté par `append` vient en tête, puis la ligne `for`, puis la condition éventuelle.

```
r = [x for x in [5, -2, 7, -8, 3] if x > 0]   # (a) -> [5, 7, 3]
r = [mot[0] for mot in ["le", "chat", "dort"]]   # (b) -> ['l', 'c', 'd']
```

**Exercice 16** *Les voyelles d'une phrase* ↑énoncé p. 156

**► Solution.** *Appels attendus, écrits avant de coder :*

```
assert voyelles("chat") == ["a"]      # cas normal
assert voyelles("") == []             # cas limite : phrase vide
assert voyelles("AIR") == ["A", "I"]  # cas piege : les majuscules comptent
```

Compréhension avec filtre : on garde chaque caractère `c` qui est dans la chaîne des voyelles ; `nb_voyelles` n'est que la longueur de ce tableau.

```
def voyelles(phrase):
    return [c for c in phrase if c in "aeiouyAEIOUY"]

def nb_voyelles(phrase):
    return len(voyelles(phrase))
```

*Autre méthode :* la même idée avec une boucle et `append` (c'est exactement ce que la compréhension condense).

```
def voyelles(phrase):
    resultat = []                # INITIALISER
    for c in phrase:             # PARCOURIR
        if c in "aeiouyAEIOUY":  # TESTER
            resultat.append(c)   # METTRE A JOUR
    return resultat              # RENVOYER
```

## Matrices (tableaux de tableaux)

**Exercice 17** *Se repérer dans une matrice* ↑énoncé p. 157

**► Solution.** On choisit d'abord la ligne (premier indice), puis la case dans la ligne (second indice). **a)** `M[0] = [1, 2, 3]`, `M[2][1] = 8`, `M[1][0] = 4`. **b)** `len(M) = 3` (nombre de lignes), `len(M[0]) = 3` (nombre de colonnes). **c)** `M[1][1] = 0`.

**Exercice 18** *Parcourir une matrice* ↑énoncé p. 157

**► Solution.** *Appels attendus, écrits avant de coder :*

```
assert somme_matrice([[1, 2], [3, 4]]) == 10        # cas normal
assert somme_matrice([[5]]) == 5                    # cas limite : une seule case
assert somme_matrice([[1, 2, 3], [4, 5, 6]]) == 21  # cas piege : matrice non carree
```

Deux boucles imbriquées (les lignes, puis les cases de chaque ligne) avec un accumulateur ou un « meilleur jusqu'ici » initialisé avec une case existante, `M[0][0]` ; une seule boucle pour une ligne donnée.

```
def somme_matrice(M):
    s = 0                # INITIALISER
    for ligne in M:      # PARCOURIR : les lignes
        for x in ligne:  # PARCOURIR : les cases de la ligne
            s = s + x    # METTRE A JOUR
    return s             # RENVOYER

def maximum_matrice(M):
    m = M[0][0]          # INITIALISER : une case qui existe
    for ligne in M:      # PARCOURIR
        for x in ligne:  # PARCOURIR
            if x > m:    # TESTER
```

```
                m = x     # METTRE A JOUR
    return m              # RENVOYER

def somme_ligne(M, i):
    s = 0                 # INITIALISER
    for x in M[i]:        # PARCOURIR
        s = s + x         # METTRE A JOUR
    return s              # RENVOYER
```



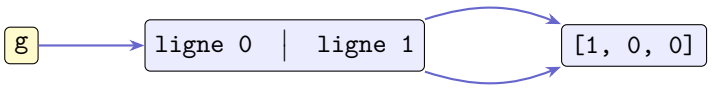
*Autre méthode* pour `somme_matrice`, par indices : `for i in range(len(M)):` puis `for j in range(len(M[i])):` et `s = s + M[i][j]`. Elle est indispensable dès qu'on a besoin de la *position* de la case.

**Exercice 19** *Créer une grille — et le piège* ↑énoncé p. 157

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert grille_zeros(2, 3) == [[0, 0, 0], [0, 0, 0]]    # cas normal
assert grille_zeros(1, 1) == [[0]]                     # cas limite : une seule case
assert grille_zeros(3, 2) == [[0, 0], [0, 0], [0, 0]]  # cas piege : n lignes, p colonnes
```

**1.** `g` vaut `[[1, 0, 0], [1, 0, 0]]` : **pas** le résultat attendu ! Avec `*`, les deux « lignes » sont *le même* objet (deux flèches vers un seul tableau) ; modifier `g[0][0]` modifie donc aussi `g[1]`.




**2.** Avec une compréhension, chaque tour crée une ligne *neuve* :

```
def grille_zeros(n, p):
    return [[0 for j in range(p)] for i in range(n)]

g = grille_zeros(2, 3)
g[0][0] = 1        # g vaut [[1, 0, 0], [0, 0, 0]] : une seule case changee
```

*Autre méthode :* `return [[0] * p for i in range(n)]`. Chaque tour réévalue `[0] * p`, donc crée une ligne neuve.

**Exercice 20** *Transposée* ↑énoncé p. 157

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert transposee([[1, 2], [3, 4]]) == [[1, 3], [2, 4]]  # cas normal
assert transposee([[7]]) == [[7]]                        # cas limite : une seule case
assert transposee([[1, 2, 3]]) == [[1], [2], [3]]        # cas piege : non carree
```

La transposée a `p` lignes et `n` colonnes : la case `(j, i)` du résultat est `M[i][j]`.

```
def transposee(M):
    n = len(M)
    p = len(M[0])
    return [[M[i][j] for i in range(n)] for j in range(p)]
```

*Autre méthode :* avec deux boucles, on construit la transposée ligne par ligne ; sa ligne `j` est la colonne `j` de `M`.

```
def transposee(M):
    resultat = []                  # INITIALISER
    for j in range(len(M[0])):     # PARCOURIR : les colonnes de M
        ligne = []                 # INITIALISER : la colonne j de M
        for i in range(len(M)):    # PARCOURIR
            ligne.append(M[i][j])  # METTRE A JOUR
        resultat.append(ligne)     # METTRE A JOUR
    return resultat                # RENVOYER
```

**Exercice 21** *Traitement d'image* ↑énoncé p. 157

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert miroir_gauche_droite([[1, 2, 3]]) == [[3, 2, 1]]            # cas normal
assert miroir_gauche_droite([[9]]) == [[9]]                        # cas limite : un pixel
assert miroir_gauche_droite([[1, 2], [3, 4]]) == [[2, 1], [4, 3]]  # cas piege : lignes gardees
```

```
def negatif(image):                                    # modifie sur place
    for i in range(len(image)):                        # PARCOURIR
        for j in range(len(image[0])):                 # PARCOURIR
            image[i][j] = 255 - image[i][j]            # METTRE A JOUR

def miroir_gauche_droite(image):                       # renvoie une NOUVELLE image
    nouvelle = []                                      # INITIALISER
    for ligne in image:                                # PARCOURIR
        p = len(ligne)
        nouvelle.append([ligne[p - 1 - j] for j in range(p)])  # METTRE A JOUR
    return nouvelle                                    # RENVOYER
```

## Dictionnaires (`dict`)

**Exercice 22** *Lecture de trace* ↑énoncé p. 158

**▶ Solution.** Dans l'ordre : `d = {"a": 1, "b": 2, "c": 3}`, puis `{"a": 10, "b": 2, "c": 3}`, puis `{"a": 10, "c": 3}`. Ensuite `x = True` (`"c"` est une clé), `y = False` (`in` teste les clés, pas les valeurs), `z = 2`.

▷ **Erreur fréquente.** croire que `3 in d` cherche parmi les valeurs : `in` ne regarde que les clés.

**Exercice 23** *Parcourir un dictionnaire* ↑énoncé p. 158

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert meilleur({"Ali": 14, "Bob": 9, "Eva": 17}) == "Eva"  # cas normal
assert meilleur({"Ada": 15, "Leo": 15}) == "Ada"            # cas limite : egalite, le premier
assert meilleur({"Ada": 0}) == "Ada"                        # cas piege : une note nulle
```

```
def moyenne(notes):
    total = 0                              # INITIALISER
    for note in notes.values():            # PARCOURIR
        total = total + note               # METTRE A JOUR
    return total / len(notes)              # RENVOYER

def admis(notes, seuil):
    return [prenom for prenom in notes if notes[prenom] >= seuil]

def meilleur(notes):
    champion = None                        # INITIALISER
    meilleure_note = -1                    # INITIALISER : toute note est superieure a -1
    for prenom in notes:                   # PARCOURIR
        if notes[prenom] > meilleure_note: # TESTER
            meilleure_note = notes[prenom] # METTRE A JOUR
            champion = prenom              # METTRE A JOUR
    return champion                        # RENVOYER
```

Pour `meilleur`, c'est une recherche de maximum où l'on retient *aussi* le prénom du champion. On part ici d'une note fictive `-1` (une *sentinelle*), valable seulement parce que toutes les notes sont positives ; le chapitre *Algorithmique : le parcours séquentiel* préférera partir d'un élément existant, ce qui marche dans tous les cas.

**Exercice 24** *Compter des votes : code à trous* ↑énoncé p. 158

**▶ Solution.**

```
def compter_votes(votes):
    resultat = {}                                  # INITIALISER
    for prenom in votes:                           # PARCOURIR
        if prenom in resultat:                     # TESTER
            resultat[prenom] = resultat[prenom] + 1  # METTRE A JOUR
        else:
            resultat[prenom] = 1                   # METTRE A JOUR
    return resultat                                # RENVOYER
```

C'est le schéma du compteur par dictionnaire du cours : une clé absente est créée avec la valeur `1`, une clé présente voit sa valeur augmenter de `1`.

**Exercice 25** *Comptage d'occurrences* ↑énoncé p. 158

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert compter_lettres("banane") == {"b": 1, "a": 2, "n": 2, "e": 1}  # cas normal
assert compter_lettres("") == {}                                      # cas limite : mot vide
assert compter_lettres("aA") == {"a": 1, "A": 1}                      # cas piege : la casse
```

Même schéma que « Compter des votes » : chaque lettre est une clé ; à la première rencontre, on la crée avec la valeur `1`, ensuite on augmente sa valeur.

```
def compter_lettres(mot):
    compteur = {}                                  # INITIALISER
    for lettre in mot:                             # PARCOURIR
        if lettre in compteur:                     # TESTER
            compteur[lettre] = compteur[lettre] + 1  # METTRE A JOUR
        else:
            compteur[lettre] = 1                   # METTRE A JOUR : premiere rencontre
    return compteur                                # RENVOYER
```

▷ **Erreur fréquente.** écrire seulement `compteur[lettre] = compteur[lettre] + 1`, sans tester la présence de la clé : à la première lettre, la clé n'existe pas encore et Python lève une `KeyError`.

**Exercice 26** *Inverser un dictionnaire* ↑énoncé p. 159

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert inverser({"a": 1, "b": 2}) == {1: "a", 2: "b"}  # cas normal
assert inverser({}) == {}                              # cas limite : dictionnaire vide
assert inverser({1: "un"}) == {"un": 1}                # cas piege : les types s'echangent
                                                       # aussi
```

On remplit un dictionnaire neuf en parcourant les couples (clé, valeur) de `d` : chaque valeur devient une clé, associée à l'ancienne clé.

```
def inverser(d):
    resultat = {}                  # INITIALISER
    for cle, valeur in d.items():  # PARCOURIR
        resultat[valeur] = cle     # METTRE A JOUR
    return resultat                # RENVOYER
```

Si deux clés partageaient la même valeur, cette valeur deviendrait une **clé unique** dans le résultat : la seconde écraserait la première, et une information serait perdue.

**Exercice 27** *Un relevé de notes à clés composées* ↑énoncé p. 159

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert moyenne_eleve(releve, "Ada") == 16.5                # cas normal
assert moyenne_eleve({("Zoe", "NSI"): 14}, "Zoe") == 14.0  # cas limite : une seule note
assert moyenne_eleve(releve, "Léo") == 12.0                # cas piege : sans les notes d'Ada
```

**1.** `releve[("Ada", "Maths")]` vaut `15`.

**2.**

```python
def moyenne_eleve(releve, eleve):
    total = 0                                   # INITIALISER
    n = 0                                       # INITIALISER
    for (nom, matiere) in releve:               # PARCOURIR : chaque cle est un couple
        if nom == eleve:                        # TESTER
            total = total + releve[(nom, matiere)]  # METTRE A JOUR
            n = n + 1                           # METTRE A JOUR
    return total / n                            # RENVOYER
```

**3.** La clé doit être **immuable** : un p-uplet ne peut pas changer après coup, un tableau si — ce qui empêcherait de retrouver la valeur.

## Le fil rouge à l'épreuve : références, alias, copie

**Exercice 28** *Alias ou copie ? (lecture de trace)* ↑énoncé p. 159

▶ **Solution.** **(a)** affiche `[1, 2, 3, 4] [1, 2, 3, 4]` : `b = a` crée un **alias**, **un seul** objet pour deux noms. **(b)** affiche `[1, 2, 3] [1, 2, 3, 4]` : la compréhension `[x for x in a]` crée un **nouvel** objet, **deux** objets indépendants.

**Exercice 29** *Effet de bord d'une fonction* ↑énoncé p. 159

▶ **Solution.** Affiche `[12, 15, 0]`. Le paramètre `t` et la variable `notes` pointent vers **le même** tableau : la méthode `append` le modifie, donc le changement est visible à l'extérieur (effet de bord). Pour ne pas modifier l'original, on travaille sur une copie :

```python
def ajoute_zero(t):
    nouveau = [x for x in t]   # copie independante
    nouveau.append(0)
    return nouveau
```

**Exercice 30** *Le piège en dictionnaire* ↑énoncé p. 159

▶ **Solution.** Affiche `50 ['clef']`.

- `joueur1 = modele` est un alias : `joueur1["vie"] = 50` change bien `modele["vie"]`.
- La compréhension crée bien un *nouveau* dictionnaire `joueur2`, mais elle ne copie que le *premier niveau* : la valeur associée à `"sac"` (le tableau) reste **partagée** : `joueur2["sac"].append("clef")` modifie donc aussi `modele["sac"]`. Il faudrait une copie *profonde*, qui recopie aussi le tableau : `joueur2["sac"] = [x for x in modele["sac"]]`.

## Synthèse : petits problèmes

**Exercice 31** *Carnet de notes* ↑énoncé p. 160

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```python
assert moyenne_eleve(carnet, "Alice") == 12.0          # cas normal
assert moyenne_eleve({"Zoe": [17]}, "Zoe") == 17.0     # cas limite : une seule note
assert moyenne_eleve({"Lou": [9, 10]}, "Lou") == 9.5   # cas piege : moyenne non entiere
```

Pour ajouter, on crée l'élève absent avec un tableau d'une note, sinon on ajoute la note à son tableau ; la moyenne est un accumulateur sur ce tableau ; `moyennes` réutilise `moyenne_eleve` pour chaque clé.

```python
def ajouter_note(carnet, eleve, note):
    if eleve in carnet:
        carnet[eleve].append(note)
    else:
        carnet[eleve] = [note]

def moyenne_eleve(carnet, eleve):
    notes = carnet[eleve]
    total = 0                                   # INITIALISER
```

```
    for note in notes:                                  # PARCOURIR
        total = total + note                            # METTRE A JOUR
    return total / len(notes)                           # RENVOYER

def moyennes(carnet):
    resultat = {}                                       # INITIALISER
    for eleve in carnet:                                # PARCOURIR
        resultat[eleve] = moyenne_eleve(carnet, eleve)  # METTRE A JOUR
    return resultat                                     # RENVOYER
```

## Exercice 32 *Panier d'une boutique* ↑énoncé p. 160

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert total({"pomme": 4}, {"pomme": 0.5}) == 2.0             # cas normal
assert total({}, prix) == 0                                   # cas limite : panier vide
assert total({"stylo": 2}, {"stylo": 1.5, "gomme": 3}) == 3.0 # cas piege : gomme hors panier
```

```
def total(panier, prix):
    s = 0                                   # INITIALISER
    for produit, q in panier.items():       # PARCOURIR
        s = s + q * prix[produit]           # METTRE A JOUR
    return s                                # RENVOYER

def ajouter(panier, produit, q):
    if produit in panier:
        panier[produit] = panier[produit] + q
    else:
        panier[produit] = q

def produit_plus_gros_sous_total(panier, prix):
    champion = None                         # INITIALISER
    meilleur_sous_total = -1                # INITIALISER : sentinelle
    for produit, q in panier.items():       # PARCOURIR
        sous_total = q * prix[produit]
        if sous_total > meilleur_sous_total:  # TESTER
            meilleur_sous_total = sous_total  # METTRE A JOUR
            champion = produit                # METTRE A JOUR
    return champion                         # RENVOYER
```

Comme pour `meilleur` (exercice « Parcourir un dictionnaire »), le sous-total fictif `-1` est une sentinelle, valable parce que les sous-totaux sont positifs.

## Exercice 33 *Vers le bac — le jeu du démineur (grille)* ↑énoncé p. 160

**▶ Solution.** *Appels attendus, écrits avant de coder :*

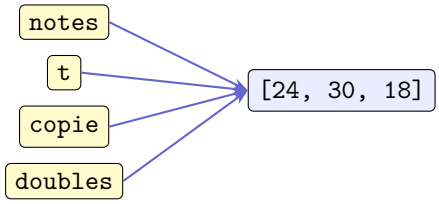
```
assert compter_voisines(champ, 1, 1) == 3  # cas normal
assert compter_voisines(champ, 0, 0) == 2  # cas limite : un coin, 3 voisines seulement
assert compter_voisines(champ, 2, 2) == 0  # cas piege : la case elle-meme ne compte pas
```

On balaie les 9 cases `(a, b)` du carré $3 \times 3$ centré sur `(i, j)` ; on ne compte une case que si elle est **dans la grille** et que ce n'est **pas** la case `(i, j)` elle-même.

```
def compter_voisines(champ, i, j):
    n = len(champ)
    p = len(champ[0])
    total = 0                                           # INITIALISER
    for di in [-1, 0, 1]:                               # PARCOURIR
        for dj in [-1, 0, 1]:                           # PARCOURIR
            a = i + di
            b = j + dj
            dans_grille = a >= 0 and a < n and b >= 0 and b < p
            if dans_grille and not (di == 0 and dj == 0):  # TESTER
                total = total + champ[a][b]             # METTRE A JOUR
    return total                                        # RENVOYER
```

**Exercice 34** *Une copie selon un assistant d'IA* ↑énoncé p. 160

**▶ Solution.** **1.** Non. L'affichage est `[24, 30, 18] [24, 30, 18]` : le tableau d'origine `notes` a été modifié, contrairement à ce qui était demandé. Dessin : `notes`, puis `t` et `copie` pendant l'appel, puis `doubles` au retour ; ces quatre noms pointent tous vers **un seul** tableau.




**2.** L'erreur est la ligne `copie = t` : elle ne copie rien, elle crée un **alias** (deux noms qui pointent vers le *même* tableau, fil rouge du cours). Modifier `copie[i]`, c'est modifier `t`. Il faut construire un nouvel objet, `copie = [x for x in t]` (ou `t[:]`) :

```
def doubler(t):
    copie = [x for x in t]        # INITIALISER : NOUVEL objet, independant de t
    for i in range(len(copie)):   # PARCOURIR
        copie[i] = copie[i] * 2   # METTRE A JOUR
    return copie                  # RENVOYER

notes = [12, 15, 9]
doubles = doubler(notes)
print(doubles, notes)             # [24, 30, 18] [12, 15, 9]
```

**3.** Afficher le tableau d'origine **après** l'appel : c'est exactement ce que la demande promettait (« sans modifier ») et donc la première chose à tester. Un commentaire qui dit « copie » ne prouve pas qu'il y a copie.

## Entraînement

**Exercice 35** *Indices et tranches* ↑énoncé p. 161

**▶ Solution.** Dans l'ordre `t[0]`, `t[-1]`, `t[1:3]`, `t[len(t) - 2]`, `len(t)` : a) `7`, `5`, `[3, 9]`, `1`, `5` ; b) `'lun'`, `'jeu'`, `('mar', 'mer')`, `'mer'`, `4` ; c) `'M'`, `'o'`, `'on'`, `'c'`, `6`. `t[len(t) - 2]` est l'avant-dernier élément, comme `t[-2]`.

**Exercice 36** *Dictionnaires : prédire l'état* ↑énoncé p. 161

**▶ Solution.** États successifs, puis valeurs des expressions :

a)

```
{'pomme': 4, 'poire': 0, 'kiwi': 7}
{'pomme': 6, 'poire': 0, 'kiwi': 7}
{'pomme': 6, 'kiwi': 7}
2
False
```

b)

```
{'Ada': 16, 'Léo': 15, 'Zoé': 17}
{'Ada': 17, 'Léo': 15, 'Zoé': 17}
True
False
3
```

`7 in stock` et `15 in ages` valent `False` : ce sont des valeurs, pas des clés. Dans b), la clé `"Ada"` existait déjà : sa valeur est remplacée, aucune clé n'est ajoutée.

**Exercice 37** *Compréhensions : prédire le tableau* ↑énoncé p. 161

**▶ Solution.** a) `[0, 3, 6, 9]` b) `[1, 2, 3]` c) `[4, 7]` (0 n'est pas strictement positif) d) `['nn', 'ss', 'ii']` e) `[1, 3, 5, 7, 9]`.

## Réponses des questions de la fiche de révision

↑ questions p. 171

### Se tester

1. un dictionnaire (accès par clé : le prénom) (§ II)
2. `[1, 2]` : la compréhension fabrique un **nouveau** tableau, `b` n'est pas un alias de `a` (§ IV.F)
3. `[0, 1, 2, 0, 1]` (§ IV.D)
4. les 2 lignes sont le même objet : changer une case change les deux lignes (§ V.A)
5. `3` lignes, `2` colonnes, et `6` (ligne d'indice 2, colonne d'indice 1) (§ V)

### Et avant ?

1. $1101_2$, car $13 = 8 + 4 + 1$ (ch. 2, § III)
2. on n'additionne pas une chaîne et un entier (`TypeError`) : convertir avec `int` (ch. 1, § II.D)

CORRIGÉS DU CHAPITRE 4

# Algorithmique : le parcours séquentiel



## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 La carte la plus haute

↑énoncé p. 178

**Exercice 1** *Jouer*

▶ **Solution.** **1.** Oui, si la règle est bien appliquée : le chercheur trouve toujours la plus haute carte.

**2.** Il la **garde** : il n'a encore rien en main, et c'est pour l'instant la plus haute carte vue.

**3.** « À chaque nouvelle carte, si elle est plus haute que celle que je tiens, alors je la prends et je jette l'ancienne ; sinon je la jette. »

**4.** Avec un roi, oui : 13 est la plus haute valeur possible, rien ne peut le battre (c'est une information *en plus* sur le paquet). Avec un 10, non : la dernière carte peut être un valet, une dame ou un roi. En général, **on ne peut pas s'arrêter avant la fin** : pour être sûr d'avoir le maximum, il faut avoir tout regardé.

**Exercice 2** *Compter le travail*

▶ **Solution.** **5.** La première carte est prise sans comparaison, puis une comparaison par carte suivante : 9 comparaisons pour 10 cartes, 51 pour 52 cartes, $n-1$ pour $n$ cartes. *(Dans le cours,* `for x in t` *compare aussi* `t[0]` *avec lui-même : $n$ comparaisons, ce qui ne change rien à l'ordre de grandeur.)*

**6.** Pile rangée dans l'ordre **strictement** croissant (10 valeurs différentes) : le chercheur change de carte à chaque fois (9 changements). Plus haute carte en premier (par exemple ordre décroissant) : il ne change jamais. Dans les deux cas, il fait **toujours** 9 **comparaisons** : le nombre de comparaisons ne dépend pas de l'ordre des cartes, seulement de leur nombre.

**7.** Le camarade annonce 0 : aucune température ne bat sa carte imaginaire. C'est faux (0 n'est même pas dans la pile ; la réponse est $-1$). La bonne façon de commencer est de **garder la première vraie carte**, comme à la question 2.

**8.** On retient un **nombre**, qui vaut 0 au départ, et on lui ajoute 1 à chaque carte rouge. Ici, partir de 0 est correct : avant d'avoir vu une carte, on a bien compté zéro carte rouge.

**Exercice 3** *Ce qu'on a découvert*

▶ **Solution.** **9.**

- On examine les cartes **une par une, de la première à la dernière, sans en sauter**.
- À tout moment, la carte en main est **la plus haute des cartes déjà vues** (le « meilleur jusqu'ici »).
- Pour $n$ cartes, il faut $n-1$ comparaisons : si le paquet double, le travail **double** aussi (il est proportionnel au nombre de cartes).

- Même schéma : on **prépare** une information au départ (la première carte, ou le compteur à 0), on la **met à jour** à chaque carte, on l'**annonce** à la fin.

# CORRIGÉS DES EXERCICES



> **Remarque**
> On donne **une** version simple ; d'autres écritures sont possibles.

## Le patron du parcours

**Exercice 1** *Lecture de trace* ↑énoncé p. 188

▶ **Solution.** `mystere([4, 1, 7, 2])` renvoie `7`. La fonction calcule le **maximum** du tableau… mais **seulement s'il est positif** : le champion initial est `0`. Sur un tableau tout négatif, comme `[-3, -7]`, elle renvoie `0`, qui n'est **pas** un élément du tableau : résultat **faux**. Il fallait initialiser avec `t[0]`.

**Exercice 2** *Reconnaître le patron* ↑énoncé p. 188

▶ **Solution.** Même patron que la somme ; on change l'initialisation (l'élément neutre : `1` pour un produit, la chaîne vide pour une concaténation) et la mise à jour.

```
1  def produit(t):
2      p = 1                          # INITIALISER : neutre de la multiplication
3      for x in t:                    # PARCOURIR
4          p = p * x                  # METTRE A JOUR
5      return p                       # RENVOYER
6
7  def concatener(mots):
8      resultat = ""                  # INITIALISER : neutre de la concatenation
9      for mot in mots:               # PARCOURIR
10         resultat = resultat + mot  # METTRE A JOUR
11     return resultat                # RENVOYER
```

## Décliner : comptages, moyennes

**Exercice 3** *Compter dans un intervalle : code à trous* ↑énoncé p. 188

▶ **Solution.**

```
1  def nb_entre(t, a, b):
2      c = 0                      # INITIALISER : compteur, neutre 0
3      for x in t:                # PARCOURIR
4          if x >= a and x <= b:  # TESTER : mise a jour conditionnelle
5              c = c + 1          # METTRE A JOUR
6      return c                   # RENVOYER
```

`nb_entre([12, 5, 18, 9, 15], 10, 15)` renvoie bien `2` (les éléments `12` et `15`). Les deux conditions sont reliées par `and` : il faut *à la fois* `x >= a` et `x <= b`.

▷ **Erreur fréquente.** relier les deux conditions par `or` : tout nombre est supérieur à `a` *ou* inférieur à `b`, et la fonction compte alors tous les éléments.

**Exercice 4** *Taux de réussite et moyenne pondérée* ↑énoncé p. 189

**▶ Solution.** *Appels attendus pour* `moyenne_ponderee`*, écrits avant de coder :*

```
assert moyenne_ponderee([12, 8, 15], [2, 1, 1]) == 11.75  # cas normal
assert moyenne_ponderee([14], [3]) == 14.0                # cas limite : une seule note
assert moyenne_ponderee([10, 20], [1, 3]) == 17.5         # cas piege : diviser par 4, pas par
                                                          # 2
```

```
def taux_reussite(notes):
    c = 0                                 # INITIALISER
    for note in notes:                    # PARCOURIR
        if note >= 10:                    # TESTER
            c = c + 1                     # METTRE A JOUR
    return 100 * c / len(notes)           # RENVOYER

def moyenne_ponderee(notes, coefs):
    total = 0                             # INITIALISER : somme ponderee
    total_coefs = 0                       # INITIALISER : somme des coefficients
    for i in range(len(notes)):           # PARCOURIR
        total = total + notes[i] * coefs[i]   # METTRE A JOUR
        total_coefs = total_coefs + coefs[i]  # METTRE A JOUR
    return total / total_coefs            # RENVOYER
```

Pour `moyenne_ponderee`, on parcourt par **indice** : le même `i` donne la note et son coefficient. Les deux accumulateurs sont mis à jour dans la même boucle : un seul parcours. Sur l'exemple : $(12 \times 2 + 8 + 15)/(2 + 1 + 1) = 47/4 = 11{,}75$.

▷ **Erreur fréquente.** diviser par `len(notes)` au lieu de la somme des coefficients : on obtiendrait $47/3$, qui n'est pas la moyenne pondérée.

## Recherche séquentielle

**Exercice 5** *Un multiple dans le tableau* ↑énoncé p. 189

**▶ Solution.**

```
def a_un_multiple(t, k):
    for x in t:          # PARCOURIR
        if x % k == 0:   # TESTER
            return True  # RENVOYER : on s'arrete des qu'on trouve
    return False         # RENVOYER : parcours fini sans succes
```

`a_un_multiple([7, 10, 3, 20], 5)` fait **2** tours : `10` est trouvé au deuxième tour, et `return` arrête aussitôt la fonction. `a_un_multiple([7, 10, 3, 20], 6)` fait **4** tours : aucun multiple de 6, il faut tout parcourir avant de répondre `False`. C'est le pire cas.

▷ **Erreur fréquente.** écrire `else: return False` dans la boucle : la fonction répond dès le premier élément, et `a_un_multiple([7, 10, 3, 20], 5)` renverrait `False`.

**Exercice 6** *Où est-il ?* ↑énoncé p. 189

**▶ Solution.** *Appels attendus pour* `dernier_indice`*, écrits avant de coder :*

```
assert dernier_indice([6, 3, 8], 3) == 1     # cas normal
assert dernier_indice([], 3) == -1           # cas limite : tableau vide
assert dernier_indice([4, 1, 4, 2], 4) == 2  # cas piege : la derniere, pas la premiere
```

Pour la **première** voyelle, on renvoie dès qu'on trouve. Pour la **dernière** occurrence, en parcourant de gauche à droite, on ne peut pas s'arrêter tôt : une autre occurrence peut encore venir. On retient donc le dernier indice vu, jusqu'au bout du tableau.

```
def premiere_voyelle(mot):
    for i in range(len(mot)):    # PARCOURIR
        if mot[i] in "aeiouy":   # TESTER
            return i             # RENVOYER
```

```
    return -1                   # RENVOYER : aucune voyelle

def dernier_indice(t, x):
    resultat = -1               # INITIALISER : absent jusqu'ici
    for i in range(len(t)):     # PARCOURIR
        if t[i] == x:           # TESTER
            resultat = i        # METTRE A JOUR : on ecrase, on garde le dernier
    return resultat             # RENVOYER
```

*Autre méthode* pour `dernier_indice` : parcourir les indices **à l'envers**, du dernier au premier : la première occurrence rencontrée est alors la dernière du tableau, et on peut s'arrêter dès qu'on la trouve.

```
def dernier_indice(t, x):
    for i in range(len(t) - 1, -1, -1):  # PARCOURIR : a l'envers
        if t[i] == x:                    # TESTER
            return i                     # RENVOYER
    return -1                            # RENVOYER : absent
```

### Exercice 7 *Recherche sous condition* ↑énoncé p. 189

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert premier_negatif([4, 7, -2, 9, -5]) == -2  # cas normal
assert premier_negatif([]) == None               # cas limite : tableau vide
assert premier_negatif([3, 0, 5]) == None        # cas piege : 0 n'est pas negatif
```

Recherche séquentielle : on renvoie l'élément dès que la condition `x < 0` est vraie ; si la boucle se termine sans succès, on renvoie `None`.

```
def premier_negatif(t):
    for x in t:         # PARCOURIR
        if x < 0:       # TESTER
            return x    # RENVOYER
    return None         # RENVOYER : aucun negatif
```

## Minimum, maximum et champion

### Exercice 8 *Le plus petit* ↑énoncé p. 190

▶ **Solution.**

```
def minimum(t):
    m = t[0]        # INITIALISER : un vrai element
    for x in t:     # PARCOURIR
        if x < m:   # TESTER
            m = x   # METTRE A JOUR
    return m        # RENVOYER
```

**Invariant** : à chaque tour, `m` est le minimum des éléments déjà visités. Initialiser `m` à `0` serait faux : sur `[3, 8, 5]`, aucun élément n'est inférieur à `0`, et la fonction renverrait `0`, qui n'est même pas dans le tableau (c'est le défaut de l'exercice « Lecture de trace »). On initialise avec un élément *existant* : `t[0]`.

▷ **Erreur fréquente.** placer `m = t[0]` *dans* la boucle : le champion est remis à `t[0]` à chaque tour, et la fonction renvoie seulement le plus petit de `t[0]` et du dernier élément.

**Exercice 9** *Le mot le plus long : lignes à remettre dans l'ordre* ↑énoncé p. 190

**▶ Solution.**

```
def mot_le_plus_long(mots):
    champion = mots[0]                # INITIALISER
    for mot in mots:                  # PARCOURIR
        if len(mot) > len(champion):  # TESTER : strict
            champion = mot            # METTRE A JOUR
    return champion                   # RENVOYER
```

À noter : `champion = mots[0]` se place **avant** la boucle (initialisation du champion) ; `return champion` est au niveau de la boucle, **après** elle (sinon on renverrait dès le premier tour). Sur l'exemple, on obtient `"girafe"` : quand arrive `"souris"`, le test `len(mot) > len(champion)` compare `6 > 6`, qui est faux. La comparaison **stricte** garde donc le **premier** mot de longueur maximale.

**Exercice 10** *La position du champion* ↑énoncé p. 190

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert indice_du_min([5, 2, 8]) == 1     # cas normal
assert indice_du_min([7]) == 0           # cas limite : un seul element
assert indice_du_min([4, 1, 6, 1]) == 1  # cas piege : egalite, le premier
```

On retient l'**indice** du champion, `i_min`, initialisé à `0` ; on parcourt par indice et l'on compare `t[i]` à la valeur du champion, `t[i_min]`.

```
def indice_du_min(t):
    i_min = 0                 # INITIALISER
    for i in range(len(t)):   # PARCOURIR
        if t[i] < t[i_min]:   # TESTER : strict, on garde le PREMIER
            i_min = i         # METTRE A JOUR
    return i_min              # RENVOYER
```

▷ **Erreur fréquente.** écrire `t[i] <= t[i_min]` : en cas d'égalité, le champion est remplacé, et l'on renvoie le *dernier* indice du minimum.

**Exercice 11** *Amplitude* ↑énoncé p. 190

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert amplitude([3, 9, 1, 4]) == 8      # cas normal
assert amplitude([5]) == 0               # cas limite : un seul element
assert amplitude([-2, -7, -4]) == 5      # cas piege : que des negatifs
```

```
def amplitude(t):
    plus_petit = t[0]                # INITIALISER
    plus_grand = t[0]                # INITIALISER
    for x in t:                      # PARCOURIR
        if x < plus_petit:           # TESTER
            plus_petit = x           # METTRE A JOUR
        if x > plus_grand:           # TESTER
            plus_grand = x           # METTRE A JOUR
    return plus_grand - plus_petit   # RENVOYER
```

Cette solution fait **un seul** parcours (on cherche min et max en même temps). Deux parcours séparés donneraient le même résultat, en deux fois plus de tours.

**Exercice 12** *Moyenne de fin de trimestre* ↑énoncé p. 190

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert moyenne_sans_pire([10, 4, 14, 12]) == 12.0  # cas normal
assert moyenne_sans_pire([9, 15]) == 15.0          # cas limite : deux notes
assert moyenne_sans_pire([6, 12, 6]) == 9.0        # cas piege : pire note en double
```

On retire une fois la plus mauvaise note de la somme, et on divise par `len(t) - 1`. La somme et la pire note se calculent dans le même parcours.

```python
def moyenne_sans_pire(t):
    pire = t[0]                          # INITIALISER : champion du minimum
    total = 0                            # INITIALISER
    for x in t:                          # PARCOURIR
        total = total + x                # METTRE A JOUR
        if x < pire:                     # TESTER
            pire = x                     # METTRE A JOUR
    return (total - pire) / (len(t) - 1)  # RENVOYER
```

**Exercice 13** *Le deuxième plus grand* ↑énoncé p. 190

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```python
assert deuxieme_max([4, 9, 7, 2]) == 7      # cas normal
assert deuxieme_max([5, 3]) == 3            # cas limite : deux valeurs
assert deuxieme_max([3, 9, 9, 7, 2]) == 7   # cas piege : le maximum en double
```

On tient à jour **deux** champions : le plus grand (`premier`) et le deuxième plus grand *distinct* (`second`), initialisés à `None` (« pas encore de champion »). Pour chaque élément `x`, deux cas utiles : `x` bat le record (l'ancien premier passe deuxième) ; ou `x` est strictement plus petit que `premier` et bat `second`. Un élément *égal* à `premier` ne change rien.

```python
def deuxieme_max(t):
    premier = None                       # INITIALISER : plus grand vu
    second = None                        # INITIALISER : 2e plus grand DISTINCT
    for x in t:                          # PARCOURIR
        if premier == None or x > premier:  # TESTER : nouveau record
            second = premier             # METTRE A JOUR
            premier = x                  # METTRE A JOUR
        elif x < premier:                # TESTER : entre second et premier ?
            if second == None or x > second:
                second = x               # METTRE A JOUR
    return second                        # RENVOYER
```

On écrit ici `== None` ; le chapitre *Spécifier et mettre au point ses programmes* présentera l'écriture recommandée `is None`.

## Invariants de boucle

**Exercice 14** *Que contient la variable ?* ↑énoncé p. 191

▶ **Solution.** **1.** Invariant : à chaque tour, `c` contient le **nombre d'éléments pairs déjà visités**.
**2.** À la fin, tous les éléments ont été visités : la fonction renvoie le **nombre d'éléments pairs** du tableau.

▷ **Erreur fréquente.** énoncer l'invariant sur « tous les éléments du tableau » : il ne porte que sur les éléments **déjà visités** ; c'est seulement à la sortie de la boucle que ce sont tous les éléments.

**Exercice 15** *Énoncer un invariant* ↑énoncé p. 191

▶ **Solution.** **1.** Invariant : à chaque tour, `p` contient le **produit des éléments déjà visités** (et `p = 1`, produit vide, avant le premier tour).
**2.** Comme l'invariant est vrai à chaque tour, il l'est encore au dernier : quand tous les éléments ont été visités, `p` est le produit de *tout* le tableau — le résultat est donc correct.

**Exercice 16** *Une preuve d'invariant en deux étapes* ↑énoncé p. 191

▶ **Solution.** **1. Initialisation.** Avant le premier tour, aucun élément n'a été visité et `c = 0` : il y a bien `0` élément positif parmi les éléments visités (il n'y en a aucun). $P$ est vraie.

**2. Hérédité.** Supposons $P$ vraie avant un tour : `c` est le nombre de positifs déjà vus. On visite `x`, deux cas :

- `x > 0` : on fait `c = c + 1`. `x` étant positif, le nombre de positifs vus augmente de 1 : `c` est à jour.

- `x <= 0` : `c` ne change pas. `x` n'étant pas positif, le nombre de positifs vus ne change pas non plus : `c` est à jour.

Dans les deux cas, $P$ est encore vraie après le tour.

**3. Conclusion.** $P$ est vraie au départ et se conserve : elle est donc vraie à chaque tour, en particulier à la fin, quand tous les éléments ont été visités. `c` contient alors le nombre d'éléments strictement positifs de *tout* le tableau. □

## Le coût : linéaire ou quadratique ?

**Exercice 17** *Compter les tours* ↑énoncé p. 191

**▶ Solution.** **(a)** `n` tours de boucle : coût **linéaire**. **(b)** deux boucles imbriquées sur `t` : $n \times n = n^2$ tours : coût **quadratique**.

▷ **Erreur fréquente.** ne compter que les $n$ tours de la boucle intérieure : elle est exécutée en entier à chacun des $n$ tours de la boucle extérieure.

**Exercice 18** *Min et max en une seule passe* ↑énoncé p. 191

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert mini_maxi([4, 9, 1, 6]) == (1, 9)     # cas normal
assert mini_maxi([5]) == (5, 5)              # cas limite : un seul element
assert mini_maxi([-3, -8, -1]) == (-8, -1)   # cas piege : que des negatifs
```

```
def mini_maxi(t):
    mini = t[0]         # INITIALISER
    maxi = t[0]         # INITIALISER
    for x in t:         # PARCOURIR
        if x < mini:    # TESTER
            mini = x    # METTRE A JOUR
        if x > maxi:    # TESTER
            maxi = x    # METTRE A JOUR
    return mini, maxi   # RENVOYER
```

Un seul parcours ($n$ tours) au lieu de deux ($2n$ tours) : deux fois moins de **tours de boucle**, mais autant de **comparaisons** (deux par tour, soit $2n$ dans les deux cas). Le coût reste **linéaire**.

**Exercice 19** *Détecter un doublon* ↑énoncé p. 192

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert a_un_doublon([3, 7, 2, 7]) == True   # cas normal
assert a_un_doublon([5]) == False           # cas limite : un seul element
assert a_un_doublon([1, 2, 3]) == False     # cas piege : ne pas comparer x a lui-meme
```

Deux boucles imbriquées sur les indices : on compare chaque élément à ceux qui sont *après* lui (`j` part de `i + 1`), et l'on s'arrête dès qu'on trouve deux éléments égaux.

```
def a_un_doublon(t):
    for i in range(len(t)):             # PARCOURIR
        for j in range(i + 1, len(t)):  # PARCOURIR : chaque paire une fois
            if t[i] == t[j]:            # TESTER
                return True             # RENVOYER
    return False                        # RENVOYER : aucun doublon
```

Dans le pire cas (aucun doublon), on examine toutes les paires, soit environ $\frac{n^2}{2}$ comparaisons : coût **quadratique**. [ au-delà du programme ]

**Exercice 20** *Deux tableaux* ↑énoncé p. 192

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert element_commun([1, 5, 9], [4, 9, 2]) == True  # cas normal
assert element_commun([], [1, 2]) == False           # cas limite : un tableau vide
assert element_commun([1, 2], [5, 2]) == True        # cas piege : le commun n'est pas en tete
```

```
def element_commun(t1, t2):
    for x in t1:             # PARCOURIR
        for y in t2:         # PARCOURIR
            if x == y:       # TESTER
                return True  # RENVOYER
    return False             # RENVOYER : aucun element commun
```

Dans le pire cas (aucun élément commun), on compare chaque élément de `t1` à chaque élément de `t2`, soit $n \times n = n^2$ comparaisons : coût **quadratique**. [au-delà du programme]

**Exercice 21** *Moyenne, variance — et le piège du coût* ↑énoncé p. 192

▶ **Solution.** *Appels attendus pour* `variance`*, écrits avant de coder :*

```
assert variance([2, 4, 4, 4, 5, 5, 7, 9]) == 4.0  # cas normal
assert variance([5]) == 0.0                       # cas limite : une seule valeur
assert variance([1, 2]) == 0.25                   # cas piege : moyenne non entiere (1.5)
```

**1, 2, 4.** Les trois fonctions :

```
def moyenne(t):                          # 1.
    s = 0                                # INITIALISER
    for x in t:                          # PARCOURIR
        s = s + x                        # METTRE A JOUR
    return s / len(t)                    # RENVOYER

def variance_naive(t):                   # 2.
    n = len(t)
    s = 0                                # INITIALISER
    for x in t:                          # PARCOURIR
        s = s + (x - moyenne(t)) ** 2    # METTRE A JOUR : moyenne(t) a CHAQUE tour
    return s / n                         # RENVOYER

def variance(t):                         # 4.
    n = len(t)
    m = moyenne(t)                       # calculee UNE seule fois
    s = 0                                # INITIALISER
    for x in t:                          # PARCOURIR
        s = s + (x - m) ** 2             # METTRE A JOUR
    return s / n                         # RENVOYER
```

**2.** `variance_naive` est **correcte** : elle calcule bien la variance. **3.** À chacun des `n` tours, `moyenne(t)` effectue `n` additions : total $\approx n \times n = n^2$ additions. Coût **quadratique**. **4.** `variance` fait un parcours pour la moyenne puis un pour la somme : $\approx 2n$ opérations, coût **linéaire**. **5.** Les deux renvoient `4.0` sur l'exemple. Sur un million de valeurs, on utilise `variance` : la version naïve demanderait $\approx 10^{12}$ opérations (des heures), la version efficace $\approx 2 \times 10^6$ (immédiat).

## Synthèse : petits problèmes

### Exercice 22 *Relevé météo* ↑énoncé p. 192

▶ **Solution.** *Appels attendus pour* `plus_longue_serie_chaude`*, écrits avant de coder :*

```
assert plus_longue_serie_chaude([25, 31, 28, 33, 34, 35], 30) == 3  # cas normal
assert plus_longue_serie_chaude([], 30) == 0                        # cas limite : aucun jour
assert plus_longue_serie_chaude([30, 30, 30], 30) == 0              # cas piege : 30 n'est pas
                                                                    # > 30
```

1 : position du champion (on retient l'indice du maximum) ; 2 : un compteur ; 3 : deux variables, la longueur de la série en cours (remise à 0 un jour plus froid) et le record.

```
def jour_le_plus_chaud(temperatures):
    i_max = 0                                   # INITIALISER
    for i in range(len(temperatures)):          # PARCOURIR
        if temperatures[i] > temperatures[i_max]:  # TESTER
            i_max = i                           # METTRE A JOUR
    return i_max                                # RENVOYER

def nb_jours_gel(temperatures):
    c = 0                                       # INITIALISER
    for temp in temperatures:                   # PARCOURIR
        if temp < 0:                            # TESTER
            c = c + 1                           # METTRE A JOUR
    return c                                    # RENVOYER

def plus_longue_serie_chaude(temperatures, seuil):  # au-dela du programme
    record = 0                                  # INITIALISER
    courante = 0                                # INITIALISER : serie en cours
    for temp in temperatures:                   # PARCOURIR
        if temp > seuil:                        # TESTER
            courante = courante + 1             # METTRE A JOUR
            if courante > record:               # TESTER
                record = courante               # METTRE A JOUR
        else:
            courante = 0                        # METTRE A JOUR : serie cassee
    return record                               # RENVOYER
```

### Exercice 23 *Vers le bac — contrôle de saisie* ↑énoncé p. 192

▶ **Solution.** *Appels attendus pour* `toutes_valides`*, écrits avant de coder :*

```
assert toutes_valides([12, 8, 15]) == True   # cas normal
assert toutes_valides([0, 20]) == True       # cas limite : les bornes sont valides
assert toutes_valides([12, 8, -3]) == False  # cas piege : l'intrus est en dernier
```

1 : recherche séquentielle d'une note invalide : on renvoie `False` dès qu'on en trouve une, `True` seulement après la boucle. 2 : on garde les *indices* des notes hors de l'intervalle.

```
def toutes_valides(notes):
    for note in notes:             # PARCOURIR
        if note < 0 or note > 20:  # TESTER
            return False           # RENVOYER : une note hors bornes suffit
    return True                    # RENVOYER

def indices_invalides(notes):
    return [i for i in range(len(notes)) if notes[i] < 0 or notes[i] > 20]
```

Pour `indices_invalides`, on parcourt par **indice** (c'est l'indice qu'on garde, pas la note). *Autre méthode :* la boucle avec `append` que la compréhension condense.

```
def indices_invalides(notes):
    resultat = []                          # INITIALISER
    for i in range(len(notes)):            # PARCOURIR
        if notes[i] < 0 or notes[i] > 20:  # TESTER
```

```
            resultat.append(i)              # METTRE A JOUR
    return resultat                         # RENVOYER
```

**Exercice 24** *Une recherche selon un assistant d'IA* ↑énoncé p. 193

▶ **Solution.** **1.** Non. Trace de `est_present([3, 8, 5], 5)` : premier tour, `element = 3` ; `3 == 5` est faux, on passe dans le `else` et la fonction renvoie **`False`** — alors que `5` est bien dans le tableau. La fonction ne regarde en fait que le *premier* élément (elle ne répond `True` que pour `x = 3`). **2.** L'erreur est le `else: return False` *dans* la boucle : on conclut à l'absence avant d'avoir tout regardé. Le `return False` doit être **après** la boucle :

```
def est_present(t, x):
    for element in t:       # PARCOURIR
        if element == x:    # TESTER
            return True     # RENVOYER
    return False            # RENVOYER : parcours fini sans succes
```

Avec cette correction, `est_present([3, 8, 5], 5)` renvoie `True` et `est_present([3, 8, 5], 7)` renvoie `False`. **3.** Tester la fonction sur un exemple où `x` n'est *pas* en première position : un seul appel suffisait. Un `return False` placé dans un `for` est presque toujours suspect.

## Entraînement

**Exercice 25** *Dérouler la recherche du maximum* ↑énoncé p. 193

▶ **Solution.** Valeurs de `m` après chaque tour : a) `5, 5, 8, 8, 8` : résultat `8` (le second `8` ne remplace pas le premier : `8 > 8` est faux). b) `-4, -4, -2, -2` : résultat `-2`. c) `7` : un seul tour, résultat `7`.

▷ **Erreur fréquente.** partir de `m = 0` au lieu de `t[0]` : sur b), `m` resterait à `0`, qui n'est pas dans le tableau.

**Exercice 26** *Recherche séquentielle : combien de tours ?* ↑énoncé p. 193

▶ **Solution.** a) `True` en **3** tours (`1` est le troisième élément). b) `False` en **4** tours : `5` est absent, il faut tout examiner. c) `False` en **0** tour : le tableau est vide. d) `True` en **1** tour : le meilleur cas. Le pire cas est b) : l'élément est absent et l'on examine les $n$ éléments.

# CORRIGÉS DES TP

## TP Un an de météo à la loupe ↑énoncé p. 196

### 1 Prise en main

**Exercice 1** *Ouvrir les données*

▶ **Corrigé.**

1. 365 jours.
2. `dates[31]` vaut `"2025-02-01"` (le 1er février : l'indice 0 est le 1er janvier, et janvier compte 31 jours) ; `tmax[31]` vaut `14.8` et `pluie[31]` vaut `0.0`.
3. Le 31 décembre est à l'indice `364` (`len(dates) - 1`) ; `mois_de("2025-07-14")` renvoie `7`.
4. Les quatre tableaux sont « parallèles » : c'est le **même indice** `i` qui relie une température à sa date. Un parcours par élément (`for x in tmax`) perd cette position.

## 2 Le bulletin climatique

**Exercice 2** *Accumuler et compter*

▶ Corrigé.

```python
def somme(t):
    s = 0
    for x in t:
        s = s + x
    return s

def moyenne(t):
    return somme(t) / len(t)

def nb_au_moins(t, seuil):
    c = 0
    for x in t:
        if x >= seuil:
            c = c + 1
    return c
```

Résultats : moyenne des maximales **20,3 °C** ; cumul de pluie **599 mm** ; **56** jours de pluie (`nb_au_moins(pluie, 1)`) ; **8** jours de chaleur (`nb_au_moins(tmax, 30)`) ; **97** nuits tropicales (`nb_au_moins(tmin, 20)`).

**Exercice 3** *Des champions datés*

▶ Corrigé.

```python
def indice_min(t):
    i_min = 0
    for i in range(len(t)):
        if t[i] < t[i_min]:          # < strict : en cas d'egalite on garde le premier
            i_min = i
    return i_min

def indice_ecart_max(tmin, tmax):
    i_best = 0
    for i in range(len(tmin)):
        if tmax[i] - tmin[i] > tmax[i_best] - tmin[i_best]:
            i_best = i
    return i_best
```

Nuit la plus froide : **2025-01-12**, 4,5 °C. Plus grande amplitude : **2025-05-14**, 8,0 °C (le bord de mer amortit les écarts, d'où des amplitudes modestes).

**Invariant** (question 3) : « à chaque tour, `i_best` est l'indice d'un jour d'amplitude maximale parmi le jour 0 et les jours **déjà visités** par la boucle » (même formulation que le champion du cours).

- *Initialisation* : avant le premier tour, aucun jour n'est encore visité par la boucle et `i_best = 0` : c'est bien le jour d'amplitude maximale parmi le seul jour 0. Vrai.
- *Hérédité* : si c'est vrai avant le tour `i`, soit l'amplitude du jour `i` dépasse strictement celle du jour `i_best` et `i_best` devient `i`, soit elle ne la dépasse pas et `i_best` reste un jour d'amplitude maximale. Dans les deux cas, la propriété est vraie pour les jours `0` à `i`.

En fin de boucle tous les jours sont visités : `i_best` est un jour d'amplitude maximale de l'année. □

**Exercice 4** *Chercher la première fois*

▶ **Corrigé.**

```
def premier_jour_au_moins(t, seuil):
    for i in range(len(t)):
        if t[i] >= seuil:
            return i                # on s'arrete des qu'on a trouve
    return -1
```

Premier jour à 30 °C : indice 181, soit le **2025-07-01** ; aucun jour à 35 °C (la fonction renvoie `-1`). Pour le seuil 30 : **182** comparaisons ; pour le seuil 35 : **365** comparaisons, c'est le **pire cas** (valeur absente, tout le tableau est parcouru).



**Exercice 5** *Mois par mois*

▶ **Corrigé.**

```
def moyenne_mois(dates, t, mois):
    s = 0
    n = 0
    for i in range(len(t)):
        if mois_de(dates[i]) == mois:
            s = s + t[i]
            n = n + 1
    return s / n

moyennes = [moyenne_mois(dates, tmax, m) for m in range(1, 13)]
m_chaud = 0                         # indice du champion (0 = janvier)
for k in range(12):
    if moyennes[k] > moyennes[m_chaud]:
        m_chaud = k
```

*Autre méthode* pour le tableau des moyennes : `moyennes = []`, puis une boucle `for m in range(1, 13)` qui fait `moyennes.append(moyenne_mois(dates, tmax, m))`. Moyennes mensuelles des maximales (°C), de janvier à décembre : `13.0, 12.6, 14.0, 18.5, 22.5, 25.8, 27.7, 27.2, 24.8, 22.9, 18.7, 15.6`. Mois le plus chaud : **juillet** (`m_chaud + 1 = 7`).

Question 3 : 12 parcours de 365 jours, soit 4 380 tours. En un seul parcours : deux tableaux `sommes = [0]*12` et `effectifs = [0]*12`, et pour chaque jour `k = mois_de(dates[i]) - 1` ; `sommes[k] += t[i]` ; `effectifs[k] += 1`. Coût : 365 tours.

**Exercice 6** *Imprimer le bulletin*

▶ **Corrigé.** Sortie du programme corrigé (extrait ; le symbole degré est omis) :

```
=== Bulletin climatique 2025 ===
Moyenne des maximales : 20.3 C
Cumul de pluie        : 599 mm
Jours de pluie (>= 1 mm) : 56
Jours de chaleur (>= 30 C) : 8
Nuits tropicales (min >= 20 C) : 97
Nuit la plus froide : 2025-01-12 4.5 C
Plus grand écart : 2025-05-14 8.0 C
Premier jour à 30 C : 2025-07-01
Mois le plus chaud : 7
```

## 3 Le piège quadratique, chronomètre en main

**Exercice 7** *Même résultat ?*

**▶ Corrigé.** Les deux renvoient `32.785...` (égalité exacte ici ; sur d'autres données, les arrondis des calculs à virgule peuvent différer au-delà de la 12e décimale). Écart-type : $\sqrt{32{,}79} \approx \mathbf{5{,}7\,°C}$.

**Exercice 8** *Construire un chronomètre*

**▶ Corrigé.**

```
def chronometre(f, t):
    debut = time.perf_counter()
    f(t)
    fin = time.perf_counter()
    return fin - debut

def mesures(tailles):
    temps_naif = []
    temps_lin = []
    for n in tailles:
        t = tableau_aleatoire(n)
        temps_naif.append(chronometre(variance_naive, t))
        temps_lin.append(chronometre(variance, t))
    return temps_naif, temps_lin
```

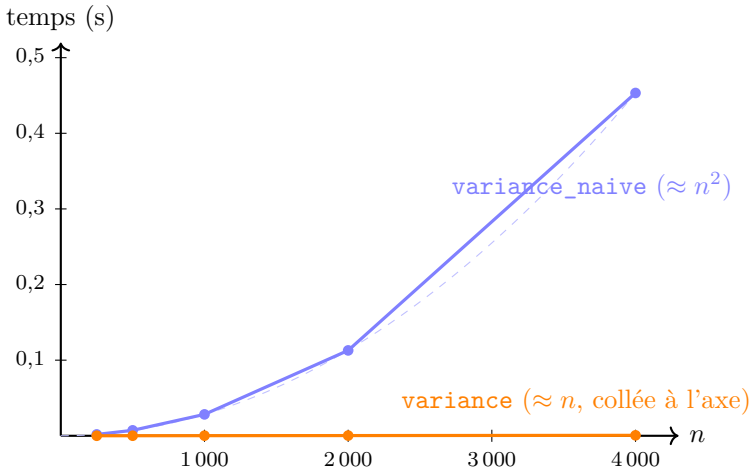
Mesures obtenues sur un ordinateur portable (vos valeurs seront différentes, mais *les rapports* doivent être proches) :

| `n` | `variance_naive` (s) | rapport | `variance` (s) | rapport |
|---|---|---|---|---|
| 250 | 0,00180 | — | 0,0000360 | — |
| 500 | 0,00720 | ×4,0 | 0,0000750 | ×2,1 |
| 1 000 | 0,0283 | ×3,9 | 0,000156 | ×2,1 |
| 2 000 | 0,113 | ×4,0 | 0,000288 | ×1,8 |
| 4 000 | 0,453 | ×4,0 | 0,000598 | ×2,1 |

**Exercice 9** *Lire les mesures*

▶ **Corrigé.**

1. Quand `n` double, le temps de `variance_naive` est multiplié par **4** ($= 2^2$) : coût **quadratique**. Celui de `variance` est multiplié par **2** : coût **linéaire**.
2. La courbe naïve est une **parabole** qui « décolle » ; celle de `variance` est une droite, mais si basse (moins d'une milliseconde contre près d'une demi-seconde) qu'elle se confond avec l'axe à cette échelle.
3. $64\,000 = 4\,000 \times 2^4$ : $0{,}45 \times 4^4 \approx 116$ s, environ **2 minutes**. Pour $10^6 = 4\,000 \times 250$ : $0{,}45 \times 250^2 \approx 28\,000$ s, soit **près de 8 heures**. Pour `variance` : $0{,}0006 \times 16 \approx 0{,}01$ s puis $0{,}0006 \times 250 \approx$ **0,15 s**.
4. Mesure réelle de `variance` sur $10^6$ valeurs : **0,15 s** — la prédiction linéaire tombe juste.




*Mesures du tableau ; en pointillés, la parabole $c\,n^2$ passant par le dernier point.*

## 4 Défi : la moyenne glissante

**Exercice 10** *Version directe*

▶ **Corrigé.**

```
1  def glissante_naive(t, k):
2      res = []
3      for i in range(k - 1, len(t)):
4          s = 0
5          for j in range(i - k + 1, i + 1):
6              s = s + t[j]
7          res.append(s / k)
8      return res
```

Elle fait $(n - k + 1) \times k$ additions, de l'ordre de $k \times n$. Pour $k = 7$, c'est linéaire (7 fois plus que nécessaire) ; mais pour $k = n/2$, cela fait environ $n^2/4$ : **quadratique**. Mesuré sur $n = 20\,000$, $k = 10\,000$ : **5,3 s** contre **0,002 s** pour la version glissante.

**Exercice 11** *Faire glisser la fenêtre*

▶ **Corrigé.**

```
1  def glissante(t, k):
2      s = 0
3      for j in range(k):                # premiere fenetre : t[0] a t[k-1]
4          s = s + t[j]
5      res = [s / k]
6      for i in range(k, len(t)):
7          s = s + t[i] - t[i - k]       # on ajoute le nouveau jour, on retire le plus ancien
```

```
        res.append(s / k)
    return res
```

Vérification : sur `tmax` avec $k = 7$, les deux fonctions renvoient 359 valeurs égales à $10^{-9}$ près. Coût : $k + (n - k) = n$ opérations, **linéaire** quel que soit $k$. C'est la leçon de la variance : **ne pas recalculer** ce que l'on connaît déjà (ici, la somme de la fenêtre précédente). Sur 30 jours, la courbe lissée part de 13 °C (fin janvier), oscille autour de 13-14 °C puis passe par un minimum de 12,2 °C (fenêtre finissant le 10 mars), monte ensuite jusqu'à un maximum de 28,3 °C (fenêtre finissant le 24 juillet) puis redescend : on lit le **cycle des saisons**, débarrassé des variations d'un jour à l'autre.

## Bilan attendu (exemple)

*Toutes les fonctions du bulletin suivent le même patron : initialiser, mettre à jour à chaque jour, renvoyer. Pour les records, on garde l'indice d'un champion, et l'invariant « le champion est le meilleur des jours déjà vus » garantit le résultat. Un parcours coûte de l'ordre de n opérations : quand on double les données, le temps double (nos mesures : ×2). Si l'on refait un parcours à chaque tour (`moyenne(t)` dans la boucle), le coût devient $n^2$ : le temps est multiplié par 4 à chaque doublement, et un million de valeurs demanderait des heures au lieu de 0,15 s. Règle : sortir de la boucle tout calcul qui n'en dépend pas, et réutiliser ce qu'on a déjà calculé.*

## Réponses des questions de la fiche de révision

↑ questions p. 203

### Se tester

1. une propriété vraie avant la boucle et conservée à chaque tour, donc vraie à la sortie (§ IV)
2. `2` (le `0` n'est pas strictement positif) (§ II)
3. linéaire : `n` comparaisons (élément absent) (§ V)
4. `0`, faux (le maximum est `-2`) : initialiser avec `t[0]` (§ IV)
5. une seule fois, avant la boucle (dans la boucle : coût quadratique) (§ V.A)

### Et avant ?

1. `2` et `[8, 2]` (ch. 3, § IV.B)
2. $2^8 = 256$, les entiers de 0 à 255 (ch. 2, § II.B)
3. `6`, soit $0 + 1 + 2 + 3$ (ch. 1, § IV.B)

# 5 Spécifier et mettre au point ses programmes



## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 La chasse aux bugs

↑ énoncé p. 210

**Exercice 1** *La mention au bac*

**▶ Solution.** **1.** `17` : attendu `"Tres bien"`, obtenu `"Tres bien"` ; `15` : `"Bien"` et `"Bien"` ; `8` : `"Refuse"` et `"Refuse"`. Les trois essais sont justes, mais ils ne prouvent **rien** : ils ne passent jamais par la branche `"Assez bien"`, ni près d'une frontière.

**2.** `12` : attendu `"Assez bien"` (« à partir de 12 »), obtenu `"Admis"`. En cause : `elif note > 12:`, qui exclut 12 ; il faut `>=`. Les notes strictement supérieures à 12 sont bien traitées (`12.5` ou `13` donnent `"Assez bien"`) : seule la valeur exacte de la frontière révèle l'erreur.

**3.** Les **valeurs limites** : 10, 12, 14, 16 (chaque frontière), plus les extrémités 0 et 20. Les erreurs se cachent presque toujours sur les frontières entre deux cas (`>` au lieu de `>=`), qu'un essai « au milieu » ne touche jamais.

**4.** `mention(23)` renvoie `"Tres bien"` et `mention(-5)` renvoie `"Refuse"`, sans aucune alerte. C'est l'appel qui est fautif (la commande dit « une note sur 20 »), mais le programme le laisse passer en silence : il vaudrait mieux qu'il **refuse** une note hors de [0 ; 20], en s'arrêtant avec un message clair.

**Exercice 2** *La moyenne d'une classe*

**▶ Solution.** **5.** `moyenne([0, 10, 20])` : la boucle additionne `notes[1]` et `notes[2]`, `total` vaut 30, résultat $30/3 = 10{,}0$ (attendu 10 : juste). `moyenne([0, 0])` : $0/2 = 0{,}0$ (juste).

**6.** Par exemple `moyenne([10, 10])` : attendu 10, obtenu 5,0 (ou `moyenne([12, 14, 16])` : attendu 14, obtenu 10,0). La boucle commence à l'indice `1` : la **première note est oubliée** dans la somme, alors qu'on divise bien par le nombre total de notes. Il faut `range(len(notes))`.

**7.** Dans les deux essais, la première note valait 0 : l'oublier ne change pas la somme. Le programme a donné le bon résultat **par hasard**.

**8.** `moyenne([])` provoque une **division par zéro** : comme `len(notes)` vaut 0, Python s'arrête et signale l'erreur `ZeroDivisionError`. La commande devrait préciser ce qui est permis : « le tableau ne doit pas être vide » (ou bien ce que la fonction doit renvoyer dans ce cas).

**Exercice 3** *Ce qu'on a découvert*

**▶ Solution.** **9.**

- Un programme qui donne le bon résultat sur quelques essais **peut quand même être faux**.
- Les cas qui font le plus souvent échouer un programme sont **les valeurs limites** (les frontières entre deux cas, les extrémités), **le tableau vide**, et les **données hors du domaine prévu**.

- Pour savoir si un résultat est juste, il faut d'abord savoir précisément **ce que la fonction doit faire, et pour quelles données** (la commande du client).
- Refaire ces essais à la main est long : on aimerait **les écrire une fois pour toutes, et que la machine les vérifie** à chaque modification.

# CORRIGÉS DES EXERCICES

## Spécifier et documenter

### Exercice 1 *Précondition ou postcondition ?* ↑énoncé p. 224

**Corrigé.** (a) **précondition** (condition sur l'argument `x`) ; (b) **postcondition** (garantie sur le résultat) ; (c) **postcondition**.

▷ **Erreur fréquente.** classer (c) en précondition parce qu'il y est question de « positif ou nul », comme en (a) : c'est le *résultat* qui est positif, donc une garantie de sortie.

### Exercice 2 *Prototyper puis documenter* ↑énoncé p. 224

**Corrigé.** Le prototype donne le nom et les paramètres ; la docstring, juste en dessous, dit le rôle, ce qu'on attend des arguments (précondition) et ce qu'on garantit sur le résultat (postcondition).

```
def puissance(x, n):
    """Renvoie x eleve a la puissance n.
    Precondition : n est un entier >= 0.
    Postcondition : renvoie x**n (et 1 si n == 0)."""
```

## Programmation défensive

### Exercice 3 *Poser un garde-fou* ↑énoncé p. 224

**Corrigé.**

```
def puissance(x, n):
    assert n >= 0, "l'exposant doit etre positif ou nul"  # VERIFIER : la precondition
    r = 1                                                 # INITIALISER
    for i in range(n):                                    # PARCOURIR
        r = r * x                                         # METTRE A JOUR
    return r                                              # RENVOYER
```

`puissance(2, -3)` déclenche `AssertionError: l'exposant doit etre positif ou nul` : le programme s'interrompt immédiatement, à l'endroit du vrai problème.

▷ **Erreur fréquente.** écrire la condition à l'envers, `assert n < 0, ...` : un `assert` énonce ce qui doit être **vrai** ; il échoue quand la condition est fausse.

### Exercice 4 `assert` *ou* `None` *?* ↑énoncé p. 224

**Corrigé.** *Appels attendus pour la version* `None`*, écrits avant de coder :*

```
assert premier_element_ou_none([4, 7]) == 4  # cas normal
assert premier_element_ou_none([9]) == 9     # cas limite : un seul element
assert premier_element_ou_none([]) is None   # cas piege : tableau vide, la sentinelle
```

Deux stratégies : avec `assert`, la précondition est à la charge de l'appelant, qui vérifie *avant* l'appel ; avec `None`, la fonction accepte tout et l'appelant teste le résultat *après*.

```python
def premier_element(t):        # version defensive
    assert len(t) > 0, "tableau vide"
    return t[0]

def premier_element_ou_none(t):  # version sentinelle
    if len(t) == 0:
        return None
    return t[0]
```

Appels respectant chaque stratégie :

```python
if len(t) > 0:              # avec assert : on garantit la precondition
    print(premier_element(t))

r = premier_element_ou_none(t)   # avec None : on teste le resultat
if r is not None:
    print(r)
```



**Exercice 5** *Le piège de* `-1` ↑énoncé p. 224

**Corrigé.** a) Sur `t = [10, 20, 30]`, si `v` est absente la fonction renvoie `-1`. Mais `t[-1]` vaut `30` : le code appelant qui ferait `t[indice_de(v, t)]` croirait avoir trouvé `v` alors qu'il lit le **dernier** élément. b) Le code appelant doit donc **tester** le résultat avant de s'en servir : `r = indice_de(v, t)` puis `if r != -1 : print(t[r])`. c) L'autre convention, `None`, évite le piège (ce n'est jamais un indice) ; les deux sont correctes, il faut suivre celle de l'énoncé.

## Tester ses programmes

**Exercice 6** *Tests pour* `puissance` ↑énoncé p. 225

**Corrigé.** Des résultats calculés à la main, sur des cas variés : le cas limite `n = 0`, `n = 1`, des cas « normaux » et une base particulière.

```python
assert puissance(2, 0) == 1     # cas limite n = 0
assert puissance(2, 1) == 2
assert puissance(2, 10) == 1024
assert puissance(5, 3) == 125
assert puissance(1, 100) == 1
```

▷ **Erreur fréquente.** calculer le résultat attendu avec la fonction testée, par exemple `assert puissance(2, 10) == puissance(2, 10)` : un tel test passe toujours et ne vérifie rien.

**Exercice 7** *Trouver le contre-exemple* ↑énoncé p. 225

**Corrigé.** La condition `i < len(t) - 1` arrête la boucle **un cran trop tôt** : le **dernier** élément n'est jamais comparé, et `i` vaut au plus `len(t) - 1` à la sortie. Le `return i < len(t)` renvoie donc `True` pour **tout tableau non vide**, que `v` soit présent ou non. Sur `t = [2, 5, 9]` et `v = 9`, la boucle s'arrête à `i = 2` sans avoir comparé le `9` : la réponse `True` est juste, mais par hasard. Les tests « présent » passent tous ; seul un test « absent » révèle l'erreur :

```python
assert appartient(9, [2, 5, 9]) == True   # passe (par hasard)
assert appartient(2, [2, 5, 9]) == True   # passe
assert appartient(4, []) == False         # passe
assert appartient(4, [1, 2]) == False     # ECHOUE : renvoie True
```

Correction : la condition doit être `i < len(t)`. La boucle s'arrête alors soit sur `v`, soit au bout du tableau ; `i < len(t)` dit lequel des deux.

▷ **Erreur fréquente.** ne tester que des valeurs présentes dans le tableau : pour une fonction de recherche, il faut aussi un cas où la valeur est absente.

```python
def appartient(v, t):
    i = 0                           # INITIALISER
    while i < len(t) and t[i] != v:  # PARCOURIR : jusqu'a v ou jusqu'au bout
```

```
        i = i + 1                    # METTRE A JOUR
    return i < len(t)                # RENVOYER : vrai seulement si on s'est arrete sur v
```

*Autre méthode :* un `for` qui renvoie `True` dès qu'on trouve, et `False` seulement après le parcours complet.

```
def appartient(v, t):
    for x in t:          # PARCOURIR
        if x == v:       # TESTER
            return True  # RENVOYER : trouve
    return False         # RENVOYER : parcours fini sans succes
```

**Exercice 8** *Deux erreurs pour le prix d'une* ↑énoncé p. 225

**Corrigé. Erreur 1 — tableau vide :** `trouvee` n'est jamais définie, l'appel lève `NameError` (ou `UnboundLocalError`). **Erreur 2 — seule la dernière case compte :** `trouvee` est réécrite à chaque tour, donc elle vaut le résultat de la comparaison sur le *dernier* élément seulement.

```
assert appartient(1, []) == False      # ECHOUE : NameError (trouvee non definie)
assert appartient(1, [1, 2]) == True   # ECHOUE : renvoie False (dernier = 2)
```

Correction : renvoyer `True` dès qu'on trouve, `False` après le parcours complet.

**Exercice 9** *Un bon jeu de tests* ↑énoncé p. 225

**Corrigé.** `est_pair` est booléenne : il faut un cas `True` et un cas `False`, le cas limite `0`, et des négatifs.

```
assert est_pair(4) == True
assert est_pair(7) == False
assert est_pair(0) == True     # cas limite
assert est_pair(-2) == True    # negatif pair
assert est_pair(-3) == False   # negatif impair
```

**Exercice 10** *Les tests d'abord* ↑énoncé p. 225

**Corrigé. a)** et **b)** Les trois doctests sont écrits *avant* le corps de la fonction : ce sont les trois appels attendus.

```
def nb_occurrences(c, mot):
    """Renvoie le nombre de fois que le caractere c apparait dans la chaine mot.

    >>> nb_occurrences("o", "bonbon")
    2
    >>> nb_occurrences("o", "")
    0
    >>> nb_occurrences("a", "Ananas")
    2
    """
    n = 0                    # INITIALISER
    for lettre in mot:       # PARCOURIR
        if lettre == c:      # TESTER
            n = n + 1        # METTRE A JOUR
    return n                 # RENVOYER

import doctest
doctest.testmod()
```

Le premier exemple est le cas normal, le deuxième le cas limite (chaîne vide : aucun tour de boucle), le troisième le cas piège : `"A"` et `"a"` sont deux caractères **différents**, seuls les deux `"a"` minuscules comptent. Si les trois exemples passent, `testmod()` n'affiche rien.
**c)** Avec `n = 1`, les **trois** exemples échouent (`3`, `1` et `3` au lieu de `2`, `0` et `2`) : pour chacun, `testmod()` affiche l'appel, le résultat attendu (*Expected*) et le résultat obtenu (*Got*), puis le bilan `***Test Failed*** 3 failures.`

▷ **Erreur fréquente.** écrire le résultat attendu en le recopiant depuis la console *après* avoir codé : le doctest valide alors le code tel qu'il est, erreurs comprises. Le résultat attendu se calcule à la main, avant.

### Exercice 11 *Des tests qui démasquent* ↑énoncé p. 225

**Corrigé.** **a)** Version 1 : `range(len(t) - 2)` oublie la **dernière paire** ; sur `[1, 3, 2]` elle répond `True`. Version 2 : `>=` rejette deux éléments **égaux** ; sur `[2, 2]` elle répond `False`. Version 3 : le `else: return True` conclut dès la **première paire** bien rangée ; sur `[1, 2, 0]` elle répond `True`.

**b)** **Deux `assert`** suffisent :

```
assert est_trie([2, 2]) == True       # fait echouer la version 2
assert est_trie([1, 2, 0]) == False   # fait echouer les versions 1 et 3
```

Une version correcte (`range(len(t) - 1)` et `>`, `return True` seulement après la boucle) passe les deux.

**c)** Non : sur `[]`, `[5]`, `[1, 2, 3]` et `[3, 1, 2]`, les trois versions répondent exactement comme une version correcte. Un jeu de tests « au hasard » peut donc passer entièrement sur un programme faux : il faut **viser** les cas où une erreur plausible se manifeste (égalités, désordre en **fin** de tableau, désordre après un début bien rangé).

## Corriger les erreurs (déboguer)

### Exercice 12 *La chasse au bug* ↑énoncé p. 226

**Corrigé.** (a) Sur `[3, 8, 5]` elle renvoie `0` : faux. (b) Le champion `m` est initialisé à `0` (une *valeur*, pas un élément) : aucun élément positif n'est jamais plus petit que 0, donc `m` reste 0. Il faut initialiser `m` au **premier élément**. (c) Version corrigée :

```
def minimum(t):
    """Renvoie le plus petit element de t. Precondition : t non vide."""
    assert len(t) > 0, "tableau vide"  # VERIFIER : la precondition
    m = t[0]                           # INITIALISER : un vrai element
    for x in t:                        # PARCOURIR
        if x < m:                      # TESTER
            m = x                      # METTRE A JOUR
    return m                           # RENVOYER

assert minimum([3, 8, 5]) == 3
assert minimum([-1, -4, -2]) == -4
assert minimum([7]) == 7
```

▷ **Erreur fréquente.** « corriger » en remplaçant `0` par une autre valeur (`100`, `1000`…) : il existera toujours un tableau qui piège la fonction ; seul un élément du tableau est un bon point de départ.

### Exercice 13 *Débogage guidé par l'affichage* ↑énoncé p. 226

**Corrigé.** Un `print(nb)` en fin de corps de boucle afficherait `1, 1, 1, 1` (ou `1, 1, 1` s'il est placé dans le `if`) : `nb` est **remis à 1** à chaque élément positif au lieu d'être **incrémenté**. Il faut `nb = nb + 1`.

```
def nb_positifs(t):
    nb = 0                # INITIALISER
    for x in t:           # PARCOURIR
        if x > 0:         # TESTER
            nb = nb + 1   # METTRE A JOUR : on incremente
    return nb             # RENVOYER

assert nb_positifs([3, 8, -2, 5]) == 3
assert nb_positifs([]) == 0
assert nb_positifs([-1, -2]) == 0
```

▷ **Erreur fréquente.** placer le `print` après la boucle : on ne voit que la valeur finale, pas son évolution tour par tour, qui révèle l'erreur.

**Exercice 14** *Une fonction spécifiée par un assistant d'IA* ↑énoncé p. 226

**Corrigé.** (a) Non. Pour `n = 0`, la condition `n > 0` est fausse dès le départ : la boucle n'est jamais exécutée et la fonction renvoie `0`. Les deux premiers tests passent (`4` et `1`), le troisième échoue :

```
Traceback (most recent call last):
  File "nb_chiffres.py", line 12, in <module>
    assert nb_chiffres(0) == 1
AssertionError
```

(b) La fonction ne respecte pas sa propre spécification sur le cas limite `0`. On compte un chiffre d'office et on boucle tant qu'il en reste au moins un autre (`n >= 10`) :

```
1  def nb_chiffres(n):
2      """Renvoie le nombre de chiffres de l'entier n.
3      Precondition : n >= 0. Par convention nb_chiffres(0) vaut 1."""
4      compteur = 1                  # INITIALISER : au moins un chiffre
5      while n >= 10:                # PARCOURIR : tant qu'il reste un autre chiffre
6          n = n // 10               # METTRE A JOUR
7          compteur = compteur + 1   # METTRE A JOUR
8      return compteur               # RENVOYER
```

Les trois `assert` passent. (c) Exécuter le jeu de tests fourni au lieu de croire la phrase « ils passent tous » : l'assistant a écrit le bon test (le cas limite) sans le lancer. Un test qu'on n'exécute pas ne vérifie rien.

## Invariant de boucle

**Exercice 15** *Une docstring pour* `division_euclidienne` ↑énoncé p. 227

**Corrigé.** La précondition interdit $b = 0$ (et un $a$ négatif) ; la postcondition reprend la définition de la division euclidienne : $a = q \times b + r$ avec $0 \leqslant r < b$.

```
1  def division_euclidienne(a, b):
2      """Renvoie (q, r), quotient et reste de la division de a par b.
3      Precondition : a >= 0 et b > 0.
4      Postcondition : a == q * b + r, 0 <= r et r < b."""
```

**Exercice 16** *Énoncer un invariant* ↑énoncé p. 227

**Corrigé.** Invariant, juste avant le tour d'indice `i` : `r == x ** i` (r est `x` multiplié par lui-même `i` fois). Vrai au départ (`i = 0`, `r = 1 = x**0`). Préservé : si `r == x**i`, alors après `r = r * x` et le passage à `i+1` on a `r == x**(i+1)`. À la sortie, la boucle a fait `n` tours : l'invariant, appliqué « juste avant un tour d'indice `n` » qui n'a pas lieu, donne `r == x**n`, le résultat attendu (pour `n = 0`, aucun tour : `r = 1 = x**0`).

**Exercice 17** *Un meilleur nom* ↑énoncé p. 227

**Corrigé.** Sur `[1, 2, 3]`, la fonction renvoie `6` : c'est une somme. Le nom le dit, la docstring précise le cas du tableau vide, l'invariant décrit ce que contient `s` au début de chaque tour.

```
1  def somme(t):
2      """Renvoie la somme des elements du tableau t (0 si t est vide)."""
3      s = 0                     # INITIALISER
4      for i in range(len(t)):   # PARCOURIR
5          # invariant : s == t[0] + t[1] + ... + t[i-1]
6          s = s + t[i]          # METTRE A JOUR
7      return s                  # RENVOYER
8
9  assert somme([]) == 0
10 assert somme([5]) == 5
11 assert somme([1, 2, 3, 4]) == 10
```

## Exercice 18 *Invariant et* `assert` ↑énoncé p. 227

**Corrigé.**

```
def nb_pairs_debut(t, i):
    """Nombre d'elements pairs parmi t[0], ..., t[i - 1] (fonction de verification)."""
    total = 0                              # INITIALISER
    for k in range(i):                     # PARCOURIR
        if t[k] % 2 == 0:                  # TESTER
            total = total + 1              # METTRE A JOUR
    return total                           # RENVOYER

def nb_pairs(t):
    c = 0                                  # INITIALISER
    for i in range(len(t)):                # PARCOURIR
        assert c == nb_pairs_debut(t, i)   # VERIFIER : invariant, c = nb de pairs des i
                                           # premiers
        if t[i] % 2 == 0:                  # TESTER
            c = c + 1                      # METTRE A JOUR
    return c                               # RENVOYER
```

(a) Juste avant le tour d'indice `i`, `c` est le nombre d'éléments pairs parmi les `i` premiers éléments `t[0]`, ..., `t[i - 1]`.

(b) Pour l'écrire en `assert`, il faut savoir calculer ce nombre *autrement* : c'est le rôle de la fonction de vérification `nb_pairs_debut`. (Elle refait le travail à chaque tour, ce qui ralentit le programme : on ne la garde que pendant la mise au point.)

(c) Pendant la mise au point, Python **vérifie l'invariant à chaque tour** : si une erreur casse la logique du comptage, l'`AssertionError` se déclenche *tout de suite*, au tour fautif, au lieu de laisser un résultat faux apparaître à la fin.

## Exercice 19 *Une division qui se trompe parfois* ↑énoncé p. 228

**Corrigé. a)**

```
def division(a, b):
    """Renvoie (q, r), quotient et reste de la division euclidienne de a par b.
    Precondition : a et b entiers, a >= 0 et b > 0.
    Postcondition : a == q * b + r, 0 <= r et r < b."""
```

**b)** `division(6, 3)` renvoie `(1, 3)` au lieu de `(2, 0)` (de même `division(7, 7)` renvoie `(0, 7)`) : l'erreur apparaît dès que `r` atteint exactement la valeur `b`.

**c)** L'`assert` de l'invariant ne se déclenche jamais : le corps de la boucle conserve bien `a == q * b + r`. C'est l'`assert` de la postcondition qui échoue (`r` vaut `b`, donc `r < b` est faux). L'erreur est donc dans la **condition** de la boucle, qui s'arrête un tour trop tôt : il faut `r >= b`.

**d)**

```
def division(a, b):
    assert a >= 0 and b > 0, "il faut a >= 0 et b > 0"  # VERIFIER : la precondition
    q = 0                                               # INITIALISER
    r = a                                               # INITIALISER
    while r >= b:                                       # PARCOURIR
        assert a == q * b + r                           # VERIFIER : invariant (mise au point)
        r = r - b                                       # METTRE A JOUR
        q = q + 1                                       # METTRE A JOUR
    assert a == q * b + r and 0 <= r and r < b          # VERIFIER : postcondition
    return q, r                                         # RENVOYER

assert division(17, 5) == (3, 2)
assert division(6, 3) == (2, 0)
assert division(7, 7) == (1, 0)
assert division(2, 5) == (0, 2)
assert division(0, 4) == (0, 0)
```

Sans l'`assert` de précondition, `division(5, 0)` ne se termine jamais : `r` reste égal à `5`, la condition `r >= 0` reste vraie, la boucle est **infinie**. L'`assert` transforme ce blocage silencieux en une erreur immédiate et explicite.

## Diversité des langages et bibliothèques

**Exercice 20** *Compilé ou interprété ?* ↑énoncé p. 228

**Corrigé.** a) Interprété : le même fichier tourne partout où l'interpréteur est installé, sans retraduction. b) Compilé : on produit un exécutable rapide, mais il faut recompiler à chaque modification. c) Interprété : la traduction se fait à la volée, donc l'erreur n'apparaît qu'en atteignant la ligne. d) Compilé : la traduction complète a lieu avant l'exécution, ce qui permet de signaler les erreurs en amont.

**Exercice 21** *Statique ou dynamique ?* ↑énoncé p. 228

**Corrigé.** a) (C) statique, type **écrit** (`int`) ; (Python) dynamique, rien à écrire ni à deviner avant l'exécution ; (OCaml) statique, type **deviné** (inféré) : OCaml déduit que `n` est un entier.

b) Non : les deux appels passent par la *même* branche (`code == "normal"`) ; la dernière ligne n'est jamais exécutée. L'appel `prix_ttc(10, "reduit")` l'atteint et déclenche `TypeError: can only concatenate str (not "float") to str` : on ne peut pas coller un texte et un nombre. Correction : `return "prix reduit : " + str(prix * 1.055)` (ou renvoyer simplement le nombre `prix * 1.055`). Un langage à typage **statique** aurait refusé ce programme *avant* l'exécution. Leçon : le jeu de tests doit passer par **toutes** les branches.

**Exercice 22** *Le bon outil* ↑énoncé p. 229

**Corrigé.** a) JavaScript (il s'exécute dans le navigateur). b) SQL (langage de requêtes sur des tables). c) C (compilé, très proche de la machine). d) Python (bibliothèques d'analyse de données, écriture rapide).

**Exercice 23** *Vrai ou faux, et pourquoi* ↑énoncé p. 229

**Corrigé.** a) Faux : aucun langage n'est le meilleur ; chacun est un compromis adapté à certaines tâches. b) Faux : un *algorithme* est indépendant du langage ; seul le *programme* qui le traduit en dépend. c) Vrai : Python est multi-paradigme (impératif, fonctionnel, objet). d) Faux : c'est l'inverse en général ; un langage compilé est traduit une fois pour toutes et s'exécute typiquement *plus vite* qu'un langage interprété.

**Exercice 24** *Langage de programmation ou pas ?* ↑énoncé p. 229

**Corrigé.** **Python, C, JavaScript** : oui, on y écrit des algorithmes (variables, boucles, conditions). **HTML** : non, il *décrit* une page ; c'est un langage formalisé, mais pas un langage de programmation. **SQL** : c'est un langage de *requêtes*, qui *interroge* une base de données en décrivant le résultat voulu ; ce n'est pas un langage de programmation *généraliste*.

**Exercice 25** *Lire un programme dans un langage inconnu* ↑énoncé p. 229

**Corrigé.** (a) Commun : une **fonction** nommée avec un paramètre et une **valeur de retour** (`retourne`). (b) Diffère : mot-clé `fonction` au lieu de `def`, `fin` pour fermer le bloc (pas d'indentation obligatoire), pas de « : ». En Python :

```
1 def double(x):
2     return x * 2
```

**Exercice 26** *Lire une documentation* ↑énoncé p. 229

**Corrigé.** (a) Précondition implicite : `a <= b`. (b) `random.randint(1, 6)`. (c) Non : la documentation (la spécification) suffit pour l'utiliser correctement, sans lire son code — c'est tout l'intérêt d'une bibliothèque.

## Entraînement

**Exercice 27** *Lire un message d'erreur* ↑énoncé p. 230

**Corrigé.** a) `IndexError` : `i` va jusqu'à `3`, or les indices de `notes` vont de `0` à `2` ; il faut `range(len(notes))`, et le programme affiche `36`. b) `ZeroDivisionError` : pour un tableau vide, `len(t)` vaut `0`. La précondition « `t` non vide » doit être vérifiée, par exemple avec `assert len(t) > 0, "tableau vide"` en tête de la fonction. c) `KeyError` : la clé `"kiwi"` n'existe pas dans le dictionnaire ; tester d'abord `if "kiwi" in stock:` avant de lire `stock["kiwi"]`.

▷ **Erreur fréquente.** lire le message de haut en bas et s'arrêter à la première ligne : l'information utile est en bas (type et explication), juste sous la ligne fautive.

**Exercice 28** *Un test qui échoue* ↑énoncé p. 230

**Corrigé.** a) `assert est_majeur(18) == True` échoue : au seuil, `18 > 18` est faux ; corriger en `return age >= 18`. b) `assert somme_jusqua(3) == 6` échoue (résultat `3`, soit $0+1+2$) : `range(n)` va de `0` à `n - 1` ; corriger en `range(1, n + 1)`. c) `assert est_croissant(1, 1, 2) == True` échoue : avec deux valeurs égales, `1 < 1` est faux ; corriger en `return a <= b and b <= c`.

# CORRIGÉS DES TP

## TP Le bulletin bogué ↑énoncé p. 233

### Les sept bugs du fichier de départ

Chaque test du programmeur évite précisément le cas qui révèle le bug (première note nulle, coefficients tous égaux à 1, `x` déjà multiple de 0,5, aucune note égale à 10, pas d'ex aequo, première note la plus basse, première note déjà sous le seuil).

| Fonction | Ligne fautive | Correction |
|---|---|---|
| `moyenne` | `range(1, len(notes))` oublie la 1re note<br>et tableau vide : `ZeroDivisionError` | `range(len(notes))`<br>`if len(notes) == 0: return None` |
| `moyenne_ponderee` | divise par `len(notes)` | divise par la somme des coefficients |
| `arrondi_demi` | `int(x * 2) / 2` tronque | `int(x * 2 + 0.5) / 2` |
| `nb_admis` | `x > 10` | `x >= 10` |
| `rang` | `r = 0` et `x >= m` | `r = 1` et `x > m` |
| `nb_progressions` | `range(len(notes))` : `notes[-1]` pour `i = 0` | `range(1, len(notes))` |
| `premiere_sous` | `i = i + 1` hors de la boucle | l'indenter dans le `while` |

## 1 Prise en main : un bulletin qui a l'air juste

**Exercice 1** *Le bulletin de Lina, à la main*

▶ **Corrigé.** **1.** Affichage obtenu :

```
Tous les tests passent.
Bulletin de Lina
Moyenne : 16.0 / 20
Notes d'au moins 10 : 2 sur 4
Progressions : 2
Rang : 0 sur 6
```

**2.** a) Moyenne pondérée : $\dfrac{15\times1+9\times2+12\times1+10\times2}{1+2+1+2}=\dfrac{65}{6}\approx10{,}83$, arrondie au demi-point le plus proche : **11**. b) Notes d'au moins 10 : **3** (15, 12, 10). c) Progressions : **1** (de 9 à 12). d) Rang : deux moyennes sont strictement supérieures à 11 (13,5 et 15), donc rang **3** sur 6 (ex aequo avec l'autre 11).
**3.** Toutes les lignes sont fausses ; un rang `0` est impossible (le meilleur rang vaut 1).
**4.** Elle prouve seulement que le code donne le bon résultat *sur ces sept cas* ; elle ne prouve pas qu'il est correct en général (règle de Dijkstra : les tests montrent la présence de bugs, jamais leur absence).

## 2 Spécifier : écrire le contrat

**Exercice 2** *Des docstrings à la place des commentaires*

▶ **Corrigé.** **1.** Voir les docstrings de la version corrigée (exercice 6). **2.** Le nom suggère des élèves « admis » alors qu'on compte des *notes* : `nb_notes_au_moins_10` est plus juste ; on garde l'ancien nom pour ne pas casser le reste du logiciel (la docstring lève l'ambiguïté). **3.** Postcondition : `1 <= rang(moyennes, m) <= len(moyennes)` — elle aurait suffi à rejeter le `0` du bulletin de Lina.

## 3 Tester pour démasquer

**Exercice 3** *Un vrai jeu de tests*

▶ **Corrigé.** **1.** `moyenne` : la première note vaut 0, donc l'oublier ne change rien ; pas de cas vide. `moyenne_ponderee` : coefficients tous égaux à 1 (la somme des coefficients vaut alors `len(notes)`). `arrondi_demi` : `12.5` est déjà un multiple de 0,5. `nb_admis` : aucune note égale à la frontière 10. `rang` : pas d'ex aequo. `nb_progressions` : la première note est la plus petite. `premiere_sous` : la réponse est l'indice 0, la boucle ne fait jamais un deuxième tour.
**2–3.** Jeu de tests possible et résultat **réellement obtenu sur la version d'origine** :

| Test | Attendu | Obtenu (origine) |
|---|---|---|
| `moyenne([12, 8, 16])` | `12` | `8.0` |
| `moyenne([])` | `None` | `ZeroDivisionError` |
| `moyenne_ponderee([10, 16], [1, 2])` | `14` | `21.0` |
| `arrondi_demi(12.8)` | `13.0` | `12.5` |
| `arrondi_demi(12.25)` | `12.5` | `12.0` |
| `nb_admis([10, 9.5, 20])` | `2` | `1` |
| `rang([15, 12, 15], 15)` | `1` | `2` |
| `nb_progressions([12, 8, 10])` | `1` | `2` |
| `premiere_sous([12, 14, 5, 3], 10)` | `2` | boucle infinie |
| `premiere_sous([12, 14], 10)` | `None` | boucle infinie |

Tests qui passent sur la version d'origine et qu'on garde (cas normaux et limites) : `arrondi_demi(12.2) == 12.0`, `arrondi_demi(0) == 0`, `nb_admis([]) == 0`, `rang([15, 12, 15], 12) == 3`, `nb_progressions([14]) == 0`, `premiere_sous([], 10) is None`...

### Exercice 4 *Un bug sans message d'erreur*

▶ **Corrigé.** **1.** Pour `i = 0`, on compare `notes[0]` à `notes[-1]`, c'est-à-dire à la **dernière** note : un indice négatif est valide en Python (on compte depuis la fin), donc aucune erreur. Sur `[12, 8, 10]`, `12 > 10` compte une « progression » fantôme. **2.** Avec `[8, 10, 12]`, la première note est la plus petite : `8 > 12` est faux, la comparaison parasite ne compte rien. C'est le piège signalé dans le cours à propos de la convention `-1`.

## 4 Déboguer et corriger

### Exercice 5 *Enquêter avec des affichages*

▶ **Corrigé.** **1.** L'écran se remplit de `i = 0` sans fin (`Ctrl + C`) : `i = i + 1` est indenté au niveau du `while`, donc exécuté *après* la boucle, jamais pendant ; `notes[0] = 12` n'étant pas sous le seuil, la boucle ne progresse pas.
**2.** `somme` vaut 42 et l'on divise par 2 (le nombre de notes) : `21.0`. Il faut diviser par la somme des coefficients, 3, ce qui donne 14.
**3.** `x = 15` : `r = 1` ; `x = 12` : `r = 1` ; `x = 15` : `r = 2`. Résultat 2 au lieu de 1. Avec `>=`, on compte `m` elle-même et ses ex aequo : sans ex aequo, cela fait exactement « 1 + nombre de moyennes strictement supérieures », d'où un résultat juste par hasard.

### Exercice 6 *Tout corriger, tout relancer*

▶ **Corrigé.** Version corrigée complète (avec la programmation défensive de l'exercice 7 et l'invariant de l'exercice 8) :

```
def notes_valides(notes):
    """Renvoie True si toutes les notes sont comprises entre 0 et 20."""
    for x in notes:
        if x < 0 or x > 20:
            return False
    return True

def moyenne(notes):
    """Renvoie la moyenne des notes, ou None si le tableau est vide.
    Precondition : chaque note est entre 0 et 20.
    Postcondition : None si notes == [], sinon un nombre entre 0 et 20."""
    assert notes_valides(notes), "note hors de [0, 20]"
    if len(notes) == 0:
        return None
    somme = 0
    for i in range(len(notes)):
        somme = somme + notes[i]
    return somme / len(notes)

def moyenne_ponderee(notes, coefs):
    """Renvoie la moyenne des notes ponderee par les coefficients.
    Preconditions : notes non vide, meme longueur que coefs,
    notes entre 0 et 20, coefficients strictement positifs.
    Postcondition : un nombre entre 0 et 20."""
    assert len(notes) > 0, "aucune note"
    assert len(notes) == len(coefs), "autant de coefficients que de notes"
    assert notes_valides(notes), "note hors de [0, 20]"
    somme = 0
    somme_coefs = 0
    for i in range(len(notes)):
        assert coefs[i] > 0, "coefficient nul ou negatif"
        somme = somme + notes[i] * coefs[i]
```

```
33          somme_coefs = somme_coefs + coefs[i]
34      return somme / somme_coefs
35
36  def arrondi_demi(x):
37      """Renvoie x arrondi au demi-point le plus proche
38      (au demi-point superieur en cas d'egalite).
39      Precondition : x >= 0."""
40      assert x >= 0, "x doit etre positif"
41      return int(x * 2 + 0.5) / 2
42
43  def nb_admis(notes):
44      """Renvoie le nombre de notes superieures ou egales a 10.
45      Precondition : chaque note est entre 0 et 20."""
46      assert notes_valides(notes), "note hors de [0, 20]"
47      nb = 0
48      for x in notes:
49          if x >= 10:
50              nb = nb + 1
51      return nb
52
53  def rang(moyennes, m):
54      """Renvoie le rang de m : 1 + le nombre de moyennes strictement
55      superieures a m (les ex aequo ont le meme rang).
56      Precondition : m est l'une des moyennes du tableau.
57      Postcondition : 1 <= resultat <= len(moyennes)."""
58      assert m in moyennes, "m doit etre une moyenne de la classe"
59      r = 1
60      for x in moyennes:
61          if x > m:
62              r = r + 1
63      return r
64
65  def nb_progressions(notes):
66      """Renvoie le nombre d'indices i >= 1 tels que notes[i] > notes[i - 1].
67      Precondition : chaque note est entre 0 et 20."""
68      assert notes_valides(notes), "note hors de [0, 20]"
69      nb = 0
70      for i in range(1, len(notes)):
71          if notes[i] > notes[i - 1]:
72              nb = nb + 1
73      return nb
74
75  def premiere_sous(notes, seuil):
76      """Renvoie l'indice de la premiere note strictement inferieure a seuil,
77      ou None s'il n'y en a pas.
78      Precondition : chaque note est entre 0 et 20."""
79      assert notes_valides(notes), "note hors de [0, 20]"
80      i = 0
81      while i < len(notes):
82          # invariant : les notes d'indices 0 a i - 1 sont toutes >= seuil
83          for k in range(i):
84              assert notes[k] >= seuil
85          if notes[i] < seuil:
86              return i
87          i = i + 1
88      return None
89
90  def bulletin(nom, notes, coefs, moyennes_classe):
91      """Affiche le bulletin de l'eleve nom."""
92      m = arrondi_demi(moyenne_ponderee(notes, coefs))
93      print("Bulletin de", nom)
94      print("Moyenne :", m, "/ 20")
95      print("Notes d'au moins 10 :", nb_admis(notes), "sur", len(notes))
```

```
    print("Progressions :", nb_progressions(notes))
    print("Rang :", rang(moyennes_classe, m), "sur", len(moyennes_classe))
    i = premiere_sous(notes, 10)
    if i is not None:
        print("Premiere note sous la moyenne : devoir numero", i + 1)
```

Jeu de tests final (tous verts) :

```
assert moyenne([0, 10, 20]) == 10
assert moyenne([12, 8, 16]) == 12        # la premiere note compte
assert moyenne([15]) == 15
assert moyenne([]) is None               # cas vide : sentinelle
assert moyenne_ponderee([12, 14], [1, 1]) == 13
assert moyenne_ponderee([10, 16], [1, 2]) == 14   # coefficients differents
assert moyenne_ponderee([8], [3]) == 8
assert arrondi_demi(12.5) == 12.5
assert arrondi_demi(12.8) == 13.0        # arrondi vers le haut
assert arrondi_demi(12.2) == 12.0
assert arrondi_demi(12.25) == 12.5       # egalite : demi-point superieur
assert arrondi_demi(0) == 0
assert nb_admis([8, 12, 15]) == 2
assert nb_admis([10, 9.5, 20]) == 2      # 10 compte
assert nb_admis([]) == 0
assert rang([12, 15, 9], 12) == 2
assert rang([12, 15, 9], 15) == 1
assert rang([15, 12, 15], 15) == 1       # ex aequo premiers
assert rang([15, 12, 15], 12) == 3
assert nb_progressions([8, 10, 12]) == 2
assert nb_progressions([12, 8, 10]) == 1  # le 1er devoir n'a pas de precedent
assert nb_progressions([14]) == 0
assert nb_progressions([]) == 0
assert premiere_sous([5, 12, 14], 10) == 0
assert premiere_sous([12, 14, 5, 3], 10) == 2
assert premiere_sous([12, 14], 10) is None
assert premiere_sous([], 10) is None
assert notes_valides([0, 20, 12.5]) == True
assert notes_valides([-1, 12]) == False
assert notes_valides([20.5]) == False
assert notes_valides([]) == True
```

**2.** On ajoute `0.5` **après** avoir multiplié par 2 et **avant** `int` : `int(x * 2 + 0.5)`. `int` tronque (garde la partie entière) ; décaler de 0,5 fait passer à l'entier suivant tout nombre dont la partie décimale est au moins 0,5 : c'est un arrondi à l'entier le plus proche de $2x$, donc au demi-point le plus proche de `x`. `12.2` → `int(24.9)/2 = 12.0` ; `12.25` → `int(25.0)/2 = 12.5` ; `12.8` → `int(26.1)/2 = 13.0`.

**3.** Le bulletin corrigé correspond au calcul de l'exercice 1 :

```
Bulletin de Lina
Moyenne : 11.0 / 20
Notes d'au moins 10 : 3 sur 4
Progressions : 1
Rang : 3 sur 6
Premiere note sous la moyenne : devoir numero 2
```

**4.** Le rapport de bugs reprend le tableau du début de ce corrigé (symptôme = test de l'exercice 3).

## 5 Programmation défensive

**Exercice 7** *Des garde-fous*

▶ **Corrigé.** **1–2.** Voir le code ci-dessus. Messages obtenus :

```
moyenne_ponderee([12, 25], [1, 1]) -> AssertionError: note hors de [0, 20]
moyenne_ponderee([12, 14], [1])    -> AssertionError: autant de coefficients que de notes
moyenne_ponderee([12, 14], [1, 0]) -> AssertionError: coefficient nul ou negatif
arrondi_demi(-3)                   -> AssertionError: x doit etre positif
rang([12, 15], 13)                 -> AssertionError: m doit etre une moyenne de la classe
```

**3.** Celui qui appelle doit **tester le résultat** (`if r is not None:`) avant de l'utiliser dans un calcul ou un affichage ; sinon l'erreur surgit plus loin (`TypeError` sur `None`).
**4.** Voir les deux dernières lignes de `bulletin`. Pour Lina, la première note sous 10 est le 9 du devoir d'indice 1, d'où « devoir numero 2 ».

## 6 Prouver une boucle : invariant et variant

**Exercice 8** *La boucle de* `premiere_sous`*, version corrigée*

▶ **Corrigé.** **1.** « … sont toutes **supérieures ou égales à `seuil`** ».
**2.** Avant le premier tour, `i = 0` : il n'y a aucune note d'indice 0 à −1, la propriété est vraie (rien à vérifier). Si elle est vraie au début d'un tour et qu'on ne sort pas par le `return`, c'est que `notes[i] >= seuil` ; après `i = i + 1`, les notes d'indices 0 à `i - 1` (l'ancien `i` compris) sont toutes au moins égales au seuil.
**3.** On sort de la boucle sans `return` quand `i < len(notes)` devient faux, c'est-à-dire `i == len(notes)`. L'invariant dit alors que *toutes* les notes sont au moins égales au seuil : renvoyer `None` est bien correct. (Si l'on sort par `return i`, l'invariant garantit que c'est la *première* note sous le seuil.)
**4.** Voir la petite boucle `for k in range(i)` du code corrigé (à retirer une fois la mise au point finie : elle rend la fonction plus lente).
**5.** Variant : `len(notes) - i`, entier positif tant qu'on est dans la boucle, qui diminue de 1 à chaque tour. Dans la version d'origine, `i` ne change jamais pendant la boucle : la quantité reste constante, ce n'est pas un variant — et la boucle est effectivement infinie dès que `notes[0] >= seuil`.

**Exercice 9** *Un invariant pour un compteur*

▶ **Corrigé.** Invariant : « après avoir parcouru les `k` premières notes, `nb` est le nombre de notes supérieures ou égales à 10 parmi elles ». Vrai au départ (`k = 0`, `nb = 0`), préservé car on ajoute 1 exactement quand la note examinée est au moins 10. À la fin, toutes les notes ont été parcourues : `nb` est le résultat attendu.

## 7 Défi : au-delà de Python

**Exercice 10** *Le même calcul en C*

▶ **Corrigé.** **1.** Commun : une fonction, une affectation `somme = somme + notes[i]`, une boucle bornée, un accumulateur initialisé à 0, un `return`. Particulier au C : types déclarés (`float`, `int`), accolades à la place de l'indentation, points-virgules, boucle `for (init; condition; pas)`, la taille `n` du tableau passée en paramètre.
**2.** Statique : chaque variable a un type **déclaré** (`int somme`). `somme = somme + "a";` est refusé **à la compilation**, avant toute exécution (le compilateur signale par exemple *incompatible pointer to integer conversion*). En Python, la même faute ne se verrait qu'à l'exécution de la ligne.
**3.** `somme / n` = `25 / 2` = `12` en division entière : le programme affiche `12.000000` au lieu de `12.5`. Tout test dont la somme n'est pas un multiple du nombre de notes le démasque ; les tests « ronds » comme `{10, 10}` ou `{0, 10, 20}` le laissent passer. `(float)` **convertit** `somme` en flottant avant

la division, qui devient alors décimale (affichage corrigé : `12.500000`).
**4.** Non dans les deux cas : ce sont des erreurs de **logique**, pas de langage ; le programme est bien formé et bien typé. Un compilateur attrape les fautes de syntaxe et de type, pas les erreurs d'algorithme : les tests restent indispensables, quel que soit le langage.

### Exercice 11 *Mille tests au hasard*

▶ **Corrigé.** [ au-delà du programme ] **1.** L'oracle, puis **2.** les 1 000 tirages :

```
import random

def verifie_arrondi(r, x):
    """Renvoie True si r est un multiple de 0.5 a au plus 0.25 de x."""
    return int(r * 2) == r * 2 and r - x <= 0.25 and x - r <= 0.25

for i in range(1000):
    x = random.randint(0, 2000) / 100
    assert verifie_arrondi(arrondi_demi(x), x)
```

**2.** La version corrigée passe les 1 000 tirages.
**3.** Pour la version d'origine, copiée sous le nom `arrondi_demi_origine`, on compte les échecs au lieu de s'arrêter au premier :

```
nb_echecs = 0
for i in range(1000):
    x = random.randint(0, 2000) / 100
    if not verifie_arrondi(arrondi_demi_origine(x), x):
        nb_echecs = nb_echecs + 1
print(nb_echecs)
```

Elle échoue sur **environ la moitié** des tirages (environ 480 sur 1 000, soit 48 % : exactement 960 des 2 001 valeurs possibles `k / 100`) : tous les `x` dont la partie décimale, comptée à partir du dernier demi-point, dépasse 0,25. Un seul tirage malchanceux suffit à la démasquer ; l'oracle juge sans connaître la réponse exacte.

## Bilan attendu

- **1.** « Le succès d'un jeu de tests ne garantit pas la correction d'un programme ». Point commun des sept tests : un seul cas, le plus « gentil », jamais de valeur limite ni de cas particulier du cahier des charges.
- **2.** Réponse personnelle ; attendus fréquents : les tests aux frontières (`nb_admis`, `rang`, `arrondi_demi`), l'affichage pour la boucle infinie, le variant pour l'expliquer.
- **3.** Non : on a seulement vérifié un nombre fini de cas. Pour une boucle, l'invariant et le variant apportent une vraie **preuve** ; pour le reste, un bon jeu de tests rend un bug *peu probable*, pas impossible.

## Réponses des questions de la fiche de révision

↑ questions p. 241

### Se tester

1. postcondition (une garantie sur le résultat) (§ I)
2. une `AssertionError` « vide » arrête le programme (§ III)
3. `0` au lieu de `3` : initialiser avec `t[0]` (§ V)
4. pas forcément : un cas non testé peut échouer (§ IV)
5. `t[-1]` est un indice valide (le dernier élément) : tester `r != -1` d'abord (§ III)

### Et avant ?

1. `1`, l'élément neutre de la multiplication (ch. 4, § II)
2. `00000001` inversé donne `11111110`, puis +1 : `11111111` (ch. 2, § VI.B)

CORRIGÉS DU CHAPITRE 6

# Les algorithmes de tri

6

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES



### ACTIVITÉ 1 Trier sans tricher

↑ énoncé p. 248

**Exercice 1** *Chacun sa méthode*

▶ **Solution.** **1–3.** Réponses variables : en général entre 15 et 30 comparaisons. On vérifie en retournant les cartes. Les méthodes inventées ressemblent presque toujours à l'une des deux méthodes de l'exercice 2 (chercher le plus petit, ou ranger au fur et à mesure) ; on peut aussi couper le paquet en deux, trier chaque moitié, puis les réunir : c'est une autre méthode, plus rapide, qu'on retrouvera plus tard.

**4.** Le minimum de la classe est souvent autour de 16 à 20. Une même méthode ne donne pas toujours le même nombre : avec la méthode « main de cartes », cela dépend de l'ordre de départ. *Remarque :* on ne peut pas garantir de trier 8 cartes en moins de 16 comparaisons, quelle que soit la méthode (il y a $8! = 40\,320$ ordres possibles, et chaque comparaison divise au mieux les possibilités par deux : $2^{15} < 40\,320 \leq 2^{16}$). Hors programme.

**Exercice 2** *Deux méthodes à l'essai*

▶ **Solution.** **5.** 7 comparaisons pour la plus petite des 8 (la championne affronte les 7 autres), puis 6, 5, …, 1 : en tout $7+6+5+4+3+2+1 = 28$.

**6.** Non : on fait toujours les 28 comparaisons, que les cartes soient mélangées, rangées ou à l'envers.

**7.** Cartes déjà rangées : chaque nouvelle carte est comparée à sa voisine de gauche, plus petite, et ne bouge pas : 7 comparaisons seulement.

**8.** Ordre inverse : chaque nouvelle carte doit remonter tout au début ; la $2^{\text{e}}$ carte fait 1 comparaison, la $3^{\text{e}}$ en fait 2, …, la $8^{\text{e}}$ en fait $7$ : $1+2+\cdots+7 = 28$.

**9.** La méthode 2 : sur des cartes presque rangées, elle ne fait que 7 ou 8 comparaisons, contre 28 pour la méthode 1, qui ne profite pas de l'ordre déjà présent.

**Exercice 3** *Ce qu'on a découvert*

▶ **Solution.** **10.** $15+14+\cdots+1 = \frac{16\times 15}{2} = 120$ comparaisons, environ **4 fois** plus que 28 : doubler le nombre de cartes quadruple (à peu près) le travail.

**11.** « Trier, c'est réorganiser les éléments pour qu'ils soient **dans l'ordre croissant**, en n'utilisant que deux opérations : **comparer** deux éléments et les **échanger** (ou les déplacer). Pour savoir si une méthode est rapide, on compte **les comparaisons**. »

# CORRIGÉS DES EXERCICES

> **Remarque**
> On donne **une** version simple ; d'autres écritures sont possibles. Tous les tris se font **en place**.

## Le tri par sélection

**Exercice 1** *Trace du tri par sélection* ↑énoncé p. 259

▶ **Solution.** Tableau montré **au début de chaque étape**, donc avant l'échange : l'état après l'échange de l'étape `i` est celui de la ligne suivante (la ligne « arrivée » pour la dernière étape). Zone grisée déjà triée, et en orange le minimum de la zone restante, amené à sa place par un échange.

| étape | tableau | | | | | | action |
|---|---|---|---|---|---|---|---|
| i=0 | 7 | 2 | 5 | 3 | 8 | 1 | min = 1 (case 5) → échange avec la case 0 |
| i=1 | 1 | 2 | 5 | 3 | 8 | 7 | min = 2 : déjà en place |
| i=2 | 1 | 2 | 5 | 3 | 8 | 7 | min = 3 (case 3) → échange avec la case 2 |
| i=3 | 1 | 2 | 3 | 5 | 8 | 7 | min = 5 : déjà en place |
| i=4 | 1 | 2 | 3 | 5 | 8 | 7 | min = 7 (case 5) → échange avec la case 4 |
| arrivée | 1 | 2 | 3 | 5 | 7 | 8 | tableau trié |

▷ **Erreur fréquente.** chercher le minimum dans tout le tableau à chaque étape : on retrouverait toujours `1`. Il faut le chercher seulement dans la partie non triée, de la case `i` à la fin.

**Exercice 2** *L'ingrédient : trouver le minimum* ↑énoncé p. 259

▶ **Solution.** Position du champion : on retient l'indice du plus petit élément vu, en partant de la case `debut`, et l'on compare les cases suivantes.

```
1  def indice_min(t, debut):
2      i_min = debut                       # INITIALISER : champion provisoire
3      for j in range(debut + 1, len(t)):  # PARCOURIR : la portion t[debut:]
4          if t[j] < t[i_min]:             # TESTER
5              i_min = j                   # METTRE A JOUR
6      return i_min                        # RENVOYER
```

▷ **Erreur fréquente.** initialiser `i_min = 0` au lieu de `debut` : la case `0`, hors de la portion étudiée, peut être renvoyée.

**Exercice 3** *Remettre le tri par sélection dans l'ordre* ↑énoncé p. 259

▶ **Solution.** On retrouve les deux temps du cours : chercher le minimum de `t[i:]`, puis l'échanger avec `t[i]`.

```
1  def tri_selection(t):
2      n = len(t)
3      for i in range(n):                  # PARCOURIR : chaque position i
4          i_min = i                       # INITIALISER : champion provisoire
5          for j in range(i + 1, n):       # PARCOURIR : la partie non triee
6              if t[j] < t[i_min]:         # TESTER
7                  i_min = j               # METTRE A JOUR
8          temp = t[i]                     # METTRE A JOUR : echange de t[i] et t[i_min]
9          t[i] = t[i_min]
10         t[i_min] = temp
```

À noter : `i_min = i` se place *avant* la boucle sur `j` (dans la boucle sur `i`) ; les trois lignes de l'échange sont *après* la boucle sur `j`, au même niveau que `i_min = i`. Avec `[5, 3, 8, 1]`, on obtient bien `[1, 3, 5, 8]`.

**Exercice 4** *Le tri par sélection* ↑énoncé p. 259

**▶ Solution.** *Appels attendus, écrits avant de coder (le tri modifie le tableau en place) :*

```
t = [5, 3, 8, 1]
tri_selection(t)
assert t == [1, 3, 5, 8]  # cas normal
t = []
tri_selection(t)
assert t == []            # cas limite : tableau vide
t = [4, 2, 4, 1]
tri_selection(t)
assert t == [1, 2, 4, 4]  # cas piege : une valeur en double
```

```
def tri_selection(t):
    n = len(t)
    for i in range(n):            # PARCOURIR : chaque position i
        i_min = indice_min(t, i)  # reutilise l'exercice precedent
        temp = t[i]               # METTRE A JOUR : echange via variable temporaire
        t[i] = t[i_min]
        t[i_min] = temp
```

Sur un tableau vide (`n = 0`), la boucle ne s'exécute pas : rien à trier, c'est correct. Sur un tableau déjà trié, chaque `indice_min` renvoie `i` : les échanges sont neutres, le tableau reste trié.

▷ **Erreur fréquente.** échanger sans variable temporaire, `t[i] = t[i_min]` puis `t[i_min] = t[i]` : après la première ligne, l'ancienne valeur de `t[i]` est perdue et le minimum se retrouve en double.

**Exercice 5** *Dans l'autre sens* ↑énoncé p. 260

**▶ Solution.** *Appels attendus, écrits avant de coder (le tri modifie le tableau en place) :*

```
t = [3, 8, 1, 5]
tri_selection_decroissant(t)
assert t == [8, 5, 3, 1]  # cas normal
t = [7]
tri_selection_decroissant(t)
assert t == [7]           # cas limite : un seul element
t = [1, 2, 3]
tri_selection_decroissant(t)
assert t == [3, 2, 1]     # cas piege : deja trie, mais dans l'autre sens
```

On cherche le **maximum** au lieu du minimum : le seul changement est le sens de la comparaison (`>` au lieu de `<`).

```
def tri_selection_decroissant(t):
    n = len(t)
    for i in range(n):              # PARCOURIR
        i_max = i                   # INITIALISER
        for j in range(i + 1, n):   # PARCOURIR
            if t[j] > t[i_max]:     # TESTER : seul changement, > au lieu de <
                i_max = j           # METTRE A JOUR
        temp = t[i]                 # METTRE A JOUR : echange
        t[i] = t[i_max]
        t[i_max] = temp
```

## Le tri par insertion

**Exercice 6** *Trace du tri par insertion* ↑énoncé p. 260

▶ **Solution.** Tableau montré **après chaque insertion** : l'élément orange est celui qu'on vient d'insérer dans la partie gauche triée.

| | tableau après insertion | | | | | |
|---|---|---|---|---|---|---|
| départ | 7 | 2 | 5 | 3 | 8 | 1 |
| insérer 2 | 2 | 7 | 5 | 3 | 8 | 1 |
| insérer 5 | 2 | 5 | 7 | 3 | 8 | 1 |
| insérer 3 | 2 | 3 | 5 | 7 | 8 | 1 |
| insérer 8 | 2 | 3 | 5 | 7 | 8 | 1 |
| insérer 1 | 1 | 2 | 3 | 5 | 7 | 8 |

▷ **Erreur fréquente.** confondre avec la sélection : en insertion, la partie gauche est triée, mais ses éléments ne sont pas forcément à leur place définitive (le `1`, inséré en dernier, décale tous les autres).

**Exercice 7** *Compléter le tri par insertion* ↑énoncé p. 260

▶ **Solution.** Les cinq trous, dans l'ordre : `n`, `i - 1`, `t[j] > x`, `t[j]`, `j + 1`.

```
def tri_insertion(t):
    n = len(t)
    for i in range(1, n):           # PARCOURIR
        x = t[i]                    # INITIALISER : l'element a inserer
        j = i - 1                   # INITIALISER
        while j >= 0 and t[j] > x:  # PARCOURIR vers la gauche, tant que plus grand
            t[j + 1] = t[j]         # METTRE A JOUR : decalage a droite
            j = j - 1               # METTRE A JOUR
        t[j + 1] = x                # METTRE A JOUR : x pose dans le trou
```

Sur `[4, 2, 5, 1]`, on obtient `[1, 2, 4, 5]`. On pose `x` en `t[j + 1]` car la boucle s'est arrêtée sur une case `j` qui *ne doit pas* bouger (ou sur `j = -1`).

▷ **Erreur fréquente.** poser `x` en `t[j]` : on écrase l'élément qui a arrêté la boucle, ou bien, pour `j = -1`, on écrit dans la dernière case du tableau.

**Exercice 8** *Qui a fait quoi ?* ↑énoncé p. 260

▶ **Solution.** On **ne peut pas** être certain. La partie gauche `[1, 2, 3]` est triée dans les deux cas. Mais l'indice décisif est ceci : en tri par **sélection**, la partie triée de gauche contient *à coup sûr les plus petits éléments du tableau, à leur place définitive*. Ici `[1, 2, 3]` sont bien les trois plus petits : c'est **compatible** avec une sélection (3 étapes faites). Mais c'est *aussi* compatible avec une insertion, si le tableau initial commençait déjà par de petites valeurs. Sans connaître le tableau de départ, on ne peut donc pas trancher. *(En revanche, si la partie gauche triée contenait une valeur qu'on sait **ne pas** être parmi les plus petites, ce serait forcément une insertion.)*

**Exercice 9** *Le tri par insertion* ↑énoncé p. 260

▶ **Solution.** *Appels attendus, écrits avant de coder (le tri modifie le tableau en place) :*

```
t = [5, 3, 8, 1, 9, 2]
tri_insertion(t)
assert t == [1, 2, 3, 5, 8, 9]  # cas normal
t = []
tri_insertion(t)
assert t == []                  # cas limite : tableau vide
t = [9, 7, 5, 3]
tri_insertion(t)
assert t == [3, 5, 7, 9]        # cas piege : chaque x recule jusqu'a la case 0
```

Pour chaque `i` à partir de `1` : on met `t[i]` de côté dans `x`, on décale vers la droite les éléments plus grands que `x`, puis on pose `x` dans le trou laissé.

```
def tri_insertion(t):
    n = len(t)
    for i in range(1, n):           # PARCOURIR
        x = t[i]                    # INITIALISER : l'element a inserer
        j = i - 1                   # INITIALISER
        while j >= 0 and t[j] > x:  # PARCOURIR vers la gauche, tant que plus grand
            t[j + 1] = t[j]         # METTRE A JOUR : decalage a droite
            j = j - 1               # METTRE A JOUR
        t[j + 1] = x                # METTRE A JOUR : x pose dans le trou
```

**Exercice 10** *Compter les décalages* ↑énoncé p. 260

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert tri_insertion_compte([4, 1, 3, 2]) == 4  # cas normal
assert tri_insertion_compte([1, 2, 3]) == 0     # cas limite : deja trie
assert tri_insertion_compte([2, 2, 1]) == 2     # cas piege : 2 egal a 2 ne se decale pas
```

On ajoute un compteur incrémenté à chaque décalage.

```
def tri_insertion_compte(t):
    n = len(t)
    nb = 0                          # INITIALISER : compteur de decalages
    for i in range(1, n):           # PARCOURIR
        x = t[i]                    # INITIALISER
        j = i - 1                   # INITIALISER
        while j >= 0 and t[j] > x:  # PARCOURIR vers la gauche
            t[j + 1] = t[j]         # METTRE A JOUR : decalage
            nb = nb + 1             # METTRE A JOUR : on le compte
            j = j - 1               # METTRE A JOUR
        t[j + 1] = x                # METTRE A JOUR
    return nb                       # RENVOYER
```

**1.** Sur `[1, 2, 3, 4, 5]` : **0** décalage — la condition `t[j] > x` est fausse d'emblée à chaque étape. C'est le *meilleur cas*, coût linéaire.
**2.** Sur `[5, 4, 3, 2, 1]` : **10** décalages, soit $\frac{n(n-1)}{2}$ pour $n = 5$. Chaque nouvel élément doit remonter tout au début : c'est le *pire cas*, coût quadratique.

## Le coût d'un tri

**Exercice 11** *Linéaire ou quadratique ?* ↑énoncé p. 261

**▶ Solution. 1. Vrai.** Le tri par sélection compare toujours $\frac{n(n-1)}{2}$ paires, que le tableau soit trié ou non : il ne « profite » jamais d'un tableau déjà ordonné.
**2. Vrai.** Sur un tableau déjà trié, la boucle `while` de l'insertion ne décale rien : coût linéaire, bien meilleur.
**3. Faux.** Un coût quadratique varie comme $n^2$ : doubler $n$ multiplie le temps par **4**, pas par 2.

▷ **Erreur fréquente.** raisonner comme pour un coût linéaire : $(2n)^2 = 4n^2$, le temps est multiplié par 4 ; pour un tableau 10 fois plus grand, il l'est par 100.

**Exercice 12** *Compter les comparaisons de la sélection* ↑énoncé p. 261

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert nb_comparaisons_selection([5, 3, 8, 1, 9]) == 10  # cas normal
assert nb_comparaisons_selection([7]) == 0               # cas limite : un seul element
assert nb_comparaisons_selection([1, 2, 3, 4, 5]) == 10  # cas piege : deja trie, autant
```

```
def nb_comparaisons_selection(t):
    n = len(t)
    c = 0                           # INITIALISER : compteur
```

```
    for i in range(n):                # PARCOURIR
        i_min = i                     # INITIALISER
        for j in range(i + 1, n):     # PARCOURIR
            c = c + 1                 # METTRE A JOUR : une comparaison t[j] < t[i_min]
            if t[j] < t[i_min]:       # TESTER
                i_min = j             # METTRE A JOUR
        temp = t[i]                   # METTRE A JOUR : echange
        t[i] = t[i_min]
        t[i_min] = temp
    return c                          # RENVOYER
```

**1.** Pour $n = 5$ : $\frac{5\times4}{2} = 10$ comparaisons. Pour $n = 10$ : $\frac{10\times9}{2} = 45$.

**2.** Quel que soit le contenu du tableau, on obtient toujours $\frac{n(n-1)}{2}$ : le nombre de comparaisons ne dépend que de la **taille**, ce qui confirme le coût quadratique.

**Exercice 13** *Vérifier qu'un tableau est trié* ↑énoncé p. 261

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert est_trie([2, 5, 9]) == True   # cas normal
assert est_trie([]) == True          # cas limite : tableau vide
assert est_trie([4, 4, 6]) == True   # cas piege : deux egaux, pas une descente
```

```
def est_trie(t):
    for i in range(len(t) - 1):   # PARCOURIR : jusqu'a l'avant-derniere case
        if t[i] > t[i + 1]:       # TESTER : une descente, pas trie
            return False          # RENVOYER
    return True                   # RENVOYER : aucune descente
```

▷ **Erreur fréquente.** écrire `range(len(t))` : au dernier tour, `t[i + 1]` n'existe pas et Python lève une `IndexError`.

Coût **linéaire** (un seul parcours), avec arrêt anticipé dès la première descente. Elle sert de **test** : après avoir appliqué un tri, on peut vérifier que `est_trie(t)` vaut `True` et que `t` contient bien les mêmes éléments qu'au départ.

## Exercice guidé : le tri à bulles

**Exercice 14** *Construire le tri à bulles* ↑énoncé p. 261

▶ **Solution.** *Appels attendus pour* `tri_bulle`*, écrits avant de coder :*

```
t = [5, 3, 8, 1, 9, 2]
tri_bulle(t)
assert t == [1, 2, 3, 5, 8, 9]  # cas normal
t = [7]
tri_bulle(t)
assert t == [7]                 # cas limite : un seul element
t = [9, 7, 5, 3]
tri_bulle(t)
assert t == [3, 5, 7, 9]        # cas piege : trie a l'envers
```

**1.** Un passage compare chaque couple de voisins et les remet en ordre :

```
def un_passage(t):
    for j in range(len(t) - 1):  # PARCOURIR : les couples de voisins
        if t[j] > t[j + 1]:      # TESTER
            temp = t[j]          # METTRE A JOUR : echange via variable temporaire
            t[j] = t[j + 1]
            t[j + 1] = temp
```

Sur `[5, 3, 8, 1, 9, 2]`, un passage donne `[3, 5, 1, 8, 2, 9]` : le **9** (le plus grand) se retrouve en dernière position.

**2.** Le plus grand élément, dès qu'il est atteint par le parcours, est plus grand que son voisin de droite : il est donc échangé, et « poussé » d'un cran vers la droite à *chaque* comparaison suivante. Il glisse ainsi jusqu'au bout : après un passage complet, il est forcément tout à droite.

**3.** Puisque chaque passage place au moins un élément (le plus grand restant) à sa position définitive à droite, $n-1$ passages suffisent. Le $i$-ème passage peut s'arrêter plus tôt (la fin est déjà triée) :

```
def tri_bulle(t):
    n = len(t)
    for i in range(n - 1):          # PARCOURIR : n - 1 passages
        for j in range(n - 1 - i):  # PARCOURIR : la fin, deja triee, est ignoree
            if t[j] > t[j + 1]:     # TESTER
                temp = t[j]         # METTRE A JOUR : echange
                t[j] = t[j + 1]
                t[j + 1] = temp
```

**4.** [ au-delà du programme ] Avec un drapeau `echange` :

```
def tri_bulle_optimise(t):
    n = len(t)
    for i in range(n - 1):          # PARCOURIR
        echange = False             # INITIALISER : aucun echange pour l'instant
        for j in range(n - 1 - i):  # PARCOURIR
            if t[j] > t[j + 1]:     # TESTER
                temp = t[j]         # METTRE A JOUR : echange
                t[j] = t[j + 1]
                t[j + 1] = temp
                echange = True      # METTRE A JOUR
        if not echange:             # TESTER : aucun echange, c'est deja trie
            return
```

Sur un tableau déjà trié, le **premier** passage ne fait aucun échange : la fonction s'arrête après **un seul** passage (coût linéaire).

## Synthèse : vers le bac et au-delà

**Exercice 15** *Classement d'une compétition* ↑énoncé p. 262

▶ **Solution.** *Appels attendus pour* `classer`*, écrits avant de coder :*

```
r = [("Ana", 53), ("Bob", 47), ("Cid", 61)]
assert classer(r) == [("Bob", 47), ("Ana", 53), ("Cid", 61)]              # cas normal
assert classer([]) == []                                                  # cas limite : aucun
                                                                          # concurrent
assert classer([("Ana", 50), ("Zoe", 47)]) == [("Zoe", 47), ("Ana", 50)]  # cas piege : par
                                                                          # temps, pas par nom
```

**1.** On reprend le tri par sélection en comparant les **temps**, c'est-à-dire le second élément `resultats[j][1]` de chaque p-uplet :

```
def classer(resultats):
    n = len(resultats)
    for i in range(n):                              # PARCOURIR
        i_min = i                                   # INITIALISER
        for j in range(i + 1, n):                   # PARCOURIR
            if resultats[j][1] < resultats[i_min][1]:  # TESTER : on compare les temps
                i_min = j                           # METTRE A JOUR
        temp = resultats[i]                         # METTRE A JOUR : echange
        resultats[i] = resultats[i_min]
        resultats[i_min] = temp
    return resultats                                # RENVOYER
```

**2.** Une fois le tableau classé, les trois plus rapides occupent les cases 0, 1 et 2 ; on garde le **nom** (indice 0) de chacun, en construisant la liste par compréhension :

```
def podium(resultats):
    classer(resultats)
    return [resultats[i][0] for i in range(3)]
```

*Autre méthode :* écrire directement les trois noms, `return [resultats[0][0], resultats[1][0], resultats[2][0]]`.

**Exercice 16** *La médiane* ↑énoncé p. 262

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert mediane([7, 2, 9, 4, 5]) == 5       # cas normal
assert mediane([8]) == 8                   # cas limite : un seul element
t = [9, 1, 5]
assert mediane(t) == 5 and t == [9, 1, 5]  # cas piege : t ne doit pas etre modifie
```

On trie une **copie** (pour ne pas modifier l'original) et on lit l'élément central.

```
def mediane(t):
    copie = [x for x in t]   # copie : l'original n'est pas modifie
    tri_selection(copie)
    return copie[len(copie) // 2]
```

*Autre méthode :* le tri par insertion convient tout aussi bien : `tri_insertion(copie)` à la place de `tri_selection(copie)`.

Une fois le tableau trié, la médiane est *simplement* l'élément du milieu : le tri a fait tout le travail. (Sans tri, il faudrait un algorithme bien plus subtil.)

**Exercice 17** *Chasse aux doublons* ↑énoncé p. 262

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert a_un_doublon([4, 2, 7]) == False  # cas normal
assert a_un_doublon([]) == False         # cas limite : tableau vide
assert a_un_doublon([5, 1, 5]) == True   # cas piege : doublons non voisins au depart
```

Après tri, deux valeurs égales sont forcément **voisines** : un seul parcours suffit à les repérer.

```
def a_un_doublon(t):
    copie = [x for x in t]
    tri_selection(copie)              # cout quadratique
    for i in range(len(copie) - 1):   # PARCOURIR : les voisins
        if copie[i] == copie[i + 1]:  # TESTER
            return True               # RENVOYER
    return False                      # RENVOYER : aucun voisin egal
```

Une fois le tri effectué, le parcours de comparaison des voisins est **linéaire**. Le coût total est donc dominé par le tri : quadratique.

**Exercice 18** *Presque trié* ↑énoncé p. 262

**▶ Solution. 1.** Sur `[2, 1, 4, 3, 6, 5]`, le tri par **insertion** ne fait que **7** comparaisons (et **3** décalages : chaque petit élément recule d'une seule case) : le travail est proportionnel au **désordre**. Le tri par **sélection**, lui, refait ses $\frac{n(n-1)}{2} = 15$ comparaisons, sans profiter du bon ordre initial.
**2.** [ au-delà du programme ] On préfère nettement le tri par **insertion** pour des données presque en ordre : son coût s'approche du linéaire. C'est pour cela qu'il est utilisé comme brique dans des tris performants (comme Timsort) sur les portions déjà quasi triées.

**Exercice 19** *Les k plus petits* ↑énoncé p. 262

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert k_plus_petits([53, 47, 61, 50, 49, 58], 3) == [47, 49, 50]  # cas normal
assert k_plus_petits([4, 1], 0) == []                              # cas limite : k = 0
assert k_plus_petits([5, 1, 5, 3], 3) == [1, 3, 5]                 # cas piege : une valeur en double
```

**1.** D'après l'invariant du tri par sélection, après `k` étapes les `k` premières cases contiennent les `k` plus petits éléments, triés et à leur place définitive : on peut s'arrêter là.

```
def k_plus_petits(t, k):
    copie = [x for x in t]  # INITIALISER : on ne modifie pas t
    for i in range(k):      # PARCOURIR : k etapes seulement
```

```
        i_min = indice_min(copie, i)
        temp = copie[i]      # METTRE A JOUR : echange
        copie[i] = copie[i_min]
        copie[i_min] = temp
    return copie[0:k]        # RENVOYER
```

**2.** Les trois étapes font $99+98+97 = 294$ comparaisons, contre $\frac{100\times 99}{2} = 4\,950$ pour un tri complet : environ 17 fois moins. Le coût est de l'ordre de $k \times n$ : **linéaire** en $n$ quand $k$ est fixé.

### Exercice 20 — *Trier des mots par longueur* ↑énoncé p. 263

**▶ Solution.** *Appels attendus, écrits avant de coder (le tri modifie le tableau en place) :*

```
mots = ["pomme", "kiwi", "figue", "noix", "ananas"]
tri_longueur(mots)
assert mots == ["kiwi", "noix", "pomme", "figue", "ananas"]  # cas normal
mots = []
tri_longueur(mots)
assert mots == []                                            # cas limite : aucun mot
mots = ["cc", "a", "bb"]
tri_longueur(mots)
assert mots == ["a", "cc", "bb"]                             # cas piege : ex aequo, ordre de
                                                             # depart garde
```

**1.** Seule la comparaison change : on compare les longueurs.

```
def tri_longueur(mots):
    for i in range(1, len(mots)):                # PARCOURIR
        x = mots[i]                              # INITIALISER
        j = i - 1                                # INITIALISER
        while j >= 0 and len(mots[j]) > len(x):  # PARCOURIR vers la gauche
            mots[j + 1] = mots[j]                # METTRE A JOUR : decalage
            j = j - 1                            # METTRE A JOUR
        mots[j + 1] = x                          # METTRE A JOUR
```

**2.** Avec `>=`, un mot recule aussi derrière les mots de **même** longueur : `"noix"` passe devant `"kiwi"`, `"figue"` devant `"pomme"`. On obtient `['noix', 'kiwi', 'figue', 'pomme', 'ananas']` : trié par longueur, mais l'ordre de départ des ex æquo est inversé. Avec `>` strict, on s'arrête dès qu'on rencontre un mot de même longueur : l'ordre de départ est conservé (on dit que le tri est *stable*).
**3.** Non. Sur `["abc", "xyz", "a"]` : à l'étape 0, le plus court est `"a"`, échangé avec `"abc"` → `["a", "xyz", "abc"]`, et plus rien ne bouge ensuite. `"abc"` et `"xyz"` ont la même longueur, mais leur ordre de départ est inversé : l'échange a fait « sauter » `"abc"` par-dessus `"xyz"`.

### Exercice 21 — *Un tri selon un assistant d'IA* ↑énoncé p. 263

**▶ Solution.** **1.** Non. Sur `[5, 2, 4, 1]` : étape `i = 0`, minimum `1` (case 3), on écrit `t[0] = 1` → `[1, 2, 4, 1]` : le `5` a **disparu**, écrasé. Étape `i = 1` : minimum `1` (case 3) → `[1, 1, 4, 1]`. Étape `i = 2` : → `[1, 1, 1, 1]`. Étape `i = 3` : inchangé. Résultat `[1, 1, 1, 1]` : le tableau n'est pas trié, il a perdu ses éléments. **2.** L'erreur est la ligne `t[i] = t[i_min]` : on *recopie* le minimum au lieu de l'**échanger** avec `t[i]`, et la valeur qui était en `t[i]` est perdue. Correction (échange via une variable temporaire, comme dans le cours) :

```
        temp = t[i]
        t[i] = t[i_min]
        t[i_min] = temp
```

Avec cette correction, `[5, 2, 4, 1]` devient `[1, 2, 4, 5]`. **3.** Exécuter la fonction sur un petit tableau et **afficher le résultat** : `[1, 1, 1, 1]` saute aux yeux. Un tri ne doit jamais changer le *contenu* du tableau, seulement l'ordre : vérifier que toutes les valeurs sont encore là est le premier test à faire.

## Entraînement

**Exercice 22** *Dérouler le tri par sélection* ↑énoncé p. 263

**▶ Solution.** a) min `1` → `[1, 4, 3, 2]` ; min `2` → `[1, 2, 3, 4]` ; min `3`, déjà en place → `[1, 2, 3, 4]`. b) min `3` → `[3, 7, 5, 9]` ; min `5` → `[3, 5, 7, 9]` ; min `7`, déjà en place. c) min `1` → `[1, 3, 3, 2]` ; min `2` → `[1, 2, 3, 3]` ; min `3`, déjà en place.

**Exercice 23** *Dérouler le tri par insertion* ↑énoncé p. 263

**▶ Solution.** a) `[1, 4, 3, 2]`, `[1, 3, 4, 2]`, `[1, 2, 3, 4]` : $1+1+2=4$ décalages. b) `[1, 2, 4, 3]`, `[1, 2, 4, 3]`, `[1, 2, 3, 4]` : 1 décalage (tableau presque trié). c) `[7, 9, 5, 3]`, `[5, 7, 9, 3]`, `[3, 5, 7, 9]` : $1+2+3=6$ décalages, le pire cas pour 4 éléments.

## Prouver un tri

**Exercice 24** *Invariant et terminaison* ↑énoncé p. 264

**▶ Solution.** **1.** Sélection, tableau au début de chaque étape : `i = 1` : `[2, 4, 9, 6, 7]` ; `i = 2` : `[2, 4, 9, 6, 7]` ; `i = 3` : `[2, 4, 6, 9, 7]` ; `i = 4` : `[2, 4, 6, 7, 9]`. À chaque fois, `t[0..i-1]` (`[2]`, `[2, 4]`, `[2, 4, 6]`, `[2, 4, 6, 7]`) est triée et formée des `i` plus petites valeurs du tableau : l'invariant est vérifié.

**2.** Insertion, tableau au début de chaque étape : `i = 1` : `[6, 4, 9, 2, 7]` ; `i = 2` : `[4, 6, 9, 2, 7]` ; `i = 3` : `[4, 6, 9, 2, 7]` ; `i = 4` : `[2, 4, 6, 9, 7]` ; à la fin : `[2, 4, 6, 7, 9]`. À chaque fois, `t[0..i-1]` contient exactement les `i` premières valeurs de départ (`6` ; `6, 4` ; `6, 4, 9` ; `6, 4, 9, 2`), rangées dans l'ordre croissant : l'invariant est vérifié.

**3.** Non : `t[0..2] = [4, 6, 9]`, alors que les trois plus petits sont 2, 4, 6. Ce n'est pas une contradiction : l'invariant de l'insertion ne parle que des éléments *déjà rencontrés* (les trois premiers de départ), pas des plus petits du tableau ; le 2, encore à droite, viendra se glisser devant eux. C'est l'invariant de la **sélection** (« les `i` plus petits, à leur place définitive ») qui garantit qu'un élément placé ne bougera plus.

**4.** Le tri par sélection ne contient que deux boucles `for` : elles sont **bornées** (`n` tours pour l'externe, moins de `n` pour l'interne). Il n'y a aucune boucle qui pourrait tourner indéfiniment : le tri se termine toujours.

**5.** Dans la boucle `while j >= 0 and t[j] > x`, la quantité `j` est un **variant** : c'est un entier, positif ou nul tant que la boucle tourne (la condition impose `j >= 0`), et il diminue de 1 à chaque tour (`j = j - 1`). La boucle s'arrête donc. Comme `j` part de `i - 1`, elle fait au plus `i` tours à l'étape `i`. Sur `t` : 1 tour (étape 1, on décale 6), 0 (étape 2), 3 (étape 3, le 2 recule au début), 1 (étape 4, on décale 9). La boucle `for`, elle, est bornée : le tri par insertion se termine.

# CORRIGÉS DES TP

## TP Le banc d'essai des tris

↑énoncé p. 267

### 1 Les jeux d'essai

**Exercice 1** *Fabriquer des listes*

▶ Corrigé.

```
def triee(n):
    return [i for i in range(n)]

def inversee(n):
    return [i for i in range(n, 0, -1)]
```

Question 3 : la liste triée est le **meilleur cas** de l'insertion, la liste inversée son **pire cas**, la liste aléatoire un cas « moyen ». Ce sont aussi des cas limites où se cachent les bugs (indice $-1$, boucle qui ne s'arrête pas…) : un tri qui ne marche que sur des listes au hasard n'est pas testé.

### 2 Des tris qui comptent leur travail

**Exercice 2** *Le tri par sélection instrumenté*

▶ Corrigé.

```
def tri_selection_compte(t):
    comparaisons = 0
    echanges = 0
    n = len(t)
    for i in range(n):
        i_min = i
        for j in range(i + 1, n):
            comparaisons = comparaisons + 1
            if t[j] < t[i_min]:
                i_min = j
        if i_min != i:              # on ne compte que les vrais echanges
            temp = t[i]
            t[i] = t[i_min]
            t[i_min] = temp
            echanges = echanges + 1
    return comparaisons, echanges
```

**Exercice 3** *Le tri par insertion instrumenté*

▶ Corrigé.

```
def tri_insertion_compte(t):
    comparaisons = 0
    decalages = 0
    for i in range(1, len(t)):
        x = t[i]
        j = i - 1
        while j >= 0 and t[j] > x:
            comparaisons = comparaisons + 1
            t[j + 1] = t[j]
            decalages = decalages + 1
```

```
            j = j - 1
        if j >= 0:                    # le test t[j] > x qui a arrete la boucle
            comparaisons = comparaisons + 1
        t[j + 1] = x
    return comparaisons, decalages
```

Sur `[5, 3, 8, 1, 9, 2]` : 11 comparaisons et 8 décalages (insérer 3 : 1 décalage ; 8 : 0 ; 1 : 3 ; 9 : 0 ; 2 : 4).

## 3 Le banc d'essai : compter

**Exercice 4** *Lancer le banc*

▶ **Corrigé.** Résultats obtenus (les lignes « aléatoire » changent d'une exécution à l'autre, les autres sont exactes) :

| n | liste | sél. : comparaisons | sél. : échanges | ins. : comparaisons | ins. : décalages |
|---|---|---|---|---|---|
| 100 | aléatoire | 4 950 | 96 | 2 395 | 2 302 |
| | triée | 4 950 | 0 | 99 | 0 |
| | inversée | 4 950 | 50 | 4 950 | 4 950 |
| 200 | aléatoire | 19 900 | 193 | 9 625 | 9 430 |
| | triée | 19 900 | 0 | 199 | 0 |
| | inversée | 19 900 | 100 | 19 900 | 19 900 |
| 400 | aléatoire | 79 800 | 394 | 42 279 | 41 882 |
| | triée | 79 800 | 0 | 399 | 0 |
| | inversée | 79 800 | 200 | 79 800 | 79 800 |

**Exercice 5** *Faire parler les nombres*

▶ **Corrigé.**

1. Non : $4\,950 = \frac{100\times99}{2}$, $19\,900 = \frac{200\times199}{2}$, $79\,800 = \frac{400\times399}{2}$, quelle que soit la liste.
2. Sélection : $\times 4{,}0$ à chaque doublement ; insertion sur liste aléatoire : $\times 4{,}0$ puis $\times 4{,}4$ (le hasard fait varier). Multiplier par 4 quand $n$ double : coût **quadratique**.
3. $n-1$ comparaisons (99, 199, 399) et aucun décalage : coût **linéaire**.
4. Sur liste inversée, les deux font $\frac{n(n-1)}{2}$ comparaisons (et l'insertion autant de décalages) : c'est le pire cas de l'insertion. Sur liste aléatoire, un élément inséré dans une partie triée de taille $i$ recule *en moyenne* jusqu'au milieu, soit $i/2$ cases ; le total vaut environ $\frac{n^2}{4}$ au lieu de $\frac{n^2}{2}$.
5. Quand déplacer un élément coûte cher (gros enregistrements, écritures lentes sur un support) : la sélection fait au plus $n-1$ échanges, l'insertion de l'ordre de $n^2$ décalages.
6. `[6, 5, 4, 3, 2, 1]` → `[1, 5, 4, 3, 2, 6]` → `[1, 2, 4, 3, 5, 6]` → `[1, 2, 3, 4, 5, 6]`, puis plus rien à échanger : 3 échanges. Chaque échange place **deux** éléments à la fois (le minimum devant, et le grand élément qu'il remplace part à sa place symétrique) : la seconde moitié du travail est déjà faite.

## 4 Le chronomètre

**Exercice 6** *La course*

▶ **Corrigé.**

```
def chronometre(tri, t):
    copie = [x for x in t]          # on ne trie jamais deux fois la meme liste
    debut = time.perf_counter()
    tri(copie)
    return time.perf_counter() - debut
```

Sans copie, le premier tri chronométré trierait la liste `t` elle-même : le suivant recevrait une liste **déjà triée** (le meilleur cas de l'insertion), et la comparaison serait faussée.

Mesures réellement obtenues (ordinateur portable récent) :

| `n` | sélection (s) | insertion (s) | `sorted` (s) | sélection / `sorted` |
|---|---|---|---|---|
| 500 | 0,0120 | 0,0118 | 0,0000420 | $\approx 290$ |
| 1 000 | 0,0470 | 0,0432 | 0,0000780 | $\approx 600$ |
| 2 000 | 0,186 | 0,187 | 0,000170 | $\approx 1\,100$ |
| 4 000 | 0,746 | 0,730 | 0,000370 | $\approx 2\,000$ |

**Exercice 7** *Lire les temps*

▶ **Corrigé.**

1. Les deux tris : $\times 4$ environ (3,9 ; 4,0 ; 4,0) : quadratiques. `sorted` : $\times 2$ environ (1,9 ; 2,2 ; 2,2), un peu plus que 2 : c'est le coût $n \log_2 n$ évoqué dans le cours. L'écart avec nos tris **se creuse** à chaque doublement (dernière colonne).
2. Deux paraboles presque superposées pour la sélection et l'insertion ; `sorted` reste collée à l'axe.
3. Non, les temps sont presque égaux. À chaque tour de sa boucle, l'insertion fait plus que comparer : un décalage (une écriture dans le tableau), une décrémentation, un test `j >= 0`. Compter les comparaisons donne l'**ordre de grandeur** du coût, pas la durée exacte : deux algorithmes de même coût quadratique peuvent différer d'un facteur constant, dans un sens ou dans l'autre.
4. Sur `triee(4000)` : sélection 0,74 s, insertion 0,000 74 s, un rapport d'environ **1 000**. Cohérent : $\frac{n(n-1)}{2}$ comparaisons contre $n-1$, rapport $\frac{n}{2} = 2\,000$ en opérations.
5. Insertion sur `presque_triee(1000, k)` : 999, 1 004, 1 019, 1 095 comparaisons pour $k = 0$, 5, 20, 100 (la sélection : toujours 499 500). Le coût reste $\approx n + k$ : **linéaire** tant que le désordre est faible. Excellent choix pour une liste triée à laquelle on ajoute quelques éléments, ou des données qui arrivent presque dans l'ordre (horodatages).
6. $10^6 = 4\,000 \times 250$ : $0{,}75 \times 250^2 \approx 47\,000$ s, soit **environ 13 heures** — le cours ne mentait pas. `sorted` trie un million de valeurs en une fraction de seconde.

## 5 Voir le tri travailler

**Exercice 8** *Des barres dans la console*

▶ Corrigé.

```
def afficher_barres(t, i_actif):
    for k in range(len(t)):
        marque = ""
        if k == i_actif:
            marque = "  <--"
        print(t[k], "#" * t[k] + marque)
    print()

def tri_insertion_visu(t, pause):
    afficher_barres(t, -1)
    for i in range(1, len(t)):
        x = t[i]
        j = i - 1
        while j >= 0 and t[j] > x:
            t[j + 1] = t[j]
            j = j - 1
        t[j + 1] = x
        print("Étape", i, ": on insère", x)
        afficher_barres(t, j + 1)     # x vient d'etre pose en j + 1
        time.sleep(pause)

def tri_selection_visu(t, pause):          # question 4
    afficher_barres(t, -1)
    n = len(t)
    for i in range(n):
        i_min = i
        for j in range(i + 1, n):
            if t[j] < t[i_min]:
                i_min = j
        temp = t[i]
        t[i] = t[i_min]
        t[i_min] = temp
        print("Étape", i + 1, ": le minimum", t[i], "va en position", i)
        afficher_barres(t, i)
        time.sleep(pause)
```

Question 3 : après l'étape $i$, les cases `0` à `i` sont triées (mais des éléments plus petits arriveront encore). Question 4 : `tri_selection_visu` (code ci-dessus) est le tri du cours, avec un affichage après chaque échange ; à l'écran, la zone de gauche n'est **plus jamais modifiée**, alors qu'avec l'insertion les barres déjà rangées glissent encore vers la droite.

## 6 Défi : le classement en direct

**Exercice 9** *Insérer à sa place*

▶ Corrigé.

```
def inserer(classement, x):
    classement.append(x)
    j = len(classement) - 2
    comparaisons = 0
    while j >= 0 and classement[j] > x:
        comparaisons = comparaisons + 1
        classement[j + 1] = classement[j]
        j = j - 1
    if j >= 0:                        # le test qui a arrete la boucle
        comparaisons = comparaisons + 1
```

```
    classement[j + 1] = x
    return comparaisons

def direct_par_insertion(arrivees):
    classement = []
    total = 0
    for x in arrivees:
        total = total + inserer(classement, x)
    return classement, total
```

Sur `aleatoire(300)` : classements identiques ; **environ 21 500** comparaisons par insertion (la valeur exacte dépend du tirage) contre **4 499 950** en retriant (environ 200 fois plus).

Quand $k$ coureurs sont classés, insérer coûte au plus $k$ comparaisons (**linéaire** en $k$), alors que retrier par sélection en coûte toujours $\frac{(k+1)k}{2}$ (**quadratique** en $k$). Sur $n$ arrivées, on totalise de l'ordre de $\frac{n^2}{4}$ dans le premier cas (aléatoire) et exactement $1+3+6+\cdots=\frac{(n+1)n(n-1)}{6}$ dans le second : pour $n=300$, $\frac{301\times300\times299}{6}=4\,499\,950$, le nombre mesuré.

## Bilan attendu (exemple)

*Les deux tris sont quadratiques : quand la liste double, comparaisons et temps sont multipliés par 4 (nos mesures : 0,19 s puis 0,75 s pour 2 000 puis 4 000 valeurs). La sélection fait toujours $\frac{n(n-1)}{2}$ comparaisons ; l'insertion n'en fait que $n-1$ sur une liste triée et reste presque linéaire sur une liste presque triée : c'est elle qu'il faut choisir quand les données sont déjà en ordre ou arrivent une à une. En revanche, la sélection déplace très peu les éléments. En pratique,* `sorted` *est des centaines à des milliers de fois plus rapide et l'écart grandit avec $n$ : on l'utilise toujours. Limite : un chronomètre mesure aussi ce que fait l'ordinateur à côté, et des tris de même coût peuvent avoir des temps différents ; ce sont les rapports qui comptent.*

## Réponses des questions de la fiche de révision

↑ questions p. 273

### Se tester

1. Le tri par sélection (§ II)
2. `[1, 3, 4, 2]` (§ III.A)
3. $\frac{10\times9}{2}=45$, quel que soit l'ordre (§ IV)
4. L'ancienne valeur de `t[i]` est écrasée : il faut `temp` (§ I)
5. Insertion (presque aucun décalage : coût proche du linéaire) (§ IV.A)

### Et avant ?

1. oui : c'est un cas limite, souvent oublié (ch. 5, § IV)
2. `[4, 7, 9]` (ch. 3, § IV.D)

CORRIGÉS DU CHAPITRE 7

# La recherche dichotomique

7

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 Le nombre mystère

↑énoncé p. 280



**Exercice 1** *Entre 1 et 100*

▶ **Solution.** **1.** Réponses variables, typiquement entre 4 et 12 coups ; un joueur chanceux peut gagner en 1 coup. Ceux qui proposent 1, 2, 3… peuvent aller jusqu'à 100 coups.

**2.** La stratégie gagnante qui émerge presque toujours : « proposer le milieu, puis le milieu de ce qui reste ».

**Exercice 2** *La meilleure stratégie*

▶ **Solution.** **3.** **50** (le milieu) : quelle que soit la réponse, on élimine environ la moitié des nombres. Proposer 10, par exemple, ne laisse que 9 nombres si la réponse est « plus petit », mais 90 si elle est « plus grand » : au pire, c'est bien pire.

**4.** Au pire **50** nombres (de 51 à 100 ; il en reste 49 si c'est « plus petit »).

**5.** $100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$ : six coups pour réduire à un seul nombre, et un septième pour le proposer. Au pire **7 coups**.

**6.** Le maître du jeu malin obtient en général 7 coups (parfois 6), jamais plus : la meilleure stratégie **garantit** 7 coups, quel que soit le secret. *Remarque : on raisonne sur le* ***pire cas****, comme on le fera pour le coût d'un algorithme.*

**Exercice 3** *Entre 1 et 1000*

▶ **Solution.** **7.** On est tenté de répondre 70 (« dix fois plus de nombres, dix fois plus de coups ») : c'est l'intuition « linéaire », et elle est fausse. La bonne prédiction est **10**.

**8.** On constate 10 coups au plus (9 ou 10 contre le maître du jeu malin).

**9.** Coups au pire : $100 \to 7$ ; $200 \to 8$ ; $1\,000 \to 10$ ; $2\,000 \to 11$ ; $1\,000\,000 \to 20$. Doubler le nombre de possibilités ajoute **un seul coup** ; le multiplier par 1 000 (environ $2^{10}$) n'en ajoute que **10**.

**Exercice 4** *Ce qu'on a découvert*

▶ **Solution.** **10.** « La meilleure stratégie consiste à proposer le **milieu** des nombres encore possibles : chaque coup **divise** le nombre de possibilités par **deux**. Avec $k$ coups, on peut départager jusqu'à $2^k - 1$ nombres (le dernier coup doit proposer le bon nombre) : comme $2^{10} - 1 = \mathbf{1\,023}$, il suffit de **10** coups pour 1 000 nombres. » Ce nombre de coups, le nombre de fois qu'on peut diviser par 2 avant d'arriver à 1, s'appelle le **logarithme en base 2** : $\log_2(1\,000) \approx 10$.

**11.** Non : avec seulement « gagné / perdu », on n'apprend rien sur la position du secret et il faut essayer les nombres un par un (jusqu'à 1 000 coups). Ce qui rend la stratégie possible, c'est que les nombres sont **rangés dans l'ordre** : la réponse « plus grand / plus petit » permet d'éliminer d'un coup toute une moitié.

# CORRIGÉS DES EXERCICES

> **Remarque**
> On donne **une** version simple ; d'autres écritures sont possibles. On note `t = [3, 8, 11, 16, 19, 24, 30, 33, 38, 47, 52]`.

## Dérouler la recherche à la main

**Exercice 1** *Un élément présent* ↑énoncé p. 288

▶ **Solution.** Recherche de `x = 33` :

| tour | `deb` | `fin` | `mil` | `t[mil]` | décision |
|---|---|---|---|---|---|
| 1 | 0 | 10 | 5 | 24 | $33 > 24$ : à droite (`deb` $= 6$) |
| 2 | 6 | 10 | 8 | 38 | $33 < 38$ : à gauche (`fin` $= 7$) |
| 3 | 6 | 7 | 6 | 30 | $33 > 30$ : à droite (`deb` $= 7$) |
| 4 | 7 | 7 | 7 | 33 | $33 = 33$ : **trouvé** (indice 7) |

Trouvé en **4 tours**.

▷ **Erreur fréquente.** calculer le milieu avec `(deb + fin) / 2` : on obtient un flottant (`5.0`), et `t[5.0]` provoque une `TypeError`. Le milieu s'obtient avec la division entière `//`.

**Exercice 2** *Un élément absent* ↑énoncé p. 288

▶ **Solution.** Recherche de `x = 20` :

| tour | `deb` | `fin` | `mil` | `t[mil]` | décision |
|---|---|---|---|---|---|
| 1 | 0 | 10 | 5 | 24 | $20 < 24$ : à gauche (`fin` $= 4$) |
| 2 | 0 | 4 | 2 | 11 | $20 > 11$ : à droite (`deb` $= 3$) |
| 3 | 3 | 4 | 3 | 16 | $20 > 16$ : à droite (`deb` $= 4$) |
| 4 | 4 | 4 | 4 | 19 | $20 > 19$ : à droite (`deb` $= 5$) |

Maintenant `deb = 5 > fin = 4` : la zone de recherche est **vide**, la boucle s'arrête et l'algorithme renvoie `False`. 20 est absent (il aurait dû se trouver entre 19 et 24).

▷ **Erreur fréquente.** s'arrêter dès que `deb == fin` : il reste alors une case à examiner (ici 19, au tour 4). La boucle continue tant que `deb <= fin`.

**Exercice 3** *Le rôle du tri* ↑énoncé p. 288

▶ **Solution.** **1.** Sur `[9, 7, 5, 3, 1]`, recherche de `x = 9` :

| tour | `deb` | `fin` | `mil` | `t[mil]` | décision |
|---|---|---|---|---|---|
| 1 | 0 | 4 | 2 | 5 | $9 > 5$ : à droite (`deb` $= 3$) |
| 2 | 3 | 4 | 3 | 3 | $9 > 3$ : à droite (`deb` $= 4$) |
| 3 | 4 | 4 | 4 | 1 | $9 > 1$ : à droite (`deb` $= 5$) |

`deb = 5 > fin = 4` : l'algorithme renvoie `False`.

**2.** Or `9` est bel et bien présent (en tête !). L'algorithme le rate parce qu'il **suppose** que « à droite se trouvent les plus grands » : c'est faux ici, puisque le tableau n'est pas trié. Chaque décision « à gauche / à droite » n'a de sens que sur un tableau **trié**. Sans tri, la dichotomie donne un résultat *faux*.

## Écrire l'algorithme

**Exercice 4** *Compléter la recherche dichotomique* ↑énoncé p. 288

▶ **Solution.** Les six trous, dans l'ordre : `len(t) - 1`, `deb <= fin`, `(deb + fin) // 2`, `mil + 1`, `mil - 1`, `False`.

```
def recherche_dichotomique(t, x):
    deb = 0                # INITIALISER : premier indice
    fin = len(t) - 1       # INITIALISER : dernier indice
    while deb <= fin:      # PARCOURIR : tant que la zone n'est pas vide
        mil = (deb + fin) // 2
        if t[mil] == x:    # TESTER
            return True    # RENVOYER : trouve
        elif x > t[mil]:   # TESTER
            deb = mil + 1  # METTRE A JOUR : moitie droite
        else:
            fin = mil - 1  # METTRE A JOUR : moitie gauche
    return False           # RENVOYER : zone vide, x absent
```

Avec `t = [3, 8, 11, 16, 19, 24, 30, 33, 38, 47, 52]`, l'appel avec `24` renvoie `True` et l'appel avec `10` renvoie `False`. À noter : `fin` est le **dernier indice** (`len(t) - 1`, pas `len(t)`) ; on écrit `mil + 1` et `mil - 1` car `mil` vient d'être testé.

▷ **Erreur fréquente.** écrire `deb = mil` au lieu de `deb = mil + 1` : sur `[1, 3]` avec `x = 3`, `mil` vaut `0` à chaque tour, `deb` reste à `0` et la boucle ne s'arrête jamais.

**Exercice 5** *La recherche dichotomique* ↑énoncé p. 289

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert recherche_dichotomique([3, 8, 11, 16, 19], 16) == True  # cas normal
assert recherche_dichotomique([], 5) == False                  # cas limite : tableau vide
assert recherche_dichotomique([3, 8, 11, 16, 19], 19) == True  # cas piege : x dans la
                                                               # derniere case
```

La zone de recherche va de `deb` à `fin`. Tant qu'elle n'est pas vide, on compare `x` au milieu : trouvé, on renvoie `True` ; sinon on garde la moitié où `x` peut encore être. Zone vide : `x` est absent.

```
def recherche_dichotomique(t, x):
    deb = 0                # INITIALISER : premier indice
    fin = len(t) - 1       # INITIALISER : dernier indice
    while deb <= fin:      # PARCOURIR : tant que la zone n'est pas vide
        mil = (deb + fin) // 2
        if t[mil] == x:    # TESTER
            return True    # RENVOYER : trouve
        elif x > t[mil]:   # TESTER
            deb = mil + 1  # METTRE A JOUR : moitie droite
        else:
            fin = mil - 1  # METTRE A JOUR : moitie gauche
    return False           # RENVOYER : zone vide, x absent
```

`recherche_dichotomique(t, 38)` renvoie `True` ; `recherche_dichotomique(t, 25)` renvoie `False`.

**Exercice 6** *Renvoyer la position* ↑énoncé p. 289

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert indice_dichotomique([3, 8, 11, 16, 19], 16) == 3  # cas normal
assert indice_dichotomique([7], 7) == 0                  # cas limite : un seul element
assert indice_dichotomique([3, 8, 11], 9) == -1          # cas piege : absent, entre deux
                                                         # valeurs
```

On renvoie `mil` au lieu de `True`, et `-1` au lieu de `False`.

```
def indice_dichotomique(t, x):
    deb = 0                # INITIALISER
```

```
    fin = len(t) - 1        # INITIALISER
    while deb <= fin:       # PARCOURIR
        mil = (deb + fin) // 2
        if t[mil] == x:     # TESTER
            return mil      # RENVOYER : la position
        elif x > t[mil]:    # TESTER
            deb = mil + 1   # METTRE A JOUR
        else:
            fin = mil - 1   # METTRE A JOUR
    return -1               # RENVOYER : absent
```

**Exercice 7** *Compter les tours* ↑énoncé p. 289

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert nb_tours([3, 8, 11, 16, 19], 16) == 2  # cas normal
assert nb_tours([], 5) == 0                   # cas limite : tableau vide, aucun tour
assert nb_tours([3, 8, 11, 16, 19], 20) == 3  # cas piege : absent, on compte aussi
```

On ajoute à la recherche un compteur `n`, augmenté au début de chaque tour et renvoyé aux *deux* sorties (trouvé ou zone vide).

```
def nb_tours(t, x):
    deb = 0                 # INITIALISER
    fin = len(t) - 1        # INITIALISER
    n = 0                   # INITIALISER : le compteur de tours
    while deb <= fin:       # PARCOURIR
        n = n + 1           # METTRE A JOUR : un tour de plus
        mil = (deb + fin) // 2
        if t[mil] == x:     # TESTER
            return n        # RENVOYER : sortie "trouve"
        elif x > t[mil]:    # TESTER
            deb = mil + 1   # METTRE A JOUR
        else:
            fin = mil - 1   # METTRE A JOUR
    return n                # RENVOYER : sortie "zone vide"
```

**1.** Pour 7 dans `[0, 1, 1, 2, 3, 5, 8, 13, 21]` (9 éléments), on examine 3 (mil 4), 8 (mil 6), 5 (mil 5), puis la zone est vide : **3 tours**, et 7 est absent.

**2.** Par exemple, chercher 7 dans `[0, 1, 2, 3, 4, 5, 6, 7]` : on examine 3, 5, 6, 7 — soit **4** valeurs. *(Tout tableau de 8 à 15 éléments dont l'élément cherché est au « fond » convient.)*

**Exercice 8** *La première occurrence* ↑énoncé p. 289

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert premiere_occurrence([1, 3, 5, 5, 8], 5) == 2  # cas normal
assert premiere_occurrence([], 4) == -1              # cas limite : tableau vide
assert premiere_occurrence([4, 4, 4, 4], 4) == 0     # cas piege : que des x, la premiere est
                                                     # en tete
```

**1.** Quand on trouve `x`, on le **mémorise** comme candidat, puis on continue à chercher dans la moitié **gauche**, où peut se trouver une occurrence plus à gauche. La zone diminue toujours de moitié : le coût reste logarithmique.

```
def premiere_occurrence(t, x):
    deb = 0                 # INITIALISER
    fin = len(t) - 1        # INITIALISER
    resultat = -1           # INITIALISER : pas encore trouve
    while deb <= fin:       # PARCOURIR
        mil = (deb + fin) // 2
        if t[mil] == x:     # TESTER
            resultat = mil  # METTRE A JOUR : candidat, on cherche encore a gauche
            fin = mil - 1   # METTRE A JOUR
        elif x > t[mil]:    # TESTER
            deb = mil + 1   # METTRE A JOUR
```

```
        else:
            fin = mil - 1    # METTRE A JOUR
    return resultat          # RENVOYER
```

Sur `[2, 4, 4, 4, 7, 9]` avec `x = 4` : `mil = 2` (trouvé, `resultat = 2`, `fin = 1`), puis `mil = 0` (`2 < 4`, `deb = 1`), puis `mil = 1` (trouvé, `resultat = 1`, `fin = 0`) ; la zone est vide, on renvoie `1`.
**2.** Avec un tableau qui ne contient que des `x`, par exemple `[4] * 1000` : la dichotomie tombe sur le milieu (indice 499), puis il faut reculer case par case jusqu'à l'indice 0, soit environ $n/2$ pas. Le coût redevient **linéaire** dans le pire cas.

## La terminaison

**Exercice 9** IA IA IA *Pourquoi ça s'arrête* ↑énoncé p. 289

▶ **Solution.** **1.** À chaque tour qui ne renvoie pas `True` :

- cas « à droite » : `deb` devient `mil + 1` $>$ `deb` : la largeur `fin - deb` diminue ;
- cas « à gauche » : `fin` devient `mil - 1` $<$ `fin` : la largeur `fin - deb` diminue aussi.

Dans les deux cas, `fin - deb` **diminue strictement** (d'au moins 1).
**2.** Une quantité entière qui décroît strictement ne peut pas décroître indéfiniment : tôt ou tard `deb > fin`, la condition de la boucle devient fausse, l'algorithme se termine. Une telle quantité est un **variant de boucle**.

## Le coût logarithmique

**Exercice 10** IA IA IA *Combien d'étapes ?* ↑énoncé p. 290

▶ **Solution.** On compte combien de fois diviser la taille par 2 pour atteindre 1, puis on ajoute la comparaison faite sur la dernière case restante.
**1.** Au pire, la zone de recherche passe par $8 \to 4 \to 2 \to 1$ : **3** divisions, puis une dernière comparaison sur la case restante, soit **4** comparaisons au maximum (c'est `nb_tours_max(8)`, car $2^3 = 8 \leq 8 < 2^4$).
**2.** $16 \to 8 \to 4 \to 2 \to 1$ : **4** divisions, plus la dernière comparaison, soit **5** comparaisons au maximum.
**3.** Pour 1 000 : $\log_2(1000) \approx 10$, donc **une dizaine** de comparaisons (à comparer aux 1000 d'un parcours !).

▷ **Erreur fréquente.** répondre $n/2$ (4 pour 8 éléments, 500 pour 1000) : on ne divise pas la taille par 2 une seule fois, mais à *chaque* tour.

**Exercice 11** IA IA IA *Le plus petit* `k` *tel que* $2^k > n$ ↑énoncé p. 290

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert nb_tours_max(100) == 7  # cas normal
assert nb_tours_max(1) == 1    # cas limite : un seul element
assert nb_tours_max(8) == 4    # cas piege : 2**3 = 8 n'est pas > 8
```

On double une puissance de 2 tant qu'elle ne dépasse pas `n`, en comptant les doublements.

```
def nb_tours_max(n):
    k = 0                          # INITIALISER
    puissance = 1                  # INITIALISER : vaut 2**k
    while puissance <= n:          # PARCOURIR : tant que 2**k ne depasse pas n
        puissance = puissance * 2  # METTRE A JOUR
        k = k + 1                  # METTRE A JOUR
    return k                       # RENVOYER
```

`nb_tours_max(100)` vaut 7 (car $2^6 = 64 \leq 100 < 128 = 2^7$).

**Exercice 12** *Le pire cas, en vrai* ↑énoncé p. 290

**▶ Solution.**

```
1 for n in [100, 1000, 10000, 100000, 1000000]:
2     t = [0] * n
3     print(n, nb_tours(t, 1))   # 1 est absent : pire cas
```

On obtient environ `7, 10, 14, 17, 20`. À chaque fois que `n` est **multiplié par 10**, le nombre de tours n'augmente que de **3 ou 4** : la croissance est **logarithmique**, extraordinairement lente. Une recherche séquentielle, elle, ferait jusqu'à `n` comparaisons (donc ×10 à chaque fois).

## Le jeu du nombre mystère

**Exercice 13** *Deviner en un minimum d'essais* ↑énoncé p. 290

**▶ Solution. 1.** La meilleure première proposition est `50`, le **milieu** : quelle que soit la réponse, on élimine la moitié des possibilités (soit 1–49, soit 51–100).

▷ **Erreur fréquente.** proposer 1, puis 2, puis 3… : on n'élimine qu'un nombre par essai, et il faut jusqu'à 100 essais.

**2.** En proposant toujours le milieu, le nombre de possibilités restantes, au pire, passe par $100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$ : six essais pour réduire à un seul nombre possible, et un septième pour le proposer. **7 essais** suffisent donc dans le pire des cas. C'est exactement `nb_tours_max(100)` $= 7$ : le jeu *est* une recherche dichotomique.

**Exercice 14** *L'ordinateur devine : la preuve* ↑énoncé p. 290

**▶ Solution. 1.** C'est une recherche dichotomique : `mini` et `maxi` jouent le rôle de `deb` et `fin` (les bornes de la zone de recherche), `proposition` celui de `mil`. Le « tableau trié » est la suite des entiers `1, 2, ..., maxi`, rangés dans l'ordre croissant : la valeur à la position `i` est `i` elle-même, il n'est donc pas besoin de la stocker.

**2.** À chaque essai raté :

- cas « trop petit » : `mini` devient `proposition + 1`, et `proposition` $\geqslant$ `mini`, donc `mini` augmente d'au moins 1 ;
- cas « trop grand » : `maxi` devient `proposition - 1`, et `proposition` $\leqslant$ `maxi`, donc `maxi` diminue d'au moins 1.

Dans les deux cas, l'entier `maxi - mini` **diminue strictement**. Grâce à la précondition, le secret reste toujours entre `mini` et `maxi` (on n'écarte que des nombres trop petits ou trop grands) : la zone ne peut donc jamais devenir vide. Comme `maxi - mini` ne peut pas décroître indéfiniment en restant $\geqslant 0$, on atteint au plus tard `mini == maxi` : la proposition est alors le secret lui-même, `trouve` devient `True` et la boucle s'arrête. La quantité `maxi - mini` est un **variant de boucle**.

**3.** Le programme **ne s'arrête jamais**. Les propositions sont 50, 75, 88, 94, 97, 99, 100, toutes « trop petites » ; on arrive à `mini = 101` et `maxi = 100`. La zone est vide, mais la boucle continue car `trouve` reste `False` : `proposition` vaut `(101 + 100) // 2 = 100`, encore trop petite, `mini` reprend la valeur `101`… et rien ne change plus. Le variant ne décroît plus : la preuve de la question 2 utilisait la précondition, qui n'est pas respectée. La condition `while mini <= maxi` (le `while deb <= fin` du cours) arrête la boucle dès que la zone est vide, même si l'appel est incorrect ; on pourrait aussi vérifier la précondition par un `assert` en début de fonction.

**4.** Après chaque essai raté, le nombre de candidats restants est au moins divisé par deux. Dans le pire des cas, le nombre d'essais est donc le plus petit $k$ tel que $2^k >$ `maxi`, c'est-à-dire `nb_tours_max(maxi)` : **7** essais pour 100 ($2^6 = 64 \leqslant 100 < 128 = 2^7$), **10** pour 1000 ($2^{10} = 1024$) et **20** pour un million ($2^{20} = 1\,048\,576$). L'ordinateur naïf, lui, peut avoir besoin de `maxi` essais : 100, 1000, un million. Coût **logarithmique** contre coût **linéaire**, comme dans le cours.

**Exercice 15** *La racine carrée entière* ↑énoncé p. 291

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert racine_entiere(50) == 7  # cas normal
assert racine_entiere(0) == 0   # cas limite : n = 0
assert racine_entiere(49) == 7  # cas piege : carre parfait, 7 * 7 <= 49
```

On cherche, dans l'intervalle des entiers de `0` à `n`, le plus grand `r` tel que `r * r <= n`. Même schéma que pour la première occurrence : on mémorise le meilleur candidat et on continue à chercher du côté où il pourrait y en avoir un meilleur.

```
def racine_entiere(n):
    deb = 0                 # INITIALISER
    fin = n                 # INITIALISER
    r = 0                   # INITIALISER : meilleur candidat
    while deb <= fin:       # PARCOURIR
        mil = (deb + fin) // 2
        if mil * mil <= n:  # TESTER
            r = mil         # METTRE A JOUR : mil convient ; peut-etre plus grand ?
            deb = mil + 1   # METTRE A JOUR
        else:
            fin = mil - 1   # METTRE A JOUR : mil est trop grand
    return r                # RENVOYER
```

`racine_entiere(50)` et `racine_entiere(49)` renvoient 7.

*Autre méthode :* une recherche séquentielle, qui essaie `1, 2, 3`... tant que le carré suivant ne dépasse pas `n` :

```
def racine_entiere(n):
    r = 0                          # INITIALISER
    while (r + 1) * (r + 1) <= n:  # PARCOURIR : tant que le carre suivant convient
        r = r + 1                  # METTRE A JOUR
    return r                       # RENVOYER
```

Plus simple, mais bien plus lente : pour `n = 1000000`, l'intervalle contient environ $10^6$ entiers ; la dichotomie le réduit en environ **20** tours ($2^{20} \approx 10^6$), contre 1 000 essais en testant `0, 1, 2`... un par un jusqu'à `1000`.

**Exercice 16** *Le mot dans le dictionnaire* ↑énoncé p. 291

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert est_dans_dico(["chat", "chien", "loup", "ours"], "loup") == True  # cas normal
assert est_dans_dico([], "chat") == False                                 # cas limite :
                                                                          # dictionnaire vide
assert est_dans_dico(["chat", "chien", "loup"], "Chat") == False          # cas piege :
                                                                          # majuscule
```

**1.** L'algorithme est *identique* : les comparaisons `<`, `==` fonctionnent déjà sur les chaînes (ordre alphabétique).

```
def est_dans_dico(dico, mot):
    deb = 0                     # INITIALISER
    fin = len(dico) - 1         # INITIALISER
    while deb <= fin:           # PARCOURIR
        mil = (deb + fin) // 2
        if dico[mil] == mot:    # TESTER
            return True         # RENVOYER
        elif mot > dico[mil]:   # TESTER : ordre alphabetique
            deb = mil + 1       # METTRE A JOUR
        else:
            fin = mil - 1       # METTRE A JOUR
    return False                # RENVOYER
```

**2.** $\log_2(100\,000) \approx 16{,}6$, donc au plus **17** comparaisons pour un dictionnaire de cent mille mots. La puissance de la dichotomie !

*Autre méthode :* une recherche séquentielle, par exemple `return mot in dico`. Elle marche même sur un dictionnaire non trié, mais elle examine les mots un par un : jusqu'à **100 000** comparaisons au lieu de 17.

**Exercice 17** *La dichotomie selon un assistant d'IA* ↑énoncé p. 292

**▶ Solution. 1.** Non. Trace de `contient([3, 9, 1, 7], 1)` :

- `deb = 0`, `fin = 3`, `mil = 1`, `t[1] = 9` ; `1 > 9` est faux → `fin = 0` ;
- `deb = 0`, `fin = 0`, `mil = 0`, `t[0] = 3` ; `1 > 3` est faux → `fin = -1` ;
- `deb > fin` : la boucle s'arrête et la fonction renvoie **`False`**, alors que `1` est bien dans le tableau (case 2).

**2.** Le code est celui du cours et il est juste ; l'erreur est dans l'explication : « trié ou non ». La dichotomie **ne s'applique qu'à un tableau trié** : c'est parce que le tableau est trié qu'on peut éliminer une moitié entière sans la regarder. Correction : préciser la précondition (`t` trié, à écrire dans la docstring) ; si le tableau ne l'est pas, utiliser une recherche séquentielle, ou le trier d'abord. Sur `[1, 3, 7, 9]`, la même fonction renvoie bien `True`. **3.** Tester la fonction sur un petit tableau **non trié** contenant la valeur cherchée : un seul appel révèle le problème. Plus généralement, une affirmation du type « fonctionne dans tous les cas » appelle un contre-exemple.

## Entraînement

**Exercice 18** *Dérouler la dichotomie* ↑énoncé p. 292

**▶ Solution.** Lignes de la forme (`deb`, `fin`, `mil`, `t[mil]`) : a) (0, 8, 4, 13) à droite ; (5, 8, 6, 21) **trouvé** : `True` en 2 tours. b) (0, 8, 4, 13) à gauche ; (0, 3, 1, 5) à gauche ; (0, 0, 0, 2) **trouvé** : `True` en 3 tours. c) (0, 8, 4, 13) à droite ; (5, 8, 6, 21) à gauche ; (5, 5, 5, 18) à gauche ; puis `deb = 5 > fin = 4` : `False` en 3 tours (`14` aurait été entre `13` et `18`). d) (0, 8, 4, 13) à droite ; (5, 8, 6, 21) à droite ; (7, 8, 7, 26) à droite ; (8, 8, 8, 30) **trouvé** : `True` en 4 tours.

**Exercice 19** *Combien de valeurs examinées au plus ?* ↑énoncé p. 292

**▶ Solution.** a) $2^2 = 4 \leqslant 7 < 8 = 2^3$ : **3**. b) $8 \leqslant 15 < 16$ : **4**. c) $32 \leqslant 50 < 64$ : **6**. d) $128 \leqslant 200 < 256$ : **8**. e) $2^{19} = 524\,288 \leqslant 10^6 < 2^{20} = 1\,048\,576$ : **20**.

# CORRIGÉS DES TP

## TP L'annuaire et le devin ↑énoncé p. 294

## 1 Le devin

**Exercice 1** *Programmer le devin*

▶ Corrigé.

```
def devin(bas, haut):
    essais = 0
    while bas <= haut:
        proposition = (bas + haut) // 2
        essais = essais + 1
        reponse = input("Je propose " + str(proposition) + " (+, - ou =) ? ")
        if reponse == "=":
            print("Trouvé en", essais, "essais !")
            return essais
        elif reponse == "+":
            bas = proposition + 1
        else:
            haut = proposition - 1
    print("Impossible : vos réponses se contredisent. Tricheur !")
    return -1
```

2. Jamais plus de **7** essais entre 1 et 100 : chaque réponse `+` ou `-` divise au moins par deux la zone, et $2^7 = 128 > 100$ (après 6 réponses, il reste au plus une valeur).
3. Entre 1 et 1 000 000 : souvent 18 à 20 essais ; au pire **20**, car $2^{20} = 1\,048\,576 > 10^6$. Exemple réel (secret 2026) : 20 essais, propositions 500000, 250000, …, 2027, 2026.

**Exercice 2** *Démasquer le tricheur*

▶ Corrigé.

1. Propositions 5 (`+`, `bas = 6`), 8 (`-`, `haut = 7`), 6 (`+`, `bas = 7`), 7 (`-`, `haut = 6`) : `bas = 7 > haut = 6`, la boucle s'arrête. Sans la modification, la fonction se termine sans rien dire (elle renvoie `None`).
2. Ce sont les deux dernières lignes du code ci-dessus (après la boucle).
3. Si le joueur est honnête, son nombre est **toujours** dans `[bas ; haut]` : c'est vrai au départ, et chaque réponse vraie n'élimine que des valeurs impossibles (invariant). Une zone vide ne contient aucun nombre : l'invariant est violé, donc au moins une réponse était fausse.
4. `haut - bas` est un entier qui diminue strictement à chaque réponse `+` ou `-` (on écarte la proposition et toute une moitié) : c'est un **variant**. Il finit par devenir négatif, et la boucle s'arrête — que le joueur mente ou non.

## 2 L'annuaire

**Exercice 3** *Prise en main*

▶ Corrigé.

1. 16 000 abonnés, de `"ADAM Adam"` à `"VINCENT Zoe"`.
2. Code ci-dessous ; `est_trie(noms)` renvoie `True`. La dichotomie n'est **correcte** que sur un tableau trié : on vérifie l'hypothèse avant de s'en servir.
3. `"LE GALL Adam"` est **avant** `"LEBLANC Adam"` (indices 8 100 et 8 400) : les deux chaînes diffèrent au 3e caractère, une espace contre un `B`, et l'espace a un code plus petit.

```
def est_trie(t):
    for i in range(len(t) - 1):
        if t[i] > t[i + 1]:
            return False
    return True
```

**Exercice 4** *Deux façons de trouver un numéro*

▶ Corrigé.

```
def numero_sequentiel(noms, numeros, nom):
    comparaisons = 0
    for i in range(len(noms)):
        comparaisons = comparaisons + 1
        if noms[i] == nom:
            return numeros[i], comparaisons
    return None, comparaisons

def numero_dicho(noms, numeros, nom):
    comparaisons = 0
    deb = 0
    fin = len(noms) - 1
    while deb <= fin:
        mil = (deb + fin) // 2
        comparaisons = comparaisons + 1
        if noms[mil] == nom:
            return numeros[mil], comparaisons
        elif nom > noms[mil]:
            deb = mil + 1
        else:
            fin = mil - 1
    return None, comparaisons
```

## 3 Mesurer

**Exercice 5** *Quelques recherches à la loupe*

▶ Corrigé.

| nom cherché | séquentielle | dichotomique |
|---|---|---|
| `"ADAM Adam"` (le premier) | 1 | 13 |
| `"DUPONT Alice"` | 4 305 | 14 |
| `"MARTIN Hugo"` | 10 334 | 14 |
| `"VINCENT Zoe"` (le dernier) | 16 000 | 14 |
| `"ZORRO Zoe"` (absent) | 16 000 | 14 |

1. La séquentielle ne gagne que pour une douzaine de tout premiers noms (il lui faut moins de comparaisons que les 13 ou 14 de la dichotomie), soit moins d'un cas sur mille.
2. Le nom du milieu, `noms[7999]` = `"LANGLOIS Zoe"` (premier `mil` $= (0 + 15\,999)//2$).

**Exercice 6** *En moyenne, et au chronomètre*

▶ Corrigé.

```
def chrono_recherches(noms, numeros, recherche, cibles):
    debut = time.perf_counter()
    for nom in cibles:
        recherche(noms, numeros, nom)
    return time.perf_counter() - debut

cibles = [noms[random.randint(0, len(noms) - 1)] for k in range(1000)]
```

Mesures réellement obtenues :

- moyennes sur 1000 recherches : séquentielle **8 141** comparaisons ($\approx n/2$ : un nom tiré au hasard est en moyenne au milieu) ; dichotomique **13,0** ;

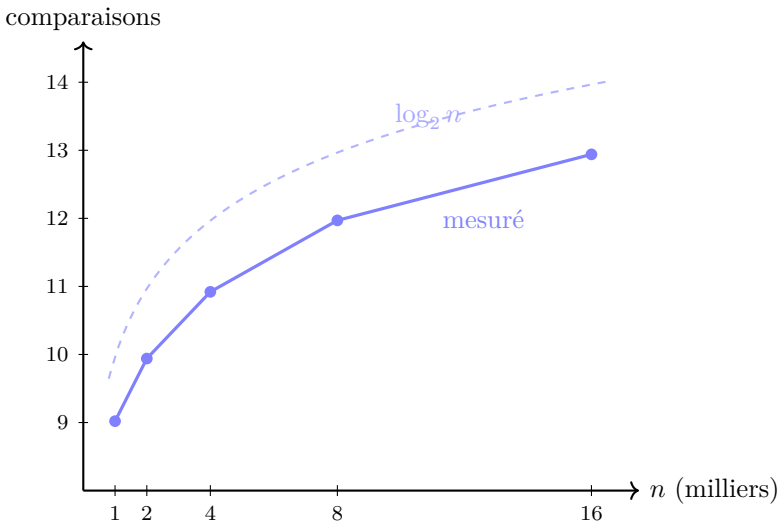
- 1000 recherches chronométrées : séquentielle **0,59 s**, dichotomique **0,003 s**, rapport $\approx 190$ ;
- `nom in noms` : **0,080 s**, sept fois plus rapide que notre séquentielle (le langage C), mais **25 fois plus lente** que notre dichotomie en Python. Avec $10^8$ abonnés (6 250 fois plus), `in` prendrait environ $6\,250 \times 0{,}08 \approx 500$ s, alors que la dichotomie ne ferait qu'une douzaine de comparaisons de plus par recherche : toujours quelques millisecondes. **Un bon algorithme bat un langage rapide** dès que les données sont grandes.

### Exercice 7 *La courbe du logarithme*

▶ Corrigé.

| **taille `n`** | 1 000 | 2 000 | 4 000 | 8 000 | 16 000 |
|---|---|---|---|---|---|
| **comparaisons (moyenne)** | 9,02 | 9,94 | 10,92 | 11,97 | 12,94 |
| $\log_2 n$ | 9,97 | 10,97 | 11,97 | 12,97 | 13,97 |

2. **+1** comparaison à chaque doublement, comme $\log_2 n$ (la moyenne vaut environ $\log_2 n - 1$ : on trouve souvent avant la fin). C'est la phrase du cours : « doubler la taille du tableau n'ajoute qu'une seule étape ».
3. Deux courbes parallèles, qui montent de moins en moins vite : la courbe mesurée est juste sous $\log_2 n$.
4. $\log_2(8 \times 10^9) \approx 32{,}9$ : au plus **33** comparaisons ; jusqu'à **8 milliards** en séquentiel.




*L'axe vertical commence à 8 : la courbe mesurée suit $\log_2 n$ à une unité près.*

## 4 L'autocomplétion

### Exercice 8 *Où commence le bloc ?*

▶ Corrigé.

```
1  def premier_indice_au_moins(noms, debut_nom):
2      deb = 0
3      fin = len(noms) - 1
4      res = len(noms)
5      while deb <= fin:
6          mil = (deb + fin) // 2
7          if noms[mil] >= debut_nom:
8              res = mil                # candidat ; on cherche encore plus a gauche
9              fin = mil - 1
10         else:
11             deb = mil + 1
```

```
    return res

def suggestions(noms, prefixe, k):
    res = []
    i = premier_indice_au_moins(noms, prefixe)
    while i < len(noms) and len(res) < k and noms[i][:len(prefixe)] == prefixe:
        res.append(noms[i])
        i = i + 1
    return res

def nb_commencant_par(noms, prefixe):
    i = premier_indice_au_moins(noms, prefixe)
    c = 0
    while i < len(noms) and noms[i][:len(prefixe)] == prefixe:
        c = c + 1
        i = i + 1
    return c
```

2. Dans l'ordre alphabétique, un préfixe est plus petit que tout mot qui le prolonge (`"DUP" < "DUPONT Adam"`) : tous les noms qui commencent par `"DUP"` sont donc `>= "DUP"`. Et tout nom placé avant ce bloc est `< "DUP"`. Le premier nom `>= "DUP"` (ici l'indice 4 300, `"DUPONT Adam"`) ouvre donc le bloc — s'il commence bien par `"DUP"` ; sinon, le bloc est vide.
4. **500** abonnés commencent par `"MAR"` (MARCHAND, MARIE, MARTIN, MARTINEZ, MARTY : $5 \times 100$) et **1 600** par `"LE"` (16 noms).
5. Environ $\log_2 n$ comparaisons pour trouver le début du bloc, puis au plus $k$ (plus une) pour le lire : $\log_2 n + k$. Avec $k = 5$ et un milliard de noms, une quarantaine d'opérations : instantané.

## 5 Bonus : la dichotomie sur les nombres à virgule

**Exercice 9** *Encadrer une racine carrée*

▶ Corrigé.

```
def racine(a, precision):
    bas = 0
    haut = a
    tours = 0
    while haut - bas > precision:
        mil = (bas + haut) / 2
        tours = tours + 1
        if mil * mil <= a:
            bas = mil
        else:
            haut = mil
    return (bas + haut) / 2, tours
```

`racine(2, 0.000001)` renvoie `(1.41421365..., 21)` ; `2 ** 0.5` vaut `1.41421356...` : écart inférieur à $10^{-6}$. Pour $a = 1\,000\,000$ : **40** tours. La largeur est divisée par 2 à chaque tour ; il faut $t$ tours avec $a/2^t \leq 10^{-6}$, soit $t \geq \log_2(a \times 10^6)$ : $\log_2(2 \times 10^6) \approx 20{,}9$ donne 21, et $\log_2(10^{12}) \approx 39{,}9$ donne 40.

## Bilan attendu (exemple)

*La dichotomie exige un tableau trié. Elle compare à l'élément du milieu et élimine une moitié à chaque tour : dans notre annuaire de 16 000 noms, 13 comparaisons en moyenne contre 8 141 pour la recherche séquentielle, et 1000 recherches en 0,003 s au lieu de 0,59 s. Quand on double la taille, il ne faut qu'une comparaison de plus (9, 10, 11, 12, 13 pour 1 000 à 16 000 noms) : c'est le coût logarithmique, environ* $\log_2 n$*, soit 33 comparaisons pour 8 milliards de noms. La même idée sert sans tableau : le devin trouve un nombre entre 1 et un million en 20 questions, et l'on encadre* $\sqrt{2}$ *à* $10^{-6}$ *près en 21 tours. Le chronomètre confirme le comptage, mais ajoute que la vitesse du langage compte aussi :* `in`*, écrit en C, bat notre recherche séquentielle, sans rattraper la dichotomie.*

## Réponses des questions de la fiche de révision

↑ questions p. 300

### Se tester

1. Il doit être trié (§ II)
2. `(2 + 9) // 2 = 5` (§ II)
3. 10 (car $2^9 = 512 \leq 1000 < 1024 = 2^{10}$) (§ IV)
4. Positif ou nul : un entier positif qui décroît strictement ne peut pas décroître indéfiniment, donc la boucle s'arrête (§ III)
5. Un indice doit être un entier : `//` donne la division entière (§ II)

### Et avant ?

1. linéaire : aucun décalage, un seul parcours (ch. 6, § IV.A)
2. 1 000, quand l'élément est absent (ch. 4, § V)
3. `False` : `and` s'arrête dès que la première condition est fausse, la division n'est pas faite (ch. 1, § III.A)

CORRIGÉS DU CHAPITRE 8

# 8 Les données en tables

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 Ouvrir un fichier CSV

↑énoncé p. 309

**Exercice 1** *Dans un éditeur de texte*

▶ **Solution.** **1. 41** lignes. La première, `prenom,sexe,annee,nombre`, ne décrit pas un prénom : elle donne le **nom des colonnes**.

**2.** La **virgule**.

**3.** En **2025**, le prénom **Jade** a été donné **2 925** fois à des **filles** (`F`). On devine sans peine `prenom`, `annee`, `nombre` ; `F/G` (fille/garçon) se déduit des prénoms.

**Exercice 2** *Dans un tableur*

▶ **Solution.** **4. 4** colonnes, **40** lignes de données.

**5.** Tout se retrouve dans **une seule colonne** : il n'y a aucun point-virgule dans le fichier, donc le tableur ne sait pas où couper. C'est exactement le piège du « fichier à la française » (séparateur `;`) lu avec la virgule, ou l'inverse.

**6.** Le tableur affiche en général **6** au lieu de `06` : il a « deviné » un nombre et supprimé le zéro. L'éditeur de texte montre bien `06`, car le fichier ne contient que du **texte**. Le problème : un code n'est pas un nombre (`2A` ne pourrait pas en être un), et si l'on réenregistre depuis le tableur, le fichier est modifié sans qu'on l'ait voulu. *À retenir pour Python : tout ce qu'on lit dans un CSV est une chaîne.*

**Exercice 3** *Chercher une information à la main*

▶ **Solution.** **7. Gabriel**, **4 625** fois. On n'a regardé que les lignes où le sexe vaut `G` *et* l'année `2025` (10 lignes) ; on a ignoré les filles et l'année 2023. On a donc d'abord **filtré** les lignes, puis cherché le **maximum**.

**8.** $3\,177 + 3\,070 = \mathbf{6\,247}$ (deux lignes `Louise` à additionner).

**9.** Menton a pour code `06` dans `communes.csv` ; dans `departements.csv`, le code `06` correspond aux Alpes-Maritimes, région **Provence-Alpes-Cote d'Azur** (écrite sans accent dans le fichier). Il a fallu les **deux fichiers**, reliés par la colonne commune `dep`. Pour Ajaccio (code `2A`), c'est impossible : ce code n'apparaît pas dans `departements.csv`.

**Exercice 4** *Ce qu'on a découvert*

▶ **Solution.** **10.** « Un fichier CSV est un simple fichier **texte**. Sa première ligne, l'**en-tête**, donne le nom de chaque **colonne**. Chaque ligne suivante est un **enregistrement** : elle décrit un individu. Sur une ligne, les valeurs sont séparées par un **séparateur**, qu'il faut connaître pour bien lire le fichier. »

# CORRIGÉS DES EXERCICES

> **Remarque**
> On donne **une** version simple ; d'autres écritures sont possibles.

## Lire une table, lire un CSV

**Exercice 1** *Vocabulaire* ↑énoncé p. 323

▶ **Solution.** **1.** Le séparateur est le **point-virgule** `;`. Il y a **4 descripteurs** et **3 enregistrements**. **2.** Descripteurs : `titre`, `editeur`, `duree`, `age_min`. Pour `Dixit`, `duree` vaut 30. **3.** « À la française » : le séparateur est le point-virgule, comme il se doit quand on veut réserver la virgule aux décimaux.

▷ **Erreur fréquente.** compter la ligne d'en-tête parmi les enregistrements (4 au lieu de 3) : elle contient les noms des descripteurs, pas des données.

## Représenter une table en Python

**Exercice 2** *Une table à la main* ↑énoncé p. 323

▶ **Solution.** Une liste de trois dictionnaires, un par enregistrement, avec les descripteurs pour clés ; les valeurs sont des chaînes, comme après une lecture par `csv.DictReader`.

```
jeux = [
    {"titre": "Les Aventuriers du Rail", "editeur": "Days of Wonder",
     "duree": "45", "age_min": "8"},
    {"titre": "Dixit", "editeur": "Libellud", "duree": "30", "age_min": "8"},
    {"titre": "7 Wonders", "editeur": "Repos Prod",
     "duree": "30", "age_min": "10"},
]
```

**Exercice 3** *Accès* ↑énoncé p. 323

▶ **Solution.** `jeux[1]["titre"]` → `'Dixit'` ; `jeux[0]["age_min"]` → `'8'` ; `len(jeux)` → 3.

Éditeur de `7 Wonders` : `jeux[2]["editeur"]`.

▷ **Erreur fréquente.** inverser les deux accès, `jeux["editeur"][2]` : `jeux` est une *liste*, on y choisit d'abord une fiche par son indice ; c'est la fiche, un dictionnaire, qui se lit par clé.

**Exercice 4** *Descripteurs automatiques* ↑énoncé p. 324

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert descripteurs([{"a": "1", "b": "2"}]) == ["a", "b"]  # cas normal
assert descripteurs([{"nom": "Ada"}]) == ["nom"]           # cas limite : une seule colonne
assert descripteurs([{"x": "1"}, {"x": "2"}]) == ["x"]     # cas piege : chaque nom une seule
                                                           # fois
```

Les noms de colonnes sont les clés de la première fiche ; une compréhension sur ce dictionnaire parcourt ses clés.

```
def descripteurs(table):
    return [cle for cle in table[0]]
```

## Importer, et le piège du texte

**Exercice 5** *Un fichier mal formé* ↑énoncé p. 324

**▶ Solution.** Le fichier est « à la française » (séparateur `;`) mais lu avec le séparateur par défaut (la virgule) : chaque ligne entière est prise pour **une seule valeur**. On corrige en précisant le séparateur :

```
csv.DictReader(f, delimiter=";")
```

**Exercice 6** *Encodage* ↑énoncé p. 324

**▶ Solution.** L'`encoding` n'a pas été précisé (le fichier est en UTF-8 mais lu autrement). On corrige :

```
open(fichier, encoding="utf-8", newline="")
```

**Exercice 7** *Lecture de trace* ↑énoncé p. 324

**▶ Solution.** Elle renvoie `20`. Elle **compte le nombre de fiches de filles** dans la table (10 en 2023 + 10 en 2025).

**Exercice 8** *Le piège du texte* ↑énoncé p. 324

**▶ Solution.**

```
>>> prenoms[0]["nombre"]
'3177'
>>> prenoms[0]["nombre"] + prenoms[1]["nombre"]
'31773168'
>>> int(prenoms[0]["nombre"]) + int(prenoms[1]["nombre"])
6345
```

Sur des **chaînes**, `+` *colle* les textes (concaténation). Il faut `int(...)` pour obtenir une vraie **addition**.

▷ **Erreur fréquente.** convertir après l'addition, `int(prenoms[0]["nombre"] + prenoms[1]["nombre"])` : les chaînes sont déjà collées, on obtient `31773168`. On convertit chaque valeur *avant* de calculer.

## Rechercher et filtrer

**Exercice 9** *Filtrer* ↑énoncé p. 324

**▶ Solution.** On part d'une liste vide et on y ajoute chaque fiche qui vérifie les deux conditions, reliées par `and`.

```
def selection(table, sexe, annee):
    resultat = []                                          # INITIALISER : liste vide
    for ligne in table:                                    # PARCOURIR
        if ligne["sexe"] == sexe and ligne["annee"] == annee:  # TESTER
            resultat.append(ligne)                         # METTRE A JOUR
    return resultat                                        # RENVOYER
```

▷ **Erreur fréquente.** appeler `selection(prenoms, "F", 2023)` avec l'entier `2023` : `"2023" == 2023` est faux, et la sélection est vide, sans aucun message d'erreur.

**Exercice 10** *En compréhension* ↑énoncé p. 324

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert len(selection(prenoms, "F", "2023")) == 10  # cas normal
assert selection(prenoms, "F", "1990") == []       # cas limite : aucune fiche ne convient
assert selection(prenoms, "F", 2023) == []         # cas piege : l'annee est une chaine
```

La condition du `if` de `selection` devient le filtre de la compréhension, et l'élément gardé est la fiche elle-même.

```python
def selection(table, sexe, annee):
    return [l for l in table if l["sexe"] == sexe and l["annee"] == annee]
```

**Exercice 11** *Rechercher une fiche* ↑énoncé p. 325

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```python
assert chercher(prenoms, "Gabriel", "2023")["nombre"] == "4527"  # cas normal
assert chercher(prenoms, "Kevin", "2025") == None                # cas limite : prenom absent
assert chercher(prenoms, "Jade", "2025")["nombre"] == "2925"     # cas piege : la Jade de 2023
                                                                 # vient avant
```

Recherche séquentielle : on renvoie la fiche dès que les deux critères sont vérifiés, et `None` seulement après avoir tout parcouru.

```python
def chercher(table, prenom, annee):
    for ligne in table:                                          # PARCOURIR
        if ligne["prenom"] == prenom and ligne["annee"] == annee:  # TESTER
            return ligne                                         # RENVOYER : trouvee, on
                                                                 # s'arrete
    return None                                                  # RENVOYER : parcours fini
                                                                 # sans succes
```

▷ **Erreur fréquente.** placer `return None` dans un `else`, à l'intérieur de la boucle : la fonction répond dès la première fiche et ne trouve que le premier prénom de la table.

**Exercice 12** *Deux critères* ↑énoncé p. 325

**▶ Solution.**

```python
[l["prenom"] for l in prenoms
 if l["sexe"] == "G" and l["annee"] == "2025" and int(l["nombre"]) > 3000]
# -> ['Gabriel', 'Noah', 'Leo', 'Raphael', 'Louis', 'Jules']
```

Le `int(...)` est indispensable : sans lui, on comparerait une chaîne à un entier (erreur), ou un tri de texte donnerait un résultat faux.

## Calculer sur une colonne

**Exercice 13** *Compter* ↑énoncé p. 325

**▶ Solution.** Patron du compteur : `n` part de 0 et augmente de 1 pour chaque fiche de ce sexe.

```python
def compter(table, sexe):
    n = 0                          # INITIALISER
    for ligne in table:            # PARCOURIR
        if ligne["sexe"] == sexe:  # TESTER
            n += 1                 # METTRE A JOUR
    return n                       # RENVOYER
```

`compter(prenoms, "G")` → 20.

**Exercice 14** *Total et moyenne* ↑énoncé p. 325

**▶ Solution.** *Appels attendus pour* `total`*, écrits avant de coder :*

```python
assert total(prenoms, "G", "2023") == 32684  # cas normal
assert total(prenoms, "G", "1990") == 0      # cas limite : aucune fiche retenue
t = [{"sexe": "F", "annee": "2023", "nombre": "900"},
     {"sexe": "F", "annee": "2023", "nombre": "3070"}]
assert total(t, "F", "2023") == 3970         # cas piege : additionner des nombres, pas du
                                             # texte
```

Patron de l'accumulateur : on ajoute `int(ligne["nombre"])` pour chaque fiche retenue ; la moyenne divise la somme par `len(table)`.

```python
def total(table, sexe, annee):
    s = 0                                    # INITIALISER
    for ligne in table:                      # PARCOURIR
```

```
        if ligne["sexe"] == sexe and ligne["annee"] == annee:  # TESTER
            s = s + int(ligne["nombre"])                       # METTRE A JOUR : convertir !
    return s                                                   # RENVOYER

def nombre_moyen(table):
    total_val = 0                                              # INITIALISER
    for ligne in table:                                        # PARCOURIR
        total_val = total_val + int(ligne["nombre"])           # METTRE A JOUR
    return total_val / len(table)                              # RENVOYER
```

`total(prenoms, "G", "2023")` → 32684. `nombre_moyen(filles23)` → 2628.7.

▷ **Erreur fréquente.** oublier `int` dans l'accumulateur : `s + ligne["nombre"]` ajoute une chaîne à un entier, et Python lève une `TypeError`.

**Exercice 15** *Champion et lanterne rouge* ↑énoncé p. 325

▶ **Solution.** *Appels attendus pour* `plus_donne`*, écrits avant de coder :*

```
assert plus_donne(prenoms) == ("Gabriel", 4625)                          # cas normal
assert plus_donne([{"prenom": "Ada", "nombre": "12"}]) == ("Ada", 12)    # cas limite : une
                                                                         # seule fiche
t = [{"prenom": "Ana", "nombre": "900"}, {"prenom": "Eve", "nombre": "3070"}]
assert plus_donne(t) == ("Eve", 3070)                                    # cas piege : "900" >
                                                                         # "3070" en texte
```

```
def plus_donne(table):
    champion = table[0]                                  # INITIALISER : la premiere fiche
    for ligne in table:                                  # PARCOURIR
        if int(ligne["nombre"]) > int(champion["nombre"]):  # TESTER
            champion = ligne                             # METTRE A JOUR
    return champion["prenom"], int(champion["nombre"])   # RENVOYER

def moins_donne(table):
    champion = table[0]                                  # INITIALISER
    for ligne in table:                                  # PARCOURIR
        if int(ligne["nombre"]) < int(champion["nombre"]):  # TESTER
            champion = ligne                             # METTRE A JOUR
    return champion["prenom"], int(champion["nombre"])   # RENVOYER
```

`plus_donne(prenoms)` → `('Gabriel', 4625)` ; `moins_donne(prenoms)` → `('Anna', 2129)`. Sans `int`, on comparerait des *chaînes*. Sur ce fichier, le champion serait le bon *par chance* : tous les effectifs ont 4 chiffres. Mais dès que les longueurs diffèrent, `"900"` serait jugé « plus grand » que `"3070"` (comparaison caractère par caractère) et le champion serait faux.

**Exercice 16** *Coût* ↑énoncé p. 325

▶ **Solution.** À chaque tour de boucle (il y en a $n$), `nombre_moyen(table)` **re-parcourt** toute la table ($n$ opérations) : coût total $n \times n$, **quadratique**. On rend le coût **linéaire** en calculant la moyenne **une seule fois, avant** la boucle :

```
m = nombre_moyen(table)
for ligne in table:
    print(ligne["nombre"], m)
```

## Trier une table

**Exercice 17** *Trier par nombre* ↑énoncé p. 326

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
assert trier_par_nombre(filles23)[0]["prenom"] == "Louise"  # cas normal
assert trier_par_nombre([]) == []                           # cas limite : table vide
t = [{"prenom": "Ana", "nombre": "900"}, {"prenom": "Eve", "nombre": "3070"}]
assert trier_par_nombre(t)[0]["prenom"] == "Eve"            # cas piege : comparer des entiers
```

`sorted` avec une clé qui convertit le `nombre` en entier, et `reverse=True` pour aller du plus donné au moins donné ; on filtre d'abord, on garde les 3 premiers à la fin.

```
def trier_par_nombre(table):
    return sorted(table, key=lambda ligne: int(ligne["nombre"]), reverse=True)

top = trier_par_nombre(selection(prenoms, "G", "2025"))
print([l["prenom"] for l in top][:3])   # ['Gabriel', 'Noah', 'Leo']
```

**Exercice 18** *Alphabétique* ↑énoncé p. 326

**▶ Solution.** On change seulement la **clé** (et on enlève `reverse`) :

```
sorted(filles23, key=lambda l: l["prenom"])
# -> Alba, Alice, Alma, Ambre, Anna, Emma, Jade, Louise, Romy, Rose
```

**Exercice 19** *Le tri qui ment* ↑énoncé p. 326

**▶ Solution.** Avec `key=lambda l: l["nombre"]`, on trie les `nombre` **comme des mots**. Deux chaînes se comparent caractère par caractère : `"900"` commence par `'9'`, `"3070"` par `'3'` ; comme `'9' > '3'`, la chaîne `"900"` est jugée « plus grande » que `"3070"`. Sur `filles23`, le résultat est pourtant le bon classement, mais *par chance* : tous les effectifs ont 4 chiffres, et pour des chaînes de chiffres de même longueur, l'ordre des mots coïncide avec l'ordre des nombres. Il suffirait d'un prénom donné 900 fois pour que le classement devienne faux : il faut `int(...)`.

## Fusionner deux tables

**Exercice 20** *Remettre la jointure dans l'ordre* ↑énoncé p. 326

**▶ Solution.**

```
def jointure(gauche, droite, cle):
    resultat = []                                   # INITIALISER : la table fusionnee, vide au depart
    for lg in gauche:                               # PARCOURIR : chaque fiche de gauche...
        for ld in droite:                           # PARCOURIR : ... comparee a chaque fiche de droite
            if lg[cle] == ld[cle]:                  # TESTER : meme valeur de la cle
                fusion = {c: lg[c] for c in lg}     # copie de la fiche de gauche
                for c in ld:
                    fusion[c] = ld[c]               # on ajoute les colonnes de droite
                resultat.append(fusion)             # METTRE A JOUR : on range la fiche combinee
    return resultat                                 # RENVOYER
```

À noter : `resultat = []` est *avant* les boucles (sinon on le vide à chaque tour) ; `resultat.append(fusion)` est dans le `if`, *après* la boucle sur `c` (sinon on ajouterait des fiches incomplètes) ; `return resultat` est au niveau de la fonction, après toutes les boucles.

**Exercice 21** *Jointure* ↑énoncé p. 326

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
villes = jointure(communes, departements, "dep")
assert villes[0]["region"] == "Provence-Alpes-Cote d'Azur"  # cas normal
assert jointure([], departements, "dep") == []              # cas limite : aucune commune
assert len(villes) == 7                                     # cas piege : Ajaccio (2A)
                                                  # disparait
```

```
def jointure(gauche, droite, cle):
    resultat = []                          # INITIALISER
    for lg in gauche:                      # PARCOURIR
        for ld in droite:                  # PARCOURIR
            if lg[cle] == ld[cle]:         # TESTER
                fusion = {c: lg[c] for c in lg}
                for c in ld:
                    fusion[c] = ld[c]
                resultat.append(fusion)    # METTRE A JOUR
    return resultat                        # RENVOYER

villes = jointure(communes, departements, "dep")
```

La fiche de `Nice` obtient `region` = `"Provence-Alpes-Cote d'Azur"`. **Ajaccio disparaît** : son code `dep "2A"` n'a *aucune* correspondance dans `departements`, donc la condition `lg[cle] == ld[cle]` n'est jamais vraie et aucune ligne n'est produite pour lui.

**Exercice 22** *Compter par groupe* ↑énoncé p. 326

**▶ Solution.**

```
villes = jointure(communes, departements, "dep")
n = 0                                                  # INITIALISER
for ligne in villes:                                   # PARCOURIR
    if ligne["region"] == "Provence-Alpes-Cote d'Azur":  # TESTER
        n += 1                                         # METTRE A JOUR
print(n)                                               # 3  (Nice, Menton, Marseille)
```

*Autre méthode :* construire par compréhension la liste des communes de la région, puis prendre sa longueur.

```
paca = [l["nom"] for l in villes if l["region"] == "Provence-Alpes-Cote d'Azur"]
print(len(paca))   # 3
```

## Mini-projets de synthèse

**Exercice 23** *Le palmarès à la loupe* ↑énoncé p. 327

**▶ Solution.**

```
filles25 = selection(prenoms, "F", "2025")
garcons25 = selection(prenoms, "G", "2025")

print("Fille la plus donnee :", plus_donne(filles25))     # ('Louise', 3070)
print("Garcon le plus donne :", plus_donne(garcons25))    # ('Gabriel', 4625)
print("Moyenne filles :", nombre_moyen(filles25))         # 2489.0
print("Moyenne garcons :", nombre_moyen(garcons25))       # 3231.5
top3 = trier_par_nombre(garcons25)[:3]
print("Top 3 garcons :", [l["prenom"] for l in top3])     # Gabriel, Noah, Leo
```

**Exercice 24** *Qui monte, qui descend ?* ↑énoncé p. 327

**▶ Solution.** *Appels attendus pour* `rang`*, écrits avant de coder :*

```
assert rang(prenoms, "G", "2023")["Gabriel"] == 1  # cas normal
assert rang(prenoms, "G", "1990") == {}            # cas limite : aucune fiche
assert rang(prenoms, "G", "2025")["Mael"] == 10    # cas piege : le rang commence a 1, pas a 0
```

```
def rang(table, sexe, annee):
    lignes = trier_par_nombre(selection(table, sexe, annee))
    resultat = {}                                  # INITIALISER
    for i in range(len(lignes)):                   # PARCOURIR
        resultat[lignes[i]["prenom"]] = i + 1      # METTRE A JOUR : le rang commence a 1
    return resultat                                # RENVOYER

r23 = rang(prenoms, "G", "2023")
r25 = rang(prenoms, "G", "2025")
for prenom in r23:
    if prenom in r25:
        print(prenom, r23[prenom], "->", r25[prenom])
# Noah 6 -> 2 (monte), Mael 5 -> 10 (descend)
```

**Exercice 25** *Entrants et sortants* ↑énoncé p. 327

**▶ Solution.**

```
noms2023 = [l["prenom"] for l in prenoms if l["sexe"] == "F" and l["annee"] == "2023"]
noms2025 = [l["prenom"] for l in prenoms if l["sexe"] == "F" and l["annee"] == "2025"]

deux_annees = [p for p in noms2023 if p in noms2025]
sortis = [p for p in noms2023 if p not in noms2025]     # ['Anna']
entres = [p for p in noms2025 if p not in noms2023]     # ['Adele']
print("dans les deux :", deux_annees)
print("sortis :", sortis)
print("entres :", entres)
```

*Autre méthode :* sans compréhension, avec une boucle et `append` (même idée : on garde les prénoms de 2023 absents de 2025).

```
sortis = []                 # INITIALISER
for p in noms2023:          # PARCOURIR
    if p not in noms2025:   # TESTER
        sortis.append(p)    # METTRE A JOUR
```

**Exercice 26** *Écrire un CSV [ au-delà du programme ]* ↑énoncé p. 327

**▶ Solution.** *Appels attendus, écrits avant de coder :*

```
exporter(filles23, "essai.csv")
assert charger("essai.csv") == filles23                  # cas normal
exporter([{"nom": "Ada"}], "essai.csv")
assert charger("essai.csv") == [{"nom": "Ada"}]          # cas limite : une fiche, une colonne
exporter([{"nom": "Dupont, Jean", "ville": "Nice"}], "essai.csv")
assert charger("essai.csv")[0]["nom"] == "Dupont, Jean"  # cas piege : une virgule dans une
                                                         # valeur
```

```
def exporter(table, fichier):
    with open(fichier, "w", encoding="utf-8", newline="") as f:
        ecrivain = csv.DictWriter(f, fieldnames=[cle for cle in table[0]])
        ecrivain.writeheader()
        for ligne in table:              # PARCOURIR
            ecrivain.writerow(ligne)     # une ligne du fichier par fiche

exporter(filles23, "filles_2023.csv")
verif = charger("filles_2023.csv")
print(len(verif))                        # 10
```

**Exercice 27** *Une moyenne selon un assistant d'IA* ↑énoncé p. 327

**▶ Solution.** **1.** Non. L'appel s'interrompt dès la première ligne de la table :

```
>>> moyenne_nombre(prenoms)
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

`total` est un entier (`0`) et `ligne["nombre"]` une **chaîne** (`'3177'`) : Python refuse de les additionner. **2.** L'erreur est l'affirmation « rien à convertir » : `csv` lit du *texte*, toutes les valeurs sont des `str`, même les nombres. On convertit avec `int` :

```
def moyenne_nombre(table):
    total = 0                                # INITIALISER
    for ligne in table:                      # PARCOURIR
        total = total + int(ligne["nombre"]) # METTRE A JOUR : int, car csv lit du texte
    return total / len(table)                # RENVOYER
```

**3.** Regarder une fiche avant de calculer : `prenoms[0]` affiche `'nombre': '3177'`, avec des guillemets, donc du texte (ou demander `type(prenoms[0]["nombre"])`). C'est le réflexe du cours : dès qu'on calcule sur une colonne, on convertit.

## Entraînement

**Exercice 28** *Lire et filtrer une table* ↑énoncé p. 328

**▶ Solution.** a) `'Zoé'` b) `'159'` (deux chaînes collées) c) `24` d) `['Ada', 'Zoé']` e) `['Léo']`.

**Exercice 29** *Trier une table : prédire l'ordre* ↑énoncé p. 328

**▶ Solution.** a) `['Ada', 'Léo', 'Zoé']` (ordre alphabétique des noms) ; b) `['Léo', 'Zoé', 'Ada']` (notes 9, 12, 15) ; c) `['Ada', 'Zoé', 'Léo']` (notes décroissantes) ; d) `['Zoé', 'Ada', 'Léo']`. En d), les notes sont comparées comme des **chaînes** : `"12"` et `"15"` commencent par `'1'`, qui vient avant `'9'` : `"9"` passe en dernier, comme si c'était la meilleure note. C'est le piège n° 1 : on convertit avec `int` dans la clé.

▷ **Erreur fréquente.** trier une colonne de nombres lue dans un CSV sans `int` : le tri ne signale aucune erreur, mais il range les valeurs dans l'ordre du dictionnaire, pas dans l'ordre numérique.

## Nettoyer une table

**Exercice 30** *Doublons et notes impossibles* ↑énoncé p. 328

**▶ Solution.** *Appels attendus pour* `doublons`*, écrits avant de coder :*

```
assert len(doublons(notes)) == 1  # cas normal
assert doublons([]) == []         # cas limite : table vide
t = [{"nom": "Ada", "classe": "1A"}, {"nom": "Ada", "classe": "1B"}]
assert doublons(t) == []          # cas piege : meme nom, autre classe
```

**1.** Un test de cohérence est un filtre (sans oublier `int`) :

```
def notes_impossibles(table):
    return [ligne["nom"] for ligne in table
            if int(ligne["note"]) < 0 or int(ligne["note"]) > 20]
```

`notes_impossibles(notes)` renvoie `['Zoé', 'Tom']` (23 et −2).

**2.** On mémorise les identifiants déjà vus :

```
def doublons(table):
    deja_vus = []                    # INITIALISER : identifiants rencontres
    resultat = []                    # INITIALISER
    for ligne in table:              # PARCOURIR
        cle = (ligne["nom"], ligne["classe"])
        if cle in deja_vus:          # TESTER
            resultat.append(ligne)   # METTRE A JOUR : c'est un doublon
```

```
        else:
            deja_vus.append(cle)     # METTRE A JOUR : premiere rencontre
    return resultat                  # RENVOYER
```

Il y a **1** doublon : la troisième fiche (Ada, 1A). La dernière fiche est une autre élève, Ada de **1B** : même nom, mais pas même identifiant (`nom`, `classe`).

**3.** On garde une fiche si elle est cohérente et si son identifiant n'a pas encore été gardé :

```
def nettoyer(table):
    propre = []                                          # INITIALISER
    deja_vus = []                                        # INITIALISER
    for ligne in table:                                  # PARCOURIR
        cle = (ligne["nom"], ligne["classe"])
        note = int(ligne["note"])
        if note >= 0 and note <= 20 and cle not in deja_vus:  # TESTER
            deja_vus.append(cle)                         # METTRE A JOUR
            propre.append(ligne)                         # METTRE A JOUR
    return propre                                        # RENVOYER
```

Avant nettoyage : $(15+9+15+23-2+11)/6 = 71/6 \approx 11{,}83$. Après : il reste Ada (1A), Léo et Ada (1B), soit $(15+9+11)/3 = 35/3 \approx 11{,}67$.

# CORRIGÉS DES TP



## TP La météo de la Riviera

↑énoncé p. 331

Les résultats ci-dessous portent sur les fichiers fournis (193 lignes de relevés, 6 stations). Les fonctions complètes sont regroupées en fin de document.

### 1. Prise en main

**Exercice 1** *Regarder le fichier avant de programmer*

**1.** Descripteurs : `station`, `date`, `tmin`, `tmax`, `pluie`. Séparateur : le **point-virgule**, car la virgule sert déjà de séparateur décimal (`23,1`). **2.** Températures minimale et maximale de la journée en degrés Celsius, hauteur de pluie en millimètres ; dates au format `AAAA-MM-JJ`. **3.** Le descripteur commun est `station` (un code de trois lettres).

**Exercice 2** *Charger les deux tables*

**1.** `len(releves)` vaut **193**, `len(stations)` vaut **6** ; `releves[0]` est `{'station': 'MON', 'date': '2025-08-01', 'tmin': '23,1', 'tmax': '29,0', 'pluie': '0,0'}`.

**2.** Avec `","`, l'en-tête n'est qu'une seule colonne nommée `'station;date;tmin;tmax;pluie'`, et chaque ligne est découpée au milieu des nombres : `releves[0]` vaut `{'station;date;tmin;tmax;pluie': 'MON;2025-08-01;23', None: ['1;29', '0;0', '0']}`. Le mauvais séparateur ne provoque *aucune erreur* : il produit silencieusement une table absurde.

**3.** C'est une chaîne (`str`). `float("29,0")` échoue (`ValueError`) car Python attend un **point** décimal.

**Exercice 3** *Convertir un nombre « à la française »*

```python
def en_nombre(texte):
    if texte == "":
        return None
    resultat = ""                  # on recopie le texte...
    for car in texte:
        if car == ",":
            resultat += "."        # ... en changeant la virgule en point
        else:
            resultat += car
    return float(resultat)
```

## 2. Contrôle qualité

**Exercice 4** *Les valeurs manquantes*

La compréhension

```python
[(l["station"], l["date"]) for l in releves if not est_complete(l)]
```

donne **2** lignes : `('NIC', '2025-08-07')` (`tmax` vide) et `('SOS', '2025-08-23')` (`tmin` vide).

**Exercice 5** *Les doublons*

**1.** Patron du comptage : si la clé est déjà dans `compteur`, on ajoute 1, sinon on la crée avec la valeur 1 (code complet de `compter_par_cle` et de `doublons` en fin de document).

**2.** Trois clés en double : `('MON', '2025-08-14')`, `('NIC', '2025-08-15')`, `('SOS', '2025-08-02')`. **3.** Les doublons de Monaco et de Sospel sont **identiques** (relevé transmis deux fois : sans gravité). Celui de Nice est **conflictuel** : `tmax` vaut `30,4` dans la première ligne et `30,8` dans la seconde. On ne peut pas savoir laquelle est juste : on garde la première par convention, mais il faudrait le **signaler**.

**Exercice 6** *Les valeurs incohérentes*

Trois lignes complètes mais incohérentes :

- `MEN`, 12 août : `tmin = 32,9` et `tmax = 25,8` ; `ANT`, 28 août : `tmin = 29,5` et `tmax = 20,3` → les deux colonnes ont manifestement été **inversées** à la saisie ;
- `GRE`, 5 août : `tmax = 99,9` → valeur impossible (à 1 400 m d'altitude !) : **capteur défectueux** ou code d'erreur.

**Exercice 7** *Nettoyer la table*

Il reste **185** lignes ; **8** ont été écartées : 2 incomplètes, 3 incohérentes, 3 doublons. (On pourrait « réparer » les deux inversions plutôt que les supprimer : bonne idée à discuter, mais c'est une décision d'analyste à justifier.)

## 3. Interroger la table

**Exercice 8** *Une station à la loupe*

Monaco : **31** relevés, moyenne des `tmax` : **29,1** degrés.

```python
monaco = releves_de(propre, "MON")
m = moyenne(monaco, "tmax")
print(len(monaco), f"{m:.1f}")      # 31 29.1 (affichage au dixieme)
```

### Exercice 9 *Records et comptages*

**1.** Record du mois : `('SOS', '2025-08-11', 34.4)`, à Sospel ; record de Monaco : `('MON', '2025-08-13', 32.1)`.

**2.** Monaco : **31** nuits tropicales sur 31 ; Gréolières : **0**.

**3.** Cumul de pluie (mm) et jours de pluie :

| | Monaco | Menton | Nice | Antibes | Sospel | Gréolières |
|---|---|---|---|---|---|---|
| cumul (mm) | 34,0 | 38,2 | 23,1 | 47,4 | 96,4 | 115,6 |
| jours $\geqslant 1$ mm | 4 | 2 | 4 | 3 | 7 | 9 |

```
for s in stations:
    r = releves_de(propre, s["station"])
    cumul = 0
    for ligne in r:
        cumul = cumul + ligne["pluie"]
    print(s["nom"], f"{cumul:.1f}", compter_si(r, "pluie", 1))
```

### Exercice 10 *L'épisode orageux*

```
def pluie_par_jour(table):
    cumul = {}
    for ligne in table:
        d = ligne["date"]
        if d in cumul:
            cumul[d] = cumul[d] + ligne["pluie"]
        else:
            cumul[d] = ligne["pluie"]
    return cumul

cumul = pluie_par_jour(propre)
jour_max = None
for d in cumul:
    if jour_max is None or cumul[d] > cumul[jour_max]:
        jour_max = d
print(len(cumul), jour_max, f"{cumul[jour_max]:.1f}")
print(len([d for d in cumul if cumul[d] == 0]))
```

**1.** 31 clés (une par jour d'août). **2.** Le **20 août** (132,5 mm au total), suivi de près par le **19 août** (127,8 mm) : c'est l'épisode orageux, très loin devant le 28 août (13,5 mm). **3.** **14** jours sans aucune pluie sur le réseau.

## 4. Trier

### Exercice 11 *Palmarès*

**1.** Les cinq journées les plus chaudes sont **toutes à Sospel** : 11 août (34,4), 12 août (34,1), 14 août (34,0), 13 août (33,7), 9 août (33,4). L'arrière-pays, loin de la mer, chauffe davantage ; la vague de chaleur se situe vers le 10–16 août.

```
top = sorted(propre, key=lambda l: l["tmax"], reverse=True)
for l in top[:5]:
    print(l["station"], l["date"], l["tmax"])
```

**2.** Gréolières (115,6), Sospel (96,4), Antibes (47,4), Menton (38,2), Monaco (34,0), Nice (23,1).

```
pluies = []
for s in stations:
    r = releves_de(propre, s["station"])
    pluies.append((s["nom"], moyenne(r, "pluie") * len(r)))
for c in sorted(pluies, key=lambda c: c[1], reverse=True):
    print(c[0], f"{c[1]:.1f}")
```

**3.** `['2025-08-15', '2025-08-20', '2025-09-02']` est dans l'ordre chronologique ; `['02/09/2025', '15/08/2025', '20/08/2025']` ne l'est pas (le 2 septembre passe en premier). Les chaînes se comparent caractère par caractère : avec l'année, puis le mois, puis le jour, l'ordre « des mots » coïncide avec l'ordre du temps. C'est le format international **ISO 8601**.

## 5. Croiser

**Exercice 12** *Littoral ou arrière-pays ?*

**1.** La jointure compte **181** lignes contre **185** : les 4 relevés de la station `VIL` (absente de `stations.csv`) ont disparu, faute de correspondance. Un analyste doit le **remarquer** (comparer les effectifs avant et après) et compléter la table des stations plutôt que de perdre ces données sans le dire.

**2. et 3.**

| | lignes | moyenne `tmin` | moyenne `tmax` | écart moyen |
|---|---|---|---|---|
| littoral | 121 | 22,6 | 29,4 | 6,7 |
| arrière-pays | 60 | 14,6 | 27,5 | 13,0 |

La mer, lente à se réchauffer et à se refroidir, **adoucit** les écarts : nuits chaudes, journées modérées. Dans l'arrière-pays, l'écart jour/nuit est deux fois plus grand ; l'altitude (Gréolières, 1 400 m) abaisse en plus toutes les températures, ce qui explique une moyenne des `tmax` plus basse malgré les records de Sospel.

**4.** Après

```
stations.append({"station": "VIL", "nom": "Villefranche-sur-Mer",
                 "commune": "Villefranche-sur-Mer", "altitude": "10",
                 "littoral": "oui"})
```

la jointure compte **185** lignes : plus aucun relevé n'est perdu. (Pour les questions 2 et 3, les valeurs du tableau ont été calculées *avant* cet ajout.)

```
j = jointure(propre, stations, "station")
littoral = [l for l in j if l["littoral"] == "oui"]
arriere = [l for l in j if l["littoral"] == "non"]
for groupe in (littoral, arriere):
    m_min = moyenne(groupe, "tmin")
    m_max = moyenne(groupe, "tmax")
    print(f"{m_min:.1f} {m_max:.1f} {m_max - m_min:.1f}")
```

## 6. Produire

**Exercice 13** *Le fichier* `bilan_stations.csv`

```
def en_texte(nombre):
    texte = f"{nombre:.1f}"        # un chiffre apres le point
    resultat = ""
    for car in texte:
        if car == ".":
            resultat += ","        # virgule decimale a la francaise
        else:
            resultat += car
    return resultat

bilan = []
for s in stations:
    r = releves_de(propre, s["station"])
    bilan.append({"station": s["station"], "nom": s["nom"],
                  "tmin_moy": en_texte(moyenne(r, "tmin")),
                  "tmax_moy": en_texte(moyenne(r, "tmax")),
                  "pluie_totale": en_texte(moyenne(r, "pluie") * len(r)),
```

```
                  "nuits_tropicales": compter_si(r, "tmin", 20)})
exporter(bilan, "bilan_stations.csv", ";")
```

La boucle parcourt la table `stations`, qui contient **sept** fiches depuis l'ajout de `VIL` (exercice 12, question 4). La ligne manquante de `exporter` est la boucle `for ligne in table: ecrivain.writerow(ligne)`. Fichier obtenu :

```
station;nom;tmin_moy;tmax_moy;pluie_totale;nuits_tropicales
MON;Monaco;23,4;29,1;34,0;31
MEN;Menton;22,9;29,6;38,2;30
NIC;Nice;22,4;29,2;23,1;29
ANT;Antibes;21,7;29,5;47,4;29
SOS;Sospel;17,0;31,2;96,4;0
GRE;Gréolières;12,1;23,9;115,6;0
VIL;Villefranche-sur-Mer;23,2;28,8;1,2;4
```

La fiche de `VIL` ne repose que sur **4** relevés : ses moyennes sont bien moins fiables que celles des autres stations (une trentaine de relevés chacune), il faut le signaler. Rechargé avec `charger(..., ";")`, on retrouve 7 fiches (dont toutes les valeurs sont redevenues des chaînes !). Dans un tableur réglé en français, les nombres doivent être reconnus comme tels (alignés à droite, utilisables dans une formule) ; avec des points décimaux, ce n'est pas garanti.

## 7. Pour aller plus loin

**Exercice 14** *La plus longue vague de chaleur*

```
def plus_longue_serie(table, seuil):
    meilleure = 0
    en_cours = 0
    jour_precedent = 0
    for ligne in sorted(table, key=lambda l: l["date"]):
        jour = int(ligne["date"][8:10])          # "2025-08-13" -> 13
        if ligne["tmax"] >= seuil and jour == jour_precedent + 1:
            en_cours = en_cours + 1
        elif ligne["tmax"] >= seuil:
            en_cours = 1                         # un jour manque : on repart
        else:
            en_cours = 0
        if en_cours > meilleure:
            meilleure = en_cours
        jour_precedent = jour
    return meilleure
```

Séries à 30 degrés ou plus : Monaco 9, Menton 5, Nice 7, Antibes 8, **Sospel 11**, Gréolières 0. La plus longue vague de chaleur est donc à **Sospel** (11 jours). *Si l'on ne tient pas compte des jours manquants* (version qui remet seulement le compteur à 0 sous le seuil), Menton obtient 8 au lieu de 5 : le relevé écarté du 12 août « recolle » artificiellement deux séries. La réponse finale (Sospel) ne change pas, mais la version simple n'est correcte que si l'on signale cette limite.

**Exercice 15** *La même chose avec* `pandas`

Les moyennes coïncident avec les nôtres (Sospel 31,2 ; Menton 29,6 ; Antibes 29,5 ; Nice 29,2 ; Monaco 29,1 ; Gréolières 23,9). La station `VIL` apparaît aussi (28,8) alors qu'elle avait disparu de notre première jointure : le code `pandas` ne fait **pas de jointure** avec la table des stations, il ne « perd » donc pas ces lignes. Ce que notre code a appris : *chaque* étape (séparateur, virgule décimale, doublons, lignes perdues par la jointure) est une décision ; `pandas` les rend invisibles, mais elles existent toujours.

**Exercice 16** *Compte rendu*

Attendus : 193 lignes lues, 8 écartées (2 incomplètes, 3 incohérentes dont 2 inversions et une valeur à 99,9, 3 doublons dont un conflictuel), 4 relevés perdus par la jointure ; trois résultats chiffrés exacts ; sans nettoyage, le « record » du mois aurait été **99,9 degrés à Gréolières**, et les moyennes auraient planté (valeurs vides) ou été faussées.

### Les fonctions complètes

```
def en_nombre(texte):
    if texte == "":
        return None
    resultat = ""                  # on recopie le texte...
    for car in texte:
        if car == ",":
            resultat += "."        # ... en changeant la virgule en point
        else:
            resultat += car
    return float(resultat)

def est_complete(ligne):
    return ligne["tmin"] != "" and ligne["tmax"] != "" and ligne["pluie"] != ""

def compter_par_cle(table):
    compteur = {}
    for ligne in table:
        cle = (ligne["station"], ligne["date"])
        if cle in compteur:
            compteur[cle] = compteur[cle] + 1
        else:
            compteur[cle] = 1
    return compteur

def doublons(table):
    compteur = compter_par_cle(table)
    return [cle for cle in compteur if compteur[cle] > 1]

def est_coherente(ligne):
    if not est_complete(ligne):
        return False
    tmin = en_nombre(ligne["tmin"])
    tmax = en_nombre(ligne["tmax"])
    return tmin <= tmax and -30 <= tmin <= 50 and -30 <= tmax <= 50

def nettoyer(table):
    propre = []
    deja_vus = {}
    for ligne in table:
        cle = (ligne["station"], ligne["date"])
        if est_coherente(ligne) and cle not in deja_vus:
            deja_vus[cle] = True
            propre.append({"station": ligne["station"], "date": ligne["date"],
                           "tmin": en_nombre(ligne["tmin"]),
                           "tmax": en_nombre(ligne["tmax"]),
                           "pluie": en_nombre(ligne["pluie"])})
    return propre

def releves_de(table, station):
    return [ligne for ligne in table if ligne["station"] == station]

def moyenne(table, descripteur):
    total = 0
```

```
54      for ligne in table:
55          total = total + ligne[descripteur]
56      return total / len(table)
57
58  def jour_le_plus_chaud(table):
59      champion = table[0]
60      for ligne in table:
61          if ligne["tmax"] > champion["tmax"]:
62              champion = ligne
63      return champion["station"], champion["date"], champion["tmax"]
64
65  def compter_si(table, descripteur, seuil):
66      n = 0
67      for ligne in table:
68          if ligne[descripteur] >= seuil:
69              n = n + 1
70      return n
```

## Réponses des questions de la fiche de révision

↑ questions p. 343

### Se tester

1. Les descripteurs (l'en-tête du CSV) (§ I)
2. `"31773168"` : deux chaînes concaténées (§ III)
3. Mauvais séparateur : il faut `delimiter=";"` (§ I)
4. `sorted(table, key=lambda ligne: int(ligne["nombre"]), reverse=True)` (§ VI)
5. Elle disparaît du résultat (§ VII)

### Et avant ?

1. 20 (ch. 7, § IV)
2. une condition sur les arguments, à respecter pour avoir le droit d'appeler la fonction (ch. 5, § I)
3. `é` occupe 2 octets en UTF-8, lus comme 2 caractères en Latin-1 (ch. 2, § IX.C)

CORRIGÉS DU CHAPITRE 9

# 9 Architecture des ordinateurs et systèmes d'exploitation

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 Perdu dans le terminal

↑énoncé p. 350

**Exercice 1** *Où suis-je ?*

▶ **Solution.** **1.** Le terminal affiche `/root`. Ce n'est ni un simple nom de fichier ni un simple nom de dossier, mais un **chemin** : l'adresse du dossier où l'on se trouve (le répertoire courant).

**2.** Quatre éléments : `bench.py`, `hello.c`, `hello.js`, `readme.txt`.

**3.** `cat` affiche le **contenu** d'un fichier texte (ici un petit mode d'emploi en anglais qui commence par *Some tests*).

**Exercice 2** *Se déplacer*

▶ **Solution.** **4.** `pwd` affiche `/` ; `ls` affiche `bin dev etc home lib media mnt opt proc root run sbin srv sys tmp usr var`. Le dossier `root` du départ est bien dedans : tous les dossiers partent de `/`, comme les branches d'un arbre partent de sa racine.

**5.** Après `cd root` : `/root`. Après `cd ..` : `/`. Le symbole `..` désigne le dossier **parent**, un cran au-dessus.

**6.** On revient dans `/root`, le **répertoire personnel** de l'utilisateur.

**7.** Les deux mènent à `/bin`. `/bin` commence par `/` : on part de la racine (chemin **absolu**, valable de partout). `../bin` part de l'endroit où l'on est (chemin **relatif**, qui ne marche que depuis `/root` ou un autre dossier situé juste sous la racine).

**Exercice 3** *Créer*

▶ **Solution.** **8.** `cd` (ou `cd /root`), `mkdir nsi`, `cd nsi`, `touch essai.txt` ; `ls` affiche `essai.txt`. Autre solution correcte : `mkdir nsi` puis `touch nsi/essai.txt` sans se déplacer.

**9.**

```
/root/
|-- bench.py
|-- hello.c
|-- hello.js
|-- nsi/
|   |-- essai.txt
|-- readme.txt
```

**Exercice 4** *Le fichier caché*

▶ **Solution.** **10.** Apparaissent `.`, `..`, `.ash_history`, `.cache`, `.mozilla` et `.wine` : leur nom **commence par un point**. Ce sont des éléments **cachés**, que `ls` n'affiche pas sans l'option `-a` (souvent des réglages de programmes). `.` et `..` sont le dossier courant et le dossier parent. `.ash_history` est un fichier qui garde l'historique des commandes (son contenu varie).

**11.** Par exemple : `cd .mozilla`, `ls` (on voit `extensions` et `firefox`), `cd firefox`, `ls`, `cat profiles.ini`. Le fichier contient des lignes comme `[Profile0]`, `Name=default-default`. Chemin complet :

```
/root/.mozilla/firefox/profiles.ini
```

On peut aussi faire, sans se déplacer, `cat .mozilla/firefox/profiles.ini`.

**Exercice 5** *Ce qu'on a découvert*

▶ **Solution.** **12.** Réponse possible :

- `pwd` affiche le dossier où l'on est (le **répertoire courant**) ; `ls` affiche son contenu ;
- `cd` change de dossier ; `..` désigne le dossier **parent** ;
- un chemin qui commence par `/` part de la **racine** (chemin **absolu**) ; sinon il part du répertoire courant (chemin **relatif**) ;
- un nom qui commence par un point est **caché** (visible avec `ls -a`).

# CORRIGÉS DES EXERCICES



> **Remarque**
> On donne **une** version simple ; d'autres écritures sont possibles (surtout pour les programmes LMC).

## Architecture matérielle et modèle de von Neumann

**Exercice 1** *Chaque chose à sa place* ↑ énoncé p. 365

▶ **Solution.** L'unité de commande dirige, l'UAL calcule, la mémoire range, les entrées-sorties communiquent : (1)–(c), (2)–(a), (3)–(b), (4)–(d).

**Exercice 2** *Vrai ou faux* ↑ énoncé p. 365

▶ **Solution.** **a) Vrai** : le processeur (CPU) réunit l'unité de commande, l'UAL et les registres. **b) Faux** : la RAM est *volatile*, elle s'efface à l'extinction ; c'est le **disque** qui conserve. **c) Vrai** : c'est l'idée centrale de von Neumann, programme et données dans la même mémoire. **d) Faux** : un registre est bien plus *petit* mais bien plus *rapide* que le disque.

▷ **Erreur fréquente.** croire que la RAM garde les données à l'extinction : elle est *volatile*. Seuls le disque (ou la clé USB, le SSD) conservent les fichiers sans courant.

**Exercice 3** *Ordres de grandeur* ↑ énoncé p. 366

▶ **Solution.** On associe d'abord par l'unité (mètres, nombre d'objets, hertz) : a) quelques **nanomètres** à quelques dizaines de nanomètres ; b) des **milliards** à des dizaines de milliards ; c) quelques **gigahertz**.

**Exercice 4** *La loi de Moore* ↑énoncé p. 366

**▶ Solution.** **a)** 5 doublements multiplient par $2^5 = 32$ : environ $2\,300 \times 32 \approx$ **73 600** transistors. **b)** Ce n'est **pas** une loi de la physique, mais une **observation** (une tendance) qui a longtemps guidé l'industrie. Elle ralentit car on approche de la **taille de l'atome** : un transistor de quelques atomes ne peut plus vraiment être réduit (effets physiques parasites, chaleur).

**Exercice 5** *RAM ou disque ?* ↑énoncé p. 366

**▶ Solution.** **a)** Le travail enregistré n'est **pas** perdu : il est sur le **disque** (mémoire permanente). **b)** Le travail non enregistré est **perdu** : il était en **RAM** (mémoire volatile). **c)** « Enregistrer », c'est **recopier** les données de la RAM (*volatile*) vers le disque (*permanent*).

**Exercice 6** *Capteur ou actionneur ?* ↑énoncé p. 366

**▶ Solution.** **a)** capteur (mesure la température) ; **b)** actionneur (agit : ouvre/ferme) ; **c)** capteur (mesure le son) ; **d)** actionneur (émet de la lumière) ; **e)** l'écran tactile est **les deux** : la dalle tactile est un *capteur* (mesure le doigt), l'affichage est un *actionneur* (sortie visuelle) ; **f)** actionneur (émet du son).

**Arrosage automatique :** un **capteur** d'humidité du sol mesure l'humidité (*capter*) ; le programme la compare à un seuil (*décider*) ; si le sol est trop sec, il commande la **pompe** ou l'électrovanne (**actionneur**) pour arroser (*agir*).

## Le processeur au travail : langage machine (LMC)

**Exercice 7** *Dérouler pas à pas* ↑énoncé p. 366

**▶ Solution.** **a)** Déroulé pour l'entrée 4 :

| Adr. | Instr. | ACC | `case 08` |
|---|---|---|---|
| 00 | INP | 4 | – |
| 01 | STA 08 | 4 | 4 |
| 02 | ADD 08 | 8 | 4 |
| 03 | ADD 08 | 12 | 4 |
| 04 | OUT | 12 | 4 |

**b)** La sortie est **12**. **c)** Le programme affiche le **triple** du nombre lu ($3 \times 4 = 12$), obtenu par deux additions successives.

▷ **Erreur fréquente.** croire que `ADD 08` modifie la case 08 : seul l'accumulateur change ; la case 08 garde la valeur `4` rangée par `STA 08`.

**Exercice 8** *Le plus grand des deux* ↑énoncé p. 367

**▶ Solution.** **a)** Entrées 7 puis 2 : `a`= 7, `b`= 2 ; en 04 `SUB a` donne ACC $= 2 - 7 = -5 < 0$, donc `BRP` ne saute pas ; on charge `a` et on affiche **7**. **b)** Entrées 3 puis 8 : `a`= 3, `b`= 8 ; en 04 ACC $= 8 - 3 = 5 \geq 0$, donc `BRP` saute en 08, on charge `b` et on affiche **8**. **c)** Le programme affiche le **plus grand** des deux nombres. `BRP` réalise le **test** : selon le signe de $b - a$, on choisit d'afficher $a$ ou $b$.

▷ **Erreur fréquente.** croire que `BRP` ne saute que si ACC est strictement positif : il saute aussi pour ACC $= 0$ (« positif ou nul »). Avec deux entrées égales, on affiche donc `b`.

**Exercice 9** *À vous d'écrire* ↑énoncé p. 367

**▶ Solution.** Une solution (les cases 10 et 11 stockent les deux premiers nombres) :

```
00   INP
01   STA 10
02   INP
03   STA 11
04   INP
05   ADD 10
```

```
06   ADD 11
07   OUT
08   HLT
10       DAT
11       DAT
```

Pour 10, 20, 30 : le troisième nombre (30) est dans ACC, on ajoute les cases 10 et 11 (+10 puis +20) : sortie = 60.

**Exercice 10** *Le plus petit* ↑énoncé p. 367

**▶ Solution.** On garde la même structure, mais on échange les deux chargements :

```
00   INP
01   STA a
02   INP
03   STA b
04   SUB a        ACC <- b - a
05   BRP 08       si b >= a, le plus petit est a
06   LDA b        sinon b < a : le plus petit est b
07   BRA 09
08   LDA a
09   OUT
10   HLT
11   a  DAT
12   b  DAT
```

Entrées 7 et 2 : $b-a=-5<0 \to$ on affiche `b`= 2. Entrées 3 et 8 : $b-a=5\geq 0 \to$ on affiche `a`= 3.

**Exercice 11** *Une boucle : la somme des entiers* ↑énoncé p. 367

**▶ Solution.** **a)** Déroulé pour $n=3$, à chaque passage sur `boucle` :

| passage | `i` (avant) | `somme` (après) |
|---|---|---|
| 1 | 3 | 0+3 = 3 |
| 2 | 2 | 3+2 = 5 |
| 3 | 1 | 5+1 = 6 |
| 4 | 0 | (`BRZ` → sortie) |

La sortie est **6**. **b)** Le programme calcule $1+2+\cdots+n$ (la somme des entiers de 1 à $n$). **c)** `BRZ` *teste* la condition d'arrêt (sortir quand `i`= 0) ; `BRA` *retourne* sans condition au début de la boucle. Ensemble, ils réalisent la répétition.

## Le système d'exploitation

**Exercice 12** *À quoi sert l'OS ?* ↑énoncé p. 368

**▶ Solution.** Pour chaque situation, on repère la ressource partagée ou organisée : a) gérer le **processeur** ; b) gérer les **périphériques** ; c) gérer les **utilisateurs** (droits) ; d) gérer la **mémoire** ; e) gérer les **fichiers**.

**Exercice 13** *Familles de systèmes* ↑énoncé p. 368

**▶ Solution.** a) **Windows**, **macOS**, **Linux** (UNIX). b) Android est bâti sur un **noyau Linux**. c) **Faux** : « libre » désigne la liberté d'étudier, modifier et partager le code ; ce n'est pas synonyme de gratuit (confusion avec l'anglais *free*).

**Exercice 14** *L'abstraction* ↑énoncé p. 368

**▶ Solution.** Le système d'exploitation offre des actions **simples et universelles** (« ouvrir un fichier ») et **cache** le fonctionnement réel du disque. Si chaque application devait connaître le modèle exact du matériel, il faudrait la **réécrire** pour chaque disque, chaque imprimante… : ingérable. Grâce à cette couche d'abstraction, les applications restent simples et **portables** : c'est tout l'intérêt du mille-feuille, où chaque couche ne parle qu'à celle juste en dessous.

## Ligne de commande Linux

**Exercice 15** *Se repérer* ↑énoncé p. 368

**▶ Solution.** a) `pwd`. b) De la racine jusqu'au fichier : `/home/eleve/Images/logo.png`. c) Depuis `Documents`, on remonte d'un cran (`..`, soit `/home/eleve`) puis on redescend : `../Images/logo.png`.

▷ **Erreur fréquente.** commencer un chemin relatif par `/` : `/Images/logo.png` désigne un dossier `Images` placé à la racine, qui n'existe pas.

**Exercice 16** *Naviguer* ↑énoncé p. 368

**▶ Solution.** `..` remonte d'un cran ; un chemin qui commence par `/` part de la racine ; `ls` accepte le répertoire à lister en argument. a) `cd Documents` b) `cd ..` c) `cd /etc` d) `ls Images`

**Exercice 17** *Créer, copier, déplacer* ↑énoncé p. 369

**▶ Solution.** Depuis `/home/eleve`, chaque commande vise sa cible par un chemin relatif ; renommer, c'est déplacer (`mv`) vers un nouveau nom.

```
mkdir Projets/Rendus
touch Projets/Rendus/brouillon.txt
cat Documents/cours_nsi.txt
cp Documents/tp_os.txt Projets/Rendus/
mv Projets/Rendus/tp_os.txt Projets/Rendus/tp_final.txt
mv notes.txt Documents/
```

**Exercice 18** *Corriger les erreurs* ↑énoncé p. 369

**▶ Solution.** **a)** `cd` attend un **répertoire**, pas un fichier : `cd /home/eleve/Documents`. **b)** `Images` est un *répertoire* : `rm` seul ne supprime que des fichiers et refuse. Il faut l'option `-r` : `rm -r Images`. **c)** Le fichier est dans `Documents`, pas dans le répertoire courant : `cat ../Documents/cours_nsi.txt`.

**Exercice 19** *Reconstituer une séquence* ↑énoncé p. 369

**▶ Solution.** Ordre : **B** → **C** → **A** (créer le dossier, y créer le fichier, puis l'afficher).

## Droits et permissions

**Exercice 20** *Lire les droits* ↑énoncé p. 369

**▶ Solution.** **a)** `script.sh` est un **fichier** (premier caractère `-`). Propriétaire : `rwx` (lecture, écriture, exécution) ; groupe : `r-x` (lecture, exécution, *pas* d'écriture) ; autres : `r--` (lecture seule). **b)** Un membre du groupe `nsi` **peut l'exécuter** (`x` présent) mais **ne peut pas le modifier** (pas de `w`). **c)** Le `d` indique un **répertoire**. Les « autres » ont `---` : ils **ne peuvent pas** y entrer (pas de droit `x`).

▷ **Erreur fréquente.** découper les droits sans mettre de côté le premier caractère : `-rwxr-xr--` se lit `-` (le type), puis `rwx`, `r-x`, `r--`.

**Exercice 21** *Modifier les droits* ↑énoncé p. 369

▶ **Solution.** On suit la grammaire `chmod [ugoa][+-][rwx] fichier` : pour qui, ajouter ou retirer, quel droit. a) `chmod o-r cours_nsi.txt` b) `chmod g+w cours_nsi.txt` c) `chmod u+x script.sh` d) `chmod a+r logo.png`

**Exercice 22** *Les droits en chiffres* ↑énoncé p. 370

▶ **Solution.** **a)** `rwxr-xr-x` → `755` $(4+2+1,\ 4+1,\ 4+1)$ ; `rw-r--r--` → `644` ; `rwx------` → `700` ; `r--r-----` → `440`. **b)** `640` → `rw-r-----` $(6 = 4+2$ : lecture et écriture ; $4$ : lecture ; $0$ : rien) ; `751` → `rwxr-x--x` ; `600` → `rw-------`. **c)** Propriétaire $4+2+1=7$, groupe $4+1=5$, autres $0$ : `chmod 750 script.sh`.

▷ **Erreur fréquente.** donner au `r` la valeur 1 et au `x` la valeur 4 : les poids sont 4, 2, 1 dans l'ordre `r`, `w`, `x`. Ainsi `r-x` vaut 5 et `rw-` vaut 6.

**d)** Une catégorie compte 3 droits, chacun présent ou absent : c'est un nombre binaire de 3 bits, donc au plus $111_2 = 4+2+1 = 7$ (et au moins 0). Les 9 droits étant chacun présent ou absent, il y a $2^9 = 512$ réglages ($8$ chiffres possibles pour chacune des 3 catégories : $8 \times 8 \times 8 = 512$).

**Exercice 23** *Le compte* `root` ↑énoncé p. 370

▶ **Solution.** **a)** `root` est l'**administrateur** (super-utilisateur) : il a **tous** les droits sur tous les fichiers et peut modifier les droits et les comptes. **b)** Une fausse manipulation ou un logiciel malveillant lancé en `root` peut **tout détruire**, sans garde-fou ; en compte normal, les droits **limitent** les dégâts. **c)** Les droits empêchent un utilisateur de lire ou modifier les fichiers des autres, et restreignent ce qu'un programme peut faire : ils assurent **confidentialité** et **protection** sur une machine partagée.

## Mini-problème de synthèse

**Exercice 24** *Du transistor au fichier* ↑énoncé p. 370

▶ **Solution.** **a)** Tant qu'il n'a pas enregistré, le texte est en **RAM** (mémoire volatile) : une coupure de courant le **perd**. **b)** `cat Projets/jeu.py` (chemin relatif depuis `/home/leo`). **c)** `chmod u+x Projets/jeu.py` : `u` (le propriétaire), `+x` (ajouter l'exécution) ; les autres droits ne bougent pas. (En chiffres, `chmod 700 Projets/jeu.py` donnerait `rwx------` : il retirerait aussi les droits du groupe et des autres, ce qui n'est pas demandé.) **d)** Le processeur ne comprend que le **langage machine**. Le code Python est donc d'abord **traduit** par l'interpréteur Python (en simplifiant : il le traduit en un code intermédiaire, qu'il exécute ensuite instruction par instruction) ; tout en bas, ce sont des instructions en langage machine que le processeur exécute une à une, en répétant le **cycle chercher-décoder-exécuter**. Le programme « lisible » du haut du mille-feuille finit toujours en instructions élémentaires tout en bas.

**Exercice 25** *Le modèle de von Neumann selon un assistant d'IA* ↑énoncé p. 370

▶ **Solution.** **a)** Non. Les quatre unités, le processeur, les registres, les bus et le cycle chercher-décoder-exécuter sont conformes au cours ; la dernière phrase le contredit frontalement. Le cours dit : « dans le modèle de von Neumann, le **programme est rangé dans la même mémoire que les données**, sous forme de nombres ». **b)** L'erreur est « une mémoire séparée de celle des données ». Correction : instructions et données partagent **la même mémoire**, dans des cases adressées de la même façon ; c'est même l'idée centrale du modèle (« le programme est une donnée »), celle qui rend la machine **universelle** : changer de programme, c'est changer des nombres en mémoire. On le voit dans le LMC, où une instruction et une donnée occupent le même genre de case. **c)** Relire la définition du cours (ou le schéma des quatre unités, où une seule boîte « MÉMOIRE » contient « programme + données »). Quand une réponse présente un « point essentiel », c'est celui-là qu'il faut vérifier en premier.

### Entraînement

**Exercice 26** *Lire et fixer des droits* ↑énoncé p. 371

**▶ Solution.** **1.** a) `rwxr-x---` : le propriétaire lit, écrit et exécute ; le groupe lit et exécute ; les autres n'ont aucun droit. $7 = 4+2+1$, $5 = 4+1$, $0$ : `chmod 750 f`. b) `rw-rw-r--` : propriétaire et groupe lisent et écrivent, les autres lisent seulement : `chmod 664 f`. c) `r--------` : seul le propriétaire peut lire : `chmod 400 f`. d) `rwxr-xr-x` : tous lisent et exécutent, seul le propriétaire écrit : `chmod 755 f`.

**2.** a) `chmod o-r notes.txt` ; b) `chmod g+w notes.txt` ; c) `chmod a+x notes.txt`.

**Exercice 27** *Chemins absolus et relatifs* ↑énoncé p. 371

**▶ Solution.** Chemin absolu, puis chemin relatif :

a) `/home/eleve/notes.txt` ; `../notes.txt` ;
b) `/etc/hosts` ; `../../etc/hosts` (on remonte à `/home`, puis à la racine) ;
c) `/home/eleve/Documents/cours_nsi.txt` ; `../Documents/cours_nsi.txt` ;
d) `/home/eleve/Images/logo.png` ; `home/eleve/Images/logo.png` (même chemin, sans le `/` initial).

▷ **Erreur fréquente.** oublier qu'un chemin relatif dépend du répertoire courant : `../notes.txt` désigne `notes.txt` depuis `Documents`, mais un tout autre fichier (qui n'existe pas) depuis `/home/eleve`.

# CORRIGÉS DES TP



## TP Sous le capot ↑énoncé p. 373

Les programmes LMC ci-dessous utilisent les mêmes codes d'instructions que le simulateur de Peter Higginson. Les sorties de terminal de la partie B sont celles de JSLinux (Alpine Linux 3.12.0, console x86).

### Partie A — Le Little Man Computer

**Exercice 1** *Prise en main : la somme, vue de l'intérieur*

**1.** Cases 00 à 09 : `901 308 901 309 508 109 902 000 000 000`. **2.** `INP` → 901, `STA` → 3…, `LDA` → 5…, `ADD` → 1…, `OUT` → 902, `HLT` → 000. Les deux derniers chiffres sont l'**adresse** de la case visée (`a` est en 08, `b` en 09). **3.** Le compteur de programme avance de **1** à chaque instruction (il est incrémenté au moment où l'instruction est *cherchée*) ; à l'arrêt, il vaut **08** : l'instruction `HLT` (case 07) a déjà été lue. **4.** Le programme est fait de **nombres rangés dans la même mémoire que les données** (les cases 08 et 09 contiennent les données, les cases 00 à 07 le programme) : c'est l'idée clé de von Neumann.

## Exercice 2 — *Le programme mystère*

```
00  INP
01  STA 09
02  INP
03  ADD 09
04  STA 09
05  ADD 09
06  OUT
07  HLT
```

Déroulé pour 5 puis 3 : ACC = 5, case 09 = 5 ; ACC = 3 ; ACC = 3 + 5 = 8 ; case 09 = 8 ; ACC = 8 + 8 = 16 ; sortie **16**.
Le programme calcule $2 \times (a + b)$.
Assemblé, ce code redonne exactement les nombres de l'énoncé (les cases 08 et 09 restent à 000 avant l'exécution).

## Exercice 3 — *L'écart entre deux nombres*

```
        INP
        STA a
        INP
        STA b
        SUB a
        BRP fin
        LDA a
        SUB b
fin     OUT
        HLT
a       DAT
b       DAT
```

Tests : (2, 7) → 5 ; (7, 2) → 5 ; (4, 4) → 0.
Après `SUB a`, l'accumulateur contient $b - a$. S'il est positif ou nul, c'est l'écart et l'on saute directement à `OUT` ; sinon on recalcule $a - b$. C'est un `if ... else` : `BRP` joue le rôle du test.

## Exercice 4 — *Le compte à rebours*

```
        INP
boucle  OUT
        BRZ fin
        SUB un
        BRA boucle
fin     HLT
un      DAT 1
```

Tests : 3 → 3, 2, 1, 0 ; 0 → 0.
**1.** `BRA boucle` revient en arrière ; `BRZ fin` fait sortir quand ACC vaut 0. **2.** Dans une case initialisée par `un DAT 1`. **3.** Sans `BRZ`, la boucle ne s'arrête plus d'elle-même : le programme affiche des nombres négatifs indéfiniment (il faut l'arrêter).

Autre solution correcte : ranger $n$ dans une case et faire `LDA`/`SUB`/`STA` à chaque tour (plus long mais correct).

## Exercice 5 *La multiplication sans multiplier*

```
        INP
        STA a
        INP
        STA b
boucle  LDA b
        BRZ fin
        SUB un
        STA b
        LDA res
        ADD a
        STA res
        BRA boucle
fin     LDA res
        OUT
        HLT
a       DAT
b       DAT
res     DAT 0
un      DAT 1
```

**1.** Tests : $6 \times 7 \to 42$ ; $5 \times 0 \to 0$ ; $0 \times 5 \to 0$.
**2.** La machine exécute $8b + 9$ instructions : 4 pour les entrées, 8 par tour de boucle, 2 pour le dernier test, 3 pour finir. Pour $3 \times 200$ : **1 609** instructions ; pour $200 \times 3$ : **33** seulement. Pour aller plus vite, il faut boucler sur le **plus petit** des deux nombres : comparer $a$ et $b$ au début (`SUB` puis `BRP`) et les échanger si besoin.
*Même résultat, coût très différent : c'est déjà une question d'algorithmique.*

## Exercice 6 *Le plus grand de trois*

```
        INP
        STA a
        INP
        STA b
        INP
        STA c
        LDA a
        STA max
        LDA b
        SUB max
        BRP prendb
        BRA testc
prendb  LDA b
        STA max
testc   LDA c
        SUB max
        BRP prendc
        BRA fin
prendc  LDA c
        STA max
fin     LDA max
        OUT
        HLT
a       DAT
b       DAT
c       DAT
max     DAT
```

Tests : (9, 4, 7) → 9 ; (4, 9, 7) → 9 ; (4, 7, 9) → 9 ; (7, 7, 2) → 7. Le programme renvoie le bon maximum pour chacun des 125 triplets formés avec 0, 1, 4, 7, 9.
C'est l'**invariant du champion** du chapitre *Algorithmique : le parcours séquentiel* : `max` contient le plus grand nombre vu jusque-là.
Erreur fréquente : oublier `BRA testc` (ou `BRA fin`) ; la machine « tombe » alors dans le bloc suivant et prend toujours `b` (ou `c`).

### Exercice 7 *Division euclidienne*

```
        INP
        STA a
        INP
        STA b
boucle  LDA a
        SUB b
        BRP suite
        BRA fin
suite   STA a
        LDA q
        ADD un
        STA q
        BRA boucle
fin     LDA q
        OUT
        LDA a
        OUT
        HLT
a       DAT
b       DAT
q       DAT 0
un      DAT 1
```

Tests : (17, 5) → 3 puis 2 ; (15, 5) → 3 puis 0 ; (4, 9) → 0 puis 4. Le programme donne le bon quotient et le bon reste pour tous les $a$ de 0 à 59 et $b$ de 1 à 11.
Si $b = 0$, `a - b` reste toujours positif : la boucle ne s'arrête jamais (`q` augmente sans fin). Un programme correct testerait $b$ dès le départ.
Équivalent Python :

```python
q = 0
while a >= b:
    a = a - b
    q = q + 1
print(q, a)
```

## Partie B — Linux en ligne de commande

### Exercice 8 *Où suis-je, qui suis-je ?*

**1.** `whoami` affiche `root` ; `pwd` affiche `/root` (c'est le répertoire personnel de l'administrateur). **2.** `ls -l` montre `-rw-r--r--  1 root root  151 ... readme.txt` : fichier ordinaire, propriétaire `root` en lecture-écriture, groupe et autres en lecture seule.

### Exercice 9 *Construire l'arborescence du club*

```
cd
mkdir club
cd club
mkdir archives docs projets
mkdir projets/jeu projets/site
echo 'Reunion jeudi 13h' > docs/annonce.txt
cat docs/annonce.txt
Reunion jeudi 13h
tree
```

```
.
|-- archives
|-- docs
|   `-- annonce.txt
`-- projets
    |-- jeu
    `-- site

5 directories, 1 file
```

(`tree` dessine l'arbre avec des traits continus ; `ls -R` liste le contenu de chaque répertoire, l'un après l'autre.)

### Exercice 10 *Un script et le droit* `x`

```
echo 'echo Bonjour le club' > bonjour.sh
ls -l bonjour.sh
-rw-r--r--    1 root     root            21 ...  bonjour.sh
./bonjour.sh
sh: ./bonjour.sh: Permission denied
sh bonjour.sh
Bonjour le club
chmod u+x bonjour.sh
ls -l bonjour.sh
-rwxr--r--    1 root     root            21 ...  bonjour.sh
./bonjour.sh
Bonjour le club
```

**2.** Le fichier n'a le droit `x` pour personne : le système refuse de l'exécuter, *même pour* `root`. **3.** Avec `sh bonjour.sh`, c'est le programme `sh` (l'interpréteur de commandes) qui est exécuté ; il se contente de **lire** `bonjour.sh`, ce qui ne demande que le droit `r`. **4.** Un `x` est apparu chez le propriétaire.

### Exercice 11 *Copier, déplacer, renommer, supprimer*

```
cp docs/annonce.txt archives/
mv archives/annonce.txt archives/annonce_sept.txt
mv bonjour.sh projets/jeu/
./projets/jeu/bonjour.sh
Bonjour le club
rm -r projets/site
tree
```

```
.
|-- archives
|   `-- annonce_sept.txt
|-- docs
|   `-- annonce.txt
`-- projets
    `-- jeu
        `-- bonjour.sh

4 directories, 3 files
```

`rm -r` supprime **définitivement** un répertoire et tout ce qu'il contient, sans corbeille ni confirmation : une faute de frappe sur le nom peut effacer tout un travail.

### Exercice 12 *L'administrateur n'a pas de limites*

```
echo 'code: 1234' > secret.txt
chmod 000 secret.txt
ls -l secret.txt
----------    1 root     root            11 ...  secret.txt
cat secret.txt
code: 1234
```

Personne n'a de droit sur le fichier… et pourtant `root` le lit : l'administrateur **ignore les droits de lecture et d'écriture** (seule l'exécution exige au moins un `x`, comme on l'a vu). Une erreur ou

un programme malveillant lancé en `root` n'est donc arrêté par **aucune** protection : d'où le conseil de n'utiliser ce compte qu'en cas de besoin.

**Exercice 13** *Mini-scénario : le dossier partagé*

**1.** Résultats obtenus (`notes.txt` est en `-rw-r-----`, `annonce.txt` en `-rw-r--r--`, `/tmp/partage` en `drwxr-xr-x`, tous à `root`) :

| commande (`lea`) | résultat |
|---|---|
| `cat annonce.txt` | `Sortie au musee le 12` : les autres ont `r` |
| `cat notes.txt` | `cat: can't open 'notes.txt': Permission denied` |
| `echo 'Lea vient' >> annonce.txt` | `ash: can't create annonce.txt: Permission denied` (pas de `w`) |
| `touch essai.txt` | `touch: essai.txt: Permission denied` (pas de `w` sur le répertoire) |
| `cat /root/club/docs/annonce.txt` | `Reunion jeudi 13h` : `r` sur le fichier, `x` sur les répertoires |
| `cd ; touch essai.txt` | réussit : `/home/lea` appartient à `lea` |

**2.**

```
chmod o-rx /root/club
chmod o+w /tmp/partage/annonce.txt
chmod go+r /tmp/partage/notes.txt
```

Pour `lea` ensuite : `annonce.txt` est toujours lisible, et maintenant **modifiable** (la ligne `Lea vient` s'ajoute) ; `notes.txt` devient lisible (`Lea 15, Tom 12`) ; `touch essai.txt` est toujours refusé ; `cat /root/club/docs/annonce.txt` est désormais refusé (`Permission denied`) : sans le droit `x` sur `club`, on ne peut plus *traverser* ce répertoire, même si le fichier lui-même est lisible.

**3.** `rm /tmp/partage/annonce.txt` donne `rm: can't remove '/tmp/partage/annonce.txt': Permission denied`. Supprimer un fichier, c'est **modifier le répertoire** qui le contient (retirer son nom de la liste) : il faut le droit `w` sur `/tmp/partage`, qui est en `drwxr-xr-x`. Le droit `w` sur le fichier permet d'en changer le *contenu*, pas de le faire disparaître.

## Bilan

**1.** Un `if` : un calcul (souvent `SUB`) suivi d'un saut conditionnel `BRZ` ou `BRP`, et d'un `BRA` pour sauter par-dessus le bloc « sinon ». Une boucle `while` : une étiquette en haut, un test de sortie (`BRZ`/`BRP` vers `fin`) et un `BRA` qui remonte. **2.** La mémoire ne contient que des nombres : les instructions sont codées (`ADD 09` → `109`). Les mots sont pour les humains (l'assembleur), les nombres pour la machine (le langage machine). **3.** Tableau complété :

| pour… | lire un fichier | modifier un fichier | créer/supprimer dans un répertoire |
|---|---|---|---|
| il faut le droit… | `r` | `w` | `w` (et `x`) |
| sur… | le fichier (et `x` sur les répertoires traversés) | le fichier | le répertoire |
| `root` est-il concerné ? | non | non | non |

Seule exception observée : pour *exécuter* un fichier, même `root` a besoin qu'au moins un `x` soit présent (exercice 10).

## Réponses des questions de la fiche de révision

↑ questions p. 379

### Se tester

1. l'unité de commande et l'UAL (avec les registres) (§ II)
2. 14 (il affiche le double) (§ III.C)
3. oui : il était en RAM, mémoire volatile (§ II)
4. `cat ../Documents/cours.txt` (§ V.A)
5. lire seulement ; `chmod g-w fichier` (§ VI)

### Et avant ?

1. des chaînes (`str`) : convertir avec `int` ou `float` pour calculer (ch. 8, § III)
2. `[1, 2, 4, 5]` : le minimum `1` est échangé avec `5` (ch. 6, § II)
3. `for cle, valeur in d.items():` (ch. 3, § VI.B)

CORRIGÉS DU CHAPITRE 10

# 10 Le Web : réseaux et interactions homme-machine

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 Le réseau qui perd des papiers

↑ énoncé p. 386

**Exercice 1** *Manche 1 : on envoie, c'est tout*

▶ **Solution.** **1.** `BALON` (le premier `L` est perdu).

**2.** Non : `BALON` ressemble à un mot plausible, rien ne signale le trou. L'émetteur non plus ne sait rien : il n'a aucun retour.

**3.** « Le récepteur renvoie un papier pour dire qu'il a bien reçu » (ou : « on attend la réponse avant d'envoyer la suite »). C'est exactement la règle de la manche 2.

**Exercice 2** *Manche 2 : on répond « reçu »*

▶ **Solution.** **4.** `BALLLON` (trois `L`).

**5.** Le premier `L` est bien arrivé, mais son papier « reçu » a été perdu. Faute de réponse au bout de 10 secondes, l'émetteur a **renvoyé** ce `L`, que le récepteur a recopié une deuxième fois ; puis l'émetteur a envoyé le vrai deuxième `L`.

**6.** Non : un papier `L` ressemble à un autre papier `L`. Comme `BALLON` contient *vraiment* deux `L` de suite, recevoir deux `L` d'affilée n'a rien d'anormal : impossible de savoir, au moment de la réception, si c'est un renvoi ou la lettre suivante.

**7.** Les lettres perdues sont réparées, mais un papier « reçu » perdu provoque un **doublon** : le récepteur reçoit deux fois la même donnée sans pouvoir la reconnaître.

**Exercice 3** *Manche 3 : on numérote, mais avec un seul chiffre*

▶ **Solution.** **8.** Avec la feuille de route (2e papier de lettre perdu, 4e « reçu » perdu), l'émetteur fait **8 envois** :

| lettre | `B` | `A` | `A` | `L` | `L` | `L` | `O` | `N` |
|---|---|---|---|---|---|---|---|---|
| chiffre | `0` | `1` | `1` | `0` | `1` | `1` | `0` | `1` |
| ce qui arrive | reçu | *perdu* | reçu | reçu | reçu (1) | *doublon jeté* | reçu | reçu |

(1) la lettre arrive, mais c'est son « reçu » qui est perdu.

Le 3e envoi est le renvoi du `A` perdu ; le 6e est le renvoi du second `L` (son « reçu » s'est perdu) : le récepteur attendait `0`, il reçoit `1`, déjà traité, donc il le jette.

**9.** `BALLON` : correct.

**10.** Si le récepteur jetait le doublon sans répondre, l'émetteur, toujours sans « reçu », renverrait encore et encore le même papier : la transmission serait bloquée. Le « reçu » du doublon remplace celui qui s'est perdu.

**11.** L'émetteur attend le « reçu » avant d'envoyer la suite : il n'y a jamais qu'**une** lettre en route. Le récepteur doit seulement distinguer « la même que la précédente » de « la suivante » : deux cas, donc un seul chiffre `0/1` (un **bit**) suffit.

**Exercice 4** *Mettre des mots*

▶ **Solution.** **12.** « …le récepteur renvoie un **accusé de réception**. Sans réponse au bout d'un certain **délai**, l'émetteur **renvoie** le même paquet. Pour ne pas confondre un renvoi avec un nouveau paquet, on marque chaque paquet d'un **bit** qui vaut alternativement `0` et `1`. »
**13.** Un **protocole**.

# CORRIGÉS DES EXERCICES

> **Remarque**
> On donne **une** version simple ; d'autres écritures sont possibles.

## Réseaux : comment les machines se parlent

**Exercice 1** *Internet ou Web ?* ↑énoncé p. 406

▶ **Solution.** Internet est le réseau (les machines et leurs liaisons) ; le Web est un service qui l'utilise (des pages reliées par des liens). a) **Web** ; b) **Internet** (le courriel est un service d'Internet, pas du Web) ; c) **Internet** ; d) **Web**.

▷ **Erreur fréquente.** employer « Internet » et « le Web » comme synonymes : le courriel, les jeux en ligne ou la visioconférence utilisent Internet sans passer par le Web.

**Exercice 2** *Le bon mot* ↑énoncé p. 406

▶ **Solution.** On suit le trajet d'un message : on trouve l'adresse, on découpe, on aiguille, en respectant des règles communes. a) le **DNS** traduit un nom de domaine en **adresse IP** ; b) **paquets** ; c) **routeur** ; d) **protocole**.

**Exercice 3** *Client-serveur ou pair-à-pair ?* ↑énoncé p. 406

▶ **Solution.** **a)** Des machines **clientes** *demandent* des ressources ; une machine **serveur** les *fournit*. **b)** En pair-à-pair, votre machine est **à la fois cliente et serveur** (elle reçoit et fournit). **c) Client** : elle demande des pages aux serveurs web et ne fournit rien en retour.

**Exercice 4** *Le voyage d'un paquet* ↑énoncé p. 406

▶ **Solution.** **a)** L'adresse IP de **destination** `193.51.208.14` *et* l'adresse IP **source** `192.168.1.10` (sans elle, le serveur ne saurait pas à qui répondre). *Précision* : `192.168.1.10` est une adresse **privée** ; en sortant vers Internet, la box la remplace par son adresse publique, et c'est à la box que le serveur répond, qui transmet ensuite la réponse à la machine (traduction d'adresses, NAT). **b)** Oui, deux paquets peuvent suivre des **chemins différents** ; un paquet perdu est simplement **renvoyé**.

**Exercice 5** *Dérouler le protocole du bit alterné* ↑énoncé p. 407

▶ **Solution.** **a)** Le bit `0/1` sert à **distinguer un nouveau paquet d'un paquet renvoyé** : si Bob reçoit deux fois de suite un paquet portant le même bit, le second est un **doublon**.

**b) Scénario 1 (tout arrive).** 4 paquets partent, 4 accusés reviennent : **4 paquets et 4 accusés**, aucun renvoi.

**c) Scénario 2 (le 2e paquet, étiquette `1`, se perd).** Alice envoie le paquet étiqueté `1` et attend son accusé. Comme le paquet n'arrive pas à Bob, aucun accusé n'est renvoyé ; au bout du

délai, Alice **renvoie le même paquet** `1`. Cette fois il arrive, Bob le traite et renvoie l'accusé. Le 2e paquet a donc été envoyé **2 fois**. Alice passe ensuite au paquet `0` suivant.

**d) Scénario 3 (l'accusé du 2e paquet se perd).** Bob a bien reçu et traité le 2e paquet (étiquette `1`), mais son accusé se perd. Faute d'accusé, Alice **renvoie le 2e paquet (étiquette `1`)**. Bob constate qu'il porte **le bit qu'il vient déjà de traiter** : c'est un doublon, il le **jette** (sans le traiter une deuxième fois) mais **renvoie quand même l'accusé**, pour qu'Alice puisse avancer.

## URL et protocole HTTP

**Exercice 6** IA IA IA *Décomposer une URL* ↑énoncé p. 407

▶ **Solution.** On lit l'URL de gauche à droite : protocole avant `://`, domaine jusqu'au premier `/`, puis chemin. a) protocole `https` : oui, **sécurisée** (chiffrée), on le voit au « s » de `https`. b) `www.lycee.mc`. c) `/nsi/web/cours.html`.

▷ **Erreur fréquente.** inclure le protocole ou le chemin dans le nom de domaine : le domaine est seulement `www.lycee.mc`, entre `://` et le premier `/`.

**Exercice 7** IA IA IA *Les codes de réponse HTTP* ↑énoncé p. 407

▶ **Solution.** a) `200` ; b) `404` ; c) `500` ; d) `403` ; e) `304`. Les codes en `4..` sont des erreurs du client (mauvaise adresse, accès refusé), ceux en `5..` des erreurs du serveur.

**Exercice 8** IA IA IA *Lire un échange HTTP* ↑énoncé p. 407

▶ **Solution. a)** La première ligne (`GET...`) est envoyée par le **client** ; les deux lignes suivantes (`HTTP/1.1 404...` et `Content-Type...`) forment la **réponse du serveur**. **b)** La méthode est `GET`. **c) 404** signifie « ressource **introuvable** » : elle n'a pas été trouvée. En cas de succès, le serveur renverrait `200 OK` suivi du code HTML de la page.

▷ **Erreur fréquente.** croire qu'un code `404` signale une panne du serveur : le serveur a bien répondu ; c'est la ressource demandée qui n'existe pas (les pannes du serveur sont les codes `5..`).

**Exercice 9** IA IA IA `GET` *ou* `POST` *?* ↑énoncé p. 407

▶ **Solution. a)** `GET` : on *demande* une page. **b)** `POST` : un mot de passe est sensible, il ne doit **pas** apparaître dans l'URL. **c)** `GET` : la recherche apparaît dans l'URL (`...?q=chat`), donc partageable et gardable en favori.

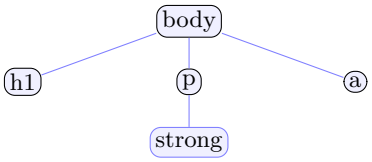
## HTML et CSS

**Exercice 10** IA IA IA *À chaque balise son rôle* ↑énoncé p. 408

▶ **Solution.** a) `<a>` ; b) `<li>` ; c) `<h1>` ; d) `<img>` ; e) `<p>` ; f) `<ul>`. La balise `<img>` n'a pas de balise fermante : elle n'entoure aucun contenu (`<img src="logo.png" alt="logo">`).

**Exercice 11** IA IA IA *Lire du HTML* ↑énoncé p. 408

▶ **Solution. a)** Titre : `<h1>` ; paragraphe : `<p>` ; lien : `<a>`. **b)** Vers `dessert.html` ; c'est un chemin **relatif** (pas de `/` ni de domaine au début). **c)** Arbre (DOM) :



**Exercice 12** *Corriger le HTML* ↑énoncé p. 408

▶ **Solution.** Les balises `<strong>` et `<p>` sont **mal imbriquées**, et le `<h2>` n'est pas fermé :

```
<p>Un paragraphe <strong>en gras</strong></p>
<h2>Un sous-titre</h2>
```

▷ **Erreur fréquente.** fermer les balises dans l'ordre où on les a ouvertes : la dernière ouverte (`<strong>`) doit être la première fermée.

**Exercice 13** *Ma première page* ↑énoncé p. 408

▶ **Solution.** On part du squelette du cours (`<!doctype html>`, puis `<head>` pour les informations sur la page et `<body>` pour ce qui s'affiche) ; le lien vers un autre site prend une adresse complète dans `href`.

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Mon site NSI</title>
  </head>
  <body>
    <h1>Mon site NSI</h1>
    <p>Bienvenue sur ma page de presentation.</p>
    <a href="https://eduscol.education.fr">Eduscol</a>
  </body>
</html>
```

**Exercice 14** *Relier le CSS* ↑énoncé p. 408

▶ **Solution.** **a)** Les `<h1>` seront en **bleu foncé** et **centrés**. **b)** L'élément qui porte `id="avertissement"` sera en **rouge**, quelle que soit sa balise : le CSS seul ne permet pas de savoir s'il s'agit d'un `<p>`, d'un `<div>`… (il faut lire le HTML). **c)** La **séparation du fond (HTML) et de la forme (CSS)** : on change l'apparence sans toucher au contenu. Pour un site de plusieurs pages, toutes les pages se relient au même fichier `.css` : une seule modification change l'apparence de tout le site, et le style reste identique d'une page à l'autre.

**Exercice 15** `id` *ou* `class` *?* ↑énoncé p. 409

▶ **Solution.** **a)** `titre` est un `id` car il désigne **une seule** balise (unique) ; `plat` est une `class` car elle est **partagée** par plusieurs balises. **b)** et **c)** :

```
#titre { color: blue; }    /* # pour l'identifiant unique */
.plat  { color: green; }   /* . pour la classe partagee : les deux plats */
```

▷ **Erreur fréquente.** inverser les symboles : `#` désigne un `id`, `.` une classe. `#plat` ne désigne aucun élément ici.

**Exercice 16** *Aligner avec Flexbox* ↑énoncé p. 409

▶ **Solution.**

```
<div class="barre">
  <a href="#">Accueil</a>
  <a href="#">Cours</a>
  <a href="#">Contact</a>
</div>
```

**a)** et **b)** :

```
.barre {
    display: flex;
    gap: 15px;                    /* a) espace entre les liens */
    justify-content: space-between;  /* b) pousse les liens aux extremites */
}
```

**b)** `space-between` **répartit** les liens sur toute la largeur, le premier collé à gauche, le dernier à droite, l'espace restant distribué entre eux. **c)** Sur le `<div>` (le **conteneur**) : c'est lui qui devient une boîte flexible, et ce sont *ses enfants* (les `<a>`) qui s'alignent.

## JavaScript : rendre la page vivante

### Exercice 17 *Prévoir l'effet* ↑énoncé p. 409

▶ **Solution.** **a)** Au clic, le paragraphe « Bonjour » devient « **Bonsoir !** ». **b)** On réagit à l'événement `click` (le clic sur le bouton), branché par l'attribut `onclick` ; `querySelector("#message")` **sélectionne** dans le DOM la balise d'`id message`. **c)** Ce traitement s'exécute **côté client** (dans le navigateur).

### Exercice 18 *Un compteur de clics* ↑énoncé p. 409

▶ **Solution.**

```html
<p id="score">0</p>
<button onclick="compter()">+1</button>
<script src="script.js"></script>
```

```javascript
function compter() {
    let e = document.querySelector("#score");
    let n = parseInt(e.innerHTML);
    e.innerHTML = n + 1;
}
```

### Exercice 19 *Additionner deux nombres* ↑énoncé p. 409

▶ **Solution.** **a)** et **b)** (version avec `addEventListener`) :

```javascript
function additionner() {
    let a = parseInt(document.querySelector("#a").value);
    let b = parseInt(document.querySelector("#b").value);
    document.querySelector("#resultat").textContent = a + b;
}
// on branche le bouton depuis le JS (pas d'onclick dans le HTML)
document.querySelector("#calc").addEventListener("click", additionner);
```

**c) Non** : le calcul se fait entièrement **côté client**, dans le navigateur.

▷ **Erreur fréquente.** additionner directement les `.value` : ce sont des chaînes, et `+` les colle (`"2" + "3"` donne `"23"`). On convertit d'abord avec `parseInt`.

### Exercice 20 *Mini-projet : un thème clair/sombre* ↑énoncé p. 410

▶ **Solution.** Une page complète possible :

```html
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Theme</title>
  <style>
    body        { font-family: sans-serif; margin: 20px; }
    .barre      { display: flex; justify-content: flex-end; gap: 8px; }
    .sombre     { background: black; color: white; }
  </style>
</head>
<body>
  <div class="barre">
    <button onclick="basculer()">Changer de theme</button>
    <button>A propos</button>
  </div>
```

```
  <h1>Mon site</h1>
  <p>Clique sur le bouton pour changer l'apparence.</p>

  <script>
    function basculer() {
        document.body.classList.toggle("sombre");
    }
  </script>
</body>
</html>
```

Au clic, `classList.toggle("sombre")` **ajoute** la classe si elle est absente, la **retire** sinon : le fond bascule du clair au sombre. La barre de boutons est alignée à droite grâce à `display:flex` + `justify-content: flex-end`.

**Exercice 21** *Client ou serveur ?* ↑énoncé p. 410

**▶ Solution. a)**

1. afficher/masquer un menu : **client** (réaction immédiate d'affichage, aucun besoin du serveur) ;
2. couleur au survol : **client** (souvent même en CSS seul) ;
3. vérifier la présence d'un « @ » avant l'envoi : **client** (confort : prévenir tout de suite), *mais* le serveur devra revérifier ;
4. vérifier un mot de passe : **serveur** (le client ne doit pas connaître les mots de passe enregistrés) ;
5. consulter une base de données : **serveur** (c'est lui qui détient et interroge la base) ;
6. enregistrer une commande : **serveur** (donnée durable et sensible).

**b)** Le JavaScript s'exécute **sur la machine de l'utilisateur** : celui-ci peut le **lire et le modifier** (outils de développement), ou même envoyer la requête sans passer par la page. Une vérification faite *seulement* côté client est donc **contournable** : le serveur, hors de portée de l'utilisateur, doit **refaire** la vérification avant d'accorder la réduction. *Règle générale : tout contrôle sensible se refait côté serveur.*

**c)** Côté client : **JavaScript**. Côté serveur : par exemple **PHP** (ou Python) — il n'est pas demandé d'en développer une expertise, seulement de savoir *qui fait quoi*.

## Les formulaires

**Exercice 22** *Lire un formulaire* ↑énoncé p. 410

**▶ Solution. a)** Vers `/inscription`, avec la méthode `post`. **b)** Sous les noms `prenom` et `courriel` (les attributs `name`). **c) Non** : avec `POST`, les données ne figurent pas dans l'URL.

▷ **Erreur fréquente.** chercher le nom d'envoi dans `type` ou dans le texte affiché : c'est l'attribut `name` qui donne l'étiquette sous laquelle la valeur arrive au serveur.

**Exercice 23** *Un formulaire de recherche* ↑énoncé p. 410

**▶ Solution. a)** Méthode `get`, pour que la recherche figure dans l'URL et puisse être gardée en favori :

```
<form action="/recherche" method="get">
    <input type="text" name="q">
    <input type="text" name="tri">
    <button type="submit">Chercher</button>
</form>
```

**b)** `https://www.recettes.mc/recherche?q=tarte&tri=prix` : l'adresse `action`, puis `?`, puis les paires `name=valeur` séparées par `&`. **c)** « soupe » dans le champ `q`, « date » dans le champ `tri`. **d) Non** : avec `get`, le mot de passe apparaîtrait dans l'URL, donc à l'écran et dans l'historique. On utilise `post` (et une page en `https`).

**Exercice 24** *Ce que retient le navigateur : les cookies* ↑énoncé p. 411

▶ **Solution.** **a)** Dans le **client** : l'en-tête `Set-Cookie` demande au navigateur de les garder. **b)** Le navigateur retransmet de lui-même les deux cookies, séparés par `;` :

```
GET /desserts.html HTTP/1.1
Cookie: theme=sombre; panier=tarte-citron
```

**c)** `theme` : 604 800 secondes, soit $604\,800/(24 \times 3600) = 7$ jours. `panier` n'a pas de durée : il est effacé à la fermeture du navigateur. **d) Non** : un cookie n'est renvoyé qu'au site qui l'a déposé (`www.recettes.mc`). **e)** Le cookie est stocké chez le client : l'utilisateur peut le lire et le **modifier** (par exemple avec les outils de développement) et payer `prix=1`. Le prix doit être calculé et vérifié **sur le serveur** ; un cookie ne doit contenir que des informations sans risque (préférence, identifiant de session).

**Exercice 25** *Le voyage d'un clic (synthèse)* ↑énoncé p. 411

▶ **Solution.** Ordre : **3** (le DNS traduit le nom en IP) → **4** (requête HTTP `GET`) → **5** (les paquets traversent les routeurs) → **2** (réponse HTTP avec le HTML) → **1** (le navigateur assemble et affiche).

**Exercice 26** `GET` *et* `POST` *selon un assistant d'IA* ↑énoncé p. 411

▶ **Solution.** **a)** Non. Le cours dit : « `GET` : les éventuels paramètres sont **visibles dans l'URL** » et « `POST` : les données ne sont **pas** dans l'URL ». En pratique, après une recherche sur un moteur (formulaire en `get`), l'URL contient bien les mots cherchés après le `?` (par exemple `...?q=lycee+monaco`). **b)** L'erreur est l'**inversion** de la dernière phrase. Correction : avec `GET`, les données sont ajoutées à la fin de l'URL après un `?` ; avec `POST`, elles voyagent dans le **corps** de la requête et n'apparaissent pas dans l'URL. Le reste (rôles de `GET` et `POST`, client-serveur, codes de statut) est juste. **c)** Un test d'une minute : soumettre un formulaire en `get` et regarder la barre d'adresse, ou ouvrir l'onglet Réseau des outils de développement (`F12`) et lire la méthode et l'URL de la requête. Le ton assuré ne prouve rien : l'inversion est l'erreur la plus facile à commettre sur deux notions symétriques.



## Entraînement

**Exercice 27** *Lire des URL* ↑énoncé p. 412

▶ **Solution.** a) protocole `http` (non sécurisé), domaine `www.lycee.mc`, chemin `/index.html`, pas de paramètre. b) `https`, `fr.wikipedia.org`, chemin `/wiki/Alan_Turing`, pas de paramètre. c) `https`, `www.recettes.mc`, chemin `/recherche`, paramètres `q` (valeur `soupe`) et `tri` (valeur `date`).

**Exercice 28** *Formulaires :* `GET`, `POST` *et l'URL* ↑énoncé p. 412

▶ **Solution.** a) `GET` : `https://www.site.mc/meteo?ville=Nice`, la valeur est visible dans l'URL. b) `POST` : `https://www.site.mc/connexion` ; `login` et `mdp` voyagent dans le corps de la requête, ils n'apparaissent pas dans l'URL (c'est la bonne méthode pour un mot de passe). c) `GET` : `https://www.site.mc/annonces?cat=velo&prix_max=100`, valeurs visibles. d) Méthode `get` (les valeurs sont dans l'URL), `action="/films"`, deux champs nommés `genre` et `annee`, où l'on avait saisi `comedie` et 2024.

▷ **Erreur fréquente.** croire que `POST` rend les données secrètes : elles ne sont pas dans l'URL, mais restent lisibles dans la requête ; seul `https` les chiffre pendant le transport.

## Réseau : encapsulation et réseau local

**Exercice 29** *Des enveloppes dans des enveloppes* ↑énoncé p. 412

▶ **Solution.** **a)** De l'extérieur vers l'intérieur :

| | | | |
|---|---|---|---|
| **en-tête MAC** : source `3C:22:FB:10:4A:01`, destination `F4:CA:E5:81:02:9B` (la box) | **en-tête IP** : source `192.168.1.23`, destination `51.75.20.30` | **en-tête TCP** : port destination 443, numéro d'ordre | **données HTTP** : `GET /horaires.html` (chiffrées, car `https`) |

**b)** L'**adresse IP de destination** (`51.75.20.30`), dans l'**en-tête IP** : le routeur ouvre la trame jusqu'à cet en-tête, sans avoir besoin de lire TCP ni HTTP. **c)** Les adresses MAC ne servent qu'à un « saut », d'une machine à la suivante sur un même réseau local : chaque routeur reconstruit une nouvelle trame vers son voisin. Les adresses IP désignent la source et la destination **finales** : elles restent les mêmes de bout en bout (sauf la traduction de l'adresse privée par la box, à la sortie du réseau local). **d)** Non : seule la couche liaison change (4G au lieu du Wi-Fi). La requête HTTP, l'en-tête TCP et l'en-tête IP s'écrivent de la même façon. Chaque couche fait son travail **sans connaître les autres** : c'est l'intérêt de l'encapsulation.

**Exercice 30** *Le réseau d'un lycée* ↑énoncé p. 413

▶ **Solution.** **a)** Point d'accès Wi-Fi : (1) ; commutateur : (2) ; pare-feu : (3) ; routeur d'accès : (4) ; serveur : (5). **b)** Le **commutateur** de la salle, (éventuellement un commutateur central), le **pare-feu**, puis le **routeur d'accès**, qui envoie la requête vers Internet. **c)** Non. `10.0.x.x` est une adresse **privée**, valable seulement dans le réseau du lycée. Le routeur d'accès la remplace par l'adresse **publique** du lycée (traduction d'adresses, NAT) : Wikipédia ne voit que cette adresse publique, commune à tous les postes du lycée. **d)** Le **pare-feu** : il bloque les échanges que l'établissement n'autorise pas.

## Programmer une interface d'après un cahier des charges

**Exercice 31** *Une inscription à l'atelier robotique* ↑énoncé p. 413

▶ **Solution.** **a)** Le titre `<h1>`, le paragraphe `#message` et la liste `#inscrits` servent seulement à **afficher** : ils ne traitent aucun événement. Les deux **champs de saisie** reçoivent la frappe au clavier ; leur contenu est lu par `.value` (ils ne déclenchent pas de fonction ici). Le **bouton** traite l'événement `click` : il appelle `inscrire()`.

**b)**

```html
<h1>Inscription à l'atelier robotique</h1>
<input type="text" id="prenom" placeholder="Prénom">
<input type="text" id="classe" placeholder="Classe (ex. 1G2)">
<button onclick="inscrire()">S'inscrire</button>
<p id="message"></p>
<ul id="inscrits"></ul>
<script src="script.js"></script>
```

**c)**

```javascript
function inscrire() {
    let prenom = document.querySelector("#prenom").value;
    let classe = document.querySelector("#classe").value;
    let message = document.querySelector("#message");
    if (prenom === "" || classe === "") {          // C4
        message.textContent = "Remplir les deux champs.";
        message.style.color = "red";
    } else {                                       // C5
```

```
        message.textContent = "Inscription enregistrée : " + prenom;
        message.style.color = "green";
        let liste = document.querySelector("#inscrits");
        liste.innerHTML += "<li>" + prenom + " (" + classe + ")</li>";
        document.querySelector("#prenom").value = "";
        document.querySelector("#classe").value = "";
    }
}
```

C6 est respectée : aucune requête n'est envoyée, tout se passe **côté client**.

**d)** Trois essais possibles :

| prénom | classe | ce qu'on doit observer |
|---|---|---|
| `Léa` | `1G2` | message vert « Inscription enregistrée : Léa » ; la ligne « Léa (1G2) » s'ajoute à la liste ; les deux champs sont vidés (C5) |
| (vide) | `1G2` | message rouge « Remplir les deux champs. » ; liste inchangée ; `1G2` reste dans son champ (C4) |
| `Hugo` | (vide) | message rouge ; liste inchangée ; `Hugo` reste dans son champ (C4) |

Un quatrième essai utile : deux inscriptions valides de suite, pour vérifier que la seconde s'**ajoute** à la première au lieu de la remplacer (d'où le `+=`).

**e)** La liste n'existe que dans la page affichée, dans la mémoire du **navigateur** (côté client) : rien n'a été envoyé au serveur. Recharger la page la redemande au serveur, qui renvoie la page d'origine, avec une liste vide. Pour conserver les inscriptions, il faut les **envoyer au serveur** : un formulaire `<form action="/inscription" method="post">` dont les champs ont un `name` (`prenom`, `classe`), et, côté serveur, un programme qui les enregistre (dans un fichier ou une base de données) et fabrique la page des inscrits. On choisit `POST` : on envoie des données à enregistrer, qui n'ont pas à apparaître dans l'URL.

▷ **Erreur fréquente.** ne tester que le cas où les *deux* champs sont vides : la condition doit être « l'un *ou* l'autre est vide », sinon une inscription sans prénom (ou sans classe) serait acceptée.



# CORRIGÉS DES TP

## TP 1 Mon premier site, pas à pas

↑ énoncé p. 416

**Ancrage BO (1re)** réaliser et analyser une page HTML/CSS simple ; rôles de HTML, CSS et JavaScript ; événements et modification du document ; distinguer ce qui relève du client et du serveur.

**Code** une version possible parmi d'autres : les textes, les couleurs et la disposition sont libres, pourvu que les balises, les sélecteurs et les fonctions demandés y soient.

## 1 La structure HTML (jalon J1)

**Exercice 1** *Découvrir le squelette*

▶ **Corrigé.** **1.** Le navigateur affiche les cinq sous-titres (« Qui sommes-nous ? », « Ce que l'on observe », « Le planning », « Vous aimez le club ? », « S'inscrire à la nuit d'observation ») et le début du tableau, sans aucune mise en forme ; l'onglet porte le titre « Mon site ». Les lignes `TODO` sont écrites entre `<!--` et `-->` : ce sont des **commentaires**, que le navigateur ignore. **2.** Cinq blocs. C'est une `class` parce que plusieurs balises portent la même étiquette, pour recevoir le même style ; un `id` doit être unique dans la page. **3.** Deux lignes (la ligne de titres et celle du lundi), de trois cases chacune.

**Exercice 2** *Titres, annonce et présentation*

▶ **Corrigé.**

```
<title>Club Astro - Accueil</title>
...
<h1>Club Astro</h1>

<p id="annonce">Prochaine nuit d'observation : vendredi à 21 h, sur le toit du lycée.</p>
...
<div class="carte">
  <h2>Qui sommes-nous ?</h2>
  <p>Le Club Astro réunit les élèves curieux du ciel. On y apprend à
     <strong>reconnaître les constellations</strong> et à se servir d'un télescope.
    </p>
</div>

<div class="carte">
  <h2>Ce que l'on observe</h2>
  <ul>
    <li>la Lune et ses cratères</li>
    <li>les planètes Jupiter et Saturne</li>
    <li>les étoiles filantes du mois d'août</li>
  </ul>
</div>
```

Le titre de l'onglet s'affiche dans l'**onglet** du navigateur (et dans les favoris ou l'historique), pas dans la page.

▷ **Erreur fréquente.** écrire les éléments de la liste directement dans `<ul>`, sans `<li>` : ils s'affichent alors à la suite, sans puces.

**Exercice 3** *Compléter le planning*

▶ **Corrigé.**

```
<tr>
  <td>Mercredi</td>
  <td>14 h</td>
  <td>Fabriquer une lunette</td>
</tr>
<tr>
  <td>Vendredi</td>
  <td>21 h</td>
  <td>Nuit d'observation</td>
</tr>
```

La première ligne contient les **titres des colonnes** : ses cases sont des `<th>`, que le navigateur affiche en gras et centrées. Les lignes suivantes contiennent les données, dans des cases ordinaires

`<td>`.

### Exercice 4 — *La barre de navigation et une deuxième page*

▶ **Corrigé. 1.**

```
<div class="barre">
  <a href="index.html">Accueil</a>
  <a href="infos.html">Infos pratiques</a>
  <a href="https://fr.wikipedia.org/wiki/Astronomie">En savoir plus</a>
</div>
```

**2.** `index.html` et `infos.html` sont des chemins **relatifs** : le navigateur cherche ces fichiers dans le dossier de la page affichée. Le lien vers Wikipédia est une adresse complète (protocole, nom de domaine, chemin). Tant que le fichier `infos.html` n'existe pas, le navigateur ne le trouve pas et affiche une page d'erreur (sur un serveur, ce serait l'erreur 404).

**3.** Le fichier `infos.html` :

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Club Astro - Infos pratiques</title>
</head>
<body>

  <h1>Club Astro</h1>

  <div class="barre">
    <a href="index.html">Accueil</a>
    <a href="infos.html">Infos pratiques</a>
    <a href="https://fr.wikipedia.org/wiki/Astronomie">En savoir plus</a>
  </div>

  <div class="carte">
    <h2>Infos pratiques</h2>
    <p>Rendez-vous en salle 204, puis sur le toit du lycée pour les observations.</p>
    <p>À apporter : une lampe rouge et des vêtements chauds.</p>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

▷ **Erreur fréquente.** enregistrer `infos.html` dans un autre dossier (Téléchargements, Bureau…) : le chemin relatif `infos.html` ne le trouve plus. Autre erreur : écrire le lien externe sans `https://` ; le navigateur cherche alors un fichier `fr.wikipedia.org` dans le dossier du site.

## 2 La mise en forme CSS (jalon J2)

### Exercice 5 — *Relier la feuille de style*

▶ **Corrigé. 1.** La règle vise toutes les balises de classe `carte` (les cinq blocs de la page d'accueil). `border` les entoure d'un trait gris continu de 2 pixels ; `padding` laisse une marge *intérieure* entre ce trait et le contenu (avec deux valeurs : 10 pixels en haut et en bas, 20 à gauche et à droite) ; `margin-bottom` laisse une marge *extérieure* de 15 pixels sous chaque carte.

**2.** Dans le `<head>`, à la place du commentaire :

```
<link rel="stylesheet" href="style.css">
```

Après actualisation, les cartes sont encadrées et espacées : la règle `.carte` s'applique.

**3.** Même ligne dans le `<head>` de `infos.html`. Une seule feuille pour tout le site : on écrit chaque règle une fois, toutes les pages ont la même apparence, et une modification du style s'applique partout d'un coup. C'est la séparation du fond (les pages HTML) et de la forme (la feuille CSS).

**Exercice 6** *Mettre en forme la page*

► **Corrigé.**

```
body {
  font-family: sans-serif;
  background-color: aliceblue;
  margin: 20px;
}

h1 {
  color: royalblue;
  text-align: center;
}

#annonce {
  background-color: gold;
  color: black;
  text-align: center;
  padding: 8px;
}

th {
  background-color: midnightblue;
  color: white;
  padding: 5px;
}

td {
  border: 1px solid gray;
  padding: 5px;
}
```

**3.** `annonce` est un **identifiant** (`id="annonce"`) : le sélecteur commence par `#`. Le point est réservé aux classes. (La ligne `color: black;` garde l'annonce lisible en mode nuit, à l'exercice 10.) **5.** Le fond change aussi sur `infos.html` : les deux pages utilisent le même fichier `style.css`.

**Exercice 7** *La barre de navigation avec Flexbox*

► **Corrigé.**

```
.barre {
  display: flex;
  gap: 10px;
  justify-content: space-between;
  align-items: center;
  background-color: midnightblue;
  padding: 10px;
  margin-bottom: 15px;
}

a {
  color: white;
}
```

**2.** `center` regroupe les liens au milieu de la barre ; `space-between` place le premier à gauche, le dernier à droite, et répartit l'espace restant entre eux ; `flex-end` les regroupe à droite. **3.** Le sélecteur par nom de balise `a` vise tous les liens (ici, ce sont tous ceux de la barre).

▷ **Erreur fréquente.** écrire `display: flex` dans la règle des liens `a` : ce sont les *enfants* ; c'est leur conteneur, `.barre`, qui doit devenir flexible.

## 3 Les interactions JavaScript (jalon J3)

**Exercice 8** *Le bouton « J'aime »*

▶ **Corrigé.** Dans `index.html` :

```
<div class="carte">
  <h2>Vous aimez le club ?</h2>
  <button onclick="aimer()">J'aime</button>
  <p>Nombre de j'aime : <span id="nbJaime">0</span></p>
</div>
```

Dans `script.js` et dans `style.css` :

```
function aimer() {
  let zone = document.querySelector("#nbJaime");
  let n = parseInt(zone.textContent);
  zone.textContent = n + 1;
}
```

```
#nbJaime {
  font-size: 24px;
}
```

**4.** Le compteur revient à 0 : actualiser recharge le fichier `index.html`, où le `<span>` contient 0. Le JavaScript n'a modifié que la page affichée, dans la mémoire du navigateur (le DOM) : rien n'a été enregistré, ni dans le fichier, ni sur un serveur.

▷ **Erreur fréquente.** oublier `parseInt` : `zone.textContent` est une chaîne, et `"0" + 1` vaut `"01"`, puis `"011"`… Le compteur colle des 1 au lieu de compter.

**Exercice 9** *Valider l'inscription*

▶ **Corrigé. 1.**

```
<div class="carte">
  <h2>S'inscrire à la nuit d'observation</h2>
  <p>Places restantes : <span id="places">3</span></p>
  <input type="text" id="nom" placeholder="Votre nom">
  <button onclick="inscrire()">S'inscrire</button>
  <p id="reponse"></p>
</div>
```

**2.** Première version, sans les places :

```
function inscrire() {
  let nom = document.querySelector("#nom").value;
  let reponse = document.querySelector("#reponse");
  if (nom === "") {
    reponse.textContent = "Veuillez écrire votre nom.";
    reponse.style.color = "red";
  } else {
    reponse.textContent = "Merci " + nom + ", à vendredi !";
    reponse.style.color = "green";
  }
}
```

**3.** et **4.** Version finale. Avec Léa, Hugo, Inès puis Tom, les trois premiers sont inscrits (les places passent à 2, 1, 0) et Tom reçoit « Désolé Tom, c'est complet ! » ; un clic avec le champ vide donne toujours « Veuillez écrire votre nom. ».

```js
function inscrire() {
  let champ = document.querySelector("#nom");
  let nom = champ.value;
  let reponse = document.querySelector("#reponse");
  let zonePlaces = document.querySelector("#places");
  let places = parseInt(zonePlaces.textContent);
  if (nom === "") {
    reponse.textContent = "Veuillez écrire votre nom.";
    reponse.style.color = "red";
  } else if (places === 0) {
    reponse.textContent = "Désolé " + nom + ", c'est complet !";
    reponse.style.color = "red";
  } else {
    zonePlaces.textContent = places - 1;
    reponse.textContent = "Merci " + nom + ", à vendredi !";
    reponse.style.color = "green";
    champ.value = "";
  }
}
```

**5.** Oui : il suffit d'actualiser la page pour retrouver 3 places, et les outils de développement (`F12`) permettent de modifier la page ou le programme. Cette vérification se fait **côté client**, sur la machine de l'utilisateur, qui la contrôle. Pour qu'elle soit sûre, il faut la faire **côté serveur** : c'est le serveur qui compterait les inscriptions reçues et refuserait la quatrième.

▷ **Erreur fréquente.** écrire `if (nom = "")`, avec un seul `=` : c'est une affectation, pas une comparaison. La variable `nom` reçoit la chaîne vide, la condition est toujours fausse, et la page répond « Merci , à vendredi ! », sans le nom.

▷ **Erreur fréquente.** oublier `parseInt` pour les places : `"0" === 0` est faux (une chaîne n'est pas un nombre) : le club n'est jamais complet, et les places affichées deviennent négatives.



### Exercice 10 *Le mode nuit*

▶ **Corrigé.** Dans `style.css`, puis dans la barre de **chaque** page :

```css
.nuit {
  background-color: black;
  color: tomato;
}
```

```html
<button id="boutonNuit">Mode nuit</button>
```

Dans `script.js` :

```js
function basculerNuit() {
  document.body.classList.toggle("nuit");
}

let boutonNuit = document.querySelector("#boutonNuit");
boutonNuit.addEventListener("click", basculerNuit);   // sans les parentheses
```

Le sélecteur de classe `.nuit` l'emporte sur le sélecteur de balise `body`, plus général : le fond devient noir même si la règle `body` en fixe un autre.

**4.** Non : chaque page est chargée à nouveau, et la classe ajoutée au `<body>` de l'accueil n'existe pas dans `infos.html`. Pour s'en souvenir, le site pourrait faire retenir la préférence par le navigateur dans un **cookie** (cours, § VIII), comme on retient une langue.

▷ **Erreur fréquente.** écrire `addEventListener("click", basculerNuit())`, avec des parenthèses : la fonction est exécutée une fois, tout de suite, au chargement (la page s'ouvre donc en mode nuit), et le clic ne fait plus rien.

▷ **Erreur fréquente.** oublier le bouton dans la barre de `infos.html` : sur cette page, `querySelector` ne trouve aucune balise d'identifiant `boutonNuit` et renvoie `null` ; la console affiche une erreur sur

la ligne de `addEventListener`.

**Exercice 11** *Bonus : le compte à rebours*

**▶ Corrigé.**

```html
<div class="carte">
  <h2>Le décollage</h2>
  <button onclick="compteARebours()">Lancer le compte à rebours</button>
  <p id="decollage"></p>
</div>
```

```javascript
function compteARebours() {
  let texte = "";
  let i = 10;
  while (i > 0) {
    texte = texte + i + "... ";
    i = i - 1;
  }
  texte = texte + "Décollage !";
  document.querySelector("#decollage").textContent = texte;
}
```

Le paragraphe affiche `10... 9... 8... 7... 6... 5... 4... 3... 2... 1... Décollage !`. La boucle s'exécute en une fraction de milliseconde, et la page n'est affichée à nouveau qu'une fois la fonction terminée : tout apparaît d'un coup. Pour montrer un nombre par seconde, il faudrait programmer une attente entre deux étapes, ce qui dépasse le programme.

## Le programme complet

Le fichier `script.js` terminé, bonus compris :

```javascript
// Programme JavaScript du Club Astro

// Le bouton "J'aime" : ajoute 1 au nombre affiche
function aimer() {
  let zone = document.querySelector("#nbJaime");
  let n = parseInt(zone.textContent);
  zone.textContent = n + 1;
}

// L'inscription : refuse un champ vide, puis un club complet
function inscrire() {
  let champ = document.querySelector("#nom");
  let nom = champ.value;
  let reponse = document.querySelector("#reponse");
  let zonePlaces = document.querySelector("#places");
  let places = parseInt(zonePlaces.textContent);
  if (nom === "") {
    reponse.textContent = "Veuillez écrire votre nom.";
    reponse.style.color = "red";
  } else if (places === 0) {
    reponse.textContent = "Désolé " + nom + ", c'est complet !";
    reponse.style.color = "red";
  } else {
    zonePlaces.textContent = places - 1;
    reponse.textContent = "Merci " + nom + ", à vendredi !";
    reponse.style.color = "green";
    champ.value = "";
  }
```

```
}

// Le mode nuit : ajoute la classe nuit au body si elle est absente, la retire sinon
function basculerNuit() {
  document.body.classList.toggle("nuit");
}

let boutonNuit = document.querySelector("#boutonNuit");
boutonNuit.addEventListener("click", basculerNuit);   // sans les parentheses

// Bonus : le compte a rebours, construit avec une boucle while
function compteARebours() {
  let texte = "";
  let i = 10;
  while (i > 0) {
    texte = texte + i + "... ";
    i = i - 1;
  }
  texte = texte + "Décollage !";
  document.querySelector("#decollage").textContent = texte;
}
```

## ACTIVITÉ 2 Simuler un réseau

↑énoncé p. 425

**Exercice 1** *Deux réseaux reliés par un routeur*

► **Solution.** **1.** Oui. Le message va de PC-A au switch S1, puis de S1 à PC-B : les deux machines sont dans le **même réseau local** (`192.168.0.x`), le routeur n'intervient pas.

**2.** Échec (« destination non joignable » ou délai dépassé). Le serveur est dans un **autre réseau** (`192.168.1.x`) : PC-A doit confier le paquet au routeur, mais sans passerelle il ne sait pas à qui le remettre.

**3.** Le `ping` réussit. Trajet : PC-A → S1 → routeur (interface `192.168.0.254`, puis `192.168.1.254`) → S2 → serveur, et la réponse fait le chemin inverse (d'où la passerelle du serveur, elle aussi indispensable).

**4.** Deux étapes : le routeur (`192.168.0.254`), puis le serveur. L'étape intermédiaire est le **routeur**, seule machine qui change le paquet de réseau ; les switchs n'apparaissent pas, car ils ne lisent pas l'en-tête IP.

**Exercice 2** *Ouvrir les enveloppes : l'encapsulation*

► **Solution.** **5.** Oui, la page d'accueil du serveur web de Filius s'affiche. Le navigateur l'a demandée avec le protocole **HTTP** (requête `GET`).

**6.** On voit passer **ARP** (recherche de l'adresse MAC de la passerelle sur le réseau local), **TCP** (ouverture de la connexion, accusés de réception) et **HTTP** (la requête `GET` et la réponse), le tout transporté par **IP**. Pour la requête `GET` : source `192.168.0.1` (PC-A), destination `192.168.1.10` (le serveur).

**7.** De l'extérieur vers l'intérieur : en-tête de trame : **adresses MAC** (de PC-A et de la prochaine machine, ici le routeur) ; en-tête IP : **adresses IP** source `192.168.0.1` et destination `192.168.1.10` ; en-tête TCP : **ports** (80 côté serveur) ; données HTTP : la **requête** `GET /`.

**8.** Le routeur ouvre la trame jusqu'à l'**en-tête IP** : il lui faut l'adresse IP de destination pour aiguiller. Le switch s'arrête à l'**en-tête de trame** : il ne lit que l'adresse MAC pour faire suivre la trame vers le bon câble. Ni l'un ni l'autre ne lit la requête HTTP.

**9.** Seule l'enveloppe extérieure (la trame : Wi-Fi au lieu d'Ethernet) changerait. La requête HTTP, l'en-tête TCP et l'en-tête IP resteraient identiques : le navigateur et le serveur n'ont rien à changer. Chaque couche fait son travail **sans connaître les autres** : on peut changer de support (câble, Wi-Fi, fibre, 4G) sans réécrire les applications. C'est l'intérêt de l'**encapsulation**.

**Exercice 3** *Le réseau d'un établissement*

▶ **Solution.** **10.** (1) **routeur d'accès** (box) : relie le réseau du lycée à Internet et traduit les adresses privées en adresse publique ; (2) **pare-feu** : filtre les échanges entre l'intérieur et l'extérieur ; (3) **commutateurs** (central et de salle) : relient par câble les machines du réseau local ; (4) **point d'accès Wi-Fi** : relie sans fil les tablettes et portables au réseau ; (5) **serveur** : stocke les fichiers des élèves, gère les comptes et les mots de passe.

**11.** Poste de la salle NSI : commutateur de salle → commutateur central → pare-feu → routeur d'accès → Internet. Tablette : point d'accès Wi-Fi → commutateur central → pare-feu → routeur d'accès.

**12.** `10.0.3.17` est une adresse **privée**, valable seulement à l'intérieur du lycée. Le **routeur d'accès** la remplace par l'adresse **publique** du lycée avant que le paquet ne sorte (traduction d'adresses, NAT) ; il redistribue ensuite la réponse au bon poste.

**13.** Non : il reste dans le réseau local. Chemin : poste → commutateur de salle → commutateur central → serveur. Ni le pare-feu ni le routeur d'accès ne sont traversés.

## TP 2 Côté serveur

↑énoncé p. 427



### 1 Un serveur sur ma machine

**Exercice 1** *Premier contact*

▶ **Corrigé.** **1.** Le client est le navigateur, le serveur est le programme Python (Flask) : deux logiciels sur la **même** machine, qui se parlent en HTTP comme s'ils étaient à deux bouts du monde. **2.** Non : le navigateur n'a reçu que la chaîne renvoyée par la fonction, du HTML. Le code Python reste sur le serveur. **3.** Une ligne du type `127.0.0.1 - - [08/Nov/2026 14:05:32] "GET / HTTP/1.1" 200 -` : méthode `GET`, chemin `/`, code `200` (OK). L'adresse `127.0.0.1` est celle du client… c'est-à-dire la machine elle-même. **4.** `404` : la ressource demandée n'existe pas (aucune route `/bidule`) ; erreur du client (famille 4). **5.** Le navigateur affiche une erreur de connexion (« Impossible de se connecter », « Connexion refusée ») : plus aucun programme n'attend de requête sur ce port. Il n'y a pas de code HTTP, puisqu'aucun serveur n'a répondu. **6.** Un site ordinaire utilise le port par défaut (80 en `http`, 443 en `https`), sous-entendu dans l'URL. Notre serveur écoute sur un autre port, 8080 : il faut le préciser.

**Exercice 2** *Une deuxième route*

▶ **Corrigé.**

```
@app.route("/apropos")
def apropos():
    return "<h1>À propos</h1><p>Site d'entraînement de la classe de 1re NSI.</p><p><a href='/'>
    Retour à l'accueil</a></p>"
```

**2.** Deux requêtes `GET` : `/apropos`, puis `/` après le clic sur le lien (plus, parfois, une requête `/favicon.ico` en 404 : le navigateur cherche l'icône de l'onglet). **3.** La route n'existe pas encore pour le serveur en marche : on obtient 404. Le programme a été lu au lancement ; il faut le relancer.

Erreur fréquente : deux fonctions de même nom (deux `def accueil`) : Flask refuse de démarrer (`AssertionError: View function mapping is overwriting...`).

## 2 Des pages fabriquées par le serveur

**Exercice 3** *Un modèle pour l'accueil*

▶ Corrigé.

```
@app.route("/")
def accueil():
    return render_template("accueil.html")
```

**3.** Erreur 404 : le lien existe dans la page, mais aucune route `/heure` n'est encore écrite côté serveur. Un lien n'est qu'une adresse ; c'est le serveur qui décide d'y répondre.

**Exercice 4** *L'heure du serveur*

▶ Corrigé.

```
@app.route("/heure")
def heure():
    maintenant = datetime.now()
    return render_template("heure.html",
                           date=maintenant.strftime("%d/%m/%Y"),
                           heure=maintenant.strftime("%H:%M:%S"))
```

```
<p>Nous sommes le {{ date }} et il est {{ heure }} (heure du serveur).</p>
```

**2.** L'heure change à chaque actualisation : chaque requête fait exécuter la fonction, qui fabrique une page **nouvelle**. Un fichier HTML ordinaire est renvoyé tel quel, identique à chaque fois (page statique) ; ici la page est **dynamique**. **3.** Non : le code source contient la date et l'heure, pas les accolades. Le serveur (le moteur de modèles de Flask) les a remplacées **avant** d'envoyer la réponse ; le navigateur ne sait même pas qu'il y avait un modèle. **4.** L'heure de Tokyo, celle du serveur, car le calcul est fait côté serveur, au moment de la requête. Un programme JavaScript s'exécuterait dans le navigateur, côté client : il afficherait l'heure de l'ordinateur de l'internaute, à Monaco. Même résultat affiché, mais pas exécuté au même endroit.

**Exercice 5** *Un compteur de visites*

▶ Corrigé.

```
compteur = {"visites": 0}       # garde en memoire vive du serveur

@app.route("/")
def accueil():
    # METTRE A JOUR
    compteur["visites"] += 1
    return render_template("accueil.html", nb_visites=compteur["visites"])
```

```
<p>Vous êtes le visiteur numéro <strong>{{ nb_visites }}</strong> depuis le
    lancement du serveur.</p>
```

**2.** Le compteur continue d'augmenter : il est stocké **sur le serveur**, dans la mémoire du programme, et partagé par tous les clients. **3.** Il repart de 0 : la variable vit en mémoire vive tant que le programme tourne. Un vrai site enregistre ce genre de donnée dans un fichier ou une base de données (Terminale). **4.** Le compteur de « j'aime » était une valeur de la page, dans le navigateur (côté client) : actualiser recharge la page d'origine et l'efface. Celui-ci est côté serveur : actualiser envoie une nouvelle requête, qui l'augmente.

> **Remarque** — *Pourquoi un dictionnaire ?*
> Un `compteur = 0` modifié dans la fonction déclencherait une erreur (`UnboundLocalError`) : une variable *affectée* dans une fonction est locale. Modifier une valeur *dans* un dictionnaire n'est pas une affectation de la variable `compteur` : cela évite le mot-clef `global`, non étudié.

## 3 Un formulaire, en POST puis en GET

**Exercice 6** *Envoyer en* `POST`

▶ Corrigé.

```html
<form action="/bonjour" method="post">
  <label for="prenom">Prénom :</label>
  <input type="text" name="prenom" id="prenom">
  <label for="classe">Classe :</label>
  <input type="text" name="classe" id="classe">
  <button type="submit">Envoyer</button>
</form>
```

```python
@app.route("/formulaire")
def formulaire():
    return render_template("formulaire.html")

@app.route("/bonjour", methods=["POST"])
def bonjour():
    prenom = request.form["prenom"]       # donnees dans le corps de la requete
    classe = request.form["classe"]
    return render_template("bonjour.html", prenom=prenom, classe=classe)
```

```html
<h1>Bonjour {{ prenom }} !</h1>
<p>Le serveur a bien reçu votre classe : {{ classe }}.</p>
```

**3.** `http://localhost:8080/bonjour`, sans le prénom : en `POST`, les données voyagent dans le **corps** de la requête, après les en-têtes. **4.** Il demande s'il faut **renvoyer** les données du formulaire : la page a été obtenue par un `POST`, et la redemander, c'est renvoyer ces données (un achat ou un message pourrait être enregistré deux fois). **5.** `405 Method Not Allowed` (famille 4, erreur du client) : taper une adresse envoie toujours un `GET`, et la route n'accepte que `POST`.

**Exercice 7** *Le même en* `GET`

▶ Corrigé.

```python
@app.route("/formulaire_get")
def formulaire_get():
    return render_template("formulaire_get.html")

@app.route("/bonjour_get", methods=["GET"])
def bonjour_get():
    prenom = request.args["prenom"]       # donnees dans l'URL, apres le ?
    classe = request.args["classe"]
    return render_template("bonjour.html", prenom=prenom, classe=classe)
```

Dans `formulaire_get.html`, seule la balise d'ouverture change : `<form action="/bonjour_get" method="get">`. L'écriture `methods=["GET"]` est facultative (c'est la valeur par défaut) mais explicite.

**3.** Par exemple `http://localhost:8080/bonjour_get?prenom=L%C3%A9a&classe=1G3` : après le `?`, les paires `prenom=...` et `classe=...`, séparées par `&`. Le `é` est codé `%C3%A9` (ses deux octets en UTF-8, en hexadécimal ; un espace devient `+`). Le navigateur affiche parfois l'adresse décodée, avec le `é`. **4.** La page salue le nouveau prénom : le formulaire n'a pas servi. Le serveur reçoit ce qu'on lui envoie, formulaire ou pas : il ne doit jamais faire confiance aux données reçues (même idée

que pour les cookies). **5.** Dans la ligne du journal : `"GET /bonjour_get?prenom=Léa&classe=1G3 HTTP/1.1" 200`. Pour le `POST`, le journal n'affiche que `"POST /bonjour HTTP/1.1" 200` : le corps n'y figure pas. Un mot de passe envoyé en `GET` se retrouverait dans les journaux du serveur. **6.** En mode `debug`, le navigateur affiche l'erreur Python (code 500) : `KeyError: 'prenom'`, sur la ligne `prenom = request.args["prenom"]` ; il n'y a aucun paramètre dans l'URL. Sans le mode `debug`, Flask répond `400 Bad Request`. **7.** L'envoi en `GET` : son URL complète, prénom compris, est dans l'historique. `GET` pour une recherche (le résultat a une adresse qu'on peut partager ou mettre en favori) ; `POST` pour un mot de passe.

## 4 Défi : une page de connexion

**Exercice 8** *Vérifier des identifiants*

**▶ Corrigé.** Trois appels attendus possibles : un cas normal (`"alice"`, `"tournesol42"` → `True`, ou un mauvais mot de passe → `False`) ; un cas limite (identifiant et mot de passe vides → `False`, ou dictionnaire vide) ; un cas piège (identifiant **inconnu** → `False` *sans erreur* ; ou `"Alice"` avec une majuscule → `False`).

```python
def verifier_identifiants(comptes, identifiant, mot_de_passe):
    """Renvoie True si identifiant est une clef de comptes et que son
    mot de passe est mot_de_passe, False sinon."""
    # TESTER
    if identifiant not in comptes:
        # RENVOYER
        return False
    # RENVOYER
    return comptes[identifiant] == mot_de_passe

def tester_connexion():
    assert verifier_identifiants(COMPTES_TEST, "bob", "pivoine7")            # fourni
    assert verifier_identifiants(COMPTES_TEST, "alice", "tournesol42")       # cas normal
    assert not verifier_identifiants(COMPTES_TEST, "", "")                   # cas limite
    assert not verifier_identifiants(COMPTES_TEST, "zoe", "tournesol42")     # piege
    print("verifier_identifiants : tous les tests passent.")
```

Le piège : `comptes[identifiant]` sans test préalable lève `KeyError` pour un identifiant inconnu ; en ligne, la page de connexion planterait (erreur 500) au lieu de dire « incorrect ». Écriture acceptée aussi : `if ... and ...: return True` puis `return False`.

**Exercice 9** *La page de connexion*

**▶ Corrigé.**

```html
<form action="/verifier" method="post">
  <label for="identifiant">Identifiant :</label>
  <input type="text" name="identifiant" id="identifiant">
  <label for="mot_de_passe">Mot de passe :</label>
  <input type="password" name="mot_de_passe" id="mot_de_passe">
  <button type="submit">Se connecter</button>
</form>
```

```python
@app.route("/connexion")
def connexion():
    return render_template("connexion.html")

@app.route("/verifier", methods=["POST"])
def verifier():
    identifiant = request.form["identifiant"]
    mot_de_passe = request.form["mot_de_passe"]
    if verifier_identifiants(COMPTES_TEST, identifiant, mot_de_passe):
        message = f"Bienvenue, {identifiant} : vous êtes connecté(e)."
```

```
    else:
        message = "Identifiant ou mot de passe incorrect."
    return render_template("resultat.html", message=message)
```

Dans `resultat.html` : `<p>{{ message }}</p>`. **4.** Le type `password` masque les caractères **à l'écran** (contre un regard par-dessus l'épaule). Il ne change rien à ce qui est envoyé : la réponse attendue, à vérifier en partie 5, est « non ».

## 5 Ce qui circule sur le réseau

**Exercice 10** *Voie A : avec Wireshark*

▶ **Corrigé.** **4.** Source et destination : `127.0.0.1` (ou `::1` en IPv6), la même machine ; port de destination 8080, port source quelconque, choisi par le système pour le navigateur. En bas : `Form item: "identifiant" = "alice"` et `Form item: "mot_de_passe" = "tournesol42"` : le mot de passe est lisible tel quel. **5.** Le flux TCP montre la requête (en rouge) puis la réponse (en bleu), par exemple :

```
POST /verifier HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 42
Referer: http://localhost:8080/connexion
...                                     <- autres en-tetes du navigateur

identifiant=alice&mot_de_passe=tournesol42

HTTP/1.1 200 OK                         <- HTTP/1.0 avec Flask 2.0
Content-Type: text/html; charset=utf-8
...
```

On retrouve exactement la structure du cours : ligne de requête, en-têtes, ligne vide, corps. `Content-Length: 42` est le nombre d'octets du corps. **6.** En `GET`, il n'y a pas de corps : les données sont dans la **première ligne**, `GET /bonjour_get?prenom=...&classe=... HTTP/1.1`.

**Exercice 11** *Voie B : sans installation, les outils de développement*

▶ **Corrigé.** **2.** Méthode `POST`, adresse `http://localhost:8080/verifier`, statut `200 OK`, adresse distante `127.0.0.1:8080` ; l'en-tête `Content-Type` vaut `application/x-www-form-urlencoded` (données de formulaire). **3.** Les données du formulaire : `identifiant: alice` et `mot_de_passe: tournesol42` ; en brut, la ligne

`identifiant=alice&mot_de_passe=tournesol42`

(la même que dans le flux TCP de la voie A). **4.** En `GET`, pas de corps : les données sont dans l'URL de la requête (rubrique « paramètres de la chaîne de requête »). **5.** Les outils de développement sont *dans* le navigateur, avant le chiffrement éventuel : sur un site `https`, ils montreraient aussi le mot de passe en clair. L'argument : le programme Python a lu et affiché les données reçues sans rien déchiffrer ; les octets sont donc arrivés tels qu'ils étaient partis, lisibles, et toute machine placée sur leur chemin aurait pu les lire de même. Seul un analyseur placé *sur le réseau*, comme Wireshark, le montre directement.

**Exercice 12** *Conclure : et avec* `https` *?*

▶ **Corrigé.** **1.** Non, si l'on pensait que `password` protégeait le transport : le mot de passe circule en clair, dans le corps de la requête. **2.** Le point d'accès Wi-Fi du café, sa box, les routeurs du fournisseur d'accès et ceux d'Internet jusqu'au serveur. Tous *pourraient* le lire, ainsi que toute personne qui écoute le Wi-Fi s'il n'est pas protégé. **3.** Non. Un routeur honnête n'en a pas besoin, mais rien ne l'empêche techniquement d'ouvrir les enveloppes suivantes : TCP, puis HTTP, sont lisibles par quiconque détient le paquet. L'encapsulation range les informations, elle ne les cache pas. **4.** Avec `https`, la partie HTTP entière (ligne de requête, en-têtes, corps) est chiffrée : Wireshark

n'afficherait que des paquets `TLS`, « Application Data », illisibles. Resteraient visibles les adresses IP et les ports (`443`), la taille et l'instant des échanges, et le nom du site (requête DNS, début de la connexion chiffrée). **5.** `POST` seul ne protège rien sur le réseau : il évite seulement que le mot de passe s'affiche dans l'URL, l'historique et les journaux. `https` avec `GET` chiffre bien le transport, mais le mot de passe reste dans l'historique et les journaux du serveur. Règle : un formulaire de connexion s'envoie en `POST` **et** sur une page `https` ; et, comme le dit le cours, le cadenas ne garantit pas que le site est honnête.

> **Remarque** — *Prolongement possible (démonstration du professeur)*
> Sur sa propre machine, capturer quelques secondes de trafic avec le filtre `tls` pendant le chargement d'un site en `https` : on voit `Client Hello` (avec le nom du site), puis seulement des `Application Data`. Le chiffrement sera étudié en Terminale (*Cryptographie*).

## 6 Bonus : un livre d'or

**Exercice 13** IA IA IA *Le livre d'or*

**► Corrigé.**

```python
livre = []      # liste de dictionnaires {"pseudo": ..., "texte": ...}

def ajouter_message(livre, pseudo, texte):
    """Ajoute {"pseudo": pseudo, "texte": texte} a la fin de livre et renvoie True,
    sauf si pseudo ou texte est vide, ou si texte depasse 200 caracteres :
    dans ce cas, livre n'est pas modifie et la fonction renvoie False."""
    # TESTER
    if pseudo == "" or texte == "" or len(texte) > 200:
        # RENVOYER
        return False
    # METTRE A JOUR
    livre.append({"pseudo": pseudo, "texte": texte})
    # RENVOYER
    return True

def tester_livre():
    essai = []
    assert ajouter_message(essai, "Léa", "Super site !")                    # fourni
    assert essai == [{"pseudo": "Léa", "texte": "Super site !"}]
    assert ajouter_message(essai, "Tom", "x" * 200)                         # cas limite : 200
                                                                            # caracteres
    assert not ajouter_message(essai, "", "Message sans pseudo")            # piege : pseudo
                                                                            # vide
    assert not ajouter_message(essai, "Zoé", "x" * 201)                     # piege : trop long
    assert len(essai) == 2                                                  # rien n'a ete
                                                                            # ajoute
    print("ajouter_message : tous les tests passent.")

@app.route("/livre")
def livre_or():
    return render_template("livre.html", messages=livre, erreur="")

@app.route("/livre/ecrire", methods=["POST"])
def livre_ecrire():
    if ajouter_message(livre, request.form["pseudo"], request.form["texte"]):
        erreur = ""
    else:
        erreur = "Message refusé : pseudo et texte obligatoires, 200 caractères au plus."
    return render_template("livre.html", messages=livre, erreur=erreur)
```

Formulaire de `livre.html` : `<form action="/livre/ecrire" method="post">` avec deux champs texte `pseudo` et `texte` et un bouton (même forme qu'à l'exercice 6). Attention au nom de la

fonction de la route : `livre` est déjà le nom de la liste, d'où `livre_or`.

**3.** Oui : la liste est sur le serveur, partagée par tous les clients (comme le compteur). **4.** Le message est publié une deuxième fois : actualiser renvoie la même requête `POST`, avec les mêmes données. Les vrais sites répondent à un `POST` par une *redirection* vers une page en `GET` pour éviter ce doublon [ au-delà du programme ] . **5.** Le texte s'affiche tel quel, balises comprises, et non en gras : dans le code source, `<` est devenu `<` (et `>` `>`). Le moteur de modèles « neutralise » ainsi le HTML écrit par un visiteur ; sinon, un visiteur pourrait glisser dans la page de *tous* les autres une balise `<script>` et y faire exécuter son programme (attaque dite XSS). Encore une fois : ne jamais faire confiance aux données reçues.

## Réponses des questions de la fiche de révision

↑ questions p. 437

### Se tester

1. Internet est le réseau ; le Web est un service (pages + liens) qui l'utilise (§ I)
2. le DNS (annuaire nom → IP) (§ II)
3. 404 ; 200 (§ IV)
4. `<p id="monPara">Bonsoir</p>` : le DOM est modifié côté client, sans recharger la page (§ VII.B)
5. `post` : avec `get`, le mot de passe serait visible dans l'URL (§ VIII)

### Et avant ?

1. le répertoire parent ; le répertoire personnel (ch. 9, § V)
2. un seul (ch. 7, § IV)
3. `1` : on s'arrête dès la première occurrence (ch. 4, § III)

CORRIGÉS DU CHAPITRE 11

# 11 Les algorithmes gloutons

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 La caisse du marchand

↑énoncé p. 444

**Exercice 1** *Rendre la monnaie en euros*

▶ **Solution.** **1.**

| Somme | Pièces et billets rendus | Nombre |
|---|---|---|
| 8 € | $5+2+1$ | 3 |
| 13 € | $10+2+1$ | 3 |
| 27 € | $20+5+2$ | 3 |
| 36 € | $20+10+5+1$ | 4 |

**2.** En général non : si un élève a fait autrement (par exemple $8=2+2+2+2$), le vérificateur trouve mieux. On ne peut pas faire moins que les nombres du tableau.

**3.** On pose d'abord le billet de 20 €, le plus grand qui ne dépasse pas 36. Règle attendue : « à chaque fois, je prends la **plus grande** pièce (ou le plus grand billet) qui ne dépasse **pas ce qu'il reste** à rendre ».

**Exercice 2** *Appliquer la règle sans réfléchir*

▶ **Solution.** **4.** $49=20+20+5+2+2$ : 5 pièces et billets. $88=50+20+10+5+2+1$ : 6.

**5.** Non : on ajoute des pièces, sans jamais revenir sur un choix déjà fait.

**Exercice 3** *Au pays de Nsiland*

▶ **Solution.** **6.** La règle donne $12+1+1+1$ : **4 pièces**.

**7.** Oui : $5+5+5$, soit **3 pièces**. Prendre la plus grosse pièce au début était un mauvais choix, et la règle ne permet pas de le défaire.

**8.** **16** ($12+1+1+1+1$ : 5 pièces, alors que $5+5+5+1$ en utilise 4) ou **20** ($12+5+1+1+1$ : 5 pièces, alors que $5+5+5+5$ en utilise 4). Pour 17, 18 et 19, la règle donne bien le minimum.

**Exercice 4** *Ce qu'on a découvert*

▶ **Solution.** **9.** Réponse possible :

- à chaque étape, la règle consiste à prendre la plus grande pièce qui ne dépasse pas ce qu'il reste à rendre : le **meilleur choix sur le moment** ;
- une fois une pièce posée, on ne revient **jamais** en arrière ;
- avec les euros, la règle donne toujours le moins de pièces possible ;
- avec les pièces de Nsiland, la règle donne une solution, mais pas toujours la meilleure.

# CORRIGÉS DES EXERCICES

> **Remarque**
> On donne **une** version simple ; d'autres écritures sont possibles.

## Problèmes d'optimisation et principe glouton

**Exercice 1** IA IA IA *Qu'optimise-t-on ?* ↑énoncé p. 453

▶ **Solution.** Pour chaque problème, on repère la quantité qui juge une solution, puis le sens dans lequel on veut la pousser. a) **minimiser** le nombre de pièces ; b) **maximiser** la valeur emportée ; c) **minimiser** la distance totale ; d) **minimiser** le nombre de CD utilisés.

▷ **Erreur fréquente.** répondre pour le sac à dos « minimiser le poids » : le poids est une *contrainte* (ne pas dépasser la capacité) ; ce que l'on optimise, c'est la valeur.

**Exercice 2** IA IA IA *Vrai ou faux* ↑énoncé p. 453

▶ **Solution.** **a) V**. **b) F** : un glouton ne revient *jamais* sur un choix. **c) F** : il n'est pas toujours optimal. **d) V** : il est en général très rapide (un seul choix par étape).

▷ **Erreur fréquente.** croire qu'un algorithme rapide et « raisonnable » donne forcément la meilleure solution : le système $\{1, 5, 8\}$ de l'exercice 6 suffit à le contredire.

**Exercice 3** IA IA IA *Les ingrédients* ↑énoncé p. 453

▶ **Solution.** a) **candidats** : les coupures disponibles ; b) **critère glouton** : la plus grande coupure qui ne dépasse pas la somme restante ; c) **faisabilité** : une coupure n'est utilisable que si elle est $\leq$ à la somme restant à rendre.

## Rendu de monnaie

**Exercice 4** IA IA IA *Dérouler à la main* ↑énoncé p. 453

▶ **Solution.** À chaque étape, la plus grande coupure qui ne dépasse pas la somme restante, autant de fois que possible. a) $8 = 5+2+1$ : **3** pièces. b) $27 = 20+5+2$ : **3**. c) $63 = 50+10+2+1$ : **4**. d) $199 = 100+50+20+20+5+2+2$ : **7**.

▷ **Erreur fréquente.** ne prendre chaque coupure qu'une seule fois : pour 199, on oublierait le second 20 et le second 2. On garde la même coupure tant qu'elle ne dépasse pas la somme restante.

**Exercice 5** IA IA IA *La fonction* `rendre` ↑énoncé p. 453

▶ **Solution.** **a)** `rendre(48)` = `[20, 20, 5, 2, 1]` (5 coupures) ; `rendre(88)` = `[50, 20, 10, 5, 2, 1]` (6 coupures). **b)** *Appels attendus, écrits avant de coder :*

```
assert nb_pieces(48) == 5      # cas normal : 20 + 20 + 5 + 2 + 1
assert nb_pieces(0) == 0       # cas limite : rien a rendre
assert nb_pieces(4) == 2       # cas piege : 2 + 2, la meme coupure deux fois
```

```
def nb_pieces(somme):
    n = 0                      # INITIALISER : le compteur remplace la liste
    for p in pieces:           # PARCOURIR
        while somme >= p:      # TESTER
            n = n + 1          # METTRE A JOUR
            somme = somme - p  # METTRE A JOUR
    return n                   # RENVOYER
```

**c)** [ au-delà du programme ] Avec une division :

```python
def rendre(somme):
    rendu = []                              # INITIALISER
    for p in pieces:                        # PARCOURIR
        rendu = rendu + [p] * (somme // p)  # METTRE A JOUR : autant de p que possible
        somme = somme % p                   # METTRE A JOUR : ce qui reste apres ces p
    return rendu                            # RENVOYER
```

**Exercice 6** *Un système qui piège le glouton* ↑énoncé p. 454

▶ **Solution.** **a)** Glouton : 8, puis 1, puis 1 → `[8, 1, 1]`, soit **3 pièces**. **b)** Oui : $5+5$, soit **2 pièces**. **c)** Le glouton **n'est pas optimal** pour le système $\{1, 5, 8\}$ : ce système n'est **pas canonique**.

▷ **Erreur fréquente.** en conclure que le glouton ne marche jamais : il est optimal pour les systèmes canoniques, comme celui des euros ; un contre-exemple montre seulement qu'il n'est pas optimal *pour ce système*.

**Exercice 7** *Quand le glouton échoue carrément* ↑énoncé p. 454

▶ **Solution.** **a)** Glouton : il prend 5… il reste 3, qu'aucune pièce (4 ou 5) ne permet de rendre : le glouton **échoue** (aucune solution trouvée). **b)** Pourtant $4+4=8$ : une solution existait. **c)** « Pas optimal » : le glouton trouve *une* solution, mais pas la meilleure (cas $\{1, 5, 8\}$ de l'exercice 6). « Échoue » : le glouton ne trouve *aucune* solution, alors qu'il en existe une (cas $\{4, 5\}$).

**Exercice 8** *Le rendu de monnaie en désordre* ↑énoncé p. 454

▶ **Solution.** **a)**

```python
def rendre_general(systeme, somme):
    rendu = []                  # INITIALISER
    for p in systeme:           # PARCOURIR
        while somme >= p:       # TESTER
            rendu.append(p)     # METTRE A JOUR
            somme = somme - p   # METTRE A JOUR
    return rendu                # RENVOYER
```

**b)** `[4, 1, 1]` : 3 pièces. Ce n'est **pas** le minimum, car $3+3=6$ n'utilise que 2 pièces : le système $\{4, 3, 1\}$ n'est pas canonique.

**Exercice 9** *Tester si un système est canonique* ↑énoncé p. 454

▶ **Solution.** *Appels attendus pour* `rendre_sys`*, écrits avant de coder :*

```python
assert rendre_sys([5, 2, 1], 9) == 3      # cas normal : 5 + 2 + 2
assert rendre_sys([5, 2, 1], 0) == 0      # cas limite : rien a rendre
assert rendre_sys([3, 2], 4) is None      # cas piege : il reste 1, le glouton n'aboutit pas
```

```python
def rendre_sys(systeme, somme):
    n = 0                       # INITIALISER
    for p in systeme:           # PARCOURIR : systeme deja trie a l'envers
        while somme >= p:       # TESTER
            n = n + 1           # METTRE A JOUR
            somme = somme - p   # METTRE A JOUR
    if somme == 0:              # TESTER : tout a-t-il ete rendu ?
        return n                # RENVOYER
    return None                 # RENVOYER : le glouton n'aboutit pas

def tester(systeme):
    for s in range(1, 31):                                  # PARCOURIR
        if rendre_sys(systeme, s) != optimal(systeme, s):   # TESTER
            print("premier ecart pour la somme", s)
            return
    print("canonique jusqu'a 30")
```

(Pour le système `[10, 6, 1]`, le premier écart apparaît pour la somme **12** : le glouton rend $10+1+1$, soit 3 pièces, alors que $6+6$ n'en utilise que 2. Pour les sommes de 1 à 11, le glouton

est optimal : de 1 à 5, on n'a que des pièces de 1 ; de 6 à 9, 6 puis des 1 ; 10 et 11 commencent par la pièce de 10.)

## Le sac à dos

**Exercice 10** *Remplir le sac* ↑énoncé p. 455

▶ **Solution.** **a)** Rapports valeur/poids : A = 30/2 = **15**, B = 20/2 = **10**, C = 24/3 = **8**. Classement : A, B, C. **b)** Glouton : A (2 kg, reste 2 kg, valeur 30), B (2 kg, reste 0, valeur 50), C (3 kg) ne rentre pas → on emporte **A + B**, poids 4 kg, **valeur 50**. **c)** Les combinaisons qui tiennent dans 4 kg : {A}=30, {B}=20, {C}=24, {A,B}=50. La meilleure est A+B = 50 : le glouton était **optimal** ici.

▷ **Erreur fréquente.** classer les objets par valeur seule (ou par poids seul) : le critère glouton est le rapport valeur/poids, la valeur apportée par kilogramme.

**Exercice 11** *Un sac à dos où le glouton se trompe* ↑énoncé p. 455

▶ **Solution.** **a)** Rapports : A = 30, B = 20, C = 18. Glouton : A (1 kg, reste 5, valeur 30), B (3 kg, reste 2, valeur 90), C (4 kg) ne rentre pas → **valeur 90**. **b)** Combinaisons qui tiennent dans 6 kg : A (30), B (60), C (72), A + B (4 kg, 90), A + C (5 kg, **102**) ; B + C pèse 7 kg. La meilleure est **A + C**, valeur **102**. Le glouton se trompe de **12**.

## Le plus proche voisin (voyageur)

**Exercice 12** *Toujours vers le plus proche* ↑énoncé p. 455

▶ **Solution.** **a)** Départ A. Plus proche : B (10). Depuis B, non visitées C (35) et D (25) → D (25). Depuis D, reste C (30) → C. Retour C→A (15). Itinéraire **A–B–D–C–A**, distance 10 + 25 + 30 + 15 = **80** km. **b)** A–C–B–D–A = 15 + 35 + 25 + 20 = 95 km, donc **plus long**. Ici la solution gloutonne (80 km) est en fait la **meilleure** — mais ce n'**est pas garanti** : sur de nombreuses villes, le glouton « plus proche voisin » est rarement optimal (cf. cours).

▷ **Erreur fréquente.** oublier le retour à la ville de départ dans la distance totale : le trajet doit revenir en A.

## Mini-problème de synthèse

**Exercice 13** *Le distributeur de billets* ↑énoncé p. 455

▶ **Solution.** **a)** Pour 90 : 50 + 20 + 20 → **3 billets**. Pour 80 : 50 + 20 + 10 → **3 billets**. **b)** Le système {10, 20, 50} est **canonique** pour les multiples de 10 jusqu'à 100 (le glouton donne le minimum pour chacun : c'est l'équivalent, à un facteur 10 près, du système {1, 2, 5}). **c)** Avec {10, 50} pour 80 : 50 + 10 + 10 + 10 → **4 billets**. C'est bien le minimum *pour ce système réduit* (le glouton reste optimal sur {10, 50}) ; simplement, faute de billet de 20, il faut un billet de plus qu'avant.

**Exercice 14** *Le rendu de monnaie selon un assistant d'IA* ↑énoncé p. 456

▶ **Solution.** **a)** Non. Avec `pieces = [4, 3, 1]` et `somme = 6`, la fonction renvoie `[4, 1, 1]` (3 pièces), alors que `[3, 3]` rend 6 avec **2 pièces**. **b)** Le *code* est bien un glouton ; l'erreur est dans l'*explication* : « quel que soit le système de pièces ». Le glouton n'est optimal que pour les systèmes **canoniques** (comme les euros). Correction : « elle donne le nombre minimal de pièces *pour un système canonique* ; pour un autre système, la solution peut ne pas être optimale, et même ne pas rendre la somme du tout s'il n'y a pas de pièce de 1 ». Par exemple, `rendre(4, [3, 2])` renvoie `[3]` : il reste 1 à rendre, alors que 2 + 2 convenait. Une version sûre vérifie à la fin que `somme == 0`. **c)** Tester la fonction sur un petit système **non canonique**, comme celui du cours ({1, 6, 10} pour 12 : le glouton donne 3 pièces, 6 + 6 en donne 2). Une affirmation en « toujours » se réfute avec un seul contre-exemple.

## Entraînement

**Exercice 15** *Le glouton dans d'autres systèmes* ↑énoncé p. 456

▶ **Solution.** a) $10+2+2$ : 3 pièces, c'est le minimum. b) $6+1+1$ : 3 pièces ; mieux : $4+4$ (2 pièces). c) $12+1+1+1$ : 4 pièces ; mieux : $5+5+5$ (3 pièces). d) $25+1+1+1+1+1$ : 6 pièces ; mieux : $10+10+10$ (3 pièces). Seul le système a) (celui des euros, réduit) est sans piège ici : les trois autres ne sont pas canoniques.

**Exercice 16** *Le sac à dos glouton* ↑énoncé p. 456

▶ **Solution.** a) Rapports : A $= 40/5 = 8$, B $= 30/4 = 7{,}5$, C $= 12/3 = 4$. Glouton : A (reste 5 kg), B (reste 1 kg), C ne rentre pas : valeur **70**. Combinaisons possibles : {A, B} = 70, {A, C} = 52, {B, C} = 42 ({A, B, C} pèse 12 kg) : le glouton est **optimal**. b) Rapports : A $= 7{,}5$, B $= C = 6$. Glouton : A (reste 2 kg), ni B ni C ne rentrent : valeur **30**. Or {B, C} pèse 6 kg et vaut **36** : le glouton se trompe de 6.

▷ **Erreur fréquente.** chercher la meilleure combinaison sans vérifier son poids : en a), {A, B, C} vaudrait 82, mais pèse 12 kg et ne rentre pas dans le sac.

# CORRIGÉS DES TP

## TP Le glouton à l'épreuve ↑énoncé p. 458

Les fonctions complètes figurent dans chaque exercice. Les **durées** dépendent de la machine : seuls leurs ordres de grandeur et leur *croissance* comptent.

### 1. Rendu de monnaie

**Exercice 1** *Le glouton fourni*

**1.** `rendu_glouton([10, 6, 1], 12)` vaut **3** $(10+1+1)$ ; `rendu_glouton([3, 2], 4)` vaut `None` (on prend 3, il reste 1, que ni 3 ni 2 ne peuvent rendre). **2.** Sans ce test, la fonction renverrait 1 pour la somme 4 en $\{2, 3\}$ : elle compterait des pièces alors que la somme **n'a pas été entièrement rendue**.

**Exercice 2** *La vraie solution, par force brute*

**1.** Pour `[10, 6, 1]` et 12 :

| $a$ | $b$ | reste | pièces |
|---|---|---|---|
| 0 | 0 | 12 | 12 |
| 0 | 1 | 6 | 7 |
| 0 | 2 | 0 | **2** |
| 1 | 0 | 2 | 3 |

Le minimum est **2** $(6+6)$ ; le glouton, lui, a suivi la dernière ligne (3 pièces).

**2.**

```
1 def optimal3(systeme, somme):
2     p1, p2, p3 = systeme
3     meilleur = None
4     for a in range(somme // p1 + 1):
5         for b in range((somme - a * p1) // p2 + 1):
6             reste = somme - a * p1 - b * p2
7             if reste % p3 == 0:                  # le reste se fait en pieces p3
8                 n = a + b + reste // p3
```

```
                if meilleur is None or n < meilleur:
                    meilleur = n
    return meilleur
```

### Exercice 3 *Chercher un contre-exemple*

```
def premier_contre_exemple(systeme, smax):
    for somme in range(1, smax + 1):
        if rendu_glouton(systeme, somme) != optimal3(systeme, somme):
            return somme
    return None
```

| système | {1,2,5} | {1,6,10} | {1,3,4} | {1,5,7} | {1,10,25} | {1,10,20} | {5,10,25} |
|---|---|---|---|---|---|---|---|
| contre-exemple | aucun | 12 | 6 | 10 | 30 | aucun | aucun |
| pièces (glouton ; optimum) | — | 3 ; 2 | 3 ; 2 | 4 ; 2 | 6 ; 3 | — | — |

Pour {5,10,25}, les sommes qui ne sont pas multiples de 5 ne peuvent être rendues ni par le glouton ni par personne : les deux fonctions renvoient `None`, ce n'est pas un contre-exemple.

### Exercice 4 *La chasse aux systèmes non canoniques*

```
total = 0
non_canoniques = 0
for a in range(2, 20):
    for b in range(a + 1, 21):
        total = total + 1
        if premier_contre_exemple([b, a, 1], 100) is not None:
            non_canoniques = non_canoniques + 1
print(total, non_canoniques)                # 171 100
```

**1.** **171** systèmes, dont **100** non canoniques (environ 58 %) : un système « au hasard » a plutôt *moins* d'une chance sur deux d'être canonique. Les euros sont un choix réfléchi. **2.** Le plus petit contre-exemple est **toujours strictement inférieur à** $a + b$ (vérifié sur les 100 systèmes).

### Exercice 5 *Une règle cachée, et deux histoires vraies*

**1.** Règle : $\{1, a, b\}$ est canonique **si et seulement si** $r = 0$ **ou** $r \geqslant a - q$. Vérification sur les 171 systèmes : 171 accords.

```
accords = 0
for a in range(2, 20):
    for b in range(a + 1, 21):
        q, r = b // a, b % a
        canonique = premier_contre_exemple([b, a, 1], 100) is None
        if canonique == (r == 0 or r >= a - q):
            accords = accords + 1
print(accords)                              # 171
```

Exemples : $\{1, 6, 10\}$ : $q = 1$, $r = 4 < 6 - 1 \to$ non canonique ; $\{1, 2, 5\}$ : $q = 2$, $r = 1 \geqslant 0 \to$ canonique. (Résultat connu pour trois pièces, admis ici.)

**2.** $\{1, 10, 25\}$ n'est **pas** canonique : pour 30 cents, le glouton donne $25 + 5 \times 1$ (6 pièces) au lieu de $3 \times 10$ (3 pièces). La pièce de 5 cents « sauve » le système américain.

**3.** `rendu_glouton([30, 24, 12, 6, 3, 1], 48)` vaut **3** $(30 + 12 + 6)$, alors que $24 + 24$ n'utilise que **2** pièces : l'ancien système britannique n'était **pas canonique**.

## 2. Le sac à dos du randonneur

**Exercice 6** *Trois critères gloutons*

**1.** Rapports : trousse 40 ; duvet 30 ; gourde 27,5 ; photo 25 ; nourriture $\approx 23{,}3$ ; doudoune 22,5 ; tente 18,75 ; réchaud 15 ; jumelles 10 ; livre 5.

**2.** trousse (1 kg) ; duvet (3 kg) ; gourde (5 kg) ; photo (6 kg) ; nourriture (9 kg) ; doudoune : 11 kg, refusée ; tente, réchaud : refusés ; jumelles (10 kg) ; livre : refusé. Sac : **10 kg, intérêt 260**.

**Exercice 7** *Programmer le glouton*

```
def rapport(objet):
    return objet[2] / objet[1]

def sac_glouton(objets, capacite, critere, decroissant):
    pris = []
    poids_total = 0
    interet_total = 0
    for objet in sorted(objets, key=critere, reverse=decroissant):
        if poids_total + objet[1] <= capacite:      # l'objet rentre encore
            pris.append(objet[0])
            poids_total = poids_total + objet[1]
            interet_total = interet_total + objet[2]
    return pris, poids_total, interet_total
```

| critère | objets pris | intérêt |
|---|---|---|
| intérêt décroissant | tente, nourriture, duvet, trousse | 245 |
| poids croissant | trousse, photo, jumelles, livre, duvet, réchaud, gourde | 225 |
| rapport décroissant | trousse, duvet, gourde, photo, nourriture, jumelles | 260 |

(À poids égal, `sorted` garde l'ordre de la liste `OBJETS` : le tri est *stable*.)

**Exercice 8** *L'optimum, en comptant en binaire*

**1.** $3 = 011$ : objets 0 et 1 ; $6 = 110$ : objets 1 et 2. Pour 10 objets : $2^{10} = \mathbf{1\,024}$ sacs.

**2.**

```
def sac_optimal(objets, capacite):
    n = len(objets)
    meilleur = ([], 0, 0)
    for k in range(2 ** n):
        pris = []
        poids_total = 0
        interet_total = 0
        for i in range(n):
            if (k // 2 ** i) % 2 == 1:      # le bit i de k vaut 1
                pris.append(objets[i][0])
                poids_total = poids_total + objets[i][1]
                interet_total = interet_total + objets[i][2]
        if poids_total <= capacite and interet_total > meilleur[2]:
            meilleur = (pris, poids_total, interet_total)
    return meilleur
```

**3.** Optimum : duvet, nourriture, trousse, gourde, doudoune — **10 kg, 270 points**. **Aucun** des trois gloutons ne le trouve. Écarts : intérêt −25 points (9,3 %) ; poids −45 (16,7 %) ; rapport −10 (3,7 %). L'optimum **laisse la tente** : pour un bivouac, c'est absurde ! L'algorithme a bien résolu le problème *posé*, mais le **modèle** est incomplet : il faudrait imposer les objets indispensables (ou leur donner un intérêt bien plus grand). À retenir : un résultat « optimal » n'est jamais meilleur que le modèle.

**Exercice 9** *Mesurer sur 200 sacs au hasard*

```python
random.seed(2026)
criteres = [("interet", interet, True), ("poids", poids, False),
            ("rapport", rapport, True)]
nb_optimal = {"interet": 0, "poids": 0, "rapport": 0}
ecart_total = {"interet": 0, "poids": 0, "rapport": 0}
for essai in range(200):
    objets = objets_au_hasard(10)
    meilleur = sac_optimal(objets, 15)[2]
    for nom, critere, decroissant in criteres:
        valeur = sac_glouton(objets, 15, critere, decroissant)[2]
        if valeur == meilleur:
            nb_optimal[nom] = nb_optimal[nom] + 1
        ecart_total[nom] = ecart_total[nom] + (meilleur - valeur) / meilleur * 100
for nom in nb_optimal:
    print(nom, nb_optimal[nom], f"{ecart_total[nom] / 200:.1f}")
```

| critère | optimum trouvé | écart moyen |
|---|---|---|
| intérêt décroissant | 114 fois sur 200 | 4,5 % |
| poids croissant | 11 fois sur 200 | 17,9 % |
| rapport décroissant | **126 fois sur 200** | **1,7 %** |

Le critère « rapport » est nettement le meilleur : optimal près de deux fois sur trois, et sinon très proche. Le critère « poids » est mauvais : il remplit le sac d'objets légers mais peu utiles.

**Exercice 10** *Le prix de la force brute*

Mesures sur notre machine : $n = 10$ : 0,00 s ; 12 : 0,02 s ; 14 : 0,08 s ; 16 : 0,36 s ; 18 : 1,6 s ; 20 : 7,3 s. Ajouter deux objets multiplie le temps par **4 environ** ($2^n$ sacs : chaque objet de plus **double** le travail). Pour 30 objets : $7{,}3 \times 2^{10} \approx 7\,500$ s, soit **2 heures** ; pour 40 : $\times 2^{10}$ encore, soit environ **90 jours**. Le glouton, lui, traite 40 objets en quelques **centièmes de milliseconde** (un tri et un parcours).

## 3. Une tournée sur la Riviera

**Exercice 11** *Longueur d'une tournée*

```python
def longueur(tournee):
    total = 0
    for i in range(len(tournee) - 1):
        total = total + distance(tournee[i], tournee[i + 1])
    return total + distance(tournee[-1], tournee[0])      # retour au depart
```

Monaco → Menton (7,1 km) → Sospel (12,0 km) → Monaco (15,5 km) : 34,7 km au total (l'écart de 0,1 vient des arrondis de chaque étape).

**Exercice 12** *Le plus proche voisin*

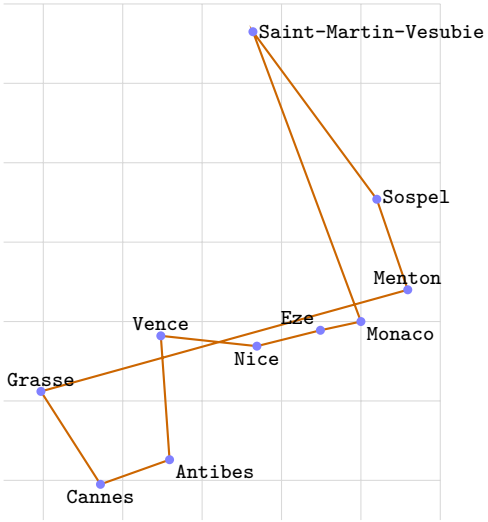
```python
def plus_proche_voisin(depart, villes):
    tournee = [depart]
    a_visiter = [v for v in villes if v != depart]
    while len(a_visiter) > 0:
        ici = tournee[-1]
        proche = a_visiter[0]
        for v in a_visiter:
            if distance(ici, v) < distance(ici, proche):
                proche = v
        tournee.append(proche)
        a_visiter = [v for v in a_visiter if v != proche]    # on la retire
    return tournee
```

Tournée : Monaco, Eze, Nice, Vence, Antibes, Cannes, Grasse, Menton, Sospel, Saint-Martin-Vesubie (et retour) : **189,6 km**. Étape maladroite : **Grasse** → **Menton** (47,9 km d'un bout à l'autre de la carte), puis le long retour Saint-Martin-Vesubie → Monaco (39,0 km) : en fonçant vers les villes proches, le glouton a laissé Menton, Sospel et Saint-Martin-Vesubie « pour la fin ».

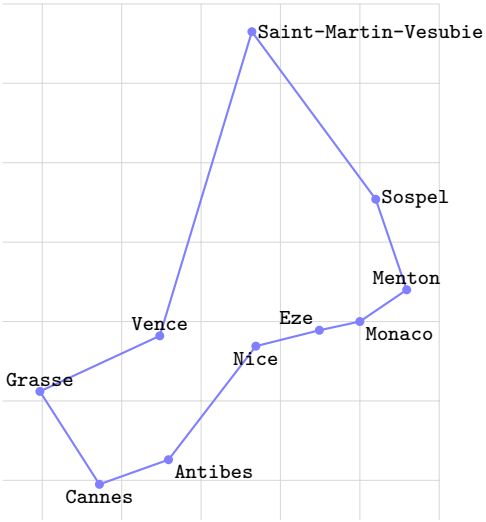
### Exercice 13 *L'optimum, et ce qu'il coûte*

```python
def tournee_optimale(depart, villes):
    autres = [v for v in villes if v != depart]
    meilleure = None
    for ordre in tous_les_ordres(autres):
        tournee = [depart] + ordre
        if meilleure is None or longueur(tournee) < longueur(meilleure):
            meilleure = tournee
    return meilleure
```

**1.** $9! = 362\,880$ ordres ; environ **3,5 s** sur notre machine. **2.** Optimum : Monaco, Menton, Sospel, Saint-Martin-Vesubie, Vence, Grasse, Cannes, Antibes, Nice, Eze : **156,7 km**. Écart du glouton : $\dfrac{189{,}6 - 156{,}7}{156{,}7} \approx$ **21 %**. **3.** $14! = 87\,178\,291\,200$, soit 240 240 fois plus que 9! : environ $3{,}5 \times 240\,240 \approx 840\,000$ s, **près de 10 jours**.




plus proche voisin depuis Monaco : 189,6 km




tournée optimale : 156,7 km

### Exercice 14 *Et si l'on partait d'ailleurs ?*

```python
meilleure = None
for v in VILLES:
    t = plus_proche_voisin(v, VILLES)
    if meilleure is None or longueur(t) < longueur(meilleure):
        meilleure = t
print(meilleure, f"{longueur(meilleure):.1f}")
```

Longueurs selon le départ : Monaco 189,6 ; Menton 184,4 ; Sospel 162,4 ; Eze 190,3 ; Nice 165,5 ; Vence 162,4 ; Grasse 162,4 ; Cannes 162,4 ; Antibes 165,7 ; Saint-Martin-Vesubie 162,4. Meilleure : **162,4 km** (par exemple depuis Sospel), soit un écart de **3,6 %** seulement — pour 10 appels du glouton, contre 362 880 tournées examinées par la force brute. Le livreur peut suivre ce circuit en partant de Monaco : c'est le même cycle.

**Exercice 15** *Décroiser la tournée*

Oui : le segment Grasse → Menton croise Vence → Antibes et Saint-Martin-Vesubie → Monaco.

L'idée : pour chaque couple de positions $i < j$, on fabrique la tournée `essai` identique à `tournee`, sauf que les villes des positions $i$ à $j$ sont parcourues **dans l'autre sens** (on les relit de $j$ à $i$, par compréhension) ; si elle est plus courte, on la garde et on refera un passage.

```
def ameliorer(tournee):
    amelioration = True
    while amelioration:
        amelioration = False
        for i in range(1, len(tournee) - 1):
            for j in range(i + 1, len(tournee)):
                retourne = [tournee[j - k] for k in range(j - i + 1)]   # de j a i
                essai = tournee[:i] + retourne + tournee[j + 1:]
                if longueur(essai) < longueur(tournee) - 0.001:   # gain >= 1 m
                    tournee = essai
                    amelioration = True
    return tournee
```

Le morceau `retourne` est construit comme dans le coup de pouce : pour `k` allant de 0 à `j - i`, la position `j - k` va de `j` à `i`. Appliquée à la tournée gloutonne depuis Monaco, on obtient **156,7 km** : ici on retombe sur l'optimum (parcouru dans l'autre sens). C'est une chance : appliquée aux tournées gloutonnes issues des autres villes de départ (sauf Menton), elle s'arrête à 162,4 km, une tournée sans croisement mais non optimale. Cette amélioration locale (appelée *2-opt*) donne en général une très bonne tournée, *sans garantie* d'optimalité. Sans la marge de 0,001 km, les erreurs d'arrondi des flottants peuvent faire accepter de fausses « améliorations » (une tournée simplement parcourue à l'envers) et changer le résultat : c'est un piège classique.

## 4. Bilan

| problème | glouton | optimum | écart | force brute |
|---|---|---|---|---|
| monnaie {1,6,10}, somme 12 | 3 pièces | 2 pièces | +50 % | — |
| sac de Léa (critère rapport) | 260 | 270 | 3,7 % | 1 024 sacs, instantané |
| 200 sacs au hasard (rapport) | optimum 126 fois sur 200 | | moyen : 1,7 % | — |
| tournée depuis Monaco | 189,6 km | 156,7 km | 21 % | 362 880 ordres, ≈ 3,5 s |
| tournée, meilleur départ | 162,4 km | 156,7 km | 3,6 % | — |

(1) Le glouton n'est **garanti** optimal que lorsqu'on l'a *démontré* pour le problème précis (rendu de monnaie dans un système canonique comme l'euro ; sac à dos *fractionnaire*). (2) La force brute devient impossible très vite : $2^n$ sacs ou $(n-1)!$ tournées, chaque objet ou ville de plus multipliant le travail (explosion combinatoire). (3) Exemples : calcul d'itinéraire à la volée, tournées de livraison quotidiennes, ordonnancement en temps réel : une solution à quelques pourcents de l'optimum, obtenue instantanément, vaut mieux qu'une solution parfaite trop tard.

## Réponses des questions de la fiche de révision

↑ questions p. 465

### Se tester

**1.** meilleur choix local à chaque étape ; jamais de retour en arrière (§ II)
**2.** `[20, 10, 5, 2]` (§ III)
**3.** $4+1+1$ (3 pièces) ; $3+3$ (2 pièces) (§ III)
**4.** le rapport valeur/poids, du plus grand au plus petit (§ IV)
**5.** non : il faut le justifier (c'est vrai pour un système canonique comme les euros) (§ III)

### Et avant ?

1. à reconnaître un paquet reçu deux fois (un doublon), que le récepteur jette (ch. 10, § II.A)
2. `sorted(table, key=lambda ligne: ligne["nom"])` (ch. 8, § VI)
3. interprété : l'interpréteur lit et exécute le programme au fur et à mesure (ch. 5, § VII)

CORRIGÉS DU CHAPITRE 12

# 12 Les $k$ plus proches voisins

## CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

### ACTIVITÉ 1 Quel est ce fruit ? ↑énoncé p. 473

**Exercice 1** *Lire le nuage de points*

▶ **Solution.** **1.** Les mandarines sont **petites** (en bas à gauche, 4 à 6 cm dans les deux sens) ; les oranges sont **grosses et rondes** (en haut à droite, 7 à 10 cm) ; les citrons sont **plus hauts que larges** (en haut à gauche). Citron/orange : la forme (allongé ou rond) ; orange/mandarine : la taille.

**2.** A est une **orange** : il est au milieu des oranges (ses deux plus proches voisins, (9 ; 8) et (8 ; 9), sont à 0,71 cm, et ses 5 plus proches voisins sont tous des oranges). Dire « il est dans le paquet des oranges », c'est déjà l'idée de voisinage.

**Exercice 2** *Demander aux voisins*

▶ **Solution.** **3.** Le plus proche de B est le citron (6,5 ; 9), à 0,71 cm : la règle 1 prédit **citron**.

**4.** Les 3 plus proches : le citron (6,5 ; 9) (0,71 cm), l'orange (8 ; 9) (1,12 cm), l'orange (8 ; 7,5) (1,41 cm) : 2 oranges contre 1 citron, la règle 3 prédit **orange**.

**5.** Les 4e et 5e sont les citrons (5,5 ; 9,5) et (6 ; 10), tous deux à 1,80 cm : 3 citrons contre 2 oranges, on revient à **citron**.

**6.** B est à la frontière entre deux sortes : la réponse **dépend du nombre de voisins** consultés. Un seul voisin, c'est fragile : un fruit mal mesuré ou mal étiqueté suffit à tromper. Avec les 15 fruits, chaque sorte a 5 voix : **égalité** pour tous les fruits, la règle ne sert plus à rien. Il faut un nombre de voisins « raisonnable », petit devant le nombre d'exemples, et de préférence impair.

**Exercice 3** *Le fruit qui fâche*

▶ **Solution.** **7.** Les binômes ne sont pas d'accord : certains voient la mandarine (5 ; 5,5), d'autres l'orange (8 ; 7,5), quelques-uns la mandarine (6 ; 4,5) ou le citron (5 ; 8).

**8.** Plusieurs réponses conviennent : mesurer à la règle, compter les carreaux, faire la somme des décalages… Toutes conviennent : l'essentiel est de **se mettre d'accord sur une mesure**.

**9.** Mandarine (5 ; 5,5) : 1,5 cm horizontalement (3 carreaux) et 1 cm verticalement (2 carreaux). Orange (8 ; 7,5) : **exactement les mêmes** décalages. C est à égale distance des deux.

**10.** Le segment est l'hypoténuse d'un triangle rectangle dont les côtés sont les deux décalages (Pythagore) : $\sqrt{1{,}5^2 + 1^2} = \sqrt{3{,}25} \approx 1{,}80$ cm, pour la mandarine comme pour l'orange.

**11.** On peut regarder plus de voisins : les 3 plus proches de C sont la mandarine (5 ; 5,5) et l'orange (8 ; 7,5) (à 1,80 cm) puis la mandarine (6 ; 4,5) (à 2,06 cm) : la règle 3 prédit **mandarine**. Autres idées possibles : tirer au sort, préférer le voisin d'une sorte plus fréquente…

**Exercice 4** *Mettre des mots*

▶ **Solution.** **12.** « . . .parmi les fruits **déjà connus** (étiquetés) ceux qui sont les plus **proches** de lui, et on prend la sorte **majoritaire** parmi eux. Il faut choisir **combien** on regarde de voisins, et une façon de mesurer la **distance** entre deux fruits. »

**13.** Des **nombres** : les mesures de chaque fruit (ses coordonnées), la sorte des fruits connus, une **formule** de distance et une règle de vote (avec que faire en cas d'égalité). Le coup d'œil doit devenir un **calcul**.

# CORRIGÉS DES EXERCICES

> **Remarque**
> On donne **une** version simple ; d'autres écritures sont possibles.

## Classification et distance

**Exercice 1** *Le vocabulaire* ↑énoncé p. 485

▶ **Solution.** **a)** Apprendre à prédire une étiquette à partir d'**exemples déjà étiquetés**. **b)** Les descripteurs sont les **mesures** (les coordonnées) de l'individu ; l'étiquette est sa **classe**. **c)** On calcule la distance de l'individu à tous les exemples, on garde les $k$ plus proches, on prend leur classe **majoritaire**.

**Exercice 2** *Calculer des distances* ↑énoncé p. 485

▶ **Solution.** On élève au carré l'écart sur chaque descripteur, on additionne (c'est $d^2$), puis on prend la racine. **a)** $\sqrt{3^2+4^2}=\sqrt{25}=\mathbf{5}$. **b)** $\sqrt{3^2+0^2+4^2}=\sqrt{25}=\mathbf{5}$. **c)** $d^2=0.2^2+0.1^2=0.04+0.01=\mathbf{0.05}$ ; $d=\sqrt{0.05}\approx\mathbf{0.22}$.

▷ **Erreur fréquente.** additionner les écarts avant de les élever au carré : $(3+4)^2=49$, et non $3^2+4^2=25$.

**Exercice 3** *Qui gagne le vote ?* ↑énoncé p. 485

▶ **Solution.** **a)** Fruit 3 ($d^2=1$, poire), 1 (2, pomme), 5 (3, pomme), 4 (5, pomme), 2 (9, poire), 6 (12, poire). **b)** 1-NN : **poire**. 3-NN : poire, pomme, pomme → **pomme**. 5-NN : 3 pommes contre 2 poires → **pomme**. La prédiction peut changer avec $k$.

▷ **Erreur fréquente.** voter avec les $k$ premiers fruits du tableau (fruits 1, 2, 3 pour $k=3$) : il faut d'abord ranger par distance, puis garder les $k$ plus *proches*.

**Exercice 4** *Dérouler le k-NN à la main (iris)* ↑énoncé p. 485

▶ **Solution.** **a)** Distances au carré de $x=(5.2;1.9)$ :

| exemple | espèce | calcul | $d^2$ |
|---|---|---|---|
| (1.4 ;0.2) | setosa | $3.8^2+1.7^2$ | 17.33 |
| (1.5 ;0.2) | setosa | $3.7^2+1.7^2$ | 16.58 |
| (4.5 ;1.5) | versicolor | $0.7^2+0.4^2$ | 0.65 |
| (4.9 ;1.5) | versicolor | $0.3^2+0.4^2$ | **0.25** |
| (5.5 ;2.1) | virginica | $0.3^2+0.2^2$ | **0.13** |
| (5.8 ;2.2) | virginica | $0.6^2+0.3^2$ | **0.45** |

**b)** 1-NN : le plus proche est une *virginica* ($d^2=0.13$) → **virginica**. **c)** 3-NN : dans l'ordre, une virginica (0.13), une versicolor (0.25) et une virginica (0.45) → vote 2 contre 1 → **virginica**.

▷ **Erreur fréquente.** calculer les racines carrées pour comparer : c'est inutile, les distances au carré sont rangées dans le même ordre que les distances.

## Programmer le $k$-NN

**Exercice 5** *Les briques de base* ↑énoncé p. 486

▶ **Solution.** `distance2` : un accumulateur et une boucle sur les indices des descripteurs. `vote` : un dictionnaire d'occurrences, puis une recherche du champion (l'étiquette de plus grand compte).

*Appels attendus pour* `vote`*, écrits avant de coder :*

```
assert vote(["pomme", "poire", "pomme"]) == "pomme"  # cas normal
assert vote(["poire"]) == "poire"                    # cas limite : un seul voisin
assert vote(["poire", "pomme", "pomme"]) == "pomme"  # cas piege : la majorite n'est pas en
                                                     # tete
```

```
def distance2(a, b):
    s = 0                                            # INITIALISER
    for i in range(len(a)):                          # PARCOURIR
        s = s + (a[i] - b[i]) ** 2                   # METTRE A JOUR
    return s                                         # RENVOYER : d2, sans racine

def vote(etiquettes):
    compte = {}                                      # INITIALISER
    for e in etiquettes:                             # PARCOURIR
        if e in compte:                              # TESTER
            compte[e] += 1                           # METTRE A JOUR
        else:
            compte[e] = 1                            # METTRE A JOUR
    meilleure = None                                 # INITIALISER : le champion
    for e in compte:                                 # PARCOURIR
        if meilleure is None or compte[e] > compte[meilleure]:  # TESTER
            meilleure = e                            # METTRE A JOUR
    return meilleure                                 # RENVOYER
```

**Exercice 6** *Le plus proche voisin, à trous* ↑énoncé p. 486

▶ **Solution.** **a)**

```
def plus_proche(exemples, x):
    dists = []                                       # INITIALISER
    for descripteurs, etiquette in exemples:         # PARCOURIR
        dists.append((distance2(x, descripteurs), etiquette))  # METTRE A JOUR
    dists.sort(key=lambda couple: couple[0])         # tri par distance croissante
    return dists[0][1]                               # RENVOYER : l'etiquette du
                                                     # plus proche
```

**b)** Après le tri, `dists` contient les couples `(distance au carré, etiquette)` rangés par distance **croissante**. `dists[0]` est le couple le plus proche ; `dists[0][1]` est donc l'étiquette du plus proche voisin. **c)** `plus_proche(iris, [4.7, 1.4])` renvoie `'versicolor'` (distance au carré 0.05).

▷ **Erreur fréquente.** renvoyer `dists[0][0]` : c'est la *distance* du plus proche voisin, pas son étiquette, qui est le second élément du couple.

**Exercice 7** *Classer* ↑énoncé p. 486

▶ **Solution.** On range les couples `(distance, etiquette)`, on les trie par distance, on garde les étiquettes des $k$ premiers, puis on vote avec la fonction `vote`.

*Appels attendus pour* `classer`*, écrits avant de coder (*`iris` *: les six exemples de l'exercice 4) :*

```
assert classer(iris, [4.7, 1.4], 3) == "versicolor"   # cas normal
assert classer(iris, [1.4, 0.2], 1) == "setosa"       # cas limite : x est un exemple
                                                      # (distance 0)
assert classer(iris, [5.4, 1.5], 3) == "virginica"    # cas piege : le plus proche
                                                      # (versicolor) perd le vote
```

```python
def k_plus_proches(exemples, x, k):
    dists = []                                          # INITIALISER
    for descripteurs, classe in exemples:               # PARCOURIR
        dists.append((distance2(x, descripteurs), classe))  # METTRE A JOUR
    dists.sort(key=lambda couple: couple[0])            # tri par distance croissante
    return [classe for (d, classe) in dists[:k]]        # RENVOYER : les k premiers

def classer(exemples, x, k):
    return vote(k_plus_proches(exemples, x, k))
```

Sur les iris avec `x=[4.7,1.4]` et $k = 3$ : `'versicolor'`.

*Autre méthode :* la liste `dists` se construit aussi par compréhension, en une ligne : `dists = [(distance2(x, d), classe) for (d, classe) in exemples]`.

## Choisir $k$, comprendre les limites

**Exercice 8** *L'effet de $k$* ↑énoncé p. 486

**▶ Solution.** **a)** Avec $k = 1$, la prédiction **copie** le seul voisin le plus proche : si c'est un exemple mal étiqueté, la réponse est fausse. **b)** Si $k$ = nombre total d'exemples, tous votent : on renvoie **toujours** la classe globalement majoritaire, *sans plus dépendre* de $x$. **c)** Un $k$ **impair** évite les **égalités** de vote entre les deux classes.

▷ **Erreur fréquente.** croire qu'un $k$ impair évite toute égalité : avec *trois* classes ou plus, un vote à 3 voisins peut donner une voix à chacune.

**Exercice 9** *Départager une égalité* ↑énoncé p. 487

**▶ Solution.** **a)** Voisins rangés : versicolor, versicolor, virginica, virginica, setosa, setosa. Égalité pour $k = 4$ (2 contre 2), $k = 5$ (2, 2 et 1) et $k = 6$ (2, 2 et 2). **b)** *Appels attendus, écrits avant de coder :*

```python
assert vote_departage(["A", "B", "B", "A", "C"]) == "B"   # cas normal (enonce)
assert vote_departage(["A"]) == "A"                       # cas limite : un seul voisin
assert vote_departage(["B", "A", "A", "B"]) == "A"        # cas piege : on retire le B
                                                          # lointain, pas le A
```

```python
def compter(voisins):
    compte = {}                     # INITIALISER
    for c in voisins:               # PARCOURIR
        if c in compte:             # TESTER
            compte[c] += 1          # METTRE A JOUR
        else:
            compte[c] = 1           # METTRE A JOUR
    return compte                   # RENVOYER

def vote_departage(voisins):
    n = len(voisins)                # INITIALISER : tous les voisins
    while n > 0:
        compte = compter(voisins[:n])
        maxi = 0                    # INITIALISER
        for c in compte:            # PARCOURIR
            if compte[c] > maxi:    # TESTER
                maxi = compte[c]    # METTRE A JOUR
        gagnants = [c for c in compte if compte[c] == maxi]
        if len(gagnants) == 1:      # TESTER : une seule classe en tete ?
            return gagnants[0]      # RENVOYER
        n = n - 1                   # METTRE A JOUR : on retire le plus lointain
```

La boucle s'arrête : avec un seul voisin, il ne peut pas y avoir d'égalité. **c)** Avec 5 voisins : A 2, B 2, C 1 (égalité) ; avec 4 : A 2, B 2 (égalité) ; avec 3 : A 1, B 2 → `"B"`. Le plus proche voisin est un A, mais une fois les voisins lointains retirés, B reste majoritaire : la règle départage par le **vote**, pas par le premier voisin. Sur les iris, avec $k = 4$, 5 ou 6, on obtient `'versicolor'`.

**Exercice 10** *Le piège de l'échelle* ↑énoncé p. 487

▶ **Solution.** **a)** Le **revenu** domine : un écart de 200 € pèse $200^2 = 40\,000$, alors qu'un écart de 10 ans ne pèse que $10^2 = 100$. L'âge devient négligeable. **b)** Il faut **normaliser** : ramener chaque descripteur à la même échelle (par exemple entre 0 et 1) pour qu'ils comptent équitablement.

**Exercice 11** *Le coût* ↑énoncé p. 487

▶ **Solution.** **a)** À **10 000** exemples (tous). **b)** « Paresseux » car il ne construit **aucun modèle** au préalable : il garde tout et compare à la demande. Coût **linéaire** en le nombre d'exemples, à chaque prédiction.

## Applications

**Exercice 12** *Recommander un poste (football)* ↑énoncé p. 487

▶ **Solution.** **a)** Distances au carré de (176; 70) :

| joueur | poste | $d^2$ |
|---|---|---|
| (180 ;75) | milieu | 41 |
| (178 ;72) | milieu | **8** |
| (192 ;88) | défenseur | 580 |
| (195 ;90) | défenseur | 761 |
| (170 ;65) | attaquant | 61 |
| (173 ;68) | attaquant | **13** |

**b)** 1-NN : le plus proche (8) est un **milieu**. 3-NN : les trois plus proches sont 8 (milieu), 13 (attaquant), 41 (milieu) → 2 milieux contre 1 → **milieu**. **c)** Ici taille (∼170–195) et poids (∼65–90) sont du **même ordre** : la normalisation n'est pas indispensable. Au contraire, âge (0–100) et revenu (0–5000) diffèrent d'un facteur ∼50, d'où l'obligation de normaliser dans l'exercice « Le piège de l'échelle ».

**Exercice 13** *Reconnaître une image $3 \times 3$ (sans machine)* ↑énoncé p. 488

▶ **Solution.** **a)** $T = [1,1,1,0,1,0,0,1,0]$, $L = [1,0,0,1,0,0,1,1,1]$, $N = [1,1,1,0,1,0,0,1,1]$. **b)** $N$ et $T$ ne diffèrent que sur le **dernier** pixel → $d^2(N,T) = \mathbf{1}$. $N$ et $L$ diffèrent sur **5** pixels → $d^2(N,L) = \mathbf{5}$. **c)** Le 1-NN prédit **T** (le plus proche). C'est cohérent : $N$ est un « T » avec un pixel d'encre en plus.

## Mini-problème de synthèse

**Exercice 14** *Évaluer la qualité* ↑énoncé p. 488

▶ **Solution.** **a)** *Appels attendus, écrits avant de coder (`iris` : les six exemples de l'exercice 4) :*

```
deux = [([4.7, 1.4], "versicolor"), ([5.4, 1.5], "versicolor")]
assert taux_reussite(iris, deux, 3) == 0.5       # cas normal : une bonne reponse sur deux
assert taux_reussite(iris, iris, 1) == 1.0       # cas limite : test = entrainement, score
                                                 # trompeur
assert taux_reussite(iris, deux[1:], 3) == 0.0   # cas piege : le plus proche est versicolor,
                                                 # le vote dit virginica
```

```
def taux_reussite(train, test, k):
    bons = 0                                          # INITIALISER
    for descripteurs, vraie in test:                  # PARCOURIR
        if classer(train, descripteurs, k) == vraie:  # TESTER : bonne prediction ?
            bons += 1                                 # METTRE A JOUR
    return bons / len(test)                           # RENVOYER : une proportion
```

**b)** Sur les exemples d'entraînement, l'évaluation est **trompeuse** (avec $k = 1$, chaque exemple est son propre voisin → 100 %). Seul un jeu de **test** (jamais vu) mesure la capacité à **généraliser**. **c)** Le meilleur $k$ dépend des données : aucune valeur ne convient à tous les problèmes, d'où l'essai

de plusieurs valeurs sur le jeu de test (c'est la façon habituelle de **choisir** $k$). Avec $k$ trop petit ($k = 1$), on copie le plus proche voisin, même aberrant : très sensible au bruit. Avec $k$ trop grand, on interroge des voisins lointains ; à l'extrême, on répond toujours la classe la plus fréquente. Le bon $k$ est un **compromis**.

**Exercice 15** *Un k-NN selon un assistant d'IA* ↑énoncé p. 488

▶ **Solution.** **a)** Non. Sur les six iris du cours, `classer(iris, [4.7, 1.4], 3)` renvoie `'setosa'` (et de même avec $k = 1$), alors que les exemples les plus proches de $x$ sont des *versicolor* ($d^2 = 0.05$). **b)** L'erreur est l'ordre dans les couples : `(classe, distance)`. Le tri `dists.sort()` d'une liste de couples se fait sur le **premier** élément, donc ici par ordre **alphabétique des classes** (*setosa* avant *versicolor*), et non par distance croissante : les « $k$ premiers » sont les $k$ premiers dans l'alphabet. On met la distance en premier (ou on trie avec une clé, comme dans le cours) :

```
def classer(exemples, x, k):
    dists = []
    for descripteurs, classe in exemples:
        dists.append((distance(x, descripteurs), classe))
    dists.sort()                    # tri par distance croissante
    compte = {}
    for d, classe in dists[:k]:
        if classe in compte:
            compte[classe] += 1
        else:
            compte[classe] = 1
    meilleure = None
    for c in compte:
        if meilleure is None or compte[c] > compte[meilleure]:
            meilleure = c
    return meilleure
```

Avec cette correction, l'appel renvoie `'versicolor'`. **c)** Afficher `dists` après le tri (les distances ne sont pas dans l'ordre croissant, cela saute aux yeux), ou tester sur l'exemple du cours dont on connaît la réponse.

## Entraînement

**Exercice 16** *Distances au carré* ↑énoncé p. 489

▶ **Solution.** a) $4^2 + 3^2 = 25$, $d = 5$. b) $2^2 + 3^2 = 13$, $d = \sqrt{13} \approx 3{,}6$. c) $2^2 + 1^2 + 3^2 = 14$, $d = \sqrt{14} \approx 3{,}7$. d) $0{,}3^2 + 0{,}2^2 = 0{,}09 + 0{,}04 = 0{,}13$, $d \approx 0{,}36$.

**Exercice 17** *Le vote des k plus proches voisins* ↑énoncé p. 489

▶ **Solution.** a) Classement : 1 (bleu), 2 (rouge), 4 (rouge), 6 (bleu), 9 (rouge). $k = 1$ : **bleu** ; $k = 3$ : 2 rouges contre 1 bleu, **rouge** ; $k = 5$ : 3 rouges contre 2 bleus, **rouge**. b) Classement : 1 (chien), 2 (chat), 3 (chat), 5 (chat), 7 (chien), 8 (chien). $k = 1$ : **chien** ; $k = 3$ : **chat** ; $k = 5$ : 3 chats contre 2 chiens, **chat** ; $k = 6$ : 3 contre 3, **égalité** : le vote ne permet pas de conclure sans règle pour départager (voir l'exercice « Départager une égalité »).

▷ **Erreur fréquente.** garder pour tout $k$ la prédiction du plus proche voisin : en a) comme en b), la prédiction change entre $k = 1$ et $k = 3$.

# CORRIGÉS DES TP

## TP Reconnaître des chiffres et des lettres manuscrits

↑ énoncé p. 491

### Partie 1 — Le $k$-NN sur 4 points

**1.2 À la main.** ▶ Solution. **1.** Distances au carré de $x = (2;2)$ :

| exemple | classe | $d^2$ | |
|---|---|---|---|
| $(1;1)$ | A | $1^2 + 1^2 = \mathbf{2}$ | |
| $(2;1)$ | A | $0^2 + 1^2 = \mathbf{1}$ | ← le plus proche |
| $(5;4)$ | B | $3^2 + 2^2 = \mathbf{13}$ | |
| $(6;5)$ | B | $4^2 + 3^2 = \mathbf{25}$ | |

**2.** Le plus proche est $(2;1)$, de classe `A` → le 1-NN prédit **A**. **3.** Les 3 plus proches : $(2;1)$ A, $(1;1)$ A, $(5;4)$ B → vote 2 A contre 1 B → le 3-NN prédit **A**.

**1.3 — 1.6 Les trous.** ▶ Solution. **(a)** `(a[i] - b[i]) ** 2` **(b)** `distance(x, descripteurs)` **(c)** `couple[0]` **(d)** `compte[c] += 1`. Ici, la fonction `distance` de `ocr.py` renvoie la distance **au carré** $d^2$ (sans racine, contrairement à celle du cours) : pour comparer des distances, cela suffit.

```python
def distance(a, b):
    s = 0
    for i in range(len(a)):
        s = s + (a[i] - b[i]) ** 2
    return s

def k_plus_proches(exemples, x, k):
    dists = []
    for descripteurs, classe in exemples:
        dists.append((distance(x, descripteurs), classe))
    dists.sort(key=lambda couple: couple[0])
    return [classe for (d, classe) in dists[:k]]

def vote(classes):
    compte = {}
    for c in classes:
        if c in compte:
            compte[c] += 1
        else:
            compte[c] = 1
    meilleure = None
    for c in compte:
        if meilleure is None or compte[c] > compte[meilleure]:
            meilleure = c
    return meilleure
```

Les tests attendus, cohérents avec le calcul à la main :

- `distance([1,1],[2,2])` $= 2$ ;
- `k_plus_proches(exemples,[2,2],3)` $=$ `['A','A','B']` ;
- `vote(["A","B","A"])` $=$ `'A'` ;
- `classer(exemples,[2,2],3)` $=$ `'A'`.

## Partie 2 — Une image est une liste de nombres

**2.1** ▶ **Solution.** `len(train)`, `len(test)` = **(3000, 500)**.

**2.2** ▶ **Solution.** La première image d'entraînement est un **2** (étiquette `'2'`) ; le jeu compte 300 images de chacun des 10 chiffres.

**2.3** ▶ **Solution.** **1.** $14 \times 14 = \mathbf{196}$ pixels ; `len(pixels)` vaut 196. **2.** Chaque nombre vaut **0** (pixel blanc) ou **1** (pixel d'encre).

**2.4** ▶ **Solution.** **1.** Les images s'obtiennent par exemple avec `zeros = [p for (p, e) in train if e == "0"]`, puis `zero_a = zeros[0]`, `zero_b = zeros[1]` (de même pour `un`). **2.** La distance entre deux « 0 » (= 39) est **plus petite** que celle entre un « 0 » et un « 1 » (= 51). C'est rassurant : deux images du **même** chiffre se ressemblent (peu de pixels diffèrent), donc elles sont *proches* au sens de la distance — c'est précisément ce qui permet au $k$-NN de reconnaître le bon chiffre.

## Partie 3 — Reconnaître les chiffres

**3.1 / 3.2** ▶ **Solution.** `test[0]` est un **1**, prédit **1**. Sur les 10 premières images de test : **10 bonnes réponses sur 10**.

**3.3** ▶ **Solution.** Trou **(e)** : `classer(exemples, pixels, k) == vraie`.

```
def taux_reussite(exemples, tests, k, limite=100):
    tests = tests[:limite]
    bons = 0
    for pixels, vraie in tests:
        if classer(exemples, pixels, k) == vraie:
            bons = bons + 1
    return bons / len(tests)
```

On obtient **0,97** sur les 100 premiers tests (plus de 9 chiffres sur 10).

**3.4** ▶ **Solution.** Pour les chiffres, le taux est **très stable**, entre 95 et 97 % :

| $k$ | 1 | 3 | 5 | 7 |
|---|---|---|---|---|
| taux | 0,95 | 0,97 | 0,95 | 0,96 |

Avec $k = 500$, on fait voter des voisins très **lointains**, qui sont souvent d'autres chiffres : la réponse dépend de moins en moins de l'image et le taux **chute** nettement (**0,69** sur les 100 premiers tests). À la limite, si tous les exemples votaient, chaque chiffre aurait ses 300 voix et le vote ne départagerait plus rien.

## Partie 4 — Aller plus loin

**4.1** ▶ **Solution.** Les images mal classées sont souvent des chiffres **mal écrits** ou **ambigus** (un 4 ouvert ressemble à un 9, un 3 à un 5…). Les erreurs sont **compréhensibles** : la machine confond ce qu'un humain trouverait *aussi* ambigu.

**4.2** ▶ **Solution.** Le taux des lettres est **moins bon** : **0,70** avec $k = 5$ sur les 100 premiers tests. Deux raisons : il y a **26 classes** de lettres contre seulement **10** de chiffres, et les lettres manuscrites ont des formes **plus variées et plus proches** les unes des autres. Confusions inévitables : **O/Q**, **I/L**, **U/V**, **O/C**…

## Partie 5 — Défi

▶ **Solution.** **1.** Le `T` fourni est bien reconnu comme `T`. Les initiales fonctionnent d'autant mieux qu'elles sont **épaisses** et **centrées**. **2.** La distance compare les images **pixel par pixel, à la même position**. Une lettre **mal centrée** ou trop **fine** n'a presque aucun pixel « en commun » avec les exemples de la même lettre : sa distance à *tous* les exemples devient grande et la prédiction déraille. Les formes arrondies dessinées de façon anguleuse tombent près d'autres lettres. C'est la limite du $k$-NN sur les images brutes : il ne « comprend » pas la forme, il compare des pixels.

## Réponses des questions de la fiche de révision

↑ questions p. 504

### Se tester

**1.** oui : il s'appuie sur des exemples déjà étiquetés (§ I)
**2.** $3^2 + 4^2 = 25$ (§ III)
**3.** chat ; chien ; chat (§ IV)
**4.** une égalité 2 contre 2 ; un $k$ impair (§ V)
**5.** normaliser les descripteurs (sinon le revenu écrase l'âge) (§ VII)

### Et avant ?

**1.** $10 + 1 + 1$ (3 pièces) ; $6 + 6$ (2 pièces) (ch. 11, § III)
**2.** seul son propriétaire (`w` seulement dans le premier groupe) (ch. 9, § VI)
**3.** quadratique : environ $n^2/2$ comparaisons (ch. 6, § IV)

# Problèmes de synthèse

> **Remarque**
> On donne une version de chaque fonction ; d'autres écritures correctes sont possibles. Le fichier complet, `corrige_synthese_premiere.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/premiere/synthese`.

## Après les chapitres 1 et 2 (Les bases de la programmation Python ; Le binaire et l'écriture des nombres)

### Exercice 1 *Le binaire avec des entiers* ↑énoncé p. 510

**1.** $13 = 2 \times 6 + 1$, $6 = 2 \times 3 + 0$, $3 = 2 \times 1 + 1$, $1 = 2 \times 0 + 1$ : en lisant les restes de bas en haut, $13 = 1101_2$. De même, $185 = 10111001_2$. Les bits sont obtenus **de droite à gauche** : le premier reste est le bit de poids faible.

**2.**

```
def en_binaire(n):
    resultat = 0
    puissance = 1
    while n > 0:
        resultat = resultat + (n % 2) * puissance
        puissance = puissance * 10
        n = n // 2
    return resultat
```

Pour `n = 0`, on n'entre pas dans la boucle et la fonction renvoie `0`.

**3.** Même idée en sens inverse : on lit les chiffres de `b` de droite à gauche (`b % 10`) et on les multiplie par $1, 2, 4, 8\ldots$

```
def en_decimal(b):
    total = 0
    puissance = 1
    while b > 0:
        total = total + (b % 10) * puissance
        puissance = puissance * 2
        b = b // 10
    return total

for n in range(301):
    assert en_decimal(en_binaire(n)) == n
```

**4. Variant** : `n`. C'est un entier positif ou nul qui diminue strictement à chaque tour (`n // 2 < n` dès que $n \geqslant 1$) ; il finit donc par valoir 0, et la condition `n > 0` devient fausse.

**5.**

```
def nb_bits(n):
    nb = 1
    while n >= 2:
        n = n // 2
        nb = nb + 1
    return nb
```

`nb_bits(255)` vaut 8 et `nb_bits(256)` vaut 9 : sur 8 bits, on code les entiers de 0 à $2^8 - 1 = 255$, et $256 = 2^8$ en demande un de plus (`100000000`).

**Exercice 2** *Le bit de parité* ↑énoncé p. 511

**1.** et **2.**

```
def nb_uns(n):
    nb = 0
    while n > 0:
        nb = nb + n % 2      # le dernier bit vaut 0 ou 1
        n = n // 2
    return nb

def bit_parite(n):
    return nb_uns(n) % 2
```

**3.** Multiplier par 2 décale tous les bits d'un cran vers la gauche (comme multiplier par 10 en décimal) et place un 0 à droite ; ajouter $b$ remplace ce 0 par $b$.

```
def emettre(n):
    return 2 * n + bit_parite(n)
```

$13 =$ `1101` a trois bits à 1 : le bit de parité vaut 1, et `emettre(13)` renvoie $27 =$ `11011`$_2$.

**4.**

```
def verifier(m):
    return nb_uns(m) % 2 == 0
```

**5.** `11111` contient cinq 1 (un bit modifié) : `verifier` renvoie `False`, l'erreur est **détectée**. `11101` contient quatre 1 (deux bits modifiés) : `verifier` renvoie `True`, l'erreur **passe inaperçue**. Le bit de parité détecte un nombre *impair* de bits modifiés, jamais un nombre pair ; il ne dit pas non plus *quel* bit est faux.

**Exercice 3** *Des entiers relatifs sur 8 bits* ↑énoncé p. 511

**1.** $12 =$ `00001100`, on inverse : `11110011`, on ajoute 1 : `11110100`, soit $128+64+32+16+4 = 244$.

**2.** Pour chaque position, le bit inversé vaut 1− le bit d'origine. La somme de $a$ et de son inversé a donc ses 8 bits à 1 : elle vaut `11111111`$_2 = 255$, d'où inversé $= 255 - a$. Le code de $-x$ est alors $(255 - x) + 1 = 256 - x$. Pour $-12$ : $256 - 12 = 244$.

**3.**

```
def code_8bits(x):
    if x >= 0:
        return x
    return 256 + x

def valeur(v):
    if v < 128:          # bit de poids fort a 0 : entier positif
        return v
    return v - 256

for x in range(-128, 128):
    assert valeur(code_8bits(x)) == x
```

**4.**

```
def addition_8bits(a, b):
    return valeur((code_8bits(a) + code_8bits(b)) % 256)
```

$12 + (-12)$ donne bien 0 ($12 + 244 = 256$, et `256 % 256` vaut 0 : la retenue qui déborde est oubliée). Mais $100 + 50$ renvoie $-106$ et $127 + 1$ renvoie $-128$ : le vrai résultat (150, 128) dépasse 127, le plus grand entier codable. Son code a un bit de poids fort à 1, qui est lu comme le signe d'un négatif. C'est un **dépassement de capacité** : la machine ne signale rien, c'est au programmeur d'y penser.

## Après les chapitres 3 et 4 (Les types construits ; Algorithmique : le parcours séquentiel)

### Exercice 4 *Le bulletin de notes* ↑énoncé p. 511

**1.** `notes["Noé"]` vaut `[8, 11, 14, 10]`, `len(notes)` vaut 3 (trois clés) et `notes["Léa"][2]` vaut 9. Pour ajouter une note : `notes["Zoé"].append(16)`.

**2.**

```
def moyenne(t):
    s = 0
    for x in t:
        s = s + x
    return s / len(t)

def moyennes(notes):
    resultat = {}
    for nom in notes:
        resultat[nom] = moyenne(notes[nom])
    return resultat
```

Avec le dictionnaire de l'énoncé : `{"Léa": 12.0, "Noé": 10.75, "Zoé": 15.0}`.

**3.**

```
def meilleur(notes):
    meilleur_nom = None
    meilleure_moy = -1          # plus petite que toute moyenne possible
    for nom in notes:
        m = moyenne(notes[nom])
        if m > meilleure_moy:
            meilleur_nom = nom
            meilleure_moy = m
    return (meilleur_nom, meilleure_moy)
```

`meilleur(notes)` renvoie `("Zoé", 15.0)`. Le champion part ici d'une moyenne fictive `-1` (une *sentinelle*), valable parce que toute moyenne est positive. *Autre méthode*, celle du cours : initialiser le champion avec un élément existant, par exemple le premier nom rencontré dans le dictionnaire.

**4.**

```
def nb_notes_sous(notes, seuil):
    nb = 0
    for nom in notes:
        for note in notes[nom]:
            if note < seuil:
                nb = nb + 1
    return nb
```

`nb_notes_sous(notes, 10)` renvoie 2 (le 9 de Léa et le 8 de Noé). Chaque note est comparée une fois : $30 \times 8 = 240$ comparaisons. Le coût est proportionnel au nombre total de notes.

### Exercice 5 *Une image en noir et blanc* ↑énoncé p. 512

**1.** L'image est un visage souriant. `image[5][3]` (ligne 5, colonne 3, en comptant à partir de 0) vaut 1 : un pixel noir de la bouche.

**2.**

```
def nb_noirs(image):
    nb = 0
    for ligne in image:
        for pixel in ligne:
            if pixel == 1:
                nb = nb + 1
    return nb
```

Pour l'image de l'énoncé : 26 pixels noirs.

**3.** `1 - pixel` échange 0 et 1 :

```
def negatif(image):
    return [[1 - pixel for pixel in ligne] for ligne in image]
```

La compréhension construit de nouveaux tableaux : `image` n'est pas modifiée.

**4.** La première ligne `00111100` représente $32 + 16 + 8 + 4 = 60$.

```
def ligne_en_octet(ligne):
    v = 0
    for bit in ligne:
        v = 2 * v + bit      # decalage a gauche, puis ajout du bit
    return v

def compresser(image):
    return [ligne_en_octet(ligne) for ligne in image]
```

`compresser(image)` renvoie `[60, 66, 165, 129, 165, 153, 66, 60]`.

*Autre méthode :* sans compréhension, on part d'un tableau vide et on ajoute l'octet de chaque ligne.

```
def compresser(image):
    octets = []
    for ligne in image:
        octets.append(ligne_en_octet(ligne))
    return octets
```

**5.** On retrouve les bits par divisions successives, du dernier (colonne 7) au premier :

```
def octet_en_ligne(v):
    ligne = [0] * 8
    for i in range(7, -1, -1):
        ligne[i] = v % 2
        v = v // 2
    return ligne

def decompresser(octets):
    return [octet_en_ligne(v) for v in octets]
```

On stocke 8 entiers au lieu de 64 : chaque pixel n'occupe plus qu'un bit, au lieu d'un entier entier. C'est ainsi que les formats d'image en noir et blanc rangent les pixels.

## Après les chapitres 5 à 8 (Spécifier et mettre au point ses programmes ; Les algorithmes de tri ; La recherche dichotomique ; Les données en tables)

### Exercice 6 *Le classement d'un cross*

↑énoncé p. 512

**1.** et **2.**

```
def plus_rapide(table):
    """Renvoie le nom du coureur au plus petit temps.
    Precondition : table est une liste NON VIDE de fiches
    {"nom", "classe", "temps"}, les temps etant des entiers positifs.
    Postcondition : le temps de la fiche renvoyee est inferieur
    ou egal a celui de toutes les autres."""
    assert len(table) > 0, "table vide"
    meilleur = table[0]
    for fiche in table:
        if fiche["temps"] < meilleur["temps"]:
            meilleur = fiche
    return meilleur["nom"]

assert plus_rapide(coureurs) == "Tom"
assert plus_rapide([{"nom": "Ana", "classe": "1G1", "temps": 900}]) \
       == "Ana"
assert plus_rapide([{"nom": "A", "classe": "1G1", "temps": 900},
                    {"nom": "B", "classe": "1G1", "temps": 600}]) == "B"
```

Non : des tests réussis montrent seulement que la fonction est juste **sur ces exemples**. Un cas non testé (deux coureurs ex æquo, par exemple) peut encore poser problème. Les tests peuvent montrer la présence d'erreurs, jamais leur absence.

**3.** On reprend le tri par insertion du cours ; on déplace des fiches entières, mais on compare leurs temps.

```
def trier_par_temps(table):
    for i in range(1, len(table)):
        fiche = table[i]
        j = i - 1
        while j >= 0 and table[j]["temps"] > fiche["temps"]:
            table[j + 1] = table[j]
            j = j - 1
        table[j + 1] = fiche
```

Ordre d'arrivée : Tom (698), Hugo (712), Sara (731), Maël (745), Inès (754), Léna (803).

**4.** Au début du tour d'indice `i`, les fiches d'indices 0 à `i - 1` sont **rangées par temps croissant** : ce sont les `i` premières fiches de départ, remises dans l'ordre. C'est un **invariant de boucle** ; à la sortie (`i` vaut `len(table)`), il dit que toute la table est triée.

**5.** `temps = [fiche["temps"] for fiche in coureurs]` vaut `[698, 712, 731, 745, 754, 803]`.

```
def position(t, x):
    deb = 0
    fin = len(t) - 1
    while deb <= fin:
        mil = (deb + fin) // 2
        if t[mil] == x:
            return mil
        elif x > t[mil]:
            deb = mil + 1
        else:
            fin = mil - 1
    return -1
```

Pour 745 : `mil = 2` (731 < 745, `deb = 3`), puis `mil = 4` (754 > 745, `fin = 3`), puis `mil = 3` : trouvé, en **3 tours**. L'indice 3 correspond à la **4e place**. Pour 760 : `mil = 2`, puis 4, puis 5 (803 > 760, `fin = 4`) ; la zone est vide après **3 tours** et la fonction renvoie `-1` : aucun coureur n'a ce temps.

## Exercice 7 *Les doublons d'une liste d'inscriptions*

↑énoncé p. 513

**1.**

```
def a_un_doublon(t):
    for i in range(len(t)):
        for j in range(i + 1, len(t)):
            if t[i] == t[j]:
                return True
    return False
```

Au pire (aucun doublon), la case 0 est comparée aux 999 suivantes, la case 1 aux 998 suivantes… soit $999 + 998 + \cdots + 1 = \frac{1000 \times 999}{2} = 499\,500$ comparaisons.

**2.** Pour `["a", "b", "c"]`, au dernier tour `i` vaut 2 et `t[i + 1]` est `t[3]`, qui n'existe pas : `IndexError: list index out of range`. Pour `["a", "a", "c"]`, la fonction renvoie `True` dès `i = 0` et n'atteint jamais le dernier indice : le test passe alors que la fonction est fausse. Il faut s'arrêter à l'avant-dernière case :

```
def a_un_doublon_trie(t):
    for i in range(len(t) - 1):
        if t[i] == t[i + 1]:
            return True
    return False
```

**3.**

```
assert not a_un_doublon_trie([])              # vide : range(-1) est vide
assert not a_un_doublon_trie(["a"])           # un seul element
assert a_un_doublon_trie(["a", "a", "b"])     # doublon au debut
assert a_un_doublon_trie(["a", "b", "b"])     # doublon a la fin
assert not a_un_doublon_trie(["a", "b", "c"])
```

Le test « doublon à la fin » et le test « aucun doublon » auraient tous deux révélé l'erreur de la question 2.

**4.** `a_un_doublon_trie(sorted(t))` fait environ $1000 \times 10 = 10\,000$ comparaisons pour le tri, puis au plus 999 : environ 11 000 en tout, contre 499 500, soit environ 45 fois moins.

**5.**

```
def doublons(t):
    resultat = []
    for i in range(1, len(t)):
        nouveau = len(resultat) == 0 or resultat[-1] != t[i]
        if t[i] == t[i - 1] and nouveau:
            resultat.append(t[i])
    return resultat
```

Le tableau trié est `["hugo.p", "ines.k", "ines.k", "lea.m", "sara.d", "tom.b", "tom.b", "tom.b"]` ; `doublons` renvoie `["ines.k", "tom.b"]`. Grâce au second test, `"tom.b"`, inscrit trois fois, n'apparaît qu'une fois.

## Après les chapitres 9 à 12 (Architecture des ordinateurs et systèmes d'exploitation ; Le Web : réseaux et interactions homme-machine ; Les algorithmes gloutons ; Les $k$ plus proches voisins)

### Exercice 8 — *Les droits d'un fichier, en binaire*

↑énoncé p. 514

**1.** `-rw-r-----` : le propriétaire peut le lire et le modifier, les membres du groupe peuvent seulement le lire, les autres n'ont aucun droit. Code : `110 100 000`, soit **640**.

**2.** et **3.**

```
def triplet_en_chiffre(triplet):
    v = 0
    for c in triplet:
        bit = 0
        if c != "-":
            bit = 1
        v = 2 * v + bit
    return v

def code_droits(ligne):
    proprietaire = triplet_en_chiffre(ligne[1:4])
    groupe = triplet_en_chiffre(ligne[4:7])
    autres = triplet_en_chiffre(ligne[7:10])
    return str(proprietaire) + str(groupe) + str(autres)
```

`code_droits("drwxr-xr-x")` renvoie `"755"` (le premier caractère, `d` pour un dossier, est ignoré).

**4.**

```
def chiffre_en_triplet(d):
    triplet = ""
    if d // 4 == 1:
        triplet = triplet + "r"
    else:
        triplet = triplet + "-"
    if (d // 2) % 2 == 1:
        triplet = triplet + "w"
    else:
```

```
        triplet = triplet + "-"
    if d % 2 == 1:
        triplet = triplet + "x"
    else:
        triplet = triplet + "-"
    return triplet

def droits(code):
    resultat = ""
    for c in code:
        resultat = resultat + chiffre_en_triplet(int(c))
    return resultat

for a in range(8):
    for b in range(8):
        for c in range(8):
            code = str(a) + str(b) + str(c)
            assert code_droits("-" + droits(code)) == code
```

**5.** Il y a 9 droits, chacun présent ou absent : $2^9 = 512$ réglages. Un chiffre code 3 bits, et le plus grand nombre de 3 bits est $111_2 = 7$. Écrire un nombre avec des chiffres de 0 à 7, c'est l'écrire en base 8 (l'**octal**), qui regroupe les bits par paquets de 3 comme l'hexadécimal les regroupe par 4.

## Exercice 9 *Un conseil de film* ↑énoncé p. 514

**1.** `https://www.cine.mc/conseil?action=8&humour=2&emotion=6` : avec `get`, les paires `name=valeur` suivent un `?` et sont séparées par `&`.

**2.**

```
def lire_reponses(url):
    reponses = {}
    for paire in url.split("?")[1].split("&"):
        morceaux = paire.split("=")
        reponses[morceaux[0]] = int(morceaux[1])
    return reponses
```

**3.** *Mission Mistral* : $(8-8)^2 + (1-2)^2 + (5-6)^2 = 0 + 1 + 1 = 2$. *Après l'orage* : $(4-8)^2 + (1-2)^2 + (8-6)^2 = 16 + 1 + 4 = 21$.

**4.** On adapte `distance`, `k_plus_proches` et `vote` du cours (distance au carré, puisqu'on ne fait que comparer) :

```
def distance2(film, profil):
    somme = 0
    for critere in ["action", "humour", "emotion"]:
        somme = somme + (film[critere] - profil[critere]) ** 2
    return somme

def k_plus_proches(films, profil, k):
    dists = []
    for film in films:
        dists.append((distance2(film, profil), film))
    dists.sort(key=lambda couple: couple[0])
    return [film for (d, film) in dists[:k]]

def conseils(films, profil, k):
    return [film["titre"] for film in k_plus_proches(films, profil, k)]

def genre_conseille(films, profil, k):
    voisins = k_plus_proches(films, profil, k)
    return vote([film["genre"] for film in voisins])
```

Distances au carré : *Mission Mistral* 2, *Course contre la nuit* 2, *Les Pirates du lagon* 14, *Après l'orage* 21, *La Lettre* 58, *Coloc en folie* 77, *Le Dîner raté* 107. Pour $k = 3$ : *Mission Mistral*, *Course contre la nuit*, *Les Pirates du lagon*, trois films d'**aventure**.

**5.** Distances au carré pour $(3;4;7)$ : *Après l'orage* 11, *La Lettre* 12, *Mission Mistral* 38, *Course contre la nuit* 40, puis les autres. Pour $k = 3$ : deux drames et une aventure, on conseille un **drame**. Pour $k = 4$ : deux drames et deux aventures, c'est une **égalité**. La fonction `vote` du cours ne remplace la classe retenue que par une classe *strictement* plus fréquente : elle garde la première rencontrée, ici `"drame"`, celle du plus proche voisin. Ce choix n'a rien d'évident : il vaut mieux choisir $k$ pour limiter les égalités, ou les départager explicitement.

**6.** Le profil est mémorisé **dans le client** (le navigateur). Deux semaines plus tard, le cookie est toujours là (2 592 000 secondes font 30 jours) : la requête contient l'en-tête `Cookie: profil=8-2-6`, que le navigateur ajoute de lui-même. Si le visiteur modifie ce cookie, il ne change que ses propres conseils : ce n'est pas grave. Le serveur doit seulement vérifier que les valeurs reçues sont bien des nombres de 0 à 10 avant de s'en servir, comme pour toute donnée venue du client.

## Exercice 10 *La soirée cinéma*

↑énoncé p. 515

**1.** Par points décroissants : *Le Dîner raté* (9 points, 120 min) est pris, il reste 120 min ; *Mission Mistral* (8 points, 90 min) est pris, il reste 30 min ; plus aucun film ne tient. Soirée de 210 min, **17 points**.

**2.** Points par minute : *Les Pirates du lagon* 0,100 ; *Mission Mistral* 0,089 ; *Le Dîner raté* 0,075 ; *La Lettre* 0,071 ; *Coloc en folie* 0,071 (un peu moins : $6/85 < 5/70$). On prend *Les Pirates du lagon* (reste 180 min), puis *Mission Mistral* (reste 90 min) ; *Le Dîner raté* ne tient pas ; on prend *La Lettre* (reste 20 min). Soirée de 220 min, **19 points**.

**3.**

```python
def soiree_gloutonne(duree_max, ordre):
    choisis = []
    reste = duree_max
    for (titre, duree, pts) in ordre:
        if duree <= reste:
            choisis.append(titre)
            reste = reste - duree
    return choisis

def points(f):
    return f[2]

def points_par_minute(f):
    return f[2] / f[1]

par_points = sorted(soiree, key=points, reverse=True)
soiree_gloutonne(240, par_points)
# Le Diner rate, puis Mission Mistral
par_minute = sorted(soiree, key=points_par_minute, reverse=True)
soiree_gloutonne(240, par_minute)
# ['Les Pirates du lagon', 'Mission Mistral', 'La Lettre']
```

**4.**

```python
def meilleure_soiree(duree_max, films):
    n = len(films)
    meilleur_choix = []
    meilleurs_points = 0
    for code in range(2 ** n):
        choix = []
        duree_totale = 0
        total_points = 0
        for i in range(n):
            if (code // 2 ** i) % 2 == 1:      # bit numero i du code
                titre, duree, pts = films[i]
                choix.append(titre)
                duree_totale = duree_totale + duree
                total_points = total_points + pts
```

```
        if duree_totale <= duree_max and total_points > meilleurs_points:
            meilleur_choix = choix
            meilleurs_points = total_points
    return meilleur_choix, meilleurs_points
```

La meilleure soirée est *Les Pirates du lagon*, *Coloc en folie* et *Mission Mistral* : 235 min et **20 points**. Aucun des deux gloutons n'était optimal (17 et 19 points), même si le second en était proche.

**5.** Avec 40 films, il faudrait essayer $2^{40} = 1\,099\,511\,627\,776$ soirées, plus de mille milliards : chaque film de plus double le travail. Un glouton ne parcourt les films qu'une fois, après un tri : il répond instantanément, avec une solution souvent bonne mais sans garantie d'être la meilleure.

# Lexique

*Les définitions sont résumées ; le renvoi (chapitre et page, cliquable) mène à la définition complète dans le cours. Les mots du langage Python sont regroupés à part, en fin de lexique.*

## A

**Accumulateur** Variable initialisée avant une boucle puis mise à jour à chaque tour (somme, compteur…) ; motif de base de tout parcours. ch. 1, p. 29

**Accusé de réception** Petit message de confirmation (ACK) renvoyé par le récepteur ; sans accusé au bout d'un délai, l'émetteur considère le paquet perdu et le renvoie. ch. 10, p. 393

**Actionneur** Dispositif qui reçoit un ordre numérique et agit sur le monde physique (une sortie) : moteur, lampe, haut-parleur… ch. 9, p. 355

**Adresse IP** Numéro d'une machine sur le réseau ; en IPv4, quatre nombres de 0 à 255, comme `193.51.208.14`. L'équivalent d'une adresse postale. ch. 10, p. 390

**Affectation** Instruction `nom = expression` (« reçoit ») : Python évalue d'abord la partie droite, puis range cette valeur dans la variable nommée à gauche. ch. 1, p. 22

**Algorithme** Méthode : suite finie et non ambiguë d'étapes qui, à partir de données, produit un résultat. Le mot vient du savant persan al-Khwârizmî. ch. 1, p. 21

**Algorithme glouton** Stratégie qui construit la solution morceau par morceau, en faisant à chaque étape le choix qui paraît le meilleur sur le moment, sans jamais revenir en arrière. ch. 11, p. 447

**Alias** Deux noms qui désignent le même objet : après `b = a`, modifier le tableau via `b` se voit via `a`. Affecter un tableau ne le copie pas. ch. 3, p. 147

**Apprentissage automatique** Plutôt que de programmer toutes les règles, on montre des exemples à la machine pour qu'elle apprenne à généraliser (*machine learning*). ch. 12, p. 475

**Arborescence, racine** Organisation des fichiers et répertoires en arbre ; sous Linux, tout part de la racine, notée `/`. ch. 9, p. 359

**Argument** Valeur fournie en entrée à une fonction lors de son appel, par exemple `3` et `4` dans `aire_rectangle(3, 4)`. ch. 1, p. 32

**ASCII** Table de 1963 qui associe à chaque caractère un numéro de 0 à 127 ; chaque caractère tient sur 7 bits, sans lettres accentuées. ch. 2, p. 103

**Assembleur** Écriture du langage machine avec des abréviations lisibles (`LDA`, `ADD`…) : une instruction assembleur correspond à une instruction machine. ch. 9, p. 356

## B

**Balise** Marque qui encadre un contenu HTML : une balise ouvrante `<p>` et une balise fermante `</p>` ; certaines, comme `<img>`, n'ont pas de fermante. ch. 10, p. 396

**Bibliothèque (module)** Ensemble de fonctions déjà écrites et testées, que l'on charge avec `import` ; on apprend à lire sa documentation plutôt qu'à la connaître par cœur. ch. 1, p. 34

**Binaire (base 2)** Écriture des nombres avec deux chiffres, `0` et `1`, chaque rang pesant une puissance de 2 : c'est ainsi que la machine représente toute information. ch. 2, p. 94

**Bit** Plus petite information (*binary digit*) : un `0` ou un `1`, deux états physiques faciles à distinguer pour un transistor. ch. 2, p. 93

**Boucle bornée** Boucle `for`, utilisée quand on connaît à l'avance le nombre de répétitions. ch. 1, p. 28

**Boucle non bornée** Boucle `while`, qui répète son bloc tant qu'une condition reste vraie, quand on ne sait pas à l'avance combien de tours il faudra. ch. 1, p. 31

## C

**Capteur** Dispositif qui mesure une grandeur du monde physique et la transforme en valeur numérique entrant dans la machine (une entrée). ch. 9, p. 355

**Chaîne de caractères** Texte (type `str`) : suite de caractères numérotés à partir de 0, écrite entre guillemets. Une chaîne est immuable. ch. 1, p. 25

**Champion (maximum, minimum)** Recherche du maximum : on initialise `m` avec `t[0]` puis on le remplace dès qu'un élément est plus grand ; `m` reste le maximum des éléments déjà vus. ch. 4, p. 182

**Charabia (mojibake)** Texte illisible (« cafÃ© ») obtenu quand on relit un fichier avec un autre encodage que celui qui a servi à l'écrire. ch. 2, p. 105

**Chemin absolu, chemin relatif** Emplacement d'un fichier : un chemin absolu part de la racine et commence par `/` ; un chemin relatif part du répertoire courant (`../Images/logo.png`). ch. 9, p. 359

**`chmod`** Commande qui modifie les droits d'un fichier, par lettres (`chmod o-r`) ou en chiffres : `chmod 754 fichier`, chaque chiffre codant en binaire les trois droits. ch. 9, p. 361

**Classe (étiquette)** Catégorie à laquelle appartient un individu (par exemple l'espèce d'une fleur), connue pour les exemples et à prédire pour un nouvel individu. ch. 12, p. 475

**Classification supervisée** À partir d'exemples déjà étiquetés (descripteurs et classe connus), prédire la classe d'un nouvel individu dont on ne connaît que les descripteurs. ch. 12, p. 475

**Clé (de dictionnaire)** Étiquette choisie qui donne accès à une valeur d'un dictionnaire. Les clés sont uniques et immuables (souvent `str`, `int` ou `tuple`). ch. 3, p. 149

**Client-serveur** Organisation des échanges où des machines clientes demandent des ressources à une machine serveur qui les fournit : c'est le modèle du Web. ch. 10, p. 392

**Compilateur, langage compilé** Un langage compilé (comme C) est traduit une fois pour toutes, avant l'exécution, par un compilateur, qui produit un fichier exécutable. ch. 5, p. 219

**Complément à deux** Codage des entiers relatifs sur $n$ bits : pour coder $-x$, on écrit $x$ en binaire, on inverse tous les bits et on ajoute 1. L'addition ordinaire reste juste. ch. 2, p. 100

**Compteur de programme** Registre qui contient l'adresse de la prochaine instruction ; il avance d'un cran à chaque cycle, sauf si une instruction de saut le modifie. ch. 9, p. 356

**Concaténation** Mise bout à bout de deux chaînes ou de deux tableaux avec `+` ; elle crée un nouvel objet sans modifier l'original. ch. 3, p. 146

**Condition** Expression dont la valeur est un booléen (`True` ou `False`), construite avec des opérateurs de comparaison et combinée avec `and`, `or`, `not`. ch. 1, p. 26

**Construction par compréhension** Écriture concise qui fabrique un tableau à partir d'un itérable : `[expression for x in itérable if condition]`, le filtre `if` étant facultatif. ch. 3, p. 145

**Cookie** Petite information que le serveur demande au navigateur de garder ; le navigateur la mémorise et la retransmet automatiquement à chaque requête vers le même site. ch. 10, p. 402

**Copie (d'un tableau)** Nouvel objet indépendant, obtenu en reconstruisant le tableau : `[x for x in a]` ou `a[:]`. Pour un tableau de tableaux, il faut recopier chaque ligne. ch. 3, p. 147

**Correction (d'un algorithme)** Propriété d'un algorithme qui donne le résultat attendu ; on la prouve par un invariant de boucle (alors que le variant prouve la terminaison). ch. 4, p. 183

**Côté client, côté serveur** Côté client, le code (JavaScript) s'exécute sur la machine de l'utilisateur après réception de la page ; côté serveur, sur la machine distante (PHP, Python…). ch. 10, p. 400

**Coût d'un algorithme** Nombre d'opérations élémentaires qu'effectue un algorithme, exprimé en fonction de la taille `n` des données. ch. 4, p. 184

**Coût linéaire** Coût proportionnel à la taille `n` des données : c'est celui d'un parcours simple. Doubler `n` double le temps. ch. 4, p. 184

**Coût logarithmique** Coût de l'ordre de $\log_2 n$ : celui de la dichotomie, qui fait au plus $\lfloor \log_2 n \rfloor + 1$ tours. Doubler `n` n'ajoute qu'une étape. ch. 7, p. 285

**Coût quadratique** Coût proportionnel à $n^2$, typique d'une boucle dans une boucle : doubler `n` multiplie le temps par 4. C'est le coût des tris par sélection et par insertion. ch. 4, p. 185 ; ch. 6, p. 255

**`cp`, `mv`, `rm` (Linux)** Commandes : `cp` copie un fichier, `mv` le déplace ou le renomme, `rm` le supprime définitivement (`rm -r` : un répertoire et son contenu). ch. 9, p. 360

**CSS** Langage qui décrit la présentation d'une page (couleurs, tailles, polices, disposition) ; il sépare la forme (CSS) du fond (HTML). ch. 10, p. 397

**CSV** Fichier texte (*Comma-Separated Values*) qui représente une table : la première ligne donne les descripteurs, chaque ligne suivante un enregistrement, les valeurs étant séparées par un séparateur (virgule, point-virgule). ch. 8, p. 312

**Cycle d'exécution** Boucle que répète sans fin le processeur : chercher l'instruction en mémoire, la décoder, l'exécuter. ch. 9, p. 356

## D

**Déboguer** Localiser et corriger une erreur : dérouler le programme à la main, placer des affichages (`print`), puis les retirer en gardant les tests. ch. 5, p. 217

**Demi-additionneur** Circuit qui additionne deux bits : la somme vaut $a$ xor $b$, la retenue vaut $a$ et $b$. ch. 2, p. 98

**Dépassement de capacité** Résultat trop grand pour le nombre de bits disponibles : il « déborde » et

l'on obtient une valeur fausse (en Python, les entiers n'ont pas cette limite). ch. 2, p. 101

**Descripteur** Nom d'une colonne d'une table (on dit aussi attribut) : ce qu'on renseigne pour chaque individu, comme `prenom` ou `annee`. ch. 8, p. 312 ; ch. 12, p. 475

**Dictionnaire (`dict`)** Ensemble de couples clé → valeur, écrit entre accolades ; on accède à une valeur par sa clé, et non par un indice. ch. 3, p. 149

**Distance euclidienne** Entre deux individus décrits par $p$ descripteurs : racine carrée de la somme des carrés des écarts, descripteur par descripteur (généralisation de Pythagore). ch. 12, p. 478

**DNS** Annuaire du réseau (*Domain Name System*) : il traduit un nom de domaine, comme `www.lycee.mc`, en adresse IP. ch. 10, p. 391

**DOM** Arbre formé par les balises emboîtées d'une page (*Document Object Model*), construit en mémoire par le navigateur et modifiable en JavaScript. ch. 10, p. 397

**Domaine de valeurs** Ensemble des valeurs que peut prendre un descripteur ; pour une jointure, les deux descripteurs doivent prendre leurs valeurs dans le même domaine. ch. 8, p. 319

**Doublon** Même fiche présente deux fois dans une table, erreur de saisie fréquente ; on la repère par un parcours qui mémorise les identifiants déjà rencontrés. ch. 8, p. 316

**Droits d'accès** Permissions d'un fichier : lecture (`r`), écriture (`w`), exécution (`x`), pour trois catégories : propriétaire (`u`), groupe (`g`), autres (`o`). ch. 9, p. 361

## E

**Élément neutre** Valeur d'initialisation d'un accumulateur qui ne change pas le résultat : `0` pour une somme, `1` pour un produit, `""` pour du texte, `[]` pour un tableau. ch. 4, p. 181

**Encapsulation** À l'envoi, chaque couche de protocoles (application, transport, réseau, liaison) ajoute son en-tête devant les données, comme des enveloppes dans des enveloppes. ch. 10, p. 390

**Encodage** Règle qui fait correspondre caractères et octets (ASCII, Latin-1, UTF-8…). Encoder : caractères vers octets ; décoder : octets vers caractères. ch. 2, p. 104 ; ch. 8, p. 313

**Enregistrement** Ligne d'une table (une fiche) : une valeur pour chaque descripteur, c'est-à-dire un p-uplet nommé ; en Python, un dictionnaire. ch. 8, p. 312

**Entrées-sorties** Organes qui font communiquer la machine avec le monde : clavier, écran, disque, réseau, capteurs et actionneurs. ch. 9, p. 354

**Événement** Ce qui se produit dans la page, souvent une action de l'utilisateur (clic, survol, touche, envoi d'un formulaire) ; on y réagit en exécutant une fonction JavaScript. ch. 10, p. 399

**Expression** Écriture formée de valeurs, de variables et d'opérateurs (`n + 1`, `a > b`…) que Python évalue pour obtenir une valeur ; c'est la partie droite d'une affectation. ch. 1, p. 22

## F

**Filtrer une table** Parcourir la table et garder dans une nouvelle liste les enregistrements qui vérifient un critère, par une boucle ou par compréhension. ch. 8, p. 315

**Flottant** Nombre à virgule stocké sur un nombre fini de bits (type `float`) ; sa valeur est le plus souvent approchée : on ne teste jamais l'égalité de deux flottants. ch. 2, p. 102

**Fonction** Bloc d'instructions nommé, défini une fois (`def`) puis appelé autant de fois qu'on veut ; il reçoit des paramètres et peut renvoyer un résultat (`return`). ch. 1, p. 32

**Formulaire** Partie d'une page (`<form>`) qui envoie des données au serveur : `action` donne l'adresse qui les reçoit, `method` la méthode (`get` ou `post`). ch. 10, p. 401

**Frontière de décision** Ligne qui sépare les régions du plan coloriées selon la classe prédite ; elle colle aux exemples pour $k$ petit, devient plus lisse quand $k$ grandit. ch. 12, p. 480

## G

**`GET`** Méthode HTTP qui demande une ressource (afficher une page) ; les éventuels paramètres sont visibles dans l'URL, après un `?`. ch. 10, p. 395

## H

**Hexadécimal (base 16)** Base à 16 chiffres (`0` à `9`, puis `A` à `F`) ; un chiffre hexadécimal représente exactement 4 bits, un octet tient en deux chiffres. ch. 2, p. 99

**HTML** Langage de balisage (*HyperText Markup Language*), et non de programmation, qui décrit la structure du contenu d'une page : titres, paragraphes, images, liens. ch. 10, p. 396

**HTTP** Protocole du Web : le client envoie une requête (méthode, chemin), le serveur renvoie une réponse (code de statut comme 200 ou 404, en-têtes, corps). HTTP est sans mémoire. ch. 10, p. 394

**HTTPS** Version sécurisée de HTTP : la communication est chiffrée. Le cadenas prouve que la liaison est chiffrée, pas que le site est honnête. ch. 10, p. 394

## I

**`id`, `class`** Attributs qui étiquettent une balise pour la retrouver en CSS ou en JavaScript : `id` est unique dans la page, `class` peut être partagée par plusieurs balises. ch. 10, p. 396

**Indentation** Décalage vers la droite (4 espaces) des lignes qui forment un bloc. En Python, l'indentation est la structure ; on ne mélange jamais espaces et tabulations. ch. 1, p. 27

**Indice** Position d'un élément dans une chaîne, un p-uplet ou un tableau, comptée à partir de 0 : de `0` à `len(t) - 1`. ch. 3, p. 141

**Instruction conditionnelle** Construction qui choisit entre plusieurs comportements selon une condition : `if`, `elif`, `else`. Seul le premier bloc dont la condition est vraie est exécuté. ch. 1, p. 26

**Instructions du LMC** `INP`/`OUT` : lire/afficher ; `LDA n`/`STA n` : charger la case `n` dans l'accumulateur ACC/l'y ranger ; `ADD`, `SUB` ; sauts `BRA`, `BRZ`, `BRP` ; `HLT` ; `DAT`. ch. 9, p. 357

**Internet** Réseau mondial : l'infrastructure de câbles, fibres, antennes et machines (routeurs) qui relient les ordinateurs entre eux. ch. 10, p. 389

**Interpréteur, langage interprété** Un langage interprété (comme Python) est traduit au fur et à mesure, pendant l'exécution, par un interpréteur nécessaire à chaque exécution. ch. 5, p. 219

**Invariant de boucle** Propriété vraie avant la boucle et préservée à chaque tour ; à la sortie, elle prouve que le résultat est correct. ch. 4, p. 183 ; ch. 5, p. 218

## J

**JavaScript** Langage de programmation exécuté dans le navigateur (côté client) qui rend la page interactive : réagir à un clic, modifier le contenu, vérifier un formulaire. ch. 10, p. 398

**Jeu d'entraînement, jeu de test** On sépare les exemples étiquetés : le jeu d'entraînement sert au vote, le jeu de test, jamais vu, mesure le taux de réussite. ch. 12, p. 481

**Jeu de tests** Ensemble de cas dont on connaît le résultat attendu (cas normaux, cas limites, cas particuliers). Son succès ne prouve pas la correction : seulement sur les cas testés. ch. 1, p. 34

**Jointure (fusion de tables)** Production d'une table où chaque enregistrement de la première est complété par les informations de la seconde ayant la même valeur pour un descripteur commun. ch. 8, p. 318

## K

**$k$ plus proches voisins ($k$-NN)** Méthode de classification : on calcule la distance du nouvel individu à chaque exemple, on garde les $k$ plus proches et on lui attribue leur classe majoritaire. ch. 12, p. 479

## L

**Langage machine** Jeu réduit d'instructions très simples (charger, additionner, comparer, sauter…), codées en binaire : le seul langage que le processeur exécute. ch. 9, p. 356

**Ligne de commande (terminal)** Interface où l'on tape des commandes au clavier, après une invite qui rappelle qui l'on est et où l'on est ; plus rapide et automatisable que l'interface graphique. ch. 9, p. 358

**Liste de dictionnaires** Représentation d'une table en Python : chaque enregistrement est un dictionnaire `{descripteur: valeur}`, la table est la liste de ces dictionnaires. ch. 8, p. 313

**Little Man Computer (LMC)** Machine-modèle de von Neumann à 100 cases mémoire, un accumulateur et un compteur de programme, programmée en assembleur et simulable en ligne. ch. 9, p. 357

**Logarithme de base 2 ($\log_2$)** Nombre $a$ de divisions par 2 qui ramènent $n$ à 1, soit $n = 2^a$ : $\log_2$ est la réciproque de la puissance de 2. ch. 7, p. 285

## M

**Matrice** Tableau de tableaux de même longueur : $n$ lignes et $p$ colonnes. `M[i][j]` est la case ligne `i`, colonne `j` ; on la parcourt avec deux boucles imbriquées. ch. 3, p. 148

**Meilleur cas, pire cas** Situations la plus favorable et la plus défavorable pour le coût : pour une recherche séquentielle, élément en tête (1 comparaison) ou absent (`n` comparaisons). ch. 4, p. 185

**Mémoire** Organe qui range à la fois le programme et les données, dans des cases numérotées par une adresse. ch. 9, p. 354

**Mémoire vive (RAM)** Mémoire de quelques giga-octets, rapide mais volatile : tout s'efface à l'extinction. Enregistrer, c'est recopier de la RAM vers le disque, permanent. ch. 9, p. 355

**`mkdir`, `touch`, `cat` (Linux)** Commandes : `mkdir` crée un répertoire, `touch` crée un fichier vide, `cat` affiche le contenu d'un fichier. ch. 9, p. 360

**Modèle de von Neumann** Organisation de quasiment tous les ordinateurs (1945) : unité de commande, UAL, mémoire et entrées-sorties ; le programme est rangé dans la même mémoire que les données. ch. 9, p. 354

**Mutable, immuable** Un objet est mutable si l'on peut changer son contenu sans en créer un nouveau (`list`, `dict`) ; immuable sinon (nombres, `str`, `tuple`). ch. 3, p. 141

## N

**Normaliser des descripteurs** Ramener des descripteurs d'échelles différentes (âge, revenu) à une même échelle, pour qu'aucun n'écrase les autres dans la distance. ch. 12, p. 482

## O

**Octet** Paquet de 8 bits (*byte*) ; il permet $2^8 = 256$ valeurs différentes. ch. 2, p. 93

**Opérateur booléen (non, et, ou)** Opérateur qui combine des valeurs booléennes : non $a$ inverse ; $a$ et $b$ vaut 1 seulement si les deux valent 1 ; $a$ ou $b$ vaut 1 dès qu'au moins un vaut 1. ch. 2, p. 96

**Oracle** Fonction, en général plus simple que le programme testé, qui juge si un résultat est juste ; elle permet de vérifier des tests générés au hasard. ch. 5, p. 217

**Ou exclusif (xor)** $a$ xor $b$ vaut 1 lorsque exactement un des deux bits vaut 1, c'est-à-dire lorsque $a$ et $b$ sont différents. ch. 2, p. 97

## P

**p-uplet nommé (tableau associatif)** Regroupement de valeurs repérées chacune par un nom de champ plutôt que par une position ; en Python, on l'écrit avec un dictionnaire. C'est une fiche, un enregistrement. ch. 3, p. 149

**p-uplet (`tuple`)** Suite ordonnée de $p$ éléments, de types éventuellement différents, qu'on ne peut pas modifier après création ; on sépare les éléments par des virgules : `(3, 4)`. ch. 3, p. 142

**Pair-à-pair** Organisation où chaque machine est à la fois cliente et serveur et échange directement avec les autres, sans serveur central. ch. 10, p. 392

**Paquet** Petit morceau d'un message découpé pour voyager sur le réseau ; il porte les adresses IP de départ et d'arrivée, et les paquets sont réassemblés à l'arrivée. ch. 10, p. 390

**Paradigme (style de programmation)** Style d'un langage : impératif (des ordres qui modifient des variables, comme en Python cette année), fonctionnel, objet, logique, événementiel. ch. 5, p. 219

**Paramètre** Nom, placé entre parenthèses dans la ligne `def`, qui désigne une entrée de la fonction ; il reçoit la valeur de l'argument lors de l'appel. ch. 1, p. 32

**Parcours séquentiel** Examen des éléments d'un tableau un par un, du premier au dernier, par élément ou par indice ; on dit aussi parcours linéaire. ch. 4, p. 180

**Polygone régulier (tortue)** Pour tracer un polygone régulier à $n$ côtés, la tortue avance puis tourne de $360/n$ degrés, $n$ fois. compl., p. 71

**Porte logique** Petit circuit de quelques transistors qui reçoit un ou deux bits et en produit un, en réalisant un opérateur booléen (ET, OU, NON…). ch. 2, p. 96

**Portée, variable locale** Les paramètres et variables créés à l'intérieur d'une fonction sont locaux : ils naissent à l'appel, disparaissent au `return` et sont invisibles de l'extérieur. ch. 1, p. 33

**`POST`** Méthode HTTP qui envoie des données au serveur pour qu'il les traite (typiquement un formulaire) ; les données ne sont pas dans l'URL. ch. 10, p. 395

**Postcondition** Garantie sur le résultat d'une fonction, valable si les préconditions sont respectées (ex. : « renvoie un nombre positif »). ch. 1, p. 33 ; ch. 5, p. 213

**Précondition** Hypothèse sur les arguments, à la charge de l'appelant, qui doit être vraie pour qu'on ait le droit d'appeler la fonction (ex. : `x >= 0`). ch. 1, p. 33 ; ch. 5, p. 213

**Problème d'optimisation** Problème qui demande, parmi de nombreuses solutions valides, d'en trouver une la meilleure possible selon une mesure de qualité à minimiser ou maximiser. ch. 11, p. 446

**Problème du sac à dos** Remplir un sac de capacité limitée en emportant le plus de valeur possible ; le glouton par rapport valeur/poids est rapide mais pas toujours optimal. ch. 11, p. 447

**Processeur (CPU)** Réunion de l'unité de commande, de l'UAL et de quelques registres : il exécute les instructions du langage machine. ch. 9, p. 354

**Programmation défensive** Façon de programmer qui teste la précondition à l'entrée d'une fonction (`assert`) ou renvoie une valeur sentinelle, pour échouer tôt et clairement. ch. 5, p. 214

**Programme** Traduction d'un algorithme dans un langage que la machine peut exécuter : une suite d'instructions qu'un ordinateur exécute pour accomplir une tâche. ch. 1, p. 21

**Protocole** Ensemble de règles précises que deux machines respectent pour communiquer (IP, TCP, HTTP…). ch. 10, p. 390

**Protocole du bit alterné** Chaque paquet porte un seul bit, alternativement 0 et 1 ; il suffit au récepteur pour distinguer un nouveau paquet d'un paquet renvoyé (doublon). ch. 10, p. 393

**Prototype** Annonce du nom d'une fonction, de ses paramètres et de ce qu'elle renvoie, par exemple `racine(x)`. ch. 1, p. 33

**`pwd`, `ls`, `cd` (Linux)** Commandes pour se repérer : `pwd` affiche le répertoire courant, `ls` liste un répertoire (`ls -l` en détail, avec les droits), `cd` change de répertoire. ch. 9, p. 360

## R

**Recherche dichotomique** Recherche dans un tableau trié : on compare à l'élément du milieu et on divise par deux la zone de recherche `[deb ; fin]` à chaque étape. ch. 7, p. 283

**Recherche séquentielle** Parcours qui cherche si un élément est présent (ou où il est) et s'arrête dès qu'il le trouve ; s'il est absent, on renvoie `-1` ou `None` selon l'énoncé. ch. 4, p. 182

**Référence (modèle mémoire)** Une variable ne contient pas l'objet : elle le désigne, comme une flèche du nom vers l'objet rangé en mémoire. Ce modèle explique alias et copies. ch. 3, p. 141

**Rendu de monnaie** Rendre une somme avec le moins de pièces possible ; le glouton prend à chaque fois la plus grande coupure qui ne dépasse pas la somme restante. ch. 11, p. 447

**Répertoire courant, répertoire parent** Le répertoire courant est celui où l'on se trouve, noté `.` ; le répertoire parent, un cran au-dessus, est noté `..` ch. 9, p. 359

**Rosace** Figure obtenue en répétant un même motif tout en tournant un peu à chaque fois : une fonction, une boucle et un petit tour. compl., p. 72

**Routeur** Machine qui aiguille les paquets : elle lit l'adresse IP de destination et choisit vers quel voisin renvoyer le paquet ; les routeurs interconnectés forment Internet. ch. 10, p. 391

## S

**Sélecteur (CSS)** Partie d'une règle CSS qui vise des balises : par nom (`h1`), par identifiant (`#menu`) ou par classe (`.bouton`). ch. 10, p. 397

**Séquence** Construction la plus simple : des instructions écrites les unes sous les autres, exécutées dans l'ordre, de haut en bas, une à la fois. L'ordre détermine le résultat. ch. 1, p. 21

**Solution optimale, solution approchée** Une solution optimale est une meilleure solution ; une solution approchée est une bonne solution, pas forcément la meilleure. ch. 11, p. 446

**Spécification** Description du contrat d'une fonction, sans dire comment elle s'y prend : son prototype, ses préconditions et ses postconditions. ch. 1, p. 33 ; ch. 5, p. 213

**Système canonique** Système de pièces (comme les euros) pour lequel le rendu de monnaie glouton donne toujours la solution optimale ; ce n'est pas le cas de tous. ch. 11, p. 449

**Système d'exploitation (OS)** Ensemble de programmes qui gèrent le matériel (processeur, mémoire, fichiers, périphériques, utilisateurs) et le mettent à disposition des applications : Windows, macOS, Linux… ch. 9, p. 358

## T

**Table de données** Organisation de l'information en lignes (les enregistrements) et colonnes (les descripteurs) ; en Python, une liste de dictionnaires. ch. 8, p. 312

**Table de vérité** Tableau qui donne la valeur d'un opérateur ou d'une expression booléenne pour chaque combinaison possible des entrées, une ligne par combinaison. ch. 2, p. 96

**Tableau (`list`)** Suite ordonnée et modifiable d'éléments, écrite entre crochets : `[12, 8, 15]`. On lit et modifie chaque élément par son indice. ch. 3, p. 144

**TCP** Protocole de transport d'Internet qui, par accusés de réception et renvois, garantit qu'aucune donnée n'est perdue ni dupliquée. ch. 10, p. 394

**Terminaison** Propriété d'un algorithme qui s'arrête toujours. Pour une boucle non bornée, on la justifie avec un variant. ch. 1, p. 31

**Trace d'exécution** Tableau qui suit l'évolution des variables tour après tour ; la dresser à la main est un réflexe essentiel pour comprendre ou déboguer un programme. ch. 1, p. 30

**Transistor** Minuscule interrupteur commandé électriquement, sans pièce mécanique, qui bascule des milliards de fois par seconde ; brique de base des portes logiques. ch. 9, p. 353

**Tri** Réorganisation des éléments d'un tableau dans l'ordre croissant, `t[0]` $\leq$ `t[1]` $\leq \cdots$, en comparant et en échangeant des éléments. ch. 6, p. 251

**Tri en place** Tri qui réarrange le tableau donné lui-même, sans en créer un nouveau. ch. 6, p. 251

**Tri par insertion** On prend les éléments un par un et on insère chacun à sa place dans la partie gauche déjà triée, en décalant les plus grands. Linéaire si le tableau est déjà trié. ch. 6, p. 254

**Tri par sélection** À chaque étape, on cherche le minimum de la partie non triée et on l'échange avec son premier élément ; la partie gauche est triée et définitive. ch. 6, p. 252

**Typage dynamique** Le type n'est connu que pendant l'exécution, quand la ligne est atteinte (c'est le cas de Python) : une erreur de type ne se manifeste qu'à ce moment-là. ch. 5, p. 220

**Typage statique** Le type de chaque variable est fixé et vérifié avant l'exécution : une opération entre types incompatibles est refusée sans même lancer le programme. ch. 5, p. 220

**Type** Nature d'une valeur, qui décide des opérations permises : entier (`int`), flottant (`float`), chaîne (`str`), booléen (`bool`)… ch. 1, p. 23

**Type construit** Type dont chaque valeur regroupe plusieurs valeurs en un seul objet : p-uplet,

tableau, dictionnaire. Par opposition, `int`, `float`, `bool`, `str` sont simples. ch. 3, p. 140

## U

**UAL (unité arithmétique et logique)** Organe du processeur qui calcule : additions, comparaisons, ET/OU logiques... ch. 9, p. 354

**Unicode, point de code** Catalogue universel qui attribue un numéro unique, le point de code (noté `U+...`), à tous les caractères de toutes les écritures. ch. 2, p. 104

**Unité de commande** Organe du processeur qui lit les instructions du programme, les décode et ordonne leur exécution. ch. 9, p. 354

**URL** Adresse d'une ressource sur le Web : protocole (`https`), nom de domaine du serveur (`www.lycee.mc`) et chemin de la ressource (`/nsi/cours.html`). ch. 10, p. 394

**UTF-8** Façon la plus répandue d'écrire les points de code Unicode en octets : 1 octet pour les caractères ASCII, 2 à 4 octets pour les autres. ch. 2, p. 104

## V

**Valeur booléenne** Valeur qui ne peut être que `1` (vrai) ou `0` (faux) ; en Python, `True` ou `False` (type `bool`). ch. 2, p. 96

**Valeur renvoyée** Résultat qu'une fonction rend au programme avec `return`, pour être réutilisé dans un calcul ; une fonction sans `return` renvoie `None`. ch. 1, p. 32

**Valeur sentinelle** Valeur renvoyée qui ne peut pas être confondue avec un résultat valide (en Python, `None`) ; celui qui appelle teste alors le résultat. ch. 5, p. 214

**Variable** Nom qui désigne une valeur rangée en mémoire ; on peut lire cette valeur et la modifier. Plus précisément, une étiquette qui pointe vers un objet. ch. 1, p. 22 ; ch. 3, p. 140

**Variant de boucle** Quantité entière positive qui décroît strictement à chaque tour de boucle ; s'il en existe un, la boucle finit par s'arrêter (terminaison). ch. 1, p. 31 ; ch. 7, p. 284

**Voyageur de commerce** Visiter toutes les villes et revenir au départ en parcourant le moins de kilomètres ; le glouton « ville la plus proche » donne une bonne solution, rarement la meilleure. ch. 11, p. 447

## W

**Web** Service d'Internet (*World Wide Web*) : ensemble des pages reliées par des liens hypertextes, consultées avec un navigateur. Ne pas le confondre avec Internet. ch. 10, p. 389

## Python : mots-clés, fonctions et syntaxe

**`%` (modulo)** Reste de la division entière : `7 % 2` vaut `1`. Un entier `a` est divisible par `b` lorsque `a % b == 0`. ch. 1, p. 28

**`+, -, *, /, **`** Opérations sur les nombres : addition, soustraction, multiplication, division (`7 / 2` vaut le flottant `3.5`), puissance (`7 ** 2` vaut `49`). ch. 1, p. 24

**`//` (quotient entier)** Quotient de la division entière : `7 // 2` vaut `3`. Il sert par exemple à calculer le milieu `(deb + fin) // 2`. ch. 1, p. 24

**`0b, bin`** Préfixe d'un littéral binaire : `0b10111001` vaut 185 ; `bin(185)` donne la chaîne `'0b10111001'` et `int("10111001", 2)` fait l'inverse. ch. 2, p. 95

**`=` et `==`** `=` affecte une valeur (« reçoit ») ; `==` teste l'égalité et renvoie un booléen. Les confondre est une erreur très fréquente. ch. 1, p. 26

**`[0] * n`** Répétition : tableau de `n` zéros. Piège : `[[0] * p] * n` répète la même ligne ; on écrit `[[0] * p for i in range(n)]`. ch. 3, p. 148

**`abs`** Valeur absolue ; on compare deux flottants par un écart minuscule : `abs(a - b) < 1e-9`, jamais par `==`. ch. 2, p. 102

**Affectation multiple `a, b = b, a`** Déballe un p-uplet dans plusieurs variables ; Python évalue d'abord tout le membre de droite, ce qui permet d'échanger deux variables en une ligne. ch. 3, p. 143

**`and, or, not`** Opérateurs booléens qui combinent des conditions. `and` et `or` sont séquentiels : évalués de gauche à droite, ils s'arrêtent dès que le résultat est connu. ch. 1, p. 26

**`append`** Méthode `t.append(x)` : ajoute `x` à la fin du tableau, sur place, et renvoie `None` (ne jamais écrire `t = t.append(x)`). ch. 3, p. 146

**`assert`** Instruction qui vérifie une condition et arrête le programme avec un message si elle est fausse ; sert de garde-fou sur une précondition et à écrire les tests. ch. 1, p. 33

**`begin_fill, end_fill`** Ordres de la tortue qui encadrent le tracé d'une figure pour la remplir de couleur. compl., p. 71

**`bool, True, False`** Type booléen et ses deux seules

valeurs, vrai et faux ; c'est la valeur d'une condition. ch. 1, p. 23

**circle, color** Ordres de la tortue : `circle(r)` trace un cercle de rayon `r`, `color("red")` choisit la couleur du trait. compl., p. 70

**csv.DictReader** Lecteur du module `csv` qui lit l'en-tête puis transforme chaque ligne en dictionnaire ; toutes les valeurs sont des chaînes ; `delimiter` précise le séparateur. ch. 8, p. 314

**d[cle], del d[cle]** `d[cle]` lit la valeur associée à une clé ; `d[cle] = v` crée ou remplace un couple ; `del d[cle]` le supprime. ch. 3, p. 150

**def** Mot-clé qui définit une fonction : `def nom(param1, param2):` suivi du bloc indenté. La définition seule n'exécute rien. ch. 1, p. 25

**Deux-points :** Signe obligatoire en fin de ligne `if`, `elif`, `else`, `for`, `while`, `def` ; les lignes du bloc qui suit sont indentées. ch. 1, p. 26

**Docstring** Texte entre triples guillemets placé juste après la ligne `def` : rôle, précondition, postcondition. On la consulte avec `help`. ch. 1, p. 33 ; ch. 5, p. 213

**elif** Contraction de *else if* : enchaîne plusieurs tests ; Python exécute le premier bloc dont la condition est vraie et ignore les autres. ch. 1, p. 27

**encode, decode** Méthodes qui convertissent un texte en octets (`"é".encode("utf-8")`) et des octets en texte, selon un encodage. ch. 2, p. 104

**float** Type des nombres à virgule (flottants), aux valeurs approchées ; `float(...)` convertit en flottant : `float("12.5")`. ch. 1, p. 24

**for** Boucle bornée : `for i in range(n):` répète le bloc pour chaque valeur ; `for x in t:` parcourt directement les éléments d'une chaîne ou d'un tableau. ch. 1, p. 28

**for x in t, for i in range(len(t))** Les deux parcours d'un tableau : par élément, quand seules les valeurs comptent ; par indice, quand on a besoin de la position ou qu'on veut modifier. ch. 3, p. 145

**forward, backward** Ordres de la tortue : `forward(d)` avance de `d` pas, `backward(d)` recule ; le trait n'apparaît qu'avec ces ordres. compl., p. 70

**goto** Ordre de la tortue : `goto(x, y)` va au point de coordonnées `(x, y)`. compl., p. 70

**help** Fonction qui affiche la docstring d'une fonction, sans voir son code. ch. 5, p. 213

**if, else** `if condition:` exécute le bloc indenté si la condition est vraie ; le bloc de `else:` est exécuté sinon. ch. 1, p. 26

**import** Instruction qui charge une bibliothèque (un module) : `import random`, `import turtle as t`, `import csv`. ch. 1, p. 34

**in** Test d'appartenance (`x in t`) ; sur un dictionnaire, `cle in d` porte sur les clés, pas sur les valeurs. Dans `for x in t`, il introduit le parcours. ch. 3, p. 150

**IndentationError** Erreur provoquée par une indentation incohérente des lignes d'un bloc. ch. 1, p. 27

**Indices négatifs t[-1]** Indices comptés depuis la fin : `t[-1]` est le dernier élément, `t[-2]` l'avant-dernier ; `t[-k]` désigne `t[len(t) - k]`. ch. 1, p. 25 ; ch. 3, p. 144

**input** Fonction qui affiche un message, attend une saisie au clavier et renvoie ce qui a été tapé, toujours sous forme de chaîne. ch. 1, p. 24

**int** Type des entiers (de taille arbitraire en Python) ; `int(...)` convertit en entier : `int("17")` vaut 17. ch. 1, p. 24

**KeyError** Erreur provoquée par `d[cle]` quand la clé est absente du dictionnaire ; tester `cle in d` avant l'évite. ch. 3, p. 150

**keys, values, items** Méthodes d'un dictionnaire : `d.keys()` les clés, `d.values()` les valeurs, `d.items()` les couples `(clé, valeur)` ; `for k in d` parcourt les clés. ch. 3, p. 150

**lambda** Petite fonction anonyme utilisée comme clé de tri : `lambda ligne: int(ligne["nombre"])` associe à une fiche sa clé de tri. ch. 8, p. 318

**left, right** Ordres de la tortue : `left(a)` et `right(a)` tournent de `a` degrés à gauche ou à droite ; ils ne changent que le cap. compl., p. 70

**len** Fonction qui renvoie le nombre d'éléments d'une chaîne, d'un p-uplet, d'un tableau ou d'un dictionnaire. ch. 3, p. 142

**M[i][j]** Case ligne `i`, colonne `j` d'une matrice ; `len(M)` est le nombre de lignes, `len(M[0])` le nombre de colonnes. ch. 3, p. 148

**NameError** Erreur provoquée par l'utilisation d'un nom qui n'existe pas, souvent une faute de frappe. ch. 1, p. 35

**None** Valeur spéciale « rien » : renvoyée par une fonction sans `return`, ou comme valeur sentinelle ; on la teste avec `r is None`. ch. 1, p. 33 ; ch. 5, p. 214

**open, with** `open(...)` ouvre un fichier (ici avec `encoding="utf-8"`) ; dans un bloc `with`, le fichier est refermé tout seul à la fin. ch. 8, p. 314

**ord, chr** `ord` donne le code (point de code) d'un caractère, `chr` fait l'inverse : `ord('A')` vaut 65, `chr(65)` vaut `'A'`. ch. 2, p. 104

**penup, pendown** Ordres de la tortue : lever et baisser le stylo, pour se déplacer sans tracer. compl., p. 70

**pop** Méthode `t.pop()` : retire et renvoie le dernier élément ; `t.pop(i)` retire et renvoie l'élément d'indice `i`. ch. 3, p. 146

**`print`** Fonction qui affiche à l'écran ce qu'on lui donne entre parenthèses ; `end=""` évite le retour à la ligne. Afficher n'est pas renvoyer. ch. 1, p. 21

**`random.randint`** Fonction du module `random` : `random.randint(1, 6)` renvoie un entier au hasard entre 1 et 6 (bornes comprises). ch. 1, p. 34

**`range`** `range(n)` : les entiers de `0` à `n-1` ; `range(a, b)` : de `a` à `b-1` ; `range(a, b, pas)` : de `pas` en `pas`. La borne `b` est exclue. ch. 1, p. 28

**`return`** Instruction qui renvoie une valeur au programme et sort immédiatement de la fonction ; les lignes situées après ne sont pas exécutées. ch. 1, p. 33

**`return a, b`** Une fonction renvoie plusieurs résultats sous forme de p-uplet, récupérés par affectation multiple : `q, r = division(17, 5)`. ch. 3, p. 143

**`sort`, `sorted`** Tris tout faits de Python (Timsort) : `t.sort()` trie en place, `sorted(t)` renvoie un nouveau tableau trié. Le chapitre programme d'abord, à la main, le tri par sélection et le tri par insertion. ch. 6, p. 256

**`sorted(..., key=...)`** Trie une table selon un descripteur : `key` donne la valeur à comparer pour chaque fiche, `reverse=True` trie du plus grand au plus petit. ch. 8, p. 318

**`str`** Type des chaînes de caractères ; `str(...)` convertit en chaîne : `str(17)` vaut `"17"`. ch. 1, p. 24

**`SyntaxError`** Erreur de syntaxe (`:` ou parenthèse oubliés…), détectée avant l'exécution : Python refuse le programme sans exécuter la moindre ligne. ch. 1, p. 35

**`t[i]` (accès par indice)** Élément d'indice `i` d'une chaîne, d'un p-uplet ou d'un tableau, de `0` à `len(t) - 1` ; dans un tableau, `t[i] = x` le remplace. ch. 3, p. 144

**`Traceback`** Message d'erreur affiché quand le programme s'arrête : fichier, numéro de ligne, ligne fautive, puis, en dernière ligne, le type d'erreur et son message. ch. 1, p. 35

**Tranche `t[a:b]`** Nouveau tableau (ou chaîne) formé des éléments d'indices `a` à `b - 1` ; `a` omis : depuis le début ; `b` omis : jusqu'à la fin. ch. 3, p. 144

**`turtle`** Module de dessin : on l'importe (`import turtle as t`), on donne des ordres à la tortue, et `turtle.done()` garde la fenêtre ouverte. compl., p. 69

**`type`** Fonction qui renvoie le type d'une valeur : `type(12.5)` donne `<class 'float'>`. ch. 1, p. 23

**`TypeError`** Erreur provoquée par une opération entre types incompatibles, comme `"a" + 1`. ch. 1, p. 35

**`while`** Boucle non bornée : `while condition:` répète le bloc tant que la condition est vraie, testée avant chaque tour. ch. 1, p. 31

**`ZeroDivisionError`** Erreur provoquée par une division par zéro. ch. 1, p. 35

# Crédits iconographiques

*Illustrations d'ouverture de chapitre* — dessins originaux de l'auteur.

### Présentation et panorama de l'année

*Capture d'écran du logiciel Spyder (logiciel libre, licence MIT)* — capture réalisée par l'auteur — licence MIT pour le logiciel
`https://www.spyder-ide.org`

*Manuscrit de l'Algèbre d'al-Khwârizmî (MS. Huntington 214)* — Bodleian Library, Oxford — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Bodleian_MS._Huntington_214_roll332_frame36.jpg`

*Gottfried Wilhelm Leibniz (v. 1695)* — Christoph Bernhard Francke — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Christoph_Bernhard_Francke_-_Bildnis_des_Philosophen_Leibniz_(ca._1695).jpg`

*Ada Lovelace (aquarelle, v. 1840)* — Alfred Edward Chalon — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Ada_Lovelace_portrait.jpg`

*Herman Hollerith* — Charles Milton Bell — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Herman_Hollerith,_head-and-shoulders_portrait,_facing_left_LCCN96502521.jpg`

*L'EDVAC* — U.S. Army — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Edvac.jpg`

*Cahier de bord du Mark II, 9 septembre 1947* — Naval Surface Warfare Center, Dahlgren — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:First_Computer_Bug,_1947.jpg`

*Iris versicolor* — U.S. Fish and Wildlife Service — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Iris_versicolor_FWS.jpg`

*Robot tortue de sol* — Valiant Technology Ltd. — CC BY-SA 3.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Turtle_draw.jpg`

### Chapitre 1 — Les bases de la programmation Python

*Ada Lovelace (aquarelle, v. 1840)* — Alfred Edward Chalon — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Ada_Lovelace_portrait.jpg`

*Note G : calcul des nombres de Bernoulli (1843)* — Ada Lovelace — domaine public — p. 21
`https://commons.wikimedia.org/wiki/File:Diagram_for_the_computation_of_Bernoulli_numbers.jpg`

*Manuscrit de l'Algèbre d'al-Khwârizmî (MS. Huntington 214)* — Bodleian Library, Oxford — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Bodleian_MS._Huntington_214_roll332_frame36.jpg`

### Complément : dessiner avec la tortue

*Robot tortue de sol* — Valiant Technology Ltd. — CC BY-SA 3.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Turtle_draw.jpg`

### Chapitre 2 — Le binaire et l'écriture des nombres

*Gottfried Wilhelm Leibniz (v. 1695)* — Christoph Bernhard Francke — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Christoph_Bernhard_Francke_-_Bildnis_des_Philosophen_Leibniz_(ca._1695).jpg`

*Leibniz, « Explication de l'arithmétique binaire » (1703)* — Gottfried Wilhelm Leibniz — domaine public — p. 94
`https://commons.wikimedia.org/wiki/File:Leibniz_binary_system_1703.png`

### Chapitre 5 — Spécifier et mettre au point ses programmes

*Cahier de bord du Mark II, 9 septembre 1947* — Naval Surface Warfare Center, Dahlgren — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:First_Computer_Bug,_1947.jpg`

*Grace Hopper en 1984* — James S. Davis (U.S. Navy) — domaine public — p. 222
`https://commons.wikimedia.org/wiki/File:Commodore_Grace_M._Hopper,_USN_(covered).jpg`

### Chapitre 6 — Les algorithmes de tri

*Trieuse de cartes perforées Hollerith* — Erik Pitti — CC BY 2.0 — p. 257
`https://commons.wikimedia.org/wiki/File:Hollerith_punch-card_sorter_(2586173692).jpg`

*L'EDVAC* — U.S. Army — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Edvac.jpg`

### Chapitre 8 — Les données en tables

*Herman Hollerith* — Charles Milton Bell — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Herman_Hollerith,_head-and-shoulders_portrait,_facing_left_LCCN96502521.jpg`

*Carte perforée Hollerith* — auteur inconnu — domaine public — p. 320
`https://commons.wikimedia.org/wiki/File:Hollerith_Punched_Card.jpg`

### Chapitre 9 — Architecture des ordinateurs et systèmes d'exploitation

*L'ENIAC* — U.S. Army — domaine public — p. 362
`https://commons.wikimedia.org/wiki/File:ARL_ENIAC_01.png`

### Chapitre 11 — Les algorithmes gloutons

*Stephen Cook en 1968* — George Bergman — CC BY-SA 4.0 — p. 451
`https://commons.wikimedia.org/wiki/File:Stephen_A._Cook_1968_(bordered).jpg`

### Chapitre 12 — Les $k$ plus proches voisins

*Iris versicolor* — U.S. Fish and Wildlife Service — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Iris_versicolor_FWS.jpg`

*Iris virginica* — Frank Mayfield — CC BY-SA 2.0 — p. 477
`https://commons.wikimedia.org/wiki/File:Iris_virginica.jpg`